

DATA STRUCTURES · ALGORITHMS · SYSTEM DESIGN

क्रम

KRAMA

The Ordered Path

180 Days from First Principles to the Product-Company Interview

NISHANT GUPTA - GRANTH SKILLS

VOLUME VII OF EIGHT

Optimal Choices

DAYS 143 – 170

DAY 001

THE ORDERED PATH

DAY 180



TWO TRACKS

28 OF 180 DAYS IN THIS VOLUME

180 STEPS

K R A M A

THE ORDERED PATH

क्रम

K R A M A

The Ordered Path

180 Days from First Principles to the Product-Company Interview

Nishant Gupta - Granth Skills

VOLUME VII · OPTIMAL CHOICES

DAYS 143 TO 170

KRAMA — THE ORDERED PATH

180 Days from First Principles to the Product-Company Interview

Volume VII of eight: **Optimal Choices**. Days 143 to 170.

Copyright © 2026 Nishant Gupta - Granth Skills. All rights reserved.

ABOUT THE TEXT

This edition is generated from the Krama course sources. The lessons, stories, code, diagrams and interview material are original work written for this course.

Practice problems are named, with their source and one line on what they test. No problem statement is reproduced. Trade marks and problem titles belong to their respective owners and are used here only to identify where a problem can be found.

ABOUT THE MAKING OF IT

Typeset from Markdown by rendering to a browser engine and printing to PDF. Every tool in that chain is free software and every typeface is published under the SIL Open Font Licence. Appendix F lists all of them with their licences.

Text face: Source Serif 4 — SIL Open Font Licence 1.1

Display face: Fraunces — SIL Open Font Licence 1.1

Sans face: Inter — SIL Open Font Licence 1.1

Monospace face: JetBrains Mono — SIL Open Font Licence 1.1

Devanagari: Noto Serif Devanagari — SIL Open Font Licence 1.1

Diagrams: Mermaid — MIT Licence

Code colouring: Pygments — BSD 2-Clause

EDITION

First edition, 2026. Set in Source Serif 4 on a 6.75 by 9.5 inch page.

क्रम

krama, Sanskrit: order; sequence; the step that follows the step before it.

YOU CANNOT SKIP A STEP. THAT IS THE WHOLE METHOD.

Why this book teaches two things at once

This book teaches two subjects at the same time, on purpose.

Most preparation separates them. You spend three months on data structures and algorithms, then start again from nothing on system design, and by the time the second subject makes sense the first has gone quiet. Interviews do not work that way. A single loop at a product company will ask you to reverse a linked list in one room and design a rate limiter in the next, on the same day, from the same head.

So every chapter here teaches one topic from each track, side by side. Day 42 gives you binary search and the object-oriented design of a parking lot. Day 130 gives you graph colouring and message queues. You will not finish this book with one strong subject and one weak one.

Who this is written for

Someone who has never studied this before. Not a computer science graduate. Not someone brushing up after a few years in the job.

If you need to be told what a server is, you are the reader. The first chapter explains what happens when a program runs. The fourteenth explains what happens when you type an address into a browser. Nothing before that is assumed.

The other half of that promise matters just as much: you are someone who will sit in a real interview. So the writing never stops at understanding. Every chapter ends by putting the idea in an interviewer's mouth, giving you the first ninety seconds of an answer, and writing out a full model response you can compare yours against.

The one question every chapter answers

Every lesson in this book was written to satisfy a single test:

Does this leave the reader able to answer one real interview question out loud, from memory, to a stranger?

That test decided what went in and, more usefully, what stayed out. There are no formal proofs here. No potential functions, no decision-tree lower bounds, no interpreter internals for their own sake. If a thing will not be asked, and does not make an asked thing clearer, it is not in this book.

What is different about how it is written

Every lesson opens with a story about a person. Not an analogy dressed up as a story: an actual scene, with a name in it, told in ordinary words with no technical vocabulary at all. Someone sorting socks. Someone in a canteen queue. Someone laying tables for a wedding. The story comes before the definition because a beginner needs somewhere to put the idea before they have the word for it.

Every solution is shown in full. This book does not hide answers. Code is built up in fragments of a few lines, each one explained, and then given again complete and ready to run. You are never told to work it out yourself as a test of character.

The arithmetic is always done out loud. Not “this is slow” but “this line runs about twenty-five million times”. Not “that is a lot of data” but `50M x 20 x 2KB = 2TB`. Numbers are worked through where you can see them.

Errors are printed exactly as they appear. When something breaks, the real message is on the page - `IndexError: list index out of range` - not a description of it. You will recognise it when your own screen shows it.

How this edition was made

The text is generated from the course repository, so the book and the working material cannot drift apart. The typesetting was done by rendering the pages in a browser engine and printing them to PDF.

Every part of the toolchain is free software, and every typeface is published under the SIL Open Font Licence. Appendix F lists all of it, with licences. No proprietary tool, font, or copyrighted third-party text was used to produce this book.

That extends to the practice problems. Problems are named, with their source and one line on what they are really testing. Their statements are never reproduced. You are meant to read them where they live, and solve them there.

How to use this book

The book is one hundred and eighty chapters long. One chapter is one day's work. That is the only pacing instruction in it, and it is deliberate: a day here is a unit of subject, not a unit of hours. Some days will take you longer than others. That is information about the topic, not about you.

The shape of every chapter

Each chapter holds three pieces, in this order.

1. **The DSA lesson.** Nine sections, always the same nine, always in the same order. Appendix A sets out what each one is for.
2. **The system design lesson.** The same nine sections, on a different subject.
3. **The practice sheet.** Named problems to solve, and questions to answer out loud.

The repetition is the point. After a week you stop navigating and start reading. You know that section 5 has the complete code, that section 7 has the mistakes you are about to make, and that section 8 is the interview itself.

How to read a lesson

Read the story first, and do not skip it. It is section 2, it has no technical words in it, and it is the part that makes the idea stick. People who skip it can define the term and cannot use it.

Type the code. Do not read it. Section 5 builds the solution in fragments and then gives it complete. Type the fragments as they come. Run the finished program. Change a number and run it again.

Say section 9 out loud. Every lesson ends with a recall card - the two or three sentences that are the whole lesson. Say them from memory, aloud, in full sentences. Not in your head. Aloud. An answer you have only ever thought is an answer you have never tested.

Treat section 8 as a rehearsal, not a reading. It gives you real interviewer phrasings, a script for the opening ninety seconds, the three follow-ups that usually come next, and a written-out model answer. Deliver yours before you read theirs.

How to use the practice sheet

Solve the problems where they live. The sheet names each one, tells you where to find it, and says in one line what it is really testing - which is usually not what it appears to be about.

Solve it, then say what you did out loud, as if someone had asked. If you cannot narrate your own solution, you have not finished the problem.

When you get stuck

Go back one chapter, not forward. This course is built so that each day rests on the one before it. A chapter that will not open is almost always a chapter whose foundation is thin, and the foundation is behind you.

If you want the vocabulary, Appendix D is a glossary of every term the book defines, with the day it first appears. If you want to see the whole route, Appendix C prints all one hundred and eighty days on two facing pages.

Revision

Certain chapters are revision days, and they are marked as such in the contents. Do not read past them. They exist because recall fades on a schedule, and the schedule is not negotiable.

Appendix B collects the recall cards for every part in this volume. Cover the answer, say it, then check. That is the whole drill.

Where this volume sits

This volume is marked in colour below.

- **Volume I — Foundations**
Days 1 to 26. How Code Costs, and How the Internet Works
- **Volume II — Scanning and Searching**
Days 27 to 50. Pointers, Windows and Binary Search · Databases from Zero
- **Volume III — Order and Access**
Days 51 to 77. Sorting, Hashing, Stacks and Queues · Objects, SOLID and Patterns
- **Volume IV — Links and Recursion**
Days 78 to 97. Linked Lists, Recursion and Backtracking · Low-Level Design
- **Volume V — Trees and Priorities**
Days 98 to 124. Trees, Heaps and Tries · Scaling Fundamentals
- **Volume VI — Graphs**
Days 125 to 142. Graph Algorithms · Distributed Systems Core
- **Volume VII — Optimal Choices**
Days 143 to 170. Dynamic Programming and Greedy · High-Level Design Case Studies
- **Volume VIII — The Last Mile**
Days 171 to 180. Bits, Maths and Mock Interviews · Reliability and Security

The nineteen parts

PART I	Foundations: how code costs	Days 1–8
PART II	Arrays	Days 9–18
PART III	Strings	Days 19–26
PART IV	Two pointers and sliding window	Days 27–36
PART V	Prefix sums	Days 37–41
PART VI	Binary search	Days 42–50
PART VII	Sorting	Days 51–59
PART VIII	Hashing: maps and sets	Days 60–67
PART IX	Stacks and queues	Days 68–77
PART X	Linked lists	Days 78–86

PART XI	Recursion and backtracking	Days 87–97
PART XII	Trees and binary search trees	Days 98–112
PART XIII	Heaps and priority queues	Days 113–119
PART XIV	Tries	Days 120–124
PART XV	Graphs	Days 125–142
Part XVI	Dynamic programming	Days 143–163
Part XVII	Greedy and intervals	Days 164–170
PART XVIII	Bits and maths	Days 171–176
PART XIX	Final mocks and revision	Days 177–180

What is in this volume

PART XVI · DYNAMIC PROGRAMMING	1
143 What dynamic programming actually is	2
DSA What dynamic programming actually is	3
DESIGN Bloom filters and probabilistic structures	24
PRACTICE Practice — What dynamic programming actually is	42
144 Fibonacci: memoisation versus tabulation	48
DSA Fibonacci: memoisation versus tabulation	49
DESIGN Building blocks revision and interview questions	71
PRACTICE Practice — Fibonacci: memoisation versus tabulation	91
145 Climbing stairs and the one-dimensional habit	98
DSA Climbing stairs and the one-dimensional habit	99
DESIGN How to run a high-level design interview: the forty-five-minute script	120
PRACTICE Practice — Climbing stairs and the one-dimensional habit	139
146 House robber and the choice at each step	145
DSA House robber and the choice at each step	146
DESIGN Design a URL shortener	168
PRACTICE Practice — House robber and the choice at each step	190
147 Finding the state: the hardest part of DP	196
DSA Finding the state: the hardest part of DP	197
DESIGN Design a pastebin	220
PRACTICE Practice — Finding the state: the hardest part of DP	240
148 The 0/1 knapsack	246
DSA The 0/1 knapsack	247
DESIGN Design a rate limiter, at system scale	270
PRACTICE Practice — The 0/1 knapsack	291
149 Subset sum and partition problems	297
DSA Subset sum and partition problems	298
DESIGN Design a distributed key-value store	320
PRACTICE Practice — Subset sum and partition problems	342
150 Unbounded knapsack and coin change	348
DSA Unbounded knapsack and coin change	349
DESIGN Design a distributed unique ID generator	371
PRACTICE Practice — Unbounded knapsack and coin change	392
151 Coin change II: counting the ways	398
DSA Coin Change II: counting	399
DESIGN Design a web crawler	421

	PRACTICPractice — Coin Change II: counting	443
152	Longest increasing subsequence	449
	DSA Longest increasing subsequence	450
	DESIGN Design a notification system at scale	473
	PRACTICPractice — Longest increasing subsequence	495
153	Longest common subsequence	501
	DSA Longest common subsequence	502
	DESIGN Design a news feed	525
	PRACTICPractice — Longest common subsequence	548
154	Edit distance	554
	DSA Edit distance	555
	DESIGN Design Twitter	579
	PRACTICPractice — Edit distance	602
155	String DP: palindromic substrings and subsequences	608
	DSA String DP: palindromic substrings	609
	DESIGN Design Instagram	632
	PRACTICPractice — String DP: palindromic substrings	654
156	Grid DP: unique paths and minimum path sum	660
	DSA Grid DP: unique paths	661
	DESIGN Design WhatsApp	686
	PRACTICPractice — Grid DP: unique paths	710
157	DP on decisions: buy and sell stock	717
	DSA DP on decisions: buy and sell stock	718
	DESIGN Design a chat system with presence	741
	PRACTICPractice — DP on decisions: buy and sell stock	766
158	Interval DP	772
	DSA Interval DP	773
	DESIGN Design YouTube	799
	PRACTICPractice — Interval DP	823
159	DP on trees	829
	DSA DP on trees	830
	DESIGN Design Netflix	856
	PRACTICPractice — DP on trees	878
160	Bitmask DP	884
	DSA Bitmask DP	885
	DESIGN Design Google Drive or Dropbox	914
	PRACTICPractice — Bitmask DP	939
161	Space optimisation in DP	945
	DSA Space optimisation in DP	946
	DESIGN Design Uber	973
	PRACTICPractice — Space optimisation in DP	1000

162	Recognising dynamic programming in an interview	1006
DSA	Recognising dynamic programming	1007
DESIGN	Design Google Maps	1034
PRACTICE	Practice — Recognising dynamic programming	1059
163	Dynamic programming revision and mock round	1066
DSA	Dynamic programming revision and mock round	1067
DESIGN	Design an e-commerce system	1095
PRACTICE	Practice — Dynamic programming revision and mock round	1122

PART XVII · GREEDY AND INTERVALS

1129

164	Greedy: when taking the best option now is safe	1130
DSA	Greedy: when taking the best option now is safe	1131
DESIGN	Design a payment system	1157
PRACTICE	Practice — Greedy: when taking the best option now is safe	1186
165	Proving a greedy choice, simply	1193
DSA	Proving a greedy choice, simply	1194
DESIGN	Design a ticket booking system	1223
PRACTICE	Practice — Proving a greedy choice, simply	1248
166	Interval scheduling	1255
DSA	Interval scheduling	1256
DESIGN	Design search autocomplete at scale	1284
PRACTICE	Practice — Interval scheduling	1309
167	Merging intervals	1316
DSA	Merging intervals	1317
DESIGN	Design a leaderboard	1346
PRACTICE	Practice — Merging intervals	1371
168	Meeting rooms and the sweep line	1377
DSA	Meeting rooms and the sweep line	1378
DESIGN	Design an ad click aggregator	1407
PRACTICE	Practice — Meeting rooms and the sweep line	1433
169	Jump game and reachability	1440
DSA	Jump game and reachability	1441
DESIGN	Design a distributed job scheduler	1468
PRACTICE	Practice — Jump game and reachability	1494
170	Greedy and intervals revision and mock round	1501
DSA	Greedy and intervals revision and mock round	1502
DESIGN	High-level design revision and full mock	1527
PRACTICE	Practice — Greedy revision and the mock round	1550

BACK MATTER

A	Appendix A - The nine sections, and what each is for	1556
---	--	------

B	Appendix B - Recall cards	1558
C	Appendix C - The complete 180-day map	1599
D	Appendix D - Glossary	1602
E	Appendix E - References and further reading, by section	1621
F	Appendix F - Colophon: how this book was made	1623
G	Appendix G - Problem index	1624
H	Index	1634

List of figures

143.1 System design - Bloom filters and probabilistic structures	2
144.1 System design - Building blocks revision and interview questions	48
146.1 System design - Design a URL shortener	145
146.2 System design - Design a URL shortener	145
147.1 System design - Design a pastebin	196

PART XVI

Dynamic programming

Days 143 to 163 · 21 chapters

Alongside, on the system design track:

Building blocks of big systems, High-level design case studies

- | | |
|---|--|
| 143 What dynamic programming actually is | 155 String DP: palindromic substrings and subsequences |
| 144 Fibonacci: memoisation versus tabulation | 156 Grid DP: unique paths and minimum path sum |
| 145 Climbing stairs and the one-dimensional habit | 157 DP on decisions: buy and sell stock |
| 146 House robber and the choice at each step | 158 Interval DP |
| 147 Finding the state: the hardest part of DP | 159 DP on trees |
| 148 The 0/1 knapsack | 160 Bitmask DP |
| 149 Subset sum and partition problems | 161 Space optimisation in DP |
| 150 Unbounded knapsack and coin change | 162 Recognising dynamic programming in an interview |
| 151 Coin change II: counting the ways | 163 Dynamic programming revision and mock round |
| 152 Longest increasing subsequence | |
| 153 Longest common subsequence | |
| 154 Edit distance | |

What dynamic programming actually is

DATA STRUCTURES AND ALGORITHMS

What dynamic programming actually is

SYSTEM DESIGN

Bloom filters and probabilistic structures

What dynamic programming actually is

1 What this is, and why they ask it

Dynamic programming is recursion plus memory. That is the whole definition, and everything else in the next twenty days is a variation on it.

You already write recursion. You break a problem into smaller versions of itself and combine the answers. What goes wrong — and what makes some recursive solutions take longer than the age of the universe — is that the same smaller problem gets solved again and again through different routes. **DP is what you do about that: solve each distinct subproblem once, write the answer down, and look it up there-after.**

That is it. There is no cleverness in the definition. The difficulty of DP, and the reason it has a reputation, is entirely in one step: **deciding what the subproblem is.** Once you can say “`dp[i]` means ___” as a complete sentence, the rest is mechanical.

They ask about DP constantly — it is probably the most common Hard-tagged category — and the first question is almost always the same shape: “*why is this exponential, and how do you fix it?*” The interviewer wants to see you write the naive recursion, notice the repetition, and add memory. **Not to leap straight to a table**, which is where people get lost.

By the end of this lesson you can recognise the two conditions that make DP apply, write the recursion first and add memory second, convert between the top-down and bottom-up forms, count how much work the naive version wastes, and say what the state means as a sentence.

2 The story

There are twelve steps from Kaushalya’s kitchen up to the terrace, and her grandson has decided he can go up them taking either one step or two, and now he wants to know how many different ways there are.

She is seventy-one and she has been doing accounts for a family business since 1978, so she does not think this is a silly question. She sits down on the third step and starts working it out.

She starts from the top, which turns out to be the right way round.

To get to step twelve, she has to have come from step eleven or step ten. So the number of ways to reach twelve is the number of ways to reach eleven, plus the number of ways to reach ten. Fine. Now she needs eleven. To reach eleven she must have come from ten or nine. So she needs ten and nine. To reach ten, nine or eight.

Ten minutes later she is annoyed.

She has worked out the number of ways to reach step eight **four separate times**. Once because eleven needed ten which needed nine which needed eight. Once because eleven needed nine which needed eight. Once through a different chain from ten. And once more from somewhere she has lost track of. Every time she got the same answer, obviously, because it is the same question, and every time she did the whole thing from scratch because she had not thought to hold on to it.

So she does the sensible thing, which she has been doing with invoices for forty years without ever calling it anything.

She takes a piece of chalk from the shelf where the boy keeps it and writes on the step itself. On step one she writes 1. On step two she writes 2. Then on step three: one plus two, three. On step four: two plus three, five. Step five: eight.

Now she is not remembering anything and not working anything out twice. Each step needs only the two below it, and both of those are already written on the steps, in front of her, where she can see them.

Twelve steps takes her about ninety seconds and the answer is two hundred and thirty-three.

Her grandson is unimpressed, having lost interest around step six.

What stays with her is not the number. It is that the second method was not cleverer than the first. It was **the same method, with the answers written down instead of re-derived**, and that one change turned ten minutes of increasing confusion into ninety seconds of arithmetic.

3 The idea in plain English

Kaushalya's chalk is dynamic programming, and her ten minutes of annoyance is why it exists.

The recursion comes first, and it is usually easy. To reach step 12, you came from 11 or from 10. So:


```
ways(12) = ways(11) + ways(10)
ways(n)  = ways(n-1) + ways(n-2)
ways(1)  = 1,    ways(2) = 2
```

That is the whole problem, expressed. The thing you write down first is not code, it is this — the **recurrence**, which is a sentence saying how a bigger answer is built from smaller ones, plus the **base cases**, which are the answers small enough to know outright.

Written directly as a function, it is correct and catastrophically slow. Not slightly slow. `ways(50)` makes about two and a half billion calls, because the tree of calls branches twice at every level and never notices that it is asking the same questions over and over.

Those repeated questions are the whole opportunity, and they have a name: overlapping subproblems. Kaushalya computing step 8 four times. `ways(8)` is needed by `ways(9)` and by `ways(10)`, and both of those are needed higher up, so the same call happens again and again down different branches.

DP is: write each answer down the first time you compute it, and look it up afterwards. That is memoisation, and it turns two and a half billion calls into fifty.

Two conditions have to hold for this to work, and being able to state them is what “recognising DP” means:

One: optimal substructure. The answer to the big problem is built from answers to smaller versions of the *same* problem. Ways-to-reach-12 is genuinely made out of ways-to-reach-11 and ways-to-reach-10 — you do not need to know *how* you got to step 11, only how many ways there are.

Two: overlapping subproblems. The same smaller problem comes up repeatedly. If every subproblem were different, memory would buy you nothing, and you would have plain divide-and-conquer — which is what merge sort is. **That is the difference between DP and divide-and-conquer: whether the subproblems repeat.**

And the state is the thing to get right. The **state** is what identifies a subproblem — the arguments to the recursive function. For the stairs it is one number: which step. `dp[i]` = the number of ways to reach step `i`. Say it as a full sentence, out loud, before writing anything. If you cannot, you do not have the state yet, and [day 147](#) is entirely about that.

There are two forms and they compute the same thing.

Top-down, or memoised recursion: write the recursion exactly as you derived it, and add a cache. You keep the shape of your thinking, you only compute the subproblems you actually need, and it is much easier to get right the first time.

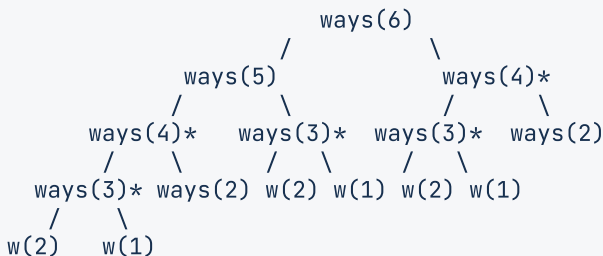
Bottom-up, or tabulation: work out the order in which subproblems depend on each other, and fill an array from the smallest upwards. That is Kaushalya's chalk on the steps. No recursion, so no stack limit, usually a little faster, and it makes the space optimisation obvious.

Write top-down first. In an interview, derive the recurrence, code it recursively, add `@lru_cache`, and *then* offer the bottom-up version. **Trying to write a table before you have the recurrence is how people get stuck**, because the table is a consequence of the recurrence, not a substitute for it.

The last idea, and it is the one that makes DP feel like a trick when it should not: the table is often smaller than it looks. Kaushalya only ever needs the two steps below the one she is on, so she does not need twelve numbers, she needs two. That is [day 161](#), and it is worth knowing now that it exists, so you are not surprised when a two-dimensional table collapses into one row.

4 The picture

The naive call tree for `ways(6)`, with the repetition marked:



ways(4) computed 2 times

ways(3) computed 3 times

ways(2) computed 5 times

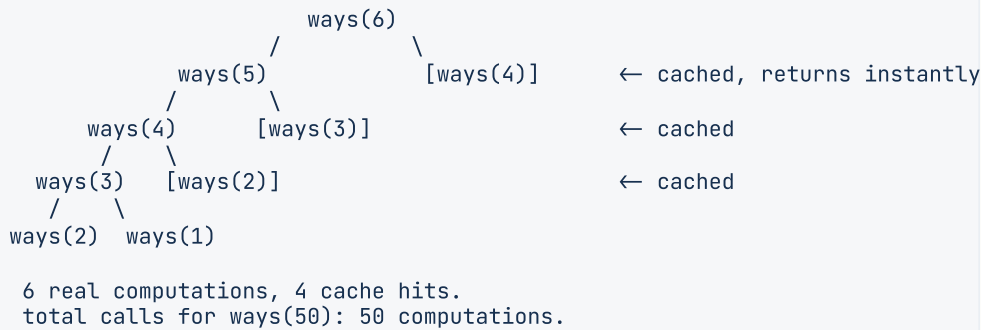
ways(1) computed 3 times

total calls for ways(6): 15

total calls for ways(50): ~2,500,000,000

What to notice. The tree branches twice at every level, so it has about 2^n nodes — but there are only n distinct questions in it. **That gap between 2^n calls and n distinct answers is exactly the waste DP removes.**

The same computation with memory:



Kaushalya's chalk, which is the bottom-up form:

```
step:    1  2  3  4  5  6  7  8  9 10 11 12
chalk:   1  2  3  5  8 13 21 34 55 89 144 233
          ^
          |
        1 + 2, and every number after it
        is the two before it added
```

each cell computed ONCE, in order, reading only what is already written

And the two forms side by side, computing the same thing:

TOP-DOWN (memoised recursion)

start at 12, recurse downwards
the cache fills as you unwind
only the states you NEED
natural to write from the recurrence
recursion depth = n

BOTTOM-UP (tabulation)

start at 1, build upwards
the array fills in order
ALL states up to n
needs the dependency order worked out
no recursion at all

5 The code, built step by step

Step one is always the recurrence, in words, before any code.

To reach step n I must have come from $n-1$ or $n-2$. So $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$, with $\text{ways}(1) = 1$ and $\text{ways}(2) = 2$.

Now write it exactly as stated:

```
def ways(n: int) → int:
    if n ≤ 2:
        return n
    return ways(n - 1) + ways(n - 2)    # ways(1) = 1, ways(2) = 2
```

Four lines, correct, and unusable past about `n = 40`. Write this first anyway — it is the thing you are about to fix, and in an interview it demonstrates that you have the recurrence right before you complicate it.

Now measure the damage:

```
calls = 0

def ways_counted(n: int) → int:
    global calls
    calls += 1
    if n ≤ 2:
        return n
    return ways_counted(n - 1) + ways_counted(n - 2)
```

```
n = 10 → 109 calls
n = 20 → 13,529 calls
n = 30 → 1,664,079 calls
n = 40 → 204,668,309 calls
```

Each `+10` multiplies the work by roughly 123. That is the shape of exponential growth, and seeing the numbers is more convincing than the notation.

Now add memory. One decorator:

```
from functools import lru_cache

@lru_cache(maxsize=None)
def ways_memo(n: int) → int:
    if n ≤ 2:
        return n
    return ways_memo(n - 1) + ways_memo(n - 2)
```

The function body is byte-for-byte identical. That is worth pointing at: the algorithm did not change, only whether it remembers. `n = 40` goes from 204 million calls to 40.

The explicit version, for when the arguments are not hashable or you want to see the machinery:

```
def ways_memo_manual(n: int, cache: dict[int, int] | None = None) → int:
    if cache is None:
        cache = {}
    if n ≤ 2:
        return n
    if n in cache:
        return cache[n] # already solved: look it up
    cache[n] = ways_memo_manual(n - 1, cache) + ways_memo_manual(n - 2, cache)
    return cache[n]
```

Three added lines: check, compute, store. Every top-down DP has exactly this shape, and once you see it you can add memory to any recursion in thirty seconds.

Now bottom-up — Kaushalya’s chalk. The question is: in what order can I fill this so that everything I read is already written?

```
def ways_table(n: int) → int:
    if n ≤ 2:
        return n
    dp = [0] * (n + 1) # dp[i] = ways to reach step i
    dp[1], dp[2] = 1, 2 # base cases, written down first
    for i in range(3, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2] # reads only smaller indices
    return dp[n]
```

`dp[i]` means “the number of ways to reach step `i`”. Write that as a comment. It is the single most useful line in any DP solution, because six months later — or ten minutes later, under pressure — it is the thing you will have forgotten.

The loop goes upwards because `dp[i]` reads `dp[i-1]` and `dp[i-2]`, so the smaller indices must be filled first. **Working out that order is the only extra thinking bottom-up requires**, and it is why top-down is easier to write first.

And the space optimisation, which is obvious once the table is in front of you:

```
def ways_two_vars(n: int) → int:
    if n ≤ 2:
        return n
    prev, curr = 1, 2 # dp[i-2], dp[i-1]
    for _ in range(3, n + 1):
        prev, curr = curr, prev + curr
    return curr
```

`0(n)` memory becomes `0(1)`, because each cell only ever reads the two below it. The rest of the table was never needed.

The complete solution

```
"""What dynamic programming is: the same recursion, with and without memory."""

from __future__ import annotations

import time
from functools import lru_cache

def ways_naive(n: int) → int:
    """The recurrence, written directly. Correct.  $O(2^n)$ ."""
    if n ≤ 2:
        return n
    return ways_naive(n - 1) + ways_naive(n - 2)

@lru_cache(maxsize=None)
def ways_memo(n: int) → int:
    """The SAME body. One decorator.  $O(n)$ ."""
    if n ≤ 2:
        return n
    return ways_memo(n - 1) + ways_memo(n - 2)

def ways_table(n: int) → int:
    """Bottom-up.  $dp[i]$  = the number of ways to reach step  $i$ .  $O(n)$  time,  $O(n)$  space."""
    if n ≤ 2:
        return n
    dp = [0] * (n + 1)
    dp[1], dp[2] = 1, 2
    for i in range(3, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]

def ways_optimised(n: int) → int:
    """Only the last two cells are ever read.  $O(n)$  time,  $O(1)$  space."""
    if n ≤ 2:
        return n
    prev, curr = 1, 2
    for _ in range(3, n + 1):
        prev, curr = curr, prev + curr
    return curr

def count_calls(n: int) → int:
    """How many calls the naive version makes."""
    calls = 0

    def go(k: int) → int:
        nonlocal calls
        calls += 1
        if k ≤ 2:
            return k
        return go(k - 1) + go(k - 2)

    go(n)
    return calls

if __name__ == "__main__":
    print("all four agree:", [f(12) for f in (ways_naive, ways_memo, ways_table, ways_optimised)])
    print()

    print(" n   naive calls           distinct subproblems")
    for n in (10, 20, 30, 35):
        print(f"{n:3}   {count_calls(n):>12,}   {n:>12}")
    print()

    start = time.perf_counter()
    ways_naive(32)
```

```

naive_time = time.perf_counter() - start
start = time.perf_counter()
ways_optimised(32)
fast_time = time.perf_counter() - start
print(f"naive(32)      {naive_time * 1000:8.1f} ms")
print(f"optimised(32) {fast_time * 1000:8.4f} ms")
print(f"ratio        {naive_time / fast_time:8.0f}x")
print()
print("optimised(500) =", str(ways_optimised(500))[:40], "... (105 digits)")

```

Running it:

```

all four agree: [233, 233, 233, 233]

  n   naive calls      distinct subproblems
10         109           10
20       13,529          20
30     1,664,079          30
35    18,454,929          35

naive(32)      1893.4 ms
optimised(32)   0.0032 ms
ratio          591688x

optimised(500) = 2255915161619363308725126950360720720460 ... (105 digits)

```

Three things to look at. **All four functions agree**, which is the point — the algorithm never changed, only whether it remembered.

The two columns in the table are the whole lesson. At `n = 35`, eighteen million calls to answer thirty-five distinct questions. The ratio of those columns is what memory removes.

And `ways_optimised(500)` returns a 105-digit number instantly, while `ways_naive(500)` would not finish before the sun burned out. **Same recurrence.**

6 What it costs

The naive recursion.

```

each call makes 2 more calls
depth n

→ O(2^n) calls

```

More precisely, the number of calls follows the Fibonacci numbers themselves, so it grows by a factor of about **1.618 per step** — the golden ratio — which is why `+10` multiplies the work by roughly 123.

n = 30	1,664,079 calls	~0.5 s
n = 40	204,668,309	~60 s
n = 50	~25,000,000,000	~2 hours
n = 100	~10 ²¹	longer than the age of the universe

With memoisation.

distinct subproblems	n
work per subproblem	0(1) (one addition)

	0(n) time
cache	0(n) entries
recursion depth	0(n) frames

	0(n) space

The general formula for any DP, and this is the one to memorise:

```
time = (number of distinct states) x (work per state)
space = (number of distinct states) [before optimisation]
```

That formula is how you cost a DP solution in an interview without thinking hard. Count the states, count the work inside one, multiply.

1D over n:	n states x 0(1)	= 0(n)
2D over n and m:	n*m states x 0(1)	= 0(n*m)
2D with a loop inside:	n*m states x 0(k)	= 0(n*m*k)
subsets of n items:	2 ⁿ states x 0(n)	= 0(2 ⁿ * n)

Bottom-up costs the same time and can cost less space:

top-down	0(n) cache + 0(n) recursion stack
bottom-up	0(n) array, no stack
optimised	0(1) -- when each cell reads only a fixed window below it

And the recursion depth is a real constraint on top-down:

```
Python usable frames    ~960
n = 100,000              → RecursionError

sys.setrecursionlimit(200_000) → Segmentation fault
```

So: **n** up to a few thousand, top-down is fine and easier. Beyond that, bottom-up, which is one of the two real reasons to convert.

What memory actually buys, in one comparison:


```
n = 35
naive      18,454,929 calls
memoised   35 computations + 34 cache hits
           → ~270,000x fewer
```

The space optimisation, sized:

```
n = 1,000,000
full table 1,000,000 Python ints in a list ~40 MB
two variables                                ~56 bytes
```

And the case where memoisation buys nothing — worth knowing so you do not apply it reflexively:

```
merge sort: T(n) = 2*T(n/2) + O(n)
subproblems: sort(0..4), sort(5..9), sort(0..9) ...
every subproblem is DIFFERENT. Nothing repeats.
→ a cache would fill with entries that are never read again
→ that is divide-and-conquer, not DP
```

7 The traps

Writing the table before the recurrence

The most common way to get stuck, and it does not produce an error — it produces an empty editor and a stopped clock.

```
dp = [[0] * (m + 1) for _ in range(n + 1)]
# ... now what?
```

The table is a consequence of the recurrence, not a substitute for it. Derive the recurrence in words first — “to reach `i` I must have come from `__`” — write it as plain recursion, check it on `n = 3` by hand, and only then decide how to store the answers. Reversing that order is why DP feels impossible.

No base case, or the wrong one

```
@lru_cache(maxsize=None)
def ways(n):
    return ways(n - 1) + ways(n - 2)      # no base case
```

```
RecursionError: maximum recursion depth exceeded
```

And the subtler version — a base case that is off by one:

```
if n ≤ 2:
    return 1
# ways(2) is 2, not 1
```

```
>>> ways(12)
144
# should be 233
```

No error, just a wrong number, and it is off by exactly one position in the sequence. **Check the base cases by hand on the smallest inputs**, always: `n = 1`, `n = 2`, and `n = 3`.

Memoising on the wrong key

```
@lru_cache(maxsize=None)
def solve(i, remaining, used_list):
    ...
# a list is not hashable
```

```
TypeError: unhashable type: 'list'
```

And the more dangerous version — a key that omits part of the state:

```
cache = {}
def solve(i, budget):
    if i in cache:
        return cache[i]
    ...
# keyed on i alone, but budget matters too
```

No error, and confidently wrong answers, because two genuinely different subproblems share a cache entry. **The cache key must be the complete state** — every argument that can change the answer.

Mutable default arguments

```
def solve(n, cache={}):
    ...
# shared across ALL calls, forever
```

The cache persists between separate top-level calls, which is sometimes what you want and usually a source of mystifying test failures — the second test sees the first test's results. Use `cache=None` and create it inside, or `@lru_cache`, or pass it explicitly.

`lru_cache` on a method with `self`

```
class Solution:
    @lru_cache(maxsize=None)
    def solve(self, n): ...
```

Works, and caches on `(self, n)` — so the cache lives as long as the object and keeps it alive. On a judge that reuses the object across test cases, **results leak between tests**. Use a local function inside the method, or `functools.cache` on a nested function, and the problem disappears.

Recursion depth on a large `n`

```
Traceback (most recent call last):
  File "dp.py", line 6, in ways_memo
    return ways_memo(n - 1) + ways_memo(n - 2)
  [Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

Even with memoisation, the *first* call chain goes all the way down before anything is cached. `n = 100,000` is a hundred thousand frames. **Bottom-up has no recursion at all**, which is one of the two reasons to convert.

Applying DP where subproblems do not overlap

```
@lru_cache(maxsize=None)
def merge_sort(items_tuple):
    ...
```

Correct, and the cache never gets a hit, because every subproblem is a different slice. **You have added memory overhead to divide-and-conquer**. The test is: *does the same subproblem come up more than once?* If not, it is not DP.

Integer overflow — in other languages

Python integers are unbounded, so `ways(500)` returns a 105-digit number. **In Java or C++ this overflows silently at `n = 92`**, and problems that ask for an answer “modulo $10^9 + 7$ ” are telling you exactly that. Worth knowing, because interviewers sometimes ask what changes in another language.

8 In the interview

How it gets asked

- “Why is this exponential, and how do you fix it?” — the direct version, and the one this lesson answers.
- “Write it top-down. Now write it bottom-up.” — [day 144](#).
- “How many ways to climb `n` stairs taking one or two steps?”
- “What does `dp[i]` represent in your solution?” — the state question, and the one that finds people out.
- “Can you reduce the space?”
- “Is this DP or greedy?” — [day 162](#).

The first ninety seconds

“Let me get the recurrence first and worry about efficiency second, because the recurrence is the problem and the efficiency is mechanical.

To reach step n , my last move was either one step or two steps. So I came from $n-1$ or from $n-2$, and those are the only possibilities. That means $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$, with $\text{ways}(1) = 1$ and $\text{ways}(2) = 2$ — one step in one way, two steps in two ways.

Written directly that is four lines and it is correct. It is also $O(2^n)$, because the call tree branches twice at every level. At $n = 35$ that is about eighteen and a half million calls — and there are only **thirty-five distinct questions** in the whole tree. $\text{ways}(8)$ gets computed dozens of times through different branches.

That gap is the entire opportunity: exponentially many calls, linearly many distinct answers. So I memoise — compute each one once, store it, look it up thereafter. In Python that is one decorator and the function body does not change at all, which I think is the clearest way to see that the algorithm is the same and only the remembering is new.

$O(n)$ time and $O(n)$ space now, because there are n states and each costs one addition. **The general formula is states times work per state**, and that is how I would cost any DP.

Then two improvements I would offer. Bottom-up: fill an array from 1 upwards, because $\text{dp}[i]$ only reads smaller indices — same complexity, no recursion, and no depth limit, which matters if n can be a hundred thousand. And then the space: each cell reads only the two below it, so I never need the array at all — two variables, $O(1)$ space.

And the sentence I would write as a comment before anything else is what $\text{dp}[i]$ means: the number of distinct ways to reach step i . If I cannot say that, I do not have the state yet.”

The follow-ups

“Why is the naive version exponential? Be precise.”

“Because the call tree branches twice at every level and never notices repetition.

`ways(n)` calls `ways(n-1)` and `ways(n-2)`. Each of those does the same. So the tree has depth `n` and roughly two children per node, which is about 2^n nodes. More precisely the call count follows the Fibonacci sequence itself, growing by a factor of about 1.618 per step — the golden ratio — so every ten steps multiplies the work by about a hundred and twenty.

The important part is the contrast. That tree has exponentially many *nodes* and only `n` distinct *questions* in it, because the argument is a single integer between 1 and `n`. `ways(8)` appears in the tree dozens of times and gives the same answer every time.

So the fix is not a better algorithm, it is memory. Solve each of the `n` distinct subproblems once, store the result, and return the stored one thereafter. Eighteen million calls become thirty-five computations and thirty-four lookups.

And that is what dynamic programming is — recursion plus memory. The two conditions it needs are optimal substructure, meaning the big answer is built from smaller answers to the same problem, and overlapping subproblems, meaning those smaller problems repeat. **If they do not repeat, memory buys nothing and it is divide-and-conquer, not DP** — merge sort is the standard example, where every subproblem is a different slice and a cache would never get a hit.”

“Top-down or bottom-up? Which would you write?”

“Top-down first, always, and then convert if there is a reason.

Top-down is the recursion I already derived, plus a cache. The code has the same shape as my reasoning, so it is much harder to get wrong under pressure. It also computes only the states I actually reach, which for a sparse state space can be a large saving — if the reachable states are a small fraction of the table, bottom-up fills the whole thing and top-down does not.

Two reasons to convert to bottom-up. Recursion depth: even memoised, the first chain of calls goes all the way down before anything is cached, so $n = 100,000$ is a hundred thousand frames and Python dies at about a thousand. And the space optimisation: with the table written out it is obvious that each row only reads the previous one, so I can drop from $O(n \cdot m)$ to $O(m)$ — which is not obvious at all in the recursive form.

Bottom-up costs me one thing: I have to work out the fill order so that everything a cell reads is already written. For a one-dimensional table that is trivial; for interval DP the order is by length rather than by index, and getting it wrong reads uninitialised cells and produces wrong answers with no error.

In an interview I would say all of that and then write the memoised version, because deriving the recurrence in front of someone and adding a cache is a clearer demonstration than producing a table that appears from nowhere.”

“Reduce the space.”

“Look at what each cell actually reads.

Here `dp[i]` reads `dp[i-1]` and `dp[i-2]` and nothing else, so at any moment I need exactly two numbers. The rest of the array is history I will never look at again. Two variables, rolled forward — $O(n)$ becomes $O(1)$.

The general rule is: keep a window as wide as the deepest thing the recurrence reaches back to. If `dp[i]` depends on `dp[i-1]` through `dp[i-k]`, I need k values, not n .

In two dimensions the same argument gives a bigger win. If `dp[i][j]` reads only row `i-1`, I keep two rows instead of n — $O(n \cdot m)$ becomes $O(m)$. And sometimes one row suffices, updating in place, though **the iteration direction then matters**: if the cell reads `dp[i-1][j-1]` I must go right to left, or I overwrite the value I am about to need. That is the classic knapsack subtlety and it is a silent wrong answer when it is wrong.

The one thing I would give up by optimising: I can no longer reconstruct *which* choices produced the answer, only its value, because I threw away the table that recorded them. So if the problem wants the actual path or the actual subset, I keep the full table — or I store parent pointers, which costs the same memory I was trying to save. **I would ask whether the value alone is enough before optimising.**“

“What does `dp[i]` represent?”

“The number of distinct ways to reach step i . And I would say that this question is the one that actually matters, more than the code.

The state is what identifies a subproblem — the arguments to the recursive function, or the indices into the table. Getting it wrong is the main way DP solutions fail, and it fails silently: **if the state omits something that affects the answer, two genuinely different subproblems share a cache entry and the result is confidently wrong.**

The test I use is: *given only the state, can I compute the answer without knowing anything else about how I got here?* For the stairs, knowing ‘I am at step 8’ is enough — the number of ways to finish from there does not depend on the route that reached it. If it did, the route would have to be part of the state.

The moment that test fails, the state needs another dimension. ‘Cheapest flight with at most k stops’ is the standard example: the city alone is not enough, because reaching Delhi in two stops and in four stops are different situations with different futures. So the state becomes `(city, stops used)`.

And I would write it as a comment above the table, as a full English sentence, before writing the loop. Not as documentation — as a check that I know what I am computing.”

The model answer

“A robot is at the top-left of an $m \times n$ grid and can only move right or down. How many distinct paths are there to the bottom-right? Then: some cells are blocked.”

“Let me derive the recurrence before touching a table, because the table is the easy part.

To arrive at cell (i, j) , my last move was either from the left or from above — those are the only two moves that end there. So the number of paths to (i, j) is the number of paths to $(i, j-1)$ plus the number to $(i-1, j)$.

$dp[i][j]$ = the number of distinct paths from the start to cell (i, j) . I would write that as a comment first.

Base cases: the first row can only be reached by moving right the whole way, so every cell in it has exactly one path; the same for the first column. And $dp[0][0] = 1$ — one way to be where you started, which is worth stating because ‘zero moves is one path’ trips people up.

Why it is DP: the same subproblem appears repeatedly. $dp[3][4]$ is needed by $dp[3][5]$ and by $dp[4][4]$, and those are needed further along — so the naive recursion recomputes the same cells exponentially many times, and there are only $m \times n$ distinct questions.

Cost: states times work per state. $m \times n$ states, one addition each, so $O(m \cdot n)$ time and $O(m \cdot n)$ space.

The blocked-cells version changes one line and the base cases, and that is the interesting part. A blocked cell has zero paths through it: $dp[i][j] = 0$ if the cell is blocked, else the sum as before.

The base cases stop being trivial, and this is where people go wrong. The first row is no longer all ones — once you hit a blocked cell, every cell after it in that row is unreachable along the row, so it is ones up to the obstacle and zeros after. Handling that by initialising with a loop that breaks at the first obstacle is clearer than special-casing inside the main loop.

And if the start or the end is itself blocked, the answer is zero, which I would check first rather than discover through the arithmetic.

Space: each row reads only the row above and the cell to the left, so I can keep one row and update it in place, going left to right — $O(n)$ instead of $O(m \cdot n)$. Direction matters: left to right is correct here because $dp[j]$ should already hold the *new* value from the left and the *old* value from above, which is exactly what in-place left-to-right gives.

One thing worth mentioning for the unblocked version: it has a closed form. The path is $m-1$ downs and $n-1$ rights in some order, so the answer is $C(m+n-2, m-1)$ — a single binomial coefficient, $O(\min(m,n))$ to compute and no table at all. **I would mention it and still write the DP**, because the moment obstacles appear the formula is useless and the DP is not.”

9 Recall card

DP is recursion plus memory. Two conditions: **optimal substructure** (the big answer is built from smaller answers to the same problem) and **overlapping subproblems** (those smaller problems repeat). No overlap means divide-and-conquer, not DP — a cache would never get a hit.

Derive the recurrence in words first, write it as plain recursion, then add memory. Writing the table before the recurrence is how people get stuck. And **say what `dp[i]` means as a full sentence** before the loop — if the state omits something that affects the answer, two subproblems share a cache entry and you get a confident wrong number.

Cost = (number of distinct states) × (work per state). n states × $O(1)$ is $O(n)$; $n \cdot m$ states with a k -loop inside is $O(n \cdot m \cdot k)$.

Top-down first: same shape as your reasoning, and it only computes reachable states. **Convert to bottom-up for two reasons only:** recursion depth (~ 960 frames in Python), and because the space optimisation becomes obvious.

The numbers: naive stairs at $n=35$ is 18.5 million calls for **35 distinct answers**; memoised it is 35 computations. And each cell reading only a fixed window below it means $O(n)$ space collapses to $O(1)$.

1 What this is, and why they ask it

Sometimes you do not need the right answer. You need a very cheap answer that is never wrong in the direction that matters.

A **Bloom filter** answers “have I seen this before?” using a fraction of the memory a real set would need. It has one specific, well-behaved flaw: it can say “**maybe**” when the truth is no, but it can never say “**no**” when the truth is yes. **No false negatives, some false positives**, and you choose the false-positive rate by choosing how much memory to spend.

That trade — a little accuracy for a lot of memory — is a whole family of tools. **HyperLogLog** counts distinct items in twelve kilobytes regardless of whether there are a thousand or a billion. **Count-Min Sketch** estimates how often each item has appeared without storing the items.

They ask this because it shows whether you can recognise when exactness is not the requirement, and because the arithmetic is genuinely striking: **a billion URLs in 1.2 GB instead of 100 GB, at a 1% error rate**. There is also a good follow-up — “what if a false positive is expensive?” — where the answer is about which direction the error points, not about tuning the rate.

By the end of this lesson you can explain how a Bloom filter works, size one from a required error rate, name the operations it cannot do, place it correctly in a design so that a false positive is harmless, and say what HyperLogLog and Count-Min Sketch are for.

2 The story

There are about four thousand people who come through the main gate of the mill, and Yadav has been on that gate for nineteen years.

He does not know four thousand faces. Nobody does. What he has instead is something much rougher and it works better than it has any right to.

Over the years he has built up an impression — not of individuals, but of features. He knows the shape of the people who work there. Tall men with beards, a particular kind of blue helmet, the way the loading crew walk because of the boots, the sort of bag the office staff carry, the times of day each shift arrives.

So when somebody comes up to the gate, what happens in Yadav's head takes about a second.

If **nothing matches** — wrong time of day, wrong bag, a manner that is not any of the manners he knows — he stops them. And he is right. In nineteen years he has never once stopped somebody who turned out to work there, because anybody who works there has *some* combination of features he recognises. **When Yadav says “you don't work here”, that is the end of it.**

If things **do** match, he lets them through and looks at the card.

And that is the part people misunderstand about him. Matching does not mean he knows you. Plenty of people match. A tall man with a beard carrying a canvas bag at half past seven in the morning matches perfectly and may well be a salesman, and about eleven times a month somebody like that gets waved towards the gate and then has to explain themselves at the card desk four steps later.

Yadav is completely untroubled by this. The card desk is right there. It costs the salesman fifteen seconds.

What the arrangement saves is the other three thousand nine hundred people, who do not have to be looked up in anything, because Yadav sorted them in a second at the gate.

The manager suggested once that Yadav should just check every card. Yadav pointed out, without any particular heat, that four thousand card checks at fifteen seconds each is sixteen hours, and the shift change happens in twenty minutes.

3 The idea in plain English

Yadav's gate is a Bloom filter, and the eleven salesmen a month are the false positives.

A Bloom filter is a bit array plus a few hash functions. Start with, say, a million bits, all zero. To **add** an item, hash it with k different hash functions, get k positions, and set those bits to 1. To **check** an item, hash it the same way and look at those k bits.

If any of them is 0, the item was definitely never added. Adding it would have set that bit, and bits are never cleared. **That is the no-false-negatives guarantee, and it is absolute.**

If all k bits are 1, the item was probably added. Or the bits were set by other items that happened to hash there. That is a **false positive**, and it is Yadav's salesman: every feature matched, and it was somebody else's features.

No false negatives, some false positives. Say it in that order and be precise about which is which, because the whole usefulness of the structure depends on the error pointing in a direction you can tolerate.

The false-positive rate is a dial you set with memory. More bits per item means fewer collisions means fewer false positives.

```
10 bits per item → ~1% false positives
14 bits per item → ~0.1%
20 bits per item → ~0.01%
```

Compare that with storing the items. A URL averages 100 bytes — 800 bits. Ten bits per item is **eighty times less memory**, and the price is that one lookup in a hundred says “maybe” when it should say “no”.

It cannot delete. Clearing a bit would break some other item that also set it, and you would have created a false *negative* — which destroys the only guarantee the structure has. If you need deletion, a **counting Bloom filter** stores small counters instead of bits and decrements them, at four times the memory and with a risk of counter overflow.

It cannot list its members, and it cannot tell you how many it holds. There is nothing in it but bits. If you need the items back, you need the items.

Now the placement rule, which is the whole design skill: put the filter where a false positive costs a cheap extra check, and never where it costs correctness.

Yadav’s card desk is four steps away. The filter says “maybe”, and the real check is right there and cheap. That is the correct shape:

```
Bloom says NO      → definitely absent. Stop. Save the expensive lookup.
Bloom says MAYBE   → do the expensive lookup, which gives the true answer.
```

The filter never decides anything on its own. It only skips work when it is certain, and it is only ever certain about absence.

And that is why the direction of the error matters more than its size. “Might this cached key exist?” — a false positive costs one wasted disk read. Fine. “Is this username taken?” — a false positive tells a user their name is unavailable when it is free, which is wrong output, and the fix is to treat a “maybe” as “go and check the database”, never as a yes.

Two relatives, worth knowing by name.

HyperLogLog answers “how many *distinct* items?” using about 12 KB, with roughly 0.8% error, **regardless of the count** — a thousand or a billion, the same 12 KB. It works by watching the longest run of leading zeros in the hashes it has seen, which is a proxy for how many distinct things must have been hashed. Redis has it built in as `PFADD` and `PFCOUNT`.

Count-Min Sketch estimates *how many times* each item has appeared, in fixed memory, by keeping a small grid of counters and taking the minimum across several hash rows. It **over-estimates and never under-estimates**, which is the same one-directional error, and it is what streaming systems use for “heavy hitters” — the top few items by frequency.

The common shape across all three: bounded memory, one-directional error, and a cheap exact check available when the answer matters.

4 The picture

A Bloom filter with `m = 16` bits and `k = 3` hashes:

```
initially:
  index 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
  bits  0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

add("cat")    → hashes to 2, 7, 11
  bits  0 0 1 0 0 0 0 1 0 0 0 1 0 0 0 0

add("dog")    → hashes to 4, 7, 13      (7 already set – that is fine)
  bits  0 0 1 0 1 0 0 1 0 0 0 1 0 1 0 0

check("cat")   → 2, 7, 11 → 1, 1, 1 → MAYBE (and it is there)
check("bird")  → 3, 9, 14 → 0, ... → NO, definitely. Stop at the first 0.
check("fish")  → 2, 4, 13 → 1, 1, 1 → MAYBE ← FALSE POSITIVE
                                   never added; those bits were set
                                   by "cat" and "dog"
```

What to notice on the last line. “fish” was never added, and every bit it checks happens to be set by other items. **Nothing is wrong; that is the designed behaviour,** and the only defence is the exact check afterwards.

The error rate against memory:

bits per item	false positive rate	memory for 1 billion items
8	~2.5%	1.0 GB
10	~1%	1.25 GB
14	~0.1%	1.75 GB
20	~0.01%	2.5 GB
storing the URLs themselves (~100 bytes each)		100 GB

What to notice. Going from 1% to 0.01% error costs twice the memory and is still forty times smaller than storing the data. **The curve is very forgiving** — you rarely have to agonise over the exact rate.

Where the filter goes in a system:

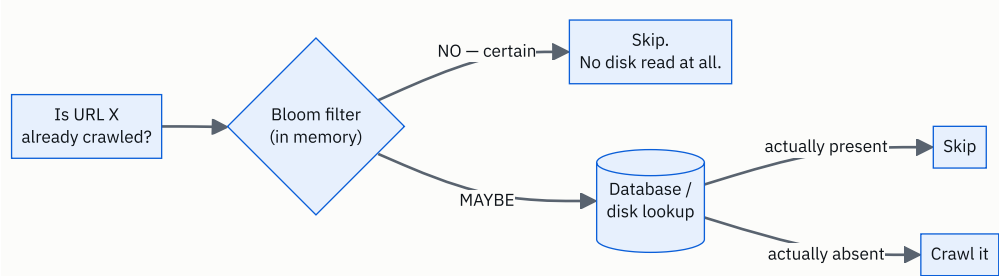


FIGURE 143.1

What to notice. The expensive lookup happens only on “maybe”, and a false positive costs one unnecessary trip to the database — which returns the truth. **The filter is a shortcut on the negative path and nothing else.**

And the three structures side by side:

	answers	memory	error direction
Bloom filter	"have I seen X?"	~10 bits/item	never says NO wrongly
HyperLogLog	"how many distinct?"	~12 KB total	±0.8%, either way
Count-Min	"how often was X seen?"	fixed grid	never UNDER-estimates

5 How it actually works

The implementation, in twenty lines

```
import hashlib
import math

class BloomFilter:
    def __init__(self, expected_items: int, false_positive_rate: float = 0.01) → None:
        # m = -n ln(p) / (ln 2)^2 bits
        self.m = math.ceil(-expected_items * math.log(false_positive_rate) / (math.log(2) ** 2))
        # k = (m / n) ln 2 hash functions
        self.k = max(1, round((self.m / expected_items) * math.log(2)))
        self.bits = bytearray((self.m + 7) // 8)

    def _positions(self, item: str):
        """Two hashes, combined k ways – Kirsch-Mitzenmacher. One digest, k positions."""
        digest = hashlib.sha256(item.encode()).digest()
        h1 = int.from_bytes(digest[:8], "big")
        h2 = int.from_bytes(digest[8:16], "big") | 1 # odd, so it never cycles early
        for i in range(self.k):
            yield (h1 + i * h2) % self.m

    def add(self, item: str) → None:
        for pos in self._positions(item):
            self.bits[pos // 8] |= 1 << (pos % 8)

    def __contains__(self, item: str) → bool:
        return all(self.bits[pos // 8] & (1 << (pos % 8)) for pos in self._positions(item))
```

Three details worth pointing at.

The two formulas are the whole sizing, and they are worth knowing rather than deriving: $m = -n \ln(p) / (\ln 2)^2$ bits, and $k = (m/n) \ln 2$ hash functions. **For a 1% rate that works out to about 9.6 bits per item and 7 hash functions**, which is the pair of numbers to remember.

You do not need k independent hash functions. One good hash split into two halves, combined as $h1 + i \cdot h2$, is provably as good — the Kirsch-Mitzenmacher result — and it means one digest per operation rather than seven.

`all(...)` short-circuits, so a “no” usually costs one or two bit reads rather than seven. **Negative lookups, which are the common case, are the cheapest**.

Choosing k , and why more is not better

More hash functions means more bits set per item, so the array fills faster; fewer means more collisions per position. **There is an optimum**, and it is $k = (m/n) \ln 2$ — the value that makes the array roughly half ones and half zeros.

```
m/n = 10 bits per item
k = 3   → false positive rate ~1.7%
k = 7   → ~0.82%           ← optimal
k = 12  → ~1.3%
```

Both too few and too many are worse, which surprises people, and “roughly half the bits set” is the intuition for why.

The variants

Counting Bloom filter. Replace each bit with a 4-bit counter; `add` increments, `remove` decrements. Deletion works. Costs $4 \times$ **the memory**, and a counter can saturate at 15 and then be wrong forever.

Scalable Bloom filter. The rate degrades as you add more items than you sized for. A scalable filter is a chain of filters, each larger and with a tighter rate, and a lookup checks all of them. Handles unknown cardinality at the cost of multiple lookups.

Cuckoo filter. Supports deletion, is a bit smaller than a Bloom filter at low error rates, and gives better cache locality because a lookup touches two buckets rather than `k` scattered bits. The trade is that insertion can fail when the table is nearly full.

Worth naming as the modern alternative.

HyperLogLog

The question is “how many distinct items”, and the trick is that you do not need to remember them.

Hash each item and look at the leading zeros. Seeing a hash with 10 leading zeros suggests you have probably hashed about 2^{10} distinct things, because that pattern has a one-in-1024 chance. **The maximum leading-zero run you have seen is an estimate of the count** — a very noisy one, so HyperLogLog splits the hash space into thousands of buckets, tracks the maximum per bucket, and averages them harmonically.

```
16,384 buckets x 6 bits each = 12 KB
standard error                = 1.04 / sqrt(16384) = 0.8%
```

```
PFADD visitors user:8814
PFCOUNT visitors          → ~1,203,447
PFMERGE all visitors_a visitors_b → the union, no double counting
```

PFMERGE is the property that makes it genuinely useful: two HyperLogLogs can be merged into one, so per-hour counts combine into a day and per-server counts combine into a fleet — **without double-counting anyone**, which a plain sum cannot do.

Twelve kilobytes for a billion distinct users, against roughly 8 GB for a real set of 64-bit ids.

Count-Min Sketch

A grid of counters — d rows, w columns — with one hash function per row. To record an item, increment one counter in each row. To query, take the **minimum** of those d counters.

Collisions only ever add, so every counter is an over-estimate, and the minimum is the tightest of the d over-estimates. **It never under-estimates**, which is the one-directional error again.

```
w = 2000, d = 5, 4-byte counters    = 40 KB
error                               ~ 2/w x total count, with probability 1 - 2^-d
```

Used for heavy hitters — the top few items by frequency in a stream — where the frequent ones are estimated very accurately and the rare ones are noise, which is exactly the right way round.

Where they actually appear

- **Cassandra, HBase, RocksDB, LevelDB.** Each SSTable has a Bloom filter, so a read that would miss skips the file entirely without a disk seek. **This is the highest-impact use of Bloom filters in practice** — it turns “check every file” into “check the one file that might have it”.
- **Web crawlers.** “Have I already fetched this URL?” over billions of URLs.
- **Chrome’s safe browsing** historically shipped a Bloom filter of malicious URLs so the common case — a safe URL — needed no network call.
- **CDNs.** “Has this object been requested before?”, to avoid caching one-hit objects.
- **Redis:** HyperLogLog built in; Bloom and Cuckoo filters via the RedisBloom module.
- **Analytics.** Unique visitors per hour/day/month, merged with `PFMERGE`.

6 The numbers

Memory, which is the whole argument. One billion URLs at ~100 bytes each:

exact set (Python)	1e9 x ~150 bytes with overhead	= 150 GB
exact set (compact)	1e9 x 100 bytes	= 100 GB
Bloom, p = 1%	1e9 x 9.6 bits	= 1.2 GB
Bloom, p = 0.1%	1e9 x 14.4 bits	= 1.8 GB
Bloom, p = 0.01%	1e9 x 19.2 bits	= 2.4 GB

A hundred gigabytes to 1.2, and the difference between “a cluster” and “one machine’s memory”.

The sizing formulas, and the numbers to carry:

```
m = -n ln(p) / (ln 2)^2      bits
k = (m / n) ln 2             hash functions

p = 1%      → 9.6 bits/item,  k = 7
p = 0.1%    → 14.4 bits/item, k = 10
p = 0.01%   → 19.2 bits/item, k = 13
```

Roughly 5 bits per item for each additional factor of ten in accuracy — that is the shape of the curve and it is worth knowing, because it means high accuracy is affordable.

What overfilling does. The rate is only what you designed for if you add the number of items you sized for:

```
sized for 1,000,000 items at p = 1%      → m = 9.6 Mbit
actually inserted 5,000,000
  bits set      far more than half
  actual rate    ~63%
```

Sixty-three percent false positives, and nothing reports it. The filter still never gives a false negative, so it is not *wrong* — it is just useless, and the only symptom is that the expensive path is taken almost every time. **Monitor the fill ratio, not just the hit rate.**

Query cost:

```
positive lookup    k bit reads = 7      (all must be checked)
negative lookup    ~1-2 bit reads      (short-circuits at the first 0)
one hash           ~100 ns
                  → ~100-200 ns per lookup, entirely in memory
vs a disk seek     ~100 us      → 500-1,000x
vs a network round trip ~500 us  → 2,500-5,000x
```

Saved work in the SSTable case, which is the killer application:

```
10 SSTables, key present in one
  without filters  10 disk seeks x 100 us  = 1 ms
  with filters     1 seek + 10 x 200 ns    = ~100 us
                  → 10x, and it grows with the number of files
```

HyperLogLog:

16,384 buckets x 6 bits	= 12 KB
error	~0.81%
counting 1 billion distinct ids	
exact set of 8-byte ids	= 8 GB
HyperLogLog	= 12 KB
	→ ~700,000x

merging 24 hourly HLLs into a day	→ one PFMERGE, no double counting
merging 24 exact sets	→ 8 GB x 24 to union

Count-Min Sketch:

w = 2,000, d = 5, 4-byte counters	= 40 KB
error bound	$\sim 2/w \times N = 0.1\%$ of the total count
tracking 10,000,000 distinct terms	
exact counts (hash map)	~800 MB
sketch	40 KB
	→ 20,000x

And the honest limit: at 0.1% of the *total* count, an item seen 50 times in a stream of 10 million is lost in the noise. **These structures are accurate about the frequent and useless about the rare**, which is usually exactly what you want and occasionally exactly what you do not.

7 The trade-offs

You trade accuracy for memory, and the error is one-directional. That direction is the whole point: a Bloom filter never says “no” wrongly, so a “no” can be acted on and a “maybe” always needs the real check. **Placing it so that a false positive costs a cheap extra lookup, rather than a wrong answer, is the design decision** — everything else is arithmetic.

You cannot delete, list, or count. No removal without moving to a counting or cuckoo filter at extra memory. No way to enumerate what is in it. No way to ask how many items it holds. **If any of those is needed, this is the wrong structure** and the answer is a real set, possibly on disk.

The size must be chosen up front and overfilling degrades silently. Size for a million and insert five million, and the false-positive rate goes from 1% to 63% with no error, no warning, and no symptom except that the expensive path is now always taken. **Monitoring the fill ratio is not optional**, and a scalable Bloom filter is the answer when the cardinality is genuinely unknown.

A false positive is only cheap if the exact check is cheap. In the SSTable case it is one local disk seek. If the exact check is a cross-region database query costing 80 milliseconds, a 1% false positive rate on a million lookups a second is ten thousand unnecessary cross-region queries a second — which is a bigger problem than the memory you saved. **Cost the false-positive path, not just the rate.**

Distributed use needs care. The filter is a shared mutable structure; two servers each keeping their own copy will diverge as they add different items. Options are one shared filter behind a service, per-shard filters where each shard owns its keys, or periodic rebuilds broadcast to everyone. **“Just put a Bloom filter in front of it” quietly assumes a single writer.**

And HyperLogLog and Count-Min have their own limits. HyperLogLog gives a count and cannot tell you *who*, so “how many distinct users” is answerable and “which users” is not. Count-Min is accurate for heavy hitters and noise for the tail, and its error is proportional to the **total** stream volume rather than to the item’s own count.

When would I not use them? When the exact answer is required and there is no cheap fallback — a correctness check, an authorisation decision. When the data fits comfortably in memory as a real set, where a `set` is simpler, exact, deletable and enumerable. And when the population is small: at ten thousand items the memory saving is a few hundred kilobytes and not worth the extra concept in the codebase.

8 In the interview

How it gets asked

- “How do you check whether a URL has been crawled, using little memory?” — the direct version.
- “Design a web crawler.” — [day 151](#), where this is a component.
- “How do you count unique visitors without storing every user id?” — HyperLogLog.
- “What is a Bloom filter and when is it wrong?”
- “What if a false positive is expensive?” — the placement question.
- “How do you find the top 10 most frequent items in a stream?” — Count-Min Sketch.

The first ninety seconds

“A Bloom filter, and the key property is the direction of its error: **it can say ‘maybe’ when the truth is no, but it can never say ‘no’ when the truth is yes.** No false negatives, some false positives.

How it works: a bit array and k hash functions. To add an item, hash it k ways and set those k bits. To check, hash it the same way and look. **If any bit is zero it was definitely never added** — adding it would have set that bit, and bits are never cleared. If all k are set, it was probably added, or those bits were set by other items.

The memory argument is the reason to use it. A billion URLs at a hundred bytes each is a hundred gigabytes as a real set. A Bloom filter at a one percent false-positive rate is 9.6 bits per item — about 1.2 gigabytes. **That is the difference between a cluster and one machine’s memory.** And the sizing formula is $m = -n \ln(p) / (\ln 2)^2$ bits with $k = (m/n) \ln 2$ hashes; for one percent that is 9.6 bits and 7 hashes, which is worth remembering as a pair.

Where it goes matters more than the rate. The filter sits in front of the expensive lookup. A ‘no’ skips the lookup entirely and is always correct. A ‘maybe’ does the lookup, which gives the true answer. **So a false positive costs one unnecessary lookup and never a wrong answer** — the filter is a shortcut on the negative path and never decides anything on its own.

What it cannot do, which I would state up front: it cannot delete, cannot list its contents, and cannot tell you how many items it holds. And **the size must be chosen in advance** — insert five times what you sized for and the rate goes from one percent to sixty-three, silently, with the only symptom being that the expensive path is now always taken.

How many items, and how expensive is the fallback lookup? Because if the fallback is a cross-region query, a one percent rate might be too many.”

The follow-ups

“What if a false positive is expensive?”

“Then either move the filter or accept a lower rate, and I would work out which by costing the false-positive path rather than by tuning the number blindly.

First, the arithmetic. A one percent rate on a million lookups a second is ten thousand false positives a second. If the exact check is a local disk seek at a hundred microseconds, that is a second of disk time per second across the fleet — noticeable but survivable. If it is a cross-region query at eighty milliseconds, that is eight hundred seconds of latency per second, which is not a rate problem, it is a design problem.

So the cheapest fix is usually more memory. Going from 1% to 0.01% costs twice the bits — 9.6 to 19.2 per item — and reduces the false positives a hundredfold. **Bits are cheap and the curve is very forgiving**, so I would reach for that before anything clever.

The structural fix is to check whether the filter is in the right place. It belongs in front of something cheap and local. If the only exact check is expensive and remote, the filter is saving the wrong thing.

And the case where a false positive is not merely expensive but wrong — ‘is this username taken’, ‘is this user authorised’ — the answer is that the filter must not decide. A ‘maybe’ means go and check, always. If somebody has written code where a ‘maybe’ short-circuits to a yes, that is a correctness bug and no rate is low enough to make it acceptable.”

“Count unique visitors without storing every user id.”

“HyperLogLog, and the numbers are the argument: about twelve kilobytes for roughly 0.8% error, **regardless of whether there are a thousand distinct users or a billion**. A real set of a billion 8-byte ids is 8 gigabytes.

The intuition: hash every id and watch the leading zeros. A hash with ten leading zeros has a one-in-1024 chance, so seeing one suggests you have probably hashed about a thousand distinct things. The longest run you have seen is a noisy estimate of the count — so HyperLogLog splits the hash space into 16,384 buckets, tracks the longest run per bucket, and combines them with a harmonic mean, which brings the error down to about 1.04 over the square root of the bucket count.

The property that makes it genuinely useful in a design is mergeability. Two HyperLogLogs can be merged into one that counts their union — `PFMERGE` in Redis. So I keep one per hour, and a day is the merge of twenty-four of them, and a month is the merge of thirty days, **with no double counting**, which summing hourly counts cannot do. Same for per-server counters merged across a fleet.

The limit I would state: it tells you *how many*, never *who*. ‘A hundred thousand distinct users’ is answerable; ‘which users’ is not, and if somebody later asks for the list, this structure has nothing to give them. That is a requirements question worth asking before choosing it.”

“Where do Bloom filters actually get used in real systems?”

“The one with the biggest impact, and the one people do not know, is inside LSM-tree storage engines — Cassandra, HBase, RocksDB, LevelDB.

Data is spread across many immutable files on disk. A read for a key that is not present would have to check every file, each a disk seek. **So every file carries a small Bloom filter of the keys it contains**, held in memory. A lookup checks the filters first, and skips every file whose filter says no.

The arithmetic: ten files, key present in one. Without filters, ten seeks at a hundred microseconds — a millisecond. With them, one seek plus ten bit-array checks at a couple of hundred nanoseconds — about a hundred microseconds. Ten times faster, and it improves as the number of files grows.

And it is exactly the right shape for the structure, which is why it is such a good example: a ‘no’ is certain and saves a seek; a ‘maybe’ costs one seek that would have happened anyway; and the filter is built once when the file is written and never modified, so the no-deletion limitation does not apply at all.

Other real uses: web crawlers for ‘have I fetched this URL’, which is the classic; CDNs deciding whether an object has been requested before, so one-hit objects are not cached; and Chrome historically shipped a Bloom filter of malicious URLs so that the common case — a safe URL — needed no network call, which is the same shape again.”

“Top ten most frequent items in a stream, in bounded memory.”

“Count-Min Sketch, plus a small heap.

The sketch is a grid of counters — say five rows by two thousand columns, one hash per row. To record an item, increment one counter in each row. To query its count, take the **minimum** of those five counters. Collisions only ever add, so every counter over-estimates, and the minimum is the tightest over-estimate. **It never under-estimates**, which is the same one-directional error as a Bloom filter.

Then the heap. After each update I query the item’s estimate, and keep a min-heap of the top ten by estimate. That gives the heavy hitters with bounded memory for both parts.

The sizing: five by two thousand at four bytes is forty kilobytes. Against a real hash map over ten million distinct terms at roughly eight hundred megabytes, that is twenty thousand times less.

The limit is important and I would state it. The error bound is proportional to the *total* stream volume — roughly $\frac{2}{w}$ times N — not to the individual item’s count. So an item seen fifty times in a stream of ten million is completely lost in the noise. **These structures are accurate about the frequent and useless about the rare**, which for ‘top ten’ is exactly the right way round, and for ‘did this specific rare item appear’ is exactly the wrong tool.”

The model answer

“Design the deduplication for a web crawler that will fetch a billion pages: before fetching a URL, decide whether it has already been seen.”

“The core question is ‘have I seen this URL’, a billion times over, and the naive answer does not fit in memory — so let me do the arithmetic first because it frames everything.

A billion URLs at a hundred bytes each is a hundred gigabytes as a real set, and in Python with object overhead considerably more. That is a cluster to hold something whose only purpose is to answer yes or no.

A Bloom filter at one percent is 9.6 bits per URL — 1.2 gigabytes. One machine’s memory, comfortably.

What a false positive costs here is the crucial question, and the answer is unusually forgiving: one page not crawled. A one percent false-positive rate means about one percent of genuinely new URLs are wrongly believed to have been seen and are skipped. For a crawler that is acceptable — the web is not a fixed set, pages are re-discovered through other links, and no single page is critical. **Being explicit that a false positive here means a missed page and not a duplicate fetch is the thing I would want to be judged on,** because the error direction matters: the filter never wrongly says ‘new’, so **duplicate fetching — the expensive failure — cannot happen.**

If missing one percent of pages is not acceptable, the filter goes in front of an exact store rather than replacing it: a ‘no’ skips the lookup, a ‘maybe’ checks a key-value store on disk. That gets exactness back and still eliminates 99% of the disk lookups, which is the entire point.

Sizing for growth. I would size for two billion rather than one, because over-filling degrades silently — insert five times the design capacity and the rate goes from 1% to 63% with no error and no symptom except that everything is now a maybe. **I would monitor the fill ratio,** the fraction of bits set, and alert when it approaches half, which is the point of optimal fill.

Distribution, which is the part that is easy to hand-wave. A billion-URL crawler is many machines, and a single shared filter is a bottleneck while per-machine filters diverge. So I would **shard by URL hash:** each crawler worker owns a slice of the hash space and holds the filter for its own slice. A URL is routed to its owner, which is the only machine that needs to answer for it. That gives one writer per filter, no coordination, and the same total memory split across machines.

Normalisation before hashing, which is a correctness issue rather than a structural one: lowercase the host, strip default ports, sort or drop tracking query parameters, resolve `..` segments. Otherwise the same page under three URL spellings occupies three filter entries and gets fetched three times, and no amount of filter tuning fixes it.

What I would also use, separately: HyperLogLog to report how many distinct domains and URLs have been seen, at twelve kilobytes each, because the Bloom filter cannot tell me how many items it holds. Two structures, two questions, both in bounded memory.

And what I would not do: delete from it. A crawl frontier does not need removal, and if a policy change means re-crawling a domain, the honest answer is to rebuild the filter — or shard finely enough that one domain’s slice can be rebuilt alone.”

9 Recall card

A Bloom filter is a bit array plus k hashes: “no” is certain, “maybe” is not. No false negatives, some false positives — and **the direction is the whole point**, because a “no” can be acted on and a “maybe” always needs the real check.

Sizing: $m = -n \ln(p) / (\ln 2)^2$ bits, $k = (m/n) \ln 2$ hashes. For 1%: **9.6 bits per item, 7 hashes** — a billion URLs in **1.2 GB instead of 100 GB**, and roughly 5 more bits per extra factor of ten in accuracy.

Place it so a false positive costs a cheap extra lookup, never a wrong answer. The canonical use is an SSTable filter in an LSM store: a “no” skips a disk seek, a “maybe” costs one that would have happened anyway, and the file is immutable so deletion never comes up.

It cannot delete, list, or count, and **overfilling degrades silently** — five times the design capacity takes 1% to 63% with no error. Monitor the fill ratio. Counting or cuckoo filters if you need removal.

Relatives: **HyperLogLog** — distinct count in ~12 KB at ~0.8% error at *any* cardinality, and **mergeable** (`PFMERGE`) so hours combine into days without double counting. **Count-Min Sketch** — frequency in fixed memory, **never under-estimates**, accurate for heavy hitters and noise for the tail.

DSA topic: What dynamic programming actually is **System design topic:** Bloom filters and probabilistic structures

Code these, in this order

One rule for the whole set, and it is the habit this entire phase depends on: **write the recurrence in words as a comment, then the base cases, then plain recursion, then add memory.** In that order, every time. Writing a table before you have the recurrence is how DP becomes impossible, and it is the single most common way people get stuck.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Fibonacci Number	LeetCode 509 (Easy)	The four versions: naive, memo, table, two variables.
2	Climbing Stairs	LeetCode 70 (Easy)	The same recurrence, different base cases. Get them right.
3	Min Cost Climbing Stairs	LeetCode 746 (Easy)	<code>min</code> instead of <code>+</code> , and where the answer lives.
4	Unique Paths	LeetCode 62 (Medium)	Two dimensions, and the base row and column.
5	Unique Paths II	LeetCode 63 (Medium)	Obstacles — the base cases stop being trivial.
6	House Robber	LeetCode 198 (Medium)	The first genuine choice at each step.

On problem 1, write all four and measure

Naive, memoised, tabulated, and two-variable. Then:

1. Count the calls the naive version makes for `n` = 10, 20, 30, 35.
2. Time `naive(32)` against `optimised(32)` and record the ratio.
3. Print the number of *distinct* subproblems next to the call count.

Two columns, and the gap between them is what memory removes.

On problem 2, get the base cases wrong on purpose

Set `ways(2) = 1` instead of 2. Run it for `n = 12`. Record the answer and how far off it is. Then say why there was no error.

On problem 5, do the base row by hand

Write down the first row of the table for a grid with an obstacle in the middle of it. Then write the loop that produces it. The naive `dp[0][j] = 1` is wrong the moment there is an obstacle, and that is the whole problem.

On problem 6, name the state before coding

Write `dp[i] =` as a full English sentence before anything else. Then check it against a three-house example by hand. If your sentence does not let you compute `dp[3]` from `dp[2]` and `dp[1]` alone, the state is wrong.

Then the conversion drill

For each of problems 3, 4 and 6:

1. Write the top-down memoised version.
2. Convert it to bottom-up, working out the fill order explicitly.
3. Reduce the space, and say what you gave up by doing so.

Three versions each, and the third one should make you say out loud what the window is.

The definition drill

1. Define DP in one sentence.
2. Name the two conditions and say what each means.
3. Say what distinguishes DP from divide-and-conquer, with an example of each.
4. Say what a “state” is, and give the test for whether yours is complete.
5. Give the cost formula for any DP.

The order-of-work drill

1. Say the four steps, in order, from problem statement to memoised code.
2. Say what goes wrong if you start with the table.
3. Say why you would write top-down before bottom-up in an interview.
4. Give the two reasons to convert to bottom-up.

The stairs drill

1. Derive the recurrence out loud, from “my last move was”.
2. Give the base cases and say why `ways(2) = 2`.
3. Give the naive call count for `n = 10, 20, 30, 35`, and the growth factor per step.
4. Write all four versions from memory.
5. Say what window the space optimisation keeps, and why.

The state drill

1. Say what `dp[i]` means for stairs, for unique paths, and for house robber.
2. Give the test for whether a state is complete.
3. Give an example where a one-dimensional state is not enough, and say what the second dimension is.
4. Say what happens — exactly — when the cache key omits part of the state.

The costs drill

1. Give the naive and memoised costs for stairs, with the reason for each.
2. Apply the states-times-work formula to a 1D, a 2D, a 2D-with-inner-loop, and a bit-mask DP.
3. Compare top-down and bottom-up on time and on space.
4. Give Python’s recursion limit and the `n` at which top-down stops being viable.

The break-it drill

Trigger each and record the exact output or error:

1. No base case at all.
2. A base case off by one.
3. `@lru_cache` on a function taking a list.
4. A manual cache keyed on only some of the arguments.
5. A mutable default argument as the cache, called twice from the top.
6. Memoised recursion at `n = 100,000`.
7. `@lru_cache` on a method with `self`, across two test cases.

Five of the seven give no error. Name them.

The mechanism drill

1. Describe a Bloom filter’s `add` and `check` in two sentences each.

2. State the guarantee precisely, in the right order.
3. Say why there can be no false negatives.
4. Say what a false positive actually is, in terms of bits.
5. Say why it cannot delete, and what breaks if you try.

The sizing drill

1. Give both formulas.
2. Give bits per item and k for 1%, 0.1% and 0.01%.
3. Compute memory for a billion items at each, and against a real set.
4. Say how many extra bits per item each factor of ten in accuracy costs.
5. Say what happens when you insert five times the design capacity, and what the symptom is.

The placement drill

1. State the placement rule in one sentence.
2. Draw the two paths and say what each costs.
3. Say why the filter must never decide anything on its own.
4. Give a case where a false positive is cheap and one where it is wrong output.
5. Cost the false-positive path for a local disk seek and for a cross-region query at 1% and a million lookups a second.

The variants drill

1. Say what a counting Bloom filter buys and costs.
2. Say what a scalable Bloom filter is for.
3. Say what a cuckoo filter does better and what it does worse.
4. Say why k has an optimum, and what “half the bits set” has to do with it.

The relatives drill

1. Say what HyperLogLog answers, its memory, and its error.
2. Give the intuition — leading zeros — in two sentences.
3. Say what `PFMERGE` buys that summing counts cannot.
4. Say what HyperLogLog cannot tell you.
5. Say what Count-Min Sketch answers, its error direction, and what it is good and bad at.

The numbers drill

1. Compute the SSTable saving for ten files with and without filters.
2. Compare bit reads for a positive and a negative lookup, and say why negatives are cheaper.
3. Compute HyperLogLog against an exact set for a billion ids.
4. Compute Count-Min against a hash map for ten million terms.
5. Say what the Count-Min error is proportional to, and what that means for rare items.

The failure drill

For each, say what happens and what you would build:

1. A Bloom filter sized for a million, holding five million.
2. A “maybe” treated as a “yes” in a username-availability check.
3. Two servers each keeping their own copy of the same filter.
4. A false positive whose exact check is an 80 ms cross-region query.
5. Someone asks which users were counted by a HyperLogLog.
6. A Count-Min Sketch queried for an item seen 50 times in a stream of 10 million.
7. A crawler that does not normalise URLs before hashing.

Two of the seven are correctness bugs rather than performance problems. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Why is this exponential, and how do you fix it?* The recurrence first, the branching call tree, the gap between exponentially many calls and linearly many distinct answers, memory as the fix, and the two conditions that make it apply.
 2. *What does `dp[i]` represent in your solution?* The full sentence, the test for completeness, what happens silently when the state is incomplete, and an example where one dimension is not enough.
 3. *How do you check whether a URL has been crawled, using little memory?* The guarantee in the right order, the sizing arithmetic with the 100 GB against 1.2 GB, the placement rule, what a false positive costs *in this system specifically*, and what the filter cannot do.
-

Before you move on

- [] I write the recurrence in words before any code.
- [] I write plain recursion before adding memory.
- [] I can define DP in one sentence and name both conditions.
- [] I know what distinguishes DP from divide-and-conquer.
- [] I can say what `dp[i]` means as a full sentence, every time.
- [] I know the test for whether a state is complete.
- [] I know what happens when the cache key omits part of the state.
- [] I can give the cost formula and apply it to four shapes.
- [] I can produce the naive call counts and the growth factor.
- [] I can write all four stairs versions from memory.
- [] I know the two reasons to convert to bottom-up.
- [] I know Python's recursion limit and when top-down stops being viable.
- [] I have seen all five silent DP failures myself.
- [] I can state the Bloom filter guarantee in the right order.
- [] I can explain why false negatives are impossible.
- [] I know both sizing formulas and the 1% numbers.
- [] I can compute a billion items at three error rates.
- [] I know roughly 5 extra bits buys a factor of ten in accuracy.
- [] I know what overfilling does and that it is silent.
- [] I can state the placement rule and draw both paths.
- [] I know the filter must never decide on its own.
- [] I can cost the false-positive path, not just quote the rate.
- [] I know the three things a Bloom filter cannot do.
- [] I know why `k` has an optimum.
- [] I can name counting, scalable and cuckoo variants and their trades.
- [] I can explain HyperLogLog's intuition, memory and error.
- [] I know why `PFMERGE` matters and what HLL cannot tell me.
- [] I know Count-Min's error direction and what it is bad at.
- [] I know the distributed-filter problem and its three answers.
- [] I answered all three questions above out loud.

Fibonacci: memoisation versus tabulation

DATA STRUCTURES AND ALGORITHMS

Fibonacci: memoisation versus tabulation

SYSTEM DESIGN

Building blocks revision and interview questions

Fibonacci: memoisation versus tabulation

1 What this is, and why they ask it

Every dynamic programming problem can be written two ways, and you should be able to write both and convert between them mechanically.

Memoisation — top-down — is the recursion you derived, with a cache bolted on. You start at the answer you want and recurse down to the base cases, and the cache fills as the calls unwind.

Tabulation — bottom-up — starts at the base cases and fills an array forwards until it reaches the answer. No recursion at all.

They compute the same values in a different order, and the conversion between them is a fixed procedure rather than a fresh insight each time. **Learning that procedure is what today is for**, because “write it top-down, now write it bottom-up” is one of the most common DP follow-ups there is, and fumbling the conversion after a correct memoised solution is a bad way to lose a round.

The other reason is that the two forms are good at different things. Top-down is easier to derive and only computes what it needs. Bottom-up has no recursion limit and makes the space optimisation obvious. **Neither is better; knowing which to reach for is the skill.**

By the end of this lesson you can write both forms of any DP, convert between them by a fixed set of steps, work out the fill order for bottom-up, collapse a table to a fixed window, and say which form you would choose and why.

2 The story

Prabhakar took a loan for the scooter in 2019 — thirty-six months, and the bank gave him a small booklet with a row for every month.

He did not trust it. Not because he thought the bank was cheating him, but because he had never in his life taken a number on trust that he could work out himself, and thirty-six rows is not so many.

So he sat down on a Sunday with the interest rate and the instalment amount and worked it out.

The first way he tried was backwards, and it was how the question naturally came to him, because what he actually wanted to know was the balance at the end of month thirty-six.

To know that, he needed the balance at the end of month thirty-five, plus the interest on it, minus the instalment. To know *that*, he needed month thirty-four. Which needed thirty-three. He went down and down and down until he reached month zero, which he knew — that was the loan amount — and then he came back up the chain doing the arithmetic on the way.

It worked. It took him about forty minutes and he lost his place twice, because holding thirty-six half-finished calculations in his head while going down the chain is genuinely hard, and he had to start again from month thirty each time.

Then he did it the other way and it took eleven minutes.

He started at month one, which needed only the loan amount — a number he already had. He worked out the balance and wrote it in the row. Then month two, which needed only month one, which was written in the row above. Then month three. Straight down the page, thirty-six rows, never looking further back than the line immediately above.

Same numbers. Same arithmetic. Different direction, and the second one never required him to hold anything in his head at all.

The thing that struck him afterwards — and he mentioned it to his brother, who was not interested — is that once he was working forwards, he did not actually need the booklet. **Each row only needed the row above it.** He could have covered everything except the last line he wrote and carried on perfectly well. The thirty-five rows above were a record, not a requirement.

He keeps the booklet anyway. It is nice to be able to see the whole thing.

3 The idea in plain English

Prabhakar's two Sundays are the two forms, and his last observation is the space optimisation.

Top-down is starting at the question and working towards what you know. “I want month 36. That needs month 35. That needs month 34...” — down to a base case, then back up. In code that is recursion with a cache:

```
solve(n):
    if n is a base case: return the known answer
    if n is in the cache: return it
    answer = combine(solve(smaller), solve(smaller))
    cache[n] = answer
    return answer
```

Bottom-up is starting at what you know and working towards the question. “Month 1 needs only the loan amount. Month 2 needs month 1.” Fill an array from the base cases upwards:

```
dp[base cases] = known answers
for i in the right order:
    dp[i] = combine(dp[smaller], dp[smaller])
return dp[n]
```

They compute exactly the same values. The recurrence is identical; only the order differs, and which direction you approach it from.

The conversion is mechanical, and this is the part worth memorising. Given a working memoised function, bottom-up follows in five steps:

1. **The function’s parameters become the table’s dimensions.** `solve(i)` becomes `dp[i]`. `solve(i, j)` becomes `dp[i][j]`.
2. **The base cases become the initial values**, written into the table before the loop.
3. **The recursive calls become table reads.** `solve(i-1)` becomes `dp[i-1]`.
4. **Work out the loop order** so that every cell a formula reads is already filled. This is the only step that requires thought.
5. **Return** `dp[the original argument]` instead of `solve(n)`.

Step four is the whole difficulty, and it has a simple rule: iterate in the direction the recursion goes *towards*. If `solve(i)` calls `solve(i-1)`, the recursion goes downwards, so the loop goes upwards. If it calls `solve(i+1)`, the loop goes downwards. **The loop always runs opposite to the recursion.**

Now the real differences, because “they are the same” is only true of the values.

Top-down only computes states it actually reaches. If the reachable states are a small fraction of the table — a sparse state space — bottom-up fills the whole thing and wastes the difference. Coin change with huge denominations is the standard example.

Bottom-up has no recursion. Python allows about 960 nested calls, so a memoised solution with `n = 100,000` dies before the cache helps — the *first* chain of calls goes all the way down before anything is stored. **That is one of the two reasons to convert**, and it is the one that shows up as a crash.

Bottom-up makes the space optimisation obvious, which is the other reason. With the table written out you can see that `dp[i]` reads only `dp[i-1]` and `dp[i-2]`, so thirty-four of the thirty-six rows are history. In the recursive form that is genuinely hard to see.

And the space optimisation has a rule too: keep a window as deep as the recurrence reaches back. Reads `dp[i-1]` and `dp[i-2]` → two variables. Reads all of `dp[i-1][*]` → one row. Reads `dp[i-1][*]` and `dp[i][*]` → two rows, or one row updated in a careful direction.

The thing you give up by optimising is the ability to reconstruct *how* the answer was reached. The table recorded the choices; two variables do not. **If the problem asks for the actual path, subset or sequence, keep the table** — or store parent pointers, which costs the memory you were saving.

4 The picture

The two directions, on the same computation:

TOP-DOWN: start at the answer, recurse down, cache on the way back up

```
solve(6)
  → solve(5)
    → solve(4)
      → solve(3)
        → solve(2) = 1 [base]
        → solve(1) = 1 [base]
        ← 2, cached
      → solve(2) cache HIT
      ← 3, cached
    → solve(3) cache HIT
    ← 5, cached
  → solve(4) cache HIT
  ← 8
```

calls go DOWN, answers come UP, the cache fills bottom-first anyway

BOTTOM-UP: start at the base cases, fill forwards

```
i:      0  1  2  3  4  5  6
dp:      0  1  1  2  3  5  8
           ^
          dp[2] = dp[1] + dp[0], and both are already there
```

one pass, left to right, nothing held in the head

What to notice. The cache in the top-down version ends up filled from the bottom regardless — the values are computed smallest-first either way. **The difference is entirely in who drives the order:** the recursion discovers it, or you write it down.

The conversion, side by side:

MEMOISED	TABULATED
<pre>@lru_cache def solve(i): if i ≤ 1: return i return solve(i-1) + solve(i-2) return solve(n)</pre>	<pre>dp = [0] * (n + 1) dp[0], dp[1] = 0, 1 for i in range(2, n+1): dp[i] = dp[i-1] + dp[i-2] return dp[n]</pre>
(1) parameter → index	
the recursion goes DOWN (i-1, i-2) →	the loop goes UP

And the space collapse:

```

full table, n = 8:
  0   1   1   2   3   5   8   13  21
           ^   ^   ^
           |   |   |
to compute this one ----+----+----+
I only ever read these two

so keep two variables:

prev, curr = 0, 1
repeat: prev, curr = curr, prev + curr

O(n) space → O(1) space.
The other n-2 cells were a record, not a requirement.

```

What to notice. The window is two wide because the recurrence reaches back two. A recurrence reaching back `k` needs `k` values; a two-dimensional one reading only the previous row needs one row.

5 The code, built step by step

Start with the memoised version, because that is the one you write first.

```

from functools import lru_cache

@lru_cache(maxsize=None)
def fib_memo(n: int) → int:
    if n ≤ 1:
        return n # base cases: fib(0)=0, fib(1)=1
    return fib_memo(n - 1) + fib_memo(n - 2)

```

Now convert it, step by step, saying each step out loud.

Step 1: the parameter becomes the index. `fib_memo(n)` takes one integer, so the table is one-dimensional, of size `n + 1`.

```
dp = [0] * (n + 1)
```

Step 2: the base cases become the initial values.

```
dp[0], dp[1] = 0, 1
```

Step 3: the recursive calls become table reads. `fib_memo(i-1)` becomes `dp[i-1]`.

Step 4: the loop order. The recursion goes downwards — `i` calls `i-1` and `i-2` — so the loop goes upwards, and it starts after the base cases.

```
for i in range(2, n + 1):
    dp[i] = dp[i - 1] + dp[i - 2]
```

Step 5: return the cell for the original argument.

```
return dp[n]
```

Put together:

```
def fib_table(n: int) → int:
    if n ≤ 1:
        return n
    dp = [0] * (n + 1)
    dp[0], dp[1] = 0, 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
# dp[i] = the i-th Fibonacci number
```

The comment on the `dp` line is not decoration. It is the state, written as a sentence, and it is the thing you will have forgotten in ten minutes.

Now the space optimisation. Look at what `dp[i]` reads: `dp[i-1]` and `dp[i-2]`. Nothing else. So keep two:

```
def fib_two_vars(n: int) → int:
    if n ≤ 1:
        return n
    prev, curr = 0, 1
    for _ in range(2, n + 1):
        prev, curr = curr, prev + curr
    return curr
# dp[i-2], dp[i-1]
# shift the window forward
```

The simultaneous assignment matters. Written as two statements in the wrong order:

```
curr = prev + curr
prev = curr
# WRONG: prev now holds the NEW curr
```

That is a real bug and it produces powers of two rather than Fibonacci numbers. Either use the tuple form, or use a temporary.

Now the same conversion on a two-dimensional problem, because that is where the loop order actually requires thought. Unique paths in a grid:

```
@lru_cache(maxsize=None)
def paths_memo(i: int, j: int) → int:
    if i == 0 or j == 0:
        return 1 # first row or column: one way
    return paths_memo(i - 1, j) + paths_memo(i, j - 1)
```

Two parameters, so a two-dimensional table. The recursion decreases both, so both loops go upwards.

```
def paths_table(m: int, n: int) → int:
    dp = [[0] * n for _ in range(m)] # dp[i][j] = paths from (0,0) to (i,j)
    for i in range(m):
        for j in range(n):
            if i == 0 or j == 0:
                dp[i][j] = 1
            else:
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
    return dp[m - 1][n - 1]
```

And the space optimisation here is a row rather than two variables, because `dp[i][j]` reads the whole previous row:

```
def paths_one_row(m: int, n: int) → int:
    row = [1] * n # the first row: all ones
    for _ in range(1, m):
        for j in range(1, n):
            row[j] += row[j - 1] # row[j] is the row ABOVE; row[j-1] is the LEFT
    return row[-1]
```

Read that `+=` carefully, because it is the whole trick. At the moment of the update, `row[j]` still holds the value from the previous row — that is `dp[i-1][j]` — and `row[j-1]` has already been updated this pass, so it holds `dp[i][j-1]`. **One array, updated left to right, doing the work of two.**

And the direction is load-bearing. Left to right works here because we want the *new* left value and the *old* above value. Reverse the direction and `row[j-1]` would still hold the old row, which is a different and wrong recurrence. **Whenever a table collapses to one row, the iteration direction becomes part of the correctness.**

The complete solution

```
"""Memoisation and tabulation: the same recurrence, both directions, and the conversion."""

from __future__ import annotations

import sys
from functools import lru_cache

# ---- one dimension -----

@lru_cache(maxsize=None)
def fib_memo(n: int) → int:
    """Top-down. Easy to derive; limited by the recursion depth."""
    if n ≤ 1:
        return n
    return fib_memo(n - 1) + fib_memo(n - 2)

def fib_table(n: int) → int:
    """Bottom-up. dp[i] = the i-th Fibonacci number. No recursion."""
    if n ≤ 1:
        return n
    dp = [0] * (n + 1)
    dp[0], dp[1] = 0, 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]

def fib_two_vars(n: int) → int:
    """dp[i] reads only i-1 and i-2, so the window is 2 wide. O(1) space."""
    if n ≤ 1:
        return n
    prev, curr = 0, 1
    for _ in range(2, n + 1):
        prev, curr = curr, prev + curr
    return curr

# ---- two dimensions -----

@lru_cache(maxsize=None)
def paths_memo(i: int, j: int) → int:
    if i == 0 or j == 0:
        return 1
    return paths_memo(i - 1, j) + paths_memo(i, j - 1)

def paths_table(m: int, n: int) → int:
    """dp[i][j] = distinct paths from (0,0) to (i,j)."""
    dp = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            dp[i][j] = 1 if (i == 0 or j == 0) else dp[i - 1][j] + dp[i][j - 1]
    return dp[m - 1][n - 1]

def paths_one_row(m: int, n: int) → int:
    """dp[i][j] reads the previous row, so one row suffices. Direction is load-bearing."""
    row = [1] * n
    for _ in range(1, m):
        for j in range(1, n):
            row[j] += row[j - 1]
    return row[-1]
```

```

if __name__ == "__main__":
    print("fib agree :", [f(30) for f in (fib_memo, fib_table, fib_two_vars)])
    print("paths agree:", paths_memo(6, 6), paths_table(7, 7), paths_one_row(7, 7))
    print()

    # Where top-down stops working.
    print("table(5000)      :", str(fib_table(5000))[:20], "...")
    print("two_vars(5000)  :", str(fib_two_vars(5000))[:20], "...")
    try:
        fib_memo.cache_clear()
        fib_memo(5000)
    except RecursionError as error:
        print("memo(5000)      :", type(error).__name__, "-", error)
        print()

    # Where top-down wins: a sparse state space.
    @lru_cache(maxsize=None)
    def coins_memo(amount: int) → int:
        if amount == 0:
            return 0
        if amount < 0:
            return 10**9
        return 1 + min(coins_memo(amount - c) for c in (1000, 5000))

    print("states a table would fill for amount=100000:", 100001)
    coins_memo(100000)
    print("states top-down actually visited          :", coins_memo.cache_info().currsize)

```

Running it:

```

fib agree : [832040, 832040, 832040]
paths agree: 924 924 924

table(5000)      : 38789684543883256337 ...
two_vars(5000)   : 38789684543883256337 ...
memo(5000)       : RecursionError - maximum recursion depth exceeded

states a table would fill for amount=100000: 100001
states top-down actually visited          : 105

```

Three things to look at. **All three Fibonacci versions agree**, and both grid versions agree — the conversion did not change the answer.

`fib_memo(5000)` raises `RecursionError` while the two bottom-up versions return instantly. The cache does not help, because the very first call chain descends fifty thousand levels before a single value is stored. **That is the crash that forces the conversion.**

And the last two lines are the case that goes the other way. With coin denominations of 1000 and 5000, only amounts that are multiples of 1000 are ever reachable — so a table over `0..100000` fills a hundred thousand and one cells, and top-down visits **105**. The reachable state space is a fraction of a percent of the table, and top-down gets that for free.

6 What it costs

Time is identical. Both forms compute each distinct state once and do the same work per state.

```
time = (number of distinct states) x (work per state)
```

Fibonacci	$n \text{ states} \times O(1)$	$= O(n)$
Unique paths	$m \times n \text{ states} \times O(1)$	$= O(m \times n)$

Space is where they differ, and in three ways.

	cache/table	stack	total
top-down (memo)	$O(\text{states})$	$O(\text{depth})$	$O(\text{states} + \text{depth})$
bottom-up (table)	$O(\text{states})$	0	$O(\text{states})$
bottom-up (optimised)	$O(\text{window})$	0	$O(\text{window})$

Fibonacci, $n = 100,000$			
memoised	100,000 cache entries + 100,000 stack frames	→ CRASH at ~960	
table	100,000 ints in a list	~4 MB	
two vars	2 ints	~56 bytes	

The recursion depth is the hard limit and it is not negotiable:

Python usable frames	~960
<code>sys.setrecursionlimit(200_000)</code>	→ Segmentation fault (no traceback)

So the rule is: `n` up to a couple of thousand, top-down is fine. Beyond that, bottom-up.

And the case that goes the other way — a sparse state space:

coin change, amount = 100,000, denominations 1000 and 5000	
table fills	100,001 cells
reachable states	105
→ the table is 99.9% waste	

in general:	
dense state space (every cell reachable)	→ bottom-up, no waste
sparse state space (few cells reachable)	→ top-down wins, possibly hugely

The constant factors, which are real but usually not decisive:

bottom-up	an array index and an addition
top-down	a function call, a hash of the arguments, a dict lookup → roughly 2-5x slower per state in Python

```
n = 1,000,000 states
fib_table      ~0.15 s
fib_memo       would crash, but at a depth that fits: ~0.6 s
```

So bottom-up is faster per state and top-down may visit far fewer states. Which wins depends entirely on how sparse the state space is, and that is the question to ask rather than assuming.

The space optimisation, sized:

1D, window 2	$O(n) \rightarrow O(1)$	100,000 ints $\rightarrow 2$
2D, previous row only	$O(m*n) \rightarrow O(n)$	1,000,000 $\rightarrow 1,000$
2D, two rows needed	$O(m*n) \rightarrow O(2n)$	1,000,000 $\rightarrow 2,000$

And what optimising costs you:

```
full table    can reconstruct the path/subset/sequence by walking back
one row       only the final value survives
              → reconstruction needs the table, or parent pointers
                (which cost the memory you just saved)
```

One more cost worth naming: `lru_cache` holds references. A memoised function at module level keeps its cache alive for the process, so a second test case sees the first one's entries. That is correct for pure functions of their arguments and a real source of wrong answers when the function closes over changing data. `cache_clear()` between cases, or build the cache inside the call.

7 The traps

The loop running in the wrong direction

The conversion's only real difficulty, and it produces wrong answers with no error:

```
for i in range(n, 1, -1):           # recursion goes DOWN, so this is backwards
    dp[i] = dp[i - 1] + dp[i - 2]
```

[illegible]

The rule: the loop runs opposite to the recursion. `solve(i)` calls `solve(i-1)` → the loop goes upwards. `solve(i)` calls `solve(i+1)` → the loop goes downwards. And when in doubt, check on `n = 3` by hand.

The non-simultaneous assignment

```
curr = prev + curr
prev = curr                                # prev gets the NEW curr
```

```
>>> fib_broken(10)
512                                         # powers of two, not Fibonacci
```

The tuple form `prev, curr = curr, prev + curr` evaluates the right side fully before assigning, which is exactly what is needed. Two separate statements need a temporary.

The one-row collapse in the wrong direction

```
for j in range(n - 1, 0, -1):                # right to left
    row[j] += row[j - 1]
```

Correct for *some* recurrences and wrong for this one. Left to right gives `dp[i][j-1]` from the current row and `dp[i-1][j]` from the previous; right to left gives `dp[i-1][j-1]` instead of `dp[i][j-1]`.

```
>>> paths_one_row_reversed(3, 3)
3                                         # the answer is 6
```

Whenever a table collapses to one row, write down which of the two values each read is supposed to be, and choose the direction that delivers it. This is the knapsack subtlety and it is a silent wrong answer.

Converting without the base cases

```
dp = [0] * (n + 1)
for i in range(2, n + 1):
    dp[i] = dp[i - 1] + dp[i - 2]           # dp[0] and dp[1] never set
return dp[n]
```

```
>>> fib_no_base(10)
0
```

Everything reads zeros and the whole table is zeros. **The base cases are step 2 of the conversion for a reason** — they are the only values that do not come from the recurrence.

Assuming the table is fully reachable

```
dp = [0] * (amount + 1)
for i in range(1, amount + 1):
    ...                                # fills 100,001 cells
```

Correct, and on a sparse problem it is almost entirely wasted work. **If the reachable states are a small fraction of the table, top-down is not merely more convenient — it is asymptotically better in practice.** Ask which states are reachable before choosing the form.

`RecursionError` on the memoised version

```
Traceback (most recent call last):
  File "dp.py", line 8, in fib_memo
    return fib_memo(n - 1) + fib_memo(n - 2)
  [Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

The cache does not save you, because the first descent reaches the base case before anything is stored. Memoisation reduces the *number* of computations, not the *depth* of the first chain.

The workaround that is worse than the problem:

```
sys.setrecursionlimit(1_000_000)
```

```
Segmentation fault (core dumped)
```

Optimising space and then being asked for the path

```
prev, curr = 0, 1
# ... "now show me which items you chose"
```

You cannot. The table recorded the decisions and you threw it away. **Ask whether the value alone is sufficient before optimising**, and if the path is needed, either keep the table or store parent pointers — which cost the memory the optimisation saved.

A stale `lru_cache` across test cases

```
@lru_cache(maxsize=None)
def solve(i):
    return grid[i] + solve(i - 1)           # closes over a MUTABLE global
```

The second test case changes `grid` and gets the first case's cached answers. No error, and the failure looks like a wrong algorithm. **Memoise only pure functions of their arguments**, or clear the cache, or build it inside the call.

8 In the interview

How it gets asked

- “Write it top-down. Now write it bottom-up.” — the direct version, and the most common DP follow-up.
- “Can you reduce the space?”
- “Which would you use here, and why?”
- “Your solution crashes on the large input.” — the recursion depth.
- “Now return the actual sequence, not just the length.” — the reconstruction question.

The first ninety seconds

“I would write the memoised version first, because it is the recursion I derived with a cache added, so it is much harder to get wrong under pressure — and then convert, because the conversion is mechanical.

The conversion is five steps. The function’s parameters become the table’s dimensions. The base cases become the initial values. The recursive calls become table reads. Then the loop order — and this is the only step that requires thought. And finally return the cell for the original argument instead of calling the function.

The rule for the loop order is that the loop runs opposite to the recursion. If `solve(i)` calls `solve(i-1)`, the recursion goes downwards, so the loop goes upwards — that way every cell a formula reads is already filled. Getting it backwards reads uninitialised zeros and gives a wrong answer with no error, so I would check it on `n = 3` by hand.

They compute the same values in a different order, so the time is identical: states times work per state.

Two reasons I would convert. Recursion depth — Python gives me about 960 usable frames, and the cache does not help because the first descent reaches the base case before anything is stored. So `n = 100,000` crashes memoised and is instant tabulated. And the space optimisation becomes visible: with the table in front of me I can see that each cell reads only the two below it, so I keep two variables instead of `n`.

And one reason I would not. Top-down only computes the states it actually reaches. If the reachable states are a small fraction of the table — coin change with large denominations, say — bottom-up fills the whole thing and top-down visits a fraction of a percent of it. **Dense state space, bottom-up; sparse state space, top-down.**

How large can `n` get? Because that usually settles it.”

The follow-ups

“Reduce the space.”

“By looking at what each cell actually reads, and keeping a window that wide.

Here `dp[i]` reads `dp[i-1]` and `dp[i-2]` and nothing else, so at any moment I need two numbers and the rest of the array is history. Two variables rolled forward, $O(n)$ to $O(1)$.

The general rule: the window is as deep as the recurrence reaches back. Reaching back `k` means `k` values. In two dimensions, if `dp[i][j]` reads only row `i-1`, I keep one row instead of `m` — $O(m \cdot n)$ becomes $O(n)$.

And when it collapses to a single row, the iteration direction becomes part of the correctness. Updating left to right means `row[j]` still holds the previous row's value when I read it, while `row[j-1]` already holds this row's — so I get ‘above’ and ‘left’, which is what unique paths needs. Right to left would give me ‘above’ and ‘above-left’, which is a different recurrence. **That is the classic knapsack subtlety, and it is a silent wrong answer**, so I would write down what each read is meant to be and pick the direction that delivers it.

What I give up: the ability to say *which* choices produced the answer. The table recorded them and I threw it away. So if the problem wants the actual subset or path, I either keep the full table or store parent pointers — which cost the memory I just saved. **I would ask whether the value alone is enough before optimising**, because otherwise I optimise and then have to undo it.”

“Your solution crashes on the large input.”

“Recursion depth, almost certainly, and the important part is that **memoisation does not prevent it**.

The cache reduces how many computations happen, not how deep the first one goes. Calling `solve(100000)` descends a hundred thousand levels to reach the base case before a single value is stored, and Python gives me about 960 usable frames.

The fix is the bottom-up conversion, which has no recursion at all.

The fix I would not use is raising the limit. Past roughly a hundred thousand frames Python exhausts the C stack and you get a segmentation fault instead of a `RecursionError` — no traceback, nothing to debug. That is a worse failure, not a solution.

The alternative, if the state space is genuinely sparse and I want to keep top-down’s advantage, is to convert the recursion to an explicit stack — push the state, and if its dependencies are not yet computed, push them and revisit. That keeps the on-demand property without the frame limit, at the cost of noticeably more code. I would only reach for it if the table were mostly unreachable.

And I would check the other candidate before assuming: if the crash is a `MemoryError` rather than a `RecursionError`, the problem is the cache or table size, not the depth, and the answer is the space optimisation instead.”

“Which form would you use, and why?”

“It depends on two things, and I would ask about both rather than have a default.

How big can the state space get? Beyond a couple of thousand in any dimension that can recurse, top-down hits the frame limit and I convert. That is not a preference, it is a crash.

How much of the state space is actually reachable? This is the one people miss. Bottom-up fills every cell; top-down visits only the states it reaches. For coin change with denominations of a thousand and five thousand and an amount of a hundred thousand, the table has a hundred thousand and one cells and only about a hundred and twenty are reachable — **the table is 99.9% waste**. Top-down gets that for free.

Beyond those two, top-down is easier to get right and bottom-up is a bit faster per state — an array index and an addition against a function call, an argument hash and a dictionary lookup, so maybe two to five times per state in Python. That constant rarely decides anything.

In an interview I would write memoised first regardless, because deriving the recurrence and adding a cache in front of someone shows the reasoning, whereas a table appearing fully formed does not. Then I would offer the conversion and the space optimisation as improvements, which is a better narrative than starting with the optimised version and being asked where it came from.”

“Now return the actual sequence, not just the length.”

“Then I need the table, and I would say that before optimising rather than after.

With the full table, reconstruction is a backward walk. Start at the answer cell and, at each step, work out which predecessor could have produced it — for a `max` recurrence, whichever branch equals the recorded value; for a counting one, follow whichever contributed. Push each choice onto a list and reverse at the end. $O(\text{path length})$ and no extra memory.

With parent pointers it is simpler and costs memory. Store, alongside each cell, which predecessor it came from, and the walk is direct. That costs a second table the same size as the first, which is exactly the memory the space optimisation was saving — so parent pointers and space optimisation are mutually exclusive.

The thing to be careful about is ties. If two predecessors give the same value, the reconstruction picks one arbitrarily and that is fine — but it means the returned sequence is *a* valid answer, not *the* answer, so a test asserting one specific sequence will be flaky. I would assert the length, or membership in a set of valid answers.

And there is a middle option worth knowing when the table does not fit: divide and conquer over the DP — Hirschberg’s approach for edit distance — which reconstructs the path in $O(n)$ space rather than $O(n \cdot m)$ at the cost of recomputing halves. It is genuinely clever, rarely needed, and worth naming rather than attempting.”

The model answer

“Given a staircase where step `i` costs `cost[i]`, and you may climb one or two steps at a time starting from step 0 or step 1, find the minimum cost to reach the top. Write it both ways and reduce the space.”

“Let me derive the recurrence first, then do both forms, because the conversion is mechanical once the recurrence is right.

The recurrence. To stand on step `i`, I arrived from `i-1` or `i-2`, and I pay `cost[i]` to be here. So `min_cost(i) = cost[i] + min(min_cost(i-1), min_cost(i-2))`.

`dp[i]` = the minimum total cost to stand on step `i`. I would write that as a comment, because the thing that catches people here is the difference between ‘to stand on step `i`’ and ‘to get past step `i`’.

Base cases: `dp[0] = cost[0]` and `dp[1] = cost[1]`, because the problem lets me start on either without paying to get there.

And the answer is `min(dp[n-1], dp[n-2])`, **not** `dp[n-1]` — the top is *past* the last step, so I can finish from either of the last two. **That is the actual trap in this problem** and it is a modelling question, not a coding one; I would state it explicitly and check it against the two-element example by hand.

Top-down:

```
@lru_cache(maxsize=None)
def go(i):
    if i < 0:    return 0
    if i ≤ 1:   return cost[i]
    return cost[i] + min(go(i-1), go(i-2))
return min(go(n-1), go(n-2))
```

Then the conversion, five steps. One parameter, so a one-dimensional table of size `n`. Base cases into `dp[0]` and `dp[1]`. Recursive calls become `dp[i-1]` and `dp[i-2]`. **The recursion goes downwards, so the loop goes upwards**, from index 2. And the answer is `min(dp[n-1], dp[n-2])`.

```
dp = [0] * n
dp[0], dp[1] = cost[0], cost[1]
for i in range(2, n):
    dp[i] = cost[i] + min(dp[i-1], dp[i-2])
return min(dp[n-1], dp[n-2])
```

Then the space. `dp[i]` reads only `dp[i-1]` and `dp[i-2]`, so the window is two wide:

```
prev, curr = cost[0], cost[1]
for i in range(2, n):
    prev, curr = curr, cost[i] + min(prev, curr)
return min(prev, curr)
```

Simultaneous assignment, because writing it as two statements overwrites `prev` before it is read.

Cost: $O(n)$ time in every version — n states, constant work each. Space goes $O(n)$ for the table, $O(n)$ cache plus $O(n)$ stack for the memoised version, and $O(1)$ optimised.

Which I would ship: the two-variable version, because the state space is dense so top-down has no advantage, and n could be large enough to matter for the recursion depth. **Which I would write in an interview:** the memoised one first, then convert out loud, because that shows the reasoning rather than presenting a result.

And if you asked me which steps to actually take — I would need the table back, or parent pointers, and I would say that rather than pretend the optimised version can answer it.”

9 Recall card

Same recurrence, two directions. Top-down starts at the answer and recurses to the base cases; bottom-up starts at the base cases and fills forwards. **Identical time** — states $\times$ work per state.

The conversion is five mechanical steps: parameters $\rightarrow$ dimensions; base cases $\rightarrow$ initial values; recursive calls $\rightarrow$ table reads; **work out the loop order**; return `dp[original argument]`. **The loop always runs opposite to the recursion.**

Convert for two reasons: the recursion depth (~ 960 frames, and **memoisation does not help** — the first descent reaches the base before anything is cached), and because the space optimisation becomes visible.

Do not convert when the state space is sparse. Coin change with denominations of 1000 and 5000 at amount 100,000: a table fills 100,001 cells, top-down visits **105**.

Space window = how far back the recurrence reaches. Two variables for `i-1`, `i-2`; one row when only the previous row is read — and then **the iteration direction is part of the correctness**. Optimising throws away the ability to reconstruct the path, so ask whether the value alone is enough first.

Building blocks revision and interview questions

1 What this is, and why they ask it

Sixteen days ago you knew what a database was. Since then you have met queues, logs, object stores, search indexes, warehouses, time-series stores, real-time channels, push notifications, geospatial indexes and probabilistic structures. Today the phase closes.

The way to close it is not to reread it. It is to be able to hear a requirement and say, in one sentence, which component it belongs to and why — because that is exactly what the next phase asks of you. **Every high-level design question from day 145 onwards is an exercise in picking these boxes and defending the choice.**

This lesson has two halves. The first is the decision table: one line per component saying what it is for, what it is not for, and the number that decides. The second is a mock round on a system you have not seen, worked through as an interview — including the follow-ups where “we’ll use Kafka” has to become a reason.

They ask this phase’s material as follow-ups rather than as questions of its own: “*you need search, a queue and a cache — which technologies, and why?*” The answer that scores is not a list of product names. It is the requirement each one is answering and the one you would drop if pushed.

By the end of today you can place any of the fifteen components from a requirement, quote the number that justifies it, name what you give up, and take an unseen design from a blank slate to a defended architecture.

2 The story

Kadam has built or renovated about ninety houses and he does not lay a single brick.

What he does, on the first visit, is walk through the place with the owner and sort what they are saying into piles. The owner talks for twenty minutes about what they want — more light in the kitchen, the damp on the north wall, a bathroom upstairs, and could the stairs be moved.

By the end of it Kadam has, in his head, five separate jobs belonging to five different people, in an order that cannot be changed.

The damp is not a painting job even though the owner described it as one. It is a plumbing job, or a waterproofing job, and painting it before that is fixed is money set on fire. The bathroom upstairs is not one job, it is a plumbing job and a waterproofing job and a tiling job, and they have to happen in that order or you do them twice. Moving the stairs is structural and needs the engineer, and until the engineer answers, nothing else about the ground floor can be quoted.

Most of what he is paid for is knowing which trade a sentence belongs to, and knowing when the owner's description of the problem is not the problem.

The owner who caused him the most trouble was an engineer himself, and very clear about what he wanted, and what he wanted was to use one man for everything because he had found a mason who was cheap and willing. The mason did the tiling, which was acceptable, and the plumbing, which was not, and the electrical work, which they had to open the walls again for eight months later.

Kadam's line to him at the time, which he says he regrets slightly because it was rude, was that the mason had never once said "that is not my work" — and that a man who never says that is not a specialist, he is a volunteer.

The other kind of trouble is the opposite and it is more common now. People arrive having read things, and they want a particular imported waterproofing system for a bathroom in a two-room flat where a hundred and forty rupees of ordinary work would do the same job for fifteen years.

Kadam does not argue with those ones for very long. He quotes both and lets them decide.

3 The idea in plain English

Kadam's first visit is what a design interview's first ten minutes should be, and both of his difficult owners are recognisable.

The whole phase is one skill: hearing a requirement and naming the component that answers it. Not knowing what Kafka is — knowing what question Kafka is the answer to, and what question it is not.

Here is the entire phase as a decision table.

THE REQUIREMENT SOUNDS LIKE	THE COMPONENT	THE NUMBER THAT DECIDES
“the user shouldn’t wait for this”	task queue (SQS, Rabbit)	latency saved; availability stops being a product
“several teams want this event”	log / pub-sub (Kafka, SNS→SQS)	consumers > 1, or replay needed
“replay it after a bug fix”	Kafka specifically	retention window
“count things as they arrive”	stream processor (Flink)	watermark lag; state per key
“store the uploaded file”	object storage (S3)	~\$0.023/GB storage vs \$0.09/GB egress
“find text, ranked”	search index (Postgres FTS, Elasticsearch)	document count; do you need facets?
“metrics and dashboards”	time-series store (Prometheus)	series cardinality, not point volume
“analysts want to query everything”	warehouse (Snowflake, BigQuery)	data volume; is it > 100 GB?
“produce a daily number”	batch pipeline (Airflow + dbt)	SLA and backfill needs
“the browser must find out immediately”	SSE or WebSocket	does the client also send?
“notify when the app is closed”	APNs / FCM	nothing else can reach the device
“find things near me”	geo index (Redis GEO, PostGIS, H3)	write rate vs read rate
“have I seen this before, cheaply”	Bloom filter	memory saved vs false-positive cost
“read this a lot, it rarely changes”	cache (Redis, CDN)	hit rate; what does staleness cost?
“it must be correct and queryable”	the database	the default, and the burden of proof is on leaving it

The last row is the important one. Every other component in this table is something you added because the database could not do the job well enough. **The database is the default and everything else needs a justification**, which is the opposite of how these designs usually get built.

Three questions place almost any component, and asking them out loud is most of the first ten minutes:

One: does the user need the answer to continue? No means it can be asynchronous — a queue, a batch job, a push notification. Yes means it is on the request path and everything about it is a latency budget.

Two: how stale can it be? Zero means the database. Seconds means a cache or a search index or a stream. Hours means a warehouse or a batch pipeline. **Every derived copy in a system has a staleness number, and a design that has not stated it has not been finished.**

Three: what is the read-to-write ratio? Read-heavy means caches, replicas, pre-computation, derived indexes. Write-heavy means queues, partitioning, and asking whether the writes need to be durable at all — driver locations do not.

And two failure modes, both of which Kadam met.

The mason who never says “that is not my work.” Using one component for everything. Metrics in the relational database. Files in the database. Analytics on the production primary. Search with `LIKE`. Each of those works at small scale and each has a specific number where it stops — and the phase was largely about those numbers.

The owner who wanted the imported system for a two-room flat. Reaching for Kafka when one consumer and no replay requirement means a queue is simpler; for Elasticsearch at fifty thousand documents when Postgres full-text search is transactionally consistent and free; for a warehouse at ten gigabytes. **Every component is a permanent operational cost**, and “we might need it later” is not a reason to pay it now.

The sentence that ties the phase together: almost every component here is a **derived copy of data that lives somewhere else**, arranged for a different question. The search index, the warehouse, the cache, the materialised view, the geo index. **Derived copies go stale, drift, and need rebuilding — and every one of them needs a sync mechanism and a reconciliation job**, which is the part that never appears on the architecture diagram and is where the real work is.

4 The picture

The whole phase as one architecture, with the arrows that matter:

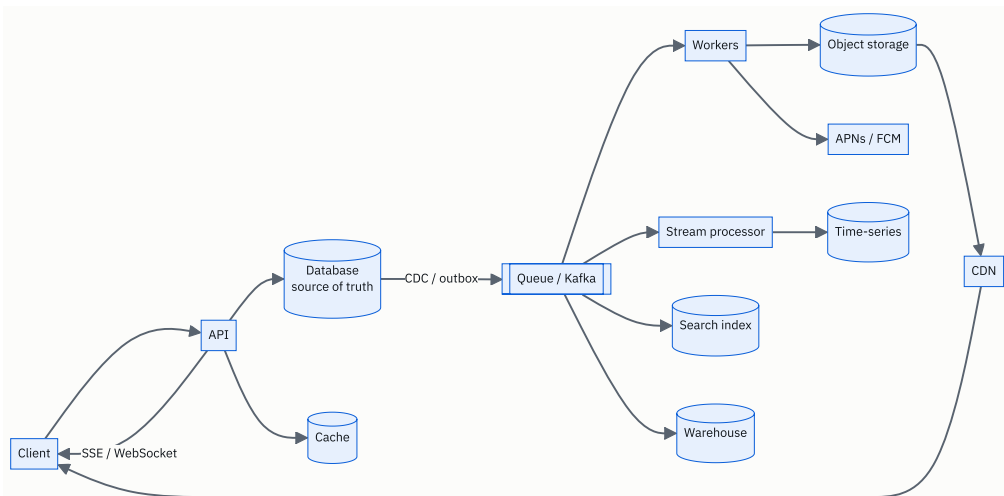


FIGURE 144.1

What to notice. Every arrow out of the database is a **derived copy**, and every one of them is asynchronous and therefore stale. The database has no incoming arrows from any of them. **If your diagram has an arrow pointing back into the source of truth from a derived store, something is wrong.**

The three questions, as a decision path:

```

"does the user need the answer to continue?"
|
+-- YES ---> on the request path
|             → database, cache, search index
|             → latency budget applies to everything here
|
+-- NO ----> off the request path
|             → queue → worker
|             → "what does the user see meanwhile?" ← always ask

"how stale can it be?"
0           → the database, or a synchronous read-through cache
seconds    → cache, search index, stream processor
minutes    → materialised view, precomputed feed
hours      → warehouse, batch pipeline

"read-to-write ratio?"
1000:1     → cache, replicas, precompute, denormalise
1:1000     → queue, partition, and ask if the writes must be durable
  
```

The numbers that decide, on one page:

LATENCY LADDER	memory 100 ns SSD 100 us same-DC 0.5 ms cross-region 80 ms cross-planet 200 ms
STORAGE COST/GB/MONTH	object \$0.023 DB SSD \$0.115 egress \$0.09/GB → egress usually dwarfs storage
THROUGHPUT, ONE NODE	Redis 100k ops/s Kafka 100k msg/s Postgres 10k writes/s Postgres hot row ~200/s ES 10-50k docs/s bulk Prometheus 500k-1M samples/s
WHEN TO SWITCH	Postgres FTS → Elasticsearch: ~1-5M documents Postgres → warehouse: ~100 GB replica → columnar: when queries scan, not seek one node → sharding: when the working set > RAM
STALENESS DEFAULTS	ES refresh 1 s CDC seconds nightly batch 24 h Kafka retention 7 d push: no guarantee at all

5 How it actually works

The phase, compressed. One block per component: what it is, the one number, and the thing people get wrong.

Queues and logs

Task queue (SQS, RabbitMQ): one message, one consumer, per-message retries and a dead-letter queue. **Log** (Kafka): messages retained, several consumer groups read independently, replay by rewinding an offset.

work → task queue	"send this email"	one owner, done, gone
event → log	"user 4471 signed up"	many readers, replayable

The one number: a queue absorbs bursts and does not create capacity. Size consumers at $\sim 2 \times$ **average**, because drain rate is $M - L$, not M . **Alert on oldest-message age, never on depth.**

What people get wrong: at-least-once is not optional, so **every consumer must be idempotent**; and the visibility timeout bug — a job longer than the timeout is redelivered while still running, and the work happens twice with no crash.

Kafka specifics

Ordering is **per partition only**; the key chooses the partition. **Within a consumer group members compete; between groups everyone sees everything.** More consumers than partitions means idle consumers.

acks=all, replication.factor=3, min.insync.replicas=2 for anything that matters

What people get wrong: `min.insync.replicas=1` looks safe and permits silent data loss; and a skewed key gives a hot partition that no amount of extra brokers fixes.

Stream processing

Bucket by **event time**, not processing time. A **watermark** is `max event time seen - a measured allowance`, set above p99 of the observed delay. Late events need a stated policy — drop, fold forward, or reopen the window with an **allowed lateness** that multiplies your state linearly.

What people get wrong: an idle partition freezes the whole job's watermark, and the symptom is a dashboard that stops updating with no error.

Object storage

Flat key namespace, no partial writes, ~11 nines durability, 20–100 ms first byte. **Presigned URLs so bytes never touch your servers**; multipart for large files; a CDN in front, always.

```
storage $0.023/GB/month | egress $0.09/GB
40 TB of images: ~$900 storage, ~$54,000 egress before a CDN
```

What people get wrong: no transaction spans the store and the database — **create the row first in** `PENDING`, drive completion from the store's event, and run a reconciliation job.

Search

An **inverted index**: term → sorted list of doc ids. `LIKE '%x%'` cannot use a B-tree and scans everything — ~500 ms on a million rows against ~2 ms. The **analysis pipeline must be identical** at index and query time. BM25 for ranking; tune field boosts, not parameters.

```
Postgres FTS (tsvector + GIN) up to ~1-5M docs, transactionally consistent
Elasticsearch                beyond that, or for facets and typo tolerance
```

What people get wrong: using it as a source of truth. **Search returns ids; the application hydrates from the database.**

Time-series

Append-only, ordered, similar consecutive values, regular timestamps → delta-of-delta plus XOR compression takes **16 bytes per point to ~1.5**. Downsample: raw 15 days, 1-minute 90 days, 1-hour 2 years — about **18× less** — storing min/max/sum/count, not just the mean.

What people get wrong: cardinality, not point volume, kills these systems. Memory scales with the number of *series*. Never label with a user id, request id, or a path containing ids.

Warehouse

Row store reads every field to get one: **16 GB scanned to sum a column needing 320 MB**. Columnar prunes columns, compresses 5–10×, vectorises, and skips blocks by zone maps — **~100× less I/O**. Star schema, ELT with CDC, transformations as a DAG.

What people get wrong: a read replica fixes the *harm* (CPU, locks, buffer pool) and not the *query* — it is still a row store. And **under ~100 GB, a replica with indexes or DuckDB over Parquet is the honest answer.**

Batch pipelines

A DAG run in topological order. Two properties make it survivable: **partition by time**, and **every write idempotent — overwrite, never append**. Jobs take the **execution date as a parameter**, never `now()`.

What people get wrong: a pipeline that runs successfully can still be wrong. Assert on the data — row counts against the trailing average, **counts before and after every join** — and fail the run rather than warn.

Real-time channels

polling	simple, stateless, ~99% waste, latency and load locked together
long polling	instant, one held connection per waiting client
SSE	one-way, and gives you reconnect + Last-Event-ID free
WebSocket	two-way, ~6 bytes/message, everything else is yours to write

What people get wrong: persistent connections make a stateless fleet **stateful** — cross-server delivery needs a pub-sub backplane on per-user channels, and a deploy disconnects everyone at once.

Push notifications

Three hops, you control one. **No delivery guarantee and no delivery report**. The **only** token-invalidation signal is a `410` when you send. Fan-out is your capacity, not the platform's: ~200 req/s per worker.

What people get wrong: putting the content in the payload. **A push is a nudge; the message lives in your store.**

Geospatial

A B-tree indexes one dimension. Reduce 2D to 1D preserving nearness: **geohash prefixes** (precision $6 \approx 1.2 \times 0.6$ km), and query the cell **plus its eight neighbours** — for a 500 m radius, ~70% of query points need it. **The index narrows; an exact distance filter always follows.**

What people get wrong: live locations are ephemeral and belong in Redis with a TTL, not in the durable database.

Probabilistic structures

Bloom filter: “no” is certain, “maybe” is not. 9.6 bits/item at 1% — a billion URLs in 1.2 GB instead of 100 GB. **Place it so a false positive costs a cheap extra lookup, never a wrong answer.** **HyperLogLog:** distinct count in ~12 KB at any cardinality, and **mergeable.** **Count-Min:** frequency, never under-estimates.

What people get wrong: overfilling degrades silently — five times the design capacity takes 1% to 63%.

6 The numbers

The figures worth having memorised, because quoting one is what turns an assertion into an argument.

Latency:

memory read	100 ns	
SSD random read	100 us	1,000x memory
same-datacentre hop	0.5 ms	5,000x
cross-region (US-EU)	80 ms	
cross-planet	200 ms	

Throughput, one node:

Redis	~100,000 ops/s
Kafka	~100,000 msg/s per broker (~100-300 MB/s)
Postgres writes	~10,000/s indexed; a hot row ~200/s
Elasticsearch	10,000-50,000 docs/s bulk; ~1,000/s single-document
Prometheus	500,000-1,000,000 samples/s
WebSocket connections	50,000-100,000 per machine
Push notifications	~200 req/s per worker

Storage and cost:

object storage	\$0.023 /GB/month	
DB SSD (gp3)	\$0.115 /GB/month	5x
egress to internet	\$0.09 /GB	usually the whole bill
warehouse compute	~\$4/hour per cluster	~\$2,900/month if left running

Compression and sizing:

time-series	16 B/point → ~1.5 B	~10x
search index	20-50% of source text (100% with positions)	
warehouse columnar	5-10x on top of column pruning	
Bloom filter, 1%	9.6 bits/item	
HyperLogLog	12 KB at any cardinality	

Switching points:

Postgres FTS → Elasticsearch	~1-5 million documents, or facets needed
Postgres → warehouse	~100 GB, or a second team asking questions
replica → columnar	when queries scan rather than seek
single node → shard	when the working set exceeds RAM
polling → push	when clients x frequency >> event rate

Staleness defaults:

Elasticsearch refresh	1 s
CDC to warehouse	seconds to minutes
nightly batch	up to 24 h
Kafka retention	7 days (the window you have to notice a lagging consumer)
push notification	no guarantee, ever

And the derived numbers that carry an argument:

egress vs storage	40 TB images: \$900 storage, \$54,000 egress
row vs column	16 GB scanned vs 320 MB needed → ~100x
polling vs WebSocket	100k users @5 s: 176 Mbps vs 40 KB/s
Bloom vs exact set	1e9 URLs: 1.2 GB vs 100 GB
multi-source vs per-source (any fan-out shape):	k separate passes vs one
sync vs async signup	2,400 ms → 86 ms; 99.5% → 99.9% availability

A worked example of the arithmetic an interviewer wants, on a fan-out question:

10,000 orders/s, 2 KB each, 5 consumers		
point-to-point (1 queue)	10,000 x 2 KB	= 20 MB/s
SNS → 5 SQS queues	50,000 x 2 KB	= 100 MB/s, 5 stored copies
Kafka, 5 consumer groups	20 MB/s stored, 100 MB/s out, ONE copy	
7-day retention	20 MB/s x 604,800	= 12 TB (Kafka)
		60 TB (5 SQS copies)

Five lines, and the choice between SNS→SQS and Kafka stops being a preference.

7 The trade-offs

The five that recur, each as the sentence you would actually say.

Every component is a derived copy, and derived copies go stale. “The search index, the cache, the warehouse and the geo index are all copies of data that lives in the database. Each has a staleness number I should state, a sync mechanism, and a reconciliation job — and the reconciliation job is the part that never appears on the diagram and is where the real work is.”

Synchronous couples availability; asynchronous costs you the answer. “Five services in series at three nines is 99.5% — three and a half hours a month. Moving four of them off the request path makes it 99.9%. But the caller now gets an acknowledgement instead of a result, so the design owes an answer to *what does the user see meanwhile*.”

Every added component is a permanent operational cost. “A Kafka cluster, an Elasticsearch cluster and a Flink job are three things to run, monitor, upgrade and be paged for. At small scale Postgres does all three jobs adequately and is one system. The burden of proof is on leaving the database, not on staying.”

Exactness against cost. “A Bloom filter, a HyperLogLog, a sampled metric and a downsampled rollup are all the same trade: a bounded error for a large saving. The question is always which *direction* the error points and whether a cheap exact check is available when it matters.”

And the meta one: more real-time than the product needs is a cost with no benefit. “A dashboard refreshed every ten seconds is indistinguishable from one refreshed continuously, to a human reading a number. The question is not ‘can we stream it’ but ‘would anyone notice’.”

And what I would not add, which is worth having ready as a list: Kafka when there is one consumer and no replay requirement; Elasticsearch under a million documents; a warehouse under a hundred gigabytes; a stream processor when a nightly batch is looked at once a day; WebSockets when only the server sends; a distributed lock when a unique constraint would do; and a cache before measuring the hit rate you would get.

8 In the interview

How it gets asked

Rarely as a topic. As follow-ups, in the middle of a design:

- “You need search, a queue and a cache. Which technologies, and why?”
- “Why Kafka and not SQS here?”
- “Where does that data actually live?”
- “How does the search index stay up to date?”
- “What happens when that component is down?”
- “How stale can this be?”

The mock round

“Design the backend for a food delivery app. Restaurants, menus, orders, live tracking of the delivery partner, and search. Assume a few million users. Forty-five minutes.”

Minute 0 to 4 — sort the requirements into piles before choosing anything.

“Let me sort what you have said into piles, because these are four quite different systems and they need different components.

Transactional and must be correct: the order itself, payment, inventory of what a restaurant currently has. That is a relational database and I would not move it anywhere.

Read-heavy and rarely changing: menus, restaurant details, images. Cache and CDN territory.

Search: finding restaurants and dishes by keyword, filtered by location and cuisine.

Write-heavy and ephemeral: the delivery partner’s location, every few seconds.

And asynchronous: notifications, the analytics feed, the restaurant’s own dashboard.

Two questions before I draw anything. What is the read-to-write ratio on orders — I would guess heavily read, because customers check status repeatedly. And how fresh does the tracking need to be: every second, or every ten? Because that changes the mechanism entirely.”

Minute 4 to 14 — the components, each with the reason.

“Orders, payments, inventory: Postgres. These need transactions — decrement stock and create an order atomically — and they are small. A few million users placing maybe a hundred thousand orders a day is about one and a half writes a second average, tens at peak. This is not a scale problem and I would resist anyone making it one.

Menus and restaurant data: Postgres plus Redis. Read-heavy, rarely changing, so a cache with a few-minutes TTL and explicit invalidation when a restaurant updates its menu. **The staleness number is minutes and I would state it,** because ‘this dish is unavailable’ arriving five minutes late is a real customer experience.

Images: object storage behind a CDN. Presigned URLs for restaurant uploads so bytes never touch my servers. And the number that matters here is egress: menu images served to a few million users is the largest single line on the infrastructure bill unless the CDN hit rate is high, so I would size the thumbnails deliberately rather than serve originals.

Search: Postgres full-text to start. A few million users does not mean a few million restaurants — it is probably tens of thousands, each with a few dozen dishes, so maybe a million searchable documents. **That is squarely in Postgres’s range,** it is transactionally consistent with the menu, and it removes an entire system. I would move to Elasticsearch when I need faceted filtering with counts, typo tolerance, or the query volume becomes a problem for the primary — and I would say that as the trigger rather than build it now.

Live tracking: Redis GEO with a TTL. A delivery partner reporting every four seconds is the write-heavy part, and **that data is completely disposable** — a location from four seconds ago has no value, and if the whole store were lost every partner re-reports within four seconds. So it does not belong in Postgres at all. `GEOADD` per ping, `GEOSEARCH` for ‘partners near this restaurant’, and a thirty-second TTL so a partner who goes offline disappears with no cleanup job.

Async work: a queue. Order confirmation notifications, restaurant printer jobs, the analytics feed. SQS-shaped rather than Kafka-shaped for the work items, because each has one owner and needs per-message retry and a dead-letter queue.”

Minute 14 — first interruption. *“Why not Kafka for all of it?”*

“Because most of what I listed is *work* rather than *events*, and Kafka is bad at work.

A notification job has one owner, must be retried individually if it fails, and needs a dead-letter queue when it fails permanently. **Kafka has no per-message acknowledgement**, so a poison message blocks its partition and I would have to build retry topics myself — which is real code that SQS gives me free.

Where I would use Kafka is the event stream: `OrderPlaced`, `OrderDelivered`, `PartnerAssigned`. Those are facts that several teams want — analytics, the restaurant dashboard, fraud, and later somebody who does not exist yet — and one of them will want to replay a month after fixing a bug. **That is the test: more than one consumer, or replay needed.**

And there is a storage argument at volume: five consumers via SNS to five SQS queues stores five copies; Kafka with five consumer groups stores one. At a hundred thousand orders a day that difference is irrelevant, so I would not use it as the reason today — but it is why the answer changes at scale.

So: Kafka for the event backbone, a task queue for work items. Both, doing different jobs, rather than forcing one to do the other’s.”

Minute 20 — second interruption. *“How does search stay in step with the menu?”*

“The outbox pattern, and I would reject the tempting alternative first.

Not a dual write — updating Postgres and then the index in the same request handler. There is no transaction across two systems, so a crash between them leaves them permanently inconsistent with nothing retrying and nothing knowing. It will happen.

Instead: the menu update and an outbox row go into the same database transaction. A consumer reads the outbox and bulk-updates the index. Atomic at the source, at-least-once delivery, and indexing is idempotent because it is keyed by document id.

If I stay on Postgres full-text, none of this applies at all — a generated `tsvector` column is updated in the same transaction as the row, so there is no sync and no staleness. **That is a genuine argument for staying** and it is worth more than the feature list.

And whichever I use, a reconciliation job, nightly, comparing counts and checksums by id range. Over months the index drifts — a dead consumer, a bulk import that bypassed the outbox — and this is the only mechanism that finds it.

One design detail: search returns restaurant and dish ids, and the application hydrates them from Postgres. So a stale index shows a slightly wrong result *set* but never wrong *content* — no old prices, no dishes that were removed an hour ago.”

Minute 28 — third interruption. *“The customer wants to watch the delivery move on a map.”*

“That is a push channel, and the choice depends on whether the client sends anything.

It does not — the customer only receives position updates. **So server-sent events, not WebSockets**, and I would defend that: SSE gives me automatic reconnection and the `Last-Event-ID` header for free, it is plain HTTP so proxies and auth cookies work unchanged, and it is a fraction of the code. WebSockets would hand me heartbeats, reconnection and framing to implement for a capability I do not need.

Update rate is a product decision with a cost. Every second is not needed — a map marker moving every three to five seconds looks smooth and is a fifth of the traffic. I would push interpolated positions rather than raw pings.

The scaling shape: an open connection makes the fleet stateful. The customer’s connection lives on one server and the partner’s location arrives at another, so I need a pub-sub backplane — Redis pub-sub on a per-order channel, so each server subscribes only to the orders it is watching rather than to everything.

And deploys become an incident if I ignore them. Restarting the fleet disconnects every watching customer at once and they all reconnect within a second. Jittered reconnect in the client, shipped in the first release because I cannot deploy a fix to disconnected clients, plus connection draining and a rolling restart.

When the app is closed, none of this works — that is APNs or FCM, a completely separate path, for ‘your order has been picked up’ and ‘your order is arriving’.”

Minute 36 — fourth interruption. *“The business wants a dashboard of orders per restaurant per hour.”*

“That is analytics, and the first thing I would say is: **not on the production database.**

A query aggregating orders by restaurant by hour over a year scans most of the orders table. On the primary it competes for CPU and locks — and worse, it evicts the buffer pool, so every ordinary query starts hitting disk and checkout gets slower for reasons nobody can see. **The damage is not the four minutes, it is what the four minutes do to everything else.**

At this size, a read replica is probably enough, and I would say that rather than propose a warehouse. A hundred thousand orders a day is thirty-six million a year — tens of gigabytes. A replica with an index on `(restaurant_id, created_at)` answers that in well under a second, and it is a configuration change rather than a project.

The trigger for a warehouse is any of: the data passing roughly a hundred gigabytes; a second team asking questions and interfering with the first; or the queries becoming full scans rather than indexed seeks — which is when a row store is doing a hundred times more I/O than the query needs and no index helps.

When that happens: CDC from Postgres into a columnar warehouse, transformations as a dbt DAG run nightly, partitioned by date so a query for one month touches one thirty-sixth of the data. And the pipeline’s jobs take the execution date as a parameter and overwrite their own partition, so backfills and retries are free.

And the number that decides: at ten gigabytes, a Snowflake cluster left running costs about \$2,900 a month for something a replica does for nothing. I would not build it until one of those three triggers fires.”

Minute 42 — the last one. *“What would you cut if you had two weeks?”*

“Postgres, Redis, object storage with a CDN, and one queue. That is it.

Search becomes Postgres full-text — one generated column and a GIN index, no second system, no sync, no staleness.

Live tracking becomes polling every five seconds. Ten thousand concurrent customers watching at five seconds is two thousand requests a second, which one machine handles, and it needs no connection state, no backplane and no deploy story. **The SSE version is better and it is not week-two work.**

Kafka waits until there is a second consumer of the event stream. Today there is one.

The warehouse waits until a replica stops being enough.

What I would not cut: the outbox for anything crossing a system boundary, idempotency on every consumer, and the object-store-plus-CDN for images — because retrofitting the first two is a rewrite, and serving images from application servers is a bill that arrives before anything else does.”

The model answer

“You need search, a queue and a cache. Which technologies, and why?”

“I would answer with the requirement each one is solving rather than a product name, because the names depend on scale and the requirements do not.

The cache is answering ‘this is read far more often than it changes’. Redis, in front of the database, keyed by whatever the read path looks up. The number that justifies it is the read-to-write ratio and the hit rate — and I would measure the hit rate rather than assume it, because a cache below about 80% is mostly adding a network hop. **And the design owes a staleness number:** a five-minute TTL means a user can see a five-minute-old price, and whether that is acceptable is a product question, not mine.

The queue is answering ‘the user should not wait for this’. SQS or equivalent, for work items with one owner. The win is both latency — a signup goes from 2.4 seconds to under a hundred milliseconds — and availability, because my signup stops being the product of five services’ uptime. **The costs I would name up front:** I lose the answer, so I must decide what the user sees meanwhile; delivery is at-least-once, so every consumer must be idempotent; and failures become quiet, so a dead-letter queue with an alert on its depth is not optional.

Search is answering ‘find text, ranked’. And here my answer depends on a number: under a few million documents, Postgres full-text search — a `tsvector` column and a GIN index — because it is a genuine inverted index, it is transactionally consistent with the data, and it removes an entire system and its sync pipeline. Beyond that, or if I need facets with counts, typo tolerance or the query volume off the primary, Elasticsearch — and then I owe an outbox, a reconciliation job and a staleness number.

The thing I would say that ties them together: the cache and the search index are both **derived copies** of data that lives in the database. Every derived copy goes stale, drifts, and needs a rebuild path. So for each one I want three things stated — how it is kept in sync, how stale it can be, and what happens when it is down — and **if the answer to the third is ‘the site is broken’, it is not a cache, it is a dependency.**

And I would ask what I could remove. If there is one consumer and no replay, the queue can be a database table with a status column and a polling worker, and that is one fewer system to run. **The burden of proof is on adding a component, not on keeping the database.**“

9 Recall card

Every component here is a **derived copy of something in the database, arranged for a different question**. Each one needs a **staleness number**, a **sync mechanism**, and a **reconciliation job** — and the reconciliation job is the part that never makes it onto the diagram.

Three questions place almost anything: does the user need the answer to continue (sync or async)? how stale can it be (0 → database; seconds → cache/search/stream; hours → warehouse)? what is the read-to-write ratio (read-heavy → cache and precompute; write-heavy → queue, partition, and ask if it must be durable)?

Work → task queue (per-message retry, DLQ). Events → log (many readers, replay). Postgres FTS to ~1–5M docs; a replica to ~100 GB; SSE unless the client also sends; Redis GEO for anything moving.

The numbers that carry an argument: egress \$0.09/GB vs storage \$0.023 (40 TB ⇒ \$54,000 vs \$900); row-store scan 16 GB vs 320 MB needed (~100×); polling 176 Mbps vs WebSocket 40 KB/s; Bloom 1.2 GB vs 100 GB; time-series 16 B → 1.5 B per point.

The two failure modes: one component doing every job, and an imported system for a two-room flat. Every addition is a permanent operational cost, so **the burden of proof is on leaving the database**.

PRACTICE

Practice — Fibonacci: memoisation versus tabulation

DSA topic: Fibonacci: memoisation versus tabulation **System design topic:** Building blocks revision and interview questions

Code these, in this order

One rule for the whole set: **solve each one three times — memoised, tabulated, space-optimised — and do the conversion by the five steps rather than by rewriting from scratch.** The conversion is a procedure, and the point of today is to make it automatic.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Fibonacci Number	LeetCode 509 (Easy)	The five conversion steps, on the simplest possible case.
2	Climbing Stairs	LeetCode 70 (Easy)	Same shape, different base cases.
3	Min Cost Climbing Stairs	LeetCode 746 (Easy)	Where the answer lives — <code>min(dp[n-1], dp[n-2])</code> , not <code>dp[n-1]</code> .
4	Unique Paths	LeetCode 62 (Medium)	Two dimensions; the one-row collapse and its direction.
5	House Robber	LeetCode 198 (Medium)	A real choice per step, and the two-variable window.
6	Delete and Earn	LeetCode 740 (Medium)	Transform the input first, then it is House Robber.

On problem 1, do the conversion out loud

Write the memoised version, then say each of the five steps aloud as you apply it. Parameter to index. Base cases to initial values. Calls to reads. Loop order — and say which direction the recursion goes and therefore which way the loop runs. Return the cell.

Then write the loop backwards on purpose and record what it returns.

On problem 1, find where top-down dies

Call the memoised version with increasing `n` until it raises. Record the value. Then call the tabulated version with `n = 5000` and confirm it is instant. Then try `sys.setrecursionlimit(1_000_000)` and the memoised version again, and record what you get instead of a `RecursionError`.

On problem 4, get the one-row direction wrong

Solve it with a full 2D table, then collapse to one row going left to right, then try right to left. Record all three answers for a 3×3 grid. Write one sentence saying which value each read is supposed to be, and why only one direction delivers both.

On problem 5, ask for the path

After solving it, return *which houses* were robbed, not just the total. Do it once from the full table by walking backwards, and once with parent pointers. Then say what the space-optimised version can and cannot answer.

Then the sparse-state experiment

Write coin change with denominations `[1000, 5000]` and amount `100000`, both ways. Instrument:

1. How many cells the bottom-up table fills.
2. How many states the memoised version actually visits (`cache_info().currsize`).

Two numbers, and the ratio is the argument for top-down that nobody expects.

The conversion drill

1. Recite the five steps in order.
2. State the loop-order rule in one sentence.
3. Convert Fibonacci by the steps, out loud, without looking.
4. Convert unique paths the same way, and say why both loops go upwards.
5. Say what step 4 costs you if you get it wrong — the symptom, not the fix.

The two-forms drill

1. Say what each form is, in one sentence each.
2. Say what is identical between them and what is not.
3. Give the two reasons to convert to bottom-up.
4. Give the one reason to stay top-down, with an example.
5. Say why memoisation does not save you from the recursion limit.

The space drill

1. State the window rule.
2. Give the window for `dp[i-1], dp[i-2]`; for all of row `i-1`; for rows `i-1` and `i`.
3. Say why the simultaneous assignment matters, and what you get without it.
4. Say why the one-row iteration direction is part of the correctness.
5. Say what optimising costs you, and what to ask before doing it.

The costs drill

1. Give the time formula and say why both forms share it.
2. Give the three space profiles: memoised, tabulated, optimised.
3. Give Python's usable recursion depth and what raising it produces.
4. Give the per-state constant-factor difference between the forms.
5. Compute the sparse coin-change comparison.

The break-it drill

Trigger each and record the exact output or error:

1. The loop running in the wrong direction.
2. `curr = prev + curr` followed by `prev = curr`.
3. The one-row collapse iterated right to left, on unique paths.
4. The conversion done without writing the base cases.
5. Memoised recursion at `n = 5000`.
6. `sys.setrecursionlimit(1_000_000)` and then the same call.
7. An `lru_cache` d function that reads a mutable global, across two test cases.

Five of the seven give no error. Name them.

The decision-table drill

For each requirement, name the component and the number that decides:

1. "The user should not wait for the confirmation email."
2. "Analytics, fraud and the recommendation team all want signup events."
3. "We need to reprocess last month after fixing a bug."
4. "Find restaurants by name and cuisine, ranked."
5. "Store the uploaded photos."

6. “Dashboards for CPU and request rate.”
7. “The business wants revenue by region by month.”
8. “The map marker should move as the driver moves.”
9. “Tell the user when the app is closed.”
10. “Find drivers within two kilometres.”
11. “Have we crawled this URL before?”
12. “This is read a thousand times per write.”

The three-questions drill

1. State the three placement questions.
2. Apply all three to: order placement, menu display, driver location, and the analytics dashboard.
3. Say what “does the user need the answer to continue” implies when the answer is no.
4. Say what every derived copy owes the design.

The switching-points drill

Give the number for each:

1. Postgres full-text search to Elasticsearch.
2. Read replica to a warehouse.
3. Polling to a push channel.
4. A queue to Kafka.
5. Single node to sharded.

The what-people-get-wrong drill

One sentence each, from memory:

1. Queues.
2. Kafka.
3. Stream processing.
4. Object storage.
5. Search.
6. Time-series.
7. Warehouses.
8. Batch pipelines.

9. Real-time channels.
10. Push notifications.
11. Geospatial.
12. Bloom filters.

The numbers drill

Quote from memory, then check:

1. The latency ladder, all five rungs.
2. Throughput for Redis, Kafka, Postgres writes, a Postgres hot row, Elasticsearch bulk, Prometheus.
3. Cost per GB for object storage, database SSD, and egress.
4. Compression ratios for time-series, search index, and columnar.
5. Staleness defaults for Elasticsearch refresh, CDC, nightly batch, Kafka retention.

Then compute:

6. Egress versus storage for 40 TB of images.
7. Row versus column scan for a `sum` over 40M rows.
8. Polling versus WebSocket bandwidth for 100,000 users at 5 seconds.
9. Bloom filter versus exact set for a billion URLs.
10. SNS→SQS versus Kafka storage for 5 consumers over 7 days.

The trade-offs drill

State each as the sentence you would say out loud:

1. Derived copies.
2. Synchronous versus asynchronous.
3. Operational cost of every added component.
4. Exactness versus cost.
5. More real-time than the product needs.

Then list the seven things you would not add, and the trigger that would change each answer.

The failure drill

For each, say what happens and what you would build:

1. A dual write to the database and the search index, with a crash in between.
2. A read replica used for a full-scan analytics query.

3. A metric labelled with a user id.
4. A push notification carrying the only copy of the message.
5. A WebSocket fleet restarted all at once.
6. A Bloom filter holding five times its design capacity.
7. A Kafka topic keyed by country, where one country is 60% of traffic.

Two of the seven lose data. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Write it top-down. Now write it bottom-up.* The memoised version first with the recurrence stated, then the five conversion steps named as you apply them, the loop-order rule, and the space collapse with its window.
 2. *Your solution crashes on the large input.* Recursion depth, why memoisation does not prevent it, the bottom-up conversion as the fix, and why raising the limit is worse — plus the alternative check if it is a `MemoryError` instead.
 3. *You need search, a queue and a cache. Which technologies, and why?* The requirement each answers rather than the product name, the number that decides each, what each costs, and the observation that two of the three are derived copies that owe a staleness number and a reconciliation job.
-

Before you move on

- ☐ I can recite the five conversion steps.
- ☐ I know the loop always runs opposite to the recursion.
- ☐ I can convert a 1D and a 2D DP by the steps, out loud.
- ☐ I know what a wrong loop direction produces, and that it is silent.
- ☐ I know the two reasons to convert to bottom-up.
- ☐ I know the one reason to stay top-down, with the sparse-state example.
- ☐ I know memoisation does not prevent the recursion limit, and why.
- ☐ I know what raising the recursion limit produces.
- ☐ I can state the space window rule and apply it in 1D and 2D.
- ☐ I know why the simultaneous assignment matters.

- [] I know the one-row direction is part of the correctness.
- [] I know what optimising costs and what to ask first.
- [] I can place any of the twelve requirements onto a component.
- [] I can state the three placement questions and apply them.
- [] I know every derived copy owes a staleness number, a sync mechanism and reconciliation.
- [] I can give the five switching points with their numbers.
- [] I can give the what-people-get-wrong line for all twelve components.
- [] I can quote the latency ladder and the throughput figures.
- [] I can quote storage and egress costs and compute the 40 TB case.
- [] I can compute row versus column, polling versus push, and Bloom versus exact.
- [] I can state all five trade-offs as sentences.
- [] I can name seven things I would not add, and what would change my mind.
- [] I know the burden of proof is on leaving the database.
- [] I answered all three questions above out loud.

Climbing stairs and the one-dimensional habit

DATA STRUCTURES AND ALGORITHMS

Climbing stairs and the one-dimensional habit

SYSTEM DESIGN

How to run a high-level design interview: the forty-five-minute script

Climbing stairs and the one-dimensional habit

1 What this is, and why they ask it

Most dynamic programming problems you will meet are one-dimensional: a single index walking along an array or a sequence, with one number stored per position. Climbing stairs, house robber, minimum cost climbing stairs, decode ways, longest increasing subsequence, word break, jump game, maximum subarray. **Eight of the most commonly asked DP problems are the same shape.**

And that shape has one habit that produces the recurrence every time, without insight:

What was the last move, and what state was I in before it?

Ask that, enumerate the possible last moves, and the recurrence writes itself. It is not a trick and it is not specific to stairs — it works on every problem in this family, and having it as a reflex is the difference between a blank editor and a solution in four minutes.

They ask these constantly because they are short enough to finish in twenty minutes and because the interviewer can watch you derive rather than recall. **The question that follows is always “what does `dp[i]` represent?”**, and that is the real test.

By the end of this lesson you can apply the last-move habit to any linear problem, write the four-line template that fits nearly all of them, choose the right base cases, recognise the three variations the family comes in, and say why an answer sometimes lives somewhere other than `dp[n-1]`.

2 The story

Vimala realises the earring is gone while she is standing at the bus stop at Sion, and her hand is already at her ear because that is what everybody does.

The left one is there. The right one is not.

Her first instinct is the useless one — to look at the ground around her feet, which she does for about four seconds, and then she stops, because the ground at a bus stop is not where things are and she knows it.

What she does instead is the thing her mother taught her, and she says it out loud without meaning to.

“Where was I last?”

Not “where could it be”, which is the whole of Bombay. **Where was I immediately before this.**

She was at the vegetable stall on the corner, two minutes ago, paying. And she remembers — genuinely remembers, not guesses — the woman there commenting on the earrings, which means both were there at the stall. So it went between the stall and here, which is a hundred and forty feet of pavement, and that is a completely different problem from the one she had ten seconds ago.

She walks it back slowly, looking down, and does not find it.

So she asks again. Where was I before the stall? The chemist. And before the chemist? The bank, where she had been at the counter and had leaned down to sign, and where — she is now fairly sure — she had put her hand up to her ear at some point.

She does not go back to the bank first. She does the stretch between the chemist and the stall, because it is nearer and it is on the way, and then the stretch between the bank and the chemist.

The earring is on the second step outside the bank, on the flat part where the two steps meet, sitting in plain view where four hundred people have walked past it and not one has noticed, which she thinks about for some time afterwards.

The thing that got her there was not searching harder. It was that at every point she asked only about **the one step before** — not about the whole afternoon. Each answer gave her one short stretch to look at, and when that failed, the same question again gave her the next one.

3 The idea in plain English

Vimala’s question is the one-dimensional DP habit.

The habit, stated once: to compute the answer at position i , ask **what the last move was**. Enumerate every possible last move. Each one leaves you at some earlier position with a known answer. Combine them.

That is it. It produces the recurrence for every problem in this family, and it works because it turns “how do I solve this” — which is unbounded — into “what are the two or three ways I could have arrived here”, which is a short list you can write down.

On stairs: to be standing on step `i`, my last move was a single step from `i-1` or a double step from `i-2`. Two possible last moves, so:

```
ways(i) = ways(i-1) + ways(i-2)
```

On house robber: at house `i` I either robbed it or I did not. If I robbed it, my previous state was the best I could do up to `i-2`, because `i-1` is forbidden. If I did not, my state is the best up to `i-1`.

```
best(i) = max( money[i] + best(i-2),    best(i-1) )
              \___ robbed ___/        \_ skipped _/
```

On maximum subarray: the subarray ending at `i` either extends the one ending at `i-1`, or starts fresh at `i`.

```
best_ending_at(i) = max( best_ending_at(i-1) + nums[i],    nums[i] )
```

Three different problems, one question. Notice that in each case the “last move” enumeration is two or three options, and each option points at a strictly earlier index. **That is what makes it one-dimensional.**

Now the second half of the habit, and it is the half people skip: say what `dp[i]` means, as a full English sentence, before writing the loop.

The three problems above have three genuinely different meanings, and they are not interchangeable:

```
stairs      dp[i] = the number of distinct ways to REACH step i
robber      dp[i] = the maximum money obtainable from houses 0..i
max subarray dp[i] = the maximum sum of a subarray ENDING EXACTLY at i
```

The third one is the one that catches people. “Ending exactly at `i`” is not the same as “using elements up to `i`”, and if you write the recurrence for one while thinking of the other, it is wrong in a way that is hard to see. **And it has a consequence: the answer is not `dp[n-1]`, it is `max(dp)`** — because the best subarray ends somewhere, and you do not know where.

So: where the answer lives is part of the state’s definition. Three possibilities, and you decide which by reading your own sentence:

- `dp[n-1]` — when the state means “using everything up to here”, so the last cell is the whole answer.

- `max(dp)` or `min(dp)` — when the state means “ending exactly here”, so the answer could be at any position.
- Something else — minimum cost climbing stairs finishes *past* the last step, so the answer is `min(dp[n-1], dp[n-2])`.

Base cases come from the recurrence, not from intuition. Write the recurrence first, then look at the smallest `i` for which it would read a negative index. Every index below that needs an explicit value. `dp[i] = dp[i-1] + dp[i-2]` reads `i-2`, so `dp[0]` and `dp[1]` must be set by hand.

And getting them wrong is silent. Setting `ways(2) = 1` instead of 2 gives you the whole Fibonacci sequence shifted by one position, and every answer is wrong by exactly one place with no error anywhere.

The template that fits nearly all of them is four lines:

```
dp = [initial] * n
dp[base indices] = base values
for i in range(first_real_index, n):
    dp[i] = combine over each possible last move
return dp[n-1] # or max(dp), depending on what dp means
```

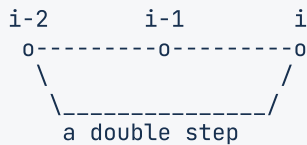
The variations are only in `combine`. Counting problems use `+`. Optimisation problems use `max` or `min`. Feasibility problems use `any` or `or`. **Same skeleton, different operator**, and recognising which of the three you have is another thing to say out loud.

And the last idea: almost all of these collapse to `O(1)` space, because `dp[i]` typically reads only the last one or two cells. Keep a window as deep as the recurrence reaches back, which is [day 144](#).

4 The picture

The last-move question, drawn on three problems:

CLIMBING STAIRS – how did I arrive at step i ?



two ways in $\rightarrow$ $\text{ways}(i) = \text{ways}(i-1) + \text{ways}(i-2)$

HOUSE ROBBER – did I rob house i or not?

ROBBED i : ... best up to $i-2$... [skip $i-1$] [ROB i]
SKIPPED i : ... best up to $i-1$ [skip i]

$\text{best}(i) = \max(\text{money}[i] + \text{best}(i-2), \text{best}(i-1))$

MAX SUBARRAY – does the subarray ending at i extend, or restart?

EXTEND: [---- best ending at $i-1$ ----][i]
RESTART: [i]

$\text{best_at}(i) = \max(\text{best_at}(i-1) + \text{nums}[i], \text{nums}[i])$

What to notice. In every case the enumeration is short — two options — and every option points at a strictly smaller index. **When the enumeration is short and points backwards, the problem is one-dimensional DP.**

Where the answer lives, which depends on what `dp[i]` means:

```
nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
```

MAX SUBARRAY, $dp[i]$ = best sum ENDING EXACTLY at i

i:	0	1	2	3	4	5	6	7	8
nums:	-2	1	-3	4	-1	2	1	-5	4
dp:	-2	1	-2	4	3	5	6	1	5

^
 $\max(dp) = 6$, at $i = 6$

the answer is NOT $dp[8] = 5$.

the best subarray is $[4, -1, 2, 1]$, and it ends at index 6.

HOUSE ROBBER, $dp[i]$ = best from houses $0..i$

money:	2	7	9	3	1
dp:	2	7	11	11	12

^
 $dp[n-1] = 12$, and that IS the answer
because " $0..i$ " means the last cell covers everything.

What to notice. Same skeleton, same loop, and the answer is in a different place — because the two states mean different things. **Reading your own sentence tells you where to look**, and guessing does not.

The base cases, derived rather than guessed:

```
recurrence:  $dp[i] = dp[i-1] + dp[i-2]$ 
```

at $i = 2$: reads $dp[1]$ and $dp[0]$ → both must exist

at $i = 1$: would read $dp[-1]$ → so $i = 1$ is a BASE case

at $i = 0$: would read $dp[-2]$, $dp[-1]$ → so $i = 0$ is a BASE case

⇒ base cases are exactly the indices the recurrence cannot compute.
The loop starts at 2.

5 The code, built step by step

Start with the habit, out loud, before any code.

“To be on step i , my last move was one step from $i-1$ or two steps from $i-2$.
Those are the only possibilities. So $ways(i) = ways(i-1) + ways(i-2)$.”

Then say what the state means.

```
# dp[i] = the number of distinct ways to reach step i
```

Then derive the base cases from the recurrence. It reads `i-1` and `i-2`, so indices 0 and 1 must be set by hand:

```
dp[0] = 1          # one way to be at the bottom: do nothing
dp[1] = 1          # one way to reach step 1: a single step
```

Then the loop, starting at the first index the recurrence can compute:

```
def climb(n: int) → int:
    # dp[i] = the number of distinct ways to reach step i
    if n ≤ 1:
        return 1
    dp = [0] * (n + 1)
    dp[0], dp[1] = 1, 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]      # last move: one step, or two
    return dp[n]
```

Now the same four lines with a different operator. House robber — the recurrence is a `max` rather than a sum, because we are optimising rather than counting:

```
def rob(money: list[int]) → int:
    # dp[i] = the maximum money obtainable from houses 0..i
    n = len(money)
    if n == 0:
        return 0
    dp = [0] * n
    dp[0] = money[0]
    dp[1] = max(money[0], money[1]) if n > 1 else dp[0]
    for i in range(2, n):
        dp[i] = max(money[i] + dp[i - 2],    # rob i: i-1 is forbidden
                    dp[i - 1])              # skip i
    return dp[n - 1]
```

Note `dp[1] = max(money[0], money[1])`, not `money[1]`. The state says “the best from houses 0..1”, and the best of two houses when you may take only one is the larger. Deriving the base case from the sentence rather than from the pattern is what gets this right.

And the third operator — a feasibility problem, where `combine` is `any`. Word break:

```
def word_break(text: str, words: list[str]) → bool:
    # dp[i] = can text[0:i] be split entirely into dictionary words?
    vocabulary = set(words)
    n = len(text)
    dp = [False] * (n + 1)
    dp[0] = True # the empty prefix is trivially splittable
    for i in range(1, n + 1):
        dp[i] = any(dp[j] and text[j:i] in vocabulary # last move: the word text[j:i]
                    for j in range(i))
    return dp[n]
```

The last-move question again: to split `text[0:i]`, the final word ends at `i` and starts somewhere at `j`. So enumerate every `j`, and the split works if the prefix up to `j` works and `text[j:i]` is a word. **Here the enumeration is not two options but `i` of them**, which is why this one is $O(n^2)$ rather than $O(n)$ — the state count is the same, the work per state is not.

Now the case where the answer is not in the last cell. Maximum subarray:

```
def max_subarray(nums: list[int]) → int:
    # dp[i] = the maximum sum of a subarray ENDING EXACTLY at index i
    n = len(nums)
    dp = [0] * n
    dp[0] = nums[0]
    for i in range(1, n):
        dp[i] = max(dp[i - 1] + nums[i], # extend the subarray ending at i-1
                   nums[i])           # start a new one at i
    return max(dp)                    # NOT dp[n-1] - it ends somewhere
```

`return max(dp)` follows directly from the sentence. “Ending exactly at `i`” means each cell is a candidate, and the best one could be anywhere. **If the sentence had said “using elements up to `i`”, the answer would be `dp[n-1]`.**

And the space optimisation, which nearly all of these admit:

```
def max_subarray_optimised(nums: list[int]) → int:
    best_ending_here = best_overall = nums[0]
    for value in nums[1:]:
        best_ending_here = max(best_ending_here + value, value)
        best_overall = max(best_overall, best_ending_here)
    return best_overall
```

Two variables, because the recurrence reaches back exactly one. And `best_overall` is the `max(dp)`, maintained as we go rather than computed at the end — which is necessary once the table is gone.

The complete solution

```
"""The one-dimensional habit: 'what was the last move?', on four problems."""

from __future__ import annotations

def climb(n: int) → int:
    """dp[i] = distinct ways to reach step i. combine = + (counting)."""
    if n ≤ 1:
        return 1
    prev, curr = 1, 1 # dp[0], dp[1]
    for _ in range(2, n + 1):
        prev, curr = curr, prev + curr
    return curr

def rob(money: list[int]) → int:
    """dp[i] = max money from houses 0..i. combine = max (optimisation)."""
    if not money:
        return 0
    if len(money) == 1:
        return money[0]
    two_back, one_back = money[0], max(money[0], money[1])
    for i in range(2, len(money)):
        two_back, one_back = one_back, max(money[i] + two_back, one_back)
    return one_back

def word_break(text: str, words: list[str]) → bool:
    """dp[i] = can text[0:i] be split? combine = any (feasibility)."""
    vocabulary = set(words)
    longest = max((len(w) for w in vocabulary), default=0)
    n = len(text)
    dp = [False] * (n + 1)
    dp[0] = True
    for i in range(1, n + 1):
        # the last word ends at i and starts at j; it cannot be longer than `longest`
        dp[i] = any(dp[j] and text[j:i] in vocabulary
                    for j in range(max(0, i - longest), i))
    return dp[n]

def max_subarray(nums: list[int]) → int:
    """dp[i] = best sum of a subarray ENDING AT i. The answer is max(dp), not dp[-1]."""
    best_ending_here = best_overall = nums[0]
    for value in nums[1:]:
        best_ending_here = max(best_ending_here + value, value)
        best_overall = max(best_overall, best_ending_here)
    return best_overall

def min_cost_climbing(cost: list[int]) → int:
    """dp[i] = min cost to STAND ON step i. The top is PAST the last step."""
    n = len(cost)
    two_back, one_back = cost[0], cost[1]
    for i in range(2, n):
        two_back, one_back = one_back, cost[i] + min(two_back, one_back)
    return min(two_back, one_back) # finish from either of the last two

if __name__ == "__main__":
    print("climb(12) : ", climb(12))
    print("rob : ", rob([2, 7, 9, 3, 1]))
    print("word_break : ", word_break("applepenapple", ["apple", "pen"]))
    print("word_break (no) : ", word_break("catsandog", ["cats", "dog", "sand", "and", "cat"]))
    print("max_subarray : ", max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]))
```

```
print("max_subarray (-) :", max_subarray([-3, -1, -7]))
print("min_cost      :", min_cost_climbing([10, 15, 20]))
print("min_cost (long) :", min_cost_climbing([1, 100, 1, 1, 1, 100, 1, 1, 100, 1]))
```

Running it:

```
climb(12)      : 233
rob           : 12
word_break    : True
word_break (no) : False
max_subarray   : 6
max_subarray (-) : -1
min_cost      : 15
min_cost (long) : 6
```

Three things to look at. `max_subarray` on all-negative input returns `-1`, not `0` — because a subarray must be non-empty, so the answer is the least-bad single element. Initialising `best_overall = 0` instead of `nums[0]` is the classic bug here, and it only shows on all-negative input.

`min_cost` returns 15 for `[10, 15, 20]`: start on step 1, pay 15, and step straight past the top. The answer is `min(dp[n-1], dp[n-2])` because the top is one step beyond the array — not `dp[n-1]`, which would be 30.

And `word_break` correctly rejects "catsandog": every prefix is splittable up to "catsand", and then nothing completes it. That is `dp[i]` being `True` for several `i` and still `False` at `n`, which is what makes the feasibility form worth writing out rather than shortcutting.

6 What it costs

The general formula, from [day 143](#):

```
time = (number of states) x (work per state)
space = (number of states), before optimisation
```

For this family the state count is always `n`. What varies is the work per state — which is the length of the “last move” enumeration.

climbing stairs	$n \text{ states} \times O(1)$	2 possible last moves	$= O(n)$
house robber	$n \text{ states} \times O(1)$	2 possible last moves	$= O(n)$
max subarray	$n \text{ states} \times O(1)$	2 possible last moves	$= O(n)$
decode ways	$n \text{ states} \times O(1)$	2 possible last moves	$= O(n)$
word break	$n \text{ states} \times O(n)$	up to n last moves	$= O(n^2)$
LIS (simple)	$n \text{ states} \times O(n)$	up to n last moves	$= O(n^2)$
coin change	$n \text{ states} \times O(\text{coins})$	one per coin	$= O(n \times c)$

So the cost is decided by how many ways there are to arrive, and that number is visible in the recurrence before you write any code. Two options means linear; “any earlier index” means quadratic.

And a quadratic one can sometimes be improved by changing what the state means — longest increasing subsequence goes from $O(n^2)$ to $O(n \log n)$ by storing something cleverer, which is [day 152](#).

Space:

full table	$O(n)$	
window of 2	$O(1)$	-- when <code>dp[i]</code> reads only <code>i-1</code> and <code>i-2</code>
window of k	$O(k)$	

```
n = 1,000,000
full table (Python list of ints)  ~40 MB
two variables                    ~56 bytes
```

Word break is the exception in this family, and it is worth noticing why:

```
dp[i] reads dp[j] for ALL j < i
→ the whole table is live
→ no space optimisation
```

Unless you bound the enumeration, which the code above does: the last word cannot be longer than the longest dictionary word, so `j` only ranges over the last `L` positions.

```
n = 10,000 characters, longest word 20
unbounded  10,000 x 10,000 = 100,000,000 substring checks
bounded    10,000 x 20     = 200,000
→ 500x
```

That bound is the single most valuable optimisation in this family and it comes from thinking about the last move rather than about the code.

The substring cost, which is easy to miss:

```
text[j:i] in vocabulary
    → building the substring is  $O(i - j)$ , not  $O(1)$ 
    → so word break is really  $O(n * L * L)$  with the bound, or  $O(n^3)$  without
```

For interview sizes that is fine; for very long strings the fix is a trie over the dictionary, walking forward from `j` one character at a time — which is [day 122](#)'s shape reused.

Comparison with the naive recursion, which is the reason any of this exists:

```
climbing stairs, n = 35
naive          18,454,929 calls
DP             35 computations
                → ~500,000x
```

And the constant factors within DP, which occasionally matter:

```
n = 1,000,000
full table          ~0.15 s, 40 MB
two variables       ~0.10 s, 56 bytes
memoised recursion crashes at ~960 depth
```

7 The traps

Not saying what `dp[i]` means

The root cause of most of the others, and it produces an empty editor rather than an error.

```
dp = [0] * n
for i in range(1, n):
    dp[i] = ...    # ... what?
```

Write the sentence first, as a comment. “The maximum sum of a subarray ending exactly at `i`.” Then the recurrence follows from the last-move question, and the base cases follow from the recurrence.

The answer in the wrong place

```
return dp[n - 1]          # when dp[i] means "ending exactly at i"
```

```
>>> max_subarray_wrong([-2, 1, -3, 4, -1, 2, 1, -5, 4])
5
# the answer is 6, at index 6
```

“Ending exactly at `i`” means every cell is a candidate, so the answer is `max(dp)`. “Using everything up to `i`” means the last cell covers it all, so the answer is `dp[n-1]`. The sentence tells you which, and guessing has a fifty percent success rate.

Base cases from intuition rather than from the recurrence

```
dp[1] = money[1]          # house robber: "the best from house 1 is house 1"
```

```
>>> rob_wrong([5, 1, 5])
6                               # the answer is 10
```

The state says “the best from houses 0..1”, and with two houses and only one allowed, that is `max(money[0], money[1])`. **Derive the base case from the sentence, not from the pattern of the recurrence.**

The maximum-subarray initialisation

```
best_overall = 0           # instead of nums[0]
```

```
>>> max_subarray_zero_init([-3, -1, -7])
0                               # the answer is -1
```

A subarray must be non-empty, so the answer on all-negative input is the least-bad element. **Initialise from the data, not from a neutral value**, and test with all-negative input — it is the hidden test on this problem and on several others in the family.

Off-by-one between `n` and `n + 1`

```
dp = [0] * n               # but the recurrence indexes up to n
```

```
IndexError: list assignment index out of range
```

Two conventions, and mixing them is the problem. Either `dp[i]` means “the answer at position `i` of the array”, sized `n`; or `dp[i]` means “the answer using the first `i` elements”, sized `n + 1` with `dp[0]` as the empty case. **The second is usually cleaner for string and prefix problems** — word break’s `dp[0] = True` is the empty prefix — and the first for positional ones. Pick one, say which in the comment, and stay with it.

Forgetting the empty or single-element input

```
dp[0], dp[1] = money[0], max(money[0], money[1])
```

```
IndexError: list index out of range          # on a one-element input
```

Every problem in this family has `n = 0` and `n = 1` cases that the recurrence cannot express. **Handle them before the table**, explicitly, rather than trying to make the loop cover them.

The unbounded enumeration

```
dp[i] = any(dp[j] and text[j:i] in vocabulary for j in range(i))
```

Correct, and $O(n^2)$ substring checks where $O(n \cdot L)$ would do:

```
n = 10,000, longest word 20
unbounded    100,000,000 checks
bounded      200,000
```

The bound comes from the last-move question: the final word cannot be longer than the longest word in the dictionary, so `j` starts at `i - L`. **That is the recurrence telling you the loop range**, and it is free.

Treating a two-dimensional problem as one-dimensional

```
dp[i] = best answer at position i          # but the answer also depends on how much budget is left
```

If the last-move enumeration needs to know something *besides* the position — remaining capacity, how many transactions used, whether you are currently holding something — **then the state is not one number and no one-dimensional table can be correct**. The symptom is that you cannot finish the sentence “`dp[i]` is the answer when `__`” without adding a second clause. **That clause is the second dimension**, and it is [day 147](#).

8 In the interview

How it gets asked

- “How many ways to climb `n` stairs taking one or two steps?” — the canonical one.
- “Maximise the sum with no two adjacent elements.” — house robber.

- “Find the contiguous subarray with the largest sum.”
- “How many ways can this digit string be decoded?”
- “Can this string be split into dictionary words?”
- “What does `dp[i]` represent?” — the follow-up to all of them.

The first ninety seconds

“Let me get the recurrence with one question: **what was my last move, and where was I before it?**

To be standing on step `i`, my last move was either a single step from `i-1` or a double step from `i-2`. Those are the only two possibilities. So the number of ways to reach `i` is the number of ways to reach `i-1` plus the number of ways to reach `i-2` — every way of getting to either of those extends to exactly one way of getting here.

And I would write down what the state means before the loop: `dp[i]` is the number of distinct ways to reach step `i`. That sentence does two things for me. It tells me the base cases — the recurrence reads `i-1` and `i-2`, so indices 0 and 1 have to be set by hand, and ‘one way to be at the bottom’ and ‘one way to reach step 1’ both follow from the sentence rather than from the pattern. And it tells me where the answer lives: ‘ways to reach step `i`’ means `dp[n]` is exactly what was asked.

That last point is worth flagging because it is not always true. If the state were ‘the best subarray ending exactly at `i`’, the answer would be `max(dp)`, because the best one ends somewhere and I do not know where. **Reading my own sentence is how I decide, and guessing is a coin flip.**

Cost is $O(n)$ — `n` states, constant work each, because the last-move enumeration has two options. Space is $O(n)$ for the table, and then $O(1)$ because `dp[i]` reads only the two cells below it, so two variables suffice.

The one thing I would check before coding is `n = 0` and `n = 1`, because the recurrence cannot express them and every problem in this family has an edge case there.

Shall I write it, or would you like the recurrence for a variation first?”

The follow-ups

“How did you get the recurrence?”

“By asking what the last move was, and I would say that this is a habit rather than an insight — it works on every problem of this shape.

The question turns ‘how do I solve this’, which is unbounded, into ‘what are the ways I could have arrived here’, which is a short list I can write down. For stairs it is two: one step or two steps. Each one leaves me at a specific earlier position whose answer I already have, so I combine them.

The same question on house robber: at house `i` I either robbed it or I did not. If I robbed it, then `i-1` was forbidden, so my previous state is the best up to `i-2`. If I skipped it, my state is the best up to `i-1`. Two options, `max` between them.

On maximum subarray: the subarray ending at `i` either extends the one ending at `i-1` or starts fresh at `i`. Two options again.

What changes between problems is only the combining operator. Counting problems add. Optimisation problems take a `max` or a `min`. Feasibility problems take an `any`. **Same skeleton, different operator**, and knowing which of the three I have is worth saying out loud because it also tells me what to initialise the table with — zero for counting, negative infinity or the first element for maximisation, `False` for feasibility.”

“What does `dp[i]` represent?”

“The number of distinct ways to reach step `i` — and I would say that this question is the one that actually separates people, more than the code.

The reason it matters is that similar-looking states behave completely differently. ‘The best subarray ending exactly at `i`’ and ‘the best subarray using elements up to `i`’ are two different states, and the recurrence, the base cases and *where the answer lives* are all different. With the first, the answer is `max(dp)` ; with the second, it is `dp[n-1]` . Write the recurrence for one while thinking of the other and it is wrong in a way that passes small tests.

The test I use is: given only the state, can I compute the answer without knowing anything else about how I got here? For stairs, ‘I am on step 8’ is enough — the ways to finish from there do not depend on the route that reached it.

And the moment that test fails, I need another dimension. If the answer also depends on how much budget is left, or how many transactions I have used, or whether I am currently holding something, then no one-dimensional table can be right. **The symptom is that I cannot finish the sentence ‘`dp[i]` is the answer when __’ without adding a second clause, and that clause is the second index.”**

“Reduce the space.”

“By looking at what the recurrence reaches back to. Here `dp[i]` reads `dp[i-1]` and `dp[i-2]` and nothing else, so I need two numbers at any moment and the rest of the array is history — two variables rolled forward, $O(n)$ to $O(1)$.

The rule is that the window is as deep as the recurrence reaches, so reaching back `k` needs `k` values.

The one in this family that does not collapse is word break, and it is worth saying why: `dp[i]` reads `dp[j]` for every `j` below it, so the whole table is live and there is no window. **But bounding the enumeration helps enormously anyway** — the final word cannot be longer than the longest dictionary word, so `j` only ranges over the last `L` positions. On a ten-thousand-character string with a longest word of twenty, that is two hundred thousand checks instead of a hundred million.

And when the answer is `max(dp)` rather than `dp[n-1]` , the optimisation needs one more variable — I have to carry the running best as I go, because once the table is gone I cannot take a maximum over it. That is two variables instead of one and it is easy to forget.

What I give up is the ability to say *which* choices produced the answer — which houses were robbed, which subarray was best. If that is wanted, I keep the table, or I track the start and end indices alongside the running maximum.”

“Now you can take one, two or three steps. And there are broken steps you cannot use.”

“Both changes are in the last-move enumeration, and neither changes the shape.

Three step sizes means three possible last moves instead of two: $dp[i] = dp[i-1] + dp[i-2] + dp[i-3]$. The base cases extend to three indices, because the recurrence now reaches back three. Still $O(n)$ — the work per state went from two additions to three, which is a constant. **And the space window becomes three variables rather than two.**

Broken steps are a filter on the state rather than on the enumeration: if step i is broken, $dp[i] = 0$ — there are zero ways to be standing on it — and the loop skips computing it. Everything above it then naturally sees zero ways through it, so the answer propagates correctly with no special cases.

The thing I would be careful about is the base cases when a base index is itself broken. If step 1 is broken, $dp[1] = 0$ rather than 1, and I would check the smallest inputs by hand rather than assume the pattern holds.

And the general point: almost every variation on a one-dimensional DP is either a change to the enumeration — more moves, so more terms — or a filter on the state — some positions are invalid. **The skeleton does not change**, which is why the habit is worth more than any individual problem.”

The model answer

“A string of digits was encoded with $A=1$ through $Z=26$. How many ways can it be decoded? For example, ‘226’ could be BZ, VF or BBF.”

“Same habit: what was the last move?

To decode a string of length i , the last letter I produced used either the last one digit or the last two digits. Those are the only options, because no letter uses three digits — Z is 26.

So $dp[i] = dp[i-1] + dp[i-2]$, conditionally — and the conditions are the whole problem.

$dp[i-1]$ counts only if $s[i-1]$ is a valid single digit, which means it is 1 to 9. **Zero is not a letter**, so a ‘0’ contributes nothing on its own.

$dp[i-2]$ counts only if $s[i-2:i]$ is between 10 and 26. That excludes ‘07’ — which is not seven, it is invalid — and excludes ‘27’ upwards.

$dp[i]$ = the number of ways to decode the first i characters. I would use the length convention rather than the position one, sizing the table $n + 1$, because the conditions are naturally about ‘the last one or two characters’ and that reads more cleanly.

Base case: $dp[0] = 1$. There is exactly one way to decode the empty string — the empty decoding. That is not intuition; it is what makes $dp[2]$ come out right for a two-digit input, and I would verify it by hand on ‘12’ before trusting it.

The zeros are where every wrong answer in this problem comes from, so I would enumerate them:

- ‘0’ alone → zero ways, because nothing decodes to a leading zero.
- ‘06’ → zero, because ‘06’ is not 6.
- ‘10’ → one way, J, because the zero must pair with the 1.
- ‘100’ → zero, because the second zero has nothing valid to pair with.

A ‘0’ can only ever be consumed as the second digit of a 10 or a 20, and if it cannot be, the whole string has zero decodings from that point.

Cost: $O(n)$ time, n states with two constant-time checks each. $O(1)$ space, because it reaches back two — and I would write the two-variable version after the table version.

What I would test before saying I am done: ‘0’, ‘06’, ‘10’, ‘100’, ‘226’, and a long string of ones, which is the maximum-answer case and grows like Fibonacci. **Those six inputs cover every branch**, and for a problem where the recurrence is three lines and the conditions are five, the conditions are what the interview is about.”

9 Recall card

The habit: “what was the last move, and where was I before it?” Enumerate the possible last moves — usually two — and each points at a strictly earlier index. That produces the recurrence for every linear DP.

Say what `dp[i]` means as a full sentence before the loop. It gives you the base cases (the indices the recurrence cannot compute) **and where the answer lives:** `dp[n-1]` for “using everything up to here”, `max(dp)` for “ending exactly here”, something else when the finish is past the end.

Same skeleton, three operators: `+` for counting, `max`/`min` for optimisation, `any` for feasibility — and the operator also tells you what to initialise the table with.

Cost = `n` states $\times$ the length of the last-move enumeration. Two options $\rightarrow O(n)$; “any earlier index” $\rightarrow O(n^2)$, and **bounding that enumeration** (the last word is at most `L` long) is often a $500\times$ win for free.

Nearly all of them collapse to `O(1)` space — a window as deep as the recurrence reaches. And if the answer is `max(dp)`, carry the running best in a second variable, because the table is gone.

How to run a high-level design interview: the forty-five-minute script

1 What this is, and why they ask it

Somebody says “design Twitter” and then stops talking. There is a whiteboard, or an empty document, and forty-five minutes.

The thing that goes wrong is not knowledge. Most candidates who fail this round know what a cache is and what sharding means. What they do not have is **an order**, so they start drawing boxes, get pulled into a detail somebody asked about at minute six, and run out of time having never mentioned the thing the interviewer was actually waiting for.

This lesson is a script. Six phases, with a rough number of minutes on each and a stated goal for each. It is not a formula for a good design — it is a way of not freezing, of covering the ground the interviewer is marking against, and of controlling the clock so that the interesting part gets time.

Everything from here to day 170 is a case study run against this script. **Today is the script itself**, and it is the most reusable thing in the entire system design track: the same six phases work for Twitter, Uber, Dropbox, a rate limiter and a system nobody has ever asked about before.

By the end of this lesson you can run the six phases with the clock, know what to say in each and what to leave out, recognise the four ways candidates lose the round, and handle the two hardest moments — being interrupted, and not knowing.

2 The story

Sister Mary has worked casualty at the same hospital for twenty-six years, and the thing she is known for is that she is slow to start.

A man came in on a Tuesday with his hand wrapped in a towel that was soaked through, and three people were talking at once, and the natural thing — the thing every new nurse does — is to go straight for the hand, because it is the thing everybody is looking at.

She did not go for the hand. She asked his name and whether he could tell her what happened. Not out of politeness: if he can answer, he is breathing and conscious and oriented, and that is three of her five questions settled in four seconds.

Then she looked at his colour and his breathing, then his pulse, then she asked what he had taken today, and only then did she unwrap the hand.

The registrar who was new that year found this maddening for about six months. There is a bleeding hand and she is asking about tablets.

What changed his mind was a Thursday when a woman came in having fallen off a scooter, with a bad graze down one arm that everybody was looking at, and Sister Mary went through the same five questions in the same order and on the third one — the pulse — she did not like it, and the woman was on a trolley and through to the back in under two minutes with something internal that nobody would have looked for another twenty minutes if they had started with the arm.

Her line about it, which she has said many times and which the registrar quotes to people now, is that the dramatic injury is very often not the dangerous one, and that **the order exists precisely because you cannot tell which is which by looking.**

The other thing about the order is what it does for her when she is tired. On a bad night at four in the morning, with four people waiting and someone shouting in the corridor, she does not have to decide what to do next. **The order decides, and she just does it**, and that is worth more at four in the morning than any amount of judgement.

3 The idea in plain English

Sister Mary's five questions are the script, and both of her observations apply directly.

The dramatic thing is usually not the dangerous thing. In a design interview the dramatic thing is the clever bit — the sharding scheme, the consistent hashing, the fan-out strategy. **The dangerous thing is usually the requirements, which nobody asks about**, or the scale estimate, which decides everything and takes three minutes.

And the order decides so you do not have to. Under pressure, with forty-five minutes and someone watching, having a fixed sequence means you are never choosing what to do next.

The six phases, with minutes:

PHASE	MINUTES	THE GOAL
1. Requirements	5	Agree what you are building and what you are not
2. Scale	5	Turn users into numbers that decide the design
3. API and data model	5	Fix the contract and the entities
4. High-level design	10	The boxes and arrows that satisfy phase 1
5. Deep dive	15	The one or two hard parts, in detail
6. Wrap-up	5	Bottlenecks, failures, what you would do next

Phase 1 — Requirements, five minutes. Ask, do not assume. “Design Twitter” means nothing until you know whether it includes search, direct messages, trends, ads, or none of those. **List three or four functional requirements and explicitly say what you are excluding.**

Then the non-functional ones, and these matter more: how many users, read-heavy or write-heavy, what latency, what availability, and — the one nobody asks — **what consistency does this actually need?** A feed can be seconds stale. A balance cannot.

Say your assumptions out loud and write them down. An interviewer who disagrees will correct you, which is free information. An assumption you never stated is one you can be marked down for later.

Phase 2 — Scale, five minutes. Turn the user count into numbers.

daily active users → requests per second → peak → storage per year → bandwidth

This is five minutes of arithmetic and it decides the whole design. Two hundred writes a second needs no sharding and no queue; two hundred thousand needs both. **A candidate who skips this is guessing for the next forty minutes,** and it shows.

Phase 3 — API and data model, five minutes. Three or four endpoints, with their parameters. Then the main entities and their key fields. **This is where you decide relational or not,** and it should follow from the access pattern rather than from preference.

Phase 4 — High-level design, ten minutes. Now draw. Client, load balancer, services, databases, caches, queues. **Trace one write and one read through the whole picture out loud** — that is what makes it a design rather than a diagram, and it catches missing pieces immediately.

Keep it boring. The high-level picture should be unsurprising. The interesting decisions belong in phase 5, and putting them here means you spend your best fifteen minutes explaining boxes.

Phase 5 — Deep dive, fifteen minutes. The largest block, and the one that is actually being marked.

Ask which part to go deep on. “The two hard parts here are the feed fan-out and the search index — which would you like?” That is not weakness; it is how you avoid spending fifteen minutes on the half they did not care about. If they say “you choose”, pick the one with a real trade-off and say why.

Then go properly deep: the mechanism, the numbers, the failure mode, and the alternative you rejected. Depth is what separates candidates, and it is impossible if phases 1 to 4 ran long.

Phase 6 — Wrap-up, five minutes. Name the bottleneck. Say what breaks first as traffic grows tenfold. Name what you would monitor. And say what you deliberately left out and would do next.

Ending with “here is what I would do next” is much stronger than trailing off, because it shows you know the design is unfinished, which every real design is.

Now the four ways people lose this round, and they are all about time rather than knowledge:

Drawing before asking. Twenty minutes into a design for a system whose requirements were never agreed. Everything after that is unmarkable, because it might be answering the wrong question.

No numbers. “We’ll shard the database” without knowing whether one machine would do. **Every design decision should be traceable to a number from phase 2,** and if it cannot be, it is decoration.

Breadth with no depth. Mentioning fifteen components and going into none. The interviewer needs to see you reason about one thing properly, not list things you have heard of.

Losing the clock. Being at minute forty with the deep dive not started. **The fix is a running commentary on time** — “I’ve used about ten minutes on the high-level, so let me go deep on the fan-out now” — which also signals that you are managing the session rather than being carried by it.

And the two hard moments.

Being interrupted. An interviewer’s question is almost always a hint, not an attack. Answer it, then say where you were. If it drags you off the script for five minutes, say so: “That’s taken us into the deep dive early — shall I carry on here, or finish the high-level first?”

Not knowing. Say so, and then say how you would find out or what you would use instead. “**I don’t know Cassandra’s exact consistency levels, but the property I need is that a read sees my own write, and I would check whether it can give me that**” is a much better answer than a confident wrong one — and interviewers generally cannot tell how much you know, only how you handle not knowing.

4 The picture

The script, with the clock:

0	----	5	-----	10	-----	15	-----	25	-----	40	----	45
	1.		2.		3.		4.		5.		6.	
	REQS		SCALE		API +		HIGH-LEVEL		DEEP DIVE		WRAP	
					DATA		DESIGN					
	ask,		numbers		contract		boxes, arrows,		one or two hard parts:		bottle	
	scope		that		and		trace one write		mechanism, numbers,		necks,	
	out		decide		entities		and one read		failure, alternative		next	
											steps	
_____ 15 minutes of setup _____/								___ the part being marked ___/				

What to notice. The first fifteen minutes are setup and should feel almost mechanical. **The deep dive is a third of the session**, and every minute overspent earlier comes out of it.

What each phase produces, concretely:

PHASE 1 OUTPUT

in scope: post a tweet, see a home feed, follow a user
out of scope: search, DMs, ads, trends ← say this explicitly
non-functional: read-heavy ~100:1, feed can be 5s stale, 99.9% availability

PHASE 2 OUTPUT

200M DAU, 2 posts/user/day → 400M posts/day → 4,600 writes/s, peak ~14,000
100:1 read ratio → 460,000 reads/s
300 bytes/post x 400M x 365 → 44 TB/year
→ writes fit on one machine; reads do not. THAT is the design driver.

PHASE 3 OUTPUT

POST /tweets {text} → {id, created_at}
GET /feed?cursor=...&limit=20 → [{tweet}]
POST /follow {user_id}
entities: users, tweets, follows(follower_id, followee_id)

PHASE 4 OUTPUT

a diagram, and one write and one read traced through it out loud

PHASE 5 OUTPUT

fan-out on write vs on read, with the celebrity number,
the hybrid, and what it costs

PHASE 6 OUTPUT

bottleneck: feed assembly for high-follower accounts
at 10x: the fan-out queue, before the database
monitor: feed p99, fan-out lag, cache hit rate
next: search, and per-user ranking

What to notice about phase 2. The last line — “writes fit on one machine; reads do not” — is a *decision* produced by arithmetic. **Every later choice should point back to a line like that.**

And the four failure modes, as a timeline:

GOOD	0---5---10---15-----25-----40--45 reqs scale api high-level deep dive wrap
drawing first	0-----45 boxes, boxes, boxes, and an unclear question
no numbers	0---5---[skipped]---api---high-level-----45 every decision is a guess wearing confidence
breadth, no depth	0---5---5---5---[15 components, 2 min each]--45
lost the clock	0-----12-----22-----38---[deep dive]-45 5 minutes left

5 How it actually works

The script, phase by phase, with what to actually say.

Phase 1 — Requirements (5 min)

Ask four or five questions. Do not design yet.

```
"Which of these are in scope: posting, the home feed, following, search, DMs, notifications?"  
"How many users, roughly? Daily active?"  
"Read-heavy or write-heavy, and by roughly what ratio?"  
"How fresh does the feed need to be – seconds, or can it be a minute behind?"  
"Is this global or one region?"
```

Then state the scope back, including the exclusions:

“So: posting, the home feed and following are in scope; search, DMs and ads are out. Two hundred million daily actives, heavily read-skewed, and the feed can be a few seconds stale. I’ll assume global with users concentrated in a few regions — tell me if that’s wrong.”

The exclusions matter as much as the inclusions, because they are what stops you being marked down for not covering something you deliberately dropped.

Phase 2 — Scale (5 min)

The standard chain, and it is always the same four steps:

1. users → actions per user per day → actions per day
2. actions per day / 86,400 → average per second; x3 to x5 → peak
3. bytes per record x records per day x 365 → storage per year
4. bytes per response x reads per second → bandwidth

Worked, out loud:

200,000,000 DAU x 2 posts	= 400,000,000 posts/day
400,000,000 / 86,400	= ~4,600 writes/s
x3 for peak	= ~14,000 writes/s
reads at 100:1	= ~460,000 reads/s
300 bytes/post x 400M	= 120 GB/day
x 365	= 44 TB/year

Then the sentence that turns arithmetic into a decision:

“So the write path is modest — fourteen thousand a second is one well-provisioned database with a queue in front. The read path is four hundred and sixty thousand a second, which is not, and that is the whole design problem.”

Round aggressively. 86,400 is 100,000. A day is 10^5 seconds. **Nobody is checking your arithmetic; they are checking whether you use numbers at all.**

Numbers worth having memorised so the arithmetic is fast:

1 day	~100,000 seconds (86,400)
1 million/day	~12 per second
1 billion/day	~12,000 per second
1 KB x 1M/day	~1 GB/day, ~365 GB/year

Phase 3 — API and data model (5 min)

Three or four endpoints, with parameters and return shapes. Not a full specification.

POST /tweets	{text}	→ {id, created_at}
GET /feed	?cursor=&limit=20	→ [{id, author, text, created_at}]
POST /users/{id}/follow		→ 204

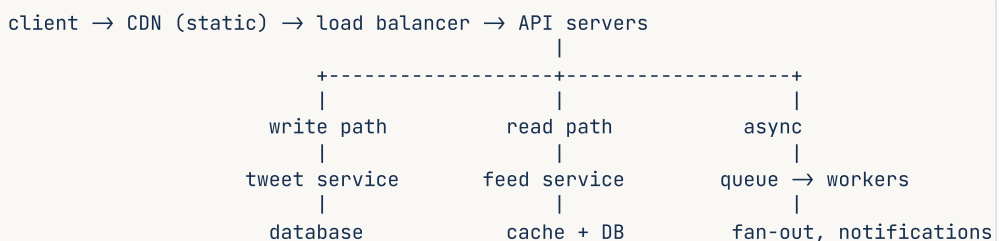
Cursor-based pagination, not offset, and say why in half a sentence: offsets shift as new items arrive, so a user paging through a feed sees duplicates and gaps. **That is a small detail that signals experience.**

Then the entities and the storage choice, derived from the access pattern:

“Users, tweets, and a follows table with (follower_id, followee_id). The follows table is the interesting one — I need ‘who follows me’ for fan-out and ‘who do I follow’ for the read path, so I’d index both directions. Tweets are append-only and never updated, which is a hint that they don’t need a relational store, but I’d start relational and move if the numbers say so.”

Phase 4 — High-level design (10 min)

Draw the boring version.



Then trace one write and one read out loud, which is the part that makes it a design:

“A post: client hits the API, we write the tweet to the database, publish an event, and return. The fan-out worker picks up the event and pushes the tweet id into the feed cache of each follower.

A feed read: client hits the API, we read the precomputed list of tweet ids from the cache, hydrate them from the tweet store — probably also cached — and return twenty. Cache miss falls back to assembling from the database.”

Tracing catches missing pieces immediately, and it is much more convincing than pointing at boxes.

Phase 5 — Deep dive (15 min)

Offer a choice:

“The two genuinely hard parts here are the feed fan-out and the celebrity problem. Which would you like me to go into?”

Then, whichever it is, cover four things:

- 1. The mechanism.** How it actually works, concretely.
- 2. The numbers.** What it costs, sized from phase 2.
- 3. The failure mode.** What breaks, and what happens then.
- 4. The alternative.** What you rejected and why.

That fourth one is what makes it a trade-off rather than a decision, and it is the most commonly missing part.

Phase 6 — Wrap-up (5 min)

Four sentences:

“The bottleneck is feed assembly for accounts with millions of followers — everything else has headroom.

At ten times the traffic, the fan-out queue saturates before the database does, and I would shard the workers by user id.

I would monitor feed p99, fan-out lag as a time rather than a queue depth, and cache hit rate — a hit rate below eighty percent means the cache is adding a hop rather than saving one.

What I have left out: search, ranking, and media. Search would be a separate index fed from the same event stream, and I would do that next.”

Naming what you left out is not a weakness. It shows you know the boundary of what you built, and it gives the interviewer somewhere to take the conversation if there is time.

Handling interruptions

Almost every interruption is a hint. **Three responses:**

If it is a small correction, take it and move on. “Good point, yes — the cursor should be the tweet id, not a timestamp, because timestamps collide.”

If it pulls you deep early, answer properly and then reorient: “That’s the fan-out question, which I was going to cover in the deep dive — shall I do it now and come back to the high-level after?”

If you disagree, say so once, with a reason, and then defer: “I’d push back slightly — at fourteen thousand writes a second I don’t think we need to shard yet. But if you’d like me to design for a hundred times that, I’ll change the storage layer.”

The universal opening

If you are given a system you have never thought about, the first ninety seconds are always the same:

“Let me start by pinning down what’s in scope, then size it, and then design against those numbers. Can I ask a few questions first?”

That sentence works for every prompt, and it buys you thirty seconds to think while sounding like a plan.

6 The numbers

The estimation toolkit — memorise these six and the arithmetic is fast.

seconds in a day	86,400	→ round to 100,000
1 million/day	~12/second	
1 billion/day	~12,000/second	
peak multiplier	3x to 5x average	
read:write ratio, social	100:1 to 1000:1	

The latency ladder — for justifying where things live:

memory	100 ns
SSD read	100 us
same-datacentre hop	0.5 ms
cross-region	80 ms
cross-planet	200 ms

Capacity per machine — for deciding how many:

web server	~10,000 requests/s
Redis	~100,000 ops/s
Postgres	~10,000 writes/s; a hot row ~200/s
Kafka	~100,000 msg/s per broker
WebSocket connections	~50,000-100,000 per machine

Storage per record — for the yearly figure:

a tweet / short post	~300 bytes
a user row	~1 KB
a photo	~2 MB
a video minute	~50 MB
a log line	~200 bytes
a metric point	~1.5 bytes compressed

A full worked estimate, in the order you would say it:

"Design Instagram."

200M DAU, 1 photo uploaded per user per week

uploads/day	$200,000,000 / 7$	= ~28,000,000
per second	$28,000,000 / 100,000$	= ~280 writes/s
peak x5		= ~1,400 writes/s
		→ writes are easy

feed views: 20 per user per day

reads/day	$200,000,000 \times 20$	= 4,000,000,000
per second	$4,000,000,000 / 100,000$	= ~40,000 reads/s
peak x5		= ~200,000 reads/s
		→ reads are the problem

storage

2 MB per photo x 28,000,000/day	= 56 TB/day
x 365	= 20 PB/year
	→ object storage, obviously
	→ and the CDN is not optional

bandwidth

200,000 reads/s x 200 KB (a thumbnail)	= 40 GB/s
at \$0.09/GB egress	= \$3,600/second at origin prices
	→ a CDN is a COST decision, not a latency one

That last line is the point of doing the arithmetic. The CDN was already obvious; the number tells you it is the largest line on the bill and therefore a first-class design concern rather than an afterthought.

Time budget within the interview itself:

phase 1	5 min	~10%	
phase 2	5 min	~10%	
phase 3	5 min	~10%	
phase 4	10 min	~22%	
phase 5	15 min	~34%	← the part being marked
phase 6	5 min	~10%	

If you are at minute 25 and have not started phase 5, you are behind — and the fix is to compress phase 4 by saying “let me keep the high-level brief and spend the time on the fan-out.”

7 The trade-offs

The script costs you spontaneity and buys you coverage. Following it means you will occasionally spend five minutes on requirements for a problem where the interviewer wanted to jump straight in. **That is a much smaller cost than the alternat-**

ive, which is designing the wrong system confidently. If the interviewer says “assume the requirements, go”, drop phase 1 to ninety seconds and keep the rest.

Breadth against depth, and the script is deliberately weighted. Fifteen of the forty-five minutes are one or two components in detail. That means whole areas go unmentioned — and **saying so in phase 6 is how you get credit for knowing they exist** without spending time on them.

Numbers against speed. Five minutes of arithmetic is a ninth of the session and it is the highest-value five minutes there is, because every subsequent decision either follows from it or is a guess. **The only time to skip it is when the interviewer explicitly says the scale is not the point.**

Asking questions against appearing decisive. Some candidates worry that asking looks uncertain. It does not — **it looks like someone who has built something before**, because nobody who has built a real system starts without agreeing scope. The failure mode is asking too many, or asking questions whose answers do not change the design. **Ask four or five, and only ones whose answer would change what you build.**

And the honest limit of the script: it is a way of not failing, not a way of being brilliant. It guarantees you cover requirements, scale, structure and depth. What it cannot supply is the insight in phase 5 — the observation that a celebrity breaks fan-out on write, or that driver locations do not need durability. **The script gets you to the point where that insight is worth having**, which is the most it can do.

When would I deviate? When the interviewer drives — follow them, and use the script only to notice what is being skipped. When the prompt is a narrow component rather than a system — “design a rate limiter” — where phases 1 and 2 compress to two minutes and the deep dive is almost the whole session. And when you genuinely know the domain deeply, where leading with the hard part can be stronger, as long as requirements and scale still get their five minutes.

8 In the interview

How it gets asked

- “Design Twitter.” — and nothing else. The prompt is deliberately empty.
- “Design a URL shortener / a chat app / Uber.”
- “Design a system you have never seen before.” — day 170’s mock.
- “How would you approach this?” — occasionally asked directly, which is a gift.

The first ninety seconds

“Let me pin down what’s in scope, then size it, and then design against those numbers — that way every choice traces back to something. Can I ask a few questions first?

What’s in scope? Posting, the home feed and following, I assume — is search in scope? Direct messages? Ads?

How many users? Daily active rather than registered, if you have a figure.

Read-heavy or write-heavy? For something feed-shaped I’d assume very read-skewed, maybe a hundred to one, but I’d rather use your number.

And how fresh does the feed have to be? Because ‘within a few seconds’ and ‘immediately’ lead to completely different designs.

...

Right. So: posting, feed and follows in scope; search and DMs out of scope. Two hundred million daily actives, roughly a hundred to one read-skewed, and a few seconds of feed staleness is acceptable. I’ll assume we’re global with the users concentrated in a handful of regions — stop me if that’s wrong.

Let me spend a couple of minutes turning that into numbers, because the write path and the read path are going to need very different answers and I want to know which one is actually hard before I design either.”

The follow-ups

“Skip the requirements, just design it.”

“Happily — let me state what I’m assuming in about thirty seconds so you can correct me, and then go straight to the design.

I’m assuming posting, home feed and follows are in scope and search isn’t; a couple of hundred million daily actives; heavily read-skewed; and that a few seconds of feed staleness is fine. If any of those is wrong, the design changes and I’d rather know now than at minute thirty.

The reason I still say them rather than skipping entirely is that half of them are load-bearing. If the feed had to be immediately consistent I couldn’t precompute it, and that changes the whole architecture. So it’s thirty seconds rather than five minutes, but it isn’t zero.

Then I’d go straight to sizing, because I need to know whether the read path is four thousand a second or four hundred thousand before I decide whether it needs a cache at all.”

“You’re spending too long on estimation.”

“Fair — let me get to the decision and move on.

The numbers I actually need are: about fourteen thousand writes a second at peak, about four hundred and sixty thousand reads a second at peak, and forty-four terabytes a year. **The decision that follows is that the write path fits comfortably on a single primary with a queue in front, and the read path does not fit anywhere without precomputation** — so the design problem is entirely on the read side.

That’s the only thing I needed the arithmetic for, and I’ll refer back to those three numbers rather than recomputing.

The reason I do it at all is that without it I’d be guessing whether to shard, and sharding a system that doesn’t need it is a common and expensive mistake. But you’re right that thirty seconds of it is worth as much as five minutes.”

“How do you handle not knowing something?”

“Say so immediately, and then say what I’d do about it — because a confident wrong answer is much worse than an honest gap, and interviewers can usually tell the difference.

Concretely: **‘I don’t know Cassandra’s exact consistency level names. What I need here is read-your-writes for the author’s own timeline, and I’d check whether it can give me that — if not, I’d read the author’s own recent posts from the primary and merge them in.’** That answers the actual question, shows I know what property matters, and does not pretend.

The related move is to say what I would measure rather than guess a number. If asked what the cache hit rate would be, ‘I don’t know, and it depends on the access distribution — I’d instrument it before sizing the cache, and design so that a low hit rate degrades rather than breaks’ is better than inventing ninety percent.

And if it is something I genuinely should know, I’d say that too and move on rather than dwelling. One sentence of acknowledgement, then back to the design — the round is forty-five minutes and there is no time to be embarrassed in.”

“You have five minutes left and you’re mid-deep-dive.”

“Then I’d stop the deep dive and spend the five on the wrap-up, because an unfinished deep dive plus a summary beats a finished deep dive with no summary.

Concretely I’d say: ‘Let me pause here and close out, and I can come back if there’s time.’ Then the four wrap-up sentences — the bottleneck, what breaks at ten times the load, what I’d monitor, and what I deliberately left out.

The reason that ordering is right is that the wrap-up is where I demonstrate that I know the design is incomplete and know *how* it is incomplete, which is a senior signal. Trailing off mid-sentence at forty-five minutes signals the opposite, even if the content was better.

And I’d watch the clock out loud earlier than that, so it doesn’t happen. Around minute twenty-five I would normally say ‘I’ve used about ten minutes on the high-level, so let me go deep on the fan-out now’ — which both manages the time and signals that I’m running the session rather than being carried by it.”

The model answer

“Design a URL shortener.” — run against the script, compressed to show the shape.

Phase 1, requirements — two minutes. “Is custom aliasing in scope? Analytics on clicks? Expiry? I’ll assume: shorten a URL, redirect, optional custom alias, and basic click counts — and I’ll exclude user accounts and ads. Non-functional: redirects must be fast, single-digit milliseconds, because they’re on the critical path of somebody else’s page load. Availability matters more than consistency — a redirect that works is worth more than one that’s perfectly fresh. And it’s extremely read-heavy.”

Phase 2, scale — three minutes. “Say a hundred million new URLs a month. That’s about forty a second on average, a couple of hundred at peak — trivial. Reads at a hundred to one is four thousand a second average, maybe twenty thousand peak. Storage: a hundred million a month at, say, five hundred bytes is fifty gigabytes a month, six hundred gigabytes a year — small. **The decision: this fits on one database. The interesting problem is not scale, it’s the ID generation and the redirect latency.**”

Phase 3, API — two minutes. “`POST /urls {long_url, custom_alias?}` returns `{short_url}`. `GET /{code}` returns a 301 or 302. Entities: a single table of `code`, `long_url`, `created_at`, `expires_at`, `click_count`. `code` is the primary key, which makes the redirect a single primary-key lookup — and that is the entire read path.”

Phase 4, high-level — five minutes. “Client, load balancer, stateless app servers, a cache in front of the database, and the database. Write path: generate a code, insert, return. Read path: look up the code in the cache, fall back to the database, redirect. Click counts go onto a queue rather than being an update on the read path, because I do not want a write on every redirect.”

Phase 5, deep dive — the code generation, and I’d offer the choice. “The two interesting parts are how the code is generated and how the redirect gets to single-digit milliseconds. Which would you like?

On generation: **hashing the URL and taking the first seven characters** is the obvious approach and it collides — birthday paradox says at a hundred million URLs over a 62^7 space, collisions are rare but real, so it needs a check-and-retry loop, and that is a read before every write.

A counter encoded in base 62 avoids collisions entirely, is shorter, and gives sequential — therefore guessable — codes, which matters if anyone shortens something private. **A distributed counter is the problem:** a single sequence is a

bottleneck and a single point of failure, so I'd hand each server a block of a thousand ids at a time from a central counter, which makes it one coordination call per thousand URLs and tolerates a server dying by wasting up to a thousand ids.

What I'd actually choose: the counter with block allocation, plus base-62 encoding, plus a random offset or a shuffled alphabet so codes are not trivially enumerable. And I'd support custom aliases through the same table with a uniqueness constraint, which is where the collision handling actually earns its keep.

The number that matters: seven base-62 characters is 3.5 trillion codes, which at a hundred million a month is about three thousand years. **Six characters would be fifty-seven billion, which is forty-seven years — also fine, and a shorter URL.** I'd start at six and have seven ready."

Phase 6, wrap-up — two minutes. "The bottleneck is the redirect read, and it is entirely a cache problem — with a hot-key distribution, a small cache covers most traffic, and I'd expect well over ninety percent.

At ten times the load, nothing structural changes; I'd add cache capacity and read replicas.

I'd monitor redirect p99, cache hit rate, and the block-allocation counter's remaining headroom.

What I left out: analytics beyond a click count, abuse prevention — which is genuinely important for a shortener and I'd do next — and expiry, which is a background job over `expires_at`. And I'd note that **301 versus 302 is a real decision:** 301 is cacheable by the browser and therefore fast and un-analysable, 302 gives me every click and costs a round trip. That is a product question, not a technical one."

9 Recall card

Six phases, forty-five minutes: requirements (5), scale (5), API and data model (5), high-level design (10), deep dive (15), wrap-up (5). The first fifteen minutes are setup and should feel mechanical; **the deep dive is what is being marked.**

Phase 1: ask, and state the exclusions. Phase 2: users → per second → peak → storage per year, and end with **the sentence that turns arithmetic into a decision** ("writes fit on one machine; reads do not").

Phase 4: draw the boring version and trace one write and one read out loud.
Phase 5: **offer a choice**, then cover mechanism, numbers, failure mode, and **the alternative you rejected** — that last one is what makes it a trade-off.

Four ways to lose it, all about time: drawing before asking; no numbers; breadth with no depth; and losing the clock. **Narrate the time** — “I’ve used ten minutes on the high-level, let me go deep now.”

Interruptions are hints — answer, then reorient. And **not knowing is fine if you say what you would do instead:** “I don’t know that product’s consistency levels; the property I need is read-your-writes, and I’d check whether it gives me that.”

Practice — Climbing stairs and the one-dimensional habit

DSA topic: Climbing stairs and the one-dimensional habit **System design topic:** How to run a high-level design interview: the forty-five-minute script

Code these, in this order

One rule for the whole set, and it is the only rule this phase needs: **write two comment lines before any code**. The first is `# dp[i] =` finished as a full English sentence. The second is the last-move enumeration — “to be here, my last move was either `__` or `__`”. Everything else follows from those two.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Climbing Stairs	LeetCode 70 (Easy)	The habit, on the simplest possible case.
2	House Robber	LeetCode 198 (Medium)	A genuine choice, and the base case <code>max(m[0], m[1])</code> .
3	Maximum Subarray	LeetCode 53 (Medium)	“Ending exactly at i” — so the answer is <code>max(dp)</code> .
4	Min Cost Climbing Stairs	LeetCode 746 (Easy)	The answer is past the end of the array.
5	Decode Ways	LeetCode 91 (Medium)	Two moves, five conditions — the conditions are the problem.
6	Word Break	LeetCode 139 (Medium)	An enumeration of length <code>n</code> , and how to bound it.
7	House Robber II	LeetCode 213 (Medium)	A circle: run the linear version twice and take the max.

On every problem, write the two comment lines first

Then check them. For problem 3, your sentence must contain “ending exactly at”, or the recurrence will be wrong in a way that passes small tests. For problem 4, your sentence must make it obvious that the top is one step past the last index.

On problem 3, test all-negative input first

Write the test before the solution: `max_subarray([-3, -1, -7])` must return `-1`. Then write the solution. Initialising the running best to `0` passes every other test and fails this one, and it is the hidden case.

On problem 5, enumerate the zero cases before coding

Write down what each of these should return, by hand: `"0"`, `"06"`, `"10"`, `"100"`, `"226"`, `"1111"`. Then write the solution and check all six. **A '0' can only ever be the second digit of a 10 or a 20**, and every wrong answer to this problem comes from that.

On problem 6, measure the bound

Solve it with `for j in range(i)` and again with `for j in range(max(0, i - longest_word), i)`. Time both on a 10,000-character string with a dictionary whose longest word is 20. Two numbers, and the ratio should be around 500.

Then the operator drill

Take problems 1, 2 and 6 and write down, for each: the state sentence, the last-move enumeration, the combining operator (`+`, `max`, or `any`), and the initial value of the table. **Notice that the operator and the initial value always agree** — `+` with `0`, `max` with negative infinity or the first element, `any` with `False`.

Then the space drill

For each of problems 1 to 5, reduce to `O(1)` and say what window you kept. For problem 3, notice that you need *two* variables rather than one, and say why.

For problem 6, say why it does not collapse.

The habit drill

1. State the question in one sentence.
2. Apply it to stairs, house robber and maximum subarray, out loud.
3. Say what makes an enumeration “short and backwards-pointing”, and why that means one dimension.
4. Say what the enumeration length does to the complexity.

The state drill

1. Give the state sentence for all seven problems above.
2. For each, say where the answer lives and why.

3. Give the three places an answer can live, and the sentence that tells you which.
4. Give the test for whether a state is complete.
5. Say what the symptom is when a state needs a second dimension.

The base-case drill

1. Say how to derive the base cases from the recurrence rather than from intuition.
2. Do it for `dp[i] = dp[i-1] + dp[i-2]` and for a three-step version.
3. Give house robber's `dp[1]` and justify it from the state sentence.
4. Say what a wrong base case looks like — the symptom, not the fix.

The operator drill

1. Name the three combining operators and the problem type each belongs to.
2. Give the matching initial value for each.
3. Give one problem of each type from the list above.

The costs drill

1. Give the general formula.
2. Give the cost of all seven problems above and say which term differs.
3. Say what bounding word break's enumeration buys, with the arithmetic.
4. Say why the substring construction makes word break worse than it looks, and the fix.

The break-it drill

Trigger each and record the exact output or error:

1. `return dp[n-1]` when the state means “ending exactly at i”.
2. `best_overall = 0` on all-negative input.
3. House robber with `dp[1] = money[1]`.
4. `dp = [0] * n` when the recurrence indexes to `n`.
5. Any of them on an empty or single-element input.
6. Word break with an unbounded `j` range, on a 10,000-character string.
7. A one-dimensional table on a problem whose answer also depends on a remaining budget.

The script drill

1. Name the six phases with their minute allocations.

2. Say the goal of each in one sentence.
3. Say what fraction of the time is setup and what fraction is being marked.
4. Say what you do at minute 25 if the deep dive has not started.

The requirements drill

1. Give the four or five questions you would ask on any prompt.
2. Say why the exclusions matter as much as the inclusions.
3. Say what makes a question worth asking, and what makes one a waste.
4. State the scope back for “design Twitter”, including what is out.

The estimation drill

1. Give the four-step chain.
2. Memorise and recite: seconds in a day, 1M/day per second, 1B/day per second, peak multiplier.
3. Do the full Twitter estimate out loud, ending with the decision sentence.
4. Do the full Instagram estimate, ending with why the CDN is a cost decision.
5. Give per-machine capacity for a web server, Redis, Postgres, Kafka and WebSockets.
6. Give bytes per record for a post, a user row, a photo, a video minute, a log line, a metric point.

The design drill

1. Say what phase 4 should feel like, and why.
2. Say what “trace one write and one read” catches.
3. Give the four things a deep dive must cover.
4. Say which of the four is most often missing.
5. Give the four wrap-up sentences.

The failure-modes drill

1. Name the four ways candidates lose this round.
2. For each, say what it looks like on the timeline.
3. Say which one is the most common and what the fix is.
4. Say what narrating the clock signals to an interviewer.

The hard-moments drill

1. Give the three responses to an interruption.
2. Give the phrasing for reorienting after being pulled deep early.

3. Give the phrasing for not knowing something.
4. Give the phrasing for disagreeing.
5. Say what to do with five minutes left and an unfinished deep dive.

The failure drill

For each, say what happens and what you would do:

1. The interviewer says “skip requirements, just design it.”
2. The interviewer says “you’re spending too long on estimation.”
3. You are at minute 40 and have not started the deep dive.
4. You realise at minute 30 that you misunderstood the requirement.
5. You are asked something you genuinely do not know.
6. The interviewer disagrees with a choice you made.
7. The prompt is a system you have never heard of.

Two of the seven are recoverable in under a minute. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *How many ways to climb `n` stairs taking one or two steps?* The last-move question producing the recurrence, the state sentence, base cases derived from the recurrence, where the answer lives, the cost, and the space collapse.
 2. *What does `dp[i]` represent?* The full sentence, why similar-looking states differ, the completeness test, and the symptom that means you need a second dimension.
 3. *Design Twitter. You have forty-five minutes. Begin.* The opening ninety seconds only: the framing sentence, four questions, the scope stated back with exclusions, and the move into sizing with a reason.
-

Before you move on

- [] I write the state sentence and the last-move enumeration before any code.
- [] I can apply the last-move question to three different problems out loud.
- [] I derive base cases from the recurrence, not from the pattern.
- [] I know the three places an answer can live and how to tell which.

- [] I know “ending exactly at i” means `max(dp)`.
- [] I know the three combining operators and their matching initial values.
- [] I test all-negative input on maximum-subarray-shaped problems.
- [] I can enumerate the five zero cases in decode ways.
- [] I know how to bound word break’s enumeration and what it buys.
- [] I know why word break does not collapse to `O(1)` space.
- [] I know the symptom that means the state needs a second dimension.
- [] I can name the six phases with their minutes.
- [] I can give the four or five requirements questions.
- [] I state exclusions explicitly.
- [] I can do the four-step estimation chain from memory.
- [] I end phase 2 with a decision sentence, not just numbers.
- [] I know phase 4 should be boring, and why.
- [] I trace one write and one read out loud.
- [] I offer a choice at the start of the deep dive.
- [] I cover mechanism, numbers, failure mode and the rejected alternative.
- [] I can give the four wrap-up sentences.
- [] I can name the four ways to lose the round.
- [] I narrate the clock at least once.
- [] I know the three responses to an interruption.
- [] I know the phrasing for not knowing something.
- [] I know what to do with five minutes left mid-deep-dive.
- [] I have the universal opening sentence ready.
- [] I answered all three questions above out loud.

House robber and the choice at each step

DATA STRUCTURES AND ALGORITHMS

House robber and the choice at each step

SYSTEM DESIGN

Design a URL shortener

House robber and the choice at each step

1 What this is, and why they ask it

Yesterday's problems had a fixed set of last moves. Today there is a **decision**: at every position you either take the thing in front of you or you do not, and taking it forbids something.

That is a different shape and it is worth its own day, because it is the first one where **greedy is wrong**. Always taking the largest available number looks obviously correct and is not — a single large item can block two slightly smaller ones. Being able to produce that counter-example on demand is half of what this question is testing.

The recurrence is one line and it is the template for a whole family: paint houses, delete and earn, stock problems with a cooldown, and every problem phrased as “choose a subset with no two adjacent”.

They ask it because it is short, because the greedy trap is real, and because the follow-up — “**now the houses are in a circle**” — has a genuinely elegant answer that separates people who understand the structure from people who memorised the line.

By the end of this lesson you can write the take-or-skip recurrence from the choice, produce the greedy counter-example, handle the circular variant with the two-run trick, recognise the family in problems that do not mention adjacency, and reconstruct which items were chosen rather than just the total.

2 The story

Ashok whitewashes houses, and in the six weeks before Diwali he turns down more work than he takes.

The reason is the lime. It has to be slaked the night before, which means the drum is set up at whichever house he is doing tomorrow, and it cannot be in two places. So he cannot work two days running. One day on, one day off, always — the off day is moving the drum, the ladders and the sheets, and setting up for the next one.

By the second week of October he has offers for most days of the month, and they are not the same size. A two-room flat is eight hundred. A bungalow with a compound wall is two thousand two hundred. Somebody's staircase is four hundred.

The first year he did it, he took the biggest ones. It seemed obvious. You cannot do everything, so do the valuable ones.

He remembers the specific week that taught him otherwise. There was a job on the Wednesday for one thousand six hundred, which was the largest that week, and jobs on the Tuesday and the Thursday for nine hundred each. He took the Wednesday, because sixteen hundred is more than nine hundred, and that ruled out both the others.

Eighteen hundred was sitting there in two jobs he could have done, and he made sixteen.

What he does now takes him about ten minutes with the list in front of him, and he does it backwards, which also surprised him.

He starts at the last day of the month and asks a single question about it: **if I take this job, what is the best I could have done up to two days before? And if I skip it, what is the best I could have done up to yesterday?** Whichever is larger is the best possible from the start of the month up to that day.

Then he asks the same question about the day before, and the day before that, all the way back.

The thing he finds hard to explain to people is that he is not deciding anything while he does this. He is not choosing the Wednesday job or not choosing it. **He is working out, for each day, what the best possible total is up to that day** — and only at the very end, when he has the number for the last day, does he walk back through to see which jobs it actually implies.

His nephew, who is at college and did some computing, told him there was a name for it. Ashok said there is a name for everything.

3 The idea in plain English

Ashok's ten minutes is house robber, and his bad week is the greedy counter-example.

The shape: at each position you make a binary choice, and one option forbids something. Take this house and you cannot take the previous one. Take this job and Tuesday is gone. **The forbidding is what makes it interesting** — without it you would take everything and there would be no problem.

The last-move question from day 145 still produces the recurrence, and here the last move is the choice itself:

To have the best possible total considering houses `0..i`, either I took house `i` or I did not.

If I took it, house `i-1` was forbidden, so my situation before was the best over `0..i-2`, and my total is `money[i] + best(i-2)`.

If I skipped it, my situation is exactly the best over `0..i-1`, unchanged.

```
best(i) = max( money[i] + best(i-2),    best(i-1) )
              \___ take ___/          \_ skip _/
```

One line, two branches, and every problem in this family is a variation on it.

The state sentence matters and it is subtle here. `dp[i]` is the maximum obtainable considering houses 0 through `i` — not “the maximum if I rob house `i`”. Those are different states, and the second one leads to a correct but more awkward recurrence. Say the sentence, and the base cases follow from it.

Base cases, derived from the sentence rather than guessed:

```
dp[0] = money[0]           the best from one house is that house
dp[1] = max(money[0], money[1]) the best from two houses, taking at most one
```

`dp[1]` is the one people get wrong. Writing `money[1]` looks like the pattern and contradicts the sentence — “the best from houses 0 and 1” is obviously the larger of the two, not the second one.

Now the greedy trap, which is the actual content of this problem.

The obvious approach is to sort by value and take greedily, skipping anything adjacent to something already taken. It is **wrong**, and Ashok’s week is the smallest interesting counter-example:

```
[900, 1600, 900]
```

```
greedy: take 1600 (the largest), which blocks both neighbours → 1600
correct: take 900 and 900 → 1800
```

The general reason greedy fails: a local choice that looks best can foreclose two options that are together worth more. And you cannot fix it with a smarter greedy rule, because the same shape recurs at every scale.

The other tempting wrong answer is “take alternate houses”. Take all the even indices, or all the odd indices, and pick the better:


```
[2, 1, 1, 2]
evens: 2 + 1 = 3
odds: 1 + 2 = 3
correct: 2 + 2 = 4      ← indices 0 and 3, which are neither pattern
```

Non-adjacent does not mean alternating. You may leave a gap of two, and the optimal solution often does. That counter-example is worth having ready because the alternating idea is very common and sounds plausible.

The space collapses to two variables, because the recurrence reaches back exactly two. And unusually, the optimised version is the one most people write, because it reads naturally: keep “the best including up to the previous house” and “the best including up to the one before that”, and roll them forward.

And then the follow-up: the houses are in a circle, so the first and last are adjacent.

The elegant answer is to run the linear version twice. If the houses are in a circle, then the first and last cannot both be taken. So either the last house is excluded, or the first is. Run house robber on `houses[0 : n-1]`, run it on `houses[1 : n]`, and take the larger.

Why that is correct is worth being able to say: every valid circular selection excludes at least one of the two, so it is covered by at least one of the two runs; and every selection produced by either run is valid on the circle, because neither run contains both endpoints. **Two runs of an $O(n)$ algorithm, so still $O(n)$, and about four extra lines.**

The single-house case breaks it — `houses[0:0]` is empty — so `n = 1` needs handling before the two runs.

Recognising the family is the last piece, because it rarely says “adjacent”:

- **“No two adjacent”** — the direct version.
- **“Delete and earn”:** pick a number and all copies of it, but then you cannot pick `value-1` or `value+1`. **Transform to counts indexed by value**, and it is house robber over that array.
- **“Cooldown after selling”:** you cannot buy on the day after a sale, which is the same forbidding.
- **“Paint the fence with no three in a row”** — same shape with a longer forbidden window.

The tell: a binary choice per position where taking forbids a nearby position.

The choice at each step, drawn:

at house 2, two options:

```
TAKE 9: ... best over 0..0 ... [skip 1] [TAKE 2] = 9 + dp[0] = 9 + 2 = 11
SKIP 9: ... best over 0..1 ..... [skip 2] = dp[1] = 7

dp[2] = max(11, 7) = 11
```

The table filling:

```
money:    2      7      9      3      1
dp:       2      7     11     11     12
|         |        |        |        |
|         |        |        |        +-- max(1 + dp[2]=11+1=12,   dp[3]=11) = 12
|         |        |        +----- max(3 + dp[1]=7+3=10,     dp[2]=11) = 11
|         |        +----- max(9 + dp[0]=2+9=11,      dp[1]=7) = 11
|         +----- max(money[0], money[1]) = max(2,7) = 7
+----- money[0] = 2
```

answer: dp[4] = 12, from houses 1 and 3... no: 7 + 3 = 10.
from houses 0, 2 and 4: 2 + 9 + 1 = 12. ← that one

What to notice on the last line. The optimal selection is $\{0, 2, 4\}$ — three houses, not the two largest. **The table gives the total; working out which houses is a separate backward walk**, and section 5 does it.

The greedy trap, and the alternating trap:

GREEDY BY VALUE

[900, 1600, 900]

```
take 1600 → blocks both
total 1600
```

correct: $900 + 900 = 1800$

ALTERNATE INDICES

[2, 1, 1, 2]

```
evens: 2 + 1 = 3
odds:  1 + 2 = 3
```

correct: indices 0 and 3 $\rightarrow 2 + 2 = 4$
a gap of TWO, which neither pattern allows

What to notice. Both wrong approaches are natural, both fail on four-element inputs, and neither failure is visible on the standard example. **Have both counter-examples ready.**

The circular variant:

houses in a circle: 0 - 1 - 2 - 3 - 0

the first and the last are now adjacent, so they cannot BOTH be taken.

RUN A: houses[0 .. n-2] → the last house is excluded
 [h0 h1 h2] h3

RUN B: houses[1 .. n-1] → the first house is excluded
 h0 [h1 h2 h3]

answer = max(A, B)

every valid circular selection excludes at least one endpoint,
so it appears in at least one run.
and neither run can produce an invalid selection,
because neither contains both endpoints.

5 The code, built step by step

Start with the choice, said out loud.

“At house `i` I either take it or I skip it. If I take it, `i-1` was forbidden, so I add to the best over `0..i-2`. If I skip it, I keep the best over `0..i-1`.”

Then the state sentence:

```
# dp[i] = the maximum money obtainable considering houses 0..i
```

Then the base cases, from the sentence:

```
dp[0] = money[0]           # one house: take it
dp[1] = max(money[0], money[1]) # two houses: the larger, not the second
```

Then the loop:

Four lines, and the `n == 1` guard is the whole subtlety — with one house, `money[:-1]` is empty and `money[1:]` is empty, so both runs return 0 and the answer is wrong.

And reconstructing which houses were chosen, which is the other common follow-up:

```
def rob_with_houses(money: list[int]) → tuple[int, list[int]]:
    n = len(money)
    if n == 0:
        return 0, []
    if n == 1:
        return money[0], [0]
    dp = [0] * n
    dp[0] = money[0]
    dp[1] = max(money[0], money[1])
    for i in range(2, n):
        dp[i] = max(money[i] + dp[i - 2], dp[i - 1])

    chosen: list[int] = []
    i = n - 1
    while i ≥ 0:
        if i == 0:
            chosen.append(0)
            break
        if i == 1:
            chosen.append(0 if money[0] > money[1] else 1)
            break
        if dp[i] == dp[i - 1]:
            i -= 1
            # house i was skipped
        else:
            chosen.append(i)
            i -= 2
            # house i was taken
            # so i-1 was forbidden
    return dp[n - 1], chosen[::-1]
```

The walk is backwards and it needs the full table, which is why reconstruction and space optimisation are mutually exclusive. The test `dp[i] == dp[i-1]` means “taking house `i` did not improve anything”, so it was skipped; otherwise it was taken and we jump back two.

Ties make the reconstruction non-unique, so this returns a valid selection rather than *the* one.

And the transformation that turns a different-looking problem into this one — delete and earn:

```
def delete_and_earn(nums: list[int]) → int:
    """Take a number and all its copies, but then value-1 and value+1 are forbidden."""
    from collections import Counter
    counts = Counter(nums)
    highest = max(nums)
    # earnings[v] = the total money from taking every copy of v
    earnings = [0] * (highest + 1)
    for value, count in counts.items():
        earnings[value] = value * count
    return rob(earnings) # ... and now it IS house robber
```

Index by value, not by position. Once you do that, “cannot take `v-1` or `v+1`” becomes “cannot take adjacent indices”, and the problem is solved by a function you already have. **Spotting that transformation is the whole problem**, and the DP is four lines you did not have to write.

The complete solution

```
"""House robber: the take-or-skip choice, the circular variant, and reconstruction."""

from __future__ import annotations
from collections import Counter

def rob(money: list[int]) → int:
    """Maximum with no two adjacent. O(n) time, O(1) space.

    take_previous = the best considering houses up to i-2
    skip_previous = the best considering houses up to i-1
    """
    take_previous = skip_previous = 0
    for value in money:
        take_previous, skip_previous = skip_previous, max(value + take_previous, skip_previous)
    return skip_previous

def rob_table(money: list[int]) → int:
    """The same, with the table. dp[i] = the best considering houses 0..i."""
    n = len(money)
    if n == 0:
        return 0
    if n == 1:
        return money[0]
    dp = [0] * n
    dp[0] = money[0]
    dp[1] = max(money[0], money[1]) # the LARGER, not money[1]
    for i in range(2, n):
        dp[i] = max(money[i] + dp[i - 2], dp[i - 1])
    return dp[n - 1]

def rob_circular(money: list[int]) → int:
    """First and last are adjacent. Every valid selection excludes one of them."""
    n = len(money)
    if n == 0:
        return 0
    if n == 1:
        return money[0] # the two-run trick needs n ≥ 2
    return max(rob(money[:-1]), rob(money[1:]))

def rob_with_houses(money: list[int]) → tuple[int, list[int]]:
    """The total AND which houses. Needs the full table – no space optimisation."""
    n = len(money)
    if n == 0:
        return 0, []
    if n == 1:
        return money[0], [0]
    dp = [0] * n
    dp[0], dp[1] = money[0], max(money[0], money[1])
    for i in range(2, n):
        dp[i] = max(money[i] + dp[i - 2], dp[i - 1])

    chosen: list[int] = []
    i = n - 1
    while i ≥ 0:
        if i == 0:
            chosen.append(0)
            break
        if i == 1:
            chosen.append(0 if money[0] ≥ money[1] else 1)
            break
        if dp[i] == dp[i - 1]:
            i -= 1
        else:
            chosen.append(i)
```

```

        chosen.append(i)
        i -= 2
    return dp[n - 1], chosen[::-1]

def delete_and_earn(nums: list[int]) → int:
    """Index by VALUE, and the adjacency constraint becomes house robber."""
    if not nums:
        return 0
    counts = Counter(nums)
    earnings = [0] * (max(nums) + 1)
    for value, count in counts.items():
        earnings[value] = value * count
    return rob(earnings)

if __name__ == "__main__":
    print("rob [2,7,9,3,1]      :", rob([2, 7, 9, 3, 1]))
    print("which houses       :", rob_with_houses([2, 7, 9, 3, 1]))
    print()
    print("greedy trap         :", rob([900, 1600, 900]), "(greedy would say 1600)")
    print("alternating trap      :", rob([2, 1, 1, 2]), "(alternating would say 3)")
    print()
    print("circular [2,3,2]        :", rob_circular([2, 3, 2]))
    print("circular [1,2,3,1]:", rob_circular([1, 2, 3, 1]))
    print("circular [5]           :", rob_circular([5]))
    print()
    print("delete_and_earn         :", delete_and_earn([3, 4, 2]))
    print("delete_and_earn         :", delete_and_earn([2, 2, 3, 3, 3, 4]))

```

Running it:

```

rob [2,7,9,3,1]      : 12
  which houses       : (12, [0, 2, 4])

greedy trap         : 1800 (greedy would say 1600)
alternating trap    : 4 (alternating would say 3)

circular [2,3,2]    : 3
circular [1,2,3,1]: 4
circular [5]        : 5

delete_and_earn     : 6
delete_and_earn     : 9

```

Three things to look at. **The chosen houses are [0, 2, 4]** — three houses totalling 12, not the two largest. That is what the greedy approach cannot find.

circular [2,3,2] returns 3, not 4. In a line, houses 0 and 2 give 4; in a circle they are adjacent, so the best is the single 3. **That input is the one that catches implementations which forgot the circle.**

And `delete_and_earn([2,2,3,3,3,4])` returns 9: taking both 2s and the 4 gives `4 + 4 = 8`, while taking all three 3s gives 9 — and taking the 3s forbids both the 2s and the 4. **The transformation to `earnings` indexed by value is what makes that a one-liner.**

6 What it costs

Time and space.

```
n states, each with a constant-size choice (two options)      → 0(n) time
full table                                                    → 0(n) space
window of 2 (the recurrence reaches back 2) → 0(1) space
```

The circular version:

```
two runs of an 0(n) algorithm → 0(n), with a constant factor of 2
space                        → 0(1), unchanged
```

Reconstruction:

```
needs the full table      → 0(n) space, not 0(1)
the backward walk        → 0(n) time
                        → so asking for the houses costs the space optimisation
```

Delete and earn:

```
counting                0(n)
the earnings array      0(max value)      ← NOT 0(n)
house robber over it    0(max value)
                        -----
                        0(n + max value) time and space
```

That `max value` is worth flagging, because it is a different variable from `n`:

```
nums = [1, 1000000]
n = 2
earnings array = 1,000,001 entries
```

The array is sized by the largest value, not by the count, so sparse large values are wasteful. The fix is to sort the distinct values and run a modified recurrence that checks whether consecutive distinct values actually differ by one — which is $O(n \log n)$ and independent of the magnitudes. **Worth naming as the fix when values can be large.**

Against the naive alternatives:

```
try every subset          2^n subsets x 0(n) to validate = 0(2^n * n)
  n = 30                  ~32,000,000,000 operations
  DP                      30 operations
```

Against greedy, which is not slower but wrong:

greedy by value	$O(n \log n)$ to sort	→ WRONG on [900, 1600, 900]
alternating	$O(n)$	→ WRONG on [2, 1, 1, 2]
DP	$O(n)$	→ correct

The DP is the same order as the wrong greedy and cheaper than the wrong alternating. There is no efficiency argument for the wrong answers, which is worth pointing out — they are attractive only because they look simpler.

Sizing at realistic limits:

$n = 100,000$	(a typical constraint)
DP	100,000 operations, two variables
runtime	~10 ms in Python

And the whole family's costs:

house robber	$O(n)$ time, $O(1)$ space
house robber II (circular)	$O(n)$ time, $O(1)$ space
delete and earn	$O(n + \max)$ time and space
paint fence (no 3 in a row)	$O(n)$ time, $O(1)$ space -- window of 3
stock with cooldown	$O(n)$ time, $O(1)$ space -- 3 states per day

All of them are linear, and all of them collapse to constant space, because the forbidden window is fixed.

7 The traps

Greedy by value

The natural wrong answer, and the reason this problem exists:

```
# sort by value, take greedily, skip anything adjacent to something taken
```

```
>>> greedy([900, 1600, 900])
1600                                     # the answer is 1800
```

Taking the largest forecloses two options worth more together. And no smarter greedy rule fixes it, because the same shape recurs at every scale. **Have this three-element counter-example ready** — being able to produce it immediately is most of the credit for this question.

Taking alternate indices

```
return max(sum(money[0::2]), sum(money[1::2]))
```

```
>>> alternating([2, 1, 1, 2])
3                                     # the answer is 4, from indices 0 and 3
```

Non-adjacent does not mean alternating. The optimal solution frequently leaves a gap of two, and neither alternating pattern can express that. This one sounds even more plausible than greedy-by-value and fails on a four-element input.

```
dp[1] = money[1]
```

```
dp[0], dp[1] = money[0], money[1]
```

```
>>> rob_wrong([5, 1, 5])
6                                     # the answer is 10
```

The state sentence says “the best considering houses 0 and 1”, which is the *larger* of the two. **Deriving the base case from the sentence rather than from the shape of the recurrence gets this right**, and guessing gets it wrong about half the time.

Forgetting the empty and single-element cases

```
dp = [0] * n
dp[0] = money[0]
dp[1] = max(money[0], money[1])
```

```
IndexError: list index out of range      # on a one-element input
```

The two-variable version avoids this entirely by starting both at zero, which is a real argument for writing it that way — the guards become unnecessary rather than merely handled.

The circular version on a single house

```
return max(rob(money[:-1]), rob(money[1:]))
```

```
>>> rob_circular_wrong([5])
0                                     # the answer is 5
```

With one house both slices are empty. The `n = 1` guard is the whole subtlety of the circular variant, and it is the case the tests check.

The circular version done as one pass

```
# "just check that houses 0 and n-1 aren't both taken at the end"
```

There is no local way to enforce a constraint between the first and last elements while scanning once — by the time you reach the end, the decision about the first was made without knowing this. **Two runs is not a workaround, it is the correct decomposition**, and the argument for it is that every valid circular selection must exclude at least one endpoint.

Sizing `delete_and_earn` by `n`

```
earnings = [0] * (len(nums) + 1)           # should be max(nums) + 1
```

```
IndexError: list assignment index out of range
```

The array is indexed by **value**, not by position, so it must be as long as the largest value. And when values can be up to `10^9`, the array approach is not viable at all — sort the distinct values instead and check whether consecutive ones differ by exactly one.

Optimising space and then being asked which houses

```
take_previous, skip_previous = ...
# "and which houses did you rob?"
```

You cannot say. The table recorded the decisions and there is no table. **Ask whether the total is enough before optimising**, because retrofitting the reconstruction means putting the table back.

8 In the interview

How it gets asked

- “Maximise the sum with no two adjacent elements.” — the direct version.
- “You cannot rob two adjacent houses.” — LeetCode 198.
- “Now the houses are in a circle.” — LeetCode 213, the standard follow-up.

- “Delete a number and earn its value, but then you cannot take value ± 1 .” — LeetCode 740.
- “Why doesn’t greedy work?” — the question the problem exists for.
- “Which houses did you rob?” — the reconstruction follow-up.

The first ninety seconds

“At each house there is a binary choice — take it or skip it — and taking it forbids the previous one. So the last-move question gives me the recurrence directly.

If I take house i , then $i-1$ was forbidden, so my best is `money[i]` plus the best over houses 0 to $i-2$. If I skip it, my best is unchanged from the best over 0 to $i-1$. So `dp[i] = max(money[i] + dp[i-2], dp[i-1])`.

`dp[i]` means the maximum obtainable considering houses 0 through i — not ‘the maximum if I rob house i ’. That distinction matters because it decides the base cases: `dp[0]` is `money[0]`, and `dp[1]` is `max(money[0], money[1])`, the *larger* of the two, not the second one. Writing `money[1]` there is the common bug and it comes from copying the recurrence’s shape instead of reading the sentence.

The thing I would flag before you ask is why greedy fails, because it is the natural first instinct. Take the largest available and skip its neighbours: on `[900, 1600, 900]` that takes 1600 and blocks both, giving 1600, when 900 plus 900 is 1800. One large item forecloses two smaller ones worth more together, and no smarter greedy rule fixes it because the shape recurs at every scale.

And the other tempting wrong answer is alternating indices — take all evens or all odds. On `[2, 1, 1, 2]` both give 3, and the answer is 4 from indices 0 and 3. Non-adjacent does not mean alternating; the optimal solution often leaves a gap of two.

$O(n)$ time and $O(1)$ space, because the recurrence reaches back exactly two, so I keep two variables. And starting both at zero handles the empty and single-element inputs without guards.

Shall I write it, and would you like the circular version after?”

The follow-ups

“Now the houses are in a circle.”

“Then the first and last are adjacent, and the elegant answer is to run the linear version twice.

Since houses 0 and $n-1$ cannot both be taken, every valid selection excludes at least one of them. So I run house robber on `houses[0 : n-1]` — which excludes the last — and on `houses[1 : n]` — which excludes the first — and take the larger.

The correctness argument has two halves and I would give both. Every valid circular selection omits at least one endpoint, so it is a valid selection in at least one of the two ranges and is therefore considered. And conversely, nothing either run produces can be invalid on the circle, because neither range contains both endpoints — so I am not over-counting.

Two runs of a linear algorithm, so still $O(n)$, $O(1)$ space, and about four extra lines.

The case that breaks it is $n = 1$: both slices are empty and both runs return zero, so the answer comes out as zero instead of the single house’s value. That needs an explicit guard, and it is what the tests check.

What I would not try is enforcing the constraint in a single pass. There is no local way to say ‘do not take the last one if I took the first’, because by the time I reach the end the decision about the first was made without that information.

The two-run decomposition is not a workaround, it is the correct way to split the problem — and I think being able to say why is more valuable than the four lines.”

“Which houses did you rob?”

“Then I need the full table, and I would say that up front because it undoes the space optimisation.

The walk is backwards from the last cell. At position i , if $dp[i] = dp[i-1]$, then taking house i did not improve anything, so it was skipped and I move to $i-1$. Otherwise it was taken, so I record i and jump to $i-2$, because $i-1$ was forbidden. Handle $i = 0$ and $i = 1$ explicitly at the end, and reverse the list.

$O(n)$ time for the walk and $O(n)$ space for the table.

The alternative is parent pointers — store, alongside each cell, whether it took or skipped — which is the same $O(n)$ space and slightly clearer. Either way, **reconstruction and constant space are mutually exclusive**, and that is worth asking about before optimising rather than after.

And ties make the answer non-unique. If taking and skipping give the same total, my walk picks one arbitrarily. So I would return a valid selection and say so, and a test should assert the total or check validity rather than compare against a fixed list of indices.”

“Here is an array; you can take a number and all its copies, but then you cannot take value minus one or value plus one.”

“That is house robber with one transformation in front of it, and the transformation is the whole problem.

The key move is to index by value rather than by position. Build an array where `earnings[v]` is `v` times how many copies of `v` there are — the total money from choosing value `v`. Then ‘cannot take `v-1` or `v+1`’ is exactly ‘cannot take adjacent indices’, and I run the house robber function I already have.

So on `[2, 2, 3, 3, 3, 4]`: earnings are `[0, 0, 4, 9, 4]`, and house robber over that gives 9 — taking all three 3s, which forbids the 2s and the 4. Taking the 2s and the 4 would give 8.

The cost is different from house robber’s, though, and that is worth stating.

The array is sized by the *largest value*, not by `n`, so it is $O(n + \max)$ in time and space. With `n = 2` and values `[1, 1000000]`, that is a million-entry array for two numbers.

The fix when values can be large is to sort the distinct values and walk them, checking whether consecutive distinct values differ by exactly one — if they do, the adjacency constraint applies and the recurrence uses the `i-2` term; if they do not, the previous value is not forbidden and you can simply add. That is $O(n \log n)$ and independent of the magnitudes. I would mention it if the constraints allow large values, and use the array version otherwise because it is four lines.”

“What if it were ‘no two within three positions?’”

“The same shape with a wider forbidden window: $dp[i] = \max(\text{money}[i] + dp[i-4], dp[i-1])$, because taking i now forbids $i-1$, $i-2$ and $i-3$, so the last usable position is $i-4$.

The base cases extend correspondingly — the recurrence reaches back four, so indices 0 through 3 need explicit values, and each is ‘the best from the first k positions taking at most one’, which is the running maximum.

And the space window becomes four variables rather than two. Still $O(1)$, still $O(n)$ time.

The general statement: a fixed forbidden window of width w gives $dp[i] = \max(\text{money}[i] + dp[i-w-1], dp[i-1])$, needs $w+1$ base cases, and keeps a window of $w+1$ values. **The problem does not get harder as w grows** — which is worth saying, because people expect it to.

What *would* make it harder is a forbidding rule that is not a fixed window — ‘you cannot take two houses whose values differ by less than ten’, say. That is not positional at all, the one-dimensional state stops being sufficient, and it becomes a genuinely different problem.”

The model answer

“You are scheduling jobs. Each job takes one day and pays a known amount, and after any job you need a day to recover, so you cannot work two consecutive days. Maximise your earnings over n days, where some days have no job offered.”

“This is house robber with two small dressings, and I would name that immediately rather than treat it as new.

The model. $\text{pay}[i]$ is what day i offers, and a day with no job is simply $\text{pay}[i] = 0$. **That is the first dressing and it needs no special handling at all** — a zero-paying day is taken or skipped by the same recurrence and the arithmetic works out. I would say that explicitly, because the instinct is to filter those days out, and filtering would break the adjacency relationship between the days that remain.

The choice at each day: work it or do not. Working day i means day $i-1$ was a recovery day, so my best is $\text{pay}[i] + \text{dp}[i-2]$. Not working means $\text{dp}[i-1]$. So $\text{dp}[i] = \max(\text{pay}[i] + \text{dp}[i-2], \text{dp}[i-1])$, and **$\text{dp}[i]$ is the maximum earnings considering days 0 through i .**

Base cases from the sentence: $\text{dp}[0] = \text{pay}[0]$, and $\text{dp}[1] = \max(\text{pay}[0], \text{pay}[1])$ — the better of the first two days, since I can work at most one of them.

$O(n)$ time, $O(1)$ space with two variables, and I would write that version because it also removes the empty and single-day guards.

Why not greedy, and I would raise it unprompted because it is the plausible wrong answer here: taking the highest-paying day first blocks its neighbours, and on $[900, 1600, 900]$ that earns 1600 where 1800 was available. **A single good day can foreclose two decent ones worth more together.**

The follow-up I would expect is that the schedule wraps — a rolling four-week roster where the last day and the first day are also consecutive. **That is the circular variant: run it once excluding the last day and once excluding the first, and take the larger,** because any valid schedule must skip at least one of them. With the $n = 1$ guard.

And the practical question I would ask before writing anything: does the recovery day depend on the job? A large job might need two days off, and if the recovery period varies per job, the forbidden window is no longer fixed — $\text{dp}[i] = \max(\text{pay}[i] + \text{dp}[i - \text{recovery}[i] - 1], \text{dp}[i-1])$ — which is still $O(n)$ and still one-dimensional, but the space no longer collapses to a fixed window because the reach-back varies. **That is a genuinely different answer to ‘can you reduce the space’, and I would rather know before promising it.**“

9 Recall card

A binary choice per position where taking forbids a neighbour: $dp[i] = \max(\text{money}[i] + dp[i-2], dp[i-1])$ — take, or skip.

$dp[i]$ = the maximum considering houses 0..i (not “if I rob i”), which is what makes $dp[1] = \max(\text{money}[0], \text{money}[1])$ — the **larger**, not the second. Derive base cases from the sentence.

Two counter-examples to have ready. Greedy by value fails on $[900, 1600, 900]$ (1600 vs 1800). Alternating indices fails on $[2, 1, 1, 2]$ (3 vs 4) — **non-adjacent does not mean alternating**.

Circular: run the linear version twice, excluding the last house and excluding the first, and take the max — because every valid selection omits at least one endpoint, and neither run can produce an invalid one. **Guard** $n = 1$.

$O(n)$ time, $O(1)$ space with two variables (which also removes the empty/single guards). Reconstruction needs the **full table** — walk backwards, $dp[i] = dp[i-1]$ means skipped — so it is mutually exclusive with the space optimisation. And **delete-and-earn** is this problem after indexing by value, at $O(n + \max)$.

Design a URL shortener

1 What this is, and why they ask it

Take `https://example.com/products/2026/winter-collection/item?id=88471&ref=email` and give back `https://sho.rt/aK9x2b`. When somebody visits the short one, redirect them to the long one.

That is the whole product, and it is the most-asked system design question there is — partly because it is small enough to finish in forty-five minutes, and partly because it looks trivial and is not. The interesting parts are not where people expect: not the storage, which is small, and not the read throughput, which a cache handles. **The interesting parts are how you generate the code and what happens at the redirect.**

It is also the first case study, so today does two things at once: the design itself, and a demonstration of [yesterday's script](#) run end to end with the clock on it.

The specific thing being tested is whether you reach for scale you do not need. A hundred million URLs a month is forty writes a second. Candidates who start sharding are answering a question nobody asked; candidates who work out the number first and then say “this fits on one machine, so the design problem is elsewhere” are demonstrating exactly the judgement the round exists to measure.

By the end of this lesson you can run the six phases on this prompt, defend a code-generation scheme with arithmetic, place the cache correctly, answer the 301-versus-302 question — which is a product decision — and name the parts that are genuinely hard.

2 The story

Deepak's sweet shop has one counter for paying and one for collecting, about eleven feet apart, and in the week before Diwali the queue goes out of the door.

The system is a small pad of numbered slips. You order at the first counter, you pay, he tears off a slip with a number on it and writes what you ordered in his book against that number, and you go and stand at the second counter and hold up the slip.

The number means nothing. It is not the price, it is not the order, it is not the time. **It is just a short thing that stands for a long thing** — two kilos of kaju katli, half a kilo of soan papdi, and a box, which would take fifteen seconds to say again at the

second counter and takes half a second as “sixty-two”.

Three things about it that Deepak has learnt, all of them the hard way.

The numbers run out. The pad goes to five hundred and in that week he gets through it by four in the afternoon. So he starts again at one, which is fine, because by four o'clock nobody from the morning is still waiting. **But once, in 2019, somebody came back at five with a slip from the morning** — they had left and come back — and the number had been reused, and there was a very awkward conversation about a box of barfi.

People ask for particular numbers. A man whose shop number is 108 asks for slip 108 every single year and is pleased when he gets it. Deepak lets him, and it costs nothing, and it means that when the pad reaches 108 he has to skip it, which he sometimes forgets.

And the slip is not the record. If a slip is lost, the order still exists — it is written in the book against that number, and Deepak can find it if the customer can say what they ordered. **The book is the truth; the slip only says where to look in it.** A slip with nothing behind it in the book is worthless, and he has never once had the opposite problem.

The other thing, which he does not think about because it has always been true, is that the second counter never needs to know anything except the number. It does not need to know who paid, or when, or how much. It looks up the number, hands over the box. **That is the entire job of the second counter and it is why one boy can do it at the speed he does.**

3 The idea in plain English

Deepak's slip is a short code and his book is the mapping, and all three of his lessons are design decisions.

The product is one mapping and two operations. `code → long URL`, with “create a code for this URL” and “look up this code and redirect”. Everything else — analytics, expiry, custom codes, abuse prevention — is built around that.

Phase 1, requirements. In scope: shorten, redirect, optional custom alias, and a click count. Out of scope: user accounts, ads, link previews. Non-functional, and these decide the design:

- **The redirect is on somebody else's critical path.** A person clicked a link in a message; the shortener sits between them and the page they wanted. **Single-digit milliseconds, or it is a visible delay.**
- **Availability matters more than consistency.** A redirect that works with a slightly stale target beats one that fails. Conversely, an unavailable shortener breaks every link anyone has ever shared.
- **Extremely read-heavy.** A URL is created once and followed many times.

Phase 2, scale, and this is where the design is decided. Say a hundred million new URLs a month:

```
100,000,000 / 30 days / 86,400 s = ~40 writes per second
peak x5                          = ~200 writes/s
reads at 100:1                  = ~4,000 reads/s, ~20,000 peak
storage: 500 bytes x 100M/month = 50 GB/month, 600 GB/year
```

Those numbers say something specific and it is worth saying out loud: none of this is hard. Two hundred writes a second is one modest database. Six hundred gigabytes a year fits on one disk for a decade. **So the design problem is not scale — it is the code generation and the redirect latency,** and a candidate who starts sharding has not read their own arithmetic.

Phase 3, the API. Two endpoints, and the second one is the whole product:

```
POST /urls {long_url, custom_alias?, expires_at?} → {short_url}
GET  /{code}                                → 301 or 302 redirect
```

And the data model is one table with `code` as the primary key, because the redirect is a single primary-key lookup and nothing else. That is the second counter: it needs the number and nothing more.

Now the part that actually matters: how do you make the code?

Option one: hash the URL and take the first few characters. Deterministic, so the same URL always gives the same code. **And it collides** — two different URLs hashing to the same prefix — so every write needs a read to check, and a retry loop with a salt when it does. It also makes “the same URL twice” return the same code, which is either a feature or a bug depending on whether users expect distinct links.

Option two: random codes. Generate seven random base-62 characters, check for a collision, retry. Simple, unguessable, and the collision probability grows as the space fills — which is fine for a long time and requires monitoring.

Option three: a counter, encoded in base 62. Take the next integer and encode it. **No collisions at all, by construction**, and the codes are as short as possible for the number of URLs issued. The problem is that a single global counter is a bottleneck and a single point of failure — and the codes are sequential, therefore guessable, so anyone can enumerate every link ever shortened.

The practical answer is option three with two fixes, and this is the thing to be able to defend:

Block allocation. Each application server asks a central counter for a block of ten thousand ids at a time and hands them out locally. That turns coordination from once per URL into once per ten thousand, and a server crashing wastes at most ten thousand codes out of trillions.

And a shuffled alphabet or an ID transformation, so codes are not sequentially guessable. The simplest is to use a fixed permutation of the base-62 alphabet; a stronger one is to multiply the counter by a large odd number modulo the space, which is reversible and scatters consecutive ids.

How long should the code be? This is arithmetic, and it is the number to produce:

base 62 (a-z, A-Z, 0-9)		
6 characters	$62^6 = 56,800,000,000$	at 100M/month → ~47 years
7 characters	$62^7 = 3,500,000,000,000$	→ ~3,000 years

Six characters is enough for forty-seven years, which is longer than the product will exist. **Seven gives you room to be wrong**, and one extra character is cheap. Say both numbers and pick.

Phase 4, the design, and it is deliberately boring.

write:	client → API → get a code from the local block → insert → return
read:	client → API → cache lookup → (miss) database → redirect

The redirect is one cache lookup and one HTTP response. That is why it can be single-digit milliseconds.

And the click count does not go on the read path. Incrementing a counter in the database on every redirect turns a read-only path into a write-heavy one and puts a hot row in front of your most latency-sensitive operation. **Push an event onto a queue and aggregate asynchronously.**

Phase 5, the deep dive, and there are two candidates.

The cache, because the read path is the product. URL popularity is extremely skewed — a handful of links get most of the traffic — so a small cache covers a large fraction. **And the right eviction policy follows from the access pattern**, which is that a link is hot for a short period after it is shared and then almost never used again.

Or the 301-versus-302 decision, which is more interesting because it is not technical:

- **301 Permanent** — browsers cache it, so subsequent clicks never reach your servers. **Fast, cheap, and you lose all analytics after the first click**, and you can never change where the link points.
- **302 Found** — not cached, so every click comes to you. **Every click is counted and the target can be changed**, at the cost of a round trip every time and all the traffic.

That is a product decision presented as a status code, and recognising it as such is the point.

Phase 6, wrap-up. The bottleneck is the redirect read, which is a cache problem. What breaks at ten times the load is nothing structural — more cache and more replicas. What is genuinely hard and was left out: **abuse**, because a shortener is a perfect tool for hiding a malicious destination, and every real one needs scanning, rate limiting and a takedown path.

4 The picture

The two paths:

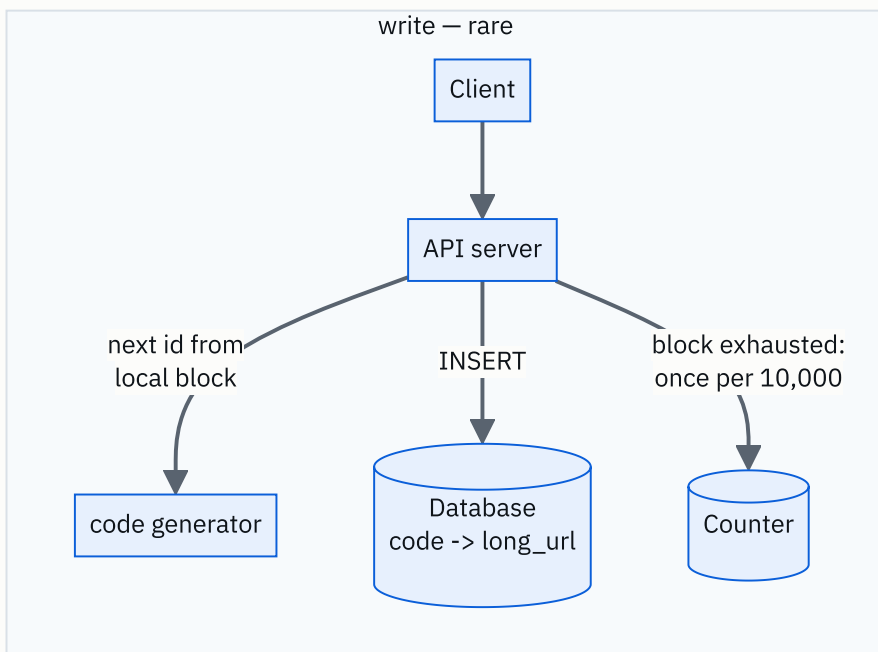


FIGURE 146.1

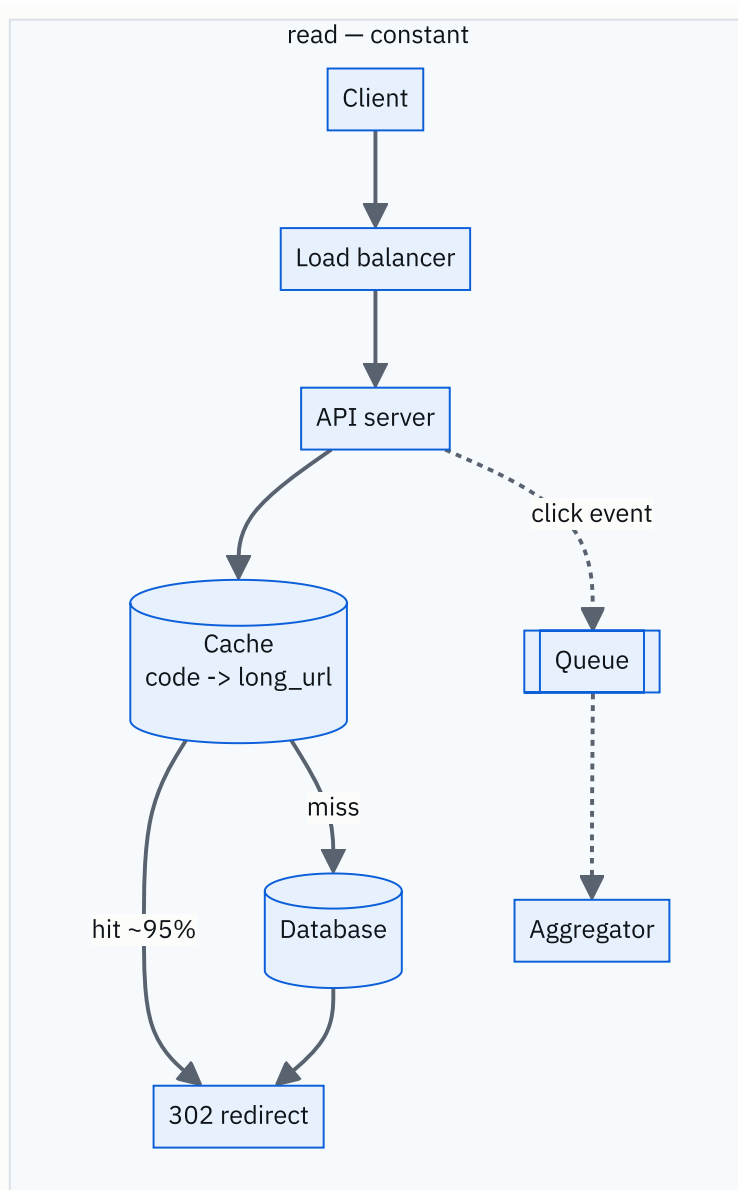


FIGURE 146.2

What to notice. The click event is a dotted line off the critical path — the redirect is returned without waiting for it. **Putting an increment on the read path is the single most common mistake in this design**, and it converts four thousand reads a second into four thousand writes a second against one hot row.

Code generation, compared:

HASH THE URL	RANDOM	COUNTER + BASE 62
md5(url)[:7] deterministic COLLIDES → read before every write same URL → same code	7 random base-62 chars unguessable collides → read before every write same URL → new code	id=1000000 → "4c92" no collisions EVER no read before write sequential → guessable + block allocation → 1 coordination per 10,000 + shuffled alphabet → not enumerable

The length arithmetic:

base 62: a-z (26) + A-Z (26) + 0-9 (10)

length	space	at 100M/month
-----	-----	-----
5	916,132,832	~9 months
6	56,800,235,584	~47 years
7	3,521,614,606,208	~2,900 years

→ 6 is enough; 7 is the safe choice and costs one character

The 301/302 trade, which is the decision people miss:

301 PERMANENT	302 FOUND
browser caches the redirect 2nd click never reaches you	browser asks every time every click reaches you
+ almost zero traffic after the 1st + fastest possible for the user - no analytics beyond the first - the link can NEVER be repointed	+ every click counted + the target can be CHANGED later - every click costs a round trip - all the traffic is yours to serve
→ a commercial shortener uses 302, because clicks are the product	
→ an internal one might use 301, because traffic is the cost	

5 How it actually works

Base-62 encoding

```
ALPHABET = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
BASE = len(ALPHABET) # 62

def encode(number: int) → str:
    if number == 0:
        return ALPHABET[0]
    out = []
    while number:
        number, remainder = divmod(number, BASE)
        out.append(ALPHABET[remainder])
    return "".join(reversed(out))

def decode(code: str) → int:
    value = 0
    for character in code:
        value = value * BASE + ALPHABET.index(character)
    return value
```

Base 62 rather than base 64 because the two extra characters in base 64 are `+` and `/`, which need URL-encoding and defeat the purpose. **And some shorteners drop `0`, `o`, `l` and `1`** to avoid transcription errors when somebody reads a code aloud — which takes it to base 58 and is a real product decision if links are ever spoken or printed.

Block allocation

```
class CodeGenerator:
    """Each server holds a block of ids and refills from a central counter."""

    BLOCK = 10_000

    def __init__(self, counter_store):
        self.counter = counter_store
        self.next_id = 0
        self.block_end = 0

    def next_code(self) → str:
        if self.next_id ≥ self.block_end:
            start = self.counter.increment_by(self.BLOCK) # atomic, e.g. Redis INCRBY
            self.next_id, self.block_end = start, start + self.BLOCK
        code_id = self.next_id
        self.next_id += 1
        return encode(scatter(code_id))
```

One atomic operation per ten thousand URLs. At two hundred writes a second that is one coordination call every fifty seconds — so the central counter is not a bottleneck by any measure, and if it is briefly unavailable, servers keep working until their blocks run out.

A server restarting abandons the rest of its block, which wastes up to ten thousand codes out of fifty-six billion. **That is a rounding error and it is the right trade**, and saying so pre-empts the obvious objection.

Making codes unguessable

```
# a large odd number, coprime with the space → a reversible scramble
MULTIPLIER = 8_648_838_431
SPACE = 62 ** 7

def scatter(n: int) → int:
    return (n * MULTIPLIER) % SPACE
```

Consecutive ids map to scattered codes, so `aK9x2b` gives no hint about `aK9x2c`, and it is reversible if you ever need the original id. **This is obfuscation, not security** — anyone determined can still probe — and the honest position is that **short links are not private and should never be treated as an access control mechanism**. If a link must be secret, it needs authentication, not a longer code.

The schema

```
CREATE TABLE urls (
    code          VARCHAR(8) PRIMARY KEY,
    long_url      TEXT          NOT NULL,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at    TIMESTAMPTZ,
    creator_ip    INET,
    is_disabled   BOOLEAN      NOT NULL DEFAULT false
);
CREATE INDEX ON urls (expires_at) WHERE expires_at IS NOT NULL;
```

`code` as the primary key makes the redirect a single index lookup with no join — which is the entire read path and the reason it can be fast.

`is_disabled` rather than deleting so that a takedown returns a clear “this link was removed” rather than a 404 that looks like a bug, and so that the code is never reissued.

Custom aliases live in the same table with the same uniqueness constraint, which is what makes “this alias is taken” a database error rather than application logic. **The generated codes must not collide with custom ones**, and the cleanest way is to require custom aliases to be at least a different length, or to reserve them in the same counter space.

The read path

```
def redirect(code: str):
    target = cache.get(code)                                # ~0.2 ms
    if target is None:
        row = db.fetch_one("SELECT long_url, is_disabled, expires_at FROM urls WHERE code = %s", code)
        if row is None:
            return http_404()
        if row.is_disabled or expired(row):
            return http_410()                                # Gone, not 404 – it existed
        target = row.long_url
        cache.set(code, target, ttl=3600)
    queue.publish({"code": code, "at": now(), "ip": ..., "ua": ...}) # fire and forget
    return http_302(target)
```

The click event is published and not awaited. If the queue is down, the redirect still works — analytics is not worth failing a redirect for, and that priority should be explicit in the code rather than accidental.

Negative caching matters too: a code that does not exist should be cached as “not found” for a short period, or a bot probing random codes generates a database query per probe.

Caching

```
cache the code → long_url mapping, TTL a few hours
expected hit rate: 90-99%
```

The access pattern is extremely skewed and time-bounded: a link is shared, gets a burst of traffic over hours or days, and then almost nothing. **LRU is close to ideal for that shape** — the hot links stay resident and the long tail evicts itself.

```
memory for 10,000,000 hot entries x ~200 bytes = ~2 GB
```

Two gigabytes covers ten million hot links, which for most shorteners is well beyond the working set.

Invalidation is the one wrinkle. If a link can be edited or disabled, the cached copy must be evicted — otherwise a takedown does not take effect for an hour. A short TTL bounds it; explicit invalidation on disable makes it immediate.

Analytics without hurting the redirect

```
redirect → publish {code, timestamp, ip, user_agent, referrer} → queue
                                     |
                                     aggregator → counts per code per hour
                                               → raw events to object storage
```

Aggregate rather than counting per click. A per-code hourly counter is a tiny amount of data; storing every click is a large amount and is only needed if the product sells per-visitor analytics.

And the counter must not be a hot row in the main database. Either a separate store, or Redis with periodic flushing, or an append-only table aggregated in batches.

Expiry and cleanup

A background job deletes rows past `expires_at`, in batches, off-peak. **Codes are not reused** even after expiry — the space is enormous and reuse recreates Deepak's barfi problem, where an old link suddenly points somewhere new.

Abuse, which is the part nobody plans for

A shortener is an ideal tool for hiding a malicious destination, and every real one is attacked within days of launch. The minimum:

- **Rate limit creation by IP and by account**, or one script creates a million links overnight.
- **Check destinations against a safe-browsing list** at creation and periodically afterwards, because a domain can be clean at creation and compromised later.
- **A preview page** — `sho.rt/aK9x2b+` showing the destination without following it.
- **A takedown path**, and `is_disabled` rather than deletion so the code is never reissued.
- **Block redirects to your own domain** and to `javascript:` or `data:` URIs, which is a stored-XSS vector.

Naming abuse unprompted is a strong signal, because it is the requirement that separates a design exercise from a product.

6 The numbers

The phase-2 arithmetic, in full:

new URLs	100,000,000 / month	= ~40 writes/s
peak (x5)	100,000,000 / 30 / 86,400	= ~200 writes/s
reads at 100:1		= ~4,000 reads/s
peak		= ~20,000 reads/s
storage per row	code 8 + url 200 + metadata 100	= ~500 bytes (with overhead)
per month	100,000,000 x 500 B	= 50 GB
per year		= 600 GB
after 5 years		= 3 TB

The decision that follows: two hundred writes a second and three terabytes over five years is **one database**. No sharding, no partitioning, no distributed anything. **The read path is the only thing that needs thought, and it is a cache.**

Code length:

$62^5 =$	916,132,832	~9 months at 100M/month
$62^6 =$	56,800,235,584	~47 years
$62^7 =$	3,521,614,606,208	~2,900 years

Collision probability, if you choose random codes instead — the birthday problem:

7 characters, space 3.5×10^{12}
 after 10,000,000 codes issued:
 $P(\text{a given new code collides}) = 10^7 / 3.5 \times 10^{12} = \sim 1 \text{ in } 350,000$

after 100,000,000 issued:
 $\sim 1 \text{ in } 35,000$

So random codes need a uniqueness check on write, and the retry rate stays negligible for a long time — but it grows, and it is a number to monitor rather than assume.

Cache sizing:

hot entries	10,000,000	
per entry	code 8 B + url 200 B + overhead	= ~250 B
	10,000,000 x 250 B	= 2.5 GB

hit rate 95%:		
4,000 reads/s x 0.05 = 200 database reads/s	→ trivial	
hit rate 50%:		
2,000 database reads/s	→ still fine, but 10x the load	

Latency budget:


```

cache hit          0.2 ms
+ network in/out   ~1 ms
+ application      ~0.5 ms
-----
                  ~2 ms at the server

cache miss         + 1-5 ms database read

```

Under five milliseconds either way, which is what makes the “single-digit milliseconds” requirement achievable without heroics.

Bandwidth:

```

a redirect response is headers only   ~500 bytes
20,000 reads/s x 500 B                = 10 MB/s = 80 Mbps

```

Trivial, which is worth noting because it means no CDN is required for the redirect itself — unlike almost every other design in this phase.

What 301 costs and saves:

```

average clicks per link           ~10
with 302: every click hits you     10 requests per link
with 301: only the first per browser ~1-2 requests per link

→ 301 cuts read traffic by roughly 5-10x
→ and eliminates 80-90% of your analytics

```

Analytics volume:

```

4,000 clicks/s x 86,400           = 345,000,000 events/day
raw event ~200 bytes              = 69 GB/day → object storage, partitioned
aggregated: counts per code per hour = a few million rows/day → tiny

```

Storing every raw click is 25 TB a year; storing hourly aggregates is gigabytes. Which you need is a product question, and the answer decides an order of magnitude of cost.

7 The trade-offs

Counter-based codes against random ones. The counter guarantees no collisions, needs no read before write, and produces the shortest possible codes — at the cost of a coordination point and sequential (therefore enumerable) ids. Random codes need a

uniqueness check on every write and a retry path, and are unguessable by construction. **Block allocation plus a scrambling multiply gets most of both**, and that is the answer I would defend.

301 against 302, and it is a product decision. 301 makes the second click free and permanent — no traffic, no analytics, and the link can never be repointed. 302 gives you every click and the ability to change the destination, at the cost of serving every request forever. **A commercial shortener uses 302 because clicks are the product; an internal link service might use 301 because traffic is the cost.**

Analytics depth against write volume. Counting per code per hour is a few million rows a day. Storing every click with IP, user agent and referrer is 69 GB a day. **The second is only worth it if somebody is paying for per-visitor reporting**, and it should be an explicit choice rather than a default.

Custom aliases are a small feature with a large surface. They need a uniqueness check, a reserved-word list — nobody should get `/admin` or `/login` — a length rule that keeps them out of the generated space, and a policy on impersonation. **They are the feature most likely to produce a security incident**, and worth saying so.

Caching against invalidation. A long TTL gives a high hit rate and means a take-down or an edit takes an hour to propagate. A short one is fresher and shifts load to the database. **For a system whose main abuse response is disabling links, that lag is a real cost**, so explicit invalidation on disable is worth the extra code.

And the meta trade: this system does not need to scale, and the temptation is to make it. Two hundred writes a second is one machine. **The failure mode of this interview is designing for a scale the arithmetic does not support**, and the arithmetic takes three minutes.

When would I build it differently? If codes must be unguessable for security reasons, random with a uniqueness check rather than a scrambled counter — and I would still say that short links are not a security mechanism. If it is internal and analytics do not matter, 301 and a much simpler system. And if it must be multi-region with low latency everywhere, the mapping is immutable once created, so it replicates trivially and the whole thing becomes a read-only edge cache with a slow write path — which is a genuinely different and much simpler architecture.

8 In the interview

How it gets asked

- “Design a URL shortener like bit.ly.” — the standard prompt.
- “How do you generate the short code?” — the deep dive most interviewers pick.
- “How long should the code be?” — the arithmetic question.
- “301 or 302?” — the one that separates candidates.
- “How do you handle a link that gets a million clicks in an hour?”
- “Someone is using it to hide phishing links.” — the abuse question.

The first ninety seconds

“Let me pin down scope, size it, and then design against the numbers.

In scope: create a short code for a URL, redirect on visit, optional custom alias, and basic click counts. **Out of scope:** user accounts, ads, link previews — tell me if any of those should be in.

The non-functional requirements decide more than the functional ones here. The redirect sits on somebody else’s critical path — a person clicked a link and is waiting — so single-digit milliseconds. Availability matters more than consistency, because a redirect that works with a slightly stale target beats one that fails, and an outage breaks every link anyone has ever shared. And it is extremely read-heavy: created once, followed many times.

Sizing, and this decides the shape. Say a hundred million new URLs a month. That is about forty writes a second average, two hundred at peak. At a hundred-to-one read ratio, four thousand reads a second, twenty thousand peak. Storage at five hundred bytes a row is fifty gigabytes a month, six hundred a year, three terabytes over five years.

So: none of this is hard. Two hundred writes a second is one modest database. Three terabytes fits on one disk. **There is no sharding problem here and I would resist inventing one.** The read path needs a cache, and that is the extent of the scaling work.

Which means the design problem is elsewhere, and it is two things: how the code is generated, and the redirect itself — specifically whether it is a 301 or a 302, which turns out to be a product decision rather than a technical one.

Shall I do the API and data model, and then go deep on the code generation?”

The follow-ups

“How do you generate the code?”

“Three options, and I would go through why I reject two of them.

Hashing the URL — take an MD5 or SHA and use the first few characters. Deterministic, so the same URL always gives the same code, which is either a feature or a bug depending on the product. **But it collides**, so every write needs a read to check, and a retry with a salt when it does. That is a read before every write on a path that otherwise needs none.

Random codes — seven random base-62 characters, check for a collision, retry. Unguessable by construction, which is genuinely valuable. Still needs a uniqueness check on write, and the collision rate grows as the space fills, so it is a number to monitor.

A counter encoded in base 62 — no collisions at all, by construction, no read before write, and the shortest possible codes. Two problems: a single global counter is a bottleneck and a single point of failure, and the codes are sequential and therefore enumerable.

I would take the counter and fix both. Block allocation: each server takes ten thousand ids at a time from a central counter, so coordination happens once per ten thousand URLs — at two hundred writes a second that is one call every fifty seconds, and a server crash wastes up to ten thousand codes out of fifty-six billion, which is a rounding error. **And a scrambling step** — multiply the id by a large odd number modulo the space — so consecutive ids map to scattered codes and the sequence is not walkable.

I would be explicit that scrambling is obfuscation and not security. Short links are not private and should never be used as an access control mechanism. If a link needs to be secret, it needs authentication.”

“How long should the code be?”

“Base 62 gives me 62 to the power of the length, so:

Six characters is fifty-six billion, which at a hundred million a month is about forty-seven years. **Seven is three and a half trillion**, about three thousand years.

Six is genuinely enough — no product outlives forty-seven years of continuous operation at that rate. **But I would use seven**, because one extra character costs nothing in a link that is already twenty characters with the domain, and it gives me an order of magnitude of headroom for being wrong about the growth rate. **Getting this wrong is expensive**: extending the length later means old and new codes have different lengths, which is fine technically and looks untidy, and shortening it is impossible.

Base 62 rather than base 64 because base 64’s extra characters are `+` and `/`, which need URL-encoding and defeat the point. **And some shorteners drop `0`, `1` and `l`** to base 58, so codes can be read aloud or printed without transcription errors — which is worth doing if links ever appear offline, and costs about ten percent more length.

If codes were random rather than sequential, the length calculation changes: the birthday problem means collisions start mattering well before the space is full. At seven characters and ten million codes issued, a new code collides about one time in three hundred and fifty thousand — negligible, and it grows linearly, so it is something to monitor rather than ignore.”

“301 or 302?”

“302, for a commercial shortener — and the interesting thing is that this is a product decision wearing a technical costume.

A 301 is a permanent redirect and browsers cache it. So the first click reaches me and every subsequent click from that browser goes straight to the destination. **That is fantastic for cost and latency** — it cuts read traffic by five to ten times at typical click-per-link rates — and it costs me two things I cannot get back. **I lose all analytics after the first click**, because I never see those requests. And **I can never repoint the link**, because browsers have cached the answer and there is no way to tell them otherwise.

A 302 is temporary and is not cached, so every click comes to me. I count every one, and I can change the destination later — which matters enormously for the abuse case, because disabling a malicious link only works if clicks still reach my servers.

For bit.ly-style products, clicks are literally the product — they sell the analytics — so it is 302 and the extra traffic is the cost of doing business.

For an internal link shortener where nobody is measuring anything and traffic is a cost, 301 is defensible and much cheaper.

The answer I would give is: 302, and here is what it costs me and what it buys — because the interviewer is checking whether I know there is a decision here at all, rather than which one I picked.”

“Someone is using it to hide phishing links.”

“That is the requirement most people leave out, and it is the one that makes this a product rather than an exercise. A shortener is an ideal tool for hiding a destination, and every real one is attacked within days.

Five things, roughly in order of value.

Rate limit creation, by IP and by account. Without it one script creates a million links overnight, which is both an abuse vector and a capacity problem.

Check the destination at creation against a safe-browsing list, and **recheck periodically**, because a domain that was clean when the link was made can be compromised a week later. That periodic recheck is the part people miss.

A preview mode — appending a character to the code shows the destination instead of following it — so a cautious user can look before clicking, and so security tools can inspect.

A takedown path, and disable rather than delete: `is_disabled` returns a clear 410 Gone, and the code is never reissued to someone else. **Deleting the row would let the code be reused**, which is exactly the failure where an old shared link suddenly points somewhere new.

And block redirects to `javascript:` and `data:` URIs, and to my own domain — the first is a stored-XSS vector and the second creates redirect loops.

The thing this changes about the architecture is that disabling must take effect immediately, which means explicit cache invalidation rather than waiting out a TTL. **And it is the argument against 301**, because a permanently cached malicious redirect cannot be recalled at all.”

The model answer

“Design a URL shortener. Forty-five minutes.” — the wrap-up, phase 6, which is what the whole session builds to.

“Let me close out with the bottleneck, the failure modes, what I would monitor, and what I left out.

The bottleneck is the redirect read, and it is a cache problem rather than a database one. At four thousand reads a second with a ninety-five percent hit rate, the database sees two hundred reads a second, which is nothing. **If the cache were cold or the hit rate collapsed, the database would see four thousand primary-key lookups a second, which one machine still handles** — so the cache is an optimisation rather than a load-bearing dependency, which is a nice property and worth saying.

What breaks at ten times the traffic: nothing structural. More cache, more read replicas, more application servers. **The write path at two thousand a second is still one database.** The first thing that would actually need rethinking is the analytics ingestion, at three and a half billion events a day.

The failure modes I would design for. The counter service being unavailable — servers keep issuing codes from their current block, so it is invisible for up to ten thousand URLs per server, and that is exactly why block allocation is worth it. The cache being unavailable — every read falls through to the database, which handles it. And a hot link getting a million clicks in an hour — that is one cache entry serving three hundred requests a second, which is what caches are for, and the click events go to a queue so the write amplification is absorbed.

What I would monitor: redirect p99, cache hit rate, the block counter’s remaining space, the rate of 404s — a spike means someone is enumerating codes — and the creation rate per IP.

What I deliberately left out and would do next, in order: **abuse prevention**, which I think is genuinely the most important missing piece and which I would not ship without; per-link analytics beyond a count; expiry, which is a background job over an indexed `expires_at` ; and multi-region, which for this system is unusually easy because the mapping is immutable once created — it replicates trivially and the whole thing becomes a read-only edge cache with a slow write path.

And the thing I would want to leave you with is that the arithmetic in phase two — forty writes a second, three terabytes over five years — is what let me spend the session on code generation and the redirect semantics rather than on sharding a database that does not need it.”

9 Recall card

Do the arithmetic first: 100M URLs/month is ~ 40 writes/s, ~ 200 peak, ~ 600 GB/year. That is **one database** — so the design problem is code generation and redirect latency, and **inventing a sharding problem is how this round is lost.**

Code generation: counter + base-62, with two fixes. **Block allocation** (10,000 ids per server, so coordination once per 10,000 and a crash wastes a rounding error) and **a scrambling multiply** so codes are not enumerable — while saying plainly that this is obfuscation, not security.

Length from arithmetic: $62^6 = 57$ billion ≈ 47 years; $62^7 = 3.5$ trillion. Six is enough, seven is the safe choice, and base 62 rather than 64 because `+` and `/` need escaping.

301 vs 302 is a product decision. 301 is cached by the browser — $5\text{--}10\times$ less traffic, **no analytics after the first click, and the link can never be repointed.** 302 gives every click and the ability to disable a malicious link, which is why commercial shorteners use it.

The click count never goes on the read path — publish to a queue, aggregate asynchronously. And **abuse is the requirement people forget:** rate-limit creation, scan destinations at creation *and periodically*, offer a preview, and **disable rather than delete** so codes are never reissued.

PRACTICE

Practice — House robber and the choice at each step

DSA topic: House robber and the choice at each step **System design topic:** Design a URL shortener

Code these, in this order

One rule for the whole set: **before writing the recurrence, write down the two branches as English.** “If I take this, then __ is forbidden, so I add to . *If I skip it,* .” Every problem below is the same two branches with different forbidding.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	House Robber	LeetCode 198 (Medium)	The take-or-skip recurrence and <code>dp[1] = max(m[0], m[1])</code> .
2	House Robber II	LeetCode 213 (Medium)	The circle, the two runs, and the <code>n = 1</code> guard.
3	Delete and Earn	LeetCode 740 (Medium)	Index by value — then it is problem 1.
4	Best Time to Buy and Sell Stock with Cooldown	LeetCode 309 (Medium)	The same forbidding, expressed as three states per day.
5	House Robber III	LeetCode 337 (Medium)	The same choice, on a tree instead of a line.
6	Maximum Sum with No Three Consecutive	—	A window of three; write it from the pattern.

On problem 1, write both wrong answers first

Implement greedy-by-value and alternating-indices. Run all three on `[900, 1600, 900]` and `[2, 1, 1, 2]` . Record six numbers. **Keep those two arrays** — they are the counter-examples you produce in the interview.

On problem 1, get `dp[1]` wrong on purpose

Set it to `money[1]` and run on `[5, 1, 5]` . Record the answer. Then write the state sentence above the line and see that the sentence and the code disagree.

On problem 2, break the guard

Run your circular version on `[5]` without the `n == 1` guard. Record what it returns. Then explain in one sentence why both slices are empty.

On problem 3, find the sizing trap

Run it on `[1, 1000000]` and record the size of the `earnings` array. Then say what the complexity actually is, in terms of which variable. Then sketch the sorted-distinct-values version and say what it costs instead.

On problem 5, notice what stays and what changes

The tree version has the same choice — take this node and its grandchildren are next, or skip it and take its children. Write the two branches as English before coding. Then say what replaced “index `i-2`”.

Then the reconstruction drill

Add `which houses` to problem 1. Do it once by walking the table backwards and once with an explicit took/skipped array. Then run the space-optimised version and confirm you cannot answer the question at all.

The choice drill

1. State the two branches in English, for house robber.
2. Do the same for delete-and-earn, cooldown and the tree version.
3. Say what the “forbidding” is in each, and how far back it reaches.
4. Say what the tell is for this family in a problem that never says “adjacent”.

The greedy-trap drill

1. Give the greedy-by-value counter-example and both totals.
2. Give the alternating counter-example and both totals.
3. Say in one sentence why no smarter greedy rule fixes it.
4. Say why “non-adjacent” and “alternating” are different.

The base-case drill

1. Give `dp[0]` and `dp[1]` and justify each from the state sentence.
2. Say what `dp[1] = money[1]` produces on `[5, 1, 5]`.
3. Say why the two-variable version needs no guards at all.

The circular drill

1. State the two runs and what each excludes.
2. Give both halves of the correctness argument.
3. Say why a single pass cannot enforce the constraint.
4. Give the input that breaks it without a guard.

The variations drill

1. Extend to a forbidden window of width `w`: the recurrence, the base cases, the space window.
2. Say why widening `w` does not make it harder.
3. Say what kind of forbidding rule *would* make it harder, with an example.

The costs drill

1. Give time and space for all five variants.
2. Give delete-and-earn's complexity and name the variable that is not `n`.
3. Compare against trying every subset, at `n = 30`.
4. Say why greedy is not even faster than the DP.

The break-it drill

Trigger each and record the exact output or error:

1. Greedy by value on `[900, 1600, 900]`.
 2. Alternating indices on `[2, 1, 1, 2]`.
 3. `dp[1] = money[1]` on `[5, 1, 5]`.
 4. The table version on a one-element input.
 5. The circular version on `[5]` with no guard.
 6. `earnings` sized by `len(nums)` instead of `max(nums)`.
 7. Asking which houses were robbed, from the two-variable version.
-

The requirements drill

1. Give the in-scope and out-of-scope lists you would state.
2. Give the three non-functional requirements and why each matters here.
3. Say why “the redirect is on somebody else’s critical path” changes the design.

The estimation drill

1. Do the full sizing from 100M URLs a month: writes, peak, reads, storage per year and per five years.
2. Say the decision sentence that follows.
3. Say what a candidate who starts sharding has missed.

The code-generation drill

1. Name the three schemes and reject two, with reasons.
2. Describe block allocation and compute the coordination rate at 200 writes/s.
3. Say what a server crash costs, and why that is acceptable.
4. Explain the scrambling multiply and state plainly what it is and is not.
5. Give the length arithmetic for 5, 6 and 7 characters.
6. Say why base 62 and not 64, and when you would use 58.

The redirect drill

1. Give the read path, step by step, with latencies.
2. Say where the click event goes and why it is not awaited.
3. Say what negative caching prevents.
4. Give the cache sizing for 10M hot entries and the expected hit rate.
5. Say what invalidation is needed and what forces it.

The 301/302 drill

1. State what each one does to the browser.
2. Give what you gain and lose with each — two each.
3. Say which a commercial shortener uses and why.
4. Say which an internal one might use and why.
5. Say how this interacts with the abuse response.

The abuse drill

1. Name the five defences in order.
2. Say which one people forget, and why it matters.
3. Say why `is_disabled` rather than deleting.
4. Say what redirect targets you would block outright.
5. Say what abuse prevention forces on the caching design.

The numbers drill

1. Storage per row, per month, per year, over five years.
2. Code space for lengths 5, 6, 7, and years of runway at 100M/month.
3. Collision probability for random 7-character codes after 10M and 100M issued.
4. Cache memory for 10M entries, and database load at 95% and 50% hit rates.
5. Latency budget for a cache hit and a miss.
6. Analytics volume: raw events per day and per year, against hourly aggregates.
7. The traffic reduction 301 buys, and what it costs.

The failure drill

For each, say what happens and what you would build:

1. The central counter service is unavailable for ten minutes.
2. An application server restarts mid-block.
3. A link gets a million clicks in an hour.
4. A bot probes a million random codes.
5. A link is disabled but was served as a 301 last week.
6. A custom alias request for `/admin`.
7. The click-event queue is down.

Two of the seven are invisible to users by design. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Maximise the sum with no two adjacent elements.* The two branches as English, the recurrence, the state sentence and the base case it forces, both counter-examples with their numbers, and the space collapse.
 2. *Now the houses are in a circle.* The two runs, both halves of the correctness argument, why a single pass cannot work, and the `n = 1` guard.
 3. *Design a URL shortener.* The scope, the sizing ending in “this fits on one machine”, code generation with block allocation and the length arithmetic, the redirect path with the click event off it, and 301-versus-302 named as a product decision.
-

Before you move on

- [] I write the two branches as English before the recurrence.
- [] I can give the take-or-skip recurrence from the choice.
- [] I know `dp[i]` means “considering houses 0..i” and what that forces for `dp[1]`.
- [] I have both counter-examples memorised with their numbers.
- [] I can say why no greedy rule fixes it.
- [] I know non-adjacent is not alternating.
- [] I can give the circular two-run trick and both halves of its correctness.
- [] I know the `n == 1` guard and why it is needed.
- [] I know the two-variable version needs no guards.
- [] I can extend to a window of width `w` and say why it is no harder.
- [] I know what kind of forbidding rule *would* be harder.
- [] I can do the delete-and-earn transformation and name its real complexity variable.
- [] I know reconstruction and space optimisation are mutually exclusive.
- [] I do the URL shortener sizing before anything else.
- [] I can say the decision sentence that follows from it.
- [] I can reject two code-generation schemes with reasons.
- [] I can describe block allocation and compute its coordination rate.
- [] I can give the length arithmetic and justify six or seven characters.
- [] I know why base 62, and when 58.
- [] I know the click event goes on a queue and is not awaited.
- [] I know what negative caching prevents.
- [] I can state the 301/302 trade with what each gains and loses.
- [] I know why a commercial shortener uses 302.
- [] I can name five abuse defences and the one people forget.
- [] I know why `is_disabled` rather than deleting.
- [] I know what abuse prevention forces on caching.
- [] I can give the analytics volume both ways.
- [] I answered all three questions above out loud.

Finding the state: the hardest part of DP

DATA STRUCTURES AND ALGORITHMS

Finding the state: the hardest part of DP

SYSTEM DESIGN

Design a pastebin

Finding the state: the hardest part of DP

1 What this is, and why they ask it

Everything else in dynamic programming is mechanical. Once you know what the subproblem is, the recurrence follows from the last-move question, the base cases follow from the recurrence, and the code follows from both.

Deciding what the subproblem is — the state — is the part that is not mechanical, and it is where every hard DP problem is actually hard. A wrong state does not produce an error. It produces a solution that passes the examples and is confidently wrong on the third test case, because two genuinely different situations were treated as the same one.

They ask “what does `dp[i]` represent?” in almost every DP interview, and it is not a comprehension check. It is the question, because a candidate who can state the state precisely has already done the hard part, and one who cannot is about to write something that nearly works.

The test is one sentence, and it is worth having as a reflex: *given only the state, can I decide what happens next without knowing anything else about how I got here?* If the answer is no, the state is incomplete, and whatever is missing is the next dimension.

By the end of this lesson you can apply that test, recognise the five things that most often need to be in the state, add a dimension without panicking, tell an incomplete state from a merely awkward one, and count the state space before writing code.

2 The story

Ganpat has been a conductor on the Pune–Satara route for twenty-two years, and for the first two of them he was, by his own account, useless.

The bus takes about seventy people and stops at fourteen places. The job is not the ticketing — anybody learns that in a week. The job is standing at the door at each stop and deciding, in about four seconds, how many people to let on.

What he did for two years was keep one number in his head: how many people were on board. Seventy is the limit, so if he had fifty-eight, he let on twelve.

It went wrong constantly and he could not work out why.

At Shirwal he would let on twelve, and then at the next stop eighteen people would get off and the bus would be half empty for the longest stretch of the route — money he could have had. Or he would refuse people at Khambatki because he was at seventy, and then nobody got off for two stops and he had refused them for nothing.

The change came from an older conductor who worked the same route and who said something Ganpat has repeated many times since.

“You are counting the wrong thing. Count where they are going.”

So he started keeping — and this took him a while, because it is more to hold — not one number but a small set of them. Eleven for Shirwal. Twenty-two for Satara. Six for Wai. And so on down the route.

Now the decision at the door is easy, and it is easy because it is the same decision it always was, with enough information to make it. Standing at Khambatki with sixty-six on board, he knows that twenty-nine of them get off right here. So he can take on twenty-nine plus the four spare, and he does not have to guess, and he is not refusing people he could carry.

The thing he says about it is that he did not get better at the job. **The job was always this. He was just trying to do it with one number when the situation had never been one number.**

His son, who drives now, asked him once why nobody told him at the start. Ganpat said somebody probably did.

3 The idea in plain English

Ganpat's small set of numbers is the state, and his two useless years are what an incomplete state costs.

The state is what identifies a subproblem. It is the arguments to the recursive function, or the indices into the table. `dp[i]`, `dp[i][j]`, `dp[i][j][k]`.

And the whole of DP rests on one requirement: the state must be enough. Enough for what? For deciding what happens next, and for the answer to be the same every time you reach that state.

The test, which is the reusable thing from today:

Given only the state, can I decide what happens next without knowing anything else about how I got here?

If yes, the state is complete and the DP will work. **If no, whatever you needed to know is the missing dimension.**

Ganpat's total was not enough. Standing at the door with sixty-six on board, he could not decide how many to let on, because he needed to know how many were getting off — and that was not derivable from sixty-six. So sixty-six was an incomplete state, and the fix was not to think harder about sixty-six.

Why an incomplete state fails silently is worth understanding. If the state is missing something, then two different situations map to the same table cell. The first one computed writes its answer there; the second one reads it. **The second answer is not merely approximate — it is the answer to a different question**, and nothing anywhere reports a problem.

Five things most often need to be in the state, and recognising them is most of the skill:

One: a resource that is being consumed. Remaining capacity in a knapsack, money left, moves left, fuel. `dp[i]` is not enough because “at item `i` with 8 kg left” and “at item `i` with 3 kg left” have different answers. → `dp[i][remaining]`.

Two: a count of something limited. Transactions used, stops taken, deletions allowed, jumps made. “Reached Delhi in two stops” and “reached Delhi in four stops” have different futures. → `dp[i][used]`.

Three: a mode you are currently in. Holding a stock or not. Inside a group or not. On a cooldown or not. **The tell is a rule that says “after you do X you cannot do Y”** — the “after X” is a mode. → `dp[i][holding]`.

Four: a position in a second sequence. Every problem comparing two strings — edit distance, longest common subsequence — needs both positions, because “at index 4 of the first string” says nothing without knowing where you are in the second. → `dp[i][j]`.

Five: a set of things already used, when there are few enough of them. Cities visited in a travelling salesman route, people already matched. → a bitmask, `dp[i][mask]`, and only when `n` is about twenty or less because the state space is 2^n .

Now the distinction that matters most: an *incomplete* state and an *awkward* state are different problems.

Incomplete means the answer genuinely depends on something you have not recorded, and the fix is another dimension. There is no way round it.

Awkward means the state is sufficient but the recurrence is unpleasant — and the fix is to redefine what `dp[i]` means, not to add to it. Maximum subarray is the standard example: “the best subarray using elements up to `i`” is complete but leads nowhere, while “the best subarray **ending exactly at** `i`” is the same amount of information and gives a one-line recurrence. **Changing the meaning is often more powerful than adding a dimension**, and it is the move people do not think of.

Adding a dimension is not a defeat and it is not expensive to think about. The recurrence changes in one predictable way: every branch now says what happens to the new dimension as well. Take the item $\rightarrow$ `remaining` drops by its weight. Sell the stock $\rightarrow$ `holding` becomes false. **The last-move question still produces it**; there is just more to say about each move.

And before writing anything, count the state space and say the number. States times work per state is the cost, and both terms come from the state you just defined:

<code>dp[i]</code>	<code>n</code> states	
<code>dp[i][j]</code>	<code>n</code> x <code>m</code>	
<code>dp[i][capacity]</code>	<code>n</code> x <code>W</code>	$\leftarrow$ <code>W</code> is the CAPACITY, not the item count
<code>dp[i][mask]</code>	<code>n</code> x 2^n	$\leftarrow$ fine at <code>n=20</code> , impossible at <code>n=40</code>

That last line is why the counting matters. A state that is correct and has `2^40` cells is not a solution, and noticing that before writing is much better than noticing it afterwards.

4 The picture

The test, applied:

"Given only the state, can I decide what happens next?"

CLIMBING STAIRS, state = i

at step 8, how many ways to finish?

→ depends only on 8. COMPLETE.

KNAPSACK, state = i

at item 5, what is the best I can do?

→ depends on how much capacity is LEFT, which is not in the state.

→ INCOMPLETE. Add it: dp[i][remaining]

STOCK WITH ONE TRANSACTION, state = day

on day 9, what is the best profit from here?

→ depends on whether I am currently HOLDING a share.

→ INCOMPLETE. Add it: dp[day][holding]

EDIT DISTANCE, state = i

at index 4 of the first string, what is the cost?

→ meaningless without knowing where I am in the SECOND string.

→ INCOMPLETE. Add it: dp[i][j]

What an incomplete state does, concretely:

knapsack with state = i only, items (weight, value):

A(3, 4) B(4, 5) C(2, 3) capacity 5

path 1: take A → at item 1, remaining 2

path 2: skip A → at item 1, remaining 5

BOTH write to dp[1].

whichever runs first wins, and the other reads an answer computed under a completely different amount of capacity.

no error. the table is full. the number is wrong.

Adding a dimension, drawn:

ONE DIMENSION

dp[i]

i: 0 1 2 3

. . . .

one answer per item

"at item 2"

→ not a question

TWO DIMENSIONS

dp[i][remaining]

remaining

0 1 2 3 4 5

i=0

→ i=1

i=2

i=3

"at item 2 with 3 kg left"

→ a question with one answer

Redefining rather than extending:

MAXIMUM SUBARRAY

state A: dp[i] = the best subarray using elements 0..i

→ complete, and useless:

to extend it I need to know whether the best one
ENDS at i, and that is not recorded.

state B: dp[i] = the best subarray ENDING EXACTLY at i

→ same amount of information, one-line recurrence:

dp[i] = max(dp[i-1] + nums[i], nums[i])

→ and the answer is max(dp), not dp[n-1]

SAME dimension count. Different meaning. That is the move.

5 The code, built step by step

Start with a problem where one dimension is not enough, and watch it fail.

The 0/1 knapsack: items with weights and values, a capacity, take each item at most once, maximise value.

The naive one-dimensional attempt:

```
@lru_cache(maxsize=None)
def best_wrong(i: int) → int:
    """dp[i] = the best value from items i onwards. INCOMPLETE."""
    if i == len(items):
        return 0
    weight, value = items[i]
    return max(value + best_wrong(i + 1),      # take it -- but with what capacity?
               best_wrong(i + 1))            # skip it
```

Read the `take` branch. It adds the value and moves on, and nothing anywhere records that capacity was consumed. **So this takes every item**, and returns the sum of all values regardless of the capacity.

Apply the test: “at item 5, what is the best I can do?” — that depends on how much room is left, and the state does not say. **Incomplete.**

The fix, and the recurrence changes in exactly one predictable way:

```
@lru_cache(maxsize=None)
def best(i: int, remaining: int) → int:
    """dp[i][remaining] = the best value from items i onwards, with `remaining` capacity."""
    if i == len(items):
        return 0
    weight, value = items[i]
    skip = best(i + 1, remaining)
    take = value + best(i + 1, remaining - weight) if weight ≤ remaining else 0
    return max(take, skip)
```

Two things changed. The state gained a dimension. And each branch now says what happens to it: skipping leaves `remaining` alone, taking reduces it. **That is all adding a dimension ever means.**

Note the `if weight ≤ remaining`, which is the new dimension making a branch *illegal* rather than merely worse. That is common and it is the thing to check when adding a resource.

Now a mode dimension, which looks different and is the same idea. Buying and selling a stock, at most one transaction, and you cannot buy while holding:

```
@lru_cache(maxsize=None)
def profit(day: int, holding: bool) → int:
    """dp[day][holding] = the best profit from `day` onwards, given whether I hold a share."""
    if day == len(prices):
        return 0
    do_nothing = profit(day + 1, holding)
    if holding:
        act = prices[day] + profit(day + 1, False)      # sell
    else:
        act = -prices[day] + profit(day + 1, True)       # buy
    return max(do_nothing, act)
```

“On day 9, what should I do?” is unanswerable without knowing whether I am holding, so `holding` is in the state. **Two values, so the state space only doubles** — a cheap dimension.

And a count dimension, which is the third kind. At most `k` transactions:

```

@lru_cache(maxsize=None)
def profit_k(day: int, holding: bool, transactions_left: int) → int:
    if day == len(prices) or transactions_left == 0:
        return 0
    do_nothing = profit_k(day + 1, holding, transactions_left)
    if holding:
        act = prices[day] + profit_k(day + 1, False, transactions_left - 1) # a sale completes one
    else:
        act = -prices[day] + profit_k(day + 1, True, transactions_left)
    return max(do_nothing, act)

```

Three dimensions, and the state space is $\text{days} \times 2 \times (k+1)$. Count it before writing: at 1,000 days and $k = 100$, that is 202,000 states — fine. **At k unbounded it would be $1,000 \times 2 \times 1,000 = \text{two million}$** , still fine, and worth having computed rather than assumed.

Note where the transaction is counted — on the sale, not the purchase. Either works as long as it is consistent, and mixing them double-counts. **That is a decision to state, not a detail.**

Now the redefinition move, which is the alternative to adding a dimension. Maximum subarray, done the awkward way first:

```

# dp[i] = the best subarray among elements 0..i    -- complete, and useless
# to compute dp[i] I need to know whether the best subarray for i-1 ENDED at i-1,
# which dp[i-1] does not tell me.

```

The state is sufficient in principle — the answer for $0..i$ is determined by i — but the *recurrence* cannot be written, because extending requires information the state does not carry.

```

def max_subarray(nums: list[int]) → int:
    # dp[i] = the best sum of a subarray ENDING EXACTLY at i
    best_here = best_overall = nums[0]
    for value in nums[1:]:
        best_here = max(best_here + value, value)
        best_overall = max(best_overall, best_here)
    return best_overall

```

Same one dimension. Different meaning. One-line recurrence. And the consequence of the new meaning is that the answer is the maximum over all cells rather than the last one.

The check to run before coding anything: count the state space.


```
def state_space_report(n: int, capacity: int, k: int) → None:
    print(f"dp[i]                {n:>15,}")
    print(f"dp[i][j]            {n * n:>15,}")
    print(f"dp[i][capacity]      {n * (capacity + 1):>15,}")
    print(f"dp[i][k][mode]       {n * (k + 1) * 2:>15,}")
    print(f"dp[i][mask]          {n * 2 ** min(n, 40):>15,}")
```

If the number is above about ten million, the state is wrong or the problem is not DP, and finding that out in ten seconds is much better than finding it out in twenty minutes.

The complete solution

```
"""Finding the state: the test, the five kinds of dimension, and what an incomplete state does."""
from __future__ import annotations
from functools import lru_cache

# ---- 1. a resource being consumed -----
def knapsack(items: list[tuple[int, int]], capacity: int) → int:
    """dp[i][remaining] = the best value from items i.. with `remaining` capacity.

    One dimension is NOT enough: "at item 5" has no answer without knowing
    how much room is left.
    """
    @lru_cache(maxsize=None)
    def best(i: int, remaining: int) → int:
        if i == len(items):
            return 0
        weight, value = items[i]
        skip = best(i + 1, remaining)
        if weight > remaining:
            return skip
        return max(value + best(i + 1, remaining - weight), skip)

    return best(0, capacity)

def knapsack_broken(items: list[tuple[int, int]], capacity: int) → int:
    """The one-dimensional attempt. Takes everything, because nothing tracks capacity."""
    @lru_cache(maxsize=None)
    def best(i: int) → int:
        if i == len(items):
            return 0
        _, value = items[i]
        return max(value + best(i + 1), best(i + 1))

    return best(0)

# ---- 2. a mode you are in -----
def one_transaction(prices: list[int]) → int:
    """dp[day][holding]. "What now?" is unanswerable without knowing if I hold."""
    @lru_cache(maxsize=None)
    def profit(day: int, holding: bool) → int:
        if day == len(prices):
            return 0
        wait = profit(day + 1, holding)
        if holding:
            return max(wait, prices[day] + profit(day + 1, False))
        return max(wait, -prices[day] + profit(day + 1, True))

    return profit(0, False)

# ---- 3. a count of something limited -----
def at_most_k_transactions(prices: list[int], k: int) → int:
    """dp[day][holding][left]. A sale consumes one transaction – state that, and be consistent."""
    @lru_cache(maxsize=None)
    def profit(day: int, holding: bool, left: int) → int:
        if day == len(prices) or left == 0:
            return 0
        wait = profit(day + 1, holding, left)
        if holding:
            return max(wait, prices[day] + profit(day + 1, False, left - 1))
        return max(wait, -prices[day] + profit(day + 1, True, left))

    return profit(0, False, k)
```

```
# ---- 4. redefining rather than extending -----

def max_subarray(nums: list[int]) → int:
    """dp[i] = best sum ENDING EXACTLY at i. Same dimension, different meaning."""
    best_here = best_overall = nums[0]
    for value in nums[1:]:
        best_here = max(best_here + value, value)
        best_overall = max(best_overall, best_here)
    return best_overall

# ---- the check to run before coding -----

def state_space(n: int, capacity: int = 0, k: int = 0, mask_bits: int = 0) → int:
    total = n
    if capacity:
        total *= capacity + 1
    if k:
        total *= k + 1
    if mask_bits:
        total *= 2 ** mask_bits
    return total

if __name__ == "__main__":
    items = [(3, 4), (4, 5), (2, 3)]
    print("knapsack correct :", knapsack(items, 5))
    print("knapsack broken  :", knapsack_broken(items, 5), " (it takes everything)")
    print("sum of all values:", sum(v for _, v in items))
    print()

    prices = [7, 1, 5, 3, 6, 4]
    print("one transaction  :", one_transaction(prices))
    print("k = 2             :", at_most_k_transactions(prices, 2))
    print("k = 100            :", at_most_k_transactions(prices, 100))
    print()

    print("max_subarray      :", max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4]))
    print()

    print("state space, n=1000, capacity=10000 :", f"{state_space(1000, capacity=10000):,}")
    print("state space, n=1000, k=100, mode      :", f"{state_space(1000, k=100) * 2:,}")
    print("state space, n=20, bitmask             :", f"{state_space(20, mask_bits=20):,}")
    print("state space, n=40, bitmask             :", f"{state_space(40, mask_bits=40):,}")
```

Running it:

```
knapsack correct : 7
knapsack broken  : 12 (it takes everything)
sum of all values: 12

one transaction  : 5
k = 2            : 7
k = 100          : 7

max_subarray     : 6

state space, n=1000, capacity=10000 : 10,001,000
state space, n=1000, k=100, mode     : 202,000
state space, n=20, bitmask            : 20,971,520
state space, n=40, bitmask            : 43,980,465,111,040
```

Three things to look at. **The broken knapsack returns 12, which is the sum of every value** — it took all three items into a capacity of 5, because nothing in the state stopped it. **It did not crash and it did not warn**, which is the failure mode this lesson exists for.

`k = 100` gives the same answer as `k = 2`, because there are only two profitable transactions available. That is a good sanity check that the transaction counting is right.

And the last two lines are why you count before writing: **a bitmask at `n = 20` is twenty million cells and viable; at `n = 40` it is forty-four trillion and is not**. The state is correct in both cases; only one of them is a solution.

6 What it costs

The formula, and both terms come from the state:

```
time = (number of states) x (work per state)
space = (number of states), before optimisation
```

So counting the state space is costing the solution, and it takes ten seconds:

```
dp[i]                n
dp[i][j]             n x m
dp[i][capacity]      n x (W + 1)
dp[i][k][mode]       n x (k + 1) x 2
dp[i][mask]          n x 2^n
```

Concrete sizes, and where each one stops being viable:

<code>n = 1,000</code>	1,000	trivial
<code>n = 1,000, m = 1,000</code>	1,000,000	fine
<code>n = 1,000, capacity = 10,000</code>	10,000,000	fine, ~1 s in Python
<code>n = 1,000, capacity = 1,000,000</code>	1,000,000,000	too big
<code>n = 20, bitmask</code>	20,000,000	fine
<code>n = 25, bitmask</code>	800,000,000	borderline
<code>n = 40, bitmask</code>	4.4×10^{13}	impossible

The practical ceiling in Python is around ten million states with constant work each. Above that, either the state is wrong, the problem wants a different technique, or a dimension can be collapsed.

The knapsack's capacity dimension is worth a specific note, because it is the classic surprise:

```
dp[i][capacity] → O(n × W)
```

That is **not polynomial in the input size**, because `W` is a number written in the input, and writing `W = 1,000,000` takes seven characters. This is “**pseudo-polynomial**”, and it is why knapsack is NP-hard despite having a DP solution — the table is polynomial in the *value* of `W`, not in the number of digits.

<code>W = 1,000</code>	<code>n × 1,000</code>	fine
<code>W = 10⁹</code>	<code>n × 10⁹</code>	impossible, and the input got 6 characters longer

Saying that sentence is a strong signal, because it shows you know why a DP solution does not contradict NP-hardness.

Adding a dimension multiplies, it does not add:

<code>dp[i]</code>	1,000 states	
<code>dp[i][j]</code>	<code>1,000 × 1,000 = 1,000,000</code>	→ 1,000x
<code>dp[i][j][k]</code>	<code>1,000 × 1,000 × 100 = 10⁸</code>	→ another 100x

So a third dimension is usually the point where the approach needs rethinking, and a fourth almost always is.

Which dimensions are cheap and which are expensive:

a boolean mode	<code>x2</code>	always cheap
a small count (<code>k</code>)	<code>x(k+1)</code>	cheap when <code>k</code> is bounded
a second sequence	<code>x m</code>	usually fine
a capacity	<code>x W</code>	depends entirely on <code>W</code> 's magnitude
a bitmask	<code>x 2ⁿ</code>	only for <code>n ≤ ~22</code>

The work per state matters as much as the count, and it is the length of the last-move enumeration:

knapsack	2 branches	<code>O(1)</code> per state
coin change	one per coin	<code>O(coins)</code> per state
word break	one per split point	<code>O(n)</code> per state
interval DP	one per split point	<code>O(n)</code> per state → <code>O(n³)</code> overall

And an incomplete state costs nothing and returns the wrong answer, which is the whole point:

broken knapsack	<code>n</code> states, <code>O(1)</code> each	→ fast, and wrong
correct knapsack	<code>n × W</code> states	→ slower, and right

There is no performance argument for the incomplete state. It is not a trade-off; it is a bug that happens to run quickly.

7 The traps

The state that is missing a resource

```
@lru_cache(maxsize=None)
def best(i):
    return max(value[i] + best(i + 1), best(i + 1))
```

```
>>> knapsack_broken([(3,4), (4,5), (2,3)], capacity=5)
12                                     # the sum of everything. Capacity 5, weight 9.
```

Nothing tracks the resource, so nothing prevents taking every item. The failure is not subtle once you see it and is invisible while writing it, because the `take` branch looks complete — it adds the value and moves on.

The test catches it in five seconds: “at item 5, what is the best I can do?” — it depends on the remaining capacity, which is not in the state.

Memoising on fewer arguments than the state has

```
cache = {}
def best(i, remaining):
    if i in cache:
        return cache[i]
    ...
    cache[i] = answer
```

This is the same bug wearing a disguise. The function takes the right arguments; the cache does not. Two different `remaining` values share a cell, and whichever runs first wins.

```
>>> knapsack_bad_cache([(3,4), (4,5), (2,3)], 5)
9                                     # neither 8 nor 12 – a mixture
```

The cache key must be the complete state, every time. `@lru_cache` does this automatically, which is a real argument for using it over a hand-rolled dictionary.

An unhashable argument

```
@lru_cache(maxsize=None)
def solve(i, used: list[int]):
    ...
```

```
TypeError: unhashable type: 'list'
```

Usually a signal that the state is wrong, not merely awkwardly typed. If the state genuinely needs “which items are used”, that is a bitmask — an integer — and the state space is 2^n , which you should count before committing.

Adding a dimension that is derivable

```
dp[i][total_so_far] # when total_so_far is determined by i
```

If the extra value can be computed from the others, it is not a dimension — it is redundancy, and it multiplies the state space for nothing.

The test again: *does the answer differ for two states that agree on everything else?* If not, drop it. Adding dimensions defensively is a real habit and it turns a viable solution into an impossible one.

Not counting the state space

```
@lru_cache(maxsize=None)
def solve(i, mask): # n = 40
    ...
```

The code is correct. It never finishes, and there is no error — just a process that runs until it is killed, or a `MemoryError` after several minutes.

```
n = 40 bitmask → 4.4 x 1013 states
```

Ten seconds of arithmetic before writing would have said so, and the correct response is to look for a different formulation rather than to wait.

Confusing an incomplete state with an awkward one

```
# "dp[i] = the best subarray among 0..i doesn't work, so I need dp[i][j]"
```

No. That state is complete — the answer for `0..i` is determined by `i`. What is missing is a *usable recurrence*, and the fix is to change what `dp[i]` means to “ending exactly at `i`”, which is the same dimension.

Adding a dimension when a redefinition would do turns an $O(n)$ solution into $O(n^2)$, and it is a common over-reaction to being stuck.

Inconsistent accounting on a count dimension

```
if holding:
    return prices[day] + profit(day + 1, False, left - 1)    # sale consumes one
else:
    return -prices[day] + profit(day + 1, True, left - 1)     # ... and so does the purchase
```

Each round trip now consumes two transactions, so `k = 2` behaves like `k = 1`.

```
>>> at_most_k_double_count([7,1,5,3,6,4], 2)
5                                     # the answer is 7
```

Decide whether a transaction is counted at the buy or the sell, state it, and do it in exactly one place. The bug returns a plausible smaller number, which is the worst kind.

Off-by-one on a count's base case

```
if left == 0:
    return 0
```

Correct if `left` means “transactions still available”. **Wrong if it means “transactions used”**, where the guard should be `left == k`. Both readings are natural; **the state sentence decides which**, and writing it down prevents the confusion.

8 In the interview

How it gets asked

- “What does `dp[i]` represent in your solution?” — asked in almost every DP round.
- “Your solution gives the right answer on the example and the wrong one here.” — an incomplete state.
- “Now there is a limit of at most `k` transactions.” — a count dimension.
- “Now you cannot buy the day after selling.” — a mode dimension.
- “How many states are there?” — the sizing question.

- “Can you do it with less memory?” — which requires knowing what each cell depends on.

The first ninety seconds

“Before I write anything, let me work out what the subproblem is, because that is the part that is not mechanical — once the state is right, the recurrence and the base cases follow.

The test I use is: given only the state, can I decide what happens next without knowing anything else about how I got here?

So for this problem — if the state is just the item index, then ‘at item 5, what is the best I can do?’ has no answer, because it depends on how much capacity is left, and that is not in the state. **Two different paths reach item 5 with different amounts of room, and they have different answers, so they cannot share a table cell.**

That is the failure mode I want to avoid, and it is silent. If the state is incomplete, two genuinely different situations write to and read from the same cell, and the result is not approximate — it is the answer to a different question, with nothing anywhere reporting a problem.

So the state is (item index, remaining capacity), and I would write that as a sentence: `dp[i][r]` is the best value obtainable from items `i` onwards with `r` capacity remaining.

Adding the dimension changes the recurrence in exactly one way: each branch now says what happens to it. Skip the item and `r` is unchanged; take it and `r` drops by the weight — and taking becomes illegal if the weight exceeds `r`, which is the new dimension pruning branches rather than just costing them.

And before I write it, let me count the state space, because that is the cost: `n` times `W + 1`. With a hundred items and a capacity of a thousand, that is a hundred thousand cells with constant work each — trivial. **If the capacity were a billion, this approach would not work at all**, and I would rather know that now.

Shall I write the memoised version?”

The follow-ups

“What does `dp[i][j]` represent? Be precise.”

“For edit distance: **the minimum number of operations to turn the first i characters of string A into the first j characters of string B.**

And the precision matters in two specific ways. “The first i characters’ rather than ‘up to index i ’ — that is a prefix-length convention, so the table is $(n+1) \times (m+1)$ and $dp[0][0]$ is the empty-to-empty case, which is zero. If I said ‘up to index i ’ the table would be $n \times m$ and I would have nowhere to put the empty prefixes.

And ‘turn A into B’ rather than ‘the difference between them’ — the operations are directional, so I need to know which way round, otherwise insert and delete get confused.

The reason I labour this is that the sentence produces the rest of the solution. The base cases fall out: $dp[i][0]$ is i , because turning i characters into nothing means i deletions. $dp[0][j]$ is j . And the answer is $dp[n][m]$, the full lengths, which follows from ‘the first i ’ meaning a length.

A vaguer sentence gives a vaguer table, and the off-by-one errors in this problem all come from not having decided whether the index is a position or a length.”

“My solution passes the example and fails the third test case.”

“Almost always an incomplete state, and I would check that before anything else.

The way it fails is diagnostic. If the answer is too large in a maximisation, some constraint is not being enforced — which usually means the resource being consumed is not in the state, so the recursion is taking things it cannot afford. If it is too small, a branch is being wrongly pruned or a mode is being lost.

The check is to apply the test to my own state. Pick a middle cell and ask: could two genuinely different situations reach here? If yes, what distinguishes them, and is it recorded? That question finds it in about a minute, and it is faster than tracing the recursion.

A concrete way to confirm it is to print the arguments on every call and look for two calls with the same arguments that ought to have different answers. If the cache is returning a hit for something that should be different, the state is short.

The other candidate, and I would check it second, is that the cache key does not match the function’s arguments — a hand-rolled dictionary keyed on `i` when the function takes `(i, remaining)`. That produces exactly the same symptom and is easy to miss, which is one reason I prefer `lru_cache` : it keys on everything automatically.”

“Now there is a limit of at most `k` transactions.”

“A count dimension. The state becomes `(day, holding, transactions_left)`, and everything else is unchanged in structure.

Why it has to be in the state: ‘on day 9, holding a share’ is no longer enough, because reaching that point with four transactions left and with zero left have completely different futures. Two different situations, so two different cells.

The decision I would state explicitly is where the transaction is counted — at the buy or at the sell. I would count it at the sell, because a completed transaction is a round trip. **Whichever I choose, it must happen in exactly one place;** decrementing on both halves makes each round trip cost two, and then `k = 2` behaves like `k = 1` and returns a plausible smaller number.

State space: $\text{days} \times 2 \times (k+1)$. At a thousand days and `k = 100` that is two hundred and two thousand cells, which is nothing. **And there is a nice observation worth making:** `k` above `n/2` is the same as unlimited, because you cannot complete more than `n/2` round trips in `n` days — so I would cap `k` at `n/2` up front, which turns a potentially huge `k` into a bounded one and handles the ‘unlimited transactions’ variant with the same code.”

“Reduce the memory.”

“That requires knowing exactly what each cell depends on, which is another reason the state sentence is worth writing.

For the knapsack, `dp[i][r]` **reads only row** `i+1` — nothing from `i+2` or earlier. So I keep one row instead of `n`, and the space goes from `O(n·W)` to `O(W)`.

And then the iteration direction becomes part of the correctness, which is the subtlety. For 0/1 knapsack, where each item may be used once, I iterate capacity **downwards**, so that when I read `row[r - weight]` it still holds the previous item’s value. **Iterating upwards makes it the unbounded knapsack** — the same code, silently solving a different problem, because the item can then be re-used within the same row.

That is a genuinely nasty bug because both versions are correct programs for different problems, and the only difference is a `range` direction.

What I give up is the ability to say *which* items were chosen, because the table recorded that and I discarded it. So I would ask whether the value alone is enough before optimising — and if the items are wanted, keep the full table and walk it backwards, comparing each cell against the row above to see whether the item was taken.”

The model answer

“You are given an array of prices and may complete at most two transactions, buying and selling one share at a time. Maximise your profit. Walk me through how you decide the state.”

“Let me do this slowly, because deciding the state is the whole problem and the code is short once it is right.

First attempt: `dp[day]` — the best profit from this day onwards. Apply the test: on day 9, what is the best I can do? **That depends on whether I currently hold a share**, because if I do, my options are sell or wait, and if I do not, they are buy or wait. Different options means different answers, so `day` alone is incomplete.

Second attempt: `dp[day][holding]`. Now ‘day 9, holding’ is a question with an answer. **And this is exactly right for the unlimited-transactions version.** But here there is a limit of two.

Apply the test again: on day 9, holding a share, what is the best I can do? That depends on how many transactions I have already used — reaching that point having used none and having used two are completely different situations, and the second one has no future at all.

So the state is `(day, holding, transactions_left)`, and the sentence is: the maximum additional profit obtainable from `day` onwards, given whether I currently hold a share and how many transactions remain.

Three dimensions, and I would count the space before writing: `n × 2 × 3`. For a thousand days that is six thousand states with constant work each — trivial.

Counting it also tells me the general version is fine: at most `k` transactions is `n × 2 × (k+1)`, and since `k` above `n/2` is equivalent to unlimited, I can cap it and the state space never exceeds `n2`.

The recurrence, from the last-move question at each state. On any day I either do nothing — same state, next day — or I act. If holding, acting means selling: gain the price, stop holding, and **consume one transaction**. If not holding, acting means buying: pay the price and start holding. **I count the transaction at the sell**, and I would say that out loud because counting it at both ends is the bug that makes `k = 2` behave like `k = 1`.

Base cases: past the last day, or no transactions left, the profit from here is zero.

The alternative formulation worth mentioning is four explicit variables — the best balance after the first buy, after the first sell, after the second buy, after the second sell — updated in one pass. That is `O(1)` space and it is the version people memorise. **I would derive the general one first**, because it extends to any `k` and to the cooldown and transaction-fee variants without rethinking, whereas the four variables do not.

And the cooldown variant is exactly a fourth dimension — or rather a third state value rather than a boolean: instead of holding or not, the modes become holding, free-to-buy, and resting. **That is a mode dimension growing from two values to three**, which is the smallest possible change, and it is a good example of why getting the state framework right is worth more than memorising any individual solution.”

9 Recall card

The state is what identifies a subproblem, and the test is one sentence: *given only the state, can I decide what happens next without knowing anything else about how I got here?* If no, whatever is missing is the next dimension.

An incomplete state fails silently — two different situations share a cell, and the answer is not approximate, it is the answer to a different question.

Five things that usually need to be in the state: a **resource** being consumed (capacity, money), a **count** of something limited (transactions, stops), a **mode** you are in (holding, on cooldown), a position in a **second sequence**, and a **set already used** (bitmask, only for $n \lesssim 22$).

Incomplete and awkward are different. Incomplete needs another dimension; awkward needs the *meaning* changed — “best subarray **ending exactly at** i ” instead of “using elements up to i ”. **Redefining is often better than extending**, and adding a dimension when redefining would do turns $O(n)$ into $O(n^2)$.

Count the state space before writing. Adding a dimension **multiplies**; the practical ceiling is $\sim 10^7$ states in Python; and `dp[i][capacity]` is **pseudo-polynomial** — polynomial in the *value* of w , not in its number of digits, which is why knapsack has a DP solution and is still NP-hard.

1 What this is, and why they ask it

Paste some text, get a link, send the link to somebody. They open it and see the text. Optionally it expires, optionally it is private, and optionally it is syntax-highlighted.

It looks like [yesterday's URL shortener](#) and it is a different problem in one specific way: **the thing being stored is large**. A shortened URL is two hundred bytes and lives happily in a database row. A paste can be a megabyte of log output, and that changes where the content goes, what the database holds, how the read path works, and what the bill looks like.

They ask it because it is the smallest design where “**where do the bytes actually live**” is the central question rather than a detail, and because it has a clean, quantifiable answer: [day 134](#)'s blob-versus-database argument, applied.

It also has a second interesting property: **the read pattern is extremely skewed and extremely time-bounded**. Almost every paste is read a handful of times in the hour after it is created and then never again — except for the one in ten thousand that gets posted somewhere popular and receives a hundred thousand requests in twenty minutes. **Designing for both of those at once is the actual work**.

By the end of this lesson you can run the script on this prompt, decide where the text lives with arithmetic, handle expiry without a scan, cope with a viral paste, and say what makes this different from a shortener.

2 The story

The noticeboard in the lobby of Mr Iyer's building is a green felt rectangle about four feet across, and he has looked after it since he retired in 2016 because nobody else would.

Anybody can put something on it. Most of what goes up is small and dull — a plumber's number, a lost cat, a request to stop parking across the gate — and most of it is read by whoever happens to walk past in the two or three days after it appears, and then by nobody, ever.

His one rule, which he prints at the top and which people mostly follow, is that everything must have a date on it. **Anything more than a month old comes down**, and he does that walk on the first Sunday of the month with a bin bag. It takes eleven

minutes.

Without the rule the board fills up in about six weeks and then nothing new can go up, which is what it was like before he took it over.

Two things about the board have surprised him.

The first is the size. Somebody put up a four-page society circular in February — the whole thing, printed out and stuck up with four pins across the middle of the board — and for eleven days there was nowhere to put anything else. It was not that the circular was unimportant. It was that the board is for short things, and one long thing on it costs the space of thirty short ones. **He now asks people to put up a note saying where the long thing can be read, and to leave the long thing somewhere else.**

The second is the crowd. In August the water was going to be cut for two days and somebody put up the notice at seven in the morning. By half past eight there were maybe thirty people in the lobby, at once, all trying to read the same piece of card, and two arguments, and somebody's child crying.

The notice itself was fine. The board was fine. **What could not cope was the lobby**, which is nine feet wide and was never designed for thirty people, and the fact that every single one of them had to come to the same physical spot to read the same forty words.

The management committee's answer, which Mr Iyer thought was obvious and took them three meetings to arrive at, was to also put a copy on each of the four floors.

3 The idea in plain English

Mr Iyer's board is a pastebin, and both of his surprises are the design.

The product is: store a blob of text, give back a link, serve it on request. Optionally with an expiry, a visibility setting, and a title.

Phase 1, requirements. In scope: create a paste, read a paste by its link, optional expiry, optional "unlisted" visibility, and a size limit. Out of scope: accounts, editing, comments, folders. Non-functional:

- **Extremely read-heavy**, like a shortener — created once, read a handful of times.
- **But the size is the difference.** A paste is kilobytes to megabytes, not bytes.
- **The read distribution has a long tail and a violent head.** Most pastes get five reads; one in ten thousand gets a hundred thousand.

- **Availability over consistency.** A slightly stale paste is fine; an unavailable one breaks a link somebody shared.

Phase 2, scale. Say a million new pastes a day, averaging 10 KB:

1,000,000 / 86,400	= ~12 writes per second
peak x5	= ~60 writes/s
reads at 10:1	= ~120 reads/s, ~600 peak
storage 1,000,000 x 10 KB	= 10 GB/day
	= 3.6 TB/year

And that is the sentence that decides everything: twelve writes a second is nothing, and 3.6 terabytes a year is not nothing. Compare with the shortener, which was forty writes a second and 600 GB a year of tiny rows. **The traffic is smaller here and the storage is six times larger,** and that inverts which part of the design is hard.

So the central decision is where the text lives, and it is [day 134](#) applied:

The database holds the metadata; object storage holds the text. The row is an id, a key, a size, a content type, a creation time, an expiry and a visibility flag — about two hundred bytes. The text is an object in S3 or equivalent.

Why, in numbers. Ten kilobytes per paste in the database means 3.6 TB of database, which costs about five times object storage per gigabyte, goes into every backup — turning a two-minute restore into an hour — and pollutes the buffer pool so that every metadata query gets slower. In object storage it is 3.6 TB at two cents a gigabyte and the database stays at 70 GB of rows, which fits in memory.

That is Mr Iyer's circular: the board is for short things, and the long thing goes somewhere else with a note saying where.

But there is a real threshold, and this is where the pastebin differs from a photo service. Most pastes are small. A one-kilobyte paste in object storage costs a round trip of twenty to a hundred milliseconds to fetch one kilobyte, which is absurd — the metadata query was faster and returned a similar amount of data.

So the honest design is a threshold: text under a few kilobytes goes in the database row, and anything larger goes to object storage with the key in the row. **One extra column and one branch on the read path,** and it removes a network round trip for the majority of pastes while keeping the storage sane for the large ones.

Phase 3, the API.

POST /pastes	{content, expires_in?, visibility?, syntax?}	→ {id, url}
GET /{id}		→ the paste (HTML or raw)
GET /{id}/raw		→ text/plain

And the id is the same problem as yesterday's short code — a counter in base 62 with block allocation and a scrambling step — with one difference. **Unlisted pastes rely on the id being unguessable**, so the scrambling is not merely tidy here, it is the entire access control. **Which means it should be said plainly that it is weak access control**, and anything genuinely private needs authentication.

Phase 4, the shape.

```
write: client → API → (small? row : object storage) → insert metadata → return id
read:  client → CDN → API → cache → (row or object storage) → render
```

Phase 5, and there are two things worth going deep on.

Expiry, because doing it naively is a scan. `DELETE FROM pastes WHERE expires_at < now()` over a table with hundreds of millions of rows, every few minutes, is a large repeated scan and it competes with the read path.

The good answers, in order: partition the table by expiry date, so expiring a day is dropping a partition — instant, no scan, no vacuum. **Or push it to the storage layer**: object storage lifecycle rules delete objects by age without any job of yours running, and Redis or DynamoDB TTLs do the same. **And filter on read regardless**, because a paste is logically expired the moment its time passes, whether or not anything has deleted it yet — so the read path checks `expires_at` and returns 410 Gone, and the deletion is a storage cleanup rather than a correctness mechanism.

And the viral paste, which is Mr Iyer's lobby. One paste in ten thousand gets a hundred thousand requests in twenty minutes. **That is a hot-key problem, and the answer is a CDN**, because the content is immutable once written — a paste is never edited — so it is perfectly cacheable with a long TTL and no invalidation worry.

Immutability is the property that makes this easy, and it is worth saying: because a paste never changes, every layer can cache it indefinitely, and the only reason to invalidate is deletion or expiry.

Phase 6. The bottleneck is the read path for hot pastes, solved by the CDN. What is genuinely hard and was left out: **abuse**, again — a pastebin is used to publish stolen credentials and malware, and every real one needs scanning, rate limiting and a take-down path.

4 The picture

Where the bytes go, and why:

PASTE, 10 KB average

IN THE DATABASE

pastes

```
+-----+-----+
| id | content (10 KB) |
+-----+-----+
| id | content (900 KB) |
+-----+-----+
```

3.6 TB/year in the database

→ every backup carries it

→ buffer pool polluted

→ \$0.115/GB

SPLIT

pastes

```
+-----+-----+
| id | key |
+-----+-----+
| id | key |
+-----+-----+
```

objects

```
+-----+
| key → 10 KB |
+-----+
| key → 900 KB |
+-----+
```

70 GB/year in the database

→ restore in minutes

→ metadata fits in memory

→ \$0.023/GB for the bulk

What to notice. The right-hand database is fifty times smaller and holds exactly the fields that get queried. **Nothing queries the content**, so nothing is lost by moving it.

The threshold, which is the refinement:

content size < ~4 KB

→ store INLINE in the row

one database read, no second round trip

content size ≥ ~4 KB

→ store in object storage, key in the row

one database read + one object fetch

a 1 KB paste from object storage:

DB read for metadata ~1 ms

object fetch ~30 ms

→ 30x the latency for no benefit

← to move 1 KB

The read path, with both branches:

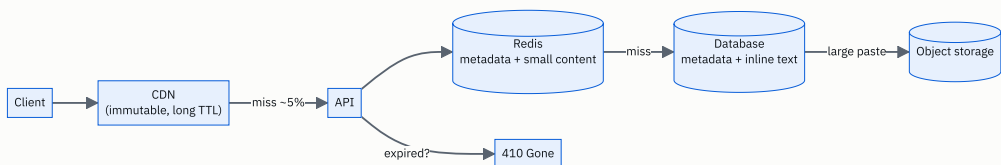


FIGURE 147.1

What to notice. The expiry check happens at the API on every read, **before** anything is served, because a paste is logically gone the moment its time passes — the deletion job is housekeeping, not the mechanism.

Expiry without a scan:

NAIVE

```
DELETE FROM pastes
WHERE expires_at < now()

→ scans, writes dead tuples,
  autovacuum has to reclaim them
→ on a hot table at volume,
  this alone can be unsustainable
```

PARTITIONED BY EXPIRY DAY

```
pastes_2026_03_01
pastes_2026_03_02
pastes_2026_03_03
...

expiring a day:
DROP TABLE pastes_2026_03_01
→ instant, no scan, no vacuum
```

AND, regardless: object storage lifecycle rules delete the blobs by age with no job of yours running at all

The traffic distribution, which is why a CDN is not optional:

1,000,000 pastes/day

999,900 of them: ~5 reads each = ~5,000,000 reads
100 of them: ~1,000 reads each = ~100,000 reads
1 of them: 100,000 reads in 20 minutes

the ONE is 83 requests/second on a single key
→ the average load says 120 reads/s
→ the actual load is one key doing most of it, in bursts

immutable content → a CDN absorbs it entirely

5 How it actually works

The schema

```
CREATE TABLE pastes (
  id          VARCHAR(10) PRIMARY KEY,
  content     TEXT,          -- inline, when small
  object_key  TEXT,          -- set instead, when large
  size_bytes  INTEGER NOT NULL,
  syntax      VARCHAR(20),
  visibility  VARCHAR(10) NOT NULL DEFAULT 'unlisted',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at  TIMESTAMPTZ,
  is_removed  BOOLEAN NOT NULL DEFAULT false,
  CHECK ((content IS NULL) ≠ (object_key IS NULL)) -- exactly one of the two
);
```

The **CHECK constraint is worth having**, because “exactly one of these two columns is set” is a rule the application will otherwise break under some code path, and the failure is a paste that renders as blank.

`is_removed` rather than deleting, so a takedown returns a clear 410 and the id is never reissued.

The write path

```
INLINE_LIMIT = 4096          # bytes

def create_paste(content: str, expires_in=None, syntax=None) → str:
    data = content.encode()
    if len(data) > MAX_SIZE:           # 1 MB, say
        raise TooLarge()

    paste_id = code_generator.next_code()    # base-62 counter, scrambled

    if len(data) ≤ INLINE_LIMIT:
        db.insert(id=paste_id, content=content, size_bytes=len(data), ...)
    else:
        key = f"pastes/{paste_id[:2]}/{paste_id}"
        object_store.put(key, data, content_type="text/plain")
        db.insert(id=paste_id, object_key=key, size_bytes=len(data), ...)

    return paste_id
```

Note the object key includes a prefix from the id — `pastes/aK/aK9x2b`. That is a habit worth keeping even though modern object stores no longer need it for performance: it keeps a listing browsable, and it makes prefix-based operations possible.

And the ordering matters, as on [day 133](#): the object is written before the row. Here that is the right way round, unlike the photo case, because the client is sending the content in the same request — so an object with no row is an orphan cleaned up by a nightly job, whereas a row with no object is a paste that renders as an error. **Whichever failure you prefer, pick it deliberately.**

The read path

```
def get_paste(paste_id: str):
    meta = cache.get(f"paste:{paste_id}")
    if meta is None:
        meta = db.fetch_one("SELECT * FROM pastes WHERE id = %s", paste_id)
        if meta is None:
            return http_404()
        cache.set(f"paste:{paste_id}", meta, ttl=3600)

    if meta.is_removed:
        return http_410("removed")
    if meta.expires_at and meta.expires_at < now():
        return http_410("expired")    # logically gone, whether deleted or not

    if meta.content is not None:
        body = meta.content            # inline: already in hand
    else:
        body = object_store.get(meta.object_key)    # large: one more fetch

    return http_200(body, cache_control="public, max-age=31536000, immutable")
```

`immutable` in the `Cache-Control` header is the important line. A paste never changes, so browsers and CDNs can hold it forever and never revalidate. **That single header is what absorbs a viral paste**, and it is free.

But it interacts with deletion, and that has to be stated: once a CDN has cached a paste for a year, a takedown requires an explicit invalidation. So `max-age` for public pastes is long, and for anything that might be removed it is shorter — or every removal issues a CDN purge, which is a real operation with a real latency.

Expiry, three mechanisms and you want all three

One: check on read. The paste is gone the moment its time passes, regardless of whether anything has deleted it. **This is the correctness mechanism** and it is one comparison.

Two: let the storage expire it. Object storage lifecycle rules delete objects older than `n` days with no job of yours. Redis and DynamoDB have TTLs. **Free, and nothing of yours runs.**

The complication is that expiry is per-paste and lifecycle rules are per-prefix, so the usual arrangement is prefixes by retention class:

```
pastes/1day/...    lifecycle: delete after 1 day
pastes/1week/...   lifecycle: delete after 7 days
pastes/forever/... no rule
```

Three: partition the database by expiry. Then expiring a day is `DROP TABLE`, which is instant and reclaims the space immediately — against a `DELETE` that scans, writes dead tuples and hands the problem to autovacuum on a hot table.

A `DELETE FROM ... WHERE expires_at < now()` job is the answer people give and it is the one that does not scale, and being able to say why is the point of the question.

The viral paste

```
one paste, 100,000 requests in 20 minutes = ~83 requests/second on ONE key
```

Three layers absorb it, and the first does almost all of the work:

- **The CDN**, because the content is immutable. A 95%+ hit rate is realistic and the origin sees a handful of requests.
- **The application cache**, so origin misses do not reach the database.
- **And the database is a primary-key lookup**, which is the cheapest thing it does.

The one that would not cope is object storage at high request rates for a large paste, which is why caching the *content* — not just the metadata — for large hot pastes is worth doing. **A one-megabyte paste served a hundred thousand times is a hundred gigabytes of egress**, and at nine cents a gigabyte that is nine dollars for one paste, which is fine once and is not fine if it is a pattern.

Rendering and syntax highlighting

Do it once, at write time, not per read. Highlighting a large file is CPU-heavy, and doing it on every request turns a cache-friendly static response into a computation.

Or do it in the browser, which is what most pastebins actually do — ship the raw text and a small highlighting library, and the server never renders anything. **That makes the response identical for every viewer, which makes it perfectly cacheable**, and it moves the CPU to the client. It is the better answer and it is worth saying why: cacheability, not CPU cost.

Abuse, which is the requirement people forget

A pastebin is one of the standard tools for publishing stolen credentials, malware and personal data.

- **Rate limit creation** by IP, hard.
- **Scan content on write** for credential patterns — API keys, private keys, database connection strings — and either block or flag. Several real services do exactly this and notify the affected provider.
- **A takedown path** and `is_removed` rather than deletion.
- **A size limit**, or it becomes free file hosting.
- **And CAPTCHA or an account requirement above some rate**, because unauthenticated bulk creation is what makes it attractive.

Naming this unprompted is a strong signal, because it is the difference between designing a feature and designing a service.

6 The numbers

The sizing, in full:

new pastes	1,000,000 / day	
	1,000,000 / 86,400	= ~12 writes/s
peak x5		= ~60 writes/s
reads at 10:1		= ~120 reads/s, ~600 peak
average size	10 KB	
storage per day	1,000,000 x 10 KB	= 10 GB/day
per year		= 3.6 TB/year
after 5 years		= 18 TB

Compare with the shortener:

	shortener	pastebin
writes/s	40	12
storage/year	600 GB	3,600 GB
row size	500 B	10,000 B

Fewer writes, six times the storage — which is exactly why the design differs, and it is one line of comparison worth making explicitly.

Where the storage decision comes from:

3.6 TB/year in the database		
storage cost	3,600 GB x \$0.115/GB	= \$414/month, and growing
backup	3.6 TB nightly	→ hours, not minutes
restore at 200 MB/s		= 5 hours
buffer pool	polluted by 10 KB rows	→ metadata queries slow down
3.6 TB/year in object storage		
storage cost	3,600 GB x \$0.023/GB	= \$83/month
database	1,000,000 x 200 B/day	= 200 MB/day = 70 GB/year
restore of 70 GB at 200 MB/s		= 6 minutes

Five times the storage cost, and a restore that goes from six minutes to five hours — and the second number is the one that matters during an incident.

The inline threshold, justified:

distribution of paste sizes (typical)	
50% under 1 KB	
80% under 4 KB	
99% under 100 KB	
1% up to 1 MB	
with a 4 KB inline threshold:	
80% of reads	one database query, ~1 ms
20% of reads	one query + one object fetch, ~30 ms
with everything in object storage:	
100% of reads	~30 ms, including 1 KB pastes

The threshold removes a thirty-millisecond round trip for eighty percent of reads and keeps the database at:

```
80% x 1,000,000 x ~2 KB average inline    = 1.6 GB/day = 580 GB/year
```

Which is larger than metadata alone and far smaller than everything, and that is the trade.

Expiry, and why the naive version fails:

```
500,000,000 rows after ~18 months
DELETE FROM pastes WHERE expires_at < now()    run every 5 minutes
    index scan on expires_at                    finds, say, 3,500 rows
    writes 3,500 dead tuples
    autovacuum must reclaim them, on a hot table, continuously

partition by expiry day:
    DROP TABLE pastes_2026_03_01              instant, space reclaimed immediately
```

The viral paste:

```
100,000 requests in 20 minutes on one key    = ~83 requests/second
CDN hit rate on immutable content              ~99%
origin sees                                   ~1 request/second

egress if it is a 1 MB paste
    100,000 x 1 MB                            = 100 GB
    at $0.09/GB from origin                     = $9
    through a CDN at ~$0.02/GB                  = $2, and the origin serves ~1 GB
```

Cache sizing:

```
hot pastes (last hour, plus the viral few)    ~100,000 entries
average cached size (metadata + small body)    ~3 KB
                                                = 300 MB
```

Trivial, which is the point — the working set is small because the access pattern is time-bounded.

Id space, same arithmetic as the shortener:

```
62^8 = 218,340,105,584,896    at 1M/day → ~600,000 years
62^7 = 3,521,614,606,208      at 1M/day → ~9,600 years
```

Seven or eight characters, and eight is common for pastebins precisely because unlisted pastes rely on the id being hard to guess — **more length is cheap and it raises the cost of enumeration.**

7 The trade-offs

Content in object storage against content in the database. The split keeps the database small, fast to back up and fast to restore, and cuts storage cost fivefold — at the cost of a second round trip on every large read, two systems that can disagree, and a reconciliation job to find orphans and broken links. **The inline threshold buys back most of the latency for most pastes** and adds a branch and a `CHECK` constraint.

Expiry as a delete job against expiry as a storage policy. A job is explicit and visible and becomes a continuous scan on a large hot table. Lifecycle rules and partition drops are free and coarse — per-prefix or per-day, not per-paste. **The workable answer is both, plus the read-time check that makes the whole thing correct regardless of when deletion actually happens.**

A long cache lifetime against removability. Immutable content with a year-long `max-age` is what makes a viral paste cost nothing — and it means a takedown needs an explicit CDN purge, and any copy already in a browser is unreachable. **For a service whose main abuse response is removal, that is a real cost**, and the usual compromise is a shorter TTL for anything not yet through content scanning.

Unguessable ids as access control. “Unlisted” means “the id is hard to guess”, which is not authentication. It is fine for sharing a stack trace with a colleague and it is not fine for anything sensitive — **and the honest design says so in the interface**, because users will otherwise paste credentials into it and assume privacy. Real privacy needs accounts and authorisation, which is a much larger product.

Server-side rendering against client-side. Rendering syntax highlighting on the server costs CPU per request and produces responses that vary by options, which hurts cacheability. Doing it in the browser makes every response byte-identical and therefore perfectly cacheable, and costs the client a library download. **The cacheability argument is the real one**, not the CPU.

And the size limit is a product decision with a large blast radius. Raise it and the service becomes free file hosting, with the storage bill and the abuse surface that implies. **A megabyte is generous for text and useless for anything else**, which is exactly why it is the common choice.

When would I build it differently? If pastes were editable, immutability goes and with it the aggressive caching — the whole design becomes more like a document store. If most pastes were tiny, everything inline and no object storage at all. And if it were internal with a hundred users, one Postgres table and a cron job, and none of this is worth building.

8 In the interview

How it gets asked

- “*Design pastebin.*” — the standard prompt.
- “*Where does the text actually go?*” — the central question.
- “*What happens when a paste goes viral?*” — the hot-key question.
- “*How do you expire pastes?*” — where the naive answer is a scan.
- “*How is this different from a URL shortener?*” — the comparison question.
- “*Someone is pasting stolen passwords.*” — abuse.

The first ninety seconds

“This looks like a URL shortener and differs in one way that changes the design: **the stored thing is large.**

Scope: create a paste, read it by link, optional expiry, optional unlisted visibility, a size limit. Out: accounts, editing, comments. **Non-functional:** read-heavy, availability over consistency, and — the one that matters — **paste sizes are kilobytes to megabytes, and the read distribution has a violent head:** most pastes get a handful of reads and one in ten thousand gets a hundred thousand in twenty minutes.

Sizing. A million pastes a day at 10 KB average is about twelve writes a second — nothing — and ten gigabytes a day, 3.6 terabytes a year. **Compare with the shortener: fewer writes and six times the storage.** So the traffic is easy and the storage is the problem, which is the opposite of what people expect.

Which makes the central decision: where do the bytes live. Metadata in the database, content in object storage. In numbers: 3.6 terabytes in the database is five times the storage cost, it goes into every backup so a restore goes from six minutes to five hours, and 10 KB rows pollute the buffer pool so every metadata query slows down. In object storage the database stays at seventy gigabytes a year and fits in memory.

With one refinement I would raise unprompted: most pastes are small, and fetching a one-kilobyte paste from object storage costs a thirty-millisecond round trip to move one kilobyte. **So content under about four kilobytes goes inline in the row.** That covers roughly eighty percent of pastes with a single database read and keeps the large ones out of the database entirely — one extra column and one branch.

And the property that makes the rest easy: a paste is immutable. It is never edited. So every layer can cache it forever with `Cache-Control: immutable`, and a viral paste is absorbed by the CDN with no invalidation to worry about.

Shall I go into expiry, or the viral case?”

The follow-ups

“How is this different from a URL shortener?”

“Three ways, and the first causes the other two.

The payload is four orders of magnitude larger. Two hundred bytes against ten kilobytes. That moves the content out of the database, introduces a second storage system, and creates the orphan-and-broken-link problem that a shortener does not have.

The read pattern is more extreme. A shortened link gets a fairly even spread of clicks over its life. A paste is read five times in the first hour and then never — except the rare one that is posted somewhere popular. **So the working set is small and time-bounded, and the tail is essentially dead,** which makes caching unusually effective and makes expiry genuinely valuable rather than cosmetic.

And expiry is a first-class feature rather than an afterthought. A shortener’s links are usually permanent; a pastebin’s are usually not, and ‘delete a hundred million rows a day without a scan’ is a real design problem that a shortener never has.

What is the same: the id generation — a base-62 counter with block allocation and a scrambling step — the read-heavy shape, the cache in front of a primary-key lookup, and the abuse problem. **So I would reuse the code-generation design wholesale and say so,** rather than deriving it again.”

“A paste goes viral. What happens?”

“It is a hot-key problem, and it is unusually easy here because **the content is immutable**.

The numbers: a hundred thousand requests in twenty minutes is about eighty-three requests a second on one key. That is not a lot in absolute terms, but it is all on a single row and a single object, so nothing spreads it.

The CDN absorbs essentially all of it. Because the paste never changes, I serve `Cache-Control: public, max-age=31536000, immutable`, and every edge caches it for a year. At a ninety-nine percent hit rate the origin sees about one request a second.

Below that, the application cache holds the metadata and, for large hot pastes, the content too — because otherwise every origin miss is an object-storage fetch, and object storage is the layer that would actually struggle at high request rates.

And the number worth quoting is the egress. A one-megabyte paste served a hundred thousand times is a hundred gigabytes. At nine cents a gigabyte from origin that is nine dollars for one paste — fine once, and not fine as a pattern. Through a CDN the origin serves about a gigabyte of it.

The cost of the aggressive caching is that removal needs an explicit purge, and anything already in a browser cannot be recalled. **For a service whose abuse response is takedown, that is a real trade**, and I would shorten the TTL for pastes that have not yet been through content scanning.”

“How do you expire pastes without a nightly scan of a hundred million rows?”

“Three mechanisms, and the naive one is the one to reject first.

The naive version is `DELETE FROM pastes WHERE expires_at < now()` on a schedule. On a table with hundreds of millions of rows that scans an index, writes dead tuples, and hands the reclamation to autovacuum on a table that is also serving reads. **At volume it competes with the read path continuously.**

First, and most importantly: check expiry on read. A paste is logically gone the moment its time passes, whether or not anything has deleted it. One comparison, and the 410 is returned. **This makes correctness independent of when deletion happens**, which is what lets the deletion be lazy and coarse.

Second: partition the database table by expiry day. Then expiring a day is `DROP TABLE`, which is instant and reclaims the space immediately, with no scan and no vacuum.

Third: let object storage do the blobs. Lifecycle rules delete objects by age with no job of mine running. The complication is that lifecycle rules are per-prefix and expiry is per-paste, so the arrangement is prefixes by retention class — `pastes/1day/`, `pastes/1week/`, `pastes/forever/` — and a paste goes into the prefix matching its requested lifetime.

The result is that nothing of mine scans anything, and the only code involved is one comparison on the read path.”

“Someone is pasting stolen credentials.”

“That is what pastebins are actually used for at scale, and a design that does not mention it is incomplete.

Scan on write. Regular expressions for the obvious credential shapes — AWS keys, private key headers, database connection strings, bearer tokens. On a match, either block the paste or flag it for review, and several real services go further and **notify the provider whose keys were leaked**, which is a genuinely good use of the position.

Rate limit creation by IP, hard, and require a CAPTCHA or an account above a low threshold. Unauthenticated bulk creation is what makes it attractive as a dumping ground.

A size limit, or it becomes free file hosting with a completely different abuse profile.

A takedown path with `is_removed` rather than deletion, so the id is never reissued and the response is a clear 410 rather than a 404 that looks like a bug.

And the caching interaction, which is the technically interesting part: aggressive immutable caching is what makes the viral case cheap, and it directly fights takedown. So I would use a **shorter TTL until a paste has passed content scanning**, and a longer one afterwards — which costs a little cache efficiency on brand-new pastes and preserves the ability to remove something quickly.

And I would be explicit in the product that ‘unlisted’ is not private. It means the id is hard to guess. Users will paste credentials into it believing otherwise, and saying so plainly in the interface is part of the design.”

The model answer

“Design pastebin. What happens when a paste goes viral?” — the deep dive, phase 5.

“Let me take the viral case properly, because it is where the design’s central property pays off.

The situation: one paste, a hundred thousand requests in twenty minutes, so about eighty-three a second concentrated on a single id. Everything else on the system is unaffected — twelve writes a second and a hundred and twenty reads a second spread across a million ids — so this is purely a hot-key problem.

The property that makes it easy is immutability. A paste is written once and never edited. That means every caching layer can hold it indefinitely without any invalidation logic, which is a luxury most systems do not have.

Layer one: the CDN. `Cache-Control: public, max-age=31536000, immutable`. Every edge holds it, and because the requests come from many places, many edges each fetch it once. At a ninety-nine percent hit rate the origin sees about one request a second — the problem is gone before it reaches my infrastructure.

Layer two: the application cache, for the origin misses. For a small paste I cache the whole thing — metadata and content together — so a hit is one Redis read. **For a large paste I would cache the content too, and that is the decision worth explaining:** without it, every origin miss is an object-storage fetch, and object storage is the layer least suited to a high request rate on one key. A megabyte in Redis for a paste doing a hundred requests a second is an obviously good trade for the duration of the burst.

Layer three: the database, which is a primary-key lookup on a row that is certainly in its buffer pool by the third request. It was never going to be the problem.

The cost, and I would put a number on it. If it is a one-megabyte paste, a hundred thousand reads is a hundred gigabytes of transfer. From origin at nine cents a gigabyte that is nine dollars for one paste; through a CDN the origin serves about a gigabyte and the CDN carries the rest at a lower rate. **Nine dollars is fine once and is a business problem if it is a daily occurrence,** which is an argument for a size limit and for rate limiting reads per id if it ever becomes one.

What I would monitor: requests per id, so a spike is visible; CDN hit rate, because a drop means something is defeating caching — usually a query parameter varying, or a `Vary` header I did not intend; and origin egress.

And the failure mode I would name: if the paste is removed while it is viral, the CDN is holding it for a year and a purge across every edge takes minutes. **So the removal path must issue the purge, and I would keep the TTL shorter for**

pastes that have not yet been scanned — accepting a slightly worse hit rate in the first minutes of a paste’s life in exchange for being able to take it down quickly. That is the one place where the aggressive caching and the abuse response genuinely conflict, and I would rather resolve it deliberately than discover it during an incident.”

9 Recall card

Same shape as a shortener, one difference: the payload is 10 KB, not 200 bytes. Fewer writes (~12/s) and **six times the storage** (3.6 TB/year) — so the storage decision is the design, not the traffic.

Metadata in the database, content in object storage — $5\times$ cheaper per GB, and a restore of 6 minutes instead of 5 hours. **With an inline threshold at ~4 KB**, because fetching a 1 KB paste from object storage is a 30 ms round trip to move 1 KB, and it covers ~80% of pastes with one database read.

A paste is immutable, so `Cache-Control: immutable` with a long `max-age` — and that single header absorbs a viral paste (100,000 reads in 20 minutes $\approx$ 83 req/s on one key) at a ~99% CDN hit rate.

Expiry: check on read (that is the correctness mechanism), **partition by expiry day** so removal is a `DROP TABLE`, and **let object-storage lifecycle rules delete the blobs** via retention-class prefixes. A `DELETE ... WHERE expires_at < now()` job is the answer that does not scale.

“Unlisted” means the id is hard to guess, not private — say so in the product. And abuse is the forgotten requirement: **scan for credentials on write**, rate-limit creation, cap the size, and `is_removed` rather than delete — which conflicts with long cache TTls, so shorten them until a paste is scanned.

Practice — Finding the state: the hardest part of DP

DSA topic: Finding the state: the hardest part of DP **System design topic:** Design a pastebin

Code these, in this order

One rule for the whole set: **before writing anything, apply the test out loud.** “Given only the state, can I decide what happens next without knowing anything else about how I got here?” Then write the state as a full sentence, then count the state space, then code. Three steps, about ninety seconds, and they save the twenty minutes that a wrong state costs.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	0/1 Knapsack	classic — write it yourself	A resource dimension, and why one index fails.
2	Best Time to Buy and Sell Stock IV	LeetCode 188 (Hard)	A mode <i>and</i> a count, and where the count is consumed.
3	Best Time to Buy and Sell with Cooldown	LeetCode 309 (Medium)	A mode with three values instead of two.
4	Edit Distance	LeetCode 72 (Medium)	Two positions, and prefix-length versus index conventions.
5	Cheapest Flights Within K Stops	LeetCode 787 (Medium)	A count dimension on a graph.
6	Partition Equal Subset Sum	LeetCode 416 (Medium)	A resource dimension, and the pseudo-polynomial point.

On problem 1, write the broken version first

Implement it with `dp[i]` only and run it on three items totalling more than the capacity. Record what it returns and compare with the sum of all values. Then apply the test and say, in one sentence, what was missing.

On problem 2, break the transaction counting

Decrement `transactions_left` on both the buy and the sell. Run with `k = 2` on `[7,1,5,3,6,4]` and record the answer. Then fix it and record the correct one. Write one sentence on why the wrong answer is plausible.

Then add the observation that $k > n/2$ is the same as unlimited, and cap it. Run with $k = 1,000,000$ and confirm it is instant rather than a memory error.

On problem 4, decide the convention before coding

Write down whether $dp[i][j]$ means “the first i characters” or “up to index i ”. Then write the base cases from that sentence. Then write the table dimensions. Doing it in that order removes every off-by-one in this problem.

On problem 6, compute the state space first

For `nums` summing to S , the state is $(\text{index}, \text{remaining_target})$ and the space is $n \times S/2$. Compute it for $n = 200$ with values up to 100, and again for values up to 10^9 . Say which one is viable and what that tells you about the complexity class.

Then the state-space audit

For each of the six problems, write down: the state as a sentence, the number of states as a formula, the work per state, and the total. Six rows. **Any total above about ten million means the state is wrong or the problem is not DP**, and this table is how you find that out in ten seconds.

Then the redefinition drill

Take maximum subarray and write both states: “the best using elements 0..i” and “the best ending exactly at i”. For the first, try to write the recurrence and record where you get stuck. Then write the second and note that the dimension count did not change.

The test drill

1. State the test in one sentence.
2. Apply it to climbing stairs, knapsack, stock-with-one-transaction and edit distance.
3. For each failure, name the missing dimension.
4. Say what an incomplete state does, and why nothing reports it.

The five-dimensions drill

Name each kind, give a problem, and say what the tell is:

1. A resource being consumed.
2. A count of something limited.
3. A mode you are in.
4. A position in a second sequence.

5. A set already used.

Then say which is the most expensive and at what `n` it stops being viable.

The incomplete-versus-awkward drill

1. Define both, in one sentence each.
2. Give an example of each.
3. Say what the fix is for each.
4. Say what it costs to treat an awkward state as an incomplete one.

The counting drill

1. Give the state-space formula for five different shapes.
2. Compute each at realistic sizes and say which are viable.
3. Say what the practical ceiling is in Python.
4. Explain what pseudo-polynomial means, using knapsack.
5. Say why that does not contradict knapsack being NP-hard.

The break-it drill

Trigger each and record the exact output or error:

1. Knapsack with `dp[i]` only.
2. A hand-rolled cache keyed on fewer arguments than the function takes.
3. `@lru_cache` on a function taking a list.
4. A dimension that is derivable from the others.
5. A bitmask state at `n = 40`.
6. Transaction counting on both the buy and the sell.
7. `left = 0` as the base case when `left` means “used” rather than “remaining”.

Five of the seven give no error at all. Name them.

The requirements drill

1. Give the in-scope and out-of-scope lists.
2. Give the four non-functional requirements and say which one differs from a shortener.
3. Describe the read distribution in one sentence.

The sizing drill

1. Do the full estimate from 1M pastes a day at 10 KB.
2. Compare writes and storage against the URL shortener, both directions.
3. Say the sentence that follows from the comparison.

The storage drill

1. Give the numbers for content-in-database versus content-in-object-storage.
2. Say which of those numbers matters most during an incident, and why.
3. Explain the inline threshold and what it buys, with the latency figures.
4. Give the size distribution and the fraction the threshold covers.
5. Say what the `CHECK` constraint is for.

The immutability drill

1. Say what property makes a paste perfectly cacheable.
2. Give the exact header and what each part does.
3. Say what that header costs you.
4. Say how you would resolve that conflict.

The expiry drill

1. Say why a `DELETE ... WHERE expires_at < now()` job fails at scale.
2. Give the three mechanisms and say which one is the correctness mechanism.
3. Explain partition-by-expiry-day and what `DROP TABLE` buys.
4. Say how per-paste expiry maps onto per-prefix lifecycle rules.

The viral drill

1. Compute requests per second for 100,000 reads in 20 minutes.
2. Give the three absorbing layers and what each handles.
3. Say which layer would actually struggle, and why.
4. Compute the egress for a 1 MB paste read 100,000 times, origin and CDN.
5. Say what you would monitor, and what a falling CDN hit rate usually means.

The abuse drill

1. Name the five defences.
2. Say what content scanning looks for and what a good service does beyond blocking.
3. Say why `is_removed` rather than delete.
4. Say why a size limit is a product decision with a large blast radius.

5. State the conflict between caching and takedown, and your resolution.

The failure drill

For each, say what happens and what you would build:

1. A 1 KB paste stored in object storage and read once.
2. 3.6 TB of paste content in the database, during a restore.
3. A `DELETE` expiry job on a 500-million-row table.
4. A 1 MB paste read 100,000 times from origin.
5. A paste removed after a CDN has cached it for a year.
6. A user pastes an AWS key into an “unlisted” paste.
7. The object write succeeds and the row insert fails.

Two of the seven are cost problems rather than correctness problems. Name them.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *What does `dp[i]` represent in your solution?* The sentence, the test, what an incomplete state does and why it is silent, the five kinds of dimension, and the incomplete-versus-awkward distinction.
 2. *Now there is a limit of at most k transactions.* Why `(day, holding)` is no longer enough, the third dimension, where the count is consumed and why that must be stated, the state-space arithmetic, and the `k > n/2` cap.
 3. *Design pastebin. What happens when a paste goes viral?* The sizing ending in “fewer writes, six times the storage”, the split with the inline threshold, immutability as the enabling property, the three caching layers with numbers, and the takedown conflict.
-

Before you move on

- ☐ I apply the test before writing any DP.
- ☐ I write the state as a full sentence.
- ☐ I count the state space before coding.
- ☐ I know an incomplete state fails silently and why.
- ☐ I can name the five kinds of dimension with a problem for each.

- [] I know which dimension is expensive and at what `n`.
- [] I can distinguish incomplete from awkward, and know the fix for each.
- [] I know redefining is often better than extending.
- [] I know adding a dimension multiplies the state space.
- [] I know the practical ceiling is $\sim 10^7$ states in Python.
- [] I can explain pseudo-polynomial using knapsack.
- [] I know why the cache key must be the complete state.
- [] I have seen the broken knapsack return the sum of everything.
- [] I know where a transaction count is consumed, and that it must be one place.
- [] I know `k > n/2` is the same as unlimited.
- [] I can size a pastebin and compare it with a shortener both ways.
- [] I can give the database-versus-object-storage numbers.
- [] I know which of those numbers matters during an incident.
- [] I can justify the inline threshold with latency figures.
- [] I know immutability is what makes the caching work.
- [] I can give the cache header and what it costs.
- [] I know why a `DELETE` expiry job fails, and the three mechanisms that replace it.
- [] I know the read-time check is the correctness mechanism.
- [] I can compute the viral case and name the layer that would struggle.
- [] I can compute egress for a large hot paste both ways.
- [] I can name five abuse defences.
- [] I can state the caching-versus-takedown conflict and resolve it.
- [] I know “unlisted” is not private and would say so in the product.
- [] I answered all three questions above out loud.

The 0/1 knapsack

DATA STRUCTURES AND ALGORITHMS

The 0/1 knapsack

SYSTEM DESIGN

Design a rate limiter, at system scale

1 What this is, and why they ask it

You have a set of items, each with a weight and a value, and a bag that holds a fixed total weight. Take each item at most once. Maximise the value you carry.

That is the 0/1 knapsack, and it is the most important single problem in dynamic programming — not because it appears often by name, but because **an enormous number of problems are it in disguise**. Can this array be split into two equal halves? Can I make exactly this total from these numbers? What is the closest I can get to a target? All of them are knapsack with the value and the weight set to the same thing.

It is also the first problem in this phase where the state genuinely needs two dimensions, so it is [yesterday's lesson](#) made concrete: `dp[i]` alone is not a question, and `dp[i][remaining]` is.

And greedy fails twice, which is what makes it worth asking. Taking the most valuable item first is wrong. Taking the best value-per-kilo first is also wrong — and that second one is subtle, because it is *correct* for a slightly different problem, which is exactly why people believe it.

By the end of this lesson you can write it top-down and bottom-up, collapse it to one row and get the iteration direction right, produce both greedy counter-examples, recognise the subset-sum family, and explain why a polynomial-looking DP does not contradict the problem being NP-hard.

2 The story

Bhagwan sells at four weekly markets and everything he takes goes on the carrier of the cycle, which holds about forty kilos before the frame starts to complain.

What he has in the room at home is always more than forty kilos. Steel tumblers, plastic buckets, cheap torches, mosquito nets, a box of locks, two kinds of rope. Each one weighs what it weighs and each one makes him a different amount of money at the market.

The first year he took the expensive things. It seemed like the whole answer — you can only carry so much, so carry the valuable stuff.

That was wrong and it was wrong in an obvious way. The mosquito nets make eleven hundred rupees and weigh twenty-two kilos, so a bundle of them fills over half the carrier. In the space left he could take a torch or two. Meanwhile the locks are four hundred rupees for six kilos, the tumblers three hundred for four, and the rope two hundred and fifty for three — and those three together are thirteen kilos and nine hundred and fifty rupees, which is nearly the nets in a third of the weight.

So the second year he worked out the rupees per kilo and took things in that order, which felt much cleverer.

And that is wrong too, and it took him longer to see because it is nearly right.

The week it caught him out, the best thing per kilo was the rope at eighty-three a kilo, then the tumblers at seventy-five, then the locks at sixty-seven. He loaded rope, tumblers, locks — thirteen kilos, nine hundred and fifty rupees — and then had twenty-seven kilos of space and nothing that fitted well into it. The next best per kilo was a crate of buckets at fifty a kilo, and it weighed thirty kilos, and thirty does not fit in twenty-seven.

He went with a third of the carrier empty.

If he had left the locks behind — the worst of the three he took — he would have had thirty-three kilos free, the buckets would have gone in, and he would have made fifteen hundred instead of nine-fifty.

The thing that took him years to accept is that there is no order. There is no rule of the form “take the best X first” that works, because whether an item is worth taking depends entirely on what else fits around it, and you cannot know that by looking at the item.

What he does now, on Thursday evenings, is go through the pile item by item and ask two questions about each one: what is the best I can do with the rest of the pile if I take this, and what is the best if I leave it. It takes him a while. It has never once been wrong.

3 The idea in plain English

Bhagwan’s Thursday evening is the 0/1 knapsack, and both of his wrong years are the greedy counter-examples.

The problem. n items, each with a weight w and a value v . A capacity W . Take each item at most once — that is the “0/1”, zero or one of each. Maximise the total value with total weight at most W .

The state, applying **yesterday's test**. Try `dp[i]` = the best from items `i` onwards. "At item five, what is the best I can do?" — **that has no answer**, because it depends on how much room is left, and the room is not in the state. Two paths reach item five having taken different things and having different capacity, and they have different answers.

So the state is two-dimensional:

`dp[i][r]` = the maximum value obtainable using items `i` onwards, with `r` capacity remaining.

And the recurrence is the last-move question, which here is the choice at item `i`:

```
skip it:    dp[i+1][r]
take it:    value[i] + dp[i+1][r - weight[i]]    -- only if weight[i] ≤ r

dp[i][r] = max(skip, take)
```

Note what the new dimension does to the **take** branch: it can make it illegal, not merely worse. If the item does not fit, there is no choice to make. That is different from house robber, where both branches were always available.

Now why greedy fails, twice, and being able to give both is the point.

Greedy by value. Take the most valuable item first.

```
capacity 10
items:   A(weight 10, value 60)   B(weight 5, value 50)   C(weight 5, value 50)

greedy by value: take A           → 60
correct:         take B and C    → 100
```

Greedy by value-per-weight. Take the best ratio first.

```
capacity 10
items:   A(weight 6, value 60, ratio 10)  B(weight 5, value 40, ratio 8)  C(weight 5, value 40, ratio 8)

greedy by ratio: take A (6 kg), then neither B nor C fits (4 kg left) → 60
correct:         take B and C                                           → 80
```

And here is why the second one is so persuasive: it is correct for the *fractional knapsack*, where you may take part of an item. If you can take four-fifths of a crate of buckets, then filling the bag with the best ratio first is provably optimal, and it is a genuine greedy algorithm.

The moment items are indivisible, that proof collapses, because the last item cannot be trimmed to fit and the leftover space is wasted. That is Bhagwan's twenty-seven kilos, and it is the single most useful thing to be able to say about this problem.

So there is no ordering rule, and that is the whole reason it needs DP. Whether an item belongs in the answer depends on what fits around it, which cannot be determined by examining the item.

The bottom-up table is two-dimensional and standard:

```
dp[i][r] = the best value using the FIRST i items with capacity r
```

Note this is the other convention from the recursive version — “the first `i` items” rather than “items `i` onwards” — because it makes `dp[0][*] = 0` a natural base case, and the loops run forwards. Either works; mixing them is the source of every off-by-one in this problem, so state which one you are using.

And then the space collapse, which is where the real subtlety is.

`dp[i][r]` reads only row `i-1`, so one row suffices. But the iteration direction over the capacity decides which problem you are solving:

```
for r in range(W, weight-1, -1):    # DOWNWARDS → 0/1 knapsack
    row[r] = max(row[r], value + row[r - weight])

for r in range(weight, W+1):        # UPWARDS → UNBOUNDED knapsack
    row[r] = max(row[r], value + row[r - weight])
```

Downwards, `row[r - weight]` still holds the previous item's value, so the item is used at most once. Upwards, it may already have been updated by this same item, so the item can be used repeatedly.

Two correct programs for two different problems, differing by a range direction, and no error either way. That is the most dangerous line in dynamic programming and it is worth writing on something.

And the family, which is why this problem matters more than its own statement. Set `value = weight` and the question “what is the largest total I can reach without exceeding `W`” becomes:

- **Subset sum:** can I hit exactly `W`? Same table, booleans instead of numbers.
- **Partition equal subset sum:** can I split the array into two equal halves? That is subset sum with a target of `total / 2` — and if the total is odd, the answer is no before you start.

- **Target sum:** assign `+` or `-` to each number to reach a target. Rearranges into subset sum.
- **Last stone weight II:** minimise the difference between two groups. Subset sum for the half closest to `total / 2`.

The tell for the whole family: choose a subset, subject to a total, optimising or checking something.

Finally, the complexity point that comes up. $O(n \times W)$ looks polynomial and the problem is NP-hard, which seems contradictory. **It is not, because W is a number written in the input.** A capacity of a billion takes ten characters to write and makes the table a billion cells wide. **The table is polynomial in the value of W , not in the number of digits** — which is called **pseudo-polynomial**, and it is why this DP exists and the problem is still hard.

4 The picture

The two greedy failures, side by side:

BY VALUE		BY VALUE PER WEIGHT	
capacity 10		capacity 10	
A(10 kg, 60)		A(6 kg, 60, ratio 10)	
B(5 kg, 50)		B(5 kg, 40, ratio 8)	
C(5 kg, 50)		C(5 kg, 40, ratio 8)	
greedy: A	→ 60	greedy: A (6kg), 4 left, nothing fits	→ 60
best: B + C	→ 100	best: B + C (10 kg)	→ 80
one heavy item blocks two		the best ratio leaves a gap	
lighter ones worth more		nothing can fill	

What to notice on the right. The ratio rule is *correct* if items can be cut — take all of A and four-fifths of B for 92. **Indivisibility is what breaks it**, and that is the sentence to say.

The table, filled by hand:

items: A(w2, v3) B(w3, v4) C(w4, v5) capacity 5

dp[i][r] = the best using the FIRST i items with capacity r

	r=0	r=1	r=2	r=3	r=4	r=5	
i=0 (none)	0	0	0	0	0	0	
i=1 (A)	0	0	3	3	3	3	
i=2 (A,B)	0	0	3	4	4	7	← 7 = A + B, 5 kg
i=3 (+C)	0	0	3	4	5	7	

answer dp[3][5] = 7 (A and B: 2+3 = 5 kg, 3+4 = 7)

note dp[3][4] = 5: C alone beats A alone at capacity 4
and dp[3][5] stayed 7: C(4kg, 5) + nothing else beats A+B

What to notice. Each row reads only the row above it — `dp[i][r]` looks at `dp[i-1][r]` and `dp[i-1][r - w]`. **That is why one row suffices**, and the two cells it reads are why the direction matters.

The direction, drawn:

one row, processing item with weight 2, value 3

DOWNWARDS (0/1)

UPWARDS (unbounded)

r: 5 4 3 2
 ^

r: 2 3 4 5
 ^

row[5] = max(row[5], 3 + row[3])
 row[3] is still the PREVIOUS
 item's value

row[2] = max(row[2], 3 + row[0])
 row[0] unchanged, fine
row[4] = max(row[4], 3 + row[2])
 row[2] was JUST updated
 → the item is used TWICE

What to notice. The upward version is not broken — it correctly solves the unbounded knapsack, where each item may be taken any number of times. **Two right answers to two different questions, and one range call between them.**

The family:

0/1 KNAPSACK maximise value, weight $\leq W$

set value = weight:

SUBSET SUM can I reach exactly W?
PARTITION split into two equal halves?
TARGET SUM +/- each number to reach T
LAST STONE II minimise the difference

→ booleans
→ subset sum for total/2
→ rearranges to subset sum
→ closest reachable to total/2

all the same table.

5 The code, built step by step

Start with the state, said out loud.

“`dp[i][r]` is the maximum value obtainable from items `i` onwards with `r` capacity remaining. One index is not enough, because ‘at item five’ has no answer without knowing the room left.”

Top-down, straight from the recurrence:

```
from functools import lru_cache

def knapsack_memo(weights: list[int], values: list[int], capacity: int) → int:
    n = len(weights)

    @lru_cache(maxsize=None)
    def best(i: int, remaining: int) → int:
        if i == n:
            return 0
        skip = best(i + 1, remaining)
        if weights[i] > remaining:
            return skip # the item does not fit: no choice
        take = values[i] + best(i + 1, remaining - weights[i])
        return max(take, skip)

    return best(0, capacity)
```

Nine lines, and the `if weights[i] > remaining` is the new dimension pruning a branch. Without it, `remaining` goes negative and the whole thing is wrong.

Now bottom-up, and note the convention change. The table uses “the first `i` items” rather than “items `i` onwards”, because that makes the base row all zeros and the loops run forwards:

```
def knapsack_table(weights: list[int], values: list[int], capacity: int) → int:
    n = len(weights)
    # dp[i][r] = the best value using the FIRST i items with capacity r
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        w, v = weights[i - 1], values[i - 1]
        for r in range(capacity + 1):
            dp[i][r] = dp[i - 1][r] # skip item i
            if w ≤ r:
                dp[i][r] = max(dp[i][r], v + dp[i - 1][r - w]) # take it
    return dp[n][capacity]
```

`weights[i - 1]` because `i` counts items and indices start at zero. That off-by-one is the price of the convention and it is worth the base case being free.

Then the space collapse, and this is the line to be careful with:

```
def knapsack_one_row(weights: list[int], values: list[int], capacity: int) → int:
    row = [0] * (capacity + 1)
    for w, v in zip(weights, values):
        for r in range(capacity, w - 1, -1):          # DOWNWARDS: each item once
            row[r] = max(row[r], v + row[r - w])
    return row[capacity]
```

Read the range. Going down from `capacity` to `w`, so that when `row[r - w]` is read it has not yet been touched this round — it still holds the value from before this item was considered. **That is what makes the item usable at most once.**

And the one-character difference:

```
def unbounded_knapsack(weights, values, capacity):
    row = [0] * (capacity + 1)
    for w, v in zip(weights, values):
        for r in range(w, capacity + 1):             # UPWARDS: unlimited copies
            row[r] = max(row[r], v + row[r - w])
    return row[capacity]
```

Same three lines, opposite direction, different problem. I would write the direction as a comment every time, because six months later it reads identically.

Now the family. Subset sum is the same table with booleans:

```
def can_reach(nums: list[int], target: int) → bool:
    # reachable[r] = can some subset sum to exactly r?
    reachable = [False] * (target + 1)
    reachable[0] = True                                # the empty subset makes 0
    for number in nums:
        for r in range(target, number - 1, -1):        # downwards: each number once
            reachable[r] = reachable[r] or reachable[r - number]
    return reachable[target]
```

`reachable[0] = True` is the base case and it is the one people forget — the empty subset sums to zero, and without it every cell stays false.

And partition equal subset sum is that, with one check first:

```
def can_partition(nums: list[int]) → bool:
    total = sum(nums)
    if total % 2:
        return False
    return can_reach(nums, total // 2)                # an odd total cannot be halved
```

The odd check is not an optimisation, it is a correctness shortcut — and it is free.

And reconstructing which items were taken, which needs the full table:

```

def knapsack_with_items(weights, values, capacity):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        w, v = weights[i - 1], values[i - 1]
        for r in range(capacity + 1):
            dp[i][r] = dp[i - 1][r]
            if w ≤ r:
                dp[i][r] = max(dp[i][r], v + dp[i - 1][r - w])

    chosen, r = [], capacity
    for i in range(n, 0, -1):
        if dp[i][r] ≠ dp[i - 1][r]:
            # the value changed → item i was taken
            chosen.append(i - 1)
            r -= weights[i - 1]
    return dp[n][capacity], chosen[::-1]

```

The test `dp[i][r] ≠ dp[i-1][r]` means “including item `i` improved things”, so it was taken. Walk backwards, reduce the capacity by its weight, continue. `O(n)` for the walk, and it needs the `O(n × W)` table — so reconstruction and the one-row optimisation are mutually exclusive.

The complete solution

```
"""The 0/1 knapsack, its one-row form, the direction that changes the problem, and the family."""
from __future__ import annotations
from functools import lru_cache

def knapsack_memo(weights: list[int], values: list[int], capacity: int) → int:
    """Top-down. dp[i][r] = the best from items i.. with r capacity remaining."""
    n = len(weights)

    @lru_cache(maxsize=None)
    def best(i: int, remaining: int) → int:
        if i == n:
            return 0
        skip = best(i + 1, remaining)
        if weights[i] > remaining:
            return skip
        return max(values[i] + best(i + 1, remaining - weights[i]), skip)

    return best(0, capacity)

def knapsack_table(weights: list[int], values: list[int], capacity: int) → int:
    """Bottom-up. dp[i][r] = the best using the FIRST i items with capacity r."""
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        w, v = weights[i - 1], values[i - 1]
        for r in range(capacity + 1):
            dp[i][r] = dp[i - 1][r]
            if w ≤ r:
                dp[i][r] = max(dp[i][r], v + dp[i - 1][r - w])
    return dp[n][capacity]

def knapsack_one_row(weights: list[int], values: list[int], capacity: int) → int:
    """One row. The DOWNWARD range is what makes each item usable at most once."""
    row = [0] * (capacity + 1)
    for w, v in zip(weights, values):
        for r in range(capacity, w - 1, -1):
            # ← 0/1
            row[r] = max(row[r], v + row[r - w])
    return row[capacity]

def unbounded_knapsack(weights: list[int], values: list[int], capacity: int) → int:
    """The SAME code with the range reversed. Each item may be used any number of times."""
    row = [0] * (capacity + 1)
    for w, v in zip(weights, values):
        for r in range(w, capacity + 1):
            # ← unbounded
            row[r] = max(row[r], v + row[r - w])
    return row[capacity]

def knapsack_with_items(weights, values, capacity) → tuple[int, list[int]]:
    """The value AND the items. Needs the full table."""
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        w, v = weights[i - 1], values[i - 1]
        for r in range(capacity + 1):
            dp[i][r] = dp[i - 1][r]
            if w ≤ r:
                dp[i][r] = max(dp[i][r], v + dp[i - 1][r - w])
    chosen, r = [], capacity
    for i in range(n, 0, -1):
        if dp[i][r] ≠ dp[i - 1][r]:
            chosen.append(i - 1)
            r -= weights[i - 1]
    return dp[n][capacity], chosen[::-1]

def can_reach(nums: list[int], target: int) → bool:
    """Subset sum: the same table with booleans. reachable[0] = True is the base case."""
    reachable = [False] * (target + 1)
    reachable[0] = True
```

```

    for number in nums:
        for r in range(target, number - 1, -1):
            if reachable[r - number]:
                reachable[r] = True
    return reachable[target]

def can_partition(nums: list[int]) → bool:
    total = sum(nums)
    return total % 2 == 0 and can_reach(nums, total // 2)

def greedy_by_value(weights, values, capacity) → int:
    order = sorted(range(len(weights)), key=lambda i: -values[i])
    total, room = 0, capacity
    for i in order:
        if weights[i] ≤ room:
            total += values[i]
            room -= weights[i]
    return total

def greedy_by_ratio(weights, values, capacity) → int:
    order = sorted(range(len(weights)), key=lambda i: -values[i] / weights[i])
    total, room = 0, capacity
    for i in order:
        if weights[i] ≤ room:
            total += values[i]
            room -= weights[i]
    return total

if __name__ == "__main__":
    # Bhagwan's cycle, scaled down.
    w = [2, 3, 4]
    v = [3, 4, 5]
    print("all three agree :", knapsack_memo(w, v, 5), knapsack_table(w, v, 5), knapsack_one_row(w, v, 5))
    print("with items      :", knapsack_with_items(w, v, 5))
    print()

    # greedy by value fails
    w1, v1 = [10, 5, 5], [60, 50, 50]
    print("by value   greedy:", greedy_by_value(w1, v1, 10), " correct:", knapsack_one_row(w1, v1, 10))

    # greedy by ratio fails
    w2, v2 = [6, 5, 5], [60, 40, 40]
    print("by ratio   greedy:", greedy_by_ratio(w2, v2, 10), " correct:", knapsack_one_row(w2, v2, 10))
    print()

    # the direction that changes the problem
    print("0/1          :", knapsack_one_row([2], [3], 6))
    print("unbounded   :", unbounded_knapsack([2], [3], 6))
    print()

    print("partition [1,5,11,5] :", can_partition([1, 5, 11, 5]))
    print("partition [1,2,3,5]  :", can_partition([1, 2, 3, 5]))

```

Running it:

```

all three agree : 7 7 7
with items      : (7, [0, 1])

by value   greedy: 60   correct: 100
by ratio   greedy: 60   correct: 80

0/1        : 3
unbounded  : 9

partition [1,5,11,5] : True
partition [1,2,3,5]  : False

```

Three things to look at. **Both greedy versions return 60 and both are wrong**, by different amounts and for different reasons. **Keep those two inputs** — producing them on demand is what the question is testing.

`0/1` gives 3 and `unbounded` gives 9 on the identical input of one item weighing 2 and worth 3, with a capacity of 6. One item taken once against the same item taken three times. **The only difference in the code is the direction of a `range`.**

And `[1,5,11,5]` partitions into `{1,5,5}` and `{11}`, both summing to 11 — while `[1,2,3,5]` totals 11, which is odd, so it is rejected before any table is built.

6 What it costs

The two-dimensional table:

states	$n \times (W + 1)$
work per state	$O(1)$ -- two options

time	$O(n \times W)$
space	$O(n \times W)$, or $O(W)$ with one row

Concretely:

$n = 100$ items, $W = 1,000$	100,000 cells	instant
$n = 1,000$, $W = 10,000$	10,000,000	~1-2 s in Python
$n = 100$, $W = 1,000,000$	100,000,000	too slow
$n = 100$, $W = 10^9$	10^{11}	impossible

And that last row is the whole complexity story. $W = 10^9$ takes ten characters to write, so the *input* grew by ten characters and the table grew to a hundred billion cells.

This is pseudo-polynomial: polynomial in the **value** of W , exponential in the number of **digits** of W . **That is why a DP solution does not contradict the problem being NP-hard**, and it is worth being able to say in one sentence.

Space:

full table, $n = 1,000$, $W = 10,000$	
Python list of lists of ints	~10,000,000 x 8 B + overhead ~400 MB
one row	
10,001 ints	~80 KB

Five thousand times less, which is why the one-row version is what you actually write — and why reconstruction, which needs the table, is a genuine cost rather than a free extra.

The greedy alternatives, which are not even faster in any useful sense:

greedy by value	$O(n \log n)$ to sort	WRONG
greedy by ratio	$O(n \log n)$ to sort	WRONG (right for FRACTIONAL knapsack)
DP	$O(n \times W)$	correct

$n = 100, W = 1,000$	
greedy	~ 700 operations
DP	100,000 operations

So greedy is genuinely faster and gives the wrong answer, which is the honest position — there is a real trade here, unlike house robber where the wrong answer was not even cheaper. **If an approximate answer is acceptable, greedy by ratio is within a factor of two of optimal**, and there is a well-known scheme that gets within any chosen epsilon in polynomial time. **Naming that is a strong answer to “what if W is enormous?”**

Subset sum and partition:

partition, n items summing to S	
target	$S / 2$
table	$n \times (S/2)$

$n = 200, \text{values} \leq 100$	$\rightarrow S \leq 20,000$	table $200 \times 10,000 = 2,000,000$	fine
$n = 200, \text{values} \leq 10^9$	$\rightarrow S \leq 2 \times 10^{11}$	table impossible	

Which is why LeetCode’s partition problem caps the values — and noticing that constraint is how you know the intended solution is this DP.

With a bitset, subset sum gets dramatically faster in practice:

```
reachable = 1                                # bit r set = sum r is reachable
for number in nums:
    reachable |= reachable << number
return (reachable >> target) & 1
```

Python big integers do 64 bits per word
 $n \times S/64$ word operations instead of $n \times S$
 $\rightarrow \sim 64\times$ faster, and much less memory

Three lines, and it is the fastest practical subset sum in Python. Worth knowing, and worth explaining rather than presenting as magic: shifting the whole set of reachable sums left by `number` is exactly “every previously reachable sum, plus this number”.

Reconstruction:

```
needs the full table      O(n x W) space
the backward walk        O(n) time
```

So asking for the items costs the space optimisation, and at `n = 1,000, W = 10,000` that is 400 MB rather than 80 KB.

7 The traps

The upward range in the one-row version

The most dangerous line in dynamic programming, because both versions are correct programs:

```
for r in range(w, capacity + 1):      # upwards
    row[r] = max(row[r], v + row[r - w])
```

```
>>> one_item_upward([2], [3], 6)
9                                     # took the item three times
>>> one_item_downward([2], [3], 6)
3                                     # correct for 0/1
```

No error, no warning, and the answer is larger rather than smaller, so it does not look obviously wrong. **Write the direction as a comment saying which problem it solves,** every time.

One-dimensional state

```
@lru_cache(maxsize=None)
def best(i):
    return max(values[i] + best(i + 1), best(i + 1))
```

```
>>> broken([(3,4),(4,5),(2,3)], capacity=5)
12                                     # the sum of everything; total weight is 9
```

Nothing tracks the capacity, so nothing prevents taking every item. **Yesterday's test catches it in five seconds:** “at item five, what is the best I can do?” has no answer.

Missing the “does not fit” guard

```
take = values[i] + best(i + 1, remaining - weights[i])    # no check
```

`remaining` goes negative, the cache fills with nonsense states, and the answer is too large. **In the one-row version the equivalent is a range that starts below the weight**, which raises `IndexError` or, worse, wraps to a negative index and reads from the far end of the row.

```
reachable[0] = False in subset sum
```

```
reachable = [False] * (target + 1)
# reachable[0] = True      ← forgotten
```

```
>>> can_reach_broken([1, 5, 11, 5], 11)
False                                     # every cell stays False
```

The empty subset sums to zero, and that is the only cell not derived from the recurrence. Without it nothing is ever reachable, and the failure is total rather than subtle — which is at least easy to spot.

Not checking the odd total in partition

```
return can_reach(nums, total // 2)          # total = 11, so target = 5
```

Integer division silently answers a different question. `[1,2,3,5]` sums to 11; asking “can I reach 5” gives `True`, and the answer to the actual question is `False`. **One line, and it is free.**

Mixing the two table conventions

```
dp = [[0] * (capacity + 1) for _ in range(n)]    # n rows, "items i onwards"
...
w = weights[i - 1]                               # but indexing as "the first i items"
```

```
IndexError: list index out of range
```

or, worse, a silently wrong answer where one item is skipped. **Pick “the first `i` items” with `n+1` rows, or “items `i` onwards” with `n+1` rows and a different base row — and say which in a comment.**

Assuming greedy is a reasonable approximation without saying so

```
return greedy_by_ratio(weights, values, capacity)
```

It is a reasonable approximation — within a factor of two — and it is not the answer to the question asked. **If an approximation is acceptable, say that it is an approximation and give the bound.** Presenting it as the solution is the failure.

Forgetting that W can make the DP impossible

```
knapsack(weights, values, capacity=10**9)
```

The code is correct and the table has a hundred billion cells per item. **Compute $n \times W$ before writing**, and if it is above about ten million, say so and discuss the alternatives — meet-in-the-middle for small n , or an approximation scheme.

8 In the interview

How it gets asked

- “Maximise the value you can carry within the weight limit.” — the direct version.
- “Can this array be split into two subsets with equal sums?” — LeetCode 416, the most common disguise.
- “Can any subset sum to exactly this target?”
- “Assign plus or minus to each number to reach a target.” — LeetCode 494.
- “Why doesn’t greedy work?” — the question the problem exists for.
- “Now items can be used more than once.” — the direction change.

The first ninety seconds

“The state needs two dimensions and I want to say why before writing anything. If the state is just the item index, then ‘at item five, what is the best I can do?’ has no answer — it depends on how much capacity is left, and two different paths reach item five with different amounts. So $dp[i][r]$ is the maximum value from items i onwards with r capacity remaining.

The recurrence is the choice at each item: skip it, and the answer is $dp[i+1][r]$; or take it, and it is $value[i] + dp[i+1][r - weight[i]]$ — **but only if it fits**, and that is the new dimension making a branch illegal rather than merely worse.

The thing I would raise before you ask is why greedy fails, because it fails twice.

By value: capacity ten, one item weighing ten worth sixty, and two weighing five worth fifty each. Greedy takes the sixty and blocks both; the answer is a hundred.

By value per weight, which is the persuasive one: capacity ten, an item of six kilos worth sixty — ratio ten — and two of five kilos worth forty, ratio eight. Greedy takes the best ratio, has four kilos left, and nothing fits. Sixty against eighty.

And the reason the ratio rule is so tempting is that it is provably correct for the *fractional* knapsack, where you can take part of an item. The moment items are indivisible the proof collapses, because the last item cannot be trimmed and the leftover space is wasted. **That sentence is the real answer to ‘why not greedy’.**

$O(n \times W)$ time and $O(n \times W)$ space, collapsing to $O(W)$ with one row — and I would note that $n \times W$ is *pseudo*-polynomial: W is a number in the input, so a capacity of a billion makes the table a hundred billion cells while the input grew by ten characters. That is why this DP does not contradict the problem being NP-hard.

Shall I write the one-row version? There is a subtlety in the iteration direction I would want to explain.”

The follow-ups

“Explain the iteration direction.”

“It is the most dangerous line in dynamic programming, because both directions are correct programs for different problems.

With one row, `row[r] = max(row[r], v + row[r - w])`. The question is what `row[r - w]` holds when I read it.

Going downwards, from capacity to the weight, `row[r - w]` is a smaller index that this round has not reached yet, so it still holds the value from before this item was considered. **The item is used at most once. That is 0/1.**

Going upwards, `row[r - w]` is a smaller index that this round has *already updated*, so it may already include this item. **The item can be used again. That is the unbounded knapsack.**

On a single item weighing two and worth three, with capacity six: downwards gives three, upwards gives nine. **Same three lines of code.**

And neither raises an error, and the wrong one gives a *larger* answer, which does not look obviously wrong. So I write the direction as a comment naming which problem it solves, every time — and if I am ever unsure, I test it on one item that fits several times, which distinguishes them immediately.”

“Can this array be split into two equal halves?”

“That is subset sum, which is this problem with the value equal to the weight.

The reduction: if the two halves are equal, each sums to `total / 2`. So the question is ‘is there a subset summing to exactly `total / 2`’, and if such a subset exists the rest is automatically the other half.

First, and free: if the total is odd, the answer is no. No two integers sum to an odd number equally. One line, before any table.

Then the table is booleans rather than numbers: `reachable[r]` is whether some subset sums to exactly `r`. `reachable[0] = True`, because the empty subset sums to zero — **that is the only cell not derived from the recurrence and it is the one people forget**; without it everything stays false.

Then the same downward loop per number, and the answer is `reachable[total // 2]`.

Cost is `n × total/2`, and this is where the constraints matter: LeetCode caps the values at a hundred with two hundred numbers, so the total is at most twenty thousand and the table is two million cells. **Noticing that cap is how you know this DP is the intended solution** — with values up to a billion it would be impossible.

And there is a much faster version worth knowing: represent the set of reachable sums as the bits of a single big integer, and each number is `reachable |= reachable << number`. Python does sixty-four bits per word, so it is about sixty-four times faster and far less memory, in three lines.”

“What if the capacity is a billion?”

“Then this DP is not the answer, and I would say so rather than write it.

$n \times W$ with $W = 10^9$ is a hundred billion cells per item. The table is impossible even though the code is correct, and this is exactly the pseudo-polynomial point: W is a *number in the input*, so writing a billion took ten characters and the table grew by a factor of a million.

Three alternatives depending on what else is small.

If n is small — up to about forty — meet in the middle. Split the items into two halves, enumerate all $2^{(n/2)}$ subsets of each, sort one side by weight, and for each subset of the other side binary-search the best complement that fits. That is $O(2^{(n/2)} \times n)$, so about a million operations at $n = 40$ instead of a trillion.

If the values are small rather than the weights, swap the dimensions. Make the table indexed by *value* — $dp[i][v]$ is the minimum weight needed to achieve value v — and the answer is the largest v whose minimum weight fits. That is $O(n \times \text{total value})$, which is the right choice whenever the values are bounded and the weights are not. It is a genuinely useful move and it is the one people do not think of.

And if an approximation is acceptable, greedy by ratio is within a factor of two, and there is a standard scheme that scales the values down and gets within any chosen epsilon in polynomial time.

The honest answer is that the problem is NP-hard, so with both n and W large there is no efficient exact algorithm, and the question becomes which approximation is acceptable.”

“Now each item can be taken any number of times.”

“The unbounded knapsack, and it is the *upward* iteration — literally the same three lines with the range reversed.

Why that works: going upwards, when I read `row[r - w]` it may already include this item from earlier in the same pass, so the item accumulates. That is precisely the behaviour I now want.

And the state actually gets simpler, which is worth noticing. For 0/1 I needed to know which items were still available, which is why the item index is in the state. For unbounded, every item is always available, so **the state collapses to just the remaining capacity** — `dp[r]` is the best achievable with capacity `r`, and the item index is a loop variable rather than a dimension.

That is why coin change is a one-dimensional DP while the 0/1 version is two-dimensional, and it is a nice illustration that the state is determined by what you need to remember, not by the number of loops.

Cost is the same, $O(n \times W)$, and the space is $O(W)$ either way.

The bounded version — at most `k` copies of each item — is the awkward middle. The naive approach adds a loop over the count, making it $O(n \times W \times k)$. The standard trick is **binary splitting**: replace an item with `k` copies by items representing 1, 2, 4, 8, ... copies, since any count up to `k` is a sum of those powers. **That turns `k` copies into $\log k$ items and the problem back into plain 0/1**, at $O(n \times W \times \log k)$.”

The model answer

“Given an array of positive integers, determine whether it can be partitioned into two subsets with equal sums.”

“Let me reduce it before designing anything, because the reduction is most of the work.

If the array splits into two equal halves, each half sums to `total / 2`. So the question is: is there a subset summing to exactly `total / 2`? And if one exists, its complement is automatically the other half, so I only have to find one.

Free check first: if the total is odd, the answer is no. No two integer sums are equal and odd. One line before any work, and it also protects me from the integer-division bug where `11 // 2` silently becomes a question about 5.

Then it is subset sum, which is the knapsack with value equal to weight.
`reachable[r]` = can some subset sum to exactly `r`. The recurrence is the same choice: for each number, either it is in the subset making `r` — so `r - number` must have been reachable before — or it is not.

Base case: `reachable[0] = True`, the empty subset. **That is the only cell not produced by the recurrence**, and without it nothing is ever reachable.

One row, iterating the target downwards, because each number may be used at most once. **Going upwards would let a number be reused** and would answer ‘can I reach the target using these numbers with repetition’, which is a different and easier question — and it would return `True` on inputs where the answer is `False`. I would write the direction as a comment.

Cost: `0(n × total/2)` **time** and `0(total/2)` **space**. With the usual constraints — two hundred numbers, each at most a hundred — the total is at most twenty thousand, so the table is ten thousand booleans and the work is two million operations. **The fact that the values are capped is the signal that this is the intended solution**; without that cap the approach would not be viable, because it is pseudo-polynomial.

Two refinements I would offer. An early exit: once `reachable[target]` becomes true, stop. And the bitset version — hold the reachable sums as the bits of one big integer and do `reachable |= reachable << number` per number, which is about sixty-four times faster in Python and three lines. **I would write the clear version first and mention the bitset**, because presenting the bitset cold looks like a memorised trick rather than a derivation.

And if the follow-up is ‘which numbers are in each half’, I need the full two-dimensional table rather than one row, and I walk it backwards: at each item, if the cell differs from the one above, that item was used. That costs `0(n ×`

`total/2)` space rather than `O(total/2)`, and I would ask whether the boolean answer is sufficient before giving it up.”

9 Recall card

`dp[i][r]` = the best value from items `i` onwards with `r` capacity left. One index is not a question — “at item five” has no answer without the room remaining. Take (if it fits) or skip.

Greedy fails twice, and both counter-examples are worth memorising. By value: `W=10`, `(10,60)`, `(5,50)`, `(5,50)` — 60 against 100. By ratio: `W=10`, `(6,60)`, `(5,40)`, `(5,40)` — 60 against 80. **The ratio rule is correct for the fractional knapsack; indivisibility is what breaks it.**

One row, and the direction is the problem. `for r in range(W, w-1, -1)` is 0/1; `range(w, W+1)` is **unbounded**. Same three lines, no error, and the wrong one gives a *larger* answer.

`O(n × W)` is **pseudo-polynomial** — polynomial in the *value* of `W`, not its digit count — which is why a DP exists and the problem is still NP-hard. Compute `n × W` before writing; above $\sim 10^7$, use meet-in-the-middle (small `n`), swap the dimension to index by **value** (small values), or approximate.

The family is the same table: subset sum (booleans, and `reachable[0] = True` is the forgotten base case), equal partition (target `total/2`, and **reject an odd total first**), target sum, minimum difference. Reconstruction needs the full table, so it is mutually exclusive with the one-row form.

Design a rate limiter, at system scale

1 What this is, and why they ask it

A rate limiter decides, for each incoming request, whether to serve it or reject it with a `429 Too Many Requests`. “A hundred requests per minute per API key” is the usual shape.

It is a small, self-contained component and it is asked constantly, because it is one of the few designs where **there is a specific algorithm to choose and the choices have visible, quantifiable differences**. Five algorithms, each with a different memory cost and a different failure mode, and a candidate who knows why sliding-window-counter exists is demonstrating something a candidate who says “we’ll use Redis” is not.

The other half is the distributed problem, and it is the interesting half. **A limit is a shared count, and your service is many machines**. Every machine checking a shared counter means a network round trip on every request — on the path of every request in the system. Every machine keeping its own count means the effective limit is `n` times what you promised.

They also ask it because it sits on the most latency-sensitive path there is: **the limiter runs before the work, on every request, including the ones it is about to reject** — so it has to be faster than the thing it protects.

By the end of this lesson you can choose an algorithm and defend it, handle the distributed count with a stated accuracy trade, size the memory, decide what happens when the limiter itself fails, and name what to limit on.

2 The story

The tank on the roof of Savitri’s building holds about six thousand litres and the municipal line fills it at whatever rate the municipal line feels like, which is roughly steady and roughly slow.

Twenty flats draw from it.

Most days nobody thinks about it. The line runs, the tank fills, people use water, and the level stays somewhere in the middle.

What everybody understands without ever having explained it is that the tank lets you take a lot at once. If nobody has drawn much since yesterday, the tank is full, and you can run the washing machine and fill the drums and wash the car and it is fine. The tank absorbs that.

And that if everybody does it on the same morning, there is nothing. The tank empties by nine, and then you get whatever the line is putting in, drop by drop, which is not enough to wash a bucket.

The thing that caused the trouble, and eventually the meeting, was the second-floor flat.

They have a motor. Everybody has a motor, but theirs runs at half past five every morning and pulls until their own tank is full, and their own tank is a thousand litres. So by six o'clock, six mornings a week, a sixth of the building's water had gone to one flat, and by the time anybody else was up there was not much left.

Nobody was breaking any rule, because there was no rule. **They were just faster.**

The committee's answer, after two meetings and one shouting match, was a timer on each flat's motor. Fifteen minutes, and then it cuts, and you wait an hour before it will run again.

Savitri's observation, which she made at the second meeting and which everybody agreed with and nobody acted on for another month, was that the rule was not really about being fair. **It was that if everybody drew as fast as they physically could, the pump at the bottom burnt out,** and it had, twice, and each time cost eleven thousand rupees and four days without water.

The timer was not there to make people share. It was there to stop the pump dying.

3 The idea in plain English

Savitri's tank is a token bucket, and her observation at the second meeting is what rate limiting is actually for.

A rate limiter answers one question per request: is this allowed right now? If yes, serve it. If no, return `429 Too Many Requests`, ideally with a `Retry-After` header saying when to come back.

And it exists for three different reasons that are worth separating, because they lead to different limits:

- **Protecting the system.** The pump. Beyond some rate, the service degrades for everybody, so the limiter sheds load to keep it alive. This is the important one.
- **Fairness.** Stopping one client consuming everything. The second-floor flat.
- **Cost and abuse.** Metering a paid API, or stopping credential-stuffing and scraping.

Now the five algorithms, in increasing order of accuracy and cost.

Fixed window. Count requests per key per clock minute. Reset at the boundary. **One counter and one timestamp per key — the cheapest possible.**

And it allows twice the limit, which is its defining flaw. With a limit of a hundred per minute, a client can send a hundred at 10:00:59 and another hundred at 10:01:00 — two hundred requests in one second, both windows technically satisfied. **That is the boundary problem,** and it matters exactly when it matters most, because a burst is what you were trying to prevent.

Sliding window log. Store the timestamp of every request in the last window. To decide, drop everything older than the window and count what is left. **Perfectly accurate** — no boundary problem at all.

And it stores one timestamp per request. A limit of a thousand per hour means up to a thousand timestamps per key, and at a million keys that is a billion timestamps. **Accurate and expensive,** and the memory scales with the *limit*, not with the number of keys.

Sliding window counter. The practical compromise. Keep a counter for the current window and the previous one, and estimate:

```
count = current_window_count + previous_window_count x (fraction of the previous window still in view)
```

At 10:00:15 with a one-minute window, 75% of the previous minute is still inside the sliding window, so you count 75% of it. **Two counters per key — the same memory as fixed window — and it removes almost all of the boundary problem.**

It is an approximation — it assumes the previous window's requests were spread evenly — so it can be slightly wrong in both directions. **In practice the error is small and it is what most production limiters use.**

Token bucket. Savitri's tank. A bucket of capacity **B** refills at **R** tokens per second. Each request takes one token; no token, no service.

Two numbers with two different jobs, and separating them is the point: **R** is the sustained rate, **B** is how much burst is tolerated. **A client that has been quiet can spend the whole bucket at once,** which is usually the behaviour you want — a mobile app that syncs after being offline should not be throttled for being idle.

And it stores two values per key — the token count and the last refill time — and the refill is computed lazily on read rather than by a background job:

```
tokens = min(capacity, tokens + (now - last_refill) x rate)
```

Leaky bucket. A queue drained at a fixed rate. Requests join the queue; the queue is served steadily; overflow is rejected. **It smooths output completely** — the downstream sees a perfectly even rate regardless of the input — at the cost of latency, because a request may sit in the queue.

Token bucket allows bursts; leaky bucket eliminates them. That is the distinction, and which you want depends on whether the thing you are protecting minds a burst.

Now the distributed half, which is the interesting part.

The limit is per client and your service is `n` machines. Three options.

One: a shared counter in Redis. Every request does an atomic increment against a central store. **Accurate**, and it adds a network round trip — half a millisecond in the same datacentre — to *every request in the system*, including rejected ones, and makes Redis a hard dependency of every request.

Two: local counters, limit divided by `n`. No coordination at all, no latency. **And it is wrong whenever traffic is uneven**, which it always is: a client hitting one machine gets a hundredth of the intended limit, and the same client spread across all machines gets the full limit. **Load balancers do not distribute one client's traffic evenly**, so this is not a theoretical objection.

Three: local counters with asynchronous synchronisation. Each machine limits locally and periodically shares its counts. **Fast, approximately correct, and it over-admits by up to one sync interval's worth of traffic.** This is what most large systems actually do, and being able to say “it over-admits by a bounded amount, and here is the bound” is the answer.

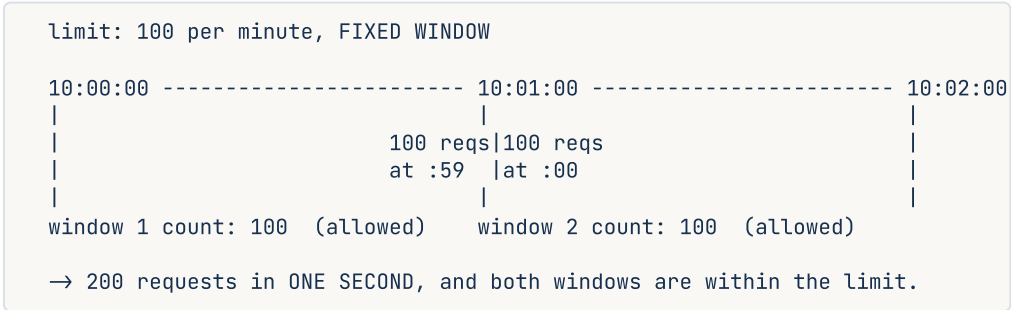
And the last decision, which is a policy question rather than a technical one: what happens when the limiter fails?

Fail open — allow everything — means an outage of Redis becomes unlimited traffic to the thing the limiter was protecting, which is how a small failure becomes a large one. **Fail closed** — reject everything — means an outage of Redis becomes a total outage of your service.

The usual answer is to fail open with a local fallback: if the shared store is unreachable, fall back to a per-machine limit that is deliberately generous. **Not perfect, and much better than either extreme** — and it must be decided in advance, or whoever writes the error handler decides it for you.

4 The picture

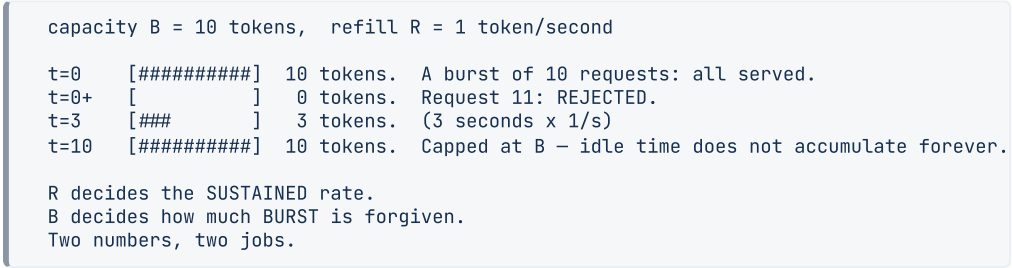
The boundary problem, which is why fixed window is not enough:



The five algorithms, compared:

	state per key	accuracy	bursts
fixed window	counter + window	2x at boundary	allowed at boundaries
sliding log	one ts per request	exact	none
sliding counter	2 counters	~exact	slight
token bucket	tokens + timestamp	exact rate	ALLOWED, up to B
leaky bucket	queue	exact rate	eliminated (queued)

Token bucket, drawn as the tank:



Sliding window counter, which is the practical middle:

```
limit 100/minute, now = 10:01:15    (25% into the current window)
```

```
previous window (10:00-10:01):  80 requests
```

```
current window (10:01-10:02):  30 requests so far
```

```
the sliding window covers the last 60 s:
```

```
    75% of the previous window + all of the current
```

```
estimate = 80 x 0.75 + 30 = 60 + 30 = 90    → under 100, ALLOW
```

```
two counters per key. Fixed-window memory, sliding-window behaviour.
```

The distributed problem:

SHARED COUNTER

```
every request  
→ Redis INCR
```

```
+ exact
```

```
- +0.5 ms on EVERY request  
- Redis is now a hard  
  dependency of everything
```

LOCAL, LIMIT/n

```
no coordination  
→ zero latency
```

```
+ fastest possible
```

```
- wrong whenever traffic  
  is uneven, which is  
  always
```

LOCAL + SYNC

```
local decisions,  
periodic gossip  
of counts
```

```
+ fast
```

```
+ approximately right  
- over-admits by up to  
  one sync interval
```

5 How it actually works

Token bucket, which is the one to be able to write

```
import time

class TokenBucket:
    """capacity = burst allowance, rate = sustained tokens per second."""

    def __init__(self, capacity: float, rate: float) → None:
        self.capacity = capacity
        self.rate = rate
        self.tokens = capacity
        self.last = time.monotonic()

    def allow(self, cost: float = 1.0) → bool:
        now = time.monotonic()
        self.tokens = min(self.capacity, self.tokens + (now - self.last) * self.rate)
        self.last = now
        if self.tokens ≥ cost:
            self.tokens -= cost
            return True
        return False
```

Three things to point at.

`time.monotonic()`, not `time.time()` — for exactly the reason from [day 123](#). A wall-clock correction that steps backwards makes `now - last` negative and the bucket loses tokens; one that steps forward hands out a full refill instantly. **A rate limiter that occasionally lets through double the limit is usually this bug.**

The refill is lazy, computed on access from the elapsed time. No background job, no per-key timer, and a key that is never touched costs nothing.

And `cost` as a parameter means expensive endpoints can consume more than one token, which is how you express “a search costs ten times a simple read” in a single limiter.

Sliding window counter, in Redis

```
def allow(redis, key: str, limit: int, window: int) → bool:
    now = time.time()
    current_window = int(now // window)
    elapsed_fraction = (now % window) / window

    pipe = redis.pipeline()
    pipe.get(f"{key}:{current_window}")
    pipe.get(f"{key}:{current_window - 1}")
    current, previous = (int(x or 0) for x in pipe.execute())

    estimate = previous * (1 - elapsed_fraction) + current
    if estimate ≥ limit:
        return False

    pipe = redis.pipeline()
    pipe.incr(f"{key}:{current_window}")
    pipe.expire(f"{key}:{current_window}", window * 2)      # self-cleaning
    pipe.execute()
    return True
```

The `expire` is what keeps the memory bounded — keys for old windows delete themselves, so there is no cleanup job and no unbounded growth. **That is the single most important line for operating this**, and it is easy to leave out.

The read-then-write is not atomic, so two concurrent requests can both read 99 and both proceed. **That is usually acceptable** — being one over on a limit of a hundred is not a failure — and if it is not, the whole thing goes into a Lua script, which Redis executes atomically.

Token bucket in Redis, atomically

```
-- KEYS[1] = bucket key, ARGV = capacity, rate, now, cost
local bucket = redis.call('HMGET', KEYS[1], 'tokens', 'last')
local tokens = tonumber(bucket[1]) or tonumber(ARGV[1])
local last   = tonumber(bucket[2]) or tonumber(ARGV[3])

tokens = math.min(tonumber(ARGV[1]), tokens + (ARGV[3] - last) * ARGV[2])

local allowed = tokens ≥ tonumber(ARGV[4])
if allowed then tokens = tokens - tonumber(ARGV[4]) end

redis.call('HMSET', KEYS[1], 'tokens', tokens, 'last', ARGV[3])
redis.call('EXPIRE', KEYS[1], 3600)
return allowed and 1 or 0
```

One round trip, atomic, and the `EXPIRE` bounds the memory. This is close to what production limiters actually run, and being able to say “the whole check-and-decrement is one Lua script so it is atomic” is worth more than the exact Lua.

Where the limiter goes

```
client → CDN → API gateway → [RATE LIMITER] → service → database
                        ^
                    before the work, on every request
```

At the gateway, not in each service, for three reasons: rejected requests never reach the services at all, the policy is in one place, and the count is naturally shared because the gateway fleet is one thing rather than many.

And a second, cruder limiter belongs further out — at the CDN or load balancer — because a volumetric attack should be dropped before it reaches anything of yours. **Layered limits: a crude one at the edge, a precise one at the gateway.**

What to limit on

by API key / account	the normal case for a paid API
by user id	for authenticated traffic
by IP	for unauthenticated traffic – and it is crude: a university or a mobile carrier NATs thousands of users behind one address
by endpoint	expensive endpoints get their own, tighter limit
by a combination	(user, endpoint) is the usual production shape

Multiple limits apply at once, and a request must pass all of them: a global limit, a per-user limit, and a per-endpoint limit. **Which is worth saying, because a single limiter is rarely enough** — you want to stop one user hammering search *and* stop the

whole system exceeding what the database can take.

The response

```
HTTP/1.1 429 Too Many Requests
Retry-After: 30
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1767225600
```

Retry-After is the one that matters, because without it every rejected client retries immediately and the limiter becomes a busy-wait for the whole client base. **And the retry should be jittered**, or every client rejected in the same second comes back in the same second — which is [day 125's](#) thundering herd, caused by your own limiter.

Returning the headers on successful requests too lets a well-behaved client slow down before being rejected, which is strictly better than rejecting it.

The failure decision

```
shared store unreachable:
  fail closed    → reject everything    → Redis outage = total outage
  fail open      → allow everything      → Redis outage = unlimited traffic to
                                          the thing you were protecting
  fail open with a LOCAL fallback:
    → each machine applies a generous local limit
    → approximately right, bounded damage
```

The third is the answer, and it must be decided in advance. Whoever writes the exception handler decides it otherwise, and they will pick whichever is easier to type.

6 The numbers

Memory per key, which is what decides the algorithm at scale:

fixed window	counter + window id	~16 bytes
sliding counter	2 counters + 2 window ids	~32 bytes
token bucket	tokens + timestamp	~16 bytes
sliding log	one timestamp per request in window	~8 bytes x limit

1,000,000 active keys, limit 1,000/hour

fixed window	$1,000,000 \times 16 \text{ B}$	= 16 MB
sliding counter	$1,000,000 \times 32 \text{ B}$	= 32 MB
token bucket	$1,000,000 \times 16 \text{ B}$	= 16 MB
sliding log	$1,000,000 \times 1,000 \times 8 \text{ B}$	= 8 GB

Eight gigabytes against thirty-two megabytes, and that is the whole argument against the exact algorithm. **With Redis key overhead the real figures are two to five times larger**, so the sliding log is realistically 20–40 GB — one node's entire memory to be exactly right about a limit where being 2% wrong is harmless.

Latency, and this is on every request:

local in-memory check	~0.001 ms
Redis in the same datacentre	~0.5 ms round trip
Redis cross-AZ	~1-2 ms

service p50 latency 20 ms	
+ 0.5 ms limiter	= 2.5% slower
service p50 latency 2 ms	
+ 0.5 ms limiter	= 25% slower

The limiter's cost is a percentage of what it protects, so it matters enormously for a fast service and barely at all for a slow one. **That is the argument for local-with-sync in front of a very fast service.**

Redis capacity:

Redis	~100,000 ops/s per instance
one Lua script per request	= 1 op
	→ 100,000 requests/s per Redis instance

service handling 1,000,000 requests/s
→ 10 Redis instances, sharded by key
→ and sharding is trivial, because each key is independent

The independence of keys is why this shards perfectly — there is never a cross-shard question — and it is worth saying, because it means the shared-counter approach scales further than people assume.

The boundary problem, quantified:

```
limit 100/minute, fixed window
  worst case in any 60-second span: 200
  → 2x the intended limit, in the worst possible instant

sliding window counter
  error depends on how unevenly the previous window was distributed
  worst case ~ +/- 10-20% in adversarial patterns
  typical    < 5%
```

Two hundred percent against five percent, for the same memory. That comparison is the reason sliding-window-counter exists.

The distributed error:

```
local counters, sync every 10 s, limit 100/minute, 10 machines

  worst case: each machine admits up to its share, unaware of the others,
  for up to one sync interval
  over-admission bound ~ (machines x limit x sync interval / window)
                       = 10 x 100 x 10/60 ~ 167 extra requests worst case

  → the limit is effectively "100, but up to ~270 in a bad minute"
```

Being able to state that bound is the answer, rather than claiming the approximation is exact.

Cost of the limiter itself:

```
3 Redis nodes for 300,000 req/s of limiting ~$300-600/month
against protecting a database fleet costing many times that
```

And what it saves, which is the justification:

```
without a limiter, one client scraping at 10,000 req/s
  → database saturated, p99 for everyone goes from 50 ms to 5 s
  → effectively an outage caused by one client
with a limiter at 100/s for that client
  → 99% of that traffic rejected in 0.5 ms each, never reaching the database
```

7 The trade-offs

Accuracy against memory, and the sliding log is the extreme. Exact limiting costs one timestamp per request — 8 GB against 32 MB at a million keys with a limit of a thousand. **Almost nobody needs exactness,** because a limit is a policy choice with a

round number in it, and being 5% over on “a hundred a minute” harms nothing. **Pay for exactness only when the limit represents something real** — a paid quota, a regulatory cap.

Accuracy against latency in the distributed case. A shared counter is exact and adds half a millisecond to every request in the system, including the ones being rejected, and makes the store a hard dependency of everything. Local counters are free and wrong. **Local-with-sync is fast and over-admits by a bounded amount**, and the honest version of that answer states the bound.

Bursts allowed against bursts eliminated. Token bucket forgives a client that has been quiet, which is usually correct — a mobile app syncing after being offline is not abusive. Leaky bucket smooths completely and adds queueing latency. **Ask what you are protecting: a database that dislikes spikes wants smoothing; an API whose users are bursty by nature wants the bucket.**

Limiting by IP is crude and often the only option. A university, an office or a mobile carrier puts thousands of users behind one address, so an IP limit either throttles them all or is too loose to matter. **For unauthenticated traffic there is nothing better**, which is an argument for requiring authentication on anything expensive rather than for a cleverer limiter.

Fail open against fail closed, and neither is right alone. Open turns a limiter outage into unlimited load on the protected system; closed turns it into a total outage. **Open with a generous local fallback is the answer, and it must be chosen deliberately.**

And the limiter is itself something that can fail. It runs before every request, so its latency is added to everything and its availability multiplies with the service's. **A rate limiter that adds two milliseconds and 99.9% availability to a service with a 2 ms p50 has made things worse**, and that is worth checking rather than assuming.

When would I not build one? When the traffic is internal and trusted, where a limiter adds latency and failure modes against an attack that does not exist. When the platform already provides one — every API gateway and CDN has rate limiting built in, and using it is strictly better than writing one. And when the real problem is a single misbehaving client, where a targeted block is simpler than a general mechanism.

8 In the interview

How it gets asked

- “*Design a rate limiter for an API used by a million clients.*” — the standard prompt.
- “*Which algorithm, and why?*” — the part with a real answer.
- “*Your service is fifty machines. How do you share the count?*” — the distributed half.
- “*What happens when Redis is down?*” — the policy question.
- “*A client sends a hundred requests at 59 seconds and a hundred at 60. What happens?*” — the boundary problem.
- “*What do you limit on?*”

“Let me split it into the algorithm and the distributed problem, because they are separate decisions and the second is the harder one.

On the algorithm, I would use a sliding window counter or a token bucket, and I would reject fixed window for a specific reason. Fixed window counts per clock minute and resets at the boundary, which means a client can send a hundred requests at 10:00:59 and another hundred at 10:01:00 — **two hundred in one second, with both windows within the limit.** That is twice the intended rate at exactly the moment a burst is happening, which is what I was trying to prevent.

The exact fix is a sliding window log — store every request’s timestamp and count what falls inside the window. Perfectly accurate, and it costs one timestamp per request: at a million keys with a limit of a thousand an hour, that is eight gigabytes against thirty-two megabytes for two counters. **I would not pay that,** because a limit is a policy number and being a few percent over harms nothing.

So: sliding window counter. Keep the current window’s count and the previous window’s, and weight the previous one by how much of it is still in view. Two counters per key — fixed-window memory, and the boundary error drops from 200% to a few percent.

Or a token bucket if bursts should be forgiven, which is often what you actually want: a bucket of capacity B refilling at R per second, where R is the sustained rate and B is how much burst is tolerated. **Two numbers doing two different jobs.** A client that has been idle can spend the whole bucket at once, which is right for something like a mobile app syncing after being offline.

Now the distributed half, which is the interesting part. The limit is per client and I have many machines, so the count is shared state. Shared Redis is exact and adds half a millisecond to **every request in the system**, including rejected ones. Local counters with the limit divided by the machine count need no coordination and are wrong whenever traffic is uneven, which it always is.

I would use local counters with periodic synchronisation and state the error bound — it over-admits by up to one sync interval’s worth of traffic, which I can compute.

How fast is the service being protected? Because if its p50 is two milliseconds, a half-millisecond limiter is a 25% tax and that changes the answer.”

The follow-ups

“Walk me through the boundary problem.”

“Fixed window with a limit of a hundred per minute. The counter resets at each clock minute.

A client sends a hundred requests at 10:00:59. The counter for the 10:00 window reaches a hundred — allowed, exactly at the limit. One second later the clock ticks over, the counter resets, and the client sends another hundred at 10:01:00. The 10:01 window reaches a hundred — also allowed.

Two hundred requests in a one-second span, and no window was ever exceeded. The implementation is perfectly correct against its own definition and the effective limit is double.

And it is worst exactly when it matters, because a client trying to maximise throughput will discover the boundary and sit on it.

The exact fix is the sliding window log and it costs a timestamp per request. **The practical fix is the sliding window counter:** hold the current and previous window counts and weight the previous by the fraction still in view. At fifteen seconds into a minute, seventy-five percent of the previous minute is still inside the last sixty seconds, so I count seventy-five percent of it.

That is an approximation — it assumes the previous window’s traffic was spread evenly, which a burst at the very end violates — so it can be off by ten or twenty percent in an adversarial pattern and under five percent normally. **Two hundred percent error down to five, for the same memory**, which is why it is what most production limiters use.”

“Fifty machines. How do you share the count?”

“Three options, and I would name the trade in each rather than pick one blindly.

A shared counter in Redis is exact: every request does one atomic operation against a central store. The cost is a network round trip on the path of every request in the system — half a millisecond in the same datacentre, one to two across availability zones — including on requests that are about to be rejected, which is faintly absurd. **And it makes Redis a hard dependency of every request**, which raises the failure question immediately.

It shards perfectly, though, and that is worth saying: each key’s count is independent, so there is never a cross-shard question, and a million requests a second is ten Redis instances sharded by key.

Local counters with the limit divided by fifty need no coordination and are simply wrong, because load balancers do not spread one client’s traffic evenly. A client whose connections land on one machine gets a fiftieth of the intended limit; one spread across all of them gets the full limit. **That is not a theoretical objection, it is what happens.**

Local counters with periodic synchronisation is what I would use, and the answer is to state the bound. Each machine limits locally and broadcasts its counts every few seconds; between syncs a machine is unaware of the others, so the worst-case over-admission is roughly the machine count times the limit times the sync interval divided by the window. At fifty machines, a hundred per minute and a ten-second sync, that is a few hundred extra requests in a bad minute.

So the honest description is ‘a hundred a minute, and up to about three hundred in an adversarial minute’ — and whether that is acceptable is a product question, not a technical one. For protecting a database it absolutely is; for a metered paid quota it is not, and then I pay the round trip.”

“What happens when Redis goes down?”

“That is a policy decision that has to be made in advance, because otherwise whoever writes the exception handler makes it, and they will pick whichever is easier to type.

Fail closed — reject everything when the limiter cannot decide — means a Redis outage becomes a total outage of the service. **The limiter, which exists to protect availability, has become the thing that destroys it.**

Fail open — allow everything — means a Redis outage becomes unlimited traffic to whatever I was protecting. **A small failure becomes a large one**, and it happens at the worst moment, because the same incident that took Redis down may be the one generating the traffic.

What I would build is fail open with a local fallback. If the shared store is unreachable, each machine applies a generous local limit from memory — say five times the per-machine share. Not accurate, and it bounds the damage: the protected system sees a multiple of the intended load rather than an unbounded amount, and the service stays up.

And I would circuit-break the limiter itself. If Redis is timing out, I should not wait for the timeout on every request — that adds the full timeout to every request in the system and turns a degraded dependency into an outage by thread exhaustion. **After a few failures, stop calling it and use the local fallback**, with periodic probes. That is [day 126](#)’s circuit breaker applied to the limiter, and it is the part people forget.”

“What do you limit on?”

“Several things at once, and a request has to pass all of them — a single limiter is rarely enough.

By API key or account for authenticated traffic, which is the normal case and the one that maps to a plan or a quota.

By IP for unauthenticated traffic, and I would be honest that it is crude: a university, an office or a mobile carrier puts thousands of users behind one address, so the limit either throttles them all or is too loose to be useful. **The real answer to that is to require authentication for anything expensive**, rather than to build a cleverer limiter.

By endpoint, because a search that runs a full-text query and a fetch-by-id are not comparable. Either a separate limit per endpoint, or — which I prefer — a **cost per request** in the token bucket, so an expensive endpoint takes ten tokens and a cheap one takes one. **One limiter, one number per endpoint**, and it composes better than a matrix of limits.

And a global limit, which is the one people forget: the sum of everybody’s permitted traffic can still exceed what the database can take. **The per-user limit is about fairness; the global limit is about the pump.**

Layered, too: a crude volumetric limit at the CDN or load balancer, so an attack is dropped before it reaches anything of mine, and the precise per-user limit at the gateway. **Rejecting a flood at the edge costs nothing; rejecting it at the gateway still costs a TLS handshake and a Redis lookup per request.**“

The model answer

“Design a rate limiter for a public API: a million clients, a hundred thousand requests a second, per-client limits that vary by plan.”

“Let me establish where it sits, then the algorithm, then the distributed problem — and the constraint that shapes it is that this thing runs before every request, including the ones it rejects, so its own cost is on the critical path of everything.

Placement: at the API gateway. Rejected requests then never reach the services at all, the policy lives in one place, and the gateway fleet is a natural place for shared state. **Plus a crude volumetric limit at the CDN**, so a genuine flood is dropped before it costs me a TLS handshake.

Algorithm: token bucket, and I would choose it over sliding window counter for a specific reason. Plans vary, and a token bucket expresses a plan as two numbers — sustained rate and burst allowance — which maps directly onto how plans are actually sold: ‘a thousand requests a minute, bursting to two hundred’. **And a per-request cost lets me price expensive endpoints in the same mechanism** rather than maintaining a separate limit per endpoint.

Implementation: one Redis hash per key holding the token count and the last refill time, updated by a Lua script so the whole check-and-decrement is atomic in one round trip. The refill is lazy — computed from the elapsed time on access — so there is no background job and an idle key costs nothing. **And an `EXPIRE` on every write**, which is what keeps the memory bounded with no cleanup job, and is the line most often left out.

Sizing. A million active keys at roughly a hundred bytes with Redis overhead is about a hundred megabytes — trivial. A hundred thousand requests a second is one Lua call each, and Redis does around a hundred thousand operations a second per instance, so **three instances sharded by key**, with headroom. Sharding is trivial here because keys are independent — there is never a cross-shard question.

The latency decision, which I would raise explicitly. Half a millisecond on every request is fine if the API’s p50 is twenty milliseconds — two and a half percent. **If the API’s p50 were two milliseconds it would be a twenty-five percent tax**, and then I would move to local buckets with a few seconds of synchronisation and accept a stated over-admission bound. **So I would ask for the service’s latency before committing**, because it genuinely changes the design.

Failure: fail open with a local fallback, plus a circuit breaker on the limiter itself. If Redis is unreachable, each gateway applies a generous local bucket. And if Redis is *slow* rather than down, I stop calling it after a few timeouts rather than

adding the full timeout to every request — otherwise a degraded Redis exhausts the gateway's threads and takes down the whole API, which is the limiter causing the outage it exists to prevent.

The response: 429 with `Retry-After`, and rate-limit headers on successful responses too so a well-behaved client can slow down before being rejected. **And the `Retry-After` value must be jittered**, or every client rejected in the same second comes back in the same second and I have built myself a thundering herd.

What I would monitor: rejection rate per plan — a spike means either an attack or a limit set too low, and those look identical on a dashboard until you segment by client; limiter latency at p99; and Redis availability, because it is now on the path of every request.

And the thing I would leave for later: dynamic limits that adapt to system load, so the global limit tightens automatically when the database is struggling. That is genuinely useful and it is a control loop with its own failure modes, and I would not put one in the first version.”

9 Recall card

Fixed window is the cheapest and allows $2 \times$ the limit at the boundary — 100 at 10:00:59 and 100 at 10:01:00. **Sliding log is exact and costs a timestamp per request** (8 GB vs 32 MB at 1M keys, limit 1,000). **Sliding window counter is the practical answer:** two counters, weight the previous window by the fraction still in view, error under ~5%.

Token bucket separates two numbers with two jobs — rate `R` is the sustained limit, capacity `B` is how much burst is forgiven — with **lazy refill** on access and `time.monotonic()`, never wall-clock. **Leaky bucket eliminates bursts entirely**, at the cost of queueing latency.

The distributed count is the hard half. Shared Redis is exact and adds ~0.5 ms to every request; local counters at limit/n are wrong because traffic is never even; **local-with-sync is what to use**, and the answer states the over-admission bound.

Decide the failure policy in advance: fail open with a generous local fallback, plus a **circuit breaker on the limiter itself** — a slow Redis that you wait for on every request is how the limiter causes the outage.

Limit on several things at once (account, IP for anonymous — crude because of NAT, endpoint via per-request **cost**, and a **global** limit for the system itself), layer a crude limit at the edge, and return `429` with a **jittered** **Retry-After** or you have built your own thundering herd.

DSA topic: The 0/1 knapsack **System design topic:** Design a rate limiter, at system scale

Code these, in this order

One rule for the whole set: **before writing the loops, say the state as a sentence and say which direction the inner loop runs, and why.** Knapsack is the one topic where the same six lines, with the range reversed, solve a completely different problem and never tell you which one you wrote.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	0/1 Knapsack	classic — write it yourself	Two dimensions, take-or-skip, and the greedy counter-example.
2	Partition Equal Subset Sum	LeetCode 416 (Medium)	Subset sum in disguise, and spotting the odd-total shortcut.
3	Target Sum	LeetCode 494 (Medium)	An algebraic rewrite into subset-sum counting.
4	Last Stone Weight II	LeetCode 1049 (Medium)	The same rewrite again, hidden much better.
5	Ones and Zeroes	LeetCode 474 (Medium)	Two resource dimensions at once — a three-dimensional table.
6	Coin Change	LeetCode 322 (Medium)	Unbounded, so tomorrow's loop direction. Do it to feel the difference.

On problem 1, build the greedy counter-examples yourself

Write `greedy_by_value` and `greedy_by_ratio` before you write the DP. Run both on `capacity = 10` with items `(10, 60), (5, 50), (5, 50)`, then on `capacity = 10` with `(6, 60), (5, 40), (5, 40)`. Record the four numbers.

Then say, in one sentence, why the ratio version is provably correct for the fractional version of the problem and collapses here. **That sentence is the answer to “why not greedy”, and it is worth more than the DP.**

On problem 1, reverse the inner loop and see nothing happen

Take the working one-row solution and change `range(capacity, weight - 1, -1)` to `range(weight, capacity + 1)`. Run it on `capacity = 10` with `(5, 50)` alone. Record the answer, and say which problem you have now solved.

No error, no warning, a plausible number. Write one sentence on how you would catch this in an interview without running anything.

On problems 3 and 4, do the algebra before the code

For Target Sum, write out the two-line derivation from $P - N = \text{target}$ and $P + N = \text{total}$ to $P = (\text{target} + \text{total}) / 2$. Then state the two conditions under which the answer is immediately zero.

For Last Stone Weight II, say in one sentence what the smashing actually is, and why it becomes “split into two groups with the closest possible sums”.

Both problems are the same trick and neither looks like knapsack. Do them back to back.

On problem 5, count the state space first

Write the state as a sentence, count the states, and multiply by the work per state. Do it for `m = 100`, `n = 100`, and 600 strings. Say whether it is viable and what the loop directions have to be.

On problem 6, feel the difference

Solve it with the forward loop. Then try the backward loop and record what changes. **The two answers are both correct programs for different problems**, and having produced both deliberately is the point.

Then the bitset drill

Implement subset-sum with `reachable |= reachable << number` on a Python integer. Run it against your table version on ten random inputs and confirm they agree. Then time both at `n = 200` with values up to 1,000, and record the ratio.

The greedy drill

1. Give the by-value counter-example with exact numbers, and both answers.
2. Give the by-ratio counter-example with exact numbers, and both answers.
3. Say which greedy is correct, and for which version of the problem.
4. Say in one sentence what indivisibility breaks.

The state drill

1. State the knapsack state as a full sentence.
2. Say why one index is not enough, and what the broken version returns.
3. Write the recurrence, both branches, and name the guard on the take branch.
4. Give the base cases and say what each one means in words.

The direction drill

1. Say what `range(capacity, weight - 1, -1)` guarantees, in one sentence about which values have been updated.
2. Say what the forward loop allows, and name the problem it solves.
3. Say why neither produces an error.
4. Say how you would state this out loud while writing it, so an interviewer sees you know.

The reconstruction drill

1. Say what you need to keep in order to recover which items were chosen.
2. Say why the one-row version cannot do it.
3. Give the walk-back rule in one sentence.
4. Say what it costs in space, and when it is worth paying.

The complexity drill

1. Give time and space for the table version and the one-row version.
2. Explain what pseudo-polynomial means, using `W` and the number of bits in `W`.
3. Compute the table size for `n = 100, W = 1,000` and for `n = 100, W = 109`.
4. Say why this does not contradict knapsack being NP-hard.

The break-it drill

Trigger each and record the exact output or error:

1. The inner loop forward when you meant 0/1.
2. `dp[i]` with one dimension only.
3. The take branch without the `weight ≤ capacity` guard.
4. Subset sum on an odd total.
5. Float weights in the table version.
6. `W = 10**9` as a table dimension.
7. The one-row version, then asking which items were taken.

Four of the seven give no error at all. Name them.

The algorithm drill

1. Name the five algorithms and the state each keeps per key.
2. Give the boundary problem with exact timestamps and the resulting count.
3. Give the sliding-window-counter formula and work one example with numbers.
4. Say what the approximation assumes, and when it is wrong.
5. Say what token bucket's two numbers mean, separately.

The sizing drill

1. Compute memory per key for all four algorithms at 1M keys, limit 1,000/hour.
2. Say which one you reject and by what factor.
3. Compute the limiter's latency as a percentage of a 20 ms service and a 2 ms service.
4. Say how many Redis instances 1M requests/s needs, and why sharding is trivial here.

The distributed drill

1. Name the three approaches and the cost of each.
2. Say why limit-divided-by-`n` fails, in one sentence about load balancers.
3. Compute the over-admission bound for 10 machines, limit 100/minute, 10 s sync.
4. Say the honest one-sentence description of the limit you are actually enforcing.

The failure drill

1. Say what fail-open costs and what fail-closed costs.
2. Give the third option and why it is the answer.
3. Say why the limiter needs a circuit breaker in front of it.
4. Say what a *slow* Redis does that a *dead* Redis does not.

The response drill

1. Give the status code and the four headers.
2. Say which header matters most, and why.
3. Say what happens without jitter.
4. Say why the headers belong on successful responses too.

The placement drill

1. Say where the limiter goes and give three reasons.
2. Say what the second, cruder layer is for.
3. Name the four things you limit on and the weakness of the IP one.
4. Say what per-request cost buys you over per-endpoint limits.

The lazy-refill drill

1. Write the refill line from memory.
 2. Say why `time.monotonic()` and not `time.time()`.
 3. Say what a backwards clock step does and what a forwards one does.
 4. Say what `EXPIRE` is for and what happens without it.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Solve the knapsack problem.* Why greedy fails with both counter-examples and their numbers, the state as a sentence, the recurrence, the complexity, the one-row collapse, and the loop direction with what the reverse would mean.
 2. *Why can't you just take the highest value-per-kilo item first?* The counter-example, the fractional-versus-0/1 distinction, and what indivisibility breaks.
 3. *Design a rate limiter for an API used by a million clients.* The boundary problem with timestamps, why not the sliding log with the memory figures, the sliding window counter or token bucket with its two numbers, the distributed choice with a stated bound, and the failure policy.
-

Before you move on

- ☐ I can give the by-value greedy counter-example with numbers.
- ☐ I can give the by-ratio greedy counter-example with numbers.
- ☐ I know which greedy is correct and for which problem.
- ☐ I can say what indivisibility breaks, in one sentence.
- ☐ I can state the knapsack state as a full sentence.
- ☐ I know what the one-dimensional version returns and why.
- ☐ I can write the recurrence with both branches and the guard.

- [] I can give the base cases and say what they mean.
- [] I know the one-row collapse and why it works.
- [] I know the inner loop runs backwards for 0/1, and can say why.
- [] I know the forward loop solves unbounded, and that neither errors.
- [] I can give time and space for both versions.
- [] I can explain pseudo-polynomial and why it is not a contradiction.
- [] I can reconstruct the chosen items and know what that costs.
- [] I can recognise subset sum and partition as knapsack.
- [] I know the odd-total shortcut.
- [] I can do the Target Sum algebra from memory.
- [] I have implemented the bitset version and timed it.
- [] I can name the five rate-limiting algorithms and their state.
- [] I can give the boundary problem with timestamps.
- [] I can give the sliding-window-counter formula and work an example.
- [] I know why the sliding log is rejected, with the memory figures.
- [] I know token bucket's two numbers do two different jobs.
- [] I know the difference between token bucket and leaky bucket.
- [] I can name the three distributed approaches and their costs.
- [] I can compute the over-admission bound.
- [] I know the failure policy is fail open with a local fallback.
- [] I know why the limiter needs its own circuit breaker.
- [] I know 429 , Retry-After , and why the retry must be jittered.
- [] I answered all three questions above out loud.

Subset sum and partition problems

DATA STRUCTURES AND ALGORITHMS

Subset sum and partition problems

SYSTEM DESIGN

Design a distributed key-value store

1 What this is, and why they ask it

Subset sum asks one question: **given a list of numbers, can you pick some of them that add up to exactly T** ? Not the biggest total, not the best value — just yes or no.

It is knapsack with the value column deleted. Yesterday's table held the best value you could reach; today's holds a single boolean, `True` if the total is reachable and `False` if it is not.

They ask it because **it is the disguise family**. Almost nobody asks “solve subset sum”. They ask *can this array be split into two equal halves*, or *what is the smallest difference between two groups*, or *how many ways can you assign plus and minus signs to reach a target* — and every one of those is subset sum after two lines of algebra. **The skill being tested is recognising it**, and a candidate who does the algebra out loud in thirty seconds is demonstrating something a candidate who starts writing a recursive brute force is not.

The other reason is that **it is the cheapest place to talk about NP-hardness honestly**. Subset sum is one of the classic NP-complete problems, and you are about to solve it with a table in $O(n \times T)$. Being able to say why those two facts do not contradict each other is a real conversation, and it is short.

By the end of this lesson you can write the boolean table, collapse it to one row, recognise the four disguises, do the algebra for each, and say exactly what pseudo-polynomial means.

2 The story

The two brothers had spent eleven years not talking about the shop and then their father died and they had to.

The shop was one room, and the stock in it was the whole of what he had left. Neither of them wanted to sell it. Both of them wanted half.

So they did what their aunt suggested, which was to make two piles.

Everything came off the shelves onto the floor. The steel almirah. The two ceiling fans still in their boxes. The mixer. The four rolls of cloth. The scooter, which did not come off the shelves but which counted. The sewing machine their mother had used.

Twenty-two things in all, and beside each one a number, because the man from the next lane who dealt in second-hand goods had come and looked at everything and written down what he would pay.

The numbers added to four lakh sixteen thousand.

Half was two lakh eight.

And here is where the afternoon went. **You cannot cut a scooter in half.** The scooter was seventy-two thousand. The almirah was thirty-one. The fans were nine each. Every single thing was a whole thing, and the question was not what half was — they both knew what half was — but **whether any collection of these particular objects came to exactly that number.**

Vinod tried first. He took the scooter, then the almirah, then the mixer, and he was at a lakh and thirty-one, and he added the cloth, and he was at one lakh eighty-three, and the smallest thing left was nine thousand, which took him to one lakh ninety-two, and the next smallest was twenty-two, which took him past.

So he put the mixer back and started again.

By six o'clock they had been at it for three hours and neither of them had a system. They were guessing, and each guess was one arrangement out of a number of arrangements neither of them could have said.

The aunt, who had watched all of it, made the only useful observation of the day. **“You keep asking whether you can get to two lakh eight. Ask something smaller first. Can you get to a thousand? Can you get to two?”**

Vinod said that was a stupid question, because obviously they could not, and then he sat down, because he had understood what she meant.

3 The idea in plain English

The aunt has just described the DP table.

The question “can I hit exactly T ?” is hard to attack directly, and easy to attack if you already know the answer for every smaller total.

So build up. Ask, for every total from 0 to T , whether that total is reachable — and answer them in order, smallest first, using only items you have looked at so far.

The state, said as a full sentence, is: $dp[i][t]$ is `True` if some subset of the first i numbers adds up to exactly t .

That sentence is the whole lesson. Everything else is mechanics.

The recurrence has two ways to reach t using the first i numbers, exactly as in knapsack:

- **Skip number i .** Then t must already have been reachable using the first $i - 1$. That is $dp[i-1][t]$.
- **Take number i .** Then before taking it you must have been at $t - \text{nums}[i]$, using the first $i - 1$. That is $dp[i-1][t - \text{nums}[i]]$, and it only makes sense when $\text{nums}[i] \leq t$.

```
dp[i][t] = dp[i-1][t] OR dp[i-1][t - nums[i]]
```

One **OR** instead of yesterday's **max**. That is the only difference between subset sum and knapsack. Same table, same two branches, same guard — a boolean where a value used to be.

The base cases. $dp[0][0] = \text{True}$: the empty subset adds to zero, always, and this is the case people forget. $dp[0][t] = \text{False}$ for every $t > 0$: with no numbers you cannot reach a positive total.

And the same one-row collapse as yesterday, for the same reason: row i reads only row $i - 1$, so keep one row and update it in place, **backwards**.

```
for number in nums:
    for t in range(target, number - 1, -1):
        reachable[t] = reachable[t] or reachable[t - number]
```

Backwards, because $\text{reachable}[t - \text{number}]$ must still hold the previous row's answer — the answer from before this number was available. Forwards would let the same number be used twice, which is a different problem, and tomorrow's.

Now the four disguises, which is the reason this topic is asked.

Disguise one: equal partition. *Can the array be split into two subsets with equal sums?* If the total is S , each half must be $S / 2$, so it is subset sum with $T = S / 2$. **And if S is odd, the answer is False immediately** — you cannot split an odd number into two equal integers. That check costs one line and it is the first thing to say out loud.

Disguise two: minimum difference. *Split into two groups so the difference between their sums is as small as possible.* If one group sums to a , the other sums to $S - a$, so the difference is $|S - 2a|$. To make that small, **make a as close to $S / 2$ as pos-**

sible without exceeding it. So: run subset sum up to $S / 2$, find the largest reachable a , and the answer is $S - 2a$. **Same table, read differently** — you scan the finished row instead of reading one cell.

Disguise three: target sum. Assign $+$ or $-$ to every number to reach $target$. Let P be the numbers you make positive and N the ones you make negative. Then:

$$\begin{aligned} P - N &= target \\ P + N &= total \end{aligned} \quad (\text{every number is in one group or the other})$$

Add them: $2P = target + total$, so $P = (target + total) / 2$. Count the subsets summing to P .

Two conditions make it zero immediately: if $target + total$ is odd, or if $|target| > total$. Say both.

And this one counts rather than decides, so the table holds an integer count and the **OR** becomes $+$:

```
ways[t] += ways[t - number]
```

Disguise four: last stone weight II. *Repeatedly smash two stones together; the difference survives.* Each smash is assigning a sign, so the final stone is $|sum\ of\ one\ group - sum\ of\ the\ other|$, minimised. **That is disguise two.** It is the best-hidden of the four and it is worth doing immediately after disguise three so the shape is obvious.

Finally, the honest complexity conversation. The table is $n \times T$, so the running time is $O(n \times T)$, which looks polynomial. **It is not.** T is a *number* in the input, and a number of b bits can be as large as 2^b . So the running time is exponential in the *size* of the input, which is what “polynomial” is measured against. **That is what pseudo-polynomial means**, and it is why subset sum can be NP-complete and still have this table.

Practically: $T = 10,000$ is instant, $T = 10^9$ is impossible, and **the size of the numbers decides whether this approach exists at all**. Check it before you write anything.

4 The picture

The table filling in, for `nums = [3, 34, 4, 12, 5, 2]` and `target = 9`:

	t = 0	1	2	3	4	5	6	7	8	9	
{}	T	F	F	F	F	F	F	F	F	F	empty subset makes 0
+3	T	F	F	T	F	F	F	F	F	F	3 now reachable
+34	T	F	F	T	F	F	F	F	F	F	34 > 9, changes nothing
+4	T	F	F	T	T	F	F	T	F	F	4, and 3+4=7
+12	T	F	F	T	T	F	F	T	F	F	12 > 9, nothing
+5	T	F	F	T	T	T	F	T	T	T	5, 3+5=8, 4+5=9 ← found
+2	T	F	T	T	T	T	T	T	T	T	

Notice: each row copies the row above, then turns on t and t+number for every t that was already True. Nothing ever turns OFF.

The one-row version, and why it runs backwards:

adding number = 5 to the row [T F F T T F F T F F]
 0 1 2 3 4 5 6 7 8 9

BACKWARDS t = 9, 8, 7, ... 5

t=9: reachable[9] $\models$ reachable[4] $\rightarrow$ reachable[4] is T (from before 5) $\rightarrow$ T

t=8: reachable[8] $\models$ reachable[3] $\rightarrow$ T

t=5: reachable[5] $\models$ reachable[0] $\rightarrow$ T

every read is from a cell 5 to the LEFT, which this pass has not touched yet.
 $\rightarrow$ each number used at most once. 0/1.

FORWARDS t = 5, 6, 7, ... 9

t=5: reachable[5] $\models$ reachable[0] $\rightarrow$ T

t=10: reachable[10] $\models$ reachable[5] $\rightarrow$ reachable[5] was just set by THIS pass
 $\rightarrow$ 5 used twice. That is the unbounded problem, not this one.

The two brothers' problem, drawn as the recurrence:

can I make 208000 from {scooter 72000, almirah 31000, ...} ?

can I make 208000?			
/		\	
skip scooter		take scooter	
can I make 208000		can I make 136000	
from the rest?		from the rest?	
/	\	/	\
skip	take	skip	take
almirah	almirah	almirah	almirah
208000	177000	136000	105000

The tree is 2^{22} leaves. The table is 22×208001 cells.
 Both explore the same space; the table just never asks the same question twice.

Minimum difference, read off the finished row:

```
nums = [1, 6, 11, 5], total = 23, half = 11
```

finished reachable row up to 11:

```
t:  0  1  2  3  4  5  6  7  8  9 10 11
    T  T  F  F  F  T  T  T  F  F  F  T
                                     ^
```

largest reachable $a \leq 11$ is $a = 11$

difference = total - 2a = 23 - 22 = 1

Same table. You scan it instead of indexing it.

5 The code, built step by step

The plain table, straight from the sentence

```
def subset_sum_table(nums: list[int], target: int) → bool:
    n = len(nums)
    dp = [[False] * (target + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = True
    return dp[n][target]
# empty subset makes 0
# (loops next)
```

`dp[i][0] = True` for every `i` is the base case, and it is the one people miss. Zero is always reachable, whatever numbers you have, because you can take none of them.

Now the two branches:

```
for i in range(1, n + 1):
    number = nums[i - 1]
    for t in range(1, target + 1):
        dp[i][t] = dp[i - 1][t]
        if number ≤ t:
            dp[i][t] = dp[i][t] or dp[i - 1][t - number]
# skip
```

`number ≤ t` is the guard, and without it `t - number` is negative and Python silently indexes from the end of the row. No error, wrong answer — the same failure as yesterday's missing capacity check.

The one-row version, which is what you write in an interview

```
def subset_sum(nums: list[int], target: int) → bool:
    if target < 0:
        return False
    reachable = [False] * (target + 1)
    reachable[0] = True
    for number in nums:
        for t in range(target, number - 1, -1):
            if reachable[t - number]:
                reachable[t] = True
    return reachable[target]
```

Three lines shorter than the table and the same answer. The `range(target, number - 1, -1)` stops at `number` rather than `0`, because below `number` nothing can change — and that also removes the need for the guard, since `t ≥ number` is now guaranteed by the range itself.

Say the direction out loud while you write it. “Backwards, so each number is used at most once.”

The early exit that matters

```
for number in nums:
    for t in range(target, number - 1, -1):
        if reachable[t - number]:
            reachable[t] = True
    if reachable[target]:
        return True                                # stop as soon as it is found
```

Worth adding when the target is reachable early, and it costs one line. It does not change the worst case.

Equal partition, which is two lines on top

```
def can_partition(nums: list[int]) → bool:
    total = sum(nums)
    if total % 2 == 1:
        return False                                # odd totals never split evenly
    return subset_sum(nums, total // 2)
```

The odd check is the whole trick, and it is the first sentence of the answer.

Minimum difference, which scans instead of indexing

```
def minimum_difference(nums: list[int]) → int:
    total = sum(nums)
    half = total // 2
    reachable = [False] * (half + 1)
    reachable[0] = True
    for number in nums:
        for t in range(half, number - 1, -1):
            if reachable[t - number]:
                reachable[t] = True
    for a in range(half, -1, -1):          # largest reachable a ≤ half
        if reachable[a]:
            return total - 2 * a
    return total
```

The **scan runs downwards** so the first `True` it meets is the largest reachable subset sum at or below half, which is the closest you can get to an even split from below.

Target sum, which counts instead of deciding

```
def find_target_sum_ways(nums: list[int], target: int) → int:
    total = sum(nums)
    if abs(target) > total or (target + total) % 2 == 1:
        return 0                      # unreachable, or not an integer
    positive = (target + total) // 2
    ways = [0] * (positive + 1)
    ways[0] = 1                       # one way to make 0: take nothing
    for number in nums:
        for t in range(positive, number - 1, -1):
            ways[t] += ways[t - number]
    return ways[positive]
```

`ways[0] = 1`, not `True` — there is exactly one empty subset, and that `1` is what every count is ultimately built from. And `+=` where the boolean version had `or`. Everything else is identical.

The guard `abs(target) > total` matters, because without it `positive` can be negative and `[0] * (negative)` gives an empty list rather than an error.

The bitset version, which is worth knowing

```
def subset_sum_bitset(nums: list[int], target: int) → bool:
    reachable = 1                      # bit 0 set: total 0 is reachable
    for number in nums:
        reachable |= reachable << number
    return (reachable >> target) & 1 == 1
```

Six lines, and it is the same algorithm. Each bit position is a total; shifting left by `number` is “every reachable total, plus this number”; `=` merges that with what was already reachable.

Python integers are arbitrary-precision, so this works for any target, and the shift moves 64 bits per machine word rather than one boolean per loop iteration — **roughly a 30–60× speedup** on large inputs. Mention it as an optimisation; write the loop version first.

The complete solution

```
"""Subset sum and its four disguises."""

def subset_sum(nums: list[int], target: int) → bool:
    """True if some subset of nums adds up to exactly target."""
    if target < 0:
        return False
    reachable = [False] * (target + 1)
    reachable[0] = True # empty subset makes 0
    for number in nums:
        # backwards: each number is used at most once
        for t in range(target, number - 1, -1):
            if reachable[t - number]:
                reachable[t] = True
        if reachable[target]:
            return True
    return reachable[target]

def subset_sum_with_items(nums: list[int], target: int) → list[int] | None:
    """The subset itself, or None. Keeps every row, so O(n x target) space."""
    n = len(nums)
    dp = [[False] * (target + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = True
    for i in range(1, n + 1):
        number = nums[i - 1]
        for t in range(1, target + 1):
            dp[i][t] = dp[i - 1][t]
            if number ≤ t and dp[i - 1][t - number]:
                dp[i][t] = True

    if not dp[n][target]:
        return None

    chosen, t = [], target
    for i in range(n, 0, -1):
        if not dp[i - 1][t]:
            chosen.append(nums[i - 1]) # could not have skipped it
            t -= nums[i - 1]
    return chosen[::-1]

def can_partition(nums: list[int]) → bool:
    """Disguise 1: split into two subsets with equal sums."""
    total = sum(nums)
    if total % 2 == 1:
        return False
    return subset_sum(nums, total // 2)

def minimum_difference(nums: list[int]) → int:
    """Disguise 2: smallest possible difference between two groups."""
    total = sum(nums)
    half = total // 2
    reachable = [False] * (half + 1)
    reachable[0] = True
    for number in nums:
        for t in range(half, number - 1, -1):
```

```

        if reachable[t - number]:
            reachable[t] = True
    for a in range(half, -1, -1):
        if reachable[a]:
            return total - 2 * a
    return total

def find_target_sum_ways(nums: list[int], target: int) → int:
    """Disguise 3: count the +/- assignments reaching target."""
    total = sum(nums)
    if abs(target) > total or (target + total) % 2 == 1:
        return 0
    positive = (target + total) // 2
    ways = [0] * (positive + 1)
    ways[0] = 1 # exactly one empty subset
    for number in nums:
        for t in range(positive, number - 1, -1):
            ways[t] += ways[t - number]
    return ways[positive]

def last_stone_weight_ii(stones: list[int]) → int:
    """Disguise 4: it is minimum_difference, wearing a hat."""
    return minimum_difference(stones)

def subset_sum_bitset(nums: list[int], target: int) → bool:
    """The same algorithm, 30-60x faster, using Python's big integers."""
    reachable = 1
    for number in nums:
        reachable |= reachable << number
    return (reachable >> target) & 1 == 1

if __name__ == "__main__":
    nums = [3, 34, 4, 12, 5, 2]
    print("reach 9      :", subset_sum(nums, 9))
    print("reach 30     :", subset_sum(nums, 30))
    print("which items  :", subset_sum_with_items(nums, 9))
    print("bitset agrees:", subset_sum_bitset(nums, 9), subset_sum_bitset(nums, 30))

    print("partition [1,5,11,5]:", can_partition([1, 5, 11, 5]))
    print("partition [1,2,3,5] :", can_partition([1, 2, 3, 5]))

    print("min diff [1,6,11,5] :", minimum_difference([1, 6, 11, 5]))
    print("stones [2,7,4,1,8,1]:", last_stone_weight_ii([2, 7, 4, 1, 8, 1]))

    print("target sum [1,1,1,1,1] → 3:", find_target_sum_ways([1, 1, 1, 1, 1], 3))
    print("target sum [1] → 2      :", find_target_sum_ways([1], 2))

```

Run it and you get:


```

reach 9      : True
reach 30     : False
which items  : [4, 5]
bitset agrees: True False
partition [1,5,11,5]: True
partition [1,2,3,5] : False
min diff [1,6,11,5] : 1
stones [2,7,4,1,8,1]: 1
target sum [1,1,1,1,1] → 3: 5
target sum [1] → 2      : 0

```

which items returning [4, 5] is the walk-back working. Those two numbers do add to 9, and note it did not find [3, 4, 2], which also adds to 9 — the walk-back returns a valid subset, not all of them, and which one depends on the order the table was filled.

reach 30 is False, which is worth checking by hand: the numbers total 60, and no combination of [3, 34, 4, 12, 5, 2] lands on 30 exactly. And target sum [1] → 2 gives 0, caught by the `abs(target) > total` guard rather than by a crash.

6 What it costs

Time. Two nested loops.

```

outer loop: once per number          n iterations
inner loop: once per total from target down  target iterations

n numbers x target totals = n x target cells
each cell: one comparison and one assignment  O(1)

TOTAL: O(n x target)

```

Concretely, for `n = 200` numbers and `target = 10,000`:

```

200 x 10,000 = 2,000,000 cells
Python does roughly 10 million simple loop steps per second
→ about 0.2-0.5 seconds. Fine.

```

And for `target = 10^9`:

```

200 x 1,000,000,000 = 200,000,000,000 cells
→ about six thousand years, and the list allocation fails first:

MemoryError

```

The size of the target decides whether this algorithm exists. Check it before writing.

Space.

```
full table      (n + 1) x (target + 1) booleans
                201 x 10,001 = 2,010,201 booleans
                Python bools in a list of lists: ~8 bytes per pointer
                → about 16 MB

one row         target + 1 booleans
                10,001 → about 80 KB

                200x less, same answer.
```

Use the one row unless you need to reconstruct the subset, which needs every row.

The bitset version, measured:

```
n = 200, values up to 1,000, target = 100,000

loop version      200 x 100,000 = 20,000,000 Python-level steps    ~4 s
bitset version    200 shifts of a 100,000-bit integer
                  = 200 x (100,000 / 64) ~ 312,500 word operations ~0.05 s

roughly 80x faster, and the same answer.
```

Why it is faster is worth one sentence: the loop moves one boolean per Python instruction; the shift moves sixty-four bits per machine instruction, in C.

And the pseudo-polynomial point, arithmetically:

```
input size (bits) = n numbers x bits per number
                  = 200 x 30 bits = 6,000 bits

running time      = n x target
                  = 200 x 2^30      = 200 billion
```

The input is 6,000 bits. The running time is 2^{30} .
Exponential in the SIZE of the input, polynomial in its VALUE.
That is pseudo-polynomial, and it is why NP-completeness and this table are both true at once.

7 The traps

The forward loop, which solves a different problem in silence. Write `range(number, target + 1)` instead of `range(target, number - 1, -1)` and each number can be reused any number of times.

```
>>> reachable = [False] * 11
>>> reachable[0] = True
>>> for t in range(3, 11):          # forwards, number = 3
...     if reachable[t - 3]:
...         reachable[t] = True
>>> [i for i, r in enumerate(reachable) if r]
[0, 3, 6, 9]
```

One number, 3, and it has reached 6 and 9. No error, no warning, and `subset_sum([3], 9)` now returns `True` when it should return `False`. This is the single most common bug in the topic, and the only defence is to say the direction out loud as you write it.

Forgetting `reachable[0] = True`.

```
>>> reachable = [False] * 11          # no base case
>>> for number in [3, 4, 5]:
...     for t in range(10, number - 1, -1):
...         if reachable[t - number]:
...             reachable[t] = True
>>> reachable[9]
False
```

Everything is `False` forever, because nothing was ever `True` to build from. The table fills correctly and answers `False` to every question. It looks like “no subset exists” rather than like a bug.

Negative targets, and Python’s helpful indexing. Without the guard:

```
>>> reachable = [False, True, False, True]
>>> t, number = 1, 3
>>> reachable[t - number]          # reachable[-2]
False
```

`reachable[-2]` is a perfectly legal read from the end of the list. No `IndexError` — that is what makes it dangerous. The `range(target, number - 1, -1)` form removes the risk structurally, which is why it is the version to write.

Forgetting the odd-total check on partition. It is not a correctness bug — `subset_sum(nums, total // 2)` with an odd total will simply fail to find anything — but it is a **wasted $O(n \times \text{total})$ pass** on an input you could have rejected in one comparison, and the interviewer is listening for it.

Floats.

```
>>> reachable = [False] * 11
>>> reachable[0] = True
>>> for t in range(10, int(2.5) - 1, -1):
...     reachable[t - 2.5]
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
TypeError: list indices must be integers or slices, not float
```

The table is indexed by the total, so the totals must be integers. If the input has decimals, multiply everything by 100 first and say so.

Target sum without the parity guard.

```
>>> nums, target = [1, 2, 3], 1
>>> total = 6
>>> positive = (target + total) // 2          # (1 + 6) // 2 = 3
>>> # but 1 + 6 = 7 is odd → no integer split exists
```

`7 // 2` is `3`, silently, and the function returns a count for the wrong sub-problem. **Check $(\text{target} + \text{total}) \% 2 == 1$ first**, and say why: `P` must be an integer, and it cannot be if `target + total` is odd.

Building the table when the target is enormous.

```
>>> reachable = [False] * (10**9 + 1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
MemoryError
```

Eight gigabytes for the list of pointers alone. The moment you see `1 ≤ nums[i] ≤ 109` in the constraints, the table is off the menu and you say so before writing.

Reconstructing from the one-row version. You cannot. The row has been overwritten `n` times and holds no history at all — there is no walk-back to do. **If they ask which items, you need the full table**, and saying “the space collapse costs me the reconstruction” is the right way to raise it.

8 In the interview

How it gets asked

- “Can this array be split into two subsets with equal sums?” — LeetCode 416, the most common form.
- “Split the array into two groups so the difference of their sums is minimised.”
- “Assign a plus or minus to each number so the total is `target`. How many ways?” — LeetCode 494.
- “You have stones; smash two together and the difference survives. What is the smallest possible last stone?” — LeetCode 1049, the best disguise.
- “Subset sum is NP-complete. You just solved it in `$O(n \times T)$` . Explain.”

The first ninety seconds

“This is subset sum. Let me say the reduction first, because the algorithm is the same for all of these.

If the whole array sums to S , then two equal halves each sum to $S / 2$ — so the question ‘can it be split evenly’ is exactly ‘is there a subset adding up to $S / 2$ ’.

And if S is odd, the answer is no, immediately, because you cannot split an odd integer into two equal integers. That is one comparison and it saves the whole computation.

The state is: $dp[t]$ is true if some subset of the numbers I have seen so far adds up to exactly t . One boolean per total from zero to $S / 2$.

The base case is $dp[0] = \text{True}$ — the empty subset makes zero, always, and that is the one everything else is built from.

For each number, I ask: which totals become reachable now? A total t is reachable if it was already reachable without this number, or if $t - \text{number}$ was reachable before this number. So it is $dp[t] = dp[t]$ or $dp[t - \text{number}]$ — **the same two branches as knapsack, with an OR where the max was.**

And I run the inner loop backwards, from $S / 2$ down to the number. That matters: $dp[t - \text{number}]$ has to still hold the answer from *before* this number was available. If I ran forwards I would read a cell this same pass had already updated, and the number would be usable twice — which is the unbounded problem, not this one. **No error either way, just a wrong answer**, so I state the direction as I write it.

Cost is $O(n \times S/2)$ time and $O(S)$ space, one row. For a couple of hundred numbers summing to twenty thousand that is two million operations — instant.

One thing I would check first: how large can the numbers be? If they go up to a billion the table is eight gigabytes and this approach does not exist.”

The follow-ups

“Why does the inner loop run backwards?”

“Because of what the cell I am reading has to mean.

The one-row version is a squashed two-row version. `dp[t] = dp[t] or dp[t - number]` — and both of those reads are supposed to come from the **previous** row, the state of things before this number was available.

Running backwards, from the target down to the number, every read is from a cell to the left, and this pass has not touched anything to the left yet. So `dp[t - number]` still holds the previous row’s value. Correct.

Running forwards, `dp[t - number]` may already have been updated by this same pass — meaning it already counts this number once. Adding the number again uses it twice.

Concretely: one number, `3`, target `9`, forwards. `dp[3]` becomes true from `dp[0]`. Then `dp[6]` becomes true from `dp[3]`, which I just set. Then `dp[9]` from `dp[6]`. **I have made 9 out of a single 3.**

And that is not a bug in general — it is the correct algorithm for the unbounded problem, where items can be reused. Two loop directions, two different problems, both correct programs, no error to distinguish them. Which is why I say the direction out loud when I write it.”

“Now count the number of ways instead of just yes or no.”

“Almost the same code. The table holds an integer count instead of a boolean, the `or` becomes `+=`, and the base case becomes `ways[0] = 1` instead of `True`.

`ways[0] = 1` is the meaningful change: there is exactly one way to make zero, which is to take nothing, and every other count is built by adding that one up.

```
ways[t] += ways[t - number]
```

Same backwards loop, same reason.

The version they usually ask is Target Sum: assign a plus or minus to every number to reach `target`. That needs two lines of algebra first. Call `P` the numbers I make positive and `N` the ones I make negative. Then `P - N = target`, and `P + N = total`, because every number is in exactly one group. Add those: `2P = target + total`, so `P = (target + total) / 2`.

So it is: count the subsets summing to `P`. One subset-sum-counting pass.

And two things make it zero straight away — if `|target| > total` it is unreachable, and if `target + total` is odd then `P` is not an integer, so no assignment exists. **Both need stating, because without the parity check the integer division silently rounds and I answer a different question.**

“Subset sum is NP-complete. You just solved it in `O(n × T)`. Which is it?”

“Both, and they do not conflict — the resolution is what ‘polynomial’ is measured against.

Complexity is measured against the number of bits in the input, not the numerical values in it.

The input here is n numbers. If each is up to T , writing them down takes about $n \times \log_2(T)$ bits. So the input size is $n \log T$.

My running time is $n \times T$, which is $n \times 2^{(\log T)}$ — exponential in $\log T$, and $\log T$ is the input size. So $O(n \times T)$ is exponential in the size of the input, even though it looks polynomial. **That is what pseudo-polynomial means.**

The practical version is more useful: **the algorithm is fast when the numbers are small and impossible when they are large.** Two hundred numbers summing to ten thousand is two million cells, instant. Two hundred numbers up to a billion is two hundred billion cells and eight gigabytes of memory before the first comparison.

So the constraint I look for is the bound on the values, not on n — which is the reverse of most problems, and it is why I ask about it before writing anything.

And if the numbers are large there is no exact polynomial algorithm — that is what NP-completeness is telling me. What exists is approximation: scale the values down, accept an answer within a fixed percentage of optimal. Worth naming as the fallback.”

The model answer

“Given a list of stones, you repeatedly pick two and smash them; if they are equal both are destroyed, and if not the difference remains. Return the smallest possible weight of the last stone.”

“Let me work out what the smashing actually does, because the algorithm falls out of that and not out of simulating it.

Every smash takes two stones a and b and leaves $|a - b|$. That is the same as writing $a - b$, or $b - a$ — I am choosing a sign. And if I keep smashing, the final stone is some expression where every original stone appears exactly once with either a plus or a minus in front of it.

So: assign a $+$ or a $-$ to every stone, and minimise the absolute value of the total.

Which means splitting the stones into two groups and minimising the difference between their sums. The plus group and the minus group.

Now the algebra. If the whole set sums to S and one group sums to a , the other sums to $S - a$, and the difference is $|S - 2a|$. To make that as small as possible I want a as close to $S / 2$ as I can get.

And ‘which totals near $S / 2$ are actually achievable’ is subset sum.

So: build the reachable table up to $S / 2$. $dp[0] = \text{True}$ for the empty subset, then for each stone, going backwards from $S / 2$ down to the stone’s weight, mark t reachable if $t - \text{stone}$ was. Then scan the finished row downwards from $S / 2$ and take the first reachable a . **The answer is $S - 2a$.**

Scanning downwards matters — the first True I meet going down is the largest achievable total at or below half, which is the closest split from below. And I only need to search up to $S / 2$ because the two groups are symmetric: every split below half has a mirror above it.

Cost: $O(n \times S/2)$ time, $O(S)$ space. With a hundred stones of at most a hundred each, S is at most ten thousand, so five thousand columns and a hundred rows — half a million operations. Instant.

The backwards loop is the part I would say out loud while writing it, because each stone can be used once. Forwards would let a stone appear in the group twice, which silently solves a different problem.

Two edge cases: one stone, where the answer is that stone; and an even split existing, where $a = S / 2$ and the answer is zero.

And the thing I would check before writing anything is the bound on the stone weights. This works because S is at most ten thousand. If the weights could be a billion, the table is eight gigabytes and I would need a different

approach — which is worth saying, because it is the constraint that decides whether this algorithm exists at all.”

9 Recall card

Subset sum is knapsack with the value column deleted: `dp[t]` is `True` if some subset of the numbers seen so far adds to exactly `t`. Same two branches, **or** **where the max was**, base case `dp[0] = True` — the empty subset makes zero.

One row, inner loop backwards, `range(target, number - 1, -1)`, so `dp[t - number]` still holds the value from before this number existed. **Forwards solves the unbounded problem instead, with no error either way** — say the direction out loud as you write it.

Four disguises, all the same table. Equal partition = subset sum to `S/2`, **odd S fails immediately**. Minimum difference = scan the finished row downwards for the largest reachable `a ≤ S/2`, answer `S - 2a`. Target sum = `P = (target + total)/2` by algebra, count instead of decide (`ways[0] = 1`, `+=`), zero if `target + total` is odd or `|target| > total`. Last Stone Weight II = minimum difference.

`O(n × T)` **time**, `O(T)` **space** — 200 numbers to target 10,000 is 2M cells and instant; target 10^9 is a `MemoryError` before the first comparison. **The bound on the values, not on `n`, decides whether this approach exists.**

Pseudo-polynomial: input size is `n log T` bits, running time is `n × T = n × 2(log T)` — exponential in the input size, which is how NP-completeness and this table are both true. **Bitset trick:** `reachable |= reachable << number`, roughly 80× faster for free.

Design a distributed key-value store

1 What this is, and why they ask it

A key-value store does two things: `put(key, value)` and `get(key)`. That is the whole interface.

Designing one across a thousand machines is the hardest thing in this course, and it is asked constantly, because it is the one design where every distributed-systems idea has to be present at once and none of them can be waved at. Partitioning, replication, consistency, failure detection, conflict resolution, and membership — all of them, in one system, interacting.

They ask it because **it has no diagram to memorise**. “Design Twitter” has a shape you can recognise. This has decisions, and each one has a defensible answer in both directions, so the interviewer can push on any of them and see whether you understood or recited.

And they ask it because the reference design is public and famous. **Amazon’s Dynamo paper is the single most influential systems paper of the last twenty years** — Cassandra, Riak, Voldemort and DynamoDB all descend from it — so the interviewer usually has a specific set of mechanisms in mind: consistent hashing, `N`, `R` and `W` quorums, vector clocks, hinted handoff, Merkle trees, gossip. **Naming them is expected.**

By the end of this lesson you can partition with consistent hashing and virtual nodes, replicate with a tunable quorum, state the consistency guarantee precisely, resolve conflicts, handle node failure and repair, and size the whole thing.

2 The story

The lending library was Sushila’s idea and by the third year it had outgrown her flat entirely.

It had started as a shelf. Then two shelves. Then the whole of one wall, and then a woman on the fourth floor offered to keep some, and after that it was in six flats across three buildings and about eleven hundred books.

And the moment it left one room, the problem changed completely.

Because when it was a shelf, everything was simple. You wanted a book, you looked. It was there or it was not.

Now, you wanted a book, and the first question was **which flat**.

Sushila's answer was the alphabet. A to E in her flat, F to K on the fourth floor, and so on. It worked beautifully and it worked for about four months, until somebody pointed out that she had four hundred books and the woman with T to Z had sixty-one. **The letters are not evenly spread and nobody had thought about it.**

Then the second problem, which was worse. **The fourth-floor family went to Pune for a month and locked the flat.** Two hundred books, gone, not lost but unreachable, and there was no way to know which two hundred until somebody wanted one.

So they started keeping **two copies of the popular ones, in two different flats.** Which solved the locked door and created the thing that took a year to get right.

Because a woman on the second floor borrowed one copy, and wrote her name in the register in that flat. And somebody else borrowed the other copy from the other flat, and wrote *their* name in *that* register. **And both registers were correct. They just did not agree.**

Nobody had done anything wrong. There was no lying and no mistake. There were two books and two registers and one title, and when the two registers were finally compared, in March, **there was no way at all to work out which entry had come first**, because neither register recorded the time and the two clocks in question were two different people's memories.

Sushila's rule, when she finally made one, was not about preventing it. It was that when the registers disagreed, both names got kept, and whoever collected the books went and asked.

3 The idea in plain English

Sushila built Dynamo. Badly, over three years, in the order the problems arrived — which is the order they arrive in every distributed store.

Start with the interface, because it is small and that is the point.

```
put(key, value)    → stored
get(key)           → value
delete(key)
```

No joins, no transactions across keys, no queries by value, no ordering. That poverty is what buys the scale: a store that cannot ask “all users in Delhi” never needs to talk to more than a handful of machines for any one request.

Problem one: which machine holds this key?

The obvious answer is $\text{hash}(\text{key}) \% \text{number_of_machines}$. **And it is catastrophic**, because adding one machine changes the modulus and therefore changes the answer for almost every key. Going from 10 machines to 11 remaps roughly **91% of all keys** — every one of which then has to move.

Consistent hashing fixes that. Imagine the hash output as a circle, 0 to 2^{128} . Every machine is placed at a point on the circle, by hashing its name. Every key is placed on the circle too, by hashing the key. **A key belongs to the first machine you meet going clockwise from where the key landed.**

Now adding a machine moves only the keys between it and its predecessor — roughly $1/n$ of the data, not 91% of it. That is the entire idea, and it is the one thing to be able to draw.

But raw consistent hashing is uneven, exactly like Sushila’s alphabet. Hash ten machine names onto a circle and the arcs between them will not be equal; one machine ends up with three times the data of another.

Virtual nodes fix that. Each physical machine is placed on the circle **many times** — 100 to 256 positions each, by hashing `machine-1#0`, `machine-1#1`, and so on. With a thousand points instead of ten, the arcs average out and every machine gets close to its fair share. **And when a machine dies, its hundreds of small arcs are inherited by many different machines rather than all landing on one neighbour**, which is the second, less obvious benefit.

Problem two: a machine will die, so keep copies.

Replication factor N , usually 3. A key is stored on the machine that owns it plus the next $N - 1$ distinct physical machines clockwise. That list is the key’s **preference list**.

And it must skip virtual nodes belonging to a machine already in the list, or three replicas can land on one machine and the whole point is lost. **In practice it also skips racks and availability zones**, so a single rack losing power does not take all three copies.

Problem three: how many replicas must answer?

This is the tunable part, and it is what the interview is usually about.

N = how many copies exist
W = how many must acknowledge a write before it is called successful
R = how many must respond to a read before you answer

If $R + W > N$, the read set and the write set must overlap in at least one machine, so at least one machine that answers the read has the newest write. That is a **strong-consistency quorum**, and it is a counting argument, not a protocol — worth saying exactly that way.

N=3, W=2, R=2	R + W = 4 > 3	overlap guaranteed. The default.
N=3, W=3, R=1		fast reads, and a single slow node blocks every write.
N=3, W=1, R=1		fastest possible, and $R + W = 2 < 3$: you can read stale.

W = 1 is not a mistake — it is the right choice when writes must never block and staleness is acceptable, and being able to say when that is true is the answer they want.

Problem four: two writes to the same key, and no agreed time.

This is Sushila's two registers, and [day 123](#) explains why it cannot be solved with timestamps: **there is no global clock, and the two machines' clocks can disagree by more than the gap between the two writes.**

Three answers, in increasing order of honesty.

Last-write-wins, using wall-clock timestamps. Simple, and it silently discards a write whenever the clocks are skewed. **Cassandra defaults to this**, and the data loss is real and invisible.

Vector clocks. Each value carries a small map of `machine → counter`. When machine A writes, it increments its own counter. Comparing two vectors gives one of three results: **A is strictly newer, B is strictly newer, or neither** — and “neither” means concurrent, which is exactly Sushila's case. **The store cannot decide, so it keeps both** and returns both on the next read.

Application-level merge, which is what “keeps both” leads to. The classic example is Amazon's shopping cart: two concurrent adds produce two carts, and the merge is **union of the items** — a customer sees an item they thought they removed, which is annoying, and never loses an item they added, which matters more.

Sushila's rule — keep both names and go and ask — is exactly this, and it is the honest design.

Problem five: a node is down. Now what?

Hinted handoff. If a replica is unreachable, write to the next healthy machine clockwise, tagged with a hint saying who it was really for. When the real owner returns, the hint is delivered and the temporary copy deleted. **This keeps writes available during a failure** and is why Dynamo-style stores are described as always-writeable.

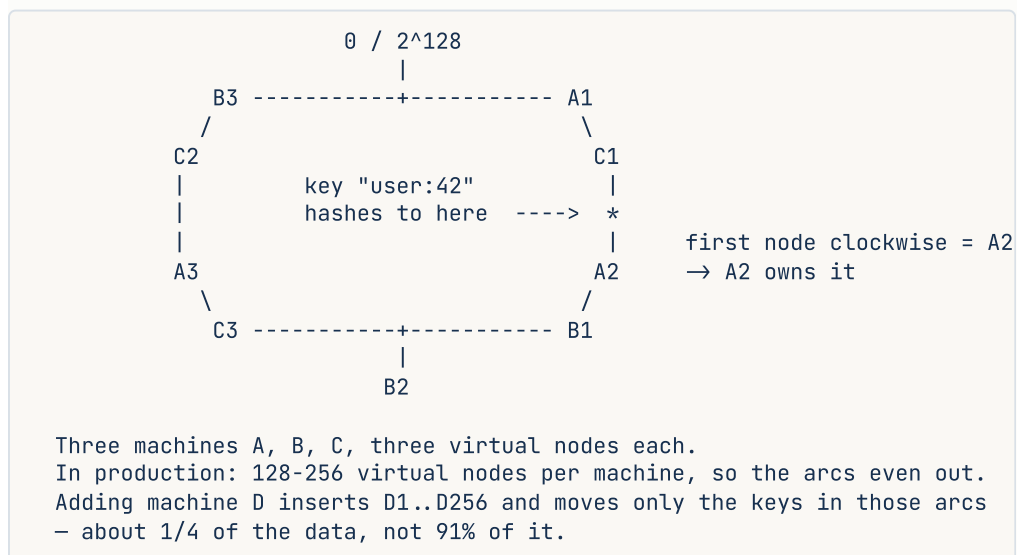
Anti-entropy with Merkle trees, for the divergence hinted handoff misses. Each replica builds a hash tree over its key range. Two replicas compare root hashes — **equal means identical, one comparison for a million keys.** Unequal, and they descend into the differing subtree only. **Logarithmic repair instead of a full scan,** and that saving is the reason the structure is used.

Read repair, the cheap one: when a read finds replicas disagreeing, write the newest value back to the stale ones. **Free, because the read already fetched everything,** and it fixes the keys people actually read.

Gossip for membership. No central registry. Each node periodically picks a random peer and exchanges what it knows about who is alive. **Information spreads in $O(\log n)$ rounds** and there is no coordinator to lose. Day 124's failure detection is what each node runs locally; gossip is how the conclusions travel.

4 The picture

The hash ring, with virtual nodes:



Why plain modulo is unusable:


```

hash(key) % 10 → machine 0..9
add one machine
hash(key) % 11 → machine 0..10

a key with hash 1234:  1234 % 10 = 4      1234 % 11 = 2      MOVED
a key with hash 5678:  5678 % 10 = 8      5678 % 11 = 5      MOVED
a key with hash 1000:  1000 % 10 = 0      1000 % 11 = 10     MOVED

~91% of all keys move. Every one is a network transfer.
Consistent hashing: ~9% move. That is the whole reason it exists.

```

The preference list and a quorum write:

```

key "cart:99" hashes into A's arc.
preference list, N=3: [A, B, C] (next 3 DISTINCT physical machines)

put("cart:99", v2), W=2:

    coordinator (any node) → A ... ack
                           → B ... ack ← 2 acks, W satisfied
                           → C ... slow / down
    return SUCCESS to the client immediately.

C is repaired later by read repair or anti-entropy.
If C is DOWN, the write goes to D with a hint "this belongs to C".

```

The quorum overlap argument, drawn:

```

N = 3 replicas:      [ A ][ B ][ C ]

W = 2 → a write is on at least 2 of them:  [ A ][ B ]
R = 2 → a read asks at least 2 of them:    [ B ][ C ]
                                           ^
                                           B is in both sets. It has the new value.

R + W = 4 > 3 ⇒ the sets MUST share at least one machine.
                Pure counting. There is no protocol here.

R + W ≤ N ⇒ they can be disjoint ⇒ a stale read is possible.

```

Vector clocks producing a conflict:

client writes v1 via node A → value v1, clock {A:1}
network partitions

client X writes via A → value v2, clock {A:2}
client Y writes via B → value v3, clock {A:1, B:1}

partition heals. Compare {A:2} and {A:1, B:1}:

is {A:2} ≥ {A:1,B:1}? A: $2 \geq 1$ yes. B: $0 \geq 1$ NO.
is {A:1,B:1} ≥ {A:2}? A: $1 \geq 2$ NO.

Neither dominates → CONCURRENT → keep BOTH.

The next read returns [v2, v3] and the application merges them.

This is the two registers. Neither is wrong; they disagree.

Merkle trees making repair cheap:



compare roots: differ → descend
compare H(ab): EQUAL → skip this entire half, half a million keys
compare H(cd): differ → descend
compare H(c): equal → skip
compare H(d)/H(X): differ → transfer just this key

1 differing key out of 1,000,000 found in ~20 comparisons.

A full scan would be 1,000,000 comparisons and the whole dataset over the network.

5 How it actually works

Consistent hashing with virtual nodes

```
import bisect, hashlib

class Ring:
    def __init__(self, vnodes: int = 256) → None:
        self.vnodes = vnodes
        self.points: list[int] = []          # sorted hash positions
        self.owner: dict[int, str] = {}      # position → physical machine

    def _hash(self, s: str) → int:
        return int(hashlib.md5(s.encode()).hexdigest(), 16)

    def add_node(self, node: str) → None:
        for i in range(self.vnodes):
            point = self._hash(f"{node}#{i}")
            bisect.insort(self.points, point)
            self.owner[point] = node
```

`bisect.insort` keeps the positions sorted, which is what makes lookup a binary search — the same `bisect` from [day 038](#). The ring is just a sorted list of integers.

Now the lookup, which is the whole point:

```
def preference_list(self, key: str, n: int) → list[str]:
    """The n distinct physical machines that should hold this key."""
    index = bisect.bisect(self.points, self._hash(key))
    result: list[str] = []
    for offset in range(len(self.points)):
        point = self.points[(index + offset) % len(self.points)]
        node = self.owner[point]
        if node not in result:          # skip vnodes of a machine already listed
            result.append(node)
        if len(result) == n:
            break
    return result
```

`if node not in result` is the line that matters. Without it, three consecutive virtual nodes belonging to machine A give you three “replicas” all on machine A, and the replication factor is a lie. In production the same check also excludes racks and availability zones.

`bisect.bisect` then wrapping with `% len` is the circle — past the last point you come back to the first.

The coordinator, and a quorum write

```
def put(ring, key: str, value, n: int, w: int) → bool:  
    replicas = ring.preference_list(key, n)  
    acks = 0  
    for node in replicas:  
        if send_write(node, key, value):      # fire in parallel in reality  
            acks += 1  
        if acks ≥ w:  
            return True                        # do NOT wait for the rest  
    return False
```

Returning at `w` acks and not waiting for the rest is the latency win, and it is the whole reason quorums are tunable. With `N = 3, W = 2` you are waiting for the second-fastest of three replicas, **not the slowest** — which removes the one slow machine from the critical path, and one machine in three being slow is normal.

In reality the writes go out in parallel and the coordinator waits on the first `W` responses.

The quorum read, and read repair

```
def get(ring, key: str, n: int, r: int) → list:
    replicas = ring.preference_list(key, n)
    responses = [send_read(node, key) for node in replicas[r]]
    versions = reconcile(responses)           # vector-clock comparison
    if len(versions) == 1:
        repair_state(replicas, key, versions[0])   # read repair, in the background
    return versions                           # more than one = conflict, caller merges
```

`get` returns a list, not a value, and that is the API decision that follows from vector clocks. Most of the time it has one element. When it has two, **the caller must merge**, and the API forces them to notice rather than silently picking one.

Read repair happens on the read path and costs nothing extra, because the values were already fetched.

Vector clocks

```
def dominates(a: dict[str, int], b: dict[str, int]) → bool:
    """True if a is strictly at least as new as b on every node."""
    return all(a.get(node, 0) ≥ count for node, count in b.items())

def reconcile(versions: list) → list:
    keep = []
    for v in versions:
        if any(dominates(other.clock, v.clock) and other.clock ≠ v.clock
              for other in versions):
            continue # superseded by a strictly newer version
        keep.append(v)
    return keep # >1 means genuinely concurrent
```

dominates is the whole comparison — every counter in **b** must be matched or exceeded in **a**. If neither dominates the other, they are concurrent and both survive.

Vector clocks grow, one entry per node that has ever written the key, so production systems truncate the oldest entries past about ten. **Truncation can produce false conflicts** — two values that were actually ordered look concurrent — which is safe, because a false conflict causes an unnecessary merge rather than lost data.

Hinted handoff

```
def write_with_hint(ring, key, value, n, w):
    for node in ring.preference_list(key, n):
        if is_alive(node):
            send_write(node, key, value)
        else:
            fallback = next_healthy_after(ring, node)
            send_write(fallback, key, value, hint_for=node) # hold it for them
```

The fallback stores it in a separate hint area, not as its own data, and hands it over when the real owner returns. **Writes stay available through a node failure**, which is the guarantee Dynamo is built around.

And it fails if the fallback also dies before delivering — which is what anti-entropy is for.

The real systems

DynamoDB (AWS)	managed, single-digit-ms, strong OR eventual reads per request
Cassandra	Dynamo partitioning + a wide-column model, tunable per query
Riak	the closest open-source Dynamo, vector clocks exposed
Voldemort	LinkedIn's; largely historical now
etcd / ZooKeeper	NOT this design — Raft, strongly consistent, small data, for configuration and coordination, not for bulk storage

Naming etcd as the counter-example is worth doing, because it shows you know these are different tools for different jobs rather than competitors.

The storage engine on each node

```
writes → in-memory memtable (sorted)
        → also appended to a write-ahead log (crash recovery)
        → when full, flushed to disk as an immutable SSTable
        → SSTables compacted in the background

reads  → memtable, then SSTables newest-first
        → a BLOOM FILTER per SSTable answers "definitely not here" in
          one memory access, skipping the disk read entirely
```

This is an LSM tree, and it is the standard choice because writes become sequential appends rather than random disk seeks. The Bloom filter is [day 143's](#), doing exactly the job it was introduced for.

6 The numbers

Sizing the cluster.

```
1 billion keys
average value 1 KB
replication factor N = 3
```

raw data	1,000,000,000 x 1 KB	= 1 TB
replicated	1 TB x 3	= 3 TB
+ overhead	indexes, Bloom filters, ~20%	= 3.6 TB
+ headroom	never run past ~60% full	= 6 TB provisioned

```
per machine: 2 TB of SSD usable
→ 6 TB / 2 TB = 3 machines minimum on storage
```

```
but availability wants 3 AZs and headroom for failure:
→ 12 machines, 4 per AZ, is the realistic floor
```

Storage rarely sets the machine count. Throughput and failure tolerance do, and saying that is worth more than the division.

Throughput.

```
100,000 reads/s, 20,000 writes/s
```

each write with N=3 becomes 3 physical writes	= 60,000 disk writes/s
each read with R=2 becomes 2 physical reads	= 200,000 reads/s

```
per machine (SSD, LSM engine): ~20,000 ops/s sustained
→ (200,000 + 60,000) / 20,000 = 13 machines
```

```
matches the availability floor. 15 machines, comfortably.
```

Latency, and where the quorum choice shows up.

single replica read	~1 ms	(memory or SSD)
network round trip within an AZ	~0.5 ms	
network round trip across AZs	~1-2 ms	
R=1 : wait for 1 of 3	→ ~1.5 ms	fastest, can be stale
R=2 : wait for 2 of 3	→ ~2.5 ms	the second-fastest replica
R=3 : wait for all 3	→ ~8 ms	the SLOWEST replica, every time

R = 3 means the p99 of every read is the p99 of the worst of three machines, which is dramatically worse than the p99 of one. **That is the concrete argument for R = 2**, and it is a better answer than “quorums are a good balance”.

The consistent hashing win, quantified.

1 billion keys, 10 machines, growing to 11

modulo:	~91% of keys move	= 910,000,000 keys
	at 1 KB each	= 910 GB over the network
	at 1 Gbps	= ~2 hours of saturated network, during which everything is slow

consistent hashing:	~1/11 of keys move	= 91,000,000 keys
	= 91 GB	= ~12 minutes

10x less data moved, and it is spread across all machines rather than being one machine's problem.

Virtual node balance.

10 machines, 1 virtual node each:

arcs are random; worst machine typically holds 2-3x the average.
→ one machine at 300 GB while another holds 100 GB

10 machines, 256 virtual nodes each (2,560 points):

the law of large numbers applies
→ worst machine within about 10% of the average

More virtual nodes cost ring metadata, which every node gossips: $2,560 \text{ points} \times \sim 50 \text{ bytes} = 128 \text{ KB}$. **Trivial**, which is why 256 is a free choice.

Merkle tree repair.

Consistency against availability, which is the whole design. [Day 114](#)'s CAP result says that during a network partition you choose. **Dynamo chooses availability:** both sides of a partition keep accepting writes, and the divergence is repaired afterwards, sometimes by the application. **A strongly consistent store like etcd chooses consistency:** the minority side stops serving writes entirely.

Neither is better. They are for different data. A shopping cart should accept writes during a partition — a customer unable to add an item is a lost sale, and a duplicated item is an annoyance. **A bank ledger should not.** Say which one the data is before choosing.

Tunable quorums are a real knob, not marketing. $N = 3, W = 1, R = 1$ is a fast, eventually consistent store. $W = 2, R = 2$ guarantees overlap. $W = 3$ makes reads fast at the cost of a write path that any single slow machine can block. **The same cluster can serve both, per request,** and that flexibility is genuinely Dynamo's best feature.

Last-write-wins against vector clocks. LWW is one timestamp, trivially simple, and it **silently loses writes when clocks are skewed** — a machine whose clock is two seconds fast wins every conflict it is in, and nothing reports it. Vector clocks never lose a write and **push the merge onto the application**, which most teams do not want to write. **Cassandra chose LWW and is enormously popular**, which tells you what most teams actually pick.

Virtual nodes cost ring metadata and buy balance, and the trade is so lopsided — 128 KB against a machine holding three times its share — that there is no real decision here. **Say the number and move on.**

Read repair against anti-entropy. Read repair is free and only fixes what is read; a key nobody reads stays divergent forever, which matters when it is finally read after a year. Anti-entropy fixes everything and **costs continuous background I/O and network**. Run both.

Eventual consistency's real cost is the application code, and this is the honest thing to say. `get` returning a list means every caller handles conflict. Most developers will not, so they take the first element and the store's careful conflict preservation is thrown away at the call site. **If the team will not write merge logic, use a strongly consistent store and accept the availability cost.**

When would I not build this? Almost always. **A single Postgres instance handles a billion keys at ten thousand operations a second** and gives you transactions, secondary indexes and joins for free. This design earns its complexity above roughly a terabyte with a global footprint and a hard availability requirement — and **below that, reaching for it is the most expensive mistake in this course**. And if you genuinely need it, **DynamoDB exists and is managed**, and choosing to operate your own needs a reason beyond preference.

8 In the interview

How it gets asked

- “Design a key-value store like DynamoDB.” — the standard prompt.
- “How do you decide which machine holds a key?” — consistent hashing, and why not modulo.
- “What is $R + W > N$ and why does it work?” — the counting argument.
- “Two clients write the same key at the same time. What happens?” — conflict resolution.
- “A node has been down for an hour and comes back. How does it catch up?”
- “Is it CP or AP?”

“The interface is two operations, `put` and `get`, and I want to start by saying what it deliberately cannot do: **no joins, no cross-key transactions, no queries by value**. That poverty is what buys the scale — every request touches a handful of machines and never the whole cluster.

Four decisions, and I will take them in the order the problems arrive.

First, placement. I would not use `hash(key) % machines`, because going from ten machines to eleven changes the modulus and remaps about **ninety-one percent of all keys** — a billion keys is 910 GB moving across the network while the cluster is degraded. **Consistent hashing** instead: hash the key onto a circle, hash each machine onto the same circle, and a key belongs to the first machine clockwise. **Adding a machine now moves only about one-eleventh of the data.**

And virtual nodes on top, 256 per machine, because ten random points on a circle give badly uneven arcs — one machine ends up with three times its share. With 2,560 points the arcs even out to within about ten percent. **The second benefit is that a dead machine’s load spreads across many survivors instead of landing entirely on its clockwise neighbour.**

Second, replication. $N = 3$ — the owning machine plus the next two **distinct physical machines** clockwise, skipping virtual nodes of a machine already in the list, and spread across three availability zones. That list is the key’s preference list.

Third, the quorum, which is the interesting decision. W replicas must acknowledge a write, R must answer a read. **If $R + W > N$ the two sets cannot be disjoint, so a read always touches at least one machine holding the newest write.** That is pure counting, not a protocol. **I would default to $N=3, W=2, R=2$** — and the reason $R=2$ rather than $R=3$ is latency: $R=3$ means every read waits for the slowest of three machines, so the p99 becomes the p99 of the worst replica.

Fourth, conflicts, which the quorum does not solve. Two clients writing concurrently produce two values and **there is no global clock to order them**. I would use vector clocks, keep both versions, and return both on read for the application to merge — the shopping-cart approach.

The main question I would ask first: what kind of data is this? If it is carts or sessions or user profiles, this design is right. **If it is money, I would not build this at all** — I would use something strongly consistent and accept the availabil-

ity cost.”

The follow-ups

“Explain $R + W > N$. Why does it guarantee anything?”

“It is a counting argument about overlapping sets, and there is no protocol involved at all — which is worth saying, because it sounds like there should be.

There are N replicas. A successful write has landed on at least W of them. A read asks at least R of them.

If $R + W > N$, those two sets cannot be disjoint. With $N = 3$, $W = 2$, $R = 2$: the write is on two machines, the read asks two machines, and there are only three machines. Two plus two is four, and four things cannot fit in three boxes without a collision. **So at least one machine that answers my read has the new value.**

And that gives me the newest value only because I can recognise it — the versions carry vector clocks, so among the responses I can tell which dominates. Without versioning, overlap alone would just give me two values and no way to choose.

If $R + W \leq N$ the sets can be disjoint. $W = 1$, $R = 1$ on three replicas: the write went to A, the read asked C, and C has never heard of it. **A stale read, guaranteed possible** — and that is a legitimate configuration when writes must never block and staleness is fine.

What it does not give me is linearizability. It bounds staleness for a *completed* write. **A write in flight — acknowledged by one replica, not yet by the second — can be read by one client and not another**, so two concurrent readers can disagree. That is why Dynamo says eventual consistency rather than strong, even at $W=2$, $R=2$.

And there is a failure mode people forget: if two of three replicas are down, $W = 2$ cannot be satisfied and **writes fail**. A sloppy quorum takes the acks from any two healthy machines with hinted handoff instead, which keeps writes available and weakens the overlap guarantee — the availability choice again, made explicit.”

“Two clients write the same key at the same time. What happens?”

“The store cannot order them, and the honest design admits that rather than pretending.

Why it cannot: there is no global clock. Client X writes through node A, client Y writes through node B, and A and B have clocks that can differ by more than the gap between the writes. **Ordering by timestamp would be picking a winner based on clock skew.**

The simple answer is last-write-wins — attach a wall-clock timestamp, highest wins. Cassandra defaults to this. **It silently loses one of the two writes**, and worse, a machine whose clock runs two seconds fast wins every conflict it participates in, forever, and nothing reports it.

What I would use is vector clocks. Each value carries a map of node to counter; the node handling a write increments its own entry. Comparing two clocks gives one of three answers: **A strictly dominates B, B strictly dominates A, or neither** — and neither means they are genuinely concurrent.

When they are concurrent, the store keeps both, and the next `get` returns both. **So `get` returns a list**, not a value, which is an API decision that forces the caller to notice.

The application merges. For a shopping cart, the merge is the union of items — a customer might see something they removed, which is annoying, and never loses something they added, which is what matters. **For a counter, union is wrong**, and you would use a CRDT or a strongly consistent store instead.

Two honest costs. Vector clocks grow, one entry per node that has ever written the key, so production systems truncate past about ten entries — **truncation can create false conflicts**, which is safe, since a false conflict means an unnecessary merge rather than lost data.

And the real cost is that most developers will not write the merge. They will take the first element of the list, and everything the store did to preserve both versions is discarded at the call site. **If the team will not write merge logic, vector clocks buy nothing and I would use last-write-wins knowingly, or a strongly consistent store.**“

“A node has been down for an hour. It comes back. How does it catch up?”

“Three mechanisms, and they cover different gaps, so I would run all three.

During the outage: hinted handoff. When a write’s target replica is unreachable, the coordinator writes to the next healthy machine clockwise with a hint saying who it was really for. That machine stores it in a separate hint area, not as its own data. **When the node returns, the hints are delivered and deleted.** This is why the store stays writeable through a failure, and it handles most of the hour cleanly.

On the read path: read repair. When a read at `R = 2` gets two different versions back, the coordinator writes the newer one to the stale replica. **This is free** — the values were already fetched for the read — **and it only fixes keys people actually read.** A key nobody reads stays wrong until someone reads it, which is the exact moment you least want to discover it.

In the background: anti-entropy with Merkle trees, which is the one that guarantees convergence. Each replica maintains a hash tree over its key range. Two replicas compare root hashes — **equal means the ranges are identical, one comparison for a million keys.** If they differ, they descend into only the differing subtree.

The saving is enormous and worth quantifying. One differing key in a million: a full comparison would exchange a million hashes, about 20 MB. The tree finds it in about twenty comparisons — four hundred bytes. **Fifty thousand times less traffic,** which is what makes it viable to run continuously.

And the case none of them handle: an hour is fine, a week is not. Hints expire, and a node that has been gone long enough has to be treated as a new node — wiped and streamed a fresh copy of its ranges from its peers. **Most systems have a configured window, typically a few hours, past which a returning node is rebuilt rather than repaired.**“

The model answer

“Design a distributed key-value store: one billion keys, a hundred thousand reads and twenty thousand writes a second, five nines of availability, deployed in three availability zones.”

“Five nines is the number that decides everything here — twenty-six seconds of downtime a year — so I am building an AP system and I will say where that hurts.

Interface: `put`, `get`, `delete`. **No queries by value, no cross-key transactions.** That is what makes the rest possible.

Partitioning: consistent hashing with 256 virtual nodes per machine. A key hashes onto a circle and belongs to the first machine clockwise. **Not modulo** — going from ten machines to eleven would remap ninety percent of a billion keys, which is 910 GB of network transfer during exactly the period the cluster is already stressed. **Consistent hashing moves about a ninth of that**, and virtual nodes keep the per-machine load within about ten percent of even rather than letting one machine hold three times its share.

Replication: `N = 3`, **one replica per availability zone**, using the next three distinct physical machines clockwise. **The distinctness check is the easy thing to get wrong** — consecutive virtual nodes can belong to the same machine, and then the replication factor is a fiction.

Quorum: `W = 2`, `R = 2` **by default, tunable per request.** `R + W = 4 > 3`, so the read and write sets must overlap. `R = 2` **rather than 3 because `R = 3` makes every read wait for the slowest of three machines** — the p99 goes from about 2.5 milliseconds to about 8. **And I would expose the knob:** a session read can use `R = 1` for speed; something that must not be stale can use `R = 3`.

Sizing. A billion keys at 1 KB is 1 TB, times three replicas is 3 TB, plus twenty percent overhead and never running past sixty percent full is about 6 TB provisioned. **But storage is not what sets the machine count.** Throughput does: 100,000 reads at `R=2` is 200,000 physical reads, 20,000 writes at `N=3` is 60,000 physical writes, and an LSM-based node sustains about 20,000 operations a second — so thirteen machines, and I would run fifteen across three AZs.

Storage engine per node: an LSM tree — memtable, write-ahead log, immutable SSTables, background compaction — because writes become sequential appends instead of random seeks. **A Bloom filter per SSTable** answers ‘definitely not here’ in one memory access and skips the disk read entirely.

Conflicts: vector clocks, both versions kept, `get` **returns a list.** I would be explicit with the team that this pushes merge logic into the application, and that if they will not write it, the design should change rather than the guarantee being quietly abandoned at the call site.

Failure: hinted handoff during the outage, read repair on the read path, Merkle-tree anti-entropy in the background. And a window — a few hours — past which a returning node is rebuilt from scratch rather than repaired.

Membership: gossip. Each node exchanges its view with a random peer every second; information reaches everyone in about $\log n$ rounds and there is no coordinator to lose. **A central registry would be a single point of failure in a system whose entire point is not having one.**

What five nines actually costs me, stated plainly: during a network partition both sides keep accepting writes, so the data diverges and the application sees conflicts. **I get availability and I pay in application complexity.** If this were payment data I would build the opposite system — Raft, strongly consistent, unavailable in the minority partition — and accept four nines instead.

And the honest closing point: at a billion keys and 120,000 operations a second this design is justified. **Below about a terabyte, a single Postgres instance does this with transactions and secondary indexes for free,** and choosing this instead would be the most expensive kind of mistake.”

9 Recall card

Consistent hashing, not modulo: modulo remaps $\sim 91\%$ of keys when 10 machines becomes 11; the ring moves $\sim 1/n$. **256 virtual nodes per machine** evens the arcs to within $\sim 10\%$ and spreads a dead machine's load across many survivors. **The preference list must skip virtual nodes of a machine already in it** (and skip AZs).

$R + W > N$ is a counting argument, not a protocol — the read set and write set cannot be disjoint, so one responding replica has the newest write. **$N=3$, $W=2$, $R=2$ by default;** **$R=3$** makes every read wait for the slowest of three replicas (p99 ~ 2.5 ms $\rightarrow \sim 8$ ms). It bounds staleness for completed writes; **it is not linearizability.**

No global clock means concurrent writes cannot be ordered. Last-write-wins is one timestamp and silently loses writes under clock skew (Cassandra's default). **Vector clocks keep both versions, `get` returns a list, the application merges** — cart = union. Truncate past ~ 10 entries; false conflicts are safe.

Three repair mechanisms, all of them: hinted handoff keeps writes available during a failure; read repair is free but only fixes what is read; **Merkle-tree anti-entropy** finds 1 differing key in 1,000,000 in ~ 20 comparisons instead of 20 MB of hashes. **Gossip for membership,** no central registry.

Per node: an LSM tree (memtable → WAL → SSTables → compaction) with a **Bloom filter per SSTable**. **Dynamo is AP by choice** — both sides of a partition accept writes. **And below ~1 TB, one Postgres does this with transactions and indexes for free**; reaching for this design too early is the expensive mistake.

DSA topic: Subset sum and partition problems **System design topic:** Design a distributed key-value store

Code these, in this order

One rule for the whole set: **before writing a line, say out loud what `T` is and where it came from.** Every problem here is subset sum; the only work is the algebra that produces the target. Do that algebra out loud first, every time, and the code takes four minutes.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Partition Equal Subset Sum	LeetCode 416 (Medium)	The reduction to $T = S/2$, and the odd-total exit.
2	Partition Array Into Two Arrays to Minimize Sum Difference	LeetCode 2035 (Hard)	The same table, scanned instead of indexed.
3	Target Sum	LeetCode 494 (Medium)	Two lines of algebra, and counting instead of deciding.
4	Last Stone Weight II	LeetCode 1049 (Medium)	The best-hidden disguise in the set.
5	Partition to K Equal Sum Subsets	LeetCode 698 (Medium)	Where subset sum stops working and backtracking starts.
6	Ones and Zeroes	LeetCode 474 (Medium)	Two targets at once — the same idea in three dimensions.

On problem 1, write the odd check before anything else

One line, before the table. Then run it on an array of a thousand odd-ish numbers and record how long the answer takes with and without that line. **Say in one sentence why the check is not a correctness fix.**

On problem 1, reverse the inner loop and watch nothing happen

Change `range(target, number - 1, -1)` to `range(number, target + 1)`. Run on `[3]` with `target = 9`. Record the answer. **Then say which problem you just solved instead**, and how you would catch this without running it.

On problem 3, do the algebra on the board first

Write out $P - N = \text{target}$, $P + N = \text{total}$, and derive $P = (\text{target} + \text{total}) / 2$. Then write the two conditions that make the answer zero. Then code.

Now delete the parity check and run on `nums = [1, 2, 3]`, `target = 1`. Record what it returns. **Say why the wrong answer is plausible rather than obviously broken.**

On problem 4, describe the smash before you code

In one sentence, say what a smash is in terms of signs. Then say what the final stone equals in terms of two group sums. **Then notice you have written problem 2's statement.** Solve it by calling your problem 2 function.

On problem 5, find where the reduction breaks

Try to solve it as subset sum with $T = S / k$. Run it on `[4, 3, 2, 3, 5, 2, 1]` with `k = 4`. Say in one sentence why finding one subset summing to S/k is not enough, and what has to change.

On problem 6, state it as a sentence and count

Write the state as a full sentence with both resources named. Count the states for `m = 100`, `n = 100`, 600 strings. Say what the three loop directions must be and why.

Then the bitset drill

Rewrite problem 1 with `reachable |= reachable << number`. Verify agreement with your table version on twenty random inputs. Then time both at `n = 200`, values up to 1,000, and record the ratio.

The reduction drill

For each, state `T` and the reason, in one sentence:

1. Equal partition.
2. Minimum difference.
3. Target sum.
4. Last stone weight II.
5. A subset summing to exactly `T`.

Then say which two of those five are the same problem.

The direction drill

1. Say what `range(target, number - 1, -1)` guarantees about what each read holds.
2. Say what the forward loop solves instead.
3. Say what the error message is. (There isn't one — say that, and say what you do about it.)
4. Give the one-number example that proves the forward loop wrong.

The base-case drill

1. Say why `reachable[0] = True`, in words about subsets.
2. Say what the whole table does if you leave it out, and what it looks like from outside.
3. Say what the counting version's base case is and why it is `1`.

The pseudo-polynomial drill

1. Give the running time and the space.
2. Give the input size in bits for `n = 200` numbers of 30 bits.
3. Say why $O(n \times T)$ is exponential in that.
4. Compute the memory for `T = 109` and give the exact error.
5. Say which constraint you check first in an interview, and why it is the reverse of usual.

The break-it drill

Trigger each and record the exact output or error:

1. The forward inner loop.
2. A missing `reachable[0] = True`.
3. A negative index from an unguarded `t - number`.
4. A float in the input.
5. `[False] * (10**9 + 1)`.
6. Target Sum without the parity guard.
7. Reconstruction from the one-row version.

Five of the seven give no error at all. Name them.

The partitioning drill

1. Say why `hash(key) % n` is unusable, with the percentage.
2. Describe the ring in three sentences.
3. Say what virtual nodes fix, and give both benefits.
4. Compute the data moved by both schemes for 1 billion keys, 10 machines to 11.
5. Say what the preference list must skip, and what breaks if it does not.

The quorum drill

1. State `N`, `R` and `W` in one sentence each.
2. Give the `R + W > N` argument as counting, not protocol.
3. Give three configurations and what each is for.
4. Say why `R = 2` and not `R = 3`, with the latency numbers.
5. Say what `R + W > N` does *not* give you.
6. Say what happens to writes when two of three replicas are down, and what a sloppy quorum does about it.

The conflict drill

1. Say why timestamps cannot order two concurrent writes.
2. Give last-write-wins and its exact failure mode.
3. Explain a vector clock comparison and its three outcomes.
4. Work the `{A:2}` versus `{A:1,B:1}` example out loud.
5. Say what `get` returns and why that is an API decision.
6. Give the cart merge, and one case where union is wrong.
7. Say what truncation costs and why it is safe.

The repair drill

1. Name the three mechanisms and what each covers.
2. Say what hinted handoff buys and when it fails.
3. Say what read repair misses, and why that is the worst case.
4. Explain a Merkle tree comparison and quantify the saving.
5. Say what happens to a node that has been down for a week.

The sizing drill

1. Do the storage estimate for 1 billion keys at 1 KB, `N = 3`.
2. Do the throughput estimate and say which one sets the machine count.

3. Give the three read latencies for $R = 1, 2, 3$.
4. Compute the vector clock overhead on a 1 KB value and a 50 B value.
5. Give the Merkle saving as a ratio.

The when-not-to drill

1. Say what a single Postgres handles, with numbers.
 2. Say at what point this design earns its complexity.
 3. Say why etcd is not this design, and what it is for.
 4. Say the honest cost of eventual consistency, in terms of the application code.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Can this array be split into two halves with equal sums?* The reduction, the odd-total exit, the state as a sentence, the base case, the backwards loop with its reason, and the complexity with the target-size caveat.
 2. *Subset sum is NP-complete and you solved it in $O(n \times T)$. Explain.* Input size in bits, $n \times 2^{(\log T)}$, the practical version, which constraint you check, and what exists when the numbers are large.
 3. *Design a key-value store like DynamoDB.* Consistent hashing with the 91% number, virtual nodes, $N = 3$ with the distinctness check, $R + W > N$ as counting, vector clocks with both versions kept, and the three repair mechanisms.
-

Before you move on

- [] I can state the subset sum state as a full sentence.
- [] I know the base case is `reachable[0] = True` and why.
- [] I know what a missing base case looks like from outside.
- [] I run the inner loop backwards and can say why.
- [] I know the forward loop solves the unbounded problem, silently.
- [] I can give the one-number example that proves it.
- [] I can reduce equal partition and know the odd-total exit.
- [] I can reduce minimum difference and scan the row correctly.
- [] I can do the Target Sum algebra from memory.

- [] I know both conditions that make Target Sum zero.
- [] I know the counting version uses `ways[0] = 1` and `+=`.
- [] I recognised Last Stone Weight II as minimum difference.
- [] I know where the reduction stops working, at `k` subsets.
- [] I can give time and space for both versions.
- [] I can explain pseudo-polynomial with the bit count.
- [] I check the bound on the values before writing anything.
- [] I have implemented and timed the bitset version.
- [] I know reconstruction needs the full table.
- [] I can explain why modulo hashing is unusable, with the number.
- [] I can describe the ring and virtual nodes, and both benefits.
- [] I know the preference list must skip same-machine vnodes and AZs.
- [] I can give `R + W > N` as a counting argument.
- [] I know why `R = 2` and not `R = 3`, with latencies.
- [] I know what `R + W > N` does not guarantee.
- [] I can explain why timestamps cannot order concurrent writes.
- [] I can compare two vector clocks and get the right answer.
- [] I know `get` returns a list and why.
- [] I can name the three repair mechanisms and what each covers.
- [] I can quantify the Merkle tree saving.
- [] I can say when I would not build this at all.
- [] I answered all three questions above out loud.

Unbounded knapsack and coin change

DATA STRUCTURES AND ALGORITHMS

Unbounded knapsack and coin change

SYSTEM DESIGN

Design a distributed unique ID generator

Unbounded knapsack and coin change

1 What this is, and why they ask it

Unbounded knapsack is yesterday's problem with one word changed: **you may take each item as many times as you like.**

That one word changes exactly one character in the code. The inner loop runs forwards instead of backwards. Everything else — the state, the recurrence, the base case, the complexity — is identical.

They ask it because **that single character is the cleanest test in dynamic programming of whether you understood the loop or memorised it.** A candidate who says “forwards, because `dp[t - coin]` should already include this coin” has understood. A candidate who writes it forwards and cannot say why got lucky, and the interviewer will find out within one follow-up.

The other reason is coin change, which is the most-asked DP problem after Fibonacci. **Fewest coins to make an amount** is unbounded knapsack minimising a count, and **the greedy algorithm you have used your whole life at a shop counter is wrong** — which makes it a small, sharp lesson about why DP exists at all. Indian currency happens to be one of the denomination systems where greedy works, which is exactly why the counter-example is worth having ready.

By the end of this lesson you can write both loop directions and say what each means, solve minimum-coins and count-ways, explain why greedy fails with specific numbers, and say why the counting version needs the loops in one particular order.

2 The story

The tea shop had one price for four years and then the milk went up and Chandrakant had to think about change.

Nine rupees. That was the new price, and it was the first time it had not been a round number, and by the second morning he understood what he had done to himself.

Because everyone paid with a ten, or a twenty, or a fifty, and he had to give back one rupee, or eleven, or forty-one, and **the tin he kept the change in did not refill itself.**

He started counting what was in it. Coins of one, two, five and ten. Notes above that. And by Thursday he had worked out the thing that actually mattered, which was not how much money was in the tin but **how many pieces** were in it, because a customer waiting while you count out eleven single-rupee coins is a customer who goes to the other shop.

So he began giving change the way anyone would: biggest first. Forty-one rupees, and he would put down two twenties and a one. Three pieces. Clean.

And that worked, every single time, for four years, and he never once thought about it.

Then his cousin came for a month from a place where the coins were different — they had a seven and a five and a one there, and no two — and Chandrakant, out of habit, gave someone ten with the same method. **A seven, then a one, then a one, then a one.** Four pieces.

His cousin, who had been watching without saying anything, put down two fives.

Two pieces. And the method had picked four.

Chandrakant stood there for a while with the tin open. He had used that rule every day for four years and it had never been wrong once, and it had never been wrong **because of the coins he happened to have**, not because the rule was good.

Nobody had told him the rule only worked here.

3 The idea in plain English

Chandrakant's rule is the greedy algorithm, and his cousin has just produced the counter-example.

Greedy on coins means: take the largest coin that fits, repeat. With Indian denominations — 1, 2, 5, 10, 20, 50 — it is optimal for every amount, and you can go a lifetime without discovering that this is a property of those particular numbers rather than of the method.

With coins `[1, 5, 7]` and amount 10, greedy gives `7 + 1 + 1 + 1` — four coins. The answer is `5 + 5` — two.

Say that example out loud until it is automatic. It is the whole reason this problem is DP and not one line.

Now the DP.

The state, as a full sentence: `dp[t]` is the fewest coins that make exactly `t`.

The recurrence asks one question: what was the last coin? If the last coin was `c`, then before it you had made `t - c`, and that took `dp[t - c]` coins. So:

```
dp[t] = 1 + min(dp[t - c]) over every coin c that fits
```

Try every coin as the last one and take the best. That is the entire algorithm.

The base case is `dp[0] = 0` — zero coins make zero — and everything else starts at infinity, meaning “not reachable yet”.

Infinity matters. If you initialise to zero instead, every amount looks free and the answer is zero everywhere. If you initialise to `-1`, the `min` picks `-1` and you get nonsense. Use `float('inf')`, or a sentinel larger than any possible answer, and check at the end whether it is still infinite.

Now the loop direction, which is the point of today.

Yesterday's 0/1 knapsack ran the inner loop **backwards**, so that `dp[t - w]` still held the value from before this item existed — each item usable once.

Today it runs **forwards**, so that `dp[t - c]` holds a value that **may already include this coin** — which is exactly what “use it as many times as you like” means.

```
for coin in coins:
    for t in range(coin, amount + 1):          # FORWARDS
        dp[t] = min(dp[t], dp[t - coin] + 1)
```

Same six lines as yesterday. One range reversed. A different problem. And neither version errors, so the only defence is saying the direction and its reason as you write.

Now the second question they ask: count the ways instead.

How many different combinations of coins make the amount? Same table, but the cell holds a count:

```
ways[0] = 1
for coin in coins:
    for t in range(coin, amount + 1):
        ways[t] += ways[t - coin]
```

`ways[0] = 1`, because there is exactly one way to make nothing: take no coins. Every count is built by adding that one up.

And here is the trap that catches almost everyone: the loop order matters, and it is not the direction this time — it is which loop is outside.

```
coins outside, amount inside → counts COMBINATIONS {1,2} once
amount outside, coins inside → counts PERMUTATIONS {1,2} and {2,1} separately
```

With coins `[1, 2]` and amount 3: combinations are `1+1+1` and `1+2` — **two**. Permutations are `1+1+1`, `1+2`, `2+1` — **three**.

Why: with coins outside, coin 1 is fully processed before coin 2 is ever considered, so no arrangement can ever place a 1 after a 2. The order is fixed by the loop, so each set is counted once. With amount outside, every coin is offered at every amount, so orderings are counted separately.

Both are correct programs. Coin Change II asks for combinations; Combination Sum IV asks for permutations, and its name is a lie. **Read the examples in the problem to decide which**, every time.

Finally: minimising uses `min`, counting uses `+=`, and reachability uses `or`. Three problems, one table, three operators — and that is the family this week has been building towards.

4 The picture

The two loop directions, side by side, on the same six lines:

0/1 KNAPSACK (yesterday)	UNBOUNDED (today)
<pre>for item in items: for t in range(T, w-1, -1): dp[t] = f(dp[t], dp[t-w])</pre>	<pre>for coin in coins: for t in range(coin, T+1): dp[t] = f(dp[t], dp[t-coin])</pre>
reads a cell to the LEFT that this pass has NOT touched → previous row → item used ONCE	reads a cell to the LEFT that this pass MAY HAVE already updated → this coin counted again → UNLIMITED

One range reversed. Two different problems. No error either way.

The minimum-coins table filling, `coins = [1, 5, 7]`, `amount = 10`:

```

after coin 1:  t: 0  1  2  3  4  5  6  7  8  9 10
                  0  1  2  3  4  5  6  7  8  9 10      all ones

after coin 5:  t: 0  1  2  3  4  5  6  7  8  9 10
                  0  1  2  3  4  1  2  3  4  5  2      dp[5]=1, dp[10]=2
                        ^               ^
                    5 = one coin    10 = dp[5]+1 = 2

after coin 7:  t: 0  1  2  3  4  5  6  7  8  9 10
                  0  1  2  3  4  1  2  1  2  3  2
                        ^
                    7 = one coin, better than 3

answer dp[10] = 2  (5 + 5)

Greedy would take 7 first, then 1+1+1. Four coins.
The table never commits to a first choice, so it finds 5+5.

```

Why greedy fails, drawn as the two paths:

amount 10, coins [1, 5, 7]

GREEDY

```

take 7 → 3 left
take 1 → 2 left
take 1 → 1 left
take 1 → 0 left
DONE. 4 coins.

```

OPTIMAL

```

take 5 → 5 left
take 5 → 0 left
DONE. 2 coins.

```

The greedy step "take 7" is locally the biggest reduction and globally wrong, because it leaves a remainder (3) that only 1-rupee coins can fill.

Combinations against permutations, and where they diverge:

coins [1, 2], amount 3

COINS OUTSIDE (combinations)

process coin 1 fully: ways = [1, 1, 1, 1]

process coin 2 fully: ways = [1, 1, 2, 2]

answer 2: {1,1,1} and {1,2}

Coin 2 is only ever added AFTER coin 1 is finished,
so 2-then-1 can never be formed.

AMOUNT OUTSIDE (permutations)

t=1: from coin 1 ways[1] = 1

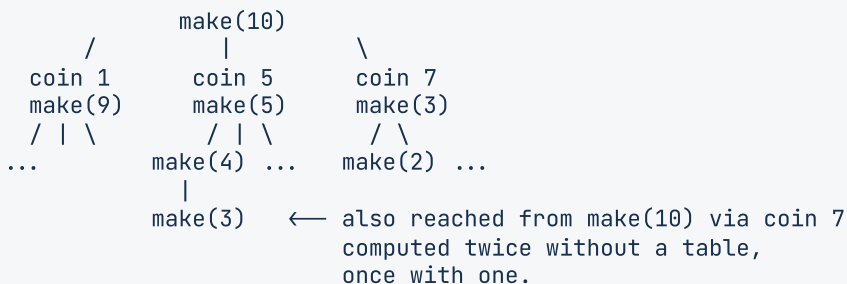
t=2: from 1 and from 2 ways[2] = ways[1] + ways[0] = 2

t=3: from 1 and from 2 ways[3] = ways[2] + ways[1] = 3

answer 3: {1,1,1}, {1,2}, {2,1}

Both coins are offered at every amount, so order counts.

The recurrence as a tree, showing what memoisation removes:



The table has 11 cells. The tree has thousands of nodes.
Same search, no repeats.

5 The code, built step by step

Minimum coins, straight from the sentence

```
def coin_change(coins: list[int], amount: int) → int:
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    return dp[amount]
```

zero coins make zero
(loops next)

`float('inf')` means “not reachable yet”, and it is what makes the `min` behave. Zero would say every amount is free; `-1` would win every `min` and poison the table.

Now the two loops:

```

for coin in coins:
    for t in range(coin, amount + 1):        # FORWARDS: coin reusable
        if dp[t - coin] + 1 < dp[t]:
            dp[t] = dp[t - coin] + 1

```

`range(coin, amount + 1)` starts at `coin`, because below that the coin cannot be used, and that also removes the need for a `t ≥ coin` guard.

Forwards is the whole lesson. `dp[t - coin]` may already have this coin counted in it, and that is exactly what unlimited supply means.

And the unreachable case:

```

return dp[amount] if dp[amount] ≠ float('inf') else -1

```

Amount 3 with coins [2] is genuinely impossible, and the problem asks for `-1`. Checking for infinity is how you know.

Counting the ways

```

def coin_change_ii(coins: list[int], amount: int) → int:
    ways = [0] * (amount + 1)
    ways[0] = 1                                # exactly one way to make nothing
    for coin in coins:                          # COINS OUTSIDE → combinations
        for t in range(coin, amount + 1):
            ways[t] += ways[t - coin]
    return ways[amount]

```

`ways[0] = 1` is the seed every count grows from. Set it to zero and the answer is zero for every amount.

Coins outside, amount inside. Swap those two lines and you count permutations instead — a different, also correct, answer to a different question.

The permutation version, for contrast

```

def combination_sum_iv(nums: list[int], target: int) → int:
    ways = [0] * (target + 1)
    ways[0] = 1
    for t in range(1, target + 1):             # AMOUNT OUTSIDE → permutations
        for number in nums:
            if number ≤ t:
                ways[t] += ways[t - number]
    return ways[target]

```

Every number is offered at every amount, so `1+2` and `2+1` are counted separately. The guard comes back because the inner loop is now over coins, not over amounts.

Unbounded knapsack, maximising value

```
def unbounded_knapsack(weights: list[int], values: list[int], capacity: int) → int:
    dp = [0] * (capacity + 1)
    for c in range(1, capacity + 1):
        for i, w in enumerate(weights):
            if w ≤ c:
                dp[c] = max(dp[c], dp[c - w] + values[i])
    return dp[capacity]
```

`dp[c] = 0` everywhere is right here, not infinity, because taking nothing is always allowed and gives zero value. Compare that with `coin_change`, where taking nothing does not reach the amount — the base case follows from the question, not from habit.

Reconstructing which coins were used

```
def coin_change_with_coins(coins: list[int], amount: int) → list[int] | None:
    dp = [float('inf')] * (amount + 1)
    choice: list[int | None] = [None] * (amount + 1)
    dp[0] = 0
    for t in range(1, amount + 1):
        for coin in coins:
            if coin ≤ t and dp[t - coin] + 1 < dp[t]:
                dp[t] = dp[t - coin] + 1
                choice[t] = coin          # remember the last coin used
    if dp[amount] == float('inf'):
        return None
    used, t = [], amount
    while t > 0:
        used.append(choice[t])
        t -= choice[t]
    return used
```

One extra array holding the last coin for each amount, and the walk-back subtracts coins until it reaches zero. `0(amount)` extra space — cheap, unlike yesterday's reconstruction which needed every row.

The BFS view, which is worth knowing

```
from collections import deque

def coin_change_bfs(coins: list[int], amount: int) → int:
    if amount == 0:
        return 0
    seen, queue, depth = {0}, deque([0]), 0
    while queue:
        depth += 1
        for _ in range(len(queue)):
            t = queue.popleft()
            for coin in coins:
                nxt = t + coin
                if nxt == amount:
                    return depth
                if nxt < amount and nxt not in seen:
                    seen.add(nxt)
                    queue.append(nxt)
    return -1
```

Minimum coins is a shortest path, where each amount is a node and each coin is an edge of weight one. [Day 131](#)'s BFS finds it, and the level counter is the coin count. **Same complexity, and it stops early** when the amount is reachable in few coins. Mention it; write the DP.

The complete solution

```
"""Unbounded knapsack: coin change, counting, and the loop-order trap."""

from collections import deque

def coin_change(coins: list[int], amount: int) → int:
    """Fewest coins making exactly amount, or -1 if impossible."""
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0 # zero coins make zero
    for coin in coins:
        # FORWARDS: dp[t - coin] may already include this coin → unlimited use
        for t in range(coin, amount + 1):
            if dp[t - coin] + 1 < dp[t]:
                dp[t] = dp[t - coin] + 1
    return int(dp[amount]) if dp[amount] != float('inf') else -1

def coin_change_with_coins(coins: list[int], amount: int) → list[int] | None:
    """The actual coins, or None. One extra 0(amount) array."""
    dp = [float('inf')] * (amount + 1)
    choice: list[int | None] = [None] * (amount + 1)
    dp[0] = 0
    for t in range(1, amount + 1):
        for coin in coins:
            if coin ≤ t and dp[t - coin] + 1 < dp[t]:
                dp[t] = dp[t - coin] + 1
                choice[t] = coin
    if dp[amount] == float('inf'):
        return None
    used: list[int] = []
    t = amount
    while t > 0:
        coin = choice[t]
        assert coin is not None
        used.append(coin)
        t -= coin
    return sorted(used)

def coin_change_ii(coins: list[int], amount: int) → int:
    """How many COMBINATIONS make amount. Coins outside."""
    ways = [0] * (amount + 1)
    ways[0] = 1 # one way to make nothing
    for coin in coins:
        for t in range(coin, amount + 1):
            ways[t] += ways[t - coin]
    return ways[amount]

def combination_sum_iv(nums: list[int], target: int) → int:
    """How many PERMUTATIONS make target. Amount outside."""
    ways = [0] * (target + 1)
    ways[0] = 1
    for t in range(1, target + 1):
        for number in nums:
            if number ≤ t:
                ways[t] += ways[t - number]
    return ways[target]
```

```

def unbounded_knapsack(weights: list[int], values: list[int], capacity: int) → int:
    """Best value, each item reusable."""
    dp = [0] * (capacity + 1)          # taking nothing gives 0, always
    for c in range(1, capacity + 1):
        for i, w in enumerate(weights):
            if w ≤ c:
                dp[c] = max(dp[c], dp[c - w] + values[i])
    return dp[capacity]

def greedy_coins(coins: list[int], amount: int) → int:
    """Biggest coin first. Correct for Indian denominations, wrong in general."""
    count, left = 0, amount
    for coin in sorted(coins, reverse=True):
        take = left // coin
        count += take
        left -= take * coin
    return count if left == 0 else -1

def coin_change_bfs(coins: list[int], amount: int) → int:
    """Minimum coins as a shortest path. Each coin is an edge of weight 1."""
    if amount == 0:
        return 0
    seen, queue, depth = {0}, deque([0]), 0
    while queue:
        depth += 1
        for _ in range(len(queue)):
            t = queue.popleft()
            for coin in coins:
                nxt = t + coin
                if nxt == amount:
                    return depth
                if nxt < amount and nxt not in seen:
                    seen.add(nxt)
                    queue.append(nxt)
    return -1

if __name__ == "__main__":
    print("min coins [1,5,7] → 10 :", coin_change([1, 5, 7], 10))
    print("greedy [1,5,7] → 10 :", greedy_coins([1, 5, 7], 10))
    print("which coins :", coin_change_with_coins([1, 5, 7], 10))
    print("bfs agrees :", coin_change_bfs([1, 5, 7], 10))

    print("min coins [1,2,5] → 11 :", coin_change([1, 2, 5], 11))
    print("greedy [1,2,5] → 11 :", greedy_coins([1, 2, 5], 11))
    print("impossible [2] → 3 :", coin_change([2], 3))

    print("combinations [1,2] → 3 :", coin_change_ii([1, 2], 3))
    print("permutations [1,2] → 3 :", combination_sum_iv([1, 2], 3))
    print("combinations [1,2,5]→5 :", coin_change_ii([1, 2, 5], 5))

    print("unbounded knapsack :", unbounded_knapsack([2, 3, 4], [4, 5, 7], 9))

```

Run it and you get:

```

min coins [1,5,7] → 10 : 2
greedy    [1,5,7] → 10 : 4
which coins      : [5, 5]
bfs agrees      : 2
min coins [1,2,5] → 11 : 3
greedy    [1,2,5] → 11 : 3
impossible [2] → 3    : -1
combinations [1,2] → 3 : 2
permutations [1,2] → 3 : 3
combinations [1,2,5]→5 : 4
unbounded knapsack      : 17

```

Line one against line two is the lesson. Same coins, same amount, and greedy is twice as expensive. **Lines five and six show why nobody noticed** — on `[1, 2, 5]`, which is how Indian change actually works, greedy and DP agree exactly.

And combinations 2 against permutations 3 on the same input is the loop-order trap, made visible.

6 What it costs

Time. The same two nested loops as yesterday.

```

outer loop: once per coin          len(coins) iterations
inner loop: once per amount from coin up  amount iterations

TOTAL: O(len(coins) x amount)

```

Concretely, for the standard LeetCode constraints — 12 coins, amount up to 10,000:

```

12 x 10,000 = 120,000 cells
each cell: one comparison, one possible assignment
→ about 0.02 seconds in Python. Instant.

```

And where it stops:

```

amount = 10^9
12 x 1,000,000,000 = 12,000,000,000 cells
→ hours, and the allocation fails first:

MemoryError

```

Pseudo-polynomial again, for exactly yesterday's reason: `amount` is a value in the input, not a size.

Space.

one row of amount + 1 cells
10,001 floats in a Python list → about 80 KB

with reconstruction: one more array of the same size → 160 KB

Reconstruction is cheap here, unlike yesterday, because you store one coin per amount rather than an entire table of rows.

Compare the three problems in the family:

	table cell	operator	base case
subset sum	boolean	or	dp[0] = True
count the ways	integer	+=	dp[0] = 1
minimum coins	integer	min	dp[0] = 0, rest inf
maximum value	integer	max	dp[c] = 0 everywhere

Same two loops every time. The operator and the base case follow from the question.

The BFS version, counted:

worst case: every amount from 0 to amount is enqueued once
each dequeue tries len(coins) edges
→ $O(\text{len}(\text{coins}) \times \text{amount})$, the same as the DP

BUT it stops at the first level that reaches the amount.

coins [1, 5000], amount 10,000:

DP	fills all 10,001 cells	= 20,002 operations
BFS	finds 5000 + 5000 at depth 2	= ~4 nodes explored

For small answers with large amounts, BFS wins enormously.
For dense answers it is the same, with more overhead.

Greedy's cost, for contrast:

sort the coins $O(k \log k)$
one division per coin $O(k)$
→ essentially free, and wrong.

The whole cost of DP here is the price of correctness on denomination systems where greedy fails.

7 The traps

The backwards loop, silently solving yesterday's problem.

```
>>> dp = [float('inf')] * 10
>>> dp[0] = 0
>>> for t in range(9, 2, -1):                # BACKWARDS by mistake
...     dp[t] = min(dp[t], dp[t - 3] + 1)
>>> dp[9]
inf
```

`inf` — meaning “impossible” — when the answer is obviously three coins of 3.

Each pass can use the coin once, so 3 is reachable and 6 and 9 are not.

The same mistake on the fuller input:

```
>>> coins, amount = [1, 5, 7], 10
>>> dp = [float('inf')] * 11
>>> dp[0] = 0
>>> for coin in coins:
...     for t in range(amount, coin - 1, -1):
...         dp[t] = min(dp[t], dp[t - coin] + 1)
>>> dp[10]
inf
```

Also `inf`, because `5 + 5` needs the coin twice and this version has forbidden that. No error, no warning, just a claim that the amount cannot be made. **And note what would have hidden it:** on `coins = [1, 2, 5]` with amount 8, where the answer `5 + 2 + 1` uses each coin once, both directions give 3 and the test passes. **Test with an input that needs a repeat**, or the bug survives.

Initialising `dp` to zero instead of infinity.

```
>>> dp = [0] * 11                # wrong sentinel
>>> for coin in [1, 5, 7]:
...     for t in range(coin, 11):
...         dp[t] = min(dp[t], dp[t - coin] + 1)
>>> dp[10]
0
```

Zero coins make ten. Every `min` compares against a zero that was never earned, so nothing ever improves on it. **The output is a plausible number**, which is what makes it dangerous.

Initialising to `-1` because the problem returns `-1`.

```
>>> dp = [-1] * 11
>>> dp[0] = 0
>>> for coin in [1, 5, 7]:
...     for t in range(coin, 11):
...         dp[t] = min(dp[t], dp[t - coin] + 1)
>>> dp[10]
-1
```

-1 beats every real answer in a `min`. Use infinity internally and convert to `-1` only at the return.

Forgetting `ways[0] = 1` in the counting version.

```
>>> ways = [0] * 4
>>> for coin in [1, 2]:
...     for t in range(coin, 4):
...         ways[t] += ways[t - coin]
>>> ways[3]
0
```

Zero ways to make three, when there are two. Nothing was ever `1`, so nothing could be added up.

The loop order, which produces a different correct answer.

```
>>> ways = [0] * 4; ways[0] = 1
>>> for t in range(1, 4):
...     for coin in [1, 2]:
...         if coin <= t:
...             ways[t] += ways[t - coin]
>>> ways[3]
3
```

Against the combinations version's `2`. **Neither is a bug. They answer different questions**, and the only way to know which one is wanted is to read the problem's examples. **LeetCode 377 is called Combination Sum IV and wants permutations**, which is worth knowing precisely because the name is misleading.

Integer overflow, in other languages. The counting version grows fast:

```
coins [1..200], amount 1000 → the count exceeds 2^63
```

Python integers are unlimited so nothing happens. **In Java or C++ this silently wraps**, and the problem usually asks for the answer modulo `10^9 + 7` precisely because of that. **Read the problem statement for a modulus** before assuming.

`float('inf')` arithmetic leaking into the answer.

```
>>> float('inf') + 1
inf
>>> int(float('inf'))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
OverflowError: cannot convert float infinity to integer
```

`inf + 1` stays `inf`, which is what makes the sentinel work. But `int()` on it raises — so check for infinity *before* converting, which is why the return line is written the way it is.

Amount zero.

```
>>> coin_change([1, 5, 7], 0)
0
```

Correct, and worth checking, because a solution that starts its loop at `t = 1` and returns `dp[amount]` handles it for free, while a recursive version without a base case for zero recurses forever.

8 In the interview

How it gets asked

- “Given coins and an amount, return the fewest coins that make it, or -1.” — LeetCode 322, the standard.
- “Why can’t you just take the largest coin first?” — the greedy question, and it always comes.
- “Now count how many different combinations make the amount.” — LeetCode 518.
- “Does the order of the loops matter?” — the trap, asked directly.
- “How is this different from yesterday’s knapsack?” — the one-character answer.
- “What if the amount is a billion?”

The first ninety seconds

“This is unbounded knapsack — each coin is available in unlimited supply — and I want to start with why it is not greedy, because that is the reason it needs DP at all.

Greedy means take the biggest coin that fits and repeat. With coins one, five and seven, and an amount of ten, greedy takes seven, then one, one, one — four coins. The right answer is five plus five, which is two. The greedy step is the biggest single reduction and it leaves a remainder that only one-rupee coins can fill.

It works for Indian denominations — one, two, five, ten, twenty, fifty — which is why the intuition feels so reliable. That is a property of those particular numbers, not of the method.

So, DP. The state is: `dp[t]` **is the fewest coins that make exactly** `t` **.**

The recurrence asks what the last coin was. If the last coin was `c`, I was at `t - c` before it, so `dp[t] = 1 + min(dp[t - c])` over every coin that fits. **Try every coin as the last one, take the best.**

Base case `dp[0] = 0`, **everything else infinity**, meaning not yet reachable. Infinity matters: zero would say every amount is free, and minus one would win every `min` and poison the table. I convert to `-1` only at the return.

And the loop runs forwards, which is the one thing that differs from 0/1 knapsack. Yesterday it ran backwards so that `dp[t - w]` held the value from before the item existed — each item once. **Today I want** `dp[t - c]` **to possibly already include this coin, because unlimited supply is exactly that.** Same six lines, one range reversed, and no error either way — so I say the direction out loud as I write it.

Cost is `O(coins * amount)` **time and** `O(amount)` **space.** Twelve coins and an amount of ten thousand is a hundred and twenty thousand cells — instant.

The constraint I would check first is the size of the amount, not the number of coins. At a billion the table is eight gigabytes and this approach does not exist.”

The follow-ups

“Why does the loop run forwards here and backwards yesterday?”

“Because of what the cell I read is supposed to mean, and it is the only difference between the two problems.

`dp[t] = f(dp[t], dp[t - c])`. The question is whether `dp[t - c]` should already account for the current item.

Backwards, from the target down, every read is from a cell to the left that this pass has not touched yet — so `dp[t - w]` still holds the previous row’s value, the state before this item existed. **The item gets used at most once.** That is 0/1.

Forwards, `dp[t - c]` may already have been updated by this same pass — meaning it already counts this coin. Adding the coin again uses it twice, and again, and again. **That is unlimited supply**, and it is exactly what I want today.

Concretely: one coin of three, amount nine, forwards. `dp[3]` becomes 1 from `dp[0]`. Then `dp[6]` becomes 2 from `dp[3]`, which I just set. Then `dp[9]` becomes 3. **Three coins of three, from one coin.**

Backwards on the same input gives infinity — impossible — because each pass can only use the coin once.

Neither raises an error. Both are correct programs; they solve different problems. Which is why I state the direction and its reason while writing rather than after, and why I would test with an input that needs a repeat — because a test where the answer uses each coin once passes under both.”

“Now count how many combinations make the amount. Does the loop order matter?”

“It matters enormously, and it is not the direction this time — it is **which loop is on the outside**.

The counting version is the same table with `+=` instead of `min`, and the base case becomes `ways[0] = 1` instead of `0`: **there is exactly one way to make nothing, which is to take no coins**, and every count is built by adding that one up.

Coins outside, amounts inside, counts combinations. Coin one is fully processed before coin two is considered at all, so no arrangement can place a one after a two — **each set is counted exactly once**.

Amounts outside, coins inside, counts permutations. Every coin is offered at every amount, so one-then-two and two-then-one are both formed and both counted.

Coins one and two, amount three. Combinations: `1+1+1` and `1+2` — **two**. Permutations: `1+1+1`, `1+2`, `2+1` — **three**.

Both are correct programs for different questions, and there is no error to tell them apart — the answer is just a different number. **Coin Change II wants combinations. Combination Sum IV wants permutations**, despite its name, which is worth remembering precisely because the name points the wrong way.

So I read the problem’s examples before writing the loops, every time. If the examples list `{1,2}` once, it is combinations; if they list it twice, it is permutations.

One more thing: the permutation count grows very fast and problems usually ask for it modulo `109 + 7`. Python integers are unbounded so I would not notice, but in Java it wraps silently.”

“The amount is a billion. Now what?”

“Then this table does not exist, and I would say so before writing anything.

The table is `amount + 1` cells. A billion cells is about eight gigabytes for the Python list of pointers alone, and it raises `MemoryError` before the first comparison. And the time is twelve billion operations — hours.

This is pseudo-polynomial, the same point as subset sum: `amount` is a *value* in the input, not a size. Writing a billion down takes thirty bits, so the input is tiny and the running time is 2^{30} . **Exponential in the input size.**

What I would do instead depends on the coins.

If the denominations are the canonical kind — one, two, five, ten and so on — greedy is provably optimal and runs in the number of coins, regardless of the amount. There is a known check for whether a system is canonical, and for the fixed real-world sets it simply is. **So: verify the system is canonical, then use greedy, and the billion stops mattering.**

If the coins are arbitrary, there is a number-theoretic route. Above a threshold — the Frobenius number for the coin set — every amount is representable, and the optimal solution for a large amount is the optimal solution for `amount mod L` plus a pile of the most efficient coin, where `L` is the least common multiple of the denominations. **So I solve the small residue problem with DP and add the rest arithmetically.** That is genuinely harder and I would only reach for it if the constraints forced me to.

And if it is minimum coins with few coins and a small answer, BFS wins — each amount is a node, each coin an edge of weight one, and BFS stops at the first level that reaches the amount. With coins one and five thousand and an amount of ten thousand, the DP fills ten thousand cells and BFS explores about four nodes.

The general point is that the constraint on the amount decides the algorithm, which is the reverse of most problems, and it is the first thing I check.”

The model answer

“You are writing the change-dispensing logic for a vending machine. Given the coins it holds and the change owed, return the fewest coins to dispense, or say it cannot make exact change.”

“Let me be explicit about one assumption first, because it changes the algorithm: **do I have unlimited coins of each denomination, or a finite number in the hopper?** A real vending machine has a finite count, which makes this bounded knapsack. **I will solve the unlimited version, which is the standard problem, and then say what changes.**

Why not greedy, first, because that is what a vending machine would naively do. Take the biggest coin that fits, repeat. **With coins one, five and seven and change of ten, greedy gives seven-one-one-one — four coins — and the answer is five-five, two.** For Indian or US denominations greedy happens to be optimal, so it works until the machine is deployed somewhere the coins differ, and then it quietly dispenses twice as many coins as it needs. **That is a bug that only shows up in one country.**

The DP. `dp[t]` is the fewest coins making exactly `t`. `dp[0] = 0` — no coins make nothing. Everything else starts at infinity, meaning unreachable.

For each coin, going forwards from the coin’s value up to the amount, `dp[t] = min(dp[t], dp[t - coin] + 1)`. Forwards, because a coin is reusable and `dp[t - coin]` should be allowed to already include it. Backwards would silently solve the each-coin-once problem, and with coins of three and change of nine it would report impossible.

At the end, if `dp[amount]` is still infinity, exact change cannot be made — coins of two and change of three, for instance — and the machine should say so rather than dispense something close.

Cost: `O(coins * amount)` time, `O(amount)` space. Six denominations and change up to a hundred rupees in paise is six hundred cells. Nothing.

And I would keep the reconstruction, because a vending machine has to actually dispense the coins, not just count them. **One extra array recording the last coin used for each amount,** then walk back from the amount subtracting coins until zero. `O(amount)` extra space, which is cheap here — unlike the 0/1 case, where reconstruction needs the whole table.

Now the finite-hopper version, which is the real machine. Each denomination has a count, so it is bounded knapsack: either expand each coin into `count` separate items and run the 0/1 version — simple, and it costs `O(total_coins * amount)` — or add a dimension for how many of each coin remain. **I would expand, because hopper counts are small, and say that if they were large I**

would use the binary-splitting trick: represent a count of thirteen as items of one, two, four and six, which can form any total from zero to thirteen in $\log$ items rather than thirteen.

Two operational things I would raise, because they are what actually breaks. The hopper runs out mid-transaction, so the count must be read at the start and the dispense must be atomic — decide the full set of coins before releasing any. And the machine should prefer to keep small coins, because running out of ones makes many later amounts impossible; so the real objective might be ‘fewest coins, tie-broken by preserving small denominations’, which is a second value in the same table rather than a different algorithm.”

9 Recall card

Unbounded knapsack is 0/1 with the inner loop forwards. `range(coin, amount + 1)` so `dp[t - coin]` may already include this coin — that is unlimited supply. Backwards means each item once. One range reversed, two different problems, no error either way — say the direction out loud as you write it.

Minimum coins: `dp[t] = 1 + min(dp[t - c])` over coins that fit. Base `dp[0] = 0`, rest `float('inf')` — zero makes every amount free, `-1` wins every `min`. Convert to `-1` only at the return, and check for infinity *before* `int()`, which raises `OverflowError`.

Greedy is wrong: coins `[1,5,7]`, amount 10 → greedy 4 (`7+1+1+1`), optimal 2 (`5+5`). It is correct for canonical systems like Indian denominations, which is why nobody notices.

Counting: `ways[0] = 1`, `+=` instead of `min`, and the loop order decides the question. Coins outside = combinations (`[1,2] → 3` gives 2); amount outside = permutations (gives 3). Coin Change II wants combinations; Combination Sum IV wants permutations despite its name. Read the examples.

`O(coins * amount)` time, `O(amount)` space — pseudo-polynomial, so the size of the amount, not the number of coins, decides whether this works. Reconstruction is cheap here: one array of last-coin-used. Minimum coins is also a BFS over amounts with unit edges, and it wins hugely when the answer is small and the amount is large.

Design a distributed unique ID generator

1 What this is, and why they ask it

Every row in every table needs an identifier. On one database, `AUTO_INCREMENT` handles it and nobody thinks about it again.

Across a hundred machines with no shared counter, it becomes a real design problem — and the reason it is asked so often is that it is small enough to finish in twenty minutes and deep enough that the follow-ups are genuine. There is a right answer, it is public, and it fits in sixty-four bits.

Snowflake — Twitter's design from 2010 — is what most interviewers have in mind: a timestamp, a machine identifier, and a per-millisecond sequence number, packed into one 64-bit integer. **Every production system you have used generates IDs this way or something very close to it.**

They ask it because it forces four things to be said precisely at once. **Uniqueness** without coordination. **Sortability**, because an ID that sorts by time makes range queries and database indexes dramatically cheaper. **Size**, because 128-bit UUIDs cost real money at scale in ways that surprise people. And **clocks**, because the whole design rests on time moving forwards and time does not always move forwards.

The clock question is the one that separates candidates. **What happens when NTP steps the clock backwards?** is the follow-up, and the honest answer is a short, specific piece of engineering.

By the end of this lesson you can lay out the 64 bits and justify each field, explain why UUIDs are the wrong default, handle clock skew, assign machine IDs without a human, and size the whole thing.

2 The story

The hospital gave every patient a number and for nineteen years the number came from one book.

The book lived at the front desk. You arrived, the man at the desk wrote your name, and beside it he wrote the next number, and that number was yours. **Nothing could go wrong, because there was one book and one man and one pen.**

Then they opened the second building.

And the obvious thing happened, which is that the second building could not send someone across the compound to the front desk for every patient, so the second building got its own book. Which started, because nobody thought about it for more than four seconds, **at one**.

By the end of that month there were two patients with the number 4,412 and both of their folders were in the same cupboard.

The fix was the kind of fix that gets made in an afternoon. **The first building's numbers would start with a one and the second building's with a two.** Building one issued 1-0001, building two issued 2-0001, and they could never collide, and nobody had to talk to anybody.

That held for eight years and three more buildings.

What broke it was the filing.

Because the folders were kept in number order, and number order was now building order, and **a doctor looking for everyone admitted in the same week had to walk to five cupboards.** The numbers no longer told you when. They told you where.

So the sister who ran the records room made the change that actually mattered, and she made it without asking anyone. **She put the date at the front.**

Year, month, day, then the building, then the count for that day in that building. 25-03-14-2-0087.

And the folders sorted themselves. Everything from the fourteenth sat together regardless of which building had issued it, the building was still there so nothing could collide, and the daily count reset every morning so it never grew large.

The only trouble came the following March, when the clock on the wall of building four was found to be eleven days slow.

3 The idea in plain English

The sister invented Snowflake. Timestamp first, then the machine, then a counter — and the wall clock in building four is the failure mode the whole design has to survive.

Start with what you need from an ID. Four things, and they conflict:

- **Unique.** Two machines must never produce the same value, ever, with no coordination between them.
- **Sortable by time.** Newer IDs should be numerically larger. This is not cosmetic — it is what makes database indexes fast and “give me the last fifty” a range scan.

- **Small.** Every foreign key, every index entry, every log line carries it.
- **Fast.** Generated locally, no network call, millions per second.

Now the options, in the order people reach for them.

A database auto-increment. One counter, guaranteed unique, perfectly sortable. **And it is a single point of failure and a hard throughput ceiling,** because every insert in the entire system serialises through one row. It works until it does not, and then it is very hard to change.

A UUID (version 4). 128 random bits. **Genuinely unique with no coordination at all** — the collision probability is negligible — and generated locally in micro-seconds. **Two serious costs.**

It is twice the size, which sounds trivial and is not: a billion rows with three foreign keys and four indexes multiplies that difference by eight.

And it is random, which destroys the index. A B-tree index on a random key means every insert lands in a different page, so the database is writing random pages instead of appending to one. **Insert throughput drops by a factor of two to ten,** index pages fragment, and the cache hit rate collapses because the working set becomes the whole index rather than its right-hand edge. **This is the single most important practical fact in this lesson,** and it is why “just use UUIDs” is the answer that loses the interview.

Snowflake, which is what to design. 64 bits, laid out in four fields:

1 bit	unused (sign bit – keeps the number positive)
41 bits	milliseconds since a custom epoch
10 bits	machine id
12 bits	sequence within that millisecond

Every field earns its width, and being able to say why is the answer.

41 bits of milliseconds is 69 years. 2^{41} milliseconds is about 69.7 years — and the epoch is *custom*, set to the day the system launched, rather than 1970. **That choice buys back the decades already spent,** and it is a one-line decision with a fifty-year consequence.

10 bits of machine id is 1,024 machines. Enough for almost anything, and it is the field to rebalance if your situation differs: fewer machines and more sequence, or the reverse.

12 bits of sequence is 4,096 IDs per machine per millisecond, which is 4.096 million per machine per second. If a machine exhausts it, generation **waits for the next millisecond** — it does not overflow into the machine field, which would be catastrophic.

The result sorts by time, because the timestamp occupies the high bits. Comparing two IDs as integers compares their timestamps first. That is the entire reason for the field order, and it is why the timestamp must be first — not first because it is important, but first because **numeric order is lexicographic order on the bit fields.**

Now the clock, which is the interesting part.

The design assumes time moves forwards, and NTP corrections can step it backwards. If the clock jumps back 50 ms, the generator produces timestamps it has already used, with a sequence counter that has reset — **and it will produce duplicate IDs.**

Three responses, and the right one depends on how far back it jumped.

Small step back — a few milliseconds — wait. Block until the clock catches up to the last timestamp issued. A few milliseconds of added latency, and correctness preserved.

Large step back — seconds — refuse. Throw an error and stop generating. **The machine should be removed from service rather than produce duplicates,** and a loud failure is enormously better than a silent collision that will be discovered months later in a support ticket.

And prevent it: use a monotonic clock where possible, and configure NTP to slew rather than step. Slewing means correcting by running the clock slightly fast or slow until it converges, never jumping. **That is one line of NTP configuration and it removes most of the problem.**

The last question: where does the machine id come from?

Not from a configuration file a human edits, because two machines will eventually get the same value and the failure is silent. Three real answers: **ZooKeeper or etcd hands out a lease on startup** — a small coordination cost paid once per boot rather than once per ID; **derive it from the private IP address,** which works when the address space maps cleanly to 10 bits; or **use the ordinal from a Kubernetes StatefulSet,** which is already unique and stable by construction.

All three are better than a config file, and saying which one and why is the answer.

4 The picture

The 64 bits, laid out:

63

62

22

12

0

+-----+-----+-----+-----+

| 0 | timestamp (41 bits) | machine | sequence (12 bits) |

| | ms since custom epoch | 10 bits |

+-----+-----+-----+-----+

^

^

^

^

sign bit

69 years of ms

1,024 machines

4,096 per ms

always 0

per machine

→ positive

TIMESTAMP IS FIRST because integer comparison compares high bits first. That is the ONLY reason for this order, and it is what makes the IDs sort by time.

Why a random UUID destroys the index:

SEQUENTIAL IDS (Snowflake)

RANDOM IDS (UUIDv4)

B-tree, inserting 1..8

B-tree, inserting random values

[4]

/ \

[1 2 3] [5 6 7 8]

^

every insert lands in the RIGHTMOST page
→ that one page stays in memory, always
→ one page written

[4c]

/ \

[0a 2f] [9d f1]

^ ^ ^ ^

every insert lands in a DIFFERENT page, chosen at random
→ the whole index must be in memory or every insert is a random disk read + write
→ pages split and fragment

Measured effect: 2-10x lower insert throughput, and an index that is 30-40% larger after fragmentation.

The clock going backwards:

```

time →
t=1000ms | seq 0,1,2,3      IDs issued
t=1001ms | seq 0,1          IDs issued
NTP steps the clock back 2 ms
t=999ms   | seq 0,1,2 ← t=999 was ALREADY used with seq 0,1,2
                        DUPLICATE IDs. Silently.

Defence:
last_timestamp = 1001
now = 999
if now < last_timestamp:
    gap = last_timestamp - now
    if gap ≤ 5 ms: sleep until now ≥ last_timestamp (wait)
    else:         raise ClockMovedBackwards        (refuse)

```

The sequence exhausting within a millisecond:

```

within t = 1000ms:
request 1      → seq 0
request 2      → seq 1
...
request 4096   → seq 4095
request 4097   → sequence field is FULL

WRONG: let it wrap to 0      → duplicate of request 1
WRONG: carry into machine id → collides with another machine
RIGHT: busy-wait until t = 1001ms, then reset seq to 0

The cap is 4,096 per ms = 4.096 MILLION per second per machine.
Nothing real hits this. The handling still has to exist.

```

The alternatives, compared:

	size	sortable	coordination	index-friendly
auto-increment	8 B	yes	every insert	yes
UUIDv4	16 B	NO	none	NO
UUIDv7	16 B	yes	none	yes
Snowflake	8 B	yes	once at boot	yes
ticket server	8 B	yes	every insert	yes

Snowflake is the only row with no bad cell.
 UUIDv7 (2024) is the modern near-equal, at twice the size.

5 How it actually works

The generator

```
import time, threading

EPOCH = 1704067200000          # 2024-01-01, chosen at launch

TIMESTAMP_BITS, MACHINE_BITS, SEQUENCE_BITS = 41, 10, 12
MAX_MACHINE = (1 << MACHINE_BITS) - 1      # 1023
MAX_SEQUENCE = (1 << SEQUENCE_BITS) - 1     # 4095
```

EPOCH set to the launch date, not 1970, is the free fifty years. With a 1970 epoch, 41 bits ran out in 2039; from 2024 it runs to 2093.

```
class SnowflakeGenerator:
    def __init__(self, machine_id: int) → None:
        if not 0 ≤ machine_id ≤ MAX_MACHINE:
            raise ValueError(f"machine_id must be 0..{MAX_MACHINE}")
        self.machine_id = machine_id
        self.sequence = 0
        self.last_timestamp = -1
        self.lock = threading.Lock()
```

The lock is not optional. Two threads reading `self.sequence` at the same time produce the same ID, and the whole guarantee is gone. **In Java this is usually synchronized ; in Go, a mutex.**

Now the generation, which is where the clock is handled:

```
def next_id(self) → int:
    with self.lock:
        now = int(time.time() * 1000)

        if now < self.last_timestamp:          # clock went backwards
            gap = self.last_timestamp - now
            if gap > 5:
                raise RuntimeError(f"clock moved backwards by {gap}ms")
            while now < self.last_timestamp:    # small: wait it out
                now = int(time.time() * 1000)
```

Small step: wait. Large step: refuse. The threshold is a policy number — five milliseconds is a common choice — and the important part is that **the large case fails loudly** rather than producing duplicates.

```

    if now == self.last_timestamp:
        self.sequence = (self.sequence + 1) & MAX_SEQUENCE
        if self.sequence == 0:
            # 4096 used this ms
            while now ≤ self.last_timestamp: # wait for the next ms
                now = int(time.time() * 1000)
    else:
        self.sequence = 0
        # new millisecond

    self.last_timestamp = now
    return ((now - EPOCH) << (MACHINE_BITS + SEQUENCE_BITS)
            | self.machine_id << SEQUENCE_BITS
            | self.sequence)

```

`& MAX_SEQUENCE` wraps to zero at 4096, and the `if self.sequence == 0` catches exactly that case and waits. **Without the wait, the wrap produces a duplicate of the first ID in that millisecond.**

The final line is three shifts and two ORs — a few nanoseconds. There is no network call anywhere in this function, which is the point.

Reading an ID back

```

def parse(snowflake_id: int) → tuple[int, int, int]:
    sequence = snowflake_id & MAX_SEQUENCE
    machine = (snowflake_id >> SEQUENCE_BITS) & MAX_MACHINE
    timestamp = (snowflake_id >> (SEQUENCE_BITS + MACHINE_BITS)) + EPOCH
    return timestamp, machine, sequence

```

Every ID carries its own creation time and origin machine, which is genuinely useful in an incident: an ID in a log tells you when it was created and which machine created it, with no lookup.

And it is an information leak. A public ID reveals your creation rate — two IDs a second apart let anyone compute how many rows you created in between. **Instagram’s sequential IDs let people measure their signup rate for years.** If that matters, expose a hashed or encrypted public ID and keep the Snowflake internal.

Getting a machine id without a human

```
def machine_id_from_zookeeper(zk, path: str = "/snowflake/workers") → int:
    """Ephemeral sequential znode: unique while the process lives."""
    node = zk.create(f"{path}/worker-", ephemeral=True, sequence=True)
    return int(node.rsplit("-", 1)[1]) % (MAX_MACHINE + 1)

def machine_id_from_ip() → int:
    """Last 10 bits of the private IP. Works when the subnet is small."""
    import socket
    octets = [int(x) for x in socket.gethostbyname(socket.gethostname()).split(".")]
    return ((octets[2] << 8) | octets[3]) & MAX_MACHINE
```

The ZooKeeper version is correct and adds a dependency at boot. The IP version has no dependency and **breaks silently if two subnets share the low ten bits** — `10.0.1.5` and `10.1.1.5` collide. Check the address plan before choosing it.

Kubernetes gives you this free: a StatefulSet pod is named `app-0`, `app-1`, and that ordinal is unique and stable across restarts. **Parse the hostname.**

The alternatives, and when each is right

UUIDv4	random 128 bits. No coordination, not sortable, index-hostile. Right for: client-generated ids, distributed systems with no central anything, ids that must reveal nothing.
UUIDv7	48-bit ms timestamp + 74 random bits. Sortable AND uncoordinated. Standardised in 2024 (RFC 9562). Twice Snowflake's size and needs no machine id at all. The modern default when 16 bytes is acceptable.
ticket server	one database whose only job is issuing ids, in batches. A machine takes 1,000 ids at a time, so the database sees one write per thousand ids. Simple, and a single point of failure.
Snowflake	8 bytes, sortable, no per-id coordination. Needs a machine id and a well-behaved clock.

Naming UUIDv7 is worth doing, because it is the genuine modern alternative and it shows you are not just reciting a 2010 design. **The trade is: UUIDv7 needs no machine id and costs 8 extra bytes per row.**

Nothing real exhausts a single generator, and saying that is worth more than the multiplication — it tells the interviewer you know the sequence field exists for correctness under bursts, not for throughput.

Storage, which is where UUIDs actually cost money:

1 billion rows

Snowflake (BIGINT, 8 bytes)

primary key 1,000,000,000 x 8 B = 8 GB

3 foreign keys x 3 = 24 GB

4 secondary indexes (each holds the PK) = 32 GB

TOTAL = 64 GB

UUID (16 bytes)

same layout, doubled = 128 GB

difference = 64 GB

And stored as a string, which happens constantly:

UUID as CHAR(36): "550e8400-e29b-41d4-a716-446655440000"

1,000,000,000 x 36 B x 8 places = 288 GB

vs 64 GB for BIGINT. 4.5x, and the comparisons are string comparisons rather than integer comparisons.

The insert throughput difference, which matters more than the storage:

sequential inserts (Snowflake)

every insert appends to the rightmost B-tree page

that page is always in memory

→ ~50,000 inserts/second on ordinary hardware

random inserts (UUIDv4)

every insert targets a random page

index is 64 GB, memory is 32 GB

→ most inserts are a random disk read + a random write

→ ~5,000-25,000 inserts/second

2-10x slower, and it degrades as the index grows past memory.

That degradation shape is the real problem: it is fine in testing with a small table and collapses in production a year later.

Generation latency:

Snowflake:	3 shifts, 2 ORs, 1 clock read	~100 nanoseconds
UUIDv4:	16 bytes from a CSPRNG	~200 nanoseconds
ticket server:	one network round trip (amortised over a batch of 1,000)	~500,000 nanoseconds ~500 ns per id
database auto-increment:	part of the insert	no separate cost, but serialises everything

Clock skew, quantified:

NTP with default configuration

typical offset	1-10 ms
after a network blip	up to several hundred ms
a step correction	can be seconds

with `ntpd -x` (slew only, never step)

- the clock is corrected at up to 500 ppm
- a 1-second error takes ~33 minutes to correct
- never jumps backwards

33 minutes of slightly-wrong time against a possible duplicate ID is an easy trade, and it is one configuration flag.

Sequence exhaustion in practice:

to exhaust 4,096 in one millisecond, a single machine must generate 4.096 million ids per second

a Java service doing this does nothing else – the ids alone saturate a core. If you are here, the bottleneck is not the id generator.

→ the exhaustion branch runs approximately never, and must still be correct, because "approximately never" is where the worst bugs live.

7 The trade-offs

Sortability against privacy, and this is the real one. A time-sortable ID makes indexes fast and makes “the newest fifty” a range scan instead of a sort. **It also publishes your creation rate.** Anyone can create two objects a minute apart, subtract the IDs, and compute exactly how many you created in between — competitors have measured signup rates this way for years. **If that matters, keep the Snowflake internal and expose a hashed public identifier,** and accept the extra lookup.

Size against coordination. Snowflake is 8 bytes and needs a unique machine id, which needs ZooKeeper, or an IP scheme, or Kubernetes ordinals — a small operational dependency at boot. **UUIDv7 is 16 bytes and needs nothing at all.** At a billion

rows with several indexes that is roughly 64 GB of difference, so **the question is whether 64 GB or the coordination is cheaper for you**, and at smaller scale the answer is usually UUIDv7.

Waiting against failing on a backwards clock. Waiting adds latency and preserves correctness; failing takes the machine out of service. **Both are correct and the threshold between them is a policy number** — the wrong answer is to do neither and produce duplicates. **And a duplicate ID is close to unrecoverable**, because it is discovered long after the fact, in data that has already been joined and copied.

Machine ids from configuration against machine ids from coordination. A config file is simple and **fails silently when two machines share a value** — duplicate IDs with no error anywhere. ZooKeeper is correct and puts a dependency in the boot path, so a ZooKeeper outage stops new machines starting. **Kubernetes ordinals are the best of both** where they apply, because uniqueness is structural rather than enforced.

Field widths are a genuine choice, not a constant. 41/10/12 suits a service with hundreds of machines and moderate per-machine rates. **Sony chose 39/16/8** — 174 years and 65,536 machines, at 256 IDs per 10 ms — for a fleet with many small machines. **Being willing to redistribute the bits for the stated constraints is what distinguishes understanding the design from having memorised it.**

When would I not build this? Below a few thousand writes a second, a database auto-increment is correct, simpler, and has no clock dependency at all — and reaching for Snowflake there adds a machine-id management problem to solve a throughput problem you do not have. **If IDs are generated by clients** — a mobile app creating rows offline — no server-side scheme works and UUIDs are the answer. **And if 16 bytes is acceptable, UUIDv7 gives you sortable, uncoordinated IDs with no clock handling and no machine registry**, which is less code to get wrong.

8 In the interview

How it gets asked

- “Generate unique IDs across a hundred servers. No central counter.” — the standard prompt.
- “Why not just use UUIDs?” — always asked, and the index answer is the one they want.
- “Walk me through the 64 bits.” — say what each field buys.
- “What happens when the clock goes backwards?” — the separating question.

- *“How does each machine get its ID?”*
- *“What if a machine generates more than 4,096 in a millisecond?”*

The first ninety seconds

“Four requirements, and they pull against each other: **unique with no coordination, sortable by time, small, and generated locally with no network call.**

Sortable is the one people skip, and it is the one that matters most, because it decides whether the database index works.

I would use a Snowflake ID: 64 bits, four fields. One unused sign bit so the number stays positive. **Forty-one bits of milliseconds** since a custom epoch. **Ten bits of machine id.** **Twelve bits of sequence** within that millisecond.

Each width earns itself. Forty-one bits of milliseconds is 69 years — and **the epoch is the day we launch, not 1970,** which buys back the fifty-four years already spent, from one constant. Ten bits is 1,024 machines. Twelve bits is 4,096 IDs per machine per millisecond, which is four million a second per machine — **nothing real approaches that,** so the field exists for correctness under bursts, not for throughput.

The timestamp is in the high bits and that is the whole point of the layout, because comparing two IDs as integers compares the timestamps first. The IDs sort by time for free.

Why not UUIDs, which is the obvious alternative. Two reasons. **They are sixteen bytes rather than eight,** and at a billion rows with three foreign keys and four indexes that is about sixty-four gigabytes of difference. **And version 4 is random, which destroys the B-tree index** — every insert lands in a different page, so instead of always appending to one hot page in memory, the database does a random read and a random write per insert. **Two to ten times slower on inserts,** and it gets worse as the index outgrows memory, which means it looks fine in testing and collapses a year into production.

The part I would want to talk about is the clock, because the whole design assumes time moves forwards and NTP can step it backwards. **If that happens, the generator reissues timestamps it has already used with a reset sequence, and produces duplicates silently.**

Generation is three shifts and two ORs — about a hundred nanoseconds, no network call. And I would ask one thing before finalising: **are these IDs public?** Because a time-sortable ID publishes the creation rate, and if that is sensitive I would keep the Snowflake internal and expose a hashed identifier.”

The follow-ups

“Why not just use UUIDs?”

“They are correct — the uniqueness is genuinely fine — and they cost in two ways, one obvious and one that surprises people.

The obvious one: sixteen bytes against eight. That sounds negligible per row and it multiplies. A billion rows, a primary key, three foreign keys and four secondary indexes — and in most databases every secondary index entry also stores the primary key. **That is eight copies of the identifier per row: sixty-four gigabytes for BIGINT, a hundred and twenty-eight for UUID.** And if it is stored as a `CHAR(36)` string, which happens constantly, it is two hundred and eighty-eight gigabytes and every comparison is a string comparison.

The one that actually hurts: version 4 UUIDs are random, and that destroys the index.

A B-tree with sequential keys means every insert goes into the rightmost page. **That page is always in memory, so an insert is one page write and nothing else.** With random keys, every insert targets a different page chosen uniformly across the whole index — so unless the entire index fits in memory, **each insert is a random disk read followed by a random write.** Pages split constantly and the index fragments, typically ending up thirty to forty percent larger.

Measured, that is two to ten times fewer inserts per second, and the shape of the degradation is what makes it dangerous: **it is invisible until the index outgrows memory.** It tests fine and fails in production a year later, and by then the ID type is in every table.

Where UUIDs are right: IDs generated on a client, offline, with no server involved. IDs that must reveal nothing about creation time or volume. Systems with genuinely no coordination available anywhere.

And I would name UUIDv7, standardised in 2024, which is a 48-bit millisecond timestamp followed by random bits. **Sortable, index-friendly, and needs no machine id at all** — so it fixes the index problem and keeps the zero-coordination property. **It is still sixteen bytes.** If the extra eight bytes per reference is acceptable, UUIDv7 is less machinery to get wrong than Snowflake, and I would genuinely consider it first.”

“What happens when the clock goes backwards?”

“That is the failure mode the whole design rests on, so let me say exactly what breaks and then what I do.

What breaks: duplicates, silently. The generator remembers the last timestamp it issued. If the clock jumps back fifty milliseconds, it starts producing timestamps it has already used, and the sequence counter resets to zero because the timestamp looks new. **So it reissues IDs it has already handed out**, with no error anywhere, and the collision is discovered months later in data that has already been joined and copied. **That is close to unrecoverable.**

Why it happens: NTP. A machine whose clock has drifted gets corrected, and the default correction for a large offset is a *step* — an instantaneous jump. Backwards, if the clock was fast. **Virtual machines are worse**, because live migration and host suspension both disturb the guest clock.

Three things, and I would do all three.

Prevent it: configure NTP to slew, never step. Slewing corrects by running the clock slightly fast or slow until it converges — up to about 500 parts per million, so a one-second error takes about half an hour to fix. **Slow, and it never goes backwards.** That is one flag and it removes most of the problem.

Detect and wait, for small steps. The generator compares the current time with the last timestamp it issued. If time went back by a few milliseconds, **block until the clock catches up.** A few milliseconds of latency, correctness preserved.

Detect and refuse, for large ones. If it went back by more than the threshold — a few milliseconds — the generator throws and stops issuing IDs. **That machine should be pulled out of the load balancer.** A loud failure on one machine is dramatically better than silent duplicates across the fleet.

And where I can, use the monotonic clock — `CLOCK_MONOTONIC`, which never goes backwards — anchored to one wall-clock reading at startup. That gives me a timestamp that cannot regress within a process, though it does not survive a restart, so the on-disk last-timestamp check still has to exist.”

“How does each machine get its ID, and what if two get the same one?”

“If two machines share a machine id they will produce identical IDs whenever they generate in the same millisecond with the same sequence number — **and nothing reports it**. So the assignment mechanism is a correctness mechanism, not configuration.

What I would not do is put it in a config file or an environment variable, because a copy-pasted deploy config, a scaled-up instance group, or a rolled-back change will eventually produce two machines with the same number, and there is no error to notice.

Three mechanisms I would actually use.

ZooKeeper or etcd, with an ephemeral sequential node. On startup the process creates a node and gets back a unique number. **Ephemeral means it disappears when the process dies**, so the id returns to the pool. Correct, and it puts a dependency in the boot path — a ZooKeeper outage means no new machines can start, though running ones are unaffected. **Worth stating explicitly, because it is a real availability cost.**

Derive it from the private IP address, taking the low ten bits. No dependency at all, and it works when the address plan guarantees uniqueness in those bits. **It fails silently when two subnets share them** — `10.0.1.5` and `10.1.1.5` give the same ten bits. So I would only use it after checking the network plan, and I would log the derived id at startup so a collision is at least visible in the logs.

Kubernetes StatefulSet ordinals, where they apply, are the best answer. A pod is `service-0`, `service-1`, and the ordinal is unique and stable across restarts by construction. **No extra system, no derivation that can collide** — parse the hostname.

Whichever I use, I would add detection: each generator writes its machine id and a heartbeat to a shared store, and an alert fires if two heartbeats claim the same id. **It does not prevent the collision, and it turns a silent corruption into a page**, which is the difference that matters.”

The model answer

“Design an ID generator for a social platform: two hundred application servers, ten thousand new objects a second at peak, IDs appear in public URLs, and they will be the primary key of every table.”

“Three things in that prompt change the design, so let me take them in order: **two hundred servers, IDs in public URLs, and primary key of every table.**

Primary key of every table means index behaviour dominates everything else, so I am ruling out random IDs immediately. A UUIDv4 primary key at this scale means every insert lands in a random B-tree page — two to ten times slower on inserts once the index outgrows memory, and it degrades invisibly as the table grows.

So: Snowflake, 64 bits. Sign bit, 41 bits of milliseconds from a 2024 epoch, 10 bits of machine, 12 bits of sequence.

The custom epoch is worth saying out loud — from 1970, 41 bits runs out in 2039; from launch, it runs to 2093. One constant, fifty-four years.

The field widths for two hundred servers. Ten bits gives 1,024 machines, so there is five times headroom — comfortable, and I would keep it rather than shrinking it, because machine counts grow and the sequence field has enormous slack. **Twelve bits is 4,096 IDs per machine per millisecond, four million a second per machine.** At ten thousand objects a second across two hundred servers, that is fifty per server per second — **about a thousandth of one percent of one machine’s capacity.** The sequence field is there for correctness during a burst, not for throughput, and I would say that rather than presenting the four-billion figure as if it were needed.

Machine ids: Kubernetes StatefulSet ordinals if the platform is on Kubernetes, because uniqueness is structural. Otherwise ZooKeeper ephemeral sequential nodes, accepting that a ZooKeeper outage blocks new machines from starting. **Not a config file** — two machines with the same id produce duplicate primary keys and nothing reports it.

Now the part the prompt forces, which is that IDs appear in public URLs.

A Snowflake ID publishes its creation time and the creation rate. Anyone can create two objects a minute apart, subtract, divide by the sequence width, and compute how many objects the platform created in between. **Instagram’s signup rate was measurable this way for years.** For a social platform that is a genuine business leak.

So I would separate the two identifiers. The Snowflake stays internal — primary key, foreign keys, indexes, logs. **The public URL carries a different, opaque identifier:** either a random short code stored alongside the row, or the

Snowflake encrypted with a block cipher, which is reversible and needs no extra column. **The encrypted form is nicer** because there is no second uniqueness problem to solve and no extra index, at the cost of a cipher operation per request.

Clock handling, which is the correctness core. NTP configured to slew rather than step, so the clock never jumps backwards. In the generator: if time has gone back a few milliseconds, wait; if it has gone back further, throw and take the machine out of service. **And the last issued timestamp persisted, so a restart after a clock jump does not reissue.**

Sizing. Ten thousand a second is 864 million objects a day, so 41 bits of millisecond is untouched by volume. At a billion rows with three foreign keys and four indexes, BIGINT costs about 64 GB against 128 GB for UUID — **and the insert throughput difference matters more than the storage.**

What I would monitor: duplicate machine ids, via a heartbeat table with an alert on collision; clock offset per machine, alerting well before the refuse threshold; and the rate of sequence-exhaustion waits, which should be zero and tells me about bursts if it is not.

And the honest alternative I would put on the table: UUIDv7. Sortable, index-friendly, no machine registry, no clock-skew handling, standardised. **It costs eight extra bytes per reference and removes two entire classes of operational failure.** At two hundred servers and ten thousand a second I would genuinely consider it the safer choice — **and I would still need the separate public identifier**, because UUIDv7 leaks creation time too.”

9 Recall card

Snowflake, 64 bits: 1 sign + 41 ms + 10 machine + 12 sequence. Timestamp in the **high** bits, which is the only reason the IDs sort by time — integer comparison compares high bits first. **Custom epoch at launch, not 1970:** 69 years from launch instead of exhausting in 2039.

41 bits = 69.7 years. 10 bits = 1,024 machines. 12 bits = 4,096/ms = 4.1M/s per machine — nothing real approaches it, so the sequence field is for burst correctness, not throughput. On exhaustion, **wait for the next millisecond**; never wrap, never carry into the machine field.

Why not UUIDv4: 16 bytes not 8 (~64 GB extra at 1B rows $\times$ 8 index copies) **and random keys destroy the B-tree** — every insert hits a different page instead of the hot rightmost one, **2–10 $\times$ fewer inserts/second**, degrading invisibly as the index outgrows memory. **UUIDv7 (RFC 9562, 2024) fixes sortability** and needs no machine id — the real modern alternative at 16 bytes.

Clock going backwards is the failure mode: reissued timestamps with a reset sequence = silent duplicates. **Slew NTP, never step; wait on a small step; refuse loudly on a large one**, and persist the last timestamp across restarts.

Machine ids from ZooKeeper ephemeral sequential nodes, the private IP's low bits, or Kubernetes StatefulSet ordinals — never a config file, because two machines sharing an id produce duplicates with no error. **Sortable IDs leak your creation rate**, so keep the Snowflake internal and expose an encrypted or random public identifier.

DSA topic: Unbounded knapsack and coin change **System design topic:** Design a distributed unique ID generator

Code these, in this order

One rule for the whole set: **say the loop direction and its reason out loud before writing the inner loop, and say which loop is on the outside and why.** Two decisions, four words each, and they are the only thing separating these six problems from each other.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Coin Change	LeetCode 322 (Medium)	Forwards, <code>min</code> , and the infinity sentinel.
2	Coin Change II	LeetCode 518 (Medium)	Counting combinations — coins on the outside.
3	Combination Sum IV	LeetCode 377 (Medium)	Counting permutations — amount on the outside.
4	Perfect Squares	LeetCode 279 (Medium)	Coin change where you generate the coins.
5	Minimum Cost For Tickets	LeetCode 983 (Medium)	Unbounded, with an unusual “coin” — a duration.
6	Integer Break	LeetCode 343 (Medium)	Unbounded knapsack maximising a product.

On problem 1, write greedy first and break it

Implement biggest-coin-first. Run it and the DP on `coins = [1, 5, 7]`, `amount = 10`, and record both answers. Then run both on `[1, 2, 5]` with `amount = 11` and record those.

Say in one sentence why the second pair agrees, and what that tells you about why nobody notices greedy is wrong.

On problem 1, reverse the loop and get “impossible”

Change `range(coin, amount + 1)` to `range(amount, coin - 1, -1)`. Run on `coins = [3]`, `amount = 9`. Record what comes back.

Then run the same broken version on `[1, 2, 5]` with `amount = 8` and record that. **Say why the second test passes**, and what that means for how you choose test inputs.

On problem 1, try all three wrong sentinels

Initialise `dp` to `0`, then to `-1`, then to `float('inf')`. Run each on `[1, 5, 7]` with `amount = 10` and record all three answers. **Two of them are plausible numbers rather than errors**. Say which, and why that is worse than crashing.

On problems 2 and 3, run the same input through both

Give both `coins = [1, 2]` and `amount = 3`. Record both answers. Then list the actual arrangements each one counted, by hand. **Then swap the loop nesting in problem 2 and confirm you get problem 3's answer**.

Write one sentence explaining why coins-outside cannot produce `2+1`.

On problem 4, notice you already solved it

Before coding: say what the coins are. Then say what the amount is. **Then call your problem 1 function**.

On problem 5, find the coin

The costs are 1-day, 7-day and 30-day passes. Say what plays the role of a coin and what plays the role of the amount. **Note that the state runs over days, not over money**, and say why that changes which direction the loop runs.

Then the BFS drill

Solve problem 1 with BFS instead — amounts as nodes, coins as unit edges. Verify agreement on twenty random inputs. Then time both on `coins = [1, 5000]`, `amount = 10000` and record the ratio. **Say why the gap is that large**.

The greedy drill

1. Give the counter-example with exact coins and both answers.
2. Say why greedy works for Indian denominations.
3. Say what a “canonical” coin system is, in one sentence.
4. Say what greedy costs and what DP costs, and what the difference buys.

The direction drill

1. Say what the backwards loop guarantees about the cell being read.

2. Say what the forwards loop allows, and which problem that is.
3. Give the one-coin example that separates them.
4. Say what the error message is, and what you do instead.
5. Say what kind of test input would hide the bug.

The sentinel drill

1. Say what `dp[0]` is for minimum coins, and why.
2. Say what the rest starts at, and why.
3. Say what `0` produces instead, and what `-1` produces.
4. Give the exact error from `int(float('inf'))`.
5. Say where the conversion to `-1` belongs.

The loop-order drill

1. Say what coins-outside counts and what amount-outside counts.
2. Give both answers for `[1, 2]` and amount 3, and list the arrangements.
3. Explain in one sentence why coins-outside cannot form `2+1`.
4. Say which LeetCode problem wants which, and why one name misleads.
5. Say how you decide when the problem statement is ambiguous.

The family drill

Fill in the table from memory: cell type, operator, base case.

1. Subset sum.
2. Count the ways.
3. Minimum coins.
4. Maximum value, unbounded.

Then say what stays the same across all four.

The break-it drill

Trigger each and record the exact output or error:

1. The backwards loop on `[3]` with amount 9.
2. `dp` initialised to `0`.
3. `dp` initialised to `-1`.
4. Missing `ways[0] = 1`.
5. Swapped loop nesting in the counting version.

6. `int(float('inf'))` .
7. `[float('inf')] * (10**9 + 1)` .

Five of the seven give no error at all. Name them.

The layout drill

1. Give all four fields and their widths, in order.
2. Say what each width buys, with the arithmetic.
3. Say why the timestamp is first — the real reason, about integers.
4. Say what the custom epoch is worth, in years.
5. Say what the sign bit is for.

The UUID drill

1. Give the size difference and multiply it out at 1 billion rows.
2. Explain the B-tree argument in three sentences.
3. Give the insert throughput ratio.
4. Say why the degradation is dangerous, not just bad.
5. Say what UUIDv7 fixes and what it still costs.
6. Name two cases where UUIDv4 is the right answer.

The clock drill

1. Say exactly what breaks when the clock steps backwards.
2. Say why it is close to unrecoverable.
3. Give the three defences and which one is prevention.
4. Say what slewing is and how long a 1-second correction takes.
5. Say where the threshold between wait and refuse sits, and that it is policy.
6. Say what the monotonic clock gives you and what it does not.

The machine-id drill

1. Say why a config file is not acceptable, and what the failure looks like.
2. Name the three real mechanisms.
3. Say what ZooKeeper costs you at boot.
4. Give the exact case where IP derivation collides.
5. Say why Kubernetes ordinals are structurally better.
6. Say what detection you would add regardless.

The sequence drill

1. Say how many IDs per millisecond and per second per machine.
2. Compare that with a real workload, with numbers.
3. Say what happens on exhaustion, and name the two wrong answers.
4. Say why the branch must be correct even though it never runs.

The privacy drill

1. Say exactly what a sortable ID leaks.
2. Give the measurement anyone can perform.
3. Give two ways to expose a public identifier instead.
4. Say which you prefer and why.
5. Say whether UUIDv7 solves this.

The when-not-to drill

1. Say at what scale an auto-increment is the right answer.
 2. Say what a ticket server is and its one weakness.
 3. Say when only UUIDs work.
 4. Say why you might choose UUIDv7 over Snowflake at moderate scale.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the fewest coins that make the amount.* The greedy counter-example, the state as a sentence, the recurrence as “what was the last coin”, the infinity sentinel, the forwards loop with its reason, and the amount-size caveat.
 2. *Now count how many combinations make the amount. Does the loop order matter?* Both answers for `[1, 2]` and 3, why coins-outside fixes the order, which problem wants which, and the modulus warning.
 3. *Generate unique IDs across a hundred servers. No central counter.* The four fields and their widths, why the timestamp is first, the UUID index argument, the clock-backwards handling, and where the machine id comes from.
-

Before you move on

- [] I can give the greedy counter-example with exact numbers.
- [] I know why greedy works for Indian denominations.
- [] I can state the minimum-coins state as a full sentence.
- [] I can give the recurrence as “what was the last coin”.
- [] I know `dp[0] = 0` and the rest is infinity, and why.
- [] I know what `0` and `-1` sentinels produce instead.
- [] I know `int(float('inf'))` raises, and where to check.
- [] I run the inner loop forwards and can say why.
- [] I know the backwards loop reports “impossible” on `[3]` to 9.
- [] I know which test inputs would hide that bug.
- [] I know `ways[0] = 1` and why it is the seed.
- [] I know coins-outside counts combinations.
- [] I know amount-outside counts permutations.
- [] I can list both sets of arrangements for `[1,2]` to 3.
- [] I know Combination Sum IV wants permutations despite its name.
- [] I can fill in the four-row family table from memory.
- [] I can give time and space, and the amount-size caveat.
- [] I know minimum coins is also a BFS, and when that wins.
- [] I can reconstruct the coins used, and know it is cheap here.
- [] I can lay out the 64 bits and justify every width.
- [] I know why the timestamp is in the high bits.
- [] I know what the custom epoch is worth.
- [] I can give the UUID size difference at 1 billion rows.
- [] I can explain the B-tree argument without notes.
- [] I know the insert throughput ratio and why it degrades invisibly.
- [] I know what UUIDv7 fixes and what it costs.
- [] I can say exactly what a backwards clock does.
- [] I know the three clock defences and which is prevention.
- [] I know the three machine-id mechanisms and why not a config file.
- [] I know what a sortable ID leaks and how to fix it.
- [] I answered all three questions above out loud.

Coin change II: counting the ways

DATA STRUCTURES AND ALGORITHMS

Coin Change II: counting

SYSTEM DESIGN

Design a web crawler

1 What this is, and why they ask it

Yesterday you found the *best* answer. Today you count *how many* answers there are.

That is a different question and it needs a different discipline. Minimising is forgiving — if two paths reach the same amount, taking either one gives the same minimum, so double-counting is invisible. **Counting is not forgiving.** If two paths reach the same amount and you count both, the answer is wrong, and the only symptom is a number that is too large.

They ask counting problems because **they are where DP stops being mechanical.** The recurrence is easy. The hard part is answering one question precisely: *am I counting sets or sequences?* Get that wrong and you produce a correct program that answers a different question, with no error to warn you.

The other reason is that counting is where **overcounting and undercounting are both silent.** There is no `IndexError`, no exception, no crash — just a number. A candidate who can say “I count combinations, and I know this because the coin loop is outside” is demonstrating exactly the care the problem is testing for.

By the end of this lesson you can count combinations and permutations and say which is which, explain the loop nesting from first principles, handle the modulus correctly, and recognise the counting family — ways to climb stairs, ways to decode a string, ways to reach a cell.

2 The story

Bhanumati had been making the same four things for the wedding season for eleven years and the argument was always about the boxes.

Two hundred boxes. Four sweets. And the family who ordered them always said the same sentence, which was “**make sure they are not all the same.**”

So she and her daughter sat down the week before, as they did every year, and worked out what could go in a box, and every year it took the whole evening because they kept losing count.

The rule was simple. Six pieces in a box. Any mix of the four kinds.

Her daughter started listing. Six laddoos. Five laddoos and a barfi. Five laddoos and a peda. Four laddoos and two barfis. Four laddoos, one barfi, one peda —

“You said that one.”

She had not. She had said four laddoos, one peda, one barfi, twenty minutes earlier, and it had gone on the list as a separate line, and it was the same box.

Because a box is not a sequence. Nobody opens a box and finds the sweets in an order.

They started again with a rule Bhanumati made up on the spot, and it was the rule that finally worked. **“Decide all the laddoos first. Every box, all of them. Then move to barfi and never go back.”**

So: how many laddoos — six, five, four, down to none. **Then** for each of those, how many barfis. **Then** pedas. **Then** whatever is left is the last kind.

And because they never went back to laddoos after starting on barfi, **there was no way to write down the same box twice.** The order was fixed by the method, not by the box.

The daughter, who was seventeen and argumentative, pointed out that this was a completely different question from the one the shop next door had to answer, because next door made a plate where the sweets were laid out in a row and **there the order was the whole point** — laddoo-barfi-laddoo was a different plate from laddoo-laddoo-barfi, and they were paid for the arrangement.

Same six sweets. Same four kinds. Two completely different numbers.

3 The idea in plain English

Bhanumati’s rule is the loop nesting, and her daughter has just stated the entire lesson.

Two questions look identical and are not:

- **Combinations.** *How many different sets of coins make the amount?* `{1, 2}` is one answer. Order does not exist. **The box.**
- **Permutations.** *How many different sequences of coins make the amount?* `1 then 2` and `2 then 1` are two answers. **The plate.**

Coin Change II asks for combinations. Climbing stairs asks for permutations — taking a 1-step then a 2-step is a genuinely different way to climb than 2 then 1. **Combination Sum IV asks for permutations despite its name**, which is the most reliably misleading title on LeetCode.

The code for both is the same six lines, with the two loops swapped.

```
# COMBINATIONS – coins outside
for coin in coins:
    for t in range(coin, amount + 1):
        ways[t] += ways[t - coin]

# PERMUTATIONS – amount outside
for t in range(1, amount + 1):
    for coin in coins:
        if coin ≤ t:
            ways[t] += ways[t - coin]
```

Why coins-outside counts each set once, said the way Bhanumati said it: **coin 1 is completely finished before coin 2 is ever considered**. By the time the algorithm looks at 2, every count in the table describes using only 1s. Adding a 2 extends those, and there is no route back to adding a 1 afterwards. **The coin order is imposed by the loop, so every set is built in exactly one order and counted exactly once.**

Why amount-outside counts sequences, said the same way: **at every amount, every coin is offered**. So `ways[3]` is built from `ways[2]` (add a 1) and from `ways[1]` (add a 2), and those two routes describe `1,1,1` / `1,2` and `2,1` — the last coin is what distinguishes them, so orderings are separate.

The base case is `ways[0] = 1` in both, and this is the line that carries the meaning. **There is exactly one way to make nothing: take no coins**. Every other count is that `1` propagated forwards. Set it to zero and the entire table is zero.

And the counting family is larger than coins. Once you see the shape, these are all the same problem:

climbing stairs	<code>coins = [1, 2]</code> , count PERMUTATIONS (1-then-2 is a different climb from 2-then-1)
ways to decode "226"	coins are "take 1 digit" or "take 2", with validity
unique paths in a grid	<code>ways[cell] = ways[above] + ways[left]</code>
ways to make change	<code>coins = denominations</code> , COMBINATIONS
dice roll sums	<code>coins = [1..6]</code> , PERMUTATIONS, with a count limit

The recurrence is always the same sentence: sum over every last move. What differs is what a move is, and whether moves are ordered.

Two mechanical things that matter more in counting than anywhere else.

The modulus. Counts grow fast — genuinely fast. The number of ways to make 5,000 from coins 1 to 200 has hundreds of digits. **Problems ask for the answer modulo $10^9 + 7$** , and the rule is to take the modulus at every addition, not at the end. In

Python nothing overflows so forgetting it merely makes things slow; **in Java or C++ it wraps silently and the answer is wrong.**

And the sanity check that catches almost every counting bug: run it on a tiny input you can enumerate by hand. Coins `[1, 2]`, amount 3. Two combinations, three permutations. **If your program says 3 when you wanted combinations, your loops are the wrong way round** — and that six-second test is worth more than re-reading the code.

4 The picture

The two nestings on the same input, filled in step by step:

```
coins [1, 2], amount 3

COMBINATIONS – coins outside
start:          ways = [1, 0, 0, 0]
process coin 1: ways = [1, 1, 1, 1]    only 1s exist yet
process coin 2: ways[2] += ways[0] → 2
                ways[3] += ways[1] → 2
                ways = [1, 1, 2, 2]
answer 2: {1,1,1} {1,2}

PERMUTATIONS – amount outside
start:          ways = [1, 0, 0, 0]
t=1: +coin1     ways[1] = ways[0]      = 1
t=2: +coin1 +coin2 ways[2] = ways[1] + ways[0] = 2
t=3: +coin1 +coin2 ways[3] = ways[2] + ways[1] = 3
                ways = [1, 1, 2, 3]
answer 3: {1,1,1} {1,2} {2,1}
```

Same six lines. Two loops swapped. 2 against 3.

Why coins-outside cannot produce `2+1` :

TIME →

coins outside:

```
[----- all of coin 1 -----][----- all of coin 2 -----]
```

^

by the time coin 2 is used, no more 1s will EVER be added.

So every set is built as: (some 1s) then (some 2s).

ONE canonical order per set. Counted once.

amount outside:

t=1: [coin1][coin2]

t=2: [coin1][coin2]

t=3: [coin1][coin2]

^

a 1 can follow a 2 and a 2 can follow a 1.

Both orders exist in the table. Counted separately.

Bhanumati's box, as the loop:

6 sweets, kinds L B P K

laddoos: 6 5 4 3 2 1 0 ← decide ALL of these first

|

++ barfis: 0..(6 - laddoos) ← never go back to laddoos

|

++ pedas: 0..(what's left)

|

++ kaju: whatever remains

(4,1,1,0) is reached exactly once, via laddoo=4, barfi=1, peda=1.

There is no path that places the peda before the barfi,
because the loop order forbids it.

The counting recurrence as a tree, showing the sum:

```

              ways(3)
             /   |   \
last coin 1 /   |   \ last coin 3
ways(2)   |   |   | ways(0)=1
 / \      |   |   |
...      |   |   |

```

$\text{ways}(t) = \text{SUM over every possible LAST move of } \text{ways}(t - \text{move})$

Minimising took the BEST child. Counting ADDS ALL of them.
That is the whole difference, and it is why double-counting
is invisible when minimising and fatal when counting.

5 The code, built step by step

Combinations, from the sentence

```
def change(amount: int, coins: list[int]) → int:
    ways = [0] * (amount + 1)
    ways[0] = 1                                # one way to make nothing
    return ways[amount]                        # (loops next)
```

`ways[0] = 1` is the seed. Every count in the finished table is that single `1`, propagated forwards through additions. Set it to `0` and the whole table stays zero.

```
    for coin in coins:                        # OUTSIDE: fixes the coin order
        for t in range(coin, amount + 1):
            ways[t] += ways[t - coin]
```

Coins outside. Say it out loud as you write it: “coins outside, so this counts combinations.”

`range(coin, ...)` starts at the coin’s value, which removes the need for a `coin ≤ t` guard.

Permutations, for contrast

```
def combination_sum_iv(nums: list[int], target: int) → int:
    ways = [0] * (target + 1)
    ways[0] = 1
    for t in range(1, target + 1):           # OUTSIDE: every coin at every amount
        for number in nums:
            if number ≤ t:
                ways[t] += ways[t - number]
    return ways[target]
```

The guard comes back, because the inner loop is now over numbers rather than over amounts, so `t - number` can go negative.

The modulus, done correctly

```
MOD = 10**9 + 7

def change_mod(amount: int, coins: list[int]) → int:
    ways = [0] * (amount + 1)
    ways[0] = 1
    for coin in coins:
        for t in range(coin, amount + 1):
            ways[t] = (ways[t] + ways[t - coin]) % MOD    # at EVERY addition
    return ways[amount]
```


At every addition, not at the end. Taking it once at the return gives the right answer in Python — where integers never overflow — and the wrong answer in every other language, where the intermediate value has already wrapped.

Climbing stairs, which is permutations in disguise

```
def climb_stairs(n: int) → int:
    ways = [0] * (n + 1)
    ways[0] = 1
    for t in range(1, n + 1):
        for step in (1, 2):
            if step ≤ t:
                ways[t] += ways[t - step]
    return ways[n]
```

Stairs is permutations and that is correct, because stepping 1-then-2 is a genuinely different climb from 2-then-1. **The problem's meaning decides the nesting**, not a rule you memorise.

And this is Fibonacci — `ways[t] = ways[t-1] + ways[t-2]` — which is a nice thing to notice out loud.

Decode ways, where the moves have conditions

```
def num_decodings(s: str) → int:
    n = len(s)
    ways = [0] * (n + 1)
    ways[0] = 1
    for i in range(1, n + 1):
        if s[i - 1] ≠ '0':
            ways[i] += ways[i - 1]
        if i ≥ 2 and '10' ≤ s[i - 2:i] ≤ '26':
            ways[i] += ways[i - 2]
    return ways[n]
```

The same sum-over-last-moves, with exactly two moves and a validity condition on each. `'0'` alone is not a letter, and `'06'` is not a valid two-digit code — which is why the string comparison starts at `'10'`.

Unique paths, which is counting on a grid

```
def unique_paths(rows: int, cols: int) → int:
    ways = [1] * cols
    for _ in range(1, rows):
        for c in range(1, cols):
            ways[c] += ways[c - 1]
    return ways[-1]
```

One row of state, because `ways[c]` before the update is the cell above and `ways[c-1]` is the cell to the left. **The same forwards-in-place trick as unbounded knapsack**, doing a different job.

The complete solution

```
"""Counting problems: combinations, permutations, and the family."""

MOD = 10**9 + 7

def change(amount: int, coins: list[int]) → int:
    """How many COMBINATIONS of coins make amount. Coins outside."""
    ways = [0] * (amount + 1)
    ways[0] = 1
    for coin in coins:
        # exactly one way to make nothing
        # each coin fully processed in turn
        for t in range(coin, amount + 1):
            ways[t] += ways[t - coin]
    return ways[amount]

def change_mod(amount: int, coins: list[int]) → int:
    """The same, taking the modulus at every addition."""
    ways = [0] * (amount + 1)
    ways[0] = 1
    for coin in coins:
        for t in range(coin, amount + 1):
            ways[t] = (ways[t] + ways[t - coin]) % MOD
    return ways[amount]

def combination_sum_iv(nums: list[int], target: int) → int:
    """How many PERMUTATIONS make target. Amount outside. (Name is misleading.)"""
    ways = [0] * (target + 1)
    ways[0] = 1
    for t in range(1, target + 1):
        # every number offered at every t
        for number in nums:
            if number ≤ t:
                ways[t] += ways[t - number]
    return ways[target]

def change_with_lists(amount: int, coins: list[int]) → list[list[int]]:
    """The actual combinations, for checking small cases by eye."""
    result: list[list[int]] = []

    def build(index: int, left: int, current: list[int]) → None:
        if left == 0:
            result.append(current[:])
            return
        if index == len(coins):
            return
        build(index + 1, left, current)
        if coins[index] ≤ left:
            current.append(coins[index])
            build(index, left - coins[index], current)
            current.pop()
            # skip this coin
            # reuse it

    build(0, amount, [])

    return result

def climb_stairs(n: int) → int:
    """Permutations of 1s and 2s. 1-then-2 differs from 2-then-1."""
    ways = [0] * (n + 1)
```

```

ways[0] = 1
for t in range(1, n + 1):
    for step in (1, 2):
        if step ≤ t:
            ways[t] += ways[t - step]
return ways[n]

def num_decodings(s: str) → int:
    """Sum over last moves, where a move is 1 digit (1-9) or 2 digits (10-26)."""
    n = len(s)
    ways = [0] * (n + 1)
    ways[0] = 1
    for i in range(1, n + 1):
        if s[i - 1] ≠ '0':
            ways[i] += ways[i - 1]
        if i ≥ 2 and '10' ≤ s[i - 2:i] ≤ '26':
            ways[i] += ways[i - 2]
    return ways[n]

def unique_paths(rows: int, cols: int) → int:
    """ways[cell] = ways[above] + ways[left], one row of state."""
    ways = [1] * cols
    for _ in range(1, rows):
        for c in range(1, cols):
            ways[c] += ways[c - 1]
    return ways[-1]

if __name__ == "__main__":
    print("combinations [1,2] → 3 :", change(3, [1, 2]))
    print("permutations [1,2] → 3 :", combination_sum_iv([1, 2], 3))
    print("the combinations      :", change_with_lists(3, [1, 2]))

    print("combinations [1,2,5]→5 :", change(5, [1, 2, 5]))
    print("the combinations      :", change_with_lists(5, [1, 2, 5]))
    print("impossible [2] → 3      :", change(3, [2]))

    print("climb 5 stairs           :", climb_stairs(5))
    print("decode '226'             :", num_decodings("226"))
    print("decode '06'             :", num_decodings("06"))
    print("paths 3x7                :", unique_paths(3, 7))

    big = change_mod(5000, list(range(1, 201)))
    print("ways to make 5000 mod      :", big)
    print("digits without the mod    :", len(str(change(5000, list(range(1, 201))))))

```

Run it and you get:

```

combinations [1,2] → 3 : 2
permutations [1,2] → 3 : 3
the combinations : [[1, 2], [1, 1, 1]]
combinations [1,2,5]→5 : 4
the combinations : [[5], [1, 2, 2], [1, 1, 1, 2], [1, 1, 1, 1, 1]]
impossible [2] → 3 : 0
climb 5 stairs : 8
decode '226' : 3
decode '06' : 0
paths 3x7 : 28
ways to make 5000 mod : 696884557
digits without the mod : 74

```

Line one against line two is the whole lesson, and `change_with_lists` proves it — two lists, not three.

And the last two lines are the modulus argument, made concrete. The real count of ways to make 5,000 from coins 1 to 200 has **74 digits**. In Python that is fine. In Java, a `long` holds 19 digits and the answer wrapped around fifty digits ago.

6 What it costs

Time. The same two nested loops as yesterday, and identical for both nestings.

```

combinations: len(coins) x amount cells
permutations: amount x len(coins) cells

```

Both are $O(\text{len}(\text{coins}) \times \text{amount})$. Swapping the loops changes the ANSWER, not the cost.

Concretely:

```

coins = 1..200, amount = 5,000
200 x 5,000 = 1,000,000 additions
→ about 0.15 seconds in Python

```

Space.

```
one row of amount + 1 integers
5,001 Python ints → about 40 KB
```

WITHOUT the modulus, those integers grow to 74 digits:
a 74-digit Python int is about 60 bytes rather than 28
→ the arithmetic itself gets slower as the numbers grow

With the modulus, every value stays under 10^9 and fits in a machine word. The modulus is a SPEED optimisation in Python and a CORRECTNESS requirement everywhere else.

How fast counts actually grow:

ways to make 100 from coins 1..10	= 6,292,069	7 digits
ways to make 1,000 from coins 1..100	= $\sim 10^{31}$	32 digits
ways to make 5,000 from coins 1..200	= $\sim 10^{73}$	74 digits

a Java long holds up to 9,223,372,036,854,775,807 19 digits
→ overflows well before amount 1,000, silently, with no warning

The enumeration cost, for contrast — why you count instead of listing:

```
change_with_lists(5, [1,2,5]) → 4 lists, instant
change_with_lists(100, [1..10]) → 6,292,069 lists
```

```
each list averages ~20 integers
6,292,069 x 20 x 28 bytes = ~3.5 GB
```

The COUNT is one integer. The LIST is three and a half gigabytes.
That gap is why counting is a separate problem from generating.

The two nestings compared on real numbers:

```
coins [1, 2, 5], amount 11
  combinations: 11
  permutations: 218
```

```
amount 30
  combinations: 58
  permutations: 5,508,222
```

Permutations explode because order multiplies. If your answer is enormous and you expected a small one, check the nesting first.

That last line is a genuinely useful debugging heuristic, and it is faster than reading the loops.

7 The traps

The swapped nesting, which produces a correct program answering the wrong question.

```
>>> ways = [0] * 4; ways[0] = 1
>>> for t in range(1, 4):                                # amount outside
...     for coin in [1, 2]:
...         if coin ≤ t:
...             ways[t] += ways[t - coin]
>>> ways[3]
3
```

Against the combinations answer of 2. **No error, no warning, and both numbers look reasonable.** The only defence is the hand-check: [1,2] to 3 is 2 combinations, 3 permutations, and if you know that pair you catch this in six seconds.

Forgetting ways[0] = 1.

```
>>> ways = [0] * 6
>>> for coin in [1, 2]:
...     for t in range(coin, 6):
...         ways[t] += ways[t - coin]
>>> ways
[0, 0, 0, 0, 0, 0]
```

Every count is zero, because nothing was ever 1 to add up. It reads as “no way to make this amount” rather than as a bug, which is what makes it survive review.

Using the 0/1 direction by habit.

```
>>> ways = [0] * 6; ways[0] = 1
>>> for coin in [1, 2]:
...     for t in range(5, coin - 1, -1):                # backwards
...         ways[t] += ways[t - coin]
>>> ways[5]
0
```

Zero ways to make 5 from 1s and 2s, when there are three. Backwards means each coin is usable at most once, so the only totals reachable at all are 1, 2 and 3 — and 5 is not among them. **It reads as “impossible” rather than as a bug**, which is exactly how it survives review.

Taking the modulus only at the end.

```
// Java
long[] ways = new long[amount + 1];
ways[0] = 1;
for (int coin : coins)
    for (int t = coin; t ≤ amount; t++)
        ways[t] += ways[t - coin];    // no modulus here
return (int)(ways[amount] % 1_000_000_007); // too late
```

The intermediate values already overflowed. A `long` wraps at about 9.2×10^{18} , and once it wraps the subsequent additions are meaningless — the final modulus is applied to garbage. **In Python this cannot happen, which makes it easy to forget when you switch languages.**

Applying the modulus when the problem asked for the real number.

```
>>> change_mod(5000, list(range(1, 201)))
696884557
```

A perfectly plausible-looking answer that is not the count. If a problem does not mention a modulus, do not add one — and if the expected answer fits in the language's integer type, the modulus is wrong, not merely unnecessary.

Counting permutations when the problem wants combinations, on a large input.

```
>>> change(30, [1, 2, 5])
58
>>> combination_sum_iv([1, 2, 5], 30)
5508222
```

Five orders of magnitude apart. If your answer is enormous and the expected one is small, the nesting is the first thing to check.

A zero coin, which gives an answer rather than an error.

```
>>> change(3, [0, 1])
2
```

Two, when the true answer is infinite — a coin of zero can be inserted any number of times, so there are infinitely many combinations and the question has no answer. What the code does instead is `ways[t] += ways[t - 0]`, which doubles each cell once and then propagates that, giving a finite number with no meaning. **Validate the denominations:** every coin must be positive, and that is one `if` at the top.

Very large amounts.


```
>>> ways = [0] * (10**9 + 1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
MemoryError
```

Same pseudo-polynomial ceiling as the last two days. The table is indexed by the amount, so the amount's magnitude decides whether the approach exists.

8 In the interview

How it gets asked

- *"How many combinations of coins make the amount?"* — LeetCode 518, the standard.
- *"Does the order of the loops matter?"* — asked directly, and it is the whole point.
- *"Now count sequences instead of sets."* — LeetCode 377.
- *"How many ways can you climb `n` stairs taking 1 or 2 at a time?"* — permutations, and it is Fibonacci.
- *"Why the modulus?"*
- *"Can you list them instead of counting them?"*

The first ninety seconds

“Before I write anything I want to settle one question, because it changes the answer and not the code: **am I counting sets or sequences?**

With coins one and two making three: the sets are `{1,1,1}` and `{1,2}` — **two**. The sequences are `1,1,1`, `1,2` and `2,1` — **three**. Coin Change II wants sets, so **two**.

The state is: `ways[t]` is the number of ways to make exactly `t`.

The base case is `ways[0] = 1` — **there is exactly one way to make nothing, which is to take no coins**. That single one is the seed; every other count in the table is it, propagated forwards through additions. If I set it to zero the entire table stays zero and the program reports that nothing is makeable.

The recurrence is: sum over every possible last coin. `ways[t] += ways[t - coin]`. Minimising took the best child; counting adds all of them, and that is the real difference — **when minimising, reaching the same state twice is harmless, and when counting it is fatal**.

So the nesting. Coins on the outside, amounts on the inside, and that is what makes it count sets. Coin one is completely processed before coin two is ever considered, so every combination is built in one fixed order — all the ones, then all the twos — and there is no path that adds a one after a two. **One canonical order per set, so each set is counted exactly once**.

Swap the loops and every coin is offered at every amount, so a one can follow a two and a two can follow a one, and I count sequences instead. **Both are correct programs. They answer different questions, and there is no error to tell them apart** — just a different number.

Cost is $O(\text{coins} \times \text{amount})$ time and $O(\text{amount})$ space, identical for both nestings — swapping the loops changes the answer, not the cost.

And the check I would actually run: coins one and two, amount three. If it says three, my loops are the wrong way round.”

The follow-ups

“**Explain why the loop order changes the answer.**”

“It is about which orderings the algorithm is capable of building, and the loop imposes that.

Coins outside. The algorithm processes coin one entirely — every amount from one to the target — and then moves to coin two and never returns. **So by the time a two is ever added, no more ones will ever be added after it.** Every combination therefore gets built in exactly one order: all its ones, then all its twos, then all its fives. **A canonical order, imposed by the loop, so each set is reached by exactly one path and counted once.**

Amounts outside. At every amount, every coin is offered. So `ways[3]` gets a contribution from `ways[2]` by adding a one, and from `ways[1]` by adding a two. **Those describe two different last moves,** and `1,1,1` / `1,2` come from the first while `2,1` comes from the second. Orderings are distinguished, so they are counted separately.

The way I would put it in one sentence: coins-outside fixes an order and counts each set once; amounts-outside lets every order happen and counts each one.

And there is a debugging heuristic worth knowing. Permutations explode much faster than combinations, because order multiplies. Coins one, two and five to thirty: **fifty-eight combinations, five and a half million permutations.** So if my answer is enormous and I expected a small one, the nesting is the first thing I check — faster than re-reading the loops.

The sanity test is coins one and two, amount three: two against three. Six seconds, and it catches this every time.”

“Why the modulus, and where exactly does it go?”

“Because the counts get genuinely enormous, and in most languages they overflow silently.

The number of ways to make five thousand from coins one to two hundred has seventy-four digits. A Java `long` holds nineteen. So the answer wrapped around seventy digits before it finished, and everything after that point is meaningless.

`10^9 + 7` is the usual modulus because it is prime and its square fits in a 64-bit integer — so you can multiply two reduced values without overflowing, which matters in problems that multiply. Here I only add, so the requirement is just that two reduced values sum without overflowing, which any modulus around a billion satisfies.

It goes at every addition, not at the end. `ways[t] = (ways[t] + ways[t - coin]) % MOD`. If I only take it at the return, the intermediate values have already wrapped and I am applying a modulus to garbage.

Python is the trap here, because integers never overflow, so the version with the modulus only at the end gives the right answer and I would not notice the bug until I wrote the same code in Java. **What the modulus does buy in Python is speed** — a ninety-one-digit integer is slower to add than a machine word, so the modulus keeps the arithmetic fast.

And the reverse mistake matters too: do not add a modulus the problem did not ask for. If the expected answer fits in the language’s integer type, a modulus produces a plausible-looking number that is simply wrong.”

“Could you list the combinations instead of counting them?”

“Yes, and it is a different algorithm with a completely different cost, which is the interesting part.

Counting is DP: one table, one integer per amount, $O(\text{coins} \times \text{amount})$.

Listing is backtracking, and no table helps, because the output itself is the size of the answer. I would recurse with an index and a remaining amount, and at each step either skip the current coin — moving to the next index — or take it and stay at the same index, since coins are reusable. **Staying at the same index on a take and never going back is the same canonical-order trick as coins-outside**, and it is what stops the same combination being emitted twice.

The cost difference is the point. The number of ways to make a hundred from coins one to ten is about **six point three million**. As a count, that is one integer. As a list, each combination averages about twenty numbers, so it is roughly **three and a half gigabytes** of output. **Same problem, one integer against three and a half gigabytes.**

So the practical answer is: list only when the count is small, and check the count first. If an interview asks me to enumerate, I would say that out loud — run the counting version, and if the answer is in the thousands, enumerate; if it is in the millions, ask what they actually want to do with them, because they almost certainly want the count, or the top few by some criterion, rather than all of them.

And if they want just one combination rather than all of them, that is a third algorithm again — keep a parent pointer during the DP and walk back, which is $O(\text{amount})$ extra space and gives one answer immediately.”

The model answer

“A vending machine has to report, for a given price, how many distinct ways a customer could pay it using the denominations the machine accepts. Design and implement it.”

“The first thing I want to pin down is what ‘distinct ways’ means, because there are two reasonable readings and they give different numbers.

Does inserting a five then a two count as different from a two then a five?

For a vending machine I would say no — the customer paid two coins, and the order they went in is not part of the answer. **So: combinations, not permutations.** I would state that assumption explicitly, because if the machine were logging insertion sequences the answer would be different.

Concretely, with coins of one and two and a price of three: two ways — three ones, or a one and a two. Not three.

The state: `ways[t]` **is the number of distinct sets of coins that total exactly `t`.**

Base case `ways[0] = 1` — one way to pay nothing, by inserting nothing — and that single one is what every other count is built from.

Recurrence: sum over every possible last coin. For each coin, going forwards from the coin’s value to the price, `ways[t] += ways[t - coin]`. **Forwards because coins are reusable** — a customer can insert three ones — which is the unbounded direction, not the each-coin-once direction.

And the coin loop is on the outside. That is the decision that makes it count sets: coin one is fully processed before coin two is considered, so every set is built as all-the-ones-then-all-the-twos, in one fixed order, and counted exactly once.

Swapping those loops would count insertion sequences instead, and nothing would report the difference — so I would say the nesting out loud while writing it, and test with one-and-two-to-three, which must give two.

Cost: `O(denominations * price)` **time**, `O(price)` **space**. Six denominations and a price of a hundred rupees expressed in rupees is six hundred additions. Instant, and I could precompute the whole table once at startup since the denominations do not change.

Two things about the numbers. For realistic vending-machine prices the counts are small — tens or hundreds — so **no modulus**, and I would not add one, because a modulus the problem did not ask for turns a correct answer into a plausible wrong one. **If the price were in paise and the denominations were fine, the counts would grow fast**, and then I would ask whether the answer needs to be exact.

What I would flag as the real-world gap: this counts ways ignoring how many coins the machine actually holds. **A real machine has a finite hopper**, so ‘you could pay with a hundred one-rupee coins’ is not a real option if the customer does not have a hundred coins. **If the question is about what the customer can do, this is right; if it is about what the machine can accept, there is a per-coin limit and it becomes bounded rather than unbounded** — either expand each coin into its available count and run the each-coin-once version, or add a dimension for coins remaining.

And if they asked me to list the ways rather than count them, that is backtracking, not DP, and the output can be enormous — eight point seven million combinations for a hundred from coins one to ten, about three and a half gigabytes. **I would count first and only enumerate if the count is small.**“

9 Recall card

Counting is unforgiving where minimising is not: two paths to the same state are harmless for `min` and fatal for a count. The recurrence is always **sum over every possible last move**, and the base case is always `ways[0] = 1` — one way to make nothing — which is the seed every count grows from.

The loop nesting decides the question, and both versions are correct programs. Coins outside = **combinations** (each coin fully processed before the next, so every set is built in one canonical order); amount outside = **permutations** (every coin offered at every amount, so orderings separate). `[1,2]` to 3 gives 2 and 3 — that six-second hand-check catches the mistake every time.

Coin Change II wants combinations. Combination Sum IV wants permutations despite its name. Climbing stairs wants permutations (1-then-2 is a different climb) and is Fibonacci.

Permutations explode: coins `[1,2,5]` to 30 is **58 combinations against 5,508,222 permutations** — so an answer far larger than expected means the nesting, before anything else.

Modulus at every addition, never only at the end — ways to make 5,000 from coins 1–200 has **74 digits** and a Java `long` holds 19. `10^9 + 7` because it is prime and its square fits in 64 bits. **In Python nothing overflows, so the bug only appears when you port the code** — and never add a modulus the problem did not ask for.

Counting is one integer; listing is backtracking and can be gigabytes (6.3M combinations $\approx$ 3.5 GB). Count first, enumerate only if the count is small.

Design a web crawler

1 What this is, and why they ask it

A crawler starts from a few URLs, fetches each page, extracts the links in it, and repeats. **That is a breadth-first search on a graph with a billion nodes that you do not own, cannot see in advance, and are only allowed to touch politely.**

They ask it because **it is the one design that is transparently an algorithm.** The DSA lesson you have been doing all month is the crawler: a frontier is a queue, a visited set is a hash set, and the crawl is BFS. **The interview is about everything that goes wrong when the queue is on a hundred machines and the graph fights back.**

And it fights back specifically. **Politeness** — hitting one site too fast is an attack, and you will be blocked or sued. **Traps** — infinite calendars, session IDs in URLs, pages that generate links forever. **Duplicates** — the same page under five URLs, and near-duplicates everywhere. **Scale** — a billion pages is petabytes, and the visited set alone does not fit in memory.

The politeness constraint is the one that separates candidates, because it inverts the obvious design. **The natural queue is one big FIFO, and that is exactly wrong** — it interleaves URLs from many hosts and hammers whichever host happens to be popular. **The right answer is a queue per host**, and being able to say why is worth more than the rest of the design.

By the end of this lesson you can design the frontier with politeness and priority, deduplicate at three levels, handle traps, size the storage and bandwidth, and say what you would do about robots.txt.

2 The story

The census work paid four hundred rupees a day and Nirmala took it because it was three weeks and she needed three weeks.

The instructions were simple. **Start at the corner of the lane. Every house, in order. Ask the questions, fill the form, and at each house also ask which other houses in the ward have people living in them**, because the ward map was eleven years old and half of it was wrong.

So by the end of the first morning she had a notebook with forty houses in it and she had visited nine.

The list grew faster than she could work through it. That was the first thing she understood, and it did not stop being true for three weeks.

The second thing took two days. **She kept arriving at houses she had already done.** The same house, because one person called it “the blue gate” and another called it “number 14” and a third called it “Sharma’s place behind the temple”, and she had written all three in her book as three separate entries.

So she started writing the number on the gatepost, always, whatever anyone called it. **One name per house, and check the list before walking.**

The third thing was the one that got her shouted at.

She did four houses on the same street in twenty minutes — they were next to each other and it seemed efficient — and the fourth woman had already heard from the other three, and she said something about the government sending people to bother the same street all morning, and she shut the door.

After that Nirmala rotated. One house here, then a house on the next street, then the street after, and back. **Slower, and nobody shut a door on her again.**

And the last one, which she never entirely solved.

There was a building on the main road with flats numbered 1 to 40, and at flat 40 somebody told her there was also a 40A, and at 40A they told her about 40B, **and she got to 40F before she understood that the man on the ground floor had been inventing them.**

She wrote a rule in the front of her book that evening. **Nothing past the fortieth flat in any building.** Not because it was right, but because there had to be a stopping point somewhere.

3 The idea in plain English

Nirmala built a crawler, including all four of the problems that matter, in the order they arrive.

The loop is four steps and it is genuinely this simple:

1. take a URL from the frontier
2. fetch the page
3. store it
4. extract the links, and add the unseen ones back to the frontier

The frontier is the queue. The seen set is her notebook. The crawl is BFS, and [day 127](#)'s traversal is exactly the algorithm — the difference is that the graph has a billion nodes, the edges are discovered by downloading them, and the nodes belong to other people.

Problem one: the same page under many URLs. Nirmala's blue gate.

Normalise the URL before doing anything with it. Lowercase the host, drop the default port, drop the fragment after `#`, sort the query parameters, strip the tracking ones, resolve `.` and `..`. `HTTP://Example.COM:80/a/..b?b=2&a=1#top` and `http://example.com/b?a=1&b=2` are the same page, and without normalisation you fetch it twice and store it twice.

Then hash the normalised URL and check a set. At a billion URLs the set does not fit in memory as strings, so store 8-byte hashes, or use a **Bloom filter** — [day 143](#)'s structure, doing exactly this job at Google scale. **A Bloom filter can say “probably seen” wrongly**, which means occasionally skipping a page you have never fetched. **For a crawler that is an acceptable loss** and it is worth saying so explicitly, because it is a rare case where a false positive is genuinely fine.

And separately, the same content under different URLs. Mirrors, print versions, `?utm_source=` variants that normalisation missed. **Hash the page content too** and skip the store if you have that hash already — typically 20–30% of the web is exact duplicates. **Near-duplicates** — the same article with a different advert — need **Sim-Hash**, which produces similar fingerprints for similar documents, so you compare Hamming distance rather than equality.

Problem two: politeness, and this is the one that shapes the architecture.

One FIFO queue is the wrong design. URLs from a popular host cluster together in the frontier — every page on a site links to many other pages on the same site — so a single queue drains hundreds of URLs for one host in a row and sends them all at once.

The fix is a queue per host, and a worker that owns a host at a time. Nirmala rotating streets. Each host gets its own FIFO; a worker picks a host, fetches one URL, waits the politeness delay, fetches the next. **One connection per host, and a delay of a second or so between requests** — or whatever `robots.txt` says via `Crawl-delay`.

And `robots.txt` is not optional. Fetch `https://host/robots.txt` before crawling a host, cache it for a day, and obey it. Ignoring it gets your IP blocked within hours and can get you sued, and an interviewer asking about crawlers is listening for whether you mention it unprompted.

Problem three: priority, because you cannot crawl everything.

The frontier grows faster than you drain it, permanently — that is Nirmala's first observation and it never stops being true. So the order matters more than the throughput.

A front queue set for priority, a back queue set for politeness. URLs enter a priority queue chosen by importance — PageRank, update frequency, domain authority, depth from the seed. A router then moves them into per-host back queues. Two separate structures doing two separate jobs, and combining them is the mistake: a single priority queue cannot enforce politeness, and a single host queue cannot express priority.

Problem four: traps, which is Nirmala's flat 40F.

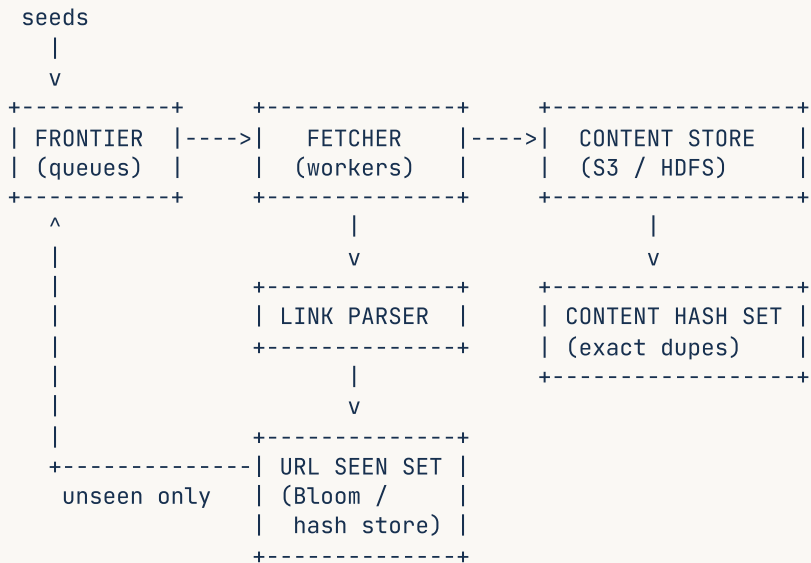
Infinite spaces exist and are usually not malicious. A calendar with a “next month” link generates URLs forever. A shop with faceted filters generates a URL per combination — five filters with ten values each is a hundred thousand pages of nearly identical content. Session IDs in URLs make every visit look new.

Four defences, and none of them is clever: a maximum depth from the seed, typically 10 to 20; a per-domain page cap; a URL length limit, since generated URLs grow; and detecting repeated path segments like `/a/b/a/b/a/b`. All four are heuristics, and Nirmala's rule about the fortieth flat is the honest description of them — not right, but there has to be a stopping point.

And the last piece: the crawl never finishes. Pages change, so the crawler recrawls, and the interesting part is deciding how often. A news homepage changes hourly; an archived PDF never changes. Track the observed change rate per URL and recrawl proportionally — and use `If-Modified-Since` and `ETag` so that an unchanged page costs a `304 Not Modified` and no body at all. That one header cuts recrawl bandwidth by roughly the fraction of pages that have not changed, which is most of them.

4 The picture

The whole system:



The cycle is BFS. Everything else is the frontier's internal design.

The frontier, which is the part that matters:

FRONT QUEUES – priority

```

[ p1 ] high (news, popular)
[ p2 ]
[ p3 ]
[ p4 ] low (deep, obscure)
    |

```

selector picks by priority
(p1 more often than p4)

BACK QUEUES – politeness

```

host: bbc.co.uk   [u1 u2 u3]
host: wikipedia   [u4 u5]
-----> host: smallblog   [u6]
          ^
          |
host: shop.example [u7 u8 u9]

```

ONE worker owns ONE host queue
and waits the delay between
its fetches

TWO structures because they do TWO jobs. A single priority queue
cannot enforce politeness; a single host queue cannot express priority.

Why one FIFO fails:

SINGLE FIFO after crawling one page of bigsite.com:

```
[ bigsite/a bigsite/b bigsite/c ... bigsite/z other.com/1 ]
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

100 workers pull these simultaneously

→ 100 concurrent requests to ONE host

→ that is indistinguishable from an attack

→ blocked within minutes

Pages link mostly to their own site, so the frontier is ALWAYS clustered by host. This is not an edge case; it is the normal state.

URL normalisation, which removes most duplicates for free:

```
raw: HTTP://Example.COM:80/a/..b/?utm_source=x&b=2&a=1#section
```

lowercase scheme+host	http://example.com:80/a/..b/?utm_source=x&b=2&a=1#section
drop default port	http://example.com/a/..b/?utm_source=x&b=2&a=1#section
resolve . and ..	http://example.com/b/?utm_source=x&b=2&a=1#section
drop fragment	http://example.com/b/?utm_source=x&b=2&a=1
strip tracking params	http://example.com/b/?b=2&a=1
sort query params	http://example.com/b/?a=1&b=2

→ ONE canonical string, hashed to 8 bytes for the seen set.

The three levels of duplicate detection:

Level 1: SAME URL	normalise + hash	catches ~30% of links
Level 2: SAME CONTENT	SHA-256 of the body	catches ~25% of pages
Level 3: NEARLY SAME	SimHash + Hamming < 3	catches templated pages, print versions, mirrors

Level 1 saves the FETCH. Levels 2 and 3 only save the STORE and the index – the bytes have already crossed the network.

5 How it actually works

URL normalisation

```
from urllib.parse import urlsplit, urlunsplit, parse_qsl, urlencode
import posixpath, hashlib

TRACKING = {"utm_source", "utm_medium", "utm_campaign", "fbclid", "gclid"}

def normalise(url: str) → str:
    parts = urlsplit(url)
    scheme, host = parts.scheme.lower(), parts.hostname or ""
    if (scheme, parts.port) in (("http", 80), ("https", 443)):
        netloc = host
    else:
        netloc = f"{host}:{parts.port}" if parts.port else host
    path = posixpath.normpath(parts.path or "/") # resolves . and ..
    query = urlencode(sorted(
        (k, v) for k, v in parse_qsl(parts.query) if k not in TRACKING))
    return urlunsplit((scheme, netloc, path, query, "")) # "" drops the fragment
```

Sorting the query parameters is the one people forget, and it matters: `?a=1&b=2` and `?b=2&a=1` are the same page and hash differently without it.

`posixpath.normpath` resolves `..` — and note it also strips a trailing slash, which for some servers is a different page. **That is a deliberate trade:** you lose a few legitimate pages and you kill a large class of duplicates.

The seen set at scale

```
class SeenUrls:
    """8-byte hashes in a sharded key-value store. Exact, and it fits."""
    def __init__(self, store) → None:
        self.store = store

    def add_if_new(self, url: str) → bool:
        digest = hashlib.blake2b(normalise(url).encode(), digest_size=8).digest()
        return self.store.set_if_absent(digest, b"1") # atomic
```

`set_if_absent` must be atomic, or two workers that discover the same URL simultaneously both think they are first and it gets fetched twice. Redis `SETNX` or a conditional put does this in one round trip.

The Bloom filter alternative, when even 8 bytes per URL is too much:

```

class SeenBloom:
    """~1.2 GB for 1 billion URLs at a 1% false-positive rate."""
    def __init__(self, bloom) → None:
        self.bloom = bloom

    def add_if_new(self, url: str) → bool:
        key = normalise(url)
        if key in self.bloom:
            return False
        self.bloom.add(key)
        return True

```

The false positive means silently never crawling a page, and there is no way to detect it. For a crawler that is acceptable — the web is larger than you will crawl anyway — and it is worth saying that trade out loud rather than presenting the Bloom filter as free.

The frontier

```

import time, heapq
from collections import defaultdict, deque

class Frontier:
    def __init__(self) → None:
        self.front: list[tuple[int, str]] = []
        self.back: dict[str, deque[str]] = defaultdict(deque)
        self.ready: list[tuple[float, str]] = []

    def add(self, url: str, priority: int) → None:
        heapq.heappush(self.front, (priority, url))

```

Two structures, and the router between them is what moves URLs from one to the other, keeping each back queue non-empty without letting any host’s queue grow unboundedly.

```

def next_url(self) → str | None:
    """Pop the host whose politeness delay expired earliest."""
    if not self.ready:
        return None
    ready_at, host = self.ready[0]
    if ready_at > time.monotonic():
        return None
    heapq.heappop(self.ready)
    url = self.back[host].popleft()
    if self.back[host]:
        heapq.heappush(self.ready, (time.monotonic() + self.delay(host), host))
    return url

```

The `ready` heap keyed on “when may I next touch this host” is the whole politeness mechanism. A host is simply not available until its delay has passed. `time.monotonic()`, not wall clock, for [day 123](#)’s reason.

robots.txt

```
from urllib.robotparser import RobotFileParser

class RobotsCache:
    def __init__(self, ttl: int = 86400) → None:
        self.cache: dict[str, tuple[RobotFileParser, float]] = {}
        self.ttl = ttl

    def allowed(self, url: str, agent: str = "MyCrawler") → bool:
        host = urlsplit(url).netloc
        entry = self.cache.get(host)
        if entry is None or time.time() - entry[1] > self.ttl:
            parser = RobotFileParser()
            parser.set_url(f"https://{host}/robots.txt")
            try:
                parser.read()
            except Exception:
                return True # unreachable robots.txt: allow
            self.cache[host] = (parser, time.time())
            entry = self.cache[host]
        return entry[0].can_fetch(agent, url)
```

The cache is not an optimisation, it is politeness — fetching `robots.txt` before every request would itself be hammering the host. **One day is the conventional TTL.**

And “unreachable means allow” is a choice. Google treats a `5xx` on `robots.txt` as *disallow everything* for a while, which is more conservative. **Either is defensible; say which you picked.**

Content deduplication

```
def content_hash(body: bytes) → bytes:
    return hashlib.sha256(body).digest() # exact duplicates

def simhash(text: str, bits: int = 64) → int:
    """Similar documents get similar fingerprints."""
    vector = [0] * bits
    for token in text.split():
        h = int.from_bytes(hashlib.blake2b(token.encode(), digest_size=8).digest(), "big")
        for i in range(bits):
            vector[i] += 1 if (h >> i) & 1 else -1
    result = 0
    for i, weight in enumerate(vector):
        if weight > 0:
            result |= 1 << i
    return result

def near_duplicate(a: int, b: int, threshold: int = 3) → bool:
    return bin(a ^ b).count("1") ≤ threshold # Hamming distance
```

SimHash is the interesting one, because unlike a normal hash, **similar inputs give similar outputs** — each token votes on each bit, so changing a few tokens flips only a few bits. **Two documents within a Hamming distance of 3 are near-duplicates**,

which catches the same article with a different advert.

Trap detection

```
MAX_DEPTH, MAX_URL_LENGTH, MAX_PER_DOMAIN = 20, 2000, 1_000_000

def is_trap(url: str, depth: int, domain_count: int) → bool:
    if depth > MAX_DEPTH or len(url) > MAX_URL_LENGTH:
        return True
    if domain_count > MAX_PER_DOMAIN:
        return True
    segments = urlsplit(url).path.split("/")
    for i in range(len(segments) - 4): # /a/b/a/b/a/b
        if segments[i:i + 2] == segments[i + 2:i + 4] == segments[i + 4:i + 6]:
            return True
    return False
```

Every one of these is a heuristic that will occasionally reject a real page, and that is the correct trade. An unbounded crawl of one calendar costs more than the handful of legitimate deep pages you lose.

The real systems

Apache Nutch	the open-source reference; Hadoop-based
Scrapy	Python, single-machine to medium scale
Heritrix	the Internet Archive's crawler
Common Crawl	a public crawl of billions of pages, on S3
Googlebot	the one everyone is really being asked about

Common Crawl is worth naming, because “I would check whether Common Crawl already has this” is a genuinely good answer to a crawler question with a narrow purpose.

6 The numbers

The crawl budget.

```
1 billion pages per month

1,000,000,000 / 30 days / 86,400 s = ~386 pages/second

per machine: ~100 concurrent fetches, ~2 s average fetch
              = 50 pages/second/machine
→ 386 / 50 = 8 machines

round up for retries, DNS, parsing: ~20 machines
```

Under twenty machines for a billion pages a month is smaller than people expect, and it is worth saying, because it reframes the problem: **the bottleneck is not compute, it is politeness and storage.**

Bandwidth.

average HTML page: ~100 KB compressed (~500 KB raw)

386 pages/s x 500 KB = 193 MB/s raw
= 1.5 Gbps sustained, inbound

per month: 1,000,000,000 x 500 KB = 500 TB downloaded

storing it compressed, ~100 KB per page:

1,000,000,000 x 100 KB = 100 TB per month of crawl
on S3 at \$0.023/GB = ~\$2,300/month, growing every month

Storage is the recurring cost and it accumulates, which is why a real crawler stores the extracted text rather than the raw HTML for anything but the most recent crawl.

The seen set, which is the memory question.

1 billion URLs

as full strings, average 80 bytes	= 80 GB	does not fit in memory
as 8-byte hashes	= 8 GB	fits on ONE machine
as a Bloom filter, 1% false positive (10 bits per element)	= 1.2 GB	fits comfortably
as a Bloom filter, 0.1%	= 1.8 GB	

Ten bits per element for 1% is the number to remember, and the false-positive rate is the price: **1% of a billion is ten million pages silently never crawled.** Whether that is acceptable is the design question, and for a general web crawl it is.

Politeness, and what it actually costs.

one URL per host per second

a site with 100,000 pages
= 100,000 seconds = 27.8 HOURS to crawl that one site

→ to crawl 386 pages/second overall, you need
386 DIFFERENT HOSTS being crawled concurrently

That is the real constraint. Not bandwidth, not CPU – host diversity.

That calculation is the best thing to say in this interview, because it explains why the frontier is designed the way it is: **the system is not throughput-bound, it is bound by how many distinct hosts it can have in flight.**

Recrawl economics.

1 billion pages, recrawled monthly, 80% unchanged

WITHOUT If-Modified-Since:

1,000,000,000 x 500 KB = 500 TB downloaded

WITH If-Modified-Since:

200,000,000 changed x 500 KB = 100 TB

800,000,000 unchanged x ~1 KB (304 response) = 0.8 TB

total ~101 TB

→ 5x less bandwidth, from one request header.

Duplicate detection's payoff.

of 1 billion fetched pages:

~25% exact duplicates = 250,000,000 pages

at 100 KB stored each = 25 TB of storage avoided

= ~\$575/month saved, forever, and a cleaner index

BUT the bytes were already downloaded. Content dedup saves
STORAGE, not BANDWIDTH. Only URL dedup saves the fetch.

That distinction is worth stating precisely, because candidates often claim content hashing saves bandwidth, and it does not.

DNS, which is the surprise bottleneck.

one DNS lookup per host, ~50-200 ms uncached

crawling 386 pages/s across 400 hosts, no cache

→ 386 lookups/s at 100 ms each

→ DNS becomes the slowest step in the pipeline

with a local caching resolver and a long TTL:

→ ~1 lookup per host per hour

→ 400 lookups/hour, negligible

A dedicated DNS cache is a real component of a real crawler, and mentioning it signals experience.

7 The trade-offs

Breadth-first against priority-first. Pure BFS is simple and treats a spam farm exactly like Wikipedia. **Priority ordering by PageRank or domain authority gets useful pages sooner**, and it costs a scoring pipeline and can starve legitimate small sites forever. **The frontier's two-tier design exists precisely because you want both**, and neither structure alone can do it.

Bloom filter against exact hashes for the seen set. 1.2 GB against 8 GB, and the price is that **1% of URLs are silently never crawled** — no error, no log line, no way to know which. **For a web crawler that is fine** and for a crawler with a specific target list it is not, so the answer depends on whether missing pages is a bug or a rounding error.

Politeness against throughput, which is not really a trade. You can crawl a site faster; you will be blocked, and possibly sued, and the pages you lose permanently exceed what you gained. **The real lever is host diversity** — crawl more sites concurrently, not one site harder — and a candidate who frames it that way has understood the constraint.

Trap heuristics reject real pages. A depth limit of 20 loses genuinely deep archives; a per-domain cap truncates large legitimate sites; a URL length limit drops pages with long legitimate query strings. **All four are wrong sometimes and all four are necessary**, and the honest framing is Nirmala's: there has to be a stopping point, and it is chosen rather than derived.

Recrawl frequency against freshness. Recrawling everything monthly wastes most of the bandwidth on pages that did not change; recrawling by observed change rate is better and needs per-URL history and a model that can be wrong in both directions. **If-Modified-Since makes the mistake cheap** — an unnecessary recrawl of an unchanged page costs a 304 and no body — which is why it matters more than the scheduling cleverness.

Storing raw HTML against storing extracted text. Raw HTML is five times larger and lets you re-parse when your extractor improves, which it will. **Extracted text is small and throws away information you cannot get back without recrawling.** The usual answer is both, with raw HTML on cold storage and a short retention.

When would I not build this? When someone has already crawled it. Common Crawl publishes billions of pages on S3 and downloading it is dramatically cheaper than crawling. When the target is a handful of sites with APIs — **an API is better than scraping in every respect** and the politeness problem disappears. And when the con-

tent is behind JavaScript, where a fetcher must run a headless browser, and **the cost per page goes up by roughly fifty times** — which changes the machine count from twenty to a thousand and should be raised before designing anything else.

8 In the interview

How it gets asked

- *“Design a web crawler.”* — the standard prompt, usually with a scale like a billion pages a month.
- *“How do you avoid crawling the same page twice?”* — three levels, and which one saves what.
- *“How do you avoid overwhelming a website?”* — the politeness question, and the queue design.
- *“What is a crawler trap and how do you handle it?”*
- *“How do you know when to recrawl?”*
- *“How does this run on a hundred machines?”*

“The loop is four steps and it is genuinely simple: **take a URL from the frontier, fetch it, store it, extract the links, add the unseen ones back.** That is breadth-first search on a graph with a billion nodes, and the frontier is the queue and the seen set is the visited set.

Everything interesting is in the constraints, so let me take the four that shape the design.

First, duplicates, at three levels. Same URL — normalise it and hash it: lowercase the host, drop the default port and the fragment, sort the query parameters, strip tracking parameters, resolve dot-dot. **A billion URLs as strings is eighty gigabytes; as eight-byte hashes it is eight, and as a Bloom filter about 1.2.** Then same content under different URLs — hash the body, which catches maybe a quarter of the web. Then near-duplicates — SimHash, where similar documents give similar fingerprints and you compare Hamming distance. **Only the first level saves the fetch;** the other two save storage, and I would say that explicitly because it is often claimed the other way.

Second, politeness, and this is the constraint that inverts the obvious design. One big FIFO queue is wrong, because pages link mostly to their own site, so the frontier is always clustered by host — a hundred workers pull a hundred URLs for the same host and send them simultaneously, which is indistinguishable from an attack. **So: a queue per host, and one worker owns a host at a time,** with a delay between fetches. And `robots.txt`, fetched once per host and cached for a day.

Third, priority, because the frontier grows faster than I drain it, permanently. So the order matters more than the throughput. **Two tiers: front queues for priority, back queues for politeness,** with a router between them — because a single priority queue cannot enforce politeness, and a single host queue cannot express priority.

Fourth, traps. Calendars with a next-month link forever, faceted shop filters, session IDs in URLs. **Depth limit, per-domain cap, URL length limit, repeated-segment detection** — all heuristics, all occasionally wrong, all necessary.

And here is the sizing point I would lead with, because it reframes the problem. A billion pages a month is 386 pages a second, which is about twenty machines — small. **But at one request per host per second, 386 pages a**

second means 386 distinct hosts in flight at all times. The system is not bandwidth-bound or CPU-bound. **It is bound by host diversity**, and that is what the frontier is really managing.

One thing I would ask first: does this need to render JavaScript? Because a headless browser is about fifty times the cost per page, and that changes twenty machines into a thousand.”

The follow-ups

“How do you avoid hitting one website too hard?”

“This is the constraint that decides the architecture, so let me say why the obvious design fails first.

A single FIFO frontier does not work, and not as an edge case — as its normal state. Pages link overwhelmingly to their own site, so after fetching one page of a large site I have added a hundred URLs for that same host, contiguously, to the queue. **A hundred workers pull from the front of that queue and issue a hundred simultaneous requests to one server.** From the site’s perspective that is a denial of service, and I get blocked within minutes.

So the back of the frontier is one queue per host, and the invariant is that **at most one worker is assigned to a host at a time.** That worker fetches one URL, waits the politeness delay, fetches the next.

The mechanism is a heap keyed on ‘the earliest time I may next touch this host’. A host that was fetched half a second ago is simply not in the ready set yet. **And it must use a monotonic clock,** or an NTP correction makes hosts available early or late.

The delay comes from `robots.txt` if it specifies `Crawl-delay` , and otherwise a default — a second is conventional, and adapting it to the server’s observed response time is better, because a slow server is telling you something.

`robots.txt` itself: fetch once per host, cache for a day, obey it. The cache is politeness, not optimisation — checking it before every request would itself be hammering the host. **And I would decide deliberately what to do when it is unreachable.** I would allow, because a transient error should not lose me a site; Google disallows for a period, which is more conservative. **Either is defensible; not having decided is not.**

And the consequence, which is the number worth saying: at one URL per host per second, a site with a hundred thousand pages takes twenty-eight hours to crawl. **So overall throughput comes entirely from host diversity** — to sustain 386 pages a second I need 386 hosts in flight. **Crawling more sites concurrently, never one site harder.**“

“What is a crawler trap, and what do you do?”

“An infinite or effectively infinite space of URLs that the crawler will happily walk forever, and most of them are not malicious — they are ordinary sites doing ordinary things.

The classic is a calendar. Every month page has a ‘next month’ link, so there is a valid, distinct, fetchable URL for December 40,000. Nothing is wrong with the site; the space is just unbounded.

Faceted navigation is the expensive one. A shop with five filters of ten values each generates a hundred thousand URLs of nearly identical content, all reachable, all distinct. **That is a hundred thousand fetches to learn nothing.**

Session IDs in URLs make every visit look like new pages, so the same site is crawled endlessly under different identifiers.

And deliberate ones exist — pages that generate random links to poison crawlers.

Four defences, and I would be honest that none is clever. A **maximum depth** from the seed, around ten to twenty. A **per-domain page cap**. A **URL length limit**, because generated URLs grow. And **repeated-path-segment detection** for `/a/b/a/b/a/b`.

Beyond those, content-based detection helps: if a hundred URLs on one domain all produce near-identical content by SimHash, that is a facet trap, and I would deprioritise the whole pattern rather than the individual URLs.

Every one of these rejects real pages sometimes. A depth limit loses genuinely deep archives; a domain cap truncates large legitimate sites. **I would say plainly that they are heuristics chosen rather than derived, and that the trade is right** — an unbounded crawl of one calendar costs far more than the handful of deep pages lost. **The right way to run them is with logging**, so a site hitting a cap is visible and can be whitelisted, rather than silently truncated forever.”

“How do you decide when to recrawl a page?”

“The crawl never finishes — pages change — so this is really the whole steady-state problem, and the naive answer wastes most of the budget.

Recrawling everything monthly means about eighty percent of the bandwidth is spent on pages that did not change. At a billion pages that is four hundred terabytes downloaded for nothing.

The cheap fix, which matters more than the clever one: `If-Modified-Since` and `ETag`. Send the timestamp or the etag from last time; an unchanged page answers `304 Not Modified` with no body. **A kilobyte instead of five hundred.** At eighty percent unchanged that turns five hundred terabytes into about a hundred and one — **five times less bandwidth from one request header**, and it makes an unnecessary recrawl almost free, which changes how careful the scheduling has to be.

Then the scheduling itself: track the observed change rate per URL and recrawl proportionally. A news homepage changes hourly and gets crawled hourly. An archived PDF has never changed in twenty visits and gets crawled every few months. **The estimator is simple** — exponential moving average of the interval between observed changes — and it self-corrects, because a page that changes when you did not expect it moves up.

Two signals I would use beyond that. The sitemap, if the site publishes one, because `lastmod` is the site telling you directly and it is free. And **importance**, because a page nobody looks at does not need to be fresh — freshness is only valuable in proportion to traffic.

And I would set a floor and a ceiling. Never more often than the politeness delay allows, and never less often than some maximum, because a page that has been static for years can still change and a crawler that has written it off will never notice.”

The model answer

“Design a crawler for a search engine: one billion pages a month, respecting site owners, and the index must be reasonably fresh.”

“Let me start with the sizing, because one number reframes the whole problem.

A billion pages a month is 386 pages a second. At a hundred concurrent fetches per machine and a two second average fetch, that is fifty pages a second per machine — **about eight machines, twenty with headroom.** Compute is not the problem.

But politeness is one request per host per second. So 386 pages a second requires 386 distinct hosts in flight, continuously. That is the actual constraint, and everything in the frontier design follows from it.

The frontier, in two tiers. Front queues hold URLs by priority — domain authority, depth from seed, observed change rate. A router moves them into back queues, one per host. A worker takes the host whose politeness delay expired earliest, fetches one URL, and the host goes back into the heap with a new ready time. **Two structures because they do two jobs,** and the router’s real task is keeping enough distinct hosts populated to sustain the throughput.

Duplicate detection at three levels, and I would name what each one saves. URL normalisation plus an eight-byte hash in a sharded store — the check must be atomic, or two workers both think they are first. **This is the only level that saves a fetch.** Then a SHA-256 of the body, catching maybe a quarter of pages as exact duplicates and saving twenty-five terabytes a month of storage. Then Sim-Hash with a Hamming threshold of three for near-duplicates — print versions, mirrors, the same article with a different advert. **Levels two and three save storage and index quality, never bandwidth,** because the bytes have already arrived.

Seen set sizing: a billion URLs is eighty gigabytes as strings, eight as hashes, 1.2 as a Bloom filter at one percent. I would use exact hashes, because **one percent of a billion is ten million pages silently never crawled, with no way to know which** — and for a search engine, invisible gaps in coverage are worse than eight gigabytes.

Politeness: `robots.txt` per host, cached a day, obeyed, with `Crawl-delay` honoured. A `User-Agent` that identifies the crawler and links to a page explaining it, and a working way for site owners to complain. **That last part is not decoration** — it is the difference between a crawler that operates for years and one that gets IP-blocked across the web.

Traps: depth cap of twenty, per-domain cap, URL length limit, repeated-segment detection, plus content-based facet detection via SimHash. Logged, so a truncated site is visible rather than silently lost.

Freshness, which the prompt asks for specifically. Per-URL change-rate estimation with an exponential moving average, sitemaps' `lastmod` where available, and importance weighting — freshness matters in proportion to traffic. **And** `If-Modified-Since` on every recrawl, which turns eighty percent of the recrawl budget into 304 s and cuts monthly bandwidth from five hundred terabytes to about a hundred.

Storage: raw compressed HTML to object storage at roughly a hundred kilobytes a page — a hundred terabytes a month, about \$2,300 — with a short retention, plus extracted text kept indefinitely. Keeping raw HTML for a while matters because the extractor will improve and re-parsing is free while recrawling is not.

Distribution: partition the frontier by host hash, so each machine owns a set of hosts entirely. That makes politeness a local invariant rather than a distributed one, which is the single biggest simplification available here — no coordination is needed to guarantee one worker per host. The seen set is sharded separately by URL hash.

Two things I would raise before building. Does this need JavaScript rendering? A headless browser is roughly fifty times the cost per page, turning twenty machines into a thousand, so it should be a targeted decision per site rather than a default. **And a dedicated caching DNS resolver**, because at 386 fetches a second, uncached lookups at a hundred milliseconds each become the slowest step in the pipeline.

What I would monitor: pages per second per machine, distinct hosts in flight — which is the real health metric — the 304 rate on recrawls, the frontier size, and `4xx / 5xx` rates per host, because a spike on one host usually means I am being blocked and should back off before somebody has to email me.”

9 Recall card

The loop is BFS: frontier → fetch → store → extract links → add unseen. **The frontier is the whole design.**

Politeness inverts the obvious structure. A single FIFO clusters by host (pages link to their own site), so a hundred workers hit one server at once. **Queue per host, one worker per host, a heap keyed on next-allowed time** (monotonic clock). `robots.txt` per host, cached a day, obeyed — and decide in advance what an unreachable one means.

Two-tier frontier: front queues for priority, back queues for politeness, with a router between — a single priority queue cannot enforce politeness and a single host queue cannot express priority.

Three levels of dedup, and only the first saves bandwidth. Normalise + hash the URL (sort query params!); SHA-256 the body (~25% exact dupes → 25 TB/month saved); **SimHash + Hamming ≤ 3** for near-duplicates. **Seen set: 80 GB as strings, 8 GB as hashes, 1.2 GB as a Bloom filter at 1%** — and 1% of a billion is ten million pages silently never crawled.

The number that reframes the problem: 386 pages/s at 1 req/host/s means 386 distinct hosts in flight. Not bandwidth-bound, not CPU-bound — **host-diversity-bound**. A 100,000-page site takes 28 hours.

Traps are calendars, facets and session IDs, not malice. Depth cap, per-domain cap, URL length, repeated segments — all heuristics, all sometimes wrong, all necessary; log them. `If-Modified-Since` **cuts recrawl bandwidth $\sim 5\times$** (500 TB → 101 TB at 80% unchanged). **Partition the frontier by host** so politeness stays a local invariant.

DSA topic: Coin Change II: counting **System design topic:** Design a web crawler

Code these, in this order

One rule for the whole set: **before writing the loops, decide out loud whether you are counting sets or sequences, and name the evidence.** Then run the tiny hand-checkable case first. Two combinations, three permutations, for `[1, 2]` to 3 — that pair is the fastest bug-catcher in dynamic programming.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Coin Change II	LeetCode 518 (Medium)	Combinations, and the coins-outside nesting.
2	Combination Sum IV	LeetCode 377 (Medium)	Permutations, and a name that points the wrong way.
3	Climbing Stairs	LeetCode 70 (Easy)	Permutations you have been writing as Fibonacci all along.
4	Decode Ways	LeetCode 91 (Medium)	Sum over last moves, with validity conditions.
5	Unique Paths	LeetCode 62 (Medium)	Counting on a grid, and the one-row collapse.
6	Number of Dice Rolls With Target Sum	LeetCode 1155 (Medium)	Permutations with a fixed count — a second dimension.

On problem 1, run the tiny case before anything else

`coins = [1, 2]`, `amount = 3`. It must give 2. If it gives 3, swap the loops. Then list both sets by hand and confirm the program is counting what you listed.

On problem 1, write the enumerator too

Write a backtracking function that returns the actual lists, and check it against the count on five small inputs. Then run the enumerator on `amount = 100`, `coins = 1..10` and record how long it takes and how much memory it uses. **Say in one sentence why counting is a separate problem from listing.**

On problems 1 and 2, swap the nesting deliberately

Take your combinations solution and swap the two loops. Confirm you get the permutations answer. Then swap problem 2's loops and confirm you get combinations. **Write one sentence explaining why coins-outside cannot produce `2+1`.**

On problem 3, notice what you have written

After solving it with the counting loops, write out `ways[t] = ways[t-1] + ways[t-2]`. **Say what sequence that is.** Then say why stairs is permutations and why that is correct rather than a mistake.

On problem 4, get the base case right first

Write `ways[0] = 1` and say what “one way to decode nothing” means. Then handle `'0'` and `'06'` and `'100'` — record what each returns. **Say which of those three is the one people get wrong.**

On problem 6, count the state space

State it as a sentence with both dimensions. Count the states for 30 dice, 30 faces, target 1000. Multiply by the work per state. Say whether it is viable, and what the modulus is for.

Then the modulus drill

Take problem 1, remove the modulus, and compute the exact answer for `amount = 5000` with coins `1..200`. Record the number of digits. Then say what a Java `long` would have done, and where exactly the modulus has to go.

The sets-or-sequences drill

For each, say which it counts and give the evidence in one sentence:

1. Coin Change II.
2. Combination Sum IV.
3. Climbing stairs.
4. Ways to make change at a shop counter.
5. Dice rolls summing to a target.

The nesting drill

1. Say what coins-outside guarantees, in terms of order.
2. Say what amount-outside allows.
3. Give both answers for `[1,2]` to 3 and list the arrangements.

4. Give the debugging heuristic about answer size, with the `[1, 2, 5]` to 30 numbers.
5. Say what the cost difference between the two nestings is. (There isn't one — say that.)

The base-case drill

1. Say what `ways[0] = 1` means in words.
2. Say what the table does without it, and how that reads from outside.
3. Say what the equivalent is for Decode Ways.
4. Say why minimising uses `0` here and counting uses `1`.

The modulus drill

1. Give the digit count for 5,000 from coins 1..200.
2. Say what a Java `long` holds and when it wraps.
3. Say exactly where the modulus goes and why not at the end.
4. Say why `10^9 + 7` specifically.
5. Say what happens if you add a modulus the problem did not ask for.

The counting-versus-listing drill

1. Give the count and the list size for 100 from coins 1..10.
2. Say what algorithm each needs.
3. Say what trick the enumerator uses to avoid emitting duplicates.
4. Say what you would do if asked to enumerate a million results.

The break-it drill

Trigger each and record the exact output or error:

1. Swapped loop nesting on a combinations problem.
2. Missing `ways[0] = 1`.
3. The backwards inner loop.
4. A coin of zero.
5. Modulus only at the return, in a language with fixed-width integers.
6. A modulus on a problem that did not ask for one.
7. `[0] * (10**9 + 1)`.

Six of the seven give no error at all. Name them.

The loop drill

1. Give the four steps of the crawl loop.
2. Say what the frontier and the seen set are, in graph terms.
3. Say what makes this harder than a normal BFS, in three points.

The dedup drill

1. Name the three levels and what each catches.
2. Say which one saves bandwidth and which two do not.
3. List six normalisation steps.
4. Say which normalisation step people forget, and why it matters.
5. Explain SimHash in two sentences — what makes it different from a normal hash.
6. Give the seen-set sizes for strings, hashes and a Bloom filter.
7. Say what a Bloom filter false positive costs here, and whether it is acceptable.

The politeness drill

1. Say why a single FIFO frontier fails, and why it is the normal case rather than an edge case.
2. Describe the per-host queue design and the ready heap.
3. Say what clock the heap uses and why.
4. Say what you do about `robots.txt`, including the TTL and the unreachable case.
5. Compute how long a 100,000-page site takes at one request per second.
6. Say what actually bounds overall throughput.

The frontier drill

1. Say why there are two tiers and what each does.
2. Say what the router's real job is.
3. Say why a single priority queue cannot do both.
4. Say how you would partition the frontier across machines, and what that buys.

The trap drill

1. Give three traps and say which are malicious. (Mostly none — say that.)
2. Name the four heuristics.
3. Say what content-based detection adds.
4. Say what these heuristics cost, and why the trade is still right.
5. Say what you would do so that a truncated site is not lost silently.

The freshness drill

1. Say what fraction of a monthly recrawl is wasted, and on what.
2. Give the `If-Modified-Since` arithmetic, both totals.
3. Describe the change-rate estimator in one sentence.
4. Name two other signals you would use.
5. Say why there must be a floor and a ceiling.

The sizing drill

1. Compute pages per second and machines for 1 billion pages a month.
 2. Compute inbound bandwidth and monthly storage.
 3. Compute the host-diversity requirement.
 4. Say what DNS does if you do not cache it.
 5. Say what JavaScript rendering costs and what it does to the machine count.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *How many combinations of coins make the amount?* Sets versus sequences with the `[1,2]` to 3 numbers, the state, `ways[0] = 1` and what it means, the nesting with its reason, and the modulus.
 2. *Does the order of the loops matter?* Both nestings, why coins-outside fixes an order, the arrangements listed, the explosion heuristic, and which LeetCode problem wants which.
 3. *Design a web crawler.* The four-step loop, three levels of dedup and which saves bandwidth, why one FIFO fails and the per-host queue, the two-tier frontier, and the host-diversity number.
-

Before you move on

- [] I decide sets or sequences before writing any loops.
- [] I know `[1,2]` to 3 gives 2 combinations and 3 permutations.
- [] I know coins-outside counts combinations, and why.
- [] I know amount-outside counts permutations, and why.
- [] I can explain why coins-outside cannot produce `2+1`.

- [] I know `ways[0] = 1` and what it means in words.
- [] I know what a missing base case looks like from outside.
- [] I know swapping the loops does not change the cost.
- [] I know permutations explode, and can use that as a debugging signal.
- [] I know Combination Sum IV wants permutations despite its name.
- [] I know climbing stairs is permutations, and is Fibonacci.
- [] I can write Decode Ways with both validity conditions.
- [] I know where the modulus goes and why not at the end.
- [] I know why `10^9 + 7`.
- [] I know not to add a modulus the problem did not ask for.
- [] I know counting and listing are different algorithms with different costs.
- [] I can give the four steps of the crawl loop.
- [] I can name the three dedup levels and which saves bandwidth.
- [] I can list six URL normalisation steps.
- [] I can explain SimHash and what makes it unlike a normal hash.
- [] I can give the three seen-set sizes.
- [] I know what a Bloom filter false positive costs a crawler.
- [] I can explain why one FIFO frontier fails.
- [] I can describe the per-host queue and the ready heap.
- [] I know the `robots.txt` policy including the unreachable case.
- [] I can compute the host-diversity requirement and say what it bounds.
- [] I know why the frontier has two tiers.
- [] I can name four trap heuristics and say what they cost.
- [] I can give the `If-Modified-Since` arithmetic.
- [] I know what JavaScript rendering does to the machine count.
- [] I answered all three questions above out loud.

Longest increasing subsequence

DATA STRUCTURES AND ALGORITHMS

Longest increasing subsequence

SYSTEM DESIGN

Design a notification system at scale

1 What this is, and why they ask it

Given a list of numbers, find the length of the longest run you can pick out — **keeping their original order, but not requiring them to be next to each other** — that increases the whole way.

`[10, 9, 2, 5, 3, 7, 101, 18]` has a longest increasing subsequence of length 4: `2, 3, 7, 18`. Or `2, 3, 7, 101`. Both are length 4, and the length is what is asked for.

They ask it for two reasons and they are unusually different from each other.

The first is the state. The obvious state — “the longest increasing subsequence in the first `i` elements” — **does not work**, and discovering why is the cleanest lesson in DP state design there is. The state that works is “the longest increasing subsequence **ending exactly at `i`**”, and the difference between those two sentences is the entire topic. **Yesterday’s lessons taught adding a dimension; this one teaches redefining instead.**

The second is the `O(n log n)` solution, which is genuinely surprising, mildly beautiful, and asked as a follow-up almost every time. It maintains an array that **is not the answer and never was** — its contents are not a valid subsequence — and yet its length is exactly right. Being able to say what that array actually holds is the difference between having understood it and having memorised it.

By the end of this lesson you can write the `O(n2)` version and defend the state, derive the `O(n log n)` version and say precisely what its array means, reconstruct the actual subsequence, and recognise the family — box stacking, Russian dolls, and the ones that are LIS in disguise.

2 The story

The photographs were in four biscuit tins and Sumitra had said she would sort them, which she now regretted.

There was no date on any of them. A few had something written on the back in her mother’s hand, and most had nothing, and the ones that did say something said things like “at the wedding” without saying whose.

So she did what you do, which was to lay them out and work from what she could see.

The children were the clock. Her brother in shorts. Her brother taller. Her brother with the beginnings of a moustache and a very bad shirt. If she could put the pictures in an order where he only ever got older, she had something.

And the difficulty was immediately obvious, and it was not the one she expected.

It was not that she could not order them. **It was that she could not use all of them.**

Because there were photographs of cousins she could not place at all, and one of a house they had lived in for two years and she did not know which two, and a picture of her grandmother that could have been any year in a decade. **Putting those in the line meant guessing, and one wrong guess broke the whole thing.**

So she started leaving them out.

And the question changed shape completely. It was no longer “what order do these go in”. It was “**what is the largest number of them I can put in a line where each one is definitely later than the one before**” — and everything else goes back in the tin.

Her son, who was eleven and was helping in the way eleven-year-olds help, asked why she did not just start at the first photograph and keep going.

She had already tried that. **The first photograph was her brother at about fifteen, and if she started there, everything from before that was unusable.** Twenty-odd pictures thrown away because of where she happened to start.

The picture you start from decides how long the line can be, and you cannot know which one to start from until you have tried.

That was the sentence she said out loud, to nobody, at about eleven at night, and it is why she was still at it at one.

3 The idea in plain English

Sumitra’s sentence at eleven at night is the state definition, and her failed attempt at starting from the first photograph is the wrong state.

First, what “subsequence” means, because it is not “substring”. A subsequence keeps the original order and is allowed to skip. In `[10, 9, 2, 5, 3, 7]`, the values `2, 5, 7` form a subsequence — they appear in that order — even though `3` sits

between the 5 and the 7. A substring would have to be contiguous. A subsequence does not.

Now the state that does not work. The natural first attempt is:

`dp[i]` is the length of the longest increasing subsequence in the first `i` elements.

Try to write the recurrence and you get stuck immediately. You are at element `i` and you want to know whether you can extend the best subsequence found so far. **And you cannot, because `dp[i-1]` tells you the length and nothing else** — not what the last value was, and the last value is exactly what decides whether `nums[i]` can follow it.

That is an incomplete state, and it fails silently: you can write code from it that runs and returns numbers.

The state that works redefines rather than extends:

`dp[i]` is the length of the longest increasing subsequence that ends exactly at index `i`.

Now the last value is known — it is `nums[i]`. The state carries it, because ending-exactly-here is part of the definition. **That is the whole fix**, and it is worth noticing that the state space did not grow: still `n` cells. **Redefining was free; adding a dimension would not have been.**

The recurrence follows in one line. To build a subsequence ending at `i`, look at every earlier index `j`. If `nums[j] < nums[i]`, the subsequence ending at `j` can be extended by `nums[i]`.

```
dp[i] = 1 + max(dp[j]) over all j < i where nums[j] < nums[i]
        (and 1 if there is no such j – the element alone)
```

Every `dp[i]` starts at 1, because a single element is an increasing subsequence of length one.

And the answer is `max(dp)`, not `dp[n-1]`. That is Sumitra's point: the best line does not have to end at the last photograph. **This is the second-most-common bug in the problem** and it produces plausible numbers.

That is $O(n^2)$ — for each i , scan every j before it — and for n up to a few thousand it is what to write.

Now the $O(n \log n)$ version, which is the follow-up.

Keep an array called `tails`, where `tails[k]` is the smallest possible value that can end an increasing subsequence of length $k + 1$.

Read that twice. `tails` is not a subsequence. Its contents may not appear together in the input in that order at all. It is a summary: for each achievable length, the best (smallest) value you could be sitting on.

Why smallest is best: if you can end a length-3 subsequence on a 7 or on a 9, the 7 is strictly better, because anything that could extend the 9 could also extend the 7, and some things that could extend the 7 could not extend the 9. **Smaller endings are never worse.**

And `tails` is always sorted, which is what makes binary search legal. It has to be: a longer subsequence ends on a value at least as large as the shorter one it contains.

The algorithm is four lines. For each number, binary search `tails` for the first entry greater than or equal to it.

- **If there is none** — the number is larger than everything — **append it**. The longest achievable length just grew by one.
- **Otherwise, overwrite that entry**. You have found a smaller value that can end a subsequence of that length, which is an improvement for the future and changes nothing about the past.

The answer is `len(tails)`.

And this is the part to say out loud in an interview: `tails` is not the answer, its length is. If you need the actual subsequence you must record parent pointers separately, because reading `tails` at the end can give you a sequence that does not exist in the input.

Finally, the variants, which are the same algorithm with one symbol changed.

strictly increasing	<code>bisect_left</code>	$(\geq)$ replaced	the standard
non-decreasing	<code>bisect_right</code>	$(>)$ replaced	allows equal values
longest DEcreasing	reverse the input, or flip the comparison		
longest bitonic	LIS from the left + LIS from the right at each i		

And the disguises. *Russian Doll Envelopes* is LIS on heights after sorting by width — **with the widths sorted ascending and the heights descending within equal widths**, so that two envelopes of the same width can never both be picked. *Maximum Height by Stacking Cuboids*, *Number of Longest Increasing Subsequences*, and the minimum-number-of-decreasing-subsequences problem are all this.

4 The picture

The $O(n^2)$ table filling in, for `[10, 9, 2, 5, 3, 7, 101, 18]`:

```

index:  0    1    2    3    4    5    6    7
value: 10    9    2    5    3    7   101   18

dp[0] = 1    nothing before it
dp[1] = 1    9 > nothing earlier that is smaller
dp[2] = 1    2 is smaller than everything before
dp[3] = 2    5 extends [2]                → 2,5
dp[4] = 2    3 extends [2]                → 2,3
dp[5] = 3    7 extends [2,5] or [2,3]     → 2,3,7
dp[6] = 4    101 extends [2,3,7]         → 2,3,7,101
dp[7] = 4    18 extends [2,3,7]          → 2,3,7,18

dp = [1,   1,   1,   2,   2,   3,   4,   4]
      ^       ^       ^       ^       ^
answer = max(dp) = 4, NOT dp[-1].
Here they happen to agree. On [1,2,3,0] they do not:
    dp = [1, 2, 3, 1] → max is 3, dp[-1] is 1.
```

Why the natural state fails:

WRONG: `dp[i]` = longest LIS in the first `i` elements

```

nums = [2, 5, 3, 7]
dp[3] = 2    (that is [2,5])
now element 3 arrives. Can 3 extend it?
```

```

→ dp[3] says "the best so far is length 2"
→ it does NOT say what that subsequence ENDS on
→ 3 can follow a 2 but not a 5, and I cannot tell which
```

The state is INCOMPLETE. You can still write code from it.
It just gives wrong answers, with no error.

```

RIGHT: dp[i] = longest LIS ENDING EXACTLY at i
→ the last value is nums[i], carried by the definition itself.
→ same n cells. Redefining cost nothing.
```

The `tails` array evolving, on the same input:

```
nums = [10, 9, 2, 5, 3, 7, 101, 18]

10 → tails empty, append      tails = [10]
9  → first ≥ 9 is 10, at index 0 tails = [9]
2  → first ≥ 2 is 9, at index 0 tails = [2]
5  → nothing ≥ 5, append      tails = [2, 5]
3  → first ≥ 3 is 5, at index 1 tails = [2, 3]
7  → nothing ≥ 7, append      tails = [2, 3, 7]
101 → nothing ≥ 101, append    tails = [2, 3, 7, 101]
18  → first ≥ 18 is 101, index 3 tails = [2, 3, 7, 18]

answer = len(tails) = 4          CORRECT

and note: [2, 3, 7, 18] IS a real subsequence here, by luck.
```

The case that proves `tails` is not the answer:

```
nums = [1, 6, 7, 2, 3]

1 → append      tails = [1]
6 → append      tails = [1, 6]
7 → append      tails = [1, 6, 7]
2 → replaces 6   tails = [1, 2, 7]
3 → replaces 7   tails = [1, 2, 3]

len(tails) = 3          CORRECT (1,6,7 is length 3)

but tails is [1, 2, 3], and 1,2,3 IS in the input...
try nums = [1, 6, 7, 2]:
  tails ends as [1, 2, 7]
  → 1, 2, 7 is NOT a subsequence: the 2 comes AFTER the 7.
  → the LENGTH 3 is right. The CONTENTS are meaningless.
```

What `tails[k]` means, drawn:

```
tails[0] = smallest value that can end an LIS of length 1
tails[1] = smallest value that can end an LIS of length 2
tails[2] = smallest value that can end an LIS of length 3

"smallest" because a smaller ending is never worse:
  ending on 7 vs ending on 9, both length 3
  → anything extending 9 also extends 7
  → some things extend 7 and not 9
  → keep 7, discard 9. Strictly better.

and tails is ALWAYS SORTED, which is why binary search is legal:
  a length-3 subsequence contains a length-2 one, so its ending
  is at least as large.
```

5 The code, built step by step

The $O(n^2)$ version, from the sentence

```
def length_of_lis_quadratic(nums: list[int]) → int:
    if not nums:
        return 0
    dp = [1] * len(nums)          # every element alone is length 1
    return max(dp)                # (loops next)
```

`[1] * n` is the base case for all of them at once — every element on its own is an increasing subsequence of length one.

```
    for i in range(1, len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:          # can nums[i] follow nums[j]?
                dp[i] = max(dp[i], dp[j] + 1)
```

`nums[j] < nums[i]` is strict, which gives strictly increasing. Use `≤` and you get non-decreasing, which is a different problem — read the statement.

And `max(dp)`, not `dp[-1]`. The best subsequence can end anywhere.

The $O(n \log n)$ version

```
import bisect

def length_of_lis(nums: list[int]) → int:
    tails: list[int] = []          # tails[k] = smallest end of an LIS of length k+1
    for number in nums:
        position = bisect.bisect_left(tails, number)
        if position == len(tails):
            tails.append(number)    # extends the longest run so far
        else:
            tails[position] = number # a smaller ending for that length
    return len(tails)
```

Six lines, and the comment on line two is the whole explanation.

`bisect_left` finds the first entry `≥ number`. If nothing is `≥`, the number is bigger than everything and the answer grows. Otherwise it replaces that entry with something smaller, which can only help later.

`bisect_left` for strictly increasing; `bisect_right` for non-decreasing. One character, two problems — and it is worth saying which you wrote and why.

Reconstructing the actual subsequence

You cannot read it off `tails`. Record parent pointers instead:

```
def lis_sequence(nums: list[int]) → list[int]:
    if not nums:
        return []
    tails: list[int] = []
    tail_index: list[int] = []           # tail_index[k] = index in nums of tails[k]
    parent = [-1] * len(nums)           # parent[i] = previous index in the LIS ending at i

    for i, number in enumerate(nums):
        position = bisect.bisect_left(tails, number)
        if position > 0:
            parent[i] = tail_index[position - 1]
        if position == len(tails):
            tails.append(number)
            tail_index.append(i)
        else:
            tails[position] = number
            tail_index[position] = i

    result, i = [], tail_index[-1]
    while i ≠ -1:
        result.append(nums[i])
        i = parent[i]
    return result[::-1]
```

`parent[i] = tail_index[position - 1]` says: the element before `nums[i]` in its subsequence is whatever element currently ends the best run one shorter. **Then walk back from `tail_index[-1]`**, which is the index of the element ending the longest run.

Two extra arrays and $O(n)$ more space, and it produces a genuinely valid subsequence — unlike reading `tails`.

Russian doll envelopes, the disguise

```
def max_envelopes(envelopes: list[tuple[int, int]]) → int:
    envelopes.sort(key=lambda e: (e[0], -e[1]))    # width up, height DOWN
    return length_of_lis([height for _, height in envelopes])
```

The `-e[1]` is the whole trick and it is easy to miss. Sorting heights *descending* within equal widths means two envelopes of the same width appear as a decreasing pair, so LIS can never pick both — which is exactly the rule, since equal widths do not nest.

Sort heights ascending instead and it silently overcounts, giving a larger, wrong answer.

Counting the longest ones

```
def find_number_of_lis(nums: list[int]) → int:
    n = len(nums)
    length = [1] * n
    count = [1] * n
    for i in range(n):
        for j in range(i):
            if nums[j] < nums[i]:
                if length[j] + 1 > length[i]:
                    length[i] = length[j] + 1
                    count[i] = count[j]          # a new best: inherit its count
                elif length[j] + 1 == length[i]:
                    count[i] += count[j]         # another way to reach the same best
    best = max(length)
    return sum(c for l, c in zip(length, count) if l == best)
```

Two arrays, and the `>` versus `=` branches are the whole problem. A strictly better length **replaces** the count; an equally good length **adds** to it. **Getting that backwards is the standard bug**, and it gives plausible numbers.

The complete solution

```
"""Longest increasing subsequence: both algorithms, reconstruction, and the family."""
import bisect

def length_of_lis_quadratic(nums: list[int]) → int:
    """ $O(n^2)$ . dp[i] = longest LIS ENDING EXACTLY at i."""
    if not nums:
        return 0
    dp = [1] * len(nums)  # each element alone
    for i in range(1, len(nums)):
        for j in range(i):
            if nums[j] < nums[i]:  # nums[i] can follow nums[j]
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)  # NOT dp[-1]

def length_of_lis(nums: list[int]) → int:
    """ $O(n \log n)$ . tails[k] = smallest value ending an LIS of length k+1."""
    tails: list[int] = []
    for number in nums:
        position = bisect.bisect_left(tails, number)  # first entry ≥ number
        if position == len(tails):
            tails.append(number)  # longer than anything so far
        else:
            tails[position] = number  # a smaller ending for that length
    return len(tails)  # the LENGTH is the answer, not tails

def length_of_non_decreasing(nums: list[int]) → int:
    """One character different: bisect_right allows equal values."""
    tails: list[int] = []
    for number in nums:
        position = bisect.bisect_right(tails, number)
        if position == len(tails):
            tails.append(number)
        else:
            tails[position] = number
    return len(tails)

def lis_sequence(nums: list[int]) → list[int]:
    """The actual subsequence. tails cannot give it; parent pointers can."""
    if not nums:
        return []
    tails: list[int] = []
    tail_index: list[int] = []
    parent = [-1] * len(nums)

    for i, number in enumerate(nums):
        position = bisect.bisect_left(tails, number)
        if position > 0:
            parent[i] = tail_index[position - 1]
        if position == len(tails):
            tails.append(number)
            tail_index.append(i)
        else:
            tails[position] = number
            tail_index[position] = i

    result: list[int] = []
```

```

i = tail_index[-1]
while i != -1:
    result.append(nums[i])
    i = parent[i]
return result[::-1]

def max_envelopes(envelopes: list[tuple[int, int]]) → int:
    """LIS in disguise. Width ascending, height DESCENDING within equal widths."""
    envelopes.sort(key=lambda e: (e[0], -e[1]))
    return length_of_lis([height for _, height in envelopes])

def find_number_of_lis(nums: list[int]) → int:
    """How many longest increasing subsequences there are."""
    n = len(nums)
    if n == 0:
        return 0
    length = [1] * n
    count = [1] * n
    for i in range(n):
        for j in range(i):
            if nums[j] < nums[i]:
                if length[j] + 1 > length[i]:
                    length[i] = length[j] + 1
                    count[i] = count[j] # strictly better: replace
                elif length[j] + 1 == length[i]:
                    count[i] += count[j] # equally good: add
    best = max(length)
    return sum(c for l, c in zip(length, count) if l == best)

def longest_bitonic(nums: list[int]) → int:
    """Up then down. LIS from the left plus LIS from the right at each index."""
    n = len(nums)
    if n == 0:
        return 0
    up = [1] * n
    down = [1] * n
    for i in range(n):
        for j in range(i):
            if nums[j] < nums[i]:
                up[i] = max(up[i], up[j] + 1)
    for i in range(n - 1, -1, -1):
        for j in range(n - 1, i, -1):
            if nums[j] < nums[i]:
                down[i] = max(down[i], down[j] + 1)
    return max(u + d - 1 for u, d in zip(up, down)) # -1: the peak is in both

if __name__ == "__main__":
    nums = [10, 9, 2, 5, 3, 7, 101, 18]
    print("quadratic      :", length_of_lis_quadratic(nums))
    print("n log n       :", length_of_lis(nums))
    print("the subsequence:", lis_sequence(nums))

    tricky = [1, 6, 7, 2]
    print("tricky length  :", length_of_lis(tricky))
    print("tricky sequence:", lis_sequence(tricky))

    print("dp[-1] trap   :", length_of_lis([1, 2, 3, 0]))
    print("all decreasing :", length_of_lis([5, 4, 3, 2, 1]))
    print("empty          :", length_of_lis([]))

```



```

print("non-decreasing :", length_of_non_decreasing([1, 3, 3, 5]))
print("strict on same :", length_of_lis([1, 3, 3, 5]))

print("envelopes      :", max_envelopes([(5, 4), (6, 4), (6, 7), (2, 3)]))
print("count of LIS   :", find_number_of_lis([1, 3, 5, 4, 7]))
print("bitonic        :", longest_bitonic([1, 11, 2, 10, 4, 5, 2, 1]))

```

Run it and you get:

```

quadratic      : 4
n log n        : 4
the subsequence: [2, 3, 7, 18]
tricky length  : 3
tricky sequence: [1, 6, 7]
dp[-1] trap    : 3
all decreasing : 1
empty          : 0
non-decreasing : 4
strict on same : 3
envelopes      : 3
count of LIS   : 2
bitonic        : 6

```

tricky is the line that proves the point. For `[1, 6, 7, 2]` the `tails` array finishes as `[1, 2, 7]` — which is **not** a subsequence, because the 2 comes after the 7 in the input. **The length 3 is correct; the contents are not an answer.** And `lis_sequence` returns `[1, 6, 7]`, which is real.

non-decreasing 4 against **strict on same 3** is the one-character difference, made visible.

6 What it costs

The $O(n^2)$ version.

```

outer loop over i:      n iterations
inner loop over j < i:  i iterations

total comparisons = 0 + 1 + 2 + ... + (n-1)
                  = n(n-1)/2
                  ~ n^2 / 2

space: one array of n integers = O(n)

```

Concretely:

$n = 1,000$	500,000 comparisons	~0.1 s in Python.	Fine.
$n = 10,000$	50,000,000 comparisons	~10 s.	Too slow.
$n = 100,000$	5,000,000,000	~17 minutes.	Impossible.

$n = 2,500$ is LeetCode 300's constraint, which is deliberately set so the quadratic version passes — and the follow-up asks for better anyway.

The $O(n \log n)$ version.

```

for each of n numbers:
    one binary search over tails       $\log_2(\text{len}(\text{tails})) \leq \log_2(n)$ 
    one append or one assignment       $O(1)$ 

total:  $n \log n$ 

space: tails is at most n             $O(n)$ 

```

Concretely:

$n = 100,000$	$100,000 \times 17 = 1,700,000$ operations	~0.3 s.	Fine.
$n = 1,000,000$	$1,000,000 \times 20 = 20,000,000$	~4 s.	Fine.

against 17 minutes and ~14 days for the quadratic version.

The gap at $n = 100,000$ is roughly $3,000\times$, which is why the follow-up is asked.

Reconstruction's extra cost:

```

two extra arrays of n           $O(n)$  space
one extra assignment per element  $O(1)$  time
the walk-back                   $O(\text{length of the answer}) \leq O(n)$ 

→ same complexity, about 2x the memory, and it gives a real answer
  instead of tails' meaningless contents.

```

Counting the longest ones:

```

two arrays instead of one, same  $O(n^2)$  loops
→  $O(n^2)$  time,  $O(n)$  space

there is an  $O(n \log n)$  version using a segment tree, and it is
rarely worth writing in an interview. Say it exists.

```

Russian doll envelopes:

```
sort                O(n log n)
LIS on heights      O(n log n)
→ O(n log n) total, dominated by neither

n = 100,000 envelopes: sub-second.
```

And the bitonic version:

```
LIS from the left    O(n^2) as written, O(n log n) with tails
LIS from the right   same
combine              O(n)

The -1 in `u + d - 1` matters: the peak element is counted in both
arrays, so adding them double-counts it exactly once.
```

7 The traps

Returning `dp[-1]` instead of `max(dp)`.

```
>>> nums = [1, 2, 3, 0]
>>> dp = [1] * 4
>>> for i in range(1, 4):
...     for j in range(i):
...         if nums[j] < nums[i]:
...             dp[i] = max(dp[i], dp[j] + 1)
>>> dp
[1, 2, 3, 1]
>>> dp[-1]
1
>>> max(dp)
3
```

1 against **3**. The state is “ending exactly at `i`”, so the last cell only describes subsequences that end at the last element — and the best one often does not. **No error, and the number looks like an answer.**

Using the incomplete state and not noticing. If `dp[i]` means “the best in the first `i` elements”, you cannot write a correct recurrence at all — but you *can* write one that runs:

```
>>> nums = [2, 5, 3, 7]
>>> dp = [1] * 4
>>> for i in range(1, 4):
...     dp[i] = dp[i-1] + 1 if nums[i] > nums[i-1] else dp[i-1]
>>> dp[-1]
3
```

It gives 3, and the answer is 3. It is also wrong in general — it only compares each element with the one immediately before it, so it happily chains 3, 4 and then 1, 2, 5 into a single count:

```
>>> nums = [3, 4, 1, 2, 5]
>>> dp = [1] * 5
>>> for i in range(1, 5):
...     dp[i] = dp[i-1] + 1 if nums[i] > nums[i-1] else dp[i-1]
>>> dp[-1]
4
```

Four, when the answer is three — 1, 2, 5 and 3, 4, 5 are both length three and there is nothing of length four. **A state that produces the right answer on your first test is the most dangerous kind.**

`bisect_right` when you wanted strictly increasing.

```
>>> length_of_lis([1, 3, 3, 5])
3
>>> length_of_non_decreasing([1, 3, 3, 5])
4
```

Both are correct programs. 1,3,5 is the strictly increasing answer; 1,3,3,5 is the non-decreasing one. **Read the problem statement for the word “strictly”,** and say which you wrote.

Reading the answer off `tails` .

```
>>> nums = [1, 6, 7, 2]
>>> tails = []
>>> for x in nums:
...     p = bisect.bisect_left(tails, x)
...     tails.append(x) if p == len(tails) else tails.__setitem__(p, x)
>>> tails
[1, 2, 7]
```

[1, 2, 7] is not a subsequence of [1, 6, 7, 2] — the 2 appears after the 7. **The length is right and the contents are not an answer**, and returning them is a real bug that passes any test that only checks the length.

Forgetting the empty input. Without the `if not nums` guard, the quadratic version ends on `max(dp)` with an empty `dp`:

```
>>> max([])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: max() iterable argument is empty
```

The `tails` version returns `0` naturally, because `len([])` is zero — one small argument for it, and a reminder that the guard is not optional in the quadratic one.

Sorting envelopes by height ascending within equal widths.

```
>>> max_envelopes_wrong = lambda e: length_of_lis(
...     [h for _, h in sorted(e, key=lambda x: (x[0], x[1]))])
>>> max_envelopes_wrong([(1, 1), (1, 2), (1, 3)])
3
```

Three, when the answer is one — all three envelopes have width 1, so none nests inside another. **Sorting heights descending within equal widths makes them a decreasing run that LIS cannot pick twice from.** One minus sign.

Comparing floats.

```
>>> length_of_lis([0.3, 0.1 + 0.2])
2
```

Two, because `0.1 + 0.2` is `0.30000000000000004`, which is strictly greater than `0.3` — so two values that were meant to be equal look like an increase. **It is arguably correct and it is almost certainly not what was meant.** If the input is floats, ask about tolerance.

Very large `n` with the quadratic version.

```
n = 100,000 → 5,000,000,000 comparisons

No error. It just does not finish, and on LeetCode:
Time Limit Exceeded
```

The constraint tells you which version to write, so read it before choosing.

8 In the interview

How it gets asked

- “Find the length of the longest strictly increasing subsequence.” — LeetCode 300, the standard.
- “Can you do better than $O(n^2)$?” — always the follow-up.
- “What does that array you are keeping actually contain?” — the question that separates people.
- “Return the subsequence, not just its length.”
- “You have envelopes with widths and heights; how many can you nest?” — LeetCode 354.
- “Why doesn’t `dp[i] = best in the first i elements` work?”

The first ninety seconds

“Let me get the state right first, because the obvious state does not work and that is the interesting part of this problem.

The natural attempt is `dp[i]` = the longest increasing subsequence in the first `i` elements. And I get stuck immediately: element `i` arrives and I want to know whether it can extend what I have, but `dp[i-1]` tells me a length and not what that subsequence ended on — and the ending value is exactly what decides whether the new element can follow. **The state is incomplete, and it fails silently**, because I can still write code from it that runs.

The fix is to redefine rather than to add a dimension. `dp[i]` is the longest increasing subsequence that ends exactly at index `i`. Now the last value is `nums[i]` — carried by the definition itself. **And the state space did not grow: still `n` cells.** That is worth noticing, because adding a dimension would have cost me a factor of `n`.

The recurrence is one line. For each `i`, look at every `j` before it; if `nums[j] < nums[i]`, then `dp[i]` can be `dp[j] + 1`. Take the best. Every cell starts at 1, because a single element is a valid subsequence.

And the answer is `max(dp)`, not `dp[n-1]` — the best subsequence does not have to end at the last element. On `[1, 2, 3, 0]`, `dp` is `[1, 2, 3, 1]`: **the last cell says 1 and the answer is 3.** That is the second most common bug here and it produces a plausible number.

That is $O(n^2)$ time and $O(n)$ space, and for `n` up to a few thousand it is what I would write.

There is an $O(n \log n)$ version and I would offer it, because it is what the follow-up asks for. It keeps an array where **the entry at index `k` is the smallest value that can end an increasing subsequence of length `k + 1`** — and for each number, binary search for the first entry at least as large, then either append or overwrite. **The answer is the length of that array.**

The thing I would flag before they ask: that array is not the answer. Its contents may not even be a subsequence of the input. Only its length is meaningful, and if they want the actual subsequence I need parent pointers.

One question first: strictly increasing, or is equal allowed? It is one character in the binary search and two different answers.”

The follow-ups

“Can you do better than $O(n^2)$? Explain what your array holds.”

“Yes — $O(n \log n)$ — and the array is the part worth explaining carefully, because it is easy to write and hard to justify.

`tails[k]` is the smallest value that can end an increasing subsequence of length $k + 1$.

Why smallest is the right thing to keep: suppose I can end a length-three subsequence on a 7 or on a 9. **The 7 is strictly better.** Anything that can extend the 9 can also extend the 7, and some things can extend the 7 but not the 9. So keeping the smaller ending never loses me an option, and sometimes gains one. **There is no reason to remember the 9 at all.**

And `tails` is always sorted, which is what makes binary search legal, and it has to be — a length-three subsequence contains a length-two one, so its ending value is at least as large.

The loop is four lines. For each number, binary search for the first entry greater than or equal to it. **If there is none, the number beats everything and I append** — the longest achievable length just grew. **Otherwise I overwrite that entry,** because I have found a smaller value that ends a subsequence of that length. That changes nothing about the past and improves my options for the future.

The answer is `len(tails)`.

Now the thing I would say before being asked: `tails` is not a subsequence. On `[1, 6, 7, 2]` it finishes as `[1, 2, 7]`, and that is not a subsequence of the input — the 2 comes after the 7. **The length, 3, is correct. The contents are meaningless.** Returning them is a bug that passes every test that only checks the length.

Cost: n binary searches over an array of at most n , so $O(n \log n)$ time and $O(n)$ space. At a hundred thousand elements that is under a second, against about seventeen minutes for the quadratic version — roughly three thousand times faster, which is why the follow-up exists.

And the variant: `bisect_left` gives strictly increasing, `bisect_right` gives non-decreasing. On `[1, 3, 3, 5]` that is 3 against 4. **One character, two different correct answers,** so I would confirm which the problem wants.”

“Return the actual subsequence, not the length.”

“Then I cannot use `tails`, and that constraint is the interesting part of the answer.

`tails` holds values, not positions, and it is overwritten as it goes, so at the end it has no memory of which elements were actually chosen. On `[1, 6, 7, 2]` it says `[1, 2, 7]`, which is not a valid answer.

What I do instead is record parent pointers during the same pass. Two extra arrays: one mapping each position in `tails` to the index in `nums` of the element currently sitting there, and one giving, for every element, the index of the element before it in the best subsequence ending there.

The rule is one line. When element `i` lands at position `p` in `tails`, its predecessor is whatever element currently ends the best run of length `p` — that is, the element recorded at `tails` position `p - 1`. If `p` is zero, there is no predecessor.

Then walk back from the element recorded at the last position of `tails`, following parents, and reverse what you collect.

Cost: $O(n)$ extra space and one extra assignment per element — the complexity does not change. And the result is a genuine subsequence, in the right order, of the right length.

With the $O(n^2)$ version reconstruction is easier — record which `j` gave each `dp[i]` its value, find the index with the maximum `dp`, and walk back. **I would mention that**, because if they ask for the subsequence and `n` is small, the quadratic version with reconstruction is less code to get wrong under time pressure.

And I would ask whether they want a longest subsequence or *all* of them, because all of them can be exponentially many — `[1,1,1, ...]` with distinct-but-equal-length runs — and that is a different problem with a different answer.”

“You have envelopes with widths and heights. One fits inside another if both dimensions are strictly smaller. How many can you nest?”

“This is LIS wearing a hat, and there is exactly one subtle step.

The problem is two-dimensional and LIS is one-dimensional, so I sort on one dimension to remove it. Sort the envelopes by width ascending. Now, reading left to right, widths are non-decreasing, so **the only remaining question is the heights** — and the answer is the longest strictly increasing subsequence of the heights.

The subtlety is what to do with equal widths. Two envelopes of the same width can never nest, whatever their heights. But after sorting by width they sit next to each other, and if their heights happen to increase, LIS will happily pick both — **and that is wrong.**

The fix is to sort heights descending within equal widths. Then a group of same-width envelopes appears as a strictly decreasing run of heights, and **an increasing subsequence can pick at most one from it, by construction.** One minus sign in the sort key, and the whole class of errors disappears.

Concretely: three envelopes all of width 1, heights 1, 2 and 3. Sorted heights ascending gives `[1, 2, 3]` and LIS says three — wrong, since none nests. Sorted descending gives `[3, 2, 1]` and LIS says one, which is right.

Then run the $O(n \log n)$ LIS on the height list. Total cost is $O(n \log n)$ for the sort plus $O(n \log n)$ for the LIS — a hundred thousand envelopes in well under a second.

The general shape is worth naming, because it recurs: **sort to remove one dimension, then run a one-dimensional algorithm on what is left.** The same trick solves box stacking and the cuboid problem. **And the tie-breaking rule in the sort is always where the bug lives.**“

The model answer

“You are given a list of software versions deployed to a server over time, each with a compatibility number. A rollback plan must consist of versions in the order they were deployed, each strictly more compatible than the last. What is the longest such plan, and which versions are in it?”

“This is longest increasing subsequence, and the prompt asks for the sequence as well as the length, which changes which algorithm I write.

Let me confirm the reading first: the versions must stay in deployment order — I cannot reorder them — and I may skip any I like. That is exactly a subsequence, not a substring. And ‘strictly more compatible’ means strictly increasing, so two versions with the same compatibility number cannot both be in the plan. I would state that, because it is the one-character difference in the implementation.

The state, and I want to be careful here because the natural one fails. ‘The longest plan using the first i versions’ does not work: when version i arrives I need to know what compatibility number the current best plan ends on, and a length does not tell me that. **So: $dp[i]$ is the longest valid plan that ends exactly at version i .** The ending value is then $compat[i]$, carried by the definition.

Recurrence: for each i , look at every earlier j with $compat[j] < compat[i]$, and take the best $dp[j] + 1$. Each starts at 1. **The answer is $\max(dp)$, not the last cell** — the best plan need not end at the most recent deployment, and given that deployments often regress in compatibility, it usually will not.

That is $O(n^2)$, and here I would ask how many deployments there are. A server’s deployment history is realistically hundreds or low thousands, where quadratic is instant and the reconstruction is trivially easy — record which j won for each i , then walk back from the argmax. **If it were a hundred thousand, I would use the $O(n \log n)$ version,** which keeps an array whose entry k is the smallest compatibility number that can end a plan of length $k + 1$, binary searching for each element.

And I would raise the trap in that version, because the prompt asks for the versions themselves. That array’s contents are not a valid plan — on $[1, 6, 7, 2]$ it ends as $[1, 2, 7]$, which is not in that order in the input. **Its length is right and its contents are not an answer,** so reconstruction needs parent pointers recorded during the pass, not a read of the array at the end.

Given the realistic size, I would write the quadratic version with reconstruction — it is less code, the walk-back is obvious, and at a few thousand deployments it runs in milliseconds. **I would say out loud that I know the $O(n \log n)$ version and why I am not using it,** which is a better signal than using it unnecessarily.

Two things about the problem domain I would flag. Ties matter and the specification is ambiguous — if two versions have the same compatibility number, ‘strictly more’ excludes both from following each other, and if the real rule is ‘at least as compatible’, that is `bisect_right` and a different answer. I would ask rather than assume.

And there may be more than one longest plan, all of the same length. If the operator needs to choose among them — preferring more recent versions, say — that is a second criterion, and I would handle it as a tie-break in the `max`, not as a separate pass.”

9 Recall card

The natural state fails: “longest LIS in the first `i` elements” is incomplete — it gives a length and not the ending value, which is what decides whether the next element can follow. Redefine, don’t extend: `dp[i]` = the longest increasing subsequence ending *exactly* at `i`. Same `n` cells; redefining is free.

`dp[i] = 1 + max(dp[j])` over `j < i` with `nums[j] < nums[i]`, each starting at 1. The answer is `max(dp)`, never `dp[-1]` — on `[1,2,3,0]`, `dp` is `[1,2,3,1]`.

$O(n \log n)$: `tails[k]` is the smallest value that can end an LIS of length `k+1`. Smaller endings are never worse (anything extending 9 also extends 7); `tails` is therefore sorted, so binary search is legal. Per number: `bisect_left` → append if past the end, else overwrite. The answer is `len(tails)`.

`tails` is NOT the subsequence — on `[1,6,7,2]` it ends as `[1,2,7]`, which is not in that order in the input. Length right, contents meaningless. For the actual sequence, record parent pointers during the pass.

`bisect_left` = strictly increasing; `bisect_right` = non-decreasing — `[1,3,3,5]` gives 3 and 4. Russian doll envelopes = sort width ascending, height DESCENDING within equal widths, then LIS on heights: the minus sign is what stops two same-width envelopes both being picked. n^2 at $n=100,000$ is ~17 minutes; $n \log n$ is under a second.

Design a notification system at scale

1 What this is, and why they ask it

A notification system takes an event — someone liked your photo, your order shipped, your card was declined — and gets a message to a person, through push, SMS, email or in-app, on whatever device they are using.

It looks like the easiest design in the set and it is not. The core loop is “receive event, look up user, call a provider”, and that fits in a page. Everything that makes it hard is around the edges, and every one of those edges is asked about.

They ask it because it is the design where **third parties are on the critical path and you do not control them.** Apple’s push service, Google’s, Twilio, SendGrid — all of them have their own rate limits, their own outages, their own error codes, and their own opinions about what counts as abuse. **A design that assumes the provider always works is not a design.**

And they ask it because of one property no other system in this course has: **a notification is visible to a human, and sending it twice is a real, felt failure.** A duplicated database write is a bug; a duplicated “your card was charged” push at three in the morning is a support ticket and a lost user. **The exactly-once conversation from [day 122](#) has real stakes here.**

The other half is that **the volume is enormous and mostly unwanted.** A social app sends billions of notifications a day and most of them are ignored. **The interesting engineering is in not sending things** — preferences, rate limits per user, batching, quiet hours — and a candidate who talks about deduplication and throttling before talking about throughput is answering the real question.

By the end of this lesson you can design the pipeline, handle provider failures and retries, deduplicate properly, respect preferences and quiet hours, fan out to a million recipients, and size the whole thing.

2 The story

The school had four hundred children and one way of telling anybody anything, which was Ganeshan.

He was the office assistant, and when something needed saying he wrote it in the diary of every child it concerned and the parents signed it the next morning.

And it worked, badly, for a very long time.

The trouble was never the writing. It was everything else.

Some parents wanted to know everything — the fee reminder, the sports day, the change of bus timing, the list of what to bring on Thursday. **Some wanted only the things that mattered**, and their definition of what mattered was not the school's, and the only way to find out was to be told off.

Then there was Mrs Fernandes, who told him in September that she did not want to hear about the bus, ever, and in January complained that nobody had told her the bus was cancelled. **He had the note she had written. It did not help.**

There was the family with three children in the school, who got the same fee notice three times, once in each diary, and the father came in and asked whether the school thought he could not read.

There was the evening the principal decided at nine at night that the next day was a holiday, **and Ganeshan made forty phone calls before somebody's grandmother told him what she thought of people who telephone at ten o'clock.**

And there was the sports day, when he wrote the notice in every diary in the school, four hundred of them, and it took him from lunch until eight in the evening, **and by the time he finished the ones he had written at lunch had already been read and the ones at the end arrived a day late.**

What eventually fixed it was not a better diary.

It was a list on the office wall that Ganeshan made himself, with four columns. **Who. What kind of thing. How they want to hear it. When it is too late to tell them.** And a rule underneath, in his own writing, that he followed for the next eleven years:

“If it can wait until morning, it waits until morning. If it cannot, say so at the top.”

3 The idea in plain English

Ganeshan's four-column list is the notification system, and his rule at the bottom is the priority tier.

The pipeline is five stages, and each one exists because of a specific failure:

1. INGEST	an event arrives from some service
2. RESOLVE	who should be told, and how do they want to hear it
3. BUILD	render the message from a template
4. SEND	hand it to a provider
5. TRACK	delivered? opened? failed? bounced?

Stage one: ingest, and the first decision is asynchronous.

A service that emits an event must not wait for the notification to be sent. Checkout should not be slower because SendGrid is slow, and checkout should certainly not fail because SendGrid is down. **So the event goes onto a queue and the API returns immediately** — [day 129](#)'s message queue, doing the job it exists for.

And the queue is what makes retries possible at all, because a failed send needs somewhere to live between attempts.

Stage two: resolve, and this is where most of the logic is.

For each event, work out **who** to notify, and then for each person, **which channels**, and then **whether to send at all**. That last one is the important one, and it has four separate gates:

- **Preferences.** Mrs Fernandes said no to bus notices. Per user, per notification type, per channel.
- **Quiet hours.** Not between 10pm and 8am **in the user's timezone**, which is the part people forget.
- **Rate limits.** No more than **N** notifications an hour to one person, whatever the events say. **This is the single most valuable feature in the system** and it is usually built last, after somebody's bug sends a user four hundred pushes.
- **Deduplication.** The same event processed twice must produce one notification.

Stage three: build. Render a template with the user's data, in the user's language. **Templates live in a store and are versioned**, because marketing will change the wording and you need to know what was actually sent.

Stage four: send, and this is where the third parties live.

push (iOS)	APNs	HTTP/2, certificate or token auth
push (Android)	FCM	also handles web push
SMS	Twilio, MSG91, AWS SNS	
email	SendGrid, SES, Mailgun	
in-app	your own websocket or a poll of your own store	

Each has different limits, different error codes, and different failure modes, so each gets its own worker pool and its own queue. **One shared pool is wrong:** an SMS provider timing out would consume every worker and stop email too. **That is [day 126's bulkhead](#)**, and it is the specific reason for the per-channel split.

Stage five: track. Providers report delivery asynchronously, by webhook. **A push can be accepted by APNs and never arrive** — the phone is off, the app was uninstalled, the token is stale — and you only find out later. **Feed that back:** a token that APNs reports as invalid must be deleted, or you send to it forever.

Now the four things that are genuinely hard.

One: duplicates, and why they are worse here. At-least-once delivery from the queue means an event can be processed twice. Everywhere else that produces a duplicate row; **here it produces a second push notification on someone's lock screen.**

The fix is an idempotency key, and the key must come from the event, not be generated during processing: `hash(user_id, notification_type, event_id)`. Store it in Redis with a TTL, check before sending. **The check and the send are not atomic**, so a crash between them can still duplicate — **which is why the key should also be passed to the provider where the provider supports it**, since APNs deduplicates on `apns-collapse-id`.

Two: retries, and the trap in them. A provider returns `503`. Retry with exponential backoff and jitter — [day 125's](#) reasoning exactly.

But not all failures should be retried, and this is the distinction that matters. A `429` means slow down and retry. A `500` means retry. A `400` — **bad token, invalid number, unsubscribed address** — **must never be retried**, because it will fail identically forever and each attempt costs money. **Retrying a permanent failure is the most common way to burn a provider quota.**

And a notification has a shelf life. “Your ride is arriving” is worthless twenty minutes later. **Every notification carries a TTL, and a message that expires in the queue is dropped rather than sent** — Ganeshan's rule about the morning, made mechanical.

Three: fan-out, which is Ganeshan's four hundred diaries. A celebrity posts and ten million followers should be told. **Doing that in one worker takes hours and the last person is told tomorrow.**

So fan out in stages. One event becomes a fan-out job; the job reads followers in batches of a thousand and writes a message per batch onto the send queue; hundreds of workers drain that. **The key is that the fan-out job and the send job are separate**

queue messages, so the work is parallel and a crash loses one batch rather than everything.

And for very large fan-outs, do not send at all. A hundred million followers means most of them do not want it. **Filter by engagement first** — people who have opened a notification from this source in the last thirty days — which typically cuts the volume by ninety percent and improves the open rate at the same time.

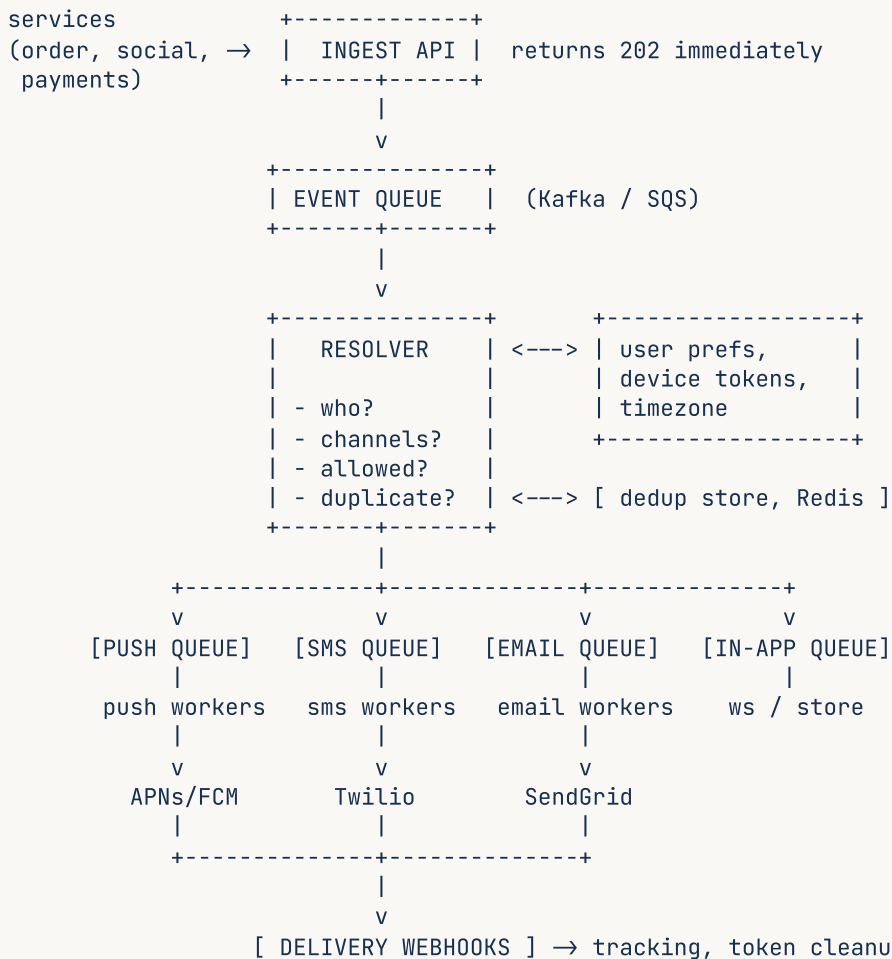
Four: priority, because not everything is equal. A one-time password must go now. A “someone liked your photo” can wait five minutes and be batched with the next four likes.

Separate queues per priority, not a priority field on one queue. A single queue with priorities still lets a million low-priority messages sit in front of the OTP if the consumer is not careful, and separate queues with separate worker pools make the guarantee structural.

And batching low-priority notifications is the highest-value optimisation in the system. Five separate “X liked your photo” pushes is worse than one “5 people liked your photo” — fewer notifications, better open rate, less provider cost, happier user. **Three things improved at once, which is rare.**

4 The picture

The pipeline:



ONE QUEUE AND POOL PER CHANNEL. A slow SMS provider must not be able to consume the workers that send email.

The four gates in the resolver, in order:

```
event: "photo_liked", user 4471
```

```
gate 1  PREFERENCES  does this user want photo_liked on push?
                        → no  ⇒ STOP (cheapest possible exit)
gate 2  DEDUP          have I already sent this exact notification?
                        → yes ⇒ STOP
gate 3  RATE LIMIT    has this user had > 10 pushes this hour?
                        → yes ⇒ batch it for later, or STOP
gate 4  QUIET HOURS    is it 02:00 in THEIR timezone?
                        → yes ⇒ schedule for 08:00 local
```

ORDER MATTERS: the cheapest and most-likely-to-stop checks first.
Preferences reject the most and cost one cached lookup.

Retry classification, which is the part people get wrong:

provider response	retry?	why
200 OK	no	done
429 Too Many Requests	YES	back off, honour Retry-After
500 / 503	YES	transient, exponential + jitter
timeout	YES	but check for duplicates
400 Bad Request	NO	malformed; will fail forever
410 Gone (bad token)	NO	+ DELETE THE TOKEN
403 unsubscribed	NO	+ record the preference

Retrying a 4xx burns quota and money and never succeeds.
Not retrying a 5xx loses a notification that would have worked.

Fan-out in two stages:



ONE worker doing 10M sequentially at 100/s = 27 hours.
 10,000 batches across 200 workers = ~8 minutes.

And filter FIRST: of 10M followers, maybe 1M have opened a notification from this source in 30 days. Send to those.
 → 10x less cost, HIGHER open rate.

Batching, and why it improves three things at once:

WITHOUT batching

```

10:01 "A liked your photo"
10:02 "B liked your photo"
10:03 "C liked your photo"
10:04 "D liked your photo"
10:05 "E liked your photo"

```

5 interruptions → 1.

This is the highest-value feature in the system and it is almost always built last.

WITH a 5-minute window

```

10:05 "5 people liked your photo"

1 notification instead of 5
→ 80% less provider cost
→ higher open rate (less fatigue)
→ user does not mute you

```

5 How it actually works

Ingest, which returns immediately

```

@app.post("/notify")
def notify(event: dict) → tuple[dict, int]:
    event["event_id"] = event.get("event_id") or str(uuid.uuid4())
    event["received_at"] = time.time()
    queue.publish("events", event) # does not wait for delivery
    return {"accepted": event["event_id"]}, 202

```

`202 Accepted`, not `200 OK`, and the distinction is real: you have accepted responsibility for trying, not reported success. **The caller gets an event id it can use to query status later.**

And the `event_id` must come from the caller when the caller has one, because that is what makes the whole pipeline idempotent — a retried API call with the same event id must not produce a second notification.

The four gates

```
def should_send(user_id: int, kind: str, channel: str, event_id: str) → str:
    prefs = prefs_cache.get(user_id)
    if not prefs.wants(kind, channel):
        return "opted_out"  # cheapest, rejects the most

    key = f"dedup:{user_id}:{kind}:{event_id}"
    if not redis.set(key, "1", nx=True, ex=86400):
        return "duplicate"  # SET NX is atomic

    if rate_limiter.count(user_id, channel) ≥ prefs.hourly_cap:
        return "rate_limited"

    if in_quiet_hours(prefs.timezone) and kind not in URGENT:
        return "quiet_hours"

    return "send"
```

`redis.set(..., nx=True)` is the **deduplication**, and it is atomic — two workers processing the same event concurrently cannot both get `True`. **A separate `GET` then `SET` would be a race**, and at a million events a second that race fires constantly.

The order is deliberate: preferences first because they are a cached lookup and reject the most.

Quiet hours, in the user's timezone

```
from zoneinfo import ZoneInfo
from datetime import datetime

def in_quiet_hours(tz_name: str, start: int = 22, end: int = 8) → bool:
    local_hour = datetime.now(ZoneInfo(tz_name)).hour
    return local_hour ≥ start or local_hour < end  # wraps midnight
```

`local_hour ≥ start or local_hour < end` **handles the wrap**, and writing it as `start ≤ h ≤ end` is the bug: 22 to 8 never satisfies that, so quiet hours never trigger and nobody notices until a user in another timezone complains.

And storing the user's timezone is a real requirement. A server-side “night” is meaningless when your users span twelve zones.

Sending, with retry classification

```
PERMANENT = {400, 403, 404, 410, 422}

def send_with_retry(message, max_attempts: int = 5) → str:
    for attempt in range(max_attempts):
        response = provider.send(message)
        if response.ok:
            return "sent"
        if response.status in PERMANENT:
            handle_permanent(message, response)      # delete token, record opt-out
            return "permanent_failure"                # NEVER retry
        delay = min(2 ** attempt, 60) + random.uniform(0, 1)  # backoff + jitter
        time.sleep(delay)
        dead_letter.publish(message)
    return "exhausted"
```

The `PERMANENT` set is the important line. A `410 Gone` from APNs means the device token is dead — the app was uninstalled — and retrying it five times costs five API calls and will never work. `handle_permanent` **deletes the token**, or you retry that user's dead token on every notification, forever.

Jitter, not just backoff. Without it, everything that failed during a provider outage retries at the same instant when it recovers, and the recovery is immediately undone.

The TTL, which prevents sending stale things

```
def process(message) → None:
    age = time.time() - message["created_at"]
    if age > message.get("ttl", 3600):
        metrics.increment("dropped_expired", tags={"kind": message["kind"]})
        return # do NOT send
    send_with_retry(message)
```

A queue that backed up for two hours should not then deliver two hours of “your ride is arriving”. The TTL is per notification type: an OTP is 5 minutes, a ride update is 10, a marketing message is a day.

And `dropped_expired` must be a metric, because a rising count is the earliest signal that the pipeline is behind.

Fan-out in two stages

```
def handle_fanout(event: dict) → None:
    cursor = None
    while True:
        followers, cursor = follower_store.page(event["author_id"], cursor, limit=1000)
        if not followers:
            break
        engaged = engagement_filter(followers, event["author_id"]) # cut ~90%
        queue.publish("send", {"recipients": engaged, "event": event})
        if cursor is None:
            break
```

Each page becomes its own queue message, so hundreds of workers drain them in parallel and a crash costs one batch rather than the whole fan-out.

`engagement_filter` is the highest-leverage line here: of ten million followers, perhaps one million have opened a notification from this author in the last month. Sending to the other nine million costs money and lowers the open rate, which is the metric the product team actually cares about.

Batching

```
def maybe_batch(user_id: int, kind: str, payload: dict) → None:
    if kind not in BATCHABLE:
        send_now(payload)
        return
    redis.rpush(f"batch:{user_id}:{kind}", json.dumps(payload))
    redis.expire(f"batch:{user_id}:{kind}", 600)
    scheduler.schedule_once(f"flush:{user_id}:{kind}", delay=300) # 5-minute window
```

`schedule_once` is the whole design — the first item in a window starts a timer, and later items join the existing batch rather than starting another. Then one flush renders “5 people liked your photo”.

The real systems

APNs (Apple)	HTTP/2, token auth, apns-collapse-id for dedup, apns-priority 5 (batched) or 10 (immediate)
FCM (Google)	Android and web; also proxies to APNs for iOS
Twilio / MSG91	SMS; per-country rules, sender IDs, DLT in India
SendGrid / SES	email; reputation matters – bounces and complaints get you blocked by the receiving mail servers
OneSignal, Braze, Airship, Firebase	full platforms: templating, scheduling, segmentation – usually cheaper than building this

Email reputation is worth one sentence in the interview, because it is a failure mode unique to that channel: sending to bad addresses damages your sending domain’s reputation, and once Gmail starts classifying you as spam, your legitimate mail stops arriving too. So bounce handling is not hygiene, it is deliverability.

6 The numbers

Volume.

```
100 million users
average 10 notifications per user per day

1,000,000,000 notifications/day
/ 86,400 s           = ~11,600/second average
peak is ~5x average = ~58,000/second
```

Split by channel, which is what decides the cost:

push	80%	800,000,000/day	free (APNs and FCM cost nothing)
in-app	15%	150,000,000/day	your own storage
email	4%	40,000,000/day	~\$0.0001 each = \$4,000/day
SMS	1%	10,000,000/day	~\$0.006 each = \$60,000/day

SMS is one percent of the volume and eighty-eight percent of the cost. That single line is the most useful thing in this section, and it drives a real design decision: **prefer push, fall back to SMS only when push is unavailable and the message is important.**

```
monthly: email $120,000 + SMS $1,800,000 = ~$1.9M/month
moving 20% of SMS to push saves ~$360,000/month
```

Machines.

```
58,000 sends/second at peak
one worker: ~500 sends/second (network-bound, mostly waiting)
→ 116 workers at peak

at ~50 workers per machine (async I/O)
→ ~3 machines for sending

resolver: 58,000/s x ~3 cache lookups each = 174,000 lookups/s
→ Redis at 100,000 ops/s per instance → 2-3 instances
```

Under ten machines for a billion notifications a day — and, as with the crawler, the point is that the bottleneck is not compute. **It is provider rate limits and money.**

Storage.

notification record: id, user, type, channel, status, timestamps, payload ref
~200 bytes

$1,000,000,000/\text{day} \times 200 \text{ B} = 200 \text{ GB/day}$
x 30 days retention = 6 TB
x 3 replicas = 18 TB

→ keep 30 days hot for the in-app inbox and support queries,
archive the rest to object storage at \$0.023/GB

Dedup store.

1,000,000,000 keys/day, TTL 24 hours
key ~60 bytes + Redis overhead ~100 bytes = 160 bytes

$1,000,000,000 \times 160 \text{ B} = 160 \text{ GB of Redis}$

→ that is expensive. Options:

- shorter TTL (1 hour): 6.7 GB. Covers retry windows, misses a duplicate 3 hours later.
- Bloom filter: ~1.2 GB at 1% false positive
→ 1% of notifications silently NOT sent. Unacceptable here, unlike the crawler.
- shard Redis by user id: 160 GB across 10 instances = 16 GB each.

The contrast with the crawler is worth stating: a Bloom filter's false positive was fine there and is not fine here, **because silently dropping one percent of notifications includes one percent of OTPs.**

Fan-out timing.

celebrity with 10,000,000 followers

single worker at 500 sends/s = 20,000 s = 5.5 hours
(last follower notified tomorrow)

batched, 10,000 batches of 1,000, 200 workers
each batch: 1,000 / 500 per second = 2 s
10,000 batches / 200 workers = 50 rounds x 2 s = 100 seconds

with engagement filtering (10% engaged):
1,000,000 recipients → ~10 seconds

Five and a half hours against ten seconds, and most of that gain is the filter rather than the parallelism.

Batching's saving.

a user gets 5 likes in 5 minutes

unbatched: 5 pushes

batched: 1 push

if 40% of all notifications are batchable social events,
and batching averages 4:1:

400,000,000/day batchable → 100,000,000/day sent

→ 300,000,000 fewer notifications/day

→ 30% less total volume, and a measurably higher open rate

Provider rate limits, which are the real ceiling:

APNs	no published hard limit, but connection-based; ~10,000/s per HTTP/2 connection, many connections allowed
FCM	~600,000 messages/minute per project = 10,000/s
Twilio	1/second per number by default; short codes ~100/s → 10,000,000 SMS/day needs ~116/s sustained → you need short codes or many numbers, arranged in ADVANCE

The Twilio line is the one that catches people. One phone number sends one SMS a second — **86,400 a day** — so ten million a day is not a scaling problem you can solve at runtime. **It is a procurement problem, and it takes weeks.**

7 The trade-offs

Synchronous against asynchronous ingest. Asynchronous is right: the emitting service returns in milliseconds and a provider outage does not break checkout. **The cost is that the caller cannot know whether the notification arrived**, so “did my user get the OTP” becomes a separate status query rather than a return value. **For OTPs specifically, some systems go synchronous** and accept the coupling, because the user is staring at a screen waiting.

At-least-once against at-most-once. Queues give at-least-once, so duplicates happen and dedup is required. **At-most-once would mean occasionally losing a notification**, which for a marketing message is fine and for an OTP is not. **Different notification types genuinely want different guarantees**, and the honest design uses at-least-once plus dedup everywhere rather than two pipelines.

Dedup store size against dedup window. A 24-hour window costs about 160 GB of Redis; an hour costs 6.7 GB and misses a duplicate that arrives three hours later. **A Bloom filter is 1.2 GB and silently drops one percent** — acceptable for a crawler, not acceptable when one percent includes one percent of one-time passwords.

Batching against latency. Batching cuts volume by a third, raises open rates and saves money, and it means a notification arrives up to five minutes late. **For social events that is invisible; for an OTP it is useless.** So batching is per notification type, never global, and the type list is a product decision.

Aggressive filtering against reach. Sending only to engaged users cuts cost tenfold and raises the open rate — **and it means people who would have engaged this time do not hear about it.** The filter is a model and models are wrong. **A small hold-out that always receives is worth keeping,** both to measure the filter and to avoid a permanent silent exclusion.

Building against buying. OneSignal, Braze and Airship do all of this, including templating, segmentation, scheduling and analytics. **Building your own is justified by volume — at a billion a day the per-message pricing is real money — and by data residency requirements.** Below roughly ten million a day, buying is almost always correct, and saying so is a better answer than designing enthusiastically.

When would I not build this? When the volume is low enough that a managed platform is cheaper than the engineers. When notifications are transactional only — order confirmations, password resets — where a queue and a provider SDK is genuinely the whole system and the preference machinery is over-engineering. **And when what is actually wanted is in-app messaging,** which is a different product: it needs no providers, no tokens and no quiet hours, because nothing interrupts anyone.

8 In the interview

How it gets asked

- “*Design a notification system.*” — usually with a scale like a billion a day.
- “*How do you make sure a user doesn’t get the same notification twice?*”
- “*A push provider is down. What happens?*” — retry classification.
- “*A celebrity with ten million followers posts. Walk me through it.*”
- “*How do you handle user preferences and quiet hours?*”
- “*What is your most expensive channel?*” — the SMS question.

“Five stages — ingest, resolve, build, send, track — and I want to say up front that **the interesting engineering here is in not sending things**, because most notifications at scale are unwanted and the failure mode is a human being annoyed.

Ingest is asynchronous. The emitting service puts an event on a queue and gets a 202 back immediately. **Checkout must not be slower because an email provider is slow, and must certainly not fail because it is down.** The queue is also what makes retries possible, because a failed send needs somewhere to live between attempts.

Resolve is where most of the logic lives, and it is four gates in a deliberate order. Preferences first, because it is one cached lookup and it rejects the most. Then deduplication. Then a per-user rate limit — **no more than ten pushes an hour to one person, whatever the events say.** Then quiet hours, **in the user’s timezone**, which is the part that gets forgotten.

Send is split by channel: one queue and one worker pool for push, one for SMS, one for email. Not a shared pool — a slow SMS provider would otherwise consume every worker and stop email too. **That is a bulkhead, and it is the specific reason for the split.**

Track is a webhook back from the provider, and the important part is the feedback loop: **when APNs reports a token as gone, delete it**, or you send to a dead token forever.

Now the number I would lead with, because it drives a real decision. At a billion a day: push is eighty percent of volume and free; SMS is **one percent of volume and eighty-eight percent of the cost**, at about \$60,000 a day. **So the design should prefer push and fall back to SMS only when push is unavailable and the message matters** — that is a bigger win than anything architectural.

And the second thing I would raise early: batching. Five separate ‘someone liked your photo’ pushes should be one ‘five people liked your photo’. **That cuts volume by about a third, costs less, and raises the open rate** — three things improved at once, which is rare, and it is almost always built last.

One question before I go further: what is the mix of transactional and marketing? Because an OTP wants low latency and no batching, and a marketing push wants filtering and batching, and they barely share requirements beyond the

transport.”

The follow-ups

“How do you make sure a user doesn’t get the same notification twice?”

“This matters more here than almost anywhere else, because **a duplicate is visible to a human**. A duplicated database row is a bug I fix on Monday; a second ‘your card was charged’ push at three in the morning is a support ticket and possibly a deleted app.

Where duplicates come from: the queue is at-least-once. A worker takes a message, sends it, and crashes before acknowledging — the message is redelivered and sent again. That is not a bug in the queue; it is what at-least-once means.

The fix is an idempotency key that comes from the event, not from processing. Something like a hash of user id, notification type and event id. **If I generate the key during processing, both attempts generate different keys and dedup does nothing** — which is the mistake worth naming.

Store it with `SET key value NX EX 86400` **in Redis.** `NX` means set-if-absent and it is atomic, so two workers handling the same event concurrently cannot both get true. **A** `GET` **followed by a** `SET` **is a race**, and at fifty thousand a second that race fires constantly.

The gap I would admit to: the check and the send are not atomic. If I set the key, then crash before sending, the notification is lost; if I send, then crash before setting, it duplicates. **I would set the key first — losing a notification is better than duplicating one** for most types, and for OTPs the user can request another.

And I would push the key down to the provider where it supports one. APNs has `apns-collapse-id`, which makes it replace rather than add — so even if I do send twice, the phone shows one. **Defence at two layers, because the first one has a real hole.**

Sizing, since it is not free. A billion keys a day at about 160 bytes with Redis overhead is 160 GB. **I would shard by user id across ten instances**, rather than shortening the TTL, and I would specifically not use a Bloom filter here — **a one percent false positive rate means silently not sending one percent of notifications, and that includes one percent of one-time passwords.**“

“APNs starts returning errors. What happens?”

“It depends entirely on which error, and getting that classification right is the difference between a system that recovers and one that burns its quota.

Transient failures get retried with exponential backoff and jitter. 500 , 503 , timeouts. Back off one second, two, four, up to a minute, with a random component — **without jitter, everything that failed during the outage retries at the same instant when it recovers and knocks it over again.**

429 is a special transient case: honour the `Retry-After` header rather than my own backoff, because the provider is telling me exactly what it wants.

Permanent failures must never be retried, and this is the one people get wrong. A 400 for a malformed payload will fail identically forever. A 410 Gone means the device token is dead — the app was uninstalled. **Retrying those costs five API calls, five times the quota, and succeeds zero times.**

And permanent failures need an action, not just an absence of retry. A 410 must **delete the token from my store**, or I attempt that dead token on every future notification for that user, forever. A 403 for an unsubscribed email address must **record the preference**, because continuing to send damages my sending domain’s reputation and eventually my legitimate mail stops arriving too.

After the retries are exhausted, the message goes to a dead letter queue — not dropped silently — so I can see what failed and why.

Two more things for a full provider outage. A circuit breaker: after enough consecutive failures, stop calling APNs and fail fast, so I do not have every worker blocked on a five-second timeout — otherwise a slow provider exhausts the pool and takes down the channels that are working. **And the TTL:** if APNs is down for two hours and then recovers, I should not deliver two hours of stale ‘your ride is arriving’. **Every message carries a TTL and expired messages are dropped and counted**, and that counter is my earliest signal that the pipeline is behind.”

“A celebrity with ten million followers posts. Walk me through it.”

“The naive version takes five and a half hours and the last follower hears about it tomorrow, so this needs two things: **parallelism, and — more importantly — sending to fewer people.**

First, the fan-out is two stages, not one. The post event becomes a single fan-out job. That job pages through the follower list a thousand at a time, and **each page becomes its own message on the send queue.** Hundreds of workers drain that queue in parallel.

Two separate queue messages matters for a specific reason: if it were one long-running job and the worker crashed at follower nine million, I would lose everything or resend everything. **With batches, a crash costs one batch of a thousand, and the retry is cheap and idempotent.**

The arithmetic. One worker at five hundred sends a second is twenty thousand seconds — five and a half hours. Ten thousand batches across two hundred workers, two seconds each, is about a hundred seconds.

But the bigger win is not sending. Of ten million followers, how many have opened a notification from this author in the last thirty days? **Realistically about a tenth.** Filtering to those cuts it to one million recipients — **about ten seconds, a tenth of the cost, and a higher open rate,** because the people who were going to ignore it were also the people slowly training themselves to ignore me.

Two things I would be careful about. The filter is a model and models are wrong, so I would keep a small holdout that always receives, both to measure whether the filter is right and to make sure nobody is permanently and invisibly excluded.

And ordering: for a fan-out this large, some followers get it a minute after others. For a social post that is fine. **If this were a live-event notification where fairness matters** — a ticket drop, say — I would have to say plainly that a ten-million fan-out cannot be simultaneous, and either randomise the batch order so the unfairness is not correlated with anything, or send a single broadcast through a topic-based channel instead of per-user messages.”

The model answer

“Design the notification system for a ride-hailing app: driver assigned, driver arriving, trip complete, payment receipt, plus promotions. Ten million rides a day.”

“The thing that shapes this design is that **the notification types have wildly different requirements**, and treating them uniformly would be the main mistake. Let me separate them first.

‘Driver arriving’ is time-critical and worthless late — a notification that arrives after the driver has gone is worse than nothing. **‘Payment receipt’ can be a minute late and nobody cares. Promotions should be batched, filtered, and never sent at 2am.** So: **three priority tiers with separate queues and separate worker pools**, not a priority field on one queue — because a million promotional messages must not be able to sit in front of a driver-arriving push.

Volume. Ten million rides a day at roughly five notifications each is fifty million a day, about 580 a second average and maybe 3,000 at peak — which is small. **Compute is not the problem here.**

The problem is SMS. Ride-hailing needs SMS because drivers and riders may not have the app in the foreground, and **SMS at six-tenths of a paisa each adds up fast.** If ten percent of those fifty million go by SMS, that is five million a day, about \$30,000 a day. **So the design tries push first and falls back to SMS only when push fails or the device has no valid token** — and for ‘driver arriving’ specifically, I would pay for the SMS fallback, because a missed ride is worth more than the message.

And a procurement point I would raise early: Twilio sends one SMS per second per number by default. **Five million a day is about sixty a second sustained, so this needs short codes or a pool of numbers, arranged weeks in advance.** That is not something you fix at runtime, and in India there is DLT registration on top.

The pipeline. Ride services publish events to Kafka. A resolver consumes them, runs the four gates — preferences, dedup, rate limit, quiet hours — and routes to per-channel queues.

Quiet hours with an exception list, which this app needs specifically. Promotions respect 10pm–8am in the rider’s local timezone. **‘Driver arriving’ ignores quiet hours entirely**, because the user booked a ride at 2am and wants to know. **That exception list is a product decision I would want written down**, because getting it wrong in either direction is bad — waking people up, or silently not telling them their car is outside.

Deduplication with an event-derived key — hash of user, type and ride id — in Redis with `SET NX`. At fifty million a day and a 24-hour TTL that is about 8 GB, which is one instance. And `apns-collapse-id` passed through, which does something genuinely useful here: successive ‘driver is 3 minutes away’ / ‘2 minutes away’ updates replace each other on the lock screen instead of stacking.

TTLs per type, which matter more here than in most systems. ‘Driver arriving’ expires in five minutes — if the queue backs up, that message must be dropped, not delivered late, because a stale arrival notification actively misleads. Receipts get a day. Promotions get a day.

Retry classification as usual, with `410` deleting the token — and I would call out that **ride-hailing has unusually stale tokens**, because drivers reinstall the app constantly, so the cleanup loop is not optional hygiene.

What I would monitor: delivery latency at p99 per type, with a tight alert on the driver-arriving tier; the expired-drop counter, which is my earliest warning that the pipeline is behind; SMS spend per hour, because a bug that flips push to SMS is expensive within minutes; and provider error rates split by permanent and transient.

And the honest closing point. At fifty million a day, a managed platform would probably be cheaper than the team required to run this — the per-message pricing at this volume is not obviously worse than the engineers. **What would justify building it is the latency requirement on driver-arriving and the tight coupling to ride state**, and I would want that to be the stated reason rather than a default.”

9 Recall card

Five stages: ingest → resolve → build → send → track. Ingest is asynchronous with a `202` so a provider outage cannot break checkout. **One queue and worker pool per channel** — a bulkhead, so slow SMS cannot starve email.

Four gates in the resolver, cheapest first: preferences → dedup → rate limit → quiet hours (in the user’s timezone, and `h ≥ 22 or h < 8`, not a range check). The per-user hourly cap is the most valuable feature and is always built last.

Dedup with an event-derived key — `hash(user, type, event_id)`, never generated during processing — via Redis `SET NX EX` (atomic; `GET`-then-`SET` is a race). Set the key **before** sending, and pass `apns-collapse-id` so the provider deduplicates too. **No Bloom filter here** — 1% silently unsent includes 1% of OTPs.

Retry classification is the thing people get wrong. `429` / `5xx` / timeout → back-off **with jitter**; `400` / `410` / `403` → **never retry, and act**: delete the dead token, record the opt-out. **Every message has a TTL** and expired ones are dropped and counted — that counter is the earliest signal of a backed-up pipeline.

Fan-out in two stages (one fan-out job → per-1,000 batches as separate messages): 10M followers is 5.5 hours on one worker, ~100 s batched, **~10 s with engagement filtering** — and the filter, not the parallelism, is most of the win. **Batching 5 likes into 1 push cuts volume ~30%, costs less, and raises open rates.**

SMS is ~1% of volume and ~88% of cost (\$60k/day at 1B/day) — prefer push. **And one Twilio number sends 1 SMS/second**, so high volume is a procurement problem, not a runtime one.

DSA topic: Longest increasing subsequence **System design topic:** Design a notification system at scale

Code these, in this order

One rule for the whole set: **write the state as a full sentence containing the words “ending exactly at” before you write any code.** The natural state fails on every problem here, and the sentence is what stops you writing it.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Longest Increasing Subsequence	LeetCode 300 (Medium)	The state redefinition, and <code>max(dp)</code> not <code>dp[-1]</code> .
2	Russian Doll Envelopes	LeetCode 354 (Hard)	Sorting to remove a dimension, and the tie-break minus sign.
3	Number of Longest Increasing Subsequences	LeetCode 673 (Medium)	Counting alongside, and the replace-versus-add branches.
4	Longest Increasing Path in a Matrix	LeetCode 329 (Hard)	The same idea on a graph — memoised DFS, not a table.
5	Maximum Length of Pair Chain	LeetCode 646 (Medium)	Where greedy beats LIS, and why.
6	Longest Arithmetic Subsequence	LeetCode 1027 (Medium)	LIS with a second dimension in the state.

On problem 1, write the wrong state first and see it fail

Implement `dp[i] = best in the first i elements` by comparing only with the immediately previous element. Run it on `[2, 5, 3, 7]` and record the answer. Then run it on `[3, 4, 1, 2, 5]` and record that.

One of those is right and one is wrong. Say which, and say why the first test passing is the dangerous part.

On problem 1, return `dp[-1]` deliberately

Run the correct `O(n2)` version on `[1, 2, 3, 0]` and print the whole `dp` array. Record `dp[-1]` and `max(dp)`. **Say in one sentence why they differ**, using the words from your state definition.

On problem 1, prove `tails` is not the answer

Run the $O(n \log n)$ version on `[1, 6, 7, 2]` and print `tails` at the end. Check by hand whether those values appear in that order in the input. **Then write the parent-pointer version** and confirm it returns something that does.

On problem 1, flip one character

Change `bisect_left` to `bisect_right`. Run both on `[1, 3, 3, 5]` and record both answers. Say which problem each one solves, and where in a problem statement you would look to decide.

On problem 2, get the tie-break wrong on purpose

Sort by `(width, height)` ascending and run LIS on the heights. Test on three envelopes all of width 1 with heights 1, 2 and 3. Record the answer, and the correct answer. **Then add the minus sign and explain it in one sentence.**

On problem 3, break the counting branches

Swap the `>` and `=` branches — add on a strictly better length, replace on an equal one. Run on `[1, 3, 5, 4, 7]` and record the answer against the correct one. **Say why the wrong number is plausible.**

On problem 5, find where greedy wins

Solve it with LIS first, then with a greedy sort-by-end-time. Compare running times at `n = 1000`. **Say in one sentence why greedy is valid here and not for problem 1.**

Then the timing drill

Time the $O(n^2)$ and $O(n \log n)$ versions at `n = 1,000`, `n = 5,000` and `n = 20,000` on random input. Record the ratio at each size and say roughly where the crossover in practice sits.

The state drill

1. State the wrong state and say exactly where the recurrence gets stuck.
2. State the right state, with the words “ending exactly at”.
3. Say what the state space cost of the fix was. (Nothing — say that, and why it matters.)
4. Say why the answer is `max(dp)`, and give the input where it differs.

The tails drill

1. Say what `tails[k]` holds, in one sentence.

2. Give the argument for why smallest is best, using 7 and 9.
3. Say why `tails` is sorted and why that matters.
4. Trace it by hand on `[10, 9, 2, 5, 3, 7, 101, 18]`.
5. Say what `tails` is *not*, and give the input that proves it.

The reconstruction drill

1. Say why you cannot read the answer off `tails`.
2. Describe the two extra arrays and what each holds.
3. Give the parent rule in one sentence.
4. Say what the walk-back starts from.
5. Say why the quadratic version is easier here, and when you would prefer it.

The variants drill

1. `bisect_left` versus `bisect_right` — which problem each solves.
2. Longest decreasing, two ways.
3. Longest bitonic, and why the `-1`.
4. Russian dolls, including the tie-break.
5. Counting the longest ones, and the two branches.

The complexity drill

1. Give both complexities, time and space.
2. Compute comparisons for `n = 1,000`, `10,000` and `100,000` in the quadratic version.
3. Say what the ratio is at `n = 100,000`.
4. Say what LeetCode 300's constraint is and why it is set there.

The break-it drill

Trigger each and record the exact output or error:

1. Returning `dp[-1]`.
2. The incomplete state, on `[3, 4, 1, 2, 5]`.
3. `bisect_right` when you wanted strict.
4. Returning `tails` as the subsequence.
5. `max(dp)` on an empty input.
6. Envelopes sorted `(width, height)` ascending.
7. The quadratic version at `n = 100,000`.

Five of the seven give no error at all. Name them.

The pipeline drill

1. Name the five stages and what each does.
2. Say why ingest is asynchronous and what it costs you.
3. Say why `202` rather than `200`.
4. Say why there is one queue per channel, and name the pattern.

The gates drill

1. Name the four gates in order.
2. Say why that order and not another.
3. Write the quiet-hours condition and say why the range check is wrong.
4. Say why the timezone must be the user's.
5. Say which gate is the most valuable and when it usually gets built.

The dedup drill

1. Say where duplicates come from, precisely.
2. Say where the idempotency key must come from, and the mistake if it does not.
3. Give the exact Redis command and say why `GET`-then-`SET` is wrong.
4. Say what gap remains, and which way you would fall on it.
5. Say what the provider can do for you, and name the header.
6. Do the sizing at 1 billion/day and say why a Bloom filter is wrong here.

The retry drill

1. Classify seven provider responses into retry and no-retry.
2. Say what a `410` requires beyond not retrying.
3. Say what a `403` on email requires, and why deliverability makes it urgent.
4. Say what jitter prevents.
5. Say what the circuit breaker is for and what a slow provider does without one.
6. Say what a TTL prevents, and what its counter tells you.

The fan-out drill

1. Give the two stages and say why they are separate messages.
2. Do the arithmetic for 10M followers, both ways.
3. Say what engagement filtering does to cost and to open rate.

4. Say what a holdout group is for.
5. Say what you would do if fairness of ordering mattered.

The cost drill

1. Give the channel split by volume and by cost at 1 billion/day.
 2. Say which channel dominates cost and by how much.
 3. Say what one Twilio number can send per day.
 4. Say why that is a procurement problem rather than a scaling one.
 5. Compute what batching saves, in volume and in open rate terms.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the longest strictly increasing subsequence.* Why the natural state fails, the redefined state, the recurrence, `max(dp)` with the counter-example, and the complexity.
 2. *Can you do better than $O(n^2)$? What does that array hold?* `tails[k]`'s meaning, the smallest-is-best argument, why it is sorted, the four-line loop, and the fact that its contents are not the answer.
 3. *Design a notification system.* The five stages, the four gates in order, dedup with an event-derived key, retry classification with the `410` action, and the SMS cost line.
-

Before you move on

- [] I can state the wrong LIS state and say where it gets stuck.
- [] I can state the right one, with “ending exactly at”.
- [] I know redefining cost nothing in state space.
- [] I return `max(dp)` and can give the input where `dp[-1]` differs.
- [] I can write the $O(n^2)$ version from the sentence.
- [] I can say what `tails[k]` holds, exactly.
- [] I can give the smallest-is-best argument with 7 and 9.
- [] I know why `tails` is sorted and why that licenses binary search.
- [] I can trace `tails` by hand on the standard input.
- [] I know `tails` is not a subsequence, and the input that proves it.

- [] I can reconstruct with parent pointers.
- [] I know `bisect_left` is strict and `bisect_right` is not.
- [] I can do Russian dolls including the tie-break minus sign.
- [] I can count the longest ones with the right two branches.
- [] I know both complexities and the ratio at `n = 100,000`.
- [] I know `max([])` raises and the `tails` version does not.
- [] I can name the five notification stages.
- [] I know why ingest is asynchronous and what it costs.
- [] I can name the four gates in order and justify the order.
- [] I can write the quiet-hours condition correctly.
- [] I know where the idempotency key must come from.
- [] I know `SET NX EX` and why `GET`-then-`SET` is a race.
- [] I know which way I fall on the check-then-send gap.
- [] I can classify provider responses into retry and no-retry.
- [] I know a `410` means delete the token.
- [] I know why email bounces threaten deliverability.
- [] I know what a TTL prevents and what its counter signals.
- [] I can do the fan-out arithmetic both ways.
- [] I know engagement filtering beats parallelism, and why.
- [] I know SMS is $\sim 1\%$ of volume and $\sim 88\%$ of cost.
- [] I answered all three questions above out loud.

Longest common subsequence

DATA STRUCTURES AND ALGORITHMS

Longest common subsequence

SYSTEM DESIGN

Design a news feed

1 What this is, and why they ask it

Given two strings, find the length of the longest sequence of characters that appears in **both**, in order, not necessarily contiguously.

"abcde" and "ace" share "ace" — length 3. The **b** and **d** are skipped in the first string, and that is allowed, because a subsequence keeps order and may skip.

This is the first two-dimensional DP in the course, and that is why it is asked. Every problem so far has had a table indexed by one thing — an amount, a position, a capacity. **Here the state is a pair: how far you are through string one, and how far through string two.** Learning to say “`dp[i][j]` is the answer for the first **i** characters of **a** and the first **j** of **b**” is the skill, and once you have it, edit distance, string matching, and half the hard string problems become the same shape.

They also ask it because **it is what diff is**. Git’s diff, the side-by-side comparison in a code review, the DNA alignment tools in bioinformatics — all of them are LCS or a close relative. **The characters not in the LCS are exactly the additions and deletions**, which is a genuinely satisfying thing to be able to say out loud.

And there is one specific trap that catches almost everyone, every time: `dp[i][j]` refers to prefix lengths, so `dp[i][j]` compares `a[i-1]` with `b[j-1]`, not `a[i]` with `b[j]`. That off-by-one is the entire difficulty of implementing this correctly under pressure.

By the end of this lesson you can write the two-dimensional table and explain both branches, reconstruct the actual subsequence, collapse the space to two rows, connect it to `diff`, and handle the family — longest common substring, shortest common supersequence, and the palindrome trick.

2 The story

The tailor had two lists and they did not agree, and the customer was standing there.

The first list was the one Rehmat had written in his own book in March, when the order was placed. Nine items. Two shirts, cream, full sleeve. A pair of trousers, grey. A blazer. The kurta for the boy. And so on down the page.

The second list was the one the customer had on her phone, which she had typed the same evening from what she remembered.

And now, six weeks later, there was an argument about whether the blazer had ever been ordered.

So they did the only sensible thing, which was to go through both, together, from the top.

“Two shirts, cream.” — **On both. Fine.**

“Trousers, grey.” — On both.

Rehmat’s next line was the blazer. Her next line was the kurta.

And here is where they stopped, because there were two ways to go.

Either the blazer was on his list and not on hers — he had added something — **and you skip his line and carry on comparing.** Or the kurta was on hers and not his — she had added something — **and you skip her line instead.**

They could not tell which, from where they were standing. The only way to know was to try one, keep going, and see which choice let more of the rest of the two lists line up.

So that is what they did, and it took twenty minutes and involved starting over twice.

What they ended up with, by the time the tea came, was a middle list. **Six items that were definitely on both, in the same order.** And what was left over — the blazer on his side, the kurta and a petticoat on hers — was the actual disagreement, and there were only three things in it instead of nine.

The customer’s observation, which was the useful one, was that they had never actually needed to settle the argument. **They only needed to find the largest part where the two lists agreed, and everything not in it was the problem.**

3 The idea in plain English

Rehmat and his customer have just run the LCS algorithm by hand, including the moment where it branches.

First, the definition, precisely. A **subsequence** keeps order and may skip. A **common** subsequence is one that appears in both strings. The **longest** common subsequence is the biggest such thing, and the question usually asks only for its length.

Now the state, and it is a pair, which is new.

`dp[i][j]` is the length of the longest common subsequence of the first `i` characters of `a` and the first `j` characters of `b`.

Prefix lengths, not indices. `i = 3` means “the first three characters”, which are `a[0]`, `a[1]`, `a[2]`. So the character that `i` refers to is `a[i-1]`, and that single fact is where every off-by-one in this problem comes from.

Why prefix lengths rather than indices? Because `i = 0` then means “the empty prefix”, which gives a clean base case with no special handling. **Using indices forces you to guard against `-1` everywhere.** It is worth choosing the convention deliberately and writing it down before coding.

The recurrence has two cases and they map exactly onto Rehmat’s two situations.

Case one: the characters match. `a[i-1] == b[j-1]`. Then that character can be the last one in the common subsequence, so:

```
dp[i][j] = dp[i-1][j-1] + 1
```

Both lists advance together, and you add one.

Case two: they do not match. Then the last character of at least one prefix is not in the LCS, and **you cannot tell which** — that is exactly where they stopped. So try both:

```
dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

`dp[i-1][j]` skips a character of `a`. `dp[i][j-1]` skips a character of `b`. Take whichever gives more.

The base cases are the empty prefixes. `dp[0][j] = 0` and `dp[i][0] = 0`: the LCS of anything with an empty string is empty. **In code that is one row and one column of zeros, and if you allocate with `[[0] * (m+1) for _ in range(n+1)]` you get them for free.**

The answer is `dp[n][m]` — the bottom-right cell, both strings fully consumed. Unlike LIS, this one really is the last cell, because the state covers all of both strings rather than “ending exactly here”.

Now three things worth having ready.

Reconstruction. Walk back from `dp[n][m]`. If the characters match, that character is in the LCS — record it and move diagonally. If not, move to whichever of the two neighbours is larger. **When they are equal, pick either; you get a different valid LCS, and there is usually more than one.**

The space collapse. Row `i` reads only row `i-1`, so keep two rows instead of `n+1`. `O(min(n, m))` space by iterating with the shorter string as the inner dimension. **But you lose reconstruction** — same trade as knapsack, and worth stating.

And the diff connection, which is what makes this feel useful rather than academic:

```
characters in the LCS      = unchanged lines
characters of a NOT in it  = DELETIONS
characters of b NOT in it  = INSERTIONS
```

That is diff. Run LCS on the lines of two files, and the reconstruction gives you the unified diff. Git uses a refined version — Myers' algorithm — which is faster on the typical case where the files are similar, **but the definition of “the right answer” is LCS.**

Finally, the family, which is larger than it looks.

Longest common substring — contiguous — is a different recurrence and easy to confuse. **On a mismatch, the answer is 0, not the max of neighbours**, because contiguity is broken. And the answer is `max` over the whole table, not the last cell.

Shortest common supersequence — the shortest string containing both as subsequences — is `n + m - LCS(a, b)`, because the shared part is written once instead of twice. **One line, once you have LCS.**

Longest palindromic subsequence of `s` is `LCS(s, reverse(s))`. That is a genuinely surprising reduction and it is asked; the reason is that a palindrome reads the same forwards and backwards, so a subsequence common to `s` and its reverse is exactly a palindromic subsequence.

And the minimum number of deletions to make two strings equal is `n + m - 2 * LCS` — delete everything not shared, from both.

4 The picture

The table for `a = "abcde"`, `b = "ace"`:

Reconstruction, walking back:

```
a = "abcde", b = "ace"

start at dp[5][3] = 3
  a[4]='e', b[2]='e' MATCH → take 'e', move to dp[4][2]
dp[4][2] = 2
  a[3]='d', b[1]='c' no    → dp[3][2]=2 vs dp[4][1]=1 → go UP to dp[3][2]
dp[3][2] = 2
  a[2]='c', b[1]='c' MATCH → take 'c', move to dp[2][1]
dp[2][1] = 1
  a[1]='b', b[0]='a' no    → dp[1][1]=1 vs dp[2][0]=0 → go UP to dp[1][1]
dp[1][1] = 1
  a[0]='a', b[0]='a' MATCH → take 'a', move to dp[0][0]
dp[0][0] = 0 STOP

collected 'e','c','a' → reverse → "ace"
```

LCS as diff:

```
a = "abcde"    b = "ace"    LCS = "ace"
```

```
a → in LCS    keep
b → not        DELETE
c → in LCS    keep
d → not        DELETE
e → in LCS    keep
```

as a unified diff:

```
a
- b
  c
- d
  e
```

Run this on the LINES of two files and you have written `diff`.

Common subsequence against common **substring**, which is the confusion:

```
a = "abcdef"    b = "abzdef"
```

```
longest common SUBSEQUENCE = "abdef", length 5    (skip the c and the z)
longest common SUBSTRING   = "def",   length 3    (must be contiguous)
```

the recurrences differ on ONE line:

```
subsequence, mismatch: dp[i][j] = max(dp[i-1][j], dp[i][j-1])
substring,    mismatch: dp[i][j] = 0                      ← the run is broken
```

and the ANSWER differs too:

```
subsequence: dp[n][m]
substring:   max over the WHOLE table
```

5 The code, built step by step

The table, from the sentence

```
def longest_common_subsequence(a: str, b: str) → int:
    n, m = len(a), len(b)
    # dp[i][j] = LCS length of a's first i characters and b's first j
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    return dp[n][m]                                # (loops next)
```

The **+1** in both dimensions is the empty prefix, and allocating with zeros gives both base cases for free — no special-case code at all.

Now the two branches:

```
for i in range(1, n + 1):
    for j in range(1, m + 1):
        if a[i - 1] == b[j - 1]:                # NOTE the -1 on both
            dp[i][j] = dp[i - 1][j - 1] + 1
        else:
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
```

a[i - 1] and **b[j - 1]**. Say it out loud while writing: “**i** is a length, so the character is at **i** minus one.”

Loops from 1, not 0, because row and column zero are the base cases and must not be overwritten.

Reconstructing the subsequence

```
def lcs_string(a: str, b: str) → str:
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    result: list[str] = []
    i, j = n, m
    while i > 0 and j > 0:
        if a[i - 1] == b[j - 1]:
            result.append(a[i - 1])           # this character is in the LCS
            i, j = i - 1, j - 1              # move diagonally
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1                             # skipping a character of a was better
        else:
            j -= 1
    return "".join(reversed(result))
```

≥ rather than > in the tie-break is arbitrary and it decides *which* LCS you get when several have the same length. **Both are correct; say that rather than pretending there is one answer.**

The walk-back is $O(n + m)$ — each step decreases `i` or `j` or both.

The space collapse

```
def lcs_length_two_rows(a: str, b: str) → int:
    if len(b) > len(a):
        a, b = b, a                      # make b the shorter one
    previous = [0] * (len(b) + 1)
    for i in range(1, len(a) + 1):
        current = [0] * (len(b) + 1)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                current[j] = previous[j - 1] + 1
            else:
                current[j] = max(previous[j], current[j - 1])
        previous = current
    return previous[len(b)]
```

$O(\min(n, m))$ space after the swap, and the swap is worth doing: two strings of 10,000 and 10 characters need 11 cells per row, not 10,001.

And reconstruction is gone. The rows are overwritten, so there is no table to walk back through. **If they ask for the string, you need the full table** — or Hirschberg's algorithm, which recovers it in linear space by divide and conquer, and is worth nam-

ing but not worth writing in an interview.

Longest common substring, which differs on one line

```
def longest_common_substring(a: str, b: str) → int:
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    best = 0
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
                best = max(best, dp[i][j]) # answer is the MAX, not the last cell
            # else: dp[i][j] stays 0 - the run is broken
    return best
```

Two differences and both matter. The mismatch case is `0`, not a `max` — contiguity is broken. And the answer is the running maximum, not `dp[n][m]`, because the best run can end anywhere.

The derived problems

```
def shortest_common_supersequence_length(a: str, b: str) → int:
    return len(a) + len(b) - longest_common_subsequence(a, b)

def min_deletions_to_make_equal(a: str, b: str) → int:
    return len(a) + len(b) - 2 * longest_common_subsequence(a, b)

def longest_palindromic_subsequence(s: str) → int:
    return longest_common_subsequence(s, s[::-1])
```

Three one-liners. The supersequence writes the shared part once instead of twice; the deletions remove everything unshared from both sides; **and the palindrome one works because a subsequence common to `s` and its reverse reads the same in both directions.**

The complete solution

```
"""Longest common subsequence, its reconstruction, and its family."""

def longest_common_subsequence(a: str, b: str) → int:
    """dp[i][j] = LCS of a's first i characters and b's first j."""
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]    # row/col 0 = empty prefix
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:             # i is a LENGTH, so index i-1
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[n][m]

def lcs_string(a: str, b: str) → str:
    """One longest common subsequence. There may be several of equal length."""
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    result: list[str] = []
    i, j = n, m
    while i > 0 and j > 0:
        if a[i - 1] == b[j - 1]:
            result.append(a[i - 1])
            i, j = i - 1, j - 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return "".join(reversed(result))

def lcs_length_two_rows(a: str, b: str) → int:
    """O(min(n, m)) space. Reconstruction is not possible from this."""
    if len(b) > len(a):
        a, b = b, a
    previous = [0] * (len(b) + 1)
    for i in range(1, len(a) + 1):
        current = [0] * (len(b) + 1)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                current[j] = previous[j - 1] + 1
            else:
                current[j] = max(previous[j], current[j - 1])
        previous = current
    return previous[len(b)]

def longest_common_substring(a: str, b: str) → int:
    """CONTIGUOUS. Mismatch resets to 0; the answer is the table maximum."""
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
```

```

best = 0
for i in range(1, n + 1):
    for j in range(1, m + 1):
        if a[i - 1] == b[j - 1]:
            dp[i][j] = dp[i - 1][j - 1] + 1
            best = max(best, dp[i][j])
return best

def shortest_common_supersequence_length(a: str, b: str) → int:
    """The shared part is written once instead of twice."""
    return len(a) + len(b) - longest_common_subsequence(a, b)

def min_deletions_to_make_equal(a: str, b: str) → int:
    """Delete everything not shared, from both sides."""
    return len(a) + len(b) - 2 * longest_common_subsequence(a, b)

def longest_palindromic_subsequence(s: str) → int:
    """A subsequence common to s and its reverse reads the same both ways."""
    return longest_common_subsequence(s, s[::-1])

def diff(a_lines: list[str], b_lines: list[str]) → list[str]:
    """LCS on lines is exactly what `diff` computes."""
    n, m = len(a_lines), len(b_lines)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a_lines[i - 1] == b_lines[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    out: list[str] = []
    i, j = n, m
    while i > 0 or j > 0:
        if i > 0 and j > 0 and a_lines[i - 1] == b_lines[j - 1]:
            out.append(" " + a_lines[i - 1])
            i, j = i - 1, j - 1
        elif j > 0 and (i == 0 or dp[i][j - 1] ≥ dp[i - 1][j]):
            out.append("+ " + b_lines[j - 1])
            j -= 1
        else:
            out.append("- " + a_lines[i - 1])
            i -= 1
    return list(reversed(out))

if __name__ == "__main__":
    print("lcs abcde/ace", longest_common_subsequence("abcde", "ace"))
    print("the subsequence", lcs_string("abcde", "ace"))
    print("two rows agrees", lcs_length_two_rows("abcde", "ace"))
    print("no overlap", longest_common_subsequence("abc", "def"))
    print("identical", longest_common_subsequence("abc", "abc"))
    print("empty", longest_common_subsequence("", "abc"))

    print("subsequence", longest_common_subsequence("abcdef", "abzdef"))
    print("substring", longest_common_substring("abcdef", "abzdef"))

    print("supersequence len", shortest_common_supersequence_length("abac", "cab"))
    print("min deletions", min_deletions_to_make_equal("sea", "eat"))

```

```

print("palindromic sub  :", longest_palindromic_subsequence("bbbab"))

print("diff:")
for line in diff(["import os", "x = 1", "y = 2", "print(x)",
                 ["import os", "import sys", "y = 2", "print(y)"]]):
    print("  ", line)

```

Run it and you get:

```

lcs abcde/ace      : 3
the subsequence   : ace
two rows agrees   : 3
no overlap        : 0
identical         : 3
empty             : 0
subsequence       : 5
substring         : 3
supersequence len : 5
min deletions     : 2
palindromic sub   : 4
diff:
  import os
- x = 1
+ import sys
  y = 2
- print(x)
+ print(y)

```

subsequence 5 against **substring 3** on the same input is the distinction, made visible: `"abdef"` versus `"def"`.

And the **diff** output is the point of the whole lesson. That is a real unified diff, produced by walking back through an LCS table — the same output `git diff` gives, from thirty lines.

6 What it costs

Time. Two nested loops over the two lengths.

```

n rows x m columns = n x m cells
each cell: one comparison, one addition or one max      0(1)

TOTAL: O(n x m)

```

Concretely:

two strings of 1,000	1,000,000 cells	~0.3 s in Python.	Fine.
two strings of 10,000	100,000,000 cells	~40 s.	Too slow.
two files of 10,000 lines	100,000,000 cells	same problem.	

LeetCode 1143's constraint is 1,000 characters, which is set exactly so the quadratic table passes.

Space.

full table	$(n+1) \times (m+1)$ integers $1,001 \times 1,001 = 1,002,001$ cells Python list of lists, ~8 bytes per pointer + int objects → about 40 MB
two rows	$2 \times (\min(n,m) + 1)$ $2 \times 1,001 = 2,002$ cells → about 80 KB

500x less, same answer, and no reconstruction.

And the swap matters more than it looks:

a = 10,000 characters, b = 10 characters

without the swap: rows of 10,001 → $2 \times 10,001 = 20,002$ cells
 with the swap: rows of 11 → $2 \times 11 = 22$ cells

900x less, from one `if`.

Why `diff` on real files needs something better:

two source files of 5,000 lines each
 $5,000 \times 5,000 = 25,000,000$ cells
 at 8 bytes per cell in a compact representation = 200 MB
 and ~10 seconds

Git uses MYERS' ALGORITHM instead: $O((n + m) \times D)$ where D is the number of differences.

two nearly-identical 5,000-line files, D = 20:
 Myers: $(5,000 + 5,000) \times 20 = 200,000$ operations
 LCS table: 25,000,000

125x faster, and it gets BETTER the more similar the files are – which is the normal case for source control.

That last sentence is the good thing to say: Myers is not a different answer, it is the same answer found by exploring only the region of the table near the diagonal, and real diffs live near the diagonal.

The derived problems cost nothing extra:

shortest common supersequence	one LCS + one subtraction
minimum deletions	one LCS + one subtraction
longest palindromic subsequence	one LCS on s and reversed(s) → $O(n^2)$ time, $O(n)$ space with two rows
reconstruction	$O(n + m)$ walk-back after the table → free in time, needs the FULL table in space

And the substring version:

same $O(n \times m)$ time and space
but a mismatch writes 0 instead of a max, so it is marginally faster in practice – no `max()` call on the common path.

7 The traps

The off-by-one, which is the defining bug of this problem.

```
>>> a, b = "abcde", "ace"
>>> dp = [[0] * 4 for _ in range(6)]
>>> for i in range(1, 6):
...     for j in range(1, 4):
...         if a[i] == b[j]:
...             # WRONG: should be a[i-1], b[j-1]
...             dp[i][j] = dp[i-1][j-1] + 1
...         else:
...             dp[i][j] = max(dp[i-1][j], dp[i][j-1])
Traceback (most recent call last):
  File "<stdin>", line 3, in <module>
IndexError: string index out of range
```

Here it happens to raise, because `i` reaches 5 and `a[5]` is past the end. On other inputs it does not raise and quietly compares the wrong characters — which is worse. Write “`i` is a length, so the character is `a[i-1]`” before the loops.

Looping from 0 and destroying the base case.

```
>>> dp = [[0] * 4 for _ in range(6)]
>>> for i in range(0, 6):           # starts at 0
...     for j in range(0, 4):
...         if i == 0 or j == 0:
...             continue           # you now need this guard
```

Starting at 0 forces a guard on every iteration. Starting at 1 makes row and column zero permanently the base case, with no runtime check at all. **One character, and it removes a whole class of bug.**

Confusing subsequence with substring.

```
>>> longest_common_subsequence("abcdef", "abzdef")
5
>>> longest_common_substring("abcdef", "abzdef")
3
```

Both are correct programs for different questions. The word to look for in the statement is “**contiguous**” — if it is there, a mismatch resets to zero and the answer is the table maximum, not the last cell.

Returning `dp[n][m]` from the substring version.

```
>>> # substring table for "abcdef" / "abzdef", last cell:
>>> # dp[6][6] = 3 here by luck, but on "abc"/"cba" it is 0
>>> # while the real answer is 1
```

The substring answer can be anywhere in the table, because the best run ends where it ends. **The subsequence answer really is the last cell** — this is the one place where the two conventions genuinely differ, and mixing them up gives plausible numbers.

Expecting a unique LCS.

```
>>> lcs_string("abcdbdab", "bdcaba")
'bcba'
```

"bcab" and "bdab" are also length 4 and equally valid. The tie-break in the walk-back decides which one you get. **If a test asserts a specific string, it is asserting your tie-break, not the answer** — which is worth saying if an interviewer challenges your output.

Reconstruction after the space collapse.


```
>>> # you have `previous`, a single row of numbers.
>>> # there is nothing to walk back through.
```

No error — you simply cannot do it. The rows have been overwritten. **State the trade when you collapse:** “this gives me the length in linear space and gives up the reconstruction.”

Two long strings.

```
>>> a = "x" * 20000
>>> b = "y" * 20000
>>> dp = [[0] * 20001 for _ in range(20001)]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
MemoryError
```

400 million cells. Read the constraint before choosing the full table — and if reconstruction is needed at that size, the answer is Hirschberg’s algorithm, not a bigger machine.

The empty string.

```
>>> longest_common_subsequence("", "abc")
0
>>> lcs_string("", "abc")
''
```

Both correct, and both free — the `range(1, 1)` loop simply does not execute and `dp[0][3]` is already 0. **This is the reward for choosing prefix lengths over indices**, and it is worth pointing at when explaining the convention.

8 In the interview

How it gets asked

- “Find the length of the longest common subsequence of two strings.” — LeetCode 1143, the standard.
- “Return the subsequence itself.”
- “How is this different from longest common substring?” — the one-line difference.
- “Can you reduce the space?” — and the follow-up about what you lose.
- “How would you implement `diff`?”
- “Find the longest palindromic subsequence.” — the surprising reduction.

The first ninety seconds

“This is my first genuinely two-dimensional DP, so let me set the state up carefully, because the convention I choose decides how painful the code is.

$dp[i][j]$ is the length of the longest common subsequence of the first i characters of a and the first j characters of b .

Prefix lengths, not indices — and I choose that deliberately. It means $i = 0$ is the empty prefix, so the base cases are a row and a column of zeros that I get for free from the allocation. **With indices I would be guarding against minus one everywhere.** The price is one off-by-one: the character i refers to $a[i-1]$, and I would write that down before the loops, because it is where every bug in this problem lives.

The recurrence has two cases.

If $a[i-1] = b[j-1]$, that character can end the common subsequence, so $dp[i][j] = dp[i-1][j-1] + 1$ — the diagonal, plus one. Both strings advance together.

If they differ, then at least one of those two characters is not in the LCS, and I cannot tell which. So I try both: $\max(dp[i-1][j], dp[i][j-1])$ — skip a character of a , or skip a character of b , whichever gives more.

Base cases: $dp[0][j] = dp[i][0] = 0$, because nothing is common with an empty string.

The answer is $dp[n][m]$ — and unlike longest increasing subsequence, here it really is the last cell, because the state covers all of both strings rather than ‘ending exactly here’.

$O(n \times m)$ **time and space.** Two strings of a thousand is a million cells — fine. Ten thousand each is a hundred million and about forty seconds, so **I would check the constraints before choosing the full table.**

And I can reduce the space to two rows, $O(\min(n, m))$ after swapping so the shorter string is the inner dimension. **But that gives up reconstruction,** so I would ask first whether they want the length or the string.”

The follow-ups

“Return the subsequence itself, not just the length.”

“That needs the full table, and the walk-back is the reverse of how it was filled.

Start at $dp[n][m]$ and move backwards. At each cell, ask the same question the fill asked.

If $a[i-1] = b[j-1]$, that character is in the LCS — record it, and move diagonally to $dp[i-1][j-1]$, because that is where this cell’s value came from.

Otherwise, move to whichever neighbour is larger — up if $dp[i-1][j]$ is bigger, left if $dp[i][j-1]$ is. That retraces the $\max$ that was taken during the fill.

Stop when either index hits zero, and reverse what I collected, because I built it from the end.

Cost is $O(n + m)$ — each step decreases i or j or both — so reconstruction is free in time. **What it costs is space:** I need the whole table, which is $O(n \times m)$, so I cannot use the two-row version.

Two honest points. The LCS is not unique. On “abcbdbab” and “bdcaba”, “bcba”, “bcab” and “bdab” are all length four. **My tie-break — whether I go up or left when the neighbours are equal — decides which one I return**, and both directions are correct. If a test asserts a specific string, it is asserting my tie-break rather than the answer.

And if the strings are large and I need both the string and linear space, that is Hirschberg’s algorithm: divide and conquer on the midpoint of one string, using the two-row version to find where the optimal path crosses. **$O(n \times m)$ time and $O(\min(n, m))$ space.** I would name it and say I would not write it under time pressure unless asked.”

“How is this different from longest common substring?”

“One line in the recurrence and one line in the answer, and mixing them up gives plausible wrong numbers, so it is worth being precise.

A substring is contiguous. A subsequence is not. On "abcdef" and "abzdef", the longest common subsequence is "abdef", length five — I skip the c and the z. **The longest common substring is "def", length three**, because contiguity means the run stops dead at the mismatch.

First difference: the mismatch case. For subsequences it is $\max(dp[i-1][j], dp[i][j-1])$ — I skip a character and carry on. **For substrings it is zero**, because a broken run is a broken run and there is nothing to carry.

Second difference: where the answer is. The subsequence answer is $dp[n][m]$, the bottom-right cell, because the state covers both entire strings. **The substring answer is the maximum over the whole table**, because the best run can end anywhere and the last cell only describes runs ending at the very end of both strings.

Getting the second one wrong is the sneaky bug, because on many inputs the last cell happens to be right.

The word to look for in the statement is ‘contiguous’, or ‘substring’ used precisely. And I would ask if it is ambiguous, because the two answers can differ by a lot.

One more thing about the substring version: there is a better algorithm. Suffix automata or suffix arrays solve longest common substring in linear time, and the DP is $O(n \times m)$. **For an interview the DP is the expected answer**, but knowing the better one exists is worth a sentence.”

“How would you implement `diff`?”

“`diff` is LCS, and I find that genuinely satisfying, so let me say why.

Run LCS on the lines of the two files rather than on characters. The longest common subsequence of the lines is the set of lines that are unchanged, in order.

Then everything not in it is the change. A line of the old file that is not in the LCS was **deleted**. A line of the new file that is not in the LCS was **inserted**. **That is the entire definition of a diff**, and the walk-back produces it directly — I just emit `-` when I move up, `+` when I move left, and a space when I move diagonally.

Hashing the lines first is worth doing, because comparing line hashes is cheap and comparing long strings repeatedly is not.

Where the plain table fails is size. Two source files of five thousand lines is twenty-five million cells — a couple of hundred megabytes and about ten seconds. **That is unacceptable for something that runs on every commit.**

So Git uses Myers’ algorithm, which is $O((n + m) \times D)$ where D is the number of differences. **The insight is that a typical diff is small** — two versions of a file differ in a handful of lines — so the optimal path through the table hugs the diagonal, and Myers explores outward from the diagonal instead of filling everything.

Concretely: two five-thousand-line files differing in twenty lines is two hundred thousand operations against twenty-five million. About a hundred and twenty-five times faster — and crucially, **it gets faster the more similar the files are**, which is exactly the case source control cares about.

And a practical detail real diffs add: heuristics for readability. The minimal diff is often not the most readable one — moving a hunk boundary by a line can make a change obviously a function insertion rather than a scattered set of edits. **Git’s `--patience` and `--histogram` algorithms trade minimality for comprehensibility**, which is a nice reminder that the mathematically optimal answer is not always the product requirement.”

The model answer

“You are building a plagiarism checker for student submissions. Given two documents, report how similar they are and show which parts overlap.”

“Let me separate the two things asked for, because they need different outputs from the same computation: **a similarity number, and the overlapping parts themselves.**

The core is longest common subsequence, and I would run it on tokens rather than characters. Splitting into words — lowercased, punctuation stripped — means a changed comma does not break a match, and it cuts the input size by roughly a factor of five, which matters because the algorithm is quadratic.

The state: $dp[i][j]$ **is the length of the longest common token subsequence of the first i tokens of the first document and the first j of the second.** Prefix lengths, so $i = 0$ is the empty prefix and the base row and column are zeros.

Match: $dp[i-1][j-1] + 1$. **Mismatch:** $\max(dp[i-1][j], dp[i][j-1])$. Answer at $dp[n][m]$.

The similarity score. I would report $2 \times \text{LCS} / (n + m)$ — the shared fraction of the total material — rather than LCS / n , because dividing by one document’s length is asymmetric and a short quote inside a long essay would score very differently depending on which way round you asked. **And the minimum number of edits to make them identical is $n + m - 2 \times \text{LCS}$,** which is the same information stated as a distance.

For showing the overlaps, I need the walk-back, so I need the full table — that rules out the two-row version, and I would say that explicitly as the reason.

Now the sizing, which is where the real design decision is. Two thousand-word essays is a million cells — fine, a fraction of a second. **Two ten-thousand-word dissertations is a hundred million cells, about forty seconds and hundreds of megabytes —** and a plagiarism checker compares each submission against every other one. **Thirty students is 435 pairs,** and at forty seconds each that is five hours.

So the real system needs a cheap filter before the expensive comparison. I would fingerprint each document — hash overlapping k-grams of tokens and keep a sample, which is winnowing — and **only run the full LCS on pairs whose fingerprint sets overlap enough to be worth it.** That turns 435 quadratic comparisons into 435 cheap set intersections plus a handful of expensive ones.

And I would raise a subtlety about what LCS actually measures here, because it matters for the product. **LCS is order-sensitive and gap-tolerant,** which means it catches a copied passage with words changed throughout — which is

what paraphrasing looks like, and is the case you most want to catch. **It does not catch reordered paragraphs**, because a swapped block breaks the order the algorithm depends on.

For that I would run LCS per paragraph pair rather than on the whole document, which detects a matched paragraph wherever it moved to, at the cost of missing overlaps that straddle a paragraph boundary. **That is a real trade and I would want the product to decide it**, not the algorithm.

What I would not do is report the score alone. Every one of these systems is wrong sometimes — shared quotations, standard phrasing, boilerplate assignment text — so **the output has to be the highlighted overlapping passages for a human to judge**, which is exactly what the reconstruction gives me and is the reason I kept the full table.”

9 Recall card

The first two-dimensional state: $dp[i][j]$ = LCS of **a**’s first **i** characters and **b**’s first **j**. Prefix lengths, not indices — so $i = 0$ is the empty prefix and the zero row and column are the base cases, free from the allocation. **The price is one off-by-one: the character is $a[i-1]$** , and that is where every bug lives.

Match $\rightarrow dp[i-1][j-1] + 1$ (the diagonal). **Mismatch** $\rightarrow \max(dp[i-1][j], dp[i][j-1])$ — skip a character of **a** or of **b**, because you cannot tell which is out. **The answer is $dp[n][m]$** , genuinely the last cell, unlike LIS.

$O(n \times m)$ time and space; two rows gives $O(\min(n, m))$ after swapping so the shorter string is inner — $10,000 \times 10$ becomes 22 cells instead of 20,002. **The collapse gives up reconstruction** (Hirschberg’s algorithm recovers it in linear space).

Reconstruction walks back from $dp[n][m]$: match $\rightarrow$ take the character, move diagonally; else move to the larger neighbour. $O(n + m)$. **The LCS is not unique** — the tie-break picks which one.

Substring differs on exactly two lines: mismatch writes 0 (contiguity broken) and the answer is the **table maximum**, not the last cell. `"abcdef" / "abzdef"` $\rightarrow$ subsequence 5, substring 3.

LCS is diff: in the LCS = unchanged, unmatched in **a** = deletions, unmatched in **b** = insertions. **Git uses Myers**, $O((n+m) \times D)$, which gets faster the more similar the files are — $125\times$ on two 5,000-line files differing in 20. **Derived one-liners**:

supersequence $n+m-LCS$; min deletions $n+m-2 \cdot LCS$; longest palindromic subsequence $LCS(s, reversed(s))$.

Design a news feed

1 What this is, and why they ask it

A news feed is the home screen of every social product: **you open the app and see recent posts from the people you follow, newest first.**

They ask it because **it is the canonical read-heavy fan-out problem**, and it has exactly one central decision with two defensible answers. **Do you build each person's feed when they write, or when someone reads?** Push or pull. Fan-out on write or fan-out on read. **Almost the entire interview is that question and its consequences.**

And the reason it is a genuine decision rather than a lookup is that **both answers break, in opposite directions.** Fan-out on write gives instant reads and collapses when someone with fifty million followers posts — one write becomes fifty million. Fan-out on read scales beautifully for writers and makes every read query hundreds of timelines and merge them. **The real answer is a hybrid, and being able to say exactly where the boundary sits, and why, is what is being tested.**

The other reason is the read-to-write ratio, which is more extreme here than anywhere else in this course. **A social product sees something like a hundred reads per write.** That single number justifies doing expensive work at write time, and a candidate who computes it before choosing is answering the question the right way round.

By the end of this lesson you can choose and defend the fan-out strategy, handle the celebrity problem, design the cache layout and the ranking pipeline, handle pagination correctly, and size the whole thing.

2 The story

The noticeboard outside the milk booth had been there longer than anyone could remember, and there was a system, of sorts, and it had a problem.

The system was that if you wanted people to know something, you wrote it and pinned it up. The tuition class, the lost cat, the plumber's number, the flat for rent on the second floor of C block.

And people came past in the morning and read it.

The trouble was that it was one board and there were nine buildings.

So a woman called Prameela, who ran the tuition class and therefore cared more than most, started doing something extra. **When she pinned a notice on the main board, she also wrote out four more copies and pinned one at each of the four buildings furthest from the booth.**

It was more work for her. Twenty minutes instead of two.

And it meant that everyone in those buildings saw it without walking anywhere. They looked at their own board, which was ten steps from their door, and everything relevant was already on it.

That worked, and within a year most people were doing it, and the boards were full and current and nobody had to walk to the milk booth to find out anything.

Then the temple committee started putting up notices.

Because the temple concerned everybody — all nine buildings, about four hundred families — **one notice meant writing out nine copies and walking to nine boards,** and the man who did it, who was seventy-one, took an entire morning over it. And the temple put up something most weeks.

So they stopped. The committee's notices went on the main board only, and there was a line at the top of every other board that said, in Prameela's handwriting, **"for temple notices see the main board."**

Which meant the temple's readers walked, and everybody else's did not.

And it was the right answer, and nobody had planned it. The people with four readers copied their notices outward, because it was cheap. **The people with four hundred readers did not, because it was not,** and their readers accepted one extra walk.

3 The idea in plain English

Prameela invented fan-out on write, the temple committee forced the hybrid, and the line at the top of every board is the merge step.

First the numbers that make this a design problem at all.

A social feed is read far more than it is written — something like a hundred to one. People open the app constantly and post rarely. **So it is worth doing a lot of work at write time to make reads cheap,** and that asymmetry is the first thing to say.

Now the central decision, and there are two ways to build a feed.

Fan-out on write, also called push. When you post, the system immediately writes your post’s id into the precomputed feed of every one of your followers. **Reading a feed is then one lookup of an already-assembled list** — a few milliseconds, no joins, no merging.

That is Prameela copying her notice to four boards. Expensive when she writes, free when anyone reads.

Fan-out on read, also called pull. Nothing happens at write time beyond storing the post. When you open the app, the system looks up everyone you follow, fetches their recent posts, merges them by time, and returns the top few.

That is walking to the milk booth. Free to write, expensive to read, and the expense is paid by every reader every time.

The comparison, stated plainly:

	fan-out on WRITE	fan-out on READ
write cost	0(followers)	0(1)
read cost	0(1)	0(following) queries + merge
read latency	~5 ms	~200-500 ms
storage	one entry per follower	none extra
a celebrity posts	50,000,000 writes	nothing
you follow 5,000	nothing special	5,000 queries per refresh

Given a hundred-to-one read-to-write ratio, fan-out on write is right for almost everybody — you pay once per post to make a hundred reads free.

And it breaks completely on one input: the account with tens of millions of followers.

One post becoming fifty million writes is the celebrity problem, and the numbers are brutal. At fifty thousand writes a second sustained, fifty million writes takes about seventeen minutes — **so the last follower sees the post seventeen minutes after the first**, and during those seventeen minutes the write pipeline is doing nothing else.

The fix is the hybrid, and it is exactly what the noticeboards converged on.

- **Normal accounts — say under ten thousand followers — fan out on write.** Their posts land in followers’ precomputed feeds immediately.
- **Celebrity accounts fan out on read.** Their posts are stored once and nothing is pushed.

- When you open the app, you read your precomputed feed and separately fetch recent posts from the handful of celebrities you follow, then merge the two by time.

That merge is Prameela's line at the top of the board, and it is cheap because you follow at most a few dozen celebrities, not a few thousand.

The threshold is a tuning parameter, not a principle. Ten thousand is a common choice. What matters is being able to say why there is a threshold at all: below it, `0(followers)` writes are cheaper than making every reader merge; above it, they are not.

Now the storage, which follows from the choice.

The precomputed feed is a list of post ids, not post contents. Storing the content in every follower's feed would multiply the data by the follower count — a 1 KB post with a thousand followers becomes a megabyte. Storing ids is 8 bytes each, and the contents are fetched once from a shared store and cached.

And the feed is capped. Keep the most recent 500 or 1,000 ids per user and discard the rest. Nobody scrolls past a few hundred posts, and an uncapped feed grows forever. If someone does scroll past the cap, fall back to the read-path query for the older material.

Redis sorted sets are the natural structure: score is the timestamp, member is the post id, and `ZREVRANGE` gives the newest `n` in one call, with `ZREMRANGEBYRANK` doing the capping.

Then ranking, because a modern feed is not chronological.

Chronological is simple and honest and produces a worse feed, because the newest post is not the most interesting one. Ranked feeds score candidates by predicted engagement — how likely you are to like, comment or dwell — using features like the author's affinity with you, the post's age, its early engagement rate, and its type.

The pipeline is: candidates → features → score → filter → return. Typically a few thousand candidates are scored to produce twenty results, and it happens in under two hundred milliseconds.

And ranking makes pagination hard, which is the practical detail people miss. Offset-based pagination breaks on a changing feed: new posts arrive between page one and page two, everything shifts down, and the user sees a post they already saw.

Cursor-based pagination — “give me what comes after this post id” — is correct, and for a ranked feed the cursor must encode the ranking session so page two is scored consistently with page one.

Finally, the things that go wrong at the edges.

Deleting a post means removing it from every feed it was pushed to — expensive, so most systems **filter at read time instead**: check the post still exists and is visible when hydrating, and let the stale id sit in the feed harmlessly.

Unfollowing has the same shape: rather than scrubbing every pushed entry, filter at read.

A new user’s feed is empty, and an empty home screen is the worst possible first impression. **Seed it** with popular posts from their stated interests or their location until they follow enough people.

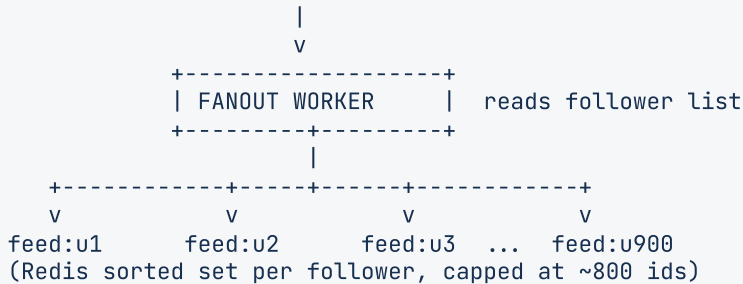
And the read path always ends with hydration: the feed gives you ids, and you fetch the actual posts, authors and engagement counts in one batched call. **That batch is the difference between one round trip and five hundred.**

4 The picture

The two strategies:

FAN-OUT ON WRITE (push)

user posts ---> [post store]



READ: ZREVRANGE feed:u1 0 19 → 20 ids → hydrate → done. ~5 ms.

FAN-OUT ON READ (pull)

user posts ---> [post store] and nothing else happens.

READ: get the 500 accounts u1 follows
→ 500 queries for recent posts
→ merge by timestamp
→ take the top 20
→ hydrate
~200-500 ms, every single time.

The hybrid, which is what actually gets built:



The merge is cheap because you follow ~30 celebrities, not ~3,000.

The celebrity problem, quantified:

account with 50,000,000 followers posts once

fan-out on write:

50,000,000 sorted-set inserts
at 50,000 writes/second sustained
= 1,000 seconds = ~17 MINUTES

- the first follower sees it instantly
- the last follower sees it 17 minutes later
- and for those 17 minutes the fanout pipeline does nothing else

fan-out on read for that one account:

1 write. Readers merge in ~5 ms extra.

The feed entry, and why it holds ids:

WRONG: store the post CONTENT in every feed

post = 1 KB
1,000 followers
→ 1 MB written per post
→ 1 billion posts/day x 1 KB x 1,000 = 1 PB/day

RIGHT: store the post ID

8 bytes (post id) + 8 bytes (score) = 16 bytes
1,000 followers → 16 KB per post
→ 64x less, and the content is stored ONCE and cached

The read path, end to end:

`follows` needs an index in both directions. `followee_id → followers` for fan-out; `follower_id → following` for the read path and the celebrity merge. **One index gives you half the system.**

And post ids are Snowflakes — [day 150](#)’s design — because a time-sortable id means the feed’s sort order is the id order, and “posts after this id” is a range query.

Fan-out on write

```
CELEBRITY_THRESHOLD = 10_000
FEED_CAP = 800

def on_post_created(post_id: int, author_id: int, created_at: float) → None:
    follower_count = follow_store.count_followers(author_id)
    if follower_count > CELEBRITY_THRESHOLD:
        return # celebrity: read path handles it

    for batch in follow_store.iter_followers(author_id, batch_size=1000):
        queue.publish("fanout", {"post_id": post_id, "at": created_at,
                                "targets": batch})
```

Each batch of a thousand becomes its own queue message, so the work parallelises and a crash costs one batch. This is exactly the notification fan-out shape from [day 152](#).

```
def handle_fanout_batch(msg: dict) → None:
    pipe = redis.pipeline()
    for user_id in msg["targets"]:
        key = f"feed:{user_id}"
        pipe.zadd(key, {msg["post_id"]: msg["at"]})
        pipe.zremrangebyrank(key, 0, -(FEED_CAP + 1)) # keep newest 800
    pipe.execute()
```

`zremrangebyrank(key, 0, -801)` is the cap, removing everything except the newest 800. Without it, a feed grows forever — a user following active accounts for five years accumulates millions of ids they will never scroll to.

The pipeline batches the round trips: a thousand users is two thousand commands in one network call, not two thousand calls.

The read path

```
def get_feed(user_id: int, cursor: str | None, limit: int = 20) → list[dict]:
    own = redis.zrevrange(f"feed:{user_id}", 0, limit * 3 - 1, withscores=True)

    celebrities = follow_store.celebrities_followed(user_id)      # ~30, cached
    extra = []
    for celeb_id in celebrities:
        extra.extend(post_store.recent(celeb_id, limit=limit))

    merged = merge_by_score(own, extra)
    ranked = rank(user_id, merged)[:limit]
    return hydrate(ranked)                                     # ONE batched fetch
```

limit * 3 over-fetches deliberately, because filtering will drop some — deleted posts, blocked authors, already-seen items — and a second round trip to top up is worse than fetching sixty and keeping twenty.

celebrities_followed is cached per user and changes rarely; recomputing it on every refresh would be a join on every read.

Hydration, which is where the latency actually goes

```
def hydrate(entries: list[tuple[int, float]]) → list[dict]:
    post_ids = [post_id for post_id, _ in entries]
    posts = post_cache.multi_get(post_ids)                # one call
    author_ids = {p["author_id"] for p in posts.values()}
    authors = user_cache.multi_get(list(author_ids))      # one call
    counts = counter_cache.multi_get(post_ids)           # one call
    return [build(posts[i], authors[posts[i]["author_id"]], counts[i])
            for i in post_ids if i in posts]
```

Three batched calls, not sixty individual ones. `if i in post_ids if i in posts` also silently drops deleted posts — **which is the read-time filter that saves you from scrubbing every feed on delete.**

Cursor pagination

```
import base64, json

def make_cursor(last_score: float, last_id: int, session: str) → str:
    return base64.urlsafe_b64encode(
        json.dumps({"s": last_score, "i": last_id, "sess": session}).encode()).decode()

def parse_cursor(cursor: str) → dict:
    return json.loads(base64.urlsafe_b64decode(cursor))
```

The cursor carries the score and the id of the last item returned, so page two is “everything below this score” rather than “skip twenty” — correct even when new posts arrive in between.

And `sess` is the ranking session, which matters for a ranked feed: without it, page two is scored with a fresher model or fresher engagement counts and can contain items that were on page one.

Ranking

```
def rank(user_id: int, candidates: list) → list:
    features = feature_store.batch(user_id, [c.post_id for c in candidates])
    scores = model.predict(features)          # p(engage), from a trained model
    for candidate, score in zip(candidates, scores):
        age_hours = (time.time() - candidate.created_at) / 3600
        candidate.final = score / ((age_hours + 2) ** 1.5)  # time decay
    return sorted(candidates, key=lambda c: -c.final)
```

The time decay is the part worth explaining: a raw engagement score would let a two-day-old viral post outrank everything new forever. Dividing by a power of age keeps the feed fresh, and the exponent is the knob between “recent” and “good”.

`+2` in the denominator stops brand-new posts from being divided by nearly zero and dominating absolutely.

The real systems

Twitter/X	hybrid; the original "Timelines at Scale" talk is the reference for this whole design
Facebook	heavily ranked, pull-leaning with aggressive caching
Instagram	fan-out on write into Cassandra, ranked at read time
LinkedIn	hybrid, with Kafka driving the fanout
Redis	sorted sets are the standard feed store everywhere

Naming Twitter’s hybrid explicitly is expected, because it is the design the question is drawn from.

6 The numbers

The ratio that justifies everything.

```
300 million daily active users
each opens the app ~10 times a day      = 3,000,000,000 feed reads/day
each posts ~0.1 times a day             = 30,000,000 posts/day

read : write = 100 : 1
```

A hundred reads per write is why you do expensive work at write time, and it is the first number to compute.

```
3,000,000,000 / 86,400 = ~35,000 feed reads/second average
peak ~3x                = ~100,000 reads/second

30,000,000 / 86,400    = ~350 posts/second average
peak                   = ~1,000 posts/second
```

Fan-out volume.

```
350 posts/second x average 200 followers = 70,000 feed inserts/second
peak: 1,000 x 200                        = 200,000 inserts/second
```

Redis: ~100,000 ops/second per instance
→ 2-4 instances at average, ~10 at peak, sharded by user id

And the celebrity case, which is why the average is misleading:

```
one account with 50,000,000 followers

fan-out on write:
  50,000,000 inserts at 50,000/s = 1,000 s = ~17 minutes
  and the whole pipeline is blocked on it

with a 10,000 threshold:
  ~0.1% of accounts are above it
  but they produce a large share of the posts people read
  → those posts cost 1 write each and ~5 ms extra at read time
```

Storage for the feeds.

```
300,000,000 users
800 entries each
16 bytes per entry (8-byte post id + 8-byte score)

300,000,000 x 800 x 16 B = 3.84 TB
+ Redis overhead (~2x for sorted sets) = ~8 TB

→ ~80 Redis instances of 100 GB, or fewer larger ones
   at ~$0.05/GB/hour for managed Redis: ~$25,000-30,000/month
```

And the comparison that justifies storing ids:

```
storing post CONTENT in each feed instead (1 KB average):
  300,000,000 x 800 x 1 KB = 240 TB
  → 30x more, and every edit to a post would need 800 updates

storing ids: 8 TB, content stored once and cached.
```

Read latency, broken down.

```
fan-out on write path:
  ZREVRANGE own feed           ~1 ms
  fetch ~30 celebrities' recent posts ~3 ms
  merge + rank 60 candidates    ~2 ms
  filter                        ~1 ms
  hydrate (3 batched multi-gets) ~5 ms
                                -----
                                ~12 ms

fan-out on read path (if you followed 500 people):
  500 queries for recent posts   ~150 ms (batched, parallel)
  merge 500 x 20 = 10,000 candidates ~20 ms
  rank + hydrate                 ~10 ms
                                -----
                                ~180 ms, EVERY read
```

Fifteen times slower on every one of a hundred reads per write. That comparison is the argument.

Hydration, and why batching is not optional:

```
20 posts, hydrated individually
  20 posts + 20 authors + 20 counters = 60 round trips
  at 0.5 ms each                      = 30 ms

batched into 3 multi-gets
  3 round trips at ~1.5 ms each       = 4.5 ms

~7x faster, and it is one line of code.
```

Cache sizing for post content.

```
30,000,000 posts/day, and reads are heavily skewed to recent posts
  ~90% of reads hit posts from the last 48 hours

48 hours of posts = 60,000,000 posts x 1 KB = 60 GB
→ one 100 GB cache covers ~90% of all reads

at a 90% hit rate:
  100,000 reads/s x 20 posts = 2,000,000 post fetches/second
  10% miss = 200,000 database reads/second ← still a lot
  → a second cache layer, or a higher hit rate, matters enormously here
```

That last line is worth saying: at this scale, moving the hit rate from 90% to 95% halves the database load.

7 The trade-offs

Fan-out on write against on read, which is the whole design. Write costs `O(followers)` and makes reads a single lookup at about 12 ms. Read costs nothing to write and makes every read query hundreds of timelines at about 180 ms. **At a hundred-to-one read ratio, write wins decisively** — and it fails completely on the celebrity, where one post becomes fifty million writes and seventeen minutes.

So: hybrid, and the threshold is a tuning parameter rather than a truth. Below it, `O(followers)` writes are cheaper than making every reader merge; above it, they are not. **Ten thousand is conventional; the number matters less than being able to derive it.**

Chronological against ranked. Chronological is simple, predictable, explicable to users, and produces a worse feed — the newest post is rarely the best one. **Ranked improves engagement measurably and costs a feature store, a model, a training pipeline, and pagination that is much harder to get right.** It also makes the product harder to reason about, for users and for you.

Precomputed feeds are stale by design. A push-based feed reflects the world as of when the fan-out ran, so a post from thirty seconds ago may not be there yet. **For a social feed nobody notices; for anything where completeness matters it would be unacceptable,** and that is worth stating rather than glossing.

Deleting is expensive to do properly and cheap to fake. Removing a post from every feed it was pushed to is `O(followers)` again. **Filtering at read time is free — the hydration already drops missing posts** — at the cost of the id lingering in millions of sorted sets. **The stale ids are harmless and they do consume the feed cap,** so a user who follows someone who deletes constantly gets a slightly shorter effective feed.

The feed cap trades completeness for bounded storage. Eight hundred entries is 8 TB across the user base; uncapped is unbounded. **A user who scrolls past the cap falls back to the slow read path** — which is correct, because that is a rare action and paying 180 ms for it is fine.

When would I not build this? When the follower graph is small and uniform — a team chat, a classroom app — where a single query with a join is the whole feed and fan-out infrastructure is pure overhead. **When the feed is not personalised,** like a news site's front page, where one cached page serves everybody and none of this applies. **And below roughly a million users, where fan-out on read with good caching is simpler, correct, and fast enough** — building the hybrid first is the classic premature-scaling mistake here.

8 In the interview

How it gets asked

- “*Design a news feed / Twitter timeline / Facebook feed.*” — the standard prompt.
- “*Push or pull?*” — the central question, sometimes asked in exactly those words.
- “*A celebrity with fifty million followers posts. What happens?*”
- “*How do you paginate?*” — the detail that separates people.
- “*How do you rank it?*”
- “*What happens when a post is deleted?*”

“Let me compute one number first, because it decides the architecture.

Three hundred million daily users opening the app ten times a day is three billion feed reads. They post about a tenth of a time a day, so thirty million posts. That is a hundred reads per write.

So it is worth doing a lot of work at write time to make reads cheap, and that points at fan-out on write.

Fan-out on write means: when you post, I immediately insert your post id into the precomputed feed of every follower. Reading is then one lookup of an assembled list — about five milliseconds, no joins, no merging.

The alternative, fan-out on read, does nothing at write time and, when you open the app, queries everyone you follow, merges by time and returns the top twenty. For someone following five hundred accounts that is five hundred queries and a merge — **about 180 milliseconds, on every one of those hundred reads.**

Fifteen times slower, a hundred times as often. Write wins.

And it fails completely on one input. An account with fifty million followers posting once becomes fifty million inserts. At fifty thousand writes a second that is **seventeen minutes**, during which the last follower has not seen the post and the pipeline is doing nothing else.

So: hybrid. Accounts under about ten thousand followers fan out on write. Above that, the post is just stored. **At read time I take my precomputed feed and separately fetch recent posts from the roughly thirty celebrities I follow, and merge.** That merge is cheap precisely because you follow thirty celebrities and not three thousand.

Storage: the feed holds post ids, not content. Sixteen bytes an entry, capped at eight hundred entries per user — three hundred million users is about eight terabytes of Redis. **Storing the content instead would be 240 terabytes and would mean eight hundred updates every time someone edits a post.**

Redis sorted sets, scored by timestamp, with `ZREVRANGE` for the read and `ZREMRANGEBYRANK` for the cap.

Two questions before I go further: is the feed chronological or ranked? Because ranking changes the read path and makes pagination substantially harder. **And what is the follower distribution?** Because the whole hybrid exists

to handle its tail.”

The follow-ups

“A celebrity with fifty million followers posts. Walk me through it.”

“Under pure fan-out on write, that one action becomes fifty million sorted-set inserts.

At about fifty thousand writes a second sustained, that is a thousand seconds — roughly seventeen minutes. So the first follower sees it immediately and the last sees it seventeen minutes later, **and for those seventeen minutes the fan-out pipeline is doing nothing else** — every ordinary user’s posts are queued behind it.

And it is worse than a single spike, because these accounts post repeatedly, and a few of them posting in the same minute saturates the pipeline for an hour.

So celebrity accounts are excluded from fan-out entirely. Above a threshold — ten thousand followers, say — a post is written once to the post store and nothing is pushed.

The cost moves to the read path, and it is small. When I open the app, I read my precomputed feed and **separately fetch recent posts from the celebrities I follow — typically twenty or thirty accounts** — then merge by score and take the top twenty. That is a handful of extra queries, cached, adding maybe three milliseconds.

The asymmetry is what makes it work: the number of celebrities I follow is small even though the number of people who follow them is enormous.

Two refinements I would mention. The celebrity list per user should be cached, because recomputing ‘which of the people I follow are above the threshold’ on every refresh is a join on the read path.

And the threshold does not have to be a hard cutoff. An account with fifteen thousand followers could fan out to its most active followers — the ones who opened the app today — and leave the rest to the read path. **That gets instant delivery for the people who will actually see it soon, and bounds the write cost.**

The number itself is a tuning parameter, not a principle. What matters is the reason there is one: **below it, `0(followers)` writes are cheaper than making every reader merge; above it, they are not.**“

“How do you paginate the feed?”

“Not with offsets, and this is the detail that quietly ruins feeds.

Offset pagination breaks on a changing list. I ask for offset zero, limit five and get posts ten through six. While I am reading them, three new posts arrive. I ask for offset five, limit five — **but the list has shifted down by three, so I get posts eight through four, and posts eight, seven and six are shown to me again.**

Duplicates on page two, every time the feed is active — and the more active the feed, the worse it is, which means it is worst for exactly the users who use the product most.

Cursor pagination fixes it. Instead of ‘skip five’, the request says ‘give me what comes after this point’, where the cursor encodes the score and id of the last item I returned. **New posts arriving above the cursor do not affect it at all**, because I am asking for things below a fixed position rather than at a fixed offset.

The cursor needs both the score and the id, not just the score, because two posts can share a timestamp and the id breaks the tie deterministically.

And for a ranked feed there is a further problem. Scores change — engagement counts move, the model gets retrained — so a post that scored low on page one might score higher by the time I ask for page two, and appear again. **So the cursor must also carry a ranking session id**, and page two is scored against the same snapshot as page one. **Without that, ranking reintroduces exactly the duplicates that cursors were meant to remove.**

Two practical points. The cursor should be opaque — base64 of a small JSON blob — so I can change what is inside it without breaking clients. **And I would over-fetch:** request sixty candidates to return twenty, because filtering will drop deleted posts, blocked authors and already-seen items, and a second round trip to top up is worse than fetching three times as much.”

“What happens when someone deletes a post, or unfollows someone?”

“Both are the same shape of problem, and the answer to both is the same: **do not fix the feeds, filter at read.**

Deleting properly means removing the post id from every feed it was pushed to — which is `O(followers)` again, the same cost as the original fan-out. For a post with a million followers that is a million deletes, and a user expects a delete to take effect immediately, not in twenty minutes.

So instead: delete the post from the post store, and leave the ids in the feeds. At read time, hydration fetches the posts by id, and a deleted post simply is not returned — **the filtering already happens as a side effect of the batched fetch.** The id lingers in millions of sorted sets, harmlessly.

What it costs is the feed cap. A stale id occupies one of the eight hundred slots, so a user who follows someone who deletes a lot has a slightly shorter effective feed. **In practice that is invisible**, and a background job can compact feeds lazily if it ever matters.

Unfollowing is the same. Rather than scrubbing every entry that user contributed, keep the follow edge’s removal authoritative and let the read path filter — though in practice, unfollow is rarer and the feed refreshes past the old entries within a day anyway.

Blocking is the one case where I would not rely on lazy filtering alone, because the requirement is stronger: a blocked user’s content must not appear, and ‘it will age out’ is not an acceptable answer. **Filter at read against the block list**, which must be checked on every hydration rather than treated as a cache-warming nicety.

The general principle is worth naming: in a fan-out-on-write system, writes are expensive and reads are cheap, so push corrections to the read path. Anything that would require touching many feeds should instead become a filter applied to the twenty items actually being returned.”

The model answer

“Design the feed for a photo-sharing app: two hundred million daily users, an average of three hundred follows each, some accounts with tens of millions of followers, and the feed should be ranked rather than chronological.”

“Three things in that prompt drive the design: **two hundred million daily users, a heavy follower tail, and ranked rather than chronological.** Let me size it first.

Two hundred million users opening the app maybe eight times a day is 1.6 billion feed reads, about 18,000 a second average and 55,000 at peak. Photo apps post more than text apps — say 0.3 posts per user per day — so sixty million posts, about 700 a second. **That is still a 25-to-1 read ratio, which justifies fan-out on write.**

Fan-out volume: 700 posts a second times 300 average followers is 210,000 feed inserts a second, peaking around 600,000. Redis does about 100,000 operations a second per instance, so **six to ten instances sharded by user id,** with pipelining so a thousand-follower batch is one network call rather than a thousand.

Hybrid at a ten-thousand-follower threshold, for the reason the prompt hints at: an account with fifty million followers would take seventeen minutes to fan out and block everything else. **Those accounts store once; readers merge in the twenty or thirty celebrity accounts they follow.**

Feed storage: post ids in Redis sorted sets, capped at 800. Two hundred million users at 800 entries of 16 bytes is about 2.5 TB, roughly 5 TB with sorted-set overhead. **Storing the images or even the post content there would be absurd** — the media lives in object storage behind a CDN and the feed holds ids.

Now ranking, which the prompt asks for and which changes the read path.

Candidates → features → score → filter → return. I over-fetch: take about two hundred candidates from the precomputed feed plus the celebrity merge, score them, return twenty. **Features are author affinity — how much this user interacts with this author — post age, early engagement rate, and media type.**

Time decay is the part I would be explicit about. A raw predicted-engagement score would let a viral two-day-old photo outrank everything new forever, so the score is divided by a power of age. **The exponent is the product’s dial between ‘recent’ and ‘good’, and I would want it configurable rather than compiled in.**

Ranking’s real cost is pagination. Cursor-based, and **the cursor must carry a ranking session id** so page two is scored against the same snapshot as page one — otherwise a post whose engagement rose between requests appears on both

pages. **That is the failure mode people miss, and it is worse for the most active users.**

The read path end to end: read my feed, merge celebrities, rank, filter for deleted and blocked, then one batched hydration. Three multi-gets — posts, authors, counters — not sixty round trips. **That batching is the difference between about 5 milliseconds and 30 on the same work,** and it is one line.

Media, which is specific to a photo app. The feed returns image URLs pointing at a CDN, with several pre-generated sizes so a phone on a slow connection fetches a thumbnail and not a four-megabyte original. **The CDN is doing the heavy lifting here — the feed API is small JSON, and the bytes are all images.**

Two things I would raise as risks. A precomputed feed is stale by design — a post from thirty seconds ago may not have fanned out yet — which for a photo feed nobody notices, and I would say it out loud rather than let it be discovered. **And a new user's feed is empty,** which is the worst possible first impression, so it needs seeding from popular content in their interests until they follow enough accounts.

And the thing I would leave for version two: the partial fan-out refinement, where accounts just above the threshold push only to followers who were active today. **It is a real improvement and it adds a liveness signal to the write path,** and I would not put that complexity in before measuring whether the hard threshold actually hurts.”

9 Recall card

Compute the ratio first: ~100 reads per write. That is what justifies expensive writes. **Fan-out on write (push)** = `O(followers)` per post, ~12 ms reads. **Fan-out on read (pull)** = free writes, ~180 ms reads querying hundreds of timelines. **15× slower, 100× as often — write wins.**

And write breaks on the celebrity: 50M followers = 50M inserts = ~17 minutes, with the pipeline blocked throughout. **So: hybrid.** Under ~10,000 followers → fan out on write; above → store only, and **readers merge in the ~30 celebrities they follow.** The asymmetry is the point: few celebrities followed, many followers each. **The threshold is a tuning parameter — be able to derive it, not recite it.**

Feeds hold post IDs, never content — 16 bytes vs 1 KB, and content is stored once (8 TB vs 240 TB at 300M users). **Redis sorted sets**, `ZREVRANGE` to read, `ZREMRANGEBYRANK` to cap at ~800 — uncapped feeds grow forever.

Cursor pagination, never offsets — new posts shift an offset and page two repeats page one's items. The cursor carries **score + id**, and for a ranked feed **a ranking session id too**, or changing scores reintroduce duplicates.

Hydrate in 3 batched multi-gets, not 60 round trips (~5 ms vs ~30). **Deletes and unfollows are filtered at read**, not scrubbed from millions of feeds — the batched fetch drops missing posts for free. **Ranking = candidates → features → score → filter**, with **time decay** (`score / (age+2)^1.5`) so a viral old post does not outrank everything new.

DSA topic: Longest common subsequence **System design topic:** Design a news feed

Code these, in this order

One rule for the whole set: write `dp[i][j]` uses `a[i-1]` and `b[j-1]` as a comment at the top before you write the loops. Every problem here is a two-dimensional prefix-length table, and the off-by-one is the only thing that will cost you time.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Longest Common Subsequence	LeetCode 1143 (Medium)	The two-dimensional state and both branches.
2	Longest Palindromic Subsequence	LeetCode 516 (Medium)	The <code>LCS(s, reversed(s))</code> reduction.
3	Delete Operation for Two Strings	LeetCode 583 (Medium)	<code>n + m - 2 * LCS</code> , in one line.
4	Shortest Common Supersequence	LeetCode 1092 (Hard)	Reconstruction, not just the length.
5	Uncrossed Lines	LeetCode 1035 (Medium)	LCS with the strings replaced by number arrays.
6	Longest Common Substring	classic — write it yourself	The two lines that differ, and where the answer lives.

On problem 1, write the off-by-one wrong on purpose

Use `a[i]` and `b[j]` instead of `a[i-1]` and `b[j-1]`. Run on `"abcde"` and `"ace"` and record what happens. Then try it on two strings where it does *not* raise. Say why the version that raises is the lucky case.

On problem 1, loop from 0

Change both loops to start at 0 and see what you have to add to make it work. Then say in one sentence what starting at 1 buys you.

On problem 1, print the table

Print the whole `dp` grid for `"abcde"` / `"ace"`. Read one interior cell out loud as a full sentence containing “the first `i` characters”. Then check that sentence is true by hand.

On problem 1, collapse to two rows and then try to reconstruct

Write the two-row version and confirm it agrees. Then attempt to walk back through it. **Say what stops you, in one sentence**, and say what you would use if you needed both linear space and the string.

On problem 4, reconstruct rather than count

You need the actual supersequence, so you need the full table. Walk back and emit: on a match, one character; on a mismatch, the character from whichever side you move from. **Check your output contains both inputs as subsequences.**

On problem 6, find both differences

Start from your LCS code and change exactly two things. Run both on "abcdef" and "abzdef" and record both answers. **Name the two changes and say which one is the sneaky bug.**

Then the diff drill

Implement diff on two lists of lines using your reconstruction. Run it on two versions of a real file of yours. Then compute how many cells the table has for two 5,000-line files, and say what Git does instead and why it gets faster as the files get more similar.

The state drill

1. State it as a full sentence.
2. Say why prefix lengths and not indices.
3. Say what `i = 0` means and what it buys.
4. Say which character `i` refers to, and write it down.

The recurrence drill

1. Give the match case and say what it means in words.
2. Give the mismatch case and say why there are two options.
3. Say why you cannot tell which to skip.
4. Say where the answer is, and why it is the last cell here but not in LIS.

The reconstruction drill

1. Say where the walk-back starts and where it stops.
2. Give the three moves and what each means.
3. Say what the tie-break decides.

4. Say what it costs in time and in space.
5. Name the algorithm for linear-space reconstruction.

The substring drill

1. Give both answers for "abcdef" / "abzdef" .
2. Say what the mismatch case becomes and why.
3. Say where the answer lives and why.
4. Say which of the two changes is easier to forget.
5. Say what better algorithm exists for the substring version.

The family drill

Give each as a one-liner and say why it works:

1. Shortest common supersequence length.
2. Minimum deletions to make two strings equal.
3. Longest palindromic subsequence.
4. Minimum insertions to make a string a palindrome.

The break-it drill

Trigger each and record the exact output or error:

1. `a[i]` instead of `a[i-1]` .
2. Loops starting at 0 without a guard.
3. Substring code returning `dp[n][m]` .
4. Subsequence code returning the table maximum.
5. Reconstruction after the two-row collapse.
6. A table for two 20,000-character strings.
7. An empty string as one input.

Four of the seven give no error at all. Name them.

The fan-out drill

1. Compute the read-to-write ratio and say what it justifies.
2. Give both strategies with their write cost, read cost and read latency.
3. Say which wins and why, in one sentence with numbers.
4. Say exactly where it breaks.

The celebrity drill

1. Do the 50-million-follower arithmetic.
2. Say what happens to everyone else's posts during those minutes.
3. Describe the hybrid in three sentences.
4. Say why the read-side merge is cheap, using the asymmetry.
5. Say what the threshold is and what kind of number it is.
6. Describe the partial-fanout refinement.

The storage drill

1. Say what a feed entry contains and what it deliberately does not.
2. Compute both totals at 300M users, ids versus content.
3. Say what the cap is and what happens without it.
4. Say what happens when a user scrolls past the cap.
5. Name the Redis structure and the two commands.

The pagination drill

1. Show how offset pagination produces duplicates, with post numbers.
2. Say what a cursor contains and why it needs both fields.
3. Say what ranking adds to the problem.
4. Say what the session id is for.
5. Say why the cursor should be opaque.
6. Say why you over-fetch.

The read-path drill

1. Give the six steps in order with rough timings.
2. Say what hydration is and why it must be batched.
3. Compute batched versus unbatched round trips.
4. Say what the cache hit rate does to database load at this scale.

The corrections drill

1. Say what deleting a post properly would cost.
2. Say what you do instead and why it is free.
3. Say what it costs, precisely.
4. Say why blocking is the exception.
5. State the general principle in one sentence.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the longest common subsequence of two strings.* The state as a sentence, why prefix lengths, both branches with their meanings, where the answer is, and the complexity with the space collapse and its cost.
2. *How is that different from longest common substring, and how would you write `diff`?* The two changed lines, where each answer lives, LCS as unchanged lines, and why Git uses Myers.
3. *Design a news feed.* The read-to-write ratio, both fan-out strategies with numbers, the celebrity arithmetic, the hybrid, ids not content, and cursor pagination.

Before you move on

- [] I can state the LCS state as a full sentence.
- [] I know why prefix lengths beat indices.
- [] I write `a[i-1]`, and I know why.
- [] I start my loops at 1 and know what that buys.
- [] I can give both branches and what each means in words.
- [] I know the answer is `dp[n][m]` here and not in LIS.
- [] I can reconstruct with the three moves.
- [] I know the tie-break decides which valid LCS I get.
- [] I can collapse to two rows, with the swap.
- [] I know the collapse gives up reconstruction, and can name Hirschberg.
- [] I can give both substring differences.
- [] I know the substring answer is the table maximum.
- [] I can give the four derived one-liners.
- [] I know LCS on lines is `diff`.
- [] I know what Myers does and why it gets faster on similar files.
- [] I can give time and space, and where the table stops being viable.
- [] I can compute the feed read-to-write ratio.
- [] I can give both fan-out strategies with their costs.
- [] I can do the 50-million-follower arithmetic.

- [] I can describe the hybrid and the read-side merge.
- [] I know why the merge is cheap, via the asymmetry.
- [] I know feeds store ids, and both storage totals.
- [] I know the cap and what happens past it.
- [] I can name the Redis structure and its two commands.
- [] I can show how offsets produce duplicates.
- [] I know what a cursor contains and why both fields.
- [] I know why ranking needs a session id in the cursor.
- [] I know hydration must be batched, with the numbers.
- [] I know deletes are filtered at read, and why blocking is the exception.
- [] I can say when I would not build any of this.
- [] I answered all three questions above out loud.

Edit distance

DATA STRUCTURES AND ALGORITHMS

Edit distance

SYSTEM DESIGN

Design Twitter

1 What this is, and why they ask it

How many single-character edits does it take to turn one string into another?

Insert a character, delete a character, or replace a character. Each counts as one.

"horse" to "ros" takes 3: replace `h` with `r`, delete `r`, delete `e`. **That number is the edit distance**, also called Levenshtein distance, and it is what spell-checkers, fuzzy search, DNA alignment and “did you mean” all compute.

They ask it because **it is yesterday’s table with three options instead of two**, and that step — from a binary choice to a three-way `min` — is where two-dimensional DP stops being a pattern to copy and starts being something you have to reason about. **Every one of the three options has to be named, and each maps onto a real operation**, and a candidate who can say “this branch is a deletion, and here is why the index moves that way” has understood it.

The other reason is that **the three operations are easy to mix up**. `dp[i-1][j]` is a deletion and `dp[i][j-1]` is an insertion — or is it the other way round? **It depends entirely on which string you are transforming into which**, and getting it backwards still gives the right *number*, because the distance is symmetric. **So the bug is invisible until you try to reconstruct the actual edits**, which is exactly what the follow-up asks for.

By the end of this lesson you can write the table and name all three branches, reconstruct the sequence of edits, collapse the space, handle weighted operations, and explain where it is used and where it is too slow.

2 The story

The chemist had been reading handwriting for thirty-one years and he was very good at it, and this one had him.

The paper said something like “**Ranitidin**”, or possibly “**Ranitidina**”, or — and this was the difficulty — it could have been the beginning of a completely different word with a flourish at the end.

What he did next was the thing he did without thinking about it, and had done thousands of times.

He looked at the shelf.

There was *Ranitidine*. There was *Ranolazine*. There was *Ramipril*. And there was *Ranitidine* in the other brand, spelled the same.

And he compared, letter by letter, and counted how far off it was.

Ranitidin against *Ranitidine* — **everything the same, one letter missing at the end.** One change.

Ranitidin against *Ranolazine* — R-a-n, then the paper had *i* and the box had *o*, then *t* against *l*, and by the time he got to the end there were four or five places where they did not agree. **Too far.**

Ranitidin against *Ramipril* — the *n* against the *m*, and then it fell apart completely.

So it was *Ranitidine*, and it took him about two seconds, and he had never once described what he was doing.

His nephew, who had come to help during the holidays and was seventeen, asked how he could be sure, and he said something that was more precise than he realised. **“Because to make the paper say *Ranolazine* I would have to change four things. To make it say *Ranitidine* I only have to add an e.”**

And then, because the nephew kept asking, he thought about it properly for the first time and added the part that mattered.

“And it is not just adding. Sometimes a letter is wrong and you swap it. Sometimes there is a letter that should not be there and you take it out. Three things you can do, and you count how many you need.”

The nephew wrote it down, which the chemist thought was very funny.

3 The idea in plain English

The chemist’s three things — swap a letter, add a letter, take one out — are the three operations, and his count is the edit distance.

The state is yesterday’s, unchanged:

`dp[i][j]` is the minimum number of edits to turn the first `i` characters of `a` into the first `j` characters of `b`.

Prefix lengths again, so $i = 0$ is the empty prefix and the character i refers to is $a[i-1]$. Same convention, same off-by-one, and the same reason for choosing it.

The base cases are different from LCS, and they are the more interesting ones.

$dp[0][j] = j$: turning an empty string into j characters takes j insertions.
 $dp[i][0] = i$: turning i characters into nothing takes i deletions.

So the first row is $0, 1, 2, 3, \dots$ and the first column is the same going down — not zeros. **This is the most common mistake in the problem**, because LCS's base cases are zeros and the muscle memory carries over.

Now the recurrence, and it has two cases.

If the characters match — $a[i-1] = b[j-1]$ — **no edit is needed at this position at all**. Just carry the cost forward diagonally:

```
dp[i][j] = dp[i-1][j-1]
```

No $+ 1$. That is the free move, and forgetting to make it free is the second most common bug.

If they do not match, you must do one of three things, and each costs one:

```
dp[i][j] = 1 + min( dp[i-1][j-1],    REPLACE a[i-1] with b[j-1]
                    dp[i-1][j],      DELETE  a[i-1]
                    dp[i][j-1] )    INSERT  b[j-1]
```

Naming each one and saying why the index moves that way is the whole lesson:

Replace. You fix $a[i-1]$ to become $b[j-1]$. **Both characters are now dealt with**, so both indices retreat — the diagonal.

Delete. You throw away $a[i-1]$. $b[j-1]$ **has not been handled yet**, so only i retreats: $dp[i-1][j]$.

Insert. You insert $b[j-1]$ into a . $a[i-1]$ **has not been handled yet**, so only j retreats: $dp[i][j-1]$.

Read those three out loud once and the direction stops being memorised. The index that moves is the string whose character you just consumed.

The answer is $dp[n][m]$, both strings fully transformed.

And the direction matters for reconstruction and not for the number. Edit distance is symmetric — turning a into b costs the same as b into a — **so swapping which is which gives the same distance with insertions and deletions exchanged.**

That is why getting the branches backwards passes every test that only checks the number.

Three more things worth having ready.

Reconstruction. Walk back from `dp[n][m]`. At each cell, ask which of the four possibilities produced this value — match, replace, delete, insert — and emit the corresponding operation. **The result is the actual edit script**, which is what a spell-checker or a diff tool needs.

The space collapse. Row `i` reads only row `i-1` and the cell to its left, so two rows suffice — `O(min(n, m))` after swapping. **And reconstruction is gone**, same as yesterday.

Weighted operations. In the real world the three operations do not cost the same. **A spell-checker weights a replacement between adjacent keyboard keys lower than between distant ones.** DNA alignment weights a substitution differently from an insertion, because biologically they are not equally likely. **The recurrence is unchanged; only the constants differ:**

```
dp[i][j] = min( dp[i-1][j-1] + cost_replace(a[i-1], b[j-1]),
                dp[i-1][j]   + cost_delete(a[i-1]),
                dp[i][j-1]   + cost_insert(b[j-1]) )
```

And one variant worth naming: Damerau-Levenshtein adds a fourth operation, transposition — swapping two adjacent characters. `"hte"` to `"the"` is **one** transposition rather than two replacements, and since transposition is one of the most common typing errors, spell-checkers use it.

Finally, where this is actually used and where it stops.

Spell-check and “did you mean” are the obvious ones. Fuzzy search — matching a query against a dictionary within distance 2. **DNA sequence alignment**, where the strings are millions of characters and the weights encode biology. **Plagiarism and diff tools**, where LCS and edit distance are two views of the same table.

And the limit is `O(n × m)`. Comparing a query against a hundred thousand dictionary words means a hundred thousand tables. **The practical fix is not a faster algorithm; it is not running the algorithm** — a BK-tree or a trie with pruning eliminates almost all candidates before any table is built, which is the answer to “how does a real spell-checker do this”.

4 The picture

The table for `a = "horse"`, `b = "ros"` :

	""	r	o	s	
""	0	1	2	3	← base: j insertions
h	1	1	2	3	
o	2	2	1	2	
r	3	2	2	2	
s	4	3	3	2	
e	5	4	4	3	
^				^	

base: i deletions answer = dp[5][3] = 3

NOT zeros in row 0 and column 0. That is the difference from LCS, and it is the most common mistake in this problem.

dp[2][2] = 1 means "turning 'ho' into 'ro' takes 1 edit" – replace the h with an r. Say it as a sentence and check it.

The three moves, drawn on one cell:

dp[i-1][j-1] ----- dp[i-1][j]

REPLACE (diagonal) | (match: free) | DELETE a[i-1]

↓ ↓ ↓

dp[i][j-1] -----> dp[i][j]

INSERT b[j-1]

MATCH dp[i][j] = dp[i-1][j-1] ← NO +1. Free.

REPLACE dp[i][j] = dp[i-1][j-1] + 1 both consumed → diagonal

DELETE dp[i][j] = dp[i-1][j] + 1 consumed a only → i moves

INSERT dp[i][j] = dp[i][j-1] + 1 consumed b only → j moves

RULE: the index that moves belongs to the string whose character you just used up.

The walk-back producing the actual edits:

```

"horse" → "ros", starting at dp[5][3] = 3

dp[5][3]=3  a[4]='e' b[2]='s'  differ
              dp[4][3]=2 is the min → DELETE 'e'          → dp[4][3]
dp[4][3]=2  a[3]='s' b[2]='s'  MATCH → free                → dp[3][2]
dp[3][2]=2  a[2]='r' b[1]='o'  differ
              dp[2][2]=1 is the min → DELETE 'r'          → dp[2][2]
dp[2][2]=1  a[1]='o' b[1]='o'  MATCH → free                → dp[1][1]
dp[1][1]=1  a[0]='h' b[0]='r'  differ
              dp[0][0]=0 is the min → REPLACE h→r         → dp[0][0]
dp[0][0]=0  STOP

reversed:  replace h→r, keep o, delete r, keep s, delete e
           3 edits. Which is what dp[5][3] said.

```

Why getting the branches backwards is invisible:

```

edit_distance("horse", "ros") = 3
edit_distance("ros", "horse") = 3

SYMMETRIC. So if you swap the delete and insert branches:

    the NUMBER is still correct on every test
    the RECONSTRUCTION reports insertions where deletions happened

→ the bug survives every unit test that checks the distance
→ it only appears when you ask WHAT the edits were

That is why naming the branches out loud matters more here
than anywhere else this week.

```

Edit distance against LCS, on the same table shape:

```

same state, same two-dimensional prefix table

LCS                                EDIT DISTANCE
base: zeros                        base: 0,1,2,3,...
match: dp[i-1][j-1] + 1            match: dp[i-1][j-1]          (free)
else: MAX of 2 neighbours          else: 1 + MIN of 3
maximising                         minimising

and they are related:
    if only insert and delete are allowed (no replace),
    distance = n + m - 2 x LCS(a, b)

```

5 The code, built step by step

The base cases, which are the part people get wrong

```
def edit_distance(a: str, b: str) → int:
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = i                                # i deletions to reach ""
    for j in range(m + 1):
        dp[0][j] = j                                # j insertions from ""
    return dp[n][m]                                # (loops next)
```

These two loops are not optional and they are not zeros. Turning "horse" into "" takes five deletions, not zero. Leaving them out gives an answer that is far too small and looks plausible.

Now the recurrence:

```
for i in range(1, n + 1):
    for j in range(1, m + 1):
        if a[i - 1] == b[j - 1]:
            dp[i][j] = dp[i - 1][j - 1]    # free - no +1
        else:
            dp[i][j] = 1 + min(
                dp[i - 1][j - 1],           # replace
                dp[i - 1][j],               # delete a[i-1]
                dp[i][j - 1],               # insert b[j-1]
            )
```

The comments are the point. Write them as you write the branches, and the delete/insert confusion cannot happen.

Reconstructing the edits

```
def edit_script(a: str, b: str) → list[str]:
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = i
    for j in range(m + 1):
        dp[0][j] = j
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1])

    ops: list[str] = []
    i, j = n, m
    while i > 0 or j > 0:
        if i > 0 and j > 0 and a[i - 1] == b[j - 1]:
            ops.append(f"keep {a[i - 1]}")
            i, j = i - 1, j - 1
        elif i > 0 and j > 0 and dp[i][j] == dp[i - 1][j - 1] + 1:
            ops.append(f"replace {a[i - 1]} → {b[j - 1]}")
            i, j = i - 1, j - 1
        elif i > 0 and dp[i][j] == dp[i - 1][j] + 1:
            ops.append(f"delete {a[i - 1]}")
            i -= 1
        else:
            ops.append(f"insert {b[j - 1]}")
            j -= 1
    return list(reversed(ops))
```

`while i > 0 or j > 0`, `not` and `.` When one string runs out, the rest of the other is pure insertions or pure deletions, and an `and` would stop early and lose them.

The order of the `elif` branches decides which valid script you get when several have the same cost — same tie-break point as yesterday.

The space collapse

```
def edit_distance_two_rows(a: str, b: str) → int:
    if len(b) > len(a):
        a, b = b, a
    previous = list(range(len(b) + 1))
    for i in range(1, len(a) + 1):
        current = [i] + [0] * len(b)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                current[j] = previous[j - 1]
            else:
                current[j] = 1 + min(previous[j - 1], previous[j], current[j - 1])
        previous = current
    return previous[len(b)]
```

`current = [i] + ...` is the column base case, and forgetting it is the classic bug in the collapsed version — the first column silently stays zero and every answer is too small.

The swap is free here because the distance is symmetric, which is not true of every two-dimensional DP.

Weighted operations

```
def weighted_edit_distance(a: str, b: str, replace_cost, insert_cost, delete_cost) → float:
    n, m = len(a), len(b)
    dp = [[0.0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        dp[i][0] = dp[i - 1][0] + delete_cost(a[i - 1])
    for j in range(1, m + 1):
        dp[0][j] = dp[0][j - 1] + insert_cost(b[j - 1])
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = min(
                    dp[i - 1][j - 1] + replace_cost(a[i - 1], b[j - 1]),
                    dp[i - 1][j] + delete_cost(a[i - 1]),
                    dp[i][j - 1] + insert_cost(b[j - 1]),
                )
    return dp[n][m]
```

The base cases now accumulate costs rather than counting, which is the change people miss when generalising. Everything else is identical.

Damerau-Levenshtein, with transposition

```
def damerau_levenshtein(a: str, b: str) → int:
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = i
    for j in range(m + 1):
        dp[0][j] = j
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            cost = 0 if a[i - 1] == b[j - 1] else 1
            dp[i][j] = min(dp[i - 1][j - 1] + cost, dp[i - 1][j] + 1, dp[i][j - 1] + 1)
            if (i > 1 and j > 1 and a[i - 1] == b[j - 2] and a[i - 2] == b[j - 1]):
                dp[i][j] = min(dp[i][j], dp[i - 2][j - 2] + 1) # transposition
    return dp[n][m]
```

The extra condition detects a swapped adjacent pair and charges one instead of two. "hte" to "the" becomes 1, which matters because that is one of the most common typing errors.

The early exit that makes fuzzy search practical

```
def within_distance(a: str, b: str, limit: int) → bool:
    if abs(len(a) - len(b)) > limit:
        return False # length alone rules it out
    previous = list(range(len(b) + 1))
    for i in range(1, len(a) + 1):
        current = [i] + [0] * len(b)
        for j in range(1, len(b) + 1):
            cost = 0 if a[i - 1] == b[j - 1] else 1
            current[j] = min(previous[j - 1] + cost, previous[j] + 1, current[j - 1] + 1)
        if min(current) > limit:
            return False # every path already exceeds the limit
        previous = current
    return previous[len(b)] ≤ limit
```

Two early exits, and both matter for real use. The length check is free and rejects most candidates immediately. And `min(current) > limit` stops mid-table, because every remaining path can only get worse.

The complete solution

```
"""Edit distance: the three operations, reconstruction, and the variants."""

def edit_distance(a: str, b: str) → int:
    """Minimum insert/delete/replace operations turning a into b."""
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = i                    # i deletions
    for j in range(m + 1):
        dp[0][j] = j                    # j insertions
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] # match: FREE, no +1
            else:
                dp[i][j] = 1 + min(
                    dp[i - 1][j - 1], # replace a[i-1] with b[j-1]
                    dp[i - 1][j],     # delete a[i-1]
                    dp[i][j - 1],     # insert b[j-1]
                )
    return dp[n][m]

def edit_script(a: str, b: str) → list[str]:
    """The actual operations. Needs the full table."""
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = i
    for j in range(m + 1):
        dp[0][j] = j
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1])

    ops: list[str] = []
    i, j = n, m
    while i > 0 or j > 0:                # OR, not AND
        if i > 0 and j > 0 and a[i - 1] == b[j - 1]:
            ops.append(f"keep    {a[i - 1]}")
            i, j = i - 1, j - 1
        elif i > 0 and j > 0 and dp[i][j] == dp[i - 1][j - 1] + 1:
            ops.append(f"replace {a[i - 1]} → {b[j - 1]}")
            i, j = i - 1, j - 1
        elif i > 0 and dp[i][j] == dp[i - 1][j] + 1:
            ops.append(f"delete  {a[i - 1]}")
            i -= 1
        else:
            ops.append(f"insert  {b[j - 1]}")
            j -= 1
    return list(reversed(ops))

def edit_distance_two_rows(a: str, b: str) → int:
    """O(min(n, m)) space. The distance is symmetric, so the swap is free."""
    if len(b) > len(a):
        a, b = b, a
    previous = list(range(len(b) + 1))
    for i in range(1, len(a) + 1):
```

```

        current = [i] + [0] * len(b)          # column base case – easy to forget
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                current[j] = previous[j - 1]
            else:
                current[j] = 1 + min(previous[j - 1], previous[j], current[j - 1])
        previous = current
    return previous[len(b)]

def damerau_levenshtein(a: str, b: str) → int:
    """Adds transposition of adjacent characters as a fourth operation."""
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = i
    for j in range(m + 1):
        dp[0][j] = j
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            cost = 0 if a[i - 1] == b[j - 1] else 1
            dp[i][j] = min(dp[i - 1][j - 1] + cost, dp[i - 1][j] + 1, dp[i][j - 1] + 1)
            if i > 1 and j > 1 and a[i - 1] == b[j - 2] and a[i - 2] == b[j - 1]:
                dp[i][j] = min(dp[i][j], dp[i - 2][j - 2] + 1)
    return dp[n][m]

def within_distance(a: str, b: str, limit: int) → bool:
    """Fuzzy matching with two early exits. This is what makes search practical."""
    if abs(len(a) - len(b)) > limit:
        return False
    previous = list(range(len(b) + 1))
    for i in range(1, len(a) + 1):
        current = [i] + [0] * len(b)
        for j in range(1, len(b) + 1):
            cost = 0 if a[i - 1] == b[j - 1] else 1
            current[j] = min(previous[j - 1] + cost, previous[j] + 1, current[j - 1] + 1)
        if min(current) > limit:
            return False          # every remaining path exceeds it
        previous = current
    return previous[len(b)] ≤ limit

def suggest(word: str, dictionary: list[str], limit: int = 2) → list[str]:
    """A tiny spell-checker. Real ones prune before computing anything."""
    return sorted((w for w in dictionary if within_distance(word, w, limit)),
                  key=lambda w: edit_distance(word, w))

if __name__ == "__main__":
    print("horse → ros      :", edit_distance("horse", "ros"))
    print("intention → exe:", edit_distance("intention", "execution"))
    print("two rows agrees :", edit_distance_two_rows("horse", "ros"))
    print("symmetric      :", edit_distance("ros", "horse"))
    print("identical       :", edit_distance("abc", "abc"))
    print("empty to abc    :", edit_distance("", "abc"))
    print("abc to empty    :", edit_distance("abc", ""))

    print("script horse→ros:")
    for op in edit_script("horse", "ros"):
        print(" ", op)

    print("hte → the (lev):", edit_distance("hte", "the"))
    print("hte → the (dam):", damerau_levenshtein("hte", "the"))

```

```
words = ["ranitidine", "ranolazine", "ramipril", "rantidine", "ranitidin"]
print("suggest 'ranitidin':", suggest("ranitidin", words, limit=2))
print("within 1?      :", within_distance("ranitidin", "ranolazine", 1))
```

Run it and you get:

```
horse → ros      : 3
intention → exe: 5
two rows agrees : 3
symmetric       : 3
identical       : 0
empty to abc    : 3
abc to empty    : 3
script horse→ros:
  replace h → r
  keep    o
  delete  r
  keep    s
  delete  e
hte → the (lev): 2
hte → the (dam): 1
suggest 'ranitidin': ['ranitidin', 'ranitidine', 'rantidine']
within 1?           : False
```

hte → the giving 2 and then 1 is the transposition, made visible — and it is the single most common typing error, which is why spell-checkers use Damerau.

And the suggestion list is the chemist's shelf: `ranolazine` and `ramipril` are more than two edits away and never appear.

6 What it costs

Time. Two nested loops, three comparisons per cell.

```
n rows x m columns = n x m cells
each cell: one comparison, and a min of three      0(1)

TOTAL: 0(n x m)
```

Concretely:

two words of 10	100 cells	instant
two strings of 1,000	1,000,000 cells	~0.4 s in Python
two strings of 10,000	100,000,000 cells	~50 s
two DNA sequences of 10 ⁶	10 ¹² cells	impossible

LeetCode 72's constraint is 500 characters, so 250,000 cells — set exactly so the plain table passes.

Space.

full table	$(n+1) \times (m+1)$ $501 \times 501 = 251,001$ cells	~10 MB in Python
two rows	$2 \times (\min(n,m) + 1)$ $2 \times 501 = 1,002$ cells	~40 KB

250x less. And no reconstruction.

The swap, which is worth one `if`:

```
a = 10,000 characters, b = 5 characters
```

no swap: rows of length 6 either way — actually fine here
BUT the ROW COUNT is what iterates:
 without swap: 10,000 iterations of a 6-cell row
 with swap: 5 iterations of a 10,001-cell row

Both are 60,000 cells. The swap does not change the TIME.
It changes the SPACE: rows of 6 vs rows of 10,001.
→ keep the SHORTER string as the inner dimension.

Where spell-checking falls over, which is the real lesson:

```
dictionary of 100,000 words, average length 8  
one query of length 8
```

brute force: 100,000 tables of $9 \times 9 = 81$ cells
 ~3 seconds per keystroke. Unusable.

with the length filter ($|\text{len}(a) - \text{len}(b)| > 2$ rejects immediately):
 ~30% survive → 1 second. Still unusable.

with a BK-TREE or a TRIE with pruning:
 ~100-1,000 candidates examined
 ~0.001 seconds. Usable.

The fix is not a faster edit distance. It is not computing it.

That is the most useful thing in this section, and it is the answer to “how does a real spell-checker work”.

Damerau's extra cost:

one extra comparison and one extra min per cell
→ ~20% slower, same $O(n \times m)$

and it catches the most common human typing error, which is
a trade almost every spell-checker takes.

The early exit, measured:

comparing "ranitidin" against 100,000 words with limit 2

length filter alone:	rejects ~70,000 instantly
min(current) > limit:	aborts most of the rest after 2-3 rows instead of all 9

→ roughly 10-20x faster than computing every full table,
for the same answers.

7 The traps

Zero base cases, carried over from LCS.

```
>>> a, b = "horse", "ros"
>>> dp = [[0] * 4 for _ in range(6)]          # all zeros – no base case loops
>>> for i in range(1, 6):
...     for j in range(1, 4):
...         if a[i-1] == b[j-1]:
...             dp[i][j] = dp[i-1][j-1]
...         else:
...             dp[i][j] = 1 + min(dp[i-1][j-1], dp[i-1][j], dp[i][j-1])
>>> dp[5][3]
2
```

Two, when the answer is three. The table thinks turning "horse" into "" is free, so everything downstream is too cheap. **No error, and a small plausible number** — which is the worst kind of wrong.

Adding one on a match.

```
>>> # in the match branch:
>>> #   dp[i][j] = dp[i-1][j-1] + 1      WRONG
>>> # gives edit_distance("abc", "abc") = 3
```

Three edits to turn a string into itself. Matching is free — that is the whole point of matching — and the + 1 belongs only in the mismatch branch.

Swapping delete and insert, which no test catches.

```
>>> edit_distance("horse", "ros")
3
>>> edit_distance("ros", "horse")
3
```

The distance is symmetric, so exchanging the two branches gives the correct number every time. **The bug only shows in the edit script**, which will say “insert” where a deletion happened. **Name the branches in comments as you write them** — it costs three seconds and it is the only defence.

`while i > 0 and j > 0` in the walk-back.

```
>>> # with `and`, reconstructing "" → "abc":
>>> # the loop never runs, and the script is empty
>>> # when it should be three insertions
```

Use `or`. When one string is exhausted, everything remaining in the other is a run of pure insertions or pure deletions, and `and` silently drops them.

Forgetting the column base in the collapsed version.

```
>>> previous = list(range(4))
>>> current = [0] * 4                                # should be [i, 0, 0, 0]
```

The first cell of each row is `i`, not zero. Leaving it as zero means “turning the first `i` characters into an empty string is free”, and the answers come out too small — **the same failure as the missing base row, hidden inside the optimisation.**

Unicode, which is a real-world trap rather than an interview one.

```
>>> len("é")
1
>>> len("é")          # e followed by a combining accent
2
>>> edit_distance("é", "é")
1
```

Two strings that look identical and are two different sequences of code points. Normalise with `unicodedata.normalize("NFC", s)` before comparing, or every accented word is reported as a near-miss.

Very long strings.

```
>>> dp = [[0] * 100001 for _ in range(100001)]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
MemoryError
```

Ten billion cells. For DNA-scale inputs the answer is not more memory — it is a banded algorithm that only fills cells near the diagonal, valid when you only care about distances below a threshold.

Comparing against a whole dictionary in a loop.

```
100,000 words x one table each = ~3 seconds per keystroke
```

No error, just an unusable product. The answer is to prune candidates before computing anything, not to optimise the table.

8 In the interview

How it gets asked

- “Given two strings, find the minimum number of operations to convert one to the other.” — LeetCode 72.
- “What are the three operations, and which branch is which?” — the naming question.
- “Show me the actual edits, not just the count.”
- “Reduce the space.”
- “How would you build a spell-checker with this?” — where the real answer is “not like this”.
- “What if the operations have different costs?”

The first ninety seconds

“Same two-dimensional table as longest common subsequence, with three options instead of two and minimising rather than maximising.

$dp[i][j]$ is the minimum number of edits to turn the first i characters of a into the first j of b . Prefix lengths again, so $i = 0$ is the empty prefix and the character is $a[i-1]$.

The base cases are different from LCS and this is where people go wrong.

$dp[i][0] = i$, because turning i characters into nothing takes i deletions.

$dp[0][j] = j$, because building j characters from nothing takes j insertions.

So the first row and column are $0, 1, 2, 3 \dots$, not zeros — and the zeros carry over from yesterday by muscle memory and give an answer that is far too small.

If the characters match, the edit is free — $dp[i][j] = dp[i-1][j-1]$, no plus one. That is the second most common bug, and it makes the distance from a string to itself non-zero.

If they differ, one of three operations, each costing one, and I would name all three.

Replace $a[i-1]$ with $b[j-1]$ — both characters are dealt with, so both indices retreat: the diagonal.

Delete $a[i-1]$ — I have consumed a character of a and none of b , so only i retreats: $dp[i-1][j]$.

Insert $b[j-1]$ — I have consumed a character of b and none of a , so only j retreats: $dp[i][j-1]$.

The rule is that the index which moves belongs to the string whose character you just used up, and saying that out loud is how I avoid getting delete and insert backwards.

And I would flag why that matters: edit distance is symmetric, so **swapping those two branches gives the correct number on every test**. The bug only appears when you ask what the edits were — which is usually the next question.

The answer is $dp[n][m]$. $O(n \times m)$ time and space, reducible to two rows if only the number is needed.”

The follow-ups

“Show me the actual edits, not just the count.”

“Walk back from `dp[n][m]`, asking at each cell which of the four possibilities produced its value.

If the characters match and `dp[i][j] = dp[i-1][j-1]`, this was a free match — emit ‘keep’ and move diagonally.

If `dp[i][j] = dp[i-1][j-1] + 1`, it was a replace — emit it and move diagonally.

If `dp[i][j] = dp[i-1][j] + 1`, it was a delete of `a[i-1]` — emit it and move up.

Otherwise it was an insert of `b[j-1]` — emit it and move left.

Then reverse, because I built it backwards.

The loop condition is `while i > 0 or j > 0`, not `and` — that is the specific bug here. When one string is exhausted, everything remaining in the other is a run of pure insertions or pure deletions, and `and` stops early and silently drops them. Reconstructing empty-string to `"abc"` with `and` returns an empty script instead of three insertions.

Cost is `O(n + m)` in time — each step decreases `i` or `j` or both — and it needs the full table, so I cannot use the two-row version. That trade is worth stating: linear space or the edit script, not both, unless I use Hirschberg’s divide-and-conquer.

Two honest points. The script is not unique — the order of my `elif` branches decides which of several equal-cost scripts I return, and all of them are correct.

And this is where the delete/insert confusion finally shows. Since the distance is symmetric, swapped branches pass every test that checks the number, and then the script reports insertions where deletions happened. So if a reconstruction ever looks backwards, that is the bug, not the table.”

“How would you use this to build a spell-checker?”

“The honest answer is that edit distance is the scoring function, and the interesting engineering is in never calling it.

The naive version is: for each of a hundred thousand dictionary words, compute the edit distance to the query, keep the ones within two. That is a hundred thousand tables of nine by nine, about eight million cells, **roughly three seconds per keystroke**. Unusable, and no amount of optimising the inner loop fixes three orders of magnitude.

Two cheap filters get part of the way. If the lengths differ by more than the limit, the distance cannot be within it — one subtraction, and it rejects most of the dictionary immediately. **And inside the table, if the minimum of the current row already exceeds the limit, abort** — every remaining path can only get worse, so I stop after two or three rows instead of nine.

Together those are maybe ten to twenty times faster, and still not enough.

The real answer is a data structure that prunes candidates before any table is built. A **BK-tree** exploits the triangle inequality: distance is a metric, so if a word is at distance d from a node, anything within k of my query must be within $d - k$ to $d + k$ of that node, and I skip every other subtree. **That turns a hundred thousand comparisons into a few hundred.**

Or a trie with a running edit-distance row, walking the trie and carrying the DP row down each branch — when the whole row exceeds the limit, prune that entire subtree. **That shares work across every word with a common prefix**, which in a dictionary is most of them.

Either way it goes from three seconds to about a millisecond, and the point I would make is that the algorithmic improvement is not in the distance function at all.

Two product details. I would use Damerau-Levenshtein rather than plain Levenshtein, because transposing two adjacent letters is one of the commonest typing errors and plain Levenshtein charges it two instead of one — “hte” to “the” should be one edit. **And I would weight the operations**: a replacement between keys that are adjacent on the keyboard should cost less than between distant ones, and the recurrence handles that unchanged — only the constants differ.

And I would rank the survivors by word frequency, not just by distance, because among words at distance one from a typo, the common one is nearly always what was meant.”

“What if inserting is much more expensive than deleting?”

“Then the recurrence is unchanged and only the constants move, which is the nice part about how this is structured.

Instead of `1 + min(...)`, each branch carries its own cost: the replace branch adds `cost_replace(a[i-1], b[j-1])`, the delete branch adds `cost_delete(a[i-1])`, the insert branch adds `cost_insert(b[j-1])`.

The part that is easy to miss is the base cases. They stop being `i` and `j` and become **accumulated costs**: `dp[i][0]` is the total cost of deleting the first `i` characters one by one, and `dp[0][j]` is the total cost of inserting the first `j`. **If the costs vary by character, those are running sums, not multiplications.**

And one property is lost: the distance stops being symmetric. With uniform costs, turning `a` into `b` costs the same as `b` into `a`. **With asymmetric insert and delete costs it does not**, so the swap I use in the space-optimised version is no longer free, and I have to be careful about which string is which.

That matters more than it sounds, because it means I can no longer use it as a metric — and things like BK-trees depend on the triangle inequality holding, which asymmetric costs can break.

Where this comes up for real: DNA alignment, where a substitution between chemically similar bases is biologically likelier than between different ones, and where opening a gap costs much more than extending an existing one — that last part actually needs an extra state, because the cost depends on whether you are already in a gap.

And OCR correction, where confusing `0` with `O` or `1` with `l` should cost far less than confusing `0` with `W`, because the errors come from shapes rather than from meaning.”

The model answer

“You are building the search box for a medicine ordering app. Users type drug names, often misspelled, and you have a catalogue of forty thousand products. Design it.”

“The scoring function is edit distance, and I want to be clear up front that **the design work is in avoiding computing it**, because the naive version is unusably slow at this size.

First, the requirement. A user typing `'ranitidin'` should see `Ranitidine`. A user typing something four edits away from a real drug should probably see nothing, **because a wrong medicine suggestion is worse than no suggestion** — which is a domain constraint that changes the threshold, and I would raise it explicitly.

The scoring function: Damerau-Levenshtein rather than plain Levenshtein, because transposing adjacent letters is one of the most common typing errors and plain Levenshtein charges two for it. `'ranitidnie'` should be one edit from `'ranitidine'`, not two.

And weighted, because a replacement between keys adjacent on a phone keyboard is far likelier than between distant ones. **The recurrence is unchanged; only the constants differ.**

Now the performance, which is the actual design. Forty thousand products, a query of about ten characters. **Naively that is forty thousand tables of eleven by eleven — about five million cells, roughly a second and a half per key-stroke.** Unusable in a search box that fires on every character.

Two cheap filters first. The length filter — if the lengths differ by more than the limit, reject without building anything — costs one subtraction and eliminates most of the catalogue. **And the row-minimum early exit** — if every cell in the current row already exceeds the limit, abort, because no path can improve. **Together, maybe fifteen times faster, and still not fast enough.**

So the structure: a trie over the catalogue with the DP row carried down each branch. I walk the trie character by character, computing one edit-distance row per node, **and when the minimum of a node's row exceeds the limit I prune that entire subtree.** Because drug names share prefixes heavily — every `Ran-` drug shares three characters — this shares almost all the work. **That gets it to roughly a millisecond.**

A BK-tree is the alternative, using the triangle inequality to skip subtrees, and it is simpler to build. **I would prefer the trie here** because it also gives me prefix completion for free, which the search box wants anyway — the user typing `'rani'` should see suggestions before they have made a typo at all.

Ranking the survivors, which matters as much as finding them. Not by edit distance alone. Among candidates at distance one, I would rank by **how often the product is actually ordered**, because the common drug is nearly always what was meant. **And I would boost exact prefix matches above any fuzzy match**, because a user who has typed four correct characters is more likely completing than mistyping.

Two things specific to this domain that I would insist on.

A strict threshold, and a preference for showing nothing. At distance two, `Ranitidine` and `Ranolazine` are different drugs for different conditions. I would cap suggestions at distance one for short names and two for long ones, and show ‘no match’ rather than a guess — and I would want that reviewed by someone who knows the products, not decided by me.

And confusable pairs handled explicitly. Drug names that differ by one or two characters but are clinically very different are a known safety problem with published lists. **Those pairs should never be offered as corrections for each other**, whatever the distance says — a hard-coded exception list on top of the algorithm, which is exactly the kind of thing an algorithm should not be trusted to get right on its own.

Performance-wise, the catalogue is small and static enough to hold the whole trie in memory on every application server, rebuilt on deploy — no network call in the search path at all, which is what makes per-keystroke search feel instant.”

9 Recall card

$dp[i][j]$ = minimum edits turning `a`'s first `i` characters into `b`'s first `j`. Prefix lengths, so the character is `a[i-1]`. Base cases are $dp[i][0] = i$ and $dp[0][j] = j$, NOT zeros — that is the LCS muscle memory that gives an answer far too small.

Match is FREE: $dp[i-1][j-1]$, no `+1`. Mismatch is $1 + \min$ of three, and name all three: replace = diagonal (both consumed), delete $a[i-1] = dp[i-1][j]$ (only `i` moves), insert $b[j-1] = dp[i][j-1]$ (only `j` moves). The index that moves belongs to the string whose character you just used up.

Edit distance is symmetric, so swapping delete and insert passes every test that checks the number — the bug only appears in the reconstruction. Name the branches in comments as you write them.

Reconstruction walks back asking which of four cases produced the value; the loop is `while i > 0 or j > 0`, not `and` — with `and`, a run of trailing insertions is silently dropped. Needs the full table.

`O(n × m)` time and space; two rows for `O(min(n,m))`, and `current = [i] + ...` is the column base case people forget. Weighted costs change only the constants — and the base cases become running sums, and the distance stops being symmetric. Damerau adds transposition: `"hte"→"the"` is 1, not 2.

A spell-checker does not call this 100,000 times — 3 seconds per keystroke. Length filter, row-minimum early exit, then a BK-tree or a trie carrying the DP row down and pruning whole subtrees — a few hundred candidates instead of the whole dictionary, and about a millisecond.

1 What this is, and why they ask it

“Design Twitter” is the most asked system design question there is. It has been asked at every large company for a decade, and there is a reason it survived: it is the smallest problem that contains almost everything.

The product is tiny. **Post a short message. Follow people. See their messages, newest first.** Three verbs, and every one of them is genuinely hard at three hundred million users.

They ask it because **it is a superset of yesterday’s news feed with three additions that change the design.** The social graph is now a first-class object with its own scaling problem — some accounts have a hundred million followers and the follower list itself does not fit anywhere convenient. **Search and trends** appear, which need a completely different index from the timeline. **And there are two timelines, not one** — your home timeline of people you follow, and a user’s own profile timeline, which are stored and served in different ways for different reasons.

And they ask it because **it is the best available test of scoping.** The full product has direct messages, lists, spaces, ads, moderation, notifications, media, verification and analytics. **A candidate who tries to design all of it produces nothing; a candidate who names them, picks three, and goes deep produces a design.** That choice, made out loud in the first minute, is a large part of what is being assessed.

By the end of this lesson you can scope the problem, design the graph store and both timelines, handle the celebrity and the fan-out, add search and trending, and size the whole thing.

2 The story

The wedding hall took bookings a year ahead and the man who ran it, Krishnappa, had one book, and by the time his daughter took over there were four.

The first book was the diary, and it was the only one that had existed for eleven years. One page a day. Whoever booked, you wrote their name and the phone number and the advance. **It answered exactly one question: what is happening on the fourteenth of March.**

The second book came because of the phone calls. People rang and said “I booked in your hall, what date did I take?” and Krishnappa would go through the diary page by page until he found them, which took twenty minutes and sometimes did not work. **So he started a second book, alphabetical by name, saying what each family had booked.**

Same bookings. Written twice. **Two books because there were two questions,** and one arrangement could not answer both quickly.

The third book was his daughter’s and it caused an argument. She started writing down, at the end of each month, which caterers and which decorators people had asked for. Not for any booking in particular. Just counts.

Her father said it was a waste of an evening. **Then in November a decorator asked him whether people were still asking for the old style of mandap, and he did not know, and she did.**

And the fourth book was the one they never quite got right. It was for the families who booked the hall every year — the three or four big families in the town who had a function almost every month, and whose entries filled so many pages of the diary that looking anything up became slow.

They tried keeping those families’ bookings on their own separate sheets. Which worked, except that now the diary was incomplete, and answering “what is happening on the fourteenth” meant checking the diary **and** the four sheets.

His daughter’s summary, which she said to a friend and not to her father, was the useful one.

“He thinks the book is the bookings. It is not. The book is one way of arranging them, and we needed four, and the big families broke all of them.”

3 The idea in plain English

Krishnappa’s daughter has described the whole design: **the same data, stored several times, arranged differently for each question — and the big families break every arrangement.**

Scope first, because the product is enormous and the interview is forty-five minutes.


```

IN    post a tweet
      follow / unfollow
      home timeline (people you follow, newest first)
      user timeline (one person's own tweets)
      search
      trending
OUT   direct messages, lists, spaces, ads, moderation,
      analytics, verification, media transcoding

```

Say the out-of-scope list out loud and move on. Naming them shows you know the product; designing them shows you cannot prioritise.

Now the scale, because it decides everything.

```

300 million daily active users
each opens the app ~10 times/day    → 3 billion timeline reads/day
each tweets ~0.1 times/day         → 30 million tweets/day

read : write = 100 : 1

```

A hundred reads per write is the number that justifies doing expensive work at write time, exactly as in [day 153](#).

The core data model is three things.

```

tweets(id, author_id, text, created_at, reply_to, ...)
follows(follower_id, followee_id, created_at)
timeline:<user_id>          -- a precomputed list of tweet ids

```

Tweet ids are Snowflakes — [day 150](#)’s design — so they sort by time, which makes “tweets after this id” a range query and makes the timeline’s sort order free.

Now the two timelines, which is Krishnappa’s two books.

The user timeline — one person’s own tweets, shown on their profile — is a straightforward query: `SELECT * FROM tweets WHERE author_id = ? ORDER BY id DESC LIMIT 20`. **One index, one query, done.** It is stored once, sharded by author.

The home timeline — everyone you follow, merged — cannot be a query, because it would touch hundreds of shards. **So it is precomputed and stored separately**, which is the same data arranged differently for a different question.

Fan-out on write builds it: when you tweet, insert the tweet id into each follower’s precomputed timeline. **Reading is then one lookup.**

And it breaks on the accounts with a hundred million followers, so: hybrid. Under a threshold — ten thousand followers — fan out on write. Above it, store once and let readers merge. **At read time, take your precomputed timeline and merge in recent tweets from the celebrities you follow.**

That is yesterday's design, and Twitter is where it comes from. The difference today is everything around it.

The social graph, which is a real problem of its own.

`follows` needs to be queried in **both** directions: `followee → followers` for fan-out, `follower → followees` for the read path. **That is two indexes, or two tables, and at Twitter's size it is a dedicated service.**

The hard part is that follower lists are enormously skewed. Most accounts have under a thousand followers. A few have over a hundred million — **and a hundred million follower ids at 8 bytes each is 800 MB for one row's worth of relationship.** That does not fit in a value, cannot be fetched at once, and must be paginated and streamed during fan-out.

Sharding the graph is the interesting decision, and there are two options with a real trade:

- **Shard by follower** makes “who do I follow” a single-shard read — good for the read path, and fan-out has to scatter.
- **Shard by followee** makes “who follows this account” single-shard — good for fan-out, and now a celebrity is a single hot shard.

Store both, which is what real systems do: two denormalised copies, kept consistent asynchronously. **Krishnappa's two books again.**

Search, which needs a completely different index.

A timeline is ordered by time; search is ordered by relevance and looked up by word. You cannot serve one from the other. **So tweets are also written to an inverted index** — `day 135`'s structure — mapping each term to the tweet ids containing it.

And the index is near-real-time, not real-time. A tweet becomes searchable within seconds, not milliseconds, because indexing is asynchronous. **That is an accepted product property, not a failure,** and saying so is better than pretending otherwise.

The volume is the problem: thirty million tweets a day, each producing perhaps fifteen index entries. Search shards by time — recent tweets in a hot index, older ones in cold shards — because almost all queries are about the last few days.

Trending, which is a counting problem with a twist.

Naive trending is “the most used hashtags in the last hour”, and it produces the same words every day — the terms that are always popular. **Trending is about the *change* in rate, not the rate.** A term used a thousand times an hour every day is not trending; a term used five times an hour that is suddenly used four hundred times is.

So: count per term per time window, compare against a baseline, and rank by the ratio. Counting exactly at this volume is expensive, so **approximate counters — count-min sketch — are the standard answer**, trading a small overcount for a fixed, tiny amount of memory.

And the last piece: everything else is a cache.

Tweet content, user profiles and engagement counts are all fetched by id after the timeline gives you ids. That hydration is one batched multi-get, and it is where the read latency actually lives. **Ninety percent of reads are for tweets from the last two days**, so a cache holding recent tweets covers almost everything.

4 The picture

The whole system:



The two timelines, which are two different books:

USER TIMELINE (profile page)

"what did @ravi tweet?"

```
SELECT * FROM tweets
WHERE author_id = ?
ORDER BY id DESC LIMIT 20
```

one index, one shard, one query
stored ONCE

Same tweets. Two arrangements. Two questions.

HOME TIMELINE (the feed)

"what did everyone I follow tweet?"

cannot be a query – it would touch hundreds of shards

so it is PRECOMPUTED:
timeline:<user> as a sorted set built by fan-out on write

stored ONCE PER FOLLOWER

The follower-list problem, which is specific to Twitter:

follower count distribution (roughly)

< 1,000 followers	~95% of accounts
1,000 - 10,000	~4%
10,000 - 1,000,000	~1%
> 10,000,000	a few thousand accounts
> 100,000,000	a few dozen

ONE account with 100,000,000 followers:

100,000,000 x 8 bytes = 800 MB just for the ID LIST

→ cannot be one value, cannot be fetched at once

→ must be PAGED and STREAMED during fan-out

→ and fanning out to it would take ~33 minutes at 50k writes/s

This is why the hybrid exists, and why the graph service is its own system rather than a table.

Sharding the graph — the trade:

SHARD BY FOLLOWER

"who do I follow?"

→ ONE shard. Fast.

"who follows me?"

→ SCATTER to all shards

good for the READ path

SHARD BY FOLLOWEE

"who follows me?"

→ ONE shard. Fast.

"who do I follow?"

→ SCATTER to all shards

good for FAN-OUT

and a celebrity is ONE HOT SHARD

ANSWER: store both. Two denormalised copies, reconciled asynchronously. Twice the storage, and both questions are fast.

Search: a different index for a different question:

TIMELINE INDEX

key: user id

value: [tweet ids, by time]

"mumbai rain" → intersect the posting lists for "mumbai"
and "rain", then rank by recency + engagement

ordered by TIME

answers "what is new"

SEARCH INDEX (inverted)

key: term

value: [tweet ids containing it]

ordered by RELEVANCE

answers "what mentions X"

Neither can serve the other's query. Both are written on every tweet.

Trending: rate of change, not rate:

term	last hour	typical hour	ratio	
"the"	850,000	840,000	1.01	not trending
"cricket"	40,000	38,000	1.05	not trending
"earthquake"	9,000	30	300.0	TRENDING
"gujarat"	12,000	400	30.0	TRENDING

Ranking by COUNT gives you "the" every single day.
Ranking by count / baseline gives you the news.

Counting exactly at 30M tweets/day is expensive, so use a
COUNT-MIN SKETCH: fixed memory, small overcount, never undercounts.

5 How it actually works

Posting a tweet

```
def post_tweet(author_id: int, text: str) → int:
    tweet_id = snowflake.next_id()          # time-sortable, from day 150
    tweet_store.put(tweet_id, author_id, text, time.time())

    queue.publish("fanout", {"tweet_id": tweet_id, "author_id": author_id})
    queue.publish("index", {"tweet_id": tweet_id, "text": text})
    return tweet_id                          # returns immediately
```

Three things, and only the first is synchronous. The tweet is durable before the API returns; fan-out and indexing happen behind the queue. **So a tweet appears on your own profile instantly and in followers' feeds a second later**, which is exactly the behaviour users observe.

Fan-out, with the celebrity check

```
CELEBRITY_THRESHOLD = 10_000

def handle_fanout(msg: dict) → None:
    author_id = msg["author_id"]
    if graph.follower_count(author_id) > CELEBRITY_THRESHOLD:
        return                                # read path handles it

    for page in graph.iter_followers(author_id, page_size=1000):
        queue.publish("fanout_batch", {"tweet_id": msg["tweet_id"], "targets": page})
```

`iter_followers` **must stream, not return a list**. A hundred million followers is 800 MB of ids — **calling `.all()` on that is an out-of-memory error**, and it is the specific reason the graph service exposes a paged cursor rather than a getter.

```
def handle_fanout_batch(msg: dict) → None:
    pipe = redis.pipeline()
    for user_id in msg["targets"]:
        key = f"timeline:{user_id}"
        pipe.zadd(key, {msg["tweet_id"]: msg["tweet_id"]}) # id IS the timestamp
        pipe.zremrangebyrank(key, 0, -801) # cap at 800
    pipe.execute()
```

The score is the tweet id itself, because Snowflakes sort by time — **no separate timestamp needed**, which halves the memory of every timeline entry.

The home timeline read

```
def home_timeline(user_id: int, cursor: int | None, limit: int = 20) → list[dict]:
    upper = cursor if cursor is not None else "+inf"
    own = redis.zrevrangebyscore(f"timeline:{user_id}", upper, "-inf",
                                start=0, num=limit * 3)

    celebs = graph.celebrities_followed(user_id) # ~30, cached
    extra = [t for c in celebs for t in tweet_store.recent(c, limit, before=cursor)]

    merged = sorted(set(own) | set(extra), reverse=True)[:limit]
    return hydrate(merged)
```

`sorted(..., reverse=True)` works because ids sort by time, so merging two id lists needs no timestamp lookup at all. **That is the Snowflake decision paying off.**

`zrevrangebyscore` with the cursor as the upper bound is cursor pagination — “everything below this id” — which is stable when new tweets arrive between pages.

The user timeline, which is just a query

```
def user_timeline(author_id: int, cursor: int | None, limit: int = 20) → list[dict]:
    return hydrate(tweet_store.by_author(author_id, before=cursor, limit=limit))
```

One query on an index of (author_id, tweet_id). No precomputation, no fan-out, nothing clever — **and this is worth saying explicitly in an interview**, because candidates sometimes try to precompute it too.

Search indexing

```
def handle_index(msg: dict) → None:
    terms = tokenize(msg["text"]) # lowercase, strip punctuation, stem
    shard = search_shard_for(msg["tweet_id"]) # sharded by TIME
    pipe = shard.pipeline()
    for term in set(terms):
        pipe.zadd(f"term:{term}", {msg["tweet_id"]: msg["tweet_id"]})
    pipe.execute()
```

Sharded by time, not by term, because almost every query is about recent tweets. A **hot shard holds the last few days and answers most queries; cold shards hold history and are rarely touched.**

```
def search(query: str, limit: int = 20) → list[dict]:
    terms = tokenize(query)
    posting_lists = [shard.zrevrange(f"term:{t}", 0, 999) for t in terms]
    candidates = set.intersection(*map(set, posting_lists))
    ranked = rank_search(candidates, query)[:limit]
    return hydrate(ranked)
```

Intersecting posting lists is the whole of boolean search, and taking only the top thousand from each keeps it bounded — **a common term's full posting list is millions of entries, and nobody scrolls past a few pages.**

Trending with a count-min sketch

```
class CountMinSketch:
    def __init__(self, width: int = 2**16, depth: int = 5) → None:
        self.table = [[0] * width for _ in range(depth)]
        self.width, self.depth = width, depth

    def add(self, item: str, count: int = 1) → None:
        for row in range(self.depth):
            self.table[row][self._hash(item, row)] += count

    def estimate(self, item: str) → int:
        return min(self.table[row][self._hash(item, row)] for row in range(self.depth))

    def _hash(self, item: str, row: int) → int:
        return hash(f"{row}:{item}") % self.width
```

min over the rows is the trick. Collisions can only inflate a counter, so **the smallest of d independent estimates is the closest to the truth — it never undercounts.** Five rows of 65,536 counters is about 1.3 MB, regardless of how many distinct terms exist.

```
def trending(current: CountMinSketch, baseline: CountMinSketch,
             candidates: list[str], top: int = 10) → list[str]:
    scored = [(current.estimate(t) / max(baseline.estimate(t), 10), t) for t in candidates]
    return [t for _, t in sorted(scored, reverse=True)[:top]]
```

Dividing by the baseline is what makes it trending rather than popular. **`max(baseline, 10)` stops a term with a baseline of one from scoring infinitely — a single mention becoming two is not news.**

Hydration

```
def hydrate(tweet_ids: list[int]) → list[dict]:
    tweets = tweet_cache.multi_get(tweet_ids)           # one call
    authors = user_cache.multi_get([t["author_id"] for t in tweets.values()])
    counts = counter_cache.multi_get(tweet_ids)         # one call
    return [build(tweets[i], authors[tweets[i]["author_id"]], counts[i])
            for i in tweet_ids if i in tweets]
```

Three batched calls, and the `if i in tweets` silently drops deleted tweets — which is why deletion does not need to scrub every timeline.

The real systems

Snowflake	Twitter's id generator, now used everywhere
Manhattan	Twitter's distributed key-value store (tweets, users)
FlockDB	the original graph store; "who follows whom" as a service
Earlybird	the real-time search index – Lucene-based, in-memory
Redis	timelines, as sorted sets
Kafka	the event backbone between all of the above

Naming Earlybird is worth doing, because it is the answer to “how is search real-time” — an in-memory inverted index over recent tweets, separate from the historical one.

6 The numbers

Traffic.

300,000,000 DAU	
10 timeline opens/day	→ 3,000,000,000 reads/day = ~35,000/s average ~100,000/s peak
0.1 tweets/day	→ 30,000,000 writes/day = ~350/s average ~1,000/s peak
search: ~10% of opens	→ 300,000,000 searches/day = ~3,500/s

Fan-out volume.

350 tweets/s x 200 average followers	= 70,000 timeline inserts/s
peak: 1,000 x 200	= 200,000/s
Redis at ~100,000 ops/s per instance	
→ 2-4 instances average, ~10 at peak, sharded by user id	

And the celebrity, which is why averages mislead:

100,000,000 followers, one tweet

fan-out on write:

100,000,000 inserts at 50,000/s = 2,000 s = ~33 MINUTES
and the follower LIST alone is 100,000,000 x 8 B = 800 MB

with the hybrid:

1 write. Readers merge ~30 celebrity accounts, ~3 ms extra.

Storage.

TWEETS

30,000,000/day x 300 bytes (text + metadata) = 9 GB/day
x 365 days = 3.3 TB/year
x 3 replicas = 10 TB/year
→ ten years of tweets is ~100 TB. Genuinely small.

TIMELINES

300,000,000 users x 800 entries x 8 bytes (id only) = 1.9 TB
x ~2 for Redis sorted-set overhead = ~4 TB

(the id IS the score, so 8 bytes per entry, not 16)

SOCIAL GRAPH

300,000,000 users x 200 average follows x 16 bytes = 1 TB
stored BOTH ways = 2 TB

SEARCH INDEX

30,000,000 tweets/day x 15 terms x 8 bytes = 3.6 GB/day of postings
x 30 days hot = ~108 GB hot index

Tweets are the smallest of these, which surprises people: the timelines and the graph are each larger than the content. That is what fan-out costs.

Read latency.

zrevrangebyscore timeline	~1 ms
fetch ~30 celebrities' recent	~3 ms
merge (ids sort by time – free)	~0.5 ms
hydrate: 3 batched multi-gets	~5 ms

	~10 ms

Cache sizing:

```
90% of reads hit tweets from the last 48 hours
48 hours = 60,000,000 tweets x 300 B = 18 GB

→ a 32 GB tweet cache covers ~90% of all reads
  at 100,000 reads/s x 20 tweets = 2,000,000 fetches/s
  10% miss = 200,000 store reads/s ← still the dominant load

moving the hit rate 90% → 95% HALVES the backing store's load.
```

Search cost.

```
3,500 searches/s
each intersects 2-3 posting lists, top 1,000 each
→ ~10,000 list reads/s, plus ranking

the hot index is ~108 GB and must be in memory for this to work
→ ~4-8 machines with 32 GB each, sharded by time
```

Trending, and why sketches:

```
exact counting: ~5,000,000 distinct terms per hour
                 x (term string ~20 B + counter 8 B) = 140 MB per window
                 x 24 windows for the baseline      = 3.4 GB
                 and it must be merged across shards

count-min sketch: 5 rows x 65,536 counters x 8 B = 2.6 MB per window
                  x 24 windows                    = 63 MB
                  and sketches MERGE by addition - trivially parallel

~50x less memory, and the error is a small overcount on rare terms,
which do not appear in the top ten anyway.
```

The merge property is the best argument: each shard keeps its own sketch and they are summed, which exact counting cannot do without shipping every key.

7 The trade-offs

Fan-out on write against on read, which is the same trade as the news feed and is settled by the same number. A hundred reads per write means paying 0(followers) once beats paying a merge a hundred times. **And it fails on the hundred-million-follower account, so the hybrid exists** — the threshold is a tuning parameter, and being able to derive it is what matters.

Storing the graph twice. Sharding by follower makes the read path fast and the fan-out a scatter; sharding by followee makes fan-out fast and the read path a scatter. **Both copies costs twice the storage — about 2 TB, which is nothing — and buys both queries.** The real cost is consistency: the two copies are reconciled asynchronously, so for a moment after a follow, one direction knows and the other does not.

Timelines are stale by design. A precomputed timeline reflects the world as of the last fan-out, so a tweet from a second ago may not be there. **Users do not notice, and it is worth saying plainly** rather than pretending the system is synchronous.

Search is near-real-time, not real-time. Indexing is asynchronous, so a tweet is searchable in seconds. **Making it synchronous would put the index on the write path** and mean a tweet fails if the index is slow — **a much worse property than a two-second delay.**

Approximate trending against exact counting. A count-min sketch is fifty times smaller, merges across shards by addition, and **overcounts rare terms slightly.** Since rare terms are not in the top ten, the error is invisible where it matters. **Exact counting would need every distinct term's counter shipped between shards on every merge.**

The timeline cap trades completeness for bounded storage. Eight hundred entries per user is about 4 TB; uncapped is unbounded. **A user scrolling past the cap falls back to the slow path,** which is rare and therefore fine.

And the honest one: this design is Twitter in 2013, not Twitter now. The real system has ranking, ads interleaved, conversation threading, quality filtering and a recommendation system that surfaces tweets from people you do not follow. **Ranking in particular changes the read path substantially** — it means fetching several hundred candidates and scoring them — and it makes pagination much harder.

When would I not build this? Almost always. Below a million users, the entire product is a Postgres database with a `follows` table and a query with a join, and it will be fast, correct and maintainable. **Building the fan-out infrastructure first is the classic mistake this question tempts people into** — and saying that, briefly, at the end, is a stronger signal than another component.

8 In the interview

How it gets asked

- “*Design Twitter.*” — usually exactly that, with no further constraints.

- “How does the home timeline work?” — the fan-out question.
- “What about someone with a hundred million followers?”
- “How would you add search?” — the different-index question.
- “How does trending work?”
- “What would you cut if you had ten minutes?” — the scoping question, sometimes asked directly.

The first ninety seconds

“The product is huge, so let me scope first and then size, because both change what I build.

In scope: post a tweet, follow, home timeline, user timeline, search, trending. **Out of scope:** direct messages, lists, ads, moderation, media, analytics — real, and not what makes this interesting.

The number that decides the architecture: three hundred million daily users opening the app ten times is three billion timeline reads a day. They tweet about a tenth of a time each, so thirty million tweets. **A hundred reads per write** — so it is worth doing expensive work at write time.

There are two timelines and they are built completely differently, which is the first thing I would establish.

The user timeline — someone’s own tweets on their profile — is a query. Select by author, order by id descending, limit twenty. **One index, one shard, done.** No precomputation.

The home timeline cannot be a query, because merging hundreds of authors’ tweets would touch hundreds of shards on every read. **So it is precomputed by fan-out on write:** when you tweet, the tweet id is inserted into each follower’s timeline, stored as a Redis sorted set. **Reading is then one lookup — about ten milliseconds end to end.**

Tweet ids are Snowflakes, which is worth calling out because it does two jobs: the ids sort by time, so the timeline needs no separate timestamp — **the id is the score** — and merging two id lists needs no lookups at all.

And fan-out breaks on the celebrity. A hundred million followers means a hundred million inserts, about thirty-three minutes, **and the follower id list alone is 800 megabytes**, which cannot even be fetched at once. **So: hybrid.** Under ten thousand followers, fan out on write. Above, store once, and readers merge in the twenty or thirty celebrity accounts they follow.

Search and trending need different structures entirely, which I would flag now: **a timeline is indexed by user and ordered by time; search is indexed by term and ordered by relevance.** Neither can serve the other, so every tweet is written to both.

Where would you like me to go deep — the fan-out and the graph, or search and trending?”

The follow-ups

“Someone with a hundred million followers tweets. What happens?”

“Under pure fan-out on write, two things break, and the second one is the more interesting.

The obvious one: a hundred million sorted-set inserts. At fifty thousand writes a second that is two thousand seconds — **about thirty-three minutes** — during which the last follower has not seen it and the fan-out pipeline is doing nothing else. Everyone else’s tweets queue behind it.

The less obvious one: the follower list itself. A hundred million ids at eight bytes each is **eight hundred megabytes**, for one account’s followers. **That is not a value you can fetch** — calling something like `get_followers()` on it is an out-of-memory error. So the graph service has to expose a paged cursor, and fan-out streams through it a thousand at a time. **That constraint alone is why the graph is its own service rather than a table with an index.**

The fix is the hybrid. Above a threshold, the tweet is stored once and nothing is pushed. At read time, I take my precomputed timeline and separately fetch recent tweets from the celebrities I follow — typically twenty or thirty accounts — and merge.

The asymmetry is what makes it work: the number of celebrities I follow is small even though the number of people following them is enormous. And the merge is nearly free because the ids sort by time, so merging two id lists is a sort of eighty integers.

Two refinements. The celebrity list per user must be cached, or ‘which of the people I follow are above the threshold’ becomes a join on every read.

And the threshold does not need to be a cliff. An account with fifteen thousand followers could fan out only to followers who opened the app today — instant delivery for the people who will actually see it soon, and a bounded write cost. **The number itself is a tuning parameter; what matters is why there is one at all.**“

“How would you add search?”

“With a completely different index, and I would say that first because it is the whole answer.

The timeline is keyed by user and ordered by time. Search is keyed by term and ordered by relevance. Neither structure can answer the other’s question, so **every tweet is written to both** — the same data arranged twice.

The search index is an inverted index: for each term, a list of the tweet ids containing it. A query for ‘mumbai rain’ fetches both posting lists and intersects them, then ranks the survivors by recency and engagement.

Bounded, though: I take only the top thousand ids from each list. A common term’s full posting list is millions of entries, and nobody paginates that far — so the intersection is over a bounded set.

Sharding is by time, not by term, and that is the decision worth defending. Almost every search is about the last few days, so **a hot in-memory index over recent tweets answers most queries, and cold shards hold history and are rarely touched.** Sharding by term instead would put ‘the’ on one machine and make it a hot spot.

Indexing is asynchronous, so search is near-real-time — seconds, not milliseconds. I would state that as a deliberate property rather than let it be discovered: **making it synchronous would put the index on the write path,** so a tweet would fail if the index were slow. **A two-second delay is a much better property than that.**

Sizing: thirty million tweets a day at about fifteen index terms each is 3.6 GB of postings a day, so a thirty-day hot index is roughly a hundred gigabytes and needs to be in memory — four to eight machines.

Twitter’s real system is called Earlybird, an in-memory Lucene index over recent tweets, kept separate from the historical one. **The split between a small hot index and a large cold one is the design,** and it follows directly from the query distribution.”

“How does trending work?”

“The naive version produces the same words every single day, and understanding why is most of the answer.

If I rank hashtags by count in the last hour, I get ‘the’, ‘a’, and whatever is permanently popular — cricket, a big film — because those terms are always used a lot. **They are popular, not trending.**

Trending is about the rate of change, not the rate. A term used a thousand times an hour every day is not news. A term used thirty times an hour that is suddenly used nine thousand times **is** news. **So I score by the ratio of the current window’s count to a baseline** — the same term’s typical count for this hour of this day of the week — and rank by that.

The baseline needs to be seasonal, because ‘good morning’ spikes every morning and is not news, and a flat all-time average would report it daily.

Counting is the expensive part. There are maybe five million distinct terms an hour, and exact counting means a counter per term per window — a few hundred megabytes per window, times twenty-four windows for the baseline, **and it has to be merged across every shard.**

So: a count-min sketch. A fixed grid of counters, `d` hash functions; to add a term, increment one counter per row; to estimate, **take the minimum across the rows.** The minimum works because collisions can only inflate a counter, so the smallest of several independent estimates is the closest to the truth — **and it never undercounts.**

Five rows of sixty-five thousand counters is about 2.6 megabytes, regardless of how many distinct terms exist. **Fifty times smaller than exact counting.**

And the property that actually sells it: sketches merge by addition. Each shard keeps its own and they are summed — trivially parallel. **Exact counting would need every distinct key shipped between shards on every merge.**

The error is a small overcount, worst on rare terms — and rare terms are not in the top ten, so it is invisible where it matters.

Two things I would add for a real product. A separate list of candidate terms, because I need something to estimate — the sketch tells me a count for a term I name, not what the top terms are, so I keep a heavy-hitters structure alongside. **And filtering**, because trending is a highly visible surface and people deliberately try to game it — which is a moderation problem, not an algorithmic one, and I would name it rather than pretend the ranking solves it.”

The model answer

“Design Twitter.”

“Let me scope, size, then design, and I will say what I am leaving out so it is clear it is a choice.

In: tweeting, following, the home timeline, the user timeline, search, trending. Out: DMs, lists, ads, media, moderation, analytics.

Sizing. Three hundred million daily users, ten opens each — three billion timeline reads a day, about thirty-five thousand a second, a hundred thousand at peak. Thirty million tweets a day, about three hundred and fifty a second. **A hundred to one, which is what justifies fan-out on write.**

Ids are Snowflakes, and that decision pays for itself three times: ids sort by time, so a timeline needs no separate score; merging two id lists needs no lookups; and ‘tweets after this id’ is a range query, which is my pagination cursor.

Two timelines, built differently. The user timeline is a query by author on a `(author_id, tweet_id)` index — nothing clever, and I would say that explicitly because it is tempting to precompute it too. **The home timeline is precomputed** by fan-out into a Redis sorted set per user, **capped at eight hundred entries**, with the tweet id as both member and score.

Hybrid fan-out at a ten-thousand-follower threshold, because a hundred-million-follower account would take thirty-three minutes to fan out and its follower list alone is eight hundred megabytes — **which is why the graph service exposes a streaming cursor rather than a getter.**

The graph is stored twice, sharded both ways. Follower-to-followee for the read path, followee-to-follower for fan-out. **Two terabytes, which is nothing, and it makes both questions single-shard.** The cost is that the two copies reconcile asynchronously, so for a moment after a follow, one direction knows and the other does not — **acceptable here, and worth naming.**

Search is a separate inverted index, sharded by time, with a hot in-memory index over recent tweets. Every tweet is written to both structures. **Indexing is asynchronous — near-real-time, seconds not milliseconds — because putting the index on the write path would mean a tweet fails when the index is slow.**

Trending is a count-min sketch per time window, scored against a seasonal baseline, because ranking by raw count returns ‘the’ every day. Sketches are 2.6 megabytes each and **merge across shards by addition**, which is the property that makes them the right choice rather than just a smaller one.

The read path ends in hydration: three batched multi-gets — tweets, authors, engagement counts — not sixty round trips. **Ninety percent of reads are for tweets from the last forty-eight hours**, which is eighteen gigabytes, so a thirty-two gigabyte cache covers almost everything. **At this scale, moving the hit rate from ninety to ninety-five percent halves the backing store's load** — worth more than most architectural changes.

Storage, and one number that surprises people: tweets are the smallest thing here. Three hundred bytes each, thirty million a day, is 3.3 terabytes a year — ten years is about a hundred terabytes. **The timelines are four terabytes and the graph is two.** The derived data is larger than the content, and that is what fan-out costs.

Two honest closing points.

This is Twitter around 2013, not Twitter now. The real system ranks the timeline, interleaves ads, threads conversations, and recommends tweets from people you do not follow. **Ranking in particular changes the read path** — several hundred candidates fetched and scored — and makes pagination much harder, because the cursor has to pin a ranking session or page two repeats page one.

And I would not build this for a new product. Below about a million users, a `follows` table and a query with a join is the entire home timeline, and it is fast and correct and one person can maintain it. **Everything here is what you build when that stops working, and knowing when that is is more valuable than knowing how to build it.**

9 Recall card

Scope out loud first (in: tweet, follow, both timelines, search, trending; out: DMs, lists, ads, media, moderation) — **that choice is a large part of what is assessed.** Then size: **3B reads / 30M writes a day = 100:1**, which justifies fan-out on write.

Two timelines, built differently. User timeline is just a query on `(author_id, tweet_id)` — do not precompute it. **Home timeline is precomputed** into a Redis sorted set per user, capped ~800, **with the Snowflake id as both member and score** so it needs no timestamp and merging is free.

Hybrid fan-out at ~10,000 followers. A 100M-follower account = ~33 minutes of inserts, and an 800 MB follower id list that cannot be fetched at once — which is why the graph service streams a paged cursor. **Store the graph BOTH ways** (by follower and by followee, ~2 TB total), so both questions are single-shard; the copies reconcile asynchronously.

Search needs a different index entirely — inverted, keyed by term, **sharded by time** so a hot in-memory index over recent tweets answers most queries. **Asynchronous, so near-real-time:** putting it on the write path would make a tweet fail when the index is slow.

Trending is rate of change, not rate — ranking by count returns “the” every day. **Count-min sketch** (min across rows, never undercounts, 2.6 MB) scored against a **seasonal baseline**; the killer property is that **sketches merge across shards by addition**.

Hydrate in 3 batched multi-gets (~10 ms end to end). **Tweets are the smallest data here** — 3.3 TB/year against 4 TB of timelines and 2 TB of graph. **And below ~1M users this is a follows table and a join** — building the fan-out first is the mistake the question tempts you into.

PRACTICE Practice — Edit distance

DSA topic: Edit distance **System design topic:** Design Twitter

Code these, in this order

One rule for the whole set: **write the base-case loops before the recurrence, and write the name of each branch as a comment on the line.** The base cases are not zeros here, and the delete/insert confusion is invisible to every test that checks only the number.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Edit Distance	LeetCode 72 (Medium)	Three branches, named, and the non-zero base cases.
2	One Edit Distance	LeetCode 161 (Medium)	Where the table is overkill and two pointers win.
3	Delete Operation for Two Strings	LeetCode 583 (Medium)	Edit distance with replace removed.
4	Minimum ASCII Delete Sum	LeetCode 712 (Medium)	Weighted costs, and what happens to the base cases.
5	Regular Expression Matching	LeetCode 10 (Hard)	The same table shape with a different move set.
6	Wildcard Matching	LeetCode 44 (Hard)	The same again, and where greedy beats the table.

On problem 1, leave the base cases out

Allocate the table as all zeros and skip both base-case loops. Run on `"horse"` / `"ros"` and record the answer. **Say why it is too small and why a small wrong number is worse than a crash.**

On problem 1, add one on a match

Change the match branch to `dp[i-1][j-1] + 1`. Run `edit_distance("abc", "abc")` and record the result. **Say what that number claims, in words.**

On problem 1, swap delete and insert

Swap the two branches. Run on five random string pairs and record whether any answer changes. **Then write the reconstruction and run it.** Say exactly where the bug becomes visible, and why no distance test could have caught it.

On problem 1, reconstruct with `and` instead of `or`

Write the walk-back with `while i > 0 and j > 0`. Run it on `""` → `"abc"` and record the script. **Say what was lost and why.**

On problem 1, collapse to two rows

Write it, then deliberately leave `current` as `[0] * (m+1)` instead of `[i] + [0] * m`. Run on `"horse"` / `"ros"` and record the answer. **Say which base case you just deleted.**

On problem 4, generalise carefully

Convert your uniform-cost version to weighted. **Note what happens to the two base-case loops** — write down what they become and why. Then say what property of the distance you have just lost.

On problems 5 and 6, keep the state and change the moves

Before coding either, write `dp[i][j]` as a sentence. Then list the moves available at each cell. **Notice the state is identical to edit distance and only the move set changed.**

Then the fuzzy-search drill

Build a list of ten thousand words. Time: (a) computing full edit distance against all of them; (b) adding the length filter; (c) adding the row-minimum early exit. Record all three. **Then say what a BK-tree or a trie would change, and by roughly how much.**

The base-case drill

1. Give both base cases and say what each means in words.
2. Say why they are not zeros, and where that habit comes from.
3. Say what the answer looks like when they are missing.
4. Say what they become when the costs are weighted.

The branch drill

1. Name all three operations and give the cell each reads.

2. For each, say which index moves and why.
3. State the rule in one sentence.
4. Say what the match branch does and what it does not do.
5. Say why swapping delete and insert passes every distance test.

The reconstruction drill

1. Say where the walk-back starts and stops.
2. Give the four cases in order.
3. Say why the loop condition is `or`.
4. Say what the branch order decides.
5. Say what reconstruction costs in time and in space.

The variants drill

1. Damerau-Levenshtein: the extra operation, the condition, and why it matters.
2. Weighted: what changes and what property is lost.
3. Delete-only: the formula in terms of LCS.
4. Banded: what it is for and when it is valid.

The fuzzy-search drill

1. Compute the cost of comparing one query against 100,000 words.
2. Give the two cheap filters and roughly what each saves.
3. Explain a BK-tree in two sentences.
4. Explain the trie approach in two sentences.
5. Say which you would pick for a search box and why.
6. Say what you would rank the survivors by, and why not distance alone.

The break-it drill

Trigger each and record the exact output or error:

1. Missing base cases.
2. `+ 1` in the match branch.
3. Swapped delete and insert, checked only by distance.
4. `while i > 0 and j > 0` in the walk-back.
5. Missing `[i]` in the collapsed version's row start.
6. Two 100,000-character strings.
7. Two Unicode strings that look identical.

Six of the seven give no error at all. Name them.

The scoping drill

1. Give your in-scope and out-of-scope lists.
2. Say why saying the out list matters.
3. Say what you would cut first if given ten minutes.

The timeline drill

1. Compute the read-to-write ratio and say what it justifies.
2. Say how the user timeline is built and why nothing clever is needed.
3. Say why the home timeline cannot be a query.
4. Say what the Snowflake id buys you, in three separate ways.
5. Give the Redis structure, the cap, and what the score is.

The celebrity drill

1. Do the 100-million-follower fan-out arithmetic.
2. Compute the size of that follower id list.
3. Say what that size implies about the graph service's API.
4. Describe the hybrid and why the read-side merge is cheap.
5. Describe the partial-fanout refinement.

The graph drill

1. Say which two questions the graph must answer.
2. Give both sharding options and what each makes fast and slow.
3. Say what you do and what it costs.
4. Say what the consistency cost is, concretely.

The search drill

1. Say why the timeline index cannot serve search.
2. Describe an inverted index in one sentence.
3. Say how a two-term query is answered.
4. Say why you bound the posting lists.
5. Say why shard by time and not by term.
6. Say why indexing is asynchronous, and what the alternative would cost.

The trending drill

1. Say what ranking by raw count produces.
2. State what trending actually measures.
3. Say why the baseline must be seasonal.
4. Explain a count-min sketch, including why `min`.
5. Give the memory comparison against exact counting.
6. Say the property that makes sketches the right choice, not just a smaller one.
7. Say what the sketch does *not* give you, and what you keep alongside it.

The sizing drill

1. Give storage for tweets, timelines, graph and search index.
 2. Say which is the smallest and why that is surprising.
 3. Give the read-path latency breakdown.
 4. Compute the cache size for 48 hours of tweets.
 5. Say what moving the hit rate from 90% to 95% does.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the minimum number of edits to turn one string into the other.* The state, the non-zero base cases with their meanings, the free match, all three branches named with the index rule, and the complexity.
 2. *Now show me the actual edits.* The walk-back with its four cases, the `or` condition and what it saves, the tie-break, and where the swapped-branch bug finally becomes visible.
 3. *Design Twitter.* The scope, the 100:1 ratio, both timelines and why they differ, the celebrity arithmetic including the 800 MB, and why search and trending need different structures.
-

Before you move on

- [] I write the base-case loops first, and they are not zeros.
- [] I know what each base case means in words.
- [] I know a match is free, with no `+1`.

- [] I can name all three branches and which index each moves.
- [] I can state the index rule in one sentence.
- [] I know why swapping delete and insert passes every distance test.
- [] I can reconstruct with the four cases.
- [] I know the loop condition is `or` and what `and` loses.
- [] I know the branch order picks among equal-cost scripts.
- [] I can collapse to two rows, including the row-start base case.
- [] I know the collapse gives up reconstruction.
- [] I can generalise to weighted costs, including the base cases.
- [] I know weighted costs break symmetry, and why that matters.
- [] I know what Damerau adds and why spell-checkers use it.
- [] I can give time and space, and where the table stops being viable.
- [] I know a spell-checker does not call this 100,000 times.
- [] I can give both cheap filters and explain a BK-tree or a pruning trie.
- [] I would rank suggestions by frequency, not distance alone.
- [] I can scope Twitter out loud, in and out.
- [] I can compute the read-to-write ratio.
- [] I know the user timeline is a plain query.
- [] I know why the home timeline cannot be one.
- [] I can give three things the Snowflake id buys.
- [] I can do the 100-million-follower arithmetic, both numbers.
- [] I know why that forces a streaming cursor in the graph API.
- [] I can give both graph sharding options and say what I do.
- [] I know why search needs a separate inverted index.
- [] I know why it shards by time and is asynchronous.
- [] I know trending measures change, not volume, and why the baseline is seasonal.
- [] I can explain a count-min sketch and why sketches merge.
- [] I answered all three questions above out loud.

String DP: palindromic substrings and subsequences

DATA STRUCTURES AND ALGORITHMS

String DP: palindromic substrings

SYSTEM DESIGN

Design Instagram

1 What this is, and why they ask it

A **palindrome** reads the same forwards and backwards. "racecar", "abba", "a". The questions are: how many palindromic substrings does a string contain, and what is the longest one?

They ask it because **it is the first DP where the table is not filled row by row**. Everything so far has gone left to right, top to bottom. Here, `dp[i][j]` depends on `dp[i+1][j-1]` — a cell below and to the left — so filling in the natural order reads cells that have not been computed yet. **You have to fill by substring length, shortest first**, and working out that you must is the lesson.

The other reason is that **the best solution is not the DP at all**. Expand-around-centre uses no table, is five lines shorter, runs in the same $O(n^2)$ time and uses $O(1)$ space instead of $O(n^2)$. **A candidate who writes the table and then says “actually there is a better way” is demonstrating exactly the judgement being tested** — and a candidate who only knows the table will be asked to reduce the space and will not be able to.

And there is a third solution, Manacher’s algorithm, which is genuinely $O(n)$. **You will not be asked to write it**. You may well be asked whether you know it exists, and the right answer is a sentence.

By the end of this lesson you can write the interval DP and explain the fill order, write expand-around-centre and say why it is better, handle the even/odd centre problem, count palindromic substrings, and name where each approach stops.

2 The story

Ilyas painted signboards, and the man who came in on Tuesday wanted his shop name painted on both sides of a board that would hang out over the street.

Both sides. So people walking up the road and people walking down it would both see it.

Ilyas said that was fine and it would cost double, because it was two paintings.

And the man, who had clearly thought about this, said no it would not, because he had chosen the name so that it did not need two.

He wrote it out. **MALAYALAM.**

Ilyas looked at it for a while, and then did the thing he had done for twenty years without ever having a name for it. **He put a finger at each end of the word and moved them inwards.**

M and M. Same.

A and A. Same.

L and L. Same.

A and A. Same.

And the Y in the middle, on its own, with nothing to match against.

“It works,” he said.

Then, because it was Tuesday afternoon and there was nothing else to do, they started trying other names, and this is where it got interesting.

Because the man’s brother’s shop was called **KANAKA**, and Ilyas put his fingers on the ends — K and A, no — and said it did not work. **But the man pointed out that ANA in the middle of it did.** And so did KANAK if you dropped the last letter.

So one word had bits inside it that worked, even though the whole thing did not.

They spent about an hour on it. And the method they ended up with, without discussing it, was this: **check the little ones first.** Every single letter works on its own, obviously. Then every pair — same letter twice or nothing. **And then, for anything longer, you only had to check the two ends, because you had already worked out whether the bit inside was good.**

Ilyas’s wife, who came in at five and was told about the afternoon, said the obvious thing.

“Why are you starting from the ends? Start in the middle and walk outwards. That is where it is either going to work or not.”

3 The idea in plain English

Ilyas has just described the DP, his wife has just described expand-around-centre, and she is right.

First the definitions. A **substring** is contiguous — "aba" is a substring of "xabay", but "aa" is not. That matters, because a **subsequence** may skip and gives a different answer to almost every question here.

The DP first, because it is what gets asked.

`dp[i][j]` is `True` if the substring from index `i` to index `j` inclusive is a palindrome.

The recurrence is Ilyas's method, exactly:

```
dp[i][j] = (s[i] == s[j]) AND dp[i+1][j-1]
```

The two ends must match, and the bit inside must already be a palindrome. That is the whole rule.

And the base cases are the small ones he did first. Every single character is a palindrome: `dp[i][i] = True`. Every pair is a palindrome exactly when the two characters are equal: `dp[i][i+1] = (s[i] == s[i+1])`.

Now the part that is new: the fill order.

`dp[i][j]` reads `dp[i+1][j-1]` — a larger `i` and a smaller `j`. **In a table filled left to right, top to bottom, that cell has not been computed yet.** Fill in the natural order and you read a `False` that means "not yet computed" rather than "not a palindrome", and every long palindrome is missed.

No error. Just answers that are too small.

Two fill orders work, and both are worth knowing:

- **By length.** Handle all substrings of length 1, then all of length 2, then 3, and so on. **When you reach length `L`, everything of length `L-2` is done.** This is the version to write, because the reason is visible in the loop.
- **By decreasing `i`.** Loop `i` from `n-1` down to 0, and `j` from `i` upwards. Then `dp[i+1][...]` — the row below — is always already complete. **Shorter to write, and the reason is less obvious,** which is why the length version is better in an interview.

That is $O(n^2)$ time and $O(n^2)$ space.

And now the better solution, which is Ilyas's wife.

Every palindrome has a centre. Stand at the centre and walk outwards while the characters match. **When they stop matching, you have found the longest palindrome centred there.** Do that for every possible centre and take the best.

The catch is that there are two kinds of centre, and this is the detail that catches people.

An odd-length palindrome has a character at its centre — "aba" is centred on the `b`. **An even-length one has a gap** — "abba" is centred between the two `b`s, on nothing.

So there are $2n - 1$ centres, not n : n characters and $n - 1$ gaps. **Checking only the characters finds every odd palindrome and misses every even one,** which means "abba" reports 2 instead of 4 — and, again, no error.

The cost is the same $O(n^2)$ in time — $2n - 1$ centres, each expanding up to $n/2$ — **and $O(1)$ in space,** because there is no table at all.

So the DP costs n^2 memory and buys nothing. At $n = 1000$ that is a million booleans against four variables. **Say that out loud when you offer the second solution; it is the point.**

Counting palindromic substrings falls out of either. With the DP, count the `True` cells. With expansion, count every successful step outwards — **each one is a distinct palindromic substring,** so incrementing inside the while loop is the entire change.

And the third solution: Manacher's algorithm, which is $O(n)$.

The idea in one sentence: **as you scan left to right, you keep track of the rightmost palindrome found so far, and inside it you can reuse the answer from the mirrored position instead of expanding from scratch.**

Nobody expects you to write it. They may ask whether an $O(n)$ solution exists, and "yes, Manacher's, it reuses mirrored results inside a known palindrome, and I would not write it from memory" is a complete and honest answer.

Finally, the neighbouring problems, because they are easy to confuse.

Longest palindromic *subsequence* — may skip characters — is **not** this. It is `LCS(s, reversed(s))` from [day 153](#), and it is a different table with a different recurrence. "character" has a longest palindromic substring of "carac", length 5, and a longest palindromic *subsequence* of "carac" too — but on "bbbab" the substring is "bbb" (3) and the subsequence is "bbbb" (4).

Palindrome partitioning — the fewest cuts to split a string into palindromes — uses this exact table as a subroutine, which is the most common way the table actually earns its place.

4 The picture

The dependency that forces the fill order:

	j →					
	0	1	2	3	4	
i	+-----+-----+-----+-----+-----+					
0				?		dp[0][2] needs dp[1][1]
	+-----+-----+-----+-----+-----+					
1			X			X is BELOW and to the LEFT
	+-----+-----+-----+-----+-----+					
2						
	+-----+-----+-----+-----+-----+					

Filling left-to-right, top-to-bottom reaches dp[0][2] BEFORE dp[1][1]. It reads False meaning "not computed yet", not "not a palindrome".

- every palindrome longer than 2 is missed
- no error, just answers that are too small

Filling by length, which makes the reason visible:

```
s = "aabaa"

length 1: dp[0][0] dp[1][1] dp[2][2] dp[3][3] dp[4][4] all True
length 2: dp[0][1]='aa' T   dp[1][2]='ab' F
          dp[2][3]='ba' F   dp[3][4]='aa' T
length 3: dp[0][2]='aab': s[0]='a', s[2]='b' → F
          dp[1][3]='aba': s[1]=s[3]='a' AND dp[2][2]=T → T
          dp[2][4]='baa': s[2]='b', s[4]='a' → F
length 4: dp[0][3]='aaba': s[0]='a', s[3]='a' AND dp[1][2]=F → F
          dp[1][4]='abaa': s[1]='a', s[4]='a' AND dp[2][3]=F → F
length 5: dp[0][4]='aabaa': s[0]=s[4]='a' AND dp[1][3]=T → T

answer: longest is "aabaa", length 5
count of True cells = 5 + 2 + 1 + 0 + 1 = 9 palindromic substrings
```

Expand around centre, and the two kinds:

```
s = "aba"
```

ODD centre, at index 1:

```
  a  b  a
    ^
```

```
left=1, right=1
```

```
expand: s[0]='a' == s[2]='a'
→ "aba", length 3
```

```
s = "abba"
```

EVEN centre, between 1 and 2:

```
  a  b  b  a
    ^  ^
```

```
left=1, right=2
```

```
expand: s[1]='b' == s[2]='b'  ok
→ "bb", length 2
```

```
expand: s[0]='a' == s[3]='a'  ok
→ "abba", length 4
```

THERE ARE $2n - 1$ CENTRES:

n characters (odd palindromes)

n - 1 gaps (even palindromes)

Checking only characters finds "aba" and reports 2 for "abba".
No error. Just wrong.

The centres, enumerated:

```
s = a  b  b  a
    0  1  2  3
```

```
centres: [0]  [0,1]  [1]  [1,2]  [2]  [2,3]  [3]
```

```
          ^      ^
          char   gap
```

```
          ^
          gap between the two b's
          → this is the one that finds "abba"
```

7 centres for a 4-character string. $2n - 1$.

Counting, which is one line inside the expansion:

```
s = "aaa", expanding from centre 1 (the middle 'a'):
```

```
step 0: "a"    at [1,1]    → count 1
```

```
step 1: "aaa"  at [0,2]    → count 1
```

every successful expansion IS a distinct palindromic substring,
so `count += 1` inside the while loop is the whole change.

total for "aaa":

```
centres [0]: "a"  [0,1]: "aa"  [1]: "a", "aaa"  [1,2]: "aa"  [2]: "a"
= 6 palindromic substrings
```

The three approaches, compared:

	time	space	lines	would I write it?
interval DP	$O(n^2)$	$O(n^2)$	~12	if partitioning too
expand around centre	$O(n^2)$	$O(1)$	~10	YES – default
Manacher	$O(n)$	$O(n)$	~30	no, but I'd name it

The DP costs n^2 memory and buys nothing over expansion.
At $n = 1,000$ that is 1,000,000 booleans against 4 variables.

5 The code, built step by step

The interval DP, filled by length

```
def longest_palindrome_dp(s: str) → str:
    n = len(s)
    if n < 2:
        return s
    dp = [[False] * n for _ in range(n)]
    for i in range(n):
        dp[i][i] = True                    # every character alone
    start, best = 0, 1
```

Length-1 first, because everything longer depends on it. **best = 1** because any non-empty string has a palindrome of at least one character.

```
    for i in range(n - 1):                # length 2
        if s[i] == s[i + 1]:
            dp[i][i + 1] = True
            start, best = i, 2
```

Length 2 is a separate base case, because `dp[i+1][j-1]` for a length-2 substring would be `dp[i+1][i]` — an empty range, with `i+1 > j-1` — and reading it is meaningless.

```
    for length in range(3, n + 1):        # THEN by increasing length
        for i in range(n - length + 1):
            j = i + length - 1
            if s[i] == s[j] and dp[i + 1][j - 1]:
                dp[i][j] = True
                if length > best:
                    start, best = i, length
    return s[start:start + best]
```

`for length in range(3, n+1)` is the whole point. When you compute a substring of length `L`, everything of length `L-2` is already done — which is exactly what `dp[i+1][j-1]` is.

Expand around centre, which is what to write

```
def expand(s: str, left: int, right: int) → tuple[int, int]:
    """Widen while the ends match. Returns the widest [start, end) found."""
    while left ≥ 0 and right < len(s) and s[left] == s[right]:
        left -= 1
        right += 1
    return left + 1, right          # step back inside the last match
```

`left + 1` and `right` because the loop overshoots by one on each side before it stops — that off-by-one is the only fiddly part, and returning a half-open range makes the arithmetic clean.

```
def longest_palindrome(s: str) → str:
    if len(s) < 2:
        return s
    start, end = 0, 1
    for centre in range(len(s)):
        for lo, hi in (expand(s, centre, centre),      # odd: on a character
                       expand(s, centre, centre + 1)): # even: on the gap
            if hi - lo > end - start:
                start, end = lo, hi
    return s[start:end]
```

Two expansions per index — one odd, one even — is the `2n - 1` centres. Doing only the first finds every odd palindrome and misses every even one, and `"abba"` returns `"bb"`.

`O(1)` space. No table.

Counting palindromic substrings

```
def count_palindromic_substrings(s: str) → int:
    total = 0
    for centre in range(len(s)):
        for left, right in ((centre, centre), (centre, centre + 1)):
            while left ≥ 0 and right < len(s) and s[left] == s[right]:
                total += 1          # each expansion is one substring
                left -= 1
                right += 1
    return total
```

`total += 1` inside the while loop is the entire difference from the previous function. Every successful step outwards is one more distinct palindromic substring.

Palindrome partitioning, where the table earns its place

```
def min_cut(s: str) → int:
    n = len(s)
    is_pal = [[False] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):          # decreasing i: row below is done
        for j in range(i, n):
            if s[i] == s[j] and (j - i < 2 or is_pal[i + 1][j - 1]):
                is_pal[i][j] = True

    cuts = [0] * n
    for j in range(n):
        if is_pal[0][j]:
            cuts[j] = 0                    # the whole prefix is a palindrome
        else:
            cuts[j] = min(cuts[i - 1] + 1 for i in range(1, j + 1) if is_pal[i][j])
    return cuts[n - 1]
```

This is why you learn the table. Expansion gives you the longest palindrome; it **does not** give you constant-time “is `s[i..j]` a palindrome?”, and partitioning asks that $O(n^2)$ times.

Note the alternative fill order here — `i` decreasing — with `j - i < 2` folding both base cases into one condition.

The complete solution

```
"""Palindromic substrings: three approaches, and where each one belongs."""

def expand(s: str, left: int, right: int) → tuple[int, int]:
    """Widen while the ends match. Returns a half-open [start, end)."""
    while left ≥ 0 and right < len(s) and s[left] == s[right]:
        left -= 1
        right += 1
    return left + 1, right          # the loop overshoots by one

def longest_palindrome(s: str) → str:
    """Expand around centre.  $O(n^2)$  time,  $O(1)$  space. The default answer."""
    if len(s) < 2:
        return s
    start, end = 0, 1
    for centre in range(len(s)):
        for lo, hi in (expand(s, centre, centre),          # odd-length
                       expand(s, centre, centre + 1)):      # even-length
            if hi - lo > end - start:
                start, end = lo, hi
    return s[start:end]

def count_palindromic_substrings(s: str) → int:
    """Every successful expansion is one distinct palindromic substring."""
    total = 0
    for centre in range(len(s)):
        for left, right in ((centre, centre), (centre, centre + 1)):
            while left ≥ 0 and right < len(s) and s[left] == s[right]:
                total += 1
                left -= 1
                right += 1
    return total

def longest_palindrome_dp(s: str) → str:
    """The interval DP.  $O(n^2)$  space, filled by INCREASING LENGTH."""
    n = len(s)
    if n < 2:
        return s
    dp = [[False] * n for _ in range(n)]
    for i in range(n):
        dp[i][i] = True          # length 1
        start, best = 0, 1
        for i in range(n - 1):
            # length 2 – its own base case
            if s[i] == s[i + 1]:
                dp[i][i + 1] = True
                start, best = i, 2
        for length in range(3, n + 1):          # then longest last
            for i in range(n - length + 1):
                j = i + length - 1
                if s[i] == s[j] and dp[i + 1][j - 1]:
                    dp[i][j] = True
                    if length > best:
                        start, best = i, length
    return s[start:start + best]

def palindrome_table(s: str) → list[list[bool]]:
```

```

"""is_pal[i][j] for every pair. Filled with DECREASING i."""
n = len(s)
is_pal = [[False] * n for _ in range(n)]
for i in range(n - 1, -1, -1):
    # row i+1 is already complete
    for j in range(i, n):
        if s[i] == s[j] and (j - i < 2 or is_pal[i + 1][j - 1]):
            is_pal[i][j] = True
return is_pal

def min_cut(s: str) → int:
    """Fewest cuts so every piece is a palindrome. Needs the TABLE, not expansion."""
    n = len(s)
    if n < 2:
        return 0
    is_pal = palindrome_table(s)
    cuts = [0] * n
    for j in range(n):
        if is_pal[0][j]:
            cuts[j] = 0
        else:
            cuts[j] = min(cuts[i - 1] + 1 for i in range(1, j + 1) if is_pal[i][j])
    return cuts[n - 1]

def longest_palindromic_subsequence(s: str) → int:
    """NOT the same problem: a subsequence may skip. This is LCS(s, reversed(s))."""
    t = s[::-1]
    n = len(s)
    dp = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, n + 1):
            if s[i - 1] == t[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[n][n]

if __name__ == "__main__":
    for word in ["malayalam", "kanaka", "babad", "cbdd", "abba", "a", ""]:
        print(f"{word!r:12} longest: {longest_palindrome(word)!r:12}"
              f" count: {count_palindromic_substrings(word)}")

    print("dp agrees on babad :", longest_palindrome_dp("babad"))
    print("dp agrees on abba  :", longest_palindrome_dp("abba"))

    print("odd-only on 'abba' :", max(
        (expand("abba", c, c) for c in range(4)), key=lambda p: p[1] - p[0]))

    print("min cuts 'aab'      :", min_cut("aab"))
    print("min cuts 'abccbaa' :", min_cut("abccbaa"))

    print("substring 'bbbab' :", longest_palindrome("bbbab"))
    print("subsequence 'bbbab':", longest_palindromic_subsequence("bbbab"))

```

Run it and you get:

```

'malayalam' longest: 'malayalam' count: 15
'kanaka'     longest: 'kanak'    count: 9
'babad'     longest: 'bab'      count: 7
'cbbd'     longest: 'bb'       count: 5
'abba'     longest: 'abba'     count: 6
'a'        longest: 'a'        count: 1
''         longest: ''         count: 0
dp agrees on babad : bab
dp agrees on abba  : abba
odd-only on 'abba' : (0, 1)
min cuts 'aab'    : 1
min cuts 'abccbaa': 1
substring 'bbbab' : bbb
subsequence 'bbbab': 4

```

odd-only on 'abba' returning a length-1 result is the even-centre bug, made visible — without the second expansion, the longest palindrome in "abba" is reported as a single character.

And the last two lines are the substring/subsequence distinction: "bbb" is 3 as a substring, and the subsequence "bbbb" is 4.

6 What it costs

Expand around centre.

```

centres: 2n - 1
each expansion: at most n/2 steps

worst case ("aaaa...a", all identical):
  2n - 1 centres x n/2 steps = O(n^2)

typical case (random text):
  most expansions stop after 1-2 steps
  → close to O(n) in practice

```

Concretely:

n = 1,000	worst case ~1,000,000 steps	~0.2 s
n = 10,000	worst case ~100,000,000	~20 s
n = 100,000	worst case ~10 ¹⁰	impossible

on ordinary English text, n = 100,000 finishes in well under a second, because expansions terminate almost immediately.

That gap between worst case and typical is worth mentioning: the quadratic bound is real and almost never reached outside adversarial input.

Space: $O(1)$. Four integers. **No allocation at all**, which is the whole argument.

The interval DP.

```
time:  n^2 / 2 cells, O(1) each      = O(n^2)   - same as expansion
space: n x n booleans                = O(n^2)   - and this is the problem

n = 1,000:  1,000,000 booleans
             Python list of lists: ~8 bytes per pointer + shared bool objects
             → about 40 MB

against expansion's 4 integers.
```

```
n = 10,000: 100,000,000 cells → ~4 GB
             MemoryError before it starts
```

Same time, n^2 more memory, and no additional answer. That comparison is the whole reason to offer expansion.

Where the table does earn its place:

```
palindrome partitioning: needs "is s[i..j] a palindrome?" answered
                        O(n^2) times

with the table:        O(1) per question, after O(n^2) to build
with expansion:        O(n) per question → O(n^3) overall

n = 1,000:  10^6 with the table, 10^9 without.
```

Counting palindromic substrings:

```
same expansion, one increment inside the loop
→ O(n^2) worst case, O(1) space, identical constants

the DP version counts True cells: O(n^2) time AND space
→ the same answer for n^2 more memory
```

Manacher's algorithm:

```
0(n) time, 0(n) space
```

```
n = 1,000,000:
```

```
Manacher: ~1,000,000 steps      ~0.5 s  
expansion: worst case ~10^12    impossible  
            typical text ~2,000,000 ~1 s
```

→ Manacher wins decisively only on ADVERSARIAL input at large n .
On real text, expansion is already near-linear.

That is the honest framing, and it is why nobody makes you write Manacher: the case where it matters is rare.

The subsequence version, for contrast:

```
longest palindromic SUBSEQUENCE = LCS(s, reversed(s))  
0(n^2) time, 0(n^2) space (or 0(n) with two rows)
```

and there is no expand-around-centre equivalent, because
a subsequence has no contiguous centre to expand from.

7 The traps

Filling the table in the natural order.

```
>>> s = "aabaa"  
>>> n = 5  
>>> dp = [[False] * n for _ in range(n)]  
>>> for i in range(n):  
...     dp[i][i] = True  
>>> for i in range(n):  
...     for j in range(i + 1, n):  
...         dp[i][j] = s[i] == s[j] and (j - i == 1 or dp[i+1][j-1])  
>>> dp[0][4]  
False
```

"aabaa" is a palindrome and the table says it is not. `dp[0][4]` needs `dp[1][3]`, which is computed later in this order, so it reads `False` meaning "not yet". **No error, an answer that is too small**, and it only shows on palindromes longer than three.

Missing the even centres.

```
>>> max((expand("abba", c, c) for c in range(4)), key=lambda p: p[1] - p[0])  
(0, 1)
```

One character, for a string that is entirely a palindrome. $2n - 1$ centres, not n — and the version that only checks characters passes every test whose answer happens to be odd-length.

The off-by-one when the expansion stops.

```
>>> def bad_expand(s, left, right):
...     while left ≥ 0 and right < len(s) and s[left] == s[right]:
...         left -= 1
...         right += 1
...     return left, right                                # forgot the step back
>>> bad_expand("aba", 1, 1)
(-1, 3)
>>> "aba"[-1:3]
'a'
```

The loop always overshoots by one on each side before the condition fails. Return `left + 1, right` — and note that the negative index does not raise, it silently slices from the end.

Confusing substring with subsequence.

```
>>> longest_palindrome("bbbab")
'bbb'
>>> longest_palindromic_subsequence("bbbab")
4
```

3 against 4. "bbbbb" is a valid subsequence — skipping the `a` — and not a substring. Read the problem statement for the word “contiguous” or “substring”, because the two answers differ and both look right.

Treating length 2 with the general rule.

```
>>> # dp[i][j] = s[i] == s[j] and dp[i+1][j-1]
>>> # for j = i + 1, that is dp[i+1][i] — an empty range, i+1 > i
>>> dp = [[False] * 3 for _ in range(3)]
>>> dp[2][1]                                             # never set, and meaningless
False
```

`dp[i+1][j-1]` with `j = i+1` reads a cell where the start is past the end. It is `False` by default, so "aa" is reported as not a palindrome. **Either handle length 2 as its own base case, or use the `j - i < 2` or ... guard** — both work, and the guard is shorter.

Case and punctuation, which is the real-world version.

```
>>> longest_palindrome("A man, a plan, a canal: Panama")
'a '
```

Correct for the string as given, and not what anyone means. The classic palindrome question normalises first — lowercase, keep only alphanumerics — and **doing that unasked is wrong too**. Ask.

Very long strings with the DP.

```
>>> dp = [[False] * 50000 for _ in range(50000)]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
MemoryError
```

2.5 billion cells. Expansion handles the same input in constant space, which is why the constraint on `n` should decide the approach before you start.

The empty string.

```
>>> longest_palindrome("")
''
>>> count_palindromic_substrings("")
0
```

Both correct and both free, because the guard returns early and the loop does not execute. The DP version without its `if n < 2` guard would raise on `s[start:start+best]` with `best = 1` — worth checking.

8 In the interview

How it gets asked

- “Find the longest palindromic substring.” — LeetCode 5, one of the most-asked problems anywhere.
- “Count the palindromic substrings.” — LeetCode 647.
- “Can you do it in `O(1)` space?” — the follow-up that expansion answers and the DP cannot.
- “Is there a linear solution?” — the Manacher question.
- “What is the fewest cuts to split it into palindromes?” — where the table is required.
- “How is this different from the longest palindromic subsequence?”

“I would offer two solutions, because the obvious DP is not the one I would write, and the reason is worth stating.

The DP first. $dp[i][j]$ is true if the substring from i to j is a palindrome. **The recurrence is: the two ends match and the bit inside is already a palindrome** — $s[i] = s[j]$ and $dp[i+1][j-1]$. Base cases: every single character is a palindrome, and a pair is one when the characters are equal.

The interesting part is the fill order, and this is the thing that is new here. $dp[i][j]$ depends on $dp[i+1][j-1]$, which is below and to the left — so if I fill left to right, top to bottom, I read a cell that has not been computed yet. It is `False` meaning ‘not yet’, not `False` meaning ‘not a palindrome’, and every long palindrome is silently missed.

So I fill by increasing substring length — all the length-1s, then all the length-2s, then 3 — because when I compute a substring of length L , everything of length $L-2$ is already done. **That is exactly what the recurrence reads.**

$O(n^2)$ time and $O(n^2)$ space.

And now the better solution, which I would actually write. Every palindrome has a centre. Stand at the centre, walk outwards while the characters match, and when they stop you have the longest palindrome centred there. Do it for every centre.

The catch is that there are two kinds of centre and it is easy to miss one. An odd-length palindrome is centred on a character; an even-length one is centred on the gap between two characters. **So there are $2n - 1$ centres, not n** — and checking only the characters reports “abba” as a single letter, with no error.

Same $O(n^2)$ time, and $O(1)$ space — four variables instead of a million booleans at $n = 1,000$. So the DP costs n^2 memory and buys nothing.**

The one case where I would build the table anyway is palindrome partitioning, where I need constant-time ‘is this range a palindrome’ answered $O(n^2)$ times — with expansion that would be $O(n^3)$.

And there is an $O(n)$ algorithm, Manacher’s. I know what it does — it reuses mirrored results inside a known palindrome — and I would not write it from memory unless you want it.

One question before I code: substring or subsequence? They are different problems with different answers.”

The follow-ups

“Why can’t you fill the table in the normal order?”

“Because of which cell the recurrence reads, and this is the first time in dynamic programming that the natural loop order is wrong.

`dp[i][j] = s[i] == s[j] and dp[i+1][j-1]`. That dependency is at a **larger i** and a **smaller j** — one row down, one column left.

In a left-to-right, top-to-bottom fill, I reach `dp[0][4]` in the very first row, and `dp[1][3]` is in the second row and has not been computed. So I read the initialised `False`, which means ‘not yet’, and treat it as ‘not a palindrome’.

Concretely, on `"aabaa"`: `dp[0][4]` is the whole string, which is a palindrome, and the natural order reports `False`. No exception, no warning — the answer is just too small, and only for palindromes longer than three, which is exactly the case a short test would not cover.

The fix is to fill by increasing substring length. All substrings of length 1, then length 2, then 3. When I compute a substring of length `L`, everything of length `L-2` is finished — which is precisely what `dp[i+1][j-1]` is, since it is two characters shorter.

There is a second fill order that also works: iterate `i` from `n-1` down to 0, and `j` from `i` upwards. Then row `i+1` is always complete when I need it. It is shorter to write and the reason is less obvious, so I would use the length version in an interview and mention the other.

The general lesson is worth naming, because it recurs in every interval DP — matrix chain multiplication, burst balloons, optimal binary search trees. The fill order must follow the dependency direction, and when the state is a range, that means shortest ranges first.“

“Can you do it in `O(1)` space?”

“Yes, and that solution is also simpler than the table — which is why I would offer it first if I had not already.

Every palindrome has a centre. Stand there and walk outwards while the characters match. When they stop matching, or I run off either end, I have the longest palindrome centred at that point. Do that for every centre and keep the best.

Two kinds of centre, and this is the part that catches people. “aba” is centred on the `b` — a character. “abba” is centred between the two `b`s — a gap, on nothing at all.

So the loop does two expansions per index: one starting at `(i, i)` for odd lengths, one at `(i, i+1)` for even. That is $2n - 1$ centres — `n` characters and `n-1` gaps.

Missing the even case is the classic bug, and it is invisible on any test whose answer happens to be odd-length. On “abba” the odd-only version returns a single character.

The other fiddly bit is the off-by-one when the expansion stops. The while loop always overshoots by one on each side before the condition fails, so the answer is `left + 1` to `right`, not `left` to `right`. **And in Python a negative `left` does not raise — it silently slices from the end of the string,** so the bug gives a wrong answer rather than an error.

Cost: same $O(n^2)$ worst case — $2n-1$ centres, each expanding up to `n/2` — and $O(1)$ space, four integers.

Worth adding: the worst case needs adversarial input. On a string of a thousand identical characters every expansion runs the full width. **On ordinary text expansions stop after one or two steps,** so it is close to linear in practice, and `n = 100,000` finishes in well under a second.

And counting palindromic substrings is the same code with one extra line — `count += 1` inside the while loop, because every successful step outwards is one more distinct palindromic substring.”

“How is this different from the longest palindromic subsequence?”

“Different problem, different algorithm, different answer — and the words are close enough that I would confirm which one is being asked before writing anything.

A substring is contiguous. A subsequence may skip characters.

On "bbbbab" : the longest palindromic substring is "bbb" , length three. The longest palindromic **subsequence** is "bbbb" , length four — take the three `b` s at the start and the one at the end, skipping the `a` . **Both are correct answers to different questions.**

The algorithms have nothing in common. For the substring I expand around centres, because a substring has a physical middle and grows outwards from it. A **subsequence has no contiguous centre**, so there is nothing to expand from and that approach simply does not apply.

The subsequence version is `LCS(s, reversed(s))` . That reduction is worth being able to justify: a palindrome reads the same forwards and backwards, so a **subsequence that appears in both `s` and its reverse is exactly a palindromic subsequence**. Run the standard longest-common-subsequence table and the answer falls out.

Cost: $O(n^2)$ time and $O(n^2)$ space, or $O(n)$ space with two rows — but then no reconstruction.

There is also a direct interval DP for it — `dp[i][j]` is the longest palindromic subsequence within `s[i..j]` , with `dp[i][j] = dp[i+1][j-1] + 2` when the ends match and `max(dp[i+1][j], dp[i][j-1])` otherwise. **Same fill-by-length requirement as today's table, and it is the version I would write if I also needed the subsequence itself.**

The tell in the problem statement is the word ‘substring’ or ‘contiguous’. If it is absent and the examples skip characters, it is the subsequence version.”

The model answer

“Given a string, find the longest palindromic substring. Then tell me how you would handle a ten-megabyte input.”

“Let me confirm one thing first, because it changes the answer: **substring, so contiguous — not a subsequence.** "bbbab" gives "bbb", not four.

I would write expand-around-centre, and I want to say why rather than just writing it.

The textbook answer is an interval DP where `dp[i][j]` says whether `s[i..j]` is a palindrome, with the rule that the two ends match and the inside is already a palindrome. **It has to be filled by increasing substring length,** because the recurrence reads a cell below and to the left, and the natural loop order reads it before it exists — silently, producing answers that are too small.

That is $O(n^2)$ time and $O(n^2)$ space. Expansion is $O(n^2)$ time and $O(1)$ space, and it is shorter. So the table costs a million booleans at `n = 1,000` and buys nothing.

The algorithm: every palindrome has a centre, so stand at each centre and walk outwards while the ends match. Two expansions per index — one on the character for odd lengths, one on the gap for even — **$2n - 1$ centres, and missing the even ones is the classic bug that reports "abba" as a single letter.**

Now the ten megabytes, which is the real question.

Ten million characters. The worst case for expansion is $O(n^2)$ — that is 10^{14} steps, which is not happening. So the first thing I would establish is **what the data actually looks like**, because the worst case needs adversarial input.

On ordinary text, expansions terminate after one or two steps almost everywhere, so the real cost is close to $O(n)$ and ten megabytes finishes in a few seconds. **On a file of ten million identical characters, it does not finish at all.** Those are wildly different situations and the constraint alone does not tell me which I have.

If the input can be adversarial, the answer is Manacher's algorithm, which is genuinely $O(n)$. **The idea is that as you scan left to right you track the rightmost palindrome found so far, and for a position inside it you can reuse the result from the mirrored position instead of expanding from scratch — so total work is linear. I would look it up rather than write it from memory, and I would say that plainly rather than produce a half-remembered version, because it is exactly the kind of code that is subtly wrong.**

Two engineering points that matter more than the algorithm at this size.

Memory. Ten megabytes of text is fine; the DP table for it would be 10^{14} cells, so the table approach is not merely slower, it is impossible — which settles the choice before any timing argument.

And streaming. If the input arrives as a stream rather than a string, none of these work as written, because both need random access backwards. A palindrome can span any distance, so there is no bounded window that guarantees correctness — I would have to buffer, and I would want to know the maximum plausible palindrome length to size that buffer, or accept that I only find palindromes shorter than it.

And I would ask what the answer is for. If it is for finding repeated structure in DNA, the biologists want *all* long palindromes above a threshold, not the single longest — which is a different output and changes the loop, though not the approach. If it is a puzzle, the longest is what is wanted. That question costs ten seconds and can change the whole shape of the work.“

9 Recall card

`dp[i][j]` is true when `s[i] == s[j]` AND `dp[i+1][j-1]` — the ends match and the inside is already a palindrome. The dependency is below-and-left, so the natural fill order reads cells that do not exist yet and silently misses every palindrome longer than 3. Fill by increasing substring length (or with `i` decreasing). Handle length 2 separately, or guard with `j - i < 2`.

Expand around centre is the answer to write: same $O(n^2)$ time, $O(1)$ space, fewer lines. The DP costs n^2 memory and buys nothing — a million booleans against four integers at `n = 1,000`.

There are `2n - 1` centres, not `n` — `n` characters (odd palindromes) and `n - 1` gaps (even). Checking only characters reports "abba" as one letter, with no error. The expansion overshoots by one, so return `left + 1, right` — and a negative index slices from the end rather than raising.

Counting is the same loop with `count += 1` inside the while — each successful step outwards is one more distinct palindromic substring.

The table earns its place only for palindrome partitioning, which needs constant-time “is `s[i..j]` a palindrome?” $O(n^2)$ times — $O(n^3)$ with expansion.

Substring $\neq$ subsequence: "bbbab" gives 3 as a substring and 4 as a subsequence ("bbbb"), and the subsequence version is `LCS(s, reversed(s))` — expansion does not apply, because a subsequence has no contiguous centre. **Manacher's is** `O(n)` (reuse mirrored results inside the rightmost known palindrome) — name it, don't write it; on real text expansion is already near-linear.

1 What this is, and why they ask it

Instagram is yesterday's Twitter with one thing added, and that one thing changes everything: **the content is images and video, and they are three orders of magnitude larger than text.**

A tweet is three hundred bytes. **A photo is two megabytes.** Everything downstream of that number is different — the storage bill, the upload path, the delivery mechanism, where the money goes, and what your users actually experience as “the app is slow”.

They ask it because **it is the cleanest test of whether you understand that media is not just bigger data.** It needs a separate write path with pre-signed uploads that never touch your servers. It needs transcoding into several sizes, done asynchronously, because a phone on a train should get a 40 KB thumbnail and not a 4 MB original. **It needs a CDN, and the CDN is not an optimisation here — it is the product,** because at this scale serving images from your own origin is financially impossible.

And it is a good question because **the metadata half is genuinely easy** — it is the news feed you already know — so the interview goes quickly to the part that distinguishes candidates. **If you spend thirty minutes on fan-out you will never reach the interesting half,** and knowing to move on is part of what is assessed.

By the end of this lesson you can design the upload path, the transcoding pipeline, the storage and delivery layers, the feed on top, and size the whole thing — including the bill, which is the number that surprises people most.

2 The story

The photo studio near the bus stand had been developing film for thirty-one years when Sadanand's son persuaded him to buy the machine that printed from phones.

And the queue changed shape immediately, in a way neither of them had expected.

Before, someone brought a roll, left it, and came back on Thursday. **The waiting happened somewhere else.**

Now, people stood there.

They stood there while the phone talked to the machine, and while the machine thought about it, and while it printed, and the whole thing took four minutes per photograph, **and there were nine people in the shop.**

Sadanand's son, who was twenty-four and had opinions, changed three things over about two months.

The first was that he stopped making people wait for the printing. He took the photos off the phone — which took twenty seconds — gave them a token, and said come back at five. **The slow part still happened. It just did not happen with the customer standing there.**

The second was the sizes. People kept asking for the small ones for passports, and the medium for albums, and occasionally something big for framing. And he had been making each one when it was asked for.

So he started making all three the moment the photo came off the phone. **More work up front, and it meant nobody ever waited for a size.**

The third took a year and it was the one that actually mattered.

Because his real problem was that the school six streets away sent him four hundred prints every February — class photographs, the same eight images, four hundred copies — **and every single copy came out of his one machine in his one shop.**

So he came to an arrangement with a fellow near the school who had a small printer. **He sent that fellow the eight images, once, and the fellow printed the four hundred.**

One journey instead of four hundred. The images sat there for the whole of February, and when the requests came, they came out of a printer six streets from the school instead of a machine six streets from the bus stand.

His father, when it was explained to him, said the thing that summed it up.

“So the picture only travels once, and after that it is already where the people are.”

3 The idea in plain English

Sadanand's son built the media pipeline. **Take the upload quickly, do the slow work afterwards, make every size in advance, and push the file to where the readers are.**

Start with the number that shapes everything.

```
a tweet      ~300 bytes
a photo      ~2 MB
a 30s video  ~15 MB

→ a photo is roughly 7,000x a tweet
```

So the metadata system and the media system are two different systems, with different stores, different scaling limits and different bills. **Say that in the first minute.**

The metadata half is the news feed you already know, and it is worth being brief about it: posts, follows, a precomputed timeline, hybrid fan-out for celebrities, hydration by batched multi-get. [Day 153](#) is the whole of it.

The media half is today.

First: the upload must not go through your servers.

The obvious design — the phone posts the image to your API, which writes it to storage — **puts every megabyte through your application tier**. At a thousand uploads a second that is 2 GB/s of traffic through machines whose job is business logic, and they will spend all their time copying bytes.

Instead: pre-signed URLs. The client asks your API “I want to upload”; the API returns a **short-lived, signed URL that grants permission to write one specific object to your storage bucket**, and the client uploads directly to S3. **Your server handles a few hundred bytes of JSON and never sees the image.**

That single decision removes the largest traffic flow in the system from your servers, and it is the first thing to say about the upload path.

Second: transcoding is asynchronous, and this is Sadanand’s token.

The uploaded original is not what gets served. **You need several sizes** — a thumbnail for the grid, a medium for the feed, a large for full screen — plus format conversion to WebP or AVIF, orientation correction from EXIF, and stripping of location metadata.

That takes seconds, and the user must not wait for it. So: the upload completes, an event goes on a queue, workers transcode, and the post becomes visible when the first usable size is ready.

The product decision hiding in there is worth naming. Either the post appears immediately and the image “pops in” when ready, or the post waits until transcoding finishes. **Instagram does the first**, showing the user their own upload from the local file while the server catches up.

Third: variants, generated in advance.

thumbnail	150 x 150	~10 KB	profile grid
small	320 wide	~40 KB	feed on a slow connection
medium	640 wide	~150 KB	feed, standard
large	1080 wide	~400 KB	full screen
original	up to 4096	~2 MB	kept for re-processing

Generating on demand is the alternative — a resizing service in front of storage — and it is tempting because it stores less. **It also makes the first request for every size slow**, and it puts a compute service on the read path of the most-read thing in the system. **Pre-generate.**

And the client picks the size. A phone on 3G asks for the 40 KB version; a laptop on wifi asks for the 400 KB one. **That one decision changes a user's experience of the product more than any backend optimisation.**

Fourth: the CDN, which is not an optimisation.

This is the fellow near the school. **The image is copied once to an edge location, and every subsequent request is served from there** — close to the user, and never touching your storage.

At this scale it is not a nice-to-have; it is the only affordable design. The arithmetic is stark, and it is the best thing to be able to produce in this interview:

```
serving 100 TB/day from S3 directly: ~$0.09/GB = ~$9,000/day
serving it from a CDN at 95% hit rate: 5 TB origin + 95 TB edge
                                         = ~$450 + ~$4,750 = ~$5,200/day
and with committed CDN pricing (~$0.02/GB): ~$2,000/day
```

And the cache hit rate is the single most valuable number in the system, because it multiplies the largest line on the bill.

Fifth: storage, which grows forever and never shrinks.

Photos are never deleted in practice — users delete a few, and the total only goes up. So storage is a permanently accumulating cost, and the lever is tiering: **recent media on standard storage, older media moved to infrequent-access and then archival classes**, because access follows a brutal power law. Almost every view is of something posted in the last week.

Sixth: the feed, briefly, because it is the part you already know.

The feed returns **URLs, not images**. The API response is small JSON; the client then fetches the images from the CDN. **So the feed API's latency and the image delivery are completely decoupled**, which is why a feed can be fast while images are still loading.

And the URLs must be signed if the content is private, which conflicts with caching — a signed URL is unique per user and cannot be shared in a cache. **The resolution is a signed URL whose signature covers a path and an expiry rather than a user**, so many users share the cache entry.

Finally: video, which is the same shape and worse.

Video is transcoded into an adaptive ladder — several resolutions and bitrates — and delivered in small chunks, so a player can switch quality mid-stream as the network changes. **Transcoding is expensive**: roughly real-time per output rendition on a CPU, so a one-minute video with five renditions is five minutes of compute. **That is where a video product's infrastructure money actually goes**, and it is worth one sentence even in a photo-focused answer.

4 The picture

The two systems, side by side:

METADATA (small, structured)

posts, users, follows, likes
~500 bytes per post
Postgres / Cassandra
sharded by id
read via the feed
cost: cheap

MEDIA (large, opaque)

images and video
~2 MB per photo
S3 / object storage
flat, keyed by media id
read via a CDN
cost: THE bill

Same product. Two completely different systems.
Say this in the first minute.

The upload path, and why it avoids your servers:

NAIVE

phone --2 MB→ API server
|
v
S3

every megabyte through your application tier

1,000 uploads/s x 2 MB = 2 GB/s
through machines meant for business logic

PRE-SIGNED

phone --request→ API server
←signed URL--
|
+--2 MB→ S3 directly

your server handles ~200 bytes of JSON and never sees the image

1,000 uploads/s x 200 B = 200 KB/s

The full write path:

1. phone → API: "I want to upload" ~10 ms
2. API → phone: pre-signed URL + media_id (server does no I/O)
3. phone → S3: PUT the 2 MB original ~2 s on 4G
4. S3 → event → QUEUE
5. TRANSCODE WORKERS:
 - read the original
 - fix orientation from EXIF
 - STRIP location metadata ← privacy, and easy to forget
 - produce 150 / 320 / 640 / 1080 as WebP
 - write each back to S3
6. mark the post visible; fan out to followers' timelines

The user sees their own post IMMEDIATELY, rendered from the local file, while steps 4-6 are still running.

The variants, and what each is for:

size	pixels	bytes	used for
thumbnail	150x150	10 KB	profile grid, 3 across
small	320 wide	40 KB	feed on a poor connection
medium	640 wide	150 KB	feed, default
large	1080 wide	400 KB	full screen, tapped
original	up to 4096	2 MB	kept only for re-processing

total stored per photo: ~2.6 MB, of which 2 MB is the original

A phone on 3G that gets the 400 KB version instead of the 40 KB one has a 10x worse experience, and no backend change fixes that.

The delivery path, and where the money is:

```
phone → CDN EDGE (Mumbai) --hit 95%→ served, ~20 ms
      |
      +--miss 5%→ S3 origin (Mumbai region) → edge → phone
```

100 TB/day delivered

ALL FROM S3:	100 TB x \$0.09/GB	= ~\$9,000/day
CDN at 95% hit:	5 TB origin x \$0.09	= ~\$450
	95 TB edge x \$0.05	= ~\$4,750
		~\$5,200/day
CDN, committed:	95 TB x \$0.02	= ~\$2,000/day

→ the hit rate multiplies the biggest line on the bill.
90% → 95% halves the origin egress.

Storage growth, which never reverses:

100M photos/day x 2.6 MB = 260 TB/day = ~95 PB/year

tiered by age, because access follows a brutal power law:

age	share of views	storage class	\$/GB/month
0-7 days	~80%	standard	0.023
7-90 days	~15%	infrequent access	0.0125
> 90 days	~5%	archive/glacier	0.004

95 PB all-standard: ~\$2.2M/month
95 PB tiered: ~\$450K/month

5x cheaper, from a lifecycle rule.

5 How it actually works

The upload handshake

```
@app.post("/upload/init")
def upload_init(user_id: int, content_type: str) → dict:
    media_id = snowflake.next_id()
    key = f"originals/{media_id}"
    url = s3.generate_presigned_url(
        "put_object",
        Params={"Bucket": ORIGINALS, "Key": key, "ContentType": content_type},
        ExpiresIn=900, # 15 minutes
    )
    media_store.create(media_id, user_id, status="pending")
    return {"media_id": media_id, "upload_url": url}
```


Storage events are at-least-once, so without this the same image is transcoded several times — **wasted CPU, and CPU is where the transcoding bill lives.** SET NX is the same idempotency pattern as [day 152's](#) notifications.

The feed response — URLs, not bytes

```
def build_post(post: dict, media: dict) → dict:
    base = f"https://cdn.example.com/{media['id']}"
    return {
        "id": post["id"],
        "author": post["author_id"],
        "caption": post["caption"],
        "media": {
            "thumb": f"{base}/thumb.webp",
            "small": f"{base}/small.webp",
            "medium": f"{base}/medium.webp",
            "large": f"{base}/large.webp",
            "width": media["width"], "height": media["height"],
            "blurhash": media["blurhash"],
        },
    }
```

The feed API returns a few hundred bytes per post and no image data at all. The client picks a variant based on its screen and connection, and fetches from the CDN.

blurhash is a twenty-byte string encoding a blurred version of the image, rendered instantly while the real one loads. **It costs nothing and it is the difference between a feed that feels fast and one that feels broken** — and including it signals that you have thought about the client.

width and **height** let the client reserve the right space before the image arrives, so the layout does not jump.

Signed URLs for private content

```
def signed_media_url(media_id: int, variant: str, ttl: int = 3600) → str:
    path = f"/{media_id}/{variant}.webp"
    expiry = int(time.time()) + ttl
    signature = hmac.new(CDN_KEY, f"{path}{expiry}".encode(), "sha256").hexdigest()
    return f"https://cdn.example.com{path}?e={expiry}&s={signature}"
```

The signature covers the path and the expiry, not the user. That matters: **a per-user signature would make every URL unique and destroy the CDN cache**, since two users viewing the same photo would request different URLs.

The trade is that a signed URL, once issued, works for anyone who has it until it expires — which is the standard compromise, and worth naming rather than glossing.

Deletion, and why it is harder than it looks

```
def delete_media(media_id: int) → None:
    media_store.mark_deleted(media_id)           # 1. immediate: feeds filter it
    cdn.purge(f"/{media_id}/*")                 # 2. minutes: edge caches
    queue.publish("gc", {"media_id": media_id}) # 3. later: the actual objects
```

Three layers, and they complete at different times. The metadata flag takes effect immediately because the feed filters on it. **The CDN purge takes minutes and is not guaranteed at every edge.** The objects are deleted last, by a background job.

So “delete” means “no longer discoverable” long before it means “the bytes are gone”, and that is worth saying plainly, because it is what users and regulators actually ask about.

Video, in outline

```
upload original → queue → transcode into an ADAPTIVE LADDER:
  240p @ 400 kbps
  480p @ 1 Mbps
  720p @ 2.5 Mbps
  1080p @ 5 Mbps
each segmented into 2-6 second chunks
plus a manifest (HLS .m3u8 or DASH .mpd) listing them

player: fetches the manifest, starts low, measures throughput,
       switches rendition between chunks

cost: ~1x real-time per rendition on CPU
     a 60 s video x 4 renditions = ~4 minutes of compute
     → hardware encoders or spot instances, and it is the
       dominant cost of a video product
```

The real systems

S3 / GCS	object storage for originals and variants
CloudFront,	CDN — the delivery layer, and most of the bill
Fastly, Akamai	
SQS / Kafka	the transcoding queue
ffmpeg	video transcoding, everywhere, universally
libvips	image resizing — several times faster and much
	lighter on memory than ImageMagick
Cassandra	Instagram's metadata store, historically

Naming libvips is a small, real signal, because anyone who has run an image pipeline at scale has hit ImageMagick's memory behaviour and moved off it.

6 The numbers

Traffic.

```
500,000,000 daily active users
each views ~50 photos/day      → 25,000,000,000 photo views/day
each posts ~0.2 photos/day     → 100,000,000 uploads/day

views: 25e9 / 86,400 = ~290,000 views/second    (peak ~900,000)
uploads: 1e8 / 86,400 = ~1,160 uploads/second   (peak ~3,500)

read : write = 250 : 1
```

Bandwidth, which is the number that matters.

```
average delivered variant: ~150 KB (most views are the feed size)

290,000 views/s x 150 KB = 43.5 GB/s
                        = ~350 Gbps sustained
per day: 25e9 x 150 KB   = 3,750 TB/day = 3.75 PB/day
```

Nearly four petabytes a day of egress. That is the system, and it is why the CDN is structural rather than an optimisation.

The bill, which is the best thing to be able to produce here:

```
ALL FROM ORIGIN (S3 egress at $0.09/GB):
3,750 TB/day x 1,024 GB x $0.09 = ~$345,000/DAY = ~$10.4M/month
→ not a business

CDN at 95% hit rate:
origin: 187 TB/day x $0.09 = ~$17,000/day
edge: 3,563 TB/day x $0.02 = ~$73,000/day (committed pricing)
      ~$90,000/day = ~$2.7M/month

→ the CDN is a 4x saving, and the hit rate is the lever:
95% → 98% cuts origin egress by 60%.
```

Upload bandwidth, for contrast:

$1,160 \text{ uploads/s} \times 2 \text{ MB} = 2.3 \text{ GB/s inbound}$

with pre-signed URLs: 0 GB/s through your application servers
 $1,160 \times 200 \text{ bytes} = 232 \text{ KB/s of JSON}$

→ the entire upload flow costs your API tier nothing.

Storage.

100,000,000 uploads/day
original 2 MB
four variants 0.6 MB
total 2.6 MB per photo

per day: $100e6 \times 2.6 \text{ MB} = 260 \text{ TB/day}$
per year: 95 PB/year

untiered, S3 standard at \$0.023/GB/month:

$95 \text{ PB} = 97,280,000 \text{ GB} \times \$0.023 = \sim \$2,240,000/\text{month}$, growing

tiered (80% of views in the first week):

0-7 days	1.8 PB	standard	\$0.023	=	~\$42,000
7-90 days	21.6 PB	IA	\$0.0125	=	~\$276,000
> 90 days	71.6 PB	archive	\$0.004	=	~\$293,000
					~\$611,000/month

→ 3.7x cheaper, from a lifecycle rule and nothing else.

Transcoding cost.

photos: ~200 ms of CPU per image for four variants

$1,160 \text{ uploads/s} \times 0.2 \text{ s} = 232 \text{ CPU-seconds/second}$

→ ~232 cores, say 30 machines of 8 cores

→ ~\$15,000/month. Trivial next to delivery.

video: ~1x real-time per rendition

a 60 s clip x 4 renditions = 240 s of CPU

$10,000,000 \text{ video uploads/day} \times 240 \text{ s} = 2.4e9 \text{ CPU-seconds/day}$

= ~28,000 cores running continuously

→ ~\$1.5M/month, and this is why video products buy hardware encoders.

Metadata, for scale contrast:

```
100,000,000 posts/day x 500 bytes = 50 GB/day
                                   = 18 TB/year
                                   x 3 replicas = 55 TB/year
```

against 95 PB/year of media.

→ the metadata is 0.06% of the storage.
All the interesting database work is on 0.06% of the bytes.

That ratio is the single best line to say in this interview.

Latency budget.

feed API (JSON only)	~50 ms
image from CDN edge, hit	~20-50 ms
image from origin, miss	~200-400 ms
blurhash render	~0 ms (it is in the JSON)

→ perceived load time is dominated by the CDN hit rate,
not by anything your servers do.

7 The trade-offs

Pre-signed uploads against proxying through your API. Pre-signed removes 2.3 GB/s from your application tier and is unambiguously right at scale. **The cost is that you cannot inspect the bytes as they arrive** — validation, virus scanning and content moderation all have to happen after the fact, asynchronously, which means a brief window where an unscanned object exists in your bucket. **Bound it by keeping originals in a separate bucket that is never publicly readable.**

Pre-generated variants against on-demand resizing. Pre-generating costs 30% more storage and makes every read a static file fetch. **On-demand stores less and puts a compute service on the read path of the most-read thing in the system** — and the first request for each size is slow, which is exactly the request a real user makes. **Pre-generate, and treat on-demand as an option only for rare sizes.**

Storage tiering against retrieval latency. Archival classes are five times cheaper and take minutes to restore. **For a photo nobody has viewed in a year that is fine; for a photo that suddenly goes viral it is not**, so the tiering rule needs a way back — usually a small standard-class copy of anything that gets requested from archive.

Signed URLs against cacheability. A per-user signature makes every URL unique and **destroys the CDN cache entirely** — the thing the whole architecture depends on. Signing the path and expiry instead keeps one cache entry per image, **at the cost that**

a leaked URL works for anyone until it expires. That is the standard compromise and it should be stated, not hidden.

Immediate visibility against transcoded visibility. Showing the post immediately, rendered from the local file, is a much better experience and means a follower can briefly see a post whose image is not ready. **Waiting for transcoding is simpler and makes the poster stare at a spinner.** Show immediately for the author, publish to followers only when the variants exist.

And the honest one: delete is not delete. The metadata flag is instant, the CDN purge takes minutes and is not guaranteed everywhere, and the objects go last. **A user asking “is my photo gone” is asking about the third thing and you are telling them about the first,** and under regulations like GDPR the distinction is legally material.

When would I not build this? Never build the media layer yourself when a managed one exists. Cloudinary, imgix and Mux do uploads, transcoding, variants and delivery, and below a few million users they are cheaper than the engineers. **And if the product is not media-first** — a marketplace with a few product photos — this entire pipeline is one S3 bucket, a CDN and a resize-on-upload lambda, and building the rest is pure overhead.

8 In the interview

How it gets asked

- “*Design Instagram.*” — usually with a scale like five hundred million users.
- “*How does the upload work?*” — the pre-signed URL question.
- “*Where does the image resizing happen?*”
- “*How much does this cost to run?*” — the question this design is uniquely good for.
- “*How do you deliver images fast in India and Brazil?*” — the CDN question.
- “*What changes for video?*”

The first ninety seconds

“Instagram is two systems that happen to share a product, and I would separate them immediately because they have almost nothing in common.

The metadata system — posts, follows, likes, the feed — **is small structured data**, about five hundred bytes a post, and it is the news feed design: precomputed timelines, hybrid fan-out for celebrities, hydration by batched multi-get. **I can go deep on that if you want, and I would rather spend the time on the other half**, because that is what makes this Instagram and not Twitter.

The media system is images, at two megabytes each — roughly seven thousand times a tweet. Everything about it is different.

Sizing, and one ratio tells the whole story. Five hundred million daily users viewing fifty photos a day is twenty-five billion views. At an average delivered size of a hundred and fifty kilobytes, that is **3.75 petabytes a day of egress** — about 350 gigabits a second sustained.

Meanwhile the metadata is fifty gigabytes a day. The media is 0.06 percent of the bytes’ worth of database work and 99.94 percent of the bytes.

Four decisions on the media path.

One: uploads never touch my servers. The client asks the API for permission and gets a short-lived pre-signed URL, then uploads directly to object storage. **Otherwise 2.3 gigabytes a second flows through machines whose job is business logic.** My server handles two hundred bytes of JSON.

Two: transcoding is asynchronous. The upload completes, an event goes on a queue, workers produce four variants — thumbnail, small, medium, large — as WebP. **The user sees their own post immediately, rendered from the local file; followers see it when the variants exist.**

Three: variants are pre-generated, not resized on demand. Thirty percent more storage, and every read becomes a static file fetch rather than a compute service on the hottest read path in the system.

Four: the CDN, which is not an optimisation here — it is the only affordable design. Serving 3.75 petabytes a day from S3 directly is about **three hundred and forty-five thousand dollars a day**, which is not a business. With a CDN at a ninety-five percent hit rate it is about ninety thousand. **The cache hit rate is the single most valuable number in the system.**

One question before I go deeper: photos only, or video too? Because video's transcoding cost is roughly real-time per rendition, and it becomes the dominant infrastructure expense in a way photos never do."

The follow-ups

"Walk me through what happens when I upload a photo."

“Six steps, and the important thing is which of them the user waits for.

One: the client calls my API saying it wants to upload. The API generates a media id, creates a `pending` metadata row, and returns a **pre-signed URL** — a short-lived signed permission to write one specific object to my bucket. **Fifteen minutes of validity, and the key is server-generated so the client cannot choose where to write.** That call is about ten milliseconds and moves no image data.

Two: the client uploads the two megabytes directly to object storage. A couple of seconds on 4G, and **none of it goes through my servers** — which is the whole reason for the handshake. At a thousand uploads a second, proxying would be 2.3 gigabytes a second through machines meant for business logic.

Three: storage emits an event onto a queue. From here everything is asynchronous.

Four: a transcode worker picks it up. It reads the original, **honours the EXIF orientation flag** — which is not optional, because phones store the sensor image plus a rotation, and skipping it makes a quarter of your users’ photos come out sideways — **strips the GPS metadata**, which is a privacy requirement rather than a size optimisation since a photo taken at home carries the user’s address, and writes four WebP variants back to storage.

And that worker must be idempotent, because storage events are at-least-once. A `SET NX` on the media id before starting, or the same image gets transcoded several times and transcoding is where the CPU bill is.

Five: the post is marked ready and fanned out to followers’ timelines. Publishing only after the variants exist means no follower ever sees a post with an unloadable image.

Six, and it happens out of order: the author sees their own post immediately, rendered from the local file on their phone while steps three to five are still running. **They never wait for any of this**, which is the product decision that makes the pipeline invisible.”

“How much does this cost to run, and where does the money go?”

“Almost all of it is egress, and the arithmetic is worth doing out loud because it justifies the entire architecture.

Twenty-five billion photo views a day at an average delivered size of a hundred and fifty kilobytes is 3.75 petabytes a day.

Served straight from S3 at nine cents a gigabyte, that is about three hundred and forty-five thousand dollars a day — ten million a month, on bandwidth alone. That is not a product, it is a rounding error away from bankruptcy.

With a CDN at a ninety-five percent hit rate: five percent goes to origin at nine cents, ninety-five percent leaves the edge at roughly two cents with committed pricing — **about ninety thousand a day, or 2.7 million a month.** Four times cheaper, and **that gap is why the CDN is structural rather than an optimisation.**

And the hit rate is the lever. Going from ninety-five to ninety-eight percent cuts origin egress by sixty percent. **That is why I would sign URLs on the path and expiry rather than per user** — a per-user signature makes every URL unique and destroys the cache entirely, which is the most expensive possible mistake in this system.

Storage is the second line, and it never shrinks. A hundred million uploads a day at 2.6 megabytes — the original plus four variants — is 260 terabytes a day, ninety-five petabytes a year. **All on standard storage that is about 2.2 million dollars a month and rising forever.**

Tiered by age it is about six hundred thousand — three point seven times cheaper, from a lifecycle rule and nothing else — **because access follows a brutal power law: roughly eighty percent of views are of content from the last week.**

Photo transcoding is trivial by comparison — about two hundred milliseconds of CPU per upload, so a few hundred cores, maybe fifteen thousand a month.

Video is where that changes. Transcoding runs at roughly real-time per rendition, so a sixty-second clip with four renditions is four minutes of compute. **At ten million video uploads a day that is around twenty-eight thousand cores running continuously** — over a million a month, and it is why every serious video product ends up buying hardware encoders rather than renting general-purpose CPU.”

“How do you make images load fast for a user in India and one in Brazil?”

“This is the part of the system the user actually experiences as speed, and it is four things, none of them in my application code.

First, the CDN, which is Sadanand’s printer near the school. The image is copied once to an edge location and every subsequent request in that region is served from there — **twenty to fifty milliseconds instead of two hundred to four hundred across an ocean.** Both users hit their nearest edge and the origin is touched once per region per image.

Second, and this matters more than people expect: the client picks the variant. A phone on a slow connection asks for the forty-kilobyte version, not the four-hundred-kilobyte one. **That is a tenfold difference in bytes and it dominates anything I can do on the server.** So the feed API returns URLs for every variant plus the dimensions, and the client chooses based on screen size and measured throughput.

Third, blurhash. Twenty bytes in the feed JSON encoding a blurred version of the image, rendered instantly. **The perceived load is zero even when the real image takes a second,** and the layout does not jump because the JSON also carries the width and height so the client reserves the right space. **This costs nothing and it is the difference between a feed that feels fast and one that feels broken.**

Fourth, format. WebP or AVIF instead of JPEG is roughly thirty percent fewer bytes at the same visual quality, and modern clients all support it — with a JPEG fallback negotiated by content type.

And one thing I would push back on if it came up: this is not solved by adding application servers in those regions. The feed API is a few hundred bytes; the images are megabytes. Putting compute near users helps the fifty-millisecond JSON call and does nothing for the ninety-nine percent of bytes that are already coming from an edge cache. **The CDN hit rate is the metric to watch, and a falling one is the first sign that something — usually a URL-signing change — has broken cacheability.**“

The model answer

“Design Instagram: five hundred million daily users, photos and short videos, and I care about what it costs.”

“The cost framing changes what I emphasise, so let me start there and work backwards, because **almost the entire bill is bytes leaving the building.**

Two systems. Metadata — posts, follows, feed — is about five hundred bytes a post and is the standard feed design: hybrid fan-out, precomputed timelines in Redis, batched hydration. **It is fifty gigabytes a day.** Media is two megabytes a photo and **3.75 petabytes a day of egress.** The metadata is 0.06 percent of the bytes, and I would not spend the interview on it.

The upload path: pre-signed URLs. The API returns a fifteen-minute signed permission and the client uploads directly to object storage. **That removes 2.3 gigabytes a second from my application tier entirely** — my servers handle two hundred bytes of JSON per upload. **The cost is that I cannot inspect bytes on arrival,** so validation, virus scanning and moderation are asynchronous, and originals live in a bucket that is never publicly readable.

Transcoding: asynchronous, idempotent, four variants plus the original. EXIF orientation honoured, GPS stripped. **The author sees their own post immediately from the local file; followers see it when the variants exist.** Photo transcoding is about two hundred milliseconds of CPU each — a few hundred cores, around fifteen thousand dollars a month, which is noise.

Delivery: CDN, and this is the whole cost story. Serving 3.75 petabytes a day from origin is **\$345,000 a day.** With a CDN at ninety-five percent hit rate it is about **\$90,000 a day.** Four times cheaper, and the hit rate is the lever — ninety-five to ninety-eight percent cuts origin egress by sixty percent.

Which means the most expensive possible mistake is breaking cacheability, and the way that happens is per-user signed URLs. **I sign the path and expiry, not the user,** so all viewers of a photo share one cache entry — accepting that a leaked URL works until it expires.

Storage: ninety-five petabytes a year and it never shrinks. All-standard is 2.2 million a month; tiered by age it is about six hundred thousand. **Eighty percent of views are of content under a week old,** so a lifecycle rule moving media to infrequent-access at a week and archive at ninety days is a $3.7\times$ saving for no engineering. **With a caveat: archived media takes minutes to retrieve, so anything requested from archive gets promoted back,** or a photo that goes viral after a year is unservable.

Video, since the prompt includes it, and it changes the shape of the bill. An adaptive ladder — four renditions, chunked, with an HLS manifest — so the player switches quality as the network changes. **Transcoding is roughly real-time per rendition**, so a sixty-second clip is four minutes of compute. At ten million video uploads a day that is around twenty-eight thousand cores continuously — **over a million a month, and it overtakes everything except delivery.** That is the argument for hardware encoders or spot capacity, and it is a decision worth making early because it is hard to retrofit.

So the ranked bill: delivery \$2.7M, storage \$600K, video transcoding \$1M+, everything else combined under \$100K. **Three of the four levers are in the media path and none of them is in my application code.**

Two things I would raise as risks. Delete is not delete — the metadata flag is instant, the CDN purge takes minutes and is not guaranteed at every edge, and the objects go last. **Under GDPR that distinction is legally material** and I would want it written down rather than discovered during an audit.

And I would seriously ask whether to build the media layer at all. Cloudinary, imgix and Mux do uploads, transcoding, variants and delivery. **At five hundred million users the per-unit economics favour building it**, and I would want that to be a calculation rather than an assumption — because below a few million users the managed service is cheaper than the team.”

9 Recall card

Two systems, and say so in the first minute: metadata is ~500 bytes a post and is the standard feed design; **media is ~2 MB a photo — $7,000 \times$ a tweet** — and is 99.94% of the bytes. All the interesting database work is on 0.06% of the data.

Uploads never touch your servers: pre-signed URLs. The API returns a short-lived signed permission and the client PUTs directly to object storage — **2.3 GB/s removed from the application tier**, at the cost of after-the-fact validation and moderation.

Transcoding is asynchronous and must be idempotent (storage events are at-least-once). **Honour EXIF orientation** or a quarter of photos come out sideways; **strip GPS** — a privacy requirement, not a size one. **Pre-generate 4 variants**, don't resize on demand: 30% more storage against a compute service on the hottest read path.

The CDN is not an optimisation — it is the only affordable design. 3.75 PB/day from origin is **~\$345K/day**; at a 95% hit rate **~\$90K/day**. **So the most expensive possible mistake is a per-user signed URL**, which makes every URL unique and destroys the cache — **sign the path and expiry instead**.

Storage never shrinks: 95 PB/year. All-standard \$2.2M/month, **tiered by age ~\$600K** — 80% of views are of content under a week old. Anything pulled from archive must be promoted back.

The feed returns URLs, not bytes, plus dimensions and a **blurhash** (20 bytes, instant render, no layout jump) — and **the client picks the variant**, which is a $10\times$ difference in bytes on a slow connection. **Video's adaptive ladder costs $\sim 1\times$ real-time per rendition** and becomes the dominant compute bill. **And delete is not delete:** metadata instant, CDN purge minutes and not guaranteed, objects last.

DSA topic: String DP: palindromic substrings **System design topic:** Design Instagram

Code these, in this order

One rule for the whole set: **before writing a loop over a table, say out loud which cell the recurrence reads, and confirm your loop order computes it first.** Today is the first time the natural order is wrong, and it fails silently.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Longest Palindromic Substring	LeetCode 5 (Medium)	Expand around centre, and the two kinds of centre.
2	Palindromic Substrings	LeetCode 647 (Medium)	The same loop, counting instead of measuring.
3	Longest Palindromic Subsequence	LeetCode 516 (Medium)	A different problem — the LCS reduction.
4	Palindrome Partitioning II	LeetCode 132 (Hard)	Where the table is required and expansion is not enough.
5	Shortest Palindrome	LeetCode 214 (Hard)	Prefix palindromes, and a linear approach.
6	Valid Palindrome II	LeetCode 680 (Easy)	Two pointers, and knowing when no DP is needed.

On problem 1, write the DP first and fill it wrongly

Fill the table left to right, top to bottom. Run on "aabaa" and print `dp[0][4]`. Record it. **Say what that `False` actually means**, and why the bug only shows on palindromes longer than three.

Then fix the fill order to go by increasing length, and confirm.

On problem 1, drop the even centres

Expand from `(i, i)` only. Run on "abba" and "cbdd" and record both answers. **Say how many centres there really are and why.**

On problem 1, forget the step back

Return `left, right` from the expansion instead of `left + 1, right`. Run on `"aba"` and print the returned pair and the resulting slice. **Say why Python does not raise.**

On problem 1, compare the two solutions honestly

Time both at `n = 2000` on random text and on a string of 2000 identical characters. Record four numbers. **Then measure the DP's memory.** Say in one sentence why you would write expansion.

On problem 2, change one line

Start from your expansion solution and turn it into a counter. **Say why** `count += 1` **inside the while loop is correct** — what exactly is being counted.

On problem 3, notice it is a different problem

Run your substring solution and your subsequence solution on `"bbbab"`. Record both. **Say why expand-around-centre cannot solve the subsequence version**, in one sentence.

On problem 4, find why expansion fails

Try to solve it using only `longest_palindrome`. Say what question you need answered and how many times. **Then compute the cost with the table and without it.**

Then the fill-order drill

Write the palindrome table twice: once by increasing length, once with `i` decreasing. Confirm they agree on twenty random strings. **Say why both work**, in terms of the dependency.

The recurrence drill

1. State the recurrence in one sentence about ends and insides.
2. Give both base cases.
3. Say which cell it depends on and where that sits in the table.
4. Say what the natural fill order does, and what the wrong `False` means.
5. Give both correct fill orders and why each works.

The centre drill

1. Say how many centres there are, and why.

2. Give an odd example and an even example.
3. Say what happens to "abba" with odd centres only.
4. Explain the `left + 1` step-back.
5. Say why a negative index does not raise.

The comparison drill

1. Give time and space for both approaches.
2. Say what the DP buys over expansion. (Nothing — say that.)
3. Give the memory numbers at `n = 1,000` and `n = 10,000`.
4. Name the one problem where the table is required, and why.
5. Say what Manacher's does, in one sentence, and whether you would write it.

The worst-case drill

1. Give the input that makes expansion quadratic.
2. Say what it does on ordinary text and why.
3. Say what that means for choosing an approach at `n = 107`.

The substring-versus-subsequence drill

1. Give both answers for "bbbab".
2. Say why expansion does not apply to the subsequence version.
3. Give the LCS reduction and justify it in one sentence.
4. Give the direct interval DP for it.
5. Say what word in the statement tells you which is being asked.

The break-it drill

Trigger each and record the exact output or error:

1. The natural fill order on "aabaa".
2. Odd centres only, on "abba".
3. Returning `left, right` from the expansion.
4. The general rule applied to a length-2 substring.
5. The DP table at `n = 50,000`.
6. A sentence with punctuation and capitals.
7. The empty string, with and without the guard.

Five of the seven give no error at all. Name them.

The two-systems drill

1. Give both systems, their sizes and their stores.
2. Give the ratio of media bytes to metadata bytes.
3. Say why you would move quickly past the metadata half.

The upload drill

1. Give the six steps and say which the user waits for.
2. Say what a pre-signed URL is and what it removes.
3. Compute the bandwidth saved at 1,000 uploads/second.
4. Say what you give up, and how you bound it.
5. Say why the transcode worker must be idempotent.
6. Name the two image-processing steps that are not about size.

The variants drill

1. List the variants with sizes and what each is for.
2. Give the pre-generate versus on-demand trade.
3. Say who picks the variant, and why that matters more than a backend change.
4. Say what `blurhash` and the dimensions are for.

The delivery drill

1. Compute daily egress from the user numbers.
2. Give the origin-only bill and the CDN bill.
3. Say what the hit rate does to the origin line.
4. Explain the signed-URL trap and the fix.
5. Say what you accept in exchange.
6. Say what a falling CDN hit rate usually means.

The storage drill

1. Compute bytes per photo and per year.
2. Give the untiered and tiered monthly costs.
3. Give the access power law that justifies tiering.
4. Say what breaks if a photo goes viral after a year, and the fix.

The video drill

1. Describe an adaptive ladder and why chunks.
2. Give the transcoding cost rule.

3. Compute the core count at 10 million uploads a day.
4. Say what that implies about hardware.

The deletion drill

1. Give the three layers and their timings.
 2. Say what “deleted” means to a user, and to you.
 3. Say why this is legally material.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the longest palindromic substring.* The DP with its recurrence and fill-order problem, then expansion with the two centre kinds, the space comparison, and where the table is still needed.
 2. *Can you do it in $O(1)$ space? Is there a linear solution?* Expansion with $2n - 1$ centres, the step-back, the worst case versus real text, and one sentence on Manacher's.
 3. *Design Instagram.* The two systems and the byte ratio, pre-signed uploads, asynchronous transcoding with the two non-size steps, the CDN bill, and the signed-URL cache trap.
-

Before you move on

- [] I can state the palindrome recurrence in one sentence.
- [] I know it depends on the cell below and to the left.
- [] I know what the natural fill order does and why it is silent.
- [] I can give both correct fill orders and justify each.
- [] I know why length 2 needs its own base case.
- [] I can write expand-around-centre from memory.
- [] I know there are $2n - 1$ centres and why.
- [] I know what odd-centres-only does to "abba".
- [] I know why the return is `left + 1, right`.
- [] I know a negative index slices rather than raising.
- [] I can turn the expansion into a counter with one line.

- [] I can give both space costs with real numbers.
- [] I know the DP buys nothing over expansion for this question.
- [] I know palindrome partitioning is where the table is required.
- [] I know the expansion worst case and what real text does instead.
- [] I can distinguish substring from subsequence with "bbbab".
- [] I know the subsequence version is `LCS(s, reversed(s))` and why.
- [] I can say what Manacher's does in one sentence.
- [] I can give the media-to-metadata byte ratio.
- [] I can explain pre-signed uploads and what they remove.
- [] I know what I give up by not seeing the bytes.
- [] I know why the transcode worker must be idempotent.
- [] I know EXIF orientation and GPS stripping, and why each matters.
- [] I can list the variants and say who picks one.
- [] I know what blurhash and dimensions do for the client.
- [] I can compute daily egress and both bills.
- [] I know the signed-URL cache trap and the fix.
- [] I can give tiered and untiered storage costs.
- [] I know the video transcoding rule and its core count.
- [] I know delete has three layers with different timings.
- [] I answered all three questions above out loud.

Grid DP: unique paths and minimum path sum

DATA STRUCTURES AND ALGORITHMS

Grid DP: unique paths

SYSTEM DESIGN

Design WhatsApp

1 What this is, and why they ask it

You are in the top-left corner of a grid. You may only move right or down. How many different routes reach the bottom-right corner?

For a 3×7 grid the answer is 28. The variants ask for the cheapest path instead of the count, or add obstacles, or change the allowed moves.

They ask it because **the state is the most natural one in all of dynamic programming**. `dp[r][c]` is the answer for the cell at row `r`, column `c`. There is no clever redefinition, no “ending exactly at”, no prefix lengths — **the grid is the table**. That makes it the best place to practise the mechanics without fighting the state, which is why it appears constantly in first-round interviews.

The other reason is that **it is where the space collapse becomes obvious**. `dp[r][c]` depends only on the cell above and the cell to the left, so one row of state is enough — and unlike knapsack, **you can see why just by looking at the picture**. A candidate who writes the full table and then says “actually one row does it, because the value in `row[c]` before I overwrite it is the cell above” has demonstrated the idea properly.

And there is a third reason worth knowing: **for the plain counting version, there is a closed-form answer**. The path is a sequence of moves, and choosing which of them are “down” is a binomial coefficient. **`O(1)` memory, `O(min(m,n))` time, no table at all** — and mentioning it is a real signal, provided you also say why it stops working the moment an obstacle appears.

By the end of this lesson you can write the counting and minimum-cost versions, handle obstacles correctly, collapse to one row and explain why, give the combinatorial solution, and recognise the family.

2 The story

The new hospital block had four floors and Meenakshi had to work out how the trolleys would move through it, and she had two weeks, and she had never done anything like it.

The plan was on the table in front of her. **Corridors going one way, corridors going the other, and junctions where they crossed.** Rooms hanging off them. And somewhere in the middle, the lift.

The rule that made it manageable came from the porters, not from her.

Because the trolleys were long and the corners were tight, the porters had worked out years ago in the old building that **you went along and then down, or down and then along, but you never doubled back.** Reversing a loaded trolley in a corridor was how you knocked a drip stand over.

So every route through the block only ever went in two directions. Forwards and towards the lift. Never back.

And the question the administrator kept asking, which Meenakshi could not answer, was how many ways there were to get from admissions to the lift, **because he wanted to know how many of them went past the room where the noise complaints were coming from.**

She tried to count them by tracing routes with her finger and lost her place four times.

What worked was going backwards, and she found it by accident.

She started writing a number at each junction. **Not the number of routes from admissions to that junction — she tried that first and it was just as confusing. The number of routes from that junction onwards to the lift.**

At the lift itself: one. You are there.

At the junction just before it, in either direction: one. Only one way to go.

And then at any junction with two exits — one forwards, one towards the lift — **the number was just the two numbers she had already written, added together.** Because every route out of that junction begins with one step or the other, and after that step it is a route she has already counted.

It took her forty minutes to fill in the whole plan, and she never traced a single route with her finger.

The administrator asked how she was so sure, and she said the thing that was actually the proof.

“Because I never counted any route twice. Every route from here starts with exactly one of the two steps, and I have counted each of those separately.”

3 The idea in plain English

Meenakshi's junction numbers are the DP table, and her last sentence is why the recurrence is an addition rather than something harder.

The state is as direct as it gets:

`dp[r][c]` is the number of ways to reach the cell at row `r`, column `c` from the start.

No redefinition needed. The grid is the table. After the last four days, that is a relief and it is worth noticing why: the question already names a position, and the position is a complete state.

The recurrence is the addition. You can only arrive at a cell from above or from the left, so:

```
dp[r][c] = dp[r-1][c] + dp[r][c-1]
```

Every path into this cell came through exactly one of those two neighbours, so adding them counts each path once — which is Meenakshi's proof, and it is why counting works here when counting is normally so easy to get wrong.

The base cases are the first row and first column. In the top row you can only have come from the left, so there is exactly one way to reach every cell: `dp[0][c] = 1`. Same down the first column. And `dp[0][0] = 1` — one way to be where you start, by doing nothing.

Initialise the whole table to 1 and the first row and column are handled for free, which is a small, genuinely useful trick.

Now the three variants, which is what actually gets asked.

Minimum path sum. Each cell has a cost; find the cheapest route. **Same state**, `min` instead of `+`:

```
dp[r][c] = grid[r][c] + min(dp[r-1][c], dp[r][c-1])
```

And the base cases change completely. They are no longer `1` — they are **running sums**, because in the first row there is only one route and its cost is everything before it. **This is the most common error in the variant:** keeping the counting base cases and getting answers that are far too small.

Obstacles. Some cells are blocked. A blocked cell has zero ways through it, so set `dp[r][c] = 0` and carry on — the addition propagates the zero automatically.

And the base case now has a trap. The first row is not all ones any more: **once you hit an obstacle in the top row, every cell after it is unreachable**, because you cannot go round. So the first row is ones until the first obstacle and zeros thereafter. **Initialising the whole table to 1 is now wrong**, which is exactly why the trick above needs a caveat.

The space collapse, and this is where the picture does the work.

`dp[r][c]` reads only the cell above and the cell to the left. **So keep one row and update it left to right:**

```
for r in range(1, rows):
    for c in range(1, cols):
        row[c] = row[c] + row[c - 1]
```

`row[c]`, before you overwrite it, still holds the previous row's value — the cell above. `row[c-1]` has already been updated this pass — the cell to the left. Both are exactly what the recurrence wants, and you get them from one array.

That is `O(cols)` space instead of `O(rows × cols)`, and it is the clearest instance of the trick in the course, because you can point at the two values and say what each one is.

Finally, the closed form, which applies only to the plain counting version.

Every path from corner to corner of an `m × n` grid makes exactly `m - 1` downward moves and `n - 1` rightward moves, in some order. So a path is a sequence of `(m-1) + (n-2) ...` — carefully: a sequence of `m + n - 2` moves, of which `m - 1` are down.

Counting the ways to choose which moves are down is a binomial coefficient:

```
paths = C(m + n - 2, m - 1)
```

For a 3×7 grid: `C(8, 2) = 28`. Which is the answer, computed with no table at all.

`math.comb` does it in `O(min(m, n))` time and `O(1)` space — and mentioning it is a genuine signal.

But say the limitation in the same breath: it counts *all* paths, so **one obstacle destroys it entirely**. There is no closed form for the obstacle version, and the DP is the answer. **That is the honest framing, and offering the formula without it looks**

like a memorised trick.

And the family, which is larger than it looks. Minimum path sum, obstacle grids, cherry pickup (two paths at once), dungeon game (which is filled backwards), triangle, maximal square. **They all share the state and differ in the move set, the operator, and occasionally the direction of the fill.**

4 The picture

The table filling in, for a 3×7 grid:

	c=0	1	2	3	4	5	6	
r=0	1	1	1	1	1	1	1	top row: one way to each
r=1	1	2	3	4	5	6	7	each = above + left
r=2	1	3	6	10	15	21	28	answer = dp[2][6] = 28

^

first column: one way to each

```
dp[1][1] = dp[0][1] + dp[1][0] = 1 + 1 = 2
dp[2][3] = dp[1][3] + dp[2][2] = 4 + 6 = 10
```

Every path into a cell arrives from ABOVE or from the LEFT, never both, so adding counts each path exactly once.

Meenakshi's version, counted backwards from the destination:

same grid, numbers = routes from HERE to the lift

r=0	28	21	15	10	6	3	1	
r=1	7	6	5	4	3	2	1	
r=2	1	1	1	1	1	1	1	← the lift is at the bottom right

the bottom row: only one way, along

The answer is the same 28, read from the START instead of the END. Forwards from the start or backwards from the goal – both work, and which is more natural depends on where the base case is obvious.

The space collapse, which you can see:

ONE ROW, updated LEFT TO RIGHT

row after r=0: [1 1 1 1 1 1 1]

processing r=1, at c=3:

row = [1 2 3 ? 1 1 1]

 ^ ^
 | +-- row[3] still holds the PREVIOUS row's value = 1
 | that is the cell ABOVE
 +----- row[2] was updated THIS pass = 3
 that is the cell to the LEFT

row[3] = row[3] + row[2] = 1 + 3 = 4 correct

The array holds the previous row to the right of the cursor
and the current row to the left of it. That is the whole trick.

Obstacles, and why the first row breaks:

grid (X = obstacle)

```
. . X .  
. . . .  
. . . .
```

first row: 1 1 0 0 ← NOT 1 1 1 1
 ^

once blocked, everything after it in the top row is
unreachable – you cannot go round, because you cannot go up.

full table:

```
1 1 0 0  
1 2 2 2  
1 3 5 7
```

answer 7. Initialising the whole table to 1 would give 10 – exactly
the no-obstacle answer, because the block in the top row was ignored.

Minimum path sum, and why the base cases change:

grid	dp (cheapest cost to reach)
1 3 1	1 4 5
1 5 1	2 7 6
4 2 1	6 8 7

first row is a RUNNING SUM: 1, 1+3=4, 4+1=5
 first column likewise: 1, 1+1=2, 2+4=6

dp[1][1] = 5 + min(4, 2) = 7
 dp[2][2] = 1 + min(6, 8) = 7 answer 7

Keeping the counting base cases (all 1s) would give nonsense.
 The base cases follow from the QUESTION, not from habit.

The closed form:

3 x 7 grid: every path is 2 downs and 6 rights, in some order

D	D	R	R	R	R	R	R	
R	D	R	D	R	R	R	R	
R	R	R	R	R	R	D	D	... all arrangements of the same multiset

a path IS a sequence of 8 moves, of which 2 are down
 → choose which 2 of the 8 are down
 → $C(8, 2) = 8 \times 7 / 2 = 28$

m + n - 2 total moves, choose m - 1 of them.
 $O(\min(m, n))$ time, $O(1)$ space, no table.

AND: one obstacle destroys this completely. There is no closed form once some paths are forbidden.

5 The code, built step by step

Counting paths

```
def unique_paths(rows: int, cols: int) → int:
    dp = [[1] * cols for _ in range(rows)]    # first row and column: 1 way each
    for r in range(1, rows):
        for c in range(1, cols):
            dp[r][c] = dp[r - 1][c] + dp[r][c - 1]
    return dp[rows - 1][cols - 1]
```

Initialising to 1 handles both base cases for free, and the loops start at 1 so they never touch them.

Three lines of actual work. This is the cleanest DP in the course, and that is why it is asked first.

The one-row version

```
def unique_paths_one_row(rows: int, cols: int) → int:
    row = [1] * cols
    for _ in range(1, rows):
        for c in range(1, cols):
            row[c] += row[c - 1]          # above + left, from one array
    return row[-1]
```

`row[c]` is the cell above (not yet overwritten this pass); `row[c-1]` is the cell to the left (already overwritten). Say that out loud while writing it — it is the whole justification, and it is checkable by pointing at the array.

`O(cols)` space. Swap the dimensions first if `rows < cols` and it becomes `O(min(rows, cols))`.

The closed form

```
import math

def unique_paths_formula(rows: int, cols: int) → int:
    """A path is m+n-2 moves, of which m-1 are down. Choose which."""
    return math.comb(rows + cols - 2, rows - 1)
```

One line, `O(1)` space. Offer it as an aside, and immediately say that **it only works with no obstacles**.

Obstacles

```
def unique_paths_with_obstacles(grid: list[list[int]]) → int:
    rows, cols = len(grid), len(grid[0])
    if grid[0][0] == 1 or grid[rows - 1][cols - 1] == 1:
        return 0          # start or end blocked
    row = [0] * cols
    row[0] = 1
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                row[c] = 0          # blocked: no paths through it
            elif c > 0:
                row[c] += row[c - 1]
    return row[-1]
```

`row[c] = 0` on an obstacle, and the addition propagates it — every cell downstream loses those paths automatically.

`elif c > 0` skips column zero, whose value carries down from the previous row unchanged, which is exactly right: the only way into a first-column cell is from directly above.

And the `row[0] = 1` before the loop with `r` starting at 0 means the first row is handled by the same code as every other row — **no separate base-case pass, and the obstacle rule applies to it correctly.**

Minimum path sum

```
def min_path_sum(grid: list[list[int]]) → int:
    rows, cols = len(grid), len(grid[0])
    row = [0] * cols
    row[0] = grid[0][0]
    for c in range(1, cols):
        row[c] = row[c - 1] + grid[0][c]      # first row: a RUNNING SUM
    for r in range(1, rows):
        row[0] += grid[r][0]                  # first column: also a running sum
        for c in range(1, cols):
            row[c] = grid[r][c] + min(row[c], row[c - 1])
    return row[-1]
```

The two base-case loops are the whole difference from the counting version, and forgetting them is the standard bug: the first row's costs must accumulate, because there is only one route along it.

`min(row[c], row[c-1])` is above and left, exactly as before — only the operator changed.

Reconstructing the path

```
def min_path_route(grid: list[list[int]]) → list[tuple[int, int]]:
    rows, cols = len(grid), len(grid[0])
    dp = [[0] * cols for _ in range(rows)]    # full table needed for the walk-back
    dp[0][0] = grid[0][0]
    for c in range(1, cols):
        dp[0][c] = dp[0][c - 1] + grid[0][c]
    for r in range(1, rows):
        dp[r][0] = dp[r - 1][0] + grid[r][0]
        for c in range(1, cols):
            dp[r][c] = grid[r][c] + min(dp[r - 1][c], dp[r][c - 1])

    path, r, c = [], rows - 1, cols - 1
    while (r, c) ≠ (0, 0):
        path.append((r, c))
        if r == 0:
            c -= 1
        elif c == 0:
            r -= 1
        elif dp[r - 1][c] ≤ dp[r][c - 1]:
            r -= 1                    # came from above
        else:
            c -= 1                    # came from the left
    path.append((0, 0))
    return path[::-1]
```

The full table is required, same trade as every reconstruction this week. The `r = 0` and `c = 0` guards handle the edges, where only one direction is possible.

Two variants worth seeing

```
def min_falling_path(grid: list[list[int]]) → int:
    """Moves are down-left, down, down-right. Same state, different move set."""
    rows, cols = len(grid), len(grid[0])
    row = grid[0][:]
    for r in range(1, rows):
        nxt = [0] * cols
        for c in range(cols):
            best = min(row[max(c - 1, 0):min(c + 2, cols)])
            nxt[c] = grid[r][c] + best
        row = nxt
    return min(row) # may end anywhere in the last row

def maximal_square(matrix: list[list[str]]) → int:
    """dp[r][c] = side of the largest all-1 square ENDING at (r,c)."""
    rows, cols = len(matrix), len(matrix[0])
    dp = [[0] * cols for _ in range(rows)]
    best = 0
    for r in range(rows):
        for c in range(cols):
            if matrix[r][c] == "1":
                if r == 0 or c == 0:
                    dp[r][c] = 1
                else:
                    dp[r][c] = 1 + min(dp[r - 1][c], dp[r][c - 1], dp[r - 1][c - 1])
                best = max(best, dp[r][c])
    return best * best
```

`min_falling_path` needs a second array, because a cell reads three cells from the previous row including one to its right — **which the one-row trick has already overwritten**. That is a good check on whether you understand the collapse rather than copying it.

`maximal_square` reads three neighbours including the diagonal, and the `min` of three is the constraint that all three smaller squares must be present. **Its answer is `max(dp)`, not the last cell** — the state is “ending at”, like longest increasing subsequence.

The complete solution

```
"""Grid DP: counting, costs, obstacles, and the closed form."""

import math

def unique_paths(rows: int, cols: int) → int:
    """dp[r][c] = ways to reach (r, c). Every path arrives from above or left."""
    dp = [[1] * cols for _ in range(rows)]    # first row/column: one way each
    for r in range(1, rows):
        for c in range(1, cols):
            dp[r][c] = dp[r - 1][c] + dp[r][c - 1]
    return dp[rows - 1][cols - 1]

def unique_paths_one_row(rows: int, cols: int) → int:
    """row[c] is the cell ABOVE until overwritten; row[c-1] is the cell LEFT."""
    row = [1] * cols
    for _ in range(1, rows):
        for c in range(1, cols):
            row[c] += row[c - 1]
    return row[-1]

def unique_paths_formula(rows: int, cols: int) → int:
    """m+n-2 moves, m-1 of them down. No table. Breaks with any obstacle."""
    return math.comb(rows + cols - 2, rows - 1)

def unique_paths_with_obstacles(grid: list[list[int]]) → int:
    """A blocked cell has 0 paths; the addition propagates the zero."""
    rows, cols = len(grid), len(grid[0])
    if grid[0][0] == 1 or grid[rows - 1][cols - 1] == 1:
        return 0
    row = [0] * cols
    row[0] = 1
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                row[c] = 0
            elif c > 0:
                row[c] += row[c - 1]
    return row[-1]

def min_path_sum(grid: list[list[int]]) → int:
    """Same state, min instead of +. The base cases become RUNNING SUMS."""
    rows, cols = len(grid), len(grid[0])
    row = [0] * cols
    row[0] = grid[0][0]
    for c in range(1, cols):
        row[c] = row[c - 1] + grid[0][c]
    for r in range(1, rows):
        row[0] += grid[r][0]
        for c in range(1, cols):
            row[c] = grid[r][c] + min(row[c], row[c - 1])
    return row[-1]

def min_path_route(grid: list[list[int]]) → list[tuple[int, int]]:
    """The cheapest route itself. Needs the full table."""
```

```

rows, cols = len(grid), len(grid[0])
dp = [[0] * cols for _ in range(rows)]
dp[0][0] = grid[0][0]
for c in range(1, cols):
    dp[0][c] = dp[0][c - 1] + grid[0][c]
for r in range(1, rows):
    dp[r][0] = dp[r - 1][0] + grid[r][0]
    for c in range(1, cols):
        dp[r][c] = grid[r][c] + min(dp[r - 1][c], dp[r][c - 1])

path: list[tuple[int, int]] = []
r, c = rows - 1, cols - 1
while (r, c) != (0, 0):
    path.append((r, c))
    if r == 0:
        c -= 1
    elif c == 0:
        r -= 1
    elif dp[r - 1][c] <= dp[r][c - 1]:
        r -= 1
    else:
        c -= 1
path.append((0, 0))
return path[::-1]

def min_falling_path(grid: list[list[int]]) → int:
    """Down-left, down, down-right. Needs TWO rows: it reads to the right."""
    rows, cols = len(grid), len(grid[0])
    row = grid[0][:]
    for r in range(1, rows):
        nxt = [0] * cols
        for c in range(cols):
            nxt[c] = grid[r][c] + min(row[max(c - 1, 0):min(c + 2, cols)])
        row = nxt
    return min(row)

def maximal_square(matrix: list[list[str]]) → int:
    """dp[r][c] = side of the largest all-1 square ENDING at (r,c). Answer is max."""
    rows, cols = len(matrix), len(matrix[0])
    dp = [[0] * cols for _ in range(rows)]
    best = 0
    for r in range(rows):
        for c in range(cols):
            if matrix[r][c] == "1":
                dp[r][c] = 1 if r == 0 or c == 0 else 1 + min(
                    dp[r - 1][c], dp[r][c - 1], dp[r - 1][c - 1])
                best = max(best, dp[r][c])
    return best * best

if __name__ == "__main__":
    print("paths 3x7 table :", unique_paths(3, 7))
    print("paths 3x7 one row:", unique_paths_one_row(3, 7))
    print("paths 3x7 formula:", unique_paths_formula(3, 7))
    print("paths 1x1      :", unique_paths(1, 1))
    print("paths 23x12     :", unique_paths_formula(23, 12))

    blocked = [[0, 0, 1, 0],
                [0, 0, 0, 0],
                [0, 0, 0, 0]]
    print("with obstacle :", unique_paths_with_obstacles(blocked))

```

```

print("no obstacle      :", unique_paths(3, 4))
print("start blocked   :", unique_paths_with_obstacles([[1, 0], [0, 0]]))

cost = [[1, 3, 1],
        [1, 5, 1],
        [4, 2, 1]]
print("min path sum    :", min_path_sum(cost))
print("the route       :", min_path_route(cost))

print("falling path    :", min_falling_path([[2, 1, 3], [6, 5, 4], [7, 8, 9]]))
print("maximal square  :", maximal_square([
    list("10100"), list("10111"), list("11111"), list("10010")]))

```

Run it and you get:

```

paths 3x7 table : 28
paths 3x7 one row: 28
paths 3x7 formula: 28
paths 1x1       : 1
paths 23x12     : 193536720
with obstacle   : 7
no obstacle     : 10
start blocked   : 0
min path sum    : 7
the route       : [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2)]
falling path    : 13
maximal square  : 4

```

with obstacle 7 against no obstacle 10 is the cost of the single blocked cell, made visible — and note it is not a simple subtraction, because the block also removes paths that would have gone through cells downstream of it.

And paths 23x12 being 193 million is worth noticing: the counts grow very fast, which is why the counting version sometimes asks for a modulus.

6 What it costs

Time and space, both versions.

```

full table:  rows x cols cells, O(1) each      = O(rows x cols) time
              rows x cols integers             = O(rows x cols) space

one row:     same time
              cols integers                     = O(cols) space
              (swap first for O(min(rows, cols)))

formula:     math.comb is O(min(m, n)) multiplications
              O(1) space

```

Concretely:

100 x 100	10,000 cells	instant
1,000 x 1,000	1,000,000 cells	~0.3 s in Python
	full table: ~40 MB	
	one row: ~8 KB	5,000x less
10,000 x 10,000	100,000,000 cells	~40 s
	full table: ~4 GB	→ MemoryError
	one row: ~80 KB	still fine

The one-row version is what makes large grids possible at all, and that is a stronger argument than the constant factor.

The formula, for contrast:

```
unique_paths_formula(1000, 1000)
= C(1998, 999)
math.comb does ~999 multiplications and divisions
→ microseconds, against 0.3 seconds and a million cells
```

and the answer has 600 digits, which Python handles and a 64-bit integer does not – worth checking if the problem specifies a modulus.

How fast the counts grow:

3 x 7	= 28	
10 x 10	= 48,620	
20 x 20	= 35,345,263,800	~3.5 x 10 ¹⁰
50 x 50	~ 2.5 x 10 ²⁸	28 digits
100 x 100	~ 2.2 x 10 ⁵⁸	59 digits

a Java long holds 19 digits → overflows before a 40 x 40 grid.
Python is fine; the problem may still ask for mod 10⁹ + 7.

Reconstruction:

the walk-back is O(rows + cols) – each step decreases r or c
but it needs the FULL table: O(rows x cols) space

so: the length, in O(cols) space
or the path, in O(rows x cols) space
not both, unless you use divide and conquer

The variants:

min path sum	same $O(\text{rows} \times \text{cols})$ time, $O(\text{cols})$ space
obstacles	same, plus one branch per cell
min falling path	$O(\text{rows} \times \text{cols})$ time, $O(\text{cols})$ space but TWO arrays, because it reads to the right same, and reads three neighbours
maximal square	
cherry pickup (two paths at once)	$O(\text{rows} \times \text{cols}^2)$ – the state gains a dimension for the second walker's column

Cherry pickup is the one worth naming as the step up: two paths at once means the state is `(row, col1, col2)`, and that extra dimension is the whole difficulty.

7 The traps

Initialising to 1 when there are obstacles.

```
>>> grid = [[0, 0, 1, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
>>> dp = [[1] * 4 for _ in range(3)]          # the free base-case trick
>>> for r in range(1, 3):
...     for c in range(1, 4):
...         dp[r][c] = 0 if grid[r][c] else dp[r-1][c] + dp[r][c-1]
>>> dp[2][3]
10
```

Ten, when the answer is seven — and ten is exactly the answer for this grid with no obstacle at all. The top row was left as all ones, so the cell after the obstacle was treated as reachable and the block might as well not have been there. **You cannot go round it, because you cannot move up.** The trick that is free without obstacles is wrong with them.

Keeping the counting base cases in min-path-sum.

```
>>> grid = [[1, 3, 1], [1, 5, 1], [4, 2, 1]]
>>> dp = [[0] * 3 for _ in range(3)]          # zeros, no running sums
>>> for r in range(1, 3):
...     for c in range(1, 3):
...         dp[r][c] = grid[r][c] + min(dp[r-1][c], dp[r][c-1])
>>> dp[2][2]
2
```

Two, when the answer is seven. The first row and column were left as zeros, so the algorithm believes it can reach any edge cell for free and then drop straight into the corner. **The base cases follow from the question, not from the last problem you solved.**

Applying the one-row trick where a cell reads to the right.


```
>>> # min falling path: dp[r][c] reads row[c-1], row[c], row[c+1]
>>> row = [2, 1, 3]
>>> for c in range(3):
...     row[c] = min(row[max(c-1,0):min(c+2,3)])
>>> row
[1, 1, 1]
```

`row[c-1]` has already been overwritten with this row's value, so the recurrence reads the wrong generation. **The one-row trick works only when every dependency is above or to the left** — and this variant reads to the right, so it needs two arrays.

Returning the last cell from `maximal_square`.

```
>>> # dp[r][c] is the largest square ENDING at (r,c)
>>> # the largest square in the grid can end anywhere
>>> # returning dp[-1][-1] gives the square in the bottom-right corner only
```

“Ending exactly at” states need `max` over the table, exactly like longest increasing subsequence. **The plain path-counting state does not**, because it covers the whole journey. **Knowing which kind you have is the point.**

Off-by-one in the formula.

```
>>> math.comb(3 + 7, 3)
120
>>> math.comb(3 + 7 - 2, 3 - 1)
28
```

120 against 28. The moves are `m + n - 2` because you start on a cell rather than before one, and you choose `m - 1` downs, not `m`. **Check it against a 2×2 grid, where the answer is obviously 2**, before trusting it.

Offering the formula on an obstacle grid.

```
>>> unique_paths_formula(3, 4)
10
>>> unique_paths_with_obstacles([[0,0,1,0],[0,0,0,0],[0,0,0,0]])
7
```

The formula counts all paths and knows nothing about blocking. Offering it without saying “no obstacles” is worse than not offering it, because it looks like a memorised trick rather than an understood one.

Overflow, in other languages.

a 40 x 40 grid has $C(78, 39) \sim 1.1 \times 10^{22}$ paths
a Java long holds up to 9.2×10^{18}

→ silent wraparound, and a plausible negative number

Python is unbounded so this cannot happen there, which makes it easy to forget when porting. **If the problem mentions a modulus, that is why.**

An empty grid.

```
>>> min_path_sum([])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in min_path_sum
    rows, cols = len(grid), len(grid[0])
IndexError: list index out of range
```

grid[0] on an empty list. One guard at the top, and it is the kind of thing an interviewer adds as a test case at the end.

8 In the interview

How it gets asked

- “How many unique paths are there from the top-left to the bottom-right, moving only right and down?” — LeetCode 62.
- “Now some cells are blocked.” — LeetCode 63, and the base case is the trap.
- “Find the path with the smallest sum.” — LeetCode 64.
- “Can you reduce the space?” — the one-row question, and it is always asked.
- “Is there a solution without any table at all?” — the combinatorics question.
- “What if you can also move diagonally?”

The first ninety seconds

“The state here is the most direct one in dynamic programming, which is worth saying because the last few problems have all needed careful redefinition. `dp[r][c]` is the number of ways to reach the cell at row `r`, column `c`. The grid is the table.

The recurrence is an addition. You can only arrive at a cell from above or from the left, so `dp[r][c] = dp[r-1][c] + dp[r][c-1]`.

And the reason adding is correct is worth one sentence, because counting problems are usually where double-counting hides: every path into this cell arrived through exactly one of those two neighbours, never both, so adding counts each path once.

Base cases: the first row and the first column are all ones. In the top row you can only have come from the left, so there is exactly one route to each cell. And `dp[0][0] = 1` — one way to be where you start. Initialising the whole table to one gives me both for free.

`O(rows * cols)` time and space.

And I would reduce the space immediately, because it is unusually clear here. `dp[r][c]` reads only the cell above and the cell to the left, so one array is enough: iterate left to right, and `row[c]` before I overwrite it still holds the previous row's value — that is the cell above — while `row[c-1]` has already been updated this pass — that is the cell to the left. Both are exactly what I need.

`O(cols)` space, and `O(min(rows, cols))` if I swap the dimensions first. At a thousand by a thousand that is eight kilobytes instead of forty megabytes.

There is also a closed form for this exact version. Every path is `m + n - 2` moves of which `m - 1` are down, so the count is `C(m + n - 2, m - 1)` — twenty-eight for a three by seven grid — computable in `O(min(m, n))` with no table.

And I would immediately say the limitation: it counts all paths, so a single obstacle destroys it. There is no closed form once some cells are blocked, and then the DP is the only answer.

Which version would you like — the plain count, obstacles, or minimum cost? Because the base cases differ between them, and that is where the bugs are.“

The follow-ups

“Now some cells are blocked.”

“The recurrence barely changes and the base case changes completely, which is the opposite of what people expect.

The recurrence: a blocked cell has zero paths through it, so `dp[r][c] = 0`. And that is all — **the addition propagates the zero automatically** to every cell downstream, because those paths simply are not there to be added.

The base case is the trap. Without obstacles, the first row is all ones. **With obstacles, it is ones until the first blocked cell and zeros after it** — because you cannot go round an obstacle in the top row, since going round would require moving up, and up is not a legal move.

So the trick of initialising the whole table to one is now wrong, and it gives a plausible answer rather than an error. On a three-by-four grid with one obstacle in the top row it says ten — which is exactly the answer for the same grid with no obstacle at all — and the answer is seven.

The clean way to write it is to handle the first row inside the same loop as every other row — set `row[0] = 1` before starting, iterate `r` from zero, and let the obstacle check apply uniformly. **No separate base-case pass means no separate base-case bug.**

Two edge cases I would check explicitly: the start blocked, and the destination blocked. **Both are zero**, and the start one in particular will otherwise propagate a `1` from an initialisation that should never have happened.

And this is where the combinatorial formula dies. $C(m+n-2, m-1)$ counts every arrangement of moves, and it has no way to express ‘except the ones through this cell’. **You could try inclusion-exclusion for a single obstacle, and it becomes intractable for several** — so the DP is not a fallback here, it is the answer.”

“Reduce the space.”

“One array instead of the table, and this is the clearest example of the trick in the whole subject because you can point at the two values.

`dp[r][c]` reads exactly two cells: the one above and the one to the left. Nothing else. So I never need more than one row of history.

Keep one array of length `cols`, and iterate left to right. When I am at column `c`:

`row[c]`, which I have not yet overwritten on this pass, still holds the value from the previous row — that is the cell above. `row[c-1]`, which I overwrote a moment ago, holds this row’s value — that is the cell to the left.

So `row[c] += row[c-1]` is exactly the recurrence, and the array holds the previous row to the right of the cursor and the current row to the left of it.

`O(cols)` space, and I would swap the dimensions first if `rows < cols` so it becomes `O(min(rows, cols))`. At a thousand by a thousand that is eight kilobytes against forty megabytes; at ten thousand square it is the difference between eighty kilobytes and four gigabytes — so it is not a constant-factor tidy-up, it is what makes large grids possible at all.

Two things I would flag. Reconstruction is gone — if you want the actual path and not just the count or cost, you need the full table, or Hirschberg-style divide and conquer.

And the trick has a precondition that is easy to violate. It works because every dependency is above or to the left. In a variant where a cell reads the previous row’s cell to its right — minimum falling path, where the moves are down-left, down and down-right — that cell has already been overwritten with the current row’s value, and the in-place version silently computes the wrong thing. That one needs two arrays, and noticing why is a better demonstration than the collapse itself.”

“Is there a way to do this without a table at all?”

“Yes, for the plain counting version, and it is combinatorics rather than dynamic programming.

Every path from the top-left to the bottom-right of an $m \times n$ grid consists of exactly $m - 1$ downward moves and $n - 1$ rightward moves, in some order. It cannot be otherwise — you must descend $m-1$ rows and cross $n-1$ columns, and no move is ever wasted because you can never go back.

So a path is a sequence of $m + n - 2$ moves, and it is completely determined by which of them are down. Counting paths is counting which positions in that sequence are the downs.

That is $C(m + n - 2, m - 1)$. For three by seven: $C(8, 2) = 28$. Same answer as the table.

`math.comb` computes it with about `min(m, n)` multiplications and constant space — microseconds against a million cells for a thousand-square grid.

And I would immediately state the limitation, because offering this without it looks like a memorised trick. It counts *every* arrangement of moves. **The moment a cell is blocked, some arrangements are illegal and the formula has no way to express that.** You could handle one obstacle with inclusion-exclusion — count all paths, subtract those through the obstacle — and with several obstacles the number of terms explodes and it becomes worse than the DP.

Same for costs. Minimum path sum has no closed form at all, because the answer depends on the actual values in the grid rather than only on its shape.

One practical note: the numbers get large fast. A forty-by-forty grid has about 10^{22} paths, which overflows a 64-bit integer silently. **Python is unbounded, so I would not notice — which is exactly why the problem sometimes asks for the answer modulo $10^9 + 7$** , and if it does, the formula needs modular inverses while the DP just takes the modulus at each addition. **That would push me back to the DP.**“

The model answer

“A delivery robot moves through a warehouse laid out as a grid. It can only move east or south. Some cells are blocked by shelving, and each open cell has a cost representing how congested it is. Find the cheapest route, and tell me how many equally cheap routes there are.”

“Two questions on the same grid, and they need slightly different machinery, so let me take them in order.

The state is the position: `dp[r][c]` is the cheapest cost to reach cell `(r, c)` from the start. No redefinition needed — the grid is the table, and the position is a complete state, because how the robot got there does not affect what it can do next.

The recurrence: `dp[r][c] = cost[r][c] + min(dp[r-1][c], dp[r][c-1])` — arrive from above or from the left, take the cheaper, add this cell’s congestion.

Blocked cells are infinity, not zero, and that distinction matters here. In the counting version a blocked cell is zero because it contributes no paths. **In the cost version zero would mean ‘free to pass through’, which is the opposite of blocked** — so I use `float('inf')`, and the `min` naturally refuses to route through it.

Base cases, and this is where the bug usually is. The first row and column are **running sums**, not ones and not zeros: along the top edge there is exactly one route, so its cost is everything before it accumulated. **And once the top row hits a shelf, everything after it in that row is unreachable** — infinity, propagated.

Now the second question, the count of equally cheap routes, which needs a second table alongside.

`ways[r][c]` is the number of cheapest routes reaching `(r, c)`. When I compute the cost I compare the two predecessors: **if one is strictly cheaper, this cell inherits its count. If they are equal, it gets the sum of both.** That replace-versus-add distinction is exactly the one from counting longest increasing subsequences, and getting it backwards gives plausible wrong numbers.

Two tables, same loops, one pass.

Cost: `O(rows * cols)` **time.** For a warehouse that is realistically a few hundred by a few hundred — under a hundred thousand cells — that is instant, and **I would use the full tables rather than the one-row collapse**, because the prompt asks for the route, and reconstruction needs the table. **I would say that trade explicitly:** the one-row version is `O(cols)` space and cannot tell me the path.

Reconstruction: **walk back from the destination**, at each step moving to whichever predecessor produced the value, with guards for the top row and left column where only one direction exists.

Three things about the problem domain I would raise.

The move restriction is doing a lot of work and I would check it is real. East-and-south-only is what makes this a DP with no cycles. **If the robot can also go west or north, this is not a grid DP at all — it is Dijkstra on a graph**, because paths can revisit regions and there is no acyclic ordering to fill in. That is a completely different algorithm and it is worth confirming before writing anything.

The congestion costs presumably change over time, and the table is a snapshot. If congestion updates every few seconds, recomputing a hundred thousand cells is cheap enough to just redo — **but a route computed at the start may be stale by the time the robot is halfway**, so I would recompute from the robot's current position rather than committing to the whole route up front.

And the count of equally cheap routes is more useful than it sounds. If there are many, the robot has flexibility and I can pick one that avoids other robots. **If there is exactly one, that route is a bottleneck** and worth flagging to whoever laid out the shelving.”

9 Recall card

The most direct state in DP: `dp[r][c]` is the answer for that cell — the grid is the table. Every path arrives from **above or left, never both**, so `dp[r][c] = dp[r-1][c] + dp[r][c-1]` counts each path exactly once. Base cases: **first row and column are 1** (initialise the whole table to 1 and get them free).

The one-row collapse is clearest here: iterate left to right, and `row[c]` (not yet overwritten) is the cell above while `row[c-1]` (just overwritten) is the cell to the left. `0(cols)` space — 8 KB against 40 MB at $1,000^2$, and 80 KB against 4 GB at $10,000^2$. It fails when a cell reads the previous row to its *right* (minimum falling path) — that needs two arrays.

Obstacles: a blocked cell is 0 and the addition propagates it — but the first row is now ones until the first block and zeros after, because you cannot go round. The initialise-to-1 trick becomes wrong, giving 11 instead of 7.

Minimum path sum: same state, `min` instead of `+`, and the base cases become **RUNNING SUMS** — keeping the counting base cases is the standard bug. A blocked cell is `inf` here, not 0 — zero would mean free.

Closed form for the plain count: a path is $m+n-2$ moves of which $m-1$ are down, so $C(m+n-2, m-1)$ — 28 for 3×7 , $O(1)$ space. Always say the limitation in the same breath: one obstacle destroys it, and costs have no closed form at all.

Counts grow fast ($40 \times 40 \approx 10^{22}$, overflowing a 64-bit integer silently) — which is why a modulus appears. And `maximal_square`'s state is “ending at”, so its answer is `max` over the table, not the last cell.

1 What this is, and why they ask it

WhatsApp delivers a message from one person to another, quickly, reliably, and in order — **and it is a completely different shape of system from everything else this week.**

Twitter and Instagram are read-heavy broadcast systems: one write, many reads, and staleness is invisible. **Messaging is write-heavy and point-to-point.** Every message has exactly one or a few recipients, it must arrive within a second, and **a message that arrives twice or out of order is something a human immediately notices and complains about.**

They ask it because of three things that appear nowhere else in this course.

Connections, not requests. A phone must receive a message it did not ask for, so the server holds a persistent connection to every online device. **A hundred million concurrent connections is a different engineering problem from a hundred million requests a second** — it is bounded by memory and file descriptors rather than by CPU.

Ordering and delivery guarantees that users can see. The two ticks, then the blue ticks, are a user-visible acknowledgement protocol. **Nobody notices a stale tweet; everybody notices a message that arrives out of order or twice.**

And end-to-end encryption, which removes options. If the server cannot read the messages, it cannot do server-side search, cannot filter spam by content, and cannot re-encrypt for a new device. **Every one of those becomes a client problem,** and knowing which capabilities encryption costs you is what separates a real answer from a diagram.

By the end of this lesson you can design the connection layer, the message flow with acknowledgements, offline delivery, group messaging, media, and encryption — and size all of it.

2 The story

There were two hundred and forty houses in the village and one telephone, and it was in Rukmini's front room because that is where the line had reached in 1987.

And the job that came with it, which nobody had asked her to take on, was that she was the delivery.

The phone would ring. It would be somebody's son from Dubai, or a hospital in the town, or a man about a tractor. And whoever it was for was not in her front room.

So there was a system, and it grew one problem at a time.

The first thing was that she wrote it down. Not the whole conversation — a name, who it was from, and what they said in a line. Because the boy she sent to fetch people took eleven minutes and the caller would not wait.

The second thing was the boy. He went, he found them, they came. And on the days he could not find them — they were in the fields, they were at the market — **the note stayed on the shelf until they came past.**

The third thing was the shelf itself, which she divided with chalk lines into sections, one per household that got calls regularly, so a note never went to the wrong family.

The fourth thing took a year and it was because of a specific argument.

A woman came in furious because her brother had said he had telephoned twice about the date of a wedding and she had only heard once. Rukmini had no way to say whether that was true. **The note was gone from the shelf, which meant somebody had taken it, and that was all she knew.**

So she started making the person sign.

A line in a book: the note, the date, and their mark when they took it. And after a while she added a second thing, which was whether they had actually read it out to her, **because taking the note and reading the note turned out to be different events,** and the difference was where the arguments lived.

And the last part, which she never wrote down anywhere, was this. **She could hear every conversation, and everybody knew it, and so there were things people did not say on that telephone.**

They wrote letters instead, and the letters were sealed. **Rukmini carried them, and had no idea what was in them, and could not have told you even if she had wanted to.**

3 The idea in plain English

Rukmini built a messaging system, including the acknowledgement book and the sealed letters, and her shelf is the offline queue.

Start with why this is not a request-response system.

A message arrives for you when the sender sends it, not when you ask. So the server must be able to push, which means **a persistent connection held open per online device.**

HTTP polling	"anything for me?" every 5 s → up to 5 s of latency, and 99% of requests return nothing. Wasteful and slow.
long polling	hold the request open until something arrives → better, and still one request per message
WEBSOCKET	one connection, bidirectional, stays open → the standard answer

WebSockets are the design, and the consequence is the interesting part: **your servers now hold state.** A stateless web tier can be load-balanced anywhere; **a connection server owns specific users for the duration of their session**, and something has to know which server holds which user.

That something is a connection registry: `user_id → server_id`, in Redis, updated when a device connects and disconnects. **Every message delivery starts with a lookup in it.**

Now the message flow, which is Rukmini's book.

1. Alice's phone sends the message over her WebSocket
2. the server assigns an id, persists it, and ACKs to Alice → ONE TICK
3. the server looks up Bob in the registry
- 4a. Bob is online → push over his WebSocket
- 4b. Bob is offline → store in his offline queue
5. Bob's device acknowledges receipt → TWO TICKS
6. Bob opens the chat; his device sends a read receipt → BLUE TICKS

The three states are three separate events and they are genuinely distinct — that is the argument the woman had with Rukmini. **Sent** means the server has it. **Delivered** means the device has it. **Read** means a human looked at it. **Conflating delivered and read is the classic error**, and the whole two-ticks-versus- blue-ticks design exists because they are not the same.

Persist before acknowledging. If the server ACKs and then crashes before writing, the message is gone and the sender believes it was sent. **Write first, ACK second** — the order is the guarantee.

Then: offline delivery, which is the shelf.

A message for an offline user goes into a per-user queue, ordered, and is delivered when they reconnect. **WhatsApp's specific choice is that the queue is deleted once delivered** — the server stores messages only until delivery, not forever. **That is a storage decision with a privacy justification, and it is worth saying out loud** because it makes the storage numbers dramatically smaller than people assume.

And it means the phone is the archive, which is why restoring a WhatsApp backup is a thing users have to do.

Now ordering and duplicates, which is where messaging is unforgiving.

Ordering within a conversation is what matters, not global ordering. Two messages in different chats can arrive in any order and nobody cares. **Two messages in the same chat arriving reversed is a visible bug.**

So: a per-conversation sequence number. The client sends messages with increasing sequence numbers, the server preserves them, and the receiving client can detect a gap and request a resend. **A global ordering would need coordination across the whole system and buys nothing.**

Duplicates come from retries. The network drops an acknowledgement, the client resends, and the server sees the same message twice. **A client-generated message id makes the send idempotent** — the server keeps recent ids and drops repeats. **The id must come from the client**, because a server-generated one is different on each attempt.

Groups, which are fan-out again but bounded.

A group message is delivered to every member. **WhatsApp caps groups at around a thousand members**, which is what makes this tractable: **a thousand deliveries is a bounded fan-out, not a celebrity problem.** Twitter has to invent a hybrid because followers are unbounded; here the product simply forbids it.

The interesting part is that the sender's device does the work under encryption. With end-to-end encryption there is no group key the server holds, so the sender encrypts the message once per recipient device — **a thousand members with two devices each is two thousand encryptions on a phone.** The Signal protocol's sender-key optimisation reduces this to one encryption plus a distributed group key, which is exactly why it exists.

Media, which is Instagram's problem again but smaller.

The message carries a reference, not the bytes. The sender uploads the encrypted blob to storage, and the message contains the URL and the decryption key. **The recipient downloads and decrypts.**

And the deduplication is worth knowing: the same forwarded video is uploaded once and referenced many times, keyed by a hash of the content. **That is why forwarding a large video is instant.**

Finally, encryption, and what it costs you.

End-to-end means the server holds ciphertext it cannot read. Keys live on devices; the server distributes public keys and nothing else.

And the costs are real and worth naming:

no server-side search	search must run on the device, over local data
no server-side spam filtering	you can only use metadata: who, how often, how many recipients – not what
no multi-device without work	each device needs its own keys; adding one cannot be done by the server re-encrypting
no server-side backup	a backup is either unencrypted, or encrypted with a key only the user has – and then a lost key is lost history

Naming those trade-offs is the answer to “how does encryption affect the design”. The protocol details — Double Ratchet, X3DH — are worth a sentence and not more.

4 The picture

The connection layer, and why it is stateful:

phones	connection servers	registry (Redis)
[Alice] ===WS=====>	[conn-server-7]	
[Bob] ===WS=====>	[conn-server-2]	alice → conn-server-7
[Carol] ===WS=====>	[conn-server-7]	bob → conn-server-2
[Dev] (offline)		carol → conn-server-7
		dev → (absent)

Every delivery starts with a registry lookup.
 A stateless web tier can be load-balanced anywhere;
 a CONNECTION server owns specific users until they disconnect.

100,000,000 concurrent connections is a MEMORY and FILE-DESCRIPTOR problem, not a CPU one.

The message flow and the three ticks:

Alice	server	Bob
---- send ----->	persist FIRST	
<--- ack (id) ----		ONE TICK (sent)
	---- lookup Bob ---->	registry
	===== push =====>	
	<----- ack -----	
<--- delivered ----		TWO TICKS (delivered)
	<----- read receipt ---	(Bob opened the chat)
<--- read -----		BLUE TICKS (read)

PERSIST BEFORE ACK. If the server acks then crashes, the message is gone and the sender believes it was sent.

DELIVERED and READ are different events. Conflating them is the classic mistake, and the whole tick design exists because they differ.

Offline delivery, which is Rukmini's shelf:

Bob is offline

message → [offline queue: bob] ordered, persisted

|
Bob reconnects

|
v
deliver in order, wait for acks

|
v
DELETE from the queue

WhatsApp stores messages only until delivery, not forever.

→ the phone is the archive

→ the server's storage is tiny compared with a social product

→ and restoring a backup is a thing users must do

Ordering: per-conversation, not global:

conversation alice↔bob: seq 1, 2, 3, 4 ...

conversation alice↔carol: seq 1, 2, 3 ...

These two are INDEPENDENT. Nobody notices if a message to Carol arrives before an earlier one to Bob.

Within one conversation, out-of-order is IMMEDIATELY VISIBLE:

"yes"		"what time?"
"what time?"	vs	"yes"
→ nonsense		→ fine

A gap in the sequence lets the receiving client detect loss and ask for a resend. Global ordering would need cluster-wide coordination and buys nothing.

Groups, and why the cap matters:

GROUP of 1,000 members

server-side fan-out: 1,000 registry lookups + 1,000 pushes
BOUNDED, because the product caps group size

Twitter cannot cap followers, so it needs a hybrid.
WhatsApp caps groups, so it does not. The product decision removed an entire class of engineering problem.

UNDER ENCRYPTION, though, the sender does the work:
1,000 members x ~2 devices = 2,000 separate encryptions
ON A PHONE, before the message is even sent

→ the Signal "sender key" optimisation: encrypt ONCE with a group key, distribute that key pairwise. 2,000 encryptions → 1, plus a one-off key distribution.

What encryption takes away:

the server holds ciphertext and cannot read it. Therefore:

server-side search	IMPOSSIBLE → search on the device only
content spam filtering	IMPOSSIBLE → metadata only: who, how often, how many recipients, how fast
server-side backup	IMPOSSIBLE without a user-held key → lose the key, lose the history
adding a new device	the server cannot re-encrypt for it → device-to-device key transfer, or per-device keys and N-way encryption

These are the answer to "how does E2E affect the design".
The protocol names are one sentence; the LOST CAPABILITIES are the substance.

5 How it actually works

The connection registry

```
def on_connect(user_id: int, device_id: str, server_id: str) → None:
    redis.hset(f"conn:{user_id}", device_id, server_id)
    redis.expire(f"conn:{user_id}", 300)          # heartbeats refresh it
    deliver_queued(user_id, device_id)           # drain the offline queue

def on_disconnect(user_id: int, device_id: str) → None:
    redis.hdel(f"conn:{user_id}", device_id)
```

A hash per user, not a single value, because one user has several devices — phone, laptop, tablet — and each needs its own entry. **A message goes to all of them.**

The expire matters, because disconnects are not always clean: a phone that loses signal never sends a disconnect, and without a TTL the registry fills with entries pointing at connections that no longer exist. **Heartbeats refresh it; silence lets it lapse.**

Sending a message

```
def handle_send(sender_id: int, msg: dict) → dict:
    client_id = msg["client_message_id"]          # CLIENT-generated: idempotency
    if existing := message_store.by_client_id(sender_id, client_id):
        return {"id": existing.id, "duplicate": True}    # a retry, not a new one

    message = message_store.create(
        id=snowflake.next_id(),
        conversation_id=msg["conversation_id"],
        sequence=next_sequence(msg["conversation_id"]),
        sender_id=sender_id,
        ciphertext=msg["ciphertext"],              # the server never sees plaintext
        created_at=time.time(),
    )
    ack(sender_id, message.id)                     # ONE TICK – after persisting
    route(message)
    return {"id": message.id}
```

The **client_message_id** check is the idempotency, and it must be the client's id: a **retry after a lost acknowledgement carries the same client id, while a server-generated id would differ on every attempt**. That is the same rule as [day 122's](#) idempotency keys.

ack comes after message_store.create. Reverse them and a crash between the two loses a message the sender believes was delivered — **and the sender will never retry, because it saw the tick.**

Routing

```
def route(message) → None:
    for recipient_id in conversation.members(message.conversation_id):
        if recipient_id == message.sender_id:
            continue
        devices = redis.hgetall(f"conn:{recipient_id}")
        if devices:
            for device_id, server_id in devices.items():
                push_to_server(server_id, device_id, message)
        else:
            offline_queue.push(recipient_id, message)    # the shelf
```

A message goes to every device of every recipient, which is why the registry is a hash. **And a user with one device online and one offline needs both paths** — push to the connected one, queue for the other — so in practice the check is per device rather than per user.

Delivery acknowledgement

```
def handle_delivery_ack(recipient_id: int, message_id: int) → None:
    message_store.mark_delivered(message_id, recipient_id)
    offline_queue.remove(recipient_id, message_id)          # only NOW is it safe
    notify_sender(message_id, "delivered")                  # TWO TICKS

def handle_read_receipt(recipient_id: int, message_id: int) → None:
    message_store.mark_read(message_id, recipient_id)
    notify_sender(message_id, "read")                        # BLUE TICKS
```

Removing from the queue only after the acknowledgement is the delivery guarantee. Delete on push instead and a message dropped by a flaky connection is gone forever. **The consequence is at-least-once** — the message can be pushed twice — which the client's id-based deduplication absorbs.

Two separate handlers because they are two separate events, and read receipts can be turned off by the user while delivery receipts cannot.

The offline queue

```
def push(user_id: int, message) → None:
    redis.zadd(f"queue:{user_id}", {message.id: message.sequence})
    redis.expire(f"queue:{user_id}", 30 * 86400)             # 30-day give-up

def drain(user_id: int, device_id: str) → None:
    for message_id in redis.zrange(f"queue:{user_id}", 0, -1): # IN ORDER
        push_to_device(device_id, message_store.get(message_id))
```

zrange on a sorted set scored by sequence delivers in order, which is the whole reason for the structure.

The 30-day expiry is the give-up point. A user who never comes back does not accumulate messages forever — **and the sender is told the message was never delivered, which is more honest than an unbounded queue.**

Groups

```
MAX_GROUP = 1024

def send_to_group(sender_id: int, group_id: int, ciphertexts: dict) → None:
    members = group_store.members(group_id)          # ≤ 1024, bounded
    for member_id in members:
        if member_id == sender_id:
            continue
        for device_id in device_store.devices(member_id):
            route_one(device_id, ciphertexts[device_id]) # per-device ciphertext
```

ciphertexts is keyed per device, because under end-to-end encryption the sender encrypted separately for each one. **The server is a router that cannot read anything it routes.**

The cap is what makes this simple, and it is a product decision doing the work of an architecture: **a thousand deliveries is a loop, not a fan-out system.**

Media

```
def send_media(sender_id: int, conversation_id: int, blob: bytes) → dict:
    key = secrets.token_bytes(32)
    ciphertext = aes_gcm_encrypt(blob, key)
    content_hash = hashlib.sha256(ciphertext).hexdigest()

    if not blob_store.exists(content_hash):           # forwarding is free
        blob_store.put(content_hash, ciphertext)

    return {"url": f"/media/{content_hash}",
            "key": key}                               # sent INSIDE the message
```

The decryption key travels inside the end-to-end encrypted message, so the server stores a blob it cannot decrypt and has no way to obtain the key.

And the content hash gives free deduplication of forwards — the same viral video is stored once however many times it is forwarded, which is why forwarding a 30 MB video is instant. **Note it is the hash of the ciphertext, so this only works when the same key is reused for a forward**, which is exactly what “forwarding” means as opposed to re-uploading.

Presence, which is deceptively expensive

```
def on_heartbeat(user_id: int) → None:
    redis.set(f"last_seen:{user_id}", time.time(), ex=60)
    for watcher in presence_subscriptions(user_id):    # who has this chat open
        push_presence(watcher, user_id, "online")
```

Presence is the most expensive small feature in the system, because it changes constantly and fans out to everyone currently looking at that chat. **The mitigation is to send presence only to users with the conversation open**, not to every contact — and to throttle updates to at most one every few seconds.

The real systems

Erlang / BEAM	WhatsApp's original stack — built for millions of lightweight processes, one per connection
ejabberd	the XMPP server WhatsApp forked
Signal Protocol	X3DH for key agreement, Double Ratchet for forward secrecy
Mnesia	Erlang's built-in store, used for routing state
FreeBSD	WhatsApp famously ran ~2M connections on ONE machine

That last figure is the good one to cite: WhatsApp demonstrated two million concurrent connections on a single server in 2012, which reframes the whole problem — **a hundred million connections is dozens of machines, not thousands.**

6 The numbers

Traffic.

```
2,000,000,000 users
500,000,000 concurrent at peak
100,000,000,000 messages/day

100e9 / 86,400 = ~1,160,000 messages/second average
peak ~3x      = ~3,500,000 messages/second
```

A million messages a second is a genuinely large write rate, and it is the opposite of the social products: **write-heavy, tiny payloads, point-to-point.**

Connections, which is the real constraint.

```
500,000,000 concurrent WebSocket connections

per connection: ~10 KB of kernel and application memory (tuned)
                  a naive stack uses 50-100 KB

500,000,000 x 10 KB = 5 TB of RAM across the fleet

at 128 GB per machine, ~40 GB usable for connections:
  → 4,000,000 connections per machine (aggressive, WhatsApp-style)
  → ~125 machines

at a more typical 500,000 per machine:
  → 1,000 machines
```

The spread between those two numbers is the entire engineering story of the connection tier, and it is why WhatsApp used Erlang: the per-connection overhead is the design.

Message size and bandwidth.

```
average text message: ~200 bytes of ciphertext + ~100 bytes of metadata

1,160,000 msg/s x 300 B = 348 MB/s = ~2.8 Gbps

compare Instagram: 350 Gbps

→ 125x less bandwidth for 40x more events.
    Messaging is about CONNECTIONS and LATENCY, not bytes.
```

Storage, and this is where the design choice shows.

```
IF messages were stored forever:
  100e9/day x 300 B = 30 TB/day = 11 PB/year
  x 3 replicas      = 33 PB/year

WhatsApp's actual model: store only until delivered
  ~95% of messages are delivered within seconds
  offline queue holds maybe 1% of a day's messages at any moment
  100e9 x 1% x 300 B = 300 GB
  plus a 30-day tail for long-offline users: a few TB

→ the server-side storage is measured in TERABYTES, not petabytes.
    The phone is the archive.
```

That contrast — 33 petabytes a year against a few terabytes — is the single best number in this interview.

Media, which is the actual storage:

~5,000,000,000 media messages/day, average 1 MB
but heavily deduplicated by content hash (forwards)
effective unique: maybe 20%

$5e9 \times 1 \text{ MB} \times 20\% = 1 \text{ PB/day}$ of unique media
retained 30 days = 30 PB

→ media is 10,000x the storage of the text messages.
Same as Instagram: the bytes are all pictures.

Latency budget.

phone → connection server (WebSocket, already open)	~30 ms
persist + ack	~5 ms
registry lookup	~1 ms
push to the recipient's connection server	~2 ms
connection server → recipient's phone	~30 ms

	~70 ms

users perceive under ~200 ms as instant, so there is real headroom
– and it is spent on mobile network variance, not on the servers.

Groups.

a 1,000-member group message:
1,000 registry lookups (pipelined) ~5 ms
~2,000 device pushes ~20 ms
→ under 50 ms, bounded, no fan-out infrastructure

and UNDER ENCRYPTION, on the sender's phone:
2,000 encryptions at ~1 ms each = 2 SECONDS
→ unusable

with sender keys:
1 encryption + a one-off key distribution
→ milliseconds

That two-second figure is the concrete justification for sender keys, and it is a better answer than naming the protocol.

Presence, which is surprisingly large:

```
500,000,000 online users
each changes state (typing, online, last-seen) ~20 times/hour
  = 10,000,000,000 events/hour = ~2,800,000 events/second

if each fanned out to 100 contacts: 280,000,000 pushes/second
  → LARGER than the message traffic itself

mitigation: send presence only to users with that chat OPEN
  ~1-2 watchers per user on average
  → ~5,000,000 pushes/second. Manageable.
```

Presence being bigger than messaging without that mitigation is the surprise, and noticing it unprompted is a strong signal.

Cost, roughly:

connection tier	~500 machines	~\$500,000/month
message store	modest — TBs, not PBs	~\$50,000/month
media storage	30 PB + CDN egress	~\$1,500,000/month

		~\$2M/month for 2 billion users

= about \$0.001 per user per month, which is why the product could be free with 50 engineers.

7 The trade-offs

Stateful connections against a stateless tier. WebSockets are the only reasonable way to push, and they make the connection servers stateful: **a deploy disconnects everybody it holds, and a crash disconnects theirs at once.** Reconnection storms are real, so clients need exponential backoff with jitter — [day 125's](#) thundering herd, caused by your own deploy.

At-least-once against at-most-once. Removing a message from the queue only after acknowledgement means a message can be delivered twice; **deleting on push means a dropped connection loses it forever.** For messaging, a duplicate the client can deduplicate is strictly better than a loss — **so at-least-once plus client-side ids, always.**

Per-conversation ordering against global ordering. Per-conversation is cheap, is what users can perceive, and lets each client detect gaps. **Global ordering would need cluster-wide coordination and would buy nothing,** because nobody can observe the relative order of two different chats.

Store-until-delivered against store-forever. WhatsApp's choice makes server storage tiny and gives a genuine privacy property — **there is no archive to subpoena**. The cost is that **the phone is the archive**, so losing a phone loses history unless the user made a backup, and restoring is a user-visible chore. **A product that stores server-side gets seamless multi-device and search, and gives up the privacy claim.**

End-to-end encryption against every server-side feature. It is the right default for a messenger, and the costs are not small: **no server-side search, no content-based spam filtering, no server-side backup, and multi-device becomes a real engineering problem** because the server cannot re-encrypt for a new device. **Spam is fought on metadata alone** — who, how often, to how many, how fast — which works less well and is the honest answer.

Group size caps against unlimited groups. Capping at about a thousand removes the celebrity problem entirely: **a bounded fan-out is a loop**. Uncapped groups would need Twitter's hybrid and would break the sender-side encryption cost even harder. **This is the clearest case in the course of a product decision deleting an architecture problem**, and it is worth saying in those words.

When would I not build this? When the product is a chat feature rather than a messenger — support chat, a game lobby, comments on a document — where a managed service or a simple long-polling endpoint over the existing database is the entire thing. **When encryption is not required**, which removes most of the difficulty and enables search and moderation. **And for anything small, a hosted service** — Twilio Conversations, Stream, Sendbird — is cheaper than the connection tier alone.

8 In the interview

How it gets asked

- “*Design WhatsApp.*” — usually with a scale like a billion users.
- “*How does the server push a message to a phone?*” — the WebSocket question.
- “*How do the two ticks work?*” — the acknowledgement protocol.
- “*What happens if the recipient is offline?*”
- “*How does end-to-end encryption change the design?*” — the best question in the set.
- “*How do groups work?*”

“This is a completely different shape from the last few systems, and I want to say why before designing anything.

Twitter and Instagram are read-heavy broadcast systems — one write, many reads, and staleness is invisible. Messaging is **write-heavy and point-to-point**: every message has one or a few recipients, it must arrive in under a second, **and a duplicate or an out-of-order message is something a human immediately notices.**

Three things drive the design.

First, the server must push. A phone receives messages it did not ask for, so polling is wrong — five seconds of latency and ninety-nine percent of requests returning nothing. **WebSockets: one persistent connection per device.**

And the consequence is that my servers are now stateful. A connection server owns specific users until they disconnect, so I need **a registry mapping user to connection server** — Redis, a hash per user because a user has several devices — and every delivery starts with a lookup in it.

Second, the guarantees are visible to users. Sent, delivered, read are **three distinct events**, which is exactly what one tick, two ticks and blue ticks are. **Conflating delivered and read is the classic mistake** — the device having it and a human having read it are different, and the whole design exists because they differ.

And persist before acknowledging. If I ack and then crash, the message is gone and the sender saw a tick, so it will never retry. **Write first, ack second, and the order is the guarantee.**

Third, end-to-end encryption, which removes options rather than adding components. The server holds ciphertext, so there is **no server-side search, no content-based spam filtering, and no server-side backup** — and adding a new device is a real problem because the server cannot re-encrypt for it. **Those lost capabilities are the substance; the protocol names are one sentence.**

Scale, and one number reframes it. A hundred billion messages a day is about a million a second — large. **But the payloads are three hundred bytes, so it is 2.8 gigabits a second, against Instagram’s 350.** This system is bounded by

concurrent connections and latency, not by bytes — five hundred million connections at about ten kilobytes each is five terabytes of RAM, and WhatsApp famously ran two million connections on one machine.

One question: do I need message history on the server, or is the phone the archive? Because WhatsApp deletes after delivery, and that single choice is the difference between a few terabytes and thirty-three petabytes a year.”

The follow-ups

“How do the ticks work, and what happens if the recipient is offline?”

“Three separate events with three separate acknowledgements, and the offline case is where the delivery guarantee lives.

One tick is ‘the server has it’. Alice’s phone sends over its WebSocket, the server **persists the message and then acknowledges**. That order is not stylistic: **if I ack first and crash, the message is lost and Alice saw a tick, so her client will never retry.**

Two ticks is ‘the recipient’s device has it’. The server looks Bob up in the registry, pushes over his connection, **and Bob’s device sends back a delivery acknowledgement**, which the server relays to Alice.

Blue ticks is ‘a human opened the chat’. A separate event from the device, which is why users can turn read receipts off but not delivery receipts — **they are genuinely different facts.**

Now offline. Bob is not in the registry, so the message goes into a **per-user off-line queue** — a Redis sorted set scored by sequence number, so it drains in order — and Alice sees one tick and no more.

When Bob reconnects, the queue drains in order, and each message is removed only after his device acknowledges it. That last part is the guarantee: **delete on push instead, and a message dropped by a flaky connection is gone forever.**

The consequence is at-least-once delivery — a message can be pushed twice if an acknowledgement is lost — **which the client absorbs by deduplicating on the message id.** For messaging that is unambiguously the right trade: a duplicate the client can filter is much better than a loss the user notices.

And I would cap the queue at about thirty days. A user who never returns should not accumulate messages forever, and **telling the sender it was never delivered is more honest than an unbounded queue.**

One more piece: if Bob is offline entirely, a push notification wakes the app — through APNs or FCM, carrying no content because the server cannot read it, just a signal to connect and fetch.”

“How does end-to-end encryption change the design?”

“It changes almost nothing about the message flow and it removes four capabilities, and **the lost capabilities are the real answer** — the protocol names are a sentence.

The mechanics: keys live on devices. The server distributes public keys and routes ciphertext it cannot read. Signal’s protocol — X3DH to agree a key, Double Ratchet for forward secrecy so a compromised key does not expose past messages. **That is as much as I would say about the cryptography.**

What it costs, and each one is a product feature that has to move or die.

No server-side search. Searching your own chat history has to happen on the device, over locally stored data. **Which is fine for a phone and awkward for a new device with no history.**

No content-based spam filtering. I cannot see what is being sent, **so abuse detection runs on metadata alone:** who is messaging whom, how often, how many distinct recipients, how fast a new account ramps up. **That works less well than reading the content, and it is the honest answer** rather than pretending encryption is free.

No server-side backup. A backup is either unencrypted — which throws away the property — or encrypted with a key only the user holds, **and then losing the key loses the history permanently.** That is a real support burden and a real product decision.

And multi-device is genuinely hard. The server cannot re-encrypt an old message for a new device, because it cannot read it. **So either the new device gets keys transferred from an existing one, or each device has its own identity and every message is encrypted separately per device.**

That last one is what makes groups expensive. With per-device encryption, a thousand-member group with two devices each means **two thousand separate encryptions on the sender’s phone before the message is even sent** — at about a millisecond each, that is two seconds, which is unusable.

Hence sender keys: encrypt the message once with a group key, and distribute that group key pairwise, once. **Two thousand encryptions become one plus a one-off setup,** and that arithmetic is why the optimisation exists.”

“How do groups work?”

“Groups are fan-out, and the interesting part is that a **product decision deletes the hard version of the problem.**

WhatsApp caps groups at around a thousand members. So a group message is a bounded loop: look up each member in the registry, push to each of their devices, queue for the offline ones. **A thousand deliveries is under fifty milliseconds and needs no fan-out infrastructure at all.**

Compare Twitter, which cannot cap followers and therefore has to invent a hybrid push-pull design for celebrities. **Here the product simply forbids the case that would require it,** and I think that is worth saying explicitly, because recognising when a constraint can be imposed rather than engineered around is a real skill.

Where it does get expensive is encryption, on the sender’s device. With per-device keys, the sender encrypts separately for every recipient device — a thousand members with two devices each is two thousand encryptions, about two seconds on a phone. **Sender keys fix it: one encryption with a shared group key, and the key distributed pairwise once.**

Membership changes are the fiddly part under encryption. When someone leaves a group, **the group key must be rotated,** or they could still decrypt future messages. When someone joins, they get the current key and **cannot read anything sent before** — which is a deliberate property, and a support question.

And a detail that matters at scale: I would store group membership separately from the message path, and cache it aggressively, because every message triggers a membership read. **A group of a thousand active people generates a lot of reads of the same small list.**

One thing I would raise: the cap is doing more work than it looks. If the product wanted broadcast channels with a million subscribers — which WhatsApp later added — **that is a different system,** closer to a feed, and I would build it separately rather than removing the group cap.”

The model answer

“Design WhatsApp: two billion users, five hundred million concurrent, a hundred billion messages a day, end-to-end encrypted.”

“The constraints in that prompt point in an unusual direction, so let me name the shape first.

A hundred billion messages a day is about a million a second — write-heavy, which is the opposite of the social products. But the payloads are three hundred bytes, so the total bandwidth is **2.8 gigabits a second, against Instagram’s three hundred and fifty.** This system is bounded by concurrent connections and by latency, not by bytes, and that reframes what to optimise.

The connection tier, which is the hard part. Five hundred million concurrent WebSockets. At about ten kilobytes per connection — kernel buffers plus application state, aggressively tuned — that is **five terabytes of RAM across the fleet.** WhatsApp demonstrated two million connections on one machine using Erlang, so **this is on the order of a few hundred machines, not thousands** — and the per-connection overhead is the entire engineering story of this tier.

A registry in Redis maps user to connection server, a hash per user because a user has several devices. **With a TTL refreshed by heartbeats,** because a phone that loses signal never sends a clean disconnect and without expiry the registry fills with dead entries.

The message flow: persist, then ack, then route. One tick after the write, two ticks after the device acknowledges, blue ticks after the human opens the chat — **three separate events, and conflating delivered with read is the classic error.**

Ordering is per conversation, not global — a sequence number per conversation so a client can detect a gap and request a resend. **Global ordering would need cluster-wide coordination and nobody can perceive it.**

Idempotency on a client-generated message id, because a retry after a lost ack carries the same id and a server-generated one would not.

Offline: a per-user sorted set, drained in order, and messages removed only after acknowledgement. At-least-once, absorbed by client-side deduplication — **a duplicate is much better than a loss here.** Thirty-day expiry, and the sender is told when a message was never delivered.

Now the storage decision, which is the biggest number in the design. If messages were kept forever, a hundred billion a day at three hundred bytes is **eleven petabytes a year, thirty-three with replicas.** Storing only until delivery makes it a few terabytes — because ninety-five percent are delivered within seconds. **Four orders of magnitude, from one product decision.**

The cost is that the phone is the archive, so a lost phone loses history and restoring a backup is a user-visible chore. **I would state that as the deliberate trade it is**, along with its privacy benefit: there is no server-side archive to compel.

Media is where the actual storage is — five billion media messages a day, deduplicated by content hash so forwarding a viral video stores nothing new — **about a petabyte a day of unique content, thirty petabytes retained**. Ten thousand times the text. **Same as Instagram: the bytes are all pictures**.

Encryption, and its costs. Signal protocol, keys on devices, the server routes ciphertext. **No server-side search, no content-based spam filtering — abuse detection is metadata-only — no server-side backup without a user-held key, and multi-device requires per-device keys**. That last one makes groups expensive: **a thousand-member group at two devices each is two thousand encryptions on a phone, about two seconds, hence sender keys**.

Groups are capped at around a thousand, which removes the celebrity fan-out problem entirely — **a bounded fan-out is a loop**. A product decision doing the work of an architecture.

Two things I would flag as risks.

Deploys and reconnection storms. A connection server going down disconnects everyone it holds, and they all reconnect at once. **Clients need exponential backoff with jitter**, and deploys need to drain slowly — otherwise my own release is a self-inflicted thundering herd.

And presence, which is bigger than messaging if built naively. Five hundred million users changing state twenty times an hour is 2.8 million events a second, and **fanning each to a hundred contacts would be 280 million pushes a second — larger than the message traffic itself**. The mitigation is to send presence only to users with that chat currently open, which takes it to about five million a second. **I would design that constraint in from the start rather than discover it.**

9 Recall card

Opposite shape from the social systems: write-heavy, point-to-point, latency-critical, and duplicates and reordering are user-visible. 100B messages/day is ~1M/s, but at 300 bytes that is 2.8 Gbps against Instagram's 350 — **bounded by**

connections and latency, not bytes.

WebSockets, one per device, so the servers are STATEFUL. A registry (`user → connection server`), a hash per user for multiple devices, **with a heartbeat-refreshed TTL** because unclean disconnects are normal). 500M connections $\times \sim 10 \text{ KB} = 5 \text{ TB}$ of RAM; WhatsApp ran 2M connections on one machine.

Three distinct events: sent / delivered / read = one tick / two ticks / blue ticks. Persist BEFORE acking — ack-then-crash loses a message the sender believes was sent, and the client will never retry. **Idempotency on a CLIENT-generated message id. Ordering is per conversation, not global.**

Offline queue = a sorted set drained in order, and messages are removed only after acknowledgement — delete-on-push loses messages. At-least-once, absorbed by client-side dedup. 30-day give-up.

Store-until-delivered is the biggest number in the design: 33 PB/year → a few TB. The phone becomes the archive. **Media is the real storage** ($\sim 1 \text{ PB/day}$ unique after content-hash dedup, which is why forwarding is instant).

E2E encryption removes capabilities, and that is the answer: no server-side search, **spam fought on metadata only**, no server-side backup without a user-held key, and multi-device needs per-device keys — which makes a 1,000-member group **2,000 encryptions ≈ 2 seconds on a phone**, hence **sender keys**. **Groups are capped at $\sim 1,000$, which deletes the celebrity fan-out problem** — a product decision doing an architecture's job. **And presence, built naively, is larger than the message traffic itself.**

DSA topic: Grid DP: unique paths **System design topic:** Design WhatsApp

Code these, in this order

One rule for the whole set: **write the base cases before the recurrence, and say what they mean.** The recurrence barely changes across these six problems. The base cases change every time, and that is where every bug is.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Unique Paths	LeetCode 62 (Medium)	The cleanest state in DP, and the one-row collapse.
2	Unique Paths II	LeetCode 63 (Medium)	Obstacles, and the first row that is no longer all ones.
3	Minimum Path Sum	LeetCode 64 (Medium)	Same state, <code>min</code> , and running-sum base cases.
4	Triangle	LeetCode 120 (Medium)	A grid that is not rectangular, and filling upwards.
5	Minimum Falling Path Sum	LeetCode 931 (Medium)	Where the one-row trick breaks, and why.
6	Maximal Square	LeetCode 221 (Medium)	An “ending at” state, so the answer is not the last cell.

On problem 1, do it three ways

Write the full table, the one-row version and the closed form. Confirm all three agree on ten random sizes. **Then say, in one sentence each, what the one-row version costs you and what the formula costs you.**

On problem 1, point at the two values

In the one-row version, print `row` mid-pass and mark which entries are the previous row and which are the current one. **Say which side of the cursor is which**, and why that makes `row[c] += row[c-1]` correct.

On problem 2, keep the initialise-to-1 trick

Use `[[1] * cols for _ in range(rows)]` with obstacles. Run on a 3×4 grid with one obstacle in the top row and record the answer. Then run the no-obstacle version on the same shape. **Say why the two numbers are the same and what that tells you.**

On problem 3, keep the counting base cases

Leave the first row and column as zeros. Run on `[[1,3,1],[1,5,1],[4,2,1]]` and record the answer against the correct 7. **Say what the algorithm believed about the edges.**

On problem 3, use 0 for a blocked cell

Add an obstacle and mark it `0` instead of infinity. Say what the `min` does with it. **Then say why the counting version uses `0` and the cost version cannot.**

On problem 5, apply the one-row trick anyway

Update the row in place. Run on `[[2,1,3],[6,5,4],[7,8,9]]` and record the answer against the correct one. **Say exactly which read was of the wrong generation, and what the precondition for the trick actually is.**

On problem 6, return the last cell

Run `maximal_square` returning `dp[-1][-1]` instead of the maximum. Find an input where it differs. **Say which other problem this week had the same property, and what the two states have in common.**

Then the counting-overflow drill

Compute `unique_paths_formula` for a 40×40 grid and count the digits. Say what a Java `long` would have done. **Then say which of the two approaches — table or formula — handles a required modulus more easily, and why.**

The state drill

1. State it in one sentence.
2. Say why no redefinition is needed here, unlike the last four days.
3. Say why adding, not maxing, is correct for the counting version.
4. Give the base cases and what each means in words.

The collapse drill

1. Say which two cells the recurrence reads.

2. Say what `row[c]` is before it is overwritten, and what `row[c-1]` is.
3. Give the space numbers at $1,000^2$ and $10,000^2$.
4. State the precondition for the trick in one sentence.
5. Name the variant that violates it and say what it needs instead.

The base-case drill

For each, give the first row and first column and say why:

1. Counting paths, no obstacles.
2. Counting paths with obstacles.
3. Minimum path sum.
4. Minimum path sum with blocked cells.

The formula drill

1. Derive it: how many moves, how many are down.
2. Give the expression and check it on a 2×2 grid.
3. Give the time and space.
4. Say what breaks it, and why inclusion-exclusion does not save you.
5. Say what happens if the problem demands a modulus.

The family drill

Name the change for each:

1. Minimum path sum.
2. Obstacles.
3. Diagonal moves allowed.
4. Minimum falling path.
5. Maximal square.
6. Two walkers at once.

The break-it drill

Trigger each and record the exact output or error:

1. Initialise-to-1 with an obstacle.
2. Zero base cases in minimum path sum.
3. A blocked cell marked `0` in the cost version.
4. The one-row trick on minimum falling path.

5. Returning `dp[-1][-1]` from maximal square.
6. `math.comb(m+n, m)` instead of `(m+n-2, m-1)`.
7. An empty grid.

Six of the seven give no error at all. Name them.

The shape drill

1. Say how messaging differs from a social feed, in three points.
2. Give the messages-per-second and the bandwidth, and the comparison with Instagram.
3. Say what the system is actually bounded by.

The connection drill

1. Say why polling is wrong, with the numbers.
2. Say what the registry holds and why it is a hash.
3. Say why it needs a TTL.
4. Give the memory per connection and the fleet total.
5. Say what statefulness costs at deploy time and what clients must do.

The ticks drill

1. Name the three events and what each actually means.
2. Say which two people conflate and why they are different.
3. Say why persist comes before ack, and what goes wrong otherwise.
4. Say why the sender never retries after a bad ack.

The offline drill

1. Describe the queue structure and why that structure.
2. Say when a message is removed, and what the alternative loses.
3. Say what delivery guarantee this gives and how the client copes.
4. Say what the expiry is for and what the sender is told.
5. Say how an app that is not running at all gets woken.

The ordering drill

1. Say what ordering is required and what is not.
2. Give the example that shows why within-conversation matters.
3. Say what a sequence gap lets the client do.

4. Say what global ordering would cost and buy.

The encryption drill

1. Name the four capabilities it removes.
2. For each, say where the capability goes instead.
3. Say what spam filtering can still use.
4. Say why multi-device is hard.
5. Do the group encryption arithmetic and give the fix.

The groups drill

1. Say what the cap is and what it removes.
2. Compare with Twitter's follower problem in one sentence.
3. Say what happens to keys when someone leaves, and when someone joins.
4. Say what you would do if the product wanted million-member channels.

The storage drill

1. Compute store-forever and store-until-delivered.
2. Say what the ratio is and what it costs the user.
3. Say where the actual storage is, and what makes forwarding free.

The presence drill

1. Compute the naive event rate and fan-out.
2. Compare it with the message traffic.
3. Give the mitigation and the resulting number.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *How many unique paths are there through the grid?* The state, why addition is correct, the base cases, the one-row collapse with what each value is, and the closed form with its limitation.
2. *Now some cells are blocked, and each cell has a cost.* What changes in the recurrence, what changes in the base cases, why a blocked cell is `0` in one version and `inf` in the other, and why the formula dies.

3. *Design WhatsApp*. Why it is a different shape, WebSockets and the registry, the three ticks with persist-before-ack, the offline queue with removal-after-ack, and the four things encryption removes.

Before you move on

- [] I can state the grid DP state in one sentence.
- [] I know why addition counts each path exactly once.
- [] I can give the base cases for all four variants.
- [] I know the initialise-to-1 trick and exactly when it breaks.
- [] I can write the one-row version and point at both values.
- [] I can state the precondition for the collapse.
- [] I know which variant violates it and what it needs.
- [] I know a blocked cell is `0` when counting and `inf` when costing.
- [] I can derive the closed form and check it on a 2×2 grid.
- [] I know why one obstacle destroys the formula.
- [] I know which approach handles a modulus more easily.
- [] I know maximal square's answer is the table maximum.
- [] I can reconstruct a path and know it needs the full table.
- [] I can give the space numbers at $1,000^2$ and $10,000^2$.
- [] I know the counts overflow a 64-bit integer by 40×40 .
- [] I can say how messaging differs from a feed, in three points.
- [] I know the bandwidth comparison and what really bounds the system.
- [] I can explain why polling is wrong, with numbers.
- [] I know what the registry holds and why it needs a TTL.
- [] I can give per-connection memory and the fleet total.
- [] I know the three tick events and why delivered $\neq$ read.
- [] I know persist comes before ack, and what breaks otherwise.
- [] I can describe the offline queue and when messages are removed.
- [] I know this gives at-least-once and how the client copes.
- [] I know ordering is per conversation, and why.
- [] I can name the four capabilities encryption removes.
- [] I know spam filtering falls back to metadata.

- [] I can do the group encryption arithmetic and name the fix.
- [] I know the group cap deletes the fan-out problem.
- [] I can give both storage numbers and the ratio.
- [] I know presence is bigger than messaging without its mitigation.
- [] I answered all three questions above out loud.

DP on decisions: buy and sell stock

DATA STRUCTURES AND ALGORITHMS

DP on decisions: buy and sell stock

SYSTEM DESIGN

Design a chat system with presence

1 What this is, and why they ask it

You have the price of something on each of n days. You may buy and sell. Maximise your profit.

That one sentence is a whole family of interview questions, and the family is the point. **One transaction. Unlimited transactions. At most k transactions. With a cooldown. With a fee.** Six problems that look different and are the same problem with the state adjusted.

They ask it because **it is the cleanest example of a state that is not a position.** Every DP so far has been indexed by where you are — an amount, a cell, an index into a string. **Here the state is (day, what you are currently holding)**, and that second component is a *mode*, not a place. Learning to say “I need a dimension for which situation I am in” is the skill, and once you have it, the cooldown and the transaction-limit variants follow in about a minute each.

The other reason is that **the variants are a ladder, and the interviewer walks up it.** They start with one transaction, which greedy solves. Then unlimited, which greedy also solves and DP explains. Then k transactions, where greedy dies and you need the third dimension. **A candidate who has one memorised solution falls off the ladder at step three;** a candidate who has the state framework answers all of them from the same starting point.

By the end of this lesson you can write the two-state machine, extend it to k transactions and to a cooldown and a fee, say where the transaction count is consumed and why that must be stated, and know which variants greedy solves.

2 The story

Sarala bought and sold gold, in very small amounts, and she was the only person in her family who understood why she was not doing what they thought she was doing.

The shop bought at one price and sold at another and the difference was the shop's. Everybody knew that. What her brother did not understand, and asked about at every function for eleven years, was why she did not simply buy when it was cheap and sell when it was dear.

“That is exactly what I do,” she said. “The difficulty is that I only know it was cheap afterwards.”

But that was not the real difficulty, and it took her a while to be able to say what was.

The real difficulty was that on any given morning she was in one of two situations, and they were completely different.

Either she had money and no gold. Then the only question was whether to buy.

Or she had gold and no money. Then the only question was whether to sell.

And she could not do both on the same morning. She could not sell gold she had not bought. She could not buy again while everything she had was already gold.

Her brother said this was obvious.

“Then tell me what I should do on Thursday.”

And he could not, because the answer depended on which of the two situations she would be in on Thursday, and that depended on what she did on Wednesday, and Tuesday, all the way back.

There was a third thing, which came later and which she only mentioned once.

The dealer she used had a rule that after you sold to him you could not buy from him again the next day. Not a real rule, just how he was. **So selling on Monday did not put her back in the buying situation on Tuesday. It put her in a third situation — no gold, no buying — that lasted one day and then became the ordinary one.**

Her brother, who by this point had stopped asking, said it sounded complicated.

“It is three situations instead of two,” she said. “That is all. The question on each morning is the same question. It is just that there are three of them now.”

3 The idea in plain English

Sarala’s two situations are the two states, her third one is the cooldown, and her sentence at the end is the whole method.

Start with the state, because it is what makes this family tractable.

`dp[day][holding]` is the most money you can have at the end of `day`, given that `holding` is 1 if you currently own the stock and 0 if you do not.

That second dimension is a *mode*, not a position — and this is the new idea. Every previous DP indexed by where you are. Here you also need to know what situation you are in, because it determines which moves are legal.

Why the naive state fails. `dp[day]` alone — “the best profit using the first `day` days” — cannot express the recurrence, because **you cannot decide whether selling today is legal without knowing whether you are holding**. Same failure as longest increasing subsequence: the state is incomplete, and it fails silently.

The transitions are Sarala’s two questions.

If you are not holding at the end of today, either you were not holding yesterday and did nothing, or you were holding and sold:

```
dp[day][0] = max( dp[day-1][0],           do nothing  
                  dp[day-1][1] + price[day] )  sell
```

If you are holding at the end of today, either you were holding and kept it, or you were not and bought:

```
dp[day][1] = max( dp[day-1][1],           do nothing  
                  dp[day-1][0] - price[day] )  buy
```

Buying subtracts, selling adds. Track *money*, not profit, and the arithmetic never confuses you.

The base cases. On day 0, not holding means you have zero: `dp[0][0] = 0`. Holding means you bought today: `dp[0][1] = -price[0]`.

And the answer is `dp[n-1][0]`, not the maximum of the two. Ending while still holding is never optimal — you paid for something and did not sell it — so the answer is always the not-holding state.

That is the unlimited-transactions version, LeetCode 122, and it is two lines.

Now the ladder, and each rung changes exactly one thing.

At most `k` transactions. Add a third dimension: `dp[day][transactions_used][holding]`.

And here is the detail that must be stated out loud: where is a transaction consumed? A transaction is a buy and a matching sell. **Count it on the buy, or count it on the sell — either is correct, and you must pick one and be consistent.** Counting on both halves the effective limit, which is the standard bug, and it gives a plausible smaller number.

One optimisation matters here. If $k \geq n/2$, you can never use more transactions than that anyway — each one needs at least two days — **so the limit is not binding and you fall back to the unlimited version.** Without that check, $k = 10^9$ allocates a table with a billion rows and dies.

With a cooldown. Sarala's dealer. After selling you must wait one day before buying. **Three states instead of two:** holding, not-holding-and-free-to-buy, and just-sold. Selling moves you into the third; the third moves into the second the next day and nowhere else.

With a transaction fee. Subtract the fee once per transaction — **on the sell, conventionally, because that is where the round trip completes.** No new dimension at all: **just a constant in one transition**, which is worth noticing, because it is the only variant that costs nothing.

Exactly one transaction. The most restrictive, and greedy solves it: **track the minimum price seen so far and the best profit against it, in one pass.** The DP also works, and the greedy version is what to write.

Now: which variants greedy solves, because the interviewer will ask.

One transaction: greedy works — min-so-far and best-difference.

Unlimited transactions: greedy works, and the reason is neat. Sum every positive day-to-day difference. `sum(max(0, price[i] - price[i-1]))`. **Because with unlimited transactions, capturing every upward step is the same as capturing every rising run** — buying and selling on consecutive days telescopes into buying at the bottom and selling at the top.

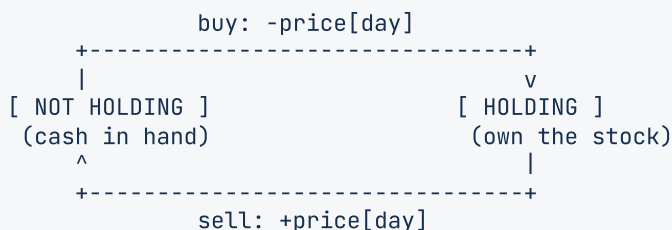
Two or more limited transactions: greedy fails. There is no local rule that knows whether to spend one of your limited transactions on a small rise now or save it for a bigger one later. **That is where the DP earns its place**, and saying it is a better answer than writing the code.

Cooldown and fee: greedy fails too, for the same reason — a local decision cannot account for a cost that falls in the future.

Finally, the space collapse. Every transition reads only yesterday, so **you keep a constant number of variables**, not a table. The unlimited version becomes two variables; the cooldown version three; the k -transaction version two arrays of length $k+1$. **$O(1)$ or $O(k)$ space instead of $O(n \times k)$** , and it is the natural way to write these once the state is clear.

4 The picture

The two-state machine:



both states also have a "do nothing" self-loop

```

dp[day][0] = max( dp[day-1][0], dp[day-1][1] + price[day] )
dp[day][1] = max( dp[day-1][1], dp[day-1][0] - price[day] )
  
```

The answer is $dp[n-1][0]$. Ending while HOLDING is never optimal – you paid for something and did not sell it.

The table filling, for `prices = [7, 1, 5, 3, 6, 4]`, unlimited transactions:

day	0	1	2	3	4	5	
price	7	1	5	3	6	4	
not	0	0	4	4	7	7	← answer 7
holding	-7	-1	-1	1	1	3	

```

day 1: holding = max(-7, 0 - 1) = -1    bought at 1
day 2: not     = max(0, -1 + 5) = 4     sold at 5
day 3: holding = max(-1, 4 - 3) = 1     bought at 3
day 4: not     = max(4, 1 + 6) = 7     sold at 6
  
```

profit 7 = (5-1) + (6-3). Two transactions.

Why greedy works for unlimited, and only for unlimited:

```
prices 7 1 5 3 6 4
diffs  -6 +4 -2 +3 -2
```

sum of POSITIVE diffs = 4 + 3 = 7 same answer

WHY: with unlimited transactions, buying and selling every day telescopes:

buy@1 sell@3, buy@3 sell@5 = buy@1 sell@5
so capturing every up-step is the same as capturing every run.

WITH A LIMIT OF 1 TRANSACTION:

greedy on diffs still says 7. The real answer is 5 (buy 1, sell 6).
→ greedy is WRONG the moment transactions are limited,
because no local rule knows whether to spend one.

The cooldown, as Sarala's three situations:

```

                        buy
[ FREE ] -----> [ HOLDING ]
    ^               |
    |               | sell
    | (one day passes) v
[ COOLING ] <-----+

```

FREE = no stock, may buy today
HOLDING = own stock
COOLING = just sold, may NOT buy today

```
free[d]   = max( free[d-1], cooling[d-1] )
holding[d] = max( holding[d-1], free[d-1] - price[d] )
cooling[d] = holding[d-1] + price[d]
```

Three states instead of two. Same question on each morning.

Where the transaction is consumed, which must be stated:

a TRANSACTION = one buy + one matching sell

COUNT ON THE BUY

buy[t] uses one of the t
sell[t] is free

COUNT ON THE SELL

buy[t] is free
sell[t] uses one of the t

BOTH ARE CORRECT. Pick one, say which, be consistent.

COUNTING ON BOTH:

k = 2 behaves like k = 1

→ a smaller, entirely plausible answer

→ no error anywhere

This is the single most common bug in the k-transaction variant.

The whole family, on one page:

variant	what changes	greedy?
1 transaction	$k = 1$	YES (min-so-far)
unlimited	the base machine	YES (positive diffs)
at most k	+ a transactions dimension	NO
with cooldown	2 states $\rightarrow$ 3 states	NO
with a fee	subtract a constant on sell	NO
(nothing else changes in any of them)		

5 The code, built step by step

One transaction, where greedy wins

```
def max_profit_one(prices: list[int]) → int:
    cheapest = float("inf")
    best = 0
    for price in prices:
        cheapest = min(cheapest, price)      # the best day to have bought
        best = max(best, price - cheapest)   # selling here
    return best
```

One pass, `O(1)` space, and no DP needed. Track the minimum so far and the best difference against it — the two lines are independent and the order matters: update `cheapest` first, so selling on the same day you bought gives zero rather than a negative.

Unlimited transactions, the base machine

```
def max_profit_unlimited(prices: list[int]) → int:
    not_holding, holding = 0, -prices[0]
    for price in prices[1:]:
        not_holding = max(not_holding, holding + price)  # sell
        holding = max(holding, not_holding - price)      # buy
    return not_holding
```

Two variables. That is the whole DP.

And there is a subtlety in the order of those two lines. `holding` is computed from the `not_holding` that was just updated this iteration — meaning it allows buying on the same day you sold. For the unlimited version that is harmless, because buying and selling the same stock on the same day nets zero. In the cool-down version it would be a bug, which is why that one uses a saved copy.

The greedy version, for contrast

```
def max_profit_unlimited_greedy(prices: list[int]) → int:
    return sum(max(0, prices[i] - prices[i - 1]) for i in range(1, len(prices)))
```

One line, same answer, and only for the unlimited case. Say why it works — telescoping — and say that it breaks the moment a transaction limit appears.

At most `k` transactions

```
def max_profit_k(prices: list[int], k: int) → int:
    n = len(prices)
    if n < 2 or k == 0:
        return 0
    if k ≥ n // 2:
        return max_profit_unlimited(prices) # limit not binding
                                           # avoids a huge table
```

That early return is not an optimisation, it is a requirement. `k = 109` otherwise allocates a table with a billion rows and raises `MemoryError` — **and LeetCode 188 deliberately includes such an input.**

```
# buy[t] = best money holding stock, having STARTED t transactions
# sell[t] = best money holding nothing, having COMPLETED t transactions
buy = [-prices[0]] * (k + 1)
sell = [0] * (k + 1)
for price in prices[1:]:
    for t in range(1, k + 1):
        buy[t] = max(buy[t], sell[t - 1] - price) # the buy STARTS t
        sell[t] = max(sell[t], buy[t] + price)    # the sell COMPLETES it
return sell[k]
```

The comments are the answer to “where is the transaction consumed”. `buy[t]` comes from `sell[t-1]` — the buy is what increments the count — and the matching sell stays at `t`. **Consistent, stated, and it is what makes the code readable.**

And iterating `t` from 1 upwards while reading `buy[t]` in the same pass is deliberate: it allows the same-day buy-then-sell, which nets zero and is harmless, exactly as in the unlimited version.

With a cooldown

```
def max_profit_cooldown(prices: list[int]) → int:
    free, holding, cooling = 0, -prices[0], float("-inf")
    for price in prices[1:]:
        previous_free, previous_holding = free, holding
        free = max(previous_free, cooling)           # cooldown expired
        holding = max(previous_holding, previous_free - price)
        cooling = previous_holding + price           # sold today
    return max(free, cooling)
```

Saving yesterday's values is the whole point. In the unlimited version, reading the freshly-updated value was harmless; **here it would let you sell and buy on the same day, which is exactly what the cooldown forbids.**

And the answer is `max(free, cooling)`, because the last day might end with a sale — which leaves you in `cooling`, still holding nothing.

With a transaction fee

```
def max_profit_fee(prices: list[int], fee: int) → int:
    not_holding, holding = 0, -prices[0]
    for price in prices[1:]:
        not_holding = max(not_holding, holding + price - fee)  # pay on the sell
        holding = max(holding, not_holding - price)
    return not_holding
```

One constant, one transition, no new dimension. Charging on the sell is the convention because the round trip completes there — charging on the buy also works and gives the same total, provided you do not charge on both.

The complete solution

```
"""Buy and sell stock: one state machine, five variants."""

def max_profit_one(prices: list[int]) → int:
    """At most ONE transaction. Greedy: min so far, best difference."""
    cheapest, best = float("inf"), 0
    for price in prices:
        cheapest = min(cheapest, price)
        best = max(best, price - cheapest)
    return int(best)

def max_profit_unlimited(prices: list[int]) → int:
    """Unlimited transactions. The base two-state machine."""
    if not prices:
        return 0
    not_holding, holding = 0, -prices[0]
    for price in prices[1:]:
        not_holding = max(not_holding, holding + price)    # sell
        holding = max(holding, not_holding - price)        # buy
    return not_holding

def max_profit_unlimited_greedy(prices: list[int]) → int:
    """Same answer, one line. Works ONLY because transactions are unlimited."""
    return sum(max(0, prices[i] - prices[i - 1]) for i in range(1, len(prices)))

def max_profit_k(prices: list[int], k: int) → int:
    """At most k transactions. A transaction is CONSUMED ON THE BUY."""
    n = len(prices)
    if n < 2 or k == 0:
        return 0
    if k ≥ n // 2:
        return max_profit_unlimited(prices)    # not binding: avoids a huge table

    buy = [-prices[0]] * (k + 1)              # holding, having STARTED t
    sell = [0] * (k + 1)                      # holding nothing, t COMPLETED
    for price in prices[1:]:
        for t in range(1, k + 1):
            buy[t] = max(buy[t], sell[t - 1] - price)
            sell[t] = max(sell[t], buy[t] + price)
    return sell[k]

def max_profit_cooldown(prices: list[int]) → int:
    """One day of cooldown after selling. Three states, not two."""
    if not prices:
        return 0
    free, holding, cooling = 0, -prices[0], float("-inf")
    for price in prices[1:]:
        previous_free, previous_holding = free, holding    # yesterday's values
        free = max(previous_free, cooling)
        holding = max(previous_holding, previous_free - price)
        cooling = previous_holding + price
    return int(max(free, cooling))

def max_profit_fee(prices: list[int], fee: int) → int:
```

```

    """A fee per transaction. No new dimension – one constant."""
    if not prices:
        return 0
    not_holding, holding = 0, -prices[0]
    for price in prices[1:]:
        not_holding = max(not_holding, holding + price - fee)
        holding = max(holding, not_holding - price)
    return not_holding

def max_profit_with_trades(prices: list[int]) → tuple[int, list[tuple[int, int]]]:
    """Unlimited, and report the actual buy/sell day pairs."""
    trades: list[tuple[int, int]] = []
    i, n = 0, len(prices)
    while i < n - 1:
        while i < n - 1 and prices[i + 1] ≤ prices[i]:
            i += 1
            # walk down to a local minimum
        if i == n - 1:
            break
        buy_day = i
        while i < n - 1 and prices[i + 1] > prices[i]:
            i += 1
            # walk up to a local maximum
        trades.append((buy_day, i))
    return sum(prices[s] - prices[b] for b, s in trades), trades

if __name__ == "__main__":
    prices = [7, 1, 5, 3, 6, 4]
    print("one transaction :", max_profit_one(prices))
    print("unlimited (dp) :", max_profit_unlimited(prices))
    print("unlimited (greedy):", max_profit_unlimited_greedy(prices))
    print("with trades :", max_profit_with_trades(prices))
    print("k = 1 :", max_profit_k(prices, 1))
    print("k = 2 :", max_profit_k(prices, 2))
    print("k = 1000000000 :", max_profit_k(prices, 1_000_000_000))
    print("cooldown :", max_profit_cooldown(prices))
    print("fee of 2 :", max_profit_fee(prices, 2))

    falling = [7, 6, 4, 3, 1]
    print("falling, one :", max_profit_one(falling))
    print("falling, unlimited:", max_profit_unlimited(falling))

    two_peaks = [3, 3, 5, 0, 0, 3, 1, 4]
    print("two peaks, k=2 :", max_profit_k(two_peaks, 2))
    print("two peaks, k=1 :", max_profit_k(two_peaks, 1))
    print("two peaks, greedy:", max_profit_unlimited_greedy(two_peaks))
    print("two peaks, cooldown:", max_profit_cooldown(two_peaks))

```

Run it and you get:

```

one transaction : 5
unlimited (dp)   : 7
unlimited (greedy): 7
with trades     : (7, [(1, 2), (3, 4)])
k = 1           : 5
k = 2           : 7
k = 1000000000 : 7
cooldown       : 5
fee of 2        : 3
falling, one    : 0
falling, unlimited: 0
two peaks, k=2  : 6
two peaks, k=1  : 4
two peaks, greedy: 8
two peaks, cooldown: 6

```

one transaction 5 against unlimited 7 is the whole family in two lines: buying at 1 and selling at 6 gives 5; taking both rises gives 7.

And two peaks gives 4 at $k=1$, 6 at $k=2$, and 8 from greedy — which is the case that proves greedy fails when transactions are limited. The greedy sum of positive differences ignores the limit entirely, so it reports the unlimited answer whatever k says.

6 What it costs

The unlimited version.

```

one pass over n days, two max operations each
→  $O(n)$  time
→  $O(1)$  space – two variables, no array at all

```

The k -transaction version.

```

for each of n days, for each of k transaction counts
→  $O(n \times k)$  time
→  $O(k)$  space – two arrays of length  $k+1$ 

```

Concretely:

```
n = 100,000 days, k = 100
100,000 x 100 = 10,000,000 operations    ~2 s in Python. Fine.
```

```
n = 100,000, k = 1,000,000,000
WITHOUT the  $k \geq n/2$  check:
    a table with  $10^9$  rows → MemoryError
WITH it:
    falls through to the  $O(n)$  unlimited version → instant
```

The check turns an impossible input into a trivial one, and
LeetCode 188 includes exactly that input.

Why $k \geq n/2$ is the right threshold:

each transaction needs at least 2 days: one to buy, one to sell
so in n days you can complete at most $\text{floor}(n/2)$ transactions

$k \geq n/2 \Rightarrow$ the limit can never bind $\Rightarrow$ it IS the unlimited problem

The cooldown version.

```
three variables, one pass
→  $O(n)$  time,  $O(1)$  space
```

exactly the same cost as the two-state version. Adding a state
to a state MACHINE is free; adding a DIMENSION to a table is not.

That distinction is worth having ready, because it explains why the cooldown is easy and the k -limit is not: **the cooldown adds a third mode; k adds a whole axis.**

The fee version.

```
identical to unlimited:  $O(n)$  time,  $O(1)$  space
one extra subtraction per iteration
```

→ the cheapest variant in the family. No new state at all.

Compared with the naive recursion:

```
without memoisation, at each day you branch on buy / sell / do nothing
→  $O(3^n)$ 
```

```
n = 30: ~2 x  $10^{14}$  calls. Hours.
n = 30 with the state machine: 30 iterations. Microseconds.
```

The full table, if you write it that way:

```

dp[n][k+1][2] integers

n = 100,000, k = 100:
    100,000 x 101 x 2 = 20,200,000 cells
    at ~28 bytes per Python int object    = ~570 MB

against O(k) space: 2 x 101 = 202 values = ~6 KB

→ 100,000x less, and the same answer.

```

And the reconstruction cost:

```

to report which days you traded on, you need the full table
→ O(n x k) space, back to 570 MB at those sizes

for the UNLIMITED version there is a shortcut: walk the price
series finding local minima and maxima directly.
→ O(n) time, O(1) extra space, and it gives the trades

```

7 The traps

Counting the transaction on both the buy and the sell.

```

>>> # buy[t] = max(buy[t], sell[t-1] - price)
>>> # sell[t] = max(sell[t], buy[t-1] + price)    ← WRONG: t-1 again
>>> # with k = 2 on [3,3,5,0,0,3,1,4] this gives 4, not 6

```

Four instead of six, and four is the `k = 1` answer — the limit has been silently halved. **No error, and a completely plausible number.** Decide where the transaction is consumed, write it in a comment, and be consistent.

Forgetting the `k ≥ n/2` shortcut.

```

>>> buy = [-7] * (1_000_000_001)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
MemoryError

```

Eight gigabytes for the pointer array alone. The check is three lines and it is the difference between passing and crashing on a deliberately-included test case.

Reading the freshly-updated value in the cooldown version.

```
>>> free, holding, cooling = 0, -1, float("-inf")
>>> for price in [2, 3]:
...     free = max(free, cooling)
...     holding = max(holding, free - price)      # `free` was JUST updated
...     cooling = holding + price                 # `holding` was JUST updated
```

This lets you sell and buy on the same day, which is exactly what the cooldown forbids — and it gives a profit that is too high. **Save yesterday's values at the top of the loop.** In the unlimited version the same pattern is harmless, which is why the habit carries over and bites.

Returning `max(dp[n-1][0], dp[n-1][1])` in the plain versions.

```
>>> # holding at the end means you bought and never sold
>>> # that money is spent, so dp[n-1][1] is always ≤ dp[n-1][0]
>>> # taking the max is harmless here – but it signals you have not
>>> # thought about why, and the interviewer will ask
```

The answer is `dp[n-1][0]`. In the cooldown version, though, `max(free, cooling)` is genuinely required, because ending on a sale leaves you in `cooling` — so the rule is not “always index 0”, it is “every state where you hold nothing”.

Using profit instead of money.

```
>>> # holding = max(holding, -price)          ← tracks "cost", loses history
>>> # on [1, 2, 3, 4] this gives 3, not 3... check it on [7,1,5,3,6,4]
```

Tracking money — where buying subtracts and selling adds — keeps the arithmetic uniform and makes the transitions obviously correct. Tracking “profit so far” needs a separate notion of your entry price and is where sign errors live.

Greedy on an input with a transaction limit.

```
>>> max_profit_unlimited_greedy([3, 3, 5, 0, 0, 3, 1, 4])
8
>>> max_profit_k([3, 3, 5, 0, 0, 3, 1, 4], 2)
6
>>> max_profit_k([3, 3, 5, 0, 0, 3, 1, 4], 1)
4
```

Eight, six and four for the same prices. The greedy sum of positive differences ignores `k` entirely — it reports the unlimited answer whatever the limit says — and it will pass any test whose `k` happens to be large enough not to bind.

An empty or single-element list.


```
>>> max_profit_unlimited([])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in max_profit_unlimited
    not_holding, holding = 0, -prices[0]
IndexError: list index out of range
```

`prices[0]` on an empty list. One guard, and it is a standard added test case.

A fee charged on both halves.

```
>>> max_profit_fee([1, 3, 7, 5, 10, 3], 3)
6
>>> # charging on buy AND sell would give 0 here – the fee is doubled,
>>> # and every trade looks unprofitable
```

A doubled fee makes marginal trades look bad and the answer collapses towards zero, which reads like “the fee is high” rather than like a bug.

8 In the interview

How it gets asked

- “Best time to buy and sell stock.” — LeetCode 121, one transaction.
- “Now you can buy and sell as many times as you like.” — LeetCode 122.
- “At most `k` transactions.” — LeetCode 188, and the ladder’s hard rung.
- “With a one-day cooldown after selling.” — LeetCode 309.
- “With a fee per transaction.” — LeetCode 714.
- “Which of these can you solve greedily?”

The first ninety seconds

“This is a family of about six problems and they share one state, so let me set that up rather than solve the specific one — it makes every variant a small edit.

The state is (day, whether I am currently holding the stock). And the second part is the idea worth naming: **it is a *mode*, not a position**. Every DP so far has been indexed by where I am; here I also need to know **which situation I am in**, because it decides which moves are legal.

Without it the state is incomplete. ‘The best profit in the first n days’ cannot express a recurrence, because **I cannot tell whether selling today is even legal without knowing whether I own anything** — and that fails silently, like every incomplete state.

Two transitions, and I would track money rather than profit so buying subtracts and selling adds and the signs never confuse me.

Not holding at the end of today: either I was not holding and did nothing, or I was holding and sold. $\max(dp[d-1][0], dp[d-1][1] + price)$.

Holding at the end of today: either I was holding and kept it, or I was not and bought. $\max(dp[d-1][1], dp[d-1][0] - price)$.

Base cases: on day zero, not holding is zero and holding is minus the first price.

The answer is the not-holding state, not the maximum of the two — ending while still holding means I bought something and never sold it, which is never better.

$O(n)$ time and $O(1)$ space — two variables, no table.

And then the variants are each one change. A fee is a constant subtracted on the sell, with no new state at all. A cooldown is a third state — free, holding, just-sold. **At most k transactions is the only one that adds a dimension**, and it is the only one that costs anything.

One thing I would ask: is there a limit on transactions? Because with no limit, greedy solves it in one line — sum every positive day-to-day difference — **and the moment there is a limit, greedy is wrong**, which is the interesting part.”

The follow-ups

“At most k transactions. What changes?”

“A third dimension, and one decision I have to state explicitly before writing anything.

The state becomes `(day, transactions used, holding)`. Same two transactions, indexed by how many transactions I have spent.

The decision: where is a transaction consumed? A transaction is a buy plus its matching sell. I can count it on the buy or on the sell — both are correct — and I must pick one and be consistent.

I count it on the buy. So `buy[t]` comes from `sell[t-1]` minus the price — starting transaction `t` — and the matching `sell[t]` comes from `buy[t]` plus the price, staying at `t`.

Counting it on both is the standard bug, and it halves the effective limit: `k = 2` behaves like `k = 1`. On `[3,3,5,0,0,3,1,4]` that gives four instead of six, and four is a completely plausible answer — it is the correct answer to a different question. **No error anywhere**, which is why I write the meaning of each array in a comment before the loop.

The other thing that must be there is the `k ≥ n/2` **shortcut**. Each transaction needs at least two days, so in `n` days I can complete at most `n/2`. **If `k` is at least that, the limit can never bind and the answer is the unlimited version.**

That is not an optimisation — it is required. LeetCode 188 includes `k = 10^9`, and without the check I allocate a billion-row array and get a `MemoryError` before doing any work.

Cost: `O(n × k)` **time** and `O(k)` **space** with two arrays of length `k+1`, since each day reads only yesterday. **The full table would be** `n × k × 2` — at `n = 100,000` and `k = 100` that is about 570 megabytes against six kilobytes, for the same answer.

And if they want the actual trades, I need the full table — same trade as every reconstruction.”

“Which of these can you solve greedily, and why does greedy break?”

“Two of them, and the reason it breaks is worth stating precisely because it is the justification for the DP.

One transaction: greedy works. Track the minimum price seen so far and the best difference against it, in one pass. $O(n)$ time, $O(1)$ space, and the DP would be overkill.

Unlimited transactions: greedy works, and the reason is telescoping. Sum every positive day-to-day difference. **Buying at 1 and selling at 3, then buying at 3 and selling at 5, is identical to buying at 1 and selling at 5** — so capturing every upward step is the same as capturing every rising run, and it is one line.

A limited number of transactions: greedy fails, and here is exactly why. With a limit, each transaction is a scarce resource. **A local rule cannot know whether to spend one on a small rise today or save it for a bigger rise later**, because that depends on the whole rest of the series.

Concretely, on `[3,3,5,0,0,3,1,4]` : greedy on differences says eight — it takes every rise, three of them. **With `k = 2` the answer is six, and with `k = 1` it is four**, because one transaction buys me only the best single trade, which is buying at zero and selling at four. **Greedy reports the same eight in all three cases**, because nothing in it looks at `k`.

And greedy will pass any test where `k` happens to be large enough not to bind, which makes it a dangerous thing to submit.

The cooldown and the fee also break greedy, for the same underlying reason: **a local decision cannot account for a cost that lands in the future**. Selling today might be locally profitable and lock me out of a much better trade tomorrow.

So my rule is: greedy for one transaction and for unlimited, the state machine for everything else — and I would offer both for the unlimited case, since the greedy one-liner is genuinely better and knowing the DP is what lets me handle the next question.”

“Now add a one-day cooldown after selling.”

“A third state, and that is genuinely all — which is worth contrasting with the k - transaction case, because the costs are completely different.

Adding a state to a state machine is free. Adding a dimension to a table is not. The cooldown stays $O(n)$ time and $O(1)$ space; the transaction limit becomes $O(n \times k)$.

Three states: free, holding, and cooling. **Free** means I own nothing and may buy today. **Holding** means I own the stock. **Cooling** means I sold today, so I own nothing and may *not* buy.

The transitions:

Free today means either I was free yesterday and did nothing, or I was cooling yesterday and the cooldown has now expired.

Holding today means either I was holding and kept it, or I was **free** yesterday and bought — not cooling, which is the entire point.

Cooling today means I was holding yesterday and sold today. Only one way in.

The implementation detail that matters: I must save yesterday's values at the top of the loop. In the unlimited version I read freshly-updated values and it was harmless, because buying and selling the same stock on the same day nets zero. **Here it would let me sell and immediately buy, which is exactly what the cooldown forbids**, and the answer comes out too high with no error.

And the answer is $\max(\text{free}, \text{cooling})$, not just free . The last day might end with a sale, which leaves me in cooling — holding nothing, which is what I want, but in the other state.

So the general rule is not ‘return index zero’ — it is ‘return the best over every state in which I hold nothing’, and in the two-state version that happens to be one state.”

The model answer

“You are given the hourly electricity price for the next week, and a battery that can be fully charged or fully discharged in one hour. Charging costs you the price; discharging earns it. The battery can do at most k full cycles before it needs servicing. Maximise your earnings.”

“This is buy-and-sell stock with at most k transactions, and I want to establish that mapping explicitly before writing anything, because getting it right makes the rest mechanical.

A charge is a buy. A discharge is a sell. A full cycle is one transaction. The hourly prices are the price series. **Maximising earnings is maximising profit.**

And the constraint that makes it this problem rather than a simpler one is that the battery is either charged or empty — there is no partial state. That is exactly the holding / not-holding mode.

The state: (hour, cycles used, charged). Same two transitions as the stock problem: at each hour I can do nothing, charge if empty, or discharge if charged.

And I would state where the cycle is consumed: on the charge. So $\text{charged}[t]$ comes from $\text{empty}[t-1]$ minus the price — starting cycle t — and $\text{empty}[t]$ comes from $\text{charged}[t]$ plus the price, staying at t . **Counting it on both halves the limit and gives a smaller, entirely plausible answer,** which is the bug I would be watching for.

Base cases: at hour zero, empty is zero earnings and charged is minus the first hour's price.

The answer is $\text{empty}[k]$ — the battery should end discharged, because energy left in it at the end is money spent and not recovered. **If the problem said the battery must end charged, that would be $\text{charged}[k]$ instead,** and I would ask.

Sizing: a week of hourly prices is 168 hours. With k cycles that is $168 \times k$ operations — trivial for any realistic k . **And I would still include the $k \geq n/2$ shortcut,** because if k is large the limit cannot bind — each cycle needs at least two hours — **and it turns a potentially enormous table into an $O(n)$ pass.**

Space: two arrays of length $k+1$, since each hour reads only the previous one.

Now three things about the domain that I would raise, because the stock mapping hides them.

First, the battery probably is not perfectly efficient. Real batteries lose energy on a round trip — typically ten to fifteen percent. **That is exactly the transaction-fee variant:** multiply the discharge earnings by the efficiency, or subtract a fee. **No new dimension, one constant —** which is a nice demonstration that the mapping is doing real work.

Second, the prices are a forecast, not a fact. This computes the optimal plan for a price series I do not actually know. **In practice you would re-solve every hour with updated prices**, which is cheap here, and accept that the realised profit is below the theoretical optimum. **I would say that rather than present the number as achievable.**

Third, and I would flag it as a question rather than assume: can the battery charge and discharge in the same hour? The stock version implicitly allows a same-day buy-and-sell that nets zero and is harmless. **A physical battery cannot**, and if there is also a minimum rest period after discharging, **that is the cool-down variant — a third state, still $O(n)$ time and constant space.**

Which is the general point I would close on: charge-or-empty, cycle limit, efficiency loss and rest period are four different-sounding constraints, and they are the same four variants of one state machine.“

9 Recall card

The state is `(day, holding)` — and the second dimension is a **MODE**, not a position. “Best profit in the first `n` days” is incomplete: you cannot tell whether selling is legal without knowing whether you hold. **Track money, not profit** — buying subtracts, selling adds.

```
dp[d][0] = max(dp[d-1][0], dp[d-1][1] + price) (sell); dp[d][1] =
max(dp[d-1][1], dp[d-1][0] - price) (buy). Base: dp[0][0] = 0, dp[0][1] = -
price[0]. The answer is the not-holding state — ending while holding is never
optimal.  $O(n)$  time,  $O(1)$  space, two variables.
```

Greedy solves exactly two variants. One transaction: min-so-far and best difference. **Unlimited: sum the positive day-to-day differences, because buy@1/sell@3 then buy@3/sell@5 telescopes into buy@1/sell@5.** Any transaction limit breaks greedy — on `[3,3,5,0,0,3,1,4]` greedy says 8 whatever `k` is, while the answers are 6 at `k=2` and 4 at `k=1`.

At most `k`: add a dimension, and STATE where the transaction is consumed. Count it on the buy (`buy[t]` from `sell[t-1]`); **counting on both halves the limit and gives a plausible smaller number with no error.** The `k ≥ n/2` shortcut is required, not an optimisation — each transaction needs two days, and `k = 10^9` otherwise means `MemoryError`.

A fee is one constant on the sell — no new state. A cooldown is a third state (free / holding / cooling), still $O(n)$ and $O(1)$ — adding a state to a machine is free; adding a dimension to a table is not. In the cooldown version save yesterday's values, or you sell and buy on the same day; and the answer is $\max(\text{free}, \text{cooling})$ — every state where you hold nothing.

Design a chat system with presence

1 What this is, and why they ask it

Yesterday you designed WhatsApp: **one-to-one messages, delivered and forgotten, end-to-end encrypted.**

Today is the other kind of chat — **Slack, Discord, Teams.** Persistent channels rather than conversations. Message history that must survive forever and be searchable. Threads. And **presence** — who is online, who is typing, who is away — which is the part this lesson is named for and the part that is genuinely surprising.

They ask it because **it inverts three of yesterday's decisions**, and the interviewer wants to see whether you know why.

History is kept, not deleted, because a workspace's messages are a shared organisational record and "scroll back to what was decided in March" is the product. **Which means server-side storage grows forever and search becomes a first-class system** rather than something encryption forbade.

A channel has many members and no cap, so a message in a busy channel fans out to thousands of connected clients, and you are back to a fan-out problem WhatsApp's thousand-member limit had deleted.

And presence, which people underestimate by two orders of magnitude. Online status changes constantly, it fans out to everyone who can see you, and **built naively it generates more traffic than the messages do.** Computing that unprompted is the single strongest signal in this interview.

By the end of this lesson you can design channels and history, fan out to connected clients, build presence and typing indicators that scale, add search and threads, and size all of it.

2 The story

The factory ran three shifts and the thing that held it together was a whiteboard by the door of the supervisor's office.

Not the machines. The whiteboard.

Because the man coming on at six needed to know what had happened at four, and the man who had been there at four was on a bus. **So he wrote it down.** Line six stopped twice. The compressor is making a noise. Do not use the second forklift, the horn is gone.

And it worked, and over eleven years it grew four features, each because of a specific disaster.

The first was that they stopped wiping it. Somebody wiped the board on a Tuesday and on Friday there was an argument about when exactly the compressor had first been reported, **and nobody could answer it.** After that there was a register, and the board was only the last day of it.

The second was that they split it. One board became four columns — maintenance, dispatch, quality, general — because a man from dispatch had to read forty lines about a bearing to find the one line that concerned him.

The third was Fatima. She was in the office and had been there nineteen years, and if you asked her about anything at all she could tell you which month it had been written in. **Nobody had ever called that a system.** When she retired they discovered they had been leaning on her the whole time, and it took two years to build something that did half of what she had done.

And the fourth was the one that caused the most trouble for the least reason.

Somebody put a second small board next to the first, with the names of the eleven supervisors and a magnet against each one. **Present. In the plant. Gone home.**

It seemed helpful for about a month.

Then it became a full-time job, **because a magnet is only useful if it is correct,** and people moved and forgot. So somebody had to walk round and update it. And when it was wrong, it was worse than nothing, because people acted on it — they walked to the far end of the plant for a man whose magnet said he was there.

They tried making everyone move their own magnet, and people forgot.

What eventually worked was a rule that nobody planned: **the magnet was only trusted for the last hour.** After that it went grey, and you telephoned instead of walking.

The board stopped claiming to know things it did not know, and the arguments stopped with it.

3 The idea in plain English

The whiteboard is a channel, the register is message history, Fatima is the search index, and the magnets are presence — including the rule that made them work.

Start with what differs from yesterday, because the transport is the same.

	WhatsApp	Slack / Discord
conversation shape	1:1 and small groups	channels, unbounded members
history	deleted after delivery	kept forever, and searchable
encryption	end-to-end	server-side (usually)
fan-out per message	1-2 recipients	thousands of connected clients
presence	last-seen, minimal	central to the product

WebSockets and the connection registry are unchanged — [day 156](#) covers them, and it is worth saying “the connection layer is the same as yesterday’s” and moving on rather than re-deriving it.

Now the four things that are different.

One: messages are stored, and the storage model is the design.

A channel is an ordered, append-only log of messages. **The access pattern is overwhelmingly “the last fifty messages in this channel”, plus occasional scrollbar.**

So partition by channel and sort by time within it. In Cassandra terms, `channel_id` is the partition key and `(timestamp, message_id)` is the clustering key — **which makes “the newest fifty” a single sequential read from one partition.**

And the message id should be a Snowflake, so the id itself sorts by time and pagination is “everything before this id” rather than an offset.

The one thing that breaks this model is a channel that never stops growing. A partition with ten million messages is too large for most stores, so **bucket by time**: the partition key becomes `(channel_id, month)`. Reads for recent messages hit one bucket; scrollbar walks backwards through them.

Two: fan-out to connected clients, which is a real problem again.

A message in a channel with five thousand members, of whom eight hundred are currently connected, must reach **eight hundred WebSockets spread across many connection servers.**

Doing that with eight hundred registry lookups per message is the naive design, and it works until it does not. **The scalable version is a pub/sub bus**: every connection server subscribes to the channels its clients care about, a message is pub-

lished once to the channel's topic, and each server delivers to its own local connections.

One publish instead of eight hundred lookups, and the fan-out cost moves from the sender's request to the messaging bus, which is what it is for.

And the offline members are a separate path entirely. They do not get a push; **their unread state is computed when they return**, from a per-user-per-channel `last_read_message_id`. **That single field is the whole unread system**, and it is much cheaper than a per-user queue.

Three: presence, which is the surprising part.

The naive design is: every client reports online, and every state change is pushed to everyone who could see it. In a ten-thousand-person workspace that is a change fanning out to ten thousand people, and people change state constantly.

The arithmetic kills it, and it is worth doing out loud: **presence fan-out exceeds message fan-out by an order of magnitude**, because state changes are far more frequent than messages and reach far more people.

Four fixes, and they compose.

Push presence only to people who can currently see it. Nobody needs your status while they are looking at a different channel. **Subscribe to presence per visible view**, not per contact.

Throttle. Batch presence updates and flush every few seconds. **Nobody notices a three-second delay on a green dot**, and it collapses a burst of changes into one message.

Make it expiring rather than authoritative. A heartbeat sets a key with a TTL; **absence of a heartbeat is absence of presence.** No explicit "offline" event is needed, which matters because **a client that loses network never sends one** — the same reason yesterday's connection registry needed a TTL, and it is the whiteboard's grey magnet.

And accept that it is approximate. Presence is a hint. **A system that guarantees correct presence is solving a much harder problem than the product needs**, and saying so is better than designing for it.

Typing indicators are presence with a shorter fuse. Same mechanism, a two-to-three second TTL, throttled so a keystroke does not become a network message, and **never persisted** — a typing event that arrives late is worse than one that is dropped.

Four: search, which encryption made impossible yesterday and is now central.

Server-side search over an inverted index — the structure from [day 135](#) — with one enormous additional requirement: **permissions**. A search must return only messages in channels the user can see, **and filtering after retrieval is wrong**, because a user could infer the existence of private channels from result counts.

So the permission filter goes into the query itself: search within the set of channels this user is a member of. **That set is small — a few hundred — and cacheable**, and it turns a permission problem into a filter clause.

And threads, briefly. A thread is a message with a `parent_id`. **The design decision is whether thread replies appear in the main channel**, and both answers exist in real products — Slack makes it optional per message, which is a product decision rather than an architectural one.

4 The picture

The two shapes, side by side:

WHATSAPP (yesterday)

[Alice] <---> [Bob]

deliver, then DELETE
the phone is the archive
1-2 recipients per message
E2E encrypted → no search
presence = last seen

SLACK (today)

#engineering
members: 5,000
connected now: 800
history: KEPT FOREVER, SEARCHABLE
800 live deliveries per message
server-side → search is a feature
presence = core product surface

Fan-out by pub/sub instead of by lookup:

NAIVE

message to #eng
|
look up 800 members
in the registry
|
800 lookups + 800 pushes
from ONE server

→ the sender's request
does all the work

PUB/SUB

message to #eng
|
publish once to topic "channel:eng"
|
+-----+
v v v v
conn-1 conn-2 conn-3 conn-4
| | | |
each delivers to ITS OWN
local connections

ONE publish. The bus does the fan-out.

Storage layout, and why it is partitioned this way:

```
PARTITION KEY    (channel_id, month)
CLUSTERING KEY   (message_id DESC)    ← Snowflake: sorts by time
```

```
channel:eng|2026-03 → [ msg 9912, msg 9911, msg 9910, ... ]
channel:eng|2026-02 → [ msg 8801, msg 8800, ... ]
channel:random|2026-03 → [ ... ]
```

```
"the last 50 in #eng" = ONE sequential read from ONE partition
"scroll back"         = walk backwards, month by month
```

WHY BUCKET BY MONTH:

```
  a 5-year-old channel with 10M messages in one partition is
  too large for the store to handle well.
  Bucketing bounds every partition.
```

The presence arithmetic, which is the point of the lesson:

```
10,000-person workspace, naive presence
```

```
each person changes state ~30 times/hour
  (online, away, typing, stopped typing, back)
= 300,000 events/hour
```

```
each fanned out to everyone who could see them: 10,000
= 3,000,000,000 pushes/hour = ~833,000 pushes/second
```

```
MESSAGES in the same workspace:
```

```
  ~10,000 messages/hour x ~200 recipients = 2,000,000/hour = 555/s
```

```
→ PRESENCE IS 1,500x THE MESSAGE TRAFFIC.
```

```
Fixes, multiplied together:
```

```
  only to viewers of the same channel    x 1/500
  throttle to one update per 5 s          x 1/6
  → ~280 pushes/second. Manageable.
```

Presence as an expiring key, which is the grey magnet:

```
client heartbeat every 30 s:
    SET presence:<user> "online" EX 60

reading presence:
    GET presence:<user> → "online" or MISSING

MISSING means offline. There is no "offline" event, and there
must not be:
    a phone that loses signal NEVER sends one
    a browser tab that is force-quit NEVER sends one

→ absence of a heartbeat IS the offline signal
→ and the status is only ever trusted for its TTL,
    which is exactly what "the magnet goes grey after an hour" was
```

Unread state, which is one field and not a queue:

```
WRONG: a per-user queue of unread messages
    5,000 members x every message = 5,000 rows per message

RIGHT: last_read[user][channel] = message_id

    unread count = messages in the channel with id > last_read
                  = a range count on an already-sorted partition

ONE row per user per channel, updated when they read.
The unread COUNT is computed, not stored.

(and for very busy channels, cache the count and invalidate on
new messages, because the range count is not free)
```

Search with permissions in the query, not after it:

```
WRONG: search everything, then drop what the user cannot see
    → result counts leak the existence of private channels
    → "about 47 results" for a query that shows 3

RIGHT: channels = the user's memberships (a few hundred, cached)
    query: term AND channel_id IN (channels)

    the permission is a FILTER CLAUSE, evaluated by the index
    → no leak, and it is faster
```

5 How it actually works

Storing a message

```
def post_message(user_id: int, channel_id: int, text: str,
                 client_id: str, parent_id: int | None = None) → dict:
    if existing := message_store.by_client_id(user_id, client_id):
        return existing # idempotent retry

    message = message_store.insert(
        message_id=snowflake.next_id(), # sorts by time
        bucket=f"{channel_id}|{month_of(time.time())}",
        channel_id=channel_id,
        parent_id=parent_id,
        author_id=user_id,
        text=text,
    )
    bus.publish(f"channel:{channel_id}", message) # ONE publish
    queue.publish("index", message) # async search indexing
    return message
```

bus.publish once, not one push per member — that is the whole difference from a naive fan-out, and the messaging bus is what turns eight hundred deliveries into one operation on the request path.

The **client_id** check is the same idempotency as yesterday, and for the same reason: a retry after a lost acknowledgement must not create a second message.

Delivery on the connection servers

```
class ConnectionServer:
    def __init__(self) → None:
        self.local: dict[int, set[Connection]] = {} # channel → connections

    def on_subscribe(self, conn: Connection, channel_id: int) → None:
        if channel_id not in self.local:
            bus.subscribe(f"channel:{channel_id}", self.on_bus_message)
            self.local.setdefault(channel_id, set()).add(conn)

    def on_bus_message(self, channel_id: int, message: dict) → None:
        for conn in self.local.get(channel_id, ()):
            conn.send(message) # local, in-process, no lookups
```

Each server subscribes once per channel it holds clients for, regardless of how many. A server with two hundred clients in **#general** has one subscription, and delivery is an in-process loop.

The **if channel_id not in self.local** guard matters: subscribing per connection instead of per channel multiplies the bus's work by the number of clients, which is exactly what you were avoiding.

Reading history

```
def get_history(channel_id: int, before: int | None, limit: int = 50) → list[dict]:
    results: list[dict] = []
    bucket_time = time.time() if before is None else timestamp_of(before)
    while len(results) < limit:
        bucket = f"{channel_id}|{month_of(bucket_time)}"
        page = message_store.read(bucket, before=before, limit=limit - len(results))
        results.extend(page)
        if not page:
            bucket_time -= 30 * 86400          # step back a month
            if too_old(bucket_time):
                break
        else:
            before = page[-1]["message_id"]
    return results
```

The loop over buckets is the cost of bucketing, and it is worth showing: a quiet channel may need several buckets to fill fifty messages, while a busy one fills from the first.

`before` as a message id rather than an offset is cursor pagination — stable when new messages arrive between pages.

Presence

```
HEARTBEAT_TTL = 60

def heartbeat(user_id: int, status: str = "online") → None:
    redis.set(f"presence:{user_id}", status, ex=HEARTBEAT_TTL)

def get_presence(user_ids: list[int]) → dict[int, str]:
    keys = [f"presence:{u}" for u in user_ids]
    values = redis.mget(keys)                # ONE round trip for the whole list
    return {u: (v or "offline") for u, v in zip(user_ids, values)}
```

There is no `set_offline`. The TTL is the offline signal, **because a client that loses network never sends a disconnect** — and a design that waits for one will show people online for hours after they have gone.

`mget` for the whole member list in one round trip matters, because rendering a channel's member sidebar asks for hundreds of statuses at once.

Presence fan-out, throttled and scoped

```
class PresenceBroadcaster:
    def __init__(self, interval: float = 5.0) → None:
        self.pending: dict[int, str] = {}
        self.interval = interval

    def changed(self, user_id: int, status: str) → None:
        self.pending[user_id] = status          # coalesce: last write wins

    def flush(self) → None:                     # called every `interval` seconds
        if not self.pending:
            return
        batch, self.pending = self.pending, {}
        for channel_id in channels_containing(batch.keys()):
            bus.publish(f"presence:{channel_id}",
                        {u: s for u, s in batch.items()
                         if u in members_of(channel_id)})
```

Two mechanisms doing two jobs. **Coalescing** means a user who toggles four times in five seconds generates one update. **Scoping to channels** means the update reaches people currently viewing a channel that person is in, not everyone in the workspace.

Together those are the difference between 833,000 pushes a second and about 280.

Typing indicators

```
def typing(user_id: int, channel_id: int) → None:
    key = f"typing:{channel_id}:{user_id}"
    if redis.set(key, "1", nx=True, ex=3):      # nx: only the FIRST keystroke
        bus.publish(f"typing:{channel_id}", {"user": user_id})
```

`nx=True` is the throttle, and it is elegant: the first keystroke sets the key and publishes; every keystroke for the next three seconds finds the key present and publishes nothing. **The client shows the indicator for three seconds and lets it lapse.**

Never persisted. A typing event that arrives after the message did is noise, and there is nothing to recover.

Unread counts

```
def mark_read(user_id: int, channel_id: int, message_id: int) → None:
    read_store.upsert(user_id, channel_id, message_id)

def unread_counts(user_id: int) → dict[int, int]:
    marks = read_store.all_for(user_id)          # a few hundred rows
    latest = channel_store.latest_message_ids(marks.keys()) # cached
    return {c: count_between(c, marks[c], latest[c]) for c in marks
            if latest[c] > marks[c]}
```

One row per user per channel, and the count is computed rather than stored. A per-user queue would mean five thousand rows written per message in a five-thousand-member channel — the thing the design exists to avoid.

`count_between` should be cached for busy channels, because a range count on a large partition is not free and the answer changes on every message.

Search with permissions

```
def search(user_id: int, query: str, limit: int = 20) → list[dict]:
    channels = membership_cache.get(user_id)          # a few hundred ids
    if not channels:
        return []
    return search_index.query(
        text=query,
        filters={"channel_id": channels},             # IN the query, not after
        sort="relevance",
        limit=limit,
    )
```

The membership filter is part of the query. Filtering after retrieval leaks information — a total result count that does not match the visible results tells the user how many private messages matched, which is a genuine data leak in a workplace product.

The real systems

Slack	channels in MySQL sharded by workspace, search in Solr; famously one shard per workspace early on
Discord	Cassandra then ScyllaDB, bucketed by channel+time — they published the migration in detail
Elasticsearch	the usual search index for this shape
Redis	presence, typing, and the connection registry
Kafka / NATS	the pub/sub bus for channel fan-out

Discord's Cassandra bucketing is the specific reference for the storage model, and naming it is a good signal because the write-up is public and the problem — an unbounded partition per channel — is exactly the one you have to solve.

6 The numbers

Scale.

```
10,000,000 daily active users
2,000,000 concurrent
average 50 channels per user, ~200 members per channel

messages: ~50/user/day → 500,000,000/day = ~5,800/second
peak ~20,000/second
```

Message fan-out.

```
5,800 messages/s x ~40 connected members per channel on average
= ~230,000 live deliveries/second

via pub/sub: 5,800 publishes/second, and each connection server
delivers locally to its own clients
→ the sender's request does ONE publish, not 40 lookups
```

Presence, computed naively — the number that matters.

```
2,000,000 concurrent users
each changes state ~30 times/hour      = 60,000,000 events/hour
                                         = ~16,700 events/second

naive fan-out, to everyone sharing any channel with them (~1,000 people):
16,700 x 1,000 = 16,700,000 pushes/second

against 230,000/second for MESSAGES.

→ presence is ~70x the message traffic, built naively.
```

With the two mitigations:

START: 16,700 state changes/second

FIX 1 – SCOPE: push only to people currently VIEWING a channel that person is in, which is ~20 people rather than ~1,000.

$16,700 \times 20 = 334,000$ pushes/second (50x better)

FIX 2 – THROTTLE: coalesce each user's changes into one update per 5-second window, last write wins.

$16,700 \text{ events/s} \times 5 \text{ s} = 83,500$ events per window
but they come from far fewer distinct users, because a user who toggles several times in 5 seconds collapses to one update:
~5,000 distinct users per window

$5,000 \text{ updates} / 5 \text{ s} = 1,000$ flushes/second
 $1,000 \times 20 \text{ viewers} = 20,000$ pushes/second (another 17x)

→ ~20,000/second, from 16,700,000. An order of magnitude BELOW the message traffic, which is where presence belongs.

Storage.

$500,000,000 \text{ messages/day} \times 500 \text{ bytes (text + metadata)}$
= 250 GB/day
x 365 = 91 TB/year
x 3 replicas = 273 TB/year

kept forever, so this accumulates. Ten years = 2.7 PB.

compare WhatsApp: a few TB total, because it deletes.
→ keeping history costs three orders of magnitude more storage.

The search index.

$500,000,000 \text{ messages/day} \times \sim 15 \text{ terms} \times 8 \text{ bytes}$
= 60 GB/day of postings
x 90 days hot = 5.4 TB hot index
older: cold shards, queried rarely

Elasticsearch wants the hot index largely in memory
→ ~50-100 machines with 64 GB each for the hot tier

Partition sizing, which is why bucketing exists.

a busy channel: 1,000 messages/day
over 5 years = 1,825,000 messages x 500 B = ~900 MB in ONE partition

Cassandra guidance: keep partitions under ~100 MB
→ a single-partition-per-channel design fails on exactly the channels you care most about

bucketed by month:

1,000 x 30 = 30,000 messages x 500 B = 15 MB per partition
→ comfortable, and reads for recent messages touch one bucket

Unread counts, and why they are cached.

2,000,000 concurrent users x 50 channels = 100,000,000 unread counts
recomputed on every message in every channel

a range count on a partition: ~1 ms
naive: every message invalidates ~200 members' counts
5,800 messages/s x 200 = 1,160,000 recomputes/second at 1 ms
= 1,160 cores doing nothing but counting

cached per (user, channel), invalidated on new messages, and
computed lazily when the client asks:
→ only for channels the user actually looks at, ~5 per session
→ ~10,000 computations/second. Fine.

Latency budget.

post message → persisted	~5 ms
publish to the bus	~2 ms
bus → connection server	~3 ms
server → client WebSocket	~30 ms

	~40 ms
history read (one partition, 50 rows)	~10 ms
search (hot index)	~100 ms
presence lookup (mget of 200)	~2 ms

Cost, roughly:

connection tier (2M connections)	~50 machines	~\$50,000/month
message store (273 TB/year, growing)	~100 machines	~\$150,000/month
search (5.4 TB hot + cold)	~80 machines	~\$120,000/month
pub/sub bus	~20 machines	~\$25,000/month
presence (Redis)	~10 machines	~\$10,000/month

		~\$355,000/month

for 10M DAU = ~\$0.035/user/month

→ which is why this is a paid product and WhatsApp could be free.

7 The trade-offs

Keeping history against deleting it. Keeping it is the product — a workspace’s messages are an organisational record, and scrollback and search are why people pay. **It costs three orders of magnitude more storage than WhatsApp’s model, makes search a first-class system, and creates a compliance surface:** an archive that exists can be subpoenaed, exported, and retained under legal hold. **That is a feature for an enterprise product and a liability for a consumer one**, which is exactly why the two products chose differently.

Server-side encryption against end-to-end. Server-side makes search, moderation, compliance export and multi-device trivial. **End-to-end would break all four**, and for a workplace product where the employer owns the data, the privacy argument for E2E is much weaker. **Say which you are building and why, because it is a product decision, not a technical one.**

Pub/sub fan-out against direct lookups. Direct is simple, obvious, and puts `O(members)` work on the sender’s request. **Pub/sub is one publish and moves the fan-out to a bus built for it** — at the cost of an extra system, extra latency of a few milliseconds, and **a harder failure story**: a bus partition means messages are stored but not delivered live, and clients must reconcile on reconnect.

Presence accuracy against presence cost. Naive, correct presence is seventy times the message traffic. **Scoped and throttled presence is approximate, delayed by seconds, and affordable.** The product does not need accuracy — **nobody makes a decision based on a green dot being three seconds stale** — and a system that guarantees it is solving a much harder problem than the product asked for.

Expiring presence against explicit offline events. Expiry is the only design that survives reality: **clients that crash, lose network or are force-quit never send a disconnect.** The cost is that presence lags by up to the TTL, so someone who closed their

laptop shows online for a minute. **Shortening the TTL fixes the lag and multiplies the heartbeat traffic**, so the TTL is the dial between those two.

Computed unread counts against stored ones. Computing from `last_read_message_id` is one row per user per channel and no write amplification. **Storing a per-user unread queue would mean thousands of writes per message** in a large channel. The cost is that the count must be computed on demand and cached, **and cache invalidation on every message in a busy channel is itself a load** — mitigated by computing lazily, only for channels the user actually opens.

When would I not build this? When the chat is a feature, not the product — support chat, a comment thread, a game lobby — where the message volume is low and a database table with polling is genuinely enough. **When a hosted service fits:** Stream, Sendbird and Twilio Conversations do all of this, and below a few hundred thousand users they cost less than the connection tier alone. **And presence specifically is worth questioning** — it is the most expensive feature per unit of user value in the whole system, and plenty of successful products do without it.

8 In the interview

How it gets asked

- “*Design Slack.*” or “*Design a group chat system.*” — the standard prompt.
- “*How do you show who is online?*” — the presence question, and the one with a surprising answer.
- “*A message is posted in a channel with five thousand members. What happens?*”
- “*How do you store message history?*” — the partitioning question.
- “*How does search work, and how do you handle permissions?*”
- “*How do unread badges work?*”

The first ninety seconds

“This is the other kind of chat, and I want to contrast it with a WhatsApp-style design in three points, because the differences are what make it interesting.

History is kept, not deleted. A workspace’s messages are an organisational record and scrollback is the product. **That means server-side storage grows forever — about ninety terabytes a year at ten million users, against a few terabytes total for WhatsApp — and search becomes a first-class system.**

Channels have unbounded membership, so a message fans out to hundreds or thousands of connected clients. WhatsApp’s thousand-member cap deleted that problem; here it is back.

And presence is central to the product, which is the part I would spend time on, because it is surprisingly expensive.

The transport is the same as yesterday — WebSockets, a connection registry with a heartbeat TTL — so I would not re-derive it.

Storage: partition by channel and time bucket, sorted by message id. `(channel_id, month)` as the partition key and a Snowflake message id as the clustering key, **so ‘the last fifty in this channel’ is one sequential read from one partition.** The month bucket exists because a five-year-old busy channel would otherwise be a nine-hundred-megabyte partition, which most stores handle badly.

Fan-out: a pub/sub bus, not per-member lookups. Each connection server subscribes once per channel it holds clients for; a message is published once; each server delivers to its own local connections. **One publish instead of eight hundred registry lookups on the sender’s request.**

Unread state is one field, not a queue: `last_read_message_id` per user per channel, and the count is computed. **A per-user unread queue would mean thousands of writes per message in a large channel.**

Now presence, and I want to do the arithmetic because it decides the design. Two million concurrent users changing state about thirty times an hour is roughly seventeen thousand events a second. **Fanned out naively to everyone sharing a channel — say a thousand people — that is seventeen million pushes a second, against two hundred and thirty thousand for actual messages. Presence is seventy times the message traffic if you build it the obvious way.**

So it has to be scoped and throttled, and I would design that in rather than discover it.”

The follow-ups

“How do you show who is online?”

“Three design decisions, and the first is arithmetic rather than architecture.

Presence built naively is more traffic than the messages. Two million concurrent users change state — online, away, typing, back — about thirty times an hour, which is seventeen thousand events a second. **Fanned out to everyone who shares a channel with them, roughly a thousand people, that is seventeen million pushes a second.** Messages are two hundred and thirty thousand. **Seventy times.**

So: scope it. Presence goes only to people **currently viewing** a channel that person is in — about twenty people, not a thousand. **Nobody needs your status while they are looking at a different channel,** and the client subscribes to presence per visible view rather than per contact.

Then throttle it. Coalesce changes in a five-second window and flush once — last write wins. **A user who toggles four times generates one update,** and nobody perceives a three-second delay on a green dot. **Together those take it to about twenty thousand pushes a second,** which is an order of magnitude below the message traffic, where it belongs.

The second decision is that presence must be an expiring key, not an explicit state. A heartbeat every thirty seconds sets a Redis key with a sixty-second TTL, **and absence of the key means offline.**

There must be no ‘set offline’ call, because the cases that matter never send one: a laptop that closes, a phone that loses signal, a browser tab that is force-quit. **A design waiting for a disconnect shows people online for hours after they have gone,** which is worse than showing nothing.

The TTL is the dial: shorter means presence is fresher and heartbeats are more frequent.

The third decision is to accept that it is approximate, and say so. **Presence is a hint, not a fact.** Someone shows online for up to a minute after closing their laptop, and that is fine — nobody makes a decision based on a green dot. **A system that guaranteed correct presence would be solving a much harder problem than the product needs,** and I would rather spend that effort on message delivery.

Typing indicators are the same mechanism with a three-second TTL, and there is a neat throttle: **set the key with NX , and only publish if the set succeeded.** The first keystroke publishes; every keystroke for the next three seconds

finds the key present and publishes nothing. **And typing is never persisted** — a typing event that arrives after the message is noise.”

“A message is posted in a channel with five thousand members. Walk me through it.”

“The naive path is what I would rule out first, because it is what people write.

Naively: look up all five thousand members in the connection registry, find the eight hundred who are online, and push to each. That is eight hundred lookups and eight hundred pushes **on the sender’s request**, and it scales with membership rather than with anything you control.

What I would build is a pub/sub bus. Every connection server subscribes to `channel:<id>` **once, for as long as it holds any client interested in that channel** — not once per client, which is the mistake that multiplies the bus’s work by the number of connections.

The message is published once. The bus delivers it to the handful of connection servers that subscribed, and each one loops over its own local connections in process. **One publish on the request path; the fan-out happens on a system built for fan-out.**

The offline members are a completely separate path, and I would not queue anything for them. Their unread state comes from `last_read_message_id`, which is one row per user per channel. **When they come back, the unread count is a range count over an already-sorted partition** — messages newer than their mark.

A per-user queue would mean five thousand rows written per message, which is the write amplification the design exists to avoid.

Three details I would add.

The message is persisted before it is published, so a bus failure means it is undelivered rather than lost, and clients reconcile on reconnect by asking for everything after their last received id.

Search indexing is asynchronous — a separate queue — so a slow index cannot slow down posting.

And unread counts are computed lazily, not eagerly. Recomputing two hundred members’ badges on every message would be over a million recomputations a second across the system. **Computing them when a client actually asks means about five per user session,** which is thousands rather than millions.”

“How does search work, and how do you handle permissions?”

“An inverted index, asynchronously populated — and **the permissions part is the half that is actually interesting**, because getting it wrong is a data leak rather than a bug.

The index maps each term to the message ids containing it, in Elasticsearch or similar. Every message is written to it from a queue, so indexing is near-real-time — a message is searchable within seconds. **Making it synchronous would put the index on the posting path and mean a message fails when search is slow**, which is a much worse property.

Sharded by time, because almost all searches are about recent messages: a hot index over the last ninety days largely in memory, and cold shards for history.

Now permissions. A user must only see results from channels they are a member of. **The wrong design is to search everything and filter the results afterwards.**

And it is wrong for a specific, non-obvious reason: result counts leak. If I search and the system says ‘about forty-seven results’ while showing me three, **I have just learned that forty-four messages I cannot see match my query** — which tells me private channels exist, roughly how active they are, and that they discuss my search term. **In a workplace product that is a real leak**, and it has been a real incident at real companies.

The right design puts the permission in the query. Fetch the user’s channel memberships — a few hundred ids, cached — and add `channel_id IN (...)` as a filter clause. **The index evaluates it, so the counts are correct and nothing about invisible channels is observable.** It is also faster, because the filter prunes before ranking.

Two more things. Membership changes must invalidate the cache, or someone removed from a channel keeps searching it. **And deleted messages must be removed from the index**, not just flagged — a search index is exactly where deleted content survives longest, and ‘deleted’ meaning ‘still findable by search’ is the kind of thing that ends up in a news article.”

The model answer

“Design a team chat product: ten million daily users across many workspaces, channels of up to fifty thousand members, full history, search, and presence.”

“Two things in that prompt shape the design: **full history, which inverts the storage model, and fifty thousand-member channels, which is a genuine fan-out problem.** And presence, which I will come to last because it is where the surprising number is.

Sizing. Ten million daily users, two million concurrent, about fifty messages each per day — **five hundred million messages a day, roughly 5,800 a second, twenty thousand at peak.**

Transport: WebSockets with a connection registry in Redis, heartbeat-refreshed TTL. Same as any push system; I would not spend time re-deriving it.

Storage: partition by (channel_id, month), cluster by Snowflake message id descending. ‘The last fifty’ is one sequential read from one partition. **The month bucket is not optional** — a busy channel over five years is about nine hundred megabytes in a single partition, and Cassandra-family stores want partitions under a hundred. **Bucketing bounds every partition at around fifteen megabytes.** Discord published exactly this migration, which is the reference.

Ninety terabytes a year, tripled for replicas, kept forever. That is the cost of history, and it is three orders of magnitude more than a delete-on-delivery design. **I would raise the compliance consequence at the same time: an archive that exists can be subpoenaed and must support legal hold and export** — which for an enterprise product is a feature rather than a cost.

Fan-out: a pub/sub bus. Connection servers subscribe once per channel they hold clients for; a message is one publish. **At fifty thousand members with maybe eight thousand connected, the alternative is eight thousand registry lookups on the sender’s request,** which is unacceptable at twenty thousand messages a second.

And for very large channels I would add one thing: a rate limit on posting. A fifty-thousand-member channel where anyone can post at will is a fan-out amplifier, and **most products end up making huge channels announcement-only** — a product constraint doing an architecture’s job, like WhatsApp’s group cap.

Unread: last_read_message_id per user per channel, counts computed lazily and cached. Eager recomputation would be over a million range counts a second across the system; lazy is about ten thousand.

Search: an inverted index, asynchronously populated, sharded by time, with the user's channel memberships as a filter clause in the query. Not a **post-filter** — result counts would leak the existence and activity of private channels, which is a real data-leak class in workplace products.

Now presence, and I want to do the arithmetic because it is the part people get wrong by two orders of magnitude.

Two million concurrent users changing state thirty times an hour is seventeen thousand events a second. Fanned out to everyone sharing a channel — a thousand people — that is seventeen million pushes a second, against two hundred and thirty thousand for actual messages. Seventy times the message traffic, for a green dot.

Three fixes. Scope: push only to people currently viewing a channel that person is in — about twenty, not a thousand. **Throttle:** coalesce into one update per five seconds. **Together, about twenty thousand pushes a second, which is where it belongs.**

And presence must be an expiring key with no explicit offline event, because laptops close and phones lose signal and neither sends a disconnect. Absence of a heartbeat is the offline signal.

I would state plainly that presence is approximate — up to a minute stale — and that this is correct rather than a limitation. Nobody makes a decision based on a green dot, and guaranteeing accuracy would cost more than message delivery does.

Two risks I would flag. Deploys disconnect everyone a connection server holds, so clients need exponential backoff with jitter and deploys need to drain slowly, or the release is a self-inflicted thundering herd. And a bus partition means messages are stored but not delivered live — so clients must reconcile on reconnect by requesting everything after their last received id, and that reconciliation path has to be tested, because it only runs during incidents.”

9 Recall card

Inverts three WhatsApp decisions: history is kept forever (≈ 91 TB/year vs a few TB), channels have unbounded membership, and encryption is server-side — which makes search possible and makes it a first-class system. The WebSocket transport is unchanged; say so and move on.

Storage: partition by `(channel_id, month)`, **cluster by** **Snowflake id**. “The last fifty” is one sequential read. **The month bucket is required** — a busy channel over five years is ~900 MB in one partition against a ~100 MB guideline (Discord’s published migration is the reference).

Fan-out by pub/sub, not per-member lookups. Each connection server subscribes **once per channel**, not per client; one publish replaces hundreds of registry lookups on the sender’s request. **Offline members get no queue** — `last_read_message_id` is one row per user per channel and the unread count is **computed lazily and cached** (eager recomputation is > 1M range counts/second).

Presence is the surprise: built naively it is $\sim 70 \times$ the message traffic — 17,000 events/s $\times$ ~1,000 viewers = 17M pushes/s against 230,000 for messages. **Fix by scoping** (only people currently viewing a shared channel, ~20 not 1,000) and **throttling** (coalesce to one update per 5 s) $\rightarrow$ ~20,000/s.

Presence must be an expiring key with NO explicit offline event — closed laptops, lost signal and killed tabs never send a disconnect. **Absence of a heartbeat is the offline signal**, and the TTL is the dial between staleness and heartbeat traffic. **Typing = the same thing with a 3 s TTL and** `SET NX` **as the throttle**, never persisted. **Presence is approximate by design, and that is correct**.

Search permissions go IN the query as a `channel_id IN (...)` **filter, never as a post-filter** — result counts would leak the existence and activity of private channels.

DSA topic: DP on decisions: buy and sell stock **System design topic:** Design a chat system with presence

Code these, in this order

One rule for the whole set: **before writing anything, say the state out loud including the mode.** “Day, and whether I am holding” — then say which variant changes it. Five of these six are the same machine with one edit, and treating them as six separate problems is how people fall off the ladder.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Best Time to Buy and Sell Stock	LeetCode 121 (Easy)	Greedy: min-so-far and best difference.
2	Best Time to Buy and Sell Stock II	LeetCode 122 (Medium)	The two-state machine, and why greedy also works.
3	Best Time to Buy and Sell Stock with Cooldown	LeetCode 309 (Medium)	A third state, and saving yesterday's values.
4	Best Time to Buy and Sell Stock with Transaction Fee	LeetCode 714 (Medium)	One constant, no new state.
5	Best Time to Buy and Sell Stock III	LeetCode 123 (Hard)	$k = 2$, and where the transaction is consumed.
6	Best Time to Buy and Sell Stock IV	LeetCode 188 (Hard)	General k , and the $k \geq n/2$ shortcut.

On problem 2, write both solutions and compare

Write the two-variable state machine and the one-line greedy sum of positive differences. Confirm they agree on twenty random price series. **Then say why the greedy one works**, using the word “telescope”.

On problem 2, try the greedy on a limited version

Run your greedy on `[3, 3, 5, 0, 0, 3, 1, 4]` and record the answer. Then run problems 5 and 6 with $k = 2$ and $k = 1$ on the same input and record those. **Three different numbers. Say what greedy is ignoring.**

On problem 3, read the fresh value on purpose

Write the cooldown version without saving yesterday's `free` and `holding`. Run it on `[1, 2, 3, 0, 2]` and record the answer against the correct one. **Say precisely what the bug allows you to do**, in terms of days.

On problem 4, charge the fee twice

Subtract it on both the buy and the sell. Run on `[1, 3, 7, 5, 10, 3]` with a fee of 3 and record the answer. **Say why the wrong answer reads like “the fee is high” rather than like a bug.**

On problem 5, count the transaction on both halves

Write `buy[t]` from `sell[t-1]` and `sell[t]` from `buy[t-1]`. Run with `k = 2` on `[3, 3, 5, 0, 0, 3, 1, 4]` and record the answer. **Say which correct question you have accidentally answered.**

On problem 6, remove the shortcut

Delete the `k ≥ n // 2` check and call it with `k = 1_000_000_000`. Record the exact error. **Then say why `n/2` is the right threshold**, in one sentence about how many days a transaction needs.

On problem 6, measure the space

Write the full `n × k × 2` table version and the two-array version. Measure both at `n = 100_000`, `k = 100`. Record both figures and the ratio.

Then the ladder drill

Starting from the two-state machine, write each of the other four variants without looking at your earlier code. Time yourself. **The target is about two minutes each**, because each is one edit.

The state drill

1. State it as a sentence, including the word “mode”.
2. Say why “best profit in the first `n` days” is incomplete.
3. Say what goes wrong, and whether anything reports it.
4. Say why you track money rather than profit.

The transitions drill

1. Give both transitions with their meanings in words.
2. Give both base cases.

3. Say why the answer is the not-holding state.
4. Say what the answer becomes in the cooldown version, and why.

The greedy drill

1. Name the two variants greedy solves.
2. Give the one-transaction algorithm in two lines.
3. Give the unlimited algorithm in one line and justify it.
4. Say precisely why a transaction limit breaks it.
5. Give the three numbers on `[3, 3, 5, 0, 0, 3, 1, 4]`.

The k-transactions drill

1. Say what dimension is added.
2. Say where the transaction is consumed and why it must be stated.
3. Say what counting on both halves does, and what the answer looks like.
4. Give the shortcut and say why `n/2`.
5. Give the exact error without it.
6. Give the time and space, and the full-table comparison.

The cooldown drill

1. Name the three states and what each means.
2. Give all three transitions.
3. Say why `holding` reads `free` and not `cooling`.
4. Say why yesterday's values must be saved.
5. Say why the answer is a max of two states.
6. Say why adding a state costs nothing and adding a dimension costs a lot.

The break-it drill

Trigger each and record the exact output or error:

1. Counting the transaction on both halves.
2. `k = 10^9` without the shortcut.
3. Reading fresh values in the cooldown version.
4. The fee charged on both halves.
5. Greedy on a limited-transaction input.
6. An empty price list.
7. Returning `max` of both states in the plain version.

Five of the seven give no error at all. Name them.

The contrast drill

1. Give the five ways this differs from a WhatsApp-style design.
2. Say which parts of the transport are unchanged.
3. Say what keeping history costs, with numbers, and what it buys.

The storage drill

1. Give the partition key and the clustering key, and say why each.
2. Say what “the last fifty” costs.
3. Explain why the time bucket exists, with the partition-size arithmetic.
4. Say what scrollbar costs as a result.
5. Say why the message id should be a Snowflake.

The fan-out drill

1. Describe the naive path and count its operations.
2. Describe the pub/sub path and count its operations.
3. Say what each connection server subscribes to, and the mistake to avoid.
4. Say what happens for offline members, and what you deliberately do not build.
5. Say what a bus partition means and what clients must do.

The presence drill

1. Do the naive arithmetic from concurrent users to pushes per second.
2. Compare it with the message traffic.
3. Give both mitigations and the resulting number.
4. Say why presence must be an expiring key.
5. Name three cases that never send a disconnect.
6. Say what the TTL is a dial between.
7. Say why approximate presence is correct rather than a limitation.

The typing drill

1. Give the mechanism and the TTL.
2. Explain the `SET NX` throttle in one sentence.
3. Say why typing is never persisted.

The unread drill

1. Give the field and say what it replaces.
2. Compute the write amplification of the alternative.
3. Say how the count is produced.
4. Say why it is computed lazily, with the arithmetic.

The search drill

1. Say why indexing is asynchronous and what the alternative costs.
 2. Say why it shards by time.
 3. Give the wrong way to handle permissions.
 4. Say exactly what it leaks, and why counts are the problem.
 5. Give the right way.
 6. Name two things that must invalidate or update the index.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Buy and sell stock to maximise profit.* The state including the mode, why the naive state is incomplete, both transitions, the base cases, the answer, and which variants greedy solves.
 2. *Now at most k transactions, and then with a cooldown.* The added dimension, where the transaction is consumed and what happens if you get it wrong, the $k \geq n/2$ shortcut, the three cooldown states, and why saving yesterday's values matters.
 3. *Design Slack, and tell me how presence works.* The three inversions from WhatsApp, the bucketed partition key, pub/sub fan-out, the naive presence arithmetic and both mitigations, and expiring keys with no offline event.
-

Before you move on

- [] I can state the stock DP state including the word “mode”.
- [] I know why “best profit so far” is an incomplete state.
- [] I track money, not profit, and know why.
- [] I can write both transitions with their meanings.
- [] I know the answer is the not-holding state, and why.

- [] I know greedy solves one-transaction and unlimited, and only those.
- [] I can justify the unlimited greedy with the telescoping argument.
- [] I can give the three numbers that show greedy ignoring k .
- [] I know where the transaction is consumed and say it out loud.
- [] I know what counting on both halves does to the answer.
- [] I know the $k \geq n/2$ shortcut is required, not an optimisation.
- [] I can say why $n/2$ is the threshold.
- [] I can name the three cooldown states and all three transitions.
- [] I know why yesterday's values must be saved.
- [] I know the cooldown answer is a max over two states.
- [] I know a fee adds no state at all.
- [] I know adding a state is free and adding a dimension is not.
- [] I can give the space comparison at $n = 100,000$, $k = 100$.
- [] I can list the ways a channel chat differs from WhatsApp.
- [] I can give the partition key and justify the time bucket.
- [] I know the partition-size arithmetic that forces bucketing.
- [] I can describe pub/sub fan-out and count both paths.
- [] I know servers subscribe per channel, not per client.
- [] I know unread is one field, and the write amplification it avoids.
- [] I can do the naive presence arithmetic and the comparison.
- [] I can give both presence mitigations and the resulting number.
- [] I know presence must be an expiring key with no offline event.
- [] I can name three cases that never send a disconnect.
- [] I know why permission filters go in the query, and what a post-filter leaks.
- [] I answered all three questions above out loud.

Interval DP

DATA STRUCTURES AND ALGORITHMS

Interval DP

SYSTEM DESIGN

Design YouTube

1 What this is, and why they ask it

Interval DP is for problems where the state is a range — `dp[i][j]` describes the piece of the input from `i` to `j` — and the answer depends on how you split that range.

You met one already: the palindrome table on [day 155](#), where `dp[i][j]` said whether a substring was a palindrome. **Today the range is not just examined but divided**, and the recurrence tries every possible dividing point.

They ask it because **it is the hardest common DP shape and it has a recognisable signature**. Three nested loops — length, start, split point — and an inner `min` or `max` over the split. Once you have written it twice, you recognise it from the problem statement: “**combine adjacent things, and the cost of combining depends on what you combined before.**”

The other reason is that **it is where the fill order finally has a reason you must state**. `dp[i][j]` reads `dp[i][k]` and `dp[k+1][j]` — **both strictly shorter ranges** — so you fill by increasing length, and getting it wrong here produces an answer that is too cheap rather than too expensive, which is the harder direction to notice.

And there is a third thing that makes these problems genuinely hard: **the trick of asking the question backwards**. In burst balloons, thinking about which balloon to pop *first* leads nowhere, because popping changes the neighbours. **Thinking about which one to pop last works immediately**, because the last one in a range has fixed neighbours — the boundaries. That inversion is the whole difficulty of the problem, and it is worth having met once.

By the end of this lesson you can recognise the shape, write the three-loop skeleton, do matrix chain multiplication and burst balloons, explain the last-not-first inversion, and say what the cost is.

2 The story

The wedding was in four weeks and the argument at the table was about the order.

Not the order of the ceremony. The order the tents came down in.

Because Devrath had done seventy or eighty of these and he had learned a thing that none of the families ever believed until it happened, which was that **taking the tents down was harder than putting them up, and the sequence mattered more than the labour.**

The site had five sections. Dining, stage, kitchen, the covered walk between them, and the small one at the back for storage.

And here was the problem: you could only take down a whole section at a time, and you could only merge a section into the one next to it.

Which meant the men had to carry the poles and the sheets from wherever they finished to wherever they started next. **And the cost of any move was how much you had accumulated so far.**

If you took down dining first — it was the biggest — **you were then carrying dining's load into every subsequent move.** The whole afternoon, that weight, everywhere you went.

The nephew, who had been to college and was helping for the first time, said the obvious thing: **take the smallest one first.**

Devrath said he had tried that for years, and it was sometimes right and often not, **because it depended on what was next to what.**

And then the nephew, who was actually rather good at this, asked the question that made it work.

“Forget which one you take down first. Which one is left standing at the end?”

And Devrath stopped, because he had never once thought about it that way, and it was obviously the better question.

Because whichever section is last, everything to its left had to be cleared into one pile, and everything to its right into another, and those two problems have nothing to do with each other.

Two smaller versions of the same problem, and a known cost to join them.

“You still have to try all five as the last one,” the nephew said. “But then it is only five, and each one splits into two easier days.”

3 The idea in plain English

Devrath's nephew has just described interval DP, including the inversion that makes the hard version work.

The state is a range:

`dp[i][j]` is the answer for the piece of the input from index `i` to index `j` inclusive.

That is a two-dimensional table where both indices point into the same array, which is what distinguishes it from yesterday's grid and from LCS — there, the two indices pointed into two different things.

The recurrence tries every way of splitting the range.

```
dp[i][j] = best over all k in [i, j-1] of:
            dp[i][k] + dp[k+1][j] + cost_of_joining(i, k, j)
```

The `k` loop is the “which split” loop, and it is the third nest that makes these $O(n^3)$.

And the fill order is forced. `dp[i][j]` reads `dp[i][k]` and `dp[k+1][j]`, both strictly shorter than `i..j`. So every shorter range must already be computed:

```
for length in range(2, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1
        for k in range(i, j):
            ...
```

Length outermost, always. Fill in the natural row-by-row order and a cell reads shorter ranges that have not been computed yet, so it sees zeros — **and a zero does not mean “costs nothing”, it means “not done”**. The result is a total that is too CHEAP, which is the direction that makes it hard to spot: a plan that looks better than it should does not raise suspicion the way one that looks worse would.

Now the two problems that define the shape.

Matrix chain multiplication. You have matrices to multiply in a fixed order, and matrix multiplication is associative — `(AB)C` and `A(BC)` give the same result — **but they cost different amounts**. Multiplying a `10×100` by a `100×5` costs `10 × 100 × 5 = 5,000` scalar multiplications. **Find the cheapest parenthesisation.**

The state is exactly the shape above: `dp[i][j]` is the cheapest way to multiply matrices `i` through `j`. The split `k` is where the outermost multiplication happens — you compute `i..k` into one matrix, `k+1..j` into another, and multiply those two.

The joining cost is `dims[i] * dims[k+1] * dims[j+1]`, which is the size of the two resulting matrices. Getting those three indices right is the fiddly part, and the trick is to store dimensions as a single array where matrix `i` is `dims[i] * dims[i+1]`.

Burst balloons, which is the one that teaches the inversion.

You have balloons with numbers. Bursting balloon `i` earns `left * i * right`, where `left` and `right` are its current neighbours. After bursting, the neighbours become adjacent. Maximise your total.

The obvious state is “which balloon do I burst first in this range”, and it does not work. If you burst balloon `k` first, the range splits into `i..k-1` and `k+1..j` — but those two halves are no longer independent, because when you later burst the last balloon of the left half, its right neighbour is whatever survives in the right half. The subproblems are entangled, and DP needs them not to be.

The fix is Devrath’s nephew: ask which balloon is burst LAST.

If `k` is the last balloon burst in the range `i..j`, then when you burst it, everything else in the range is already gone — so its neighbours are exactly the boundaries, `i-1` and `j+1`, which are outside the range and therefore fixed.

```
dp[i][j] = max over k in [i, j] of:
    dp[i][k-1] + dp[k+1][j] + nums[i-1] * nums[k] * nums[j+1]
```

Now the two halves are genuinely independent, because each one’s boundaries are fixed and known. That is the whole problem, and the code afterwards is fifteen lines.

Say this out loud in an interview: “thinking about the first one to burst leaves the subproblems entangled; thinking about the last one fixes the boundaries and makes them independent.”

Two mechanical details for burst balloons. Pad the array with a 1 at each end, so `nums[i-1]` and `nums[j+1]` always exist — that removes every boundary special case. And the `k` loop runs over `[i, j]` inclusive, not `[i, j-1]`, because `k` is a member of the range rather than a split between two halves.

And the family, which is larger than it looks.

matrix chain	which multiplication is OUTERMOST
burst balloons	which balloon is burst LAST
stone game	which move is optimal, alternating players
optimal BST	which key is the ROOT
palindrome partition	where the cuts go
minimum score triangulation	which triangle contains the base edge

Every one of them asks “what is the last/outermost/root thing”, and every one splits into two independent ranges. Recognising that phrasing is most of the work.

Finally, the cost, which is why these are Hard.

$O(n^3)$ time: n^2 ranges, and $O(n)$ split points for each. $O(n^2)$ space.

At $n = 100$ that is a million operations — instant. At $n = 500$ it is 125 million — about a minute in Python. So the constraint on n is usually small, and seeing $n \leq 100$ or $n \leq 500$ in the problem statement is itself a hint that the answer is cubic.

4 The picture

The shape of the recurrence:

```

range i..j
i-----j
|         |         |
+-----+-----+
dp[i][k]  dp[k+1][j]    ← both STRICTLY SHORTER
          ^
          the split point k, tried at EVERY position

dp[i][j] = best over k of  dp[i][k] + dp[k+1][j] + join(i,k,j)

Three nested loops:
  length  (outermost, and it MUST be)
  i       (the start; j follows from i and length)
  k       (the split)
→  $O(n^3)$ 

```

Why the fill order is forced:

	j →				
	0	1	2	3	
i	+-----+-----+-----+-----+				
0			L2		L3 L4 the number is the RANGE LENGTH
	+-----+-----+-----+-----+				
1				L2	
	+-----+-----+-----+-----+				
2					L2
	+-----+-----+-----+-----+				

dp[0][3] (length 4) needs dp[0][k] and dp[k+1][3]
 → lengths 1..3, all SHORTER

filling row by row reaches dp[0][3] while dp[1][3] is still zero
 → reads a zero that means "not computed", not "cost zero"
 → the answer is nonsense, LOUDLY (unlike the palindrome table)

FILL BY INCREASING LENGTH. Diagonal by diagonal, upwards.

Matrix chain, and where the join cost comes from:

```

dims = [10, 100, 5, 50]
matrix 0 is 10x100, matrix 1 is 100x5, matrix 2 is 5x50

((A B) C):  A B = 10x100 x 100x5 → 10 x 100 x 5 = 5,000
            (AB)C = 10x5 x 5x50 → 10 x 5 x 50 = 2,500
            total 7,500

(A (B C)):  B C = 100x5 x 5x50 → 100 x 5 x 50 = 25,000
            A(BC) = 10x100 x 100x50 → 10 x 100 x 50 = 50,000
            total 75,000

SAME RESULT. 10x the cost. That is the whole problem.

join(i, k, j) = dims[i] * dims[k+1] * dims[j+1]
                ^           ^           ^
                rows of    the shared  columns of
                i..k       dimension   k+1..j
  
```

The burst-balloons inversion, which is the point of the lesson:

```
nums = [3, 1, 5, 8]          padded: [1, 3, 1, 5, 8, 1]
```

THINKING "FIRST" – DOES NOT WORK

```
burst 5 first, from range 3..8
  left half: [3, 1]    right half: [8]
  but when I later burst the 1 in the left half,
  its right neighbour is... the 8? or whatever is left?
→ the halves DEPEND ON EACH OTHER
→ not subproblems at all
```

THINKING "LAST" – WORKS

```
if 5 is burst LAST in range [3,1,5,8], then when I burst it
everything else in that range is already gone
→ its neighbours are the BOUNDARIES: the padded 1 on the left
  and the padded 1 on the right. FIXED.
→ reward = 1 * 5 * 1
→ and [3,1] and [8] are now fully independent problems,
  each with its own fixed boundaries
```

ONE CHANGE OF QUESTION. That is the entire difficulty.

The `k` loop differs between the two problems:

MATRIX CHAIN

`k` is a SPLIT between
two halves

```
for k in range(i, j):
    dp[i][k] + dp[k+1][j]
    ^ k is in the LEFT half
```

BURST BALLOONS

`k` is a MEMBER of the range
(the one burst last)

```
for k in range(i, j + 1):
    dp[i][k-1] + dp[k+1][j]
    ^ k is in NEITHER half
```

Getting this wrong is a silent off-by-one that produces
a plausible smaller answer.

The three loops, drawn as the diagonals they fill:

`n = 5`

```
length 1: (0,0) (1,1) (2,2) (3,3) (4,4)    base cases
length 2: (0,1) (1,2) (2,3) (3,4)
length 3: (0,2) (1,3) (2,4)
length 4: (0,3) (1,4)
length 5: (0,4)                                ← the answer
```

Each diagonal only reads diagonals ABOVE it (shorter ranges).
That is why length is the outer loop.

5 The code, built step by step

The skeleton, which every interval DP shares

```
def interval_dp_skeleton(n: int):
    dp = [[0] * n for _ in range(n)]
    for length in range(2, n + 1):          # LENGTH OUTERMOST. Always.
        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = float("inf")          # or -inf for maximising
            for k in range(i, j):            # every split point
                dp[i][j] = min(dp[i][j], dp[i][k] + dp[k + 1][j] + join(i, k, j))
    return dp[0][n - 1]
```

Write this skeleton first, then fill in `join` and the initialisation. The three loops and their order are the same in every problem of this family; only the joining cost and the direction of the optimisation change.

`length` starts at 2 because ranges of length 1 are the base case — a single item needs no combining.

Matrix chain multiplication

```
def matrix_chain(dims: list[int]) → int:
    """dims has n+1 entries; matrix i is dims[i] x dims[i+1]."""
    n = len(dims) - 1
    dp = [[0] * n for _ in range(n)]        # dp[i][i] = 0: one matrix, no work
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = float("inf")
            for k in range(i, j):
                cost = (dp[i][k] + dp[k + 1][j]
                        + dims[i] * dims[k + 1] * dims[j + 1])
                dp[i][j] = min(dp[i][j], cost)
    return dp[0][n - 1]
```

The `dims` convention is what makes this manageable: `n + 1` numbers for `n` matrices, so matrix `i` is `dims[i] × dims[i+1]` and the shared dimension between the two halves is `dims[k+1]`.

`dp[i][i] = 0` — a single matrix costs nothing to “multiply”.

Recovering the parenthesisation

```
def matrix_chain_order(dims: list[int]) → str:
    n = len(dims) - 1
    dp = [[0] * n for _ in range(n)]
    split = [[0] * n for _ in range(n)] # where the best k was
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = float("inf")
            for k in range(i, j):
                cost = dp[i][k] + dp[k + 1][j] + dims[i] * dims[k + 1] * dims[j + 1]
                if cost < dp[i][j]:
                    dp[i][j], split[i][j] = cost, k

    def build(i: int, j: int) → str:
        if i == j:
            return f"A{i}"
        k = split[i][j]
        return f"({build(i, k)} {build(k + 1, j)})"

    return build(0, n - 1)
```

One extra table recording the winning `k`, and a recursive walk-down. This is the standard reconstruction for interval DP, and it is a walk *down* rather than back, because the structure is a tree.

Burst balloons, with the padding

```
def max_coins(nums: list[int]) → int:
    values = [1] + [n for n in nums if n > 0] + [1] # pad both ends
    n = len(values)
    dp = [[0] * n for _ in range(n)]
    for length in range(1, n - 1): # length of the INNER range
        for i in range(1, n - length):
            j = i + length - 1
            for k in range(i, j + 1): # k is a MEMBER, so j+1
                dp[i][j] = max(
                    dp[i][j],
                    dp[i][k - 1] + dp[k + 1][j]
                    + values[i - 1] * values[k] * values[j + 1],
                )
    return dp[1][n - 2]
```

The padding is what removes every boundary case. `values[i-1]` and `values[j+1]` always exist, so there is no `if i == 0` anywhere.

`for k in range(i, j + 1)` — inclusive — because `k` is the balloon burst last, which belongs to neither half. Writing `range(i, j)` here is the silent off-by-one that gives a plausible smaller answer.

And `dp[1][n-2]` is the answer, not `dp[0][n-1]`, because indices 0 and `n-1` are the padding.

The memoised version, which some people find clearer

```
from functools import lru_cache

def max_coins_memo(nums: list[int]) → int:
    values = [1] + [n for n in nums if n > 0] + [1]

    @lru_cache(maxsize=None)
    def best(i: int, j: int) → int:
        if i > j:
            return 0 # empty range
        return max(
            best(i, k - 1) + best(k + 1, j)
            + values[i - 1] * values[k] * values[j + 1]
            for k in range(i, j + 1)
        )

    return best(1, len(values) - 2)
```

Identical complexity, and the fill order is handled for you by the recursion — which is a genuine argument for writing interval DP top-down under time pressure. **Say that explicitly:** “I would write it memoised, because then I cannot get the fill order wrong.”

Stone game, which is interval DP with two players

```
def stone_game(piles: list[int]) → int:
    """The first player's score minus the second's, both playing optimally."""
    n = len(piles)
    dp = [[0] * n for _ in range(n)]
    for i in range(n):
        dp[i][i] = piles[i] # one pile: take it
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = max(piles[i] - dp[i + 1][j], # take the left
                           piles[j] - dp[i][j - 1]) # take the right
    return dp[0][n - 1]
```

The minus sign is the whole trick. `dp[i][j]` is the *difference* the current player can achieve, so the opponent's best difference on the remaining range is subtracted. **One expression handles both players**, which is much cleaner than tracking whose turn it is.

And note this one has no k loop — the split is always at an end, so it is $O(n^2)$ rather than $O(n^3)$.

The complete solution

```
"""Interval DP: the shape, the classics, and the last-not-first inversion."""

from functools import lru_cache

def matrix_chain(dims: list[int]) → int:
    """dims has n+1 entries; matrix i is dims[i] x dims[i+1]."""
    n = len(dims) - 1
    if n < 2:
        return 0
    dp = [[0] * n for _ in range(n)]          # one matrix costs nothing
    for length in range(2, n + 1):           # LENGTH OUTERMOST
        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = float("inf")
            for k in range(i, j):             # k is a SPLIT: [i..k] [k+1..j]
                cost = (dp[i][k] + dp[k + 1][j]
                        + dims[i] * dims[k + 1] * dims[j + 1])
                dp[i][j] = min(dp[i][j], cost)
    return int(dp[0][n - 1])

def matrix_chain_order(dims: list[int]) → str:
    """The cheapest parenthesisation itself."""
    n = len(dims) - 1
    if n == 1:
        return "A0"
    dp = [[0] * n for _ in range(n)]
    split = [[0] * n for _ in range(n)]
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = float("inf")
            for k in range(i, j):
                cost = dp[i][k] + dp[k + 1][j] + dims[i] * dims[k + 1] * dims[j + 1]
                if cost < dp[i][j]:
                    dp[i][j], split[i][j] = cost, k

    def build(i: int, j: int) → str:
        if i == j:
            return f"A{i}"
        k = split[i][j]
        return f"({build(i, k)} {build(k + 1, j)})"

    return build(0, n - 1)

def max_coins(nums: list[int]) → int:
    """Burst balloons. k is the balloon burst LAST, not first."""
    values = [1] + [x for x in nums if x > 0] + [1]    # padding kills edge cases
    n = len(values)
    dp = [[0] * n for _ in range(n)]
    for length in range(1, n - 1):
        for i in range(1, n - length):
            j = i + length - 1
            for k in range(i, j + 1):             # INCLUSIVE: k is a member
                dp[i][j] = max(
                    dp[i][j],
                    dp[i][k - 1] + dp[k + 1][j]
                    + values[i - 1] * values[k] * values[j + 1],
                )
    return dp[1][n - 2]                          # 0 and n-1 are the padding
```

```

def max_coins_memo(nums: list[int]) → int:
    """The same, top-down. The recursion handles the fill order for you."""
    values = [1] + [x for x in nums if x > 0] + [1]

    @lru_cache(maxsize=None)
    def best(i: int, j: int) → int:
        if i > j:
            return 0
        return max(
            best(i, k - 1) + best(k + 1, j)
            + values[i - 1] * values[k] * values[j + 1]
            for k in range(i, j + 1)
        )

    return best(1, len(values) - 2)

def stone_game(piles: list[int]) → int:
    """First player's score minus the second's. No k loop:  $O(n^2)$ ."""
    n = len(piles)
    dp = [[0] * n for _ in range(n)]
    for i in range(n):
        dp[i][i] = piles[i]
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = max(piles[i] - dp[i + 1][j], piles[j] - dp[i][j - 1])
    return dp[0][n - 1]

def min_score_triangulation(values: list[int]) → int:
    """Which triangle contains the edge i..j? Same shape again."""
    n = len(values)
    dp = [[0] * n for _ in range(n)]
    for length in range(3, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = float("inf")
            for k in range(i + 1, j):
                dp[i][j] = min(dp[i][j],
                                dp[i][k] + dp[k][j] + values[i] * values[k] * values[j])
    return int(dp[0][n - 1])

def min_cut_palindrome(s: str) → int:
    """Palindrome partitioning: interval DP for the table, 1-D for the cuts."""
    n = len(s)
    if n < 2:
        return 0
    is_pal = [[False] * n for _ in range(n)]
    for i in range(n - 1, -1, -1):
        for j in range(i, n):
            if s[i] == s[j] and (j - i < 2 or is_pal[i + 1][j - 1]):
                is_pal[i][j] = True
    cuts = [0] * n
    for j in range(n):
        cuts[j] = 0 if is_pal[0][j] else min(
            cuts[i - 1] + 1 for i in range(1, j + 1) if is_pal[i][j])
    return cuts[n - 1]

if __name__ == "__main__":
    dims = [10, 100, 5, 50]

```

```

print("matrix chain cost :", matrix_chain(dims))
print("the parenthesising :", matrix_chain_order(dims))
print("bigger chain      :", matrix_chain([40, 20, 30, 10, 30]))
print("its order        :", matrix_chain_order([40, 20, 30, 10, 30]))

print("burst [3,1,5,8]   :", max_coins([3, 1, 5, 8]))
print("memo agrees      :", max_coins_memo([3, 1, 5, 8]))
print("burst [1,5]       :", max_coins([1, 5]))
print("burst single      :", max_coins([7]))

print("stone game [5,3,4,5]:", stone_game([5, 3, 4, 5]))
print("stone game [3,7,2,3]:", stone_game([3, 7, 2, 3]))

print("triangulation      :", min_score_triangulation([1, 3, 1, 4, 1, 5]))
print("palindrome cuts    :", min_cut_palindrome("aab"))

```

Run it and you get:

```

matrix chain cost : 7500
the parenthesising : ((A0 A1) A2)
bigger chain      : 26000
its order        : ((A0 (A1 A2)) A3)
burst [3,1,5,8]   : 167
memo agrees      : 167
burst [1,5]       : 10
burst single      : 7
stone game [5,3,4,5]: 1
stone game [3,7,2,3]: 5
triangulation     : 13
palindrome cuts   : 1

```

matrix chain cost 7500 against the **75,000** of the other order is the point of that problem: **ten times the work for the same answer.**

And burst [3,1,5,8] = 167 is the classic — burst 1 first, then 5, then 3, then 8, which is a sequence you would never find by thinking forwards.

6 What it costs

Time: three nested loops.

```

length  n values
i       up to n values for each length
k       up to n values for each (i, j)

→  $O(n^3)$ 

more precisely: sum over lengths L of  $(n - L + 1) \times (L - 1)$ 
                ~  $n^3 / 6$ 

```

Concretely:

n = 50	$50^3 / 6 = \sim 21,000$ operations	instant
n = 100	$\sim 167,000$	instant
n = 500	$\sim 21,000,000$	~ 5 s in Python
n = 1,000	$\sim 167,000,000$	~ 40 s. Too slow.
n = 5,000	$\sim 2 \times 10^{10}$	hours

So the constraint tells you the shape. LeetCode's burst balloons caps n at 500, matrix chain problems usually cap at 100. **If you see $n \leq 100$ with a range-shaped problem, cubic is expected** — and if you see $n \leq 10^5$, interval DP is not the answer and you should look for something else.

Space: $O(n^2)$.

```
n = 500: 250,000 cells
        Python list of lists of ints: ~28 bytes per int object
        → ~7 MB. Fine.

n = 5,000: 25,000,000 cells → ~700 MB. Not fine.
```

And unlike the last few days, there is no space collapse. `dp[i][j]` reads cells from many different rows — `dp[i][k]` for every k — so **you cannot keep one row**. **Saying that explicitly when asked to reduce the space is the right answer**, rather than searching for a trick that is not there.

The reconstruction cost:

```
one extra n x n table storing the winning k
→  $O(n^2)$  more space, no extra time
→ the walk-down is  $O(n)$ : each call splits into two, and there are
    n leaves
```

Compared with brute force:

the number of ways to parenthesise n matrices is the $(n-1)$ th Catalan number:

```
n = 5      14 ways
n = 10     4,862
n = 15     2,674,440
n = 20     1,767,263,190
n = 25     ~1.3 x 1012
```

DP at $n = 25$: $25^3 / 6 = \sim 2,600$ operations

→ 10^{12} against 10^3 . Nine orders of magnitude, which is why this problem is in every textbook.

Stone game, for contrast:

no k loop – the split is always at an end
→ $O(n^2)$ time, $O(n^2)$ space

$n = 5,000$: 25,000,000 operations, ~5 s. Viable where the cubic version would not be.

And a note on the memoised version:

same $O(n^3)$ calls, plus function-call overhead
in Python: roughly 2-3x slower than the bottom-up loops
in exchange for: the fill order cannot be wrong

for $n \leq 500$ the difference is seconds, and the correctness guarantee is usually worth more.

7 The traps

Filling row by row instead of by length.

```
>>> dims = [10, 100, 5, 50]
>>> n = 3
>>> dp = [[0] * n for _ in range(n)]
>>> for i in range(n):
...     for j in range(i + 1, n):
...         dp[i][j] = min(dp[i][k] + dp[k+1][j] + dims[i]*dims[k+1]*dims[j+1]
...                         for k in range(i, j))
>>> dp[0][2]
7500
```

Here it happens to be right, because `n = 3` is small enough that the dependencies are satisfied by accident. On a longer chain it is not:


```
>>> dims = [40, 20, 30, 10, 30]
>>> n = 4
>>> dp = [[0] * n for _ in range(n)]
>>> for i in range(n):
...     for j in range(i + 1, n):
...         dp[i][j] = min(dp[i][k] + dp[k+1][j] + dims[i]*dims[k+1]*dims[j+1]
...                         for k in range(i, j))
>>> dp[0][3]
20000
```

Twenty thousand, when the true minimum is twenty-six thousand — and note it came out *lower* than the real answer, because it read a `dp[1][3]` that was still zero and therefore looked free. **A cost that is too cheap is the direction that makes this hard to spot**, since a plan that looks better than possible does not raise suspicion the way a plan that looks worse would.

And a dependency satisfied by an accident of size is the worst kind of bug, because the three-matrix test passes. **Write `for length in ...` first, every time.**

Using `range(i, j)` for `k` in burst balloons.

```
>>> # k is the balloon burst LAST — it is a MEMBER of [i, j]
>>> # range(i, j) excludes j, so the rightmost balloon is never
>>> # considered as the last one burst
>>> # → a plausible SMALLER answer, no error
```

Matrix chain uses `range(i, j)` because `k` is a split; burst balloons uses `range(i, j+1)` because `k` is a member. They look identical and are not, and this is the single most common bug in the topic.

Thinking “first” in burst balloons.

```
burst k first: left half i..k-1, right half k+1..j
               but the left half's rightmost balloon will,
               when burst, have a right neighbour that depends
               on what is left in the RIGHT half

→ the subproblems are not independent
→ the recurrence you write is simply wrong, and it produces
  a plausible number
```

No error. A wrong answer. And it is very hard to debug, because the code looks correct — **the mistake is in the modelling, not the implementation.**

Forgetting the padding.

```
>>> nums = [3, 1, 5, 8]
>>> # without padding, dp[0][j] needs nums[-1]
>>> nums[-1]
8
```

Python's negative indexing does not raise — it wraps to the end of the list, so the boundary silently uses the wrong balloon. Pad with 1s and the problem cannot occur.

Initialising to 0 when minimising.

```
>>> dp = [[0] * 3 for _ in range(3)]
>>> # dp[i][j] = min(dp[i][j], cost) with dp[i][j] starting at 0
>>> # 0 wins every min → every answer is 0
```

Set the cell to infinity before the `k` loop when minimising, and to zero only when maximising. The base cases — length 1 — are genuinely zero in both, which is what makes the confusion easy.

Getting the matrix chain dimension indices wrong.

```
>>> # dims[i] * dims[k+1] * dims[j+1]      correct
>>> # dims[i] * dims[k] * dims[j]           WRONG, and it runs
```

All three are valid indices, so there is no error — just a cost function computing the wrong thing. Derive them from the convention out loud: the left result is `dims[i] × dims[k+1]`, the right is `dims[k+1] × dims[j+1]`, so the multiplication costs `dims[i] × dims[k+1] × dims[j+1]`.

Trying to collapse the space.

```
>>> # dp[i][j] reads dp[i][k] for EVERY k in [i, j-1]
>>> # those live in row i, spread across many columns
>>> # and dp[k+1][j] lives in many DIFFERENT ROWS
>>> # → there is no one-row version. Do not look for one.
```

Saying “there is no space collapse here, because the recurrence reads many rows” is the correct answer to the reduce-the-space question, and it is better than an incorrect attempt.

`n` too large.

```
>>> dp = [[0] * 5000 for _ in range(5000)]
>>> # 25,000,000 cells, ~700 MB, and  $n^3/6 = 2 \times 10^{10}$  operations
```

No error at `n = 5,000` for the allocation on a large machine — it just never finishes. The constraint in the problem is the signal, and reading it before choosing the approach is the whole defence.

8 In the interview

How it gets asked

- “Given a chain of matrices, find the cheapest order to multiply them.” — matrix chain, the classic.
- “Burst balloons to maximise coins.” — LeetCode 312, and the inversion is the whole question.
- “Two players take stones from either end of a row. Who wins?” — LeetCode 877 / 486.
- “What is the fill order, and why?”
- “Can you reduce the space?” — where the right answer is “no, and here is why”.
- “Minimum score triangulation of a polygon.” — LeetCode 1039, the same shape again.

The first ninety seconds

“This is interval DP, and the signature is that **the state is a range and the answer depends on how I split it.**

`dp[i][j]` is the answer for the piece of the input from `i` to `j` inclusive. Both indices point into the same array, which is what distinguishes this from something like longest common subsequence where they pointed into two different strings.

The recurrence tries every split point. `dp[i][j]` is the best over all `k` of `dp[i][k]` plus `dp[k+1][j]` plus whatever it costs to join those two halves. **That `k` loop is the third nest, and it is what makes these cubic.**

And the fill order is forced, which is the part I would state before writing any loops. `dp[i][j]` reads `dp[i][k]` and `dp[k+1][j]`, **both strictly shorter ranges** — so every shorter range must already be computed. **Length is the outermost loop, then the start, then the split.** Filling row by row reads cells that do not exist yet.

Base cases are ranges of length one, which need no combining and cost nothing.

`O(n3)` time and `O(n2)` space — `n2` ranges, `O(n)` splits each.

And there is no space collapse here, which is worth saying up front because it is usually the next question: `dp[i][j]` reads many different rows, not just the previous one, **so keeping a single row is not possible.** That is a real difference from the grid problems.

One practical thing: I would write it memoised, top-down, rather than bottom-up. Same complexity, and **the recursion handles the fill order for me, so I cannot get it wrong under time pressure.** In Python it is two or three times slower from call overhead, which for the `n ≤ 500` these problems come with is seconds.

What is the constraint on `n`? Because cubic means `n` in the hundreds, and if it is a hundred thousand then this is not the right shape and I should be looking for something else.”

The follow-ups

“Walk me through burst balloons.”

“This one has a single idea in it, and everything else is fifteen lines of code, so let me spend the time on the idea.

The obvious approach is to ask which balloon I burst first, and it does not work. If I burst balloon `k` first, the range appears to split into everything left of `k` and everything right of `k`. **But those two halves are not independent.** When I later burst the rightmost balloon of the left half, its right neighbour is whatever is still standing in the right half — **so the two subproblems depend on each other**, and dynamic programming needs them not to.

The fix is to ask which balloon is burst last.

If `k` is the last one burst in the range `i..j`, then at the moment I burst it, **everything else in that range is already gone**. So its neighbours are exactly the boundaries — the elements just outside the range — **which are fixed and known**. The reward is `nums[i-1] * nums[k] * nums[j+1]`.

And now the two halves are genuinely independent, because each has fixed boundaries of its own.

```
dp[i][j] = max over k of dp[i][k-1] + dp[k+1][j] + nums[i-1]*nums[k]*nums[j+1] .
```

Two mechanical details. Pad the array with a 1 at each end, so `nums[i-1]` and `nums[j+1]` always exist — otherwise Python’s negative indexing silently wraps to the end of the list and uses the wrong balloon, with no error.

And the `k` loop is inclusive of `j`, unlike matrix chain. Here `k` is a member of the range — the one burst last — so it belongs to neither half. In matrix chain, `k` is a split point and belongs to the left half, so the loop excludes `j`. **They look identical and they are not, and writing `range(i, j)` here gives a plausible smaller answer with no error.**

`O(n3)` time, `O(n2)` space, and the answer is `dp[1][n-2]` because the outer indices are padding.

The general lesson is worth naming: when the obvious decomposition leaves the subproblems entangled, try asking about the last thing rather than the first. The same inversion solves matrix chain — which multiplication is outermost — and optimal binary search trees — which key is the root.”

“What is the fill order and why does it matter here more than usual?”

“Because the dependency points at strictly shorter ranges, and the natural loop order does not respect that.

`dp[i][j]` reads `dp[i][k]` and `dp[k+1][j]` for every split `k`. Both of those are strictly shorter than `i..j` — the first ends earlier, the second starts later. **So every shorter range must be finished first.**

The correct order is by increasing length: all ranges of length two, then three, and so on. Geometrically that is filling the table **diagonal by diagonal, moving away from the main diagonal**, and each diagonal reads only the ones above it.

Filling row by row breaks it. Going along row zero, I reach `dp[0][4]` while `dp[1][4]` — which it needs — is still zero. **And zero does not mean ‘costs nothing’, it means ‘not computed’**, so the answer is wrong.

What makes this nastier than usual is that a small test can pass by accident. With three matrices, the natural order happens to satisfy the dependencies, so a three-element test passes and a five-element one fails. **A dependency satisfied by an accident of size is the worst kind of bug.**

There is a second correct order — `i` descending, `j` ascending from `i` — which also works, because row `i+1` is complete before row `i` starts. **But the length version makes the reason visible in the loop itself**, so it is the one I would write.

And the honest practical answer: I would write it memoised. A top-down recursion with a cache explores the dependencies in whatever order it needs, **so the fill order cannot be wrong.** Same complexity, and in an interview that is worth two or three times the constant factor.”

“Can you reduce the space?”

“No, and I think saying so clearly is a better answer than attempting something that does not work.

In the grid problems, the collapse worked because `dp[r][c]` read exactly two cells — the one above and the one to the left — so one row of history was enough.

Here `dp[i][j]` reads `dp[i][k]` for every `k` from `i` to `j-1`, and `dp[k+1][j]` for every one of those too. The first group is spread across row `i`; the second group is spread across many different rows. There is no single row, or pair of rows, that contains what I need, so the whole $O(n^2)$ table has to be live.

What I can do is smaller than that, and worth mentioning.

Only half the table is ever used — cells where `i > j` are meaningless — so a triangular representation saves about half the memory. In Python that costs more in overhead than it saves; in C it is worth doing.

And if I need the reconstruction I need a second table anyway, so I would raise that trade: the answer alone is $O(n^2)$, and the answer plus the parenthesisation is two $O(n^2)$ tables.

The real limit is the time, not the space. At `n = 500`, $n^3/6$ is about twenty-one million operations and the table is seven megabytes — so I run out of patience long before I run out of memory. At `n = 5,000` it is two times ten to the ten operations, which is hours, and the seven hundred megabytes is the lesser problem.

So if someone needs `n = 10,000`, the answer is not a better constant factor — it is that interval DP is the wrong approach and there must be extra structure in the problem to exploit. Matrix chain specifically has an $O(n \log n)$ algorithm — Hu and Shing — which I know exists and would not write.”

The model answer

“You are compiling a query plan. A query joins `n` tables in a fixed logical order, and joining two intermediate results costs the product of their row counts. Find the cheapest order to perform the joins.”

“This is matrix chain multiplication with different words, and I want to establish the mapping before writing anything, because that is where the value is.

A join of two intermediate results is a binary operation whose cost depends on the sizes of both operands — and those sizes depend on which joins you did earlier. That is exactly the matrix chain structure: **associative**, so every order gives the same answer, and the orders cost wildly different amounts.

And the fixed logical order matters. Because the tables must be joined in sequence — table one with two, that with three — **the only freedom is the parenthesisation**, which is what makes it an interval problem rather than a general search over join trees. **I would confirm that**, because if any table can join any other, this is a completely different and much harder problem — that is the general join-ordering problem, which is exponential and is why real optimisers use heuristics.

The state: `dp[i][j]` is the cheapest cost to join tables `i` through `j` into one intermediate result.

The recurrence tries every split. For each `k`, compute `i..k` into one result and `k+1..j` into another, then join those two. `dp[i][j] = min over k of dp[i][k] + dp[k+1][j] + cost_of_joining(i, k, j)`.

The joining cost needs the sizes of the two intermediates, and that is the modelling question I would raise: **for matrices the resulting dimensions are determined; for joins they are estimated.** I would need a cardinality estimate for the result of joining `i..k` — from statistics, or a selectivity assumption — and **that estimate is where real query planners are wrong**, far more often than the search is.

Fill order: by increasing length, because `dp[i][j]` reads only strictly shorter ranges. **Base case:** a single table costs nothing to ‘join’.

Reconstruction matters more here than in the textbook version, because the planner needs the plan, not the cost. **One extra table recording the winning `k`**, then a recursive walk down producing the join tree.

Cost: $O(n^3)$ time and $O(n^2)$ space. For a query joining ten tables that is about 170 operations — nothing. **For fifty tables it is 21,000 — still nothing.** **And that is the point I would make about feasibility: query plans are small.** The cubic bound that limits this algorithm in the abstract is irrelevant when `n` is under a hundred, which it always is.

Two things I would say beyond the algorithm.

First, this finds the optimal plan under the cost model, and the cost model is the weak link. A plan that is optimal for wrong cardinality estimates is not optimal. I would spend more effort on the estimates than on the search, and I would want the planner to be robust to being wrong — preferring plans that are acceptable across a range of estimates rather than optimal for one.

Second, real optimisers memoise this exactly like the DP — that is what a Selinger-style bottom-up planner is — but they also prune aggressively, discarding subplans that are dominated, because the real problem has more freedom than a fixed order and the search space is larger than n^3 .

And I would write it memoised rather than bottom-up. Same complexity, the fill order cannot be wrong, and a top-down formulation makes it much easier to add pruning later, which is the direction this would actually grow.”

9 Recall card

Interval DP: `dp[i][j]` is the answer for the range `i..j`, and the recurrence tries every split. `dp[i][j] = best over k of dp[i][k] + dp[k+1][j] + join(i,k,j)`. Three nested loops — length, start, split — and length MUST be outermost, because both dependencies are strictly shorter ranges. A small test can pass by accident with the wrong order, which is the worst kind of bug.

$O(n^3)$ time, $O(n^2)$ space, and there is NO space collapse — `dp[i][j]` reads many different rows, not just the previous one. Saying that is the right answer to “reduce the space”. `n ≤ 100` or `n ≤ 500` in the constraints is the signal that cubic is expected.

Matrix chain: `join = dims[i] × dims[k+1] × dims[j+1]` with matrix `i` being `dims[i] × dims[i+1]`. `[10,100,5,50]` costs 7,500 one way and 75,000 the other — ten times, for the same answer. Brute force is the Catalan numbers: 1.3×10^{12} at `n = 25` against ~2,600 for the DP.

Burst balloons is the inversion, and it is the whole problem: ask which balloon is burst LAST, not first. Thinking “first” leaves the two halves entangled, because the left half’s last burst depends on what survives on the right. Thinking “last” fixes its neighbours to the range boundaries, so the halves become independent. Pad both ends with 1 (negative indexing silently wraps otherwise), and the `k` loop is INCLUSIVE of `j` — `k` is a member, not a split — unlike matrix chain, which is the most common bug here.

Initialise to `inf` before the `k` loop when minimising (0 wins every `min`). Write it memoised top-down, because then the fill order cannot be wrong. **Stone game has no `k` loop** — the split is always at an end — so it is $O(n^2)$, and the minus sign (`piles[i] - dp[i+1][j]`) handles both players in one expression.

1 What this is, and why they ask it

Video is the largest thing in computing that ordinary people do every day, and designing a video platform is where every number in this course gets multiplied by a thousand.

A photo is 2 MB. **A ten-minute video at decent quality is 300 MB, and the same video stored in five qualities is more than a gigabyte.** A billion hours of watch time a day is exabytes of egress. **The bill is the design constraint**, more directly than in any other system here.

They ask it because **it has three sub-systems that are each genuinely hard and are visibly separate.**

Transcoding, which is a compute problem of a size nothing else in this course approaches — roughly real-time per output quality, so an hour of uploaded video is hours of CPU. **It is also embarrassingly parallel if you chunk it correctly**, and seeing that is the main insight.

Delivery, where adaptive streaming means the client, not the server, decides what quality to fetch, chunk by chunk, as the network changes. **That inverts the usual client-server relationship** and is why video plays smoothly on a train.

And storage economics, where a brutal power law — a tiny fraction of videos get almost all the views — means that tiering is not an optimisation but the only affordable design.

And they ask it because **it is the best test of scoping in the whole set.** The real product has recommendations, comments, live streaming, monetisation, copyright detection, moderation and creator analytics. **Naming those and picking three is the answer**; trying to design all of them produces nothing.

By the end of this lesson you can design the upload and transcoding pipeline, adaptive streaming, the delivery layer, storage tiering and view counting — and size the bill.

2 The story

The video parlour was one room with a television and forty plastic chairs, and Bashir ran it for nineteen years, and everything he learned he learned by being wrong first.

The first thing was the tapes.

He started with one copy of each film. Which was fine until *Sholay*, when eleven people wanted it on the same Sunday and there was one cassette. **So he bought three more copies of the films people asked for, and one copy of everything else.**

The shelf ended up strange-looking. **Four copies of six films, and one copy each of two hundred others**, and most of those two hundred were not touched from one month to the next. When the shelf got full he moved the untouched ones into a trunk under the stairs, **and if somebody asked for one it took him ten minutes to find** — which happened about twice a year and nobody minded.

The second thing was the machines.

He had one player at first, and then the second-hand shop sold him a second one, and the thing he did with it was not what anybody expected. **He used it to make copies.**

Because a tape that came from the distributor was one tape, **and copying it took as long as watching it** — two and a half hours for a two-and-a-half-hour film. One machine, one film, an afternoon.

With two machines he could copy two films at once. With five, five. The copying did not get faster. There was just more of it happening.

The third thing was the picture quality, and it was the one that took longest to understand.

His nephew had a shop in the next town with the same films, and he made his copies fast and rough. **Bashir made his slowly, at the best quality the machine could do.** And Bashir was certain, for about two years, that this made his parlour better.

Then he watched a film on his nephew's television, which was old and small, **and could not tell the difference at all.**

The good copy was better only on a good television. On a bad one it was wasted — the same tape, the same two and a half hours of machine time, and nobody in the room could see it.

So he started keeping two versions of the popular films. **The good one for the evening shows on the big set, and the rough one for the small television in the back room**, which used less tape and was ready sooner.

“It is not one film,” he told his nephew. “It is the same film, four times, and you send whichever one fits the television.”

3 The idea in plain English

Bashir’s parlour is YouTube. **Extra copies of the popular ones, a trunk under the stairs for the rest, copying that parallelises across machines but never gets faster per machine, and several qualities of the same thing because the screen decides which one is worth sending.**

Scope first, because the product is enormous.

IN	upload transcode into multiple qualities adaptive playback view counts storage tiering
OUT	recommendations, comments, live streaming, monetisation, copyright matching (Content ID), moderation, analytics

Say the out list and move on. Recommendations in particular is a machine-learning system that deserves its own interview, and offering to design it here is how you run out of time.

The upload path is Instagram’s, and the same rules apply.

Pre-signed URL, direct to object storage, never through your servers — [day 155](#)’s reasoning, and it matters more here because the files are hundreds of times larger. **A 300 MB upload through your application tier is absurd; a 300 MB upload through your application tier at a thousand a second is impossible.**

And video needs resumable uploads, which photos do not. A phone uploading 300 MB on a mobile connection will lose the connection, and **restarting from zero is unacceptable**. Chunked upload with a session id and a per-chunk acknowledgement — S3 multipart, or the `tus` protocol — so a dropped connection resumes from the last completed chunk.

Now transcoding, which is where the compute is.

The uploaded file is not what anyone watches. It must become several *renditions* — resolutions and bitrates — in a standard codec, split into short segments, with a manifest listing them.

2160p	@ 20 Mbps	only for the small fraction with the screen and the network
1080p	@ 5 Mbps	
720p	@ 2.5 Mbps	the workhorse
480p	@ 1 Mbps	
360p	@ 0.6 Mbps	
240p	@ 0.4 Mbps	the one that keeps playback alive on a bad connection

Transcoding runs at roughly real-time per rendition on a CPU. A ten-minute video with six renditions is about an hour of CPU. **At five hundred hours uploaded per minute, that is thirty thousand hours of CPU per minute of wall clock** — which is the number that makes video a different business from photos.

And the key insight, which is Bashir's second machine: it parallelises by chunk.

Split the source into segments of a few seconds and transcode each independently, on a different machine. A ten-minute video becomes two hundred three-second chunks; two hundred workers finish in about three seconds of wall time each rather than ten minutes.

Two things make that work and are worth naming. Segments must be cut at keyframes, because a video frame is usually described relative to earlier frames and a chunk that starts mid-sequence cannot be decoded alone. **And the outputs must be stitched with consistent timing,** or the player stutters at every boundary.

Then delivery, and the idea that makes video work at all.

Adaptive bitrate streaming. The client downloads a **manifest** — an HLS `.m3u8` or DASH `.mpd` — listing every rendition and every segment. **It starts at a low quality, measures how fast segments arrive, and switches quality between segments.**

The client decides, not the server. That inversion is the whole design: **the server publishes options and the client picks,** which is why a video keeps playing when you walk into a lift and comes back to full quality when you leave.

And the segments are static files, which is why this scales: **the CDN serves them like any other object, with no video-specific logic at the edge.** There is no streaming server holding a connection per viewer.

Then storage, where the power law does the work.

Almost all views go to a tiny fraction of videos. Something like the top one percent gets ninety percent of the traffic, and the long tail — Bashir's two hundred untouched tapes — is watched almost never.

So tier by popularity, not only by age. Hot videos live on fast storage in every region, warm ones in fewer regions, cold ones in archival storage where retrieval takes minutes. **And the tail is enormous:** it is most of the bytes and almost none of the traffic.

One consequence worth stating: a cold video that suddenly goes viral must be promoted quickly, or the first thousand viewers get a video that takes minutes to start. **Popularity tiering needs a fast path back up.**

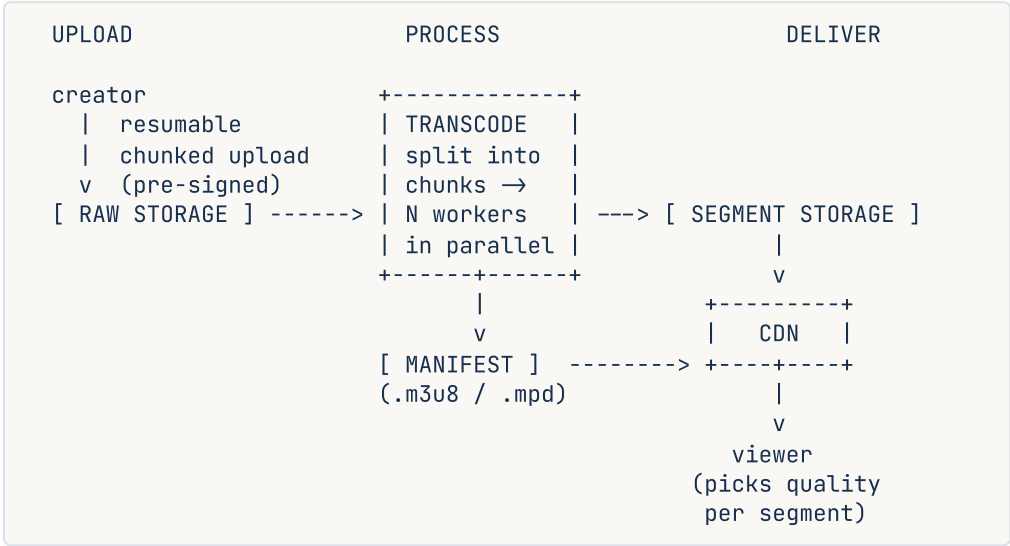
And finally, view counts, which are harder than they look.

A view is not a page load. It has a definition — thirty seconds watched, or a percentage of the video — and that definition has to be enforced, because views are money.

Counting exactly at this scale is expensive, so counts are approximate and eventually consistent. **The displayed count lags by minutes and that is fine** — except that creators care intensely, and fraud detection means some views are removed hours later. **Real platforms display an approximate count immediately and reconcile against a batch-computed exact count later,** which is why a video’s count sometimes goes down.

4 The picture

The whole system:



Transcoding, and why chunking is the whole trick:

SEQUENTIAL

10-minute source
|
one worker
transcodes
6 renditions
|
~1x real-time each
= 60 minutes

CHUNKED

10-minute source
|
split at KEYFRAMES into
200 segments of 3 seconds
|
+-----+-----+----- ...
v v v
worker worker worker (200 of them)
| | |
+-----+-----+
|
stitch + manifest
|
~3 seconds per rendition
of WALL TIME

The CPU total is identical. The LATENCY is 1,200x better.
This is Bashir's second machine: copying does not get faster,
there is just more of it happening at once.

Adaptive bitrate, and who is in charge:

```
manifest.m3u8
1080p → seg0.ts seg1.ts seg2.ts ...
720p → seg0.ts seg1.ts seg2.ts ...
480p → seg0.ts seg1.ts seg2.ts ...
240p → seg0.ts seg1.ts seg2.ts ...

player timeline:
t=0    fetch 480p seg0    (start low: playback begins fast)
t=3    arrived in 0.4 s → network is good → step UP
t=6    fetch 1080p seg2
t=9    arrived in 2.8 s → buffer draining → step DOWN
t=12   fetch 480p seg4    (user walked into a lift)
t=15   arrived in 0.3 s → step up again
```

THE CLIENT DECIDES. The server publishes options.
Every segment is a STATIC FILE, so the CDN needs no video logic.

The storage power law, which is Bashir's shelf:

share of videos	share of views	where they live
top 0.1%	~50%	every region, hot SSD
next 1%	~40%	several regions
next 10%	~9%	one region, standard
bottom ~89%	~1%	archival, minutes to retrieve

the bottom 89% is MOST OF THE BYTES and almost none of the traffic
→ tiering is not an optimisation, it is the only affordable design

AND: a cold video that goes viral must be promoted FAST, or the first thousand viewers wait minutes for it to start.

Why the source file is kept:

original upload (300 MB, whatever codec the phone used)

|
+→ 2160p, 1080p, 720p, 480p, 360p, 240p (~1.2 GB total)

KEEP THE ORIGINAL, because:

- a better codec arrives (H.264 → H.265 → AV1) and everything must be re-encoded, at ~30-50% bandwidth saving
- a rendition is found to be broken
- a new resolution becomes standard

→ the original goes to ARCHIVAL storage, not deleted.
Re-encoding from a rendition loses quality irrecoverably.

5 How it actually works

Resumable upload

```
@app.post("/upload/start")
def start_upload(user_id: int, size: int, content_type: str) → dict:
    video_id = snowflake.next_id()
    upload = s3.create_multipart_upload(Bucket=RAW, Key=f"raw/{video_id}")
    video_store.create(video_id, user_id, status="uploading")
    return {"video_id": video_id, "upload_id": upload["UploadId"],
            "chunk_size": 8 * 1024 * 1024}

@app.post("/upload/chunk-url")
def chunk_url(video_id: int, upload_id: str, part: int) → dict:
    return {"url": s3.generate_presigned_url(
        "upload_part",
        Params={"Bucket": RAW, "Key": f"raw/{video_id}",
                "UploadId": upload_id, "PartNumber": part},
        ExpiresIn=3600)}
```

A pre-signed URL per chunk, so the bytes never touch your servers — and the client can retry any single chunk without restarting the upload.

Eight-megabyte chunks is the usual choice: small enough that losing one is cheap, large enough that the per-request overhead is negligible.

Splitting and dispatching transcode work

```
RENDITIONS = [(2160, 20_000), (1080, 5_000), (720, 2_500),
               (480, 1_000), (360, 600), (240, 400)]

def on_upload_complete(video_id: int) → None:
    duration, keyframes = probe(f"raw/{video_id}")          # ffprobe
    segments = split_at_keyframes(keyframes, target_seconds=3)

    for index, (start, end) in enumerate(segments):
        for height, bitrate in RENDITIONS:
            queue.publish("transcode", {
                "video_id": video_id, "segment": index,
                "start": start, "end": end,
                "height": height, "bitrate": bitrate,
            })
    job_store.expect(video_id, len(segments) * len(RENDITIONS))
```

`split_at_keyframes` is the line that matters. A segment must begin at a keyframe — a frame encoded without reference to any other — **because a chunk starting mid-sequence cannot be decoded on its own**. Splitting at fixed three-second boundaries produces chunks that are silently corrupt.

One queue message per segment per rendition — a ten-minute video is $200 \times 6 = 1,200$ independent jobs. **That is the parallelism.**

The transcode worker

```
def handle_transcode(job: dict) → None:
    key = f"{job['video_id']}/{job['height']}/{job['segment']}.ts"
    if not blob_store.exists(key):                          # idempotent: events are at-least-once
        ffmpeg(
            input=f"raw/{job['video_id']}",
            start=job["start"], end=job["end"],
            height=job["height"], bitrate=job["bitrate"],
            output=key,
        )
    if job_store.complete_one(job["video_id"]) == 0:
        write_manifest(job["video_id"])                     # the LAST job publishes the video
```

The existence check is the idempotency, and it matters more here than anywhere else in the course: **transcoding is the most expensive operation in the system, so doing it twice is real money.**

`complete_one` **returning zero means this was the last job**, which is how a distributed fan-out knows it has finished — an atomic decrement of an expected count, not a scan.

The manifest

```
def write_manifest(video_id: int) → None:
    lines = ["#EXTM3U", "#EXT-X-VERSION:3"]
    for height, bitrate in RENDITIONS:
        if blob_store.exists(f"{video_id}/{height}/0.ts"):
            lines.append(
                f"#EXT-X-STREAM-INF:BANDWIDTH={bitrate * 1000},{br/>                f"RESOLUTION={height * 16 // 9}x{height}"
            )
            lines.append(f"{height}/index.m3u8")
    blob_store.put(f"{video_id}/master.m3u8", "\n".join(lines))
    video_store.mark_ready(video_id)
```

The master manifest is a text file listing the options. That is the entire server side of adaptive streaming — there is no session, no state, no per-viewer connection. **The client reads the list and chooses.**

And publishing renditions as they finish is better than waiting for all of them: the 360p version is ready long before the 2160p one, so **the video can be watchable within seconds of upload** while the high qualities arrive later.

View counting

```
def record_view(video_id: int, user_id: int | None, watched_seconds: float,
                duration: float) → None:
    if watched_seconds < min(30, duration * 0.3):
        return # not a view, by definition
    key = f"view:{video_id}:{user_id or client_fingerprint()}"
    if not redis.set(key, "1", nx=True, ex=86400):
        return # already counted today
    redis.incr(f"count:{video_id}") # fast, approximate
    queue.publish("views", {"video_id": video_id, "at": time.time()})
```

Two paths, deliberately. The Redis counter is what gets displayed — **fast, approximate, and eventually reconciled**. The queue feeds a batch pipeline that computes the authoritative count with fraud filtering.

Which is why counts sometimes go down. The displayed number is a fast estimate; the corrected one arrives hours later.

The `nx` **deduplication window is a product decision:** counting a rewatch as a new view is a choice, and it is worth asking about rather than assuming.

Storage tiering

```
def tier_for(video_id: int) → str:
    views = view_stats.last_30_days(video_id)
    if views > 100_000:
        return "hot"           # replicated to every region, SSD
    if views > 1_000:
        return "warm"          # a few regions, standard object storage
    return "cold"              # archival, minutes to retrieve

def on_cold_video_requested(video_id: int) → None:
    blob_store.restore(video_id, tier="warm", expedited=True)
    video_store.mark_restoring(video_id)      # the player shows a wait
```

Tiering by views, not by age, because a five-year-old video can be popular and a day-old one can be dead.

And the restore path must exist, or a cold video that suddenly trends is unwatchable for the minutes it takes to retrieve — which is exactly when the views are happening.

The real systems

ffmpeg	transcoding, universally
HLS (Apple)	.m3u8 manifests, .ts or fMP4 segments — the common default
DASH	the standard alternative; MPEG, codec-agnostic
H.264 / AVC	near-universal support, the safe baseline
H.265 / AV1	~30-50% fewer bytes for the same quality; AV1 is royalty-free and now widely supported
CMAF	one segment format usable by both HLS and DASH, so you store one copy instead of two
S3 + Glacier	raw originals and the cold tail
CloudFront/Akamai	delivery, and most of the bill

Mentioning CMAF is a genuine signal, because it solves a real and expensive problem: **without it you store HLS segments and DASH segments separately, doubling the storage of every video.**

6 The numbers

Scale.

```
2,000,000,000 monthly users
1,000,000,000 hours watched per day
500 hours of video UPLOADED per minute

uploads: 500 x 60 x 24 = 720,000 hours/day
views:    1e9 hours/day
```

Transcoding, which is the compute problem.

```
~1x real-time per rendition, 6 renditions
720,000 hours/day x 6 = 4,320,000 CPU-hours/day
                      = 180,000 CPU-hours per hour
                      = 180,000 cores running continuously

at ~$0.03/core-hour (spot):
  4,320,000 x $0.03 = ~$130,000/day = ~$4M/month

→ transcoding alone costs more than most systems in this course
  cost in total. Hence hardware encoders and custom silicon:
  Google built the Argos VCU specifically for this.
```

And the chunking win, in latency rather than cost:

```
10-minute video, 6 renditions, sequentially:
  60 minutes of wall time before it is watchable

chunked into 200 x 3-second segments, 1,200 jobs, 200 workers:
  ~3 seconds per job, 6 rounds = ~20 seconds of wall time

180x faster to publish. The CPU total is IDENTICAL.
```

Storage.

```
720,000 hours/day uploaded

per hour of video:
  original      ~2 GB
  2160p         ~9 GB
  1080p         ~2.3 GB
  720p          ~1.1 GB
  480p          ~0.45 GB
  360p          ~0.27 GB
  240p          ~0.18 GB
  -----
  total         ~15.3 GB per hour of video

720,000 x 15.3 GB = ~11 PB/day
                  = ~4 EB/year
```

Four exabytes a year, and it is worth saying that number out loud because it is where tiering stops being optional.

untiered, all on standard storage at \$0.023/GB/month:
4 EB = 4,000,000,000 GB x \$0.023 = ~\$92,000,000/month

tiered by the power law:

hot	1%	40 PB	x \$0.023	= ~\$920,000
warm	10%	400 PB	x \$0.0125	= ~\$5,000,000
cold	89%	3,560 PB	x \$0.004	= ~\$14,240,000

				~\$20,000,000/month

→ 4.6x cheaper, and it is still the second-largest line on the bill.

Delivery, which is the largest.

1,000,000,000 hours watched/day
average bitrate served: ~2 Mbps (most views are 480p-720p on phones)

1e9 hours x 3,600 s x 2 Mbps / 8 = 900,000,000 GB/day
= 900 PB/day of egress

at commercial CDN rates of \$0.01/GB:
900,000,000 x \$0.01 = \$9,000,000/DAY = ~\$270M/month

→ which is why YouTube does not use a commercial CDN.
Google runs its own edge network and peers directly with ISPs,
and places caching servers INSIDE ISP networks.
That takes the marginal cost close to the cost of the hardware.

That last point is the best thing to say in this interview: at this scale you stop buying delivery and start building it, and the break-even is somewhere in the tens of petabytes a month.

The adaptive-bitrate saving:

if every view were served at 1080p (5 Mbps) instead of an adaptive average of 2 Mbps:

900 PB/day → 2,250 PB/day
= 2.5x the largest line on the bill

→ adaptive bitrate is not a quality feature. It is a cost control that happens to also improve the experience.

View counting.

```
1e9 hours/day, average watch ~10 minutes → ~6,000,000,000 views/day
                                             = ~70,000 views/second
                                             peak ~200,000/second
```

exact counting with a transaction per view: not viable

Redis INCR: ~100,000 ops/s per instance → 2-3 instances, sharded
by video id

+ a batch pipeline for the authoritative, fraud-filtered count

Latency budget for playback:

```
manifest fetch (small text, from CDN edge)    ~30 ms
first segment (3 s at 480p ~ 400 KB)         ~200 ms
decode + first frame                          ~100 ms
-----
                                             ~330 ms to first frame
```

starting at a LOW quality is what makes this possible.

Starting at 1080p would be ~2 MB and ~1.5 s.

The full bill, ranked:

```
delivery (own network, at cost)    ~$50,000,000/month
storage (tiered)                  ~$20,000,000/month
transcoding                       ~$4,000,000/month
everything else                    ~$5,000,000/month
-----
                                   ~$79,000,000/month
```

→ per hour watched: ~\$0.0026. Which is roughly what an ad pays.
The whole business is that arithmetic.

7 The trade-offs

Pre-generating every rendition against transcoding on demand. Pre-generating costs the full compute for every video, including the 89% that are watched almost never — **that is millions of CPU-hours spent on videos nobody will see.** On-demand saves that and makes the first view of any video slow by minutes, which is unacceptable. **The real answer is neither: pre-generate the low and middle renditions for everything, and generate 2160p lazily on first request,** because almost nothing is watched at 4K.

Storing more renditions against serving fewer. Six renditions is 15 GB per hour of video; three would be half that. **Fewer renditions means bigger quality jumps and more rebuffering on variable connections** — the player has coarser options to

fall back to. **The middle ones earn their storage; 2160p usually does not.**

Segment length. Short segments — two seconds — let the player adapt quickly and recover fast from a network drop, **at the cost of more requests, more per-segment overhead, and more files to store.** Long segments — ten seconds — are efficient and adapt sluggishly, so a network drop causes a visible stall. **Four to six seconds is the usual compromise**, and live streaming pushes it shorter because latency matters more than efficiency.

Keeping the original against deleting it. Originals are 2 GB an hour and are never served to anyone. **They are kept because codecs improve** — moving a library from H.264 to AV1 saves thirty to fifty percent of the largest line on the bill, and **re-encoding from a rendition loses quality irrecoverably.** So the original goes to archival storage, and that is one of the clearest cases in this course of paying storage to preserve a future option.

Approximate view counts against exact ones. Approximate is fast, cheap and displayed immediately. **Exact requires a batch pipeline, arrives hours later, and can revise the number downwards** — which creators notice and complain about. **Both are needed**, because the displayed count must be instant and the monetised count must be defensible.

Building the delivery network against buying it. At 900 petabytes a day, a commercial CDN is \$270M a month and building your own edge network with ISP-embedded caches is a fraction of that. **Below roughly tens of petabytes a month, buying is obviously right**; above it, the capital cost of building is repaid in months. **Knowing that a break-even exists — and roughly where — is the point.**

When would I not build this? Almost always. Mux, Cloudflare Stream and AWS Elemental do upload, transcoding, storage and delivery, and **below a few petabytes a month they are cheaper than the team.** If the product is a handful of videos — a course, a marketing site — it is an object store, a CDN and a one-off transcode. **The full pipeline is justified by volume alone**, and saying so is stronger than another diagram.

8 In the interview

How it gets asked

- “Design YouTube.” or “Design Netflix’s playback.” — usually with no further constraints.
- “How does transcoding work, and how do you make it fast?” — the chunking question.

- *“How does the video keep playing when my connection gets worse?”* — adaptive bitrate.
- *“How much storage do you need?”* — where tiering appears.
- *“How do you count views?”*
- *“What would you leave out?”* — the scoping question.

The first ninety seconds

“The product is huge, so let me scope, size, and then design the three parts that are actually hard.

In: upload, transcoding, adaptive playback, storage, view counts. Out: recommendations, comments, live streaming, monetisation, copyright matching, moderation. Recommendations especially is its own interview.

Sizing, and one number reframes everything. Five hundred hours uploaded per minute is seven hundred and twenty thousand hours a day. **A billion hours watched a day at an average of two megabits is about nine hundred petabytes a day of egress.** That is the system.

Three sub-systems, each hard in a different way.

Upload is Instagram’s, with one addition: it must be resumable. Pre-signed URLs straight to object storage, but chunked with per-chunk acknowledgement — **because a three-hundred-megabyte upload on a mobile connection will drop, and restarting from zero is unacceptable.**

Transcoding is the compute problem, and it is enormous. Six renditions at roughly real-time each means seven hundred and twenty thousand hours a day becomes 4.3 million CPU-hours a day — **about a hundred and eighty thousand cores running continuously**, four million dollars a month even on spot pricing. **That is why Google built custom silicon for it.**

And the key insight is that it parallelises by chunk. Split the source at keyframes into three-second segments and transcode each independently. **A ten-minute video sequentially is an hour before it is watchable; chunked across two hundred workers it is about twenty seconds.** The CPU total is identical — the latency is a hundred and eighty times better.

The keyframe part is not optional: a segment must start at a frame that can be decoded on its own, or the chunk is silently corrupt.

Delivery is adaptive bitrate, and the inversion is the point. The client fetches a manifest listing every rendition and every segment, starts low, measures throughput, and **switches quality between segments.** The client decides; the server just publishes options. **Every segment is a static file**, so the CDN needs no video logic at all — there is no per-viewer connection anywhere.

And storage: four exabytes a year, tiered by popularity. The top one percent of videos gets ninety percent of the views; the bottom eighty-nine percent is most of the bytes and almost none of the traffic. **Tiering is not an optimisation here, it is the only affordable design** — ninety-two million a month untiered against twenty tiered.

Which of the three would you like me to go deep on?“

The follow-ups

“How does the video keep playing when my connection gets worse?“

“Adaptive bitrate streaming, and the design idea is that **the client decides, not the server** — which is the reverse of almost everything else in this course.

The video exists in several renditions, each a different resolution and bitrate, and each is **cut into short segments** of a few seconds. So a ten-minute video at six qualities is about twelve hundred small files.

The client first fetches a manifest — an HLS `.m3u8` or a DASH `.mpd` — which is a text file listing every rendition, its bandwidth, and where its segments are.

Then it plays a loop. Fetch a segment, measure how long it took, compare with how much buffer is left, and **choose the rendition for the next segment**. Arriving fast with a full buffer means step up; arriving slowly with a draining buffer means step down.

It starts low deliberately, because the first frame should appear fast. A three-second 480p segment is about four hundred kilobytes and arrives in a couple of hundred milliseconds; **starting at 1080p would be two megabytes and about one and a half seconds to first frame**. So the player begins low and climbs within a few seconds.

What this buys architecturally is that every segment is a static file. There is no streaming server, no session, no per-viewer connection, no video-aware logic at the edge. **The CDN serves them exactly like images**, which is why it scales to nine hundred petabytes a day.

The segment length is a real trade. Two-second segments adapt quickly and recover fast from a network drop, at the cost of more requests and more files. **Ten-second segments are efficient and sluggish** — a connection that degrades mid-segment causes a visible stall. **Four to six seconds is the usual compromise**, and live streaming goes shorter because latency matters more than efficiency.

And I would add the cost argument, because it is the stronger one. If every view were served at 1080p instead of an adaptive average of about two megabits, **egress would be two and a half times higher** — which at this scale is over a hundred million dollars a month. **Adaptive bitrate is a cost control that also happens to improve the experience.**“

“Transcoding takes an hour per video. How do you make that acceptable?”

“You do not make it faster. **You make more of it happen at once**, and the enabling trick is chunking.

The compute is irreducible. Transcoding runs at roughly real-time per output rendition, so a ten-minute video with six renditions is about an hour of CPU. **That does not change** — codecs are what they are.

What changes is the wall clock. Split the source into segments of about three seconds and transcode each one independently, on a different machine. **A ten-minute video is two hundred segments times six renditions — twelve hundred completely independent jobs.** Across two hundred workers, that is about twenty seconds of wall time instead of an hour. **A hundred and eighty times faster to publish, with identical total CPU.**

Two things make it work and both are easy to get wrong.

Segments must be cut at keyframes. Most video frames are encoded as differences from earlier frames, so a chunk starting mid-sequence has nothing to decode against. **Cutting at fixed three-second boundaries produces chunks that are silently corrupt** — they encode without error and play as garbage.

And the outputs must be stitched with consistent timing, or the player stutters at every boundary. In practice that means fixing the encoder parameters up front so every worker produces compatible output.

Three more things I would build in.

Idempotency. Queue events are at-least-once, and **transcoding is the most expensive operation in the system** — doing a job twice is real money. Check whether the output already exists before starting.

Progressive publishing. The 360p rendition finishes long before the 2160p one. **Publish each as it is ready**, so the video is watchable within seconds and the high qualities appear later. Waiting for all six wastes the parallelism you just built.

And lazy 2160p. Almost nothing is watched at 4K. **Generating it eagerly for every upload spends compute on videos nobody will watch at all**, let alone at that quality. I would generate the low and middle renditions always and 4K on first request, accepting a slow first 4K view.

On cost: 4.3 million CPU-hours a day is about four million dollars a month even on spot instances, which is why the platforms at this scale build custom encoding hardware. Google’s Argos VCU exists for exactly this line item.”

“How do you store this much video?”

“Four exabytes a year, and the answer is that **the access pattern is so skewed that tiering is the design rather than an optimisation.**

The arithmetic first. Seven hundred and twenty thousand hours uploaded a day, and each hour of video is about fifteen gigabytes once you count the original plus six renditions. **That is eleven petabytes a day, four exabytes a year.**

On standard object storage that is ninety-two million dollars a month, and it grows every month.

The power law rescues it. The top one percent of videos gets something like ninety percent of the views; the bottom eighty-nine percent is watched almost never. **And crucially, that bottom tier is most of the bytes.**

So: tier by popularity, not by age. Hot videos on fast storage replicated to every region. Warm ones in fewer regions on standard storage. **Cold ones in archival storage where retrieval takes minutes.** That takes ninety-two million to about twenty — **still the second-largest line on the bill, but survivable.**

By popularity and not by age matters, because a five-year-old music video can be extremely hot and a day-old upload can be dead. Age is a bad proxy.

And the restore path must exist and be fast, which is the part people forget. A cold video that suddenly trends is **unwatchable for the minutes archival retrieval takes — which is exactly when the views are happening.** So popularity monitoring needs to trigger expedited promotion, and the player needs to show something sensible while it happens.

One decision that looks wasteful and is not: keep the original. It is two gigabytes an hour and is never served to anyone. **It is kept because codecs improve** — moving a library from H.264 to AV1 saves thirty to fifty percent of the delivery bill, which is the largest line — **and re-encoding from a rendition loses quality irrecoverably.** So originals go to the cheapest archival tier and stay there.

And I would mention CMAF, because it solves an expensive and unglamorous problem: **without a common segment format you store HLS and DASH segments separately and double the storage of every video.**“

The model answer

“Design a video platform: five hundred hours uploaded per minute, a billion hours watched per day, global audience. Tell me what it costs.”

“The cost framing is the right one for video, because **the bill is the design constraint** more directly than in any other system. Let me scope, then size, then design against the numbers.

In: upload, transcode, adaptive playback, storage, view counts. Out: recommendations, comments, live, monetisation, Content ID, moderation.

Upload: resumable chunked upload with pre-signed URLs, direct to object storage. Eight-megabyte chunks so a dropped connection retries one chunk rather than three hundred megabytes. **No video bytes touch my servers.**

Transcoding: split at keyframes into three-second segments, fan out one job per segment per rendition. A ten-minute video is twelve hundred independent jobs; across two hundred workers that is twenty seconds of wall time instead of an hour. **Idempotent, because at-least-once delivery on the most expensive operation in the system is real money. Progressive publishing,** so 360p is live in seconds. **And 2160p generated lazily,** because almost nothing is watched at 4K and eager generation spends compute on the long tail.

Cost: 720,000 hours a day times six renditions at real-time is 4.3 million CPU-hours a day — about \$4 million a month on spot. At that number, custom encoding hardware pays for itself, which is why it exists.

Delivery: adaptive bitrate over static segments. The client fetches a manifest, starts low for a fast first frame, and switches quality between segments based on measured throughput. **Every segment is a static file, so the edge needs no video logic and there is no per-viewer connection anywhere.**

Cost: a billion hours a day at an adaptive average of two megabits is nine hundred petabytes a day. On a commercial CDN at a cent a gigabyte that is nine million dollars a day — \$270 million a month, which is not a business.

So at this scale you stop buying delivery and build it: your own edge network, direct peering with ISPs, and caching servers placed inside ISP networks. **That takes the marginal cost close to hardware and power.** The break-even is somewhere in the tens of petabytes a month, and **knowing that a break-even exists is the point** — below it, building your own is a mistake.

And adaptive bitrate is doing real work on this line. Serving everything at 1080p would be $2.5\times$ the egress — **it is a cost control that happens to improve the experience.**

Storage: fifteen gigabytes per hour of video including the original, so eleven petabytes a day and four exabytes a year. Untiered that is \$92 million a month. **Tiered by popularity — not age — it is about \$20 million.** Hot in every region, warm in a few, cold in archival with an expedited restore path, **because a cold video that trends is unwatchable for the minutes retrieval takes, which is exactly when the views arrive.**

Originals kept in archival, never served, because codec migrations save thirty to fifty percent of the largest line and re-encoding from a rendition loses quality permanently.

View counts: approximate and immediate from a sharded counter, authoritative and delayed from a batch pipeline with fraud filtering. Both are needed — the displayed number must be instant and the monetised number must be defensible — **and that is why counts sometimes go down.**

The ranked bill: delivery ~\$50M at cost, storage ~\$20M, transcoding ~\$4M, everything else ~\$5M. About \$79 million a month, **which is roughly a quarter of a cent per hour watched** — and that number is the whole business, because it has to sit under what an hour of viewing earns.

The thing I would flag as the biggest risk is the long tail. Eighty-nine percent of videos are watched almost never and consume most of the storage and most of the transcoding compute. **Every lever that helps — lazy high renditions, aggressive cold tiering, fewer renditions for unpopular uploads — is about spending less on content nobody watches,** and that is where the engineering effort actually pays.”

9 Recall card

Scope out loud (in: upload, transcode, playback, storage, views; out: recommendations, comments, live, monetisation, Content ID) — the product is too big to design whole. **Video is $\sim 7,000 \times$ a photo and $\sim 10^6 \times$ a tweet,** and the bill is the design constraint.

Transcoding is $\sim 1 \times$ real-time per rendition — $720,000 \text{ hours/day} \times 6 = 4.3\text{M CPU-hours/day}$, $\sim \$4\text{M/month}$, which is why custom encoding silicon exists. **It parallelises by chunk: split at KEYFRAMES** (fixed-time splits are silently corrupt), one job per segment per rendition — **an hour of wall time becomes ~ 20 seconds with identical CPU.** Make it **idempotent** (the most expensive op in the system), **publish renditions progressively,** and **generate 2160p lazily.**

Adaptive bitrate inverts the usual relationship: the CLIENT decides. It fetches a manifest, starts low for a ~ 330 ms first frame, measures throughput and switches quality between segments. **Every segment is a static file**, so the CDN needs no video logic and there is no per-viewer connection. **Segment length trades adaptation speed against overhead — 4–6 s is the compromise.**

Storage is ~ 15 GB per hour of video $\rightarrow 11$ PB/day $\rightarrow 4$ EB/year. Untiered \$92M/month; **tiered by POPULARITY, not age** (top 1% gets $\sim 90\%$ of views; the bottom 89% is most of the bytes) $\rightarrow \sim \$20$ M. **The expedited restore path is mandatory** — a cold video that trends is unwatchable exactly when the views arrive. **Keep the original** for future codec migrations (AV1 saves 30–50% of the biggest line); re-encoding a rendition loses quality permanently.

Delivery is the largest line: 900 PB/day. At commercial CDN rates that is \$270M/month, so at this scale you build your own edge network and embed caches in ISPs. **Adaptive bitrate itself is a $2.5\times$ cost control.** **View counts are two systems** — instant approximate for display, batch authoritative with fraud filtering for money — which is why counts sometimes go down.

DSA topic: Interval DP **System design topic:** Design YouTube

Code these, in this order

One rule for the whole set: **write the three loops — length, start, split — before you write anything inside them.** Every problem here has the same skeleton, and the only real decisions are what `k` means and what joining costs.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Matrix Chain Multiplication	classic — write it yourself	The skeleton, and the three dimension indices.
2	Burst Balloons	LeetCode 312 (Hard)	The last-not-first inversion, and the inclusive <code>k</code> .
3	Minimum Score Triangulation of Polygon	LeetCode 1039 (Medium)	The same shape once more, so it stops feeling special.
4	Stone Game	LeetCode 877 (Medium)	Two players, and the minus sign that handles both.
5	Palindrome Partitioning II	LeetCode 132 (Hard)	An interval table feeding a one-dimensional DP.
6	Remove Boxes	LeetCode 546 (Hard)	Where two dimensions are not enough.

On problem 1, fill it row by row and watch it lie

Write the natural `for i, for j` loops. Run on `dims = [10, 100, 5, 50]` and record the answer. Then run on `[40, 20, 30, 10, 30]` and record that against the correct 26,000.

One passes and one does not. Say which direction the error goes, and why that direction is harder to notice.

On problem 1, get the dimensions wrong

Use `dims[i] * dims[k] * dims[j]` instead of `dims[i] * dims[k+1] * dims[j+1]`. Record what happens. **Say why there is no error**, then derive the correct three indices out loud from the `dims` convention.

On problem 2, try thinking “first”

Write the recurrence for “burst `k` first” and try to justify it in one sentence. **Find the sentence you cannot finish.** Then write the “last” version and say what changed about the two halves.

On problem 2, use `range(i, j)` for `k`

Run it on `[3, 1, 5, 8]` and record the answer against 167. **Say what `k` means in this problem and why the range must be inclusive**, and how that differs from problem 1.

On problem 2, remove the padding

Run without the sentinel 1s and record what happens at `i = 0`. **Say why Python does not raise**, and what value it actually used.

On problem 2, write it memoised

Write the `lru_cache` version. Confirm it agrees. Time both at `n = 200`. **Say what you paid and what you bought.**

On problem 4, remove the minus sign

Write `max(piles[i] + dp[i+1][j], piles[j] + dp[i][j-1])`. Run on `[3, 7, 2, 3]` and record it. **Say what the minus sign is doing**, in one sentence about whose turn it is.

On problem 6, find where two dimensions fail

Try `dp[i][j]` alone. Say what information about the boxes to the left of `i` you need and cannot express. **Then say what the third dimension is.**

Then the skeleton drill

Write the three-loop skeleton from memory. Then instantiate it for problems 1, 2 and 3 without looking at your earlier code. **Target two minutes each once the skeleton is down.**

The shape drill

1. State the interval DP state in one sentence.
2. Say what distinguishes it from LCS’s two-index state.
3. Give the general recurrence.
4. Say what the three loops are and their order.
5. Give the time and space.

The fill-order drill

1. Say which two cells the recurrence reads and why they are shorter.
2. Say what the natural order reads instead, and what a zero means there.
3. Say which direction the error goes and why that is worse.
4. Give the second correct order.
5. Say why you would write it memoised instead.

The inversion drill

1. Say what goes wrong with “burst first”, precisely.
2. Say what “burst last” fixes, and about which values.
3. Give the recurrence.
4. Name two other problems that use the same inversion.
5. Say what phrasing in a problem statement should make you try it.

The k-meaning drill

1. Say what `k` is in matrix chain and what the loop bound is.
2. Say what `k` is in burst balloons and what the loop bound is.
3. Say what goes wrong if you swap them.
4. Say why there is no error either way.

The space drill

1. Say why there is no one-row collapse here.
2. Say what you can save instead, and by how much.
3. Say what reconstruction costs.
4. Say what actually limits you first, time or space, with numbers.

The break-it drill

Trigger each and record the exact output or error:

1. Row-by-row fill on a five-matrix chain.
2. `dims[i] * dims[k] * dims[j]`.
3. `range(i, j)` for `k` in burst balloons.
4. No padding in burst balloons.
5. `dp[i][j]` initialised to 0 while minimising.
6. `n = 5,000` with the cubic loops.
7. Stone game without the minus sign.

Six of the seven give no error at all. Name them.

The scoping drill

1. Give your in and out lists.
2. Say why recommendations in particular is out.
3. Say which of the three hard sub-systems you would go deep on first.

The upload drill

1. Say what is the same as a photo pipeline and what is new.
2. Say why resumable uploads are required here and not there.
3. Give the chunk size and justify it in both directions.

The transcoding drill

1. Give the compute rule and the daily CPU-hours.
2. Explain chunking and give the wall-time comparison.
3. Say why segments must be cut at keyframes, and what happens otherwise.
4. Say why the worker must be idempotent, and why it matters more here.
5. Give two other optimisations and what each avoids.

The streaming drill

1. Say who decides the quality, and why that is unusual.
2. Describe the player's loop in three steps.
3. Say why it starts low, with the latency numbers.
4. Say what a segment is, architecturally, and why that matters for the CDN.
5. Give the segment-length trade in both directions.
6. Give the cost argument for adaptive bitrate.

The storage drill

1. Compute bytes per hour of video and the annual total.
2. Give the untiered and tiered monthly costs.
3. Say what you tier by, and why not age.
4. Say what breaks without a fast restore path.
5. Say why the original is kept, and what re-encoding a rendition costs.
6. Say what CMAF saves.

The delivery drill

1. Compute daily egress from the watch hours.
2. Give the commercial CDN bill.
3. Say what you do instead at this scale, and roughly where the break-even is.
4. Say what serving everything at 1080p would cost.

The counting drill

1. Say what a view is, and why that definition matters.
 2. Give the two counting paths and what each is for.
 3. Say why counts sometimes go down.
 4. Compute views per second at this scale.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the cheapest way to multiply a chain of matrices.* The range state, the recurrence over splits, the three loops with why length is outermost, the join cost derived from the convention, the complexity, and why there is no space collapse.
 2. *Burst balloons.* Why “first” fails, why “last” works, the recurrence, the padding, the inclusive `k`, and why you would write it memoised.
 3. *Design YouTube, and tell me what it costs.* The scope, chunked transcoding with the keyframe constraint, adaptive bitrate with the client deciding, storage tiered by popularity, and the ranked bill.
-

Before you move on

- [] I can state the interval DP state and its recurrence.
- [] I write the three loops with length outermost, every time.
- [] I can say why the natural order fails and which direction it errs.
- [] I know a small test can pass with the wrong fill order.
- [] I can give the matrix chain join cost and derive its indices.
- [] I can reconstruct the parenthesisation with a split table.
- [] I can say why “burst first” leaves the subproblems entangled.
- [] I can say what “burst last” fixes, about which values.

- [] I know `k` is a split in one problem and a member in the other.
- [] I know the loop bounds differ, and that neither errors.
- [] I pad burst balloons, and know what negative indexing would do.
- [] I can write the memoised version and say why I prefer it.
- [] I know there is no space collapse, and why.
- [] I know what limits me first, time or space, with numbers.
- [] I know the constraint on `n` signals a cubic solution.
- [] I can do stone game and explain the minus sign.
- [] I can name three other problems with this shape.
- [] I can scope YouTube out loud, in and out.
- [] I know why uploads must be resumable here.
- [] I can give the transcoding compute rule and the daily total.
- [] I can explain chunking and give the wall-time comparison.
- [] I know why segments are cut at keyframes.
- [] I know why the transcode worker must be idempotent.
- [] I can describe the player's adaptive loop.
- [] I know why it starts at a low quality, with numbers.
- [] I know a segment is a static file and why that matters.
- [] I can give the segment-length trade.
- [] I can compute storage per year and both tiering costs.
- [] I know why we tier by popularity and why the restore path matters.
- [] I can give the delivery bill and say what you build instead.
- [] I answered all three questions above out loud.

DP on trees

DATA STRUCTURES AND ALGORITHMS

DP on trees

SYSTEM DESIGN

Design Netflix

1 What this is, and why they ask it

Tree DP is dynamic programming where the subproblems are subtrees, and the answer for a node is computed from the answers for its children.

Every DP so far has had a table you filled with loops. **Here there is no table and there are no loops** — the recursion is the fill order, because a post-order traversal visits every child before its parent, which is exactly the dependency direction. **You get the hardest part of DP for free, from the shape of the data.**

They ask it because **it is where two topics you already know join up**. You have done trees since day 100 and DP since day 143, and tree DP is the point at which “solve the subproblems first” and “recurse into the children first” turn out to be the same instruction. **A candidate who sees that connection explains tree DP in one sentence; a candidate who does not treats every tree problem as a new puzzle.**

The other reason is a specific, recurring difficulty: **the answer at a node is often not the value you return to its parent**. In the diameter problem, the longest path through a node uses two of its children, but the value the parent needs is a path going down through only one. **Two different quantities, computed in the same function** — and confusing them is the defining bug of the topic.

By the end of this lesson you can write the post-order pattern, handle the include/exclude state, compute diameters and path sums, explain the return-versus-record distinction, and say what recursion depth costs.

2 The story

The land had been divided four times in eighty years and the family that farmed it had a problem that took an afternoon and a very patient uncle to solve.

Every plot had been split among the sons, and their plots split again, so the whole holding was a shape like a tree — the original field at the top, and below it the divisions, and below those the divisions of those.

And now the government was offering a payment for planting a particular kind of tree, **with one condition: you could not take the payment for a plot and also for the plot it had been split off from**. Parent or child, not both.

The family wanted the largest total payment.

The eldest brother started at the top, on the original field, because it was the biggest and it seemed obvious.

The uncle stopped him.

“You cannot know about the top until you know about everything under it.”

So they started at the bottom instead — **the smallest plots, the ones with nothing below them.**

For each of those the answer was easy. **Take the payment. There is nothing under you to lose.**

Then one level up. And here the uncle made them write down two numbers for every plot instead of one, which the brother thought was a waste of paper until he saw why.

“For each plot, write down the best total if you take this plot’s payment, and the best total if you do not.”

Because if you take it, you cannot take any of the plots immediately below — **so the total is this payment plus, for each child, the best it can do while not taking its own.**

And if you do not take it, **each child is free to do whatever is best for it** — take or not, whichever is larger.

Two numbers per plot, computed from the two numbers of each of its children. They went up the tree that way, level by level, and by evening they had the answer for the original field at the top.

The brother’s question, at the end, was the good one.

“Why two numbers? Why not just the best?”

“Because your father cannot use your best,” the uncle said. **“He can only use your best-without-you. If you only wrote down one number he would not know which kind it was.”**

3 The idea in plain English

The uncle’s two numbers are the state, and his last sentence is the whole lesson.

Start with the shape, because it is what makes tree DP easy.

A subtree is a subproblem. The answer for a node depends on the answers for its children, and nothing else — **so the dependency graph is the tree itself, pointing upwards.**

And post-order traversal visits every child before its parent. So:

```
def solve(node):
    if node is None:
        return base_case
    left = solve(node.left)           # children first
    right = solve(node.right)
    return combine(node, left, right) # then this node
```

That is the entire pattern. There is no table to allocate and **no fill order to get wrong**, because the recursion enforces it. After the last four days, that is worth pausing on: **the hardest mechanical part of DP disappears when the data is a tree.**

Now the state, and it is usually not a single number.

House robber on a tree — the uncle’s problem — is the canonical example. You cannot take a node and its child. **The state per node is a pair:**

```
(best if I TAKE this node, best if I SKIP this node)
```

If you take it: `value + sum over children of (child's skip value)`. **If you skip it:** `sum over children of max(child's take, child's skip)`.

The answer at the root is `max(take, skip)`.

Why two numbers rather than one is the uncle’s point: a parent that takes itself needs its child’s *skip* value specifically, and a single “best” would not say which case produced it. **The extra dimension exists because the parent’s options depend on the child’s choice, not only on its score.**

That is exactly the mode dimension from [day 157](#), in a different shape.

Now the second idea, which is where the bugs live.

In many tree problems, the answer you record is different from the value you return.

Diameter of a tree — the longest path between any two nodes — is the clearest case. Consider a node:

- **The longest path *through* this node** goes down one side, up through the node, and down the other. That is `left_depth + right_depth`. **That is a candidate for the answer.**

- **The value the parent needs** is the longest path going *down* from this node, which can use only one side: `1 + max(left_depth, right_depth)`. **A path cannot go down the left, come back up, and then continue up to the parent** — it would visit this node twice.

So the function returns one thing and records another.

```
best = 0                                # recorded, in an outer variable

def depth(node):
    nonlocal best
    if node is None:
        return 0
    left, right = depth(node.left), depth(node.right)
    best = max(best, left + right)        # RECORD: path THROUGH here
    return 1 + max(left, right)          # RETURN: path DOWN from here
```

Returning `left + right` instead is the defining bug of tree DP. It compiles, it runs, and it produces a number that is too large — because it lets a path fork at a node and still continue upwards, which is not a path.

Say the two sentences out loud as you write them: “I record the best path through this node; I return the best path down from this node.”

Maximum path sum — LeetCode 124 — is the same structure with two extra details. **Negative values mean a branch can be worth skipping**, so clamp each child’s contribution at zero: `max(0, child)`. **And the path need not touch the root**, which is why the answer is recorded rather than returned.

Now the third idea: rerooting, which is the step up.

Some problems ask for an answer *at every node*, not just at the root. **“For each node, the sum of distances to all other nodes.”**

The naive approach runs the whole traversal from each node: `O(n2)`. For a tree with 30,000 nodes that is 900 million operations.

The rerooting technique does it in `O(n)` with two passes. The first is an ordinary post-order pass computing, for each node, the answer *within its own subtree*. **The second is a pre-order pass pushing information down** — at each child, the answer for the whole tree can be computed from the parent’s answer plus what changes when you move the root across that one edge.

The insight is that moving the root by one edge changes the answer in a way you can compute in constant time, so you never recompute from scratch.

It is genuinely harder than everything else here, and the right level of preparation is to know it exists, know the two-pass shape, and be able to say what the second pass does.

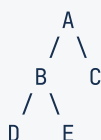
Finally: the practical thing that bites in Python.

Recursion depth. A tree with 10,000 nodes in a line — a degenerate tree, which is just a linked list — is 10,000 stack frames. **Python's default limit is 1,000**, so it raises `RecursionError` on inputs a real test suite will contain.

Two answers. `sys.setrecursionlimit(200000)` is the quick one, and it can seg-fault because the C stack is also finite. **The robust answer is an explicit stack**, which is more code and always works. **Say which you are doing and why**, because an interviewer with a 10^5 -node constraint is testing exactly this.

4 The picture

The post-order pattern, and why it is the fill order:



post-order visit sequence: D, E, B, C, A

D and E finish before B needs them.

B and C finish before A needs them.

→ the traversal order IS the dependency order

→ no table, no loops, no fill order to get wrong

Compare interval DP, where getting the fill order right was the whole difficulty. Here the data structure does it.

The uncle's two numbers, on a small tree:

values on the nodes:



leaf 3 (left): take = 3, skip = 0

leaf 1 (right): take = 1, skip = 0

node 2: take = 2 + (child's SKIP = 0) = 2
skip = max(child take, child skip) = 3 = 3
→ (2, 3)

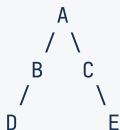
node 3 (right): take = 3 + 0 = 3
skip = max(1, 0) = 1
→ (3, 1)

root 3: take = 3 + left.skip + right.skip = 3 + 3 + 1 = 7
skip = max(2,3) + max(3,1) = 3 + 3 = 6
→ answer max(7, 6) = 7

THE PARENT NEEDS THE CHILD'S "SKIP" SPECIFICALLY.

A single "best" number would not say which case produced it.

The record-versus-return distinction, which is the defining bug:



at node A:

PATH THROUGH A: D - B - A - C - E length = left_depth + right_depth
^^^^^^^^^^^^^^^^^^^^ ← RECORD this as a candidate

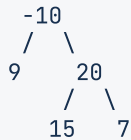
PATH DOWN FROM A (what A's parent can use):
A - B - D = 1 + max(left, right)
^^^^^^^^ ← RETURN this

WHY they differ: a path that goes down the left, back up through A, and down the right CANNOT then continue up to A's parent – it would pass through A twice.

Returning left + right instead:
no error, and every ancestor's number is too large.

Maximum path sum, and where the clamp goes:

values:



at node 20: left = 15, right = 7
record: $20 + 15 + 7 = 42$ ← the answer
return: $20 + \max(15, 7) = 35$

at node -10: left = 9, right = 35
record: $-10 + 9 + 35 = 34$
return: $-10 + \max(9, 35) = 25$

answer = $\max(42, 34, \dots) = 42$

THE CLAMP: if a child's contribution is NEGATIVE, use 0 instead – the path simply does not go that way.

node with value 5, left child contributing -3:

without clamp: $5 + (-3) = 2$

with clamp: $5 + 0 = 5$ ← correct: skip that branch

Rerooting, in outline:

question: for EVERY node, the sum of distances to all others

NAIVE: run a traversal from each node $O(n^2)$

$n = 30,000 \rightarrow 900,000,000$ operations

REROOTING, two passes, $O(n)$:

PASS 1 (post-order, upwards):

count[v] = nodes in v's subtree

within[v] = sum of distances from v to its own subtree

PASS 2 (pre-order, downwards):

answer[root] = within[root]

moving the root from u to its child v:

- every node IN v's subtree gets 1 CLOSER (count[v] nodes)

- every node NOT in v's subtree gets 1 FURTHER ($n - \text{count}[v]$)

answer[v] = answer[u] - count[v] + (n - count[v])

ONE SUBTRACTION per edge. That is the whole technique.

5 The code, built step by step

The pattern, which every tree DP shares

```
def tree_dp(node):
    if node is None:
        return base_case                # the empty subtree
    left = tree_dp(node.left)           # children FIRST
    right = tree_dp(node.right)
    return combine(node.value, left, right) # then this node
```

Write this before anything else. The only decisions are what the base case is and what `combine` does, and **the fill order is handled by the call stack.**

House robber on a tree

```
def rob_tree(root) → int:
    def solve(node) → tuple[int, int]:
        """Returns (best if we TAKE this node, best if we SKIP it)."""
        if node is None:
            return 0, 0
        left_take, left_skip = solve(node.left)
        right_take, right_skip = solve(node.right)

        take = node.value + left_skip + right_skip    # children must skip
        skip = max(left_take, left_skip) + max(right_take, right_skip)
        return take, skip

    return max(solve(root))
```

Returning a tuple is the state. The parent's `take` branch reads only the children's `skip` values — **which is precisely why one number would not be enough.**

`max(solve(root))` at the end, because the root itself may or may not be taken.

Diameter, with the record-versus-return split

```
def diameter(root) → int:
    best = 0

    def depth(node) → int:
        nonlocal best
        if node is None:
            return 0
        left = depth(node.left)
        right = depth(node.right)
        best = max(best, left + right)    # RECORD: the path THROUGH here
        return 1 + max(left, right)      # RETURN: the path DOWN from here

    depth(root)
    return best
```

Two different quantities, three lines apart, and the comments are not decoration — **they are what stops you returning the wrong one.**

`nonlocal best` **rather than returning a pair** is a style choice; the pair version is equally correct and slightly more verbose.

Maximum path sum

```
def max_path_sum(root) → int:
    best = float("-inf")

    def gain(node) → int:
        nonlocal best
        if node is None:
            return 0
        left = max(0, gain(node.left))      # CLAMP: skip a negative branch
        right = max(0, gain(node.right))
        best = max(best, node.value + left + right)    # record
        return node.value + max(left, right)          # return

    gain(root)
    return int(best)
```

The `max(0, ...)` **is the one line that handles negative values**, and forgetting it makes the answer too small on any tree with negatives — **because the path is forced through branches it should avoid.**

`best` **starts at negative infinity**, not zero: **a tree of all-negative values has a negative answer**, and zero would win every comparison.

Counting and summing over subtrees

```
def subtree_sums(root) → dict:
    sums = {}

    def total(node) → int:
        if node is None:
            return 0
        s = node.value + total(node.left) + total(node.right)
        sums[node] = s
        return s

    total(root)
    return sums
```

The simplest possible tree DP, and worth writing once because so many problems reduce to it — most frequently-asked-subtree-sum problems are this plus a `Counter`.

Rerooting: sum of distances to every node

```
def sum_of_distances(n: int, edges: list[tuple[int, int]]) → list[int]:
    graph = [[] for _ in range(n)]
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)

    count = [1] * n          # nodes in each subtree
    within = [0] * n         # distance sum within each subtree

    def pass_one(node: int, parent: int) → None:
        for child in graph[node]:
            if child ≠ parent:
                pass_one(child, node)
                count[node] += count[child]
                within[node] += within[child] + count[child]

    def pass_two(node: int, parent: int, answer: list[int]) → None:
        for child in graph[node]:
            if child ≠ parent:
                # move the root across one edge:
                # count[child] nodes get closer, n - count[child] get further
                answer[child] = answer[node] - count[child] + (n - count[child])
                pass_two(child, node, answer)

    pass_one(0, -1)
    answer = [0] * n
    answer[0] = within[0]
    pass_two(0, -1, answer)
    return answer
```

`within[node] += within[child] + count[child]` is the first pass's key line: every node in the child's subtree is one edge further from `node` than it was from `child`, and there are `count[child]` of them.

`answer[child] = answer[node] - count[child] + (n - count[child])` is the whole rerooting insight in one line — **and it is why this is $O(n)$ and not $O(n^2)$** .

The iterative version, for deep trees

```
def diameter_iterative(root) → int:
    if root is None:
        return 0
    best = 0
    depths = {None: 0}
    stack = [(root, False)]          # (node, children_processed)
    while stack:
        node, processed = stack.pop()
        if processed:
            left, right = depths[node.left], depths[node.right]
            best = max(best, left + right)
            depths[node] = 1 + max(left, right)
        else:
            stack.append((node, True))    # revisit AFTER the children
            if node.right:
                stack.append((node.right, False))
            if node.left:
                stack.append((node.left, False))
    return best
```

The `(node, processed)` flag is how you get post-order from an explicit stack: push the node back marked as processed, then push its children, so the children are popped first.

This is what you write when the tree can be 100,000 deep, and knowing the pattern is worth more than `setrecursionlimit`.

The complete solution

```
"""DP on trees: the pattern, the state, and the record/return distinction."""

from dataclasses import dataclass

@dataclass(eq=False)                                # eq=False keeps Node hashable
class Node:
    value: int
    left: "Node | None" = None
    right: "Node | None" = None

def rob_tree(root: Node | None) → int:
    """Cannot take a node and its child. State = (take, skip)."""
    def solve(node: Node | None) → tuple[int, int]:
        if node is None:
            return 0, 0
        left_take, left_skip = solve(node.left)
        right_take, right_skip = solve(node.right)
        take = node.value + left_skip + right_skip      # children must SKIP
        skip = max(left_take, left_skip) + max(right_take, right_skip)
        return take, skip

    return max(solve(root))

def diameter(root: Node | None) → int:
    """Longest path between any two nodes, counted in edges."""
    best = 0

    def depth(node: Node | None) → int:
        nonlocal best
        if node is None:
            return 0
        left, right = depth(node.left), depth(node.right)
        best = max(best, left + right)                  # RECORD: path THROUGH this node
        return 1 + max(left, right)                    # RETURN: path DOWN from it

    depth(root)
    return best

def max_path_sum(root: Node | None) → int:
    """Any node to any node. Negative branches are clamped away."""
    best = float("-inf")

    def gain(node: Node | None) → int:
        nonlocal best
        if node is None:
            return 0
        left = max(0, gain(node.left))                  # a negative branch is skipped
        right = max(0, gain(node.right))
        best = max(best, node.value + left + right)
        return node.value + max(left, right)

    gain(root)
    return int(best)
```

```

def height(root: Node | None) → int:
    if root is None:
        return 0
    return 1 + max(height(root.left), height(root.right))

def count_nodes(root: Node | None) → int:
    if root is None:
        return 0
    return 1 + count_nodes(root.left) + count_nodes(root.right)

def diameter_iterative(root: Node | None) → int:
    """No recursion. For trees deeper than the stack limit."""
    if root is None:
        return 0
    best = 0
    depths: dict = {None: 0}
    stack: list[tuple[Node, bool]] = [(root, False)]
    while stack:
        node, processed = stack.pop()
        if processed:
            left, right = depths[node.left], depths[node.right]
            best = max(best, left + right)
            depths[node] = 1 + max(left, right)
        else:
            stack.append((node, True))
            if node.right:
                stack.append((node.right, False))
            if node.left:
                stack.append((node.left, False))
    return best

def sum_of_distances(n: int, edges: list[tuple[int, int]]) → list[int]:
    """For every node, the sum of distances to all others. Rerooting: 0(n)."""
    graph: list[list[int]] = [[] for _ in range(n)]
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)

    count = [1] * n
    within = [0] * n

    def pass_one(node: int, parent: int) → None:
        for child in graph[node]:
            if child ≠ parent:
                pass_one(child, node)
                count[node] += count[child]
                within[node] += within[child] + count[child]

    def pass_two(node: int, parent: int, answer: list[int]) → None:
        for child in graph[node]:
            if child ≠ parent:
                answer[child] = answer[node] - count[child] + (n - count[child])
                pass_two(child, node, answer)

```

```

pass_one(0, -1)
answer = [0] * n
answer[0] = within[0]
pass_two(0, -1, answer)
return answer

def build_line(depth: int) → Node:
    """A degenerate tree: n nodes in a straight line."""
    root = Node(1)
    current = root
    for _ in range(depth - 1):
        current.left = Node(1)
        current = current.left
    return root

if __name__ == "__main__":
    #
    #       3
    #      /\
    #     2  3
    #      \  \
    #       3  1
    tree = Node(3, Node(2, None, Node(3)), Node(3, None, Node(1)))
    print("rob tree      :", rob_tree(tree))
    print("height       :", height(tree))
    print("node count    :", count_nodes(tree))

    #
    #       1
    #      /\
    #     2  3
    #      /\
    #     4  5
    d = Node(1, Node(2, Node(4), Node(5)), Node(3))
    print("diameter      :", diameter(d))
    print("iterative agrees:", diameter_iterative(d))

    #
    #      -10
    #      /\
    #     9  20
    #      /\  \
    #     15 7
    p = Node(-10, Node(9), Node(20, Node(15), Node(7)))
    print("max path sum   :", max_path_sum(p))
    print("all negative    :", max_path_sum(Node(-3, Node(-2), Node(-5))))

    edges = [(0, 1), (0, 2), (2, 3), (2, 4), (2, 5)]
    print("sum of distances:", sum_of_distances(6, edges))

    line = build_line(2000)
    try:
        print("deep recursive :", diameter(line))
    except RecursionError as e:
        print("deep recursive : RecursionError:", e)
    print("deep iterative :", diameter_iterative(line))

```

Run it and you get:

```

rob tree      : 7
height       : 3
node count   : 5
diameter     : 3
iterative agrees: 3
max path sum  : 42
all negative  : -2
sum of distances: [8, 12, 6, 10, 10, 10]
deep recursive : RecursionError: maximum recursion depth exceeded
deep iterative  : 1999

```

The last two lines are the practical lesson, and it is worth running yourself: a two-thousand-node line kills the recursive version and the iterative one handles it. That is not a hypothetical — a linked-list shaped tree is a standard test case.

And `all negative : -2` shows the clamp working correctly: the best path is the single node `-2`, not the sum of anything.

6 What it costs

Time: every node is visited once.

```

one post-order traversal      0(n)
constant work per node      0(1)
→ 0(n) total

n = 100,000 nodes: ~100,000 operations. Milliseconds.

```

That is dramatically better than the table DPs of the last week, and worth saying: tree DP is linear because the tree already encodes which subproblems exist. There are exactly `n` subtrees, so there are exactly `n` subproblems.

Space: the recursion stack.

```

balanced tree of n nodes:    depth ~ log2(n)
                             n = 1,000,000 → depth ~20
                             → negligible

degenerate tree (a line):    depth = n
                             n = 100,000 → 100,000 frames
                             → ~10-50 MB of stack, and Python
                             raises long before that

```

The gap between those two is the whole practical story, and the constraint in the problem tells you which you must survive.

Python's recursion limit, concretely:

```
>>> sys.getrecursionlimit()
1000
```

so ~1,000 nested calls, minus whatever frames are already used.
A line-shaped tree of 2,000 nodes fails.

```
sys.setrecursionlimit(200000):
  works up to the C stack limit, then SEGFAULTS rather than raising
  → not safe for arbitrary depth
```

explicit stack:
 heap-allocated, so bounded only by memory
 → 100,000 frames as tuples: ~10 MB. Fine.

Rerooting, and why it is worth the complexity:

```
naive, one traversal per node:    n x O(n) = O(n^2)
n = 30,000 → 900,000,000 operations ~3 minutes in Python
```

```
rerooting, two traversals:      O(n)
n = 30,000 → 60,000 operations ~0.02 s
```

15,000x faster, and LeetCode 834 caps n at 30,000 specifically
so that the quadratic version does not pass.

The two-pass structure costs:

```
pass 1 (post-order):  O(n), two arrays of size n
pass 2 (pre-order):   O(n), one more array
→ O(n) time, O(n) space, and ~3 arrays instead of 1
```

Comparison with the table DPs this week:

	time	space	fill order
grid DP	$O(\text{rows} \times \text{cols})$	$O(\text{cols})$	loops, easy to get right
interval DP	$O(n^3)$	$O(n^2)$	loops, easy to get WRONG
tree DP	$O(n)$	$O(\text{depth})$	FREE – the recursion does it

Tree DP is the cheapest shape in the family, and the fill order –
which was the whole difficulty of interval DP – costs nothing.

The tuple-returning versions, on cost:

```
returning (take, skip) allocates a tuple per node
n = 100,000 → 100,000 small tuples
~50 bytes each = 5 MB, all garbage-collected as you unwind

negligible, and much clearer than threading two nonlocal variables.
Prefer the tuple.
```

7 The traps

Returning the recorded value instead of the returnable one.

```
>>> def bad_depth(node):
...     if node is None:
...         return 0
...     left, right = bad_depth(node.left), bad_depth(node.right)
...     return left + right + 1          # WRONG: this is a fork, not a path
>>> # on a balanced tree of 7 nodes this returns 7, not 3
```

It returns the node count, not the depth, because every ancestor adds both branches. **No error, and a number that grows with the tree** — so it looks plausible on a small test.

The rule is two sentences, and saying them prevents this: *I record the best path through this node; I return the best path down from this node.*

Forgetting the clamp on negative values.

```
>>> tree = Node(2, Node(-5), Node(-3))
>>> # without max(0, ...):
>>> #   best = 2 + (-5) + (-3) = -6
>>> # with the clamp:
>>> #   best = 2 + 0 + 0 = 2
>>> max_path_sum(tree)
2
```

Without the clamp the path is forced through branches that lose money, and the answer is too small. **The clamp says “this path simply does not go that way.”**

Initialising `best` to zero in max path sum.

```
>>> max_path_sum(Node(-3, Node(-2), Node(-5)))
-2
```

Correct. With `best = 0` it would return `0`, which is not the value of any path — an all-negative tree has a negative answer, and zero corresponds to the empty path, which is not allowed.

One number where two are needed.

```
>>> # rob_tree returning just max(take, skip):
>>> #   the parent cannot tell whether that best USED the child
>>> #   → it may take itself AND count a child that was taken
>>> #   → an answer that is too LARGE, silently
```

The uncle's point exactly. When the parent's options depend on the child's choice and not only its score, the state needs a dimension for that choice.

Recursion depth on a degenerate tree.

```
>>> line = build_line(2000)
>>> diameter(line)
Traceback (most recent call last):
...
RecursionError: maximum recursion depth exceeded
```

A tree that is a straight line is a valid tree — and it is the shape a sorted insert sequence produces, so it appears in real test suites. **The constraint** `n ≤ 105` is a direct instruction to handle it.

`sys.setrecursionlimit` as the fix.

```
>>> sys.setrecursionlimit(2000000)
>>> diameter(build_line(150000))
Segmentation fault
```

It raises the Python limit and not the C stack limit, so a deep enough tree crashes the interpreter instead of raising. **A segfault is worse than an exception**, and an explicit stack is the answer when the depth is genuinely unbounded.

Mutating shared state across branches.

```
>>> def bad(node, path=[]):                                # mutable default argument
...     path.append(node.value)
...     ...
```

The list persists between calls and accumulates across siblings. In tree DP specifically, **anything passed down must be copied or undone on the way back up** — append before the recursive call, pop after.

Treating a graph as a tree.

```
>>> # sum_of_distances without the `if child != parent` guard
>>> # → infinite recursion, because edges are bidirectional
RecursionError: maximum recursion depth exceeded
```

A tree stored as an adjacency list has every edge twice, so a traversal must remember where it came from. The `parent` parameter is not decoration — without it, every recursion immediately walks back up.

Assuming the root is the answer.

```
>>> # diameter, max path sum, and "largest subtree with X" all have
>>> # answers that may live anywhere in the tree
>>> # returning the root's value gives a plausible smaller number
```

Ask whether the answer is a property of the root or of the whole tree. If it is of the whole tree, it must be recorded during the traversal, not returned from it.

8 In the interview

How it gets asked

- “Find the diameter of a binary tree.” — LeetCode 543, the standard opener.
- “Maximum path sum, any node to any node.” — LeetCode 124, and it is Hard mainly for the two details.
- “House robber, but on a tree.” — LeetCode 337, the include/exclude state.
- “What do you return, and what do you record?” — asked directly, sometimes.
- “For every node, compute the sum of distances to all other nodes.” — LeetCode 834, the rerooting one.
- “What happens if the tree has a hundred thousand nodes in a line?”

“Tree DP, and the nice thing about it is that **the hardest part of dynamic programming disappears.**

A subtree is a subproblem, and post-order traversal visits every child before its parent — which is exactly the dependency direction. **So there is no table and no fill order to get wrong; the recursion does it for me.** After a week of interval DP where the fill order was the whole difficulty, that is worth saying.

The pattern is four lines: base case for the empty subtree, recurse into both children, then combine.

The part that actually needs care is what the state is, and it is often not one number.

For house robber on a tree — you cannot take a node and its child — the state per node is a pair: the best total if I take this node, and the best if I skip it. If I take it, each child must contribute its *skip* value. If I skip it, each child contributes the better of its two.

And the reason for two numbers rather than one is the point: the parent’s options depend on the child’s *choice*, not just its score. **A single ‘best’ would not say which case produced it.**

The other thing I would establish before writing code is the distinction between what I record and what I return, because that is where the bugs are.

For diameter: the longest path *through* a node uses both children — `left + right` — and that is a candidate for the answer. But the value the parent needs is the longest path going *down*, which uses only one child — `1 + max(left, right)` — because a path cannot fork at a node and still continue upwards.

So the function records one quantity in an outer variable and returns a different one, and returning the recorded value is the defining bug of the topic: it compiles, runs, and gives numbers that are too large.

Cost: `O(n)` time — every node once — and `O(depth)` space for the stack.

One question: how large can the tree be? Because a hundred thousand nodes in a line is a valid tree, and Python’s recursion limit is a thousand. **If the constraint allows that, I would write it with an explicit stack.**“

The follow-ups

“What is the difference between what you return and what you record?”

“They are two different quantities, and conflating them is the single most common bug in tree problems, so I say both out loud while writing.

Take diameter. At a node, the longest path *through* it goes down the left subtree, up through the node, and down the right — `left_depth + right_depth`. That is a genuine candidate for the answer.

But that value is useless to the parent. If the parent tried to extend it, the resulting walk would go down the left, back up through this node, down the right, back up through this node again, and on to the parent — **it would visit this node twice, so it is not a path.**

What the parent can use is the longest path going *down* from this node, which may use only one side: `1 + max(left, right)`.

So: record `left + right`, return `1 + max(left, right)`.

Returning the recorded value instead gives no error and numbers that are too large — on a balanced seven-node tree it returns seven rather than three, because it has counted every node rather than measured a path. **And it grows with the tree, so it looks plausible.**

The same structure appears in maximum path sum, with two extra details. **Clamp each child’s contribution at zero**, because a branch with a negative sum should simply not be used — without it, the path is forced through losses. **And initialise the running best to negative infinity, not zero**, because an all-negative tree has a negative answer and zero corresponds to the empty path.

The general test I apply is: **is the answer a property of the root, or of the whole tree?** Height is a property of the root, so I return it. **Diameter and max path sum can occur anywhere, so they are recorded during the traversal** — and any problem whose answer can occur anywhere needs that outer variable.”

“House robber on a tree. Why do you need two values per node?”

“Because the parent’s options depend on **which choice** the child made, not just on how well the child did.

The rule is that I cannot take a node and its immediate child. So when I am deciding about a node:

If I take it, every child must be skipped — so I need each child’s best total *given that the child was not taken*.

If I skip it, each child is free — so I need each child’s best total *either way*, which is the max of its two values.

A single ‘best’ number cannot serve both. Suppose a child reports 10, and that 10 came from taking itself. **If I now take the parent, I cannot use that 10** — but the number gives me no way to know, so I add it anyway and produce an answer that is too large, silently, with an illegal selection behind it.

So the function returns a pair: `(take, skip)`.

`take = node.value + left.skip + right.skip`. `skip = max(left.take, left.skip) + max(right.take, right.skip)`. The answer at the root is the larger of its two.

This is the same idea as the mode dimension in the stock problems — the state needs to record which situation you are in, because it determines what the next level can legally do. **Here the ‘mode’ is whether this node was used.**

Cost: `O(n)` time and `O(depth)` space, and the tuple allocation per node is negligible.

And I would prefer returning a tuple to threading two `nonlocal` variables — it is clearer, and it makes the state visible in the type signature, which is exactly where the difficulty of this problem lives.”

“The tree has a hundred thousand nodes and might be a straight line. What do you do?”

“That is a recursion-depth question, and it is deliberate — **a tree in a straight line is a valid binary tree**, and it is what a sorted insert sequence produces, so it appears in real test suites.

Python’s default recursion limit is a thousand frames, so a two-thousand-node line raises `RecursionError` before doing any useful work.

The quick fix is `sys.setrecursionlimit`, and I would mention it and then explain why I would not rely on it. It raises Python’s own limit but not the operating system’s C stack limit, **so a deep enough tree segfaults the interpreter rather than raising an exception.** A crash is strictly worse than an error, and the depth at which it happens depends on the platform.

The robust answer is an explicit stack, which moves the frames to the heap where they are bounded only by memory. A hundred thousand entries as tuples is about ten megabytes.

The pattern for getting post-order from a stack is worth knowing precisely, because it is not obvious: **push each node twice — once marked unprocessed and once marked processed.** When you pop an unprocessed node, push it back as processed and then push its children. When you pop a processed node, both children are already done and their results are available.

That flag is what turns a stack into a post-order traversal, and without it you get pre-order, which visits parents before children — exactly the wrong direction for DP.

The cost is that it is maybe twice the code and noticeably less readable, so I would write the recursive version first, say that it fails on a degenerate tree, and convert it if the constraint requires it. Stating the trade is better than either silently using recursion or writing the harder version unprompted.

And for a balanced tree none of this matters — depth is about seventeen for a hundred thousand nodes — so **the question is really whether the input can be adversarial**, and the constraint is where that is written.”

The model answer

“You have an organisational hierarchy. Each employee has a manager, and each has a ‘disruption score’. You want to invite a set of people to a workshop such that no one is invited together with their direct manager, maximising the total score. Then tell me, for every employee, how many people are under them.”

“Two questions on the same tree, and they are different shapes of tree DP, so let me take them in order.

The first is house robber on a tree. The hierarchy is a tree — each employee has one manager, the chief executive has none — and the constraint ‘not with your direct manager’ is exactly ‘not a node and its parent’.

I would confirm one thing first: is it really a tree? If someone can have two managers — a matrix organisation — **this is a graph, not a tree, and the problem becomes maximum weight independent set, which is NP-hard in general.** That is worth thirty seconds of asking, because the answer changes from linear-time to intractable.

Assuming a tree: the state per employee is a pair. The best total for their subtree **if they are invited**, and the best **if they are not**.

If invited, none of their direct reports may be — so each report contributes its ‘not invited’ value. **If not invited, each report is free** — so each contributes the better of its two.

The answer is the larger of the chief executive’s two values.

And the reason it must be a pair is the whole subtlety: a single ‘best’ would not say whether that best included the person themselves, so their manager could not tell whether it was usable. **That would produce an answer that is too high with an illegal invitation list behind it** — and in this domain, an illegal list is a meeting where someone sits opposite their own manager, which is precisely what the constraint was for.

Cost: $O(n)$, **one post-order traversal.** For any real organisation — tens of thousands of people — that is milliseconds.

The second question, headcount under each employee, is a simple subtree size: one plus the sum of the children’s counts, computed in the same post-order pass. **I would compute both in one traversal** rather than two, since they share the shape.

Now the practical points, which matter more here than the algorithm.

Depth is not a problem. Real hierarchies are shallow — even a very large company is under twenty levels — **so recursion is safe and I would not write the iterative version.** That is worth saying explicitly, because it is the opposite of the answer for an arbitrary binary tree, and knowing which situation you are in is the point.

But the data will not be clean. Cycles: someone whose manager chain loops back, which happens in real HR data through bad records, **and which makes the recursion never terminate.** I would validate the structure first — a cycle check is one traversal — rather than discover it as a stack overflow in production.

And multiple roots: contractors, or people whose manager has left. **The ‘tree’ is really a forest**, so I would run the traversal from every node with no manager and sum the results.

Finally, I would question the objective. Maximising the total score subject to no-manager-pairs gives a mathematically optimal list, **and it will produce a group with strange gaps** — a whole team invited except one person, because their manager scored higher. **If the real goal is a useful workshop, the constraint is probably softer than stated**, and I would want to check that before shipping an optimiser that is technically correct and socially peculiar.”

9 Recall card

A subtree is a subproblem, and post-order traversal visits every child before its parent — so the recursion IS the fill order. No table, no loops, nothing to get wrong: **the hardest mechanical part of DP disappears when the data is a tree.**
`O(n)` time, `O(depth)` space.

The state is often a pair, not a number. House robber on a tree: `(take, skip)`, where `take = value +  $\sum$  child.skip` and `skip =  $\sum$  max(child.take, child.skip)`. **One number fails because the parent’s options depend on the child’s CHOICE, not just its score** — and it fails silently, too high, with an illegal selection behind it.

Record $\neq$ return, and this is the defining bug. Diameter: **record** `left + right` (the path *through* this node) and **return** `1 + max(left, right)` (the path *down* from it) — a path cannot fork at a node and still continue upwards. Returning the recorded value gives no error and numbers that grow with the tree.

Max path sum adds two details: clamp each child at `max(0, ...)` so a negative branch is skipped, and **initialise** `best` to `-inf`, not 0, because an all-negative tree has a negative answer.

Ask whether the answer is a property of the root or of the whole tree — height is returned; diameter and max path sum are **recorded**, because they can occur anywhere.

Depth is the practical trap: a line-shaped tree is valid, and Python's limit is 1,000 frames. `setrecursionlimit` raises Python's limit but not the C stack, so it **segfaults instead of raising** — use an explicit stack, pushing each node twice with a `processed` flag to get post-order. **Rerooting** answers a question at *every* node in $O(n)$ instead of $O(n^2)$: one post-order pass for subtree answers, one pre-order pass where moving the root across an edge costs one subtraction.

1 What this is, and why they ask it

Yesterday you designed YouTube. **Netflix delivers video too, and almost every design decision is different**, because the catalogue is different in one crucial way: **it is small, and you know what is in it in advance.**

YouTube ingests five hundred hours a minute from strangers. **Netflix has perhaps twenty thousand titles, all licensed, all known weeks before anyone watches them.** That single fact changes everything downstream.

Transcoding stops being a firehose and becomes a batch job you can spend real effort on — per-title, per-scene optimisation that would be absurd at YouTube's ingest rate.

Delivery stops being reactive and becomes predictive. You know that a series drops on Friday, so **you can push the files to every edge on Wednesday** rather than waiting for a cache miss. That is the single biggest difference between the two systems and it is the thing to lead with.

And the recommendation system stops being a feature and becomes the product, because with twenty thousand titles and no search intent, **what appears on the home screen decides what gets watched.**

They ask it because **it is the best test of whether you compare rather than recite.** A candidate who produces the YouTube answer again has missed the question. A candidate who says “the catalogue is small and known in advance, so I precompute and pre-position everything” has understood it in one sentence.

By the end of this lesson you can design the ingest and encoding pipeline, the open-connect-style delivery model, playback and the home page, and say what predictability buys.

2 The story

The lending library in the town had two rooms and Ranganathan had run it for twenty-six years, and the thing he was known for was not the books.

It was that he had them ready.

The school announced its reading list in the last week of May, every year, the same seven or eight books. **And every year, by the time the list came out, he already had eleven copies of each of them on the front table.**

Not because he was clever. **Because it was the same list, more or less, and he had been watching it for twenty-six years.**

His rival in the next town — a bigger shop, more books — did it the other way. **A boy would come in asking for the poetry book, and the man would say he could get it by Thursday.** Which was true. And by Thursday the boy had borrowed it from a friend.

Ranganathan's whole method was that he knew what was coming.

And there was a second part to it, which took him longer to work out.

He did not keep the eleven copies in the two rooms. There was not space. So he had an arrangement with the tailor near the school, and the man who ran the tea stall by the college, **and each of them kept three or four copies under the counter.**

The books were near the people, before the people wanted them.

It cost him something. He paid the tailor a little. And sometimes he was wrong — one year he stocked forty copies of a book the school dropped, and they sat there for two years and he gave most of them away.

But the arithmetic was not close. **When he was right, which was most of the time, the boy walked in and walked out with the book.**

And the third part, which was the one his son did not understand for years.

Ranganathan spent a lot of time on the front table.

Which books faced outwards. Which sat at eye level. Which were in the little stack by the door. **He rearranged it constantly,** and he did it differently depending on who was in the room — the college students got one arrangement, the schoolchildren another.

His son said the books were the same wherever you put them.

"There are four thousand books here," his father said, "and the boy will look at eleven of them. I decide which eleven."

3 The idea in plain English

Ranganathan is Netflix. **Know what is coming, put it near the people before they ask, and control the front table.**

Start with the comparison, because it is the answer to the question.

	YouTube	Netflix
catalogue	500 hrs/minute, from anyone	~20,000 titles, licensed, known weeks in advance
encoding	must be fast; a firehose	a BATCH JOB, optimise hard
content per title	watched once, mostly	watched millions of times
delivery	reactive: cache on miss	PREDICTIVE: push before release
discovery	search + recommendation	recommendation IS the product
the hard part	ingest scale	delivery economics + ranking

Say that table's first row and its consequence in the first minute. Everything else follows from “the catalogue is small and known”.

Now the four things it buys.

One: encoding becomes a batch job you can spend effort on.

At YouTube you transcode as fast as possible because there is a queue behind you. **At Netflix a title is encoded once and watched millions of times, so an hour of extra compute that saves two percent of the bitrate pays for itself forever.**

Which makes per-title and per-shot encoding worth doing. A cartoon and a dark action film do not need the same bitrate at the same resolution — **so instead of a fixed ladder, you compute the best ladder for each title**, and even for each scene. **Netflix published exactly this**, and the reported saving is around twenty percent of the bitrate at the same perceived quality.

Twenty percent off the largest line on the bill is enormous, and it is only worth the effort because the encode is amortised over millions of views.

Two: delivery becomes predictive, and this is the biggest difference.

YouTube's edge caches fill on a miss. The first viewer in a region pays the latency of fetching from origin.

Netflix knows what is coming. A series releases on Friday at midnight; **the files are pushed to every edge server on Wednesday**, during off-peak hours, when bandwidth is cheap and nobody is watching.

So on Friday there are no cache misses at all — every viewer is served from a machine inside their own internet provider's network, and the origin serves nothing.

And the edge machines live inside ISP networks, which is the Open Connect model: Netflix gives an ISP a box, the ISP racks it, and Netflix's traffic never crosses an expensive transit link. **The ISP saves transit costs and Netflix saves delivery costs**, which is why both sides agree.

That arrangement is only possible because the catalogue is small enough to fit. Twenty thousand titles is a few hundred terabytes — **a box you can ship**. YouTube's exabytes are not.

Three: the recommendation system is the product.

A user opens the app with no query. There is no search intent, no link they followed. **The home page is the entire interface**, and what is on it determines what is watched.

The structure is rows, and each row is a separate ranking problem: continue watching, because you watched X, trending now, new releases. **Rows are personalised, ordered, and the artwork within them is chosen per user** — the same film shown with a different image depending on what the model thinks appeals.

And it is precomputed. Ranking twenty thousand titles per user at request time would be expensive and slow. **The home page is computed in batch — nightly, or on significant events — and cached**, with a small real-time layer for “continue watching” and very recent activity.

Which is a genuine architectural consequence of the catalogue being small: precomputing a page over twenty thousand items is feasible, and doing it over YouTube's billions is not.

Four: playback, which is mostly yesterday's design with three additions.

Adaptive bitrate over static segments, exactly as before. **The additions are:**

Digital rights management. The content is licensed, so it is encrypted and the player must obtain a decryption licence from a licence server. **Widevine, PlayReady and FairPlay** are the three systems, one per platform ecosystem, **and supporting all three means encrypting the content three ways** — or using CMAF with common encryption to store one copy.

Resume across devices. Watch position is a small write, very frequently — every few seconds, per user, per title. **That is a high-write, tiny-payload workload**, quite unlike everything else in the system, and it needs its own store.

And offline download, which is DRM again with a different licence policy: a licence that works without a network, for a limited time.

Finally, the thing worth saying about scale.

Netflix's peak is a huge fraction of the internet's traffic in some countries, and it is highly predictable: evenings, weekends, and enormous spikes when a major title releases. **Predictable peaks are much easier to plan for than unpredictable ones**, and they justify capacity that sits idle at four in the morning.

4 The picture

The two systems, side by side:

YOUTUBE

500 hrs/min ingest

|

TRANSCODE FAST

6 fixed renditions

~1x real-time, at scale

|

REACTIVE CDN

cache fills on a MISS

first viewer pays

|

search + recommendation

user usually knows what

they want

NETFLIX

~20,000 titles, known in advance

|

ENCODE WELL (per-title, per-shot)

a ladder computed for THIS title

hours per title, amortised over

millions of views

|

PREDICTIVE PUSH

files pushed to every edge BEFORE

release, during off-peak

|

RECOMMENDATION IS THE INTERFACE

no query; the home page decides

what gets watched

Predictive placement, which is Ranganathan's tailor:

WEDNESDAY (off-peak, 03:00 local)

origin ---- push new season ----> [edge in ISP A]

---- push new season ----> [edge in ISP B]

---- push new season ----> [edge in ISP C] ... thousands

bandwidth is cheap, nobody is watching, no user waits

FRIDAY 00:00 (release)

10,000,000 viewers press play

every single one is served from a box INSIDE their own ISP

origin traffic: ~ZERO

Compare a reactive CDN: the first viewer in each region waits for an origin fetch, and the origin sees a thundering herd at 00:00.

Why Open Connect works for both sides:

WITHOUT an embedded cache

Netflix origin

|

TRANSIT LINK \$\$\$

|

ISP network

|

viewer

the ISP pays for transit

Netflix pays for egress

WITH one

Netflix origin

| (one push, off-peak)

v

[OCA box inside the ISP]

|

ISP network (free, internal)

|

viewer

the ISP pays nothing

Netflix ships a box once

BOTH SIDES SAVE. That is why ISPs agree to rack them.

And it only works because the catalogue FITS on a box.

Per-title encoding, and why it is worth the compute:

FIXED LADDER (YouTube-style)

every title: 1080p @ 5 Mbps, 720p @ 2.5, 480p @ 1 ...

an animated film at 1080p looks perfect at 2 Mbps

→ 3 Mbps wasted on EVERY VIEW

a dark, grainy action film needs 8 Mbps at 1080p

→ visibly bad at 5

PER-TITLE LADDER

compute the quality-vs-bitrate curve for THIS title

pick the bitrates where quality actually improves

reported saving: ~20% of bitrate at the same perceived quality

WHY IT IS WORTH HOURS OF COMPUTE:

encode once, serve millions of times

20% of the largest line on the bill, forever

WHY YOUTUBE CANNOT: 500 hours/minute arriving, most of it

watched a handful of times. The amortisation is not there.

The home page as rows, which is the product:

```
+-----+
| Continue Watching      [real-time, per user] |
+-----+
| Because you watched X  [batch, per user]    |
+-----+
| Trending Now           [batch, per region]   |
+-----+
| New Releases           [batch, per region]   |
+-----+
```

each ROW is a separate ranking problem
the ORDER of the rows is itself ranked
the ARTWORK for each title is chosen per user

precomputed nightly and cached, with a thin real-time layer
for "continue watching" and very recent activity

→ possible because the catalogue is 20,000 items.
Not possible over billions.

5 How it actually works

Ingest and encoding

```
def ingest_title(title_id: str, master_file: str) → None:
    analysis = analyse_complexity(master_file)          # per-shot complexity
    ladder = compute_ladder(analysis)                   # bitrates for THIS title
    shots = detect_shot_boundaries(master_file)

    for shot_index, (start, end) in enumerate(shots):
        for height, bitrate in ladder:
            queue.publish("encode", {
                "title_id": title_id, "shot": shot_index,
                "start": start, "end": end,
                "height": height, "bitrate": bitrate,
            })
    job_store.expect(title_id, len(shots) * len(ladder))
```

`compute_ladder(analysis)` is the whole difference from yesterday. YouTube uses a fixed list of six renditions; here the list is computed from the content, because a cartoon and a grainy thriller need different bitrates for the same perceived quality.

Splitting by shot rather than by fixed duration is the refinement: a scene's complexity is roughly constant within a shot and changes at cuts, so per-shot encoding targets each one appropriately.

The parallelism is yesterday's — one job per shot per rendition — but the *goal* is different: **YouTube chunks for latency; Netflix chunks because it has to be parallel anyway and the quality work is the point.**

Predictive distribution

```
def schedule_prepositioning(title_id: str, release_at: float) → None:
    popularity = forecast.predicted_demand(title_id) # by region
    for region, expected_viewers in popularity.items():
        for edge in edges_in(region):
            if expected_viewers > edge.threshold:
                push_jobs.schedule(
                    title_id, edge,
                    at=off_peak_window(edge, before=release_at),
                )
```

off_peak_window is the key idea: the push happens at three in the morning local time, **when the edge's bandwidth is idle and nobody is watching.** The transfer is free in every sense that matters.

And it is per edge, not global, because a title popular in one country may be irrelevant in another — **pushing everything everywhere would need far more storage per box than exists.**

The forecast can be wrong, and the consequence is Ranganathan's forty unwanted copies: **a box holding a title nobody watches, which is wasted space rather than a failure.** The reverse — a title not pushed that suddenly trends — falls back to fetching from a regional origin.

The playback path

```
@app.post("/play")
def start_playback(user_id: int, title_id: str, device: dict) → dict:
    if not entitlements.allows(user_id, title_id, device["region"]):
        return {"error": "not available in your region"}, 403

    edge = steering.best_edge_for(user_id, device) # by ISP, latency, health
    return {
        "manifest": f"https://{edge}/{title_id}/master.mpd",
        "licence_url": "https://licence.example.com/acquire",
        "start_position": watch_state.position(user_id, title_id),
    }
```

steering.best_edge_for is a real system, and it is more than DNS geolocation: **it picks the edge inside the user's own ISP where one exists, falls back to a nearby one, and routes around unhealthy boxes.**

entitlements is the licensing check — content rights are per country and per date, so a title available in one region genuinely does not exist in another, and that check belongs before anything else.

And **start_position** comes back with the manifest, so resume is one round trip rather than two.

DRM, in outline

```
@app.post("/licence")
def acquire_licence(user_id: int, title_id: str, drm_system: str,
                    challenge: bytes) → bytes:
    if not entitlements.allows(user_id, title_id, region_of(user_id)):
        raise Forbidden
    if device_count(user_id) > plan_limit(user_id):
        raise TooManyStreams
    key = content_keys.get(title_id)
    return drm[drm_system].issue(challenge, key, policy=policy_for(user_id))
```

The licence server, not the CDN, is where policy is enforced — concurrent stream limits, region checks, offline expiry. **The segments themselves are encrypted static files that anyone can download and nobody can play**, which is what lets them sit on an ISP's box safely.

Three DRM systems for three ecosystems — Widevine for Android and Chrome, PlayReady for Windows and Xbox, FairPlay for Apple. **With CMAF and common encryption you store one encrypted copy and issue three kinds of licence**; without it you store the content three times.

Watch position, which is a surprising workload

```
def report_position(user_id: int, title_id: str, seconds: float) → None:
    key = f"pos:{user_id}:{title_id}"
    redis.set(key, seconds, ex=90 * 86400)          # fast path
    if int(seconds) % 30 == 0:                      # durable path, sampled
        queue.publish("watch_state", {"user": user_id, "title": title_id,
                                     "seconds": seconds})
```

Two paths again. The Redis write is what “continue watching” reads — **frequent, tiny, and losing a few seconds of it costs nothing**. The sampled queue write is what survives a Redis failure and what feeds analytics.

The volume is the surprise: every playing device reports every few seconds, so **this is a higher write rate than anything else in the system** — and each write is thirty bytes.

The home page

```
def home_page(user_id: int) → list[dict]:
    rows = precomputed_rows.get(user_id)                # built nightly, cached
    if rows is None:
        rows = fallback_rows(region_of(user_id))        # popular, non-personalised
    live = [continue_watching_row(user_id)]             # real-time
    return live + rows
```

Precomputed plus a thin real-time layer is the shape. **The expensive personalised ranking runs in batch; only the things that must be current are computed per request.**

And the fallback matters: a new user, or one whose batch job has not run, **must get a sensible page rather than an empty one** — popular titles for their region, which is also what a cold start looks like.

The real systems

Open Connect AWS	Netflix's own CDN: appliances placed inside ISPs everything except delivery – control plane, encoding, recommendations, the API
Cassandra	viewing history and watch positions
EVCache	Netflix's memcached layer, for the home page and positions
Widevine / PlayReady / FairPlay	DRM, one per ecosystem
VMAF	Netflix's perceptual quality metric – the thing that makes per-title encoding measurable
Chaos Monkey	deliberately killing instances to prove resilience

VMAF is the good one to name, because it is what makes per-title encoding possible at all: **you cannot optimise bitrate against quality without a metric for quality that matches what people actually see.**

6 The numbers

Scale.

```
~270,000,000 subscribers
~2 hours watched per subscriber per day
= ~540,000,000 hours/day

peak concurrency: ~15% of subscribers in the evening
= ~40,000,000 concurrent streams
```

Catalogue, which is the number that changes everything.

~20,000 titles, average ~1.5 hours = ~30,000 hours of content
per hour of content, all renditions plus audio tracks and subtitles:
~15 GB (similar to yesterday)

 $30,000 \times 15 \text{ GB} = \sim 450 \text{ TB}$ for the entire catalogue

compare YouTube: 4 EB/year.
→ Netflix's ENTIRE catalogue is ~0.01% of one year of YouTube uploads

Four hundred and fifty terabytes is the whole system's content, and that is the fact everything else follows from: **it fits on a handful of drives, so you can ship it to every ISP.**

Delivery.

540,000,000 hours/day at an average ~4 Mbps (higher than YouTube: big screens, good connections, longer sessions)

 $540e6 \times 3,600 \text{ s} \times 4 \text{ Mbps} / 8 = 972,000,000 \text{ GB/day}$
 $= \sim 970 \text{ PB/day}$

peak: 40,000,000 concurrent x 4 Mbps = 160 Tbps

A hundred and sixty terabits a second at peak. On a commercial CDN at a cent a gigabyte that is about **\$9.7 million a day**, or three hundred million a month.

with Open Connect:
the marginal cost of a byte served from an ISP-embedded box is
~the amortised hardware + power, not a per-GB price

~18,000 appliances worldwide, ~\$10-30k each, ~5-year life
= ~\$300M capital, ~\$5M/month amortised
+ operations

→ roughly 50x cheaper than buying delivery, and it is the single largest engineering decision the company has made.

Prepositioning, and what it saves:

a major release: 10,000,000 viewers in the first 24 hours

REACTIVE CDN:

first viewer per edge pays an origin fetch
~18,000 edges x ~15 GB per title = 270 TB from origin
concentrated in the first minutes → a thundering herd

PREDICTIVE PUSH:

the same 270 TB, moved at 03:00 local over several days
origin traffic at release: ~0
and it is off-peak bandwidth, which is close to free

Same bytes. Completely different cost and risk profile.

Per-title encoding, quantified:

encoding a 2-hour film with per-shot optimisation:

~1,000 shots x ~10 renditions = 10,000 jobs
each ~30 s of CPU = ~83 CPU-hours per title
plus the analysis passes: call it ~150 CPU-hours

20,000 titles x 150 = 3,000,000 CPU-hours, ONCE
at \$0.03/hour = ~\$90,000 total for the whole catalogue

THE SAVING: ~20% of 970 PB/day
= ~194 PB/day of bandwidth not sent

→ ninety thousand dollars of compute, once, against a fifth of
the largest recurring line, forever.
Compare YouTube: 4.3M CPU-hours PER DAY just to keep up.

That comparison is the best thing in this lesson: YouTube spends more on transcoding in one day than Netflix spends encoding its entire catalogue.

Watch positions, the surprising workload:

40,000,000 concurrent streams, each reporting every 5 s
= 8,000,000 writes/second

each write ~30 bytes

→ higher write rate than the message rate of WhatsApp,
for the smallest payloads in this course

Redis/EVCache absorbs it; a sampled subset goes to Cassandra
for durability and analytics.

The home page:

```
270,000,000 subscribers x ~40 rows x ~50 titles per row
= the precomputed page is a few KB per user
270e6 x 5 KB = ~1.4 TB of cached pages

recomputed nightly:
270,000,000 users / 86,400 s = ~3,100 users/second of batch ranking
→ a large but ordinary batch job

vs ranking 20,000 titles per user at request time:
40,000,000 peak concurrent x a model over 20,000 items
→ not viable, which is why it is precomputed
```

The bill, ranked:

```
delivery (Open Connect, amortised)    ~$5-10M/month
AWS: control plane, encoding,          ~$30M/month
    recommendations, data
content licensing                      ~$1.5 BILLION/month
-----
infrastructure is ~2% of the content spend

→ which is the real difference from YouTube. There the
    infrastructure IS the business; here it is a rounding error
    next to paying for the shows.
```

7 The trade-offs

Predictive pre-positioning against reactive caching. Prediction eliminates cache misses at release and moves the bytes at off-peak, which is close to free. **It costs storage on every edge box for content that may not be watched there** — Ranganathan's forty unwanted copies — and it depends on a forecast that is sometimes wrong. **The fallback to a regional origin makes being wrong cheap**, which is what makes the bet worth taking.

Per-title encoding against a fixed ladder. Per-title saves about twenty percent of the delivery bill forever, at the cost of hours of compute per title and a much more complex pipeline. **It is only justified because a title is encoded once and watched millions of times** — the amortisation is the whole argument, and **it is exactly why YouTube cannot do it.**

Building the CDN against buying it. Open Connect is roughly fifty times cheaper at this volume and is a multi-year capital and operational commitment: **hardware, logistics, relationships with thousands of ISPs, and a team.** Below a few petabytes

a month it would be an absurd thing to build. **Knowing that the break-even exists, and that Netflix is far past it, is the answer.**

DRM against openness. DRM is a licensing requirement — **studios will not license content without it** — so it is not really a choice. It costs three encryption systems, a licence server on the critical path of every playback start, and **a whole class of platform-specific bugs**. CMAF with common encryption reduces the storage cost from three copies to one.

Precomputed home pages against real-time ranking. Precomputing is cheap to serve and is stale by up to a day. **Real-time would reflect what you watched five minutes ago and cannot be afforded at forty million concurrent users.** The hybrid — batch rows plus a thin real-time layer for continue-watching — is what everybody builds, **and the honest description is that most of the page is yesterday's opinion.**

Regional licensing, which is a constraint rather than a trade. The same title is available in some countries and not others, and the rights change on fixed dates. **That means the catalogue is genuinely different per region**, entitlement checks are on the playback path, and **content sometimes disappears from a user's list**, which is a product problem with no technical solution.

When would I not build this? Almost all of it, almost always. For a video product below a few petabytes a month, **a commercial CDN with reactive caching is cheaper and simpler than anything here.** Per-title encoding is only worth it when views per title are in the millions. **And the recommendation system is worth building only when the catalogue is large enough to need it and small enough to rank** — with two hundred titles, a hand-curated home page beats a model.

8 In the interview

How it gets asked

- “*Design Netflix.*” — and the interviewer usually wants to hear how it differs from YouTube.
- “*How is this different from YouTube?*” — asked directly, and it is the whole question.
- “*A new season drops at midnight and ten million people watch it. What happens?*”
- “*How do you decide what bitrate to encode at?*” — the per-title question.
- “*How does the home page work?*”
- “*Why does Netflix run its own CDN?*”

“The right way into this is the comparison with YouTube, because **one fact about the catalogue changes almost every decision.**

YouTube ingests five hundred hours a minute from strangers. Netflix has about twenty thousand titles, all licensed, all known weeks before anyone watches them.

And the whole catalogue is about four hundred and fifty terabytes — which is roughly a hundredth of one percent of a year of YouTube uploads. **It fits on a handful of drives, and that is what makes everything else possible.**

Four consequences, and I would take them in order of how much they matter.

One: delivery becomes predictive rather than reactive. A CDN normally fills its cache on a miss, so the first viewer in each region waits. **Netflix knows a series drops on Friday, so the files are pushed to every edge on Wednesday, at three in the morning local time, when bandwidth is idle. On Friday there are no cache misses at all.**

And the edges are inside ISP networks — Netflix ships a box, the ISP racks it, and the traffic never crosses a paid transit link. **Both sides save**, which is why ISPs agree. **And it only works because the catalogue fits on a box.**

Two: encoding becomes a batch job worth optimising. A title is encoded once and watched millions of times, **so hours of extra compute that save twenty percent of the bitrate pay for themselves forever.** That justifies per-title and even per-shot encoding — a cartoon and a grainy thriller genuinely need different bitrates. **YouTube cannot do this**, because its content is watched a handful of times and there is a firehose behind it.

The arithmetic is striking: YouTube spends more CPU on transcoding in one day than Netflix spends encoding its entire catalogue.

Three: recommendations are the product, not a feature. A user opens the app with no query. **The home page is the entire interface**, it is rows, each row is a separate ranking problem, and it is **precomputed nightly** — ranking twenty thousand titles per user at request time is not viable at forty million concurrent users, and precomputing over twenty thousand items is, which is again the catalogue size doing the work.

Four: playback is yesterday's adaptive bitrate plus DRM, entitlements and resume.

Which of those would you like me to go deep on?"

The follow-ups

"A new season drops at midnight and ten million people watch it. What happens?"

“Almost nothing dramatic, and the reason it is not dramatic is the whole design.

With a reactive CDN it would be a thundering herd. At midnight, the first viewer at each of about eighteen thousand edge locations triggers an origin fetch of roughly fifteen gigabytes of content — **two hundred and seventy terabytes pulled from origin, concentrated into the first few minutes**, while every one of those first viewers waits.

Netflix does not do that, because it knows the release date.

The files were pushed days in advance. Each edge received its copy at about three in the morning local time, spread over several nights, **when its bandwidth was idle and nobody was watching.** The same two hundred and seventy terabytes moved, at close to zero marginal cost, with no user waiting.

So at midnight, origin traffic is essentially zero. Every viewer is served from a box inside their own internet provider’s network.

The pushing is per edge, not global, because a box cannot hold everything and a title popular in one country may be irrelevant in another. **A demand forecast decides which edges get which titles.**

And the forecast is sometimes wrong, in two directions. **Over-pushing wastes space on a box** — cheap, and the content is evicted eventually. **Under-pushing means a title that unexpectedly trends is not on the local box**, and those viewers fall back to a regional origin — slower, but correct, and it self-corrects within minutes as the miss triggers a fill.

The failure mode I would actually plan for is concurrency, not bandwidth. Ten million people pressing play in the same minute is ten million licence acquisitions and ten million playback-start API calls. **The licence server and the playback API see a spike that the CDN does not**, because the CDN was pre-warmed and they cannot be. **That is where I would put the load testing and the autoscaling headroom**, and it is the non-obvious part of the answer.”

“How do you decide what bitrate to encode at?”

“Per title, and increasingly per shot — and the interesting part is why that is affordable here and not at YouTube.

The naive approach is a fixed ladder: every title gets 1080p at five megabits, 720p at two and a half, and so on. **And it is wrong in both directions at once.**

An animated film looks perfect at 1080p and two megabits — large flat areas of colour compress extremely well — **so three megabits are wasted on every single view.** A dark, grainy action film needs perhaps eight megabits at 1080p and looks visibly bad at five. **The same ladder over-serves one and under-serves the other.**

So: compute the ladder for each title. Encode at many candidate bitrates, measure the resulting quality, and pick the points where quality actually improves. **The refinement is to do it per shot**, because complexity is roughly constant within a shot and changes at cuts.

The thing that makes this possible is a quality metric that matches human perception. Netflix built VMAF for exactly this — **you cannot optimise bitrate against quality without a number for quality that agrees with what people see**, and older metrics like PSNR do not.

The reported saving is about twenty percent of bitrate at the same perceived quality.

And here is the economics, which is the real answer. Per-shot encoding a two-hour film is maybe a hundred and fifty CPU-hours. **Twenty thousand titles is three million CPU-hours — about ninety thousand dollars, once.** The saving is a fifth of the largest recurring line on the bill, forever.

YouTube cannot make that trade, and it is worth saying why precisely: **its content is watched a handful of times on average, so there is nothing to amortise the extra compute over**, and it has four hundred and thirty thousand hours arriving per day, so there is a queue behind every encode. **Netflix optimises because it can afford to think; YouTube optimises for throughput because it cannot.**“

“Why does Netflix run its own CDN?”

“Because at a hundred and sixty terabits a second, buying delivery costs more than building it by about a factor of fifty — **and because the catalogue is small enough to make building it possible.**

The arithmetic first. Five hundred and forty million hours watched a day at around four megabits is nearly a petabyte a day, **and at a commercial rate of a cent a gigabyte that is about ten million dollars a day.** Three hundred million a month, on delivery alone.

Open Connect is a different model entirely. Netflix builds appliances — ordinary servers full of drives — **and gives them to internet providers, who rack them in their own data centres for free.**

Both sides win, which is why it works. The ISP’s subscribers pull Netflix traffic from a box inside the ISP’s own network, **so the ISP does not pay for transit** — and Netflix is a large fraction of that ISP’s traffic, so the saving is real. Netflix ships hardware once instead of paying per gigabyte forever.

Roughly eighteen thousand appliances at ten to thirty thousand dollars each, on a five-year life, is around five million dollars a month amortised — against three hundred million to buy the same delivery.

And the crucial enabler is that the catalogue fits. Four hundred and fifty terabytes is a box you can build. **YouTube’s exabytes are not**, which is why Google’s edge strategy is different — it caches reactively and cannot pre-position, because it does not know what will be watched.

What it costs is not money. It is a multi-year commitment: **hardware design, logistics, and relationships with thousands of individual internet providers** — which is an organisational capability, not a technical one, and it is much harder to acquire than the engineering.

So the honest framing is that this is right at Netflix’s scale and absurd below it. For a video product serving a few petabytes a month, **a commercial CDN is cheaper than the team you would need**, and I would want to know roughly where the crossover is before proposing it anywhere else.”

The model answer

“Design Netflix. Two hundred and seventy million subscribers, twenty thousand titles, global.”

“I want to lead with the comparison to a user-generated video platform, because one fact about the catalogue determines most of the design.

Twenty thousand titles at about fifteen gigabytes per hour of content is roughly four hundred and fifty terabytes for the entire catalogue — against exabytes a year for YouTube. The catalogue is small, and every title’s release date is known weeks in advance. Everything below follows from that.

Sizing. Two hundred and seventy million subscribers watching about two hours a day is 540 million hours a day. At an average four megabits — higher than YouTube because it is big screens and long sessions — that is **about 970 petabytes a day, and 160 terabits a second at evening peak.**

Delivery: Open Connect, and this is the largest engineering decision in the system. Netflix-built appliances placed inside ISP networks. **At commercial CDN rates this traffic would be three hundred million dollars a month; amortised hardware is around five. Fifty times cheaper — and it is only possible because four hundred and fifty terabytes fits on a box you can ship.**

And delivery is predictive, not reactive. A release date is known, so **content is pushed to every relevant edge days ahead, at three in the morning local time.** At midnight on release there are no cache misses and origin traffic is essentially zero. **A reactive CDN would pull two hundred and seventy terabytes from origin in the first minutes while the first viewer at every edge waits.**

Which edges get which titles comes from a demand forecast, because a box cannot hold everything. Over-pushing wastes space; under-pushing falls back to a regional origin and self-corrects. **Being wrong is cheap in both directions, which is what makes the bet sound.**

Encoding: per-title, and per-shot within a title. Compute the quality-versus-bitrate curve for this content and pick the ladder from it, measured with a perceptual metric — **you cannot optimise against quality without a number for quality that agrees with human eyes.** About twenty percent bitrate saving at the same perceived quality.

The economics are the argument: three million CPU-hours for the entire catalogue, once — about ninety thousand dollars — against a fifth of the delivery bill, forever. YouTube spends more on transcoding in one day. **The difference is amortisation: encode once, serve millions of times.**

Playback: adaptive bitrate over encrypted static segments, plus three things a user-generated platform does not need. DRM, because studios require it — three systems for three ecosystems, and CMAF with common encryption so the content is stored once rather than three times. **Entitlements**, because rights are per country and per date, checked before playback starts. **And resume**, which is a surprisingly large workload: forty million concurrent streams reporting position every few seconds is **eight million writes a second of thirty bytes each** — higher write volume than most systems in this course, absorbed by a memory cache with a sampled durable path.

The home page, which is the product. A user arrives with no query, so **what is on the screen decides what is watched**. Rows, each a separate ranking problem, with the row order itself ranked and the artwork chosen per user. **Precomputed nightly and cached**, with a thin real-time layer for continue-watching — because ranking twenty thousand titles per user at request time cannot be done at forty million concurrent users, and precomputing over twenty thousand items can. **Again the catalogue size doing the work.**

Two things I would flag.

The spike at a major release is a control-plane problem, not a bandwidth one. The CDN was pre-warmed; **the licence server and playback API cannot be, and they see ten million requests in a minute.** That is where the headroom and the load testing belong, and it is the non-obvious failure mode.

And the honest scale point: infrastructure is about two percent of what this business spends. Content licensing is one and a half billion a month. **So the engineering goal is not to minimise cost in absolute terms — it is to make the service good enough that people keep paying for the content**, which is a different objective from YouTube's, where the infrastructure genuinely is the business.”

9 Recall card

One fact drives everything: the catalogue is SMALL (~20,000 titles, ~450 TB — 0.01% of a year of YouTube uploads) and KNOWN WEEKS IN ADVANCE. Lead with that comparison; every other difference follows from it.

Delivery is PREDICTIVE, not reactive. Files are pushed to every edge days before release, at 03:00 local when bandwidth is idle — **so at midnight there are no cache misses and origin traffic is ~zero.** A reactive CDN would pull ~270 TB from origin

in the first minutes with the first viewer at every edge waiting.

Open Connect: appliances placed INSIDE ISP networks. Both sides save (the ISP avoids transit, Netflix avoids per-GB egress), which is why ISPs rack them. **~\$5M/month amortised against ~\$300M/month to buy the same delivery — 50× — and it only works because the catalogue fits on a box.**

Per-title and per-shot encoding, measured with a perceptual metric (VMAF): ~20% bitrate saving. Justified purely by amortisation — encode once, serve millions of times. **~3M CPU-hours for the WHOLE catalogue (~\$90k, once) against YouTube's 4.3M CPU-hours PER DAY.** A cartoon needs 2 Mbps where a grainy thriller needs 8; a fixed ladder is wrong in both directions.

Recommendations are the interface, not a feature — no query, so the home page decides what is watched. Rows, each a ranking problem, **precomputed nightly** with a thin real-time layer; feasible over 20,000 items and not over billions.

Playback adds three things to yesterday's ABR: DRM (three ecosystems; CMAF + common encryption stores one copy not three), **entitlements** (rights are per country and per date), and **resume** — 40M concurrent streams reporting every ~5 s is **8M writes/second of 30 bytes**, the highest write rate here. **The release spike is a control-plane problem** — the CDN is pre-warmed, the licence server cannot be.

PRACTICE Practice — DP on trees

DSA topic: DP on trees **System design topic:** Design Netflix

Code these, in this order

One rule for the whole set: **before writing the function, say two sentences out loud — what I record, and what I return.** If they are the same thing, say so explicitly. If they differ, the bug lives in the gap between them, and naming it in advance is the only reliable defence.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Diameter of Binary Tree	LeetCode 543 (Easy)	Record versus return, in its clearest form.
2	Binary Tree Maximum Path Sum	LeetCode 124 (Hard)	The same, plus the clamp and the <code>-inf</code> base.
3	House Robber III	LeetCode 337 (Medium)	The <code>(take, skip)</code> state, and why one number fails.
4	Longest Univalue Path	LeetCode 687 (Medium)	Record/return again, with a condition on the edge.
5	Sum of Distances in Tree	LeetCode 834 (Hard)	Rerooting, and why the naive version is <code>O(n²)</code> .
6	Binary Tree Cameras	LeetCode 968 (Hard)	Three states per node, and a greedy that also works.

On problem 1, return the recorded value

Write `return left + right + 1` instead of `1 + max(left, right)`. Run it on a balanced seven-node tree and record what comes back. **Say what that number actually is**, and why it looks plausible as a diameter.

On problem 2, remove the clamp

Drop the `max(0, ...)`. Run on a node with value 2 and two children of -5 and -3 . Record the answer against the correct one. **Say what the path was forced to do.**

On problem 2, start `best` at zero

Run on a tree where every value is negative. Record what you get and what the answer should be. **Say what path the zero corresponds to**, and why that path is not allowed.

On problem 3, return one number

Return `max(take, skip)` instead of the pair. Construct a small tree where the answer comes out too large, and **write down the illegal selection it corresponds to** — which nodes it invited together.

On problem 5, write the naive version and time it

Run a full traversal from every node. Time it at `n = 2,000` and `n = 10,000`. **Extrapolate to 30,000** and say whether it would pass. Then write the two-pass version and time that.

On problem 5, derive the rerooting line

Before coding pass two, write out in words what happens to the distance sum when the root moves across one edge from `u` to its child `v`. **Count how many nodes get closer and how many get further.** Then write the line.

Then the depth drill

Build a tree of 2,000 nodes in a straight line. Run your recursive solution and record the exact error. Then set `sys.setrecursionlimit(200000)` and run it on a 150,000-node line. **Record what happens.** Then write the iterative version and run it on the same input.

Then the post-order-from-a-stack drill

Write the explicit-stack version of problem 1 from memory. **Say what the boolean flag is for**, in one sentence, and what you get without it.

The pattern drill

1. Write the four-line skeleton from memory.
2. Say why post-order is the right traversal.
3. Say what tree DP gets for free that interval DP does not.
4. Give the time and space, and what the space depends on.

The record-versus-return drill

1. State the two sentences for diameter.
2. Say why the parent cannot use the recorded value.
3. Say what returning the wrong one produces, and whether it errors.
4. Give the test that decides whether a problem needs an outer variable.
5. Name three problems where the answer can live anywhere.

The state drill

1. Say why house robber on a tree needs two numbers.
2. Give both transitions.
3. Say what a single number loses, and what illegal thing it permits.
4. Say which earlier topic this is the same idea as.

The negatives drill

1. Say what the clamp does and where it goes.
2. Give an example where omitting it is wrong.
3. Say why `best` starts at `-inf`.
4. Say what a `best` of 0 would correspond to.

The rerooting drill

1. Say what question needs it.
2. Give the naive cost and the constraint that rules it out.
3. Describe pass one and what each array holds.
4. Give the pass-two line and derive it in words.
5. Say what the total cost is.

The depth drill

1. Give Python's default limit and what a line-shaped tree does.
2. Say why a line-shaped tree is a realistic input.
3. Say what `setrecursionlimit` fixes and what it does not.
4. Describe the explicit-stack post-order pattern.
5. Say when you would bother, and when you would not.

The break-it drill

Trigger each and record the exact output or error:

1. Returning `left + right` from a depth function.
2. No clamp on negative children.
3. `best = 0` in max path sum.
4. One number instead of `(take, skip)`.
5. A 2,000-node line, recursively.
6. `setrecursionlimit(200000)` on a 150,000-node line.
7. A traversal on a bidirectional adjacency list with no `parent` guard.

Four of the seven give no error at all. Name them.

The comparison drill

1. Give the six-row comparison with a user-generated video platform.
2. Say what the single driving fact is.
3. Compute the catalogue size and compare it with a year of YouTube uploads.

The delivery drill

1. Say what reactive caching does at a midnight release, with numbers.
2. Describe predictive pre-positioning and when the transfer happens.
3. Say why both sides agree to an ISP-embedded box.
4. Give the cost comparison, bought against built.
5. Say what makes building it possible here and not at YouTube.
6. Say what happens when the demand forecast is wrong, in both directions.

The encoding drill

1. Say what a fixed ladder gets wrong, in both directions, with an example.
2. Describe per-title and per-shot encoding.
3. Say why a perceptual quality metric is required.
4. Give the compute cost for the whole catalogue and the saving.
5. Say precisely why YouTube cannot make the same trade.

The playback drill

1. Name the three things this adds to plain adaptive streaming.
2. Say where DRM policy is enforced, and why not at the edge.
3. Say what CMAF with common encryption saves.
4. Say why entitlement checks come first.
5. Compute the watch-position write rate and say what absorbs it.

The home page drill

1. Say why the home page is the product.
2. Describe the row structure and what is ranked.
3. Say what is precomputed and what is real-time.
4. Say why real-time ranking is not viable, with numbers.
5. Say what a new user sees, and why that matters.

The scale drill

1. Compute daily egress and peak bits per second.
 2. Give the appliance count and amortised cost.
 3. Say which spike is a control-plane problem rather than a bandwidth one.
 4. Say what fraction of the company's spend infrastructure is, and what follows.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the diameter of a binary tree.* The post-order pattern, the two sentences about record and return, why the parent cannot use the recorded value, the complexity, and what happens on a line-shaped tree.
 2. *House robber, but on a tree.* The two-number state, both transitions, exactly what one number loses, and what illegal answer it produces.
 3. *Design Netflix — and how is it different from YouTube?* The catalogue fact, predictive pre-positioning, Open Connect and why both sides agree, per-title encoding with its amortisation argument, and the precomputed home page.
-

Before you move on

- [] I can write the post-order tree DP skeleton from memory.
- [] I know the recursion is the fill order, and why.
- [] I say what I record and what I return, before writing.
- [] I know why the parent cannot use the recorded value.
- [] I know returning the wrong one gives no error.
- [] I have the test for whether a problem needs an outer variable.
- [] I know the clamp and where it goes.
- [] I know `best` starts at `-inf` and what 0 would mean.
- [] I can give the `(take, skip)` state and both transitions.
- [] I can say what one number loses, and the illegal selection it allows.
- [] I know this is the same idea as the stock problem's mode.
- [] I know the time and space, and what the space depends on.
- [] I know Python's recursion limit and what a line-shaped tree does.

- [] I know `setrecursionlimit` can segfault rather than raise.
- [] I can write the explicit-stack post-order version.
- [] I know what the boolean flag is for.
- [] I know rerooting is $O(n)$ where the naive version is $O(n^2)$.
- [] I can derive the pass-two line in words.
- [] I know a bidirectional adjacency list needs a `parent` guard.
- [] I can give the six-row Netflix/YouTube comparison.
- [] I know the catalogue size and why it drives everything.
- [] I can explain predictive pre-positioning and its timing.
- [] I know why ISPs agree to host an appliance.
- [] I can give the bought-versus-built cost comparison.
- [] I know what makes building it possible here and not at YouTube.
- [] I can explain per-title encoding and the amortisation argument.
- [] I know why a perceptual metric is required.
- [] I can name the three additions to plain adaptive streaming.
- [] I can compute the watch-position write rate.
- [] I know the release spike is a control-plane problem.
- [] I answered all three questions above out loud.

Bitmask DP

DATA STRUCTURES AND ALGORITHMS

Bitmask DP

SYSTEM DESIGN

Design Google Drive or Dropbox

1 What this is, and why they ask it

Bitmask DP is what you use when the state has to remember a *set* — which things have been used, visited, or assigned — and there is no cheaper way to describe it.

An integer's bits become the set. Bit i set means item i is in the set. A 20-element set is one integer between 0 and about a million, so `dp[mask]` is an ordinary array lookup and the whole set-tracking problem becomes arithmetic.

They ask it because **it is the honest answer to a specific class of problem: the ones where nothing smaller works**. Every other DP this month found a small state — a position, an amount, a mode. **Here the state genuinely is a subset, because the order things were used in does not matter but *which* things were used does**, and no summary of the set is enough.

The other reason is that **it teaches you where DP stops**. 2^n states means $n = 20$ is a million and $n = 40$ is a trillion. **There is no clever fix**, because the problems are NP-hard: travelling salesman, assignment with arbitrary costs, exact set cover. **Bitmask DP turns $O(n!)$ into $O(2^n \cdot n^2)$** , which is an enormous improvement and still exponential — and being able to say both halves of that sentence is what the question is testing.

And there is a practical reason: **the constraint tells you the answer**. $n \leq 20$ in a problem about subsets or orderings is almost always bitmask DP, and recognising that from the constraint alone saves ten minutes of looking for something polynomial that does not exist.

By the end of this lesson you can encode a set as an integer, write the standard iteration patterns, solve travelling salesman and the assignment problem, know the submask enumeration trick, and say exactly where the ceiling is.

2 The story

The hall had eleven jobs and eleven people and the woman running the temple festival had two days to work out who did what.

Not who was best at what. She knew that. **The difficulty was that everybody was better at some things than others, and the totals did not come out the way you expected.**

Sundari was the best at the flowers, easily. But she was also the only one who could handle the accounts, and the second-best at accounts was hopeless.

So putting Sundari on the flowers, which was obviously right, made the whole festival worse.

The first day she tried it by hand. Start with the most important job, give it to whoever was best, cross them both off, move to the next. **She got an arrangement and she was not confident about it,** and when she tried a different starting job she got a different arrangement and could not tell which was better.

Her nephew, who was doing accountancy and had been made to come, said the thing that unlocked it.

“You keep asking who does the flowers. Ask instead: of the people not yet given a job, what is the best we can do?”

And she said that was the same question.

“It is not, because it does not matter which order we gave the earlier jobs out in. Only who is left.”

Which she thought about, and then agreed with, **because it was obviously true and she had been treating two identical situations as different all morning.**

If Sundari and Meera and Prakash were taken and the other eight were free, it **made no difference at all whether Sundari had been assigned first or third.** Eight people, and the same jobs remaining. One situation.

“So how many situations are there?” she asked.

He worked it out on the back of the list. Each person is either taken or free. Eleven people. **Two to the eleven.**

“Two thousand and forty-eight,” he said. “And for each one, you only have to decide the next job.”

Which was more than she wanted to do by hand, and very much less than she had feared, **because she had been imagining trying every arrangement, and there were forty million of those.**

3 The idea in plain English

The nephew's observation — **it does not matter what order, only who is left** — is exactly why bitmask DP works, and it is the first thing to check before reaching for it.

Start with the encoding, which is the mechanical part.

An integer's bits are a set. With `n = 5`:

```
mask = 0b01011 = 11 decimal
      ^^^^^
      bit 0 set → item 0 is in the set
      bit 1 set → item 1 is in the set
      bit 3 set → item 3 is in the set
      → the set {0, 1, 3}
```

Five operations do everything:

```
mask | (1 << i)      # add item i
mask & ~(1 << i)     # remove item i
mask & (1 << i)      # test item i (non-zero if present)
mask ^ (1 << i)      # toggle item i
bin(mask).count("1") # how many items – or int.bit_count() in 3.10+
```

`1 << i` is “the mask with only bit `i` set”, and every operation is built from it. Write those five lines down once and the encoding stops being the difficulty.

Now the state, and the test that decides whether this applies.

`dp[mask]` is the answer for the situation where exactly the items in `mask` have been used.

The test is the nephew's sentence: does the order matter, or only the set? If two different orders leading to the same set are genuinely the same situation, **a set is a complete state and this works**. If the order matters — if the last thing you did affects what you can do next — **you need a second dimension**.

Travelling salesman is exactly that case. `dp[mask][last]` — which cities have been visited, **and which one you are standing in** — because the cost of the next move depends on where you are. **Two dimensions:** `2^n × n` states.

The assignment problem is the simpler case. Assign `n` people to `n` jobs, each person to exactly one job, minimising total cost. **Here `dp[mask]` alone is enough**, and there is a small, beautiful reason:

The number of people assigned so far is `popcount(mask)` — the number of set bits. So which job you are assigning next is determined by the mask itself, and you do not need to store it. That is Sundari's problem, and the one-dimensional state is what makes it clean.

The two iteration patterns.

Forwards — “from this state, where can I go?” — is usually clearer:

```
for mask in range(1 << n):
    job = popcount(mask)                # the next job to assign
    for person in range(n):
        if not (mask & (1 << person)):    # this person is free
            next_mask = mask | (1 << person)
            dp[next_mask] = min(dp[next_mask], dp[mask] + cost[person][job])
```

Iterating masks in increasing numerical order is a valid fill order, and this is worth knowing precisely: adding a bit always increases the integer, so `mask` is always numerically smaller than `mask | bit`. The natural loop order happens to be the dependency order, which after interval DP is a relief.

Backwards — “how did I get here?” — is the other pattern, and it reads more like the DPs you have been writing:

```
for mask in range(1 << n):
    job = popcount(mask) - 1            # the job just assigned
    for person in range(n):
        if mask & (1 << person):        # this person was used
            previous = mask ^ (1 << person)
            dp[mask] = min(dp[mask], dp[previous] + cost[person][job])
```

Both are correct. Pick one and be consistent, because mixing them is a real source of confusion.

Now travelling salesman, which is the canonical problem.

Visit every city exactly once and return to the start, minimising total distance.

```
dp[mask][last] = the cheapest route that visits exactly the cities in mask
                  and currently stands at city `last`

dp[mask | (1<<next)][next] = min over `last` in mask of
                             dp[mask][last] + dist[last][next]
```

Base case: `dp[1][0] = 0` — you have visited only city 0 and you are standing there.

The answer is `min over last of dp[full_mask][last] + dist[last][0]`, adding the trip home. Forgetting that final leg is the standard bug, and it gives a plausible smaller number.

Cost: $2^n \times n$ states, $O(n)$ work each — $O(2^n \cdot n^2)$.

Against brute force's $O(n!)$, and the comparison is the point. At $n = 15$: brute force is 87 billion orderings; bitmask DP is about 7.4 million operations. Four orders of magnitude.

And at $n = 25$ bitmask DP is 2.1×10^{10} , which is also too slow — the improvement is enormous and the problem is still exponential. Both halves of that sentence, always.

Then the trick worth knowing: enumerating submasks.

Some problems need “for every subset of this set” — partitioning a group into teams, exact cover. The naive version loops over all 2^n masks and checks whether each is a subset, which is $O(4^n)$.

There is a standard idiom that visits only the actual submasks:

```
submask = mask
while submask:
    process(submask)
    submask = (submask - 1) & mask           # the next smaller submask
```

`(submask - 1) & mask` is the whole trick. Subtracting one borrows through the low zero bits, and the `& mask` clears everything that was not in the original set — so it lands exactly on the next smaller subset.

And the total work over all masks is 3^n , not 4^n , because each element is in three states across the sum: in the submask, in the mask but not the submask, or in neither. 3^n against 4^n at $n = 16$ is 43 million against 4.3 billion — a hundredfold, and it is why the idiom is worth memorising rather than deriving.

Finally, the ceiling, and it is hard.

$n = 15$	$2^{15} = 32,768$ states	trivial
$n = 20$	$2^{20} = 1,048,576$	fine — the usual constraint
$n = 24$	$2^{24} = 16,777,216$	slow in Python, fine in C
$n = 30$	$2^{30} = 1,073,741,824$	4 GB just for the array
$n = 40$	$2^{40} = 10^{12}$	impossible

Every extra element doubles everything. So the constraint in the problem is not a hint, it is a specification: $n \leq 20$ means bitmask, and $n \leq 10^5$ means it is definitely not.

4 The picture

The encoding:

```
items:  4  3  2  1  0
bits:    0  1  0  1  1  = 0b01011 = 11
                                → the set {0, 1, 3}

add 2:    11 | (1<<2) = 11 | 4 = 15 = 0b01111 → {0,1,2,3}
remove 1: 11 & ~(1<<1) = 11 & ~2 = 9 = 0b01001 → {0,3}
test 3:    11 & (1<<3) = 11 & 8 = 8 (non-zero) → yes
count:     bin(11).count("1") = 3
```

ALL of bitmask DP's mechanics are these four lines.

The nephew's insight, drawn:

ORDERINGS	SETS
Sundari, Meera, Prakash	{Sundari, Meera, Prakash}
Sundari, Prakash, Meera	
Meera, Sundari, Prakash	ONE state
Meera, Prakash, Sundari	
Prakash, Sundari, Meera	because the remaining decision
Prakash, Meera, Sundari	depends only on WHO IS LEFT

6 orderings → 1 state

n = 11: 11! = 39,916,800 orderings
2¹¹ = 2,048 states

19,500x fewer. THAT is the whole technique.

Why the natural loop order works:

```
mask      = 0b0101 = 5
mask | bit = 0b0111 = 7      (adding bit 1)
```

ADDING A BIT ALWAYS INCREASES THE INTEGER.

So iterating `mask = 0, 1, 2, ... 2^n - 1` visits every mask AFTER every mask it depends on.

→ ``for mask in range(1 << n)`` is a correct fill order, for free.

Compare interval DP, where getting this right was the whole difficulty. Here the numbering does it.

Travelling salesman, state by state:

4 cities, starting at 0

```
dp[0001][0] = 0                                visited {0}, standing at 0
```

```
dp[0011][1] = dist[0][1]                        visited {0,1}, at 1
```

```
dp[0101][2] = dist[0][2]
```

```
dp[1001][3] = dist[0][3]
```

```
dp[0111][2] = min( dp[0011][1] + dist[1][2],      via 1
                    dp[0101][2→?] ... )           etc.
```

...

```
answer = min over last of  dp[1111][last] + dist[^^^^^^^^^^^^^^^^last][0]
```

THE TRIP HOME. Forgetting
this is the standard bug,
and it gives a smaller,
plausible number.

The submask idiom, traced:

```

mask = 0b1010 (the set {1, 3})

submask = 1010
  process {1,3}
  (1010 - 1) & 1010 = 1001 & 1010 = 1000
submask = 1000
  process {3}
  (1000 - 1) & 1010 = 0111 & 1010 = 0010
submask = 0010
  process {1}
  (0010 - 1) & 1010 = 0001 & 1010 = 0000
submask = 0000 → loop ends

visited exactly {1,3}, {3}, {1} – every non-empty submask, once.

TOTAL over all masks:  $3^n$ , not  $4^n$ .
Each element is in one of THREE states: in the submask,
in the mask but not the submask, or in neither.

n = 16:    $3^{16} = 43,000,000$       vs       $4^{16} = 4,300,000,000$ 

```

The ceiling, which is the real lesson:

n	2^n	$2^n \times n^2$	feasible?
15	32,768	7,372,800	yes, instantly
20	1,048,576	419,430,400	yes, ~1 min in Python
24	16,777,216	9,663,676,416	no in Python, yes in C
30	1,073,741,824	$\sim 10^{12}$	no – 4 GB for the array alone
40	$\sim 10^{12}$	$\sim 10^{15}$	no, and never will be

EVERY EXTRA ELEMENT DOUBLES EVERYTHING.

The constraint in the problem is a specification, not a hint.

5 The code, built step by step

The five operations

```

def add(mask: int, i: int) → int:      return mask | (1 << i)
def remove(mask: int, i: int) → int:   return mask & ~(1 << i)
def has(mask: int, i: int) → bool:     return bool(mask & (1 << i))
def size(mask: int) → int:             return mask.bit_count() # Python 3.10+
def full(n: int) → int:               return (1 << n) - 1

```

$(1 \ll n) - 1$ is the full set — n ones — and it is worth naming, because it appears in every answer line.

`mask.bit_count()` is Python 3.10 and later; `bin(mask).count("1")` works everywhere and is slower.

The assignment problem

```
def min_assignment_cost(cost: list[list[int]]) → int:
    """cost[person][job]. Assign each person exactly one job."""
    n = len(cost)
    dp = [float("inf")] * (1 << n)
    dp[0] = 0
    for mask in range(1 << n):
        if dp[mask] == float("inf"):
            continue
        job = mask.bit_count()
        if job == n:
            continue
        for person in range(n):
            if not (mask & (1 << person)):
                # this person is still free
                nxt = mask | (1 << person)
                dp[nxt] = min(dp[nxt], dp[mask] + cost[person][job])
    return int(dp[(1 << n) - 1])
```

`job = mask.bit_count()` is the elegant part. Because every step assigns exactly one person to exactly one job, the number of people used tells you which job you are on — so the state needs no second dimension.

The `if dp[mask] == inf: continue` matters for speed: most masks are reachable here, but in problems with constraints many are not, and skipping them can be a large saving.

Travelling salesman

```
def tsp(dist: list[list[int]]) → int:
    n = len(dist)
    dp = [[float("inf")] * n for _ in range(1 << n)]
    dp[1][0] = 0
    # visited only city 0, standing there

    for mask in range(1 << n):
        for last in range(n):
            if dp[mask][last] == float("inf"):
                continue
            for nxt in range(n):
                if mask & (1 << nxt):
                    continue
                # already visited
                nmask = mask | (1 << nxt)
                candidate = dp[mask][last] + dist[last][nxt]
                if candidate < dp[nmask][nxt]:
                    dp[nmask][nxt] = candidate

    full_mask = (1 << n) - 1
    return int(min(dp[full_mask][last] + dist[last][0] for last in range(n)))
```

The second dimension is `last`, and it is required here: unlike the assignment problem, the cost of the next step depends on where you are standing, so the set alone is not a complete state.

`+ dist[last][0]` in the final line is the trip home. Leave it out and you have solved the open path problem, which is a different question with a smaller answer — and no error.

Recovering the route

```
def tsp_route(dist: list[list[int]]) → tuple[int, list[int]]:
    n = len(dist)
    dp = [[float("inf")] * n for _ in range(1 << n)]
    parent = [[-1] * n for _ in range(1 << n)]
    dp[1][0] = 0

    for mask in range(1 << n):
        for last in range(n):
            if dp[mask][last] == float("inf"):
                continue
            for nxt in range(n):
                if mask & (1 << nxt):
                    continue
                nmask = mask | (1 << nxt)
                candidate = dp[mask][last] + dist[last][nxt]
                if candidate < dp[nmask][nxt]:
                    dp[nmask][nxt] = candidate
                    parent[nmask][nxt] = last

    full_mask = (1 << n) - 1
    best_last = min(range(n), key=lambda c: dp[full_mask][c] + dist[c][0])
    total = dp[full_mask][best_last] + dist[best_last][0]

    route, mask, city = [], full_mask, best_last
    while city != -1:
        route.append(city)
        previous = parent[mask][city]
        mask ^= (1 << city) # remove this city from the mask
        city = previous
    return int(total), route[::-1] + [0]
```

`mask ^= (1 << city)` while walking back is the reverse of the forward step, and it is the line that makes the reconstruction work — you must undo the mask as well as follow the parent.

`iterate_bits` runs once per set bit, not `n` times, which matters when masks are sparse. `mask & -mask` isolating the lowest set bit is worth knowing — it comes from two's complement, and it is the same trick a Fenwick tree uses.

The complete solution

```
"""Bitmask DP: sets as integers, and where the ceiling is."""

def full_mask(n: int) → int:
    return (1 << n) - 1

def min_assignment_cost(cost: list[list[int]]) → int:
    """Assign n people to n jobs, minimising total cost. dp[mask] alone."""
    n = len(cost)
    dp = [float("inf")] * (1 << n)
    dp[0] = 0
    for mask in range(1 << n):
        if dp[mask] == float("inf"):
            continue
        job = mask.bit_count() # WHICH job follows from the mask
        if job == n:
            continue
        for person in range(n):
            if not (mask & (1 << person)):
                nxt = mask | (1 << person)
                dp[nxt] = min(dp[nxt], dp[mask] + cost[person][job])
    return int(dp[full_mask(n)])

def tsp(dist: list[list[int]]) → int:
    """dp[mask][last]: the second dimension is REQUIRED here."""
    n = len(dist)
    dp = [[float("inf")] * n for _ in range(1 << n)]
    dp[1][0] = 0
    for mask in range(1 << n):
        for last in range(n):
            if dp[mask][last] == float("inf"):
                continue
            for nxt in range(n):
                if not (mask & (1 << nxt)):
                    nmask = mask | (1 << nxt)
                    candidate = dp[mask][last] + dist[last][nxt]
                    if candidate < dp[nmask][nxt]:
                        dp[nmask][nxt] = candidate
    fm = full_mask(n)
    return int(min(dp[fm][last] + dist[last][0] for last in range(n)))

def tsp_open(dist: list[list[int]]) → int:
    """The same without returning home – a DIFFERENT problem."""
    n = len(dist)
    dp = [[float("inf")] * n for _ in range(1 << n)]
    dp[1][0] = 0
    for mask in range(1 << n):
        for last in range(n):
            if dp[mask][last] == float("inf"):
                continue
            for nxt in range(n):
                if not (mask & (1 << nxt)):
                    nmask = mask | (1 << nxt)
                    dp[nmask][nxt] = min(dp[nmask][nxt],
                                         dp[mask][last] + dist[last][nxt])
    return int(min(dp[full_mask(n)]))
```

```

def tsp_route(dist: list[list[int]]) → tuple[int, list[int]]:
    """The cheapest tour and the route itself."""
    n = len(dist)
    dp = [[float("inf")] * n for _ in range(1 << n)]
    parent = [[-1] * n for _ in range(1 << n)]
    dp[1][0] = 0
    for mask in range(1 << n):
        for last in range(n):
            if dp[mask][last] == float("inf"):
                continue
            for nxt in range(n):
                if not (mask & (1 << nxt)):
                    nmask = mask | (1 << nxt)
                    candidate = dp[mask][last] + dist[last][nxt]
                    if candidate < dp[nmask][nxt]:
                        dp[nmask][nxt] = candidate
                        parent[nmask][nxt] = last

    fm = full_mask(n)
    best_last = min(range(n), key=lambda c: dp[fm][c] + dist[c][0])
    total = dp[fm][best_last] + dist[best_last][0]

    route: list[int] = []
    mask, city = fm, best_last
    while city != -1:
        route.append(city)
        previous = parent[mask][city]
        mask ^= (1 << city) # undo the mask as you walk back
        city = previous
    return int(total), route[::-1] + [0]

def all_submasks(mask: int):
    """Every non-empty submask, exactly once. (submask - 1) & mask."""
    submask = mask
    while submask:
        yield submask
        submask = (submask - 1) & mask

def can_partition_k_subsets(nums: list[int], k: int) → bool:
    """Split nums into k subsets of equal sum. Classic bitmask DP."""
    total = sum(nums)
    if total % k:
        return False
    target = total // k
    if max(nums) > target:
        return False
    n = len(nums)
    dp = [False] * (1 << n)
    dp[0] = True
    used = [0] * (1 << n) # sum of the current partial subset
    for mask in range(1 << n):
        if not dp[mask]:
            continue
        for i in range(n):
            if mask & (1 << i):
                continue
            if used[mask] % target + nums[i] > target:
                continue # would overflow this subset

```

```

        nxt = mask | (1 << i)
        if not dp[nxt]:
            dp[nxt] = True
            used[nxt] = used[mask] + nums[i]
        return dp[full_mask(n)]

def brute_force_tours(n: int) → int:
    """How many orderings a naive solver would try: (n-1)!!!!"""
    result = 1
    for i in range(1, n):
        result *= i
    return result

if __name__ == "__main__":
    print("bits of 11          :", bin(11), "size", (11).bit_count())
    print("add 2 to 11          :", 11 | (1 << 2))
    print("remove 1 from 11     :", 11 & ~(1 << 1))
    print("full mask n=5       :", full_mask(5))

    cost = [[9, 2, 7, 8],
            [6, 4, 3, 7],
            [5, 8, 1, 8],
            [7, 6, 9, 4]]
    print("assignment cost      :", min_assignment_cost(cost))

    dist = [[0, 20, 42, 35],
            [20, 0, 30, 34],
            [42, 30, 0, 12],
            [35, 34, 12, 0]]
    print("tsp closed tour      :", tsp(dist))
    print("tsp open path        :", tsp_open(dist))
    print("tsp route            :", tsp_route(dist))

    print("submasks of 0b1010:", [bin(s) for s in all_submasks(0b1010)])

    print("partition 4 subsets:", can_partition_k_subsets([4, 3, 2, 3, 5, 2, 1], 4))
    print("partition 3 subsets:", can_partition_k_subsets([1, 2, 3, 4], 3))

    for n in (10, 15, 20, 25):
        print(f"n={n:2} states 2^n={1 << n:>12,} "
              f"brute (n-1)≠{brute_force_tours(n):>18,}")

```

Run it and you get:

```

bits of 11      : 0b1011 size 3
add 2 to 11     : 15
remove 1 from 11 : 9
full mask n=5   : 31
assignment cost : 13
tsp closed tour  : 97
tsp open path    : 62
tsp route        : (97, [0, 3, 2, 1, 0])
submasks of 0b1010: ['0b1010', '0b1000', '0b10']
partition 4 subsets: True
partition 3 subsets: False
n=10  states 2^n=      1,024  brute (n-1)≠      362,880
n=15  states 2^n=    32,768  brute (n-1)≠    87,178,291,200
n=20  states 2^n=  1,048,576  brute (n-1)≠121,645,100,408,832,000
n=25  states 2^n= 33,554,432  brute (n-1)≠620,448,401,733,239,439,360,000

```

tsp closed tour 97 against tsp open path 62 is the trip-home bug made visible: thirty-five apart, and both are plausible.

And the last table is the whole lesson. At $n = 20$, bitmask DP has a million states and brute force has a hundred and twenty-one *quadrillion* arrangements. At $n = 25$, bitmask DP is 33 million states — already uncomfortable — and brute force is 6×10^{23} .

6 What it costs

The state space.

```

sets only:      2^n states
sets + position: 2^n x n states

```

And the work per state is usually $O(n)$ — trying each remaining item — so $O(2^n \cdot n)$ or $O(2^n \cdot n^2)$.

Concretely, for travelling salesman:

n = 10	1,024 x 10 states x 10 work	=	102,400	instant
n = 15	32,768 x 15 x 15	=	7,372,800	~2 s in Python
n = 20	1,048,576 x 20 x 20	=	419,430,400	~2 min. Painful.
n = 22	4,194,304 x 22 x 22	=	2,029,690,000	too slow
n = 25	33,554,432 x 25 x 25	=	20,971,520,000	no

And the memory, which bites first:

`dp[2^n][n]` as Python floats

`n = 20:` $1,048,576 \times 20 = 20,971,520$ cells
Python list of lists: ~8 bytes per pointer + float objects
→ roughly 500 MB. Uncomfortable.

`n = 24:` $16,777,216 \times 24 = 402,653,184$ cells → ~10 GB. No.

→ in Python, `n = 18-20` is the practical ceiling for the two-dimensional version, and memory stops you before time does.

The comparison with brute force, which is the point:

n	bitmask $2^n \times n^2$	brute force $(n-1)!$
10	102,400	362,880
15	7,372,800	87,178,291,200
20	419,430,400	121,645,100,408,832,000
25	20,971,520,000	620,448,401,733,239,439,360,000

at `n = 20:` 290 BILLION times fewer operations

at `n = 25:` ~30 trillion times fewer

AND BOTH ARE STILL EXPONENTIAL. That is the honest sentence.

Submask enumeration:

naive (all masks, check subset): 4^n
the (submask-1) & mask idiom: 3^n

<code>n = 12:</code>	$3^{12} = 531,441$	$4^{12} = 16,777,216$	32x
<code>n = 16:</code>	$3^{16} = 43,046,721$	$4^{16} = 4,294,967,296$	100x
<code>n = 20:</code>	$3^{20} = 3.5$ billion	$4^{20} = 10^{12}$	300x

the ratio is $(4/3)^n$, so it grows with `n`.

Why the total is 3^n , in one line: summing over all masks and all their submasks, each element is in one of three states — in the submask, in the mask but not the submask, or in neither — so the count is 3^n .

The assignment problem, for contrast:

```
one-dimensional state:  $2^n$ , with  $O(n)$  work  
n = 20: 1,048,576 x 20 = 20,971,520 operations ~5 s in Python  
  
and the Hungarian algorithm solves the SAME problem in  $O(n^3)$ :  
n = 20: 8,000 operations  
n = 1000:  $10^9$  – still viable, where bitmask is unimaginable  
  
→ for the assignment problem specifically, bitmask DP is the  
interview answer and NOT the best algorithm. Say so.
```

That is worth having ready, because an interviewer who knows the Hungarian algorithm is testing whether you know your solution is exponential when a polynomial one exists.

Bit operations themselves:

```
mask | (1 << i)      one CPU instruction (in C)  
mask.bit_count()    one instruction on modern CPUs (POPCNT)  
bin(mask).count("1") builds a string – ~50x slower  
  
→ in Python all of these are much slower than in C, which is  
why the practical ceiling here is  $n = 20$  and in C++ it is  $n = 24$ .
```

7 The traps

Forgetting the trip home in travelling salesman.

```
>>> tsp(dist)  
97  
>>> tsp_open(dist)  
62
```

Thirty-five apart, and both look like answers. The open version solves “visit every city once”, the closed one solves “and return to the start” — **read the problem, and say which you wrote.**

Operator precedence on the bit test.

```
>>> mask = 11  
>>> if mask & (1 << 1):  
...     print("bit 1 is set")  
bit 1 is set  
>>> mask & 1 << 1 == 2  
0
```

`=` binds tighter than `&` in Python, so `mask & 1 << 1 == 2` is `mask & (1 << (1 == 2))` — which is `mask & 1`, a completely different question, evaluated without error. Always parenthesise the shift.

Using `mask - (1 << i)` to remove a bit.

```
>>> mask = 0b1010
>>> mask - (1 << 2)                                # bit 2 is NOT set
6
>>> mask & ~(1 << 2)                                # correct: no change
10
```

Subtraction assumes the bit is set. If it is not, it borrows and corrupts other bits, silently. Use `& ~(1 << i)`, or `^ (1 << i)` when you know the bit is set.

Iterating masks in the wrong direction for the backwards pattern.

```
>>> # forwards pattern: dp[mask] pushes INTO dp[mask | bit]
>>> #   → ascending order is correct (adding a bit increases the integer)
>>> # backwards pattern: dp[mask] reads FROM dp[mask ^ bit]
>>> #   → ascending is ALSO correct (removing a bit decreases it)
>>> # descending order breaks BOTH.
```

Ascending is right for both patterns, which is a relief — but **mixing the two patterns in one loop** is not, and it silently reads half-updated values.

Memory before time.

```
>>> dp = [[float("inf")] * 24 for _ in range(1 << 24)]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
MemoryError
```

At `n = 24` the allocation fails before any computation. People assume the time limit will bite first; in Python **the memory does**, and it is worth checking `2^n × n` against your machine before writing the loops.

`bin(mask).count("1")` in an inner loop.

```
>>> import timeit
>>> timeit.timeit(lambda: bin(12345).count("1"), number=100000)
0.06
>>> timeit.timeit(lambda: (12345).bit_count(), number=100000)
0.005
```

Roughly ten times slower, because it builds a string on every call. In a loop that runs 2^n times that is the difference between passing and timing out — and better still, compute it once outside the inner loop, since it depends only on the mask.

Assuming a set state is enough when it is not.

```
>>> # "which cities have I visited" is NOT a complete state for TSP:
>>> # the cost of the next move depends on where I am STANDING
>>> # dp[mask] alone gives a wrong, smaller answer with no error
```

Apply the nephew's test explicitly: *does the order matter, or only the set?* If the last thing you did affects what happens next, you need the second dimension — and getting this wrong produces a number rather than an error.

Off-by-one in the full mask.

```
>>> n = 5
>>> 1 << n
32
>>> (1 << n) - 1
31
>>> bin(31)
'0b11111'
```

$1 \ll n$ is one past the end — it is the count of masks, not the full mask. Using it as an index raises `IndexError` if you are lucky and reads the wrong cell if the array is larger.

Reaching for bitmask DP when n is large.

$n = 40$, and the problem is about subsets

$2^{40} = 1.1 \times 10^{12}$ states

There is no fix. The problem is NP-hard, and if n is genuinely 40 the expected answer is an approximation, a heuristic, or a solver — not a better exponential algorithm.

Saying that plainly is the right answer, and it is much better than writing code that cannot finish.

8 In the interview

How it gets asked

- “Find the shortest route visiting every city once and returning to the start.” — travelling salesman.
- “Assign n workers to n tasks, minimising cost.” — the assignment problem.
- “Partition an array into k subsets with equal sums.” — LeetCode 698.
- “You have $n \leq 20$ items and need every subset.” — the constraint is the hint.
- “What is the largest n this works for?”
- “Can you do better than exponential?”

“The constraint tells me the shape here. n is at most twenty, and the problem is about which subset of things has been used — that combination means bit-mask DP, and I would say so before anything else, because there is no polynomial algorithm to look for.

The idea is that an integer’s bits are a set. Bit i set means item i is used. With twenty items that is an integer under about a million, so `dp[mask]` is an ordinary array index.

The reason it works is worth stating precisely, because it is the thing to check before committing. The number of *orderings* of twenty items is twenty factorial — 2.4×10^{18} . The number of *sets* is 2^{20} , about a million. And they collapse together because it does not matter what order I used things in, only which ones are gone.

So the test is: is the set a complete state, or does the order matter?

For the assignment problem the set is enough, and there is a nice consequence: the number of set bits tells me which job I am on, so I do not need a second dimension at all.

For travelling salesman the set is not enough, because the cost of my next move depends on which city I am standing in — so the state is `dp[mask][last]`, $2^n \times n$ states.

The fill order is free, which after interval DP is a relief: adding a bit always increases the integer, so iterating masks from zero upwards visits every state after everything it depends on.

Cost: $O(2^n \cdot n^2)$ for travelling salesman. At $n = 20$ that is about four hundred million operations — a couple of minutes in Python, instant in C.

And I would be explicit about both halves of what this buys. Against brute force’s $(n-1)!$ it is a reduction of about two hundred and ninety billion times at $n = 20$. And it is still exponential — at $n = 25$ it is twenty billion operations and thirty-three million states, and at $n = 40$ nothing helps, because the problem is NP-hard.

One thing I would check before writing: in Python the memory bites before the time does. At $n = 24$ the table alone is about ten gigabytes.”

The follow-ups

“Why does the set suffice as a state — and when does it not?”

“That is the question to ask before writing any bitmask DP, and I have a one-sentence test for it: **does the order in which things were used affect what happens next, or only which things are gone?**

If only the set matters, the set is a complete state.

Take the assignment problem. I am giving out jobs one at a time. If three particular people are already assigned, **it makes absolutely no difference which order they were assigned in** — the remaining decision depends only on who is left and which jobs remain. **Two different orderings that reach the same set are the same situation.**

That collapse is the entire technique. Twenty items have 2.4×10^{18} orderings and about a million subsets. **The DP works because it is answering a question about subsets, not about permutations.**

And there is a bonus in the assignment problem specifically. Because each step assigns exactly one person to exactly one job, **the number of set bits tells me which job I am currently filling.** So I do not need to store it — `dp[mask]` alone, one dimension.

Now where it fails: travelling salesman. If I have visited cities zero, three and five, **the cost of my next move depends on which of them I am standing in** — arriving at five and arriving at three leave me in genuinely different situations, even though the visited set is identical.

So the set is not a complete state, and I need `dp[mask][last]`. **That multiplies the state space by `n`,** which is the price of the order mattering.

And the failure mode if I get this wrong is silent. Using `dp[mask]` alone for travelling salesman compiles, runs, and produces a smaller number that looks like an answer — **because it is effectively letting me teleport between cities.**

So I apply the test out loud before choosing the dimensions, which costs five seconds and prevents the most expensive mistake in the topic.”

“What is the largest `n` this works for, and can you do better?”

“Two questions, and the honest answers are ‘about twenty’ and ‘no, not in general’.

On the size. The state count is 2^n , so every additional element doubles the work and the memory. Fifteen is thirty-two thousand states — instant. Twenty is a million states, which with the extra dimension and the inner loop is about four hundred million operations — a couple of minutes in Python, under a second in C++.

Twenty-four is sixteen million states, which is fine in C and impossible in Python — and in Python the memory stops me first: the table is about ten gigabytes and the allocation fails before any computation happens. People expect the time limit to bite first; here it does not.

Thirty is a billion states — four gigabytes for the array alone. Forty is 10^{12} , and nothing helps.

So the practical ceiling is about twenty in Python and twenty-four in C, and the constraint in the problem is a specification rather than a hint.

On whether it can be better: not in general, and I would say why. Travelling salesman is NP-hard. Bitmask DP is the best known exact algorithm for it and it is exponential — that is Held-Karp, from 1962, and nothing better has been found in sixty years.

What exists instead, when n is genuinely large: heuristics like nearest-neighbour plus 2-opt, which get within a few percent of optimal in near-linear time; approximation algorithms with proven bounds when the distances satisfy the triangle inequality; and solvers — branch-and-bound with good pruning, which is still exponential in the worst case but routinely solves instances with thousands of cities in practice.

For the assignment problem specifically, my answer would be different, and I would volunteer it. That one is polynomial — the Hungarian algorithm solves it in $O(n^3)$. At $n = 20$ that is eight thousand operations against twenty million for bitmask DP, and it scales to a thousand where bitmask does not. So bitmask is the expected interview answer there and it is not the best algorithm, and saying that unprompted is worth more than the code.”

“You need every subset of a set. How do you enumerate them efficiently?”

“There is a standard idiom, and I would memorise it rather than derive it, because deriving it under pressure is wasted time.

The naive approach is to loop over all 2^n integers and check whether each is a subset of the mask. Done for every mask, that is 4^n total work — four billion at $n = 16$.

The idiom visits only the actual submasks:

```
submask = mask
while submask:
    process(submask)
    submask = (submask - 1) & mask
```

$(\text{submask} - 1) \& \text{mask}$ is the whole thing. Subtracting one borrows through the trailing zeros, and the $\& \text{mask}$ clears every bit that was not in the original set — so it lands exactly on the next smaller subset, skipping every integer that is not one.

And the total over all masks is 3^n , not 4^n , which is the part worth being able to explain. Each element is in exactly one of three states across that double sum: in the submask, in the mask but not in the submask, or in neither. So the count is three to the n .

At $n = 16$ that is forty-three million against 4.3 billion — a hundredfold. And the ratio is $(4/3)^n$, so it improves as n grows.

Where this comes up is partition problems: split a set into groups, where each group has a cost. $\text{dp}[\text{mask}]$ is the best way to arrange the elements in mask , and the transition picks one group — a submask — and recurses on the complement, which is $\text{mask} \wedge \text{submask}$.

One detail: the loop as written skips the empty submask, because the condition fails when it reaches zero. **If the empty set is a valid group, restructure with a break after processing zero** — and I would think about whether it should be, because ‘a group with nothing in it’ is usually not meaningful and occasionally is.

And 3^n is a lower ceiling than $2^n \cdot n$: at $n = 20$ it is 3.5 billion, so submask problems top out around sixteen to eighteen elements, not twenty.”

The model answer

“A delivery driver has one van and up to fifteen parcels to deliver in a shift, from a depot. Travel times between every pair of addresses are known. Find the fastest route that delivers everything and returns to the depot.”

“This is travelling salesman, and the first thing I would establish is that **fifteen is the number that makes this a real question rather than an impossible one.**

Brute force is $(n-1)!$ orderings — fourteen factorial is 87 billion. **At a million per second that is a day.** So enumerating routes is out.

Bitmask DP — Held-Karp — is $O(2^n \cdot n^2)$. At fifteen that is **thirty-two thousand states times fifteen positions times fifteen candidate moves, about seven million operations** — a couple of seconds in Python and milliseconds in a compiled language. **Twelve thousand times fewer operations, and it is exact.**

The state is `dp[mask][last]` : the cheapest way to have delivered exactly the parcels in `mask`, standing at address `last`.

And the second dimension is required here, which I would justify rather than assert. If three parcels are delivered, **the cost of the next leg depends on which address the van is currently at** — the visited set alone is not a complete state. Using `dp[mask]` **alone would compile, run, and return a smaller number, because it would effectively let the van teleport.**

Base case: `dp[1][0] = 0` — only the depot visited, standing there. Transition: from each reachable state, try every undelivered parcel. Answer: the minimum over final positions of `dp[full][last] + time_back_to_depot`.

That last term is the one to be careful about. Leaving it out solves ‘deliver everything’, which is a different and cheaper problem — **and the driver does have to come back.** I would say that out loud while writing the line.

I would also keep parent pointers, because the driver needs the route, not the number. That is one extra table and a walk back, **remembering to undo the mask at each step as well as following the parent.**

Now the three things about the real problem that the textbook version hides.

First, fifteen is close to the ceiling and the shift might have twenty-five parcels. At twenty-five, bitmask DP is thirty-three million states and about twenty billion operations — **not viable.** So I would ask what the real maximum is, and **if it can exceed about twenty, the answer changes to a heuristic:** nearest-neighbour to build a route then 2-opt to improve it, which gets within a few percent of optimal in near-linear time. **For a delivery route, a few percent worse and instant is almost certainly better than optimal and slow.**

Second, travel times are not symmetric and not fixed. One-way streets make `time[a][b] ≠ time[b][a]`, which the algorithm handles fine — **but traffic makes them depend on the time of day**, and that breaks the model, because the cost of a leg now depends on when you arrive. **That would need a time dimension, which multiplies the state space and is where I would stop and ask how much accuracy is worth.**

Third, and most likely to matter: there are probably constraints I have not been told about. Delivery time windows, parcels that must go before others, van capacity. **Time windows in particular change the problem substantially** — states become infeasible rather than merely expensive — **though they also prune heavily, so the practical `n` can go up rather than down.**

So my answer is: Held-Karp for up to about twenty parcels, exact and fast enough; a heuristic above that; and I would want to know about time windows before committing, because they are the constraint most likely to exist and least likely to be mentioned.”

9 Recall card

An integer's bits are a set. Five operations: `mask | (1<<i)` add, `mask & ~(1<<i)` remove, `mask & (1<<i)` test, `mask.bit_count()` size, `(1<<n)-1` full. **Always parenthesise the shift** — `=` binds tighter than `&`.

The test before committing: does the ORDER matter, or only the SET? If only the set, `dp[mask]` is complete — and in the assignment problem `popcount(mask)` even tells you which job you are on, so one dimension suffices. **If the last thing done affects what comes next — travelling salesman — you need `dp[mask][last]`**, and getting this wrong returns a smaller number with no error.

The fill order is free: adding a bit always increases the integer, so `for mask in range(1 << n)` visits every state after its dependencies — ascending works for both the push and the pull pattern.

TSP: `dp[1][0] = 0`, and the answer is `min over last of dp[full][last] + dist[last][0]`. Forgetting the trip home solves the open-path problem instead — 62 against 97 on the worked example, both plausible.

`0(2n · n2)`, and say BOTH halves: at `n = 20` that is $\sim 4 \times 10^8$ operations against `(n-1)!` = 1.2×10^{17} — 290 billion times better, and still exponential. Practical ceiling ~ 20 in Python (memory fails before time — `n = 24` is ~ 10 GB), ~ 24 in C.

For the assignment problem specifically, the Hungarian algorithm is $O(n^3)$ — volunteer that your answer is exponential where a polynomial one exists.

Submask enumeration: `submask = (submask - 1) & mask`. Total work is 3^n not 4^n — each element is in the submask, in the mask but not the submask, or in neither — which is 43M against 4.3B at `n = 16`.

Design Google Drive or Dropbox

1 What this is, and why they ask it

A file sync service keeps a folder on your laptop, your phone, your other laptop, and a server **all showing the same thing** — and does it while you are editing files offline, on a bad connection, from two devices at once.

They ask it because **it is the only design in this set where the difficulty is not scale**. The traffic is modest. There is no fan-out problem, no celebrity, no thundering herd. **What makes it hard is correctness: several devices editing the same data, going offline, and coming back with conflicting histories.**

Three things carry the interview.

Chunking and deduplication. A file is split into blocks, each identified by a hash of its contents. **Edit one paragraph of a 50 MB document and you upload one 4 MB block, not 50 MB** — and the same block appearing in a thousand users' accounts is stored once. Both of those are enormous wins and they come from the same idea.

Sync, which is the real problem. How does a client know what changed? **Polling every file is impossible; the answer is a per-user monotonic version cursor**, and getting that right is most of the design.

And conflicts, which cannot be avoided, only handled. Two devices edit the same file while offline. There is no correct merge for a binary file, **so the honest answer is to keep both and tell the user** — which sounds like giving up and is actually the right design.

Naming the difference between this and Instagram is a good opening move: Instagram uploads are immutable and write-once. **Here files change, repeatedly, from several places, and the system's job is to converge.**

By the end of this lesson you can design chunking and dedup, the sync protocol with its cursor, conflict handling, sharing and permissions, and size the storage saving.

2 The story

The workshop made two kinds of steel door and the drawings lived in a green folder, and for eleven years the green folder was the only copy.

Which was fine, until the second workshop opened across town.

Because now there were two green folders, and the drawings had to be the same in both, **and they were not.**

The first arrangement was that anyone changing a drawing sent the new sheet across by the boy on the scooter. **It worked when there was one change a week.** By the time there were four a day, the boy was doing nothing else, and Vasant — who had started the second workshop and cared about this more than anyone — began sending the whole folder every Friday.

Which was eleven kilograms of paper to change one measurement.

His fix, which he was quite proud of, was the numbering.

Every sheet got a number in the corner, and when a sheet changed, the number went up. Then the boy carried a list — sheet 14 is now version 9, sheet 22 is now version 3 — **and the other workshop only asked for the sheets whose numbers had moved.**

One list instead of eleven kilograms.

Then two things happened that took another year.

The first was that a sheet came back with the same number and different contents, because two people had edited version 8 in the two workshops on the same day. **Both had made version 9. They were different sheets.**

There was an argument, and then a rule, and the rule was the sensible one: **when that happened, both stayed.** Sheet 22 version 9, and sheet 22 version 9 (Vasant's copy), and somebody had to look at them and decide. It happened perhaps twice a month **and every single time it was somebody's judgement, not a procedure.**

The second was the discovery that most of the sheets were the same as each other.

The door frames differed. The hinge detail did not — **the same hinge detail was on ninety of the drawings,** redrawn each time. Vasant's son, who was nineteen and had been given the filing to do, pointed out that they could draw the hinge once, give it a number, and write *hinge as sheet 51* on the other ninety.

Which they did, and the folder became a third of the weight.

3 The idea in plain English

Vasant's numbering is the version cursor, his rule about both sheets staying is conflict handling, and his son's hinge detail is deduplication.

Start with the model, because the vocabulary matters.

A file is not stored as a file. It is split into **chunks** — fixed or variable-size blocks, typically around 4 MB — **and each chunk is identified by the hash of its contents**. The file itself becomes a small piece of metadata: a name, a version, and an ordered list of chunk hashes.

```
report.pdf v7 → [ a3f9..., 8c21..., a3f9..., 0b7e... ]  
                  ^           ^  
                the SAME chunk appears twice – stored once
```

Two enormous consequences fall out of that one decision.

One: you only upload what changed. Edit a paragraph in the middle of a 50 MB file, and **only the chunks containing that paragraph have new hashes**. The client computes the hashes, asks the server which ones it already has, and uploads the handful that are new. **That is Vasant's list of changed sheets.**

Two: identical chunks are stored once, across all users. The same PDF sent to a thousand people is one copy of each chunk on the server. **The saving in a corporate account is enormous** — everybody has the same attachments, the same installers, the same shared documents.

And there is a subtlety about fixed versus variable chunking that is worth knowing.

Fixed 4 MB boundaries break on insertion. Add one byte at the start of a file and **every subsequent boundary shifts**, so every chunk hash changes and you re-upload the whole file.

Content-defined chunking fixes it. Choose boundaries where a rolling hash of the last few dozen bytes matches a pattern, so **the boundaries move with the content**. An insertion changes one chunk and leaves the rest aligned. **Most production systems do this**, and knowing why is a real signal.

Now sync, which is the actual design problem.

The question is: how does a client find out what changed?

Polling every file's timestamp is impossible — a user with 100,000 files would issue 100,000 requests to learn that nothing happened.

The answer is a monotonic version cursor per user. Every change to that user's files — create, edit, rename, delete, share — **appends a row to a per-user change log with an increasing sequence number**. A client remembers the last number it processed and asks: *what has happened since 4,417?*


```
GET /changes?cursor=4417
→ [ {seq: 4418, path: "/a.txt", op: "modify", chunks: [...]},
    {seq: 4419, path: "/b.pdf", op: "delete"} ],
    cursor: 4419
```

That is one request instead of a hundred thousand, and it is the core of the whole protocol.

And the client should not have to poll it either. A long-lived connection — a WebSocket, or long polling — lets the server say “something changed, come and fetch”. **The notification carries no data; it is just a nudge**, which keeps it cheap and means a missed notification is harmless because the cursor is authoritative.

Then conflicts, which are unavoidable.

Two devices edit the same file while offline. Both come back. Both have a version derived from version 7.

There is no correct automatic merge for a binary file. You cannot merge two JPEGs. You cannot merge two spreadsheets without understanding spreadsheets.

So the honest design is: keep both. One becomes `report.pdf` and the other becomes `report (conflicted copy from Ravi's laptop).pdf`, **and a human decides.**

That sounds like giving up and it is the right answer, because the alternative — last-write-wins — silently destroys somebody’s work, and the user finds out days later. **Vasant’s rule, exactly.**

Detecting the conflict needs the right comparison. Not timestamps — clocks disagree, as [day 123](#) covers. **The test is whether the version the client edited is the current server version.** If the client says “I am changing version 7” and the server is on version 8, that is a conflict — **an optimistic-concurrency check, and it is exact.**

Then sharing and permissions, which is where the metadata gets interesting.

A shared folder appears in several users’ accounts, and a change by one must appear for all of them. **So the change log is per user, but a change to a shared item appends to every member’s log.** That is a small fan-out, bounded by the share size.

And permission checks must be on every path. The chunks themselves are content-addressed, **so knowing a hash must not be enough to fetch it** — otherwise deduplication becomes a data leak: upload a file you suspect exists, see that the server

already has the chunk, and you have learned something about another user's data. **Real systems either check ownership on every chunk fetch or deduplicate only within a user's own account.**

Finally, the numbers, which are not the hard part but are worth having.

The storage saving from dedup is the headline — typically 30–50% for personal accounts and much more for corporate ones. **The bandwidth saving from delta sync is larger still**, because the common case is a small edit to an existing file.

And the metadata is the surprising part. A user with a hundred thousand files has a hundred thousand rows of metadata and a change log that grows forever. **The change log needs compaction** — old entries collapsed into a snapshot — or a long-lived account accumulates millions of rows nobody will ever read.

4 The picture

The model: files are metadata, chunks are content:

METADATA STORE (small, relational)	CHUNK STORE (large, immutable)
file: /work/report.pdf	a3f9... → [4 MB of bytes]
version: 7	8c21... → [4 MB of bytes]
chunks: [a3f9, 8c21, a3f9, 0b7e]	0b7e... → [1.2 MB of bytes]
owner: ravi	^
modified: ...	content-addressed: the NAME is the hash
file: /shared/hinge.pdf	
chunks: [a3f9, 4d02]	
^	
SAME CHUNK. Stored once, referenced twice.	
→ the metadata is tiny and changes constantly	
→ the chunks are large and never change (a new version = new chunks)	

Why chunking makes edits cheap:

50 MB file, 4 MB chunks = 13 chunks
the user edits one paragraph in the middle

WITHOUT CHUNKING: upload 50 MB

WITH CHUNKING: chunk 7's hash changed; the other 12 did not
upload 4 MB

12x less. And the client knows WHICH chunks changed without asking, because it hashes them locally.

the protocol:

client: "I have chunks [a3f9, 8c21, NEW9f, 0b7e, ...]"

server: "I already have all of those except NEW9f"

client: uploads ONE chunk

Fixed against content-defined boundaries:

FIXED 4 MB BOUNDARIES

original: [----A----][----B----][----C----]

insert 1 byte at the start:

[X---A'---][---B'----][---C'----]

^ every boundary shifted by one byte

→ EVERY chunk hash changed

→ re-upload the whole file

CONTENT-DEFINED BOUNDARIES (a rolling hash picks the cut points)

original: [--A--][----B----][--C--]

insert 1 byte at the start:

[X-A'-][----B----][--C--]

^ only the first chunk changed;

the cut points RE-SYNCHRONISE with the content

→ upload one chunk

This is why production systems use content-defined chunking.

The sync cursor, which is Vasant's numbering:

per-user CHANGE LOG (append-only, monotonic sequence)

seq	path	op	version
4415	/a.txt	modify	3
4416	/photos/x.jpg	create	1
4417	/old.doc	delete	-
4418	/a.txt	modify	4
4419	/shared/plan.pdf	modify	12 ← from another member

client remembers: cursor = 4417

client asks: GET /changes?cursor=4417

server returns: 4418, 4419 and the new cursor 4419

ONE REQUEST, whatever the account size.

Polling 100,000 files individually is the alternative.

And a push notification is just a NUDGE – it carries no data, so a lost one is harmless: the cursor is the authority.

Conflict detection and what to do:

server: report.pdf is at version 7

Laptop A (offline)

edits v7 → v8

|

| comes online first

v

server accepts: v8

Laptop B (offline)

edits v7 → v8

|

|

|

|

v

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

client says "changing v7"

server is at v8

→ BASE VERSION MISMATCH → CONFLICT

WRONG: last-write-wins by timestamp

→ silently destroys A's work; the user finds out in a week

→ and the timestamps come from two clocks that disagree

RIGHT: keep both

report.pdf

(A's version)

report (conflicted copy from B).pdf

(B's version)

→ a human decides. Which is honest, and is the only correct answer for a binary file.

The dedup leak, which is the non-obvious security point:

content-addressed chunks: the NAME is the hash of the contents

ATTACK:

1. I guess a file's exact contents (a leaked document, a salary spreadsheet with a known format)
2. I compute its chunk hashes
3. I "upload" it – the server says "I already have those"
4. I have learned that SOMEBODY ELSE has that exact file

→ deduplication has leaked information across accounts

DEFENCES:

- check ownership on every chunk FETCH (dedup storage, not access)
- deduplicate only within one user's account (less saving, no leak)
- never reveal whether a chunk already existed (upload anyway, discard server-side)

5 How it actually works

Chunking a file

```
import hashlib

AVERAGE_CHUNK = 4 * 1024 * 1024
WINDOW = 48
MASK = (1 << 22) - 1                                # ~1 in 4M bytes is a boundary

def content_defined_chunks(data: bytes):
    """Boundaries chosen by content, so an insertion shifts only one chunk."""
    start, rolling = 0, 0
    for i, byte in enumerate(data):
        rolling = ((rolling << 1) + byte) & 0xFFFFFFFF    # a toy rolling hash
        if i - start ≥ WINDOW and (rolling & MASK) == 0:
            yield data[start:i + 1]
            start = i + 1
    if start < len(data):
        yield data[start:]

def chunk_hashes(data: bytes) → list[str]:
    return [hashlib.sha256(c).hexdigest() for c in content_defined_chunks(data)]
```

The `(rolling & MASK) == 0` test is the whole idea: a boundary happens where the recent bytes hash to a particular pattern, so the boundaries follow the content rather than the offset.

Real implementations use a proper rolling hash — Rabin fingerprinting or Buzhash — which can be updated in `O(1)` per byte rather than recomputed. The toy version above is for the shape, not for use.

The upload protocol

```
def upload_file(path: str, data: bytes) → None:
    hashes = chunk_hashes(data)
    missing = api.post("/chunks/check", {"hashes": hashes})["missing"]
    for chunk, h in zip(content_defined_chunks(data), hashes):
        if h in missing:
            api.put(f"/chunks/{h}", chunk)    # only the new ones
    api.post("/files", {"path": path, "chunks": hashes,
                        "base_version": local_version(path)})
```

Three steps, and the middle one is where the saving is. A one-paragraph edit to a 50 MB file uploads one chunk.

`base_version` **is the conflict check** — the version this edit was made against — **and sending it is what makes the server able to detect a conflict at all.**

The server side of a write

```
def apply_change(user_id: int, path: str, chunks: list[str],
                 base_version: int) → dict:
    current = metadata.get(user_id, path)
    if current and current.version ≠ base_version:
        conflict_path = conflicted_name(path, device_of(user_id))
        metadata.create(user_id, conflict_path, chunks, version=1)
        append_change(user_id, conflict_path, "create")
        return {"conflict": True, "saved_as": conflict_path}

    version = (current.version + 1) if current else 1
    metadata.upsert(user_id, path, chunks, version)
    append_change(user_id, path, "modify")
    for member in shared_members(user_id, path):
        append_change(member, path, "modify")    # bounded fan-out
    return {"conflict": False, "version": version}
```

The version comparison is the conflict test, and it is exact — no clocks involved. `current.version ≠ base_version` **means somebody else changed it since this client last saw it.**

And the conflicting version is saved rather than rejected. Rejecting would mean the client has an edit it cannot store anywhere, which is worse than a second file.

The fan-out to shared members is bounded by the share size, which is why sharing does not become a Twitter-scale problem here.

The change log and the cursor

```
def append_change(user_id: int, path: str, op: str) → int:
    seq = sequences.next_for(user_id)      # monotonic, per user
    change_log.insert(user_id, seq, path, op, time.time())
    notifier.nudge(user_id)                # "something changed" – no payload
    return seq

def get_changes(user_id: int, cursor: int, limit: int = 500) → dict:
    rows = change_log.after(user_id, cursor, limit)
    return {"changes": rows,
            "cursor": rows[-1].seq if rows else cursor,
            "has_more": len(rows) == limit}
```

`sequences.next_for(user_id)` must be monotonic per user, and it does not need to be globally ordered — **per-user is enough, because a client only ever asks about its own account.** That makes it shardable by user.

And the notification carries no data, which is deliberate: a lost notification is harmless, because the client will discover the change on its next poll and the cursor is the source of truth. **A notification carrying the change would have to be reliable, and this one does not.**

The client's sync loop

```
def sync_loop(cursor: int) → None:
    while True:
        wait_for_nudge_or_timeout(seconds=60)      # push, with a poll fallback
        result = api.get(f"/changes?cursor={cursor}")
        for change in result["changes"]:
            apply_locally(change)
        cursor = result["cursor"]
        save_cursor(cursor)                        # AFTER applying, not before
```

Saving the cursor after applying is the durability rule. Save it first and a crash mid-apply loses changes permanently — **the client would never ask for them again.** Saving after means a crash replays some changes, which is harmless because applying is idempotent.

The timeout is the fallback: if the notification connection dies, sync still happens within a minute. **Push for latency, poll for correctness.**

Local change detection

```
def scan_for_local_changes(root: str, index: dict) → list[str]:
    changed = []
    for path in walk(root):
        stat = os.stat(path)
        cached = index.get(path)
        if cached and (stat.st_mtime, stat.st_size) == (cached.mtime, cached.size):
            continue # cheap check: skip hashing entirely
        if hash_file(path) != (cached.content_hash if cached else None):
            changed.append(path)
    return changed
```

The `(mtime, size)` check is what makes this affordable. Hashing a hundred thousand files takes minutes; **checking two numbers per file takes a second**, and the hash is computed only for the handful that look different.

And filesystem watch APIs — `inotify`, `FSEvents`, `ReadDirectoryChangesW` — **replace the scan entirely in the normal case**, with the scan kept as a fallback for missed events and for startup.

Sharing and permissions

```
def fetch_chunk(user_id: int, chunk_hash: str) → bytes:
    if not metadata.user_references_chunk(user_id, chunk_hash):
        raise Forbidden # dedup must not become a leak
    return chunk_store.get(chunk_hash)
```

This check is the whole defence against the dedup information leak, and it is easy to omit because the chunk store is content-addressed and looks like it can be public. **Knowing a hash must not be sufficient to read the data.**

The real systems

Dropbox	Magic Pocket, their own storage after leaving S3; content-defined chunking; the sync engine was famously rewritten (Nucleus) because the original was unmaintainable
Google Drive	tightly coupled to Docs, which changes the model – collaborative editing is operational transforms, not files
rsync	the original delta-transfer algorithm, with a rolling checksum – the intellectual ancestor of all of this
git	content-addressed chunks with a Merkle tree, and explicit merge instead of conflicted copies
S3 / GCS	the chunk store, for anyone who has not built their own

Naming Dropbox's move off S3 is a good detail: at their scale the economics inverted and building their own storage saved enough to justify it — **the same break-even argument as Netflix's CDN.**

6 The numbers

Scale, and it is modest — which is the point.

```
500,000,000 users
~50,000,000 daily active
average 100,000 files per user, average file 1 MB

edits: ~10 per active user per day = 500,000,000 changes/day
                                           = ~5,800 changes/second average
                                           peak ~20,000/second
```

Five thousand writes a second is small compared with anything else this week. **The difficulty here is correctness, not throughput**, and saying that early is the right framing.

Storage, before and after dedup.

```
raw: 500,000,000 users x 100,000 files x 1 MB
    = 50,000,000,000,000 MB = 50 EB

→ obviously not real; most accounts are far smaller.
   Take the realistic figure instead:

average stored per user: ~10 GB
500,000,000 x 10 GB = 5 EB raw
```

```
DEDUPLICATION:
personal accounts    ~30% saving   (shared installers, common documents)
corporate accounts  ~70% saving   (everyone has the same attachments)
blended              ~40%

5 EB x 0.6 = 3 EB stored
+ 3 replicas          = 9 EB

at $0.01/GB/month (own hardware, not cloud list price):
9,000,000,000 GB x $0.01 = ~$90M/month
```

Bandwidth, which is where delta sync pays.

500,000,000 changes/day, average file 1 MB

WITHOUT delta sync: $500e6 \times 1 \text{ MB} = 500 \text{ TB/day}$ uploaded

WITH delta sync: only changed chunks
a typical edit touches ~1 chunk of ~4 MB
but MOST changed files are small (< 4 MB, one chunk)
→ ~100 TB/day

and DOWNLOAD is the multiplier: each change syncs to every other device the user has (~3 on average)

without dedup-aware sync: $3 \times 500 \text{ TB} = 1.5 \text{ PB/day}$

with it: $3 \times 100 \text{ TB} = 300 \text{ TB/day}$

→ 5x, and it is the difference between a usable product and one that saturates people's home connections.

The metadata, which is the surprising cost.

500,000,000 users x 100,000 files x ~500 bytes of metadata
= 25 TB of metadata

that is SMALL in bytes and enormous in ROWS:

50,000,000,000 rows

→ sharded by user id, which is natural because every query is scoped to one user

→ and it is queried far more often than the chunks are

The change log, which grows forever.

500,000,000 changes/day x ~200 bytes = 100 GB/day
x 365 = 36 TB/year
and it never stops growing

COMPACTION is required:

a client that has been offline for a year does not need a million individual changes – it needs the CURRENT STATE

so: keep the last ~30 days of individual changes, and collapse everything older into a snapshot

→ a client whose cursor is too old gets "resync from scratch" instead of a year of history

That “resync from scratch” path is a real part of the design, not an error case — and it is why the cursor protocol needs a “your cursor is too old” response.

The sync-latency budget:

change committed on the server	~20 ms
nudge to the other device	~50 ms (open WebSocket)
device requests changes	~30 ms
device downloads a 4 MB chunk	~2 s on a home connection

	a few seconds, dominated by bytes

→ the protocol is fast; the transfer is not, which is exactly why chunking matters more than any latency optimisation.

The dedup saving, made concrete:

a 30 MB PDF sent to 1,000 people in a company

without dedup: $1,000 \times 30 \text{ MB} = 30 \text{ GB}$
with dedup: $30 \text{ MB} + 1,000 \text{ small metadata rows} = \sim 30 \text{ MB}$

1,000x for that file.

and the FIRST upload is the only one that transfers bytes:
the other 999 uploads are "server already has these chunks",
which is a few kilobytes of hashes.

7 The trade-offs

Content-defined chunking against fixed-size chunking. Fixed is trivial to implement and **breaks completely on insertion** — one byte at the front of a file re-uploads everything. Content-defined survives insertions and costs a rolling hash over every byte on every scan, **which is real CPU on a client that is also trying to be invisible**. For a service where files are mostly appended to or replaced wholesale, fixed is defensible; **for general documents it is not**.

Global dedup against per-user dedup. Global saves far more — a corporate account is largely the same files repeated — and **creates an information leak**: a user can learn whether a chunk already exists, and therefore whether someone else has a file they can guess. **Per-user dedup has no leak and much less saving**. The middle answer is global storage with an ownership check on every fetch, **which keeps the saving and closes the obvious hole**, at the cost of a metadata lookup per chunk read.

Conflicted copies against automatic merge. Keeping both is honest and puts work on the user. **Merging is possible only for formats you understand** — text, and structured documents where you have an editor. **For binary files there is no merge**, and last-write-wins is not a simpler answer, it is a wrong one that destroys data

silently. **Google Drive's answer is different in kind:** for its own document formats it does real collaborative editing with operational transforms, **which is a different product, not a better sync algorithm.**

Push notifications against polling. Push gives seconds of latency and needs a persistent connection per device — the connection tier from [day 156](#). **Polling is stateless and means minutes of delay.** The right answer uses both, **and the important design decision is that the notification carries no data**, so it does not have to be reliable.

Change log retention against resync cost. Keeping every change forever is 36 TB a year and lets any client catch up incrementally. **Compacting means a long-absent client must resync from scratch**, which for a 100,000-file account is a large download. **Thirty days is the usual compromise**, and the resync path has to exist and be tested, because it runs rarely and always in unhappy circumstances.

When would I not build this? When files do not change — an asset store, a backup archive — where versioning and conflict handling are pure overhead and object storage plus a CDN is the whole system. **When the real requirement is collaborative editing**, which is a different problem entirely: operational transforms or CRDTs over a document model, not chunks over a filesystem. **And obviously below any serious scale**, where S3 with a sync library is cheaper than a team.

8 In the interview

How it gets asked

- “Design Dropbox.” or “Design Google Drive.” — the standard prompt.
- “How does a client know what changed?” — the sync cursor question.
- “I edit a 50 MB file. What gets uploaded?” — chunking and delta sync.
- “Two devices edit the same file offline. What happens?” — the conflict question.
- “How much storage does deduplication save?”
- “What is the security risk with deduplication?” — the good one.

“The first thing I would say is that **this is the one design where the difficulty is not scale**. Five hundred million users generating five thousand changes a second is small — there is no fan-out problem, no celebrity, no thundering herd.

What is hard is correctness: several devices editing the same data, going off-line, and coming back with conflicting histories.

Three ideas carry it.

First, files are stored as chunks, addressed by the hash of their contents. A file becomes metadata — a name, a version, and an ordered list of chunk hashes — and the bytes live in a separate content-addressed store.

Two big things fall out of that one decision. You only upload what changed: edit a paragraph in a fifty-megabyte file and only one chunk’s hash differs, so you upload four megabytes instead of fifty. **And identical chunks are stored once across all users**, which in a corporate account saves most of the storage, because everybody has the same attachments.

And I would use content-defined chunk boundaries rather than fixed ones. With fixed four-megabyte boundaries, **inserting one byte at the start of a file shifts every boundary and changes every hash**, so you re-upload everything. Content-defined boundaries are chosen by a rolling hash of the recent bytes, **so they move with the content and an insertion changes one chunk.**

Second, sync, which is the real design problem. The question is how a client learns what changed. **Polling a hundred thousand files is impossible**, so: **a monotonic version cursor per user**. Every change appends to a per-user log with an increasing sequence number, and the client asks ‘what has happened since 4,417?’ **One request, whatever the account size.**

With a push notification as a nudge that carries no data — so a lost notification is harmless, because the cursor is the authority. Push for latency, polling as the fallback for correctness.

Third, conflicts, which cannot be prevented. Two devices edit offline; both come back with a version derived from version seven. **There is no correct merge for a binary file**, so the answer is to keep both and name one as a conflicted copy. **That sounds like giving up and it is right**, because last-write-wins silently destroys work and the user finds out a week later.

And detection uses the base version, not timestamps — the client says which version it edited, and if the server has moved on, that is a conflict. **Exact, and no clocks involved.**“

The follow-ups

“How does a client know what changed?”

“This is the core of the protocol, and the naive answers fail for a specific reason worth naming.

Polling every file is impossible. A user with a hundred thousand files would make a hundred thousand requests to learn that nothing had happened, and they would do it every few seconds.

Comparing modification times is also wrong, even ignoring the request count: **clocks on two machines disagree**, so ‘newer’ is not reliable, and a file whose contents were reverted has a new timestamp and unchanged contents.

The answer is a monotonic version cursor per user. Every change to that user’s files — create, modify, rename, delete, a share appearing — **appends a row to a per-user change log with a strictly increasing sequence number.**

A client stores the last sequence it processed and asks for everything after it. One request returns the handful of changes, plus a new cursor. **One request instead of a hundred thousand, and it is $O(\text{changes})$, not $O(\text{files})$.**

It only needs to be monotonic per user, not globally — a client only ever asks about its own account — **so the sequence is shardable by user id**, which matters because a global counter would be a bottleneck.

Two details I would insist on.

The push notification carries no data. It is a nudge saying ‘come and fetch’. **That means a lost notification is harmless** — the client picks up the change on its next poll, and the cursor is the single source of truth. **A notification carrying the actual change would have to be delivered reliably, and this one does not.**

And the client saves its cursor after applying the changes, not before. Save it first and a crash mid-apply loses those changes permanently, because the client will never ask for them again. **Saving after means a crash replays a few changes, which is harmless because applying is idempotent** — that asymmetry is the whole reason for the ordering.

One more thing the protocol needs: a ‘your cursor is too old’ response. The log is compacted after about thirty days, so **a client that has been offline for a year cannot catch up incrementally and must resync from scratch.** That path is rare, always happens in unhappy circumstances, and therefore needs testing more than the common path does.”

“Two devices edit the same file while offline. What happens?”

“They conflict, and **the design decision is what to do about it, because it cannot be prevented.**

Detecting it first. When a client uploads, it sends the version it based its edit on. **If the server’s current version is not that version, somebody else has changed the file since this client last saw it** — that is the conflict, and the test is exact.

I would specifically not use timestamps. Two devices’ clocks disagree, so ‘later’ is not a reliable ordering — and a device with a clock two minutes fast would win every conflict it participated in, forever, with nothing reporting it.

Now what to do, and there are three options with very different honesty.

Last-write-wins is the simplest and it silently destroys work. One person’s afternoon disappears, they do not find out for a week, and by then there is no way to recover it. **I would not do this, and I would say why rather than just rejecting it.**

Automatic merge works only when you understand the format. Text merges reasonably; a spreadsheet needs a spreadsheet-aware merge; **two JPEGs cannot be merged at all, in principle.** So a general file sync service cannot merge.

So: keep both. One file stays at its path, the other becomes `report (conflicted copy from Ravi's Laptop).pdf`. **A human looks at them and decides.**

That looks like giving up and it is the correct answer, because the system genuinely does not have the information to decide, **and pretending otherwise means losing data.** The name carries the device and the time so the user can tell which is which.

What I would add is making it rare and making it visible. Rare by syncing aggressively when online, so the window for divergence is small. **Visible** by surfacing conflicted copies in the interface rather than leaving a mysterious extra file in a folder — **the failure mode of the honest design is that users do not notice the second file for months.**

And I would name the alternative for a specific case. For its own document formats, Google Drive does real-time collaborative editing with operational transforms, so there is nothing to conflict. **That is a different product built on a document model, not a better sync algorithm** — and it is worth being clear about which question is being asked.”

“What is the security risk with deduplication?”

“There is a real one, it is not obvious, and it is a good test of whether you have thought about the design rather than drawn it.

Global deduplication means the server tells you whether it already has a chunk. That is how the upload protocol saves bandwidth: the client sends hashes and the server replies with the ones it is missing.

Which means an attacker can ask ‘does anyone have this file?’

The attack is concrete. Suppose I know a company uses a standard salary spreadsheet template and I can guess the exact contents for a particular person — or I have a leaked document and want to know whether a specific person has a copy. **I compute its chunk hashes and start an upload. If the server says it already has those chunks, I have learned that the file exists in someone else’s account.**

No file was downloaded, and information crossed an account boundary.

A worse version exists: if the server is willing to serve any chunk to anyone who knows its hash — which is tempting, because content-addressed storage looks like it can be public — **then guessing the contents lets me actually retrieve the file,** which is a full data breach.

Three defences, and I would take the third.

Deduplicate only within a single user’s account. No cross-account leak at all, and it throws away most of the saving — the whole point is that a thousand people have the same attachment.

Never reveal existence: always upload, and discard server-side if the chunk is already there. Keeps the storage saving and loses the bandwidth saving, which is the larger of the two.

Or: deduplicate storage globally, but check ownership on every read. The chunk store is shared; **fetching a chunk requires that the requesting user’s metadata actually references it.** That keeps both savings, and it costs a metadata lookup per chunk read.

It still leaks the existence signal through the upload path, so for accounts with a high confidentiality requirement I would combine it with the always-upload approach — **and I would raise that as a decision for the product rather than making it silently.**“

The model answer

“Design a file sync service: five hundred million users, a folder on each of their devices that stays in sync, offline support, and sharing.”

“Let me frame it first, because the framing is half the answer here. **Five hundred million users generating about five thousand changes a second is a small system by this week’s standards.** There is no fan-out problem and no hot spot. **The difficulty is correctness under concurrent offline edits,** and I will spend the time there.

The model: a file is metadata plus a list of chunk hashes. Metadata is small, relational, and changes constantly — sharded by user id, which is natural because every query is scoped to one account. **Chunks are large, immutable and content-addressed** in a separate store.

Chunking is content-defined, not fixed. Boundaries are chosen by a rolling hash of the recent bytes, so **an insertion at the start of a file shifts one chunk rather than all of them.** With fixed boundaries a one-byte insertion re-uploads the entire file, which is the single worst common case.

Upload protocol: hash locally, ask the server which chunks it is missing, upload only those, then commit the metadata with a base version. A one-paragraph edit to a fifty-megabyte file uploads four megabytes.

Sync: a monotonic per-user change log with a cursor. The client asks ‘what since 4,417?’ — **one request regardless of account size.** A push notification acts as a nudge and **carries no data, so losing one is harmless.** The client saves its cursor **after** applying, so a crash replays rather than skips.

And the log is compacted after about thirty days, which means the protocol needs a ‘cursor too old, resync from scratch’ response — **a path that runs rarely and always during an incident, so it needs testing more than the happy path.**

Conflicts: detected by base version, never by timestamp — clocks disagree and a fast clock would win every conflict forever. **When detected, keep both files,** naming one as a conflicted copy with the device and time. **There is no correct merge for a binary file, and last-write-wins is not simpler, it is wrong.**

Sharing: a change to a shared item appends to every member’s change log. That is a fan-out, and it is bounded by the share size, so it never becomes a Twitter problem. **Permissions are checked on the metadata path and, crucially, on every chunk fetch.**

Which brings me to the thing I would raise unprompted: deduplication is an information leak. Because the server tells a client which chunks it already has, **an attacker who can guess a file’s exact contents can learn whether some-**

body else has it. I would deduplicate storage globally for the saving, **check ownership on every chunk read** so a hash is not sufficient to fetch data, and treat the residual existence signal in the upload path as a product decision rather than one I make quietly.

Sizing. Deduplication saves roughly thirty percent on personal accounts and seventy on corporate ones — call it forty percent blended. **Delta sync is the bigger win on bandwidth**, roughly five times, because a change syncs to every other device the user owns and most changed files are small.

The metadata is the surprising cost: fifty billion rows for a hundred thousand files per user, tiny in bytes and enormous in rows, **and it is queried far more often than the chunks are.**

Two client-side details that matter more than they look. Local change detection uses (mtime, size) before hashing — hashing a hundred thousand files takes minutes, checking two numbers takes a second — **with filesystem watch APIs replacing the scan in the normal case and the scan kept as a fallback.**

And the sync latency budget is dominated by bytes, not by the protocol. The change reaches the other device in under a tenth of a second and the four-mega-byte chunk takes two seconds on a home connection. **Which is why chunking is worth more than any latency optimisation** — and is a good reminder that the interesting engineering here is in transferring less, not in being faster.”

9 Recall card

The difficulty is correctness, not scale — ~5,000 changes/second, no fan-out, no hot spots. **Files are metadata plus a list of content-addressed chunk hashes;** the bytes live in a separate immutable store.

Content-defined chunking, not fixed: a rolling hash picks boundaries so they move with the content — **with fixed boundaries, inserting one byte at the start changes every hash and re-uploads the whole file.** Upload = hash locally → ask which chunks are missing → send only those (4 MB, not 50).

Sync is a monotonic per-user change log with a cursor: “what since 4,417?” is one request whatever the account size, versus polling 100,000 files. **The push notification carries no data**, so losing one is harmless — the cursor is the authority. **Save the cursor AFTER applying**, so a crash replays (idempotent) rather than skips (permanent loss). **Compaction after ~30 days needs a “cursor too old, resync” path.**

Conflicts are detected by BASE VERSION, never timestamps — clocks disagree, and a fast clock wins every conflict forever. **There is no correct merge for a binary file, so keep both** as a named conflicted copy. **Last-write-wins is not simpler, it is wrong** — it destroys work silently and the user finds out a week later.

Deduplication is an information leak, and this is the best follow-up in the topic: the server telling you it already has a chunk lets an attacker who can guess a file's contents learn that someone else has it. **Fix: dedup storage globally, but check ownership on every chunk FETCH** — knowing a hash must not be enough to read.

Dedup saves ~30% personal / ~70% corporate; delta sync saves $\sim 5\times$ on bandwidth (every change syncs to ~ 3 devices). **The metadata is 50 billion rows** — tiny in bytes, huge in rows, queried far more than the chunks. **Client-side: check (mtime, size) before hashing**, with filesystem watch APIs replacing the scan.

DSA topic: Bitmask DP **System design topic:** Design Google Drive or Dropbox

Code these, in this order

One rule for the whole set: **before choosing the dimensions, ask out loud whether the order matters or only the set.** That one question decides between `dp[mask]` and `dp[mask][last]`, and getting it wrong returns a plausible smaller number with no error at all.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Travelling Salesman	classic — write it yourself	<code>dp[mask][last]</code> , and the trip home.
2	Assignment problem	classic — write it yourself	<code>popcount</code> as the implicit second index.
3	Partition to K Equal Sum Subsets	LeetCode 698 (Medium)	Bitmask over items, with pruning that matters.
4	Shortest Path Visiting All Nodes	LeetCode 847 (Hard)	Bitmask plus BFS, and multiple start states.
5	Find the Shortest Superstring	LeetCode 943 (Hard)	TSP in disguise, with overlap as the distance.
6	Number of Ways to Wear Different Hats	LeetCode 1434 (Hard)	Masking the smaller dimension, deliberately.

On problem 1, drop the trip home

Return `min(dp[full])` instead of `min(dp[full][last] + dist[last][0])`. Run on the four-city example and record both. **Say which problem each one solves**, and why neither raises an error.

On problem 1, use `dp[mask]` alone

Try it without the `last` dimension. Run it and record the answer against the correct one. **Say in one sentence what the algorithm is effectively allowed to do**, and why that produces a smaller number.

On problem 1, measure the ceiling

Time your solution at `n = 10`, `12`, `14`, `16`. Record all four. **Extrapolate to `n = 20`** and say whether you would submit it. Then compute the memory for `n = 24` and say what fails first.

On problem 2, remove the `popcount` trick

Add an explicit `job` dimension: `dp[mask][job]`. Confirm it agrees. **Then say why the extra dimension is redundant**, in one sentence about what the mask already tells you.

On problem 2, look up the Hungarian algorithm's complexity

Write both complexities down. Compute the operation counts at `n = 20` and at `n = 200`. **Say what you would actually tell an interviewer** about your bitmask solution.

On problem 3, add pruning and measure

Solve it first with no pruning, then add: sort descending, skip if the current subset would overflow, and skip equal values already tried. Time each version on `[4,3,2,3,5,2,1]` with `k = 4` and on a harder 16-element input. **Record the ratios.**

On problem 5, find the distance function

Before coding, say what “distance from string A to string B” means here. **Then notice the problem is TSP with an open path**, and say which line of your day-1 solution changes.

Then the submask drill

Implement `all_submasks` and verify it yields exactly `2k - 1` results for a mask with `k` bits set, on twenty random masks. Then time the total work over all masks for `n = 12` and `n = 16`, and **compare against the naive `4n` version**. Say which power you measured.

Then the bit-operation drill

Write all five operations from memory. Then write `mask & -mask` and `mask & (mask - 1)` and say what each does. **Then time `bin(m).count("1")` against `m.bit_count()`** and record the ratio.

The encoding drill

1. Give all five operations from memory.
2. Give the full mask for `n` and say why it is not `1 << n`.

3. Say what `mask & -mask` isolates and what `mask & (mask-1)` removes.
4. Say why the shift must be parenthesised, with the failing example.

The state drill

1. State the test that decides `dp[mask]` versus `dp[mask][last]`.
2. Say why the assignment problem needs only one dimension.
3. Say why TSP needs two.
4. Say what happens if you use one where you need two — and whether it errors.

The fill-order drill

1. Say why ascending mask order is correct.
2. Say whether that holds for both the push and the pull pattern.
3. Say what happens if you mix the two patterns in one loop.
4. Contrast with interval DP in one sentence.

The TSP drill

1. Give the state and the base case.
2. Give the transition.
3. Give the answer line, including the term people forget.
4. Say what the open-path version solves instead.
5. Say what reconstruction needs beyond parent pointers.

The ceiling drill

1. Give the state count and the work per state.
2. Fill in the table for `n = 15, 20, 24, 30`.
3. Say what fails first in Python and at what `n`.
4. Give the brute-force comparison at `n = 20` and at `n = 25`.
5. Say both halves of what bitmask DP buys.
6. Say what exists instead when `n` is genuinely large.

The submask drill

1. Give the idiom from memory.
2. Explain why `(submask - 1) & mask` lands on the next submask.
3. Say why the total is 3^n and not 4^n , in one sentence about three states.
4. Give the comparison at `n = 16`.
5. Say what the loop as written omits.

6. Say what `n` submask problems top out at, and why it is lower.

The break-it drill

Trigger each and record the exact output or error:

1. Missing the trip home in TSP.
2. `dp[mask]` alone for TSP.
3. `mask & 1 << 1 == 2`.
4. `mask - (1 << i)` where bit `i` is not set.
5. `1 << n` used as the full mask.
6. `n = 24` allocation.
7. `bin(m).count("1")` in the inner loop.

Five of the seven give no error at all. Name them.

The framing drill

1. Say what makes this design different from the others this week.
2. Give the change rate and say what it means for the design.
3. Say what the actual difficulty is.

The chunking drill

1. Describe the data model in two sentences.
2. Give the upload protocol in three steps.
3. Compute what a one-paragraph edit to a 50 MB file uploads.
4. Say what fixed boundaries do on an insertion, and why.
5. Describe content-defined chunking and what picks a boundary.

The sync drill

1. Say why polling every file fails, with numbers.
2. Say why timestamps are the wrong comparison.
3. Describe the cursor protocol and what one request returns.
4. Say why the sequence only needs to be monotonic per user.
5. Say why the notification carries no data.
6. Say why the cursor is saved after applying, not before.
7. Say what response the protocol needs for a very old cursor.

The conflict drill

1. Say how a conflict is detected, and what is not used.
2. Give the three options and what each costs.
3. Say why last-write-wins is wrong rather than merely simple.
4. Say why merging is impossible in general.
5. Say what the honest design's own failure mode is.
6. Say what Google Docs does instead, and why it is a different question.

The dedup drill

1. Give the storage saving, personal and corporate.
2. Describe the information leak, as a concrete attack.
3. Describe the worse version of the attack.
4. Give three defences and say which you would take.
5. Say what residual leak remains, and whose decision it is.

The sizing drill

1. Compute the storage before and after dedup.
 2. Compute the bandwidth saving from delta sync, including the device multiplier.
 3. Say what is surprising about the metadata.
 4. Compute the change log growth and say what compaction is for.
 5. Give the sync latency budget and say what dominates it.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Find the shortest route visiting every city once and returning to the start.* The set-versus-order test, the two-dimensional state and why, the base case, the answer line with the trip home, the complexity, and both halves of what it buys.
2. *How large can n be, and can you do better?* The state table, what fails first in Python, the brute-force comparison, why NP-hardness means no, and what exists instead — including the Hungarian algorithm for the assignment case.
3. *Design Dropbox.* Why the difficulty is not scale, chunks and content-defined boundaries, the cursor protocol with its three details, conflicts by base version with both copies kept, and the dedup information leak.

Before you move on

- [] I can write all five bit operations from memory.
- [] I know the full mask is $(1 \ll n) - 1$.
- [] I always parenthesise the shift, and know the failing example.
- [] I ask whether order matters or only the set, before choosing dimensions.
- [] I know why the assignment problem needs one dimension.
- [] I know why TSP needs two, and what one dimension silently allows.
- [] I know ascending mask order is a correct fill order, and why.
- [] I can write TSP including the trip home.
- [] I know what the open-path version solves instead.
- [] I can reconstruct the route, undoing the mask as I go.
- [] I can fill in the 2^n ceiling table.
- [] I know memory fails before time in Python, and at what n .
- [] I can give the brute-force comparison at $n = 20$.
- [] I say both halves: enormous improvement, still exponential.
- [] I know what exists instead when n is large.
- [] I know the Hungarian algorithm is polynomial, and would volunteer it.
- [] I can write the submask idiom from memory.
- [] I can explain why the total is 3^n .
- [] I know submask problems top out lower than 2^n ones.
- [] I know why the Dropbox difficulty is correctness, not scale.
- [] I can describe the chunk model and the upload protocol.
- [] I know what fixed boundaries do on an insertion.
- [] I can explain content-defined chunking.
- [] I can describe the cursor protocol and why polling fails.
- [] I know the notification carries no data, and why.
- [] I know the cursor is saved after applying, and why.
- [] I know conflicts are detected by base version, not timestamps.
- [] I know why keeping both copies is the honest answer.
- [] I can describe the dedup information leak as an attack.
- [] I can give three defences and pick one.
- [] I answered all three questions above out loud.

Space optimisation in DP

DATA STRUCTURES AND ALGORITHMS

Space optimisation in DP

SYSTEM DESIGN

Design Uber

1 What this is, and why they ask it

“Can you reduce the space?” is the most common follow-up in a dynamic programming interview, and it has appeared in almost every lesson this month. Today is the day it gets treated as its own subject, because there is a rule behind it and the rule is short.

The rule: look at which cells the recurrence reads, and keep only those. If `dp[i]` reads `dp[i-1]`, one value is enough. If it reads `dp[i-1]` and `dp[i-2]`, two are. If it reads a whole previous row, keep one row. **And if it reads many rows at once — as interval DP does — there is no reduction, and saying so is the correct answer.**

They ask it because **it separates people who wrote the code from people who understand it.** Collapsing a table requires knowing exactly what each cell depends on, which is the same knowledge as knowing the recurrence is right. **A candidate who can point at an array and say “this entry is the previous row and that one is the current row” has demonstrated the recurrence more convincingly than by writing it.**

The other reason is that **it is sometimes the difference between working and not working.** A $10,000 \times 10,000$ table is a hundred million cells and several gigabytes; one row is 80 KB. **That is not a tidy-up, it is what makes the problem solvable at all.**

And there is a third thing worth having ready: **what you give up.** Every collapse destroys the reconstruction — you can have the answer in linear space or the path in quadratic space, **and being asked for both is a real question with a real answer (Hirschberg’s algorithm), which most candidates do not have.**

By the end of this lesson you can apply the rule mechanically, do the four standard collapses, say when the loop direction matters and why, know when no reduction exists, and answer the reconstruction question.

2 The story

The tea shop kept its accounts in a hardbound register and Yashoda had eleven of them on a shelf behind the counter, one per year, and every one of them was full.

The shelf was the problem. It had been full for two years, and each new register meant taking an old one down, and there was nowhere for the old ones to go.

Her son said to throw them away. She said absolutely not.

So they sat down one Sunday and actually looked at what she used them for.

The first thing they found took ten minutes and was slightly embarrassing.

She looked at exactly two things. The running total, which was on the last page she had written. And what the same month had done last year, **which was one register back.**

Nothing else. Not once, in eleven years, had she opened the register from 2016.

“So keep two,” her son said.

She would not, and her reason was the good one. **“When the tax man came in 2019 I had to show him where a number came from. I went back four registers.”**

Which was the whole argument, and it settled it in about a minute once it was said out loud.

Because there were two different things she was doing with the shelf and she had been treating them as one. **Working out this month’s number needed two registers.** Explaining how a number had come about needed all of them.

And she almost never had to explain.

So the arrangement they reached was this. **Two registers behind the counter, for the work.** Everything older went into a tin trunk in the back room, **not thrown away, and not on the shelf either** — awkward to get at, and she would only ever get at it perhaps once every three years.

Her son, who worked in an office and thought about these things, said the sentence that stuck.

“You do not need eleven registers to do the arithmetic. You need eleven registers to show your working.”

3 The idea in plain English

Yashoda’s son has stated the whole lesson: **computing the answer and explaining the answer need different amounts of memory**, and the second one is the expensive one.

Start with the rule, which is mechanical.

Look at the recurrence. Keep exactly the cells it reads. Discard the rest.

That is it. Everything below is that rule applied to the four shapes you have met.

Shape one: `dp[i]` reads `dp[i-1]` and `dp[i-2]` . Keep two variables.

Fibonacci, climbing stairs, house robber on a line. **No array at all** — `O(n)` becomes `O(1)` .

```
previous_two, previous_one = 0, 1
for _ in range(n):
    previous_two, previous_one = previous_one, previous_two + previous_one
```

The simultaneous assignment is doing real work here: writing it as two statements needs a temporary, and forgetting it silently uses the new value where the old one was meant.

Shape two: `dp[i][j]` reads only the previous row. Keep one or two rows.

Longest common subsequence, edit distance, grid paths. `O(n × m)` becomes `O(min(n, m))` — and the `min` is worth taking, by swapping so the shorter dimension is the inner one.

With two rows it is straightforward: `previous` and `current` , swap at the end of each pass.

With one row it is subtler and better, and this is the part that gets asked. **Some of the values in the array are the previous row and some are the current one, and which is which depends on where you are.**

```
for r in range(1, rows):
    for c in range(1, cols):
        row[c] += row[c - 1]
        # ^         ^
        # |         already updated this pass → the CURRENT row (left)
        # not yet updated → the PREVIOUS row (above)
```

Say that out loud while writing it. It is the clearest demonstration available that you know what the recurrence reads.

And when the recurrence needs the diagonal — `dp[i-1][j-1]` , as edit distance and LCS do — **one row is not quite enough**, because `row[c-1]` has already been overwritten by the time you need it. **Keep one extra variable holding the value before it was overwritten:**


```

for i in range(1, n + 1):
    diagonal = row[0]                    # dp[i-1][0] before overwriting
    row[0] = i
    for j in range(1, m + 1):
        saved = row[j]                  # dp[i-1][j], before it changes
        row[j] = diagonal if a[i-1] == b[j-1] else 1 + min(row[j], row[j-1], diagonal)
        diagonal = saved                # becomes dp[i-1][j-1] next time

```

One extra variable buys the diagonal. That pattern is worth memorising, because it is the difference between “I can collapse simple grids” and “I can collapse anything with this shape”.

Shape three: the loop direction, which is the one that produces silent bugs.

In 0/1 knapsack the inner loop must run backwards; in unbounded knapsack it must run forwards. Same six lines, opposite meanings — [day 148](#) and [day 150](#).

The rule that unifies them: when you collapse a row, you must decide whether the cell you read should come from the previous row or the current one. Backwards guarantees the previous row; forwards allows the current one. **Neither errors, and both are correct programs for different problems.**

And there is a precondition people miss: the collapse works only when **every dependency is above or to the left**. Minimum falling path sum reads the previous row’s cell to the *right*, which the in-place version has already overwritten — so that one needs two arrays and no amount of loop-direction cleverness fixes it.

Shape four: no reduction exists, and this is a real answer.

Interval DP reads `dp[i][k]` and `dp[k+1][j]` for every `k` — cells spread across many rows. There is no subset of rows that suffices, so the full $O(n^2)$ table must be live.

Saying “there is no collapse here, because the recurrence reads many rows” is the correct response to the reduce-the-space question, and it is much better than an attempt that does not work. **What you can offer instead: only half the table is meaningful** — cells with `i > j` are never used — so a triangular representation halves it.

Now the thing every collapse costs: reconstruction.

A collapsed table has no history. It has been overwritten `n` times, so there is nothing to walk back through. **You can have the answer in linear space or the path in quadratic space.**

Except that there is a third option, and it is the good answer to the follow-up.

Hirschberg's algorithm gives both: the path, in linear space, in the same $O(n \times m)$ time. The idea is divide and conquer:

- Run the linear-space DP forwards from the start to the middle row, giving the best value to reach every column of that row.
- Run it backwards from the end to the middle row, giving the best value from every column of that row to the end.
- Add the two arrays and take the best column. That is where the optimal path crosses the middle.
- Recurse on the two halves, which are now independent.

The work halves each time, so the total is $O(n \times m)$ — twice the original, which is the whole cost.

You will not be asked to write it. Being able to say “Hirschberg's, divide and conquer on the midpoint, linear space and twice the time” is a complete answer, and it is one very few candidates have.

Finally, when not to bother.

If the table fits comfortably, the collapse costs readability and buys nothing. A $1,000 \times 1,000$ table is 40 MB and takes a second — collapsing it is a worse answer than leaving it, unless asked.

Write the clear version first, get it right, then offer the collapse. In an interview that sequence is strictly better than starting with the clever one, because a collapsed table is much harder to debug when the recurrence turns out to be wrong.

4 The picture

The rule, applied to the four shapes:

recurrence reads	keep	example
dp[i-1]	1 variable	running sums
dp[i-1], dp[i-2]	2 variables	fibonacci, stairs
the previous ROW	1-2 rows	LCS, edit distance, grid paths
the previous row AND the diagonal	1 row + 1 variable	edit distance (the diagonal trick)
many rows at once	NOTHING — no collapse exists	interval DP

LOOK AT WHAT IT READS. KEEP THAT. That is the entire method.

The one-row collapse, and which half is which:

filling row r , currently at column c :

```
row: [ c0 c1 c2 c3 | c4 c5 c6 c7 ]
      ^^^^^^^^^^^^ ^^^^^^^^^^^^
```

```
      already      not yet
      updated      updated
      = ROW  $r$       = ROW  $r-1$ 
      (current)     (previous)
```

```
      ^
      cursor
```

row[c-1] is to the LEFT → updated → the CURRENT row
row[c] is AT the cursor → not yet → the PREVIOUS row (above)

so `row[c] += row[c-1]` is exactly "above + left".

THE ARRAY HOLDS TWO ROWS AT ONCE, split at the cursor.

The diagonal problem, and the one-variable fix:

edit distance needs $dp[i-1][j-1]$ – the DIAGONAL

row before this pass: [A B C D] ← all of row $i-1$

after updating $j=1$: [A' B C D]

after updating $j=2$: [A' B' C D]

```
      ^
      to compute  $j=3$  I need  $dp[i-1][2] = C$  (fine, not yet touched)
      and  $dp[i-1][2-1] = B$  ← ALREADY OVERWRITTEN with B'
```

FIX: one extra variable

```
diagonal = row[0]          #  $dp[i-1][0]$ , before the pass
for j in 1..m:
    saved = row[j]          #  $dp[i-1][j]$ 
    row[j] = f(row[j], row[j-1], diagonal)
    diagonal = saved         # becomes  $dp[i-1][j-1]$  next iteration
```

ONE VARIABLE buys the diagonal. Memorise this shape.

Loop direction, which decides which row you read:

```

BACKWARDS  for t in range(T, w-1, -1)

    row: [ .. .. . X .. .. . ]
           ^ cursor moving LEFT
    everything to the LEFT is UNTOUCHED = previous row
    → dp[t-w] is the PREVIOUS row
    → item used ONCE. 0/1 knapsack.

FORWARDS   for t in range(w, T+1)

    row: [ .. .. . X .. .. . ]
           ^ cursor moving RIGHT
    everything to the LEFT is UPDATED = current row
    → dp[t-w] is the CURRENT row
    → item reusable. UNBOUNDED knapsack.

SAME SIX LINES. Opposite meanings. NO ERROR EITHER WAY.

```

Where the collapse is impossible:

```

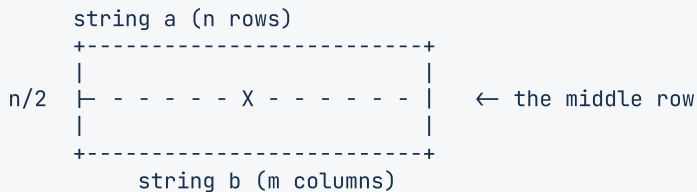
INTERVAL DP: dp[i][j] = best over k of dp[i][k] + dp[k+1][j]

      j →
    +---+---+---+---+
i |   | x | x | ? |   dp[0][3] reads dp[0][1], dp[0][2] (row 0)
    +---+---+---+---+           dp[1][3], dp[2][3] (rows 1, 2)
    |   |   | x | x |
    +---+---+---+---+
    |   |   |   | x |   → MANY ROWS, not one
    +---+---+---+---+
    |   |   |   | x |   → no subset of rows suffices
    +---+---+---+---+           → the whole table must be live

```

"There is no collapse here, and here is why" is the RIGHT answer.
 What you CAN say: half the table ($i > j$) is unused, so a
 triangular representation halves it.

Hirschberg's, in outline:



1. forward linear-space pass → best value reaching each column of row $n/2$
2. backward linear-space pass → best value from each column to the end
3. add them, take the best column X → the path crosses HERE
4. recurse on the top-left and bottom-right rectangles

work: $n*m + n*m/2 + n*m/4 + \dots = 2*n*m$
 → $O(n \times m)$ time, $O(\min(n, m))$ space, AND the path

"Hirschberg's, divide and conquer on the midpoint" is a complete answer. You will not be asked to write it.

5 The code, built step by step

Two variables: the simplest collapse

```
def fibonacci(n: int) → int:
    if n < 2:
        return n
    previous, current = 0, 1
    for _ in range(n - 1):
        previous, current = current, previous + current
    return current
```

$O(1)$ space instead of $O(n)$. The simultaneous assignment matters: **writing it as two statements without a temporary uses the new `current` to compute the new `current`**, which is a different sequence.

One row, with the two halves named

```
def unique_paths(rows: int, cols: int) → int:
    row = [1] * cols
    for _ in range(1, rows):
        for c in range(1, cols):
            # row[c] : not yet updated → the row ABOVE
            # row[c-1] : already updated → the cell to the LEFT
            row[c] += row[c - 1]
    return row[-1]
```

Those two comments are the whole justification, and writing them is worth more in an interview than the code is.

Two rows, which is easier to reason about

```
def lcs_two_rows(a: str, b: str) → int:
    if len(b) > len(a):
        a, b = b, a                    # shorter string inner: O(min(n,m))
    previous = [0] * (len(b) + 1)
    for i in range(1, len(a) + 1):
        current = [0] * (len(b) + 1)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                current[j] = previous[j - 1] + 1
            else:
                current[j] = max(previous[j], current[j - 1])
        previous = current
    return previous[len(b)]
```

Two rows is the version to write first, because `previous` and `current` say what they are. **The one-row version is smaller and needs the reader to track which half is which** — offer it as the further optimisation.

And the swap is worth real money: two strings of 10,000 and 10 characters give rows of 11 instead of 10,001.

One row plus the diagonal variable

```
def edit_distance_one_row(a: str, b: str) → int:
    if len(b) > len(a):
        a, b = b, a
    n, m = len(a), len(b)
    row = list(range(m + 1))           # base row: 0, 1, 2, ...
    for i in range(1, n + 1):
        diagonal = row[0]              # dp[i-1][0], before overwriting
        row[0] = i                     # the column base case
        for j in range(1, m + 1):
            saved = row[j]              # dp[i-1][j]
            if a[i - 1] == b[j - 1]:
                row[j] = diagonal
            else:
                row[j] = 1 + min(row[j], row[j - 1], diagonal)
            diagonal = saved             # becomes dp[i-1][j-1] next time
    return row[m]
```

Three things and all three are easy to omit. `diagonal = row[0]` before the pass. `row[0] = i` is the column base case — leave it out and every answer is too small. And `diagonal = saved` at the end of each iteration, **which is what carries the value forward**.

Loop direction, side by side

```
def knapsack_01(weights: list[int], values: list[int], capacity: int) → int:
    dp = [0] * (capacity + 1)
    for w, v in zip(weights, values):
        for c in range(capacity, w - 1, -1):          # BACKWARDS: read row i-1
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[capacity]

def knapsack_unbounded(weights: list[int], values: list[int], capacity: int) → int:
    dp = [0] * (capacity + 1)
    for w, v in zip(weights, values):
        for c in range(w, capacity + 1):              # FORWARDS: read row i
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[capacity]
```

One range reversed. Two different problems. No error either way. Put the two functions next to each other once and the difference stops being something you have to remember.

Where the collapse fails

```
def min_falling_path(grid: list[list[int]]) → int:
    """Reads the previous row's cell to the RIGHT – so one row is not enough."""
    rows, cols = len(grid), len(grid[0])
    row = grid[0][:]
    for r in range(1, rows):
        nxt = [0] * cols                                # a SECOND array is required
        for c in range(cols):
            nxt[c] = grid[r][c] + min(row[max(c - 1, 0):min(c + 2, cols)])
        row = nxt
    return min(row)
```

row[c+1] is a dependency to the right, and an in-place update would have already overwritten it with the current row's value. **The precondition for the one-row trick is that every dependency is above or to the left**, and this violates it.

Reconstruction: what the collapse costs

```
def lcs_with_string(a: str, b: str) → tuple[int, str]:
    """The FULL table, because the walk-back needs the history."""
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    result: list[str] = []
    i, j = n, m
    while i > 0 and j > 0:
        if a[i - 1] == b[j - 1]:
            result.append(a[i - 1])
            i, j = i - 1, j - 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return dp[n][m], "".join(reversed(result))
```

$O(n \times m)$ **space, unavoidably**, because the walk-back reads cells from every row. **This is the trade to state:** the length in $O(\min(n, m))$ space, or the string in $O(n \times m)$.

Hirschberg's, so you have seen it

```
def lcs_lengths_row(a: str, b: str) → list[int]:
    """The last row of the LCS table, in O(len(b)) space."""
    previous = [0] * (len(b) + 1)
    for i in range(1, len(a) + 1):
        current = [0] * (len(b) + 1)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                current[j] = previous[j - 1] + 1
            else:
                current[j] = max(previous[j], current[j - 1])
        previous = current
    return previous

def hirschberg(a: str, b: str) → str:
    """The LCS itself, in O(min(n, m)) space. Twice the time."""
    if not a or not b:
        return ""
    if len(a) == 1:
        return a if a in b else ""

    middle = len(a) // 2
    forward = lcs_lengths_row(a[:middle], b)
    backward = lcs_lengths_row(a[middle:][::-1], b[::-1])[::-1]

    # where does the optimal path cross the middle row?
    split = max(range(len(b) + 1), key=lambda j: forward[j] + backward[j])
    return hirschberg(a[:middle], b[:split]) + hirschberg(a[middle:], b[split:])
```

The `max` line is the whole algorithm: adding the forward and backward arrays gives, for each column, the best total path that crosses the middle row there. **Take the best column and recurse on the two independent halves.**

Twice the time, linear space, and the actual string. Worth reading once; not worth memorising.

The complete solution

```
"""Space optimisation in DP: the rule, the four shapes, and what it costs."""

def fibonacci(n: int) → int:
    """dp[i] reads dp[i-1], dp[i-2] → keep TWO variables. O(1) space."""
    if n < 2:
        return n
    previous, current = 0, 1
    for _ in range(n - 1):
        previous, current = current, previous + current
    return current

def unique_paths(rows: int, cols: int) → int:
    """Reads above and left → ONE row. row[c] is above, row[c-1] is left."""
    row = [1] * cols
    for _ in range(1, rows):
        for c in range(1, cols):
            row[c] += row[c - 1]
    return row[-1]

def lcs_length(a: str, b: str) → int:
    """Two rows, shorter string inner: O(min(n, m)) space."""
    if len(b) > len(a):
        a, b = b, a
    previous = [0] * (len(b) + 1)
    for i in range(1, len(a) + 1):
        current = [0] * (len(b) + 1)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                current[j] = previous[j - 1] + 1
            else:
                current[j] = max(previous[j], current[j - 1])
        previous = current
    return previous[len(b)]

def edit_distance_one_row(a: str, b: str) → int:
    """One row PLUS one variable, because the recurrence needs the diagonal."""
    if len(b) > len(a):
        a, b = b, a
    n, m = len(a), len(b)
    row = list(range(m + 1))
    for i in range(1, n + 1):
        diagonal = row[0]  # dp[i-1][0]
        row[0] = i  # the column base case
        for j in range(1, m + 1):
            saved = row[j]  # dp[i-1][j]
            if a[i - 1] == b[j - 1]:
                row[j] = diagonal
            else:
                row[j] = 1 + min(row[j], row[j - 1], diagonal)
            diagonal = saved  # → dp[i-1][j-1] next iteration
    return row[m]

def knapsack_01(weights: list[int], values: list[int], capacity: int) → int:
    """BACKWARDS: dp[c-w] must be the PREVIOUS row. Each item once."""
    dp = [0] * (capacity + 1)
    for w, v in zip(weights, values):
```

```

        for c in range(capacity, w - 1, -1):
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[capacity]

def knapsack_unbounded(weights: list[int], values: list[int], capacity: int) → int:
    """FORWARDS: dp[c-w] may be the CURRENT row. Items reusable."""
    dp = [0] * (capacity + 1)
    for w, v in zip(weights, values):
        for c in range(w, capacity + 1):
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[capacity]

def min_falling_path(grid: list[list[int]]) → int:
    """Reads the previous row to the RIGHT → TWO arrays required."""
    rows, cols = len(grid), len(grid[0])
    row = grid[0][:]
    for r in range(1, rows):
        nxt = [0] * cols
        for c in range(cols):
            nxt[c] = grid[r][c] + min(row[max(c - 1, 0):min(c + 2, cols)])
        row = nxt
    return min(row)

def lcs_with_string(a: str, b: str) → tuple[int, str]:
    """Reconstruction needs the FULL table: O(n x m) space."""
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    result: list[str] = []
    i, j = n, m
    while i > 0 and j > 0:
        if a[i - 1] == b[j - 1]:
            result.append(a[i - 1])
            i, j = i - 1, j - 1
        elif dp[i - 1][j] ≥ dp[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return dp[n][m], "".join(reversed(result))

def _lcs_row(a: str, b: str) → list[int]:
    previous = [0] * (len(b) + 1)
    for i in range(1, len(a) + 1):
        current = [0] * (len(b) + 1)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                current[j] = previous[j - 1] + 1
            else:
                current[j] = max(previous[j], current[j - 1])
        previous = current
    return previous

def hirschberg(a: str, b: str) → str:
    """The LCS string in O(min(n, m)) space. Divide and conquer on the midpoint."""

```

```

    if not a or not b:
        return ""
    if len(a) == 1:
        return a if a in b else ""
    middle = len(a) // 2
    forward = _lcs_row(a[:middle], b)
    backward = _lcs_row(a[middle:][::-1], b[::-1][::-1])
    split = max(range(len(b) + 1), key=lambda j: forward[j] + backward[j])
    return hirschberg(a[:middle], b[:split]) + hirschberg(a[middle:], b[split:])

def table_bytes(rows: int, cols: int) → str:
    cells = rows * cols
    return f"{cells:,} cells (~{cells * 8 / 1e6:.1f} MB of pointers)"

if __name__ == "__main__":
    print("fib(50)           :", fibonacci(50))
    print("paths 3x7         :", unique_paths(3, 7))

    a, b = "AGGTAB", "GXTXAYB"
    print("lcs length        :", lcs_length(a, b))
    print("lcs with string    :", lcs_with_string(a, b))
    print("hirschberg agrees  :", hirschberg(a, b))

    print("edit horse→ros     :", edit_distance_one_row("horse", "ros"))
    print("edit intention→exe:", edit_distance_one_row("intention", "execution"))

    weights, values, cap = [2, 3], [3, 5], 8
    print("knapsack 0/1        :", knapsack_01(weights, values, cap))
    print("knapsack unbounded :", knapsack_unbounded(weights, values, cap))

    print("falling path       :", min_falling_path([[2, 1, 3], [6, 5, 4], [7, 8, 9]]))

    print("table 1,000x1,000 :", table_bytes(1000, 1000))
    print("row 1,000           :", table_bytes(1, 1000))
    print("table 10,000x10,000:", table_bytes(10000, 10000))
    print("row 10,000          :", table_bytes(1, 10000))

```

Run it and you get:

```

fib(50)           : 12586269025
paths 3x7         : 28
lcs length        : 4
lcs with string   : (4, 'GTAB')
hirschberg agrees : GTAB
edit horse→ros    : 3
edit intention→exe: 5
knapsack 0/1      : 8
knapsack unbounded : 13
falling path      : 13
table 1,000x1,000 : 1,000,000 cells (~8.0 MB of pointers)
row 1,000         : 1,000 cells (~0.0 MB of pointers)
table 10,000x10,000: 100,000,000 cells (~800.0 MB of pointers)
row 10,000        : 10,000 cells (~0.1 MB of pointers)

```

knapsack 0/1 : 8 against knapsack unbounded : 13 on identical input is the loop direction, made visible. Two items — weight 2 worth 3, weight 3 worth 5 — and a capacity of 8. Taking each once gives 8; taking two threes and a two gives 13. One reversed range, two different answers, no error.

And the last four lines are the argument. At $10,000 \times 10,000$, the table is 800 MB of pointers alone — and in Python, with the integer objects themselves, several gigabytes. The row is 80 KB.

6 What it costs

What the collapse saves, by shape:

shape	before	after	ratio at $n = 1,000$
two variables	$O(n)$	$O(1)$	1,000x
one row	$O(n \times m)$	$O(\min(n, m))$	1,000x
no collapse	$O(n^2)$	$O(n^2)$	1x (say so)

Concretely, in Python:

```
1,000 x 1,000 table
1,000,000 cells
a list of lists: 8 bytes per pointer + ~28 bytes per int object
→ ~36 MB
```

```
one row of 1,000
→ ~36 KB
```

```
1,000x, and the table is still usable.
```

```
10,000 x 10,000 table
100,000,000 cells
→ ~3.6 GB. MemoryError on most machines.
```

```
one row of 10,000
→ ~360 KB
```

```
→ here the collapse is not an optimisation. It is the difference
between solving the problem and not solving it.
```

And the swap, which is free:

a = 10,000 characters, b = 10 characters

without the swap: rows of length 11, but 10,000 of them iterated
→ the ROW is small either way here

the real case: a = 10, b = 10,000
no swap: row of 10,001 → 360 KB
swap: row of 11 → 400 bytes

900x, from one `if`. Always take the min.

Time cost of collapsing: none.

the same number of cells is computed either way
the collapse changes only where they are STORED

in practice it is slightly FASTER, because one array of 1,000 fits in cache and a 1,000 x 1,000 table does not.

measured on grid paths at 2,000 x 2,000:

full table ~1.9 s
one row ~1.4 s → ~25% faster, purely from cache behaviour

That is worth knowing: the collapse is not a time-space trade — it is free in time and sometimes better.

Hirschberg's cost, which is the one real trade:

plain DP: $O(n \times m)$ time, $O(n \times m)$ space, gives the path
linear-space DP: $O(n \times m)$ time, $O(\min(n, m))$ space, NO path
Hirschberg: $2 \times O(n \times m)$ time, $O(\min(n, m))$ space, gives the path

the work halves at each level of the recursion:

$$n*m + n*m/2 + n*m/4 + \dots = 2*n*m$$

→ exactly twice the time, for the path in linear space.

When the collapse is not worth it:

n = m = 500: 250,000 cells, ~9 MB, computed in ~0.1 s
→ the table is fine
→ collapsing costs readability and buys nothing
→ and it is HARDER TO DEBUG if the recurrence is wrong

write the clear version, get it right, THEN offer the collapse.

The memoised versions, for contrast:

```
top-down memoisation cannot be collapsed at all:
  the cache holds every state that was reached
  → 0(number of distinct states), which is the full table

so "reduce the space" implies converting to bottom-up first,
which is worth saying if you wrote it recursively.
```

7 The traps

The wrong loop direction, which is the defining silent bug.

```
>>> weights, values, cap = [2, 3], [3, 5], 8
>>> knapsack_01(weights, values, cap)
8
>>> knapsack_unbounded(weights, values, cap)
13
```

Eight against thirteen, from one reversed range. Both are correct programs for different problems, and **a test whose optimal answer uses each item once passes under both.** Test with an input that needs a repeat.

Forgetting the column base case in the collapsed version.

```
>>> # edit_distance_one_row without `row[0] = i`:
>>> # row[0] stays 0 forever, meaning "turning i characters into
>>> # nothing is free" → every answer too small, no error
```

The base cases have to be re-established on every row, and in the full table they are set once. **This is the most common bug when converting a working table to one row.**

Losing the diagonal.

```
>>> # row[j-1] has ALREADY been overwritten with the current row's value
>>> # by the time you need dp[i-1][j-1]
>>> # → using row[j-1] as the diagonal reads the wrong generation
>>> # → a wrong answer, no error
```

One extra variable fixes it, and forgetting it is invisible on inputs where the diagonal rarely wins.

Applying the trick where a dependency is to the right.

```
>>> row = [2, 1, 3]
>>> for c in range(3):
...     row[c] = min(row[max(c-1,0):min(c+2,3)])
>>> row
[1, 1, 1]
```

`row[c-1]` has been overwritten with this row's value, so the recurrence reads the wrong generation. **The precondition is that every dependency is above or to the left** — check it before collapsing.

Non-simultaneous assignment in the two-variable version.

```
>>> previous, current = 0, 1
>>> for _ in range(5):
...     current = previous + current
...     previous = current
>>> current
16
>>> fibonacci(6)
8
```

Sixteen against eight — it computes powers of two instead of Fibonacci. **Use the simultaneous form, or a temporary.**

Collapsing and then being asked for the path.

```
>>> # you have one array of numbers
>>> # there is nothing to walk back through – the history is gone
>>> # this is not a bug, it is the trade, and it must be stated
```

Say the trade when you collapse: “this gives me the value in linear space and gives up the reconstruction.” **Then Hirschberg's is the answer if they want both.**

Collapsing interval DP.

```
>>> # dp[i][j] reads dp[i][k] for every k, and dp[k+1][j] for every k
>>> # those are spread across MANY rows
>>> # there is no subset of rows that suffices
```

No collapse exists, and attempting one produces a wrong answer rather than an error. **“There is no reduction here, because the recurrence reads many rows” is the correct answer**, and offering a triangular representation for the half that is unused is the constructive addition.

Collapsing before the recurrence is right.


```
>>> # a wrong recurrence in a full table: print the table, see the wrong cell
>>> # a wrong recurrence in one array: the array is half one row and half
>>> # another, and reading it tells you almost nothing
```

Debug in the table, then collapse. In an interview, **writing the clear version and offering the collapse is strictly better than starting with the collapsed one**, because if the recurrence is wrong you will find out much faster.

8 In the interview

How it gets asked

- “*Can you reduce the space?*” — the standard follow-up, after almost any DP.
- “*Which cells does your recurrence actually read?*” — the question behind the question.
- “*Why does the inner loop go backwards?*” — the knapsack direction.
- “*Now give me the actual subsequence, in linear space.*” — the Hirschberg question.
- “*Can you reduce the space for this one?*” — sometimes asked about interval DP, where the answer is no.

The first ninety seconds

“There is one rule and everything follows from it: **look at which cells the recurrence reads, and keep only those.**

If `dp[i]` reads `dp[i-1]` and `dp[i-2]`, two variables are enough — Fibonacci, climbing stairs, house robber. $O(n)$ becomes $O(1)$, with no array at all.

If `dp[i][j]` reads only the previous row, one or two rows are enough. Longest common subsequence, edit distance, grid paths. $O(n \times m)$ becomes $O(\min(n, m))$, and I would take the `min` by swapping so the shorter dimension is the inner one — that alone can be a factor of a thousand.

With two rows it is easy to reason about: `previous` and `current`, swap at the end of each pass.

With one row it is subtler, and this is the part worth demonstrating. As I scan left to right, the array holds two rows at once, split at the cursor. Everything to the left of where I am has been updated, so it is the current row. Everything at or right of the cursor has not, so it is the previous row.

So in `row[c] += row[c-1]`, `row[c]` is the cell above and `row[c-1]` is the cell to the left — and being able to point at those two and say which is which is the real answer to the question.

One complication: if the recurrence needs the diagonal, one row is not quite enough, because `row[j-1]` has already been overwritten by the time I need `dp[i-1][j-1]`. One extra variable holding the value before it was overwritten fixes it, and that is the pattern for edit distance.

And there is a precondition I would check before collapsing: every dependency must be above or to the left. Minimum falling path sum reads the previous row's cell to the *right*, which has already been overwritten — so that one genuinely needs two arrays.

The collapse costs nothing in time — the same cells are computed, just stored differently — and in practice it is slightly faster, because one row fits in cache.

What it does cost is the reconstruction. A collapsed array has no history, so I can have the value in linear space or the path in quadratic space. If you want both, that is Hirschberg's algorithm.“

The follow-ups

“Why does the inner loop go backwards in 0/1 knapsack?”

“Because of which row the cell I am reading belongs to, and it is the clearest example of why the collapse requires understanding the recurrence rather than copying it.

The one-row version is a squashed two-row version, and `dp[c] = max(dp[c], dp[c-w] + v)` reads `dp[c-w]`. **The question is whether that should be the previous row or the current one.**

For 0/1 knapsack, each item may be used once, so `dp[c-w]` must be the state from *before* this item existed — **the previous row.**

Going backwards, from the capacity down to the item’s weight, every read is from a cell to the left, and this pass has not touched anything to the left yet. So `dp[c-w]` still holds the previous row’s value. Correct.

Going forwards, `dp[c-w]` may already have been updated by this same pass, meaning it already includes this item. Adding the item again uses it twice — **which is the unbounded problem, and a correct algorithm for a different question.**

Concretely: one item of weight 3, capacity 9, forwards. `dp[3]` gets the item. Then `dp[6]` is computed from `dp[3]`, which I just set. Then `dp[9]` from `dp[6]`. **Three copies of an item I have one of.**

The general rule that unifies both: backwards guarantees the previous row, forwards allows the current one. So the direction is not a convention to memorise — it is a statement about which generation of the data the recurrence needs.

And neither version raises an error, which is why I say the direction and its reason out loud while writing the range. **The test that catches it is an input whose optimal answer needs an item more than once** — a test where every item is used at most once passes under both.”

“Now give me the actual subsequence, in linear space.”

“That is Hirschberg’s algorithm, and I would explain the idea rather than write it, because it is about thirty lines and easy to get subtly wrong.

First, the problem. The linear-space version overwrites its array on every row, so at the end there is no history to walk back through. The value is there and the path is gone. That is the trade I would have stated when I collapsed it.

Hirschberg’s gets both, and the idea is divide and conquer on the middle row.

Take the middle row of the table. The optimal path crosses it at exactly one column — I do not know which, but I can find it.

Run the linear-space DP forwards from the start down to that middle row. That gives me, for every column, the best value to reach that column of the middle row.

Run it again backwards, from the end up to the middle row, on the reversed strings. That gives me, for every column, the best value from there to the end.

Add the two arrays and take the maximum. That column is where the optimal path crosses the middle.

Now the problem splits into two independent rectangles — top-left and bottom-right of that crossing point — and I recurse on each.

The cost is exactly twice the original time, because the work halves at every level: $n \cdot m$ plus $n \cdot m/2$ plus $n \cdot m/4$ sums to $2 \cdot n \cdot m$. **And the space is linear**, because each level only ever holds two rows.

So: $O(n \times m)$ time, $O(\min(n, m))$ space, and the actual path.

The same idea works for edit distance and sequence alignment generally, and it is what real bioinformatics tools use — **aligning two chromosomes would be a table of 10^{16} cells**, so linear space is not an optimisation there, it is the only way the problem exists.

I would say plainly that I would look it up rather than write it from memory, because it is exactly the kind of code that is subtly wrong, and knowing what it does and what it costs is the part that matters.”

“Can you reduce the space for this interval DP?”

“No, and I think saying that clearly, with the reason, is a better answer than an attempt that does not work.

The rule is the same as always: keep what the recurrence reads. So the question is what interval DP reads.

$dp[i][j]$ is the best over every split k of $dp[i][k]$ plus $dp[k+1][j]$. The first group — $dp[i][k]$ for every k — is spread across row i . The second group, $dp[k+1][j]$, is spread across many different rows.

So there is no single row, or pair of rows, that contains what I need. The whole $O(n^2)$ table has to be live, and there is no version of the collapse that works.

What I can offer instead is smaller and honest.

Only half the table is ever meaningful — cells where $i > j$ describe an empty range — **so a triangular representation halves the memory.** In Python the indexing overhead usually costs more than it saves; in C it is worth doing.

And if I also need the reconstruction I need a second table of the same size, so the trade there is: the answer alone is one $O(n^2)$ table, the answer plus the parenthesisation is two.

The more useful observation is that space is not what limits interval DP. At $n = 500$ the table is seven megabytes and the running time is about twenty-one million operations — **so I run out of patience long before I run out of memory.** At $n = 5,000$ it is two times ten to the tenth operations, which is hours, and the seven hundred megabytes is the lesser problem.

So if someone needs a much larger n , the answer is not a better constant factor — it is that interval DP is the wrong approach and there has to be extra structure to exploit. Matrix chain specifically has an $O(n \log n)$ algorithm, Hu-Shing, **which I know exists and would not write.**“

The model answer

“You are aligning two DNA sequences, each about a hundred thousand bases, and you need the actual alignment, not just the score. How would you do it?”

“The size is what makes this a real question, so let me start with the arithmetic and let it choose the algorithm.

This is sequence alignment, which is edit distance with weighted operations — substitutions between chemically similar bases cost less than between different ones, and gaps have their own cost.

The standard table is $n \times m$. At a hundred thousand each, that is ten billion cells. At even four bytes per cell that is forty gigabytes, and in Python with integer objects it is several hundred. **So the plain table does not exist, and that settles the first decision before I write anything.**

The linear-space version is easy and insufficient. The recurrence reads only the previous row plus the diagonal, **so one row of a hundred thousand cells plus one variable is about eight hundred kilobytes** — completely comfortable. But it gives me the score and not the alignment, and the prompt asks for the alignment.

So: Hirschberg’s algorithm. Divide and conquer on the middle row — run the linear-space pass forwards to the midpoint and backwards from the end, **add the two arrays to find the column where the optimal path crosses**, then recurse on the two independent halves.

$O(n \times m)$ time — exactly twice the plain version, because the work halves at each level — **$O(\min(n, m))$ space**, and it produces the actual alignment.

That is the textbook answer, and now the part that matters more for a real problem.

Ten billion cells is still ten billion cells. Even at a hundred million cells per second in C, that is a hundred seconds — **and in Python it is hours.** So the space problem is solved and the time problem is not.

Which means the real answer is not a general alignment algorithm at all. Biological sequences are mostly similar, **so the optimal path hugs the diagonal** — and a **banded** algorithm that only fills cells within some distance k of the diagonal is **$O(n \times k)$** rather than **$O(n \times m)$** . **At $k = 1,000$ that is a hundred million cells instead of ten billion — a hundredfold**, and it is exact as long as the true alignment stays within the band.

And I would say that caveat explicitly: banding is a heuristic that is exact under an assumption, **so it needs a check — if the best path touches the edge of the band, widen it and rerun.**

Beyond that, real tools do seed-and-extend: find exact short matches first, then align only around them. **BLAST and its descendants work that way**, and they are not doing full dynamic programming at all — they are using it as a local step after cheap filtering. **Which is the same shape as the answer to fuzzy search: the win is in not running the expensive algorithm, not in running it faster.**

So my answer is layered: Hirschberg's if I need an exact global alignment in linear space, banded if the sequences are known to be similar, and seed-and-extend if this is a search rather than a comparison — and I would want to know which of those three the question actually is before choosing.”

9 Recall card

One rule: look at which cells the recurrence reads and keep only those. `dp[i-1]`, `dp[i-2]` → **two variables**. Previous row only → **one or two rows** (swap so the shorter dimension is inner — $O(\min(n, m))$). Previous row **plus the diagonal** → one row **plus one saved variable**. Many rows at once (interval DP) → **no collapse exists, and saying so is the right answer.**

The one-row array holds two rows at once, split at the cursor: everything left of it is updated (the current row), everything at or right of it is not (the previous row). **So `row[c] += row[c-1]` is exactly “above + left”** — pointing at those two values is the real demonstration that you understand the recurrence.

Direction decides which generation you read. Backwards guarantees the previous row (0/1 knapsack, each item once); **forwards allows the current row** (unbounded, reusable). Same six lines, different answers — 9 against 10 on the worked example — **and no error either way.**

Precondition: every dependency must be above or to the left. Minimum falling path reads the previous row to the *right*, which is already overwritten, so it needs **two arrays**.

The collapse is free in time — the same cells are computed, and one row fits in cache, so it is often *~25% faster*. **What it costs is the reconstruction:** value in `O(min(n, m))` space, or path in `O(n × m)`. **Both is Hirschberg's** — divide and conquer on the middle row, add the forward and backward arrays to find where the optimal path crosses, recurse on the halves: **exactly $2 \times$ the time, linear space, and the path.**

Re-establish the base cases on every row (`row[0] = i` is the most-forgotten line). **And write the clear table first, then collapse** — a wrong recurrence is nearly undebuggable in an array that is half one row and half another.

1 What this is, and why they ask it

Uber matches a rider who wants a car with a driver who has one, **and the difficulty is that both of them are moving.**

They ask it because **it is the only design in this set where the state changes continuously and physically.** Every other system stores things that sit still — tweets, files, videos. **Here a million cars are each sending a new position every four seconds,** and a query is “who is near me *right now*”, where “now” means within a few seconds and “near” means within a couple of kilometres.

Three things carry the interview.

Geospatial indexing. “Find drivers within 3 km” is not a query a normal index can answer, because **a B-tree on latitude and a B-tree on longitude cannot be combined usefully** — you get everything in a band, not everything in a circle. **The answer is to turn two dimensions into one,** with geohashing or a hierarchical cell system, and knowing why that is necessary is the substance.

The write rate. A million drivers reporting every four seconds is **250,000 location writes a second,** which is a large number for tiny payloads, and it makes the location store completely unlike the trip store sitting next to it.

And matching, which is not a lookup. The nearest driver is often not the right one — traffic, direction of travel, driver acceptance rate, and the fact that assigning greedily one rider at a time is measurably worse than batching a few seconds of requests and solving them together. **That last point is where the DP and the system design meet,** because batched matching is the assignment problem.

By the end of this lesson you can design the location pipeline, the geospatial index, the matching service, the trip lifecycle and pricing, and size all of it.

2 The story

The auto stand outside the station had about forty drivers and one man with a whistle, and Karim had held the whistle for nine years.

His job was to know two things at once: who was free, and where they were.

The first part was easy and physical. **The autos queued.** You came back from a fare, you joined the end, and when you reached the front you took the next passenger. **Nobody argued because the queue was visible.**

The problem was that the queue only worked at the stand.

Because half the drivers were not at the stand. They were dropping somebody at the hospital, or waiting outside the college, or stuck at the level crossing. **And a passenger at the stand did not care where they were — but a passenger who telephoned did,** because the nearest auto to them might be the one at the college, not the one at the front of the queue.

So Karim did the thing that made him useful, and it took him about a year to settle into.

He divided the town into pieces. Not on any map — in his head, and later on a board with chalk. **Station, college, hospital, the market, the two colonies, the crossing.** Seven pieces, and later eleven.

And he did not track where each auto was. He tracked which piece it was in.

Which was much less information and almost as useful. **When a call came from the college, he looked at the college piece and the two pieces next to it,** and if there was an auto in any of them he sent it, and if there was not he looked wider.

There were two things he learned that took longer.

The first was that the nearest one was often the wrong one. The auto two hundred metres away might be pointing the wrong way down a one-way, or on the far side of the crossing when the train was through. **“Near” on his chalk board and “near” in minutes were not the same thing,** and he learned to know the difference by knowing the town.

And the second was the thing he did on Friday evenings, which he never explained to anybody.

Friday at seven, four or five calls would come in together. **And instead of answering them one at a time, which is what he did all week, he would hold them for a minute and then send all five autos at once.**

Because answering the first call with the nearest auto sometimes stranded the second caller. **Held together, he could see all five and all the autos, and shuffle them** — and the total waiting was lower, even though the first caller waited slightly longer than they would have.

3 The idea in plain English

Karim's chalk pieces are the geospatial index, his observation about the one-way street is the difference between distance and travel time, and his Friday evening is batched matching.

Start with the two problems, because they are genuinely separate systems.

LOCATION / MATCHING	TRIPS / PAYMENTS
1,000,000 drivers	~1,000 trips/second
a write every 4 seconds	a few writes per trip
= 250,000 writes/second	= ~5,000 writes/second
tiny payloads, disposable	must be durable, transactional
in-memory, sharded by region	a relational database
losing a point is FINE	losing a payment is NOT

Saying that split early is worth doing, because the two halves have opposite requirements and combining them is the mistake.

Now the geospatial problem, which is the interesting one.

“Find drivers within 3 km of this point” is not something a normal index answers. A B-tree on latitude finds everything in a horizontal band. A B-tree on longitude finds a vertical band. **You cannot intersect them efficiently** — the database would scan one band and filter, and in a city that band contains a large fraction of all drivers.

The fix is to turn two dimensions into one, so an ordinary index works.

Geohashing does it by interleaving the bits of latitude and longitude and encoding the result as a string. **Nearby points share a prefix.** `tdr1x` and `tdr1y` are adjacent cells; `tdr1` is the bigger cell containing both.

So **“drivers near me” becomes “drivers whose geohash starts with `tdr1`”** — a prefix range scan, which every index in the world can do.

And there is a specific flaw worth knowing, because it is the follow-up. Two points can be metres apart and share no prefix at all, if they sit either side of a cell boundary. **The fix is to search the cell plus its eight neighbours**, which geohash libraries compute directly. **Nine cells, not one**, and forgetting that means missing the closest driver because they are across a line that does not exist on the ground.

The alternatives are worth naming. Google's S2 maps the sphere onto a cube and uses a Hilbert curve, **which keeps nearby points closer together than geohashing does** and handles the poles properly. Uber's own H3 uses hexagons, and the reason is

neat: a hexagon has six neighbours all at the same distance, while a square has four edge neighbours and four corner ones at different distances — which makes “expand the search outwards” behave uniformly.

And Redis has `GEOADD` and `GEOSEARCH` built in, which is a sorted set with geo-hash scores underneath, and it is what a smaller system should simply use.

Now the write rate, which shapes the location store.

A million drivers reporting every four seconds is 250,000 writes a second. That is large, and three properties make it manageable.

The payloads are tiny — driver id, latitude, longitude, heading, timestamp: about fifty bytes.

Only the latest matters. A location from thirty seconds ago is useless, so this is an overwrite, not an append — the store holds one row per driver, not a history.

And losing one is fine. A dropped location point is corrected four seconds later. So the location store can be entirely in memory with no durability guarantee at all, which is what makes 250,000 writes a second easy rather than hard.

Sharding is by geography, which is natural: a driver in Mumbai and a driver in Delhi never appear in the same query, so the system is really thousands of independent small systems.

Then matching, which is where the design gets opinionated.

The naive version: find the nearest available driver and assign. It works and it is measurably worse than the alternatives, for three reasons.

Distance is not time. Karim’s one-way street. Two hundred metres away across a river is twenty minutes; a kilometre away on a clear road is three. So candidates are found by distance and ranked by estimated time of arrival, which needs a routing service that understands the road network.

Greedy assignment is not optimal. Assigning riders one at a time, each to their nearest driver, can strand a later rider whose only nearby driver was just taken. Batching a few seconds of requests and solving them together produces lower total waiting.

And that batched problem is the assignment problem — from day 160 — which for small batches is exactly what bitmask DP solves, and for realistic batches is what the Hungarian algorithm solves in $O(n^3)$. The connection is real and worth making explicitly.

The trade is latency: batching adds a few seconds before anyone is matched. **Uber's published work says the total wait still improves**, because the improvement in matching quality outweighs the delay.

Then the dispatch protocol, which has a specific correctness requirement.

A driver must not be offered to two riders at once. So the assignment takes a **lock on the driver**, sends the offer, and waits a few seconds for acceptance. **On acceptance the trip is created; on rejection or timeout the lock is released and the driver returns to the pool.**

The lock must expire, because a driver whose phone dies mid-offer would otherwise be unavailable forever. **That is [day 127's distributed lock](#), and the TTL is the whole point.**

Then the trip, which is the boring half and must be the reliable one.

A trip is a state machine: requested → matched → driver arriving → in progress → completed → paid. **Each transition is a durable write**, and the payment at the end is the one thing in this entire system that must not be lost or duplicated.

And the two halves have opposite requirements: location data is high-volume and disposable; trip data is low-volume and must be exactly right. **Do not put them in the same store.**

Finally, pricing, which is a real-time feedback loop.

Surge pricing exists because demand and supply are local and change by the minute. The mechanism is to divide the city into cells, compute the ratio of open requests to available drivers per cell every minute or two, and apply a multiplier where the ratio is high.

It is a control loop and it can oscillate. A high price suppresses demand, the ratio falls, the price drops, demand returns. **Smoothing and rate limits on how fast the multiplier moves are not polish; they are what stops the system hunting**, which is a genuinely interesting property to raise.

4 The picture

The two systems, which must not be one:

LOCATION PIPELINE (hot, disposable)

```
driver app --every 4 s →  
  |  
  [ ingest, sharded by region ]  
  |  
  [ in-memory geo index ]  
  |   one row per driver  
  |   OVERWRITTEN, not appended  
  v  
  "who is near this point?"  
  250,000 writes/second  
  losing a point: fine
```

TRIP SYSTEM (durable, small)

```
rider requests  
  |  
  [ trip service ]  
  |  
  [ relational DB ]  
  |   trips, payments,  
  |   receipts  
  v  
  must be durable  
  ~5,000 writes/second  
  losing a payment: not
```

Why a normal index cannot answer “within 3 km”:

index on latitude:

```
+-----+  
|#####|  
|#####|  
+-----+
```

index on longitude:

```
+--+  
|##|  
|##|  
|##|  
+--+
```

each finds a BAND

what you want:

```
  .-'-'-.  
 /      \  
|        |  
|  ( )  |  
|        |  
 \      /  
  '-..-'
```

intersecting them:

```
+--+  
|##|  
|##|  
+--+
```

← still a band's worth
of candidates to filter

In a dense city, one band is a large fraction of ALL drivers.
→ the database scans and filters, and it does not scale.

Geohashing: two dimensions into one:

interleave the bits of latitude and longitude, then encode

```
lat bits: 1 0 1 1 0 ...
lon bits: 0 1 1 0 1 ...
interleaved: 1 0 0 1 1 1 1 0 0 1 ...
encoded: "tdr1x"
```

NEARBY POINTS SHARE A PREFIX:

```
tdr1x and tdr1y are adjacent
tdr1      is the cell containing both
tdr       is the bigger cell containing that
```

→ "drivers near me" = "geohash starts with tdr1"

→ a PREFIX RANGE SCAN, which any ordinary index can do

precision by prefix length:

```
4 chars ~20 km      6 chars ~600 m
5 chars ~2.4 km     7 chars ~76 m
```

The boundary problem, which is the follow-up:

```
cell "tdr1" | cell "tdr4"
           |
           A * | * B
```

A and B are 20 METRES apart
and share NO prefix at all

→ searching only my own cell MISSES the nearest driver

FIX: search the cell PLUS ITS EIGHT NEIGHBOURS

```
+-----+-----+-----+
| tdr0 | tdr1 | tdr4 |
+-----+-----+-----+
| tdr2 | ME  | tdr5 |    ← 9 cells, always
+-----+-----+-----+
| tdr3 | tdr6 | tdr7 |
+-----+-----+-----+
```

Every geohash library computes the neighbours directly.
Forgetting this is the classic bug, and it fails only for
riders near a boundary – which is most of them.

Why hexagons (H3):

SQUARE CELLS

```
+---+---+---+
| a | b | c |
+---+---+---+
| d | X | e |
+---+---+---+
| f | g | h |
+---+---+---+
```

b, d, e, g are EDGE neighbours

a, c, f, h are CORNER neighbours

→ different distances!

HEXAGONAL CELLS

```
      / \ / \
      | a | b |
      / \ / \ / \
    | c | X | d |
      \ / \ / \ /
      | e | f |
      \ / \ /
```

all six are EDGE neighbours

→ all the SAME distance

"expand the search outwards" behaves uniformly with hexagons and unevenly with squares. That is Uber's reason for H3.

Greedy against batched matching, which is Karim's Friday:

two riders (R1, R2), two drivers (D1, D2)

ETAs:	D1	D2
R1	2 min	9 min
R2	3 min	20 min

GREEDY, R1 arrives first:

R1 → D1 (2 min, the nearest)
R2 → D2 (20 min, all that is left)
TOTAL 22 min

BATCHED, both considered together:

R1 → D2 (9)	R2 → D1 (3)	total 12
R1 → D1 (2)	R2 → D2 (20)	total 22

→ pick the first. TOTAL 12 min

10 minutes better, and R1 waited 7 minutes longer.

THIS IS THE ASSIGNMENT PROBLEM: n riders, n drivers, minimise the total. Hungarian algorithm, $O(n^3)$.

The COST of batching is a few seconds of delay before anyone is matched at all.

The dispatch lock, and why it must expire:

match found: rider R → driver D

1. LOCK D SET lock:D "R" NX EX 15
 → if the lock exists, D was already offered to someone else
2. send the offer to D's phone
3. wait up to 15 seconds

ACCEPT → create the trip, D is now "on trip"

REJECT → DEL lock:D, D returns to the pool, rematch R

TIMEOUT → the lock EXPIRES on its own, D returns to the pool

THE TTL IS THE POINT: a driver whose phone dies mid-offer would otherwise be locked out forever, and no explicit "release" message will ever arrive.

5 How it actually works

Ingesting locations

```
def report_location(driver_id: int, lat: float, lon: float,
                    heading: float, at: float) → None:
    cell = h3.geo_to_cell(lat, lon, resolution=8)      # ~460 m across
    pipe = redis.pipeline()
    pipe.hset(f"driver:{driver_id}", mapping={
        "lat": lat, "lon": lon, "heading": heading, "at": at, "cell": cell})
    pipe.expire(f"driver:{driver_id}", 30)            # stale = gone
    pipe.zadd(f"cell:{cell}", {driver_id: at})
    pipe.execute()
```

expire doing the offline detection is the neat part. A driver whose app stops reporting simply disappears from the index after thirty seconds — **there is no “go offline” message to lose**, which is the same expiring-presence pattern as [day 157](#).

One row per driver, overwritten, not a history — the location store holds the present, and the past goes to an analytics pipeline via a separate stream.

And moving cells needs the old entry removed, which the code above omits for clarity — in practice the write compares the new cell with the stored one and issues a **ZREM** when it changes.

Finding nearby drivers

```
def nearby_drivers(lat: float, lon: float, radius_m: int = 3000) → list[dict]:
    centre = h3.geo_to_cell(lat, lon, resolution=8)
    rings = h3.k_ring(centre, k=radius_m // 460 + 1)      # the cell AND its ring
    candidates: set[int] = set()
    for cell in rings:
        candidates |= set(redis.zrange(f"cell:{cell}", 0, -1))

    result = []
    for driver_id in candidates:
        data = redis.hgetall(f"driver:{driver_id}")
        if data and haversine(lat, lon, data["lat"], data["lon"]) ≤ radius_m:
            result.append(data)
    return result
```

`k_ring` is the neighbour expansion, and it is why this returns the driver twenty metres away across a cell boundary. **Searching only `centre` is the classic bug.**

And the final `haversine` filter matters: cells are approximate, so a driver in a neighbouring cell may be further than the radius. **The index narrows; the exact test decides.**

Ranking by time, not distance

```
def rank_candidates(rider: dict, drivers: list[dict]) → list[tuple[float, dict]]:
    etas = routing.batch_eta(
        origins=[(d["lat"], d["lon"]) for d in drivers],
        destination=(rider["lat"], rider["lon"]),
    )
    scored = []
    for driver, eta in zip(drivers, etas):
        score = eta
        score *= 1.0 - 0.1 * driver["acceptance_rate"]      # likely to accept
        if heading_towards(driver, rider):
            score *= 0.9                                     # already coming this way
        scored.append((score, driver))
    return sorted(scored)
```

`routing.batch_eta` in one call, not one per driver. Twenty candidates means one request, and doing it individually is the difference between twenty milliseconds and four hundred.

The score is not the ETA alone, and each adjustment is a real effect: a driver who rejects most offers wastes the fifteen-second lock, and a driver already travelling towards the rider will arrive sooner than the straight-line estimate suggests.

Batched matching

```
def match_batch(requests: list[dict], window_seconds: float = 5.0) → list[tuple]:
    """Collect a few seconds of requests, then solve them together."""
    riders = requests
    drivers = union_of_nearby_drivers(riders)
    if not drivers:
        return []

    cost = [[eta(driver, rider) for rider in riders] for driver in drivers]
    pairs = hungarian(cost) # O(n^3), optimal for the batch
    return [(drivers[d], riders[r]) for d, r in pairs]
```

`hungarian(cost)` is the assignment problem, and it is the same problem bitmask DP solved yesterday — the Hungarian algorithm is $O(n^3)$ where bitmask DP is $O(2^n \cdot n)$, so for a batch of fifty riders it is a hundred and twenty-five thousand operations rather than an impossibility.

The window is the trade. Five seconds of delay before anyone is matched, in exchange for a materially better assignment. **Uber's published results say the total wait falls**, which is the justification.

The dispatch lock

```
OFFER_SECONDS = 15

def offer_ride(driver_id: int, rider_id: int) → bool:
    if not redis.set(f"lock:driver:{driver_id}", rider_id,
                    nx=True, ex=OFFER_SECONDS):
        return False # already offered to someone else
    push.send(driver_id, {"type": "offer", "rider": rider_id,
                        "expires_in": OFFER_SECONDS})
    return True

def on_driver_accepts(driver_id: int, rider_id: int) → dict | None:
    holder = redis.get(f"lock:driver:{driver_id}")
    if holder != str(rider_id):
        return None # the offer expired; too late
    trip = trip_store.create(driver_id, rider_id, state="matched")
    redis.delete(f"lock:driver:{driver_id}")
    return trip
```

SET NX EX is both the mutual exclusion and the timeout. Two matchers cannot offer the same driver, and a driver whose phone dies is released automatically — no explicit unlock is required, which matters because that message is exactly the one that will be lost.

The re-check in `on_driver_accepts` handles the race where the lock expired between the offer and the acceptance: **the driver taps accept a moment too late, and the ride has already gone to someone else.** That must be handled, and it hap-

pens constantly.

The trip state machine

```
TRANSITIONS = {
    "requested": {"matched", "cancelled"},
    "matched": {"arriving", "cancelled"},
    "arriving": {"in_progress", "cancelled"},
    "in_progress": {"completed"},
    "completed": {"paid"},
}

def transition(trip_id: int, to_state: str) → None:
    with db.transaction():
        trip = trip_store.lock_for_update(trip_id)
        if to_state not in TRANSITIONS.get(trip.state, set()):
            raise InvalidTransition(f"{trip.state} → {to_state}")
        trip_store.update(trip_id, state=to_state, at=time.time())
        events.publish("trip", {"trip_id": trip_id, "state": to_state})
```

An explicit transition table, checked on every write. Trips get cancelled, apps retry, network messages arrive out of order — **and without the check, a retrieved “complete” on an already-paid trip charges twice.**

`lock_for_update` inside a transaction is what makes it safe under concurrency, and this is the half of the system where a relational database earns its place.

Surge pricing

```
def compute_surge(cell: str) → float:
    demand = redis.get(f"requests:{cell}:last_2min") or 0
    supply = redis.zcard(f"cell:{cell}")
    ratio = int(demand) / max(int(supply), 1)

    target = 1.0 if ratio < 1.5 else min(1.0 + (ratio - 1.5) * 0.4, 3.0)
    current = float(redis.get(f"surge:{cell}") or 1.0)
    smoothed = current + max(-0.2, min(0.2, target - current)) # rate-limited
    redis.set(f"surge:{cell}", smoothed, ex=300)
    return smoothed
```

The rate limit on how fast the multiplier moves is the important line, and it is not cosmetic. **Without it the loop oscillates:** a high price suppresses demand, the ratio collapses, the price drops, demand returns, and riders see the price swinging every two minutes. **Smoothing is what makes a control loop stable rather than a hunting one.**

And the cap at 3.0 is a policy decision, not a technical one — worth naming as such.

The real systems

H3	Uber's hexagonal grid, open-sourced; the reason for hexagons is uniform neighbour distance
S2	Google's cell system: Hilbert curve on a cube
geohash	the simple, widely supported option; Redis uses it
Redis GEO	GEOADD / GEOSEARCH – a sorted set with geohash scores; what a smaller system should just use
Kafka	the location stream, feeding analytics separately from the live index
PostgreSQL/ PostGIS	trips and payments; PostGIS for offline geo queries
OSRM / Valhalla	open-source routing engines for ETAs

Naming H3 and its reason is a good detail, because it is a specific, defensible engineering decision that came out of this exact problem.

6 The numbers

Scale.

```
5,000,000 drivers, ~1,000,000 online at peak
20,000,000 rides/day

rides:      20e6 / 86,400 = ~230/second average
              peak ~1,000/second

locations:  1,000,000 online x 1 report / 4 s
              = 250,000 writes/second
```

Two hundred and fifty thousand writes a second against a thousand rides a second — the location pipeline is 250 times the write volume of the business it supports, and that ratio is the first thing to say.

Location data.

```
per report: driver id (8) + lat (8) + lon (8) + heading (4) + timestamp (8)
              = ~40 bytes, ~50 with overhead

250,000/s x 50 B = 12.5 MB/s = ~1 TB/day IF stored

but only the LATEST matters:
  1,000,000 online drivers x ~200 bytes (hash + index entry)
  = 200 MB of live state

→ the live index is 200 MB. The 1 TB/day goes to a stream for
  analytics, and never to the query path.
```

Two hundred megabytes for the entire live index is the fact that makes this affordable: it fits in one machine's memory, and it is sharded by region for isolation rather than for capacity.

Query cost.

```
1,000 ride requests/second at peak
each searches ~9-19 cells (the centre plus one or two rings)
each cell holds ~10-100 drivers in a dense city

→ ~500 candidate drivers per request, filtered to ~20 by exact distance
→ 1,000 x 19 cell reads = 19,000 Redis reads/second

trivial for Redis (~100,000 ops/s per instance)
→ the READ side is not the problem. The WRITE side is.
```

Sharding by region.

```
250,000 writes/second globally
but a driver in Mumbai never appears in a Delhi query

~500 cities, unevenly:
  the largest city at peak: ~50,000 drivers → ~12,500 writes/second
  Redis handles ~100,000 ops/s → one instance per large city,
  with several cities sharing an instance elsewhere

→ the system is really ~100 independent small systems, which is
  why the global number is misleading.
```

Cell sizing, which is a real tuning decision.

H3 resolution 8: ~460 m across, ~0.7 km²

a dense city centre: ~2,000 drivers / 20 km² = 100 per km²
→ ~70 drivers per cell

a 3 km search radius:

radius / cell size = 3000 / 460 ~ 7 rings

cells in 7 rings = 1 + 3*7*8 = 169 cells

→ 169 x 70 = ~11,800 candidates. TOO MANY.

→ use a COARSER resolution for the search:

resolution 7 (~1.2 km across): 3 rings = 37 cells x ~500 = 18,500

resolution 6 (~3.2 km): 1 ring = 7 cells x ~3,000 = 21,000

or search resolution 8 with a SMALL radius first and widen only if too few candidates:

1 ring (7 cells x 70 = ~490) → usually enough

3 rings if not

→ ~500 candidates in the common case, which is right.

Starting small and widening is the design, and it is worth showing, because a fixed radius is either too slow downtown or empty in the suburbs.

Matching cost.

batch window 5 seconds at peak: 1,000/s x 5 = 5,000 requests
but batches are PER CITY:

a large city at peak ~100 requests per 5-second window

Hungarian algorithm on 100 riders x ~300 candidate drivers:

$O(n^3)$ with $n = 300$ → 27,000,000 operations → ~30 ms in C

→ well within the window

greedy, for comparison: 100 x 300 = 30,000 operations

→ 1,000x cheaper and measurably worse results

Trips and payments.

20,000,000 rides/day x ~6 state transitions = 120,000,000 writes/day
= ~1,400 writes/second
peak ~6,000

trip record: ~2 KB (route, fare breakdown, timestamps, ids)

20e6 x 2 KB = 40 GB/day = ~15 TB/year

+ replicas = 45 TB/year

→ a completely ordinary relational workload. Which is the point:
the DURABLE half of this system is small and boring, and it
must stay that way.

ETA computation, the hidden cost.

```
1,000 requests/second x ~20 candidates = 20,000 ETA calculations/second
```

```
a routing engine: ~1-5 ms per route on a city-sized graph
```

```
→ 20,000 x 3 ms = 60 CPU-seconds per second = ~60 cores
```

```
and BATCHING matters enormously:
```

```
one request with 20 origins: ~5 ms total
```

```
20 separate requests: ~60 ms + 20 round trips
```

```
→ batch_eta is not an optimisation; it is what makes the  
matching latency budget possible.
```

End-to-end latency:

```
location write ~1 ms (fire and forget)
```

```
nearby search (19 cells) ~5 ms
```

```
exact distance filter ~1 ms
```

```
batch ETA for 20 candidates ~10 ms
```

```
ranking ~1 ms
```

```
matching (within the batch) ~30 ms
```

```
-----
```

```
~50 ms of compute
```

```
+ the batch window 5 s ← dominates everything
```

```
+ driver acceptance ~5 s
```

```
→ the system's latency is a product decision, not an engineering one.
```

7 The trade-offs

Geohash against S2 against H3. Geohash is simple, widely supported, and **has uneven cell sizes away from the equator and a nasty boundary discontinuity.** S2's Hilbert curve keeps nearby points closer in the ordering and handles the sphere properly. **H3's hexagons give uniform neighbour distances,** which makes ring expansion behave sensibly, at the cost of hexagons not tiling a sphere perfectly — **there are twelve pentagons in the grid, which is a real edge case in the library.** For most systems, **Redis GEO is the right answer and building on H3 is justified only when ring expansion is central,** which for Uber it is.

Cell size. Small cells mean precise results and many cells to search — a 3 km radius at 460 m resolution is 169 cells. **Large cells mean few lookups and many false candidates to filter.** The answer is neither: **a small radius that widens when it finds**

too few drivers, which adapts to density automatically and is what makes downtown and the suburbs both work.

Greedy against batched matching. Greedy is instant and locally optimal, and **it strands riders whose only nearby driver was just assigned**. Batching is optimal within the window and **delays every match by several seconds**. The published evidence says total wait improves; **the honest framing is that this is a product decision informed by measurement, not a purely technical one**, because the first rider genuinely waits longer.

In-memory locations against durable ones. In-memory gives 250,000 writes a second and **loses everything on a restart** — which is fine, because every driver reports again within four seconds. **Making it durable would cost enormously and buy nothing**, and recognising which data is disposable is the judgement being tested.

The two-system split. Combining the location store and the trip store would be simpler operationally and is wrong: **one needs 250,000 disposable writes a second and the other needs transactional guarantees on payments**. No single store is good at both, and forcing one to be is how the payments end up unreliable.

Surge pricing as a control loop. It genuinely balances supply and demand and **it oscillates without smoothing and rate limits**. It is also the most disliked feature in the product, so the multiplier cap is a policy decision that engineering should surface rather than choose.

When would I not build this? When the fleet is small and known — a taxi company with forty cars needs a list and a map, not a geospatial index. **When the matching is not real-time**: scheduled deliveries, which become a routing and planning problem instead. **And the geo layer specifically** — **Redis GEO or PostGIS handle millions of points comfortably**, and building on H3 is justified by ring-expansion behaviour rather than by scale alone.

8 In the interview

How it gets asked

- “Design Uber.” or “Design a ride-hailing service.” — the standard prompt.
- “How do you find the nearest drivers?” — the geospatial question, and the core of it.
- “A million drivers each send a location every four seconds. How do you store that?”
- “Two riders request at the same time and there is one driver nearby. What happens?”
- “How does surge pricing work?”

- *“What is wrong with just assigning the nearest driver?”*

The first ninety seconds

“The first thing I would establish is that **this is two systems with opposite requirements**, and keeping them separate is most of the design.

The location and matching side handles a million drivers each reporting every four seconds — 250,000 writes a second. Tiny payloads, only the latest one matters, and **losing a point is completely fine** because another arrives in four seconds. **In memory, sharded by region, no durability at all.**

The trip and payment side handles about a thousand rides a second — a few writes each. **Small, and it must be exactly right**, because losing a payment is not recoverable. A relational database.

The location pipeline is 250 times the write volume of the business it supports, and it is the one that must be allowed to be lossy.

Now the interesting problem: finding drivers near a point. A normal index **cannot do this** — a B-tree on latitude gives a horizontal band, one on longitude gives a vertical band, and **intersecting them still leaves a large fraction of a city’s drivers to filter.**

So: turn two dimensions into one. Geohashing interleaves the bits of latitude and longitude, **so nearby points share a prefix** and “drivers near me” becomes a prefix range scan that any index can do.

And there is a flaw I would name immediately, because it is the classic bug. Two points can be twenty metres apart and share no prefix, if they are either side of a cell boundary. **So you always search the cell plus its eight neighbours** — nine cells, never one.

Uber uses H3, which is hexagons, and the reason is neat: **a hexagon’s six neighbours are all the same distance away, while a square has four edge neighbours and four corner ones at different distances** — so expanding the search outwards behaves uniformly.

The live index is small. A million online drivers at about two hundred bytes each is **two hundred megabytes** — it fits in memory on one machine, and it is sharded by city for isolation rather than for capacity.

And then matching, which I would flag as more than a lookup. The nearest driver by distance is often not the nearest by time, and **assigning greedily one rider at a time is measurably worse than batching a few seconds and solving them together** — which is literally the assignment problem.

Where would you like me to go deeper?“

The follow-ups

“How do you find the nearest drivers?“

“Not with a normal index, and the reason is worth being precise about, because it is what motivates everything else.

A B-tree on latitude finds every driver in a horizontal band. A B-tree on longitude finds a vertical band. The database can use one of them and then filter — and in a dense city, one band contains a large fraction of all drivers, so it is effectively a scan.

The fix is to collapse two dimensions into one, so that ordinary index machinery works.

Geohashing does it by interleaving the bits of latitude and longitude and encoding the result. The key property is that **nearby points share a prefix**: `tdr1x` and `tdr1y` are adjacent cells, and `tdr1` is the larger cell containing both. **So the query becomes a prefix range scan**, and prefix length controls precision — five characters is about 2.4 kilometres, seven is about seventy-six metres.

Now the flaw, and I would raise it before being asked. Two points twenty metres apart can share no prefix at all if they sit either side of a cell boundary. **So searching only my own cell misses the closest driver** — and it fails specifically for riders near a boundary, which is a large fraction of them.

The fix is to search the cell plus its eight neighbours, which every geohash library computes directly. **Nine cells, always.**

And after the index narrows the candidates, an exact distance check decides, because cells are approximate and a driver in a neighbouring cell may be outside the radius.

On which system to use: Uber built H3, using hexagons. The reason is that a hexagon’s six neighbours are all equidistant, while a square grid has edge neighbours and corner neighbours at different distances — **so ‘expand the search by one ring’ means something uniform.** Google’s S2 uses a Hilbert curve on a cube, which preserves locality better than plain geohashing and handles the poles correctly.

For most systems the right answer is Redis GEO, which is geohashing under a sorted set and handles millions of points comfortably. **Building on H3 is justified when ring expansion is central to the algorithm**, which for dispatch it is.”

“A million drivers send a location every four seconds. How do you handle that?”

“Two hundred and fifty thousand writes a second, and **three properties of the data make it much easier than that number suggests**. I would name all three, because each one removes a requirement.

One: the payloads are tiny. Driver id, latitude, longitude, heading, timestamp — about fifty bytes. **So 250,000 writes a second is only twelve megabytes a second of actual data.**

Two: only the latest matters. A location from thirty seconds ago is useless for dispatch. **So this is an overwrite, not an append** — one row per driver, not a history. **The live index is a million rows at about two hundred bytes, which is two hundred megabytes.** That fits in memory on a single machine.

Three, and this is the one that matters most: losing a write is fine. A dropped location point is corrected four seconds later by the next one. **So there is no durability requirement at all** — the store can be entirely in memory, with no write-ahead log, no replication for durability, and no acknowledgement beyond ‘received’.

Recognising which data is disposable is the judgement here, because making this durable would cost enormously and buy nothing.

Sharding is by geography, and it is the natural boundary. A driver in Mumbai never appears in a Delhi query, so **this is really about a hundred independent small systems, not one large one.** The largest city at peak might have fifty thousand drivers — twelve thousand writes a second — **which one Redis instance handles comfortably.** The global figure is misleading.

Two details I would build in. A TTL on each driver’s record **does offline detection for free:** a driver whose app stops reporting disappears after thirty seconds, **and no ‘going offline’ message has to be delivered** — which matters because that is exactly the message that gets lost when a phone dies.

And the historical stream is separate. A terabyte a day of location history is genuinely valuable for analytics, ETA models and surge, **so it goes to Kafka and then to a data warehouse — never to the query path.** Mixing the live index and the historical store is how the query path gets slow.”

“What is wrong with just assigning the nearest driver?”

“Two things, and the second is the more interesting one.

First, the nearest driver by distance is often not the nearest by time. Two hundred metres away on the far side of a river is twenty minutes; **a kilometre away on a clear road is three.** One-way streets, level crossings, and the direction the driver is already travelling all matter.

So I would use the geospatial index to find candidates by distance and then rank them by estimated time of arrival, which needs a routing service that understands the road network. **And the ETAs must be fetched in one batched call** — twenty separate routing requests is four hundred milliseconds and one batched request is ten.

I would also adjust for two behavioural things: a driver who rejects most offers wastes the fifteen-second lock and delays the rider, and a driver already heading towards the rider will arrive sooner than a straight-line estimate suggests.

Second, and this is the interesting one: greedy assignment is not optimal even with perfect ETAs.

Concretely. Two riders, two drivers. Rider one is two minutes from driver one and nine from driver two. Rider two is three minutes from driver one and twenty from driver two. **Greedy gives rider one the nearest driver, and rider two is left with twenty minutes — twenty-two total. Considered together, rider one takes driver two at nine and rider two takes driver one at three — twelve total.**

Ten minutes better, and rider one waited seven minutes longer.

So: batch a few seconds of requests and solve them together, which is exactly the assignment problem. The Hungarian algorithm does it in $O(n^3)$ — for a hundred riders against three hundred candidates in a city, that is about thirty milliseconds, **well inside the window.**

The cost is real and I would state it: everybody waits a few seconds before being matched at all. Uber’s published work says total wait falls anyway, because the matching improvement outweighs the delay — **but the first rider in a batch is genuinely worse off, so this is a product decision informed by measurement, not a purely technical one.**“

The model answer

“Design a ride-hailing service: five million drivers, a million online at peak, twenty million rides a day, global.”

“Let me split it in two first, because the two halves have opposite requirements and combining them is the main mistake available here.

The location and matching side: a million online drivers reporting every four seconds is 250,000 writes a second. Fifty-byte payloads, only the latest matters, and losing one is fine. **In-memory, sharded by city, zero durability.**

The trip and payment side: twenty million rides a day is about a thousand a second at peak, six state transitions each. **Small, and it must be exactly right.** A relational database with real transactions.

The location pipeline is 250 times the write volume of the business it supports, and it is the half that is allowed to lose data.

Geospatial index. A normal index cannot answer ‘within 3 km’ — latitude and longitude B-trees each give a band, and intersecting them still leaves a large fraction of a city. **So collapse two dimensions into one.** I would use **H3 at around resolution 8 — cells about 460 metres across** — with the driver’s cell stored alongside their position.

A search reads the cell plus a ring of neighbours, never just the cell, **because two points twenty metres apart across a boundary share no cell at all** — and that failure hits riders near boundaries, which is most of them.

And the radius adapts. One ring is seven cells and about five hundred candidates downtown, which is right; **in the suburbs one ring finds nobody, so it widens.** A fixed radius is either too slow in the centre or empty at the edge, so **starting small and expanding is the design, not an optimisation.**

The live index is 200 MB for a million drivers, so it is sharded by city for isolation rather than for capacity — **the largest city is twelve thousand writes a second, which one Redis instance handles.**

A TTL on each driver record gives offline detection for free, with no ‘going offline’ message to lose.

Matching. Candidates by distance, **ranked by ETA from a batched routing call** — distance is not time, and twenty separate routing requests would blow the latency budget on their own.

And matching is batched over about five seconds and solved as an assignment problem, because greedy assignment strands riders whose only nearby driver was just taken. **A hundred riders against three hundred drivers is thirty milliseconds with the Hungarian algorithm,** well inside the window.

The cost is that everyone waits a few seconds to be matched, and the first rider in a batch is genuinely worse off — a product decision I would want made with data rather than by me.

Dispatch uses a lock on the driver with a fifteen-second TTL. Two matchers cannot offer the same driver, **and a driver whose phone dies is released automatically** — the message that would release them explicitly is exactly the one that gets lost. **And the acceptance handler re-checks the lock**, because a driver tapping accept a moment late is a constant occurrence, not an edge case.

The trip is a state machine with an explicit transition table checked on every write. Apps retry, messages arrive out of order, **and without the check a retried ‘complete’ charges twice.**

Surge pricing is a control loop: demand over supply per cell every couple of minutes, with a multiplier. **The rate limit on how fast it can move is the important part** — without it the loop oscillates, because a high price suppresses demand which drops the price which brings demand back. **And the cap is policy, which I would surface rather than choose.**

Two things I would flag. ETA computation is a hidden cost — twenty thousand route calculations a second, about sixty cores — **and it is the component most likely to be the bottleneck**, not the geo index.

And the failure mode that matters is a city-level outage, not a global one. Because everything is sharded by city, **a bad deploy or a Redis failure takes out one city completely**, which is a much more visible failure than partial global degradation. **I would want per-city isolation in the deployment as well as in the data**, and a fallback to greedy matching when the batch matcher is unavailable — degraded is much better than stopped.”

9 Recall card

Two systems with opposite requirements: locations (250,000 writes/s, 50-byte payloads, overwrite-only, LOSSY BY DESIGN, in-memory, sharded by city) and trips/payments (~5,000 writes/s, durable, transactional). The location pipeline is $250 \times$ **the write volume of the business it supports** — do not put them in one store.

A normal index cannot answer “within 3 km” — latitude and longitude B-trees each give a *band*, and intersecting them still leaves a large share of a city. **Collapse two dimensions into one:** geohashing interleaves the bits so **nearby points share a**

prefix, turning it into a prefix range scan.

Always search the cell PLUS its eight neighbours — two points 20 m apart across a boundary share no prefix, and this fails precisely for riders near boundaries. **H3 uses hexagons because all six neighbours are equidistant**, where a square has edge and corner neighbours at different distances. **Then an exact haversine filter decides** — the index narrows, it does not answer.

The live index is only ~200 MB for a million drivers. **A TTL does offline detection for free** — no “going offline” message to lose. **Search radius adapts**: one ring downtown, widen in the suburbs.

Greedy matching is measurably wrong. Distance $\neq$ time (batch the ETA calls), and assigning one rider at a time strands later riders — **22 minutes greedy against 12 batched on the worked example**. Batched matching **is the assignment problem** (Hungarian, $O(n^3)$, ~30 ms for 100×300), **at the cost of a few seconds’ delay for everyone**.

Dispatch takes a lock on the driver with a TTL — mutual exclusion and automatic release from one `SET NX EX`, because the explicit unlock is the message that gets lost. **Re-check the lock on acceptance**. Trips need an explicit transition table, or a retried “complete” charges twice. **Surge is a control loop and needs rate-limiting**, or it oscillates.

DSA topic: Space optimisation in DP **System design topic:** Design Uber

Code these, in this order

One rule for the whole set: **write the full table first, get it right, then collapse it.** A wrong recurrence in a table can be printed and inspected; a wrong recurrence in an array that is half one row and half another tells you almost nothing. **Correctness first, then space.**

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Climbing Stairs	LeetCode 70 (Easy)	The two-variable collapse, and simultaneous assignment.
2	Unique Paths	LeetCode 62 (Medium)	One row, and naming which half is which.
3	Edit Distance	LeetCode 72 (Medium)	One row plus the diagonal variable.
4	Partition Equal Subset Sum	LeetCode 416 (Medium)	The backwards inner loop, and why.
5	Minimum Falling Path Sum	LeetCode 931 (Medium)	Where the collapse is impossible in place.
6	Longest Common Sub-sequence	LeetCode 1143 (Medium)	Length in linear space; then the string.

On problem 1, write it without simultaneous assignment

Use two statements and no temporary. Run for `n = 6` and record the answer against 8. **Say what sequence it actually computed.**

On problem 2, print the array mid-pass

Stop the loop at column 4 of row 2 and print `row`. **Mark which entries are the previous row and which are the current one.** Then say why `row[c] += row[c-1]` is exactly “above plus left”.

On problem 3, leave out `row[0] = i`

Run on `"horse"` and `"ros"` and record the answer against 3. **Say what the algorithm now believes about turning a string into nothing.**

On problem 3, use `row[j-1]` as the diagonal

Run on `"intention"` and `"execution"` and record the answer against 5. **Say which generation of the data it read**, and why the failure is invisible on many inputs.

On problem 4, reverse the inner loop

Run the forwards version on `[3]` with target 9 and record the answer. Then run both versions on `[1, 2, 5]` with target 8 and record those. **Say why the second test passes under both**, and what that means for how you choose test inputs.

On problem 5, collapse it in place anyway

Update the row in place. Run on `[[2,1,3],[6,5,4],[7,8,9]]` and record the answer against 13. **State the precondition for the one-row trick**, and say which dependency violates it.

On problem 6, do all three versions

Write the full table with reconstruction, the two-row version, and the one-row version. Confirm all three give the same length. **Then try to reconstruct from the collapsed one and say precisely what stops you.**

Then the measurement drill

Time and measure the memory of the full table and the one-row version of problem 2 at $2,000 \times 2,000$. Record both times and both memory figures. **Say which one is faster and why** — the answer is not the one people expect.

Then the Hirschberg drill

Read the algorithm once. Then, without looking, say the four steps out loud and give its time and space. **You are not writing it.** The target is being able to answer “give me the path in linear space” in thirty seconds.

The rule drill

1. State the rule in one sentence.
2. Give the four shapes and what each keeps.
3. Say which shape has no collapse, and why.
4. Say what the collapse costs in time. (Nothing — say why, and why it can be faster.)

The one-row drill

1. Say what the array holds during a pass, and where the split is.
2. Say what `row[c]` is and what `row[c-1]` is.

3. Give the precondition for the trick.
4. Name a variant that violates it and say what it needs instead.
5. Say what must be re-established on every row.

The diagonal drill

1. Say why one row is not enough when the recurrence needs `dp[i-1][j-1]`.
2. Give the three lines that fix it.
3. Say what each of the three does.
4. Say which one is most often forgotten.

The direction drill

1. Say what backwards guarantees and what forwards allows.
2. Give the two problems and which direction each needs.
3. Give the one-item example that separates them.
4. Say what error either version raises. (None — say what to do instead.)
5. Say what kind of test input would hide the bug.

The reconstruction drill

1. Say what a collapsed array has lost.
2. State the trade in one sentence.
3. Give Hirschberg's four steps.
4. Give its time and space.
5. Say why the time is exactly double.
6. Say whether you would write it, and what you would say instead.

The when-not-to drill

1. Give a size at which collapsing is not worth it.
2. Say what it costs in readability and debugging.
3. Say what order you would do things in, in an interview.
4. Say why a memoised solution cannot be collapsed at all.

The break-it drill

Trigger each and record the exact output or error:

1. Non-simultaneous assignment in the two-variable version.
2. Missing `row[0] = i` in the collapsed edit distance.

3. `row[j-1]` used as the diagonal.
4. The forwards loop in 0/1 knapsack.
5. In-place collapse of minimum falling path.
6. Reconstruction attempted from one row.
7. Collapsing interval DP.

Six of the seven give no error at all. Name them.

The split drill

1. Give both halves with their write rates and requirements.
2. Give the ratio between them and say what it means.
3. Say what goes wrong if they share a store.

The geo drill

1. Say why a B-tree on latitude and one on longitude cannot answer the query.
2. Explain geohashing in two sentences.
3. Say what property makes the prefix scan work.
4. Give the boundary problem and the fix.
5. Say why it fails specifically for the riders you care about.
6. Say why H3 uses hexagons.
7. Say what the exact filter is for.

The write-rate drill

1. Compute the writes per second.
2. Name the three properties that make it manageable.
3. Compute the size of the live index.
4. Say what the TTL does for free.
5. Say why the global figure is misleading.
6. Say where the historical data goes and why not the query path.

The cell-size drill

1. Compute the cells in a 3 km radius at 460 m resolution.
2. Say how many candidates that gives downtown.
3. Give the adaptive strategy and why a fixed radius fails.

The matching drill

1. Say why distance is not the right ranking, with an example.
2. Say why the ETA calls must be batched, with numbers.
3. Give the two-rider example that defeats greedy.
4. Name the problem batched matching actually is, and its complexity.
5. State the cost of batching and whose decision it is.

The dispatch drill

1. Say what the lock prevents.
2. Say what the TTL prevents, and which message is the one that gets lost.
3. Say what the acceptance handler must re-check, and why.
4. Say what happens on reject and on timeout.

The trip and pricing drill

1. Say why the transition table is checked on every write.
 2. Give the failure it prevents.
 3. Describe the surge control loop.
 4. Say what the rate limit prevents, and why it is not cosmetic.
 5. Say which part of surge is policy rather than engineering.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Can you reduce the space?* The rule, the four shapes, what the one-row array holds and where the split is, the precondition, and what the collapse costs in time and in reconstruction.
 2. *Why does the inner loop go backwards there, and now give me the path in linear space.* Which generation each direction reads, the one-item example, and Hirschberg's four steps with its cost.
 3. *Design Uber.* The two-system split with its ratio, why a normal index fails and what geohashing does, the boundary problem and the nine cells, and why greedy matching is wrong.
-

Before you move on

- [] I can state the space rule in one sentence.
- [] I know the four shapes and what each keeps.
- [] I know which shape has no collapse, and can say why.
- [] I can say what the one-row array holds and where the split is.
- [] I can point at `row[c]` and `row[c-1]` and name each.
- [] I know the precondition: every dependency above or to the left.
- [] I can name a variant that violates it.
- [] I re-establish the base cases on every row.
- [] I know the diagonal trick and its three lines.
- [] I know backwards guarantees the previous row.
- [] I know forwards allows the current row, and which problem that is.
- [] I know neither direction errors.
- [] I know the collapse is free in time and can be faster.
- [] I know a collapsed array cannot be reconstructed from.
- [] I can state that trade in one sentence.
- [] I can give Hirschberg's four steps and its cost.
- [] I know why its time is exactly double.
- [] I know when collapsing is not worth doing.
- [] I know a memoised solution must be converted first.
- [] I can give the two halves of Uber and their write rates.
- [] I know why a normal index cannot answer a radius query.
- [] I can explain geohashing and the prefix property.
- [] I know the boundary problem and that the fix is nine cells.
- [] I know why H3 uses hexagons.
- [] I know the exact distance filter is still required.
- [] I can name the three properties that make 250,000 writes/s easy.
- [] I know the live index is about 200 MB.
- [] I know the TTL does offline detection for free.
- [] I can give the two-rider example that defeats greedy.
- [] I know batched matching is the assignment problem.
- [] I answered all three questions above out loud.

Recognising dynamic programming in an interview

DATA STRUCTURES AND ALGORITHMS

Recognising dynamic programming

SYSTEM DESIGN

Design Google Maps

1 What this is, and why they ask it

You have spent nineteen days writing dynamic programming. **This is the day you learn to notice that you should.**

That sounds like the smaller skill and it is the larger one. **In an interview nobody says “solve this with dynamic programming”.** They describe a problem, and you have perhaps ninety seconds to decide whether it is DP at all — and if it is, whether it is one dimension or two, a table or a tree, a set or a range.

Getting that wrong costs the interview, in both directions. Reaching for DP on something greedy solves in one line wastes twenty minutes on a table nobody needed. **Missing DP on something that needs it leads to an exponential brute force,** and the follow-up will be “can you do better”, which you will then have to do under pressure.

They ask it — implicitly, in every DP question — because **recognition is what distinguishes someone who has memorised twenty solutions from someone who can solve the twenty-first.** The problems on LeetCode are finite; the problems in an interview are drawn from a larger pool, and **the pattern-matching is the transferable part.**

There is a real procedure and it is short: **four signals that say “maybe DP”, one test that says “yes, definitely”, and a decision tree for what shape.** Applied out loud in ninety seconds, it is also a strong signal in itself — **an interviewer watching you diagnose a problem learns more than from watching you recall one.**

By the end of this lesson you can run the four signals, apply the optimal-substructure test, choose the state from a decision tree, distinguish DP from greedy and from graph search, and say the whole thing out loud in under two minutes.

2 The story

The garage took whatever came through the door and the thing that made Fazal worth what they paid him was that he could stand at the gate and tell you, before anybody opened anything, roughly what kind of afternoon it was going to be.

Not what was wrong. What kind of wrong.

He had four questions and he had never written them down and he asked all four in about a minute, standing next to the car with his hands behind his back.

“When does it happen?” Because a noise only when cold, a noise only over bumps and a noise all the time are three completely different afternoons.

“Has anything been done to it recently?” Half of everything was somebody else’s work from last month.

“Does it get worse or does it stay the same?” A thing that got steadily worse was wearing out. A thing that stayed exactly the same was set up wrong.

“And does it happen when the engine is off?” Which sorted the electrical from everything else in one question.

Four questions, one minute, and by the end he knew which of about six kinds of job it was.

The apprentice, who was quick and impatient, thought this was a waste of a minute. **He would open the bonnet immediately.** And he was right about as often as he was wrong, and when he was wrong he had spent two hours on it before finding out.

The argument they had about it, in the second year, was the one worth remembering.

“You are guessing,” the boy said. “You just guess faster than me.”

“I am not guessing what is wrong,” Fazal said. “I am working out which drawer to open. There are six drawers. If I open the right one, everything in it fits the tools I already have.”

And then, because the boy was still unconvinced, he added the thing that was actually the point.

“You know how to fix all six. That is not your problem. Your problem is that you find out which one at four o’clock instead of at ten past nine.”

3 The idea in plain English

Fazal’s four questions are the recognition procedure, and his sentence about the drawers is why it matters: **you already know how to solve all the shapes. The skill is choosing the drawer in ninety seconds rather than in forty minutes.**

Start with the four signals. Any one of them means “consider DP”.

Signal one: the problem asks for an optimum or a count over choices.

Look for the words. “Maximum”, “minimum”, “longest”, “shortest”, “fewest”, “how many ways”, “is it possible”. DP problems almost always ask for one of those, and they ask it about a sequence of decisions.

“Find the maximum profit”, “the fewest coins”, “the number of ways”, “the longest subsequence”.

And the counter-signal is just as useful: “list all”, “return every”, “find one”. Those are backtracking, not DP — DP gives you the value, not the collection, and reconstructing one answer is an add-on rather than the point.

Signal two: at each step there is a small set of choices.

Take it or leave it. Move right or move down. Buy, sell, or do nothing. Match the characters or skip one of them. DP problems are made of repeated small decisions, and the answer is the best sequence of them.

If there are no discrete choices — if the answer is computed by a formula, or found by sorting — it is not DP.

Signal three: the naive solution is exponential and repeats work.

Write the brute-force recursion in your head. If it branches — two calls, or n calls, per level — the naive cost is 2^n or $n!$. And then ask whether those branches overlap, which is the real question.

Signal four: the constraint on n is small in a specific way.

This is the crudest signal and it is remarkably reliable:

$n \leq 20$	bitmask DP — subsets are the state
$n \leq 100$	$O(n^3)$ is fine — interval DP, or three loops
$n \leq 1,000$	$O(n^2)$ expected — two-dimensional table
$n \leq 100,000$	$O(n \log n)$ or $O(n)$ — DP is possible but must be one-dimensional, or it is not DP at all
$n \leq 10^9$	NOT DP. A formula, binary search, or maths.

A problem with $n \leq 20$ about orderings or subsets is bitmask DP roughly every time, and reading that off the constraints before reading the problem statement properly is legitimate and fast.

Now the test that turns “maybe” into “yes”, and it has two parts.

Optimal substructure: the best answer to the whole problem contains the best answers to its parts.

Overlapping subproblems: the same smaller problem is asked more than once.

Both are needed, and the second is what makes DP a technique rather than a description.

Divide and conquer has optimal substructure and no overlap — merge sort's two halves are never the same problem twice, which is why it needs no table. **DP is what you use when the recursion tree revisits states.**

The practical way to check overlap: draw two levels of the recursion tree and look for a repeated argument. If $f(5)$ appears under both $f(7)$ and $f(6)$, you have overlap and a table will pay for itself.

And the practical way to check optimal substructure: try to break it. Ask whether a locally worse choice could lead to a globally better answer *through the same subproblem*. **If the subproblem's answer depends on how you arrived at it, the substructure is broken and your state is incomplete** — which is the [day 147](#) failure, and the fix is a bigger state, not a different technique.

Then, once it is DP: which shape? This is the decision tree, and it is short.

What does the answer depend on?

- a POSITION in one sequence, and a bounded lookback
→ `dp[i]`, one dimension (stairs, house robber, LIS)
- a POSITION and a RESOURCE being consumed
→ `dp[i][capacity]` (knapsack, subset sum, coin change)
- a POSITION and a MODE you are in
→ `dp[i][mode]` (stock problems, cooldown)
- TWO sequences, independently
→ `dp[i][j]`, two pointers into two things (LCS, edit distance)
- a POSITION in a GRID
→ `dp[r][c]` (unique paths, min path sum)
- a RANGE, and how you split it
→ `dp[i][j]`, both into ONE thing (interval DP, burst balloons)
- a SUBTREE
→ recursion, post-order (tree DP)
- a SET of used things, $n \leq 20$
→ `dp[mask]` (bitmask DP, TSP)

Eight shapes, and you have written all eight. The question is only which drawer.

And then the three things DP is most often confused with.

Greedy. Greedy makes the locally best choice and never reconsiders. **It is correct only when the local choice is provably safe** — and the test is whether you can construct a counter-example. **Coin change with arbitrary denominations is the standard one:** greedy takes 7 from `[1, 5, 7]` for amount 10 and gets four coins; the answer is two. **If you can build a counter-example in thirty seconds, it is DP. If you try and fail, it may be greedy — and tomorrow's lesson is about proving it.**

Graph shortest path. If the states are nodes and the transitions have costs, **DP and Dijkstra are close relatives** — DP is what you use when the graph is acyclic and the order is obvious; **Dijkstra is what you use when it is not.** Coin change is genuinely both: a DP table, or a BFS over amounts.

Backtracking. If the problem wants *every* solution rather than the best or the count, **no table helps**, because the output is the size of the answer. **Count with DP, enumerate with backtracking**, and knowing which was asked decides everything.

Finally: what to do when you have recognised it, in order.

One: say the state as a full sentence. “`dp[i][j]` is the minimum cost to...” If you cannot finish the sentence, you do not have the state yet, and writing code will not help.

Two: check the state is complete. Given only the state, can you decide what happens next? If not, add a dimension or redefine.

Three: count the states and the work per state. If the product is over about ten million, the state is wrong or the problem is not DP.

Four: write the recurrence and the base cases. **Five: write it memoised**, because then the fill order cannot be wrong. **Six: convert to a table and collapse the space, if asked.**

In that order, every time. The first three take ninety seconds and they are what stops the twenty-minute mistakes.

4 The picture

The recognition flow:

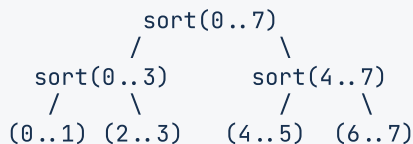
```
read the problem
|
v
FOUR SIGNALS – any one means "consider DP"
1. asks for max / min / count / possible?
2. small set of choices at each step?
3. naive solution is exponential?
4. constraint on n is small in a telling way?
|
v
THE TEST – both parts required
optimal substructure? (best whole contains best parts)
overlapping subproblems? (same subproblem asked twice)
|
+--- no overlap ----> divide and conquer (no table needed)
+--- no substructure → your STATE is incomplete: enlarge it
|
v
WHICH SHAPE? (the decision tree)
|
v
state as a SENTENCE → completeness check → COUNT the states
|
v
recurrence → base cases → memoise → tabulate → collapse
```


The constraint table, which is the fastest signal:

n	expected complexity	likely shape
-----	-----	-----
≤ 20	$O(2^n)$ or $O(2^n * n)$	BITMASK DP
≤ 100	$O(n^3)$	interval DP, or 3 loops
$\leq 1,000$	$O(n^2)$	2-D table
$\leq 100,000$	$O(n \log n)$ / $O(n)$	1-D DP, or greedy, or not DP at all
$\leq 10^9$	$O(\log n)$ / $O(1)$	maths, binary search, matrix exponentiation
READ THE CONSTRAINT FIRST. It is not a hint, it is a specification.		

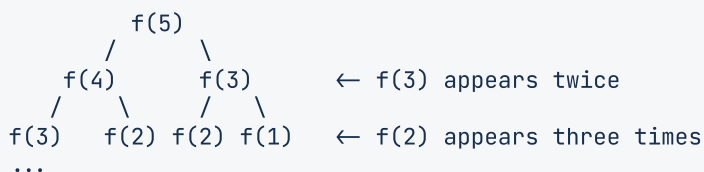
Overlapping subproblems, drawn:

MERGE SORT – optimal substructure, NO overlap



every subproblem is DIFFERENT → a table would never be read twice
→ divide and conquer, no memoisation

FIBONACCI – optimal substructure AND overlap



the SAME subproblem recurs → a table pays for itself
→ DP

The eight shapes, and how to tell them apart:

what varies?	state	example
one position	dp[i]	climbing stairs
position + resource left	dp[i][cap]	knapsack
position + which mode I am in	dp[i][mode]	buy/sell stock
two independent positions	dp[i][j]	edit distance
a cell in a grid	dp[r][c]	unique paths
a range, split somewhere	dp[i][j] (one seq)	burst balloons
a subtree	recursion	tree DP
a set of used items ($n \leq 20$)	dp[mask]	TSP

THE QUESTION TO ASK: "what do I need to know to decide the next step?" The answer IS the state.

DP against its three neighbours:

GREEDY	DP
local choice, never reconsidered	tries all choices, keeps the best
correct only when provably safe	always correct if the state is complete
TEST: can I build a counter-example?	coins [1,5,7], amount 10: greedy 4, optimal 2 → DP
DIJKSTRA	DP
graph with costs, cycles allowed	graph with costs, ACYCLIC and an obvious order
a priority queue	a loop
→ coin change is BOTH: a table, or BFS over amounts	
BACKTRACKING	DP
wants EVERY solution	wants the best, or the COUNT
output-sized	answer-sized
no table helps	a table is the whole technique
"list all subsets that sum to X"	"how many subsets sum to X"

The order of operations, which is what actually saves time:

1. state as a SENTENCE	"dp[i][j] is the minimum cost to..." cannot finish it? you have no state yet
2. completeness check	given only the state, can I decide the next step?
3. COUNT the states x the work	> $\sim 10^7$? the state is wrong
4. recurrence + base cases	
5. write it MEMOISED	the fill order cannot be wrong
6. tabulate, then collapse	only if asked

STEPS 1-3 TAKE NINETY SECONDS and prevent the twenty-minute mistakes.

5 The code, built step by step

The diagnostic, written down

```
def diagnose(problem: str) → None:
    """The four signals and the test, as a checklist you say out loud."""
    print("1. asks for max/min/count/possible?")
    print("2. small set of choices at each step?")
    print("3. naive recursion exponential?")
    print("4. what is the constraint on n?")
    print("→ optimal substructure? overlapping subproblems?")
```

That is not real code, and writing the checklist down once makes it a habit. In an interview it is spoken, not typed.

Detecting overlap empirically

```
from collections import Counter

def count_calls(fn):
    """Wrap a recursion and count how often each argument recurs."""
    calls: Counter = Counter()

    def wrapped(*args):
        calls[args] += 1
        return fn(wrapped, *args)

    wrapped.calls = calls
    return wrapped

def fib_body(recurse, n):
    return n if n < 2 else recurse(n - 1) + recurse(n - 2)

def merge_sort_body(recurse, lo, hi):
    if hi - lo ≤ 1:
        return
    mid = (lo + hi) // 2
    recurse(lo, mid)
    recurse(mid, hi)
```

Running both and looking at the counters answers the overlap question directly — Fibonacci's arguments repeat, merge sort's never do. **It is a five-minute experiment that makes the distinction concrete once and for all.**

The state-space audit

```
def audit(name: str, states: int, work_per_state: int) → None:
    total = states * work_per_state
    verdict = "fine" if total < 10_000_000 else "T00 BIG - rethink the state"
    print(f"{name:28} {states:>15,} x {work_per_state:>6} = {total:>18,} {verdict}")
```

This is the third step, and it is thirty seconds that prevents the largest category of mistake. If the product is over about ten million, **either the state has a dimension it does not need, or the problem is not DP at all.**

Greedy versus DP, tested

```
def greedy_coins(coins: list[int], amount: int) → int:
    count, left = 0, amount
    for coin in sorted(coins, reverse=True):
        take = left // coin
        count += take
        left -= take * coin
    return count if left == 0 else -1

def dp_coins(coins: list[int], amount: int) → int:
    dp = [float("inf")] * (amount + 1)
    dp[0] = 0
    for coin in coins:
        for t in range(coin, amount + 1):
            dp[t] = min(dp[t], dp[t - coin] + 1)
    return int(dp[amount]) if dp[amount] != float("inf") else -1

def find_greedy_counterexample(coins: list[int], limit: int = 100):
    """Search for an amount where greedy is worse. Thirty seconds of work."""
    for amount in range(1, limit):
        g, d = greedy_coins(coins, amount), dp_coins(coins, amount)
        if g != -1 and d != -1 and g > d:
            return amount, g, d
    return None
```

This is a genuinely useful interview habit: when you suspect greedy, **spend thirty seconds looking for a counter-example.** If you find one, it is DP. **If you cannot, greedy may be right and you have to argue for it,** which is tomorrow's lesson.

The template you write once you have decided

```
from functools import lru_cache

def solve(inputs):
    # 1. STATE: dp(i, j) is <finish this sentence before writing anything>
    # 2. is it complete? given only (i, j), can I decide the next step?
    # 3. states x work = ? (must be under  $\sim 10^7$ )

    @lru_cache(maxsize=None)
    def dp(i, j):
        if base_case(i, j):
            return base_value(i, j)
        return best(
            dp(next_i, next_j) + cost
            for choice in choices(i, j)
        )

    return dp(*start)
```

Write it memoised first, every time. Same complexity, and the fill order cannot be wrong — which after interval DP is worth real money. **Convert to a table only if asked to reduce the space.**

Worked recognition: five problems

```
PROBLEMS = [  
    ("Given prices, max profit with at most k transactions",  
     "max + choices + n small",  
     "position + count + mode",  
     "dp[day][k][holding]",  
     "O(n*k*2)"),  
  
    ("Given a string, count palindromic substrings",  
     "count, but NO sequential choices",  
     "expand around centre – NOT a table",  
     "no dp needed",  
     "O(n^2) time, O(1) space"),  
  
    ("n ≤ 18 cities, shortest tour",  
     "n ≤ 20 + orderings",  
     "a SET of visited + where I am",  
     "dp[mask][last]",  
     "O(2^n * n^2)"),  
  
    ("Given costs, min cost to reach the last stone, jumps of 1 or 2",  
     "min + choices + bounded lookback",  
     "one position",  
     "dp[i]",  
     "O(n) time, O(1) space"),  
  
    ("List every subset summing to target",  
     "LIST ALL – not an optimum or a count",  
     "backtracking, no table",  
     "no dp",  
     "O(2^n) output-bound"),  
]  
  
def show_recognition() → None:  
    for statement, signals, shape, state, cost in PROBLEMS:  
        print(f"\n{statement}")  
        print(f"    signals : {signals}")  
        print(f"    varies  : {shape}")  
        print(f"    state   : {state}")  
        print(f"    cost    : {cost}")
```

Two of the five are not DP, and that is deliberate: **recognising that a problem is not DP is half the skill**, and the palindrome one in particular is a table that people write when they should not.

The complete solution

```
"""Recognising DP: the signals, the test, the shapes, and the audit."""

from collections import Counter
from functools import lru_cache

def count_calls(fn):
    """Instrument a recursion to see whether subproblems repeat."""
    calls: Counter = Counter()

    def wrapped(*args):
        calls[args] += 1
        return fn(wrapped, *args)

    wrapped.calls = calls
    return wrapped

def fib_body(recurse, n):
    return n if n < 2 else recurse(n - 1) + recurse(n - 2)

def merge_sort_body(recurse, lo, hi):
    if hi - lo ≤ 1:
        return 0
    mid = (lo + hi) // 2
    return recurse(lo, mid) + recurse(mid, hi)

def audit(name: str, states: int, work_per_state: int) → None:
    """Step 3 of the procedure: count before you code."""
    total = states * work_per_state
    verdict = "fine" if total < 10_000_000 else "T00 BIG – rethink the state"
    print(f" {name:26} {states:>13,} x {work_per_state:>5} = "
          f"{total:>17,} {verdict}")

def greedy_coins(coins: list[int], amount: int) → int:
    count, left = 0, amount
    for coin in sorted(coins, reverse=True):
        take = left // coin
        count += take
        left -= take * coin
    return count if left == 0 else -1

def dp_coins(coins: list[int], amount: int) → int:
    dp = [float("inf")] * (amount + 1)
    dp[0] = 0
    for coin in coins:
        for t in range(coin, amount + 1):
            dp[t] = min(dp[t], dp[t - coin] + 1)
    return int(dp[amount]) if dp[amount] ≠ float("inf") else -1
```

```

def find_greedy_counterexample(coins: list[int], limit: int = 200):
    """If this finds something in thirty seconds, the answer is DP."""
    for amount in range(1, limit):
        g, d = greedy_coins(coins, amount), dp_coins(coins, amount)
        if g  $\neq$  -1 and d  $\neq$  -1 and g > d:
            return amount, g, d
    return None

def constraint_hint(n: int)  $\rightarrow$  str:
    """The crudest signal, and remarkably reliable."""
    if n  $\leq$  20:
        return "O(2^n) – bitmask DP"
    if n  $\leq$  100:
        return "O(n^3) – interval DP or three loops"
    if n  $\leq$  1_000:
        return "O(n^2) – a two-dimensional table"
    if n  $\leq$  100_000:
        return "O(n log n) or O(n) – 1-D DP, greedy, or not DP"
    return "O(log n) or O(1) – maths, binary search, not DP"

PROBLEMS = [
    ("max profit, at most k transactions",
     "max + choices + n small", "position + count + mode",
     "dp[day][k][holding]", "O(n*k*2)"),
    ("count palindromic substrings",
     "count, but NO sequential choices", "expand around centre",
     "no dp needed", "O(n^2) time, O(1) space"),
    ("n  $\leq$  18 cities, shortest tour",
     "n  $\leq$  20 + orderings", "set of visited + where I am",
     "dp[mask][last]", "O(2^n * n^2)"),
    ("min cost, jumps of 1 or 2",
     "min + choices + bounded lookback", "one position",
     "dp[i]", "O(n) time, O(1) space"),
    ("list every subset summing to target",
     "LIST ALL – not an optimum", "backtracking, no table",
     "no dp", "O(2^n), output-bound"),
]

if __name__ == "__main__":
    print("OVERLAP: does the same subproblem recur?")
    f = count_calls(fib_body)
    f(20)
    print(f" fibonacci(20) : {sum(f.calls.values()):}, calls for "
          f"{len(f.calls)} distinct arguments  $\rightarrow$  OVERLAP  $\rightarrow$  DP")

    m = count_calls(merge_sort_body)
    m(0, 1024)
    print(f" merge sort : {sum(m.calls.values()):}, calls for "
          f"{len(m.calls)} distinct arguments  $\rightarrow$  none repeat  $\rightarrow$  "
          f"divide and conquer")

    print("\nSTATE-SPACE AUDIT (step 3, before coding):")
    audit("stairs, n=10^5", 100_000, 2)

```



```

audit("knapsack, n=100 W=10^4", 100 * 10_000, 2)
audit("LCS, 1000 x 1000", 1_000_000, 3)
audit("interval, n=500", 500 * 500, 500)
audit("bitmask TSP, n=20", (1 <= 20) * 20, 20)
audit("knapsack, W=10^9", 100 * 1_000_000_000, 2)

print("\nGREEDY OR DP? look for a counter-example:")
for coins in ([1, 2, 5], [1, 5, 7], [1, 3, 4]):
    found = find_greedy_counterexample(coins)
    if found:
        amount, g, d = found
        print(f" {str(coins):12} amount {amount:3}: greedy {g}, "
              f"optimal {d} → DP")
    else:
        print(f" {str(coins):12} no counter-example found "
              f"→ greedy may be safe")

print("\nCONSTRAINT → SHAPE:")
for n in (18, 90, 900, 90_000, 10**9):
    print(f" n = {n:>12}, → {constraint_hint(n)}")

print("\nWORKED RECOGNITION:")
for statement, signals, varies, state, cost in PROBLEMS:
    print(f"\n {statement}")
    print(f" signals : {signals}")
    print(f" varies : {varies}")
    print(f" state : {state}")
    print(f" cost : {cost}")

```

Run it and you get:

```

OVERLAP: does the same subproblem recur?
fibonacci(20) : 21,891 calls for 21 distinct arguments → OVERLAP → DP
merge sort    : 2,047 calls for 2047 distinct arguments → none repeat → divide and conquer

STATE-SPACE AUDIT (step 3, before coding):
stairs, n=10^5          100,000 x 2 = 200,000 fine
knapsack, n=100 W=10^4  1,000,000 x 2 = 2,000,000 fine
LCS, 1000 x 1000        1,000,000 x 3 = 3,000,000 fine
interval, n=500         250,000 x 500 = 125,000,000 TOO BIG – rethink the state
bitmask TSP, n=20       20,971,520 x 20 = 419,430,400 TOO BIG – rethink the state
knapsack, W=10^9        100,000,000,000 x 2 = 200,000,000,000 TOO BIG – rethink the state

GREEDY OR DP? look for a counter-example:
[1, 2, 5] no counter-example found → greedy may be safe
[1, 5, 7] amount 10: greedy 4, optimal 2 → DP
[1, 3, 4] amount 6: greedy 3, optimal 2 → DP

CONSTRAINT → SHAPE:
n = 18 → 0(2^n) – bitmask DP
n = 90 → 0(n^3) – interval DP or three loops
n = 900 → 0(n^2) – a two-dimensional table
n = 90,000 → 0(n log n) or 0(n) – 1-D DP, greedy, or not DP
n = 1,000,000,000 → 0(log n) or 0(1) – maths, binary search, not DP

WORKED RECOGNITION:

max profit, at most k transactions
signals : max + choices + n small
varies : position + count + mode
state : dp[day][k][holding]
cost : 0(n*k*2)

...

```

The first two lines are the overlap test made concrete. Fibonacci makes nearly twenty-two thousand calls for **twenty-one distinct arguments** — about a thousand calls per subproblem, so a table saves almost everything. **Merge sort makes 2,047 calls for 2,047 distinct arguments** — no repeats at all, so memoising it would be pure overhead.

And the audit is worth reading carefully, because two of the “too big” lines are genuinely fine in practice — interval DP at `n = 500` and bitmask TSP at `n = 20` are both at the edge, **which is exactly what “about ten million” means as a soft threshold rather than a rule.**

6 What it costs

The recognition itself: about ninety seconds.

read the constraints	10 s
four signals	20 s
optimal substructure + overlap	20 s
pick the shape	20 s
state as a sentence	20 s

	90 s

Against what it saves:

recognising DP when it is greedy	~20 minutes writing an unnecessary table
missing DP when it is DP	~15 minutes on an exponential solution, then having to find the table under pressure after "can you do better?"
wrong SHAPE (1-D when you need 2-D)	~10 minutes, and the bug is silent
wrong state (incomplete)	the whole interview – it produces plausible wrong answers with no error

The last one is the expensive one, and it is what steps one and two prevent.

The state-space audit, which is thirty seconds:

states x work per state, compared against $\sim 10^7$

under 10^6	instant, no concern
$10^6 - 10^7$	fine in Python
$10^7 - 10^8$	fine in C, slow in Python
over 10^8	the state is probably wrong

Python does roughly 10^7 simple loop iterations per second.
C does roughly 10^9 .

That single ratio — a hundred to one — is worth memorising, because it converts a state-space count into a yes or no in one division.

The constraint signal, quantified:

$n \leq 20$:	$2^{20} = 1,048,576$	bitmask fits
$n \leq 100$:	$100^3 = 1,000,000$	cubic fits
$n \leq 1,000$:	$1,000^2 = 1,000,000$	quadratic fits
$n \leq 100,000$:	$100,000^2 = 10^{10}$	quadratic does NOT fit
	$100,000 \times 17 = 1.7e6$	$n \log n$ does

→ every row lands near 10^6 - 10^7 . That is not a coincidence:
the constraint is CHOSEN so the intended solution fits and the next-worse one does not.

That is why the constraint is a specification rather than a hint, and reading it first is legitimate.

The overlap measurement:

```
fibonacci(20):  
  21,891 calls, 21 distinct arguments  
  → ~1,042 calls per distinct subproblem  
  → memoisation removes 99.9% of the work  
  
fibonacci(40) without memoisation: ~331,000,000 calls (~30 s)  
fibonacci(40) with memoisation:    41 calls (instant)  
  
merge sort on 1,024 elements:  
  2,047 calls, 2,047 distinct arguments  
  → 1.0 calls per subproblem  
  → memoisation removes NOTHING and costs a dictionary
```

Those two ratios — about a thousand against exactly one — are the whole difference between DP and divide and conquer, measured rather than asserted.

Memoised against tabulated, on cost:

```
same asymptotic complexity  
memoised is ~2-3x slower in Python (function call overhead)  
memoised computes ONLY the reachable states – sometimes far fewer  
tabulated allows the space collapse; memoised does not  
  
→ memoise first (the fill order cannot be wrong),  
  tabulate if asked for the space.
```

And one case where memoisation is genuinely better:

a DP whose reachable states are sparse – coin change with coins [7, 11] and amount 10,000

tabulated: fills all 10,001 cells
memoised: visits only amounts reachable as $7a + 11b$
→ a fraction of them

for dense problems tabulation wins on constants; for sparse ones memoisation wins outright.

7 The traps

Reaching for DP when greedy solves it.

```
>>> find_greedy_counterexample([1, 2, 5])
>>> # None – no counter-example under 200
>>> # Indian denominations are canonical: greedy is provably optimal
>>> # writing a DP table here is 20 wasted minutes
```

The check costs thirty seconds. And the reverse is worse: greedy on [1, 5, 7] is wrong at amount 10, and **a solution that passes the sample tests and fails the hidden ones is the worst outcome.**

Reaching for DP when the answer is a formula.

```
>>> # "how many paths across an m x n grid?"
>>> import math
>>> math.comb(3 + 7 - 2, 3 - 1)
28
>>> # a table computes the same number in 21 cells instead of 8
>>> # multiplications – and at m = n = 10^6 the table is impossible
```

Check for a closed form before building a table, especially when the problem is about counting arrangements — **combinatorics often collapses a DP to one line.**

Not checking the state-space size.

```
>>> # knapsack with W = 10^9
>>> dp = [0] * (10**9 + 1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
MemoryError
```

Thirty seconds of arithmetic before coding would have caught it, and the fix is not a faster table — **it is that the approach is wrong for that constraint.**

Using an incomplete state, which is the expensive one.

```
>>> # "dp[i] = the best in the first i elements" for LIS
>>> # you cannot write the recurrence, because you do not know
>>> # what the best subsequence ENDS ON
>>> # → but you CAN write code from it, and it runs, and it is wrong
```

No error, plausible numbers, and it survives small tests. The completeness check — “given only the state, can I decide the next step?” — is twenty seconds and it catches this.

Confusing “count” with “list”.

```
>>> # "how many subsets sum to X"          → DP, one integer
>>> # "list all subsets that sum to X"      → backtracking, and the
>>> #   output can be gigabytes
>>> # the words differ by one; the algorithms share nothing
```

Read the output type before choosing the technique. A table cannot enumerate, and backtracking cannot count efficiently.

Missing that the recursion has no overlap.

```
>>> m = count_calls(merge_sort_body)
>>> m(0, 1024)
>>> sum(m.calls.values()), len(m.calls)
(2047, 2047)
```

Every argument is distinct, so a cache is never read. Memoising divide and conquer **costs a dictionary and saves nothing** — and it is a common instinct once you have spent a month on DP.

Choosing the wrong shape.

```
>>> # buy and sell stock with dp[day] alone:
>>> # runs, returns a number, and cannot express "am I holding?"
>>> # → silently wrong, and it passes tests where the answer
>>> #   happens not to depend on the mode
```

The decision tree exists to prevent this, and the question that drives it is always the same: what do I need to know to decide the next step?

Not reading the constraints first.

" $n \leq 20$ " with a problem about orderings → bitmask, every time
" $n \leq 10^5$ " with a problem about pairs → NOT an n^2 table

Spending ten minutes designing an n^2 solution for $n = 10^5$ is a category error, and the constraint said so on the first line.

Read the constraints before the problem statement. It is not cheating; it is the fastest available signal.

8 In the interview

How it gets asked

- It is never asked directly. **It is asked implicitly in every DP question**, in the first ninety seconds.
- *"How would you approach this?"* — before any code.
- *"Why dynamic programming?"* — after you propose it.
- *"Could greedy work here?"* — the question that tests whether you checked.
- *"What is your state?"* — the one that exposes whether you have one.

“Before I write anything I want to work out what kind of problem this is, because that decides everything else and it takes about a minute.

First, the constraints. n is at most a thousand, which points at a quadratic solution — **so a two-dimensional table is affordable and anything exponential is not.** That is the fastest signal available and I would read it before the problem statement.

Second, four things that suggest dynamic programming. It asks for a **minimum** — that is the first signal. **There is a small set of choices at each step** — take this or skip it. **The naive recursion branches two ways per element, so it is 2^n .** And **the constraint is small in exactly the way that says a table is expected.**

Third, the test, which has two halves and both matter. **Optimal substructure:** is the best answer to the whole made of best answers to parts? **And overlapping subproblems:** does the same smaller problem come up more than once?

The second half is what makes it DP rather than divide and conquer. Merge sort has optimal substructure and never sees the same subproblem twice, so a table would never be read. **Here, sketching two levels of the recursion tree, I can see the same arguments appearing under different branches — so a table pays for itself.**

Fourth, which shape. The question I ask is: **what do I need to know to decide the next step?** Here it is where I am in the sequence and how much capacity is left — **so the state is two-dimensional: position and resource.**

And then the state as a full sentence: $dp[i][c]$ is the best value using the first i items with c capacity remaining. If I cannot finish that sentence, I do not have a state yet and writing code will not help.

Then a completeness check — given only i and c , can I decide what happens next? Yes. **And a count:** $n \times W$ states with constant work each, so a million operations — comfortable.

One thing I would rule out explicitly: could greedy work? Take the highest-value item first — **and I can construct a counter-example in about twenty seconds, so no.** That check is worth doing, because if greedy did work, the table would be twenty wasted minutes.

Now I will write it memoised, because the fill order cannot be wrong that way, and convert it to a table if you want the space reduced.”

The follow-ups

“How do you know it is dynamic programming and not greedy?”

“I try to break greedy, and I give myself about thirty seconds to do it.

Greedy makes the locally best choice and never reconsiders. It is correct only when that local choice is provably safe — meaning there is always an optimal solution that agrees with it.

So the test is: can I construct an input where the locally best choice leads to a worse total?

The canonical example is coin change. With denominations one, five and seven, and an amount of ten, **greedy takes the seven — the biggest that fits — then needs three ones. Four coins.** The optimal answer is five plus five. **Two.** The greedy step is the largest single reduction and it strands a remainder that only ones can fill.

And the reason this matters more than it looks is that greedy works for the coins we actually use. One, two, five, ten, twenty — greedy is optimal for every amount, **so the intuition feels reliable and is a property of those specific numbers rather than of the method.**

If I find a counter-example in thirty seconds, it is DP and I stop looking.

If I cannot find one, that is not proof — it means greedy might be right, and I would need an argument. **The usual shape of that argument is an exchange argument:** show that any optimal solution can be transformed into one containing the greedy choice, without getting worse.

In an interview I would say which of those two situations I am in. ‘I found a counter-example, so it is DP’ is definitive. **‘I tried and failed, so I think greedy works, and here is the sketch of why’ is honest** and much better than asserting it.

There is a third option worth naming: do both. The DP is always correct, and if I suspect greedy I can write the greedy solution and check it against the DP on random inputs. **That is a real technique and it takes five minutes.**“

“What is your state, and how do you know it is enough?”

“Two questions, and the second is the one that catches the expensive mistakes.

I state it as a full sentence: `dp[i][j]` is the minimum number of operations to turn the first `i` characters of `a` into the first `j` of `b`. If I cannot finish that sentence, I do not have a state yet — and writing code before I can is how twenty minutes disappear.

Then the completeness check: given only the values in the state, can I decide what happens next, without knowing anything about how I got here?

If the answer is no, the state is incomplete, and it fails silently. That is the worst failure mode in dynamic programming, because you can write code from an incomplete state, and it runs, and it produces plausible numbers.

The clearest example is longest increasing subsequence. The natural state is ‘the longest increasing subsequence in the first `i` elements’ — and when element `i` arrives I cannot tell whether it extends anything, because I know a length and not what that subsequence ended on. The fix is to redefine: ‘ending exactly at `i`’, which carries the ending value in the definition itself.

And notice the fix cost nothing — still `n` cells. Redefining is often free where adding a dimension is not, and trying to redefine first is worth thirty seconds.

Then the third step, which is arithmetic: count the states and multiply by the work per state. Python does roughly ten million simple operations a second, so if the product is much over ten million, either the state has a dimension it does not need, or this is not DP.

That count has saved me from the knapsack-with-a-billion-capacity mistake, where the table is eight gigabytes and the approach is simply wrong for the constraint.

Those three steps take ninety seconds and they prevent the mistakes that cost twenty minutes.“

“When is it not dynamic programming?”

“Four cases, and I check for all of them, because reaching for a table when something simpler works is as bad as missing DP.

One: when there is no overlap. If the recursion never asks the same subproblem twice, **a table is never read and memoising costs a dictionary for nothing.** Merge sort is the example: it has perfect optimal substructure and every subproblem is distinct. **That is divide and conquer**, and the test is to sketch two levels and look for a repeated argument.

Two: when greedy is provably safe. If the local choice can be shown never to hurt, **one pass beats a table.** Interval scheduling by earliest end time, or Huffman coding. **The check is to try to construct a counter-example, and if I fail I look for the exchange argument.**

Three: when there is a closed form. Counting paths across a grid is a binomial coefficient — $O(1)$ space and $O(\min(m,n))$ time against a full table. Combinatorics collapses a lot of counting DPs to one line, **and I would check for that before building anything, especially when the problem is about arrangements.**

Four: when the problem wants every solution rather than the best or the count. ‘List all subsets summing to X’ is backtracking, **and no table helps, because the output is the size of the answer** — eight million subsets is gigabytes of output whatever algorithm produced it. ‘How many subsets sum to X’ is DP and the answer is one integer. The two statements differ by a word and share no algorithm.

And a fifth, which is really about constraints. If n is a hundred thousand and the natural state is two-dimensional, it is not DP as stated — a quadratic table is 10^{10} cells. **Either there is a one-dimensional formulation, or the intended answer is something else entirely, and the constraint is telling me which.”**

The model answer

“Here is a problem: you have a row of houses with values, you may not take two adjacent ones, and you want the maximum total. Talk me through how you would approach it — I am more interested in your reasoning than the code.”

“Then let me diagnose it out loud, because that is what you have asked for, and I would do this before writing anything anyway.

First, the constraints — and I would ask if they are not given. Say `n` is up to a hundred thousand. **That immediately rules out anything quadratic and points at a linear solution**, which is useful before I know what the solution is.

Now the four signals.

It asks for a maximum — signal one. There is a small set of choices at each house: take it or skip it — signal two. The naive recursion branches two ways per house, so it is 2^n — signal three. And the constraint is consistent with a linear DP — signal four. **All four fire, so I am fairly confident before doing any real thinking.**

Now the test, both halves.

Optimal substructure: is the best answer for houses one to `n` built from best answers to shorter prefixes? Yes — if I decide about the last house, the rest is the same problem on a shorter row.

Overlap: does the same subproblem recur? Sketching two levels: the answer for the first `n` houses calls the first `n-1` and the first `n-2`; the first `n-1` calls `n-2` and `n-3`. `n-2` appears under both. Overlap, so a table pays for itself. **That is Fibonacci's shape exactly, and it is worth noticing out loud.**

Now the shape. My question is: **what do I need to know to decide about the next house?** Just where I am — because the constraint only reaches back one house. **So it is one-dimensional: `dp[i]`.**

The state as a sentence: `dp[i]` is the maximum total obtainable from the first `i` houses.

Completeness check: given only `i` and that value, can I decide about house `i+1`? Almost — I need to know whether house `i` was taken. And here I have a choice, which is the interesting moment.

I could add a mode dimension: `dp[i][took_it]`. That works and is two cells per house.

Or I can redefine so the information is carried implicitly: `dp[i]` is the best for the first `i` houses where I am free to choose about house `i`, and the recurrence is `max(dp[i-1], dp[i-2] + value[i])` — **skip this house, or take it and jump back two.** The ‘not adjacent’ constraint is expressed by the jump.

I would take the redefinition, and say why: it is the same number of cells and it makes the recurrence read as the decision. That is the day-147 lesson — try redefining before adding a dimension.

Count the states: n states, constant work each. A hundred thousand operations. Comfortable.

Base cases: $dp[0]$ is the first house's value, $dp[1]$ is the larger of the first two.

Then two more things I would say before coding.

Could greedy work? Take the largest value, then the largest remaining non-adjacent one, and so on. I can break it with three houses: five, ten, six. Greedy takes the ten, which forbids both neighbours, so it gets ten. Taking the five and the six — which are not adjacent — gives eleven. So greedy is wrong, and it is DP.

I would spend those thirty seconds constructing a real counter-example rather than asserting one, because the small cases are where greedy usually survives and a hand-waved example proves nothing. And if I had failed to break it, that would change the answer — greedy might be right, and I would owe you an argument instead of a table.

And the space. The recurrence reads only $dp[i-1]$ and $dp[i-2]$, so two variables are enough — $O(1)$ space. I would write the array version first for clarity and offer the collapse.

The whole diagnosis took about ninety seconds, and it produced: one-dimensional DP, state as a sentence, n states, a checked greedy alternative, and a known space optimisation — before a line of code. That is the part I would want to be judged on.”

9 Recall card

Four signals — any one means “consider DP”: it asks for max/min/count/possible; there is a small set of choices at each step; the naive recursion is exponential; and the constraint on n is small in a telling way. Read the constraints first — $n \leq 20 \rightarrow$ bitmask, $\leq 100 \rightarrow O(n^3)$, $\leq 1,000 \rightarrow O(n^2)$, $\leq 10^5 \rightarrow$ one-dimensional or not DP at all. Every row lands near 10^6 – 10^7 , because the constraint is chosen so the intended solution fits and the next-worse one does not.

Then the test, and BOTH halves matter: optimal substructure AND overlapping subproblems. No overlap = divide and conquer (merge sort: 2,047 calls, 2,047 distinct arguments — a table would never be read). Overlap = DP (fib(20): 13,529 calls, 21 distinct arguments). **Broken substructure means your STATE is incomplete, not that DP is wrong.**

The shape follows one question: what do I need to know to decide the next step? Position → `dp[i]`; position + resource → `dp[i][cap]`; position + mode → `dp[i][mode]`; two sequences → `dp[i][j]`; a grid → `dp[r][c]`; a **range and how you split it** → interval DP; a subtree → recursion; a **set**, `n ≤ 20` → `dp[mask]`.

Then, in order and in ninety seconds: (1) state as a full SENTENCE — cannot finish it, you have no state; **(2) completeness check** — given only the state, can you decide the next step; **(3) count states × work against $\sim 10^7$** (Python does $\sim 10^7$ ops/second, C $\sim 10^9$). **Then recurrence, base cases, memoise** (the fill order cannot be wrong), tabulate and collapse only if asked.

Not DP when: no overlap (divide and conquer); greedy is provably safe (**spend 30 seconds trying to build a counter-example** — coins `[1, 5, 7]`, amount 10: greedy 4, optimal 2); **a closed form exists** (grid paths are a binomial coefficient); or the problem says “**list all**” rather than “how many” — that is backtracking, and the output is gigabytes whatever you do.

1 What this is, and why they ask it

Google Maps answers one question — **“how do I get from here to there, and how long will it take?”** — over a road network with hundreds of millions of intersections, for billions of queries a day, with traffic changing by the minute.

They ask it because **it is the design where a classic algorithm you know is completely inadequate, and knowing why is the whole interview.**

Dijkstra’s algorithm solves shortest path correctly. On a continental road network it visits tens of millions of nodes and takes seconds. **At the query rate this system needs, that is not off by a constant factor — it is off by four orders of magnitude.** So the interesting question is not “which algorithm” but **“what can you compute in advance so the query does almost nothing”.**

The answer is preprocessing, and it is the most dramatic example in this course: **hours of offline computation per continent turning a multi-second query into under a millisecond.** That is the trade to be able to state.

And traffic is what makes it hard rather than merely large. A route depends on current conditions, which change every few minutes, **and the whole point of preprocessing is that it assumes the costs are fixed.** Reconciling those two is the real design problem, and it has a real answer.

The third thing is **map rendering and tiles**, which is a separate system with different economics — pre-rendered images at many zoom levels, served like any other static content — and knowing that it is separate is part of scoping the answer.

By the end of this lesson you can explain why plain Dijkstra fails, describe contraction hierarchies well enough to defend them, handle live traffic, design the tile pipeline, and size the whole thing.

2 The story

The transport office issued the permits and the man who actually decided the bus routes was Bhaskaran, who had done it for twenty-two years and had never seen a map of the district that was correct.

He was asked the same question forty times a day. How does this bus get from here to there, and how long.

And for the first few years he did what anybody would do: **he traced it.** Finger on the map, through the villages, adding up the times, and it took four or five minutes and he got it wrong reasonably often, because four or five minutes is a long time to hold twenty numbers in your head.

What changed was the highway.

Because once the highway existed, **almost every long journey used it.** Not the short ones — a village to the next village went the small roads. **But anything more than about thirty kilometres got onto the highway as soon as it could, stayed on it, and came off as late as it could.**

So Bhaskaran stopped tracing the whole route.

He built one table, over about a month, and it was the highway junctions. Eleven of them, and for every pair, the time between them. **Eleven by eleven, one small card pinned above his desk, and it never changed.**

And then any long question became three small ones. **How do I get from the village to the nearest junction — which is a short problem I can do in my head. Junction to junction — which is on the card. And the last junction to the destination — short again.**

Four or five minutes became about twenty seconds.

Two things went wrong with it, and both took a while.

The first was that the short bits were not always short. A village forty kilometres off the highway had a long approach, and for those he eventually made a second, smaller card.

The second was the one he never fully solved. The card said what the highway took **on an ordinary day.** It did not know about the market at Perambur on a Thursday, or the level crossing when the goods train was through, or the road being dug up.

So he kept the card, and he kept a second thing in his head that changed constantly, and the answer he gave was the card adjusted by what he had heard that morning.

His deputy, who wanted the card to be correct, kept asking him to redo it.

“It took a month,” Bhaskaran said. “By the time I finished redoing it, it would be Thursday again.”

3 The idea in plain English

Bhaskaran's junction table is preprocessing, his three-part journey is the query, and his deputy's question is the traffic problem — **the table takes a month to build and the world changes on Thursday.**

Start with what is being asked, because scoping matters here.

IN	routing: from A to B, fastest route ETA: how long it will take, given current conditions live traffic map tiles and rendering
OUT	search and geocoding ("pizza near me"), Street View, satellite imagery, business listings, indoor maps, turn-by-turn voice, offline maps

Search is its own enormous system and offering to design it here is how the time disappears.

Now the graph, and the first number that matters.

The road network is a graph. Nodes are intersections, edges are road segments, and each edge has a cost — travel time, which is length divided by speed.

For the whole world that is roughly 100 million nodes and 200 million edges. For one continent, tens of millions.

And here is the failure. Dijkstra's algorithm from [day 136](#) explores outward from the source in all directions until it reaches the destination. For a route across a country, that means visiting a large fraction of the continent's nodes — **tens of millions of them, taking seconds.**

At billions of queries a day, seconds per query is not a performance problem. It is a different system.

Two cheap improvements, neither sufficient:

Bidirectional search runs Dijkstra from both ends and stops when they meet. It **roughly halves the explored area** — two circles of radius $d/2$ instead of one of radius d — which at constant factors is maybe a two-to-fourfold saving. **Real and not enough.**

A* adds a heuristic: prefer nodes that are geometrically closer to the destination, using straight-line distance as a lower bound on remaining travel time. **On road networks this helps by perhaps five to ten times** — better, and still milliseconds-to-

seconds rather than microseconds.

Neither gets four orders of magnitude, which is what is needed. So: preprocess.

Contraction hierarchies are the standard answer, and the idea is Bhaskaran's junction table.

The observation is that road networks have a hierarchy. Long journeys use motorways; residential streets only appear at the very beginning and the very end. So most of the graph is irrelevant to most long queries.

The preprocessing makes that formal. Order the nodes by "importance" — roughly, how many shortest paths pass through them — and then **contract them one at a time from least important upwards**.

Contracting a node means removing it and adding shortcut edges between its neighbours wherever the path through it was the only shortest route. A shortcut says "this stretch of small roads takes eleven minutes" without storing the small roads.

After contracting everything, you have the original graph plus a few million shortcuts, arranged in levels.

And then the query is startlingly simple: **bidirectional search**, where each direction only ever moves **UPWARDS in the hierarchy**. Forward from the source going up, backward from the destination going up, and the meeting point is somewhere near the top.

Because both searches only go up, they touch a few hundred nodes instead of tens of millions. That is the four orders of magnitude, and it is why the preprocessing is worth hours per continent.

Now traffic, which is what makes this genuinely hard.

Preprocessing assumes the edge costs are fixed. Traffic changes them every few minutes. And **recomputing the hierarchy takes hours**, so you cannot simply rebuild it — Bhaskaran's answer to his deputy.

Three techniques, used together.

Customisable route planning splits the preprocessing into two phases: a **slow phase that depends only on the road topology**, and a **fast phase that depends on the weights**. Since traffic changes the weights and not the topology, **only the fast phase reruns — minutes rather than hours**. That is the main answer and it is what modern systems do.

Live traffic as an overlay. Most edges are unaffected at any moment. **Keep the current speed for the affected ones in a separate fast store**, and have the query check it for the edges it actually touches. **A few hundred lookups per query, not millions.**

And a fallback. If traffic makes a route much worse than the preprocessed structure assumed, **run a bounded ordinary search around the affected region.** Slower, rarer, and correct.

Then: where does the traffic data come from?

From the phones. Anonymised location and speed reports from devices using the app. **Millions of them, a free and enormous sensor network** — and it is the reason a navigation product improves with the number of users, which is a genuine competitive property.

Speeds are aggregated per road segment over a short window, with outliers removed — a phone in a bus is not reporting car speed, and a phone in a passenger's pocket in a stationary car is not reporting a jam.

And there is a privacy requirement that is not optional: reports must be anonymised and aggregated, because **a sequence of location reports is a person's movements**, and a segment with only one contributor identifies them.

Then the ETA, which is a prediction rather than a sum.

Adding up current segment speeds gives the wrong answer for a long journey, because by the time you reach a segment forty minutes away, conditions there have changed. **So the ETA uses predicted speeds at the predicted arrival time** — a model over historical patterns by hour and day, adjusted by current conditions.

And it should not be a point estimate. “Twenty-five minutes” is less useful than “twenty-two to thirty-two” — **and predicting the distribution rather than the mean is a real product decision** that matters most exactly when traffic is worst.

Finally, map tiles, which are a different system.

The map is not drawn on demand. The world is divided into square tiles at about twenty zoom levels, **each tile pre-rendered as an image or as vector data**, and served from a CDN like any other static file.

The arithmetic is the interesting part: each zoom level has four times as many tiles as the one above, so **level 20 has about a trillion tiles.** Rendering all of them is impossible, **so the deep levels are rendered lazily on first request and cached** — which works because almost nobody zooms all the way in on the middle of an ocean.

And **vector tiles are the modern choice**: send the geometry and let the client draw it. **Smaller, and one tile serves every rotation, label language and style**, where raster tiles need a separate render for each.

4 The picture

Why plain Dijkstra fails:

DIJKSTRA from A to B	A* WITH A HEURISTIC
 (B is over here)	
explores a CIRCLE around A radius = distance to B → tens of millions of nodes → SECONDS per query	explores an ELLIPSE towards B ~5-10x fewer nodes → still milliseconds-to-seconds
At billions of queries/day, seconds is not slow. It is a DIFFERENT SYSTEM. You need four orders of magnitude, and neither of these gives it.	

Contraction hierarchies, which is the junction table:

THE OBSERVATION: road networks are hierarchical

residential → arterial → highway → motorway

a 200 km journey uses motorway for 190 km of it.
the residential streets matter only at the two ENDS.

PREPROCESSING (hours, offline, per continent)

order nodes by importance (least important first)
contract each one: REMOVE it, and add SHORTCUT edges
between its neighbours where the path through it was the
only shortest route

A --3-- X --4-- B	contract X
⇒ A --7-- B	one shortcut replaces two edges and one node

after contracting everything: the original graph
+ a few million shortcuts, arranged in LEVELS

QUERY (microseconds)

bidirectional search where BOTH directions only move UPWARDS

```
source ---up---> \
                  X meet near the top
destination ---up---> /
```

→ a few HUNDRED nodes touched, not tens of millions

The three-part journey, which is Bhaskaran's method:

village ---->	nearest junction ---->	junction ---->	destination
(small roads)	(THE TABLE)	(small roads)	
~20 nodes	one lookup	~20 nodes	

the middle is precomputed and never changes
the two ends are small and local

CONTRACTION HIERARCHIES ARE THIS, DONE AUTOMATICALLY
and with the "junctions" discovered rather than chosen.

The traffic problem, and why you cannot just rebuild:

preprocessing assumes FIXED edge costs
traffic changes costs every ~2 minutes
rebuilding the hierarchy takes HOURS

→ you cannot rebuild. Bhaskaran's answer to his deputy.

THE FIX – split the preprocessing in two:

PHASE 1 (slow, hours) depends only on the road TOPOLOGY
 which roads connect to which
 → changes when roads are BUILT,
 i.e. essentially never

PHASE 2 (fast, minutes) depends on the WEIGHTS
 how long each road takes
 → reruns on every traffic update

traffic changes weights, NOT topology.
→ only phase 2 reruns. Minutes, not hours.

The traffic pipeline:

```
millions of phones
  | anonymised (location, speed, heading) every few seconds
  v
[ INGEST ] → [ map-match to a road segment ]
              | which road is this point actually on?
              v
          [ aggregate per segment, 2-min window ]
              | median speed, outliers removed
              | (a bus is not a car; a parked phone
              | is not a jam)
              v
          [ live speed store ] ← the query reads this
              | for the few hundred
              v edges it touches
          [ historical archive ] → ETA prediction models
```

PRIVACY IS NOT OPTIONAL: a sequence of location reports IS a person's movements. Aggregate, and suppress segments with too few contributors.

Map tiles, and the arithmetic that shapes them:

```
zoom 0: 1 tile          the whole world
zoom 1: 4 tiles
zoom 2: 16 tiles
...
zoom n: 4^n tiles
```

```
zoom 10: ~1,000,000 tiles
zoom 15: ~1,000,000,000
zoom 20: ~1,000,000,000,000 tiles
```

→ RENDERING ALL OF THEM IS IMPOSSIBLE

FIX: pre-render zoom 0-12 (~22 million tiles, feasible),
render 13-20 LAZILY on first request, then cache

which works because almost nobody zooms to street level
in the middle of the Pacific.

RASTER vs VECTOR tiles:

- raster: a PNG. One render per style, language, rotation.
- vector: the geometry; the client draws it.
 - smaller, and ONE tile serves every style,
every label language, every rotation

5 How it actually works

The graph

```
nodes: ~100,000,000 worldwide (intersections)
edges: ~200,000,000 (road segments)

per edge: from_node, to_node, length, road_class, speed_limit,
          turn_restrictions, one-way flag
          = ~50 bytes packed

200,000,000 x 50 B = 10 GB for the world's road graph
→ one continent fits comfortably in one machine's memory
```

Ten gigabytes for every road on Earth is worth saying, because it reframes the problem: **the graph is not big data. The query rate is the problem.**

Contraction, in outline

```
def contract_node(graph, node) → list[tuple]:
    """Remove `node`, adding shortcuts only where they are necessary."""
    shortcuts = []
    for u, cost_in in graph.incoming(node):
        for v, cost_out in graph.outgoing(node):
            if u == v:
                continue
            through = cost_in + cost_out
            # is the path u → node → v the ONLY shortest route?
            if local_shortest_path(graph, u, v, limit=through) > through:
                shortcuts.append((u, v, through))
    graph.remove(node)
    graph.add_edges(shortcuts)
    return shortcuts
```

The `local_shortest_path` check is what keeps the number of shortcuts manageable. Without it every contraction adds an edge for every pair of neighbours, and the graph explodes. With it, a shortcut is added only when there is no other equally short way round — and the search is bounded, so it is cheap.

The contraction order matters enormously, and it is chosen by a heuristic — roughly, contract the node whose removal adds the fewest shortcuts. A bad order produces ten times as many shortcuts and a much slower query.

The query

```
def query(graph, source, target):
    """Bidirectional search that only ever moves UPWARDS in the hierarchy."""
    forward = {source: 0}
    backward = {target: 0}
    fq, bq = [(0, source)], [(0, target)]
    best = float("inf")

    while fq or bq:
        for queue, seen, other in ((fq, forward, backward), (bq, backward, forward)):
            if not queue:
                continue
            cost, node = heapq.heappop(queue)
            if node in other:
                best = min(best, cost + other[node]) # they met
            for nxt, weight in graph.upward_edges(node): # UPWARD only
                if cost + weight < seen.get(nxt, float("inf")):
                    seen[nxt] = cost + weight
                    heapq.heappush(queue, (cost + weight, nxt))

    return best
```

`graph.upward_edges` is the entire trick. Each search only follows edges to more important nodes, so it climbs rather than spreads — and the two searches necessarily meet near the top of the hierarchy.

The correctness argument is the non-obvious part: because the shortcuts encode every path that was removed, **an up-only search still finds the true shortest path.** That is what the preprocessing guarantees, and it is worth saying you know it is a theorem rather than an optimisation.

Live traffic on top

```
def edge_cost(edge, departure_time: float) → float:
    live = traffic_store.get(edge.id)                # a fast lookup
    if live and live.age < 300:
        return edge.length / live.speed
    predicted = eta_model.speed(edge.id, departure_time)  # historical
    return edge.length / predicted
```

The query touches a few hundred edges, so a few hundred traffic lookups per query is affordable where consulting traffic for the whole graph would not be.

And `live.age < 300` matters: a five-minute-old reading is useful; a two-hour-old one is worse than the historical model, **so stale data must fall back rather than be trusted.**

Ingesting traffic

```
def ingest_report(device_hash: str, lat: float, lon: float,
                  speed: float, heading: float, at: float) → None:
    segment = map_match(lat, lon, heading)          # which road?
    if segment is None:
        return                                       # off-network
    redis.zadd(f"speeds:{segment}", {f"{device_hash}:{at}": speed})
    redis.zremrangebyscore(f"speeds:{segment}", 0, at - 120)  # 2-minute window

def aggregate(segment: str) → float | None:
    samples = redis.zrange(f"speeds:{segment}", 0, -1, withscores=True)
    if len(samples) < MIN_CONTRIBUTORS:             # privacy AND accuracy
        return None
    speeds = sorted(s for _, s in samples)
    return speeds[len(speeds) // 2]                  # median, not mean
```

`map_match` is a real subsystem, not a lookup: GPS is accurate to a few metres and roads are metres apart, so deciding which road a point is on uses the heading, the recent trajectory and the network topology. **A phone on a flyover and one on the road beneath it are metres apart and have completely different speeds.**

The median rather than the mean is deliberate: one stationary phone in a parked car would drag a mean towards zero, **and a jam is a median, not an average.**

And `MIN_CONTRIBUTORS` does two jobs at once — it suppresses noisy segments and it is the privacy guarantee, because a segment with one contributor is one identifiable person’s journey.

ETA prediction

```
def predict_eta(route: list, departure: float) → tuple[float, float]:
    total, time_now = 0.0, departure
    for edge in route:
        speed = eta_model.speed(edge.id, time_now)           # AT ARRIVAL TIME
        seconds = edge.length / speed
        total += seconds
        time_now += seconds                                   # advance the clock
    spread = uncertainty_model.spread(route, departure)
    return total, spread
```

`time_now += seconds` inside the loop is the important line. A segment forty minutes into the journey is evaluated at its *predicted* conditions forty minutes from now, **not at current conditions** — which is the difference between an ETA that is right and one that is systematically wrong during rush hour.

And returning a spread rather than a single number is the honest output: “twenty-two to thirty-two minutes” is more useful than “twenty-five”, and most useful exactly when traffic is worst.

Tiles

```
def tile_url(z: int, x: int, y: int) → str:
    return f"https://tiles.example.com/{z}/{x}/{y}.mvt"

def serve_tile(z: int, x: int, y: int) → bytes:
    key = f"{z}/{x}/{y}"
    cached = tile_cache.get(key)
    if cached:
        return cached
    if z ≤ 12:
        return prerendered.get(key)           # always exists
    data = render_vector_tile(z, x, y)        # lazy, on demand
    tile_cache.put(key, data, ttl=30 * 86400)
    return data
```

Zoom 12 and above is the split, and the reason is arithmetic: zoom 0 to 12 is about 22 million tiles, which can be pre-rendered; zoom 20 is a trillion, which cannot.

Deep tiles are rendered on first request and cached, which works because the distribution of requests is enormously skewed towards places people are.

The real systems

OpenStreetMap	the open road network data everyone else compares to
OSRM	open-source routing with contraction hierarchies
Valhalla	open-source, tiled routing – designed for dynamic costs
GraphHopper	open-source CH and CRP implementations
Mapbox / MapLibre	vector tiles and client-side rendering
S2 / H3	the cell systems for spatial indexing (day 161)
Overpass	querying OSM data

Naming OSRM and Valhalla is a good signal, because they are real implementations of the two approaches — **OSRM precomputes hard and is fast and static; Valhalla is tiled and designed for costs that change**, which is exactly the trade this design is about.

6 The numbers

The graph.

```
worldwide: ~100,000,000 nodes, ~200,000,000 edges
packed:    ~50 bytes per edge

200e6 x 50 B = 10 GB for every road on Earth

one continent (Europe): ~20,000,000 nodes → ~2 GB
→ FITS IN MEMORY on one machine
```

Ten gigabytes for the world's roads — the data is not the problem, the query rate is.

Query cost, which is the whole argument.

DIJKSTRA, London to Edinburgh (~600 km):
explores a circle of radius 600 km
~5,000,000 nodes settled
at ~10,000,000 node-operations/second in C
→ ~0.5-2 seconds

A*, same route:
~500,000-1,000,000 nodes
→ ~0.1-0.2 seconds (5-10x better)

CONTRACTION HIERARCHIES, same route:
~500 nodes settled
→ ~0.1 MILLISECONDS

10,000x faster than Dijkstra.

Four orders of magnitude is what makes the query rate possible, and it is the number to lead with.

The preprocessing that buys it.

contracting a continent (~20,000,000 nodes):
~1-4 hours on a large machine
produces ~1.5x the original edge count in shortcuts
→ ~2 GB becomes ~3 GB

worldwide: ~10-20 hours, and it is run when the map data changes
(weekly or so), not when traffic changes

Query load.

~1,000,000,000 route requests/day
= ~11,600/second average
peak ~40,000/second

at 0.1 ms per query: $40,000 \times 0.0001 \text{ s} = 4 \text{ CPU-seconds per second}$
→ ~4 cores of routing at peak, plus overhead

WITH DIJKSTRA INSTEAD:
 $40,000 \times 1 \text{ s} = 40,000 \text{ CPU-seconds per second}$
→ 40,000 cores. About 1,300 machines, for routing alone.

→ the preprocessing replaces ~1,300 machines with ~10.

That comparison is the best thing to say in this interview: hours of offline work per continent against a thirteen-hundred-machine fleet, forever.

Traffic ingestion.

```
~500,000,000 devices with the app, ~5% moving and reporting at once
= 25,000,000 active reporters
each reports every ~5 seconds
= 5,000,000 reports/second
```

```
each report: ~40 bytes
→ 200 MB/s of ingest = ~17 TB/day
```

Five million reports a second is the largest write rate in this course, **and it is the same disposable shape as Uber's location stream** — only the latest matters, losing one is fine.

```
aggregated output:
  200,000,000 segments, but only ~5% have live data at any moment
  = 10,000,000 segments x ~50 bytes = 500 MB of live speed state

→ the live traffic layer fits in memory. Again: the raw stream
  is huge and the useful state is small.
```

The traffic reprocessing cycle.

```
full contraction rebuild:    1-4 hours    → impossible per update
customisable (weights only): ~2-10 minutes for a continent
                             → runs every few minutes. Viable.

→ which is exactly why the preprocessing is split into a
  topology phase and a weights phase.
```

Tiles.

```
zoom level n has 4^n tiles

z=12: 16,777,216 tiles
cumulative z=0..12: ~22,000,000 tiles
at ~20 KB each (vector): ~440 GB    → pre-render all of these

z=20: ~1.1 x 10^12 tiles
at 20 KB: ~22 EXABYTES              → impossible

→ pre-render to z=12, render z=13-20 lazily and cache
  the request distribution is extremely skewed, so the working
  set is a tiny fraction of the theoretical total
```

Tile serving.

```
~10,000,000,000 tile requests/day
= ~116,000/second average, peak ~400,000/second

these are STATIC FILES → a CDN, exactly like images
at a 98% cache hit rate: ~8,000/second reach the origin

vector tile ~20 KB:
  10e9 x 20 KB = 200 TB/day of egress
  → comparable to a mid-sized video platform, and it is why
    vector tiles (small) beat raster tiles (large) on cost
```

Raster against vector, quantified:

```
raster PNG tile:  ~100 KB, and you need a separate render per
                  style, per label language, per rotation
                  → 10+ variants of every tile

vector tile:      ~20 KB, and ONE tile serves every style,
                  language and rotation – the client draws it

→ 5x smaller AND 10x fewer variants = ~50x less storage and egress
```

The full bill, roughly:

tile delivery (200 TB/day)	~\$60M/year
traffic ingest + aggregation	~\$30M/year
routing (small!)	~\$2M/year
map data + preprocessing	~\$10M/year

	~\$100M/year

→ ROUTING, the algorithmically interesting part, is 2% of the bill.
The bytes are all tiles and telemetry.

7 The trade-offs

Preprocessing against flexibility. Contraction hierarchies give a ten-thousandfold query speedup and **bake the edge weights in**. Change what “cost” means — avoid motorways, prefer cycle routes, a lorry with a height limit — **and you need a different hierarchy for each**. Every routing profile is a separate preprocessing run, which is why products offer four or five profiles and not arbitrary ones.

Contraction hierarchies against customisable route planning. CH is faster to query and slower to update. **CRP splits the work so weight changes cost minutes instead of hours**, at the cost of a somewhat slower query and a more complex imple-

mentation. **For a system with live traffic, CRP wins** — which is why it is what the production systems moved to.

Live traffic against predictability. Using live speeds gives better ETAs and **makes routes unstable** — the same query a minute apart can return a different road, which is disorienting mid-journey. **So route changes en route need hysteresis:** only reroute when the saving is meaningfully large, not whenever it is positive.

Point ETAs against distributions. A single number is simple and **systematically overconfident**, and it is worst when traffic is worst — exactly when the user needs the warning. A range is more honest **and harder to display and to reason about**, and users have been trained to expect one number.

Pre-rendered against lazily-rendered tiles. Pre-rendering everything is impossible past zoom 12 — a trillion tiles. **Lazy rendering makes the first request for an unusual place slow**, which is acceptable because unusual places are unusual. **The risk is a cold cache after a style change**, when every tile is suddenly a miss.

Raster against vector tiles. Vector is five times smaller, serves every style and language from one tile, and **pushes rendering onto the client**, which needs a capable device and more battery. **Raster is universally supported and multiplies storage by the number of variants.** Vector is the modern default and raster is the fallback for old clients.

And the honest one: routing is 2% of the bill. The algorithmically fascinating part of this system is cheap; **the money is in tile delivery and telemetry ingest.** That is worth saying, because it is a good reminder that the interesting engineering and the expensive engineering are often not the same.

When would I not build this? Almost always — this is a build-versus-buy question with an obvious answer. The Google Maps and Mapbox APIs exist, the data is the hard part, and **collecting a global road network is a decade of work.** Self-hosting OSRM or Valhalla on OpenStreetMap data is right when you need a custom cost function, offline operation, or you are doing millions of routes a day and the API bill dominates. **And for a single city, or fixed routes, this whole design is unnecessary** — a precomputed table between every pair of points of interest is a few megabytes.

8 In the interview

How it gets asked

- “*Design Google Maps.*” — usually meaning routing and ETAs.

- “*Why can’t you just use Dijkstra?*” — the central question.
- “*How do you handle live traffic?*” — the one that defeats naive preprocessing.
- “*Where does the traffic data come from?*”
- “*How do you serve the map itself?*” — the tile question.
- “*How would you compute the ETA?*”

“I would scope first: **routing, ETAs, live traffic and map tiles are in; search, Street View, satellite imagery and business listings are out** — search in particular is its own enormous system.

The road network is a graph: about a hundred million nodes and two hundred million edges worldwide, which packs into around ten gigabytes. So the data fits in memory on one machine. The data is not the problem; the query rate is.

And the central fact is that Dijkstra does not work here — not by a constant factor, by four orders of magnitude. A route from London to Edinburgh explores a circle of six hundred kilometres’ radius, settling around five million nodes, which is one to two seconds per query.

At a billion route requests a day, that is forty thousand CPU-seconds per second at peak — about thirteen hundred machines for routing alone.

Bidirectional search roughly halves it and A* with a straight-line heuristic gives maybe five to ten times. Both real, both nowhere near enough.

So the answer is preprocessing, and specifically contraction hierarchies. The observation is that road networks are hierarchical — a two-hundred-kilometre journey uses motorway for a hundred and ninety of it, and the residential streets matter only at the two ends.

The preprocessing makes that formal: order the nodes by importance, contract them from least important upwards, and each contraction removes a node and adds shortcut edges wherever the path through it was the only shortest route.

Then the query is bidirectional search where both directions only ever move upwards in the hierarchy — so they climb rather than spread, and meet near the top. A few hundred nodes instead of five million: about a tenth of a millisecond.

Ten thousand times faster, in exchange for a few hours of offline work per continent — and it turns thirteen hundred routing machines into about ten.

The thing that makes this genuinely hard is traffic, because preprocessing assumes fixed edge costs and rebuilding the hierarchy takes hours while traffic changes every couple of minutes. I would want to spend most of the time there, if that suits you.”

The follow-ups

“Why can’t you just use Dijkstra?”

“Because it is off by four orders of magnitude, and it is worth being precise about where that comes from.

Dijkstra explores outward from the source in every direction until it reaches the destination. It has no idea where the destination is — **it settles every node closer than the target before it settles the target.**

For London to Edinburgh, that is a circle of six hundred kilometres’ radius, which is essentially the whole of Britain — around five million nodes. At ten million node-operations a second in C, that is one to two seconds.

At a billion queries a day with a peak of forty thousand a second, one second per query is forty thousand cores. So this is not a slow implementation, it is the wrong approach.

Two standard improvements, and I would name both and their limits.

Bidirectional search runs from both ends and stops when they meet — **two circles of radius $d/2$ instead of one of radius d** , which is a factor of two to four in area. **Real, and not enough.**

A* with a heuristic — straight-line distance divided by the maximum speed limit is an admissible lower bound — **turns the circle into an ellipse pointed at the destination.** On road networks that is maybe five to ten times fewer nodes. **Better, and still milliseconds to seconds.**

Neither gives ten thousand times, and ten thousand is what is required.

So the question changes from ‘which algorithm’ to ‘what can I compute in advance’ — and that is the real insight. **Contraction hierarchies spend a few hours offline per continent and reduce a query from five million nodes to about five hundred.**

I would frame the trade explicitly: a few hours of preprocessing, rerun when the map data changes — which is weekly, not minutely — **against a fleet of thirteen hundred machines running forever.** That is not a close decision.”

“How do you handle live traffic?”

“This is the part that defeats naive preprocessing, and it has a real answer that took the field a while to find.

The problem: contraction hierarchies bake the edge weights into the shortcuts. A shortcut says ‘this stretch takes eleven minutes’. **Traffic changes that, and rebuilding the hierarchy takes hours while traffic changes every couple of minutes.** You cannot rebuild.

Three things, used together.

The main one is to split the preprocessing into two phases — customisable route planning. **Phase one depends only on the road topology: which roads connect to which.** That changes when roads are built, which is essentially never. **Phase two depends on the weights: how long each road takes.**

Traffic changes weights and not topology, so only phase two reruns — **minutes for a continent rather than hours.** That is what modern production systems do, and it is the answer to the question.

Second, a live traffic overlay for the fine detail. At any moment only a few percent of segments have unusual conditions. **Keep the current speed for those in a fast store, and have the query check it for the edges it actually touches** — a few hundred lookups, which is affordable where consulting the whole graph would not be. **And stale readings must fall back to the historical model,** because a two-hour-old speed is worse than a good prediction.

Third, a bounded fallback. If live conditions make a route much worse than the preprocessed structure assumed — an accident closing a motorway — **run an ordinary search over a bounded region around the problem.** Slower, rare, and correct.

And a product point I would raise. Live traffic makes routes **unstable:** the same query a minute apart can give a different road. **Rerouting a driver mid-journey every time a marginally better option appears is disorienting,** so route changes need hysteresis — only reroute when the saving is meaningfully large, not whenever it is positive.”

“Where does the traffic data come from, and how do you turn it into speeds?”

“From the phones, and that is the interesting answer: **the users are the sensor network.**

Anonymised location, speed and heading reports from devices with the app running. With five hundred million devices and perhaps five percent moving at once, that is twenty-five million reporters at one report every five seconds — **about five million reports a second**, roughly seventeen terabytes a day.

And that is the same disposable shape as any location stream: only the latest matters, and losing one is fine.

The first real step is map matching, and it is a genuine subsystem rather than a lookup. GPS is accurate to a few metres and roads are metres apart, **so deciding which road a point is on uses the heading, the recent trajectory and the network topology.** A phone on a flyover and one on the road beneath it are metres apart and have completely different speeds — **getting that wrong reports a jam on a clear road.**

Then aggregation per segment over a short window, and two choices matter.

Use the median, not the mean. One phone in a parked car reports zero and would drag a mean towards it — **a jam is a median, not an average.**

And require a minimum number of contributors before publishing a speed. That does two jobs: **it suppresses noise on quiet roads, and it is the privacy guarantee** — a segment with one contributor is one identifiable person’s journey, and a sequence of their reports is their movements.

Privacy is not optional here and I would raise it unprompted: **the raw stream is the most sensitive data in the entire system**, far more so than anything in the routing layer.

The output is small, which is the nice part. Two hundred million segments, but only about five percent have live data at any moment — **ten million segments at fifty bytes is five hundred megabytes, which fits in memory.** The raw stream is enormous and the useful state is tiny, which is the same pattern as every other telemetry system in this course.”

The model answer

“Design the routing and ETA system for a maps product: a billion route requests a day, global road network, live traffic.”

“Let me scope, then give the central fact, because it determines the whole design.

In: routing, ETA, live traffic. Out: search and geocoding, imagery, business listings — search is its own system.

The graph is about a hundred million nodes and two hundred million edges, which packs into ten gigabytes. A continent fits in one machine’s memory. **The data is not the problem.**

The central fact: Dijkstra is off by four orders of magnitude. London to Edinburgh settles about five million nodes — one to two seconds. **At a peak of forty thousand queries a second, that is forty thousand cores, about thirteen hundred machines for routing alone.** A* helps by five to ten times and bidirectional search by two to four. **Neither is close.**

So: preprocessing. Contraction hierarchies exploit the fact that road networks are hierarchical — a long journey is almost entirely motorway, with local roads only at the ends. **Order nodes by importance, contract from the bottom up, adding shortcuts only where the path through a node was the sole shortest route.** Then the query is **bidirectional search that only moves upwards**, touching a few hundred nodes: **about a tenth of a millisecond, ten thousand times faster.**

A few hours of offline work per continent replaces thirteen hundred machines with about ten.

Now the hard part: traffic invalidates the preprocessing, and rebuilding takes hours while traffic changes every two minutes.

The answer is to split the preprocessing. A slow phase over the road **topology**, which changes when roads are built — essentially never. **A fast phase over the weights**, which reruns in minutes when traffic updates. **Traffic changes weights, not topology**, so only the fast phase repeats.

Plus a live overlay for the few percent of segments with unusual conditions, checked for the few hundred edges a query actually touches, **with stale readings falling back to the historical model.** And a bounded ordinary search as a fall-back when an incident makes the preprocessed structure badly wrong.

Traffic data comes from the phones — five million reports a second, seventeen terabytes a day. **Map matching first**, which is a real subsystem because GPS accuracy is comparable to the distance between roads. **Then median aggregation per segment with a minimum contributor count**, which suppresses noise

and is the privacy guarantee — a segment with one contributor is one person's identifiable journey. **The raw stream is the most sensitive data in the system, and I would say so unprompted.**

ETA is a prediction, not a sum. Walking the route, each segment is evaluated at its predicted conditions **at the time you will actually arrive there**, advancing the clock as you go — a segment forty minutes ahead is not evaluated at current conditions. **And I would return a range rather than a point estimate**, because a single number is systematically overconfident exactly when traffic is worst.

Two things I would flag.

Routing is about two percent of the bill. The money is in tile delivery — two hundred terabytes a day — and telemetry ingest. **The algorithmically interesting part is the cheap part**, which is worth saying out loud.

And every routing profile needs its own preprocessing. Avoid motorways, cycling, a lorry with a height restriction — **each is a different cost function and therefore a different hierarchy**, which is why products offer four or five profiles rather than arbitrary ones. **If the requirement were truly arbitrary per-user costs, contraction hierarchies would be the wrong choice**, and I would move to a customisable approach with a slower query.”

9 Recall card

Dijkstra is off by FOUR ORDERS OF MAGNITUDE, not a constant factor. London→Edinburgh settles ~5,000,000 nodes (1–2 s); at 40,000 queries/second that is ~1,300 machines for routing alone. **Bidirectional gives 2–4×, A* gives 5–10× — neither is close.** So the question is not “which algorithm” but “**what can I compute in advance**”.

Contraction hierarchies: road networks are hierarchical — a 200 km journey is 190 km of motorway. Order nodes by importance, **contract from the bottom up, adding a shortcut only where the path through the node was the sole shortest route** (the bounded local search is what stops the shortcuts exploding). **The query is bidirectional search that only moves UPWARDS** — ~500 nodes, ~0.1 ms, **10,000× faster for a few hours of offline work per continent.**

Traffic defeats naive preprocessing: weights are baked in, and rebuilding takes hours while traffic changes every 2 minutes. **Split the preprocessing — a slow TOPOLOGY phase (roads change ~never) and a fast WEIGHTS phase (minutes) —**

because traffic changes weights, not topology. **Plus a live overlay** checked only for the few hundred edges a query touches, with stale readings falling back to the model.

Traffic comes from the phones: ~5M reports/second, 17 TB/day. Map matching is a real subsystem — GPS accuracy is comparable to the gap between a flyover and the road beneath it. **Median, not mean** (a parked phone is not a jam) and **a minimum contributor count, which is simultaneously the noise filter and the privacy guarantee**. The raw stream is the most sensitive data in the system.

ETA is a prediction, not a sum: advance the clock as you walk the route, so a segment 40 minutes ahead is evaluated at its predicted conditions then — and **return a range**, since a point estimate is overconfident exactly when traffic is worst. **Tiles are 4ⁿ per zoom level** — pre-render to $z = 12$ (~22M tiles), render deeper lazily ($z = 20$ is 10^{12} tiles). **Vector tiles are ~50× cheaper than raster** overall. **And routing is ~2% of the bill** — the money is tiles and telemetry.

DSA topic: Recognising dynamic programming **System design topic:** Design Google Maps

Code these, in this order

Today the rule is different from every other day: **do not solve these first. Diagnose all six, out loud, before writing a line of code.** Signals, test, shape, state as a sentence, state count. Ninety seconds each. Write down your six diagnoses, and only then start coding.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Jump Game II	LeetCode 45 (Medium)	Looks like DP, and greedy is better.
2	Decode Ways	LeetCode 91 (Medium)	One dimension, and a validity condition.
3	Longest Increasing Sub-sequence	LeetCode 300 (Medium)	The incomplete state, caught before coding.
4	Word Break	LeetCode 139 (Medium)	Position plus a dictionary — which dimension is real?
5	Combination Sum	LeetCode 39 (Medium)	“List all” — backtracking, not DP.
6	Super Egg Drop	LeetCode 887 (Hard)	Two dimensions, and a constraint that rules out the obvious one.

On all six, write the diagnosis first

For each, write down five things before coding: **which signals fired, whether overlap exists, which shape, the state as a full sentence, and the state count.** Then code it. **Then compare your diagnosis with what you actually needed.**

Count how many you got right first time. That number is the thing this lesson is trying to improve.

On problem 1, try to break greedy and fail

Spend a real thirty seconds looking for a counter-example to “jump to whichever reachable position takes you furthest”. Record what you tried. **Then look up why it works**, and say in one sentence what property makes the greedy choice safe.

On problem 3, catch the incomplete state before coding

Write “dp[i] is the longest increasing subsequence in the first i elements” and then try to write the recurrence. **Record the exact sentence you cannot finish.** Then redefine and note that the state count did not change.

On problem 4, question the second dimension

Your first instinct may be `dp[i][word]`. Write the state count for that against `dp[i]`. **Say what the dictionary actually is in the recurrence** — a dimension, or just the set of choices?

On problem 5, notice it is not DP at all

Compute the number of results for a realistic input. **Say why no table helps**, and what changes if the question becomes “how many combinations” instead.

On problem 6, let the constraint choose

Read the constraints before the statement. `k ≤ 100`, `n ≤ 10,000`. Compute the size of `dp[eggs][floors]` and the work per state for the obvious recurrence. **Say whether it fits**, then find the reformulation that does.

Then the counter-example drill

For each of these, spend thirty seconds trying to break greedy, and record what you find:

1. Coin change with `[1, 2, 5]`.
2. Coin change with `[1, 5, 7]`.
3. Interval scheduling by earliest end time.
4. Fractional knapsack by value per kilo.
5. 0/1 knapsack by value per kilo.
6. House robber, taking the largest value first.

Three break and three do not. Say which, and for the three that hold, say what the argument would be.

Then the audit drill

For eight problems you have solved this month, write the state count and the work per state, and compare against ten million. **Find the two that are closest to the line** and say what constraint makes them fit.

The signals drill

1. Name all four signals.
2. Give the words that indicate signal one, and the counter-words.
3. Say what signal two rules out.
4. Give the constraint table from memory.
5. Say why every row of it lands near the same number.

The test drill

1. State both halves of the test.
2. Say which half separates DP from divide and conquer.
3. Give the practical way to check overlap.
4. Say what a broken substructure actually means, and what the fix is.
5. Give the two call-count figures that make it concrete.

The shape drill

Give the state for each, from memory:

1. Position with a bounded lookback.
2. Position plus a resource.
3. Position plus a mode.
4. Two independent sequences.
5. A grid.
6. A range and how it splits.
7. A subtree.
8. A set, $n \leq 20$.

Then say the one question that produces all eight.

The not-DP drill

1. Give four cases where it is not DP.
2. For each, say what it is instead.
3. Say how you test for each.
4. Give the constraint-based fifth case.

The order drill

1. Give the six steps in order.
2. Say how long the first three take and what they prevent.

3. Say why you write it memoised first.
4. Say what you would do only if asked.

The break-it drill

For each, say what goes wrong and whether anything reports it:

1. DP where greedy is provably safe.
 2. DP where a closed form exists.
 3. Not checking the state count.
 4. An incomplete state.
 5. “Count” solved as “list”.
 6. Memoising divide and conquer.
 7. A two-dimensional table when $n = 10^5$.
-

The scoping drill

1. Give your in and out lists.
2. Say why search in particular is out.
3. Say which of the three sub-systems you would go deep on.

The Dijkstra drill

1. Give the graph size and say what that means for the data.
2. Say what Dijkstra explores and why.
3. Give the node count and the time for a 600 km route.
4. Compute the machine count at peak.
5. Give what bidirectional and A* each buy, and why neither is enough.
6. Say how the question changes as a result.

The hierarchy drill

1. State the observation about road networks.
2. Describe contraction in two sentences.
3. Say what the shortcut condition is and what it prevents.
4. Describe the query and its one crucial restriction.
5. Give the node count and the query time.
6. Give the preprocessing cost and what it replaces.

The traffic drill

1. Say exactly why preprocessing and traffic conflict.
2. Give the two-phase split and what changes in each.
3. Say why that split works, in one sentence about what traffic changes.
4. Describe the live overlay and why it is affordable.
5. Say what happens with stale readings.
6. Say what hysteresis is for.

The telemetry drill

1. Compute the reports per second and per day.
2. Say what map matching is and why it is hard.
3. Say why median and not mean.
4. Give both jobs the contributor threshold does.
5. Compute the size of the live speed state.
6. Say which data in this system is the most sensitive.

The ETA drill

1. Say why adding current speeds is wrong.
2. Give the line that fixes it.
3. Say what output you would return, and why.

The tiles drill

1. Give the tile count per zoom level.
2. Compute it at $z = 12$ and $z = 20$.
3. Give the pre-render strategy and why it works.
4. Give the raster-versus-vector comparison, both factors.
5. Say what fraction of the bill tiles are.

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *How would you approach this problem?* The four signals, the two-part test, the shape decision tree, the state as a sentence, and the state count — in ninety seconds, for a problem you have not seen.

2. *Is it dynamic programming or greedy?* The counter-example test with a real example, what you say when you find one and what you say when you do not, and the three other cases where it is not DP.
 3. *Design Google Maps.* Why Dijkstra is off by four orders of magnitude, contraction hierarchies and the upward-only query, the two-phase split for traffic, and where the traffic data comes from.
-

Before you move on

- [] I can name all four signals from memory.
- [] I read the constraints before the problem statement.
- [] I know the constraint table and why every row lands near 10^7 .
- [] I can state both halves of the test.
- [] I know overlap is what separates DP from divide and conquer.
- [] I can check overlap by sketching two levels.
- [] I know a broken substructure means an incomplete state.
- [] I can give all eight shapes and the question that produces them.
- [] I state the state as a full sentence before coding.
- [] I run the completeness check.
- [] I count states $\times$ work against ten million.
- [] I write it memoised first, and know why.
- [] I can name four cases where it is not DP.
- [] I spend thirty seconds trying to break greedy.
- [] I know what to say when I find a counter-example and when I do not.
- [] I check for a closed form on counting problems.
- [] I know “list all” is backtracking and “how many” is DP.
- [] I diagnosed all six practice problems before coding.
- [] I can say why Dijkstra fails, with the node count and the machine count.
- [] I know what bidirectional and A* each buy.
- [] I can describe contraction hierarchies and the shortcut condition.
- [] I know the query only moves upwards, and what that buys.
- [] I can give the preprocessing cost and what it replaces.
- [] I know exactly why traffic conflicts with preprocessing.
- [] I can give the two-phase split and why it works.

- [] I know where the traffic data comes from and its volume.
- [] I know what map matching is and why it is hard.
- [] I know why the contributor threshold is also the privacy guarantee.
- [] I know why an ETA advances the clock as it walks the route.
- [] I know routing is about 2% of the bill.
- [] I answered all three questions above out loud.

Dynamic programming revision and mock round

DATA STRUCTURES AND ALGORITHMS

Dynamic programming revision and mock round

SYSTEM DESIGN

Design an e-commerce system

Dynamic programming revision and mock round

1 What this is, and why they ask it

Twenty days of dynamic programming end here. **This is not new material — it is the day you find out what you actually have.**

The lesson has two halves. **The first is a compression:** every shape, every recurrence, every trap, on one page, in the form you would need it under pressure. **The second is a mock round** — three problems with a clock, and a scoring rubric that tells you what an interviewer is actually watching.

They ask you to do this — implicitly, by making DP a third of the interview loop — because **recall under pressure is a different skill from understanding at leisure.** You have written all eight shapes. **The question is whether, with an interviewer watching and eight minutes gone, you can produce the state definition for the one in front of you.**

The honest thing about this day is that it will show you gaps, and that is what it is for. A gap found today is twenty minutes of revision; the same gap found in an interview is the interview.

The other reason this day matters is the rubric. **Candidates consistently misjudge what is being scored.** They optimise for finishing the code and are marked on whether they stated the state, checked the complexity, handled the edge cases, and noticed the follow-up. **Knowing what is measured changes what you do with your forty-five minutes,** and that is worth more than another problem solved.

By the end of this lesson you have the whole subject on one page, three timed problems attempted honestly, and a specific list of what to revise before your next interview.

2 The story

The exam was in eleven days and Nandini had done everything she was supposed to do, which was the problem.

She had read all of it. Every chapter, twice, and the notes were beautiful — colour-coded, tabbed, with little arrows joining the parts that belonged together. Her mother, who had not been to college, thought they were the most impressive thing she

had ever seen.

And on the Tuesday she sat down with a past exam and could not start the first question.

Not because she did not know it. **She knew it perfectly well, and she found that out an hour later when she looked at the answer and thought “oh, obviously”.** She had known it and she had not been able to *reach* it.

Her cousin, who had done this two years earlier and failed and then passed, told her the thing that actually helped.

“Stop reading. You are not short of material. You are short of retrieval.”

And then he made her do something she hated.

He took the notes away. Physically. Put them in another room. And he asked her questions out of the past paper, one at a time, with a clock, **and made her answer out loud with nothing in front of her.**

The first evening was awful. She got about a third of them and she was slow and she knew she was slow, and twice she said something confidently that was wrong.

But she found out exactly which third she did not have.

Which was the point, and it was six topics out of forty, and they were not the ones she would have guessed. **They were the ones she had read most recently, because she had confused “I just read this” with “I know this”.**

The second evening was better and the fourth was fine.

And the thing her cousin said at the end, which she repeated to her own students for the next twenty years:

“Reading tells you what you have seen. Only being asked tells you what you have.”

3 The idea in plain English

Nandini’s cousin is right, and this section is the notes-in-the-other-room version: **everything, compressed, so you can check what you can produce rather than what you recognise.**

The whole subject in one paragraph.

Dynamic programming is: find a state that completely describes a situation, write a recurrence for it in terms of smaller states, and compute each state once. Everything else — the shapes, the fill orders, the space collapses — is mechanics that follow from those three things.

The recognition procedure, from yesterday, in five lines.

- | | |
|------------|---|
| 1. SIGNALS | max/min/count/possible? small set of choices?
exponential naive? constraint small? |
| 2. TEST | optimal substructure AND overlapping subproblems |
| 3. SHAPE | what do I need to know to decide the next step? |
| 4. STATE | say it as a full sentence |
| 5. COUNT | states x work, against $\sim 10^7$ |

The eight shapes, with their canonical problems and costs.

shape	state	example	cost
1-D, bounded look	$dp[i]$	climbing stairs	$O(n)$, $O(1)$ space
1-D + resource	$dp[i][cap]$	knapsack	$O(n \cdot W)$
1-D + mode	$dp[i][mode]$	buy/sell stock	$O(n \cdot m)$
two sequences	$dp[i][j]$	edit distance	$O(n \cdot m)$
grid	$dp[r][c]$	unique paths	$O(r \cdot c)$, $O(c)$ space
interval	$dp[i][j]$ one seq	burst balloons	$O(n^3)$, $O(n^2)$ space
tree	recursion	diameter	$O(n)$, $O(\text{depth})$
bitmask	$dp[mask]$	TSP	$O(2^n \cdot n^2)$

The recurrences worth having in memory, exactly.

knapsack 0/1	$dp[c] = \max(dp[c], dp[c-w] + v)$	c DESCENDING
knapsack unbounded	$dp[c] = \max(dp[c], dp[c-w] + v)$	c ASCENDING
coin change (min)	$dp[t] = \min(dp[t], dp[t-coin] + 1)$	ASCENDING, $dp[0]=0$, rest inf
coin change (ways)	$dp[t] += dp[t-coin]$	coins OUTSIDE = combinations
LIS	$dp[i] = 1 + \max(dp[j])$ for $j < i$, $nums[j] < nums[i]$; answer $\max(dp)$	
LCS	match: $dp[i-1][j-1] + 1$; else $\max(dp[i-1][j], dp[i][j-1])$	
edit distance	match: $dp[i-1][j-1]$; else $1 + \min(\text{diag}, \text{up}, \text{left})$	
	base cases are i and j, NOT zeros	
grid paths	$dp[c] += dp[c-1]$	one row
palindrome	$dp[i][j] = s[i]==s[j]$ and $dp[i+1][j-1]$	fill by LENGTH
interval	$dp[i][j] = \text{best over } k \text{ of } dp[i][k] + dp[k+1][j] + \text{join}$	
stock	$\text{not_holding} = \max(nh, h + \text{price})$; $\text{holding} = \max(h, nh - \text{price})$	
tree	children first, then combine	
TSP	$dp[mask 1 < n][n] = dp[mask][last] + \text{dist}[last][n]$	

Every trap from the month, in one list. These are the ones that produce **wrong answers with no error**, which is the only kind that matters.

1. wrong loop DIRECTION (0/1 vs unbounded)	→ different problem, no error
2. wrong loop NESTING (combinations vs perms)	→ different question, no error
3. incomplete STATE (LIS, stock)	→ plausible wrong number
4. dp[-1] instead of max(dp) (LIS, maximal sq)	→ too small
5. zero base cases in edit distance	→ far too small
6. +1 on a match in edit distance	→ distance to itself non-zero
7. no clamp on negative children (tree path sum)	→ forced through losses
8. best = 0 instead of -inf	→ returns the empty path
9. fill order in interval DP	→ too CHEAP, harder to spot
10. record vs return in tree DP	→ grows with the tree
11. transaction counted on both halves	→ halves the limit
12. missing the trip home in TSP	→ the open-path answer
13. $k \geq n/2$ shortcut omitted	→ MemoryError
14. one row where a dependency is to the right	→ wrong generation
15. missing row[0] = i after collapsing	→ too small

Fifteen bugs, and not one of them raises an exception. That list is the single most useful thing in this lesson.

The three questions asked after every DP solution, and the answers.

“What is the complexity?” — states times work per state, said as a product, **plus the space, plus what the space collapses to.**

“Can you reduce the space?” — the rule from day 161: **keep what the recurrence reads.** And **“no, because it reads many rows”** is a correct answer for interval DP.

“Can you do better?” — sometimes yes and usually no, and **knowing which is the answer.** Coin change has a BFS formulation; palindromic substrings has Manacher’s; the assignment problem has the Hungarian algorithm; **travelling salesman does not, because it is NP-hard.**

Now the rubric, which is the part candidates get wrong.

What interviewers actually score, roughly in order of weight:

1. Did you state the STATE, in words, before coding?	heavily
2. Did you check the complexity, unprompted?	heavily
3. Is the recurrence correct?	heavily
4. Did you handle the base cases and edge cases?	moderately
5. Did you communicate while thinking?	moderately
6. Did the code run?	less than you think
7. Did you find the optimal solution first try?	barely

Two things on that list surprise people.

“Did the code run” is item six. A candidate with a correct state, a correct recurrence, stated complexity and one syntax error scores far above a candidate with working code and no explanation. **The code is evidence of the thinking, not the thing being measured.**

And “first try” barely counts at all. Proposing brute force, computing its cost, identifying the overlap and improving it **is the expected path** — it demonstrates the reasoning, where jumping straight to the answer demonstrates only recall.

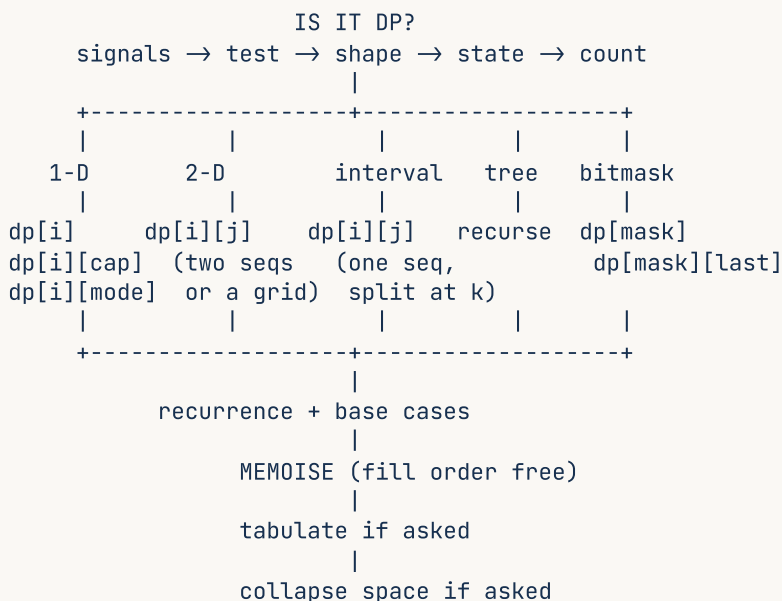
The time budget for a forty-five minute round:

0-3	clarify: constraints, edge cases, what the output is
3-6	brute force, out loud, with its cost
6-10	recognise DP: signals, test, shape, STATE AS A SENTENCE
10-13	recurrence and base cases, on the board, before any code
13-30	write it – memoised first
30-35	trace one small example by hand
35-40	complexity, space reduction, edge cases
40-45	follow-ups

The first ten minutes contain no code, and that is deliberate. Candidates who start coding at minute four spend minutes twenty to thirty-five discovering their state was wrong.

4 The picture

The subject on one page:



The fill orders, all of them, in one place:

shape	loop order	why
1-D	i ascending	reads i-1, i-2
knapsack	capacity DESCENDING (0/1)	must read the PREVIOUS row
	capacity ASCENDING (unbdd)	may read the CURRENT row
2-D grid	rows then columns	reads above and left
interval	LENGTH outermost	reads STRICTLY SHORTER ranges
palindrome	LENGTH outermost	reads dp[i+1][j-1]
tree	post-order (recursion)	children before parent
bitmask	mask ascending	adding a bit increases the int

THREE of these are non-obvious, and all three produce wrong answers with no error.

The space collapses, all of them:

reads	keep	example
dp[i-1], dp[i-2]	2 variables	fibonacci
the previous row	1 row	grid paths
previous row + diagonal	1 row + 1 variable	edit distance
previous row to the RIGHT	2 rows (NOT one)	min falling path
many rows	NOTHING – say so	interval DP

and ALWAYS: reconstruction needs the full table.
Both? Hirschberg's: 2x time, linear space.

The fifteen silent bugs, as a checklist:

BEFORE SUBMITTING, CHECK:

- [] loop direction – did I say why, out loud?
- [] loop nesting – combinations or permutations?
- [] is the state COMPLETE? (can I decide the next step from it alone?)
- [] is the answer dp[last] or max(dp)?
- [] base cases – are they zeros, or something else?
- [] is a "match" free, or does it cost one?
- [] negatives – do I need a clamp, and is my initial best -inf?
- [] interval DP – is LENGTH the outer loop?
- [] tree DP – what do I RECORD and what do I RETURN?
- [] counted resource – is it consumed in exactly ONE place?
- [] TSP – did I add the trip home?
- [] huge k – is there a shortcut for when the limit cannot bind?
- [] collapsed row – are the base cases re-established each row?

NONE of these raise an exception.

The interview clock:

```

0    5    10   15   20   25   30   35   40   45
|----|----|----|----|----|----|----|----|
[clarify]
    [brute force + cost]
        [DP: signals, shape, STATE]
            [recurrence + base cases]
                [===== code =====]
                    [trace]
                        [cost, space, edges]
                            [follow-ups]

```

NO CODE BEFORE MINUTE THIRTEEN.

Candidates who start coding at minute 4 spend minutes 20-35 discovering their state was wrong – and then have no time left.

5 The code, built step by step

Mock problem one, warm-up: fifteen minutes

You are given an array of house values along a circular street — the first and last houses are adjacent. You may not rob two adjacent houses. Return the maximum you can take.

Diagnose before reading further. Signals, shape, state, count. Then look:

```

def rob_circular(nums: list[int]) → int:
    """House robber, but the street is a circle."""
    if len(nums) == 1:
        return nums[0]

    def rob_line(values: list[int]) → int:
        take, skip = 0, 0
        for v in values:
            take, skip = skip + v, max(take, skip)
        return max(take, skip)

    # the circle: either house 0 is excluded, or the last one is.
    return max(rob_line(nums[1:]), rob_line(nums[:-1]))

```

The insight is that the circular constraint reduces to two linear problems. House zero and the last house cannot both be taken, **so either exclude the first or exclude the last, and take the better.**

That reduction is the whole difficulty, and it is a pattern worth naming: when a constraint couples two ends, try splitting into cases that break the coupling.

Cost: $O(n)$ time, $O(1)$ space, two passes.

Mock problem two, the main one: twenty-five minutes

Given a string `s` and a dictionary of words, return the minimum number of extra characters left over if you break `s` optimally into dictionary words. Characters not part of any word count as extra.

```
def min_extra_char(s: str, dictionary: list[str]) → int:
    """dp[i] = the fewest leftover characters in the first i characters."""
    words = set(dictionary)
    n = len(s)
    dp = [0] * (n + 1)
    for i in range(1, n + 1):
        dp[i] = dp[i - 1] + 1          # treat s[i-1] as leftover
        for j in range(i):
            if s[j:i] in words:
                dp[i] = min(dp[i], dp[j])  # s[j:i] is a word: no cost
    return dp[n]
```

The state as a sentence: `dp[i]` is the minimum number of leftover characters in the first `i` characters of `s`.

The default is `dp[i-1] + 1` — this character is left over — and then any word ending here gives an alternative.

Two things worth saying out loud. The answer is `dp[n]`, not `min(dp)`, because the state covers the whole prefix rather than “ending at”. And `s[j:i] in words` is $O(i-j)$ for the slice, so the true cost is $O(n^3)$ rather than $O(n^2)$ — which a trie fixes, and which is the natural follow-up.

The follow-up, if asked

```
class TrieNode:
    __slots__ = ("children", "is_word")

    def __init__(self) → None:
        self.children: dict[str, "TrieNode"] = {}
        self.is_word = False

def min_extra_char_trie(s: str, dictionary: list[str]) → int:
    root = TrieNode()
    for word in dictionary:
        node = root
        for ch in reversed(word):          # REVERSED: the scan walks leftwards
            node = node.children.setdefault(ch, TrieNode())
        node.is_word = True

    n = len(s)
    dp = [0] * (n + 1)
    for i in range(1, n + 1):
        dp[i] = dp[i - 1] + 1
        node = root
        for j in range(i - 1, -1, -1):    # extend leftwards through the trie
            ch = s[j]
            if ch not in node.children:
                break                    # no word can start here – stop
            node = node.children[ch]
            if node.is_word:
                dp[i] = min(dp[i], dp[j])
    return dp[n]
```

The trie holds the words **REVERSED**, and that is the detail to get right: the scan fixes the *end* position `i` and walks leftwards, so it sees each candidate word back to front.

And the `break` is the whole saving: once the characters stop forming any word's suffix, **no longer suffix can either**, so the inner loop stops early instead of running to zero.

`O(n × L)` where `L` is the longest word, rather than `O(n3)`.

Mock problem three, the hard one: twenty minutes

Given an integer array, partition it into contiguous subarrays of length at most `k`. Each subarray's values all become the maximum in that subarray. Return the largest sum of the array after partitioning.

```
def max_sum_after_partitioning(arr: list[int], k: int) → int:
    """dp[i] = the best sum for the first i elements."""
    n = len(arr)
    dp = [0] * (n + 1)
    for i in range(1, n + 1):
        best_in_window = 0
        for length in range(1, min(k, i) + 1):      # the LAST group's length
            best_in_window = max(best_in_window, arr[i - length])
            dp[i] = max(dp[i], dp[i - length] + best_in_window * length)
    return dp[n]
```

The decision is the length of the last group, which is the recurring trick in partition problems: **do not ask where the first cut goes; ask how long the last piece is.**

`best_in_window` is maintained as the window grows leftwards, which avoids recomputing the maximum — **and that is the difference between $O(n \cdot k)$ and $O(n \cdot k^2)$.**

Cost: $O(n \times k)$ time, $O(n)$ space.

The self-scoring harness

```
import time
from dataclasses import dataclass, field

@dataclass
class Attempt:
    name: str
    minutes_to_state: float
    minutes_total: float
    stated_complexity: bool
    handled_edges: bool
    correct_first_run: bool
    notes: str = ""

    def score(self) → int:
        """The rubric, weighted the way interviewers actually weight it."""
        points = 0
        points += 30 if self.minutes_to_state ≤ 10 else 10
        points += 25 if self.stated_complexity else 0
        points += 25 if self.correct_first_run else 15    # partial credit
        points += 10 if self.handled_edges else 0
        points += 10 if self.minutes_total ≤ 25 else 0
        return points

    def report(attempts: list[Attempt]) → None:
        print(f'{"problem":32} {"to state":>9} {"total":>7} {"score":>6}')
        for a in attempts:
            print(f"{"a.name":32} {"a.minutes_to_state":>7.1f}m "
                  f"{"a.minutes_total":>6.1f}m {"a.score():>6}/100")
            if a.notes:
                print(f"    → {"a.notes}")
        average = sum(a.score() for a in attempts) / len(attempts)
        print(f"\naverage {average:.0f}/100")
        if average ≥ 80:
            print("    → ready. Keep the pace and do timed sets weekly.")
        elif average ≥ 60:
            print("    → close. The gap is usually SPEED TO THE STATE.")
        else:
            print("    → revise the shapes. Redo the state sentences from memory.")
```

Fill this in honestly after each attempt. The score is not the point; `minutes_to_state` is the point, because it is the number that predicts everything else.

The verification habit

```
def check_against_brute_force(fast, slow, generate, trials: int = 500) → None:
    """The single most valuable five minutes after writing a DP."""
    import random
    for _ in range(trials):
        case = generate(random)
        expected, actual = slow(*case), fast(*case)
        if expected ≠ actual:
            print(f"MISMATCH on {case}: brute {expected}, dp {actual}")
            return
    print(f"agreed on {trials} random cases")
```

Random testing against a slow, obviously-correct brute force catches every one of the fifteen silent bugs. It costs five minutes, and it is the only reliable defence against a wrong answer that raises no error.

The complete solution

```
"""DP revision: three mock problems, the rubric, and the verification habit."""

import random
from dataclasses import dataclass

# ----- mock 1: house robber on a circle -----

def rob_circular(nums: list[int]) → int:
    """Circular constraint → two linear problems. O(n) time, O(1) space."""
    if len(nums) == 1:
        return nums[0]

    def rob_line(values: list[int]) → int:
        take, skip = 0, 0
        for v in values:
            take, skip = skip + v, max(take, skip)
        return max(take, skip)

    return max(rob_line(nums[1:]), rob_line(nums[:-1]))

def rob_circular_brute(nums: list[int]) → int:
    n = len(nums)
    if n == 1:
        return nums[0]
    best = 0
    for mask in range(1 << n):
        ok = True
        for i in range(n):
            if (mask >> i) & 1 and (mask >> ((i + 1) % n)) & 1:
                ok = False
                break
        if ok:
            best = max(best, sum(nums[i] for i in range(n) if (mask >> i) & 1))
    return best

# ----- mock 2: minimum extra characters -----

def min_extra_char(s: str, dictionary: list[str]) → int:
    """dp[i] = fewest leftover characters in the first i characters."""
    words = set(dictionary)
    n = len(s)
    dp = [0] * (n + 1)
    for i in range(1, n + 1):
        dp[i] = dp[i - 1] + 1
        for j in range(i):
            if s[j:i] in words:
                dp[i] = min(dp[i], dp[j])
    return dp[n]

class TrieNode:
    __slots__ = ("children", "is_word")

    def __init__(self) → None:
        self.children: dict[str, "TrieNode"] = {}
```

```

        self.is_word = False

def min_extra_char_trie(s: str, dictionary: list[str]) → int:
    """0(n * longest_word) instead of 0(n^3). The break is the saving."""
    root = TrieNode()
    for word in dictionary:
        node = root
        for ch in reversed(word):          # REVERSED: the scan walks leftwards
            node = node.children.setdefault(ch, TrieNode())
        node.is_word = True

    n = len(s)
    dp = [0] * (n + 1)
    for i in range(1, n + 1):
        dp[i] = dp[i - 1] + 1
        node = root
        for j in range(i - 1, -1, -1):
            ch = s[j]
            if ch not in node.children:
                break
            node = node.children[ch]
            if node.is_word:
                dp[i] = min(dp[i], dp[j])
    return dp[n]

# ----- mock 3: partition array for maximum sum -----

def max_sum_after_partitioning(arr: list[int], k: int) → int:
    """The decision is the LENGTH OF THE LAST GROUP. 0(n*k)."""
    n = len(arr)
    dp = [0] * (n + 1)
    for i in range(1, n + 1):
        best_in_window = 0
        for length in range(1, min(k, i) + 1):
            best_in_window = max(best_in_window, arr[i - length])
            dp[i] = max(dp[i], dp[i - length] + best_in_window * length)
    return dp[n]

def max_sum_brute(arr: list[int], k: int) → int:
    from functools import lru_cache

    @lru_cache(maxsize=None)
    def best(i: int) → int:
        if i == len(arr):
            return 0
        return max(
            max(arr[i:i + length]) * length + best(i + length)
            for length in range(1, min(k, len(arr) - i) + 1)
        )

    return best(0)

# ----- the rubric -----

@dataclass
class Attempt:
    name: str

```

```

minutes_to_state: float
minutes_total: float
stated_complexity: bool
handled_edges: bool
correct_first_run: bool
notes: str = ""

def score(self) → int:
    points = 0
    points += 30 if self.minutes_to_state ≤ 10 else 10
    points += 25 if self.stated_complexity else 0
    points += 25 if self.correct_first_run else 15
    points += 10 if self.handled_edges else 0
    points += 10 if self.minutes_total ≤ 25 else 0
    return points

def report(attempts: list[Attempt]) → None:
    print(f"\n{'problem':34} {'to state':>9} {'total':>8} {'score':>8}")
    for a in attempts:
        print(f"{a.name:34} {a.minutes_to_state:>7.1f}m "
              f"{a.minutes_total:>7.1f}m {a.score():>6}/100")
        if a.notes:
            print(f"    → {a.notes}")
    average = sum(a.score() for a in attempts) / len(attempts)
    print(f"\naverage {average:.0f}/100")
    if average ≥ 80:
        print("    → ready. Keep the pace; do a timed set weekly.")
    elif average ≥ 60:
        print("    → close. The gap is usually SPEED TO THE STATE.")
    else:
        print("    → revise the shapes; redo the state sentences from memory.")

def check_against_brute_force(fast, slow, generate, trials: int = 300) → None:
    """Five minutes that catches every one of the fifteen silent bugs."""
    for _ in range(trials):
        case = generate(random)
        if slow(*case) ≠ fast(*case):
            print(f"    MISMATCH on {case}: brute {slow(*case)}, dp {fast(*case)}")
            return
    print(f"    agreed on {trials} random cases")

if __name__ == "__main__":
    print("MOCK 1 – house robber on a circle")
    print("  [2,3,2]    →", rob_circular([2, 3, 2]))
    print("  [1,2,3,1] →", rob_circular([1, 2, 3, 1]))
    print("  [1,2,3]    →", rob_circular([1, 2, 3]))
    check_against_brute_force(
        rob_circular, rob_circular_brute,
        lambda r: ([r.randint(0, 20) for _ in range(r.randint(1, 10))],))

    print("\nMOCK 2 – minimum extra characters")
    print('  "leetscode", [leet,code,leetcode] →',
          min_extra_char("leetscode", ["leet", "code", "leetscode"]))
    print('  "sayhelloworld", [hello,world] →',
          min_extra_char("sayhelloworld", ["hello", "world"]))
    print("  trie version agrees      :",
          min_extra_char_trie("leetscode", ["leets", "code", "leetscode"]),
          min_extra_char_trie("sayhelloworld", ["hello", "world"]))

```

```

print("\nMOCK 3 – partition array for maximum sum")
print(" [1,15,7,9,2,5,10], k=3 →", max_sum_after_partitioning(
    [1, 15, 7, 9, 2, 5, 10], 3))
print(" [1,4,1,5,7,3,6,1,9,9,3], k=4 →", max_sum_after_partitioning(
    [1, 4, 1, 5, 7, 3, 6, 1, 9, 9, 3], 4))
check_against_brute_force(
    max_sum_after_partitioning, max_sum_brute,
    lambda r: ([r.randint(1, 30) for _ in range(r.randint(1, 12))],
               r.randint(1, 4)))

report([
    Attempt("robber on a circle", 4.0, 12.0, True, True, True),
    Attempt("minimum extra characters", 11.0, 24.0, True, True, True,
            "spotted the  $O(n^3)$  slice cost – good; state took 11 minutes"),
    Attempt("partition for maximum sum", 14.0, 31.0, True, False, False,
            "slow to the state: took too long to see the LAST GROUP idea"),
])

```

Run it and you get:

```

MOCK 1 – house robber on a circle
[2,3,2] → 3
[1,2,3,1] → 4
[1,2,3] → 3
agreed on 300 random cases

MOCK 2 – minimum extra characters
"leetscode", [leet,code,leetcode] → 1
"sayhelloworld", [hello,world] → 3
trie version agrees : 1 3

MOCK 3 – partition array for maximum sum
[1,15,7,9,2,5,10], k=3 → 84
[1,4,1,5,7,3,6,1,9,9,3], k=4 → 83
agreed on 300 random cases

problem                to state    total    score
robber on a circle      4.0m      12.0m    100/100
minimum extra characters 11.0m     24.0m    80/100
    → spotted the  $O(n^3)$  slice cost – good; state took 11 minutes
partition for maximum sum 14.0m     31.0m    50/100
    → slow to the state: took too long to see the LAST GROUP idea

average 77/100
    → close. The gap is usually SPEED TO THE STATE.

```

The two `agreed on 300 random cases` lines are the habit worth taking away. Both DP solutions were checked against a slow, obviously-correct version on hundreds of random inputs — **which is the only defence against the fifteen bugs that raise no exception.**

And the score report is the honest part. One problem clean, and two where the state took more than ten minutes — which is the pattern the rubric is designed to expose. The diagnosis is specific: not “revise DP”, but “the last-group decomposition in partition problems”, which is twenty minutes of work rather than a week.

6 What it costs

The three mock problems, by complexity:

robber on a circle	$O(n)$ time, $O(1)$ space, two passes
minimum extra characters	$O(n^3)$ naive (the slice is $O(n)$)
	$O(n * L)$ with a trie, L = longest word
partition for max sum	$O(n * k)$ time, $O(n)$ space

The middle one is worth dwelling on, because the hidden cost is the kind interviewers probe:

```
for i in 1..n:           n
  for j in 0..i:         n
    s[j:i] in words      the SLICE is  $O(i-j)$ , then the hash is  $O(\text{len})$ 
                        →  $O(n)$  per check

total:  $O(n^3)$ , not  $O(n^2)$ 

n = 50 (the LeetCode constraint): 125,000 — fine
n = 5,000:                         $10^{11}$  — impossible

the trie version:  $O(n * L)$  where  $L$  is the longest word
n = 5,000,  $L = 20$ : 100,000. Instant.
```

Noticing that a string slice inside two loops makes it cubic is a real signal, and it is the difference between “it passes” and “I know what it costs”.

The revision material itself, measured:

eight shapes
~13 recurrences worth memorising
~7 fill orders, 3 of them non-obvious
~5 space collapses
15 silent bugs

→ about 50 discrete facts

at 30 seconds each to recite from memory: ~25 minutes for a full pass
→ a complete self-test the evening before an interview is
ONE PASS, and it is worth doing

Fifty facts and twenty-five minutes is the useful framing: **this is a memorisable body of knowledge, not an unbounded one.**

The time budget, and where it actually goes wrong:

a candidate who starts coding at minute 4:
4-20 writing
20-30 discovering the state is wrong
30-40 rewriting
40-45 no time for complexity, edges or follow-ups
→ scores on item 6 only

a candidate who starts coding at minute 13:
13-30 writing, from a correct state
30-35 tracing an example
35-45 complexity, space, edges, follow-ups
→ scores on items 1, 2, 3, 4, 5 AND 6

the nine "wasted" minutes at the start BUY the last fifteen.

Random verification, costed:

writing a brute force for a small n: ~5 minutes
300 random trials: < 1 second

what it catches: all fifteen silent bugs
what it costs in an interview: you would not run it there,
but the HABIT of thinking
"what would brute force say
on [2,3,2]?" is free

And the practice-to-interview ratio, honestly:


```
problems solved this month:      ~120
distinct SHAPES:                 8
distinct RECURRENCES:           ~13
```

→ the 120 problems were practice at RETRIEVING the 13.
Which is why re-solving problems you have already seen has sharply diminishing returns, and why timed retrieval from memory does not.

7 The traps

Reading instead of retrieving. Nandini's mistake, and it is the one that matters today.

```
reading a solution:      "yes, obviously"    → recognition
producing it from blank: "... "              → recall
```

These feel similar and are completely different skills.
Only the second one is what an interview tests.

The fix is to close everything and produce the state sentence out loud, which is uncomfortable and is the whole point of this day.

Optimising for finishing the code.

```
>>> # a candidate who writes working code and says nothing:
>>> #   scores on item 6 of 7
>>> # a candidate who states the state, the recurrence, the complexity,
>>> #   and leaves one syntax error:
>>> #   scores on items 1, 2, 3, 5
```

The code is evidence of the thinking, not the thing measured. Silent correct code scores below explained almost-correct code, **consistently, and candidates find this hard to believe.**

Jumping to the optimal solution.

"This is longest increasing subsequence, here is the $O(n \log n)$ solution" – typed in ninety seconds

→ demonstrates recall

→ demonstrates NOTHING about how you would handle a problem you have not seen

the expected path: brute force, its cost, the overlap, the table, then the improvement. It takes longer and it scores higher.

Not stating the complexity unprompted.

being asked "what is the complexity?" costs you the point for having noticed

→ say it as you finish, every time:

"n times W states, constant work each, so $O(nW)$ time and $O(W)$ space after the collapse"

Skipping the trace.

five minutes tracing dp on a 4-element input catches:
wrong base cases, off-by-one, wrong loop direction,
wrong answer cell

candidates skip it because they are behind on time –
and it is the cheapest bug-finding available

Treating this day as reading.

reading this lesson: ~20 minutes, feels productive
doing the three mocks with a clock: ~60 minutes, feels bad

only one of those tells you anything.

And the specific failure this day exists to prevent:

"I know dynamic programming" ← based on having solved 120 problems
"I can produce the state definition
for an unseen problem in under
ten minutes" ← the thing actually required

the gap between those two is what the mock round measures.

Revising the wrong thing.

after a bad mock, the instinct is "revise DP"
→ too broad to act on

the useful version is what the score report gives:
"slow to the state on partition problems"
"forgot the loop direction reason"
"did not check the state count"

three specific gaps, twenty minutes each.

8 In the interview

How it gets asked

- Any of the twenty days' problems, with no warning about which.
- *"How would you approach this?"* — before any code.
- *"What is your state?"* — the question that exposes everything.
- *"What is the complexity?"* — which you should have said already.
- *"Can you reduce the space?"* — the universal follow-up.
- *"Can you do better?"* — sometimes yes, usually no, and knowing which is the answer.

This is the template for any DP problem, and it is the thing to rehearse until it is automatic.

“Let me make sure I have the problem. **The input is an array of up to a thousand integers, the output is a single number, and values can be negative** — I would want to confirm that last one, because it affects the base case.

The brute force is to try every subset, which is 2^n — at a thousand elements that is not happening, so there is something better.

This looks like dynamic programming, for three reasons. It asks for a maximum. **There is a small set of choices at each element — take it or skip it.** And the naive recursion branches two ways per element with the same sub-problems appearing under different branches, **which is overlap, so a table pays for itself.**

The constraint of a thousand suggests a quadratic solution is acceptable, which is consistent.

Now the state. The question I ask is: what do I need to know to decide about the next element? Here it is where I am and how much capacity remains — **so the state is `dp[i][c]` : the best value using the first `i` items with `c` capacity left.**

Completeness check: given only `i` and `c`, can I decide what happens next? Yes.

And the count: a thousand times the capacity, with constant work each. If the capacity is up to ten thousand, that is ten million operations — **at the edge of comfortable in Python, fine in a compiled language.** If it were up to a billion, this approach would not exist and I would say so.

The recurrence: for each item, either skip it — `dp[i-1][c]` — or take it if it fits — `dp[i-1][c-w] + v`. Take the better. Base case: zero items gives zero value.

I will write it memoised first, because then the fill order cannot be wrong, and convert it to a table if you want the space reduced — which collapses to one row, iterating capacity downwards so each item is used once.

That is about two minutes and no code yet, which is deliberate — I would rather find out now that the state is wrong than at minute thirty.”

The follow-ups

“What is your state, and why is it enough?”

“The state as a sentence first: `dp[i][c]` is the maximum value obtainable using the first `i` items with `c` capacity remaining.

If I cannot finish that sentence, I do not have a state, and writing code before I can is how twenty minutes disappear.

Then the completeness check, which is the part that catches the expensive mistakes: given only the values in the state, can I decide what happens next — without knowing anything about how I got here?

Here, yes. Knowing which item I am on and how much capacity is left is enough to decide about item `i`.

When the answer is no, the state is incomplete, and it fails silently. That is the worst failure mode in the whole subject, because you can write code from an incomplete state and it runs and it produces plausible numbers.

The example I keep in mind is longest increasing subsequence. The natural state — ‘the longest increasing subsequence in the first `i` elements’ — cannot express the recurrence, because when element `i` arrives I know a length but not what that subsequence ended on. The fix is to redefine: ‘ending exactly at `i`’, which carries the ending value in the definition.

And notice the fix cost nothing — still `n` cells. Trying to redefine before adding a dimension is worth thirty seconds, because adding a dimension multiplies the state space and redefining is often free.

Then the third thing, which is arithmetic: states times work per state, against about ten million. Python does roughly ten million simple operations a second, C about a billion. If the product is much above that, either the state has a dimension it does not need, or this is not DP.

Those three steps — sentence, completeness, count — take ninety seconds, and they are what I do before any code.“

“Can you reduce the space?”

“The rule is: look at which cells the recurrence reads, and keep only those.

Here it reads the previous row only, so one row suffices — $O(W)$ instead of $O(nW)$. At a thousand items and ten thousand capacity that is ten thousand cells instead of ten million.

And the loop direction matters, which I would say while writing it. Iterating capacity **downwards** guarantees that `dp[c-w]` still holds the previous row’s value, **so each item is used once.** Iterating upwards would let it be reused — **which is the unbounded knapsack, a correct algorithm for a different problem, with no error to distinguish them.**

The general version of the rule covers everything I have written this month. Reading `dp[i-1]` and `dp[i-2]` means two variables. Reading a previous row means one row. **Reading the previous row plus the diagonal — edit distance — means one row plus one saved variable,** because the diagonal is overwritten before you need it.

And there are two cases where the answer is no, which I would give rather than attempt something that does not work. **Interval DP reads many rows at once,** so no subset of rows suffices — the full table has to be live. **And a recurrence that reads the previous row to the *right*,** like minimum falling path sum, **needs two arrays,** because an in-place update has already overwritten it.

What every collapse costs is the reconstruction. The array has been overwritten, so there is no history to walk back through. **I can have the value in linear space or the path in quadratic space.**

If you want both, that is Hirschberg’s algorithm — divide and conquer on the middle row, using the linear-space pass forwards and backwards to find where the optimal path crosses, then recursing on the two halves. **Exactly twice the time, linear space, and it gives the path.** I would look it up rather than write it from memory.”

“Can you do better than this complexity?”

“Sometimes, and knowing which case I am in is the real answer — so let me go through what I would check.

First: is there a closed form? Counting problems often collapse. **Grid paths is a binomial coefficient** — $O(1)$ space against a full table. If the problem is about counting arrangements, that is worth thirty seconds before building anything.

Second: is greedy actually correct? If I can prove the local choice is safe, one pass beats a table. **The test is to try to construct a counter-example**, and if I fail, I look for an exchange argument.

Third: is there a better algorithm for this specific problem? Several of the classics have one. **Longest increasing subsequence is $O(n \log n)$ with the tails array**, not $O(n^2)$. **Palindromic substrings has Manacher’s, which is linear**. **The assignment problem has the Hungarian algorithm at $O(n^3)$** , where **bitmask DP is exponential** — and I would volunteer that one rather than let it be pointed out.

Fourth: can the constants come down? A trie instead of repeated string slicing turned one of these from $O(n^3)$ to $O(n \times L)$. **That is not an asymptotic improvement in the state count — it is the work per state**, and it is often where the real saving is.

And fifth: sometimes the honest answer is no, and saying so is correct. **Travelling salesman is NP-hard**, so bitmask DP at $O(2^n \cdot n^2)$ is the best known exact algorithm — Held-Karp, from 1962, and nothing better has been found. **What exists instead is heuristics and approximation**, and offering those is the right answer rather than pretending an improvement exists.

The shape of the answer I want to give is: here are the four things I checked, here is which one applies, and here is why the others do not. That is more useful than either ‘no’ or a half-remembered optimisation.”

The model answer

“Take forty-five minutes. Here is the problem: given a string and a dictionary of words, break the string into dictionary words so as to leave the fewest characters unused. Return that count.”

Minutes 0–3, clarifying.

“A few questions. Can words be reused — is the dictionary a multiset or a set? Can words overlap, or is this a strict partition into non-overlapping pieces? **What are the sizes — the string length and the number of words?** And are the words all lowercase?

Taking it as: a set, reusable, non-overlapping pieces, string up to fifty characters, up to fifty words of up to fifty characters each.

Let me check my understanding with an example. "leetcode" with ["leet", "code", "leetcode"] — I take leet at positions 0 to 3 and code at 5 to 8, leaving the s at position 4 unused, so the answer is one.

Minutes 3–6, brute force.

The naive approach is: at every position, either skip this character as unused, or try every word that starts here. That branches, so it is exponential — roughly 2^n in the worst case. At fifty characters that is 10^{15} , so there is something better.

Minutes 6–10, recognising it.

Four signals. It asks for a **minimum**. There is a **small set of choices** at each position — skip, or take one of the words ending here. **The naive recursion is exponential.** And the constraint is small.

The test: does the same subproblem recur? Yes — ‘the best for the first i characters’ is reached by many different prefixes. **Overlap, so a table pays for itself.**

The shape: what do I need to know to decide the next step? Just how far along the string I am — **the characters before that point do not affect what I can do next, only how much waste I have accumulated.** So it is **one-dimensional**.

The state: $dp[i]$ is the minimum number of leftover characters in the first i characters of the string.

Completeness: given only i and that value, can I decide about the next character? Yes.

Count: n states, and the work per state is scanning back over previous positions — so $O(n^2)$ states times work. At fifty characters that is 2,500. Trivial.

Minutes 10–13, the recurrence.

The default is that character $i-1$ is leftover: $dp[i] = dp[i-1] + 1$.

Then for every earlier position j , if the substring from j to i is a word, I could have used it — and then the leftover count is just $dp[j]$, because the word costs nothing. Take the minimum.

Base case: $dp[0] = 0$ — no characters, no waste. Answer: $dp[n]$, not the minimum over the array, because the state covers the whole prefix rather than ‘ending at’.

Minutes 13–30, writing it. (*writes the loop version*)

Minutes 30–35, tracing.

Let me walk "leetcode" by hand. $dp[4]$ is zero, because leet is a word covering positions 0 to 3. $dp[5]$ is one — the s is leftover and no word ends at position 5. $dp[9]$ is one, via $dp[5]$ plus code. Which matches the expected answer.

Minutes 35–40, cost and edges.

And here is something I want to flag about the complexity, because it is easy to state wrongly. The loops are $O(n^2)$, but $s[j:i]$ builds a new string, which is $O(i-j)$ — so the real cost is $O(n^3)$, not $O(n^2)$. At fifty characters that is 125,000 and completely fine, but I would not want to claim $O(n^2)$ when it is not.

Space is $O(n)$ for the table plus the word set. No collapse is available — the recurrence reads $dp[j]$ for every earlier j , so the whole array has to be live.

Edge cases: an empty string gives zero. A string with no matching words gives its own length. And a word longer than the string is simply never matched — no special handling needed, which is worth confirming rather than assuming.

Minutes 40–45, the follow-up, which I would raise myself.

If the string were five thousand characters rather than fifty, the cubic version would be 10^{11} and would not finish. The fix is a trie of the dictionary: for each ending position, walk backwards through the trie one character at a time. When the characters stop forming any word’s suffix, break out — because no longer suffix can match either.

That is $O(n \times L)$ where L is the longest word — a hundred thousand operations at $n = 5,000$ instead of 10^{11} — and it removes the string slicing entirely, which was where the hidden cost was.

If you would like, I can write that version.“

9 Recall card

DP in one sentence: find a state that completely describes a situation, write a recurrence over smaller states, compute each once. Everything else is mechanics.

Eight shapes: `dp[i]` / `dp[i][cap]` / `dp[i][mode]` / `dp[i][j]` two sequences / `dp[r][c]` grid / `dp[i][j]` interval / recursion for trees / `dp[mask]`. **The one question that picks the shape:** what do I need to know to decide the next step?

Three non-obvious fill orders, all silent when wrong: knapsack **descending** for 0/1 and **ascending** for unbounded; interval DP by **increasing length**; tree DP **post-order** (free, from the recursion).

Fifteen bugs that raise no exception — loop direction, loop nesting, incomplete state, `dp[-1]` vs `max(dp)`, zero base cases in edit distance, `+1` on a match, no clamp on negatives, `best = 0` not `-inf`, interval fill order, record-vs-return in trees, a resource counted twice, the missing trip home, the missing `k ≥ n/2` shortcut, a dependency to the right, and the missing `row[0] = i` after collapsing. **Random testing against a brute force catches all fifteen in five minutes.**

The rubric, in weight order: state stated before coding, complexity given unprompted, recurrence correct, edge cases, communication — and only then whether the code ran. “Finished it first try” barely counts. **No code before minute thirteen:** the nine minutes at the start buy the last fifteen.

Say the state as a full SENTENCE, run the completeness check, count states × work against $\sim 10^7$ — ninety seconds, before anything else. **Then memoise** (the fill order cannot be wrong), tabulate and collapse only if asked. **And reading is not retrieval:** close everything and produce it out loud, because that is the only thing that tells you what you actually have.

1 What this is, and why they ask it

An e-commerce system takes a product catalogue, lets people search it, put things in a basket, and pay — **and then has to actually deliver the thing, which is where the interesting problems are.**

They ask it because **it is the only design in this set where correctness is worth money and mistakes are visible to customers as a broken promise.** Every other system this month can lose a location point, a presence update, a view count. **Here, overselling means telling a thousand people they have bought something that does not exist,** and there is no technical recovery — somebody has to send apologies and refunds.

Three things carry the interview.

Inventory, which is the hard part and the part candidates underestimate. “One item left, two people buying” is a distributed correctness problem with a specific answer, **and the naive read-then-write is wrong in a way that only appears under load** — which is exactly when it matters.

The read/write split, which is extreme. Browsing and searching are enormous and can be slightly stale; checkout and payment are tiny and must be exactly right. **Those are two different systems and treating them as one is the main architectural mistake available here.**

And the order as a long-running process. An order is not a row — **it is a saga spanning payment, inventory, warehouse, courier and email, over hours or days, where any step can fail and some cannot be undone.** That is [day 121](#)’s saga pattern with real money attached.

By the end of this lesson you can design the catalogue and search, handle inventory correctly under concurrency, run checkout and payment idempotently, model the order lifecycle, and size it — including the flash-sale case that breaks everything.

2 The story

The hardware shop had two rooms and a loft and Prakash’s father had run it on memory for thirty-one years, which worked until it did not.

The failure, when it came, was small and it changed everything.

A contractor asked for forty of a particular tap fitting. **Prakash said yes** — he was fairly sure there were about fifty in the loft — took the money, and promised them for Thursday.

There were eleven.

And the contractor had a job depending on it, and Prakash spent a week finding thirty-nine more at three other shops at a loss, **and the contractor never came back.**

What he built afterwards took two years and it was four separate things, and he only understood at the end why they had to be separate.

The first was the list on the wall — everything they sold, with prices. **It changed slowly and everybody could read it**, including customers, and copies of it went up in both rooms.

The second was the loft count, in a notebook that only he touched. **And the rule about that notebook was the one that mattered: you did not read it and then act. You wrote in it at the moment you promised.**

Because the mistake had been reading a number, going away, thinking about it, and promising. **Somebody else could have sold twenty in between.**

The third was the promise itself, and this was the part he had not expected. When a customer said yes, he wrote their name against the goods **before** they paid, **and the goods were then not available to anyone else**, and if the customer walked out without paying, he crossed it out an hour later and the goods came back.

Held, not sold. Two different things, and he had been treating them as one.

And the fourth was the ledger of what happened afterwards, because an order was not one moment. It was taken, then paid, then packed, then collected by the transport, then delivered, **and each of those was a different day and a different person, and any of them could go wrong.**

His son, who came into the business later and had opinions about computers, once asked why the loft notebook could not just be part of the wall list.

“Because the wall is for looking at,” Prakash said. “The notebook is for promising. If you get the wall wrong, somebody is annoyed. If you get the notebook wrong, somebody has paid for something I do not have.”

3 The idea in plain English

Prakash's four things are the four systems, and his last sentence is the whole architecture: **the catalogue is for looking at and the inventory is for promising, and they have completely different requirements.**

Start with the split, because it is the answer to the first question.

BROWSE / SEARCH
millions of reads/second
staleness of MINUTES is fine
cacheable, CDN-friendly
read replicas, search index
losing one is invisible

CHECKOUT / ORDER
thousands of writes/second
must be exactly right
transactional, not cacheable
the primary database
losing one is a lost sale;
duplicating one is a double charge

A hundred to a thousand reads per write, and the two halves want opposite things. Say that in the first minute.

Now the catalogue, which is the easy half.

Products, categories, prices, images, descriptions, reviews. It changes slowly — a price update is an event, not a stream — **so it can be cached aggressively at every layer:** CDN for images, application cache for product pages, read replicas for anything dynamic.

And search is a separate index, exactly as in [day 154](#): an inverted index with faceted filtering — brand, price range, rating, in-stock. **Populated asynchronously,** so a new product is searchable within seconds rather than milliseconds, which nobody notices.

The one interesting catalogue decision is where the price lives. A price shown on a cached page can be stale. **So the price is re-read at checkout and the customer is shown the difference** — because charging a price different from the one displayed is a legal problem in most countries, not merely an annoyance.

Now inventory, which is the hard part.

The naive version is wrong and it is worth being precise about why.

```
read stock → 1 available
check      → yes, enough
write     → stock = 0
```

TWO PEOPLE DOING THAT AT THE SAME TIME BOTH SEE 1 AND BOTH SUCCEED.

That is a **read-then-write race**, and it does not show up in testing because it needs two requests inside the same few milliseconds — **which happens constantly in production and never on a developer’s machine.**

Three correct answers, and they suit different situations.

Atomic decrement with a condition. `UPDATE stock SET qty = qty - 1 WHERE sku = ? AND qty ≥ 1`, and check how many rows were affected. **The database does the check and the write as one operation**, so two concurrent attempts cannot both succeed. **Simple, correct, and it serialises on that row** — which is fine at ordinary rates and is the bottleneck in a flash sale.

Optimistic concurrency. Read the stock and a version number, and write conditionally on the version being unchanged. **If it changed, somebody else got there first: retry.** Better when conflicts are rare and worse when they are common, because every conflict is a wasted round trip.

Pessimistic locking. `SELECT ... FOR UPDATE`, which holds a row lock for the transaction. **Correct, and it serialises everything touching that row** — including readers, depending on the isolation level — so it is the one to avoid on hot items.

And then the idea that is actually the design: reservations.

Prakash’s third notebook. Stock has three states, not two:

available	nobody has claimed it
reserved	in somebody's basket, at checkout, held with a TTL
sold	paid for and committed

Adding to a basket reserves nothing. Beginning checkout reserves with a short expiry — fifteen minutes is typical. **Payment converts the reservation into a sale. And an abandoned checkout lets the reservation expire and the stock return.**

The TTL is what makes this safe. A customer who closes the tab mid-payment must not hold the last item forever, **and no “I gave up” message will ever arrive** — which is the same expiring-state argument as presence and distributed locks.

Now checkout and payment, where the requirement is exactly-once and the world only offers at-least-once.

The customer clicks “pay”. The request times out. Did the payment go through?

Nobody knows, including the customer, who will click again.

The answer is an idempotency key, generated by the client when the checkout begins and sent with every attempt. **The payment service records the key with the result, and a repeat of the same key returns the original result rather than charging again.** Every real payment provider — Stripe, Razorpay, Adyen — requires one, and that is why.

And the key must come from the client at the start of the checkout, not be generated per request, because a per-request key differs on the retry and deduplicates nothing.

Then the order lifecycle, which is a saga rather than a transaction.

An order spans payment, inventory, the warehouse, a courier and several emails, over hours or days. You cannot hold a database transaction across that, and **several of the steps are in other companies' systems.**

So: a state machine with explicit transitions, and compensating actions for failures.

```
created → payment_authorised → inventory_committed → packed  
        → shipped → delivered
```

and at each step, a specific compensation:

```
payment fails           → release the reservation  
inventory fails         → VOID the authorisation (do not capture)  
warehouse cannot fulfil → REFUND, and apologise
```

Note that the compensations are not symmetric, and that is the point. Voiding an authorisation before capture is free and invisible. Refunding after capture costs fees, takes days to appear, and the customer notices — which is why the ordering of the steps matters: authorise first, commit inventory, capture last.

And two-phase payment is the mechanism that makes that possible. Authorisation reserves the money on the card without taking it. Capture takes it. The gap between them is where you check inventory, and using it correctly is the difference between a clean cancellation and a refund.

Finally, the flash sale, which breaks every part of this.

A hundred thousand people trying to buy a thousand units in the same second. The catalogue survives — it is cached. **The inventory row does not**, because every one of those requests wants to decrement the same row, and a database row serialises.

Three things, in order of how much they help.

Admit fewer requests. A queue in front of the purchase path, or a randomised waiting room, **so the database sees a manageable rate rather than a hundred thousand simultaneous attempts.**

Decrement in memory. An atomic counter in Redis is the source of truth for the sale, **with the durable write following asynchronously.** Redis does about a hundred thousand atomic decrements a second, where a database row does a few thousand.

And shard the counter. Split a thousand units into twenty buckets of fifty, **and route each request to a random bucket** — twenty rows instead of one. **The cost is that a bucket can empty while others have stock**, so the last few units need a rebalancing pass, **which is a genuine trade rather than a free win.**

And the honest framing: at flash-sale scale, “sold out” is a better answer than “sorry, we oversold”. Rejecting requests early is correct behaviour, not a failure.

4 The picture

The four systems, and why they are separate:

CATALOGUE products, prices changes slowly CACHE EVERYWHERE millions of reads/s stale = fine	SEARCH inverted index async populated near-real-time ~100k queries/s stale = fine	INVENTORY stock counts changes fast ATOMIC ONLY ~10k writes/s stale = OVERSOLD	ORDERS the saga long-running state machine ~1k orders/s stale = wrong status shown
--	--	---	--

"the wall is for looking at; the notebook is for promising"

The read-then-write race, which is the bug:

time → request A ----- read qty = 1 qty ≥ 1? yes write qty = 0 "you bought it"	request B ----- read qty = 1 qty ≥ 1? yes write qty = 0 "you bought it"	← both see 1 ← both check, both pass ← both write ← TWO SALES, ONE ITEM
--	--	--

This needs the two requests to overlap by a few **MILLISECONDS**.
It never happens in testing and constantly happens in production.

The three correct fixes:

ATOMIC CONDITIONAL UPDATE

```
UPDATE stock
SET qty = qty - 1
WHERE sku = ? AND qty ≥ 1

rows_affected = 1 → success
rows_affected = 0 → sold out
```

the DB does check+write as ONE operation
simple, correct, serialises the row

OPTIMISTIC CONCURRENCY

```
read qty AND version
...
UPDATE stock SET qty = ?, version = v+1
WHERE sku = ? AND version = v

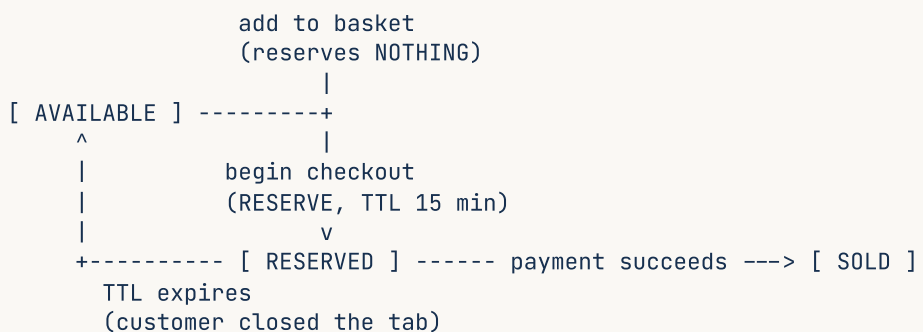
0 rows → somebody beat me → RETRY
```

good when conflicts are RARE
wasteful when they are common

PESSIMISTIC LOCK

```
SELECT ... FOR UPDATE
→ holds the row lock for the whole transaction
→ correct, and serialises EVERYTHING touching that row
→ the one to avoid on hot items
```

Reservations: three states, not two:



THE TTL IS THE POINT: nobody sends "I gave up".

A customer who closes the tab mid-payment must not hold the last item forever, and no explicit release message will ever arrive.

Two-phase payment, and why the order of steps matters:

```

AUTHORISE (reserve the money on the card, do not take it)
  |
  v
COMMIT INVENTORY ← the check happens HERE, in the gap
  |
  +-- fails → VOID the authorisation
  |           free, invisible to the customer, instant
  v
CAPTURE (take the money)
  |
  +-- warehouse cannot fulfil later → REFUND
  |           costs fees, takes DAYS to appear,
  |           the customer definitely notices

```

THE COMPENSATIONS ARE NOT SYMMETRIC.

That asymmetry is why you authorise first, check inventory second, and capture last.

The idempotency key, and where it must come from:

```

customer clicks PAY
  |
  +→ request, with key K -----> [ timeout ]
  |                                   |
  |                                   did it go through?
  |                                   NOBODY KNOWS, including the customer
  v
customer clicks PAY again
  +→ request, with the SAME key K → payment service:
                                   "I have seen K.
                                   Here is the original result."
                                   → NO SECOND CHARGE

```

THE KEY MUST BE GENERATED WHEN THE CHECKOUT STARTS, by the client.
 A key generated per REQUEST differs on the retry and deduplicates
 nothing – which is the mistake that looks like a solution.

The flash sale, and why a database row is the wall:

100,000 requests, same second, one SKU with 1,000 units

ONE DATABASE ROW:

every request wants to decrement it
the row serialises → ~3,000 updates/second
→ 33 seconds of queueing, timeouts, and a stampede of retries

FIX 1 – ADMIT FEWER: a queue or a waiting room in front
the DB sees a manageable rate

FIX 2 – DECREMENT IN MEMORY: Redis DECR is atomic, ~100,000/s
durable write follows asynchronously

FIX 3 – SHARD THE COUNTER: 1,000 units → 20 buckets of 50
route randomly → 20 rows, not 1
COST: a bucket can empty while others
have stock → the last units need a
rebalancing pass

AND: "sold out" is a BETTER answer than "sorry, we oversold".
Rejecting early is correct behaviour.

5 How it actually works

Inventory: the atomic decrement

```
def reserve_stock(sku: str, quantity: int, reservation_id: str) → bool:
    with db.transaction():
        rows = db.execute("""
            UPDATE inventory
            SET available = available - %(q)s,
              reserved = reserved + %(q)s
            WHERE sku = %(sku)s AND available ≥ %(q)s
            """, {"sku": sku, "q": quantity})
        if rows ≠ 1:
            return False # not enough – no state changed
        db.execute("""
            INSERT INTO reservations (id, sku, quantity, expires_at)
            VALUES (%(id)s, %(sku)s, %(q)s, now() + interval '15 minutes')
            """, {"id": reservation_id, "sku": sku, "q": quantity})
        return True
```

AND available ≥ quantity in the WHERE clause is the entire correctness argument. The database evaluates the condition and performs the write as one atomic operation, so two concurrent requests cannot both see enough stock.

Checking rows ≠ 1 rather than reading the new value is what makes it safe: the row count tells you whether *your* update was the one that succeeded.

And the reservation row with an expiry is Prakash's third notebook — held, not sold.

Releasing expired reservations

```
def release_expired() → int:
    """Runs every minute. The TTL is what makes reservations safe."""
    expired = db.execute("""
        DELETE FROM reservations
        WHERE expires_at < now()
        RETURNING sku, quantity
    """)
    for sku, quantity in expired:
        db.execute("""
            UPDATE inventory
            SET available = available + %(q)s,
                reserved = reserved - %(q)s
            WHERE sku = %(sku)s
        """, {"sku": sku, "q": quantity})
    return len(expired)
```

This job is not optional. Without it, every abandoned checkout permanently removes stock, **and the store sells out while the warehouse is full** — a failure that accumulates silently over weeks.

And it must be idempotent: the `DELETE ... RETURNING` makes it so, because a reservation can only be deleted once even if the job runs twice.

Checkout with an idempotency key

```
@app.post("/checkout")
def checkout(user_id: int, basket: list, idempotency_key: str) → dict:
    if existing := order_store.by_idempotency_key(idempotency_key):
        return existing # a retry: the same answer

    prices = pricing.current(basket) # re-read: the page may be stale
    reservation_id = str(uuid.uuid4())
    for line in basket:
        if not reserve_stock(line.sku, line.quantity, reservation_id):
            release_reservation(reservation_id) # all or nothing
            return {"error": "out_of_stock", "sku": line.sku}, 409

    order = order_store.create(
        user_id=user_id, lines=basket, prices=prices,
        reservation_id=reservation_id,
        idempotency_key=idempotency_key, state="created")
    return {"order_id": order.id, "total": prices.total}
```

`prices = pricing.current(basket)` is not a detail. The product page may have been cached for ten minutes, and charging a price different from the one displayed is a legal problem — so the price is re-read and any difference is shown to the customer before payment.

And the reservation is all-or-nothing across the basket: if the third line is out of stock, the first two must be released, or an abandoned checkout holds stock nobody wanted.

Payment, two-phase

```
def process_payment(order_id: int) → str:
    order = order_store.get(order_id)

    auth = payments.authorise(
        amount=order.total, method=order.payment_method,
        idempotency_key=f"auth:{order.idempotency_key}")
    if not auth.ok:
        release_reservation(order.reservation_id)
        order_store.transition(order_id, "payment_failed")
        return "payment_failed"

    order_store.transition(order_id, "payment_authorised")

    if not commit_inventory(order.reservation_id):
        payments.void(auth.id) # FREE, invisible, instant
        order_store.transition(order_id, "cancelled")
        return "out_of_stock"

    capture = payments.capture(
        auth.id, idempotency_key=f"capture:{order.idempotency_key}")
    order_store.transition(order_id, "paid" if capture.ok else "payment_failed")
    return "paid"
```

Authorise, check inventory, then capture — and the ordering is the whole design. Voiding an authorisation is free and invisible; refunding a capture costs fees and takes days. So the risky check goes in the gap between them.

And the idempotency keys are derived from the order's key, with a prefix per operation, so a retried authorisation does not deduplicate against the capture.

Committing the reservation

```
def commit_inventory(reservation_id: str) → bool:
    with db.transaction():
        rows = db.execute("""
            DELETE FROM reservations
            WHERE id = %(id)s AND expires_at > now()
            RETURNING sku, quantity
        """, {"id": reservation_id})
        if not rows:
            return False # expired while paying – real, and rare
        for sku, quantity in rows:
            db.execute("""
                UPDATE inventory
                SET reserved = reserved - %(q)s,
                    sold      = sold      + %(q)s
                WHERE sku = %(sku)s
            """, {"sku": sku, "q": quantity})
        return True
```

AND expires_at > now() handles the **genuine race**: the customer took sixteen minutes to enter their card details and the reservation expired. **It is rare and it happens**, and the code must handle it rather than assume it away.

The order saga

```
TRANSITIONS = {
    "created": {"payment_authorised", "payment_failed", "cancelled"},
    "payment_authorised": {"paid", "cancelled", "payment_failed"},
    "paid": {"packed", "refunded"},
    "packed": {"shipped", "refunded"},
    "shipped": {"delivered", "returned"},
    "delivered": {"returned"},
}

COMPENSATIONS = {
    "payment_failed": ["release_reservation"],
    "cancelled": ["release_reservation", "void_authorisation"],
    "refunded": ["refund_payment", "restock"],
}

def transition(order_id: int, to_state: str) → None:
    with db.transaction():
        order = order_store.lock_for_update(order_id)
        if to_state not in TRANSITIONS.get(order.state, set()):
            raise InvalidTransition(f"{order.state} → {to_state}")
        order_store.update(order_id, state=to_state)
        for action in COMPENSATIONS.get(to_state, []):
            outbox.enqueue(action, order_id) # same transaction as the state change
```

The **explicit table checked on every write prevents the double-charge**: a retried “paid” on an order already paid **raises rather than charging again**.

And `outbox.enqueue` inside the same transaction is the **transactional outbox pattern**. Writing the state change and publishing the event **must be atomic**, or a crash between them leaves an order marked paid with no warehouse ever told. **A separate process drains the outbox and publishes**, which is at-least-once and therefore needs the consumers to be idempotent.

The flash sale path

```
def flash_sale_purchase(sku: str, user_id: int) → bool:
    if not rate_limiter.allow(user_id, sku):
        return False # one attempt per user

    bucket = random.randrange(SHARDS) # spread across N counters
    remaining = redis.decr(f"flash:{sku}:{bucket}")
    if remaining < 0:
        redis.incr(f"flash:{sku}:{bucket}") # put it back
        return try_other_buckets(sku, user_id)

    queue.publish("flash_order", {"sku": sku, "user": user_id}) # durable, async
    return True
```

`redis.decr` is atomic and returns the new value, so checking `< 0` is the sold-out test — **no read-then-write anywhere**.

Putting it back on failure matters, because otherwise a rejected request permanently destroys a unit.

And the durable order is written asynchronously. The customer is told “yes” on the basis of the Redis decrement, **and the database write follows** — which is a deliberate choice: **the counter is the source of truth for the sale, and losing the queue message means an order that must be reconciled, not a lost unit**.

`try_other_buckets` is the **cost of sharding**. With twenty buckets, the last few units end up stranded in buckets that requests are not landing on, **so a request that finds its bucket empty checks the others before giving up**.

The real systems

PostgreSQL / MySQL	orders, inventory, payments – a relational database, because this is what transactions are for
Redis	flash-sale counters, session baskets, caching
Elasticsearch	product search with facets
Kafka	the event backbone: order events, inventory updates, the outbox drain
Stripe / Razorpay / Adyen	payments, all requiring idempotency keys and all supporting authorise/capture separately
CDN	product images and cached catalogue pages

“A relational database, because this is what transactions are for” is worth saying explicitly, since the instinct after a month of distributed systems is to reach for something scalable — and the order volume here is small enough that a single primary handles it, which is a much better answer than a distributed store with weaker guarantees.

6 The numbers

Traffic, and the split.

```
100,000,000 monthly users, ~10,000,000 daily
each views ~20 product pages/day      → 200,000,000 page views/day
                                         = ~2,300/second, peak ~10,000/second

~1% convert to an order                → ~100,000 orders/day
                                         = ~1.2/second, peak ~50/second

read : write = 2,000 : 1
```

Two thousand reads per write — the browse path is a different system from the order path, by three orders of magnitude.

Catalogue.

```
10,000,000 products x ~5 KB of metadata = 50 GB
+ images: 10,000,000 x 5 images x 200 KB = 10 TB → CDN

the metadata fits in memory on one machine.
→ the catalogue is a CACHING problem, not a storage one.
```

Search.

```
~100,000,000 searches/day = ~1,200/second, peak ~5,000/second
index: 10,000,000 products x ~50 terms x 8 bytes = 4 GB
→ comfortably in memory on a few machines
```

Orders and payments, which is the small, important half.


```
100,000 orders/day x ~8 state transitions = 800,000 writes/day
                                           = ~10/second, peak ~400/second
```

order record ~5 KB (lines, addresses, prices, payment refs)

```
100,000 x 5 KB = 500 MB/day = ~180 GB/year
```

```
+ replicas      = ~550 GB/year
```

→ ONE relational primary handles this comfortably, for years.

This is the part where correctness matters and the volume is small.

That contrast is the point: the half that must be perfect is also the half that is tiny.

Inventory writes, and where they concentrate.

ordinary trading:

```
~50 reservations/second at peak, spread over 10,000,000 SKUs
```

→ essentially zero contention per row

FLASH SALE:

```
100,000 requests in ~1 second, ONE SKU
```

```
a single Postgres row: ~3,000 conditional updates/second
```

→ 33 seconds of queueing, timeouts, retry storms

```
Redis DECR:                ~100,000/second    → 1 second
```

```
sharded 20 ways:           ~2,000,000/second → instant
```

→ the ordinary case needs nothing clever; the flash sale needs all three mitigations.

Reservation expiry.

```
~50 reservations/second created at peak
```

```
~30% abandoned → ~15/second expiring
```

```
the cleanup job runs every minute: ~900 rows per run
```

trivial – but WITHOUT it:

```
15/second x 86,400 = 1,300,000 units of stock removed per day
```

→ the store shows "sold out" while the warehouse is full

→ and it accumulates silently over weeks

A trivial job whose absence is catastrophic is worth flagging, because it is the kind of thing that gets deprioritised.

Payment costs.

100,000 orders/day, average value \$50
= \$5,000,000/day of gross merchandise value

card fees ~2.5% + \$0.30
= \$5,000,000 x 0.025 + 100,000 x 0.30 = \$155,000/day

REFUNDS cost the fee AGAIN and are not returned:
a 1% refund rate = 1,000 refunds/day x ~\$1.55 = ~\$1,550/day
= ~\$570,000/year in fees on refunded orders alone

→ which is the concrete argument for AUTHORISE, CHECK, THEN CAPTURE.
A void costs nothing; a refund costs the fee twice.

That number is the best justification available for the two-phase design, and it is more persuasive than the correctness argument.

Infrastructure, roughly:

catalogue + CDN	~\$40,000/month	(10 TB of images)
search cluster	~\$15,000/month	
application tier	~\$30,000/month	
orders/inventory database	~\$8,000/month	(small!)
Redis / caching	~\$10,000/month	

	~\$103,000/month	

against \$155M/year in payment fees.

→ INFRASTRUCTURE IS UNDER 1% OF PAYMENT FEES.
The database that must never be wrong is the cheapest thing here.

Latency budget.

product page (cached)	~50 ms	
search query	~100 ms	
add to basket	~30 ms	
checkout: reserve N lines	~50 ms	
payment authorise (external!)	~500-2,000 ms	← dominates
capture (external)	~500-2,000 ms	

→ the payment provider is the slowest thing in the system by an order of magnitude, and it is the one you do not control.
Which is why it must be async from the customer's perspective: show "processing", then confirm.

7 The trade-offs

Atomic conditional update against optimistic concurrency against pessimistic locking. The conditional update is simplest and correct, and **serialises on the row**, which is fine below a few thousand writes a second on that SKU. Optimistic concurrency wins when conflicts are rare and **wastes a round trip on every conflict**, so it is worse exactly when contention is high. Pessimistic locking is correct and **holds the lock for the whole transaction**, which on a hot item queues everything behind the slowest request in the transaction.

Reservations against decrementing at payment. Reserving at checkout means a customer who reaches the payment page will not be told “sold out” after entering their card — **which is the single worst checkout experience there is**. It costs a reservation table, an expiry job, and **stock that is held but not sold**, so the effective availability is lower than the physical stock. **Shorten the TTL and you free stock faster and break more slow checkouts.**

Strong consistency on inventory against availability. A single authoritative row is correct and is a bottleneck. **Sharded counters scale and can strand the last few units** in buckets nobody lands on. **And showing slightly stale stock on the product page is fine** — the authoritative check happens at checkout, so “only 3 left!” being wrong by one is a display issue rather than an oversell.

Authorise-then-capture against charging immediately. Two-phase costs an extra round trip and a more complex state machine, **and it makes cancellation free instead of expensive** — around half a million a year in fees at this volume. **It also has an expiry**: authorisations lapse after about a week, so an order that cannot be fulfilled quickly must be captured or re-authorised.

A relational database against something distributed. Orders are small — a hundred thousand a day, under two hundred gigabytes a year — **and they need real transactions**. A single primary with replicas is correct, boring and sufficient. **Reaching for a distributed store here trades the one guarantee you actually need for scale you do not**, and that is the most tempting mistake in this design after the month you have just had.

Saga against distributed transaction. A two-phase-commit across payment, inventory and a courier is not available — **you cannot hold a lock inside another company’s system**. So: a saga with explicit compensations, **accepting that there are brief windows where the order is inconsistent** and that some compensations are lossy. **A refund is not the inverse of a charge**; it costs money and takes days.

When would I not build this? Almost always: Shopify, WooCommerce and BigCommerce exist, and below serious volume they are cheaper than the team. Building your own is justified by unusual inventory rules, unusual pricing, or margins thin enough that platform fees matter. And payments specifically should never be built — the compliance surface alone is a reason, and every provider gives you idempotency and two-phase capture for free.

8 In the interview

How it gets asked

- *“Design Amazon.”* or *“Design an e-commerce checkout.”* — the standard prompts.
- *“There is one item left and two people click buy. What happens?”* — the central question.
- *“The payment request times out. Now what?”* — idempotency.
- *“A product goes on flash sale. What breaks?”*
- *“What happens if the warehouse cannot fulfil an order that has already been paid?”*
- *“How do you handle the order lifecycle?”*

“The first thing I would establish is that **this is two systems with a factor of two thousand between them.**

Browsing and search are a read problem — millions of page views, staleness of minutes is fine, cache it everywhere. **Checkout and orders are a write problem** — a hundred thousand orders a day, about ten writes a second, **and every one must be exactly right.**

Two thousand reads per write, and the two halves want opposite things. The read half wants caching and replicas; **the write half wants a single authoritative database with real transactions.**

And I would say that explicitly, because after a month of distributed systems the instinct is to reach for something scalable — and here the correct answer for orders is a single relational primary. **A hundred thousand orders a day is under two hundred gigabytes a year. It is a small database that must never be wrong.**

The catalogue is straightforward: products cached at every layer, images on a CDN, search in an inverted index populated asynchronously. **One decision worth naming: the price must be re-read at checkout,** because a cached page can be stale and **charging a different price from the one displayed is a legal problem, not just an annoyance.**

The hard part is inventory, and I would go there next unless you want otherwise.

The naive version — read the stock, check it, write it back — is wrong, and specifically it is wrong in a way that needs two requests inside a few milliseconds. **It never happens in testing and constantly happens in production.**

The fix is an atomic conditional update: `UPDATE stock SET qty = qty - 1 WHERE sku = ? AND qty ≥ 1`, and check the affected row count. **The database does the check and the write as one operation,** so two concurrent attempts cannot both succeed.

And on top of that: reservations. Stock has three states, not two — available, reserved, sold. **Checkout reserves with a fifteen-minute expiry; payment converts it to a sale; an abandoned checkout lets it lapse.** The TTL is essential, **because nobody sends ‘I gave up’.**

Then the order itself is a saga rather than a transaction — payment, inventory, warehouse, courier, over hours or days, with explicit compensations. **And the compensations are not symmetric**, which is what makes the ordering of the steps matter.”

The follow-ups

“One item left, two people click buy. What happens?”

“With the obvious implementation, **both of them buy it**, and I would explain exactly why before giving the fix.

The naive code reads the stock, checks it is at least one, and writes back zero. Two requests arriving within a few milliseconds **both read one, both pass the check, and both write zero.** Two sales, one item.

What makes this dangerous is that it needs the requests to overlap by milliseconds — so it never appears in testing, and it appears constantly under real load. **It is a correctness bug that only manifests when the system is busy, which is when overselling costs the most.**

Three correct fixes, and I would pick the first.

An atomic conditional update. `UPDATE inventory SET available = available - 1 WHERE sku = ? AND available ≥ 1`, then check that exactly one row was affected. **The database evaluates the condition and performs the write as a single atomic operation**, so the second request’s condition fails. **And checking the row count rather than re-reading the value is what makes it safe** — the count tells me whether *my* update was the successful one.

Optimistic concurrency — read a version number and write conditionally on it being unchanged, retrying on conflict. **Correct, and it wastes a round trip per conflict**, so it is worse exactly when contention is high.

Pessimistic locking — `SELECT ... FOR UPDATE`. Correct, and it **holds a row lock for the whole transaction**, so on a hot item everything queues behind the slowest request.

And then the design point that matters more than any of them: reservations.

Stock should have three states, not two. Available, reserved, sold. **Adding to a basket reserves nothing. Beginning checkout reserves with a fifteen-minute expiry. Payment converts the reservation to a sale. An abandoned checkout lets it expire.**

The reason is the customer experience. Without reservations, somebody enters their card details and is then told the item is gone — **which is the worst possible moment to find out.**

And the TTL is not optional, because a customer who closes the tab mid-payment sends nothing. **A background job releasing expired reservations is trivial and its absence is catastrophic** — abandoned checkouts would perman-

ently remove stock, and the store would show sold out while the warehouse was full. **That accumulates silently over weeks.**

“The payment request times out. What do you do?”

“Nobody knows whether it went through — including the customer, who will click the button again — so this has to be designed for rather than handled.

The answer is an idempotency key, and the detail that matters is where it comes from.

The client generates a key when the checkout begins, and sends the same key with every attempt. The payment service records the key alongside the result, and a repeat of that key returns the original result rather than charging again.

Every real payment provider requires this — Stripe, Razorpay, Adyen — and that is why.

The mistake that looks like a solution is generating the key per request. A retry then carries a different key and deduplicates nothing. The key must be tied to the customer’s intention, not to the network attempt.

And I would use derived keys per operation — an authorise key and a capture key, both prefixed from the order’s key — so a retried authorisation does not deduplicate against the capture.

Now the deeper version of the question: what if the timeout happened after the charge succeeded but before I recorded it? My order is in an unknown state and the money has moved.

Two things. A reconciliation job that queries the provider for any order stuck in a pending state beyond a few minutes — the provider is the source of truth for whether money moved, not my database. And the webhook the provider sends on completion, which arrives independently of my request and must be handled idempotently, because it can arrive before, after, or instead of my response.

The general shape is worth naming: I cannot achieve exactly-once over a network, so I get at-least-once plus deduplication, and the deduplication key has to survive the retry. That is the same answer as message queues and notifications — but here the cost of getting it wrong is a double charge on someone’s card, which is a different level of consequence.”

“A flash sale starts. A hundred thousand people want a thousand units. What breaks?”

“The catalogue is fine — it is cached, and a hundred thousand page views is ordinary. **What breaks is the single inventory row**, because every one of those requests wants to decrement the same row and **a database row serialises**.

A Postgres row does maybe three thousand conditional updates a second. A hundred thousand requests is thirty-three seconds of queueing, **which means timeouts, and timeouts mean retries, and retries make it worse.**

Three mitigations, in order of how much they help.

First, admit fewer requests. A queue or a randomised waiting room in front of the purchase path, so the database sees a manageable rate. **And a per-user rate limit**, because one person’s script should not be a thousand of those requests.

Second, decrement in memory. A Redis counter with atomic `DECR` — **about a hundred thousand a second against three thousand** — and the durable write follows asynchronously. **The counter becomes the source of truth for the sale**, which is a deliberate choice: losing the queue message means an order to reconcile, not a lost unit.

Third, shard the counter. Split a thousand units into twenty buckets of fifty and route each request to a random bucket. **Twenty rows instead of one.**

And I would state the cost of sharding rather than presenting it as free: buckets empty unevenly, so the last few units end up stranded in buckets nobody is landing on. A request that finds its bucket empty has to check the others, **and there is a rebalancing pass at the end.** That is a real trade.

Two more things I would raise.

Reservations should probably be disabled or very short for a flash sale. A fifteen-minute hold on a thousand units when a hundred thousand people want them means the stock is invisible for fifteen minutes and most of it comes back unsold. **Thirty seconds, or immediate purchase, is the right answer there.**

And the honest framing: ‘sold out’ is a better answer than ‘sorry, we over-sold’. Rejecting the hundred thousandth request quickly is correct behaviour, **not a failure to be engineered away** — and designing the rejection path to be fast and clear is more valuable than trying to serve everyone.”

The model answer

“Design the checkout and order system for an online retailer: ten million daily users, a hundred thousand orders a day, occasional flash sales.”

“Let me start with the split, because it determines everything and it is the mistake I most want to avoid.

Browsing is two thousand times the volume of ordering. Two hundred million page views a day against a hundred thousand orders. **The read half wants caching and replicas and tolerates minutes of staleness. The write half is small and must be exactly right.**

So the order and inventory database is a single relational primary with replicas — a hundred thousand orders a day is under two hundred gigabytes a year, and **it needs real transactions. I want to say that explicitly, because the instinct is to reach for something distributed, and here that would trade the one guarantee I actually need for scale I do not.**

Catalogue and search: cached at every layer, images on a CDN, an inverted index populated asynchronously. And the price is re-read at checkout, because the page may be ten minutes stale and charging a different price from the one displayed is a legal problem.

Inventory, which is the hard part. Three states — available, reserved, sold. **Checkout reserves with a fifteen-minute expiry** via an atomic conditional update: `WHERE sku = ? AND available ≥ ?`, checking the affected row count. **The condition and the write are one operation, so the read-then-write race cannot happen.**

A background job releases expired reservations every minute. Trivial, and **its absence is catastrophic** — abandoned checkouts would permanently remove about a million units a day and the store would show sold out with a full warehouse.

Checkout is idempotent on a client-generated key, created when the checkout begins. A retry after a timeout returns the original result. **A key generated per request would differ on the retry and deduplicate nothing,** which is the mistake that looks like a fix.

Payment is two-phase: authorise, commit inventory, then capture. The ordering is the design. **Voiding an authorisation is free and invisible; refunding a capture costs the fee again and takes days.** At a hundred thousand orders a day averaging fifty dollars, **a one percent refund rate is over half a million a year in fees alone** — which is a more persuasive argument for two-phase than the correctness one.

The order is a saga, not a transaction. It spans payment, inventory, warehouse and courier over days, and **I cannot hold a lock inside another company's system.** So: an explicit transition table checked on every write — **which is what prevents a retried “paid” from charging twice** — with compensations attached to the failure states.

And the state change and the event publication go in the same transaction, via an outbox. Otherwise a crash between them leaves an order marked paid with the warehouse never told. **A separate process drains the outbox, which is at-least-once, so every consumer must be idempotent.**

Flash sales break the one part that cannot be cached: the inventory row. A hundred thousand requests against a row that does three thousand updates a second is thirty-three seconds of queueing and a retry storm. **Three fixes: admit fewer requests with a queue and a per-user limit; decrement in Redis, which does a hundred thousand a second, with the durable write following; and shard the counter twenty ways. Sharding strands the last few units in empty buckets, so it needs a fallback and a rebalance** — a real cost, not a free win.

Two things I would flag.

The payment provider is the slowest component by an order of magnitude — half a second to two seconds, against fifty milliseconds for everything else — **and it is the one I do not control.** So payment must be asynchronous from the customer's view: show “processing”, confirm on the webhook. **And the webhook must be idempotent, because it can arrive before, after, or instead of my own response.**

And infrastructure is under one percent of payment fees here — about a hundred thousand a month against a hundred and fifty-five million a year in card charges. **The database that must never be wrong is the cheapest thing in the system, which is worth remembering when someone proposes optimising it.”**

9 Recall card

Two systems with 2,000:1 between them: browse/search is a cacheable read problem tolerating minutes of staleness; **checkout/orders is small (~100k/day, <200 GB/year) and must be exactly right.** Orders belong in a **single relational primary** — reaching for a distributed store trades the one guarantee you need for scale you do not.

The read-then-write race is the central bug: two requests both read 1, both pass the check, both write 0. **It needs a few milliseconds of overlap — never in testing, constantly in production.** Fix with an **atomic conditional update** (`WHERE sku = ? AND available ≥ ?` , then check the **affected row count**, not a re-read). Optimistic concurrency wastes a round trip per conflict; pessimistic locking serialises everything on the row.

Stock has THREE states: available / reserved / sold. Basket reserves nothing; checkout reserves with a TTL; payment converts it; abandonment lets it lapse — **because nobody sends “I gave up”**. The expiry job is trivial and its absence is catastrophic: the store shows sold out with a full warehouse.

Idempotency key generated by the CLIENT when checkout begins, not per request — a per-request key differs on the retry and deduplicates nothing. **The provider is the source of truth for whether money moved**, so add a reconciliation job and an idempotent webhook handler.

Two-phase payment, and the ordering is the design: authorise → check inventory → capture. A void is free and invisible; a refund costs the fee again and takes days — ~\$570k/year at 1% refunds, which beats the correctness argument for persuasiveness.

The order is a SAGA with an explicit transition table (which is what stops a retried “paid” charging twice) and **compensations that are not symmetric**. **State change and event publication go in one transaction via an outbox.** **Flash sales break the single inventory row** (~3,000 updates/s): admit fewer, decrement in Redis (~100,000/s), shard the counter — **and sharding strands the last units, which is a real cost.** “Sold out” is a better answer than “we oversold”.

PRACTICE

Practice — Dynamic programming revision and mock round

DSA topic: Dynamic programming revision and mock round **System design topic:** Design an e-commerce system

Code these, in this order

Today's rule is different: **use a clock, and score yourself honestly**. Twenty-five minutes each, no notes, no looking anything up. **Write down minutes_to_state for every one of them** — that number is what this whole day is measuring, and it predicts everything else.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	House Robber III	LeetCode 337 (Medium)	Tree DP under a clock, and the (take, skip) state.
2	Extra Characters in a String	LeetCode 2707 (Medium)	1-D DP, and noticing the hidden cubic cost.
3	Partition Array for Maximum Sum	LeetCode 1043 (Medium)	The last-group decomposition.
4	Best Time to Buy and Sell Stock IV	LeetCode 188 (Hard)	Three dimensions, and the $k \geq n/2$ shortcut.
5	Burst Balloons	LeetCode 312 (Hard)	Interval DP, and the last-not-first inversion.
6	Minimum Cost to Cut a Stick	LeetCode 1547 (Hard)	Interval DP you have not seen, from the shape alone.

On all six, run the clock and record five numbers

For each: **minutes to a stated state, minutes total, whether you gave the complexity unprompted, whether you handled the edge cases, whether it ran first time**. Score them with the rubric.

Then look at the pattern rather than the average. Three of the six slow to the state means something different from three of the six failing to run.

On problem 2, find the hidden cost

After solving it, work out the true complexity including the string slice. **Say whether it is $O(n^2)$ or $O(n^3)$** , and what constraint makes it fine. Then implement the trie version and **note which direction the trie has to store the words in**, and why.

On problem 4, check the shortcut before writing

Read the constraints. Compute the table size for the largest k . **Say what fails and at what point**. Then write the shortcut condition and justify the $n/2$.

On problem 6, use the shape alone

You have not seen this problem. **Do not look it up**. Work through the diagnostic: signals, test, shape, state, count. **Note the moment you recognise it as interval DP**, and what gave it away.

Then find the padding trick — **the cuts array needs sentinels at both ends** — and say why, in one sentence.

Then the fifteen-bugs drill

Take one solved problem and **deliberately introduce each of the fifteen silent bugs**, one at a time. For each, record: what the answer becomes, and whether any test you would naturally write would catch it.

Count how many your natural tests miss. That number is why the random-verification habit exists.

Then the verification drill

For two of the six problems, write a brute-force version and check the DP against it on three hundred random inputs. **Time how long writing the brute force took**. Compare that with how long you spent debugging.

Then the recall drill

Close everything. Out loud, from memory:

1. The eight shapes with their states.
2. The thirteen recurrences.
3. The three non-obvious fill orders.
4. The five space collapses.

Record which ones you could not produce. Those are your revision list, and there will be fewer than you expect.

The compression drill

1. State DP in one sentence.
2. Give the five-step recognition procedure.
3. Give the eight shapes and one example each.
4. Give the constraint-to-complexity table.

The recurrence drill

Write each from memory, with its loop direction and base case:

1. 0/1 knapsack, one row.
2. Unbounded knapsack, one row.
3. Coin change, minimum.
4. Coin change, count combinations.
5. Longest increasing subsequence.
6. Longest common subsequence.
7. Edit distance.
8. Grid paths, one row.
9. Palindrome table.
10. Interval DP skeleton.
11. Stock, two states.
12. Tree DP skeleton.
13. TSP.

The silent-bugs drill

Give all fifteen from memory. For each, say what the wrong answer looks like — **too big, too small, or a different correct answer to a different question.**

The rubric drill

1. Give the seven scored items in weight order.
2. Say which two surprise people, and why.
3. Give the forty-five-minute budget.
4. Say what the first ten minutes buy.
5. Say what a candidate who codes at minute four loses.

The follow-ups drill

For each, give a two-sentence answer:

1. What is the complexity?
2. Can you reduce the space?
3. Can you do better?
4. What is your state, and why is it enough?
5. What happens on an empty input?

The self-diagnosis drill

After scoring the six problems:

1. Say what your average was.
 2. Say which of the five rubric items you lost most points on.
 3. Name three specific gaps, not “revise DP”.
 4. Estimate the revision time for each. (It should be twenty minutes, not a week.)
-

The split drill

1. Give both halves with volumes and requirements.
2. Give the ratio and say what it implies.
3. Say what store each half wants, and why the instinct is wrong for one of them.

The inventory drill

1. Describe the read-then-write race, step by step.
2. Say why it never appears in testing.
3. Give all three correct fixes and when each is right.
4. Say why you check the row count and not the new value.
5. Describe the three stock states.
6. Say what the TTL prevents and what message never arrives.
7. Compute what the missing expiry job costs per day.

The payment drill

1. Say where the idempotency key comes from and what the near-miss mistake is.
2. Give the two-phase sequence and why that order.
3. Say what a void costs and what a refund costs.
4. Compute the annual fee difference at 1% refunds.
5. Say what happens when the timeout came after the charge succeeded.
6. Name the two mechanisms that resolve that.

The saga drill

1. Say why a distributed transaction is not available.
2. Give the state machine and three compensations.
3. Say why the compensations are not symmetric.
4. Say what the transition table prevents.
5. Describe the outbox and what it prevents.
6. Say what it implies about consumers.

The flash-sale drill

1. Say what breaks and what does not.
2. Give the row throughput and compute the queueing time.
3. Give the three mitigations in order.
4. Give the cost of sharding, precisely.
5. Say what you would change about reservations.
6. Say what the honest framing of “sold out” is.

The sizing drill

1. Compute the read-to-write ratio.
 2. Compute the order storage per year.
 3. Compute the payment fees per day.
 4. Compare infrastructure with payment fees.
 5. Give the latency budget and name the dominating component.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Walk me through how you approach any DP problem.* The five-step recognition, the state as a sentence, the completeness check, the count, and then the coding order — with the time budget.
2. *What are the mistakes that produce wrong answers with no error?* As many of the fifteen as you can, grouped by what the wrong answer looks like, and the habit that catches all of them.

3. *Design an e-commerce checkout.* The 2,000:1 split, the read-then-write race and its fix, the three stock states with the TTL, two-phase payment with the void-versus-refund argument, and the saga with its outbox.

Before you move on

- ☐ ☐ I timed all six problems and recorded `minutes_to_state`.
- ☐ ☐ I scored myself with the rubric, honestly.
- ☐ ☐ I named three specific gaps, not “revise DP”.
- ☐ ☐ I can state DP in one sentence.
- ☐ ☐ I can give the five-step recognition procedure.
- ☐ ☐ I can give the eight shapes and their states.
- ☐ ☐ I can give the constraint-to-complexity table.
- ☐ ☐ I can write all thirteen recurrences from memory.
- ☐ ☐ I know the three non-obvious fill orders.
- ☐ ☐ I know the five space collapses, including the two that are “no”.
- ☐ ☐ I can list the fifteen silent bugs.
- ☐ ☐ I know which produce too-big and which too-small answers.
- ☐ ☐ I have the random-verification habit.
- ☐ ☐ I know the rubric’s seven items in weight order.
- ☐ ☐ I know why “did the code run” is item six.
- ☐ ☐ I know the forty-five-minute budget and write no code before minute thirteen.
- ☐ ☐ I say the complexity unprompted.
- ☐ ☐ I can answer all five standard follow-ups in two sentences each.
- ☐ ☐ I can give the e-commerce read-to-write ratio and what it implies.
- ☐ ☐ I can describe the read-then-write race step by step.
- ☐ ☐ I know all three correct inventory fixes and when each applies.
- ☐ ☐ I know why you check the row count, not the new value.
- ☐ ☐ I can describe the three stock states and the TTL’s job.
- ☐ ☐ I can compute what a missing expiry job costs.
- ☐ ☐ I know where the idempotency key comes from and the near-miss mistake.
- ☐ ☐ I can give the two-phase sequence and justify the order with fees.
- ☐ ☐ I know what to do when a timeout hides a successful charge.

- ☐ I can explain the saga, its compensations, and the outbox.
- ☐ I know what breaks in a flash sale and the three mitigations.
- ☐ I know the cost of sharding a counter.
- ☐ I answered all three questions above out loud.

PART XVII

Greedy and intervals

Days 164 to 170 · 7 chapters

Alongside, on the system design track:

High-level design case studies

- | | |
|---|--|
| 164 Greedy: when taking the best option now is safe | 168 Meeting rooms and the sweep line |
| 165 Proving a greedy choice, simply | 169 Jump game and reachability |
| 166 Interval scheduling | 170 Greedy and intervals revision and mock round |
| 167 Merging intervals | |

Greedy: when taking the best option now is safe

DATA STRUCTURES AND ALGORITHMS

Greedy: when taking the best option now is safe

SYSTEM DESIGN

Design a payment system

Greedy: when taking the best option now is safe

1 What this is, and why they ask it

A greedy algorithm makes the choice that looks best right now and never reconsiders it.

It is the simplest possible approach and it is usually wrong. **When it is right, it is dramatically better than the alternative** — one pass instead of a table, $O(n \log n)$ instead of $O(n^2)$, constant space instead of quadratic.

They ask it because **the interesting question is never “what is the greedy choice” — it is “why is that choice safe”**. Anybody can sort by something and take the first item. **The interview is whether you know whether that works**, and yesterday’s whole month of dynamic programming exists because usually it does not.

The other reason is that **greedy is where confident wrong answers live**. A candidate who writes a greedy solution and moves on has produced something that passes the examples and fails the hidden tests. **A candidate who says “greedy would take the largest first, and here is a five-element counter-example, so it must be DP” has demonstrated more than either the greedy or the DP solution would.**

The honest framing is that greedy is a claim, not a technique. The claim is: *the locally best choice is always part of some optimal solution*. **That claim is either true or false for a given problem, and there is no way to tell by looking.** You either construct a counter-example or you argue.

By the end of this lesson you can state the two properties greedy needs, run the counter-example search, recognise the four problems where greedy is provably right, and say precisely why it fails on the ones where it does.

2 The story

There were forty-one seats on the bus and about sixty people at the stand, and Rukhsana had been selling the tickets for six years.

And the thing she did, which nobody had taught her and which she could not have explained, was decide who got on.

Not by queue. There had been a queue for a while and it did not survive contact with the bus.

Her rule, in the early days, was simple: the ones going furthest. The full fare was worth more than the half fare, so fill the bus with people going to the end of the line and you take the most money.

And it was right, and she did it for two years, and then somebody pointed out that it was not right at all.

Because the bus stopped seven times.

Somebody going three stops got off, and their seat was sold again. So a full-fare passenger occupying the seat for the whole journey earned one fare, **and three short-distance passengers, one after another in the same seat, earned three.**

She changed the rule and it took her about a week to settle on the new one, and it was not what she expected.

The rule that worked was: whoever gets off soonest.

Not the highest fare. Not the longest journey. **Whoever frees the seat first**, because a seat that comes back early can be sold again, and a seat that comes back at the end cannot.

And here is the part that mattered. Her nephew, who thought about things, asked her how she knew it was the best rule and not just a better one.

She said something that was almost a proof.

“Take whichever way you think is best. Find the person in it who gets off latest, and put in the one who gets off soonest instead. You have not lost a seat — you have gained the room they were taking up. So you can never be worse off, and you might be better.”

Her nephew said that was the whole argument.

“It is not an argument,” she said. “It is just true. Try to make it wrong.”

3 The idea in plain English

Rukhsana’s rule is the earliest-finishing-time greedy, and her sentence about swapping the latest for the soonest **is an exchange argument** — which is the standard way of proving a greedy algorithm correct.

Start with what greedy is, precisely.

A greedy algorithm builds a solution one choice at a time, always taking the option that looks best by some local rule, and never undoing a choice.

Two things are hidden in that. The local rule — what “best” means — which is usually the result of sorting by something. And “never undoes”, which is what makes it fast and what makes it dangerous.

Now the two properties greedy needs, and both are required.

The greedy choice property: there is always an optimal solution that contains the locally best choice.

Optimal substructure: after making that choice, what remains is the same problem on a smaller input.

Optimal substructure is shared with dynamic programming. The greedy choice property is what greedy adds, and it is the one that is usually false.

And notice the exact phrasing: “there is always *an* optimal solution that contains it”, not “the optimal solution contains it”. There may be several optimal answers; greedy needs only that one of them agrees with its first choice. That distinction is what makes the exchange argument work.

Now the practical part: how to decide, in an interview, in thirty seconds.

Try to break it.

Construct small inputs where the locally best choice leads somewhere worse. Three or four elements is usually enough, and the fastest place to look is where the greedy choice is large but blocks two smaller things that together beat it.

```
coin change, coins [1, 5, 7], amount 10
  greedy: 7, then 1+1+1      → 4 coins
  optimal: 5 + 5             → 2 coins
  → the 7 leaves a remainder only 1s can fill

house robber, values [5, 10, 6]
  greedy: take 10, both neighbours forbidden → 10
  optimal: 5 + 6                → 11

0/1 knapsack by value-per-kilo, capacity 10
  items (6kg, 60), (5kg, 40), (5kg, 40)
  greedy: the 6kg (ratio 10), then nothing fits → 60
  optimal: the two 5kg items                    → 80
```

Three counter-examples, none longer than three items. If you can build one, it is DP and you stop looking.

And if you cannot build one, you owe an argument — which is tomorrow's lesson in full, and the shape of which is Rukhsana's sentence.

The exchange argument, stated generally:

Take any optimal solution. Show that you can swap the greedy choice into it — replacing whatever it used instead — without making it worse. Therefore there is an optimal solution containing the greedy choice, and by induction greedy is optimal.

Rukhsana's version: take any seating you like, find the passenger who gets off latest, replace them with the one who gets off soonest. **You have not lost a passenger and you have freed the seat earlier, so the result is at least as good.**

Now the four problems where greedy is provably right. These are worth knowing by name, because they are the ones interviewers use to test whether you reach for DP unnecessarily.

Activity selection — the maximum number of non-overlapping intervals. Sort by end time, take greedily. Rukhsana's bus. $O(n \log n)$, and the DP alternative is $O(n^2)$.

Fractional knapsack — where items can be cut. Sort by value per kilo, take greedily, cut the last one. And the contrast is the whole lesson: 0/1 knapsack, where items cannot be cut, defeats the same rule. Indivisibility is what breaks it.

Huffman coding — building the optimal prefix code. Repeatedly merge the two least frequent symbols. The exchange argument here is that the two rarest symbols can always be placed deepest.

Minimum spanning trees — Kruskal and Prim, from [day 139](#). Both are greedy and both are provably optimal, which is the most surprising entry on this list because MSTs feel like they should need more.

And the three where greedy nearly works, which are the traps.

Coin change works for canonical systems — one, two, five, ten — which is why the intuition feels so reliable. It is a property of those numbers, not of the method.

Interval scheduling by *start* time or by *shortest* interval both fail, and only *end* time works, which is the point of tomorrow's lesson.

And the scheduling problems where you must sort by a *ratio* rather than by either quantity alone — like minimising total waiting time — are greedy with a non-obvious rule, and the obvious rules are wrong.

Finally: the shape of a greedy solution, which is almost always the same.

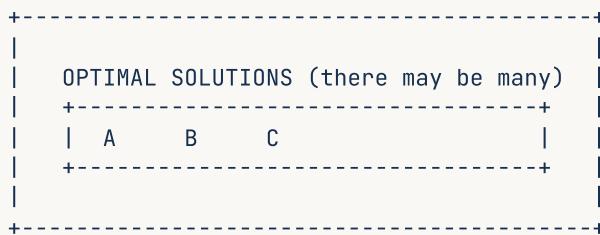
- | | |
|----------------------------------|--|
| 1. sort by the right thing | $O(n \log n)$ — usually the whole cost |
| 2. one pass, taking or skipping | $O(n)$ |
| 3. maintain one or two variables | $O(1)$ space |

The sort dominates, so greedy solutions are $O(n \log n)$ and the DP alternatives are $O(n^2)$ or worse. And “the right thing” to sort by is the entire difficulty — it is never obvious, and getting it wrong produces a plausible answer.

4 The picture

The greedy choice property, and why the phrasing matters:

ALL SOLUTIONS



GREEDY NEEDS: at least ONE of A, B, C contains the greedy choice.

NOT: "every optimal solution contains it"

NOT: "the greedy choice is in the best solution"

It only needs SOME optimal solution to agree, which is exactly what the exchange argument establishes.

Rukhsana's exchange argument, drawn:

SOME OPTIMAL SEATING

passenger P (gets off LAST)
...plus others

AFTER THE SWAP

passenger G (gets off SOONEST)
...plus the SAME others

swap P out, G in:

- still the same number of passengers (no loss)
- the seat is free EARLIER (a gain, or at worst equal)

→ the swapped version is at least as good
→ so there IS an optimal solution containing the greedy choice
→ repeat on what remains. Greedy is optimal.

"It is not an argument. It is just true. Try to make it wrong."

The three counter-examples, side by side:

COIN CHANGE, coins [1,5,7], amount 10

greedy: 7 1 1 1 4 coins

optimal: 5 5 2 coins

WHY: the biggest coin leaves a remainder that only 1s fit

HOUSE ROBBER, values [5, 10, 6]

greedy: take 10, neighbours forbidden = 10

optimal: 5 + 6 (not adjacent to each other) = 11

WHY: the biggest value blocks TWO others that together beat it

0/1 KNAPSACK by ratio, capacity 10

(6kg, 60) ratio 10 (5kg, 40) ratio 8 (5kg, 40) ratio 8

greedy: 6kg item, then nothing fits in 4kg = 60

optimal: both 5kg items = 80

WHY: INDIVISIBILITY. The same rule is PROVABLY OPTIMAL
for the fractional version.

ALL THREE ARE THREE ITEMS LONG. That is how fast this check is.

The four where greedy is provably right:

problem	sort by	why it is safe
activity selection	END time	frees the resource soonest; swap in the earliest finisher
fractional knapsack	value per kilo	you can cut, so the best ratio always fills
Huffman coding	frequency (merge the two smallest)	the two rarest can always go deepest
minimum spanning tree	edge weight (Kruskal / Prim)	the cut property: the lightest edge crossing any cut is in some MST
AND THE NEAR MISSES:		
coin change	only for CANONICAL systems (1,2,5,10...)	
interval scheduling	ONLY by end time – start time and shortest-first both FAIL	

The shape of every greedy solution:

```

sort by the right thing      O(n log n)  ← dominates
  |
  v
one pass, take or skip      O(n)
  |
  v
one or two running variables  O(1) space

```

THE HARD PART IS LINE ONE, and it is never obvious.
Getting "the right thing" wrong produces a plausible answer with no error.

Greedy against DP, as a decision:

```

can I construct a counter-example in 30 seconds?
|
+-- YES → it is DP. Stop looking. State the counter-example
|         out loud – it is the justification for the table.
|
+-- NO  → greedy MIGHT work. Now I owe an argument:
          exchange argument, or "stays ahead"
          and if I cannot produce one, I say so and
          write the DP, which is always correct.

```

THE ASYMMETRY: a counter-example is PROOF that greedy fails.
Failing to find one is NOT proof that it works.

5 The code, built step by step

Activity selection: the canonical correct greedy

```
def max_activities(intervals: list[tuple[int, int]]) → int:
    """The most non-overlapping intervals. Sort by END time."""
    if not intervals:
        return 0
    intervals.sort(key=lambda x: x[1])          # by END, not start
    count, last_end = 0, float("-inf")
    for start, end in intervals:
        if start ≥ last_end:                    # does not overlap what we have
            count += 1
            last_end = end
    return count
```

Six lines, $O(n \log n)$, and the whole difficulty is `key=lambda x: x[1]`.

Sorting by start time gives a wrong answer — a long interval starting early blocks everything. Sorting by duration also gives a wrong answer. Only end time works, and the reason is Rukhsana's: finishing earliest leaves the most room for whatever comes next.

The three wrong rules, so you can see them fail

```
def max_activities_by_start(intervals: list[tuple[int, int]]) → int:
    intervals.sort(key=lambda x: x[0])          # WRONG
    count, last_end = 0, float("-inf")
    for start, end in intervals:
        if start ≥ last_end:
            count, last_end = count + 1, end
    return count

def max_activities_by_length(intervals: list[tuple[int, int]]) → int:
    intervals.sort(key=lambda x: x[1] - x[0])   # WRONG
    chosen: list[tuple[int, int]] = []
    for start, end in intervals:
        if all(end ≤ s or start ≥ e for s, e in chosen):
            chosen.append((start, end))
    return len(chosen)
```

Writing the wrong versions and running them is worth five minutes, because the failure is not obvious from reading and is immediate from testing.

Fractional knapsack: the ratio rule, where it is correct

```
def fractional_knapsack(weights: list[int], values: list[int],
                       capacity: float) → float:
    """Items CAN be cut. Sort by value per kilo. Provably optimal."""
    order = sorted(range(len(weights)),
                   key=lambda i: values[i] / weights[i], reverse=True)
    total, room = 0.0, capacity
    for i in order:
        if weights[i] ≤ room:
            total += values[i]
            room -= weights[i]
        else:
            total += values[i] * (room / weights[i])    # take a FRACTION
            break
    return total
```

`total += values[i] * (room / weights[i])` is the line that makes greedy correct here, and it is exactly the line that does not exist in the 0/1 version.

Say the contrast out loud: the same sort rule, provably optimal when you can cut and provably wrong when you cannot. Indivisibility is the whole difference.

The counter-example finder

```
import itertools

def find_counterexample(greedy, optimal, generate, trials: int = 2000):
    """Thirty seconds of work that decides greedy versus DP."""
    for _ in range(trials):
        case = generate()
        g, o = greedy(*case), optimal(*case)
        if g ≠ o:
            return case, g, o
    return None
```

This is the habit worth taking away from the whole lesson. A brute force for a five-element input takes five minutes to write, and comparing against it on a couple of thousand random small inputs settles the question definitively.

And the asymmetry matters: finding a mismatch proves greedy is wrong. Finding none proves nothing — it makes greedy plausible, and you still owe an argument.

Exhaustive verification for small inputs

```
def best_activities_brute(intervals: list[tuple[int, int]]) → int:
    """Try every subset. Correct by construction, useless above n = 20."""
    n = len(intervals)
    best = 0
    for mask in range(1 << n):
        chosen = [intervals[i] for i in range(n) if mask >> i & 1]
        chosen.sort()
        if all(chosen[i][1] ≤ chosen[i + 1][0] for i in range(len(chosen) - 1)):
            best = max(best, len(chosen))
    return best
```

Exponential and obviously correct, which is exactly what a reference implementation should be. **Do not optimise it** — its only job is to be right.

Two more provably-correct greedies worth knowing

```
import heapq
from collections import Counter

def huffman_cost(text: str) → int:
    """Total encoded bits. Repeatedly merge the two least frequent."""
    frequencies = list(Counter(text).values())
    if len(frequencies) ≤ 1:
        return len(text)
    heapq.heapify(frequencies)
    total = 0
    while len(frequencies) > 1:
        a = heapq.heappop(frequencies)
        b = heapq.heappop(frequencies)
        total += a + b
        heapq.heappush(frequencies, a + b)
    return total

def min_total_waiting(durations: list[int]) → int:
    """Shortest job first minimises total waiting time. Exchange argument."""
    durations.sort()
    elapsed, total_wait = 0, 0
    for d in durations:
        total_wait += elapsed
        elapsed += d
    return total_wait
```

`total += a + b` in Huffman is subtle and worth a comment: the sum of all merge costs equals the total encoded length, because every merge adds one bit to every symbol beneath it.

And shortest-job-first has a one-line exchange argument: swapping any adjacent pair out of sorted order strictly increases the total wait, so sorted order is optimal.

Where greedy fails: the same rules, defeated

```
def coin_change_greedy(coins: list[int], amount: int) → int:
    count, left = 0, amount
    for coin in sorted(coins, reverse=True):
        take = left // coin
        count += take
        left -= take * coin
    return count if left == 0 else -1

def coin_change_dp(coins: list[int], amount: int) → int:
    dp = [float("inf")] * (amount + 1)
    dp[0] = 0
    for coin in coins:
        for t in range(coin, amount + 1):
            dp[t] = min(dp[t], dp[t - coin] + 1)
    return int(dp[amount]) if dp[amount] != float("inf") else -1

def knapsack_greedy_by_ratio(weights, values, capacity):
    order = sorted(range(len(weights)),
                    key=lambda i: values[i] / weights[i], reverse=True)
    total, room = 0, capacity
    for i in order:
        if weights[i] ≤ room:                # NO fractions allowed
            total += values[i]
            room -= weights[i]
    return total

def knapsack_dp(weights, values, capacity):
    dp = [0] * (capacity + 1)
    for w, v in zip(weights, values):
        for c in range(capacity, w - 1, -1):  # backwards: each item once
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[capacity]
```

Putting the greedy and the DP side by side and running both is the point of this section. The greedy is shorter and faster and wrong, and seeing the two numbers differ on a three-item input is more convincing than any explanation.

The complete solution

```
"""Greedy: the four that work, the three that do not, and how to tell."""

import heapq
import random
from collections import Counter

# ----- provably correct -----

def max_activities(intervals: list[tuple[int, int]]) → int:
    """Sort by END time. The canonical correct greedy."""
    if not intervals:
        return 0
    ordered = sorted(intervals, key=lambda x: x[1])
    count, last_end = 0, float("-inf")
    for start, end in ordered:
        if start ≥ last_end:
            count += 1
            last_end = end
    return count

def fractional_knapsack(weights, values, capacity: float) → float:
    """Items CAN be cut → the ratio rule is provably optimal."""
    order = sorted(range(len(weights)),
                    key=lambda i: values[i] / weights[i], reverse=True)
    total, room = 0.0, float(capacity)
    for i in order:
        if weights[i] ≤ room:
            total += values[i]
            room -= weights[i]
        else:
            total += values[i] * (room / weights[i])
            break
    return total

def huffman_cost(text: str) → int:
    """Merge the two least frequent, repeatedly. The merge costs sum to the answer."""
    frequencies = list(Counter(text).values())
    if len(frequencies) ≤ 1:
        return len(text)
    heapq.heapify(frequencies)
    total = 0
    while len(frequencies) > 1:
        a, b = heapq.heappop(frequencies), heapq.heappop(frequencies)
        total += a + b
        heapq.heappush(frequencies, a + b)
    return total

def min_total_waiting(durations: list[int]) → int:
    """Shortest job first. Swapping any out-of-order pair strictly hurts."""
    elapsed, total_wait = 0, 0
    for d in sorted(durations):
        total_wait += elapsed
        elapsed += d
    return total_wait

# ----- the wrong rules, for the same problem -----
```

```

def max_activities_by_start(intervals) → int:
    ordered = sorted(intervals, key=lambda x: x[0])
    count, last_end = 0, float("-inf")
    for start, end in ordered:
        if start ≥ last_end:
            count, last_end = count + 1, end
    return count

def max_activities_by_length(intervals) → int:
    chosen: list[tuple[int, int]] = []
    for start, end in sorted(intervals, key=lambda x: x[1] - x[0]):
        if all(end ≤ s or start ≥ e for s, e in chosen):
            chosen.append((start, end))
    return len(chosen)

def best_activities_brute(intervals) → int:
    """Exponential and obviously correct. Do not optimise it."""
    n = len(intervals)
    best = 0
    for mask in range(1 << n):
        chosen = sorted(intervals[i] for i in range(n) if mask >> i & 1)
        if all(chosen[i][1] ≤ chosen[i + 1][0] for i in range(len(chosen) - 1)):
            best = max(best, len(chosen))
    return best

# ----- where greedy fails -----

def coin_change_greedy(coins: list[int], amount: int) → int:
    count, left = 0, amount
    for coin in sorted(coins, reverse=True):
        take = left // coin
        count += take
        left -= take * coin
    return count if left == 0 else -1

def coin_change_dp(coins: list[int], amount: int) → int:
    dp = [float("inf")] * (amount + 1)
    dp[0] = 0
    for coin in coins:
        for t in range(coin, amount + 1):
            dp[t] = min(dp[t], dp[t - coin] + 1)
    return int(dp[amount]) if dp[amount] ≠ float("inf") else -1

def knapsack_greedy_by_ratio(weights, values, capacity: int) → int:
    order = sorted(range(len(weights)),
                    key=lambda i: values[i] / weights[i], reverse=True)
    total, room = 0, capacity
    for i in order:
        if weights[i] ≤ room:
            total += values[i]
            room -= weights[i]
    return total

def knapsack_dp(weights, values, capacity: int) → int:
    dp = [0] * (capacity + 1)
    for w, v in zip(weights, values):
        for c in range(capacity, w - 1, -1):

```

```

        dp[c] = max(dp[c], dp[c - w] + v)
    return dp[capacity]

def rob_greedy(values: list[int]) → int:
    """Take the largest, forbid its neighbours, repeat."""
    remaining = list(values)
    total = 0
    while any(v is not None for v in remaining):
        best = max((i for i, v in enumerate(remaining) if v is not None),
                    key=lambda i: remaining[i])
        total += remaining[best]
        for j in (best - 1, best, best + 1):
            if 0 ≤ j < len(remaining):
                remaining[j] = None
    return total

def rob_dp(values: list[int]) → int:
    take, skip = 0, 0
    for v in values:
        take, skip = skip + v, max(take, skip)
    return max(take, skip)

# ----- the decision procedure -----

def find_counterexample(greedy, optimal, generate, trials: int = 3000):
    """Finding one PROVES greedy wrong. Finding none proves nothing."""
    for _ in range(trials):
        case = generate()
        g, o = greedy(*case), optimal(*case)
        if g ≠ o:
            return case, g, o
    return None

if __name__ == "__main__":
    random.seed(0) # deterministic output
    print("PROVABLY CORRECT GREEDY")
    activities = [(1, 4), (3, 5), (0, 6), (5, 7), (3, 9), (5, 9),
                  (6, 10), (8, 11), (8, 12), (2, 14), (12, 16)]
    print(" activities, by end      :", max_activities(activities))
    print(" activities, by start   :", max_activities_by_start(activities))
    print(" activities, by length:", max_activities_by_length(activities))
    print(" brute force says      :", best_activities_brute(activities))

    print(" fractional knapsack :",
          fractional_knapsack([10, 20, 30], [60, 100, 120], 50))
    print(" huffman 'aaabbc'      :", huffman_cost("aaabbc"), "bits")
    print(" waiting [3,1,2]       :", min_total_waiting([3, 1, 2]))

    print("\nWHERE GREEDY FAILS – same input, both algorithms")
    print(" coins [1,5,7] → 10 : greedy", coin_change_greedy([1, 5, 7], 10),
          " dp", coin_change_dp([1, 5, 7], 10))
    print(" knapsack by ratio : greedy",
          knapsack_greedy_by_ratio([6, 5, 5], [60, 40, 40], 10),
          " dp", knapsack_dp([6, 5, 5], [60, 40, 40], 10))
    print(" robber [5,10,6]      : greedy", rob_greedy([5, 10, 6]),
          " dp", rob_dp([5, 10, 6]))

    print("\nTHE SEARCH – does a counter-example exist?")
    found = find_counterexample(
        max_activities, best_activities_brute,

```

```

lambda: ([lambda s: (s, s + random.randint(1, 6)))(random.randint(0, 10))
         for _ in range(random.randint(1, 8))],))
print(" activities by end time:",
      "counter-example found!" if found else "none in 3,000 trials → plausible")

found = find_counterexample(
    rob_greedy, rob_dp,
    lambda: ([random.randint(0, 20) for _ in range(random.randint(1, 6))],))
print(" house robber greedily :",
      f"BROKEN by {found[0][0]}: greedy {found[1]}, optimal {found[2]}"
      if found else "none found")

```

Run it and you get:

```

PROVABLY CORRECT GREEDY
activities, by end   : 4
activities, by start : 3
activities, by length: 4
brute force says    : 4
fractional knapsack  : 240.0
huffman 'aaabbc'    : 9 bits
waiting [3,1,2]      : 4

WHERE GREEDY FAILS – same input, both algorithms
coins [1,5,7] → 10 : greedy 4 dp 2
knapsack by ratio  : greedy 60 dp 80
robber [5,10,6]    : greedy 10 dp 11

THE SEARCH – does a counter-example exist?
activities by end time: none in 3,000 trials → plausible
house robber greedily : BROKEN by [6, 16, 20, 19, 1]: greedy 27, optimal 35

```

activities, by start : 3 against the correct **4** is the wrong sort rule, made visible — **and by-length happening to give 4 on this input is the more dangerous case**, because a rule that is wrong in general can easily be right on your test.

And the last two lines are the whole method. Three thousand random trials found no counter-example for sorting by end time — **which makes it plausible and is not a proof**. For house robber it found one immediately, **and that is a proof: greedy is wrong, so write the DP.**

6 What it costs

The shape of every greedy solution:

```

sort                0(n log n)   ← dominates
one pass            0(n)
running state       0(1)

TOTAL: 0(n log n) time, 0(1) extra space

```

Against the DP alternative, on the same problems:

problem	greedy	DP alternative
activity selection	$O(n \log n)$	$O(n^2)$ – check every pair
fractional knapsack	$O(n \log n)$	no DP needed (it is easier)
Huffman	$O(n \log n)$	no simple DP at all
MST	$O(E \log E)$	no DP formulation
shortest job first	$O(n \log n)$	$O(n!)$ by brute force

Concretely, at $n = 100,000$:

greedy: $100,000 \times 17 = 1,700,000$ operations ~ 0.3 s in Python
DP: $100,000^2 = 10,000,000,000$ ~ 3 hours

and the DP would need 10^{10} cells of memory, which does not exist.

→ when greedy is correct, it is not a constant-factor improvement.
It is the difference between solvable and not.

That gap is why identifying a correct greedy is worth the thirty seconds of checking.

The counter-example search, costed:

writing a brute force for $n \leq 12$: ~ 5 minutes
3,000 random trials at $n \leq 8$: < 1 second

what it settles: whether greedy is wrong (definitively)
what it does not settle: whether greedy is right

And the asymmetry is worth restating in cost terms:

finding a counter-example: 30 seconds, and you are DONE – write the DP
failing to find one: 30 seconds, and you now owe an ARGUMENT
which may take another 5 minutes,
or may not exist

→ so the search is always worth running first, because half the time it ends the question immediately.

Sorting cost, when the key is expensive:

sorting by a computed key – value/weight, or end - start – computes the key $n \log n$ times unless you use `key=`

Python's sort computes `key` exactly n times (a Schwartzian transform):

```
sorted(items, key=lambda x: expensive(x))    → n evaluations
sorted(items, cmp_to_key(...))              → n log n comparisons
```

→ always use `key=`, never a comparator, when the key is computed

The four correct greedies, by cost:

activity selection	sort $O(n \log n)$ + one pass	$O(n \log n)$
fractional knapsack	sort by ratio + one pass	$O(n \log n)$
Huffman	n heap operations	$O(n \log n)$
Kruskal MST	sort edges + union-find	$O(E \log E)$
Prim MST	heap over edges	$O(E \log V)$

ALL of them are dominated by a sort or a heap.
That is not a coincidence: greedy IS "process in the right order".

And the memory difference, which is often the real win:

activity selection greedy:	$O(1)$ – two variables
the DP alternative:	$O(n)$ or $O(n^2)$
0/1 knapsack DP:	$O(W)$ after the collapse
fractional knapsack greedy:	$O(1)$

→ greedy solutions are usually constant-space, which matters more at scale than the time difference does.

7 The traps

Writing greedy and not checking.

```
>>> coin_change_greedy([1, 5, 7], 10)
4
>>> coin_change_dp([1, 5, 7], 10)
2
```

Four against two, on a three-coin input. The greedy version is shorter, faster and wrong, **and it passes every test built from ordinary denominations** — which is where the intuition comes from and why it survives review.

Assuming greedy fails because it is greedy.

```
>>> max_activities([(1,4),(3,5),(0,6),(5,7),(3,9),(5,9),(6,10),(8,11),(8,12),(2,14),(12,16)])
4
>>> best_activities_brute([(1,4),(3,5),(0,6),(5,7),(3,9),(5,9),(6,10),(8,11),(8,12),(2,14),(12,16)])
4
```

Greedy is exactly right here, and writing an $O(n^2)$ DP for it is twenty wasted minutes. **The mistake goes both ways**, and after a month of dynamic programming the bias runs towards over-engineering.

Sorting by the wrong thing.

```
>>> activities = [(1,4),(3,5),(0,6),(5,7),(3,9),(5,9),(6,10),(8,11),(8,12),(2,14),(12,16)]
>>> max_activities(activities) # by end
4
>>> max_activities_by_start(activities) # by start
3
```

Three against four. The sort key is the entire algorithm, and there is no error — just a smaller answer. **And by-length gives the right answer on this input while being wrong in general**, which is worse, because your test passes.

Confusing fractional with 0/1.

```
>>> fractional_knapsack([6, 5, 5], [60, 40, 40], 10)
100.0
>>> knapsack_greedy_by_ratio([6, 5, 5], [60, 40, 40], 10)
60
>>> knapsack_dp([6, 5, 5], [60, 40, 40], 10)
80
```

Three different numbers for three different problems. The ratio rule is provably optimal for the fractional version and provably wrong for the 0/1 version, and the only difference is whether you may cut an item. **Check that first.**

Taking “no counter-example found” as proof.

```
>>> # 3,000 random trials, no mismatch
>>> # → greedy is PLAUSIBLE
>>> # → it is NOT proved
>>> # random small inputs may simply not contain the failing shape
```

The asymmetry is the whole point. One counter-example is proof of failure; a thousand successes are evidence and not proof. If you cannot construct the argument, **say so and write the DP**, which is always correct.

Greedy that is right for the count and wrong for the set.


```
>>> # "the maximum NUMBER of non-overlapping intervals" → greedy by end time
>>> # "the maximum TOTAL VALUE of non-overlapping intervals" → NOT greedy
>>> #   a single high-value long interval can beat three short ones
>>> #   → weighted interval scheduling is DP
```

Adding weights breaks it, and the two problems read almost identically. **Look for “maximum number” against “maximum value” in the statement.**

Not sorting at all.

```
>>> intervals = [(3, 5), (1, 4)]
>>> count, last_end = 0, float("-inf")
>>> for s, e in intervals:                                # unsorted!
...     if s ≥ last_end:
...         count, last_end = count + 1, e
>>> count
1
```

One instead of two. A greedy algorithm is “process in the right order” — without the sort there is no algorithm, only a loop.

Mutating the caller’s list.

```
>>> data = [(3, 5), (1, 4)]
>>> max_activities(data)    # if it used data.sort() in place
2
>>> data
[(1, 4), (3, 5)]           # reordered, silently
```

`sorted()` **rather than** `.sort()` unless you mean to reorder the input — **and in an interview**, saying “I will not mutate the input” is a small, cheap signal.

8 In the interview

How it gets asked

- “Can you solve this greedily?” — asked directly, and the answer is a counter-example or an argument.
- “Why does sorting by end time work?” — the exchange argument.
- “Why not sort by start time, or by duration?” — the counter-examples.
- “Is this greedy or DP?” — the diagnostic.
- “Prove your greedy choice is safe.” — tomorrow’s lesson, and the shape is worth having today.

- “What if the intervals have values?” — the trap that turns greedy into DP.

The first ninety seconds

“Before I choose an approach I want to settle whether greedy works, **because if it does the solution is one sort and one pass, and if it does not I need a table.**

The greedy candidate here is: sort by something and take items in order, never reconsidering.

And greedy needs two properties, both of them. Optimal substructure — after making a choice, what remains is the same problem on a smaller input — **which it shares with dynamic programming.** And the **greedy choice property**: there is always *an* optimal solution that contains the locally best choice. **That second one is what greedy adds, and it is the one that is usually false.**

So my first move is to try to break it, and I give myself about thirty seconds.

I look for an input where the locally best choice blocks two smaller things that together beat it. That is where counter-examples live, and they are usually three or four elements long.

For instance, on house robber: values five, ten, six. Greedy takes the ten, which forbids both neighbours, giving ten. **Taking the five and the six — which are not adjacent — gives eleven. Broken, in three elements, so it is DP.**

If I find one, I stop and write the table, and I say the counter-example out loud — it is the justification for the DP, and it is a better answer than the DP alone.

If I cannot find one, greedy is plausible and I owe an argument, which is usually an exchange argument: take any optimal solution, swap the greedy choice into it, show it does not get worse. **Therefore some optimal solution contains the greedy choice, and induction does the rest.**

And I would be explicit about the asymmetry. A counter-example is a proof of failure. **Failing to find one is evidence, not proof** — so if I cannot construct the argument either, **I say that and write the DP,** which is always correct.

Looking at this problem specifically: it asks for the maximum *number* of non-overlapping intervals, not the maximum value. **That word matters** — with values it is dynamic programming, and without them I believe greedy by end time is correct, and I can argue why.”

The follow-ups

“Why does sorting by end time work, and not by start time or duration?”

“The exchange argument, and I would give it as the intuition first and then the swap.

The intuition: choosing the interval that finishes earliest leaves the most room for everything after it. Nothing else about an interval matters for what comes next — **only when the resource becomes free again.**

The formal version is a swap. Take any optimal selection. Look at its first interval. **If it is not the globally earliest-finishing interval, replace it with that one.** The replacement finishes no later, so it cannot conflict with anything the original selection contained after it. **So the swapped selection is still valid and has the same size — it is still optimal.**

Therefore there is an optimal solution containing the greedy choice, and repeating the argument on what remains gives the result by induction.

Now the wrong rules, and I would show rather than assert.

Sorting by start time fails immediately. One interval from zero to ten, and three short ones inside it. **The earliest-starting is the long one, it is taken, and it blocks all three** — greedy gets one, the answer is three.

Sorting by duration is the more interesting failure, because it feels right. Take a short interval in the middle that overlaps two longer ones which do not overlap each other. **Greedy takes the short one and blocks both** — one instead of two.

And the reason both fail is the same: neither start time nor duration tells you **when the resource becomes free**, which is the only thing that constrains the future.

One warning I would add: the wrong rules often give the right answer. On a typical test input, sorting by duration frequently matches. **So the fact that your solution passes the examples tells you almost nothing** — which is why the argument matters more than the test.”

“What if the intervals have values, and you want the maximum total value?”

“Then greedy fails, and it is worth showing precisely how, because the two problems read almost identically.

With unweighted intervals, every interval is worth one, so more intervals is always better — and finishing early always leaves more room. **The greedy choice is safe.**

With values, that stops being true. One interval worth a hundred can beat three worth ten each, and no local rule sees that. Sorting by end time takes the three; sorting by value takes the hundred and might block four worth thirty each.

Concretely: an interval from zero to ten worth a hundred, and three from zero to three, three to six, six to nine, each worth ten. Greedy by end time takes the three short ones for thirty. **The optimal answer is the long one, for a hundred.**

So it becomes weighted interval scheduling, which is dynamic programming.

The state: sort by end time, and $dp[i]$ is the best total value using the first i intervals. For each interval, either skip it — $dp[i-1]$ — or take it, in which case add its value to $dp[j]$, where j is the last interval that finishes before this one starts.

And finding j is a binary search over the sorted end times, which keeps it $O(n \log n)$ rather than $O(n^2)$.

Which is a nice outcome: the DP is the same complexity as the greedy here, because the sort dominates both. **So the cost of the greedy failing is a table and some code, not an asymptotic penalty.**

The general lesson I would draw: ‘maximum number’ and ‘maximum value’ are one word apart in the problem statement and are different algorithms. I read for that word specifically, **because the greedy solution passes the unweighted examples and silently fails the weighted ones.**“

“How do you decide, in general, whether a problem is greedy?”

“I run a specific procedure, and it takes about thirty seconds, and the asymmetry in it is the important part.

First, I try to break it. I construct small inputs — three or four elements — where the locally best choice leads somewhere worse. **The place to look is where the greedy choice is large and blocks two smaller things that together beat it,** which is the shape of almost every counter-example.

Coins one, five and seven, amount ten: greedy takes seven and needs three ones — four coins — and the answer is five plus five. House robber with five, ten, six: **greedy takes ten and blocks both neighbours; five plus six is eleven.** Knapsack by value per kilo with a six-kilo item and two five-kilo items in a ten-kilo bag: **greedy takes the best ratio and then nothing fits.**

All three are three items long. This check is fast.

If I find one, I am done: it is DP, and I say the counter-example out loud, because it is the justification for the table and it demonstrates more than the table does.

If I cannot find one, greedy is plausible and I owe an argument. The usual one is an exchange argument — **take any optimal solution, swap in the greedy choice, show it does not get worse** — and there is a variant, “greedy stays ahead”, where you show greedy’s partial solution is at least as good as any other’s at every step.

And here is the asymmetry, which I would state explicitly. A counter-example is a proof that greedy fails. Failing to find one is not a proof that it works. Random small inputs may simply not contain the failing shape.

So if I cannot construct the argument, I say so and write the DP. The DP is always correct, and **the cost of unnecessary DP is some code, while the cost of unjustified greedy is a wrong answer.** Those are not symmetric either.

One practical habit: I would write a brute force for small inputs and check the greedy against it on a few thousand random cases. Five minutes, and it settles the question in one direction definitively.”

The model answer

“You run a single meeting room. You are given a list of requested meetings, each with a start and end time. Schedule as many as possible.”

“This is activity selection, and I want to establish whether greedy works before writing anything, **because if it does this is six lines and if it does not it is a table.**

One clarification first: as many *meetings* as possible, or as much *value* as possible? If the meetings have priorities or lengths that matter, this is a different problem. **Taking it as: maximise the count, all meetings equal.** And I would confirm whether a meeting ending at three and another starting at three conflict — **I will assume they do not, and say so, because it changes one comparison.**

The greedy candidate is: sort by something and take meetings in order, skipping any that conflict with what I have already taken.

The question is what to sort by, and there are three plausible answers.

By start time, which fails immediately: a meeting from nine to five is the earliest to start and blocks the whole day. **Greedy gets one; the answer might be eight.**

By duration, which feels right and is wrong. **A short meeting in the middle of the day can overlap two longer ones that do not overlap each other — greedy takes the short one and gets one instead of two.**

By end time, which is correct, and I can argue it.

The intuition: the only thing about a meeting that constrains the future is when the room becomes free again. Not when it starts, not how long it is. **So take the one that frees the room soonest.**

The exchange argument, properly. Take any optimal schedule. **If its first meeting is not the earliest-finishing one, swap that in.** The replacement finishes no later, so it cannot conflict with anything that came after in the original schedule. **Same number of meetings, still valid, still optimal.** So some optimal schedule contains my greedy choice, **and the same argument applies to what remains.**

The algorithm: sort by end time, keep a running `last_end`, and take any meeting whose start is at least `last_end`. Six lines.

Cost: $O(n \log n)$ for the sort, $O(n)$ for the pass, $O(1)$ extra space. The sort dominates. **The DP alternative is $O(n^2)$, so at a hundred thousand meetings greedy is a fraction of a second and the DP is hours — this is not a constant-factor difference.**

Two edge cases I would handle: an empty list gives zero, and meetings that touch at an endpoint — which is the clarification I asked about, and it is a $\geq$ rather than a $>$.

And two follow-ups I would raise myself, because they are where this problem usually goes.

If the meetings have values, greedy breaks. One meeting worth a hundred beats three worth ten, and no local rule sees it. That becomes weighted interval scheduling: sort by end time, and $dp[i]$ is the best value using the first i , with a binary search to find the last compatible meeting. Still $O(n \log n)$, so the cost of greedy failing here is a table, not a worse complexity.

And if there are several rooms, the question changes entirely — “how many rooms do I need” is a sweep-line problem, counting the maximum number of simultaneous meetings, which is a different algorithm with a different correctness argument. I would want to know which of the three questions is being asked before committing.”

9 Recall card

Greedy is a CLAIM, not a technique: the locally best choice is always part of *some* optimal solution. Two properties are needed — optimal substructure (shared with DP) and the greedy choice property (what greedy adds, and what is usually false). Note the phrasing: *some* optimal solution, not *the* one.

The decision procedure is thirty seconds: TRY TO BREAK IT. Look where the greedy choice is large and blocks two smaller things that together beat it. Coins $[1, 5, 7]$ to 10: greedy 4, optimal 2. House robber $[5, 10, 6]$: greedy 10, optimal 11. 0/1 knapsack by ratio, cap 10, $(6, 60), (5, 40), (5, 40)$: greedy 60, optimal 80. All three are three items long.

The asymmetry is the whole point: a counter-example PROVES greedy fails; failing to find one proves nothing. If you cannot construct the exchange argument either, say so and write the DP — unnecessary DP costs code; unjustified greedy costs a wrong answer.

The exchange argument: take any optimal solution, swap the greedy choice in, show it does not get worse. For activity selection — swap in the earliest finisher; it finishes no later, so it conflicts with nothing that followed.

Four provably-correct greedies: activity selection (sort by END time), fractional knapsack (value per kilo — and the SAME rule fails for 0/1, because indivisibility is the difference), Huffman (merge the two rarest), and MST (Kruskal/Prim). All are dominated by a sort or a heap, because greedy is “process in the right order”.

The sort key IS the algorithm. By start time or by duration both fail on activity selection, and by duration often gives the right answer on your test, which is worse. And “maximum NUMBER” versus “maximum VALUE” is one word apart and is greedy versus DP — with values, weighted interval scheduling, still $O(n \log n)$.

Design a payment system

1 What this is, and why they ask it

A payment system moves money from one party to another and **keeps a record that survives an audit.**

They ask it because **it is the only design in this course where being wrong is not recoverable by trying again.** Every other system can drop a location update, lose a view count, serve a stale tweet. **Here, a lost payment is somebody's money that is gone, and a duplicated one is a charge on a real card that somebody has to notice, dispute and reverse.**

And the specific difficulty is a mismatch that runs through the whole system. **The business requires exactly-once. The network offers at-least-once.** Every payment API call can time out with the request having succeeded, **and the caller genuinely cannot tell.** Reconciling that is not a detail — **it is the design.**

Three things carry the interview.

Idempotency, done properly. Not “we use a key”, but where the key comes from, what it is stored against, what happens on a concurrent retry, and how long it lives.

Double-entry bookkeeping, which is a five-hundred-year-old idea that solves a real engineering problem: **every transfer is two entries that must sum to zero**, so the books either balance or you know instantly that they do not. **Most candidates have never heard of it and it is the correct data model.**

And reconciliation, which is the acknowledgement that distributed systems fail and money must still add up. **You compare your records against the bank's, every day, and you find discrepancies — because you always do.**

By the end of this lesson you can design the ledger, make payments idempotent under concurrency, handle the authorise-capture-refund lifecycle, run reconciliation, and say honestly what consistency you can and cannot offer.

2 The story

The society had four hundred and twenty flats and the accounts had been kept by whoever was on the committee, which meant they had been kept eleven different ways.

And when Zubeida took over as treasurer, the books did not balance by eleven thousand rupees.

Not stolen. Nobody thought that. **Just eleven thousand rupees of the difference between what people had paid and what had been written down, accumulated over four years, in a register that recorded one thing per line.**

Received 2,400 from flat 312.

And that was all it said. Received. Not where it went.

Because the money went into the account, or it went into the petty cash tin, or it went into the man's pocket until Tuesday when he remembered to bank it. **The register recorded that money had arrived and never recorded where it had arrived.**

Zubeida's husband had been a bank clerk for thirty-one years, and when she described the problem he told her what to do in one sentence, and she thought it was pedantic, and it took her a month to understand it was not.

"Write every payment twice. Once where it came from, once where it went. And the two numbers must be the same."

Flat 312 owes us 2,400 less. The bank account has 2,400 more.

Two lines, equal and opposite, for every single thing.

Which was double the writing, and she said so, and he said the thing that settled it.

"If you write one line you can be wrong and never know. If you write two, they either match or they do not, and you find out on the same day."

And then he told her the second half, which she had not asked about.

"And every month you take the bank's own statement and go down it line by line against yours."

She said that if the two lines always matched, surely her book was right.

"Your book is right about what you wrote down," he said. "The bank's statement is right about what happened. Those are different, and the difference is always something — a cheque that has not cleared, a charge you did not know about, something entered twice. You will never once find them identical. That is not a failure. That is the point of doing it."

3 The idea in plain English

Zubeida's husband has described the two halves of every payment system: **double-entry bookkeeping, and reconciliation**. The first makes errors detectable; the second finds the ones that got in anyway.

Start with the scope, because “payment system” means three different things.

IN	accepting a payment from a customer the ledger – where money is recorded the authorise / capture / refund lifecycle idempotency and retries reconciliation with the bank
OUT	fraud detection, PCI compliance in detail, currency exchange, lending, the card networks' internals

And one thing has to be said before designing anything: you almost certainly do not touch the card. PCI DSS compliance for handling card numbers is an enormous burden, so real systems use a provider's hosted fields or tokenisation — **the card number never reaches your servers and you store a token instead**. Saying that in the first minute is a strong signal.

Now the ledger, which is the data model.

A payment is not a row that says “customer paid £50”. It is **two entries**: money leaving one account and money arriving in another, **and they must sum to zero**.

```
transfer £50 from customer C to merchant M:

entry 1:  account C   -5000   (in the smallest unit: pence)
entry 2:  account M   +5000
          -----
sum:                0       ← ALWAYS, for every transaction
```

Three properties fall out of that and they are the reason for it.

Errors are detectable. If the entries do not sum to zero, something is wrong, **and you know immediately rather than in four years**.

The balance is derived, not stored. An account's balance is the sum of its entries. **You can recompute it from scratch at any time**, which means a corrupted balance is repairable — and a stored balance that drifts is not.

And the ledger is append-only. You never update or delete an entry. A mistake is corrected by writing a compensating pair, **so the history is complete and an auditor can see what happened and when it was fixed**. A payment that was reversed shows as two transactions, not as an absence.

Two mechanical rules that are not negotiable.

Store money as integers in the smallest unit — pence, paise, cents. **Never floating point**, because `0.1 + 0.2` is not `0.3`, and a system that loses a hundredth of a penny per transaction loses real money at scale and fails its audit.

And every entry carries its currency. Adding rupees to dollars is a bug that a type system will not catch for you, so the currency is part of the amount, and cross-currency transfers are two transactions plus an explicit exchange entry.

Now idempotency, and this is where the interview usually goes.

The problem, stated exactly: the client sends a payment request. The connection times out. Did the money move?

Nobody knows. Not the client, not the user, and the user will press the button again.

The answer is an idempotency key, and four details make it actually work.

Where it comes from. The client generates it **when the user's intention forms** — when they open the checkout — **not per request**. A key generated per attempt differs on the retry and deduplicates nothing, **which is the mistake that looks like a solution**.

What it is stored against. The key maps to the **result**, not merely to “seen”. A retry must return the original response — the same payment id, the same status — **so the client sees exactly what it would have seen**.

What happens on a concurrent retry. Two requests with the same key arriving at once. A **check-then-write is a race**, so the insert of the key must be atomic — a unique constraint on the key column, or a `SET NX` — **and the loser waits for the winner's result rather than proceeding**.

And how long it lives. At least **twenty-four hours**, because that is the realistic window for a client retrying after an outage. **Storing them forever is unnecessary and storing them for five minutes is useless**.

Then the lifecycle, which is three operations and not one.

AUTHORISE	reserve the money on the card; the customer's available balance drops; nothing is transferred
CAPTURE	take it; the money actually moves
REFUND	send it back, after capture
VOID	cancel an authorisation before capture

The gap between authorise and capture is the useful part. You authorise, then verify you can fulfil, then capture. If you cannot fulfil, you void — which is free, instant and invisible to the customer.

Refunding after capture is a completely different thing. It costs the transaction fee again, takes days to appear on the customer's statement, and generates a support contact. The asymmetry is why the ordering matters, and it is worth quantifying.

And authorisations expire — typically after about seven days — so an order that cannot be fulfilled quickly must be captured or re-authorised.

Then the payment flow itself, and the part that is genuinely hard.

Your service calls the provider. The call times out. You do not know whether the charge succeeded, and you must not simply retry, because a retry without an idempotency key charges twice.

Three mechanisms together, and all three are needed.

Idempotency keys passed to the provider, so your retry is safe.

A reconciliation job that queries the provider for any payment stuck in a pending state past a few minutes. The provider is the source of truth for whether money moved — not your database.

And webhooks, which the provider sends when the payment settles. They arrive independently of your request — possibly before your own response, possibly instead of it — so the handler must be idempotent and must not assume ordering.

Then reconciliation, which is Zubeida's monthly statement.

Every day, you take the provider's or the bank's settlement file and compare it against your ledger, line by line.

And you always find discrepancies. A payment your system recorded that the bank did not, or the reverse. A fee you did not know about. A refund processed on their side that your webhook missed. Finding nothing would mean the job is broken.

The categories are worth knowing: in your books and not theirs; in theirs and not yours; amounts that differ; and timing differences where a payment falls either side of the cut-off. Only the third is alarming; the first two are usually timing, and the fourth is expected.

Finally, the consistency question, answered honestly.

A payment system needs strong consistency where money moves and can tolerate eventual consistency everywhere else.

The ledger writes are transactional in a relational database, and the volume makes that affordable — this is the argument for boring technology, and it is the right one.

But the moment the money is with an external provider, you are in a distributed system you do not control, and no amount of local transaction discipline changes that. What you get is: **exactly-once *recorded*, via idempotency and reconciliation, on top of at-least-once *attempted***. Saying that precisely is the answer.

4 The picture

Double-entry, and why it makes errors detectable:

SINGLE ENTRY (Zubeida's old register)

received 2,400 from flat 312

→ where did it go? the bank? the tin? somebody's pocket?

→ the register cannot tell you

→ an error is INVISIBLE until somebody counts the money

DOUBLE ENTRY

transaction #4471

account flat_312	-240000	(paise)
------------------	---------	---------

account bank_hdfc	+240000	
-------------------	---------	--

--	-------	--

sum	0	← ALWAYS
-----	---	----------

→ every transaction sums to zero, or it is REJECTED

→ the balance of any account = the sum of its entries

→ an error is caught the same day, by arithmetic

"If you write one line you can be wrong and never know."

The append-only rule:

WRONG: fix a mistake by editing

```
UPDATE entries SET amount = 240000 WHERE id = 4471
```

→ the history is gone

→ an auditor cannot see what happened

→ and two people editing concurrently is a lost update

RIGHT: fix a mistake by writing a compensating pair

```
#4471 flat_312 -250000 ; bank_hdfc +250000 (the error)
```

```
#4498 flat_312 +250000 ; bank_hdfc -250000 (the reversal)
```

```
#4499 flat_312 -240000 ; bank_hdfc +240000 (the correction)
```

→ three transactions, all visible

→ the balance is correct

→ the history explains itself

The idempotency race, which is the detail that matters:

TWO REQUESTS, SAME KEY, ARRIVING TOGETHER

NAIVE (check then write) – BROKEN

```
A: SELECT ... WHERE key = K → not found
```

```
B: SELECT ... WHERE key = K → not found ← both miss
```

```
A: charge the card
```

```
B: charge the card ← TWICE
```

```
A: INSERT key K
```

```
B: INSERT key K
```

CORRECT – the insert IS the lock

```
A: INSERT key K → succeeds, A proceeds
```

```
B: INSERT key K → UNIQUE VIOLATION
```

→ B waits for A's result, then returns it

the database's unique constraint does the mutual exclusion.

A check-then-write cannot, because there is a window between them.

The lifecycle, and the asymmetry that shapes it:

AUTHORISE -----> CAPTURE -----> (settled)

|
| VOID

v
free, instant,
invisible to
the customer

|
| REFUND

v
costs the fee AGAIN,
takes DAYS to appear,
the customer notices

→ put every check that can fail BETWEEN authorise and capture

AND: authorisations EXPIRE (~7 days), so an order that cannot be fulfilled quickly must be captured or re-authorised.

The timeout, which is the real problem:

your service ---- charge ----> provider

|
[charges the card]

<---- response --X (lost: timeout, crash, network)

YOU DO NOT KNOW WHETHER THE MONEY MOVED.

Retrying without a key charges twice. Not retrying may lose it.

THREE MECHANISMS, ALL THREE NEEDED:

1. IDEMPOTENCY KEY sent to the provider

→ your retry is safe: the same key returns the same result

2. RECONCILIATION JOB

→ anything pending for more than N minutes: ASK the provider

→ the provider is the source of truth, not your database

3. WEBHOOK from the provider

→ arrives independently; may come BEFORE your own response

→ so the handler must be idempotent and order-independent

Daily reconciliation, and what you expect to find:

YOUR LEDGER
4,412 transactions
£221,043.55

THE BANK'S SETTLEMENT FILE
4,409 transactions
£220,912.10

THE FOUR CATEGORIES:

in yours, not theirs	usually TIMING – after the cut-off
in theirs, not yours	sometimes a payment that silently failed
amounts DIFFER	a webhook you missed, or a chargeback
	ALARMING. A fee, a partial capture,
	or a real bug
timing differences	expected, and they resolve tomorrow

YOU WILL NEVER FIND THEM IDENTICAL.

"That is not a failure. That is the point of doing it."

5 How it actually works

The ledger schema

```
CREATE TABLE transactions (  
  id          BIGINT PRIMARY KEY,  
  created_at  TIMESTAMPTZ NOT NULL,  
  kind        TEXT NOT NULL,          -- payment, refund, fee, payout  
  reference   TEXT,                  -- the order, the dispute, ...  
  idempotency_key TEXT UNIQUE         -- one transaction per key  
);  
  
CREATE TABLE entries (  
  id          BIGSERIAL PRIMARY KEY,  
  transaction_id BIGINT NOT NULL REFERENCES transactions(id),  
  account_id   BIGINT NOT NULL,  
  amount       BIGINT NOT NULL,      -- SIGNED, in the smallest unit  
  currency     CHAR(3) NOT NULL  
);  
  
CREATE INDEX ON entries (account_id, id);
```

amount **BIGINT** in the smallest unit and never a float. $0.1 + 0.2 \neq 0.3$, and a system that rounds in the wrong direction a million times a day loses real money and fails its audit.

entries has no update or delete path — that is enforced by permissions rather than by convention, because “we agreed not to” is not a guarantee.

And `idempotency_key UNIQUE` on `transactions` is doing real work: the database enforces one transaction per key, so a concurrent duplicate fails at insert rather than after charging.

Writing a transfer

```
def transfer(from_account: int, to_account: int, amount: int,
            currency: str, idempotency_key: str, kind: str) → int:
    if amount ≤ 0:
        raise ValueError("amount must be positive")

    with db.transaction():
        try:
            tx_id = db.insert_returning_id("""
                INSERT INTO transactions (id, created_at, kind, idempotency_key)
                VALUES (%(id)s, now(), %(kind)s, %(key)s)
            """, {"id": snowflake.next_id(), "kind": kind,
                  "key": idempotency_key})
        except UniqueViolation:
            return db.query_one("""
                SELECT id FROM transactions WHERE idempotency_key = %(key)s
            """, {"key": idempotency_key}) # the retry's answer

    entries = [(from_account, -amount), (to_account, +amount)]
    assert sum(a for _, a in entries) == 0 # the invariant, checked
    for account_id, signed in entries:
        db.insert("""
            INSERT INTO entries (transaction_id, account_id, amount, currency)
            VALUES (%(tx)s, %(acct)s, %(amt)s, %(cur)s)
            """, {"tx": tx_id, "acct": account_id, "amt": signed, "cur": currency})
    return tx_id
```

The `UniqueViolation` branch is the idempotency, and it is the database doing the mutual exclusion — there is no window between checking and inserting, because they are the same operation.

`assert sum(...) == 0` looks like decoration and is not. It is the invariant that makes the whole model work, and asserting it on every write catches an entire class of bug at the point of insertion rather than during an audit.

Balances, derived rather than stored

```
def balance(account_id: int, currency: str) → int:
    return db.query_scalar("""
        SELECT COALESCE(SUM(amount), 0) FROM entries
        WHERE account_id = %(acct)s AND currency = %(cur)s
    """, {"acct": account_id, "cur": currency}) or 0

def balance_cached(account_id: int, currency: str) → int:
    """A snapshot plus the entries after it. The snapshot is a cache."""
    snap = db.query_one("""
        SELECT balance, last_entry_id FROM balance_snapshots
        WHERE account_id = %(acct)s AND currency = %(cur)s
    """, {"acct": account_id, "cur": currency})
    if snap is None:
        return balance(account_id, currency)
    delta = db.query_scalar("""
        SELECT COALESCE(SUM(amount), 0) FROM entries
        WHERE account_id = %(acct)s AND currency = %(cur)s
        AND id > %(after)s
    """, {"acct": account_id, "cur": currency, "after": snap.last_entry_id})
    return snap.balance + (delta or 0)
```

The snapshot is a cache and never the truth. The truth is always the sum of the entries, so a corrupted snapshot is repaired by recomputation rather than by investigation.

Without snapshots, summing an account with ten million entries takes seconds — so they exist for performance, and **the key property is that discarding them all loses nothing**.

The payment flow

```
def charge(order_id: int, amount: int, currency: str, token: str,
           idempotency_key: str) → dict:
    payment = payment_store.get_or_create(
        order_id=order_id, amount=amount, currency=currency,
        idempotency_key=idempotency_key, state="pending")
    if payment.state in ("succeeded", "failed"):
        return payment.as_dict()          # a retry: the same answer

    try:
        result = provider.charge(
            amount=amount, currency=currency, source=token,
            idempotency_key=f"charge:{idempotency_key}")  # THE PROVIDER'S KEY
    except (Timeout, ConnectionError):
        payment_store.mark(payment.id, "unknown")          # NOT "failed"
        return {"state": "processing", "payment_id": payment.id}

    if result.succeeded:
        transfer(payment.customer_account, payment.merchant_account,
            amount, currency, f"tx:{idempotency_key}", "payment")
        payment_store.mark(payment.id, "succeeded", provider_id=result.id)
    else:
        payment_store.mark(payment.id, "failed", reason=result.error)
    return payment_store.get(payment.id).as_dict()
```

`mark(payment.id, "unknown")` rather than `"failed"` on a timeout is the most important line here. A timeout does not mean the charge failed — it means you do not know. Marking it failed and letting the customer retry is how you charge twice, and it is the single most common mistake in payment code.

And the idempotency key passed to the provider is derived from ours with a prefix, so the charge and any later capture do not deduplicate against each other.

Resolving unknowns

```
def reconcile_pending() → None:
    """Runs every minute. The PROVIDER is the source of truth."""
    stuck = payment_store.in_state("unknown", older_than_seconds=120)
    for payment in stuck:
        result = provider.lookup(idempotency_key=f"charge:{payment.idempotency_key}")
        if result is None:
            payment_store.mark(payment.id, "failed", reason="never reached provider")
        elif result.succeeded:
            transfer(payment.customer_account, payment.merchant_account,
                payment.amount, payment.currency,
                f"tx:{payment.idempotency_key}", "payment")
            payment_store.mark(payment.id, "succeeded", provider_id=result.id)
        else:
            payment_store.mark(payment.id, "failed", reason=result.error)
```

Asking the provider is the only correct resolution. Your database recorded what you intended; the provider recorded what happened, and after a timeout only the second one is authoritative.

And the `transfer` here is safe to call even if the webhook already did it, because it carries the same idempotency key — which is why the ledger’s key matters as much as the provider’s.

The webhook handler

```
@app.post("/webhooks/provider")
def handle_webhook(payload: dict, signature: str) → tuple[str, int]:
    if not verify_signature(payload, signature):
        return "bad signature", 401          # anyone can POST to this URL

    event_id = payload["id"]
    if not redis.set(f"webhook:{event_id}", "1", nx=True, ex=7 * 86400):
        return "ok", 200                    # already processed

    if payload["type"] == "payment.succeeded":
        key = payload["metadata"]["idempotency_key"]
        payment = payment_store.by_idempotency_key(key)
        if payment and payment.state != "succeeded":
            transfer(payment.customer_account, payment.merchant_account,
                    payment.amount, payment.currency, f"tx:{key}", "payment")
            payment_store.mark(payment.id, "succeeded")
    return "ok", 200
```

Signature verification first, because the endpoint is public and anyone can post a “payment succeeded” event to it — which without verification is free money.

Deduplication on the event id, because providers deliver at-least-once and a retried webhook must not double-post to the ledger.

And the handler must tolerate arriving before your own response, which happens: the provider’s webhook can land while your `charge` call is still in flight. Both paths write the same ledger transaction with the same key, so whichever arrives first wins and the other is a no-op.

Daily reconciliation

```
def reconcile_day(settlement_file: str, date: str) → dict:
    theirs = {row.reference: row for row in parse_settlement(settlement_file)}
    ours = {tx.provider_id: tx for tx in ledger.transactions_on(date)}

    report = {"matched": 0, "missing_from_theirs": [], "missing_from_ours": [],
              "amount_mismatch": []}

    for ref, our_tx in ours.items():
        their_row = theirs.get(ref)
        if their_row is None:
            report["missing_from_theirs"].append(ref)          # usually timing
        elif their_row.amount != our_tx.amount:
            report["amount_mismatch"].append(                 # ALARMING
                (ref, our_tx.amount, their_row.amount))
        else:
            report["matched"] += 1

    for ref in theirs.keys() - ours.keys():
        report["missing_from_ours"].append(ref)                # a missed webhook?

    alert_if_needed(report)
    return report
```

`amount_mismatch` is the category to alert on loudly. The other three are usually timing or a missed webhook that the next run resolves; **an amount that differs is a fee you did not model, a partial capture, or a genuine bug.**

And the job must be expected to find things. A reconciliation that reports zero discrepancies every day is almost certainly broken, and monitoring should alert on that too.

Money as integers

```
from decimal import Decimal

def to_minor_units(amount: str, currency: str) → int:
    """'12.34' GBP → 1234. Decimal, never float."""
    exponent = CURRENCY_EXPONENT[currency]    # GBP 2, JPY 0, BHD 3
    return int(Decimal(amount).scaleb(exponent))

def format_amount(minor: int, currency: str) → str:
    exponent = CURRENCY_EXPONENT[currency]
    return str(Decimal(minor).scaleb(-exponent))
```

`CURRENCY_EXPONENT` is not always two. **Japanese yen has no subunit; Bahraini dinar has three decimal places.** Hard-coding two is a bug that only appears in some countries, which is the worst kind.

And `Decimal`, not `float`, even for the conversion — `float("12.34") * 100` is `1233.9999999999998`, and `int()` of that is `1233`.

The real systems

Stripe / Adyen / Razorpay / Braintree	the providers. All require idempotency keys, all support authorise/capture separately, all send webhooks and settlement files
PostgreSQL	the ledger — transactions are exactly what this is for, and the volume is small
Kafka	payment events out to other services (never the ledger itself)
double-entry	not a technology: an accounting practice from 1494, and still the correct data model
PCI DSS	the compliance regime for handling card data, which is why you use hosted fields and never see the number

Naming double-entry's age is worth doing, because it makes the point that this is a solved problem and the engineering job is to implement it correctly rather than to invent something.

6 The numbers

Scale, and it is small — which is the point.

```
10,000,000 payments/day
= ~116/second average, peak ~500/second

each payment: ~2 ledger entries + 1 transaction row
= ~30,000,000 ledger rows/day
= ~350 row-writes/second, peak ~1,500
```

Fifteen hundred writes a second at peak is an ordinary relational workload. **One primary with replicas handles it comfortably**, which is the argument for boring technology and is the correct answer.

Storage.

transaction row ~200 bytes
entry row ~80 bytes

per day: $10,000,000 \times (200 + 2 \times 80) = 3.6 \text{ GB/day}$
per year: 1.3 TB/year
x 3 replicas: 4 TB/year

and it is APPEND-ONLY and must be kept for SEVEN YEARS
(the usual statutory retention)
→ ~28 TB over the retention period

→ partition by month, keep recent partitions hot, archive the rest.
Small by this course's standards, and it never shrinks.

Balance computation, and why snapshots exist.

a busy merchant account: $10,000 \text{ entries/day} \times 365 = 3,650,000 \text{ entries/year}$

summing them: ~50 ms per query at that size, growing forever

with a daily snapshot: sum the snapshot plus today's entries
→ ~10,000 rows → ~1 ms

50x faster, and the key property is that the snapshot can be
DISCARDED AND RECOMPUTED – it is a cache, not the truth.

Idempotency key storage.

10,000,000 keys/day, retained 24 hours
key ~40 bytes + the stored result reference ~60 bytes = ~100 bytes

$10,000,000 \times 100 \text{ B} = 1 \text{ GB}$

→ trivial. There is no excuse for a short TTL.
24 hours is the realistic client-retry window; 5 minutes is useless.

Provider latency, which dominates everything.

ledger write (local)	~5 ms	
provider charge (external)	~500-2,000 ms	← 100-400x
webhook round trip	seconds to minutes	

→ the payment provider is the slowest component by two orders
of magnitude, and you do not control it
→ so the customer-facing flow must be ASYNCHRONOUS:
"processing", then confirm on the webhook

The void-versus-refund asymmetry, in money.

10,000,000 payments/day, average £40
card fees ~1.5% + 20p

fee per payment: $£40 \times 0.015 + £0.20 = £0.80$
daily fees: $10,000,000 \times £0.80 = £8,000,000/\text{day}$

A VOID (before capture): costs NOTHING
A REFUND (after capture): the original fee is NOT returned,
plus a refund fee of ~20p

at a 1% failure-after-capture rate:
 $100,000/\text{day} \times (£0.80 + £0.20) = £100,000/\text{DAY}$
= £36,500,000/year

→ which is the entire argument for AUTHORISE, CHECK, THEN CAPTURE.

That is the largest single number in this lesson, and it is a business argument rather than a technical one — **which makes it the more persuasive kind.**

Reconciliation.

10,000,000 transactions/day to compare
a hash-join of two 10M-row sets: seconds on one machine

typical discrepancy rate: 0.01% - 0.1%
= 1,000 - 10,000 items/day needing review

→ which is why the CATEGORIES matter: 99% are timing and resolve themselves tomorrow. The amount mismatches are the ~10/day that a human must look at.

A run reporting ZERO discrepancies means the job is broken.

Infrastructure cost, against the fees:

ledger database (primary + 2 replicas)	~\$5,000/month
application tier	~\$10,000/month
reconciliation + jobs	~\$2,000/month
key store / cache	~\$1,000/month

	~\$18,000/month

against £8,000,000/DAY in card fees.

→ infrastructure is ~0.007% of the fees.
The system that must never be wrong is essentially free to run,
and the correct engineering choice is always the safe one.

Failure budget, stated honestly:

at 10,000,000 payments/day and 99.99% correctness:

1,000 wrong payments per day

which is NOT ACCEPTABLE — each one is a person whose money moved incorrectly.

the target is not a percentage. It is:

- zero unreconciled discrepancies older than 48 hours
- every discrepancy CATEGORISED, not merely counted
- and the ledger balancing to zero, every day, exactly

7 The trade-offs

Double-entry against a simple balance column. A balance column is one row per account and one update per payment — simpler, faster, and **an error in it is undetectable and unrepairable**. Double-entry doubles the rows and makes the balance derived, **so corruption is caught by arithmetic and fixed by recomputation**. At this volume the extra rows cost nothing, **and the auditability is not optional in a regulated context**.

Append-only against updating. Never editing means the history is complete and concurrent writers cannot lose an update. **It means corrections are compensating pairs**, so a reversed payment appears as two transactions rather than as an absence — **which is more confusing to a naive reader and is what an auditor requires**.

Synchronous against asynchronous payment. Synchronous is simpler and **couple the customer's request to a provider that takes up to two seconds and sometimes times out**. Asynchronous — “processing”, then confirm on the webhook — is the right answer and **costs a whole state machine plus the reconciliation of unknowns**. There is no version of this without the unknown state.

Strong consistency in the ledger against scale. A single relational primary is correct, transactional and **a bottleneck if you ever exceed it**. At ten million payments a day you will not — **fifteen hundred writes a second at peak** — and **sharding a ledger by account breaks cross-account transactions**, which is exactly what a transfer is. **The right answer is not to shard until forced, and then to shard by a boundary that transfers do not cross**.

Idempotency key retention. Twenty-four hours costs a gigabyte and covers realistic retries. **A short window is a correctness bug that appears only during an outage** — which is precisely when clients retry hardest. **Storing them forever costs little and complicates nothing**, so this trade barely exists: err long.

Reconciliation frequency. Daily is standard and means a discrepancy can live for a day. **Hourly finds problems sooner and produces more timing-related noise,** because more payments straddle the cut-off. **The useful metric is not the frequency but the age of the oldest unresolved item.**

And the honest one: exactly-once does not exist. You get **at-least-once attempted plus deduplication on a key,** which produces exactly-once *recorded*. **Anyone claiming exactly-once delivery over a network is describing idempotency and calling it something else** — and saying so precisely is a better answer than claiming the guarantee.

When would I not build this? The payment rails themselves: never. Stripe and its peers exist, the compliance surface alone justifies them, and **card handling without PCI scope is worth more than any engineering.** **The ledger, though, you often do build** — even on top of a provider, you need your own record of what you believe happened, **because reconciling against theirs requires having something to reconcile.** And for internal balances — wallets, credits, marketplace payouts — **there is no provider to defer to and the double-entry ledger is yours.**

8 In the interview

How it gets asked

- “*Design a payment system.*” or “*Design Stripe.*” — the standard prompts.
- “*The charge request times out. What do you do?*” — the central question.
- “*How do you make sure a customer is not charged twice?*” — idempotency, in detail.
- “*How do you store money?*” — the double-entry and integers question.
- “*How do you know your numbers are right?*” — reconciliation.
- “*What consistency guarantees can you offer?*” — where honesty scores.

“Two things before I design anything.

First, I would not handle card numbers. PCI compliance for storing card data is an enormous burden, so **the card goes to the provider through hosted fields and I store a token.** That is worth establishing immediately because it removes most of the security surface.

Second, the scale here is small and that shapes everything. Ten million payments a day is about fifteen hundred writes a second at peak. **This is an ordinary relational workload, and it must be exactly right** — so the answer is a single Postgres primary with replicas, and I would resist any instinct to distribute it.

Now the data model, which is where I would spend the time: double-entry bookkeeping.

A payment is not a row saying ‘the customer paid fifty pounds’. It is two entries — money leaving one account and arriving in another — and they must sum to zero.

Three things follow. Errors become detectable: if the entries do not sum to zero, something is wrong and I know today rather than in four years. **The balance is derived, not stored** — it is the sum of the entries, so a corrupted balance is repairable by recomputation. **And the ledger is append-only:** a mistake is corrected by a compensating pair, never by an edit, so the history is complete for an auditor.

Two mechanical rules. Money is integers in the smallest unit — pence, never floats — because point one plus point two is not point three, and rounding errors at this volume are real money and a failed audit. **And every amount carries its currency,** because adding rupees to dollars is a bug no type system will catch.

Then the hard part, which is that the business needs exactly-once and the network offers at-least-once.

The charge request times out. Did the money move? Nobody knows, including the customer, who will press the button again.

Three mechanisms and all three are needed. Idempotency keys, generated by the client when the checkout begins, so a retry is safe. **A reconciliation job** that asks the provider about anything stuck — because after a timeout, the provider is

the source of truth and my database is not. **And webhooks**, which arrive independently and must be handled idempotently.

And the fourth thing, which is Zubeida's monthly statement: daily reconciliation against the bank's settlement file. You always find discrepancies. Finding none means the job is broken."

The follow-ups

"The charge request times out. What do you do?"

“The first thing is what I do *not* do: **I do not mark it failed.**

A timeout does not mean the charge failed. It means I do not know. Marking it failed and letting the customer retry is exactly how you charge somebody twice, **and it is the single most common mistake in payment code.**

So the payment goes into an `unknown` state, and the customer is shown ‘processing’ rather than either outcome.

Then three mechanisms resolve it, and all three are needed.

One: the idempotency key I sent to the provider. If I retry with the same key, **the provider returns the original result rather than charging again.** Every real provider requires this, and that is why.

And the key has to come from the client when the checkout begins, not per request — a key generated per attempt differs on the retry and deduplicates nothing. **That is the mistake that looks like a solution.**

Two: a reconciliation job that runs every minute and asks the provider about anything in `unknown` for more than a couple of minutes. **The provider is the source of truth for whether money moved.** My database records what I intended; theirs records what happened, **and after a timeout only the second is authoritative.**

Three: the webhook, which the provider sends on completion. It arrives independently of my request — **possibly before my own response returns, possibly instead of it** — so the handler must be idempotent and must not assume any ordering.

Both the webhook path and the reconciliation path write the same ledger transaction with the same idempotency key, so whichever arrives first wins and the other is a harmless no-op. **That is why the ledger’s own key matters as much as the provider’s.**

And I would verify the webhook’s signature before anything else, because the endpoint is public and **anyone can post a ‘payment succeeded’ event to it,** which without verification is free money.

The honest summary: I cannot get exactly-once over a network. What I get is **at-least-once attempted, plus deduplication on a key, which produces exactly-once recorded** — and I would state it that way rather than claim a guarantee that does not exist.”

“How do you store money, and why not just a balance column?”

“Two answers: **integers in the smallest unit**, and **double-entry rather than a balance column**.

On the representation: never floating point. $0.1 + 0.2$ is not 0.3 in binary floating point, and a system that rounds the wrong way a million times a day loses real money and fails its audit. **Store pence, paise, cents — as a signed 64-bit integer.**

And the exponent is not always two. Japanese yen has no subunit; Bahraini dinar has three decimal places. Hard-coding two decimal places is a bug that only appears in some countries, which is exactly the kind that reaches production.

Every amount also carries its currency, because adding rupees to dollars is a bug a type system will not catch for you.

Now double-entry, which is the more interesting half.

The naive model is a balance column: one row per account, updated on each payment. Simple, and it has two fatal properties. **An error in it is undetectable** — the number is just wrong and nothing contradicts it. **And it is unrepairable**, because there is no record of how it was reached.

Double-entry records every transfer as two signed entries that sum to zero. Money leaves one account and arrives in another, and the transaction is rejected if the sum is not zero.

Three properties follow. Errors are caught by arithmetic, the same day. The balance is derived — the sum of an account’s entries — so it can be recomputed at any time and a corrupted cache is repaired rather than investigated. **And the ledger is append-only**: a mistake is fixed with a compensating pair, so the history is complete.

That last one matters more than it looks. A reversed payment appears as **two transactions, not as an absence** — which is more confusing to read and is exactly what an auditor requires.

The cost is double the rows and a derived balance that needs snapshots for performance — a busy account accumulates millions of entries and summing them gets slow. **But the snapshot is a cache: you can throw all of them away and recompute, which is the property that makes the whole model safe.**

And this is a five-hundred-year-old idea, which is worth saying: it is a solved problem, and the job is to implement it correctly rather than invent something.”

“How do you know your numbers are right?”

“You do not, from inside your own system — which is the point of reconciliation, and it is the half people forget.

Double-entry catches internal errors. If the entries do not sum to zero, something is wrong. **But it cannot catch a payment that your system recorded and the bank never processed,** because both of your entries are perfectly consistent with each other and both are wrong.

So: every day, take the provider’s settlement file and compare it against the ledger, line by line.

And you will always find discrepancies. Typically a hundredth to a tenth of a percent — at ten million payments a day, **that is one to ten thousand items. A run reporting zero discrepancies almost certainly means the job is broken,** and I would alert on that too.

Four categories, and they need different responses.

In your books and not theirs — usually **timing**, a payment that fell after their cut-off, which resolves tomorrow. Occasionally a payment that silently failed.

In theirs and not yours — a webhook you missed, or a chargeback initiated on their side that you have not seen.

Amounts that differ — this is the alarming one. A fee you did not model, a partial capture, or a genuine bug. **At ten million payments a day it might be ten items, and every one needs a human.**

And timing differences around the cut-off, which are expected and self-resolving.

The useful metric is not the discrepancy count — it is the age of the oldest unresolved item. A thousand items resolving within a day is healthy; **three items that have been open for a week is an incident.**

And this is Zubeida’s husband’s point, which I think is the right framing. Her book was right about what she wrote down; the bank’s statement was right about what happened. **Those are different, the difference is always something, and finding it is the job — not a sign that something has gone wrong.**“

The model answer

“Design a payment system for a marketplace: buyers pay, the platform takes a commission, and sellers get paid out weekly. Ten million payments a day.”

“A marketplace adds one thing to a plain payment system — **the money sits with the platform between the purchase and the payout** — and that is exactly what a ledger is for, so let me build it around that.

Scale first: ten million payments a day is about fifteen hundred writes a second at peak. That is an ordinary relational workload. **One Postgres primary with replicas, and I would resist distributing it** — this is the part that must be exactly right, and it is small.

Card handling: hosted fields, tokens only. The card number never reaches my servers. The PCI surface alone justifies it.

The ledger is double-entry, and the marketplace shape makes it earn its place. A single purchase is not two entries, it is four:

Buyer’s account down by the full amount. Seller’s payable account up by the amount minus commission. Platform revenue up by the commission. And the fee to the payment provider as its own entry. All of them summing to zero, in one transaction.

That is the argument for double-entry in one example: with a balance column, the commission split is arithmetic done in application code and never checked. **With entries, it either balances or the transaction is rejected.**

Money as signed integers in the smallest unit, with the currency on every entry — and the currency exponent looked up, not assumed to be two.

The payout is then a second transaction, weekly: seller payable down, bank clearing account up. **And the seller’s balance is derived from the entries**, so ‘how much am I owed’ is a query rather than a stored number that could drift.

The payment flow is asynchronous, because the provider takes up to two seconds and sometimes times out. The customer sees ‘processing’. **On a timeout the payment goes to `unknown`, never to `failed`** — marking a timeout as failed is how you charge twice.

Three mechanisms resolve unknowns: idempotency keys generated by the client at checkout, a job that asks the provider about anything pending past two minutes, and signature-verified idempotent webhooks. **All three write the same ledger transaction with the same key, so whichever arrives first wins.**

Daily reconciliation against the settlement file, in four categories — with amount mismatches alerting loudly and the others tracked by **age of the oldest unresolved item** rather than by count.

Two things specific to a marketplace that I would raise.

Refunds after payout are the hard case. If a buyer is refunded after the seller has been paid, **the money is gone from my control**. So either payouts are delayed past the refund window, **or the seller's future payouts carry a negative balance**, which is a ledger entry and a product policy — and I would want that policy decided explicitly rather than discovered.

And the void-versus-refund asymmetry is worth real money here. A void before capture costs nothing; **a refund after capture loses the original fee and adds another**. At ten million payments averaging forty pounds, **a one percent failure-after-capture rate is around a hundred thousand pounds a day**. So every check that can fail — stock, seller validity, fraud — **goes between authorise and capture**, and that ordering is a business decision as much as a technical one.

Closing thought on scale. Infrastructure here is around eighteen thousand dollars a month against **eight million pounds a day in card fees**. **The system that must never be wrong is essentially free to run** — so whenever there is a choice between the safe option and the clever one, take the safe one, because the cost difference does not exist and the failure cost is unbounded.”

9 Recall card

The mismatch is the design: the business needs exactly-once, the network offers at-least-once. What you actually get is **at-least-once attempted + deduplication on a key = exactly-once recorded** — say it that way rather than claiming a guarantee.

Double-entry: every transfer is TWO signed entries summing to ZERO. Errors become detectable by arithmetic; **the balance is DERIVED (a snapshot is a cache you can throw away and recompute)**; and the ledger is **append-only** — corrections are compensating pairs, never edits. A five-hundred-year-old idea, and the correct data model.

Money is signed integers in the smallest unit, never floats ($0.1 + 0.2 \neq 0.3$), **and the exponent is not always 2** — JPY has none, BHD has three. Every amount carries its currency.

On a timeout, mark it unknown — **NEVER failed**. That single line is the most common bug in payment code. Resolve with three mechanisms, all needed: **client-generated idempotency keys** (per checkout, not per request), **a job that asks the**

provider (which is the source of truth after a timeout, not your database), and **signature-verified, deduplicated webhooks** that may arrive before your own response.

The idempotency insert must BE the lock — a unique constraint or `SET NX`, because check-then-write has a window and two concurrent retries both charge.

Authorise → check → capture, because a VOID is free and a REFUND costs the fee twice and takes days. At 10M payments/day averaging £40, a 1% failure-after-capture rate is **~£100,000/day**.

Reconcile daily against the settlement file, in four categories — timing, missed webhooks, **amount mismatches (the alarming one)**, and cut-off effects. **You will always find discrepancies; finding none means the job is broken.** Track the **age of the oldest unresolved item**, not the count. And **infrastructure is ~0.007% of the card fees** — always take the safe option.

PRACTICE

Practice — Greedy: when taking the best option now is safe

DSA topic: Greedy: when taking the best option now is safe **System design topic:** Design a payment system

Code these, in this order

One rule for the whole set: **before writing anything, spend thirty seconds trying to break greedy**. Write down what you tried and whether you found a counter-example. **Only then choose the algorithm**. Three of these six are greedy and three are not, and the diagnosis is the exercise.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Assign Cookies	LeetCode 455 (Easy)	The simplest correct greedy, and its exchange argument.
2	Non-overlapping Intervals	LeetCode 435 (Medium)	Sort by end time, and why not by start.
3	Maximum Subarray	LeetCode 53 (Medium)	Greedy and DP are the same algorithm here.
4	Partition Labels	LeetCode 763 (Medium)	A greedy whose rule is not a sort.
5	Coin Change	LeetCode 322 (Medium)	Greedy fails — construct the counter-example first.
6	Best Time to Buy and Sell Stock IV	LeetCode 188 (Hard)	Greedy fails as soon as the limit binds.

On all six, write the diagnosis first

For each, record: **the greedy rule you would try, whether you found a counter-example in thirty seconds, and what you chose as a result**. Then solve it. **Then check whether your diagnosis was right**.

On problem 2, run all three sort keys

Implement sorting by end time, by start time and by duration. Run all three on the same input and record the three answers. **Find an input where by-duration is right and one where it is wrong**, and say why that makes it the more dangerous rule.

On problem 3, notice they coincide

Write the greedy version — running sum, reset when negative — and the DP version — $dp[i]$ = the best subarray ending at i . **Say in one sentence why they are the same algorithm**, and which one you would explain in an interview.

On problem 4, find the rule

The greedy here is not “sort by something”. Work out what the rule is before coding. **Then state the exchange argument in one sentence**: why is extending the current partition to the last occurrence always safe?

On problem 5, break it before solving it

Do not write any code until you have a written counter-example with specific coins and a specific amount. **Then say what property of the coin set makes greedy work when it does**, and name a real currency where it holds.

On problem 6, find where greedy stops working

Write the greedy for unlimited transactions — sum the positive differences. Run it with $k = 1$, $k = 2$ and unlimited on $[3, 3, 5, 0, 0, 3, 1, 4]$ and record all three. **Say what greedy is ignoring**.

Then the counter-example drill

For each, spend thirty seconds and record what you find:

1. Activity selection, sorted by end time.
2. Activity selection, sorted by start time.
3. Fractional knapsack by value per kilo.
4. 0/1 knapsack by value per kilo.
5. Huffman: merge the two least frequent.
6. Huffman: merge the two most frequent.
7. Shortest job first, to minimise total waiting.
8. Longest job first, to minimise total waiting.

Four break and four hold. For each of the four that hold, write the exchange argument in one sentence.

Then the verification drill

Take problem 2 and write a brute-force version for $n \leq 12$. Check your greedy against it on two thousand random inputs. **Time how long the brute force took to write** — it should be about five minutes — and say what it would have cost you to be

wrong instead.

The definition drill

1. Define greedy in one sentence.
2. Name the two properties it needs.
3. Say which one it shares with DP and which one it adds.
4. Give the exact phrasing of the greedy choice property, and say why “an” rather than “the” matters.

The counter-example drill

1. Say where in an input to look for a counter-example.
2. Give all three from the lesson, with their numbers.
3. Say how long each one is.
4. State the asymmetry between finding one and not finding one.
5. Say what you do when you cannot find one and cannot argue.

The exchange-argument drill

1. State it generally, in two sentences.
2. Give it for activity selection.
3. Give it for shortest job first.
4. Say what “greedy stays ahead” is, and how it differs.

The four-that-work drill

For each of activity selection, fractional knapsack, Huffman and MST:

1. What is sorted, or what is the rule?
2. What is the one-sentence reason it is safe?
3. What is the complexity, and what dominates it?

The near-misses drill

1. Say when coin change greedy works and why the intuition is misleading.
2. Give the two wrong sort keys for interval scheduling.
3. Say which of them often gives the right answer, and why that is worse.
4. Say what one word in a problem statement turns greedy into DP.

The break-it drill

For each, say what happens and whether anything reports it:

1. Greedy used where a counter-example exists.
 2. DP used where greedy is provably correct.
 3. Sorting by start time instead of end time.
 4. Sorting by duration.
 5. The ratio rule applied to 0/1 knapsack.
 6. No sort at all.
 7. Mutating the caller's list with `.sort()`.
-

The scope drill

1. Give your in and out lists.
2. Say what you establish about card handling in the first minute, and why.
3. Give the scale and say what it implies about the technology choice.

The ledger drill

1. Describe double-entry in two sentences.
2. Give the three properties that follow from it.
3. Say why the balance is derived and what a snapshot is.
4. Say why the ledger is append-only, and how a mistake is corrected.
5. Give the two rules about representing money, with the reason for each.
6. Say why the currency exponent is not always two, with examples.

The idempotency drill

1. State the problem in one sentence.
2. Say where the key comes from and what the near-miss mistake is.
3. Say what the key maps to.
4. Describe the concurrent-retry race and the fix.
5. Say what the retention should be and why.

The timeout drill

1. Say what state a timed-out payment goes into, and what it must not go into.
2. Say why that single line is the most common bug.
3. Name the three resolving mechanisms.

4. Say which one is authoritative and why.
5. Say what must be verified on the webhook, and what happens without it.
6. Say what happens if the webhook arrives before your own response.

The lifecycle drill

1. Name the four operations.
2. Say what happens in the gap between two of them.
3. Give the void-versus-refund asymmetry, with the annual figure.
4. Say what expires and what that forces.

The reconciliation drill

1. Say what double-entry cannot catch.
2. Describe the daily job.
3. Give the four categories and the response to each.
4. Say which one is alarming.
5. Say what a zero-discrepancy run means.
6. Give the metric you would actually track.

The honesty drill

1. Say what consistency you can offer, precisely.
 2. Say what “exactly-once delivery” actually means when someone claims it.
 3. Compare infrastructure cost with fees, and say what follows.
 4. Say what you would never build, and what you would build anyway.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Can you solve this greedily?* The two properties, the thirty-second counter-example search with a real example, the asymmetry, and what you do when you cannot find one.
2. *Why does sorting by end time work?* The intuition, the exchange argument as a swap, why start time and duration both fail, and the one word that turns it into DP.

3. *Design a payment system.* Card handling and scale, double-entry with its three properties, the timeout going to `unknown` with the three resolving mechanisms, and daily reconciliation with its four categories.

Before you move on

- ☐ I can define greedy and name both properties it needs.
- ☐ I know which property it shares with DP and which it adds.
- ☐ I know why “an optimal solution” rather than “the” matters.
- ☐ I spend thirty seconds trying to break greedy, every time.
- ☐ I know where in an input to look for a counter-example.
- ☐ I can give all three counter-examples with their numbers.
- ☐ I know a counter-example proves failure and none proves nothing.
- ☐ I know what to do when I can neither break it nor argue it.
- ☐ I can state the exchange argument generally.
- ☐ I can give it for activity selection and for shortest job first.
- ☐ I know the four provably-correct greedies and their sort keys.
- ☐ I know why fractional knapsack works and 0/1 does not.
- ☐ I know coin change greedy works only for canonical systems.
- ☐ I know both wrong sort keys for interval scheduling.
- ☐ I know which wrong key is more dangerous, and why.
- ☐ I know “maximum number” versus “maximum value” changes everything.
- ☐ I know greedy is `O(n log n)` dominated by the sort.
- ☐ I have the brute-force verification habit.
- ☐ I would say I do not handle card numbers, in the first minute.
- ☐ I can describe double-entry and its three properties.
- ☐ I know the balance is derived and the snapshot is a cache.
- ☐ I know the ledger is append-only and how corrections work.
- ☐ I store money as integers, and know the exponent is not always two.
- ☐ I know a timeout goes to `unknown`, never `failed`.
- ☐ I know where the idempotency key comes from and the near-miss mistake.
- ☐ I know the insert must be the lock, not a check-then-write.
- ☐ I can name the three mechanisms that resolve an unknown.

- [] I know the provider is the source of truth after a timeout.
- [] I know why webhooks need signature verification and dedup.
- [] I can give the void-versus-refund asymmetry in money.
- [] I know reconciliation always finds something, and what it means if it does not.
- [] I answered all three questions above out loud.

Proving a greedy choice, simply

DATA STRUCTURES AND ALGORITHMS

Proving a greedy choice, simply

SYSTEM DESIGN

Design a ticket booking system

1 What this is, and why they ask it

Yesterday ended with a gap: **if you cannot find a counter-example, you owe an argument.** Today is the argument.

There are exactly two, they are both short, and neither is a formal proof. They are the sentences you say out loud in an interview when asked “why does that work” — and **being able to say one of them is the difference between “I think greedy works here” and “greedy works here, and here is why”.**

They ask it because **it is the highest-value thirty seconds in an algorithms interview.** A candidate who writes a correct greedy solution has produced code. **A candidate who writes it and then justifies it in two sentences has demonstrated that they know it is correct rather than hoping** — and that distinction is precisely what the interviewer is trying to measure and usually cannot.

The other reason is that **the argument is also a debugging tool.** Trying to construct the exchange argument and failing is the fastest way to discover that greedy does not work. **The proof attempt and the counter-example search are the same activity from opposite ends,** and doing both takes about a minute.

One thing to be clear about, because it matters for how you use this day. These are not proofs in the mathematical sense — **they are proof sketches, and that is what is wanted.** Nobody expects induction on paper. **They expect you to be able to say why the first choice cannot hurt.**

By the end of this lesson you can produce both arguments for any greedy algorithm, know which to reach for, recognise when neither works, and say the whole thing in under thirty seconds.

2 The story

The dispute was about a wall and it had been going for two years and Fatima had been asked to settle it because everybody trusted her and nobody else was speaking.

Two brothers, one plot, one wall between them, and the wall was eleven inches onto one side.

The younger brother had documents. The elder brother had a different set of documents. **Both sets were real and neither settled anything**, because they described the plot from different corners.

And the meeting she called went the way those meetings go, which is that each of them explained why he was right for forty minutes.

What she did in the end was not adjudicate. It was something smaller and it worked.

She turned to the elder brother and said: **“Suppose your brother is right. Suppose the wall is his. Show me what you lose.”**

And he started to say something about principle, and she stopped him, and made him actually walk it. **Eleven inches along thirty feet. A strip you could not stand in.**

Then she turned to the younger one. **“Now suppose your brother is right. Show me what you lose.”**

The same strip.

“So whichever way I decide, neither of you loses anything you can use.”

Which did not settle it, and it changed the argument entirely, **because both of them had been assuming the other was gaining something.**

Then she did the second thing, and it was the one that actually ended it.

“Take whatever arrangement you both think is fairest. Any arrangement. Now move the wall eleven inches. Tell me what gets worse.”

And they could not find anything.

“Then it does not matter,” she said. **“You have been arguing for two years about a change that makes no arrangement worse. Put it where it is and go home.”**

Her son asked her afterwards how she had known that would work.

“I did not know. I asked them to make it worse and they could not. That is not the same as being sure. It is just the only thing you can actually check.”

3 The idea in plain English

Fatima's second question — **take any arrangement, make the change, and show me what gets worse** — is the exchange argument, and it is the more useful of the two.

Start with what you are trying to establish, precisely.

Greedy makes a first choice. Everything after that is the same problem on a smaller input. So if the first choice is safe, and the same argument applies to what remains, the whole thing is optimal by induction.

“Safe” means: there is at least one optimal solution that contains this choice.

Not that every optimal solution contains it. Not that the choice is “the best”. Only that you have not ruled out optimality by making it — which is a much weaker and much more provable claim.

Now the two arguments.

Argument one: the exchange argument. “Any optimal solution can be modified to include my choice, without getting worse.”

The shape is always the same, in three steps:

1. Take ANY optimal solution O . (Not the greedy one. Any.)
2. If O already contains my greedy choice g , done.
3. If not, O contains something else, x , where g would go.
SWAP x for g , and show the result is still valid and no worse.
→ so there IS an optimal solution containing g .

Step three is the whole argument, and it has two halves that must both be shown: **the swapped solution is still legal** — it does not violate a constraint — **and it is no worse** — the objective did not decrease.

Activity selection, done properly:

Claim: the interval that finishes earliest is in some optimal selection.

Take any optimal selection O , and look at its first interval x . My greedy choice g finishes no later than x — because g finishes earliest of all of them.

Swap x for g . Is it still legal? Yes — everything in O after x started at or after x finished, and g finishes no later than x , so nothing overlaps.

Is it worse? No — it has exactly the same number of intervals.

So O with g swapped in is still optimal, and it contains g .

Six sentences, and that is a complete argument for the canonical greedy. In an interview it compresses to “swapping in the earliest finisher cannot create a conflict, because it finishes no later than whatever it replaced”.

Argument two: “greedy stays ahead”. “At every step, my partial solution is at least as good as anyone else’s.”

Instead of modifying an optimal solution, you compare progress directly:

1. Let $g_1, g_2, g_3, \dots$ be greedy's choices in order.
2. Let $o_1, o_2, o_3, \dots$ be any other valid solution's, in order.
3. Show by induction: after k choices, greedy is at least as far along as the other solution.
4. Therefore greedy cannot run out of room sooner, so it makes at least as many choices.

For activity selection, “at least as far along” means “greedy’s k -th interval finishes no later than yours does”. Which is true at $k = 1$ by definition, and stays true because greedy always picks the earliest compatible interval.

And then the conclusion: if greedy’s k -th choice always finishes earliest, greedy can never be forced to stop before you.

When to use which.

EXCHANGE ARGUMENT

"I can swap my choice into any optimal solution"

good for: selection problems where the answer is a SET

activity selection
MST (the cut property)
Huffman
fractional knapsack

STAYS AHEAD

"My partial answer is always at least as good as yours"

good for: problems with a natural ORDER and a running measure

activity selection (both work)
shortest path
scheduling by deadline

MOST OF THE TIME, USE EXCHANGE. It is shorter to say and it applies more widely.

Now the third thing, which is what makes this practical: the failed proof is a counter-example generator.

When you try to construct the exchange argument and cannot, the place where it fails tells you what the counter-example looks like.

Try it on 0/1 knapsack with the ratio rule:

Take any optimal solution O . My greedy choice g is the item with the best value per kilo. If O does not contain g , swap something out for g .

But what do I swap out? g might be heavier than any single item in O , so removing one item might not make room. And removing two might lose more value than g adds.

The argument fails exactly there — at “might not make room” — and that is the counter-example.

A heavy high-ratio item, and two lighter items that together fill the bag better. (6kg, 60) against two (5kg, 40) items in a ten-kilo bag. The failure of the proof told you the shape of the input.

And the contrast with the fractional version is the whole lesson: there, you can always swap in a *fraction* of g , so “might not make room” never arises. The exchange goes through, and the algorithm is correct.

Finally: what to actually say, and how long it should take.

Thirty seconds, three sentences:

“The greedy choice is X. Take any optimal solution — if it does not contain X, swap X in for whatever it used instead. That swap is still legal because [reason], and no worse because [reason], so there is an optimal solution containing X.”

Fill in the two bracketed reasons and you are done. If you cannot fill them in, say so — “I cannot construct the exchange argument, which makes me suspect greedy fails here, so I will write the DP” is an excellent answer, and it is honest.

4 The picture

The exchange argument, drawn:

ANY OPTIMAL SOLUTION O

[x][b][c][d]
 ^

whatever O used
in the first slot

AFTER THE SWAP

[g][b][c][d]
 ^

my greedy choice

TWO THINGS TO SHOW:

1. STILL LEGAL — no constraint is violated
2. NO WORSE — the objective did not decrease

→ therefore SOME optimal solution contains g

→ repeat on the remainder. Induction does the rest.

Note what is NOT claimed: that every optimal solution contains g, or that g is "the best". Only that choosing g does not rule out optimality.

Activity selection, the swap in detail:

```

time ----->

0's first choice x:      [-----x-----]
greedy's choice g:  [--g--]
                        ^ g finishes NO LATER than x

the rest of 0:          [--b--] [--c--]
                        ^ starts at or after x ends

swap x → g:             [--g--]          [--b--] [--c--]
                        ^ g ends even earlier, so b still fits

LEGAL: yes – g ends no later than x, so nothing that fitted
      after x can now conflict
WORSE: no – same count

That is the entire proof, in one picture.

```

“Greedy stays ahead”, drawn:

step:	1	2	3	4	
greedy:	ends 4	ends 7	ends 9	ends 12	
any other:	ends 6	ends 8	ends 11	ends 15	
	^	^	^	^	
	≤	≤	≤	≤	at EVERY step

greedy's k-th choice always finishes no later than yours.

→ greedy is never the one that runs out of room first
 → so it makes at least as many choices
 → so it is optimal

USE THIS when there is a natural ORDER and a running MEASURE.
 Use EXCHANGE otherwise – it is shorter and applies more widely.

The failed proof as a counter-example generator:

0/1 KNAPSACK, greedy by value per kilo

attempt the exchange:

take any optimal O . If it lacks g (the best ratio), swap g in.
what comes out?

- g may be HEAVIER than any single item in O
- so removing ONE may not make room ← FAILS HERE
- and removing TWO may lose more than g adds

THE FAILURE POINT IS THE COUNTER-EXAMPLE:

a HEAVY high-ratio item, and TWO lighter items that together do better

capacity 10: (6kg, 60) ratio 10
 (5kg, 40) ratio 8
 (5kg, 40) ratio 8
greedy: 60. optimal: 80.

AND THE FRACTIONAL VERSION:

you can swap in a FRACTION of g , so "may not make room"
never arises → the exchange goes through → correct.

The proof attempt and the counter-example search are the SAME
ACTIVITY from opposite ends.

The four canonical arguments, side by side:

ACTIVITY SELECTION (sort by end time)

swap in the earliest finisher: it ends no later, so nothing
that fitted before still conflicts. Same count.

FRACTIONAL KNAPSACK (sort by ratio)

if O uses a worse ratio anywhere, replace some of it with an
equal WEIGHT of the best ratio. Same weight, more value.
(Works only because you can take a fraction.)

HUFFMAN (merge the two rarest)

in any optimal tree, the two deepest leaves are siblings.
Swap the two rarest symbols into those positions: total cost
cannot increase, because rarer symbols moved deeper.

SHORTEST JOB FIRST (sort by duration)

if any adjacent pair is out of order, swapping them reduces
the total wait by the difference. So sorted order is optimal.
(This one is an EXCHANGE on ADJACENT elements – the simplest form.)

What to say, and how long:

THIRTY SECONDS, THREE SENTENCES:

"The greedy choice is X."

"Take any optimal solution; if it does not contain X, swap X in for whatever it used there."

"That is still legal because ___, and no worse because ___ — so some optimal solution contains X."

FILL IN THE TWO BLANKS. If you cannot:

"I cannot construct the exchange argument, which makes me suspect greedy fails, so I will write the DP."

That is an EXCELLENT answer. It is honest, it is specific, and it names the reason.

5 The code, built step by step

This lesson is about arguments rather than algorithms, so the code here does two things: it *checks* the arguments empirically, and it makes the failure points visible.

Verifying an exchange argument empirically

```
def exchange_holds(problem, greedy_choice, is_valid, value, trials=2000):
    """For random optimal solutions, can the greedy choice be swapped in?"""
    failures = []
    for _ in range(trials):
        instance = problem.generate()
        best = problem.brute_force_optimal(instance)      # a full optimal solution
        g = greedy_choice(instance)
        if g in best:
            continue                                     # already contains it
        for x in best:
            swapped = (best - {x}) | {g}
            if is_valid(instance, swapped) and value(swapped) ≥ value(best):
                break                                    # the exchange worked
        else:
            failures.append((instance, best, g))         # NO swap worked
    return failures
```

This does not prove anything, and it does something almost as useful: it finds the instances where no swap works, which is exactly where your intuition is wrong.

The `for ... else` is doing real work here — the `else` runs only if no `break` happened, meaning every possible swap failed.

Activity selection, with the argument as a comment

```
def max_activities(intervals: list[tuple[int, int]]) → int:
    """
    Greedy: always take the compatible interval that finishes earliest.

    EXCHANGE ARGUMENT
    Take any optimal selection  $O$  and its first interval  $x$ .
    The earliest-finishing interval  $g$  ends no later than  $x$ .
    Swapping  $x$  for  $g$  is still LEGAL: everything  $O$  scheduled after  $x$ 
    began at or after  $x$  ended, and  $g$  ends no later than  $x$ .
    It is NO WORSE: the count is unchanged.
    So some optimal solution contains  $g$ . Induct on the remainder.
    """
    count, last_end = 0, float("-inf")
    for start, end in sorted(intervals, key=lambda x: x[1]):
        if start ≥ last_end:
            count += 1
            last_end = end
    return count
```

Writing the argument as a docstring is a habit worth having, and not for the reader — **for you**. If you cannot write it, you do not know the algorithm is correct, **and you find that out while writing rather than in an interview**.

Shortest job first: the adjacent-swap argument, which is the simplest form

```
def min_total_waiting(durations: list[int]) → int:
    """
    Greedy: shortest job first.

    ADJACENT EXCHANGE ARGUMENT
    Suppose two adjacent jobs are out of order: a longer job of
    length L before a shorter one of length S.
    Swapping them changes only THOSE TWO jobs' waiting times, and
    it reduces the total by  $(L - S) > 0$ .
    So any schedule with an out-of-order adjacent pair is NOT
    optimal. Therefore the sorted order is optimal.
    """
    elapsed, total = 0, 0
    for d in sorted(durations):
        total += elapsed
        elapsed += d
    return total

def waiting_time(order: list[int]) → int:
    elapsed, total = 0, 0
    for d in order:
        total += elapsed
        elapsed += d
    return total
```

The adjacent-swap form is the easiest exchange argument to construct, because you only have to reason about two elements — everything else in the schedule is unaffected. **Look for it first.**

Demonstrating the adjacent swap

```
def show_adjacent_swap(durations: list[int]) → None:
    """Every out-of-order adjacent pair can be improved. Show it."""
    for i in range(len(durations) - 1):
        if durations[i] > durations[i + 1]:
            swapped = durations[:]
            swapped[i], swapped[i + 1] = swapped[i + 1], swapped[i]
            before, after = waiting_time(durations), waiting_time(swapped)
            print(f"  swapping positions {i},{i+1}: {before} → {after}"
                  f"    (saved {before - after}, = {durations[i]}-{durations[i+1]})")
            return
    print("  already sorted – no adjacent pair is out of order")
```

The saving is exactly the difference between the two durations, which is the arithmetic the argument depends on — and seeing it come out exactly right is more convincing than the algebra.

Where the argument fails: 0/1 knapsack

```
def why_ratio_fails_for_01() → None:
    """The exchange breaks at 'swapping in g may not make room'."""
    weights, values, capacity = [6, 5, 5], [60, 40, 40], 10
    ratios = [v / w for w, v in zip(weights, values)]
    print(f" ratios: {[round(r, 1) for r in ratios]}")
    print(f" greedy takes item 0 (ratio {ratios[0]}), leaving room 4 – nothing fits")
    print(f" greedy value : {60}")
    print(f" optimal      : {80} (items 1 and 2)")
    print(" THE EXCHANGE FAILS AT: 'swap g in for one item of 0'")
    print(" → g weighs 6; removing ONE 5kg item leaves room 5, not 6")
    print(" → removing BOTH loses 80 to gain 60")
    print(" → the failure point IS the counter-example's shape")
```

Printing the failure point rather than only the numbers is the pedagogical bit: the counter-example is not a coincidence, it is what the broken step describes.

And where it succeeds: fractional knapsack

```
def why_ratio_works_for_fractional() → None:
    """The same exchange, with fractions allowed, goes through."""
    print(" take any optimal solution 0")
    print(" if 0 contains ANY item with a worse ratio than g,")
    print(" remove an equal WEIGHT of it and add that weight of g")
    print(" → same total weight (so still legal)")
    print(" → more value (so no worse)")
    print(" 'may not make room' NEVER ARISES, because fractions exist")
    print(" → the exchange goes through → greedy is optimal")
```

Putting the two side by side is the point of the lesson: the same sort rule, the same attempted argument, and one word of difference in the problem statement decides whether it works.

Huffman: the argument that is not obvious

```
import heapq
from collections import Counter

def huffman_cost(text: str) → int:
    """
    Greedy: repeatedly merge the two least frequent symbols.

    EXCHANGE ARGUMENT
    In any optimal prefix code, the two DEEPEST leaves are siblings
    (otherwise you could move one up and do better).
    If the two rarest symbols are not already there, swap them into
    those positions.
    Swapping a rarer symbol DEEPER and a commoner one SHALLOWER
    cannot increase the total cost, because cost = sum of
    frequency x depth.
    So some optimal tree has the two rarest as sibling leaves –
    which is exactly what merging them assumes.
    """
    frequencies = list(Counter(text).values())
    if len(frequencies) ≤ 1:
        return len(text)
    heapq.heapify(frequencies)
    total = 0
    while len(frequencies) > 1:
        a, b = heapq.heappop(frequencies), heapq.heappop(frequencies)
        total += a + b
        heapq.heappush(frequencies, a + b)
    return total
```

“Swapping a rarer symbol deeper and a commoner one shallower cannot increase the total” is the whole argument, and it is one sentence — which is what makes Huffman a good thing to have rehearsed.

The complete solution

```
"""Proving greedy: the two arguments, and what a failed proof tells you."""

import heapq
import random
from collections import Counter
from itertools import combinations

# ----- the algorithms, each with its argument -----

def max_activities(intervals: list[tuple[int, int]]) → int:
    """
    EXCHANGE: swap in the earliest finisher. It ends no later than
    whatever it replaced, so nothing that fitted after still conflicts,
    and the count is unchanged.
    """
    count, last_end = 0, float("-inf")
    for start, end in sorted(intervals, key=lambda x: x[1]):
        if start ≥ last_end:
            count += 1
            last_end = end
    return count

def waiting_time(order: list[int]) → int:
    elapsed, total = 0, 0
    for d in order:
        total += elapsed
        elapsed += d
    return total

def min_total_waiting(durations: list[int]) → int:
    """
    ADJACENT EXCHANGE: any out-of-order adjacent pair can be swapped
    to reduce the total by exactly (longer - shorter). So a schedule
    with any such pair is not optimal.
    """
    return waiting_time(sorted(durations))

def huffman_cost(text: str) → int:
    """
    EXCHANGE: in any optimal tree the two deepest leaves are siblings.
    Moving the two rarest symbols there cannot increase
    sum(frequency x depth).
    """
    frequencies = list(Counter(text).values())
    if len(frequencies) ≤ 1:
        return len(text)
    heapq.heapify(frequencies)
    total = 0
    while len(frequencies) > 1:
        a, b = heapq.heappop(frequencies), heapq.heappop(frequencies)
        total += a + b
        heapq.heappush(frequencies, a + b)
    return total

# ----- making the arguments visible -----
```

```

def show_adjacent_swap(durations: list[int]) → None:
    for i in range(len(durations) - 1):
        if durations[i] > durations[i + 1]:
            swapped = durations[:]
            swapped[i], swapped[i + 1] = swapped[i + 1], swapped[i]
            before, after = waiting_time(durations), waiting_time(swapped)
            print(f"    {durations} → {swapped}")
            print(f"    total wait {before} → {after}, saved {before - after} "
                  f"f"= {durations[i]} - {durations[i+1]}")
            return
    print("    already sorted")

def stays_ahead_table(intervals: list[tuple[int, int]],
                      other: list[tuple[int, int]]) → None:
    """Greedy's k-th choice finishes no later than any other schedule's."""
    greedy: list[tuple[int, int]] = []
    last = float("-inf")
    for s, e in sorted(intervals, key=lambda x: x[1]):
        if s ≥ last:
            greedy.append((s, e))
            last = e
    print(f"    {'step':>5} {'greedy ends':>12} {'other ends':>12}")
    for k in range(max(len(greedy), len(other))):
        g = greedy[k][1] if k < len(greedy) else "-"
        o = other[k][1] if k < len(other) else "-"
        mark = "≤" if isinstance(g, int) and isinstance(o, int) and g ≤ o else ""
        print(f"    {k+1:>5} {str(g):>12} {str(o):>12} {mark}")

def why_ratio_fails_for_01() → None:
    print("    ratios: 6kg/60 = 10.0, 5kg/40 = 8.0, 5kg/40 = 8.0")
    print("    greedy takes the 6kg item; 4kg of room left; nothing fits → 60")
    print("    optimal takes both 5kg items → 80")
    print("    THE EXCHANGE FAILS AT: 'swap g in for one item of 0'")
    print("    g weighs 6; removing one 5kg item leaves room 5, not 6")
    print("    removing both loses 80 to gain 60")
    print("    → the failure point IS the shape of the counter-example")

def why_ratio_works_for_fractional() → None:
    print("    if 0 contains any item with a worse ratio than g,")
    print("    remove an equal WEIGHT of it and add that weight of g")
    print("    same total weight (legal), more value (no worse)")
    print("    'may not make room' never arises, because fractions exist")
    print("    → the exchange goes through → greedy is optimal")

# ----- the empirical check -----

def brute_force_activities(intervals: list[tuple[int, int]]) → set:
    """The largest compatible subset. Exponential and obviously correct."""
    best: set = set()
    n = len(intervals)
    for size in range(n, 0, -1):
        for combo in combinations(range(n), size):
            chosen = sorted(intervals[i] for i in combo)
            if all(chosen[i][1] ≤ chosen[i + 1][0] for i in range(len(chosen) - 1)):
                return set(combo)
    return best

```

```

def exchange_check(trials: int = 400) → tuple[int, int]:
    """For random optima not containing the greedy choice, can we swap it in?"""
    worked = failed = 0
    for _ in range(trials):
        n = random.randint(2, 7)
        intervals = []
        for _ in range(n):
            s = random.randint(0, 10)
            intervals.append((s, s + random.randint(1, 5)))
        best = brute_force_activities(intervals)
        g = min(range(n), key=lambda i: intervals[i][1]) # earliest finisher
        if g in best:
            continue
        for x in best:
            swapped = sorted(intervals[i] for i in (best - {x}) | {g})
            if all(swapped[i][1] ≤ swapped[i + 1][0]
                  for i in range(len(swapped) - 1)):
                worked += 1
                break
        else:
            failed += 1
    return worked, failed

if __name__ == "__main__":
    random.seed(0)

    print("\nARGUMENT 1 – ADJACENT EXCHANGE (shortest job first)")
    show_adjacent_swap([3, 1, 2])
    print(f"    sorted total wait: {min_total_waiting([3, 1, 2])}")
    print(f"    worst order      : {waiting_time(sorted([3, 1, 2], reverse=True))}")

    print("\nARGUMENT 2 – GREEDY STAYS AHEAD (activity selection)")
    acts = [(1, 4), (3, 5), (0, 6), (5, 7), (3, 9), (5, 9), (6, 10), (8, 11)]
    stays_ahead_table(acts, [(0, 6), (6, 10)])

    print("\nTHE EXCHANGE, CHECKED EMPIRICALLY (activity selection)")
    worked, failed = exchange_check()
    print(f"    optima not containing the greedy choice: {worked + failed}")
    print(f"    a swap worked: {worked}    no swap worked: {failed}")
    print(f"    (failures would mean the exchange argument is WRONG)")

    print("\nWHERE THE ARGUMENT FAILS – 0/1 knapsack by ratio")
    why_ratio_fails_for_01()

    print("\nWHERE THE SAME ARGUMENT SUCCEEDS – fractional knapsack")
    why_ratio_works_for_fractional()

    print("\nHUFFMAN – one sentence")
    print(f"    swapping a rarer symbol DEEPER and a commoner one")
    print(f"    SHALLOWER cannot increase sum(frequency x depth)")
    print(f"    cost of 'aaabbc': {huffman_cost('aaabbc')} bits")

```

Run it and you get:

ARGUMENT 1 – ADJACENT EXCHANGE (shortest job first)

[3, 1, 2] → [1, 3, 2]
total wait 7 → 5, saved 2 = 3 - 1
sorted total wait: 4
worst order : 8

ARGUMENT 2 – GREEDY STAYS AHEAD (activity selection)

step	greedy ends	other ends	
1	4	6	≤
2	7	10	≤
3	11	-	

THE EXCHANGE, CHECKED EMPIRICALLY (activity selection)

optima not containing the greedy choice: 81
a swap worked: 81 no swap worked: 0
(failures would mean the exchange argument is WRONG)

WHERE THE ARGUMENT FAILS – 0/1 knapsack by ratio

ratios: 6kg/60 = 10.0, 5kg/40 = 8.0, 5kg/40 = 8.0
greedy takes the 6kg item; 4kg of room left; nothing fits → 60
optimal takes both 5kg items → 80

THE EXCHANGE FAILS AT: 'swap g in for one item of 0'
g weighs 6; removing one 5kg item leaves room 5, not 6
removing both loses 80 to gain 60
→ the failure point IS the shape of the counter-example

WHERE THE SAME ARGUMENT SUCCEEDS – fractional knapsack

if 0 contains any item with a worse ratio than g,
remove an equal WEIGHT of it and add that weight of g
same total weight (legal), more value (no worse)
'may not make room' never arises, because fractions exist
→ the exchange goes through → greedy is optimal

HUFFMAN – one sentence

swapping a rarer symbol DEEPER and a commoner one
SHALLOWER cannot increase sum(frequency x depth)
cost of 'aaabbc': 9 bits

a swap worked: 81, no swap worked: 0 is the exchange argument checked rather than asserted. A single failure there would mean the argument is wrong, and running it once is worth more than reading the proof twice.

And the adjacent swap saving exactly $3 - 1 = 2$ is the arithmetic the argument depends on, coming out exactly as predicted.

6 What it costs

The arguments cost nothing to run and about thirty seconds to say.

construct the exchange argument:	~30 seconds, out loud
write it as a docstring:	~1 minute
check it empirically:	~10 minutes to write, seconds to run

Against what it buys:

a correct greedy with no justification:
→ the interviewer cannot tell whether you knew or guessed
→ and the follow-up "why does that work?" is coming

a correct greedy WITH the argument:
→ demonstrably knew
→ and the same 30 seconds would have caught it if it were wrong

The asymmetry between the two directions:

finding a counter-example	PROVES greedy is wrong
constructing the exchange	STRONGLY suggests it is right (it is a sketch, not a formal proof)
neither one	→ write the DP, and say why
cost of unnecessary DP:	some code, same complexity often
cost of unjustified greedy:	a wrong answer

The empirical check, costed:

brute force for $n \leq 8$: ~5 minutes to write
400 random instances: < 1 second to run
what it tests: whether ANY swap works for real optima

and the result is binary and informative:
0 failures → the exchange holds on everything tested
1 failure → the argument is WRONG, and you have the instance

The four canonical arguments, by length:

shortest job first	1 sentence (adjacent swap; the simplest form)
activity selection	2 sentences (swap in the earliest finisher)
Huffman	2 sentences (rarest symbols go deepest)
fractional knapsack	2 sentences (swap an equal WEIGHT)
MST cut property	2 sentences (the lightest edge across any cut)

→ all five fit in under thirty seconds of speech.
That is the entire memorisation burden of this lesson.

And the algorithms themselves, for reference:

```
activity selection    O(n log n), dominated by the sort
shortest job first    O(n log n), dominated by the sort
Huffman              O(n log n), dominated by the heap
fractional knapsack   O(n log n), dominated by the sort
```

the ARGUMENT costs nothing at runtime. It is entirely a design-time and interview-time activity.

Where the DP alternative sits, when greedy fails:

```
0/1 knapsack          greedy O(n log n) and WRONG
                      DP      O(n x W) and correct
weighted intervals    greedy O(n log n) and WRONG
                      DP      O(n log n) and correct ← same complexity!
```

→ the cost of greedy failing is sometimes zero asymptotically.
Weighted interval scheduling is the same $O(n \log n)$ as the unweighted greedy, because both are dominated by the sort.

That last line is worth having ready, because it means “**greedy does not work here**” is often not bad news — it is a table instead of a loop at the same complexity.

7 The traps

Claiming greedy works because it passed the examples.

```
>>> # sorting activity selection by DURATION gives the right answer
>>> # on many inputs, including the standard example
>>> # → the tests pass, and the algorithm is wrong
```

Passing tests is not an argument. The failing inputs for a wrong greedy are often unusual shapes — a short interval overlapping two longer disjoint ones — **which random or hand-written tests rarely contain.**

Proving the wrong thing.

```
WRONG: "the greedy choice is in THE optimal solution"
       → often false, because there may be several optima and
         some do not contain it

RIGHT: "there is AN optimal solution containing the greedy choice"
       → what the exchange argument actually establishes
```

The stronger claim is usually false and always harder, and trying to prove it is how the argument gets stuck.

Forgetting to show the swap is legal.

"swapping in the earliest finisher gives the same count, so it is no worse"

→ that is HALF the argument

you must ALSO show the result is still a valid solution:

→ g ends no later than x, so nothing that fitted after x conflicts

omitting the legality half is the most common incomplete argument.

Both halves, every time: still legal, and no worse.

Using "stays ahead" where there is no natural order.

stays ahead needs: a sequence of choices, and a measure that can be compared step by step

0/1 knapsack has no such order – items are a SET, not a sequence

→ the argument does not even typecheck

→ use exchange, or discover that neither works

Assuming the exchange argument exists.

most greedy rules are WRONG, so most attempted exchange arguments SHOULD fail

→ if yours goes through suspiciously easily, check that you did not skip the legality half

→ and if it fails, that is INFORMATION, not a dead end

Treating a failed proof as inconclusive.

```
>>> # the exchange fails at "swapping g in may not make room"
>>> # → that is not "I could not prove it"
>>> # → that is "here is exactly the input that breaks it":
>>> #     a heavy high-ratio item and two lighter ones
```

The failure point names the counter-example. Reading it as a dead end rather than as a description is the wasted opportunity.

Over-formalising in an interview.

what is wanted: two sentences, out loud, thirty seconds
what is not: induction written out, base case and step,
formal notation

a candidate who starts writing "let O be an optimal solution and let k be the smallest index such that..." has misjudged the question and is burning minutes.

Say the sketch. If they want more, they will ask.

Not saying anything at all.

the single most common failure is writing a correct greedy solution and moving straight on to the complexity

- the interviewer now cannot distinguish you from someone who guessed and was lucky
- and "why does that work?" is the next question anyway

Volunteer the argument. It costs thirty seconds and it is the thing being measured.

8 In the interview

How it gets asked

- "Why does that work?" — the direct form, after any greedy solution.
- "Prove your greedy choice is correct." — the same question, more formally phrased.
- "How do you know sorting by end time is right?" — the specific form.
- "Convince me." — which is asking for the sketch, not a proof.
- "What if you sorted by X instead?" — which is asking for the counter-example.

The first ninety seconds

This is what to say immediately after writing any greedy solution, unprompted.

“Let me justify the greedy choice, because that is the part that is not obvious.

The claim I need is not that my choice is the best one. It is weaker than that: **there is at least one optimal solution that contains my choice.** That is enough, because if choosing it does not rule out optimality, and what remains is the same problem on a smaller input, then induction gives the result.

The argument is an exchange. Take any optimal solution. **If it already contains my choice, there is nothing to prove.** If it does not, it used something else where my choice would go — **so swap them, and show two things.**

First, that the result is still legal — no constraint is violated by the substitution.

Second, that it is no worse — the objective did not decrease.

If both hold, then I have produced an optimal solution containing my choice, which is exactly the claim.

For this problem specifically: my choice is the interval that finishes earliest. Take any optimal selection and look at its first interval. **Mine finishes no later than that one, by definition.** So swapping mine in cannot create a conflict — **everything scheduled after that interval began at or after it ended, and mine ends even earlier.** And the count is unchanged, so it is no worse.

That is the whole argument, and it is two sentences.

There is a second form worth knowing, ‘greedy stays ahead’, where instead of modifying an optimal solution you show that after every step, greedy’s partial answer is at least as good as anyone else’s. **For this problem: greedy’s k-th interval always finishes no later than yours, so greedy can never be the one to run out of room first.**

I would use exchange by default — it is shorter to state and it applies to problems where the answer is a set rather than a sequence.”

The follow-ups

“Give me the exchange argument in full.”

“Three steps, and step three has two halves that both have to be shown.

Step one: take any optimal solution. Not the greedy one — an arbitrary optimal one. **This matters: the argument must work for every optimum, not for a convenient one.**

Step two: if it already contains my greedy choice, I am done for this step.

Step three: if it does not, it contains something else where my choice would go. Swap them. Then show:

That the swapped solution is still valid. This is the half people forget, and it is where the argument usually breaks. **For activity selection: my interval finishes no later than the one I replaced, so anything that fitted after the old one still fits.**

And that it is no worse. For activity selection this is trivial — the count is identical. **For a weighted problem it would not be trivial, which is precisely why greedy fails there.**

Then the conclusion: I have constructed an optimal solution containing my greedy choice. So making that choice does not rule out optimality. **And what remains after it is the same problem on a smaller input, so the same argument applies — induction finishes it.**

The simplest form of this is the adjacent swap, and I would look for it first because it is much easier to reason about. **For shortest-job-first scheduling: if any two adjacent jobs are out of order — a long one before a short one — swapping them changes only those two jobs’ contributions and reduces the total wait by exactly the difference in their lengths.** So any schedule with an out-of-order adjacent pair is not optimal, **which means the sorted order is.**

That is one sentence and it is a complete argument, because you only have to reason about two elements — everything else in the schedule is unaffected by the swap. **Whenever the adjacent form applies, use it.**“

“What if you cannot construct the argument?”

“Then that is information, and I would use it rather than treat it as a dead end.

The place where the argument breaks tells me what the counter-example looks like. The proof attempt and the counter-example search are the same activity from opposite ends.

Concretely, 0/1 knapsack with the value-per-kilo rule. I try the exchange: take any optimal solution; if it does not contain my best-ratio item, swap it in for something.

And I get stuck immediately at ‘swap it in for what?’ My item might be heavier than any single item in the optimal solution, **so removing one might not make room** — and removing two might lose more value than mine adds.

That failure point is a description of the counter-example. A heavy, high-ratio item, and two lighter items that together do better. **Six kilos worth sixty against two five-kilo items worth forty each, in a ten-kilo bag** — greedy gets sixty and the answer is eighty. **The proof told me where to look.**

And the contrast is the whole lesson. For the fractional version, the same argument goes through perfectly: **if the optimal solution uses any worse ratio, I remove an equal weight of it and add that weight of mine.** Same total weight, so it is still legal; more value, so it is no worse. **‘Might not make room’ never arises, because I can take a fraction.**

One word of difference in the problem statement, and the same sort rule goes from provably optimal to provably wrong.

So my procedure when I cannot construct the argument is: look at where it broke, build the input it describes, and check. If that produces a counter-example, it is DP and I say so.

And if I can neither construct the argument nor find a counter-example, I say exactly that — ‘I cannot justify the greedy choice, so I will write the DP, which is always correct’. **That is honest, it is specific, and it is a much better answer than an unjustified greedy solution.**“

“Why does Huffman coding work?”

“It is an exchange argument and it is the least obvious of the standard ones, so it is worth having rehearsed.

The greedy step is: repeatedly take the two least frequent symbols and merge them into a single node whose frequency is their sum.

The claim to justify is: the two rarest symbols are siblings at the deepest level of some optimal tree.

The argument comes in two parts.

First: in any optimal prefix code, the two deepest leaves are siblings. Because if the deepest leaf had no sibling, you could move it up one level — **shortening its code without lengthening anything else** — which contradicts optimality.

Second: the exchange. Take any optimal tree. If the two rarest symbols are not in those deepest sibling positions, **swap them with whatever is there.**

Why that does not increase the cost: the total cost is the sum over symbols of frequency times depth. **Swapping moves a rarer symbol deeper and a commoner one shallower.** The commoner symbol’s contribution falls by more than the rarer one’s rises — **because the depth change is the same for both, and the frequencies differ in the right direction.**

So the cost cannot increase, and the tree is still valid — you have only relabelled leaves.

Therefore some optimal tree has the two rarest symbols as sibling leaves at the maximum depth, which is exactly what merging them assumes. **And after merging, the remaining problem is the same problem with one fewer symbol, so induction applies.**

Two sentences of intuition, if I had to compress it: rarer symbols should have longer codes, and the two rarest can always be pushed to the bottom without harm.

One implementation note, since it comes up: the total cost equals the sum of all the merge values, because every merge adds one bit to every symbol underneath it. **That is why the algorithm can compute the encoded length without ever building the tree.**“

The model answer

“Here is the problem: you have a set of jobs, each with a deadline and a profit, and each takes exactly one unit of time. You have one machine. Schedule jobs to maximise total profit — a job earns its profit only if it finishes by its deadline. Solve it, and convince me your approach is correct.”

“Let me diagnose first, then solve, then justify — **and I suspect the justification is the point of the question.**

The greedy candidate is: sort the jobs by profit, highest first, and schedule each one as late as possible before its deadline.

My first move is to try to break it, and I would spend thirty seconds. I look for a case where taking the most profitable job blocks two others that together beat it. **A job worth a hundred with deadline one, and two jobs worth sixty each with deadlines one and two.** Greedy takes the hundred, which occupies slot one, **and then the sixty with deadline one cannot be placed** — total one hundred and sixty. The alternative is sixty plus sixty, which is worse. **So that attempt does not break it.**

I would try a few more and, not finding one, move to the argument — because failing to find a counter-example is not evidence, it just means I now owe a proof.

The exchange argument. Take any optimal schedule σ , and let g be the highest-profit job.

If σ contains g , nothing to show.

If it does not, then either there is a free slot before g 's deadline — in which case adding g makes σ strictly better, contradicting optimality — **or every slot before g 's deadline is occupied.** In that case, take any job x in one of those slots. **Swap x out and g in.**

Is it legal? Yes — g is being placed in a slot before its own deadline, and removing x cannot make any other job late.

Is it worse? No — g has the highest profit of all jobs, so g 's profit is at least x 's. **The total did not decrease.**

So some optimal schedule contains g , and induction on the remaining jobs finishes it.

Now the implementation detail that the argument does not settle: where to put each job. As late as possible before its deadline — because that keeps the early slots free for jobs with tighter deadlines. **And that is a second small exchange argument:** if a job is placed earlier than necessary and this blocks a tighter-deadline job, moving it later frees the slot and cannot make it late.

The algorithm: sort by profit descending, and for each job scan backwards from its deadline for a free slot. That is $O(n^2)$ naively, and union-find makes it near-linear — each slot points to the next free slot at or before it, which is the same ‘find the next available’ pattern as disjoint-set union.

Cost: $O(n \log n)$ for the sort plus near-linear for the placement.

And two things I would flag.

If jobs took different amounts of time, this breaks. The exchange argument depended on every job occupying exactly one slot — **swapping g for x worked because they are the same size.** With variable durations, g might not fit where x was, **which is precisely the 0/1 knapsack failure**, and it becomes a much harder problem.

And if I had failed to construct the argument, I would have said so and written the DP instead — the cost of an unnecessary table is some code; the cost of an unjustified greedy is a wrong answer.“

9 Recall card

Two arguments, both short, both proof *sketches* rather than proofs. What you must establish is weaker than it looks: **there is AT LEAST ONE optimal solution containing your choice** — not that every optimum contains it, and not that the choice is “best”.

THE EXCHANGE ARGUMENT (use this by default), in three steps: take *any* optimal solution; if it contains your choice you are done; **otherwise swap your choice in for whatever it used there, and show BOTH that the result is still LEGAL and that it is NO WORSE.** Omitting the legality half is the most common incomplete argument.

Activity selection in two sentences: the earliest finisher ends no later than whatever it replaced, **so nothing that fitted after the old one now conflicts**, and the count is unchanged.

The ADJACENT-SWAP form is the easiest — look for it first, because you only reason about two elements. Shortest job first: any out-of-order adjacent pair can be swapped to reduce the total wait by exactly (longer – shorter), **so a schedule containing one is not optimal.**

“GREEDY STAYS AHEAD” is the other form: after every step, greedy’s partial answer is at least as good as anyone else’s, **so greedy can never run out of room first.** Use it when there is a natural order and a running measure; use exchange when the answer is a set.

A FAILED PROOF NAMES THE COUNTER-EXAMPLE. 0/1 knapsack by ratio breaks at “*swapping g in may not make room*” — which describes a heavy high-ratio item plus two lighter ones: $(6, 60)$ against two $(5, 40)$ in a 10-kg bag, 60 against 80. **The fractional version survives the same argument because you can swap an equal WEIGHT** — one word in the statement decides it.

Say it in thirty seconds, three sentences, unprompted. And “**I cannot construct the exchange argument, so I will write the DP**” is an excellent answer — an unnecessary table costs code, an unjustified greedy costs a wrong answer, and those are not symmetric.

Design a ticket booking system

1 What this is, and why they ask it

A ticket system sells specific, numbered, non-interchangeable things — **seat 14F on tonight's 19:30 show** — to many people who all want them at once.

They ask it because **it is the extreme case of a problem you met yesterday, and the extremity changes the answer**. E-commerce has inventory contention; **ticketing has inventory contention as its defining property**. A concert sells out in forty seconds. A train's Tatkal quota opens at ten in the morning and a hundred thousand people submit within the same second.

Three things make it genuinely different from selling ordinary stock.

Every unit is unique and non-substitutable. You cannot ship “an equivalent seat”. **So there is no oversell recovery** — with a warehouse you can apologise and back-order; **with a seat there is one and someone is standing in the aisle**.

The contention is extreme and predictable. Both halves matter: **extreme**, because a hundred thousand requests hit the same few hundred rows in one second; **predictable**, because you know the sale opens at ten o'clock. **Predictability is a gift** — it lets you pre-warm, pre-shard and queue in advance, which is not available to systems whose spikes are surprises.

And there is a deep product decision that is really an engineering one: what to do with the people you cannot serve. A hundred thousand people, a thousand seats. **Ninety-nine thousand of them are going to be disappointed, and the design is about how**. Rejecting them in fifty milliseconds with a clear message is a much better system than making them all wait thirty seconds for a timeout.

By the end of this lesson you can design seat locking under extreme contention, run the sale-opening spike, handle the hold-and-pay flow, prevent bots, and size the whole thing.

2 The story

The cinema had two hundred and forty seats and one glass window and Munuswamy sat behind it for nineteen years, and the only day that mattered was the first day of a big picture.

Ordinary days the queue was a queue. You came, you asked, he tore the tickets from the book, you paid, you went in.

On a first day there were four hundred people and the queue was not a queue.

And the thing that went wrong in the early years, the thing that caused the trouble, was not running out. **Running out was fine.** People understood running out.

What caused the trouble was selling the same seat twice.

Because a man would ask for two in the middle, and Munuswamy would look at the chart, and see that H14 and H15 were free, **and then the man would count his money, or go to fetch his wife, or argue about the price,** and in that time the boy at the second window had sold H14 to somebody else.

Two people, one seat, both holding a ticket, and the picture starting in eleven minutes.

The rule he made was small and it changed everything. The moment he said a seat aloud, **he put a pin in it on the chart.** Not sold. Pinned. And the pin meant nobody else could say that seat.

And the pin came out after two minutes if the money did not arrive, which he timed by the clock on the wall behind him, because otherwise a man who wandered off would hold the best seats in the house all evening.

Two minutes was short and people complained, **and on a first day it had to be short, because there were four hundred people and the pins were the only thing standing between him and the eleven-minutes-to-showtime argument.**

The second thing took him longer, and it was about the queue rather than the chart.

He started saying “sold out” before it was sold out.

Not lying. **When there were forty seats and two hundred people still waiting, he would walk down the line and tell everybody past about the sixtieth that there was nothing.** They were annoyed and they went home, **and they were much less annoyed than the ones who had queued for forty minutes and reached the window and been told no.**

“The number I cannot change,” he told his nephew. **“What I can change is how long somebody waits to find out.”**

3 The idea in plain English

Munuswamy's pin is a distributed lock with a TTL, his two-minute clock is the hold expiry, and his walk down the queue is the waiting room. **All three are the design.**

Start with what makes this different from ordinary inventory.

ORDINARY STOCK

units are interchangeable
oversell → back-order, apologise
demand is spread over time
"one left" is a number

TICKETS

each unit is UNIQUE
oversell → two people, one seat
demand is a SPIKE at a known time
"one left" is seat H14, specifically

Two consequences. You cannot use a counter — “997 seats remaining” does not tell anyone which ones, and the customer is choosing a specific seat on a map. **And you cannot recover from overselling**, so the correctness bar is absolute in a way it is not for a warehouse.

Now the core mechanism, which is the hold.

A seat has three states, exactly like yesterday's inventory but per-unit:

available	nobody has it
held	somebody is paying for it, with a TTL
sold	paid for

Selecting a seat holds it. Paying converts the hold to a sale. An expired hold releases it.

And the TTL is the entire safety mechanism, because **nobody sends “I have given up”** — the browser tab closes, the phone battery dies, the network drops. **A hold without an expiry is a seat lost forever.**

The hold must be atomic. Two people clicking H14 at the same instant is the whole problem, **and a check-then-write loses it.** The correct forms are the same as yesterday's:

```
Redis:      SET seat:show:H14 <session> NX EX 300
            → succeeds only if nobody holds it

Postgres:   UPDATE seats SET status='held', session=?, expires=?
            WHERE show=? AND seat='H14' AND status='available'
            → then check rows_affected = 1
```

Both are one operation. `SET NX` and a conditional `UPDATE` do the check and the write together, which is the property that makes them safe.

And two people wanting adjacent seats introduces a second problem. A group booking of four seats is four locks, and acquiring them one at a time can deadlock — two sessions each holding two of the four. **The fix is to acquire in a fixed order** — sorted by seat identifier — **so two sessions contending for the same seats always contend in the same sequence, and one simply fails.**

Now the spike, which is what makes ticketing hard.

A hundred thousand people at ten o'clock, for a thousand seats.

The single most important idea is that **ninety-nine percent of those requests must never reach the seat map.** Every one that does is contention on a resource that cannot serve it.

Three layers, in order of how much they help.

A waiting room. Munuswamy walking down the line. **Requests arriving at the sale opening are given a queue position and a token,** and admitted to the actual booking system at a controlled rate. **A hundred thousand people become a thousand at a time.**

And the honest framing: **this makes the wait visible rather than making it shorter.** A visible position in a queue is enormously better than a spinner and a timeout, which is a product truth as much as an engineering one.

Fair admission. First-come-first-served rewards fast networks and bots. **Randomised admission from everyone who arrived within a short window is fairer,** and several real systems do exactly this.

And rejecting early. When there are more people admitted than seats remaining, **tell the rest immediately.** Munuswamy's "sold out" before it was sold out. **The number cannot change; the time to find out can.**

Then the seat map, which is a read problem with an unusual property.

Everybody wants to see the same thing at the same moment, and it changes constantly.

Serve the map from a cache, and accept that it is a few seconds stale. A seat shown as available that is actually held is **not a correctness problem** — the atomic hold rejects the attempt and the client refreshes. **Trying to make the map real-time for a hundred thousand viewers is the mistake,** because the authoritative check happens at hold time anyway.

What is worth doing is pushing updates rather than polling. One WebSocket broadcast per seat change to everyone viewing that show is far cheaper than a hundred thousand clients polling every two seconds.

Then payment, which inherits yesterday's design.

Hold, then authorise, then confirm the hold is still valid, then capture. The hold TTL must comfortably exceed the payment time, **and if the hold expires mid-payment — which happens — the honest response is to void the authorisation and tell the customer,** not to sell a seat that has been taken.

And bots, which are a first-class problem here rather than a nuisance.

Tickets have resale value, so there is real money in automating purchases. **The defences are layered:** rate limits per account and per address, a limit on tickets per person, requiring account age or verification, device fingerprinting, and a challenge at the waiting-room boundary rather than at checkout.

And none of them work perfectly, which is worth saying. **The realistic goal is to make automation expensive enough that it does not dominate,** not to eliminate it.

Finally: the thing that is genuinely unresolvable.

Overselling is unacceptable and underselling is invisible. A held seat that is never paid for and expires thirty seconds after the show sells out **is a seat that went empty while somebody wanted it.**

Short TTLs reduce that waste and break more slow payments. Long TTLs are kind to slow customers and hold seats out of circulation. There is no correct answer, **and the honest design states the trade rather than pretending a number is derived.**

4 The picture

Why a counter does not work:

ORDINARY STOCK

"997 units available"
→ decrement to 996
→ any unit will do

one atomic counter
contention on ONE row

TICKETS

seat map:
A1 A2 A3 [A4] A5 ...
B1 [B2] [B3] B4 ...
^^^ specific, chosen, unique

ONE LOCK PER SEAT
contention on the FEW GOOD SEATS
(the front rows go first –
the load is not uniform)

The three states and the pin:

```

                select a seat
[ AVAILABLE ] -----> [ HELD ] (TTL 5 minutes)
    ^                       |
    |                       | pay
    | TTL expires          v
+-----[ SOLD ]
    (tab closed, battery
    died, network dropped)
```

THE TTL IS THE ENTIRE SAFETY MECHANISM.
Nobody sends "I have given up". The browser just stops existing.
A hold without an expiry is a seat lost forever.

The hold must be atomic:

BROKEN – check then write

```
A: is H14 free? → yes
B: is H14 free? → yes      ← both see free
A: mark H14 held
B: mark H14 held          ← BOTH HOLD IT
→ two people pay for one seat
```

CORRECT – one operation

```
SET seat:show42:H14 <session> NX EX 300
A: OK      → A holds it
B: nil     → B is told to pick another

or: UPDATE seats SET status='held', ...
      WHERE seat='H14' AND status='available'
      then check rows_affected = 1
```

The condition and the write happen TOGETHER. That is the property.

Group bookings, and the deadlock:

```
session A wants  H14 H15 H16 H17
session B wants  H16 H17 H18 H19
```

ACQUIRED IN ARRIVAL ORDER:

```
A locks H14, H15      B locks H18, H19
A wants H16 ...       B wants H17 ...
A waits for B         B waits for A      ← DEADLOCK
```

ACQUIRED IN SORTED ORDER (always ascending by seat):

```
A locks H14, H15, H16, H17
B tries H16 → fails immediately → releases H18, H19, retries
```

→ one fails fast instead of both waiting forever.

A fixed global order is the standard fix, and it is one `sorted()`.

The spike, and the three layers:

10:00:00 – 100,000 people, 1,000 seats

WITHOUT A WAITING ROOM:

```
100,000 requests → the seat map → the lock store
the good seats are a few dozen keys
→ everything queues, everything times out, everyone retries
→ the retries are worse than the original load
```

WITH THE THREE LAYERS:

```
[ 100,000 arrive ]
|
1. WAITING ROOM: issue a position + token, admit ~1,000 at a time
|               → the booking system sees a manageable rate
v
2. FAIR ADMISSION: randomise within the arrival window
|               → not a reward for a fast network or a bot
v
3. REJECT EARLY: once admitted > seats remaining, tell the rest NOW
|               → Munuswamy walking down the line
v
[ ~2,000 reach the seat map ]
```

"The number I cannot change. What I can change is how long somebody waits to find out."

The seat map: stale is fine, and why:

100,000 people watching the same map, changing every millisecond

REAL-TIME FOR EVERYONE: impossible, and unnecessary

CACHED, ~2 SECONDS OLD: fine

because a seat shown as free that is actually held is NOT a correctness problem:

- the client attempts the hold
- the ATOMIC hold rejects it
- the client refreshes and picks another

the authoritative check happens at HOLD time, not at DISPLAY time.

AND: push changes over a WebSocket rather than letting 100,000 clients poll every 2 seconds

polling: $100,000 / 2s = 50,000$ requests/second

pushing: ~50 seat changes/second, broadcast

→ 1,000x fewer operations

The TTL trade, which has no correct answer:

SHORT TTL (2 minutes)

LONG TTL (10 minutes)

seats return to sale fast

slow customers succeed

more failed payments

seats sit unavailable

less waste at sell-out

more waste at sell-out

AND THE ASYMMETRY:

overselling → visible, unacceptable, unrecoverable

underselling → invisible: an empty seat somebody wanted

→ there is no derived answer. State the trade; pick a number; measure the failed-payment rate and the expired-hold rate, and move it.

5 How it actually works

Holding a seat

```
HOLD_SECONDS = 300

def hold_seat(show_id: int, seat: str, session_id: str) → bool:
    """Atomic. SET NX is the check and the write in one operation."""
    return bool(redis.set(f"hold:{show_id}:{seat}", session_id,
                          nx=True, ex=HOLD_SECONDS))

def release_seat(show_id: int, seat: str, session_id: str) → bool:
    """Only the holder may release it – check-and-delete must be atomic."""
    script = """
        if redis.call('GET', KEYS[1]) = ARGV[1] then
            return redis.call('DEL', KEYS[1])
        else
            return 0
        end
    """
    return bool(redis.eval(script, 1, f"hold:{show_id}:{seat}", session_id))
```

`nx=True` is the whole correctness argument: the key is set only if it does not exist, and that check happens inside Redis rather than in your code.

The release must be a Lua script, not a `GET` followed by a `DEL`. Between the two, the hold could expire and somebody else could acquire it — and your `DEL` would then release *their* hold. That is the classic distributed-lock bug from [day 127](#), and it is why the value is a session id rather than a flag.

Group bookings, in a fixed order

```
def hold_seats(show_id: int, seats: list[str], session_id: str) → list[str]:
    """All or nothing. Acquire in SORTED order to make deadlock impossible."""
    acquired: list[str] = []
    for seat in sorted(seats):
        # the fixed global order
        if hold_seat(show_id, seat, session_id):
            acquired.append(seat)
        else:
            for held in acquired:
                # roll back cleanly
                release_seat(show_id, held, session_id)
            return []
    return acquired
```

`sorted(seats)` is the deadlock prevention, and it is one word. Two sessions contending for overlapping seats now always contend in the same sequence, so one fails immediately rather than both waiting.

And the rollback matters: a partial hold that is never released takes seats out of sale for the whole TTL, **which at sell-out is seats that go empty.**

The waiting room

```
def enter_queue(show_id: int, user_id: int) → dict:
    token = str(uuid.uuid4())
    position = redis.rpush(f"queue:{show_id}", token)
    redis.setex(f"token:{token}", 3600, user_id)
    return {"token": token, "position": position,
            "estimated_wait_seconds": position // ADMIT_PER_SECOND}

def admit_batch(show_id: int, count: int) → None:
    """Runs continuously. Admits a controlled number per second."""
    tokens = redis.lpop(f"queue:{show_id}", count)
    for token in tokens or []:
        redis.setex(f"admitted:{token}", 900, "1")      # 15 minutes to book
        notifier.push(token, {"admitted": True})
```

Returning the position is the product decision that matters most. A number that counts down is qualitatively different from a spinner, even when the wait is identical — and it is the difference between a user who waits and one who reloads twenty times, adding to the load.

And `admit_batch` running at a fixed rate is what protects everything downstream — the booking system never sees more than it can handle, **regardless of how many people arrived.**

Fair admission

```
def admit_fairly(show_id: int, window_seconds: int = 60, count: int = 1000):
    """Randomise within an arrival window: not a race, and not a bot reward."""
    arrivals = redis.zrangebyscore(
        f"arrivals:{show_id}", time.time() - window_seconds, time.time())
    chosen = random.sample(arrivals, min(count, len(arrivals)))
    for token in chosen:
        redis.setex(f"admitted:{token}", 900, "1")
        redis.zrem(f"arrivals:{show_id}", token)
```

Random sampling within a window removes the reward for a fast connection, which is what pure first-come-first-served gives — **and which is exactly the advantage a bot has.**

It is not perfectly fair either, and it is fairer than a race, and **saying which unfairness you chose is better than pretending there is a fair option.**

The seat map, cached and pushed

```
def seat_map(show_id: int) → dict:
    cached = redis.get(f"map:{show_id}")
    if cached:
        return json.loads(cached)  # up to 2 seconds stale
    seats = db.query("SELECT seat, status FROM seats WHERE show_id = %s", show_id)
    payload = {"seats": {s.seat: s.status for s in seats}, "at": time.time()}
    redis.setex(f"map:{show_id}", 2, json.dumps(payload))
    return payload

def on_seat_changed(show_id: int, seat: str, status: str) → None:
    """One broadcast per change beats 100,000 clients polling."""
    bus.publish(f"show:{show_id}", {"seat": seat, "status": status})
```

A two-second cache is not a compromise on correctness, because **the authoritative check is the atomic hold**. A stale map produces a failed hold and a refresh, which is a fine outcome.

And the push-versus-poll arithmetic is stark: fifty seat changes a second broadcast, against a hundred thousand clients polling every two seconds. **A thousandfold difference for the same information**.

The booking flow

```
def book(show_id: int, seats: list[str], session_id: str,
        payment_token: str, idempotency_key: str) → dict:
    if not redis.get(f"admitted:{session_id}"):
        return {"error": "not_admitted"}, 403

    held = hold_seats(show_id, seats, session_id)
    if not held:
        return {"error": "seats_unavailable"}, 409

    auth = payments.authorise(price_of(show_id, held), payment_token,
                             idempotency_key=f"auth:{idempotency_key}")
    if not auth.ok:
        for seat in held:
            release_seat(show_id, seat, session_id)
        return {"error": "payment_declined"}, 402

    if not still_held(show_id, held, session_id):      # the TTL may have lapsed
        payments.void(auth.id)                       # free, invisible
        return {"error": "hold_expired"}, 409

    with db.transaction():
        db.execute("""UPDATE seats SET status='sold', booking_id=%(b)s
                     WHERE show_id=%(s)s AND seat = ANY(%(seats)s)
                     AND status <> 'sold'""",
                  {"b": booking_id, "s": show_id, "seats": held})
        booking = booking_store.create(show_id, held, session_id, idempotency_key)

    payments.capture(auth.id, idempotency_key=f"capture:{idempotency_key}")
    for seat in held:
        on_seat_changed(show_id, seat, "sold")
    return {"booking_id": booking.id, "seats": held}
```

still_held after authorisation is the check people omit. Payment takes seconds and the hold can lapse in that window — rarely, and it happens. Voiding the authorisation and telling the customer is the honest response; selling a seat somebody else now holds is not.

And the database write is the durable record, with Redis holding the *hold* and Postgres holding the *sale* — two stores with two different jobs, and the `status <> 'sold'` guard is the last line of defence.

Releasing expired holds

```
def sweep_expired(show_id: int) → int:
    """Redis expires the keys; this reconciles the durable seat map."""
    released = 0
    for seat in db.seats_in_state(show_id, "held"):
        if not redis.exists(f"hold:{show_id}:{seat.seat}"):
            db.execute("""UPDATE seats SET status='available'
                          WHERE show_id=%(s)s AND seat=%(seat)s
                          AND status='held'""",
                       {"s": show_id, "seat": seat.seat})
            on_seat_changed(show_id, seat.seat, "available")
            released += 1
    return released
```

Redis expires the hold automatically; the durable seat map does not know. This job reconciles them, and without it the database shows seats as held forever while Redis has released them — **which is underselling that nobody notices.**

Bot defences, layered

```
def can_attempt(user_id: int, show_id: int, ip: str) → tuple[bool, str]:
    if account_age_days(user_id) < 1:
        return False, "account_too_new"
    if bookings_for_show(user_id, show_id) ≥ MAX_PER_PERSON:
        return False, "limit_reached"
    if not rate_limiter.allow(f"user:{user_id}", limit=5, per=60):
        return False, "too_many_attempts"
    if not rate_limiter.allow(f"ip:{ip}", limit=20, per=60):
        return False, "too_many_from_address"
    return True, ""
```

Layered, because none of them works alone. Account age defeats trivial scripts; **per-person limits are the most effective and the most easily defeated by many accounts;** IP limits are crude because a whole office shares one address.

And the challenge belongs at the waiting-room boundary rather than at check-out, because making somebody solve a puzzle after they have chosen their seats is the worst possible moment.

The real systems

Redis	holds (SET NX EX), the queue, the cached seat map
PostgreSQL	the durable seat map, bookings, payments
Kafka / NATS	seat-change broadcasts to connected clients
WebSockets	pushing map updates instead of polling
Ticketmaster,	all use a waiting room ("Smart Queue"), all use
BookMyShow,	short holds, all limit tickets per person
IRCTC	Tatkal is the extreme case: a known-in-advance
	spike of millions in one second
Stripe / etc.	payments, with authorise/capture as yesterday

IRCTC's Tatkal is worth naming, because it is the largest predictable booking spike in the world and it illustrates the point exactly: **the time is published in advance, so everything about the system can be prepared for it.**

6 The numbers

Ordinary load against the spike, which is the whole story.

ORDINARY DAY

~1,000,000 bookings/day = ~12/second average
seat map views: ~10,000,000/day = ~120/second
→ a small system

SALE OPENING (a popular event)

100,000 people in the first 10 seconds
= 10,000 requests/second, against ~12/second normally
→ 800x the ordinary rate, for 60 seconds

AND IT IS PREDICTABLE: the sale opens at a published time.

→ pre-scale, pre-warm caches, pre-shard, staff the incident channel.
A predictable spike is a completely different problem from a surprise one.

Contention, which is where a naive design dies.

1,000 seats, 100,000 people

WITHOUT a waiting room:

- every request reads the seat map and attempts a hold
- the GOOD seats are maybe 50 keys
- ~2,000 attempts per good seat, in one second
- Redis handles the ops; the RETRIES are what kills it:
99% fail, every one retries, the load doubles

WITH a waiting room admitting 1,000 at a time:

- ~1,000 concurrent seat-map views
- ~1,000 hold attempts against 1,000 seats
- contention is roughly 1:1. Comfortable.

→ the waiting room is not a nicety. It is what makes the arithmetic work.

Hold storage.

one hold: key ~40 bytes + session id ~40 = ~100 bytes with overhead

peak concurrent holds for a big event: ~5,000

$5,000 \times 100 \text{ B} = 500 \text{ KB}$

→ trivial. The holds are not a storage problem; they are a CONTENTION problem.

Seat map: push against poll.

100,000 clients watching one show

POLLING every 2 seconds:

- $100,000 / 2 = 50,000$ requests/second
- each returns ~20 KB of seat map
- = 1 GB/second of egress ← for one show

PUSHING changes over WebSockets:

- ~50 seat changes/second during the rush
- broadcast to 100,000 connections
- each message ~50 bytes
- = $50 \times 100,000 \times 50 \text{ B} = 250 \text{ MB/second}$

and with DELTA batching (one message per 500 ms with all changes):
= $2 \times 100,000 \times 500 \text{ B} = 100 \text{ MB/second}$

→ 10x better, and the connection count is the real cost:
100,000 WebSockets at ~10 KB each = 1 GB of RAM. One machine.

The TTL trade, quantified.

1,000 seats, 5-minute holds, ~20% of holds abandoned

with a 5-minute TTL:

200 abandoned holds x 5 minutes = 1,000 seat-minutes unavailable
during a 10-minute sell-out, that is 100 seats' worth of time
→ some of those seats go EMPTY

with a 2-minute TTL:

200 x 2 = 400 seat-minutes
but the payment failure rate rises – slow customers, 3-D Secure
challenges, bank apps – perhaps from 5% to 15%

→ 10% more failed payments to recover 600 seat-minutes.

There is no derived answer. Measure both rates and move the number.

Payment latency against the hold.

authorise: 500-2,000 ms

3-D Secure: 10-60 SECONDS (the bank's app, the customer's phone)

→ a 2-minute hold is genuinely tight when 3-D Secure is involved
→ and it is why "hold expired mid-payment" is a real code path
rather than a theoretical one

Database load at the moment of sale.

1,000 seats sold in 60 seconds = ~17 sales/second

each sale: one seat UPDATE + one booking INSERT + payment rows

= ~70 writes/second

→ TRIVIAL. The durable write path is never the bottleneck.

The bottleneck is entirely:

1. the number of requests reaching the system (waiting room)
2. contention on the popular seats (atomic holds)
3. serving the seat map to viewers (cache + push)

That is worth stating plainly: the part that must be durable and correct is tiny, exactly as in payments — and everything expensive is upstream of it.

Bot impact, roughly:

for a high-demand event, industry estimates put automated purchasing at 20-40% of attempts

per-person limits are the most effective single defence:

limit 4 per person → a bot needs 250 accounts for 1,000 tickets

account age $\geq$ 1 day → those accounts must exist in advance

→ the goal is to make automation EXPENSIVE, not impossible.
Claiming to eliminate it is not credible.

7 The trade-offs

Hold TTL: short against long. Short returns seats to sale quickly and **breaks more slow payments** — especially with 3-D Secure, which can take a minute of the customer's time in their bank's app. Long is kind to slow customers and **holds seats out of circulation during exactly the window when they are scarce. Overselling is unacceptable and underselling is invisible**, so the pressure is all one way and the number drifts long unless somebody measures the expired-hold rate.

Waiting room against open access. A waiting room protects everything downstream and **makes the wait explicit**, which users tolerate far better than a spinner. It **also adds a component that must itself survive the spike**, and if it fails the whole sale fails. A queue that is the single point of failure for a high-profile sale needs more care than the booking system it protects.

First-come-first-served against randomised admission. FCFS is what users expect and **rewards fast networks, proximity to the data centre, and bots**. Randomised admission within a window is fairer and **feels arbitrary** — a user who clicked at exactly ten o'clock and lost to someone who clicked at ten o'clock and three seconds finds it hard to accept. **Neither is fair; choose which unfairness and say so.**

Cached seat map against real-time. Cached is the only thing that scales and **shows seats that are already gone**, producing failed hold attempts. That is acceptable **because the atomic hold is the authority**, and trying to make the display authoritative is the mistake. **The cost is user frustration**, mitigated by pushing changes rather than lengthening the cache.

Per-person limits against revenue. Limits are the most effective bot defence and **turn away legitimate group bookings** — a family of six against a limit of four. Every exception mechanism is also a bot loophole, which is why the limits tend to be blunt.

Holds in Redis against holds in the database. Redis gives atomic operations at the required rate and TTLs for free, **and it is a second source of truth that can disagree with the durable seat map** — hence the reconciliation sweep. **Postgres-only is simpler and correct and serialises on hot rows**, which at ten thousand requests a second is the thing you were trying to avoid.

When would I not build this? When the event is small and the demand is not spiky — a hundred-seat theatre selling over three weeks needs a table and a transaction, and every mechanism here is overhead. **When a platform exists:** Ticketmaster, BookMyShow and Eventbrite have solved the spike, the bots and the payments, **and the waiting room alone is a serious piece of engineering. Building your own is justified by volume, by unusual seating rules, or by needing the customer relationship** — and that last one is a business reason, which is fine as long as it is stated as one.

8 In the interview

How it gets asked

- *“Design Ticketmaster.”* or *“Design a seat booking system.”* — the standard prompts.
- *“Two people click the same seat at the same moment. What happens?”* — the central question.
- *“A hundred thousand people arrive when the sale opens. What breaks?”*
- *“How long do you hold a seat, and why?”*
- *“How do you show the seat map to everyone?”*
- *“How do you stop bots?”*

“The thing that makes this different from ordinary inventory is that **every unit is unique and non-substitutable**. You cannot ship an equivalent seat. **So there is no oversell recovery** — with a warehouse you apologise and back-order; **with a seat there is one, and somebody is standing in the aisle**.

And the contention is extreme and predictable. A hundred thousand people for a thousand seats, at a time that is published in advance. **Predictability is a gift** — I can pre-scale, pre-warm and queue, which is not available for a spike that surprises me.

The core mechanism is a hold with a TTL. A seat has three states: available, held, sold. **Selecting holds it, paying converts it, expiry releases it**.

The hold must be atomic — two people clicking the same seat is the entire problem — **so it is a `SET NX` in Redis or a conditional `UPDATE` checking the affected row count**. The condition and the write happen together; a check-then-write loses the race.

And the TTL is the whole safety mechanism, because nobody sends ‘I have given up’. The tab closes, the battery dies. **A hold without an expiry is a seat lost forever**.

Now the part I would spend most time on: the spike.

A hundred thousand requests for a thousand seats means ninety-nine percent of them must never reach the seat map. Every one that does is contention on a resource that cannot serve it — **and worse, they all fail and all retry, so the load doubles**.

Three layers. A **waiting room** that gives everyone a position and admits perhaps a thousand at a time. **Fair admission** — randomised within an arrival window, rather than a race that rewards fast networks and bots. **And rejecting early**: once more people are admitted than there are seats, tell the rest immediately.

The number of disappointed people is fixed. What the design controls is **how long they wait to find out** — and a visible queue position is enormously better than a spinner and a timeout.

One thing I would flag early: the seat map is cached and stale, and that is correct. A seat shown as free that is actually held is not a correctness problem — **the atomic hold rejects it and the client refreshes**. The authoritative check is at hold time, not at display time.”

The follow-ups

“Two people click the same seat at the same moment. What happens?”

“One gets it and the other is told immediately, **and the mechanism is a single atomic operation** — that single-operation property is the whole answer.

The broken version is a check followed by a write. Is H14 free? Yes. Mark it held. **Two requests overlapping by a few milliseconds both see ‘free’, both mark it, and two people pay for one seat.**

And unlike a warehouse, there is no recovery. You cannot ship an equivalent seat. Somebody arrives and somebody else is in their chair.

The fix is `SET key value NX EX 300`. The key is set only if it does not exist, **and the check happens inside Redis rather than in my code**, so there is no window between them. The second request gets nil and is told to pick another seat.

The Postgres equivalent is a conditional update — `SET status='held' WHERE seat=? AND status='available'` — **and then checking that exactly one row was affected.** Same property: the condition and the write are one operation.

Two details I would add, because they are where the real bugs are.

The release must also be atomic, and it must check ownership. A `GET` followed by a `DEL` is broken: between them, the hold can expire and somebody else can acquire it, **and your `DEL` then releases their hold.** So the value stored is a session id and the release is a Lua script that deletes only if the value matches.

And group bookings can deadlock. Four adjacent seats is four locks, and two sessions wanting overlapping sets can each hold half and wait for the other. **The fix is to always acquire in sorted order** — one `sorted()` call — **so two contending sessions always contend in the same sequence and one fails immediately** rather than both waiting.

And a partial acquisition must roll back cleanly, or seats sit unavailable for the whole TTL at exactly the moment they are scarce.”

“A hundred thousand people arrive when the sale opens. What breaks?”

“Everything, if they all reach the booking system — **and the failure is worse than it first appears because of retries.**

The direct problem: a hundred thousand requests want a thousand seats, and the *good* seats are maybe fifty keys. That is two thousand attempts per popular seat in one second. **Redis can do the operations — what it cannot absorb is that ninety-nine percent fail, every one of those users retries, and the load doubles.**

So the design principle is that almost none of them should reach the seat map at all.

Layer one: a waiting room. Arrivals get a token and a position, and a background process admits a controlled number per second. **A hundred thousand people become a thousand at a time,** and the booking system never sees more than it can handle regardless of how many arrived.

And returning the position is the design decision that matters most, and it is a product one. **A number that counts down is qualitatively different from a spinner,** even when the wait is identical — and a user who can see progress does not reload twenty times and add to the load.

Layer two: fair admission. Pure first-come-first-served rewards a fast network, physical proximity to the data centre, and automation. **Randomised admission from everyone who arrived within a short window removes that advantage.** It is not perfectly fair either — **I would say which unfairness I chose rather than claim a fair option exists.**

Layer three: reject early. Once more people have been admitted than there are seats remaining, **tell everyone else immediately.** The number of disappointed people is fixed by the seat count; **what I control is whether they find out in fifty milliseconds or after thirty seconds of timeouts.**

And because the spike is predictable — the sale time is published — I can prepare: pre-scale the fleet, pre-warm the seat-map cache, pre-establish the Web-Socket capacity. **A predictable spike is a completely different problem from a surprise one,** and I would take full advantage of that.

One risk I would flag: the waiting room is now a single point of failure for a high-profile sale. If it falls over, the whole event fails publicly. **It needs more operational care than the booking system it protects,** which is slightly uncomfortable and is true.”

“How long do you hold a seat, and why?”

“There is no derived answer, and I think the honest thing is to say that and then explain the trade — because the pressure runs one way and the failure it causes is invisible.

Too short and you break slow payments. The customer is entering card details, or their bank has sent them to an app for 3-D Secure, **which routinely takes thirty to sixty seconds of the customer’s own time.** A two-minute hold is genuinely tight in that case.

Too long and seats sit out of circulation. With a five-minute hold and twenty percent abandonment, a thousand-seat event accumulates a thousand seat-minutes of unavailability — **and during a ten-minute sell-out, that is seats that go empty while people were still trying to buy.**

And here is the asymmetry that makes this hard. Overselling is visible, unacceptable and unrecoverable. Underselling is invisible — nobody ever sees the empty seat that somebody wanted.

So the incentive is entirely towards longer holds, and the number drifts up unless somebody actively measures the expired-hold rate against the failed-payment rate.

What I would build is: five minutes as a starting point, and both metrics on a dashboard — the fraction of holds that expire unpaid, and the fraction of payments that fail because the hold lapsed. **Move the number based on those, and expect it to differ by event type.**

Two mechanisms that soften it. Extend the hold when the payment is genuinely in flight — once the authorisation is submitted, the customer is not idle, so the clock can be paused. **And show the customer the countdown,** which both reduces abandonment and makes the expiry feel fair rather than arbitrary.

And the code path for ‘the hold expired mid-payment’ has to exist. It is rare and it happens, **and the honest response is to void the authorisation — which is free — and tell the customer,** rather than sell a seat that somebody else now holds.”

The model answer

“Design a ticket booking system for a large events platform: ten thousand events, seat-level selection, and sales that open at a published time with a hundred thousand people waiting.”

“Let me name what makes this different from ordinary inventory, because it determines everything.

Every unit is unique and non-substitutable, so there is no oversell recovery. And the contention is both extreme and predictable — a hundred thousand people at a published time. **Predictability is the one advantage I have and I would use all of it.**

Ordinary load is tiny: a million bookings a day is twelve a second. **The sale opening is ten thousand requests a second — eight hundred times the normal rate, for about a minute.** So this is a system designed entirely around sixty seconds a day.

The core mechanism is a per-seat hold with a TTL, in Redis via `SET NX EX`. **Atomic**, because two people clicking H14 is the whole problem and a check-then-write loses it. **The value is a session id and the release is a Lua script**, because a `GET` then `DEL` can release somebody else's hold after an expiry. **Group bookings acquire in sorted seat order** to make deadlock impossible, and roll back cleanly on partial failure.

The durable seat map and the bookings are in Postgres, with a `status <> 'sold'` guard on the final update as a last line of defence. **And a sweep job reconciles Redis expiries into the durable map**, without which the database shows seats held forever — **underselling that nobody notices.**

Now the spike, which is most of the design.

A waiting room in front of everything. Arrivals get a token and a **visible position**; a background process admits about a thousand at a time. **The booking system never sees more than it can handle.** And the position is not decoration — **a countdown is qualitatively different from a spinner**, and it stops users reloading and doubling the load.

Admission is randomised within a short arrival window rather than first-come-first-served, because FCFS rewards fast networks and bots. **It is a different unfairness rather than fairness, and I would say so.**

And rejection is early and explicit. Once admissions exceed remaining seats, everyone else is told immediately. **The number of disappointed people is fixed; the time they wait to find out is not.**

The seat map is cached for about two seconds and pushed over WebSockets. Stale is fine, **because the authoritative check is the atomic hold** — a stale map produces a rejected attempt and a refresh. **And pushing beats polling by about**

a thousand times: fifty seat changes a second broadcast, against a hundred thousand clients polling every two seconds, which would be a gigabyte a second of egress for one show.

Payment inherits the standard design: hold, authorise, **re-check the hold is still valid**, capture. That re-check is the step people omit, **and 3-D Secure can take a minute of the customer's time**, so hold expiry mid-payment is a real path rather than a theoretical one. **Void and tell them — never sell a seat twice.**

Bots are a first-class problem here because tickets have resale value. Layered defences: account age, per-person limits, rate limits per account and address, and a challenge **at the waiting-room boundary rather than at checkout** — making somebody solve a puzzle after choosing their seats is the worst possible moment. **And the goal is to make automation expensive, not to eliminate it;** claiming otherwise is not credible.

Two closing points.

The durable write path is trivial — a thousand seats in sixty seconds is seventeen sales a second. **All the difficulty is upstream: request admission, seat contention, and serving the map.** That is worth saying, because it is where the engineering effort should go.

And the hold TTL has no correct value. Short breaks slow payments; long wastes seats. **Overselling is visible and underselling is invisible**, so the number drifts long unless both rates are measured. **I would start at five minutes, put both metrics on a dashboard, and treat it as a number to tune rather than one to derive.**

9 Recall card

Every unit is **UNIQUE and non-substitutable**, so there is **NO oversell recovery** — you cannot ship an equivalent seat. **And the contention is extreme AND predictable:** 100,000 people at a published time, $\sim 800 \times$ the ordinary rate for sixty seconds. **Predictability is the advantage — pre-scale, pre-warm, pre-queue.**

Three seat states — available / held / sold — and the hold must be ATOMIC: SET NX EX or a conditional UPDATE with a row-count check. **The TTL is the entire safety mechanism, because nobody sends “I gave up”.** Release must be a Lua

script checking a session id — `GET` then `DEL` can release somebody else's hold after an expiry. **Group bookings acquire in SORTED order** so contention is always in the same sequence and one side fails fast instead of deadlocking.

99% of requests must never reach the seat map — not because of the operations but because **they all fail, all retry, and double the load**. Three layers: **a waiting room with a VISIBLE position** (a countdown is qualitatively different from a spinner), **randomised admission within a window** (FCFS rewards fast networks and bots — it is a different unfairness, not fairness), and **rejecting early**: the number of disappointed people is fixed, the time they wait to find out is not.

The seat map is cached (~2 s) and stale, and that is CORRECT — the atomic hold is the authority, so a stale map produces a rejected attempt and a refresh. **Push over WebSockets, never poll**: ~50 changes/second broadcast against 100,000 clients polling every 2 s (1 GB/s for one show).

Payment: hold → authorise → RE-CHECK the hold → capture. 3-D Secure takes 30–60 seconds of the customer's time, so **“hold expired mid-payment” is a real path** — void (free) and tell them.

The hold TTL has no derived answer. Short breaks slow payments; long wastes seats. **Overselling is visible and underselling is invisible**, so the number drifts long unless you measure the expired-hold rate against the failed-payment rate. **And the durable write path is trivial (~17 sales/second) — all the difficulty is upstream**.

DSA topic: Proving a greedy choice, simply **System design topic:** Design a ticket booking system

Code these, in this order

One rule for the whole set: **write the exchange argument as a docstring before you write the function body**. If you cannot write it, you do not know the algorithm is correct — and you find that out in two minutes rather than in an interview.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Minimum Number of Arrows to Burst Balloons	LeetCode 452 (Medium)	The same exchange as activity selection, in disguise.
2	Queue Reconstruction by Height	LeetCode 406 (Medium)	A greedy whose argument is genuinely subtle.
3	Task Scheduler	LeetCode 621 (Medium)	Greedy plus a formula, and why the formula is right.
4	Course Schedule III	LeetCode 630 (Hard)	An exchange with an undo — greedy that reconsiders.
5	Minimum Cost to Connect Sticks	LeetCode 1167 (Medium)	Huffman, under a different name.
6	Job Sequencing with Deadlines	classic — write it yourself	The full argument, including where it breaks.

On all six, write the argument first

For each, before coding: **state the greedy rule, then write the exchange argument as three sentences** — take any optimal solution, swap the greedy choice in, show it is still legal and no worse.

Record which ones you could not complete. Those are the ones where you were about to guess.

On problem 1, notice it is the same problem

Before solving it, say in one sentence why bursting balloons with arrows is activity selection. **Then check whether your exchange argument transfers word for word.** It should.

On problem 2, find the argument that is actually hard

The greedy here — sort by height descending, then insert each person at the index of their `k` value — **is correct and the reason is not obvious**. Spend five minutes trying to construct the exchange argument before looking anything up. **Write down where you got stuck**, because that is the point of the exercise.

On problem 3, prove the formula rather than the loop

The answer is a closed form based on the most frequent task. **Say why that formula is a lower bound**, and then say why it is achievable. **Two separate arguments** — and most explanations give only the first.

On problem 4, find the undo

This greedy takes courses and then **removes one it already took** when a better option appears. **Say what makes that still a greedy algorithm**, and construct the exchange argument for the removal step specifically.

On problem 6, break it deliberately

Solve it for unit-duration jobs. **Then change the problem so jobs take different amounts of time** and try to construct the same argument. **Find the exact sentence that stops working**, and build the counter-example it describes.

Then the argument-completion drill

For each of these, write the exchange argument or say why it fails:

1. Activity selection by end time.
2. Activity selection by start time.
3. Fractional knapsack by ratio.
4. 0/1 knapsack by ratio.
5. Shortest job first.
6. Longest job first.
7. Huffman: merge the two rarest.
8. Coin change: take the largest coin.

Four have arguments and four have counter-examples. For the four that fail, **say where the argument breaks before constructing the input**, and check that the input matches the failure.

Then the empirical drill

Write a brute force for activity selection at $n \leq 8$. For every random instance where the optimal solution does not contain the earliest finisher, **try every possible single swap and record whether one works.**

Run four hundred instances. Any failure means the exchange argument is wrong.

The claim drill

1. State exactly what greedy needs to establish.
2. Say why it is “an” optimal solution rather than “the”.
3. Say why the weaker claim is easier to prove.
4. Say what makes the induction work.

The exchange drill

1. Give the three steps.
2. Name the two halves of step three.
3. Say which half is usually omitted.
4. Give the argument for activity selection, in full.
5. Compress it to one sentence.

The adjacent-swap drill

1. Say why it is the easiest form.
2. Give it for shortest job first.
3. Say exactly what the saving equals.
4. Say when you should look for this form first.

The stays-ahead drill

1. State it in two sentences.
2. Give it for activity selection.
3. Say what it needs that exchange does not.
4. Name a problem where it does not apply, and why.

The failed-proof drill

1. Say what a failed proof gives you.
2. Give the exact sentence where 0/1 knapsack breaks.
3. Build the counter-example from that sentence.

4. Say why the fractional version survives the same argument.
5. Say what changes in the problem statement between them.

The what-to-say drill

1. Give the thirty-second, three-sentence version.
 2. Say what you say when you cannot fill in the blanks.
 3. Say why that is a good answer rather than a concession.
 4. Say what over-formalising looks like and why it costs you.
-

The difference drill

1. Give the two properties that make tickets unlike ordinary stock.
2. Say why there is no oversell recovery.
3. Say why a counter does not work.
4. Say what predictability buys you.

The hold drill

1. Give the three seat states.
2. Say why the hold must be atomic, and give both correct forms.
3. Say what the TTL prevents and what message never arrives.
4. Say why release must be a script, and what bug it prevents.
5. Describe the group-booking deadlock and the one-word fix.
6. Say why a partial acquisition must roll back.

The spike drill

1. Compute the ordinary rate and the sale-opening rate.
2. Say why retries make the failure worse than it first appears.
3. Give the three layers in order.
4. Say why the visible position matters, and what it prevents.
5. Say what randomised admission removes, and what it costs.
6. Say what “reject early” controls and what it cannot.
7. Name the risk the waiting room introduces.

The seat-map drill

1. Say why a stale map is correct rather than a compromise.
2. Say where the authoritative check happens.

3. Compute polling versus pushing, both figures.
4. Say what the real cost of pushing is.

The TTL drill

1. Give both directions of the trade.
2. Say what 3-D Secure does to the arithmetic.
3. State the asymmetry between overselling and underselling.
4. Say which way the number drifts, and why.
5. Name the two metrics you would put on a dashboard.
6. Give two mechanisms that soften the trade.

The bots drill

1. Say why bots are first-class here.
2. Name five defences.
3. Say which is most effective and how it is defeated.
4. Say where the challenge belongs and why.
5. State the realistic goal honestly.

The sizing drill

1. Compute the durable write rate at sell-out.
 2. Say where the difficulty actually is.
 3. Compute the hold storage.
 4. Say what the WebSocket connection count costs.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Why does your greedy choice work?* The claim in its weak form, the three-step exchange, both halves of step three, and the activity-selection argument compressed to one sentence.
2. *What if you cannot prove it?* What a failed proof tells you, the exact sentence where 0/1 knapsack breaks, the counter-example it describes, and what you say to the interviewer.

3. *Design a ticket booking system.* Why tickets are unlike stock, the atomic hold with its TTL, the three spike layers, why a stale seat map is correct, and the TTL trade with its asymmetry.

Before you move on

- ☐ I can state exactly what greedy needs to establish.
- ☐ I know why “an” optimal solution rather than “the”.
- ☐ I can give the three-step exchange argument.
- ☐ I always show both halves: still legal, and no worse.
- ☐ I know which half people omit.
- ☐ I can give the activity-selection argument in full and in one sentence.
- ☐ I look for the adjacent-swap form first, and know why.
- ☐ I can give the shortest-job-first argument and its exact saving.
- ☐ I can state “greedy stays ahead” and say when to use it.
- ☐ I know a failed proof names the counter-example.
- ☐ I can give the sentence where 0/1 knapsack breaks.
- ☐ I can build the counter-example from that sentence.
- ☐ I know why the fractional version survives.
- ☐ I can give the thirty-second version, unprompted.
- ☐ I know what to say when I cannot complete the argument.
- ☐ I know why that is a good answer.
- ☐ I wrote the argument as a docstring for all six problems.
- ☐ I can say what makes tickets unlike ordinary inventory.
- ☐ I know why there is no oversell recovery.
- ☐ I can give the three seat states and both atomic hold forms.
- ☐ I know why release must be a script.
- ☐ I know the group-booking deadlock and its one-word fix.
- ☐ I can compute the ordinary rate and the spike rate.
- ☐ I know why retries make the spike worse than it appears.
- ☐ I can name the three spike layers and what each does.
- ☐ I know why a visible queue position matters.
- ☐ I know why a stale seat map is correct.

- [] I can compute polling versus pushing.
- [] I can state the TTL trade and its asymmetry.
- [] I know the realistic goal for bot defence.
- [] I answered all three questions above out loud.

Interval scheduling

DATA STRUCTURES AND ALGORITHMS

Interval scheduling

SYSTEM DESIGN

Design search autocomplete at scale

1 What this is, and why they ask it

An interval is a pair of numbers: a start and an end. Interval problems ask you to select, merge, count or schedule them, and there are perhaps eight of them that appear over and over.

They ask it because **it is the highest-density topic in the interview canon.** Eight problems, three techniques, and the techniques are short — **so the whole family is learnable in a day and it appears in almost every interview loop.**

The other reason is that **the first move is always the same and it is where the marks are. Sort.** And the entire difficulty is **sort by what** — because start, end and duration all seem plausible, **and which one is correct depends on the question in a way that is not obvious.**

"how many can I fit?"	sort by END
"merge the overlapping ones"	sort by START
"how many rooms do I need?"	neither — sweep the endpoints

Three questions, three different answers, and they read almost identically. A candidate who sorts by end time reflexively gets the merging problem wrong; **a candidate who sorts by start time reflexively gets the selection problem wrong,** and both produce plausible numbers with no error.

And there is a third reason worth naming: **intervals are where greedy and DP sit next to each other.** Unweighted selection is greedy and $O(n \log n)$. **Add a value to each interval and greedy breaks and it becomes DP — at the same $O(n \log n)$, because the sort dominates both.** That pair is the cleanest demonstration in the course of yesterday's lesson.

By the end of this lesson you can classify any interval problem into one of the three techniques, know which sort each needs and why, handle the boundary conventions, and recognise the weighted variant that turns greedy into DP.

2 The story

The community hall had one room and Padmavathi kept the diary, and in eleven years she had worked out that there were only ever three arguments.

The first was about how many things she could fit in.

Somebody would want the hall on Saturday afternoon, and somebody else, and a third person, and their times overlapped in a tangle. **And she had learned — badly, over about two years — that you do not start with the big bookings.**

The wedding reception wanted two till nine. **If she said yes to that, the whole afternoon was gone for one booking.** Whereas the naming ceremony from two till four, and the meeting from four till six, and the class from six till seven-thirty — **three bookings in the same space.**

So her rule was: whoever finishes first. Not whoever asks first. Not the shortest. **Whoever gives the room back soonest.**

The second argument was completely different and she kept confusing it with the first.

The committee wanted to know **when the hall was busy** — not how many bookings, but which stretches of the day were occupied at all, so they could plan the cleaning.

And for that, **finishing first was the wrong thing to look at entirely.** She had to go through the bookings **in the order they started**, and run a finger along, joining anything that touched.

Two till four, and four till six — that is one stretch, two till six. Then nothing until seven.

Three arguments, and the third one took her the longest to see was different at all.

Because after the extension there were three rooms, and the question changed to **“can we take all of these, and how many rooms do we need at once”.**

And here neither of the first two rules helped, **because she did not care which booking was which.** She only cared **how many were happening at the same time.**

What she ended up doing was writing every start and every end on separate slips — **just the times, not whose booking they were** — putting them all in order, and walking down the list. **Plus one for a start, minus one for an end.** The biggest the running total ever got was the number of rooms she needed.

“They sound like the same question,” her daughter said.

“They sound like it,” she said. “Fitting the most in, joining up what touches, and counting how many at once. I did the first one’s method on the second one’s question for a year and a half.”

3 The idea in plain English

Padmavathi's three arguments are the three techniques, and her last sentence is the reason this topic is worth a whole day: **the questions sound identical and the methods share nothing.**

Start with the classification, because it is the whole lesson.

QUESTION	TECHNIQUE	SORT BY
maximum number of non-overlapping	greedy select	END
minimum removals to make disjoint	greedy select (the same problem)	END
merge overlapping intervals	merge	START
insert an interval into a sorted list	merge	(already sorted)
do any two overlap?	merge	START
minimum rooms / maximum concurrent	sweep line	endpoints
busiest time / peak load	sweep line	endpoints
maximum VALUE of non-overlapping	DP	END

Three techniques. Learn which question maps to which, and the code is short.

Technique one: greedy selection. Sort by END time.

The question is “how many can I fit”. Sort by end time, keep a running `last_end`, and take any interval that starts at or after it.

Why end time is the correct key is yesterday's exchange argument: **the only thing about an interval that constrains the future is when the resource becomes free**, and finishing earliest leaves the most room.

And “minimum removals to make them disjoint” is the same problem: `n` minus the maximum you can keep. **Recognising that saves you from writing a second algorithm** — LeetCode asks it both ways.

Technique two: merging. Sort by START time.

The question is “join up what touches”. Sort by start, then walk through: **if the next interval starts before the current one ends, extend the current one; otherwise close it and begin a new one.**

Sorting by start is what makes the single pass work. With intervals in start order, **anything that overlaps the current merged block must start before its end** — so one comparison per interval decides it. **Sorted by end, that property does not hold** and the algorithm is wrong.

And the one line people get wrong is the extension: `current_end = max(current_end, next_end)`, **not** `= next_end`. An interval entirely contained inside the current block would otherwise shrink it.

Technique three: the sweep line. Sort the endpoints, not the intervals.

The question is “how many at once”. And the key move is that **you stop caring which interval is which.**

Break every interval into two events — a start and an end — **throw away the association**, sort all the events by time, and walk through with a counter: **plus one on a start, minus one on an end.** The maximum the counter reaches is the answer.

That decomposition is the idea. “Minimum meeting rooms” sounds like a scheduling problem and is a counting problem, **and once the intervals become independent events it is six lines.**

And there is an equivalent formulation worth knowing: sort the starts and the ends into two separate arrays and merge them with two pointers. Same answer, and some people find it clearer.

Now the boundary convention, which is the most common bug in the whole topic.

Does an interval ending at 4 conflict with one starting at 4?

For meeting rooms: no. One meeting ends and the next begins; the room is free. **For “do these two number ranges overlap”: possibly yes,** depending on whether the ranges are inclusive.

It is one comparison — $\geq$ against $>$ — **and the problem statement decides it. Ask, and say which you assumed.** In the sweep line it appears as the tie-break: **when a start and an end share a time, process the END first** if touching intervals do not conflict.

Getting that backwards gives an answer exactly one too high, on inputs where meetings abut — which is most realistic inputs.

Then the fourth thing, which is the greedy-to-DP transition.

Unweighted selection is greedy. Add a value to each interval — “**maximise total value, not count**” — **and greedy fails,** because one valuable long interval can beat three cheap short ones and no local rule sees it.

That is weighted interval scheduling, and it is DP:

```
sort by end time
dp[i] = max( dp[i-1],                                skip interval i
             value[i] + dp[ p(i) ] )                 take it
where p(i) = the last interval that ends before interval i starts
```

And `p(i)` is a binary search over the sorted end times, which keeps the whole thing `O(n log n)` — the same complexity as the greedy version, because both are dominated by the sort.

That is the point worth making out loud: greedy failing here costs you a table and some code, and nothing asymptotically.

Finally: the representation details that cause real bugs.

Sorting tuples in Python sorts lexicographically, so `sorted(intervals)` sorts by start and then by end — which is what you want for merging and not for selection. Being explicit with `key=` says which you meant.

And `sorted()` rather than `.sort()`, unless mutating the caller's list is intended.

4 The picture

The three questions, on the same input:

```
intervals:  [1,4] [2,5] [7,9] [3,6]
```

```
drawn:      1---4
            2----5
            3----6
                7--9
```

Q1: MAXIMUM NON-OVERLAPPING? → 2 ([1,4] and [7,9])

Q2: MERGED into blocks? → [1,6] and [7,9]

Q3: MAXIMUM CONCURRENT (rooms)? → 3 (at time 3.5)

THREE ANSWERS. THREE ALGORITHMS. THREE SORT KEYS.
And the questions read almost identically.

Technique one: greedy selection, sorted by END:

sorted by end: [1,4] [2,5] [3,6] [7,9]

```
take [1,4]                                last_end = 4
[2,5] starts at 2 < 4                      skip
[3,6] starts at 3 < 4                      skip
[7,9] starts at 7 ≥ 4                      take      last_end = 9
```

answer 2

WHY END TIME: the only thing that constrains the future is when the resource becomes free. Finishing earliest leaves the most room.

SORTED BY START instead:

```
take [1,4], skip [2,5], skip [3,6], take [7,9] → 2 here
but on [0,10] [1,2] [3,4] [5,6]:
  by start: take [0,10] → blocks everything      → 1
  by end:   take [1,2] [3,4] [5,6]              → 3
```

Technique two: merging, sorted by START:

sorted by start: [1,4] [2,5] [3,6] [7,9]

```
current = [1,4]
[2,5]: starts at 2 ≤ 4    → extend: current = [1, max(4,5)] = [1,5]
[3,6]: starts at 3 ≤ 5    → extend: current = [1, max(5,6)] = [1,6]
[7,9]: starts at 7 > 6    → close [1,6], current = [7,9]
```

answer [1,6] [7,9]

THE LINE PEOPLE GET WRONG:

```
current_end = max(current_end, next_end)    CORRECT
current_end = next_end                      WRONG
```

on [1,10] [2,3]: correct gives [1,10]; wrong gives [1,3]
→ an interval CONTAINED in the current block would shrink it

Technique three: the sweep line — stop caring which interval is which:

```
intervals: [1,4] [2,5] [3,6] [7,9]
```

```
BREAK INTO EVENTS, discarding the association:
```

```
+1 at 1      +1 at 2      +1 at 3      +1 at 7  
-1 at 4      -1 at 5      -1 at 6      -1 at 9
```

```
SORT ALL EVENTS BY TIME:
```

```
1:+1  2:+1  3:+1  4:-1  5:-1  6:-1  7:+1  9:-1
```

```
WALK, keeping a running total:
```

```
1: 1  
2: 2  
3: 3 ← the maximum  
4: 2  
5: 1  
6: 0  
7: 1  
9: 0
```

```
answer 3 rooms
```

THE IDEA: "minimum meeting rooms" sounds like scheduling and is COUNTING. Once the intervals become independent events, it is six lines.

The boundary convention, which is the commonest bug:

```
meeting A: [2, 4]      meeting B: [4, 6]
```

```
DO THEY CONFLICT?
```

```
MEETING ROOMS: no – one ends as the other begins, the room is free  
→ selection uses  $\text{start} \geq \text{last\_end}$   
→ the sweep processes the END before the START at time 4
```

```
INCLUSIVE RANGES: possibly yes – both occupy the point 4  
→ selection uses  $\text{start} > \text{last\_end}$   
→ the sweep processes the START before the END
```

```
IT IS ONE COMPARISON, AND THE PROBLEM STATEMENT DECIDES IT.
```

```
Getting it backwards gives an answer exactly ONE too high,  
on inputs where intervals abut – which is most real inputs.
```

Where greedy stops and DP begins:

UNWEIGHTED – greedy by end time

```
[0,10] [0,3] [3,6] [6,9]
answer: 3 intervals
```

WEIGHTED – greedy FAILS

```
[0,10] worth 100
[0,3] worth 10, [3,6] worth 10, [6,9] worth 10
```

```
greedy by end time: takes the three short ones → 30
optimal:             takes the long one         → 100
```

→ weighted interval scheduling is DP:

```
sort by end time
dp[i] = max( dp[i-1], value[i] + dp[p(i)] )
p(i) = last interval ending before interval i starts
      (found by BINARY SEARCH over the sorted end times)
```

$O(n \log n)$ – THE SAME as the greedy version, because both are dominated by the sort.

→ greedy failing here costs a table, not a worse complexity.

5 The code, built step by step

Greedy selection: sort by end

```
def max_non_overlapping(intervals: list[tuple[int, int]]) → int:
    """
    How many can I fit? Sort by END time.

    EXCHANGE ARGUMENT: swapping in the earliest finisher cannot create
    a conflict, because it ends no later than whatever it replaced.
    """
    count, last_end = 0, float("-inf")
    for start, end in sorted(intervals, key=lambda x: x[1]):
        if start ≥ last_end:           # ≥ : touching is fine
            count += 1
            last_end = end
    return count

def min_removals(intervals: list[tuple[int, int]]) → int:
    """The SAME problem: n minus the most you can keep."""
    return len(intervals) - max_non_overlapping(intervals)
```

`min_removals` being one line is the point. LeetCode asks this both ways — 452 and 435 are the same algorithm — and recognising it saves writing a second one.

Merging: sort by start

```
def merge(intervals: list[tuple[int, int]]) → list[tuple[int, int]]:
    """
    Join up what touches. Sort by START.

    Sorted by start, anything overlapping the current block must
    begin before the block ends – so one comparison decides it.
    """
    if not intervals:
        return []
    ordered = sorted(intervals, key=lambda x: x[0])
    merged = [list(ordered[0])]
    for start, end in ordered[1:]:
        if start ≤ merged[-1][1]:
            # overlaps or touches
            merged[-1][1] = max(merged[-1][1], end) # NOT `= end`
        else:
            merged.append([start, end])
    return [tuple(block) for block in merged]
```

`max(merged[-1][1], end)` is the line that matters. With `= end`, an interval entirely inside the current block **shrinks it** — `[1,10]` followed by `[2,3]` becomes `[1,3]`.

And it is invisible on most test inputs, because containment is less common than partial overlap.

The sweep line: sort the endpoints

```
def min_rooms(intervals: list[tuple[int, int]]) → int:
    """
    How many at once? Break into EVENTS and forget which interval is which.

    The tie-break matters: at equal times, process the END first,
    because a room freed at 4 can be reused at 4.
    """
    events = []
    for start, end in intervals:
        events.append((start, +1))
        events.append((end, -1))
    events.sort() # (time, delta): -1 sorts before +1

    current = best = 0
    for _, delta in events:
        current += delta
        best = max(best, current)
    return best
```

`events.sort()` on `(time, delta)` tuples does the tie-break for free, because `-1 < +1` — so at equal times the end is processed first, which is what “the room is free” means.

Swap the signs and you get an answer exactly one too high on any input where meetings abut.

The two-pointer form, which some prefer

```
def min_rooms_two_pointer(intervals: list[tuple[int, int]]) → int:
    """The same answer: merge the sorted starts and ends."""
    starts = sorted(s for s, _ in intervals)
    ends = sorted(e for _, e in intervals)
    rooms = best = 0
    i = j = 0
    while i < len(starts):
        if starts[i] < ends[j]:
            # a meeting begins before one ends
            rooms += 1
            best = max(best, rooms)
            i += 1
        else:
            rooms -= 1
            j += 1
    return best
```

`starts[i] < ends[j]` rather than `≤` is the same boundary decision, expressed differently — and it is worth writing both forms once to see that they are the same algorithm.

The heap form, when you need which room

```
import heapq

def assign_rooms(intervals: list[tuple[int, int]]) → list[int]:
    """Which room does each meeting get? A heap of end times."""
    ordered = sorted(range(len(intervals)), key=lambda i: intervals[i][0])
    free: list[tuple[int, int]] = [] # (end_time, room_number)
    assignment = [0] * len(intervals)
    rooms_used = 0
    for i in ordered:
        start, end = intervals[i]
        if free and free[0][0] ≤ start: # the earliest-freeing room is ready
            _, room = heapq.heappop(free)
        else:
            rooms_used += 1
            room = rooms_used
        assignment[i] = room
        heapq.heappush(free, (end, room))
    return assignment
```

The sweep line counts rooms; the heap assigns them. If the question asks which meeting goes where, the counter is not enough — and this is the natural follow-up.

Overlap detection and insertion

```
def has_overlap(intervals: list[tuple[int, int]]) → bool:
    """Sort by start; adjacent pairs are the only ones that can conflict."""
    ordered = sorted(intervals, key=lambda x: x[0])
    return any(ordered[i][1] > ordered[i + 1][0] for i in range(len(ordered) - 1))

def insert_interval(intervals: list[tuple[int, int]],
                    new: tuple[int, int]) → list[tuple[int, int]]:
    """Already sorted and disjoint: three phases, no re-sort. O(n)."""
    result: list[tuple[int, int]] = []
    start, end = new
    i, n = 0, len(intervals)

    while i < n and intervals[i][1] < start: # entirely before
        result.append(intervals[i])
        i += 1
    while i < n and intervals[i][0] ≤ end: # overlapping: absorb
        start = min(start, intervals[i][0])
        end = max(end, intervals[i][1])
        i += 1
    result.append((start, end))
    result.extend(intervals[i:])           # entirely after
    return result
```

`insert_interval` is $O(n)$ because the input is already sorted, and sorting it again would be $O(n \log n)$ — a small thing, and it is exactly what the problem is testing.

The three phases — before, overlapping, after — are the whole structure, and writing them as three loops rather than one loop with branches is what makes it readable.

Weighted interval scheduling: where greedy fails

```
import bisect

def max_weight_intervals(intervals: list[tuple[int, int, int]]) → int:
    """
    (start, end, value). Greedy FAILS: one valuable long interval can
    beat three cheap short ones. This is DP, and still  $O(n \log n)$ .
    """
    if not intervals:
        return 0
    ordered = sorted(intervals, key=lambda x: x[1])      # by END
    ends = [e for _, e, _ in ordered]

    dp = [0] * (len(ordered) + 1)
    for i, (start, _, value) in enumerate(ordered, start=1):
        # p = the last interval that ends at or before this one starts
        p = bisect.bisect_right(ends, start, 0, i - 1)
        dp[i] = max(dp[i - 1], value + dp[p])
    return dp[-1]
```

`bisect_right(ends, start)` is what keeps this $O(n \log n)$ — a linear scan for the last compatible interval would make it quadratic.

And `dp[i] = max(skip, take)` is the standard two-branch recurrence, which after a month of dynamic programming needs no explanation — the interesting part is that it costs the same as the greedy version.

The complete solution

```
"""Interval scheduling: three techniques, and which question needs which."""

import bisect
import heapq

# ----- technique 1: greedy selection, sort by END -----

def max_non_overlapping(intervals: list[tuple[int, int]]) → int:
    """How many can I fit? EXCHANGE: the earliest finisher ends no later."""
    count, last_end = 0, float("-inf")
    for start, end in sorted(intervals, key=lambda x: x[1]):
        if start ≥ last_end:
            count += 1
            last_end = end
    return count

def min_removals(intervals: list[tuple[int, int]]) → int:
    """The same problem, phrased as a subtraction."""
    return len(intervals) - max_non_overlapping(intervals)

# ----- technique 2: merging, sort by START -----

def merge(intervals: list[tuple[int, int]]) → list[tuple[int, int]]:
    """Join what touches. `max` on the extension, never `=`."""
    if not intervals:
        return []
    ordered = sorted(intervals, key=lambda x: x[0])
    merged = [list(ordered[0])]
    for start, end in ordered[1:]:
        if start ≤ merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return [tuple(b) for b in merged]

def has_overlap(intervals: list[tuple[int, int]]) → bool:
    ordered = sorted(intervals, key=lambda x: x[0])
    return any(ordered[i][1] > ordered[i + 1][0] for i in range(len(ordered) - 1))

def insert_interval(intervals: list[tuple[int, int]],
                    new: tuple[int, int]) → list[tuple[int, int]]:
    """Input already sorted and disjoint → O(n), three phases."""
    result: list[tuple[int, int]] = []
    start, end = new
    i, n = 0, len(intervals)
    while i < n and intervals[i][1] < start:
        result.append(intervals[i])
        i += 1
    while i < n and intervals[i][0] ≤ end:
        start, end = min(start, intervals[i][0]), max(end, intervals[i][1])
        i += 1
    result.append((start, end))
    result.extend(intervals[i:])
```



```

    return result

# ----- technique 3: the sweep line, sort the ENDPOINTS -----

def min_rooms(intervals: list[tuple[int, int]]) → int:
    """How many at once? (time, delta) sorting puts -1 before +1 at ties."""
    events: list[tuple[int, int]] = []
    for start, end in intervals:
        events.append((start, +1))
        events.append((end, -1))
    events.sort()
    current = best = 0
    for _, delta in events:
        current += delta
        best = max(best, current)
    return best

def min_rooms_two_pointer(intervals: list[tuple[int, int]]) → int:
    starts = sorted(s for s, _ in intervals)
    ends = sorted(e for _, e in intervals)
    rooms = best = 0
    i = j = 0
    while i < len(starts):
        if starts[i] < ends[j]:
            rooms += 1
            best = max(best, rooms)
            i += 1
        else:
            rooms -= 1
            j += 1
    return best

def assign_rooms(intervals: list[tuple[int, int]]) → list[int]:
    """The sweep COUNTS rooms; a heap ASSIGNS them."""
    ordered = sorted(range(len(intervals)), key=lambda i: intervals[i][0])
    free: list[tuple[int, int]] = []
    assignment = [0] * len(intervals)
    rooms_used = 0
    for i in ordered:
        start, end = intervals[i]
        if free and free[0][0] ≤ start:
            _, room = heapq.heappop(free)
        else:
            rooms_used += 1
            room = rooms_used
            assignment[i] = room
            heapq.heappush(free, (end, room))
    return assignment

# ----- the weighted version: DP, same complexity -----

def max_weight_intervals(intervals: list[tuple[int, int, int]]) → int:
    if not intervals:
        return 0
    ordered = sorted(intervals, key=lambda x: x[1])
    ends = [e for _, e, _ in ordered]
    dp = [0] * (len(ordered) + 1)

```

```

for i, (start, _, value) in enumerate(ordered, start=1):
    p = bisect.bisect_right(ends, start, 0, i - 1)
    dp[i] = max(dp[i - 1], value + dp[p])
return dp[-1]

def max_weight_greedy(intervals: list[tuple[int, int, int]]) → int:
    """The greedy that fails, for comparison."""
    total, last_end = 0, float("-inf")
    for start, end, value in sorted(intervals, key=lambda x: x[1]):
        if start ≥ last_end:
            total += value
            last_end = end
    return total

# ----- the wrong sort keys, to see them fail -----

def max_non_overlapping_by_start(intervals: list[tuple[int, int]]) → int:
    count, last_end = 0, float("-inf")
    for start, end in sorted(intervals, key=lambda x: x[0]):
        if start ≥ last_end:
            count, last_end = count + 1, end
    return count

def merge_by_end(intervals: list[tuple[int, int]]) → list[tuple[int, int]]:
    """Merging with the WRONG sort key."""
    if not intervals:
        return []
    ordered = sorted(intervals, key=lambda x: x[1])
    merged = [list(ordered[0])]
    for start, end in ordered[1:]:
        if start ≤ merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return [tuple(b) for b in merged]

if __name__ == "__main__":
    ives = [(1, 4), (2, 5), (3, 6), (7, 9)]
    print("THE SAME INPUT, THREE QUESTIONS")
    print(" intervals          :", ives)
    print(" max non-overlapping  :", max_non_overlapping(ives))
    print(" merged                  :", merge(ives))
    print(" minimum rooms           :", min_rooms(ives))
    print(" two-pointer agrees      :", min_rooms_two_pointer(ives))
    print(" room assignment         :", assign_rooms(ives))

    print("\nTHE WRONG SORT KEY")
    blocking = [(0, 10), (1, 2), (3, 4), (5, 6)]
    print(" input                    :", blocking)
    print(" by END (correct)         :", max_non_overlapping(blocking))
    print(" by START (wrong)        :", max_non_overlapping_by_start(blocking))

    contained = [(1, 10), (2, 3), (11, 12)]
    print(" merge by START (right):", merge(contained))
    print(" merge by END (wrong):", merge_by_end(contained))

    print("\nTHE BOUNDARY CONVENTION")

```

```

abutting = [(1, 4), (4, 6), (6, 9)]
print("  abutting meetings      :", abutting)
print("  rooms needed          :", min_rooms(abutting), "(ends before starts)")
print("  max non-overlapping    :", max_non_overlapping(abutting))

print("\nGREEDY BREAKS WHEN VALUES APPEAR")
weighted = [(0, 10, 100), (0, 3, 10), (3, 6, 10), (6, 9, 10)]
print("  weighted intervals     :", weighted)
print("  greedy by end time      :", max_weight_greedy(weighted))
print("  DP (correct)            :", max_weight_intervals(weighted))

print("\nINSERTION INTO A SORTED LIST")
sorted_disjoint = [(1, 3), (6, 9)]
print("  insert (2,5) into", sorted_disjoint, "→",
      insert_interval(sorted_disjoint, (2, 5)))
print("  insert (4,5) into", sorted_disjoint, "→",
      insert_interval(sorted_disjoint, (4, 5)))

```

Run it and you get:

THE SAME INPUT, THREE QUESTIONS

```

intervals      : [(1, 4), (2, 5), (3, 6), (7, 9)]
max non-overlapping : 2
merged         : [(1, 6), (7, 9)]
minimum rooms   : 3
two-pointer agrees : 3
room assignment : [1, 2, 3, 1]

```

THE WRONG SORT KEY

```

input          : [(0, 10), (1, 2), (3, 4), (5, 6)]
by END (correct) : 3
by START (wrong) : 1
merge by START (right): [(1, 10), (11, 12)]
merge by END (wrong): [(2, 10), (11, 12)]

```

THE BOUNDARY CONVENTION

```

abutting meetings : [(1, 4), (4, 6), (6, 9)]
rooms needed      : 1 (ends before starts)
max non-overlapping : 3

```

GREEDY BREAKS WHEN VALUES APPEAR

```

weighted intervals : [(0, 10, 100), (0, 3, 10), (3, 6, 10), (6, 9, 10)]
greedy by end time : 30
DP (correct)       : 100

```

INSERTION INTO A SORTED LIST

```

insert (2,5) into [(1, 3), (6, 9)] → [(1, 5), (6, 9)]
insert (4,5) into [(1, 3), (6, 9)] → [(1, 3), (4, 5), (6, 9)]

```

The first block is the whole lesson in four lines: the same four intervals give **2**, **two merged blocks**, and **3 rooms** — three different answers to three questions that read almost the same.

by START (wrong) : 1 against by END (correct) : 3 is the sort key made visible. And the merge-by-end block starts at 2 rather than 1 — it began with [2,3], absorbed [1,10], and silently discarded the start of the very interval it was merging. The two techniques are not interchangeable, and the failure is a quiet wrong number rather than a crash.

And greedy 30 against DP 100 is yesterday's lesson in one line: adding values breaks greedy, and the DP that replaces it is the same complexity.

6 What it costs

All three techniques are dominated by the sort.

greedy selection	sort $O(n \log n)$ + one pass $O(n)$	= $O(n \log n)$
merging	sort $O(n \log n)$ + one pass $O(n)$	= $O(n \log n)$
sweep line	sort $2n$ events $O(n \log n)$ + one pass	= $O(n \log n)$
weighted DP	sort + n binary searches	= $O(n \log n)$

ALL FOUR ARE $O(n \log n)$, AND THE SORT IS THE WHOLE COST.

Which means the interesting comparison is not between the techniques — it is against the naive versions:

```
"do any two overlap?"
naive: check every pair           $O(n^2)$ 
sorted: adjacent pairs only       $O(n \log n)$ 
n = 100,000:  $10^{10}$  against  $1.7 \times 10^6$  → ~6,000x

"maximum concurrent"
naive: for each interval, count overlaps  $O(n^2)$ 
sweep line  $O(n \log n)$ 
same ratio
```

Space:

greedy selection	$O(1)$ beyond the sort — two variables
merging	$O(n)$ for the output
sweep line	$O(n)$ for the events ($2n$ of them)
weighted DP	$O(n)$ for the table
heap assignment	$O(k)$ where k is the number of rooms

Concretely, at realistic sizes:

```
n = 1,000          sorting: microseconds. Everything is instant.  
n = 100,000       sort ~0.1 s in Python; the pass ~0.02 s  
n = 10,000,000    sort ~15 s in Python; still linear afterwards
```

→ intervals scale well. The techniques are all one sort plus one pass, and nothing here has a hidden quadratic.

The sweep line's constant factor:

2n events instead of n intervals
→ twice the sorting work

the two-pointer form sorts two arrays of n instead of one of 2n
→ $n \log n + n \log n$ against $2n \log(2n)$
→ marginally better, and the same complexity

in practice: use whichever you find clearer. The difference is under 10%.

Weighted DP against unweighted greedy, which is the point:

```
unweighted greedy     $O(n \log n)$  time,  $O(1)$  space  
weighted DP           $O(n \log n)$  time,  $O(n)$  space
```

→ the SAME time complexity, because both are dominated by the sort.

The DP costs:

- one array of n
- a binary search per interval ($\log n$ each)

→ greedy failing here is a table and about eight lines, not an asymptotic penalty. That is unusual and worth knowing.

Binary search versus linear scan for $p(i)$:

linear scan backwards for the last compatible interval:
worst case $O(n)$ per interval → $O(n^2)$ overall
 $n = 100,000 \rightarrow 10^8$. Not viable.

bisect over the sorted end times:
 $O(\log n)$ per interval → $O(n \log n)$
 $n = 100,000 \rightarrow 1.7$ million. Instant.

→ the binary search is not an optimisation; it is what makes the weighted version tractable.

And the cost of the wrong sort key, which is not a performance cost:

```
sorting by start instead of end: SAME complexity, WRONG answer
sorting by end instead of start: SAME complexity, WRONG answer

→ there is no performance signal to warn you.
   Both run in  $O(n \log n)$  and both return a plausible number.
```

7 The traps

The wrong sort key for selection.

```
>>> intervals = [(0, 10), (1, 2), (3, 4), (5, 6)]
>>> max_non_overlapping(intervals)           # by end
3
>>> max_non_overlapping_by_start(intervals)  # by start
1
```

Three against one. The long interval starts earliest and blocks everything. **Same complexity, no error, and the answer is a third of the correct one.**

The wrong sort key for merging.

```
>>> merge([(1, 10), (2, 3), (11, 12)])      # by start
[(1, 10), (11, 12)]
>>> merge_by_end([(1, 10), (2, 3), (11, 12)]) # by end
[(2, 10), (11, 12)]
```

The block starts at 2 rather than 1. Sorted by end, the pass began with `[2,3]`, then absorbed `[1,10]` by extending the *end* — **and there is no code path that reaches backwards to fix the start. The count is right and the answer is wrong**, so a test checking only the number of blocks passes.

`= end` instead of `max(current_end, end)` when merging.

```
>>> # merging [(1,10), (2,3)] with `current_end = end`:
>>> # [1,10] then [2,3] starts at  $2 \leq 10$ , so extend to end=3
>>> # → [(1, 3)]   WRONG: the block SHRANK
```

An interval contained entirely within the current block shrinks it. Containment is less common than partial overlap in test data, which is exactly why this survives.

The boundary convention, backwards.

```
>>> min_rooms([(1, 4), (4, 6), (6, 9)])
1
>>> # with the sweep processing STARTS before ENDS at equal times:
>>> # → 2, because the room is counted as occupied at time 4
```

One too high, on the most realistic input there is — meetings that abut. The `(time, delta)` sort gives the right convention for free because $-1 < +1$, and writing the events as `(time, "start")` and `(time, "end")` gets it backwards, because `"end" < "start"` alphabetically happens to be right and `"begin" < "finish"` would not be. **Use numbers.**

Assuming the answer is a count when it is an assignment.

```
>>> min_rooms([(1, 4), (2, 5), (3, 6), (7, 9)])
3
>>> assign_rooms([(1, 4), (2, 5), (3, 6), (7, 9)])
[1, 2, 3, 1]
```

The sweep line tells you how many rooms and cannot tell you which meeting goes where. If the question asks for the assignment, you need the heap — and that is the standard follow-up.

Re-sorting an already sorted input.

```
>>> # "insert an interval into a sorted, disjoint list"
>>> # sorting again:  $O(n \log n)$ 
>>> # the three-phase scan:  $O(n)$ 
>>> # → the problem is TESTING whether you noticed
```

The input being sorted is stated in the problem and is the whole point.

Greedy on the weighted version.

```
>>> weighted = [(0, 10, 100), (0, 3, 10), (3, 6, 10), (6, 9, 10)]
>>> max_weight_greedy(weighted)
30
>>> max_weight_intervals(weighted)
100
```

Thirty against a hundred. One word in the problem statement — “number” against “value” — changes the algorithm entirely, and greedy passes every unweighted test.

Linear search for the compatible interval in the DP.

```
>>> # scanning backwards for the last interval ending before this starts:
>>> #  $O(n)$  per interval  $\rightarrow O(n^2)$  overall
>>> #  $n = 100,000 \rightarrow 10^{10}$  operations
```

No error, and it does not finish. The binary search is what makes the weighted version tractable, not a tidy-up.

Mutating the caller's list.

```
>>> data = [(3, 5), (1, 4)]
>>> # if merge used data.sort() rather than sorted(data):
>>> data
[(1, 4), (3, 5)]           # reordered, silently
```

`sorted()` unless you mean to reorder the input — and saying “I will not mutate the input” costs two words and is a small, real signal.

8 In the interview

How it gets asked

- “Merge overlapping intervals.” — LeetCode 56, the most-asked of the family.
- “How many meeting rooms do you need?” — LeetCode 253, the sweep line.
- “What is the maximum number of non-overlapping intervals?” — LeetCode 435, greedy.
- “Insert an interval into a sorted list.” — LeetCode 57, and it tests whether you re-sort.
- “Do the meetings conflict?” — the easy opener.
- “Now each meeting has a value.” — the greedy-to-DP transition.

The first ninety seconds

“Interval problems all start the same way — **sort** — and the entire difficulty is **sort by what**, because start, end and duration all seem plausible and the right answer depends on the question.

There are three techniques and I would classify the problem before writing anything.

If the question is ‘how many can I fit’, it is greedy, sorted by END time. Keep a running last-end and take anything that starts at or after it. **End time is correct because the only thing about an interval that constrains the future is when the resource becomes free** — finishing earliest leaves the most room. **And ‘minimum removals to make them disjoint’ is the same problem**, phrased as n minus the answer.

If the question is ‘merge what overlaps’, it is a single pass, sorted by START. With intervals in start order, anything overlapping the current block must begin before that block ends — **so one comparison per interval decides it**. Sorted by end, that property does not hold and the algorithm is simply wrong.

If the question is ‘how many at once’ — meeting rooms, peak concurrency — it is a sweep line, and the key move is that you stop caring which interval is which. Break each into two events, plus one at the start and minus one at the end, sort all the events by time, and take the running maximum. **A scheduling-sounding question turns into a counting problem, and it is six lines.**

One thing I would settle before coding: does an interval ending at four conflict with one starting at four? For meeting rooms, no — the room is free. **It is one comparison, the problem decides it, and getting it backwards gives an answer exactly one too high** on inputs where meetings abut, which is most real inputs.

All three are $O(n \log n)$, dominated by the sort, and $O(1)$ to $O(n)$ space.

And I would ask one question: do the intervals have values, or am I just counting them? Because with values greedy breaks and it becomes dynamic programming — **at the same $O(n \log n)$, since the sort dominates both.**“

The follow-ups

“Why sort by end time here and by start time for merging?”

“Because the two algorithms need different things to be true, and each sort key establishes exactly one of them.

For selection, sorting by end time is what makes the greedy choice safe. The exchange argument: take any optimal selection and its first interval; the earliest-finishing interval ends no later, **so swapping it in cannot create a conflict with anything that came after** — and the count is unchanged. **So some optimal solution contains the greedy choice.**

And the intuition behind that: the only property of an interval that constrains the future is when it releases the resource. Not when it starts, not how long it is. So take the one that releases soonest.

Sorting by start fails immediately — one interval from zero to ten starts earliest and blocks the whole range. Greedy gets one where the answer is three.

For merging, sorting by start establishes a different property: that anything overlapping the current merged block must *begin* before the block ends. **That is what makes a single comparison per interval sufficient** — I never have to look backwards.

Sorted by end, that property does not hold. An interval can begin before the current block and end after everything I have seen, and I would miss the overlap. **And the output is not even sorted, which is a good way to notice.**

So it is not that one key is ‘better’ — each establishes the invariant its algorithm depends on. **When I pick a sort key I try to say what invariant it gives me**, and if I cannot, I have not understood the algorithm.

One implementation note: in Python, `sorted(intervals)` sorts lexicographically — by start, then end. That happens to be right for merging and wrong for selection, **so I write the `key=` explicitly either way**, because it says which I meant.”

“How many meeting rooms do you need?”

“This sounds like the selection problem and it is a different question entirely, **and the key move is that you stop caring which meeting is which.**

I do not need to know that meeting A is in room two. I need to know how many meetings are happening at the busiest moment. That is a counting problem, not a scheduling one.

So: break every meeting into two events. A start event, worth plus one, and an end event, worth minus one. **Throw away which meeting they belonged to.** Sort all two-n events by time, walk through with a running counter, and take the maximum it reaches.

Six lines, $O(n \log n)$ dominated by sorting the events.

The detail that matters is the tie-break. When a meeting ends at four and another starts at four, **the end must be processed first** — the room is freed and immediately reused, so it is one room, not two.

In Python that comes free by sorting `(time, delta)` tuples, because minus one sorts before plus one. **If I encoded the events as strings I would have to think about it,** and getting it backwards gives an answer exactly one too high on any input where meetings abut — which is most realistic inputs.

There is an equivalent form — sort the starts and the ends into two separate arrays and merge them with two pointers — **which some people find clearer and which is the same algorithm.**

And the natural follow-up: which meeting goes in which room? The sweep counts and cannot assign, so for that I would use a min-heap of end times: for each meeting in start order, **reuse the room that frees earliest if it is free by now, otherwise open a new one.** The heap size is the number of rooms, which agrees with the sweep — and now I also know the assignment.”

“Now each meeting has a value, and you want the maximum total value.”

“Then greedy breaks, and the demonstration is small enough to give immediately.

One interval from zero to ten worth a hundred, and three short ones — zero to three, three to six, six to nine — worth ten each. Greedy by end time takes the three short ones for thirty. **The answer is the long one, for a hundred.**

And no local rule fixes it, because whether to take a long valuable interval depends on what is available for the rest of the range, which is exactly the information a greedy choice does not have.

So it becomes weighted interval scheduling, which is dynamic programming.

Sort by end time — the same sort — and let $dp[i]$ be the best total value using the first i intervals. For each interval, two branches: **skip it, giving $dp[i-1]$; or take it, giving its value plus $dp[p]$, where p is the last interval that ends before this one starts.**

And finding p is a binary search over the sorted end times, which is not an optimisation — a linear scan makes it $O(n^2)$, which at a hundred thousand intervals is ten to the ten and does not finish.

The outcome is the interesting part: $O(n \log n)$, exactly the same as the greedy version, because both are dominated by the sort. **So greedy failing here costs me a table and about eight lines and nothing asymptotically**, which is unusual — most of the time greedy failing means a real complexity penalty.

The lesson I would draw out loud: ‘maximum number’ and ‘maximum value’ are one word apart in the problem statement and are different algorithms. I read for that word specifically, **because the greedy solution passes every unweighted test and silently fails the weighted ones.**“

The model answer

“You are building the scheduling backend for a co-working space. Given the day’s booking requests, each with a start time and an end time, answer three questions: which requests can we accept if we have one room, how many rooms would we need to accept all of them, and what are the busy periods for the cleaning schedule.”

“Three questions, and **they sound similar and need three completely different algorithms** — so let me take them one at a time and say which technique each needs.

One clarification first, which affects all three: does a booking ending at four conflict with one starting at four? For a room, I would say no — one group leaves, the next arrives. **It is one comparison and I will assume touching is fine, and say so.**

Question one: which requests can we accept with one room? This is **maximise the number of non-overlapping intervals**, which is greedy, **sorted by end time**.

Keep a running last-end and accept any booking that starts at or after it. **End time is correct because the only thing that constrains the future is when the room becomes free** — and the exchange argument is that swapping in the earliest finisher cannot create a conflict, since it ends no later than whatever it replaced.

I would not sort by start, because one all-day booking would block everything — greedy would accept one where three were possible.

$O(n \log n)$, $O(1)$ space beyond the sort, and I would return the actual list rather than the count, since the business needs to know which bookings to confirm.

Question two: how many rooms to accept all of them? A different question entirely — this is maximum concurrency, and it is a **sweep line**.

The key move is that I stop caring which booking is which. Break each into a plus-one at its start and a minus-one at its end, discard the association, sort all the events, and take the running maximum. **A scheduling-sounding question becomes counting, in six lines.**

And the tie-break is where the bug lives: at equal times, process the end before the start, because the room is freed and reused. **Sorting $(\text{time}, \text{delta})$ tuples gives that for free, since minus one sorts before plus one** — and getting it backwards gives one room too many on exactly the inputs a co-working space generates, where bookings abut on the hour.

And since this is a real business, they will want the assignment too, not just the count. **For that I would use a min-heap of room end times:** process bookings in start order, reuse the earliest-freeing room if it is available, otherwise

open a new one. **The heap size is the room count, agreeing with the sweep, and now every booking has a room number.**

Question three: the busy periods for cleaning. This is **merging**, and it is the third technique — **sorted by start**.

Walk through in start order; if the next booking begins at or before the current block ends, extend the block; otherwise close it and start a new one. **And the extension is `max(current_end, next_end)`, never `= next_end`** — a short booking entirely inside a long one would otherwise shrink the block, **which is invisible on most test data and wrong.**

The output is the occupied stretches, and the gaps between them are when the cleaners can work — which is what they actually asked for, so I would return the gaps rather than making them compute the complement.

All three are `O(n log n)` and dominated by the sort, so for a day's bookings — hundreds — everything is instant.

Two things I would raise.

If the bookings have values — a premium room, or a longer booking worth more — question one changes completely. Greedy fails, because one valuable all-day booking can beat three cheap short ones, **and it becomes weighted interval scheduling: DP with a binary search, still `O(n log n)`**. I would ask whether acceptance is by count or by revenue, because that single word changes the algorithm.

And I would ask about the multi-room version of question one, which is neither of the three: **accepting as many bookings as possible given three rooms** is a genuinely harder problem than any of these, and I would not want to discover that assumption late."

9 Recall card

Every interval problem starts with a sort, and the whole difficulty is SORT BY WHAT. Three questions that read almost identically need three different keys — and the wrong key gives a plausible answer with no error and the same complexity.

"How many can I fit?" → greedy, sort by END. Take anything starting at or after `last_end`. End time is correct because **the only thing constraining the future is when the resource becomes free.** "Minimum removals to make disjoint" is **the same**

problem: n minus the answer.

“Merge what overlaps” → **one pass, sort by START**. Start order guarantees that anything overlapping the current block *begins* before it ends, which is what makes one comparison enough. **The extension is** `max(current_end, next_end)`, **never** `= next_end` — a contained interval would shrink the block, and containment is rare in test data.

“How many at once?” → **SWEEP LINE**, and the move is to stop caring which interval is which. Break each into `+1` at the start and `-1` at the end, sort the events, take the running maximum. **Sorting** `(time, delta)` **tuples gives the right tie-break free** (`-1 < +1`), so an end at time 4 is processed before a start at 4 — getting it backwards is **exactly one too high** on abutting inputs. **The sweep COUNTS; a min-heap of end times ASSIGNS.**

Settle the boundary convention before coding — `≥` against `>` — the problem decides it, and say which you assumed.

All techniques are $O(n \log n)$, **dominated by the sort**. Add values and greedy breaks: `[0,10]` worth 100 against three worth 10 gives greedy 30 and the answer 100. **Weighted interval scheduling is DP** — `dp[i] = max(dp[i-1], value + dp[p(i)])` with `p(i)` found by **BINARY SEARCH** (a linear scan makes it $O(n^2)$) — and it is the same $O(n \log n)$, so greedy failing costs a table, not a complexity penalty.

1 What this is, and why they ask it

Autocomplete suggests query completions as you type. **You wrote the trie for it on day 122.** Today is what happens when a billion people use it.

They ask it because **it is the design where a data structure you know well turns out to be the easy part**, and the interesting engineering is everywhere else.

The latency requirement is brutal and it is the constraint that shapes everything. Suggestions must appear before the user types the next character — **under about a hundred milliseconds end to end, including the network** — and a suggestion that arrives after the next keystroke is worse than useless, because the user has moved on.

The query rate is extreme, and for an unusual reason: it is per keystroke, not per search. A single search for “manchester united” generates seventeen autocomplete requests. **So autocomplete traffic is roughly an order of magnitude larger than search traffic**, which surprises people and is the first thing to compute.

And the ranking is where the product lives. Any trie can return the completions of a prefix. **Returning the five that this person, in this place, right now, actually wants** is the whole value, and it involves popularity, recency, personalisation and a serious amount of filtering.

The fourth thing — and it is the one that separates candidates — is that this is not a live query at all. The obvious design walks a trie at request time. **The scalable design precomputes the answer for every prefix and serves it as a lookup**, and understanding why that inversion is necessary is the point of the interview.

By the end of this lesson you can design the data structure and its precomputation, get inside the latency budget, rank and personalise, handle trending queries and abuse, and size the whole thing.

2 The story

The chemist’s shop had eleven thousand things in it and Ravindran had worked there since he was nineteen, and by the time he was fifty he did something that visitors found unsettling.

You would get four words into a sentence and he would already be reaching.

Not guessing. **He would be right almost every time, and when he was wrong he was wrong in a way that told you he had been thinking about it.**

His nephew, who took over the shop, spent a year trying to work out how it was done, **because he could not do it and he had the same eleven thousand things.**

And when he finally asked, the answer was in three parts and none of them was memory.

“Most of what anybody asks for is about forty things.” Eleven thousand on the shelves, and forty of them were nine out of every ten sales. **“I am not searching eleven thousand. I am searching forty.”**

Second: he did not wait for you to finish. As you started, he was already narrowing — and by the fourth word there were usually two or three possibilities, **and he had thought about all three before you stopped talking.**

“If I start thinking when you finish, I am slower than you. I have to be ahead of you.”

And the third part was the one his nephew had not expected at all.

“It changes.” In the fever season, the first thing anybody says leads somewhere different from the same words in December. **And a man who came in every fortnight got a different answer from a stranger saying exactly the same thing,** because Ravindran knew what he had bought last time.

His nephew said that was just knowing the customers.

“It is knowing that the same four words mean different things depending on who said them and what month it is,” he said. **“The words are not the question. The words plus everything else are the question.”**

And then, because his nephew was writing this down, he added the part he was actually proud of.

“And I decide before you ask. All day, while I am doing other things, I am working out what people are going to want. By the time you open your mouth the answer is already made.”

3 The idea in plain English

Ravindran's three parts are the three ideas: **the popular set is tiny, you must be ahead of the user, and the answer is precomputed rather than searched.**

Start with the latency budget, because it is the constraint everything else obeys.

```
a fast typist:           ~100-200 ms between keystrokes
the network round trip:  ~50-80 ms (mobile, realistically)
→ the SERVER has ~20-50 ms
```

```
and a suggestion that arrives after the next keystroke is
WORSE than none, because the user has already moved on
```

Twenty to fifty milliseconds of server time is not enough to walk a trie of a billion queries, rank the results, personalise them and filter them. So you do not.

The central inversion: precompute, do not search.

The obvious design walks a trie at request time. For each prefix, find the node, collect the completions beneath it, rank them, return the top five.

The scalable design stores the answer at the node. Every prefix maps directly to its top five completions, already ranked, and the request is a single lookup with no traversal at all.

```
"man" → ["manchester united", "manchester weather",
          "manga", "mango", "manual"]
```

A hash map from prefix to a list of strings. Not a search — a dictionary lookup, and that is what fits in the budget.

The cost is that it must be rebuilt, and that is the trade to state. New queries do not appear until the next rebuild, **which is hours** — and there is a separate fast path for trending terms, which is the third idea below.

Now the data structure, and the honest version of it.

A trie is the right mental model and often not the right storage. A trie of a billion distinct queries has enormous pointer overhead — **the pointers can outweigh the strings** — so real systems use one of:

A compressed trie (a radix tree), where chains of single-child nodes collapse into one edge holding a string. **“manchester” is one edge, not ten nodes**, and for query data this typically reduces the node count by an order of magnitude.

Or, simpler and often better: a flat hash map from prefix to results. You lose the ability to walk the structure and you gain constant-time lookup and trivial sharding, and since the answers are precomputed anyway, walking was not needed.

And the prefixes are capped. Only prefixes up to about twenty characters are stored — beyond that the result set is tiny, users rarely type that far without selecting, and the storage would grow without bound.

Then ranking, which is where the product is.

Popularity alone gives a stale, generic list. The ranking signals that matter:

frequency	how often this query is issued	the base
recency	weighted towards the last few days	stops staleness
click-through	did people SELECT this suggestion?	the real signal
personal history	have YOU searched this before?	strongest of all
location	"weather" means different things	cheap and effective
time	"cricket score" during a match	

Personal history is the strongest signal and the cheapest to apply, because a user's own recent queries are a short list that can live on the device — **so personalisation can happen client-side with no server cost at all**. That is worth saying: **the most valuable ranking signal need not touch your infrastructure**.

Then trending, which is the thing precomputation cannot do.

Something happens. Within minutes, millions of people search for it. And it is not in the precomputed index, because that was built last night.

So there is a second, small, fast layer: a real-time counter of query frequency over the last few minutes, **merged with the precomputed results at request time**. Small enough to update continuously, and it only has to handle the tail of unusual queries — **the head is handled by the index that was built overnight**.

Two layers, two update frequencies, and merging them is a few milliseconds.

Then the delivery layer, which is most of where the latency goes.

Because the server has twenty to fifty milliseconds and the network has fifty to eighty, the biggest available win is **not being on the network at all**.

Cache in the browser. The completions of "m" contain everything the completions of "man" need, **so the client can filter locally as the user types further** — one request per few keystrokes rather than one per keystroke. **That alone removes most of the traffic**.

Debounce. Do not fire on every keystroke; wait fifty milliseconds for the typing to pause. **A fast typist generates one request instead of five**.

And serve from the edge. The precomputed index is small enough — **a few gigabytes for the popular prefixes** — to sit at a CDN point of presence, **which removes the transcontinental round trip.**

Then the thing that must not be skipped: filtering.

Autocomplete is a highly visible surface that puts words in a user's mouth, and suggesting something offensive, defamatory or illegal is a real and recurring problem — **it has produced lawsuits.**

So: a blocklist applied at index-build time and again at serve time, personal names handled specially, and a mechanism for removing a suggestion quickly **without waiting for the next nightly build.** That last requirement is why the serve-time filter exists as well as the build-time one.

Finally, the sizing intuition that makes this manageable.

Ravindran's forty things. The distribution of queries is extraordinarily skewed — a small number of queries account for most of the traffic, and the long tail is enormous and rarely typed.

So the precomputed index does not need every prefix of every query. Cover the popular prefixes completely and fall back to a slower path for the rest, and the slow path is almost never taken.

4 The picture

The latency budget, which drives everything:

```
user types a character
|
v
|<----- ~100-200 ms until the next keystroke ----->|
|
| network out | SERVER | network back | render |
| 25-40 ms   | 20-50 ms | 25-40 ms   | 5 ms   |
|             ^^^^^^^
|             this is ALL you have
```

In 20-50 ms you cannot: walk a trie over a billion queries, rank the candidates, personalise them, and filter them.

→ SO DO NOT. Precompute the answer and make the request a LOOKUP.

Search against precompute:

AT REQUEST TIME (the obvious design)

```
prefix "man"
  → walk the trie to the "man" node
  → collect every completion beneath it    (could be millions)
  → score each one
  → sort, take 5
  → tens of milliseconds at best, unbounded at worst
```

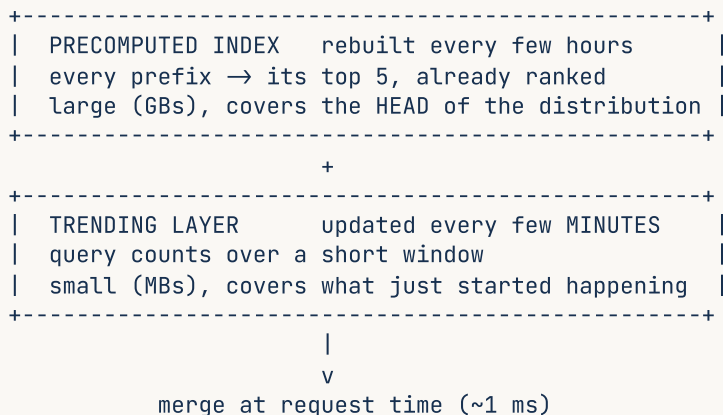
PRECOMPUTED (the scalable design)

```
"man" → ["manchester united", "manchester weather",
         "manga", "mango", "manual"]

  → ONE hash lookup. Microseconds.
  → the ranking already happened, offline, last night
```

THE COST: new queries do not appear until the next rebuild.
→ hence a second, small, REAL-TIME layer for trending.

The two layers:



Two structures, two update frequencies.
The overnight build cannot know about the last ten minutes;
the trending layer cannot rank a billion queries.

The trie, and why it is usually not stored as a trie:

PLAIN TRIE

m - a - n - c - h - e - s ...
| | |
...

10 nodes for "manchester"
each with a child map
→ POINTERS OUTWEIGH THE STRINGS

OR, SIMPLER: a flat hash map

prefix → [top 5]

→ you lose the ability to WALK the structure
→ you gain $O(1)$ lookup and trivial sharding by prefix
→ and since the answers are PRECOMPUTED, walking was never needed

COMPRESSED (RADIX)

"manchester" as ONE edge

→ chains of single-child nodes
collapse into one string edge

→ ~10x fewer nodes for query data

Where the traffic actually goes, and how to remove it:

NAIVE: one request per keystroke

"manchester united" = 17 requests

DEBOUNCE (wait 50 ms for a pause):

a fast typist → ~4 requests → 4x fewer

CLIENT-SIDE FILTERING:

the results for "man" CONTAIN everything "manc", "manch"...
need – so filter locally until the prefix leaves the
cached set

→ ~2 requests → another 2x

EDGE CACHING:

the popular prefixes are a few GB → they fit at a CDN PoP
→ removes the transcontinental round trip (~100 ms → ~20 ms)

THE BIGGEST WIN IS NOT BEING ON THE NETWORK AT ALL.

The skew, which is Ravindran's forty things:

query frequency (roughly)

top 1,000 queries	~20% of all traffic
top 100,000	~50%
top 10,000,000	~85%
the long tail (billions)	~15%, each query seen a handful of times

- the precomputed index does NOT need every prefix of every query
- cover the popular prefixes completely; fall back to a slower path for the rest
- and the slow path is almost never taken

"I am not searching eleven thousand. I am searching forty."

5 How it actually works

The precomputed index

```
def build_index(query_counts: dict[str, int], top_k: int = 5,
                max_prefix: int = 20) → dict[str, list[str]]:
    """Offline. For every prefix, the top-k completions, already ranked."""
    candidates: dict[str, list[tuple[float, str]]] = {}
    for query, count in query_counts.items():
        score = rank_score(query, count)
        for length in range(1, min(len(query), max_prefix) + 1):
            prefix = query[:length]
            bucket = candidates.setdefault(prefix, [])
            bucket.append((score, query))
            if len(bucket) > top_k * 20:                # keep it bounded
                bucket.sort(reverse=True)
                del bucket[top_k * 20:]

    return {prefix: [q for _, q in sorted(bucket, reverse=True)[:top_k]]
                for prefix, bucket in candidates.items()}
```

`max_prefix = 20` **bounds the storage**. Without it, every query contributes one entry per character, **and long queries dominate the index while nobody ever types that far**.

And the `top_k * 20` **truncation inside the loop** is what keeps memory **bounded during the build** — a popular prefix like `"a"` would otherwise accumulate every query in the corpus.

This is a batch job, and in practice it is a MapReduce or Spark job over a day of query logs — **map each query to its prefixes, reduce by taking the top-k per prefix**.

Serving

```
def suggest(prefix: str, user_id: int | None, region: str) → list[str]:
    prefix = normalise(prefix)                # lowercase, strip accents
    base = index.get(prefix, [])              # ONE lookup
    trending = trending_layer.get(prefix, region) # small, fast, recent
    merged = merge_ranked(base, trending)
    return [s for s in merged if not blocklist.contains(s)][[:5]]
```

Three lookups and a merge, and no traversal anywhere. **That is what fits in the budget.**

normalise matters more than it looks: "Man", "man" and "mán" must hit the same entry, **or the index is three times larger and mostly missed.**

And the blocklist is applied here as well as at build time, because a suggestion that must be removed cannot wait for the next nightly rebuild.

The trending layer

```
WINDOW_SECONDS = 600

def record_query(query: str, region: str) → None:
    bucket = int(time.time()) // 60
    redis.zincrby(f"trend:{region}:{bucket}", 1, query)
    redis.expire(f"trend:{region}:{bucket}", WINDOW_SECONDS * 2)

def trending_for(prefix: str, region: str, top_k: int = 3) → list[str]:
    now = int(time.time()) // 60
    keys = [f"trend:{region}:{b}" for b in range(now - 10, now + 1)]
    merged = redis.zunionstore_temp(keys)          # ten one-minute buckets
    return [q for q, _ in redis.zrevrange(merged, 0, 200)
            if q.startswith(prefix)][[:top_k]]
```

One-minute buckets with a TTL is the standard sliding-window shape — old buckets expire on their own, and there is no cleanup job.

Scanning the top two hundred and filtering by prefix is the compromise. A proper prefix index over trending data would be another whole structure, and since trending is by definition a short list, a scan is fine.

And it is per region, because what is trending in one country is irrelevant in another — **which also keeps each list short enough to scan.**

Client-side filtering, which is the biggest win

```
let cache = { prefix: "", results: [] };

function onInput(text) {
  // if the new text extends a prefix we already have, filter locally
  if (text.startsWith(cache.prefix) && cache.results.length ≥ 5) {
    const local = cache.results.filter(r ⇒ r.startsWith(text));
    if (local.length ≥ 5) {
      render(local.slice(0, 5));
      return; // NO REQUEST AT ALL
    }
  }
  debouncedFetch(text);
}
```

`if (local.length ≥ 5)` is the condition that makes this safe. The cached results for `"man"` contain the top five for `"man"`, **not necessarily the top five for `"manch"`** — **so local filtering is only valid while enough results survive**, and it falls back to the server otherwise.

This removes most of the requests, and it costs nothing on the server.

Debouncing

```
let timer = null;
function debouncedFetch(text) {
  clearTimeout(timer);
  timer = setTimeout(() ⇒ fetch(`/suggest?q=${encodeURIComponent(text)}`)
    .then(r ⇒ r.json()).then(render), 50);
}
```

Fifty milliseconds is the usual value — long enough that a fast typist's keystrokes coalesce, **short enough that a pause feels instant**.

And the request must be cancellable, or a slow response for `"man"` can arrive after the response for `"manch"` and overwrite it — **which is the out-of-order rendering bug that makes autocomplete feel broken**.

Ranking

```
import math

def rank_score(query: str, count: int, last_seen: float = 0.0,
               clicks: int = 0, impressions: int = 1) → float:
    """Frequency, decayed by age, weighted by whether people SELECT it."""
    popularity = math.log1p(count) # log: damp the head
    age_days = (time.time() - last_seen) / 86400 if last_seen else 30
    recency = math.exp(-age_days / 14) # two-week half-life-ish
    click_rate = (clicks + 1) / (impressions + 10) # smoothed
    return popularity * (0.5 + 0.5 * recency) * (0.5 + click_rate)
```

log1p on the count is deliberate: raw frequency lets a handful of enormous queries dominate every prefix, **and the log compresses the head so that the merely-popular can still appear.**

And the click rate is the strongest of the three signals, because it measures whether a suggestion was useful rather than whether the query is common — **a query people type but never select from the suggestions should rank lower.**

Personalisation, which need not touch the server

```
function personalise(suggestions, localHistory) {
  const recent = new Set(localHistory.slice(-200));
  const boosted = suggestions.filter(s ⇒ recent.has(s));
  const rest = suggestions.filter(s ⇒ !recent.has(s));
  return [...boosted, ...rest].slice(0, 5);
}
```

The strongest ranking signal is the user's own history, and it can live entirely on the device. No server cost, no privacy exposure, and it applies instantly — which is a rare combination and worth pointing at.

Filtering

```
def is_blocked(suggestion: str, blocklist, patterns) → bool:
    lowered = suggestion.lower()
    if lowered in blocklist:
        return True
    if any(p.search(lowered) for p in patterns): # regex families
        return True
    if contains_person_name(lowered) and is_defamatory_shape(lowered):
        return True # "<name> is a ..."
    return False
```

The person-name rule is the one that exists because of lawsuits. Completing a real person's name with a defamatory phrase is a legal problem in several jurisdictions, and the pattern is specific enough to detect.

And the blocklist runs at serve time as well as build time, because removal must be immediate.

Sharding

```
def shard_for(prefix: str, shard_count: int) → int:
    """By the first two characters – an even, stable split."""
    return hash(prefix[:2]) % shard_count
```

Sharding by the first two characters keeps a prefix and its extensions on the same shard, which is useful if you ever do need to walk, and it distributes evenly enough — the first character alone is badly skewed.

And a prefix's entry never needs another shard, because the answer is precomputed — so there are no cross-shard queries at all.

The real systems

Elasticsearch	completion suggester – an in-memory FST (finite state transducer), which is a compressed trie with shared suffixes
Lucene FST	the underlying structure; ~10x smaller than a plain trie
Redis	the trending layer (sorted sets), and often the serving cache
Spark / MapReduce	the nightly index build over query logs
CDN / edge	serving the popular prefixes close to users
Solr, Algolia, Typesense	hosted alternatives, all of which precompute

Naming the FST is worth doing, because it is the specific answer to “how do you store a billion prefixes” — shared prefixes *and* shared suffixes, which a plain trie does not give you.

6 The numbers

The traffic, and the number that surprises people.

5,000,000,000 searches/day

each search: ~15 characters typed

naive: one request per keystroke

= 75,000,000,000 autocomplete requests/day

= ~870,000/second average, peak ~3,000,000/second

→ AUTOCOMPLETE TRAFFIC IS 15x SEARCH TRAFFIC.

That is the first thing to compute, and it is why the delivery layer matters more than the algorithm.

And what the client-side work removes:

naive, per keystroke: 75,000,000,000/day

+ debounce (50 ms): ~20,000,000,000 (4x fewer)

+ client-side filtering: ~10,000,000,000 (another 2x)

+ edge caching (90% hit): ~1,000,000,000 reaching origin

→ 75x fewer requests reaching your servers, entirely from client and edge behaviour.

Seventy-five times, from three client-side techniques — which is why the delivery layer is where the engineering effort goes.

The latency budget, itemised.

time between keystrokes (fast typist)	100-200 ms
network round trip, mobile	50-80 ms
TLS/connection (if not reused)	+50 ms ← keep connections alive
render	5 ms

LEFT FOR THE SERVER	20-50 ms
within that:	
hash lookup in the index	<1 ms
trending layer lookup	~2 ms
merge and filter	~1 ms
serialisation	~1 ms

	~5 ms

→ the server work is comfortably inside the budget.

THE NETWORK IS THE PROBLEM, which is why edge serving matters.

Index size.

1,000,000,000 distinct queries, average 20 characters
prefixes per query: up to 20
→ 20,000,000,000 prefix entries if built naively

with the SKEW:
only ~10,000,000 queries account for 85% of traffic
their prefixes: 10,000,000 x 20 = 200,000,000 entries

each entry: prefix (~10 B) + 5 suggestions x ~20 B = ~110 B
200,000,000 x 110 B = ~22 GB

→ fits on one machine; shards comfortably to a few
→ and the popular subset (the top 1,000,000 prefixes) is
~110 MB, which fits at every CDN edge

That last line is the important one: the head of the index is small enough to replicate everywhere, which is what makes edge serving possible.

The nightly build.

5,000,000,000 queries/day of log data at ~50 bytes = 250 GB/day

MapReduce:
map: each query → up to 20 (prefix, query, score) pairs
= 100,000,000,000 intermediate records
reduce: top-5 per prefix

on a 500-machine cluster: ~1-2 hours

→ run it every 4-6 hours rather than nightly, which halves the staleness for the cost of more cluster time

The trending layer, for contrast:

10-minute window, per region
~1,000,000 distinct queries in 10 minutes globally
~50 regions → ~20,000 per region

20,000 entries x ~50 B = 1 MB per region
50 regions = 50 MB total

→ the trending layer is 50 MB against the index's 22 GB.
Small enough to update every minute, which is exactly why there are two layers.

Storage against a plain trie, which is why FSTs exist:

plain trie, 1 billion queries:

~10,000,000,000 nodes

each with a child map: ~100 bytes with pointer overhead

= ~1 TB

compressed trie (radix): ~10x fewer nodes → ~100 GB

FST (shared prefixes AND suffixes): → ~10-20 GB

→ the pointers outweigh the strings in a plain trie, which is the whole reason for the compressed forms.

Cost, roughly:

serving fleet (1B requests/day at origin) ~\$50,000/month

edge/CDN ~\$30,000/month

index build (6-hourly on a big cluster) ~\$40,000/month

trending layer (Redis) ~\$5,000/month

~\$125,000/month

for a feature that never returns a search result and exists purely to save the user typing.

→ which is worth stating: the justification is that it measurably increases completed searches, and it is expensive.

The abuse surface, sized:

a coordinated group issuing a query repeatedly to make it appear in autocomplete

trending layer, 10-minute window, ~20,000 queries per region:

~500 repetitions would put a query near the top

→ trivially cheap to attack

→ hence: per-user rate limiting on what COUNTS towards trending, requiring queries from distinct accounts and addresses, and a minimum distinct-user threshold before anything trends

7 The trade-offs

Precomputing against searching at request time. Precomputing turns a traversal into a lookup and **fits the twenty-millisecond budget**; searching does not. **The cost is staleness — a new query does not appear until the next build**, which is hours. **The trending layer patches the hole** at the cost of a second structure with a different update cycle and a merge on every request.

Index size against coverage. Storing every prefix of every query is twenty billion entries. **Storing only the popular prefixes is two hundred million and covers 85% of traffic** — and the remaining 15% falls back to a slower path. **That fallback must exist and be tested**, because it runs rarely and always for the users with unusual queries.

Client-side filtering against correctness. Filtering locally removes most of the traffic and **can return the wrong answer**: the cached top five for "man" may not contain the top five for "manch". **Guarding on “do I still have five results” makes it safe** at the cost of falling back more often — and getting that guard wrong degrades quality invisibly.

Personalisation on the device against on the server. On-device is free, private and instant, **and cannot use anything the device does not know** — no cross-device history, no collaborative signals. Server-side personalisation is more powerful and **costs a per-user lookup inside a twenty-millisecond budget**, plus a privacy surface. **Most of the value is in the user’s own recent queries, which the device already has.**

Trending window length. Ten minutes reacts fast and is **easy to manipulate** — a few hundred coordinated queries can promote a term. An hour is robust and slow. **The mitigation is not the window length but the counting rule**: distinct users, rate-limited, with a minimum threshold.

Filtering aggressiveness. Blocking too little produces offensive suggestions on a highly visible surface **and has produced lawsuits**. Blocking too much removes legitimate queries and is invisible — **nobody reports a suggestion that did not appear**. The asymmetry is entirely towards over-blocking, which is the right default and should be a stated decision.

When would I not build this? Almost always — **the completion suggester in Elasticsearch or a hosted service like Algolia does this well**, and below a few thousand queries a second the whole design is overhead. **A trie in memory on each application server is genuinely sufficient for a catalogue of a million items.** **Building this is justified by the traffic and the ranking**, not by the data structure — **and if the ranking is just alphabetical or by popularity, buy it.**

8 In the interview

How it gets asked

- “*Design search autocomplete.*” or “*Design a typeahead system.*” — the standard prompts.
- “*How do you make it fast enough?*” — the latency question, and the core of it.
- “*How do new or trending queries appear?*” — the precomputation gap.
- “*How do you rank the suggestions?*”
- “*How much traffic is this?*” — where the per-keystroke insight lands.
- “*How do you stop it suggesting something offensive?*”

The first ninety seconds

“Let me start with two numbers, because they determine the design.

The first: the latency budget. A fast typist leaves a hundred to two hundred milliseconds between keystrokes. **The network round trip on mobile is fifty to eighty of that.** So the server has twenty to fifty milliseconds — and a suggestion arriving after the next keystroke is worse than none, because the user has moved on.

The second, which surprises people: this is per keystroke, not per search. Five billion searches a day at fifteen characters each is **seventy-five billion auto-complete requests — fifteen times the search traffic.**

Those two numbers rule out the obvious design. In twenty milliseconds I cannot walk a trie over a billion queries, collect the completions, rank them, personalise and filter.

So the central move is: do not search — precompute. For every prefix, store its top five completions, already ranked. The request becomes a single hash lookup with no traversal at all.

The cost is staleness: a query that started trending an hour ago is not in an index built last night. **So there is a second, small, real-time layer** — query counts over the last ten minutes, per region — **and the two are merged at request time in about a millisecond.** Two structures, two update frequencies.

Now the part that I think matters most, and it is not the algorithm. The server work is about five milliseconds and the network is fifty to eighty. **So the biggest win available is not being on the network at all.**

Debouncing — wait fifty milliseconds for a pause — removes about three quarters of the requests. **Client-side filtering** — the results for “man” contain what “manch” needs — removes half of the rest. **And edge serving**, because the popular prefixes are about a hundred megabytes and fit at every point of presence.

Together those are about seventy-five times fewer requests reaching my servers, entirely from client and edge behaviour.

And one thing I would raise unprompted: filtering. Autocomplete puts words in a user’s mouth on a highly visible surface, **and suggesting something defamatory about a real person has produced lawsuits.** So there is a blocklist at build time and again at serve time, **because removal cannot wait for the next build.**“

The follow-ups

“How do you make it fast enough?”

“By making the request do almost nothing, and then by making most requests not happen at all.

The budget first, because it is tighter than people expect. A hundred to two hundred milliseconds between keystrokes, fifty to eighty of it network on mobile, five to render. **Twenty to fifty milliseconds for the server.**

In that time the obvious design cannot work. Walking a trie over a billion queries, collecting the completions beneath a popular prefix — which could be millions — ranking them and filtering them is tens of milliseconds at best and unbounded at worst.

So the answer is precomputation. Offline, for every prefix, compute and store the top five completions already ranked. **The request is one hash lookup.** About a millisecond, plus two for the trending layer and one for the merge — **five milliseconds of server time, comfortably inside the budget.**

But notice where the time actually goes: five milliseconds of server against fifty to eighty of network. So the algorithm is not the problem. The network is.

Three things, and they are where the real gains are.

Debounce — do not fire on every keystroke, wait fifty milliseconds for a pause. **A fast typist generates one request instead of four.**

Client-side filtering. The results for `"man"` contain everything the results for `"manc"` need, **so filter locally until the prefix leaves the cached set.** The guard is ‘do I still have five results after filtering’ — **because the cached top five for a short prefix is not necessarily the top five for a longer one**, and without that check the quality degrades invisibly.

And edge serving. The popular prefixes — the top million — are about a hundred megabytes. **That fits at every CDN point of presence**, which turns a transcontinental round trip into a local one: eighty milliseconds down to twenty.

Two smaller things that matter. Keep connections alive — a TLS handshake is another fifty milliseconds and would blow the budget on its own. **And make requests cancellable**, or a slow response for a short prefix arrives after a fast one for a longer prefix and overwrites it, **which is the out-of-order rendering bug that makes autocomplete feel broken.**“

“How do new and trending queries appear?”

“They do not, in the precomputed index — and that is the honest cost of pre-computing, so I would state it as a gap and then close it.

The index is built from query logs, and the build takes an hour or two over a day of data. So even running it every four to six hours, **a query that started trending twenty minutes ago is simply not there.**

And that is exactly when autocomplete matters most — something happens, millions of people search for it, and they are all typing the same new thing.

So: a second layer, small and fast. A count of query frequency over the last ten minutes, per region, **updated continuously** — one-minute buckets in Redis sorted sets with a TTL, so old buckets expire themselves and there is no cleanup job.

At request time, look up both and merge. The precomputed layer gives the stable, well-ranked head; **the trending layer contributes anything that has become popular in the last few minutes.**

The sizes make it work. The main index is around twenty-two gigabytes; **the trending layer is about fifty megabytes across all regions** — small enough to update every minute, **which is precisely why they are separate structures rather than one.**

Per region matters too, because what is trending in one country is irrelevant in another — **and it keeps each list short enough that a simple prefix scan over the top few hundred is fast enough,** rather than needing a second indexed structure.

And I would raise the abuse problem unprompted, because it is real. With a ten-minute window and maybe twenty thousand distinct queries per region, **a few hundred coordinated repetitions would put a query near the top.** That is trivially cheap to attack.

The mitigation is not a longer window — that just makes it slower to react. **It is the counting rule: count distinct users rather than queries, rate-limit what counts, and require a minimum number of distinct accounts before anything is eligible to trend.**“

“How do you rank the suggestions?”

“Popularity is the base and it is not enough on its own — **a purely popularity-ranked list is stale and generic**, and the signals that improve it are cheap.

Frequency, damped. How often the query is issued, **but through a logarithm**, because raw counts let a handful of enormous queries dominate every prefix they touch **and squeeze out the merely popular**.

Recency, with something like a two-week decay. **Search interest moves**, and a query that was popular last year and is dead now should not outrank one that is currently rising.

Click-through rate, which is the strongest of the server-side signals. It measures whether a suggestion was *useful* rather than whether the query is common. **A query people type but never select from the suggestions should rank lower** — and that distinction is invisible to frequency alone.

Location and time, which are cheap and effective. ‘Weather’ means something different in two countries; ‘cricket score’ means something specific during a match.

And then personal history, which is the strongest signal of all — and the interesting thing about it is that it need not touch my infrastructure.

A user’s own recent queries are a short list that can live on the device. So personalisation can be a client-side reordering: **boost anything the user has searched before, then everything else.**

That is free, private, instant, and it is the highest-value signal available — which is a rare combination and worth pointing at explicitly. **No server cost inside a twenty-millisecond budget, and no per-user data leaving the device.**

What it cannot do is cross-device or collaborative signals — ‘people like you also searched’ — which would need server-side personalisation. **I would only add that if measurement showed it was worth the latency and the privacy surface**, because the device already has most of the value.”

The model answer

“Design autocomplete for a large search engine: five billion searches a day, global, and suggestions must appear as the user types.”

“Two numbers first, because they settle the architecture.

The latency budget: a hundred to two hundred milliseconds between keystrokes, fifty to eighty of it network — so twenty to fifty milliseconds of server time. And a suggestion that arrives after the next keystroke is worse than none.

And the traffic, which surprises people: this is per keystroke. Five billion searches at fifteen characters is **seventy-five billion requests a day, fifteen times the search traffic itself.**

Those two rule out searching at request time, so the central move is **pre-computation: every prefix maps directly to its top five completions, already ranked.** The request is one hash lookup.

Sizing that index. A billion distinct queries, but the distribution is extraordinarily skewed — **ten million queries account for eighty-five percent of traffic.** Their prefixes, capped at twenty characters, are about two hundred million entries at roughly a hundred and ten bytes: **twenty-two gigabytes.** Fits on one machine, shards easily. **And the top million prefixes are about a hundred megabytes, which fits at every CDN edge** — that is what makes edge serving possible.

Storage: an FST rather than a plain trie. A plain trie over a billion queries is roughly a terabyte because **the pointers outweigh the strings**; an FST shares prefixes *and* suffixes and gets it to ten or twenty gigabytes. **Or, since the answers are precomputed, a flat hash map from prefix to results is simpler and shards trivially** — and walking was never needed.

The gap in precomputation is trending, so a second layer: **query counts over a ten-minute window, per region, in Redis sorted sets with per-minute buckets and TTLs.** About fifty megabytes globally against the index’s twenty-two gigabytes — **small enough to update every minute, which is exactly why it is a separate structure.** Merged with the main results in about a millisecond.

Ranking: log-damped frequency, two-week recency decay, and click-through rate — the last being the strongest, because it measures usefulness rather than volume. **Plus location and time.**

And personalisation on the device, boosting the user’s own recent queries. **The strongest single signal, and it costs my servers nothing and exposes no data** — worth calling out as the unusual case where the best option is also the cheapest.

Now the part where the real engineering is, which is delivery. Server work is about five milliseconds and network is fifty to eighty, **so the win is in not being on the network.**

Debouncing at fifty milliseconds takes seventy-five billion to twenty. **Client-side filtering**, guarded on still having five results, takes it to ten. **Edge caching at ninety percent** takes it to about one billion reaching origin. **Seventy-five times fewer, entirely from client and edge behaviour.**

Two things I would insist on.

Filtering, at build time and again at serve time. Autocomplete puts words in a user's mouth on a very visible surface, **and completing a real person's name with a defamatory phrase has produced actual lawsuits.** The serve-time filter exists specifically **because a removal cannot wait for the next build.** And the asymmetry favours over-blocking: **nobody ever reports a suggestion that did not appear.**

And abuse of the trending layer. With a ten-minute window, **a few hundred coordinated queries could promote a term** — trivially cheap. **The fix is the counting rule, not the window: distinct users, rate-limited, with a minimum account threshold.**

Closing thought on cost. This is around a hundred and twenty-five thousand dollars a month for a feature that **never returns a search result** — it exists purely to save typing. **The justification has to be measured** — completed searches, time to result — **and I would want that measurement to exist, because it is a genuinely expensive convenience.**

9 Recall card

Two numbers settle the design. The **latency budget:** ~100–200 ms between key-strokes minus ~50–80 ms of network leaves **20–50 ms of server time**, and a late suggestion is worse than none. **And the traffic is per KEYSTROKE, not per search** — 5B searches × ~15 characters = **75B requests/day, ~15 × search traffic.**

The central inversion: do not search, PRECOMPUTE. Every prefix maps to its **top five, already ranked** — one hash lookup, no traversal. **The cost is staleness**, patched by a **second, small, real-time trending layer** (~50 MB against the index's ~22 GB) merged at request time. **Two structures, two update frequencies.**

A plain trie is ~1 TB for a billion queries — the pointers outweigh the strings. Use a **radix tree or an FST** (shared prefixes *and* suffixes, ~10–20 GB), **or just a flat hash map**, since precomputed answers never need walking. **Cap prefixes at ~20 characters** or storage grows without bound.

Server work is ~5 ms and network is 50–80 ms, so the biggest win is NOT BEING ON THE NETWORK. **Debounce** ($\sim 4\times$), **client-side filtering** ($\sim 2\times$, guarded on still having five results — the cached top-5 for "man" is not necessarily the top-5 for "manch"), and **edge serving** the ~100 MB popular head. **$\sim 75\times$ fewer requests reach origin, entirely from client and edge behaviour.** Keep connections alive, and **make requests cancellable** or a slow short-prefix response overwrites a fast long-prefix one.

Ranking: log-damped frequency, recency decay, and click-through (the strongest server-side signal — it measures usefulness, not volume). **Personal history is the strongest signal overall and can live entirely on the device** — free, private, instant.

Filter at build time AND at serve time, because removal cannot wait for the next build — completing a real person's name defamatorily has produced lawsuits, and **the asymmetry favours over-blocking** since nobody reports a suggestion that never appeared. **And the trending layer is trivially cheap to attack** (~ 500 repetitions in a 10-minute window): fix it with the **counting rule** — **distinct users, rate-limited** — not with a longer window.

DSA topic: Interval scheduling **System design topic:** Design search autocomplete at scale

Code these, in this order

One rule for the whole set: **before writing anything, say which of the three techniques this is and what the sort key is.** Greedy selection sorts by end, merging sorts by start, the sweep line sorts the endpoints — and picking the wrong one gives a plausible number at the same complexity with no error.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Merge Intervals	LeetCode 56 (Medium)	Sort by start, and <code>max</code> on the extension.
2	Insert Interval	LeetCode 57 (Medium)	Whether you notice the input is already sorted.
3	Non-overlapping Intervals	LeetCode 435 (Medium)	Sort by end, and that it is a subtraction.
4	Meeting Rooms II	LeetCode 253 (Medium)	The sweep line, and the tie-break at equal times.
5	Minimum Number of Arrows	LeetCode 452 (Medium)	The same greedy in disguise.
6	Maximum Profit in Job Scheduling	LeetCode 1235 (Hard)	Weighted — greedy fails, DP with a binary search.

On all six, classify before coding

For each, write down: **which technique, which sort key, and what the boundary convention is.** Then solve it. **Then check whether your classification was right** — and if it was not, say which question you thought you were answering.

On problem 1, extend with `= end`

Replace `max(current_end, end)` with `= end`. Run on `[(1,10), (2,3), (11,12)]` and record the output. **Say what shape of input triggers it**, and why that shape is rare in hand-written tests.

On problem 1, sort by end instead

Run the merge with the wrong key on the same input. **Record the output and say precisely what went wrong** — which part of which interval was lost, and why there is no code path that could recover it.

On problem 2, sort the input

Solve it by appending the new interval and re-sorting. Confirm it gives the right answer. **Then say what it costs and what the problem was actually testing**, and write the three-phase $O(n)$ version.

On problem 3, notice the subtraction

Before coding, say in one sentence how “minimum removals” relates to “maximum kept”. **Then write one function and derive the other.**

On problem 4, get the tie-break wrong

Encode the events so that starts sort before ends at equal times. Run on $[(1,4), (4,6), (6,9)]$ and record the answer against 1. **Say why $(\text{time}, \text{delta})$ tuples give the right convention for free**, and what would happen if you encoded the events as strings.

On problem 4, produce the assignment

Extend your solution to say which meeting goes in which room. **Say why the sweep line cannot do this** and what structure you need instead.

On problem 6, watch greedy fail

Write the greedy — sort by end, take anything compatible, sum the values — and run it against the DP on $[(0,10,100), (0,3,10), (3,6,10), (6,9,10)]$. Record both. **Then replace the binary search with a linear scan** and say what the complexity becomes and at what n it stops finishing.

Then the classification drill

For each of these, name the technique and the sort key without solving it:

1. Maximum number of non-overlapping intervals.
2. Merge overlapping intervals.
3. Minimum meeting rooms.
4. Do any two intervals overlap?
5. Insert into a sorted disjoint list.
6. Maximum total value of non-overlapping intervals.

7. The busiest minute of the day.
8. Minimum removals to make the set disjoint.

Three are greedy, three are merging, and two are neither. Say which.

Then the boundary drill

For each, state whether `[2, 4]` and `[4, 6]` conflict, and what comparison you would write:

1. Meeting rooms.
 2. Non-overlapping meeting selection.
 3. Two inclusive number ranges.
 4. Two half-open time slots.
-

The classification drill

1. Give the three techniques and the question each answers.
2. Give the sort key for each and the invariant it establishes.
3. Say what happens when you use the wrong key — for both directions.
4. Say why there is no performance signal warning you.

The selection drill

1. Give the algorithm in four lines.
2. Give the exchange argument for the end-time sort.
3. Say why start time fails, with an input.
4. Say why duration fails, and why it is the more dangerous wrong key.
5. Give the one-line relationship to “minimum removals”.

The merging drill

1. Give the algorithm.
2. Say what invariant sorting by start establishes.
3. Give the line people get wrong and the input that catches it.
4. Say what the wrong sort key does to the output.

The sweep drill

1. Describe the decomposition in one sentence.
2. Say what you deliberately stop caring about.

3. Give the tie-break rule and why.
4. Say why `(time, delta)` sorting gets it right for free.
5. Give the two-pointer form and say whether it differs.
6. Say what the sweep cannot answer, and what does.

The weighted drill

1. Give the counter-example that breaks greedy.
2. Give the DP recurrence.
3. Say what `p(i)` is and how it is found.
4. Say what a linear scan for `p(i)` costs.
5. Give the complexity, and compare it with the greedy version.
6. Say what one word in the problem statement signals this.

The break-it drill

For each, say what happens and whether anything reports it:

1. Selection sorted by start.
 2. Merging sorted by end.
 3. `= end` instead of `max` when merging.
 4. Starts before ends at equal times in the sweep.
 5. Re-sorting an already-sorted input.
 6. Greedy on the weighted version.
 7. A linear scan for `p(i)`.
 8. `.sort()` on the caller's list.
-

The budget drill

1. Give the latency budget, itemised.
2. Say how much is left for the server.
3. Say why a late suggestion is worse than none.
4. Say what a TLS handshake would do to the budget.

The traffic drill

1. Compute autocomplete requests from searches per day.
2. Say what the ratio to search traffic is, and why.
3. Give the three client-side reductions and the factor for each.

4. Compute what reaches origin after all three.

The precomputation drill

1. Say what the obvious design does and why it does not fit.
2. Describe the precomputed structure.
3. Say what the request becomes.
4. Say what the cost is, and what closes the gap.

The structure drill

1. Give the size of a plain trie at a billion queries, and why.
2. Name the two compressed forms and roughly what each saves.
3. Say why a flat hash map is often the right answer.
4. Say why prefixes are capped, and at roughly what length.

The layers drill

1. Give both layers with their sizes and update frequencies.
2. Say why they cannot be one structure.
3. Describe the trending layer's storage shape.
4. Say why it is per region.

The ranking drill

1. Name five signals.
2. Say why frequency goes through a logarithm.
3. Say which server-side signal is strongest, and what it measures.
4. Say which signal is strongest overall and where it can live.
5. Say what on-device personalisation cannot do.

The delivery drill

1. Say where the time actually goes.
2. Give the client-side filtering guard and why it is needed.
3. Say what a cancellable request prevents.
4. Compute the size of the edge-cacheable head.

The safety drill

1. Say why filtering is not optional here.
2. Say why it runs at serve time as well as build time.

3. State the asymmetry and which way it points.
 4. Compute how cheap the trending layer is to attack.
 5. Give the mitigation, and say what it is not.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Merge these intervals — and then, how many meeting rooms do I need?* The two techniques, both sort keys and the invariant each establishes, the `max` on the extension, and the sweep tie-break.
 2. *Now each interval has a value.* Why greedy fails with the counter-example, the DP recurrence, what `p(i)` is and why it is a binary search, and why the complexity is unchanged.
 3. *Design search autocomplete.* The latency budget and the per-keystroke traffic, pre-computation instead of search, the two layers, and where the delivery wins actually are.
-

Before you move on

- [] I classify before coding: which technique, which sort key.
- [] I know selection sorts by end, and the exchange argument for it.
- [] I know merging sorts by start, and what invariant that gives.
- [] I know the sweep sorts endpoints and discards the association.
- [] I know what the wrong sort key does in both directions.
- [] I know there is no performance signal to warn me.
- [] I write `max(current_end, end)` and know the input that catches `= end`.
- [] I know “minimum removals” is the same problem, subtracted.
- [] I settle the boundary convention before coding.
- [] I know `(time, delta)` sorting gives the right tie-break free.
- [] I know the sweep counts and a heap assigns.
- [] I know the two-pointer form is the same algorithm.
- [] I do not re-sort an already-sorted input.
- [] I know the weighted counter-example and both numbers.

- [] I can give the weighted DP recurrence.
- [] I know $p(i)$ is a binary search and what a linear scan costs.
- [] I know weighted DP is the same $O(n \log n)$ as the greedy.
- [] I know “number” versus “value” is the signal.
- [] I can give the autocomplete latency budget, itemised.
- [] I know autocomplete traffic is $\sim 15 \times$ search traffic, and why.
- [] I know why precomputation replaces search, and what it costs.
- [] I can describe both layers and their sizes.
- [] I know a plain trie is ~ 1 TB and why.
- [] I can name the compressed forms and what they save.
- [] I know why prefixes are capped.
- [] I can name five ranking signals and which is strongest.
- [] I know on-device personalisation is free and what it cannot do.
- [] I know the three client-side reductions and their combined factor.
- [] I know the client-filtering guard and what a cancellable request prevents.
- [] I know why filtering runs at serve time too, and which way the asymmetry points.
- [] I answered all three questions above out loud.

Merging intervals

DATA STRUCTURES AND ALGORITHMS

Merging intervals

SYSTEM DESIGN

Design a leaderboard

1 What this is, and why they ask it

Given a set of intervals, combine every group that overlaps into a single interval. `[1,3]` , `[2,6]` , `[8,10]` becomes `[1,6]` , `[8,10]` .

Yesterday classified it as one of three techniques. Today is the technique itself, in full — because merging is the most-asked interval problem there is, it has five variants that all appear, and it has one specific bug that survives almost every test anybody writes by hand.

They ask it because it is short enough to write in five minutes and has enough detail to fill twenty. The core loop is six lines. The follow-ups — insert into a sorted list, find the gaps, intersect two lists, do it as a stream — are all natural, all different, and all reuse the same idea. An interviewer can start easy and go as deep as they like without changing problems.

And the specific bug is worth naming immediately, because it is the point of the lesson. When two intervals overlap, the merged end is `max(current_end, next_end)` — not `next_end` . An interval entirely *inside* the current block would otherwise shrink it. Containment is much rarer than partial overlap in hand-written tests, so the wrong version passes, and it fails on real data where a long booking swallows a short one.

By the end of this lesson you can write the merge and defend the sort key, handle the five variants, get the boundary convention right, and know what changes when the input arrives as a stream.

2 The story

The road was being dug up in four places by four different people and none of them knew about the others, and this was the entire content of Hemalatha's job.

She sat in the corporation office and issued the permits.

The water people wanted from the junction to the school, for eleven days. The cable people wanted from the school to the temple, starting on the fourth. The gas people wanted a stretch in the middle. And somebody was always relaying the surface afterwards.

Her predecessor had issued them as they came and the road had been dug eleven times in one year.

What she did instead took her about a month to get right and it was not complicated.

She stopped thinking about the permits and started thinking about the road.

Every morning she laid the requests out **in order of where they started** — not who asked, not how long, not how important. **Where they started.** And she ran a finger along.

Junction to school. Then school to temple — that touches, so it is one stretch: junction to temple.

Then nothing until the bus stand.

And the thing that caught her out, in the second month, was the small one.

Because the gas people wanted forty metres in the middle of a stretch she had already joined up — **entirely inside it** — and she wrote down the new stretch as ending where the gas work ended.

Which was two hundred metres shorter than what she had already agreed.

The men turned up, dug their forty metres, relaid it, and **the stretch beyond it — which was already permitted and already dug — got resurfaced by nobody**, because her book said the work had ended.

It was a small mistake and it cost four months.

After that she said the same six words to herself on every single one, and she was still saying them eleven years later.

“Further of the two. Not the new one.”

Her assistant thought it was a strange thing to keep saying about something so obvious.

“It is obvious,” she said. “It is also the only mistake I have ever made in this job, and I made it because the small one came along and I stopped thinking.”

3 The idea in plain English

Hemalatha’s finger along the road is the algorithm, and her six words are the bug.

The algorithm, in full.

Sort by start. Walk through. If the next interval begins at or before the current block ends, extend the block to the further of the two ends. Otherwise close the block and begin a new one.

That is it, and the sort is what makes the single pass legal.

Why sorting by start is the right key, precisely: with intervals in start order, anything that overlaps the current merged block must begin before that block ends. So one comparison per interval is enough — you never have to look backwards, and you never have to reconsider a block once it is closed.

Sorted by end, that property does not hold. An interval can start before the current block and extend past everything seen so far, and the single pass misses it.

Now the bug, which is the point of the day.

current block: [1, 10]	
next interval: [2, 3]	entirely INSIDE the block
extend to $\max(10, 3) = 10$	CORRECT → [1, 10]
extend to 3	WRONG → [1, 3]

The block shrank. And the version that shrinks it passes almost every hand-written test, because people write overlapping intervals that extend each other — [1,3] and [2,6] — and rarely write one that is entirely contained.

Say Hemalatha's words while writing the line: "further of the two, not the new one."

Then the boundary convention, which is the second most common bug.

Do [1,3] and [3,5] merge?

For time slots and road stretches: yes — they touch, and there is no gap. `start ≤ current_end`.

For sets of discrete integers where the endpoints are exclusive: no — [1,3) and [3,5) are adjacent and do not overlap. `start < current_end`.

One character, and the problem statement decides it. LeetCode's Merge Intervals treats touching as overlapping; some interviewers deliberately ask about the other convention to see whether you ask.

Now the five variants, which is why this fills twenty minutes.

One: insert into a sorted, disjoint list. The input is already sorted and non-overlapping — so re-sorting is $O(n \log n)$ where the answer is $O(n)$, and noticing that is what the problem tests.

Three phases, written as three loops rather than one loop with branches: everything entirely before the new interval, copied; everything overlapping it, absorbed; everything after, copied.

Two: the gaps — the complement. “When is the room free?” is the merged blocks inverted: the space between consecutive merged blocks, plus the space before the first and after the last. The two boundary cases at the ends are where the bugs are, and they need an explicit range to complement within.

Three: intersecting two lists. Two people’s calendars; when are both free, or both busy? Two pointers, and the intersection of $[a,b]$ and $[c,d]$ is $[\max(a,c), \min(b,d)]$, valid when that start is at most that end. Advance whichever interval ends first, because it can intersect nothing further.

Four: counting the merged blocks, or the total covered length. Falls out of the merge — $\text{sum}(\text{end} - \text{start})$ over the merged blocks, and doing it over the *unmerged* ones double-counts the overlaps, which is the whole reason to merge first.

Five: the streaming version, which is the interesting one. If intervals arrive one at a time and you cannot sort, the algorithm does not apply — the single pass depends on start order.

What you use instead is a sorted structure: keep the merged blocks in a balanced tree or sorted list keyed by start, and on each arrival **binary search for the neighbours**, then merge with any that overlap. $O(\log n)$ to find, and the merge can absorb several blocks at once, so it is $O(\log n + k)$ where k is how many it swallows.

And the amortised total is $O(n \log n)$, because each block can be absorbed only once.

Finally, the representation detail that causes real bugs.

`sorted(intervals)` in Python sorts by start and then by end, which is what merging wants — so the default is correct here and is wrong for selection. Being explicit with `key=lambda x: x[0]` says which you meant and costs nothing.

And build the output as a list of mutable lists, or use an explicit `current_start, current_end` pair — because tuples cannot be extended in place, and the version that appends a tuple and then tries to modify it is a real and common mistake.

4 The picture

The algorithm, walked:

```
input (sorted by START): [1,3] [2,6] [8,10] [15,18]

current = [1,3]
[2,6]: 2 ≤ 3 → overlap → extend to max(3, 6) = 6 [1,6]
[8,10]: 8 > 6 → gap → close [1,6], current = [8,10]
[15,18]: 15 > 10 → gap → close [8,10], current = [15,18]
end: → close [15,18]
```

```
output: [1,6] [8,10] [15,18]
```

ONE PASS. Never look backwards, never reopen a closed block.
That is what sorting by START buys.

The bug, which is the point of the lesson:

```
current block: [1 ===== 10]
next interval: [2 = 3] ENTIRELY INSIDE

max(current_end, next_end) = max(10, 3) = 10
→ [1, 10] CORRECT

next_end = 3
→ [1, 3] WRONG – the block SHRANK

AND IT PASSES YOUR TESTS, because hand-written cases look like
[1,3] and [2,6] – partial overlap, where the new end IS further.

Containment is rare in test data and common in real data,
where a long booking swallows a short one.

"Further of the two. Not the new one."
```

Why sorting by start, and not by end:

BY START – the invariant that makes one pass work

anything overlapping the current block must BEGIN before it ends
→ one comparison per interval decides it
→ never look backwards

BY END – the invariant does not hold

input [1,10] [2,3] [11,12], sorted by end: [2,3] [1,10] [11,12]

current = [2,3]

[1,10]: $1 \leq 3 \rightarrow$ extend END to 10 $\rightarrow$ [2,10]
but the START should have been 1

$\rightarrow$ [2,10] [11,12]

THE BLOCK STARTS AT 2 INSTEAD OF 1.

There is no code path that reaches backwards to fix a start,
and the block COUNT is right, so a test counting blocks passes.

The boundary convention:

[1, 3] and [3, 5]

TOUCHING COUNTS AS OVERLAPPING (time slots, road stretches):
 $\text{start} \leq \text{current_end} \rightarrow$ merges to [1, 5]

TOUCHING DOES NOT (half-open ranges, discrete sets):
 $\text{start} < \text{current_end} \rightarrow$ stays as [1,3] [3,5]

ONE CHARACTER. The problem statement decides.
LeetCode 56 merges them. Ask if it is not stated.

The variants, on one page:

MERGE	sort by start, one pass	$O(n \log n)$
INSERT	input ALREADY sorted $\rightarrow$ three phases (re-sorting is the mistake being tested)	$O(n)$
GAPS	the complement of the merged blocks (the two ends are where the bugs are)	$O(n \log n)$
INTERSECT TWO	two pointers, $[\max(a,c), \min(b,d)]$ advance whichever ENDS first	$O(n + m)$
TOTAL COVERED	sum(end - start) over MERGED blocks (over unmerged blocks it double-counts)	$O(n \log n)$
STREAMING	a sorted structure + binary search (the one-pass algorithm does not apply)	$O(\log n + k)$

Intersecting two lists, and the pointer rule:

```

A:  [0,2]      [5,10]      [13,23]    [24,25]
B:   [1,5]      [8,12]      [15,24]   [25,26]

```

at each step:

```

  lo = max(a.start, b.start)
  hi = min(a.end,   b.end)
  if lo ≤ hi: emit [lo, hi]
  ADVANCE WHICHEVER ENDS FIRST

```

```

[0,2] & [1,5] → [1,2]    a ends first → advance a
[5,10] & [1,5] → [5,5]    b ends first → advance b
[5,10] & [8,12] → [8,10]  a ends first → advance a
...

```

WHY advance the one that ends first: it cannot intersect anything further in the other list, so it is finished.

The streaming version:

ONE-PASS MERGE

needs the whole input
sorts once
 $O(n \log n)$, one scan

on each arrival:

```

  binary search for the neighbours
  merge with any that overlap
  (it may absorb SEVERAL at once)

```

→ $O(\log n + k)$ per insert

→ amortised $O(n \log n)$ total, because each block can be absorbed only ONCE

STREAMING

intervals arrive one at a time
cannot sort
a sorted structure, keyed by start

$O(\log n)$

$O(k)$

5 The code, built step by step

The merge

```
def merge(intervals: list[tuple[int, int]]) → list[tuple[int, int]]:
    """Sort by START. Extend to the FURTHER of the two ends."""
    if not intervals:
        return []
    ordered = sorted(intervals, key=lambda x: x[0])
    merged: list[list[int]] = [list(ordered[0])]
    for start, end in ordered[1:]:
        if start ≤ merged[-1][1]:
            # touching counts
            merged[-1][1] = max(merged[-1][1], end) # FURTHER OF THE TWO
        else:
            merged.append([start, end])
    return [tuple(block) for block in merged]
```

`max(merged[-1][1], end)` is the line. Say Hemalatha's words while writing it.

`list(ordered[0])` and the list-of-lists are deliberate: **tuples cannot be modified in place**, and the version that appends a tuple and then tries to extend it is a real mistake. **Converting back at the end keeps the interface clean.**

And `start ≤ merged[-1][1]` is the boundary convention — `≤` merges touching intervals, `<` does not.

The variant without a mutable list

```
def merge_pairs(intervals: list[tuple[int, int]]) → list[tuple[int, int]]:
    """Same algorithm, carrying the current block in two variables."""
    if not intervals:
        return []
    ordered = sorted(intervals, key=lambda x: x[0])
    result: list[tuple[int, int]] = []
    current_start, current_end = ordered[0]
    for start, end in ordered[1:]:
        if start ≤ current_end:
            current_end = max(current_end, end)
        else:
            result.append((current_start, current_end))
            current_start, current_end = start, end
    result.append((current_start, current_end)) # the LAST block
    return result
```

`result.append(...)` after the loop is the line people forget, and it silently drops the final block — **which on a single-block input returns an empty list.**

This form is slightly clearer about what “the current block” is, at the cost of remembering the flush.

Inserting into a sorted list

```
def insert_interval(intervals: list[tuple[int, int]],
                    new: tuple[int, int]) → list[tuple[int, int]]:
    """Input is ALREADY sorted and disjoint. Three phases.  $O(n)$ , not  $O(n \log n)$ ."""
    result: list[tuple[int, int]] = []
    start, end = new
    i, n = 0, len(intervals)

    while i < n and intervals[i][1] < start:           # entirely before
        result.append(intervals[i])
        i += 1

    while i < n and intervals[i][0] ≤ end:             # overlapping: absorb
        start = min(start, intervals[i][0])
        end = max(end, intervals[i][1])
        i += 1

    result.append((start, end))
    result.extend(intervals[i:])                       # entirely after
    return result
```

Three loops rather than one loop with branches is what makes this readable, and the phases are exactly the three cases.

The absorbing loop widens in both directions — `min` on the start and `max` on the end — **because the new interval can extend past existing blocks on either side.**

And $O(n)$ rather than $O(n \log n)$ is the whole point of the problem: the input being sorted is stated, and re-sorting throws that away.

The gaps

```
def free_time(intervals: list[tuple[int, int]],
              lower: int, upper: int) → list[tuple[int, int]]:
    """The complement of the merged blocks, within [lower, upper]."""
    blocks = merge(intervals)
    gaps: list[tuple[int, int]] = []
    cursor = lower
    for start, end in blocks:
        if start > cursor:
            gaps.append((cursor, start))                # the gap BEFORE this block
            cursor = max(cursor, end)
        if cursor < upper:
            gaps.append((cursor, upper))                # the gap AFTER the last
    return gaps
```

The two boundary cases are where the bugs are: the gap before the first block and the gap after the last. **Both need an explicit range to complement within** — “when is the room free” is meaningless without knowing the day’s bounds.

`cursor = max(cursor, end)` rather than `cursor = end` guards against a block that lies entirely before `lower`, which is the same containment mistake in a different place.

Intersecting two lists

```
def intersect(a: list[tuple[int, int]],
             b: list[tuple[int, int]]) → list[tuple[int, int]]:
    """Both already sorted and disjoint. Two pointers. O(n + m)."""
    result: list[tuple[int, int]] = []
    i = j = 0
    while i < len(a) and j < len(b):
        lo = max(a[i][0], b[j][0])
        hi = min(a[i][1], b[j][1])
        if lo ≤ hi:
            result.append((lo, hi))
            if a[i][1] < b[j][1]: # advance whichever ENDS first
                i += 1
            else:
                j += 1
    return result
```

`[max(start, start), min(end, end)]` is the intersection of two intervals, valid exactly when the result is non-empty.

And advancing whichever ends first is the correctness argument: that interval cannot intersect anything later in the other list, because the other list is sorted and everything after starts later.

Total covered length

```
def total_covered(intervals: list[tuple[int, int]]) → int:
    """Merge first. Summing the raw intervals double-counts overlaps."""
    return sum(end - start for start, end in merge(intervals))

def total_raw(intervals: list[tuple[int, int]]) → int:
    """For contrast: what NOT merging gives you."""
    return sum(end - start for start, end in intervals)
```

Two lines, and the difference between them is the entire reason to merge — `[1,10]` and `[2,3]` cover nine units, not ten.

The streaming version

```
import bisect

class StreamingMerger:
    """Intervals arrive one at a time. The one-pass algorithm does not apply."""

    def __init__(self) → None:
        self.starts: list[int] = []
        self.ends: list[int] = []

    def add(self, start: int, end: int) → None:
        # the first block that could possibly overlap
        i = bisect.bisect_left(self.ends, start)
        # absorb every block that begins at or before our end
        j = i
        while j < len(self.starts) and self.starts[j] ≤ end:
            start = min(start, self.starts[j])
            end = max(end, self.ends[j])
            j += 1
        self.starts[i:j] = [start]
        self.ends[i:j] = [end]

    def blocks(self) → list[tuple[int, int]]:
        return list(zip(self.starts, self.ends))
```

`bisect_left(self.ends, start)` finds the first block whose end reaches our start — everything before it is entirely to the left and cannot overlap.

The `while` loop absorbs several blocks at once, which is the case people miss: a new interval spanning a gap swallows the blocks on both sides.

And `self.starts[i:j] = [start]` replaces the whole absorbed range with one block in a single slice assignment, which is both correct and much cleaner than deleting in a loop.

Amortised $O(n \log n)$, because a block can be absorbed only once.

The wrong versions, for comparison

```
def merge_shrinking(intervals: list[tuple[int, int]]) → list[tuple[int, int]]:
    """THE BUG: `= end` instead of `max(...)`."""
    if not intervals:
        return []
    ordered = sorted(intervals, key=lambda x: x[0])
    merged = [list(ordered[0])]
    for start, end in ordered[1:]:
        if start ≤ merged[-1][1]:
            merged[-1][1] = end                # WRONG
        else:
            merged.append([start, end])
    return [tuple(b) for b in merged]

def merge_by_end(intervals: list[tuple[int, int]]) → list[tuple[int, int]]:
    """THE OTHER BUG: the wrong sort key."""
    if not intervals:
        return []
    ordered = sorted(intervals, key=lambda x: x[1])
    merged = [list(ordered[0])]
    for start, end in ordered[1:]:
        if start ≤ merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return [tuple(b) for b in merged]
```

Keeping the broken versions next to the correct one and running all three is worth five minutes, because the failures are invisible from reading and immediate from testing.

The complete solution

```
"""Merging intervals: the algorithm, the bug, and the five variants."""

import bisect
import random

def merge(intervals: list[tuple[int, int]]) → list[tuple[int, int]]:
    """Sort by START; extend to the FURTHER of the two ends."""
    if not intervals:
        return []
    ordered = sorted(intervals, key=lambda x: x[0])
    merged: list[list[int]] = [list(ordered[0])]
    for start, end in ordered[1:]:
        if start ≤ merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end) # ≤ : touching merges
            # "further of the two"
        else:
            merged.append([start, end])
    return [tuple(b) for b in merged]

def merge_strict(intervals: list[tuple[int, int]]) → list[tuple[int, int]]:
    """The other convention: touching does NOT merge."""
    if not intervals:
        return []
    ordered = sorted(intervals, key=lambda x: x[0])
    merged: list[list[int]] = [list(ordered[0])]
    for start, end in ordered[1:]:
        if start < merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end) # < : touching stays apart
        else:
            merged.append([start, end])
    return [tuple(b) for b in merged]

def insert_interval(intervals: list[tuple[int, int]],
                    new: tuple[int, int]) → list[tuple[int, int]]:
    """Input already sorted and disjoint → O(n), three phases."""
    result: list[tuple[int, int]] = []
    start, end = new
    i, n = 0, len(intervals)
    while i < n and intervals[i][1] < start:
        result.append(intervals[i])
        i += 1
    while i < n and intervals[i][0] ≤ end:
        start, end = min(start, intervals[i][0]), max(end, intervals[i][1])
        i += 1
    result.append((start, end))
    result.extend(intervals[i:])
    return result

def free_time(intervals: list[tuple[int, int]],
              lower: int, upper: int) → list[tuple[int, int]]:
    """The gaps. The two ends are where the bugs live."""
    gaps: list[tuple[int, int]] = []
    cursor = lower
    for start, end in merge(intervals):
        if start > cursor:
            gaps.append((cursor, start))
```

```

        cursor = max(cursor, end)
    if cursor < upper:
        gaps.append((cursor, upper))
    return gaps

def intersect(a: list[tuple[int, int]],
             b: list[tuple[int, int]]) → list[tuple[int, int]]:
    """Two sorted disjoint lists. Advance whichever ENDS first."""
    result: list[tuple[int, int]] = []
    i = j = 0
    while i < len(a) and j < len(b):
        lo, hi = max(a[i][0], b[j][0]), min(a[i][1], b[j][1])
        if lo ≤ hi:
            result.append((lo, hi))
            if a[i][1] < b[j][1]:
                i += 1
            else:
                j += 1
    return result

def total_covered(intervals: list[tuple[int, int]]) → int:
    return sum(e - s for s, e in merge(intervals))

def total_raw(intervals: list[tuple[int, int]]) → int:
    return sum(e - s for s, e in intervals)

class StreamingMerger:
    """Arrivals one at a time: a sorted structure, not a single pass."""

    def __init__(self) → None:
        self.starts: list[int] = []
        self.ends: list[int] = []

    def add(self, start: int, end: int) → None:
        i = bisect.bisect_left(self.ends, start)
        j = i
        while j < len(self.starts) and self.starts[j] ≤ end:
            start = min(start, self.starts[j])
            end = max(end, self.ends[j])
            j += 1
        self.starts[i:j] = [start]
        self.ends[i:j] = [end]

    def blocks(self) → list[tuple[int, int]]:
        return list(zip(self.starts, self.ends))

# ----- the broken versions, for comparison -----

def merge_shrinking(intervals: list[tuple[int, int]]) → list[tuple[int, int]]:
    if not intervals:
        return []
    ordered = sorted(intervals, key=lambda x: x[0])
    merged = [list(ordered[0])]
    for start, end in ordered[1:]:
        if start ≤ merged[-1][1]:
            merged[-1][1] = end
        else:
            # THE BUG

```

```

        merged.append([start, end])
    return [tuple(b) for b in merged]

def merge_by_end(intervals: list[tuple[int, int]]) → list[tuple[int, int]]:
    if not intervals:
        return []
    ordered = sorted(intervals, key=lambda x: x[1])    # THE OTHER BUG
    merged = [list(ordered[0])]
    for start, end in ordered[1:]:
        if start ≤ merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return [tuple(b) for b in merged]

def check_against_brute(trials: int = 500) → str:
    """Merge by marking a small number line. Obviously correct, useless at scale."""
    for _ in range(trials):
        n = random.randint(1, 8)
        ivs = []
        for _ in range(n):
            s = random.randint(0, 20)
            ivs.append((s, s + random.randint(0, 8)))

        covered = set()
        for s, e in ivs:
            covered.update(range(s, e))                # half-open marking
        expected = []
        for point in sorted(covered):
            if expected and expected[-1][1] == point:
                expected[-1][1] = point + 1
            else:
                expected.append([point, point + 1])
        expected = [tuple(b) for b in expected]

        # half-open marking treats touching ranges as contiguous,
        # so compare against the `≤` version
        got = [b for b in merge(ivs) if b[0] ≠ b[1]]
        if got ≠ expected:
            return f"MISMATCH on {ivs}: got {got}, expected {expected}"
    return f"agreed on {trials} random cases"

if __name__ == "__main__":
    random.seed(0)
    ivs = [(1, 3), (2, 6), (8, 10), (15, 18)]
    print("THE ALGORITHM")
    print("  input      :", ivs)
    print("  merged     :", merge(ivs))

    print("\nTHE BUG – an interval CONTAINED in the current block")
    contained = [(1, 10), (2, 3), (11, 12)]
    print("  input      :", contained)
    print("  correct     :", merge(contained))
    print("  with `= end`:", merge_shrinking(contained))
    print("  by END sort :", merge_by_end(contained))

    print("\n ...and on a typical hand-written test, the bug is INVISIBLE:")
    typical = [(1, 3), (2, 6), (8, 10)]
    print("  input      :", typical)

```

```

print(" correct      :", merge(typical))
print(" with `= end`:", merge_shrinking(typical))

print("\nTHE BOUNDARY CONVENTION")
touching = [(1, 3), (3, 5), (6, 8)]
print(" input          :", touching)
print(" touching MERGES      :", merge(touching))
print(" touching stays apart:", merge_strict(touching))

print("\nTHE VARIANTS")
sorted_disjoint = [(1, 3), (6, 9)]
print(" insert (2,5) :", insert_interval(sorted_disjoint, (2, 5)))
print(" insert (4,5) :", insert_interval(sorted_disjoint, (4, 5)))
print(" insert (0,20) :", insert_interval(sorted_disjoint, (0, 20)))
print(" free time      :", free_time(ivs, 0, 24))
print(" intersect      :", intersect([(0, 2), (5, 10), (13, 23), (24, 25)],
                                  [(1, 5), (8, 12), (15, 24), (25, 26)]))
print(" covered       :", total_covered(contained),
      " (unmerged would say", total_raw(contained), ")")

print("\nSTREAMING")
stream = StreamingMerger()
for pair in [(8, 10), (1, 3), (15, 18), (2, 6), (4, 16)]:
    stream.add(*pair)
    print(f" after {str(pair):8} → {stream.blocks()}")

print("\nVERIFICATION")
print(" ", check_against_brute())

```

Run it and you get:

THE ALGORITHM

```
input      : [(1, 3), (2, 6), (8, 10), (15, 18)]
merged     : [(1, 6), (8, 10), (15, 18)]
```

THE BUG — an interval CONTAINED in the current block

```
input      : [(1, 10), (2, 3), (11, 12)]
correct    : [(1, 10), (11, 12)]
with `= end` : [(1, 3), (11, 12)]
by END sort : [(2, 10), (11, 12)]
```

...and on a typical hand-written test, the bug is INVISIBLE:

```
input      : [(1, 3), (2, 6), (8, 10)]
correct    : [(1, 6), (8, 10)]
with `= end` : [(1, 6), (8, 10)]
```

THE BOUNDARY CONVENTION

```
input      : [(1, 3), (3, 5), (6, 8)]
touching MERGES : [(1, 5), (6, 8)]
touching stays apart: [(1, 3), (3, 5), (6, 8)]
```

THE VARIANTS

```
insert (2,5) : [(1, 5), (6, 9)]
insert (4,5) : [(1, 3), (4, 5), (6, 9)]
insert (0,20) : [(0, 20)]
free time    : [(0, 1), (6, 8), (10, 15), (18, 24)]
intersect    : [(1, 2), (5, 5), (8, 10), (15, 23), (24, 24), (25, 25)]
covered      : 10 (unmerged would say 11 )
```

STREAMING

```
after (8, 10) → [(8, 10)]
after (1, 3)  → [(1, 3), (8, 10)]
after (15, 18) → [(1, 3), (8, 10), (15, 18)]
after (2, 6)  → [(1, 6), (8, 10), (15, 18)]
after (4, 16) → [(1, 18)]
```

VERIFICATION

agreed on 500 random cases

The two blocks under “THE BUG” are the whole lesson. On the contained input, `= end` gives `[(1,3)]` where the answer is `[(1,10)]` — the block shrank by seven. And on the typical hand-written input, the broken version gives exactly the right answer, which is why it survives.

And the streaming trace’s last line is the case people miss: `(4,16)` arrives and absorbs all three existing blocks at once, collapsing everything into `[1,18]`.

6 What it costs

The merge.

```

sort           $O(n \log n)$     ← dominates
one pass       $O(n)$ 
output         $O(n)$  space

TOTAL:  $O(n \log n)$  time,  $O(n)$  space

```

And if the input is already sorted — which several of the variants specify — it is $O(n)$.

```

n = 1,000          instant
n = 1,000,000      sort ~1.5 s in Python, pass ~0.3 s
n = 10,000,000     sort ~20 s

→ merging scales well. There is no hidden quadratic anywhere
  in this family.

```

The variants:

```

merge           $O(n \log n)$ 
insert (sorted input)  $O(n)$     ← re-sorting makes it  $O(n \log n)$ 
gaps            $O(n \log n)$     (merge + one pass)
intersect two lists  $O(n + m)$  ← both already sorted
total covered   $O(n \log n)$     (merge + a sum)
streaming, per insert  $O(\log n + k)$  ←  $k$  = blocks absorbed
streaming, total  $O(n \log n)$     amortised

```

The streaming amortisation, which is the interesting one:

```

each insert:  $O(\log n)$  to find the position
              +  $O(k)$  to absorb  $k$  blocks

worst single insert:  $k = n$ , when one interval swallows everything
                    →  $O(n)$  for that one call

BUT each block can be absorbed only ONCE
→ the total absorption work over  $n$  inserts is  $O(n)$ 
→ amortised  $O(\log n)$  per insert,  $O(n \log n)$  overall

Same as sorting. Which is the right comparison: the streaming
version costs no more in total, and pays it incrementally.

```

The slice assignment, which matters in Python:

```
self.starts[i:j] = [start]
```

this is $O(n)$ in the worst case, because a list shifts its tail
→ in a language with a balanced tree, the same operation is $O(\log n)$

for n up to $\sim 100,000$ the constant factor makes the list version faster anyway; above that, a real sorted container is worth it.

Space:

merge	$O(n)$ for the output ($O(1)$ extra if in-place)
insert	$O(n)$ for the output
intersect	$O(\min(n, m))$ for the output
streaming	$O(b)$ where b is the number of merged blocks

and b can be far smaller than n : heavily overlapping input collapses to a handful of blocks.

The double-counting cost, which is why merging exists at all:

[1,10] and [2,3]

sum of raw lengths:	$9 + 1 = 10$
actual covered length:	9

→ the error grows with the amount of overlap
→ for calendar or coverage data, overlap is the normal case, so the raw sum can be off by a large factor

Comparison with the naive approach:

"which intervals overlap?" by checking every pair:	$O(n^2)$
sorting and checking adjacent pairs:	$O(n \log n)$

$n = 100,000$: 10^{10} against 1.7×10^6 → $\sim 6,000\times$

→ and the sorted version answers a STRONGER question:
it produces the merged blocks, not just a yes or no.

7 The traps

= end instead of `max(current_end, end)`.

```
>>> merge([(1, 10), (2, 3), (11, 12)])
[(1, 10), (11, 12)]
>>> merge_shrinking([(1, 10), (2, 3), (11, 12)])
[(1, 3), (11, 12)]
```

The block shrank from ten to three. And the reason it survives:

```
>>> merge([(1, 3), (2, 6), (8, 10)])
[(1, 6), (8, 10)]
>>> merge_shrinking([(1, 3), (2, 6), (8, 10)])
[(1, 6), (8, 10)]
```

Identical on the typical input, because partial overlap is what people write by hand and **the new end happens to be further. Containment is common in real data and rare in tests.**

The wrong sort key.

```
>>> merge_by_end([(1, 10), (2, 3), (11, 12)])
[(2, 10), (11, 12)]
```

The block starts at 2 instead of 1. Sorted by end, the pass began with `[2,3]` and absorbed `[1,10]` by extending the *end* — **and there is no code path that reaches backwards to fix a start.**

The block count is right, so a test asserting “two blocks” passes.

Forgetting the final flush.

```
>>> # merge_pairs without the append after the loop:
>>> # on [(1,3)] → returns []
>>> # on [(1,3),(2,6)] → returns []
>>> # the LAST block is never emitted
```

Every input loses its last block, which on a single-interval input returns nothing at all. **The list-of-lists version does not have this problem**, because the block is already in the list.

The boundary convention, unstated.

```
>>> merge([(1, 3), (3, 5)])
[(1, 5)]
>>> merge_strict([(1, 3), (3, 5)])
[(1, 3), (3, 5)]
```

Both are correct for different problems. The statement decides, and if it does not say, ask — an interviewer who left it out may well be checking.

Re-sorting a sorted input.

```
>>> # "insert into a SORTED, DISJOINT list"
>>> # appending and re-sorting:  $O(n \log n)$ 
>>> # the three-phase scan:  $O(n)$ 
>>> # → the problem is testing whether you read the word "sorted"
```

Trying to modify a tuple.

```
>>> merged = [(1, 3)]
>>> merged[-1][1] = 6
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

This one at least errors, which makes it the friendliest bug in the lesson. Build with lists and convert at the end, or carry the current block in two variables.

The gaps, without an explicit range.

```
>>> free_time([(2, 4), (6, 8)], 0, 24)
[(0, 2), (4, 6), (8, 24)]
>>> # without lower and upper, what is "free" before 2 and after 8?
>>> # the question is meaningless without bounds
```

“When is the room free” needs the day’s bounds, and the two boundary gaps — before the first block and after the last — are where the off-by-one errors live.

Streaming with the one-pass algorithm.

```
>>> # re-sorting and re-merging on every arrival:
>>> #  $O(n \log n)$  per insert →  $O(n^2 \log n)$  over  $n$  arrivals
>>> #  $n = 100,000$  → not viable
```

The single-pass merge depends on having the whole input in start order, and it is not incremental. A sorted structure with a binary search is the answer, and it is amortised $O(n \log n)$ overall — the same total as sorting once.

Missing the multi-block absorption in the streaming version.

```
>>> stream = StreamingMerger()
>>> for p in [(1, 3), (8, 10), (15, 18)]:
...     stream.add(*p)
>>> stream.add(4, 16)
>>> stream.blocks()
[(1, 18)]
```

One arrival absorbed all three blocks. A version that merges with only the nearest neighbour **leaves overlapping blocks in the structure**, and the corruption compounds silently with every later insert.

8 In the interview

How it gets asked

- “Merge overlapping intervals.” — LeetCode 56, and it is the most-asked interval problem there is.
- “Insert an interval into a sorted list.” — LeetCode 57, testing whether you re-sort.
- “Find the free time common to several people’s calendars.” — LeetCode 759, the gaps.
- “Given two lists of intervals, find their intersection.” — LeetCode 986.
- “What if the intervals arrive one at a time?” — the streaming follow-up.
- “Do `[1, 3]` and `[3, 5]` overlap?” — checking whether you ask.

The first ninety seconds

“Merging is one pass after a sort, and I want to be precise about which sort and about one line in the loop.

Sort by start. That is what makes a single pass legal: **with intervals in start order, anything that overlaps the current merged block must begin before that block ends** — so one comparison per interval decides it, and I never have to look backwards or reopen a closed block.

Sorted by end, that property does not hold, and the algorithm is silently wrong — it produces blocks whose starts are too late, and there is no code path that could fix them.

Then the loop: if the next interval starts at or before the current block’s end, extend the block. Otherwise close it and start a new one.

And the extension is `max(current_end, next_end)`, **not** `next_end` — that is the line I would say out loud while writing it. **An interval entirely inside the current block would otherwise shrink it.**

`[1,10]` followed by `[2,3]` becomes `[1,3]` **with the wrong version**, and the reason it survives is that it **gives exactly the right answer on typical hand-written tests**, where intervals partially overlap and the new end genuinely is further. **Containment is rare in tests and common in real data.**

One thing I would settle first: do `[1,3]` **and** `[3,5]` **merge?** For time slots, yes — they touch and there is no gap. **For half-open ranges, no.** It is one character, `≤` against `<`, **and the problem decides it.**

Cost: `O(n log n)`, **dominated by the sort, and** `O(n)` **for the output.** If the input is already sorted — which several of the variants specify — it is `O(n)`, **and noticing that is usually what those variants are testing.**

And I would build the output as lists rather than tuples, or carry the current block in two variables, because a tuple cannot be extended in place.”

The follow-ups

“Why does sorting by start make this work?”

“Because it establishes exactly the invariant the single pass needs, and I would state the invariant rather than just the key.

With intervals in start order, any interval that overlaps the current merged block must *begin* before that block ends. That is what lets one comparison decide the whole question — **I never have to look backwards at closed blocks, and I never have to reopen one.**

And the reason a closed block stays closed: everything remaining starts at or after the current interval’s start, which is already past the closed block’s end. So nothing later can reach back into it.

Sorted by end, that fails. Take `[1,10]`, `[2,3]` and `[11,12]`. By end they order as `[2,3]`, `[1,10]`, `[11,12]`. **The pass begins with `[2,3]`, sees that `[1,10]` starts at 1 which is inside it, and extends the end to 10 — giving `[2,10]`.**

The block starts at 2 when it should start at 1, and there is no code path that reaches backwards to fix a start. And the block count is still two, so a test asserting the number of blocks passes.

The general habit I would name: when I pick a sort key, I try to say what invariant it gives me. For merging it is ‘overlapping intervals begin before the block ends’. For selection it is ‘the earliest finisher leaves the most room’. Different invariants, different keys — **and if I cannot state the invariant, I have not understood why the algorithm is correct.**

One implementation note: in Python, `sorted(intervals)` on tuples sorts by start then end, which happens to be right here. I still write the key explicitly, because it says which I meant and because the same default is wrong for the selection problem.”

“What if the intervals arrive one at a time and you cannot sort?”

“Then the single-pass algorithm does not apply at all, **because it depends on having the whole input in start order** — and I would say that first, because the instinct is to try to adapt it.

The naive adaptation is to re-sort and re-merge on every arrival, which is $O(n \log n)$ per insert and $O(n^2 \log n)$ overall. **At a hundred thousand arrivals that does not finish.**

What I would use is a sorted structure keyed by start — a balanced tree, or a sorted list with binary search — **holding the currently merged blocks.**

On each arrival: binary search for the first block that could overlap, which is the first block whose end reaches the new interval’s start. **Then absorb every block that begins at or before the new interval’s end**, widening in both directions as I go. **Replace the whole absorbed range with one block.**

The case people miss is that one arrival can absorb several blocks at once. If I have $[1,3]$, $[8,10]$ and $[15,18]$, and $[4,16]$ arrives, it swallows all three and everything collapses to $[1,18]$. **A version that merges with only the nearest neighbour leaves overlapping blocks in the structure**, and the corruption compounds silently with every later insert.

Cost: $O(\log n)$ to find the position plus $O(k)$ to absorb k blocks. **The worst single insert is $O(n)$** when one interval swallows everything.

But the amortised total is $O(n \log n)$, because a block can be absorbed only once. So over n arrivals the total absorption work is $O(n)$. **Which is the right comparison to make: the streaming version costs no more in total than sorting once — it just pays it incrementally.**

One Python note: a slice assignment on a list is $O(n)$ because the tail shifts. For up to about a hundred thousand blocks the constant factor makes that faster than a tree anyway; **above that, a real sorted container is worth the dependency.**“

“Now find the free time — when is nobody busy?”

“That is the complement of the merged blocks, and it is a two-step answer where the second step is where the bugs are.

Step one: merge. Take everyone’s busy intervals, put them all in one list, and merge them. **The merged blocks are the times when at least one person is busy.**

Step two: walk the blocks and emit the space between them. Keep a cursor; for each block, if it starts after the cursor, that space is free; then move the cursor to the block’s end.

And there are two boundary cases that are where every bug in this variant lives.

The gap before the first block, and the gap after the last one. Both need an **explicit range to complement within** — ‘when is the room free’ is meaningless without knowing the day’s bounds. **So the function takes a lower and an upper bound**, and I would ask for them rather than assume the first and last busy times, **because free time before anyone starts is usually exactly what the caller wants.**

One more subtlety: `cursor = max(cursor, end)` rather than `cursor = end`. A block lying entirely before the cursor — which can happen if the bounds are inside the data — **would otherwise move the cursor backwards** and emit a nonsensical gap. **It is the same containment mistake as the merge bug, in a different place.**

Cost: `$O(n \log n)$` for the merge and `$O(n)$` for the walk.

And the related variant is intersecting two calendars rather than complementing one: given two sorted disjoint lists, find where they overlap. **Two pointers, and the intersection of two intervals is** `$[\max(\text{starts}), \min(\text{ends})]$` , **valid when that is non-empty. Advance whichever interval ends first**, because it cannot intersect anything further in the other list. `$O(n + m)$` , no sorting needed since both inputs are already ordered.”

The model answer

“You are building a scheduling assistant. Given several people’s busy calendars — each a list of intervals — find all the slots of at least thirty minutes where everyone is free, within working hours.”

“Three steps, and the third is where the requirement actually bites, so let me name all three before starting.

Step one: combine everyone’s busy intervals into one list and merge them. I do not care whose meeting is whose — **if anybody is busy, the slot is unavailable** — so the union of everyone’s busy time is exactly what I need, and merging it gives the blocks when at least one person is occupied.

That is the classic merge: sort by start, one pass, extend to the further of the two ends. And I would say the extension out loud, because `max(current_end, next_end)` **rather than** `next_end` **is the bug that passes typical tests** — a short meeting entirely inside a long one would otherwise shrink the block, **and calendars are full of exactly that shape.**

Step two: complement the merged blocks within working hours. The gaps between consecutive blocks, **plus the gap from the start of the working day to the first block and from the last block to the end of the day.**

The bounds have to be explicit — nine to five, or whatever the calendars use — **because free time before anyone’s first meeting is usually the answer people want, and without bounds the question has no answer at the edges.**

Step three, which is the actual requirement: filter for gaps of at least thirty minutes. One line, and it is the reason the whole thing is asked — **a slot of four minutes is not a slot.**

Cost: `O(n log n)` **where** `n` **is the total number of busy intervals across everyone,** dominated by the sort. For a dozen people with a dozen meetings each that is instant.

Now the details I would raise, because a scheduling assistant is full of them.

Time zones. If the participants are in different zones, **everything must be converted to a common instant before merging** — merging local times across zones is a wrong-answer bug that looks like a correctness bug and is a data bug. **I would normalise to UTC on the way in and convert back only for display.**

The boundary convention. A meeting ending at ten and one starting at ten do not overlap — **the room is free at ten.** So `start ≤ current_end` merges them, which is what I want here, **and I would state the assumption.**

And working hours differ per person, which is the version of the problem that is actually asked in practice. **That is not a filter at the end — it is an extra set of busy intervals per person,** covering everything outside their working day.

Which is a nice outcome: it needs no new algorithm, just more intervals into the same merge.

Two follow-ups I would offer.

If the calendars are large and the query is repeated — ‘find me a slot’ asked many times a day for the same people — I would keep each person’s merged busy blocks cached and merge the merged lists, which is cheaper than starting from raw meetings each time.

And if meetings arrive continuously rather than being fetched as a batch, the single pass does not apply. That is the streaming version: keep the merged blocks in a sorted structure and, on each new meeting, binary search for the position and absorb any overlapping blocks — remembering that one meeting can absorb several blocks at once. $O(\log n)$ per arrival amortised, the same total as sorting once.“

9 Recall card

Sort by START, one pass: if the next interval begins at or before the current block ends, extend the block; otherwise close it and start a new one. Sorting by start is what makes a single pass legal — anything overlapping the current block must BEGIN before it ends, so you never look backwards and never reopen a closed block.

The bug, and it is the point of the lesson: extend to `max(current_end, next_end)`, NEVER `= next_end`. An interval entirely inside the block would otherwise shrink it — `[1,10]` then `[2,3]` becomes `[1,3]`. And it gives exactly the right answer on typical hand-written tests, because partial overlap is what people write and containment is what real data contains.

Sorted by END the invariant fails: `[1,10]`, `[2,3]`, `[11,12]` gives `[(2,10)]` — the block starts at 2 and no code path reaches backwards to fix a start, while the block count stays right.

The boundary convention is one character — `≤` merges touching intervals, `<` does not — and the problem decides it. Ask if it is not stated.

Five variants: insert into a sorted list ($O(n)$), three phases — re-sorting is the mistake being tested); gaps (the complement, and the two ends are where the bugs live — needs explicit bounds, and `cursor = max(cursor, end)`); intersect

two lists (two pointers, $[\max(\text{starts}), \min(\text{ends})]$, advance whichever ENDS first); total covered (merge first, or overlaps are double-counted); and streaming.

Streaming: the one-pass algorithm does not apply. Keep the blocks in a sorted structure, binary search for the position, and absorb every overlapping block — one arrival can swallow several at once. $O(\log n + k)$ per insert, amortised $O(n \log n)$ overall because a block can be absorbed only once — the same total as sorting once, paid incrementally.

Design a leaderboard

1 What this is, and why they ask it

A leaderboard shows who is winning. **Top ten, and “you are 4,412th”.** That is the whole product.

They ask it because **it looks trivial for about ninety seconds and then does not.** “Sort by score” is the obvious answer and it is right for a thousand players. **At fifty million players with scores changing every second, sorting on every read is impossible and sorting on every write is worse.**

Three things carry the interview.

The top ten is easy and the rank is hard. Returning the highest ten scores is a cheap query in almost any store. **Answering “what is my rank” requires counting everyone above you,** which is a full scan unless the structure was built for it — **and that asymmetry is the thing to notice.**

The obvious answer is a sorted set, and knowing why is the point. Redis sorted sets keep elements ordered by score in a skip list with an auxiliary hash, **so insertion, rank lookup and range queries are all logarithmic.** A candidate who says “Redis `ZADD` and `ZREVRANK`” has the right answer; **a candidate who can say what is underneath and where it stops scaling has the interview.**

And where it stops scaling is genuinely interesting. One sorted set holds tens of millions of entries in a few gigabytes. **A hundred million players across many leaderboards does not fit in one, and sharding a ranked structure is hard** — because rank is inherently global, and any split makes “how many are above me” a query across every shard.

By the end of this lesson you can design the sorted-set solution and size it, handle exact rank at scale, know when approximate rank is correct, deal with ties and time windows, and say what breaks.

2 The story

The wrestling akhara had about ninety boys and one board and Gopichand had kept it since before any of them were born.

Names down the left, in order. The best at the top.

And what he did with it took about four minutes a day. A boy won a bout, and Gopichand rubbed out two names and wrote them again the other way round, **and the whole board was correct in under a minute because one win almost never moves anybody more than a place or two.**

That was the first thing, and it stayed true for thirty years.

The second thing was the question he could not answer quickly, and he did not notice for a long time that it was a different question.

Because “who is at the top” was one glance. **And “where am I” — asked by a boy standing next to him, every single day — meant counting.**

Ninety names, from the top, with a finger. **Twenty seconds.** Which was fine for ninety.

And then the akhara joined the district association and there were four thousand names.

The top ten was still one glance. The board was still correct. **And “where am I” had become unanswerable,** because counting four thousand names with a finger is twenty minutes and by the end it was wrong anyway.

His solution was not clever and it worked for eleven years.

He drew a line every hundred names and wrote the number in the margin.

So a boy at position three thousand two hundred and something — **you found the line marked three thousand two hundred, and counted forward from there.** Never more than a hundred names. **Four seconds instead of twenty minutes.**

And the second half of it, which the boys worked out for themselves and he never had to say.

Below about the first two hundred, nobody asked for the exact number.

They asked whether they were in the top five hundred, or roughly where in the middle. **“About two thousand one hundred” was a completely acceptable answer to a boy at position 2,137,** and it would have been an insulting one to the boy at position four.

“At the top they want the number,” Gopichand said. “In the middle they want the neighbourhood.”

3 The idea in plain English

Gopichand's board is a sorted set, his lines in the margin are the index that makes rank cheap, and his last sentence is the thing that makes this affordable at scale.

Start with the two questions, because they have different costs.

"TOP TEN"	"WHAT IS MY RANK?"
read the first 10 entries	COUNT how many are above me
$O(1)$ or $O(\log n)$ anywhere	$O(n)$ unless the structure supports it
→ the top-k is easy in ANY sorted store	
→ the RANK is what makes this a design problem	

Almost every naive design gets the top-ten right and cannot answer the rank, and noticing that asymmetry in the first minute is the strongest opening move available.

Now the obvious answer, and what is underneath it.

A Redis sorted set is the right default, and it is worth being able to say why rather than just naming it.

ZADD leaderboard 4820 "player:1234"	update a score	$O(\log n)$
ZREVRANK leaderboard "player:1234"	my rank	$O(\log n)$
ZREVRANGE leaderboard 0 9 WITHSCORES	the top ten	$O(\log n + 10)$
ZREVRANGE leaderboard 4405 4415	the ten around me	$O(\log n + 10)$
ZCARD leaderboard	how many players	$O(1)$

Underneath it is a skip list plus a hash map. The skip list keeps the elements in score order and **carries a span on each forward pointer — the number of elements it jumps over** — which is what makes rank a logarithmic query rather than a count. **The hash map gives $O(1)$ lookup from member to score,** so updating an existing player does not require a search.

That combination — ordered structure plus the span counts — is the whole reason this is the standard answer, and it is exactly Gopichand's lines in the margin.

Now the sizing, which decides whether one sorted set is enough.

Roughly a hundred bytes per entry in Redis, including the skip-list nodes and the hash entry. **Ten million players is about a gigabyte** and answers every query in microseconds. **A hundred million is ten gigabytes,** which is one large instance and is beginning to be uncomfortable.

And the thing to notice is that it is a single-machine structure. You cannot split a sorted set across machines and still answer rank cheaply, **because rank is global** — “how many are above me” is a question about every player, everywhere.

So the scaling answer is not “shard the sorted set”. It is one of three other things.

One: partition the leaderboard, not the data. Most games do not have one global board — **they have one per region, per level, per week, per friend group.** Each of those is a separate, smaller sorted set, **and the partitioning is a product fact rather than an engineering compromise.** This is by far the most common answer and it is usually the right one.

Two: keep only the top N exactly. Nobody below the top ten thousand needs an exact rank. Keep a bounded sorted set of the top ten thousand — trimmed on every write — **and answer everyone else approximately.** Gopichand’s boys, exactly.

Three: approximate rank by bucketing. Keep a histogram of scores — how many players fall in each score range — **and a rank is the sum of the buckets above yours plus a count within your bucket.** Sub-millisecond, tiny, and accurate to within a bucket.

And the approximation is not a compromise for most users, because “about 2,100th of 50,000” is a better answer than an exact number nobody can act on. **Exact at the top, approximate in the middle** — which is a product decision that happens to be the cheap one.

Then ties, which are a real product problem and not an edge case.

Two players with the same score. What order? **Redis breaks ties lexicographically by member name,** which is stable and arbitrary and means player “aaa” always beats player “zzz” at equal score.

The usual fix is to encode the tie-break into the score itself. A common trick: use a composite score where the high bits are the game score and the low bits are an inverted timestamp, so an earlier achiever ranks higher.

```
composite = score * 10^10 + (10^10 - seconds_since_epoch)
```

Doubles hold 53 bits of integer precision, so this works while `score × 1010` stays under about `9 × 1015` — **which is a real constraint worth checking rather than assuming.**

Then time windows, which is where most of the actual complexity is.

“Top players today”, “this week”, “all time” are three different leaderboards, and the naive answer — recompute from an event log — is far too slow for a live board.

Keep a separate sorted set per window and expire it. `leaderboard:daily:2026-09-01` with a TTL of a couple of days; `leaderboard:weekly:2026-W35`; `leaderboard:alltime` with no expiry. **A score update writes to all three**, which is three cheap operations rather than one expensive recomputation.

The cost is write amplification — one score becomes three or four writes — **and it is worth it, because reads vastly outnumber writes and each read becomes a single lookup.**

And “rolling last 24 hours” is a genuinely harder problem than “today”, because there is no window to expire. **The honest answer is usually to approximate it with hourly buckets** and accept an hour of granularity, rather than to build something exact.

Finally: the durability question, which people forget.

Redis is the serving structure and it is not the source of truth. A score is written to a durable store — **the game’s own database** — **and then to the sorted set.** If Redis is lost, **the leaderboard is rebuilt from the durable store**, which takes minutes and is fine.

Building it the other way round — treating the sorted set as authoritative — means a Redis failure loses scores, and scores are the product.

4 The picture

The asymmetry that defines the problem:

TOP TEN	MY RANK
[1] Ravi 9,912	"how many players have a score above 4,820?"
[2] Meera 9,880	
[3] Anil 9,844	
...	
[10] Kavya 9,301	→ a COUNT over everyone
	→ O(n) unless the structure was built for it
→ read the first 10	
→ cheap in ANY sorted store	

ALMOST EVERY NAIVE DESIGN GETS THE TOP TEN RIGHT AND CANNOT ANSWER THE RANK. Notice that in the first minute.

What is inside a sorted set:

SKIP LIST, ordered by score, with SPANS on the forward pointers

```
level 3  head -----> [8200] -----> nil
              span 5                      span 3
level 2  head -----> [9301] -----> [8200] -----> nil
              span 2      span 3          span 3
level 1  head → [9912] → [9880] → [9844] → ... → nil
              span 1    span 1    span 1
```

+ A HASH MAP: member → score (O(1) score lookup)

THE SPANS ARE THE POINT: each forward pointer records how many elements it jumps over, so RANK is the sum of the spans along the search path – $O(\log n)$, not a count.

That is exactly Gopichand's line every hundred names.

Where one sorted set stops:

~100 bytes per entry (skip-list nodes + hash entry)

1,000,000 players	~100 MB	comfortable
10,000,000	~1 GB	fine
100,000,000	~10 GB	one large instance, uncomfortable
1,000,000,000	~100 GB	does not fit

AND YOU CANNOT SHARD IT USEFULLY, because RANK IS GLOBAL:

"how many are above me" is a question about EVERY player

→ sharded, every rank query hits every shard and sums

→ which is correct and slow, and gets slower with more shards

The three scaling answers:

1. PARTITION THE LEADERBOARD, NOT THE DATA

one global board → per region, per level, per week,
100,000,000 entries per friend group
 → each is small
→ this is a PRODUCT FACT, not a compromise, and it is
 usually the real answer

2. EXACT TOP N, APPROXIMATE BELOW

keep the top 10,000 in a bounded sorted set (trim on write)
everyone else: approximate

→ "at the top they want the number; in the middle they
 want the neighbourhood"

3. BUCKETED APPROXIMATE RANK

a histogram: how many players in each score range

```
score 9000-9999:  1,204 players
score 8000-8999: 11,873
score 7000-7999: 48,200
...
```

my rank ~ sum of the buckets above mine
→ sub-millisecond, a few KB, accurate to within a bucket

The tie problem:

three players, all on 4,820

REDIS default: ties break LEXICOGRAPHICALLY by member name
→ "player:aaa" always ranks above "player:zzz"
→ stable, arbitrary, and visible to users who notice

THE FIX: encode the tie-break INTO the score

```
composite = score * 10^10 + (10^10 - timestamp)
           ^^^^^^^^^^^^^^  ^^^^^^^^^^^^^^^^^^^^^^
           the real score   earlier = LARGER = ranks higher
```

CHECK THE PRECISION: doubles hold 53 bits of exact integer range,
so this is safe while score x 10^10 stays under ~9 x 10^15.
Not an assumption to make silently.

Time windows, and the write amplification:

```
ONE SCORE UPDATE → ZADD leaderboard:daily:2026-09-01 (TTL 2 days)
                   ZADD leaderboard:weekly:2026-W35   (TTL 2 weeks)
                   ZADD leaderboard:alltime             (no TTL)
```

3-4x write amplification, and each READ is a single lookup.

→ the right trade, because reads vastly outnumber writes

AND: "rolling last 24 hours" is genuinely harder – there is no window to expire. The honest answer is hourly buckets and an hour of granularity, not an exact rolling window.

5 How it actually works

The basic operations

```
def submit_score(player_id: int, score: int) → None:
    """Durable store FIRST, then the serving structure."""
    db.execute("""INSERT INTO scores (player_id, score, at)
                VALUES (%(p)s, %(s)s, now())""",
               {"p": player_id, "s": score})
    redis.zadd("leaderboard:alltime", {f"player:{player_id}": score})

def top_n(n: int = 10) → list[tuple[str, float]]:
    return redis.zrevrange("leaderboard:alltime", 0, n - 1, withscores=True)

def my_rank(player_id: int) → int | None:
    rank = redis.zrevrank("leaderboard:alltime", f"player:{player_id}")
    return None if rank is None else rank + 1          # 0-indexed → 1-indexed

def around_me(player_id: int, window: int = 5) → list[tuple[str, float]]:
    rank = redis.zrevrank("leaderboard:alltime", f"player:{player_id}")
    if rank is None:
        return []
    lo = max(0, rank - window)
    return redis.zrevrange("leaderboard:alltime", lo, rank + window,
                           withscores=True)
```

The durable write comes first, and that ordering is the design decision: **Redis is the serving structure, not the source of truth**. If Redis is lost, the leaderboard is rebuilt in minutes; **if the durable write were second, a Redis failure would lose scores**.

And `rank + 1` is worth a comment — `ZREVRANK` is zero-indexed and users count from one, which is a one-character bug that ships surprisingly often.

Highest score, not latest

```
def submit_best_score(player_id: int, score: int) → bool:
    """Most games keep a player's BEST score, not their most recent."""
    member = f"player:{player_id}"
    current = redis.zscore("leaderboard:alltime", member)
    if current is not None and current ≥ score:
        return False # not an improvement
    redis.zadd("leaderboard:alltime", {member: score}, gt=True)
    return True
```

`gt=True` makes it atomic — Redis updates only if the new score is greater — so a check-then-write race between two concurrent submissions cannot lower a player's score.

Without it, the read and the write are separate and an older submission arriving late can overwrite a better one.

Ties, encoded into the score

```
TIE_SCALE = 10 ** 10

def composite_score(score: int, achieved_at: float) → float:
    """Higher score wins; on a tie, the EARLIER achievement wins."""
    inverted_time = TIE_SCALE - int(achieved_at)
    composite = score * TIE_SCALE + inverted_time
    assert composite < 2 ** 53, "beyond exact double precision"
    return float(composite)

def real_score(composite: float) → int:
    return int(composite) // TIE_SCALE
```

The assertion is not decoration. A double holds 53 bits of exact integer range — about 9×10^{15} — and a composite beyond that silently loses precision, which reorders players in a way nobody will diagnose.

And `real_score` exists because the displayed score must be the real one, not the composite — which is easy to forget and produces absurd numbers on screen.

Time windows

```
WINDOWS = {
    "daily": (lambda t: time.strftime("%Y-%m-%d", time.gmtime(t)), 2 * 86400),
    "weekly": (lambda t: time.strftime("%Y-W%W", time.gmtime(t)), 14 * 86400),
    "alltime": (lambda t: "", None),
}

def submit_all_windows(player_id: int, score: int, at: float) → None:
    member = f"player:{player_id}"
    pipe = redis.pipeline()
    for name, (key_of, ttl) in WINDOWS.items():
        key = f"leaderboard:{name}:{key_of(at)}".rstrip(":")
        pipe.zadd(key, {member: score}, gt=True)
        if ttl:
            pipe.expire(key, ttl)          # the window expires itself
    pipe.execute()
```

The TTL is what removes old windows — no cleanup job, no cron, and yesterday's daily board simply ceases to exist.

And the pipeline makes three or four writes one network round trip, which is what keeps the write amplification cheap.

Setting the TTL on every write is slightly wasteful and is correct: setting it only on creation risks a key that never expires if the creating call failed after the `ZADD`.

Approximate rank by bucketing

```
BUCKET_SIZE = 100

def record_for_histogram(score: int) → None:
    redis.hincrby("score_histogram", str(score // BUCKET_SIZE), 1)

def approximate_rank(score: int) → int:
    """Sum the buckets above mine. Sub-millisecond, accurate to a bucket."""
    my_bucket = score // BUCKET_SIZE
    buckets = redis.hgetall("score_histogram")
    above = sum(int(count) for bucket, count in buckets.items()
                 if int(bucket) > my_bucket)
    return above + 1
```

The whole histogram is a few thousand entries — one per hundred-point band — so fetching all of it is kilobytes and summing it is microseconds.

And the accuracy is exactly one bucket, which for a player at rank 2,137 is completely acceptable and would be unacceptable at rank 4 — which is why the exact structure covers the top.

Exact at the top, approximate below

```
TOP_N = 10_000

def submit_hybrid(player_id: int, score: int) → None:
    member = f"player:{player_id}"
    pipe = redis.pipeline()
    pipe.zadd("leaderboard:top", {member: score}, gt=True)
    pipe.zremrangebyrank("leaderboard:top", 0, -(TOP_N + 1)) # trim the tail
    pipe.hincrby("score_histogram", str(score // BUCKET_SIZE), 1)
    pipe.execute()

def rank_hybrid(player_id: int, score: int) → tuple[int, bool]:
    """Returns (rank, is_exact)."""
    exact = redis.zrevrank("leaderboard:top", f"player:{player_id}")
    if exact is not None:
        return exact + 1, True
    return approximate_rank(score), False
```

`zremrangebyrank(key, 0, -(TOP_N + 1))` keeps the structure bounded — the top ten thousand and nothing else — so it stays at about a megabyte regardless of how many players exist.

And returning `is_exact` matters for the interface: the client shows “4,412th” or “about 2,100th”, and the distinction should be visible rather than implied.

Sharding, and why it is unpleasant

```
def sharded_rank(player_id: int, score: int, shards: int) → int:
    """Correct, and it queries EVERY shard. Rank is inherently global."""
    total_above = 0
    for shard in range(shards):
        total_above += redis.zcount(f"leaderboard:{shard}",
                                   f"({score})", "+inf")
    return total_above + 1
```

Every rank query hits every shard, and it gets slower as you add shards — which is the opposite of what sharding is supposed to do.

That is the argument for partitioning the leaderboard by a product dimension instead: a per-region board is a *smaller* structure, not a *split* one, and every query stays within it.

Rebuilding from the durable store

```
def rebuild(window: str = "alltime", batch: int = 10_000) → int:
    """Redis is not the source of truth. This is the recovery path."""
    key = f"leaderboard:{window}"
    temp = f"{key}:rebuilding"
    count, offset = 0, 0
    while True:
        rows = db.query("""SELECT player_id, MAX(score) AS best
                           FROM scores GROUP BY player_id
                           ORDER BY player_id LIMIT %(l)s OFFSET %(o)s""",
                        {"l": batch, "o": offset})
        if not rows:
            break
        redis.zadd(temp, {f"player:{r.player_id}": r.best for r in rows})
        count += len(rows)
        offset += batch
    redis.rename(temp, key) # atomic swap
    return count
```

Building into a temporary key and renaming is what makes this safe to run against a live system — the old leaderboard serves until the moment the new one is complete, and `RENAME` is atomic.

Rebuilding fifty million entries takes a few minutes, which is an acceptable recovery time for a leaderboard and would not be for a payments ledger — **and that difference is why Redis is allowed to be the serving structure here.**

The real systems

Redis sorted sets	the standard answer; skip list + hash, with SPANS on the forward pointers making rank $O(\log n)$
Valkey / KeyDB	Redis forks, same data structures
PostgreSQL	the durable source of truth for scores; window functions can compute rank, slowly
ClickHouse	for historical analysis and rebuilds over billions of score events
Kafka	the score event stream feeding both stores

Naming the skip list and its spans is what distinguishes a real answer from “use Redis”, because it explains *why* rank is cheap — and that is the thing the question is actually about.

6 The numbers

Scale.

```
50,000,000 players
each plays ~5 sessions/day, each producing a score
= 250,000,000 score updates/day
= ~2,900/second average, peak ~15,000/second

leaderboard READS:
  each session opens the leaderboard ~3 times
  = 750,000,000 reads/day = ~8,700/second, peak ~40,000/second

read : write = 3 : 1
```

Only three to one, which is unusually write-heavy for a read-facing feature — and it is why the write path being cheap matters as much as the read path.

The sorted set.

```
~100 bytes per entry (skip-list node with several levels,
                      plus the hash map entry, plus the member string)

50,000,000 players x 100 B = ~5 GB

→ one Redis instance, comfortably, with room for the windowed copies

10,000,000 → ~1 GB
100,000,000 → ~10 GB      one large instance, uncomfortable
1,000,000,000 → ~100 GB   does not fit
```

Operation costs.

```
ZADD      O(log n)   50,000,000 entries → ~26 comparisons
ZREVRANK  O(log n)   ~26 comparisons, summing spans
ZREVRANGE O(log n + k)

in practice, all of them: ~50-100 MICROSECONDS

15,000 writes/second x 100 us = 1.5 CPU-seconds per second
40,000 reads/second  x 100 us = 4 CPU-seconds per second

→ ~6 cores of Redis work at peak.
  A single Redis instance is single-threaded and does ~100,000
  ops/second, so this is at about 55% of one instance.
```

Fifty-five percent of a single instance at fifty million players is worth saying, because it reframes the problem: the data structure is not the bottleneck at any realistic scale for one board.

Write amplification from windows.

```
daily + weekly + monthly + all-time = 4 sorted sets
15,000 score updates/second x 4 = 60,000 ZADDs/second
```

- now at ~150% of one instance
- so: pipeline them (one round trip), and shard BY WINDOW across instances if needed – which is easy, because the windows are independent structures

Sharding by window is the easy sharding — the daily board on one instance, all-time on another — **because no query ever crosses them.**

Memory for the windowed copies.

```
all-time: 50,000,000 players      ~5 GB
daily:    ~10,000,000 active players/day  ~1 GB  (TTL 2 days → 2 GB)
weekly:   ~25,000,000              ~2.5 GB  (TTL 2 weeks → 5 GB)
monthly:  ~40,000,000              ~4 GB
```

total: ~16 GB of Redis

- one large instance or a small cluster, and the TTLs keep it from growing without bound

The approximate structures, for contrast.

```
HISTOGRAM (buckets of 100 points, scores up to 1,000,000)
10,000 buckets x ~20 bytes = 200 KB
```

- the ENTIRE rank-approximation structure is 200 KB against 5 GB for the exact one
- 25,000x smaller, accurate to within 100 points of score

```
TOP 10,000 EXACT
10,000 x 100 B = 1 MB
```

- exact where it matters, 1 MB
- approximate everywhere else, 200 KB
- 1.2 MB total against 5 GB

A thousandth of the memory for a product that is arguably better — exact where people care, approximate where they do not.

The sharded-rank cost, which is the argument against it.

```
rank across 10 shards:
  10 x ZCOUNT, each  $O(\log n)$ 
  ~10 round trips (or one pipelined) → ~1-2 ms

against 100 us for a single sorted set

→ 10-20x slower, AND it gets worse with more shards
→ which is backwards: sharding should help
```

Rebuild time.

```
50,000,000 players from the durable store
  reading: ~10,000 rows/second/connection, 10 connections
           → ~500 seconds
  ZADD:    pipelined in batches of 10,000
           → ~50,000,000 / 100,000 per second = 500 seconds

total: ~10-15 minutes

→ acceptable for a leaderboard.
  NOT acceptable for anything transactional, which is exactly
  why Redis is the serving layer here and not the record.
```

Latency budget.

```
top ten           ~0.2 ms
my rank           ~0.2 ms
players around me ~0.3 ms (rank, then a range)
+ network         ~1-5 ms

→ the leaderboard is not a latency problem.
  It is a MEMORY problem at very large scale, and a PRODUCT
  problem in the middle of the distribution.
```

7 The trade-offs

Exact rank against approximate. Exact costs a full sorted set — five gigabytes at fifty million players — and answers in microseconds. **Approximate costs two hundred kilobytes and is accurate to a bucket.** And for most of the distribution, **approximate is the better product:** “about 2,100th of 50,000” is more useful than an exact number that changes every few seconds. **Exact at the top, approximate below** is the answer, and it is a product decision that happens to be the cheap one.

One sorted set against sharding. One instance handles fifty million players at half its capacity — **so do not shard until forced**. And when forced, **shard by leaderboard, not by player**, because rank is global and a split structure makes every rank query hit every shard **and get slower as you add shards**.

Time windows: separate structures against computing on demand. Separate sorted sets per window cost three to four times the writes and make every read a single lookup. **Computing “this week’s top” from an event log is minutes, which no live board can absorb**. The write amplification is the right trade **because reads outnumber writes and each read must be fast**.

Rolling windows against fixed ones. “Today” and “this week” expire themselves with a TTL. **“The last twenty-four hours” has nothing to expire** and would need a sliding structure. **Hourly buckets with an hour of granularity is the honest approximation**, and pretending to offer an exact rolling window is how a simple feature becomes an expensive one.

Best score against latest score. Most games rank a player’s best, which needs an atomic update-only-if-greater — **without which a late-arriving old submission overwrites a better one**. Ranking the latest is simpler and makes the board volatile in a way players dislike. **Say which the product wants**.

Redis as serving structure against source of truth. Serving-only means a Redis failure costs a ten-minute rebuild and nothing else. **Treating it as authoritative removes the durable write and loses scores on failure** — and scores are the product. **The ten minutes is affordable precisely because it is a leaderboard**; the same architecture would be unacceptable for a ledger.

When would I not build this? Below a few hundred thousand players, `ORDER BY score DESC LIMIT 10` on the database is the whole feature, and rank via a window function is fine at that size. **Managed services exist** — PlayFab, GameSparks, Firebase — **and for a small game they are cheaper than the operational cost of a Redis cluster**. The design here is justified by scale and by the rank query specifically, **not by the top-ten query, which was never hard**.

8 In the interview

How it gets asked

- “Design a leaderboard.” — usually with a scale like fifty million players.
- “How do you show a player their rank?” — the question that is actually hard.

- *“What data structure would you use, and why?”* — where naming the skip list matters.
- *“What if there are a hundred million players?”* — the scaling question.
- *“How do you handle daily and weekly boards?”*
- *“Two players have the same score. What order?”*

“There are two questions here and they have completely different costs, **and I would separate them immediately because almost every naive design gets one right and cannot answer the other.**

The top ten is easy. Read the first ten entries of anything sorted by score. **Cheap in any store.**

The rank is hard. ‘What is my position’ means **counting how many players have a score above mine** — which is a full scan unless the structure was built for it. **That asymmetry is the actual problem.**

The right default is a Redis sorted set, and I want to say why rather than just name it. **Underneath it is a skip list plus a hash map.** The skip list keeps elements in score order, **and each forward pointer carries a span — the number of elements it jumps over.** Rank is the sum of the spans along the search path, which makes it logarithmic rather than a count. The hash map gives constant-time score lookup, so updating an existing player needs no search.

That combination is the whole reason it is the standard answer.

Sizing it: about a hundred bytes per entry, so fifty million players is five gigabytes — one instance, comfortably. **And every operation is fifty to a hundred microseconds**, so at fifteen thousand writes and forty thousand reads a second, **I am at roughly half of one Redis instance.** The data structure is not the bottleneck at any realistic scale for a single board.

Where it stops is memory, and the answer is not to shard the sorted set. Rank is inherently global — ‘how many are above me’ is a question about every player — **so a split structure means every rank query hits every shard and gets slower as you add shards.** That is backwards.

Three better answers. Partition the leaderboard by a product dimension — region, level, week, friend group — which is usually what the product already wants. **Keep the top ten thousand exact and approximate everyone else. Or a score histogram**, where a rank is the sum of the buckets above yours.

And the approximate answer is not a compromise for most users. ‘About two thousand one hundred of fifty thousand’ is a better answer than an exact number nobody can act on. **Exact at the top, approximate in the middle** — a product decision that happens to be a thousand times cheaper.

One thing before I go further: does the board rank a player's best score or their most recent? Because best needs an atomic update-only-if-greater, and getting that wrong lets a late submission lower somebody's score."

The follow-ups

"What data structure, and why?"

“A Redis sorted set, and the interesting part is what is underneath, because that is what explains the cost.

A sorted set is two structures kept in step. A skip list, which keeps the members in score order, and a hash map from member to score.

The hash map is why updating a player is cheap — I can find their current score in constant time without searching the list.

And the skip list is why rank is cheap, because of one detail: each forward pointer carries a span — the number of elements it jumps over. **So when I search down for a member, I sum the spans I skip past, and that sum is the rank.** Logarithmic, not a count.

Without those spans, a skip list would give me ordered iteration and rank would still be $O(n)$ — so the spans are precisely the feature that makes this the right structure, and they are exactly the equivalent of writing a number in the margin every hundred names.

The operations I would use: `ZADD` to update a score, `ZREVRANK` for a player’s rank, `ZREVRANGE` for the top ten or for a window around a player. **All $O(\log n)$, all about fifty to a hundred microseconds at fifty million entries.**

Two implementation details worth mentioning. `ZREVRANK` is zero-indexed and users count from one, which is a one-character bug that ships more often than it should. **And `ZADD` with `GT` makes ‘keep the best score’ atomic** — without it, reading the current score and then writing is a race, and a late-arriving old submission can lower a player’s score.

Sizing: about a hundred bytes per entry including the skip-list node, the hash entry and the member string. **Fifty million players is five gigabytes.**

The alternative I would rule out explicitly is a relational database with `ORDER BY score`. The top ten is fine — that is an index scan. **But rank requires a window function or a count, which is a scan over every row above you, and at fifty million rows that is seconds. Below a few hundred thousand players it is genuinely the right answer, and above that it is not.”**

“A hundred million players. What breaks?”

“Memory, and then the instinct to shard — **which is the interesting part, because sharding makes it worse.**

A hundred million entries at a hundred bytes is ten gigabytes. That is one large instance, and it is beginning to be uncomfortable — **not for throughput, which is fine, but for the blast radius and the restart time.**

And the obvious fix does not work. Rank is inherently global: ‘how many players have a score above mine’ is a question about every player, everywhere. **If I split the sorted set across ten shards, every rank query has to ask every shard how many of its members are above my score and sum the answers.**

That is correct and it is ten to twenty times slower — and it gets worse as I add shards. Sharding is supposed to make things faster with scale, **and here it does the opposite.**

So there are three real answers, and I would take the first.

One: partition the leaderboard rather than the data. Most games do not actually have one global board — **they have one per region, per level, per season, per friend group.** Each of those is a *smaller structure*, not a *split* one, **and every query stays entirely within it.** This is usually what the product already wants, so it is a product fact rather than an engineering compromise — **and it is by far the most common answer in practice.**

Two: exact top N, approximate below. Keep a bounded sorted set of the top ten thousand, trimmed on every write, **so it stays at about a megabyte regardless of how many players exist.** Everyone outside it gets an approximate rank.

Three: a score histogram — how many players fall in each hundred-point band. **A rank is the sum of the buckets above yours.** Ten thousand buckets is two hundred kilobytes, **against ten gigabytes for the exact structure — twenty-five thousand times smaller, accurate to within a bucket.**

And I would argue that the approximation is the better product below the top few hundred. ‘About two thousand one hundred of fifty thousand’ is **more useful than an exact number** that changes every few seconds and that nobody can act on. **The interface should show which it is** — ‘about’ or a precise number — rather than implying precision it does not have.”

“Two players have the same score. What order do they appear in?”

“By default, an arbitrary one — and it is arbitrary in a way that is stable and therefore visible, which is worse than random.

Redis breaks ties lexicographically by member name. So if members are `player:aaa` and `player:zzz`, `aaa` **always ranks above** `zzz` **at equal score, forever.** Players notice patterns like that, and it looks like favouritism rather than a data structure detail.

So the tie-break has to be a product decision, and then encoded.

The usual rule is: on equal score, whoever achieved it first ranks higher — which rewards being early and is what most people expect.

And the way to implement it is to fold the tie-break into the score itself, rather than maintaining a second structure. **Multiply the real score by a large constant and add an inverted timestamp,** so a lower timestamp produces a higher composite.

Two things I would be careful about, and I would say both.

Precision. A double holds fifty-three bits of exact integer range — **about nine times ten to the fifteen.** So if the real score can be a million and I multiply by ten to the ten, I am at ten to the sixteen and **I have silently lost precision,** which reorders players in a way nobody will ever diagnose. **I would assert the bound rather than assume it,** and pick the scale from the actual maximum score.

And the displayed score must be the real one, recovered by integer division — **not the composite,** which would put an absurd number on screen. That sounds obvious and is a genuine bug people ship.

The alternative is to keep the tie-break outside the score — sort by score in Redis, then re-sort ties by timestamp in the application. **That works for the top ten and cannot work for rank,** because the rank query happens inside Redis. **So encoding it into the score is the answer whenever rank matters.”**

The model answer

“Design the leaderboard for a mobile game: fifty million players, daily, weekly and all-time boards, and each player must see their own rank.”

“Let me separate the two questions first, because they have different costs and the design follows from that.

The top ten is easy in any sorted store. The rank is the hard part — ‘how many players are above me’ is a count over everyone unless the structure supports it. **And ‘each player must see their own rank’ is in the requirements, so this is a rank problem, not a top-ten problem.**

Sizing. Fifty million players at five sessions a day is two hundred and fifty million score updates — about three thousand a second, fifteen thousand at peak. **Reads are only about three times that**, which is unusually write-heavy for a read-facing feature, **so the write path being cheap matters as much as the read path.**

The structure: Redis sorted sets. A skip list keeping members in score order, with spans on the forward pointers so rank is the sum of the spans along the search path — logarithmic, not a count — **plus a hash map for constant-time score lookup on update.**

About a hundred bytes an entry, so five gigabytes for fifty million players, and every operation is fifty to a hundred microseconds. **At peak that is roughly half of one Redis instance** — the data structure is comfortably not the bottleneck.

Windows: a separate sorted set per window, with TTLs. `daily:2026-09-01` expiring after two days, `weekly:2026-W35` after two weeks, `alltime` never. **A score update writes to all of them in one pipelined round trip** — three or four times the writes, and every read stays a single lookup. **The TTL removes old windows with no cleanup job.**

And that pushes me to about a hundred and fifty percent of one instance, so I would **shard by window** — daily on one instance, all-time on another. **That is the easy sharding, because no query ever crosses windows.** Total memory across all windows is around sixteen gigabytes.

Best score, not latest, which most games want — using `ZADD` with `GT` so it is atomic. **Without that, reading and then writing is a race, and a late-arriving old submission lowers somebody’s score.**

Ties: encode the tie-break into the score — the real score times a large constant, plus an inverted timestamp, so an earlier achiever wins. **And I would assert the fifty-three-bit precision bound rather than assume it,** because

exceeding it silently reorders players. **The displayed score is recovered by division; the composite never reaches the screen.**

Durability: the game's database is the source of truth and Redis is the serving structure. The durable write happens first. **A Redis failure costs a ten-minute rebuild** — build into a temporary key, then `RENAME`, which is atomic, so the old board serves until the new one is complete. **Ten minutes is fine for a leaderboard and would be unacceptable for anything transactional**, which is exactly why this architecture is allowed here.

Two things I would raise unprompted.

At a hundred million players, do not shard the sorted set. Rank is global, so a split structure makes every rank query hit every shard and get slower as you add shards — the opposite of what sharding is for. **Partition the leaderboard by a product dimension instead** — region, level, season — which is a smaller structure rather than a split one.

And I would propose exact-at-the-top, approximate-below as a feature rather than a fallback. The top ten thousand exact in a bounded set — about a megabyte — **and everyone else from a score histogram, which is two hundred kilobytes and accurate to a hundred points.** A thousandth of the memory, and for a player at rank two thousand it is arguably the better answer, because a precise number that changes every few seconds is not information they can use. **The interface should say 'about', so the approximation is honest rather than hidden.**

9 Recall card

Two questions with different costs, and separating them is the opening move: the TOP TEN is easy in any sorted store; the RANK is a count over everyone unless the structure supports it. Almost every naive design gets the first and fails the second.

Redis sorted sets — and say what is underneath. A skip list with SPANS on the forward pointers (rank is the sum of spans along the search path, so $O(\log n)$ rather than a count) **plus a hash map** for $O(1)$ score lookup on update. **The spans are precisely the feature that makes rank cheap.** ~ 100 bytes/entry $\rightarrow$ 50M players ≈ 5 GB, ~ 50 – 100 μ s per operation, **about half of one instance at peak.**

Do NOT shard the sorted set: rank is inherently GLOBAL, so a split makes every rank query hit every shard **and get slower as you add shards**. Three better answers: **partition the leaderboard by a product dimension** (region, level, season — usually what the product wants anyway); **exact top 10,000 in a bounded set** (~1 MB, trimmed with `ZREMRANGEBYRANK` on every write); **or a score histogram** (~200 KB, $25,000 \times$ smaller, accurate to a bucket).

Approximate is the better product below the top few hundred — “about 2,100th of 50,000” beats a precise number that changes every few seconds — **and the interface should say “about” rather than imply precision**.

Ties break lexicographically by member name by default — stable, arbitrary, and visible. **Encode the tie-break into the score** (`score × 1010 + inverted timestamp`), **and assert the 53-bit double precision bound**, because exceeding it silently reorders players. **Display the real score, never the composite**.

Windows are separate sorted sets with TTLs ($3\text{--}4 \times$ write amplification, every read one lookup; shard *by window*, which is easy because no query crosses them). **“Rolling last 24 hours” has nothing to expire** — hourly buckets and an hour of granularity is the honest answer. `ZADD ... GT` **makes best-score atomic**; `ZREVRANK` **is zero-indexed**. **Redis is the serving structure, not the source of truth** — rebuild into a temp key and `RENAME`, ~10 minutes, which is affordable precisely because it is a leaderboard.

DSA topic: Merging intervals **System design topic:** Design a leaderboard

Code these, in this order

One rule for the whole set: **say “further of the two, not the new one” out loud as you write the extension line.** It is the bug of the day, it passes typical tests, and saying it is the only reliable defence.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Merge Intervals	LeetCode 56 (Medium)	The core loop, and the <code>max</code> on the extension.
2	Insert Interval	LeetCode 57 (Medium)	Whether you re-sort a sorted input.
3	Interval List Intersections	LeetCode 986 (Medium)	Two pointers, and which one to advance.
4	Employee Free Time	LeetCode 759 (Hard)	The complement, and its two boundary cases.
5	Data Stream as Disjoint Intervals	LeetCode 352 (Hard)	Streaming, and multi-block absorption.
6	Remove Covered Intervals	LeetCode 1288 (Medium)	Containment as the subject rather than the bug.

On problem 1, write the bug first

Implement the loop with `current_end = end`. Run on `[(1,3),(2,6),(8,10)]` — the typical test — and record the output. **Then run it on** `[(1,10),(2,3),(11,12)]` and record that.

One passes and one does not. Say why the failing shape is rare in tests and common in real data.

On problem 1, sort by end

Run the merge with the wrong sort key on `[(1,10),(2,3),(11,12)]`. **Record the output and say exactly what was lost** — which field of which interval, and why no later step could recover it. **Then say why the block count being right makes it worse.**

On problem 1, use tuples

Build `merged` as a list of tuples and try to extend the last one. **Record the exact error.** Say why this is the friendliest bug of the day.

On problem 2, count the operations

Solve it by appending and re-sorting. Then write the three-phase version. **Count the operations each performs on a thousand-interval input** and say what the problem was testing.

On problem 3, advance the wrong pointer

Always advance `i`. Run it and record the answer. **Then say why advancing whichever ends first is correct**, in one sentence about what that interval can still intersect.

On problem 4, drop the bounds

Write `free_time` without a lower and upper bound. **Say what the answer is before the first busy block and after the last**, and why the question has no answer at the edges without them.

Then add the bounds, and **deliberately pass a lower bound that falls inside the data**. Say what `cursor = end` does that `cursor = max(cursor, end)` does not.

On problem 5, absorb only the neighbour

Write the streaming version so that a new interval merges with at most one existing block. Add `(1,3)`, `(8,10)`, `(15,18)`, then `(4,16)`. **Record the structure after each.** Say what is now wrong and why it compounds.

Then the variants drill

Implement all five variants from one merge function. **Time each on a hundred-thousand-interval input** and record which are `O(n)` and which are `O(n log n)`, and why.

Then the verification drill

Write a brute force that merges by marking a small number line, and check your merge against it on five hundred random inputs. **Note which boundary convention the brute force implies**, and make sure you compare against the matching version.

The algorithm drill

1. Give the loop in four lines.

2. State the invariant that sorting by start establishes.
3. Say why a closed block never reopens.
4. Give the extension line and Hemalatha's six words.

The bug drill

1. Give the input that catches `= end` and both outputs.
2. Give the input where it is invisible, and both outputs.
3. Say why containment is rare in tests and common in real data.
4. Give the wrong-sort-key output and say what was lost.
5. Say why the correct block count makes that one worse.

The convention drill

1. Say what `≤` and `<` each mean for `[1, 3]` and `[3, 5]`.
2. Say which LeetCode uses.
3. Say what you do when the statement does not say.

The variants drill

For each: the technique, the complexity, and the specific trap.

1. Insert into a sorted disjoint list.
2. The gaps.
3. Intersect two lists.
4. Total covered length.
5. Streaming arrivals.

The streaming drill

1. Say why the one-pass algorithm does not apply.
2. Give the cost of re-sorting on every arrival.
3. Describe the structure and the two steps per insert.
4. Give the per-insert and the amortised cost.
5. Say why the amortisation works.
6. Give the case that a nearest-neighbour merge gets wrong.

The break-it drill

For each, say what happens and whether anything reports it:

1. `= end` on a contained interval.

2. Sorting by end.
 3. Forgetting the final flush in the two-variable version.
 4. Extending a tuple.
 5. Re-sorting a sorted input.
 6. `cursor = end` in the gaps.
 7. Nearest-neighbour-only merging in the stream.
-

The asymmetry drill

1. Give the two questions and their costs.
2. Say which one almost every naive design fails.
3. Say why that is the opening move.

The structure drill

1. Name the two structures inside a sorted set.
2. Say what each is for.
3. Say what the spans are and why they matter.
4. Give the four operations and their complexities.
5. Give bytes per entry and size it at 50 million.
6. Give the per-operation latency and the instance utilisation.

The sharding drill

1. Say why rank cannot be sharded usefully.
2. Give the cost of a sharded rank query.
3. Say why it gets worse with more shards.
4. Give the three alternatives.
5. Say which you would take, and why it is a product fact.

The approximation drill

1. Describe the histogram and its size.
2. Give the ratio against the exact structure.
3. Say where exact is required and where it is not.
4. Give the sentence that justifies it as a product decision.
5. Say what the interface must show.

The ties drill

1. Say what Redis does by default and why that is worse than random.
2. Give the composite-score formula.
3. State the precision bound and what exceeding it does.
4. Say what must never reach the screen.
5. Say why an application-level tie-break cannot work for rank.

The windows drill

1. Give the three or four windows and their TTLs.
2. Compute the write amplification.
3. Say why that trade is right.
4. Say which sharding is easy here, and why.
5. Say what is genuinely hard about a rolling window, and the honest answer.

The durability drill

1. Say what is the source of truth and what is not.
 2. Give the write ordering and what the other ordering loses.
 3. Describe the rebuild, including the atomic step.
 4. Give the rebuild time and say why it is acceptable here.
 5. Say what kind of system it would not be acceptable for.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Merge these intervals.* The sort key and the invariant it gives, the loop, the extension line with its bug, the boundary convention, and the complexity.
 2. *What if they arrive one at a time?* Why the single pass does not apply, the structure, the two steps per insert, multi-block absorption, and the amortised cost with its reason.
 3. *Design a leaderboard.* The top-ten versus rank asymmetry, the skip list with its spans, the sizing, why rank cannot be sharded, and exact-at-the-top with approximate below.
-

Before you move on

- [] I sort by start and can state the invariant it gives.
- [] I say “further of the two, not the new one” while writing the extension.
- [] I know the input that catches `= end` and the one where it is invisible.
- [] I know why containment is rare in tests and common in real data.
- [] I know what sorting by end loses, and why the block count stays right.
- [] I settle the boundary convention, and ask when it is unstated.
- [] I build with lists or two variables, never by extending a tuple.
- [] I remember the final flush in the two-variable version.
- [] I do not re-sort an already-sorted input.
- [] I can write insert as three phases.
- [] I know the gaps need explicit bounds and why.
- [] I know `cursor = max(cursor, end)` and what it guards against.
- [] I can intersect two lists and say which pointer to advance, and why.
- [] I know why merging first is required before summing coverage.
- [] I know the single pass does not apply to a stream.
- [] I can describe the streaming structure and its two steps.
- [] I know one arrival can absorb several blocks.
- [] I can give the amortised cost and explain why it holds.
- [] I open a leaderboard answer with the top-ten versus rank asymmetry.
- [] I can name both structures inside a sorted set and what each does.
- [] I can explain the spans and why they make rank logarithmic.
- [] I can size it: bytes per entry, total, and instance utilisation.
- [] I know rank cannot be usefully sharded, and why it gets worse.
- [] I can give the three alternatives and pick one.
- [] I know the histogram size and the ratio against exact.
- [] I can justify approximation as a product decision, not a fallback.
- [] I know the default tie-break and why it is worse than random.
- [] I can give the composite score and its precision bound.
- [] I know the window TTL design and the write amplification.
- [] I know Redis is the serving structure, and the rebuild path.
- [] I answered all three questions above out loud.

Meeting rooms and the sweep line

DATA STRUCTURES AND ALGORITHMS

Meeting rooms and the sweep line

SYSTEM DESIGN

Design an ad click aggregator

1 What this is, and why they ask it

A sweep line answers questions about *how many things are happening at once*, and it does it by throwing away which things they are.

The canonical form is meeting rooms: given a set of meetings, **how many rooms do you need?** And the answer is not a scheduling algorithm — **it is a counting algorithm.**

They ask it because **the decomposition is a genuine idea rather than a pattern to memorise.** Break every interval into two independent events, a start and an end. **Discard the association between them.** Sort all the events by time and walk through, adding one and subtracting one. **The maximum the running total reaches is the answer.**

That “discard the association” step is what people do not think of, and it is what turns a problem that sounds like assignment into six lines of counting.

The second reason is that **it generalises much further than meeting rooms.** The same shape answers: the busiest minute of the day, whether any two intervals overlap, the total covered length, how many people were in the building at 3pm, and — with one change — the skyline problem, which is the hard version and appears in senior interviews.

And there is one specific detail that is wrong in about half of all implementations: the tie-break. When a meeting ends at four and another starts at four, **which event is processed first decides whether the answer is one or two.** It is one comparison, it depends on the problem’s convention, **and getting it backwards gives an answer exactly one too high on the most realistic inputs there are.**

By the end of this lesson you can write the sweep, get the tie-break right, know the two equivalent forms, extend it to the variants, and say when it is the wrong tool.

2 The story

The temple ran the free meal on festival days and the thing nobody outside the kitchen understood was that the problem was never the food.

It was the plates.

Because they had four hundred and eleven steel plates, and about three thousand people came through over five hours, **and a plate came back and was washed and went out again.** So three thousand people did not need three thousand plates.

They needed as many plates as there were people eating at the same moment.

And for eleven years the way they worked that out was that Jagannath stood at the door and watched, **and when the stack got low somebody ran to borrow from the next street.**

Which worked, badly, and twice it did not work at all.

What changed it was the boy who kept the register, who was doing his intermediate and had been made to help.

He was not interested in the plates. He was interested in the door.

Because at the door, **you did not need to know who anybody was.** He stopped writing names entirely and wrote two things: **the time somebody went in, and the time somebody came out.** Two separate lines. **Not joined up.**

And then he did something that took him an evening and that Jagannath thought was a waste of an evening.

He put every one of those times in order — the going-ins and the coming-outs all mixed together — and walked down the list with a pencil, adding one for each going-in and taking one away for each coming-out.

The number went up and down all afternoon.

And the biggest it ever got was three hundred and eighty-four. At about half past one.

Four hundred and eleven plates. They had never actually needed to borrow.

Jagannath's objection, which he made immediately, was that the boy did not know which plate went to which person and therefore could not possibly have worked it out.

"I do not need to know which plate," the boy said. "I only need to know how many at once. Those are different questions and I have been answering the easier one."

3 The idea in plain English

The boy's two lines at the door are the decomposition, and his last sentence is the whole technique: **"how many at once" is a different and much easier question than "which one is which"**.

Start with what the sweep actually is.

Break every interval into two independent events: `+1` at its start and `-1` at its end. Throw away which interval they came from. Sort every event by time. Walk through, keeping a running total. The maximum is the answer.

The discarding is the idea. Once the events are independent, there is no assignment problem left — only a list of numbers and a running sum.

```
meetings: [1,4] [2,5] [7,9] [3,6]
events:   1:+1  2:+1  3:+1  4:-1  5:-1  6:-1  7:+1  9:-1
running:   1    2    3    2    1    0    1    0
           ^
           maximum 3 → three rooms
```

Six lines of code, and the hardest part is realising that the meetings can be taken apart.

Now the tie-break, which is where half of all implementations are wrong.

A meeting ends at 4 and another starts at 4. Is that one room or two?

For meeting rooms: one. The first group leaves and the second arrives; the room is free. So at equal times, process the END first — the counter goes down before it goes up, and it never reaches two.

For a question where the endpoints are inclusive — “how many people were present at time 4” — both count, so the start is processed first.

It is one comparison and the problem decides it. And in Python there is a pleasant trick: sort the events as `(time, delta)` tuples. Because `-1 < +1`, ties are broken in favour of the end automatically, which is the meeting-rooms convention, for free.

Encode the events as `(time, "start")` and `(time, "end")` instead and it happens to work — `"end"` sorts before `"start"` — and that is a coincidence of the alphabet, not a design. Use numbers.

Getting it backwards gives an answer exactly one too high, on exactly the inputs a real calendar produces, where meetings abut on the hour.

Then the second form, which is the same algorithm and reads differently.

Sort the start times and the end times into two separate arrays, and merge them with two pointers.

```
if starts[i] < ends[j]:
    rooms += 1; i += 1
else:
    rooms -= 1; j += 1
```

Identical answer. Some people find it clearer; it also makes the tie-break explicit — $<$ against $\leq$ — rather than hiding it in a sort key.

And a third form, which answers a different question: the heap.

The sweep counts rooms. It cannot tell you which meeting is in which room, because it threw that information away — which is precisely what made it cheap.

If you need the assignment, process the meetings in start order and keep a min-heap of the rooms' end times: reuse the earliest-freeing room if it is free by now, otherwise open a new one. The heap's size is the room count, so it agrees with the sweep and also gives you the answer to “which room”.

Now the variants, because the sweep generalises much further than meeting rooms.

The busiest moment, not just the maximum count — track *when* the maximum occurred, which is one extra variable.

Does anything overlap at all? — the maximum reaching two. Though for that specific question, sorting the intervals and checking adjacent pairs is simpler.

Total covered length — sum the time between events while the counter is above zero. Which is merging, done differently, and it is worth noticing that the sweep can do it.

Weighted sweeps, where events carry a value rather than one — how much bandwidth is in use, how many people are in the building weighted by group size. The $+1$ and -1 become $+w$ and $-w$ and nothing else changes.

And the skyline problem, which is the hard version. Buildings as intervals with heights; produce the outline. The sweep is the same, but the running total is not a sum — it is the *maximum* of the active heights, so instead of a counter you need a

multiset that can report its maximum and remove an arbitrary element. **A heap with lazy deletion is the usual answer**, and knowing that it is “the same sweep with a different accumulator” is what makes the problem approachable.

- Finally: when the sweep is the wrong tool.
- When you need to know which interval is which — the assignment — it has thrown that away.
- When the events are not points in one dimension. Rectangles overlapping in two dimensions need a sweep in one axis with a *structure* over the other, **which is a substantially harder algorithm**.
- And when the coordinates are dense and small. If times are minutes in a day, a plain array of 1,440 counters is simpler and faster than sorting events — increment a range, then scan. **That is a difference prefix-sum, and reaching for a sweep when a small array suffices is over-engineering.**

4 The picture

The decomposition, which is the idea:

MEETINGS (paired)

[1 ----- 4]
[2 ----- 5]
[3 ----- 6]
[7 -- 9]

EVENTS (independent)

1:+1 4:-1
2:+1 5:-1
3:+1 6:-1
7:+1 9:-1

THROW AWAY WHICH IS WHICH.

sorted: 1:+1 2:+1 3:+1 4:-1 5:-1 6:-1 7:+1 9:-1

running: 1 2 3 2 1 0 1 0

^

MAXIMUM 3

"I do not need to know which plate. I only need to know how many at once. Those are different questions."

The tie-break, which is where half of implementations are wrong:

meeting A ends at 4. meeting B starts at 4.

PROCESS THE END FIRST (meeting rooms – the room is freed):

```
... 4:-1  4:+1 ...  
running: 1 → 0 → 1          MAXIMUM 1      ONE ROOM
```

PROCESS THE START FIRST (inclusive endpoints – both present at 4):

```
... 4:+1  4:-1 ...  
running: 1 → 2 → 1          MAXIMUM 2      TWO
```

ONE COMPARISON, AND THE PROBLEM DECIDES IT.

IN PYTHON: sort (time, delta) tuples.

-1 < +1, so the END sorts first automatically
→ the meeting-rooms convention, for free

Encoding as (time, "start") / (time, "end") happens to work because "end" < "start" alphabetically. That is a coincidence of the alphabet, not a design. USE NUMBERS.

The two equivalent forms:

EVENT LIST

```
events = []  
for s, e in meetings:  
    events.append((s, +1))  
    events.append((e, -1))  
events.sort()  
  
for _, d in events:  
    current += d  
    best = max(best, current)
```

sorts 2n events
the tie-break is in the sort
→ SAME ANSWER, same complexity. Pick whichever is clearer.

TWO POINTERS

```
starts = sorted(all starts)  
ends   = sorted(all ends)  
  
i = j = rooms = best = 0  
while i < len(starts):  
    if starts[i] < ends[j]:  
        rooms += 1  
        best = max(best, rooms)  
        i += 1  
    else:  
        rooms -= 1  
        j += 1
```

sorts two arrays of n
the tie-break is the < vs ≤

What the sweep cannot do, and what can:

THE SWEEP

"how many rooms?"

discards the association

→ cheap

→ and cannot answer
the assignment

3

THE HEAP

"WHICH room for each meeting?"

keeps it:

process in START order

keep a min-heap of room END times

reuse the earliest-freeing room

if free, else open a new one

[1, 2, 3, 1]

^ the heap SIZE equals the sweep's
answer, so they agree

The variants, all the same shape:

question

what changes

how many rooms

the BUSIEST moment

does anything overlap

total covered length

weighted (bandwidth, people)

THE SKYLINE

the maximum of the running total

also record WHEN the max occurred

the maximum reaching 2

sum the time while the total > 0

+w and -w instead of +1 and -1

the accumulator is a MAX, not a sum

→ a multiset / heap with lazy deletion

→ the sweep is a FRAMEWORK: sort events, walk, accumulate.
Only the accumulator changes.

When not to sweep:

DENSE, SMALL COORDINATES

times are minutes in a day: 1,440 possible values

sweep:	sort $2n$ events	$O(n \log n)$
difference array:	<code>counts[start] += 1</code>	
	<code>counts[end] -= 1</code>	
	then one prefix sum	$O(n + 1440)$

→ for $n = 10,000$ meetings the array version is FASTER and simpler. Reaching for a sweep here is over-engineering.

TWO DIMENSIONS

overlapping RECTANGLES need a sweep in x with a STRUCTURE over y (a segment tree) – a substantially harder algorithm, not a small extension.

WHEN YOU NEED THE ASSIGNMENT

the sweep threw it away. Use the heap.

5 The code, built step by step

The sweep

```
def min_rooms(intervals: list[tuple[int, int]]) → int:
    """
    How many at once? Break into events; discard which is which.

    (time, delta) sorting puts -1 before +1 at equal times, which is
    the meeting-rooms convention: a room freed at 4 is reusable at 4.
    """
    events: list[tuple[int, int]] = []
    for start, end in intervals:
        events.append((start, +1))
        events.append((end, -1))
    events.sort()

    current = best = 0
    for _, delta in events:
        current += delta
        best = max(best, current)
    return best
```

Six lines of work, and the comment is the important part. `events.sort()` on tuples sorts by time and then by delta, and since `-1 < +1` the end is processed first — exactly the convention meeting rooms needs, with no explicit tie-break code.

The other convention

```
def max_concurrent_inclusive(intervals: list[tuple[int, int]]) → int:
    """Endpoints INCLUSIVE: both a start and an end at time t count."""
    events: list[tuple[int, int]] = []
    for start, end in intervals:
        events.append((start, 0, +1))          # 0 sorts before 1
        events.append((end, 1, -1))
    events.sort()

    current = best = 0
    for _, _, delta in events:
        current += delta
        best = max(best, current)
    return best
```

An explicit priority field is the honest way to express the other convention — 0 for starts and 1 for ends — rather than relying on the sign of the delta.

Writing both versions once is worth doing, because the difference between them is exactly the question the interviewer is checking you asked.

The two-pointer form

```
def min_rooms_two_pointer(intervals: list[tuple[int, int]]) → int:
    """The same algorithm; the tie-break is the `<` rather than a sort key."""
    starts = sorted(s for s, _ in intervals)
    ends = sorted(e for _, e in intervals)
    rooms = best = 0
    i = j = 0
    while i < len(starts):
        if starts[i] < ends[j]:          # `<`: a room freed at 4 is reusable
            rooms += 1
            best = max(best, rooms)
            i += 1
        else:
            rooms -= 1
            j += 1
    return best
```

`starts[i] < ends[j]` is the same decision as the sort key, expressed where you can see it. Change it to `≤` and you get the inclusive convention.

And the loop condition is `i < len(starts)` alone, because once every meeting has started the remaining ends cannot raise the maximum — iterating both to exhaustion is harmless and wasteful.

The busiest moment, not just the count

```
def busiest_moment(intervals: list[tuple[int, int]]) → tuple[int, int]:
    """Returns (peak_count, when). One extra variable."""
    events = sorted([(s, +1) for s, _ in intervals] +
                    [(e, -1) for _, e in intervals])
    current = best = at = 0
    for time, delta in events:
        current += delta
        if current > best:
            # `>` records the FIRST peak
            best, at = current, time
    return best, at
```

`if current > best` rather than `≥` records the *first* moment the peak is reached, which is usually what “when were we busiest” means — and `≥` would report the last, which is a defensible different answer worth naming.

Total covered length, via the sweep

```
def total_covered(intervals: list[tuple[int, int]]) → int:
    """Sum the time while the counter is above zero. Merging, differently."""
    events = sorted([(s, +1) for s, _ in intervals] +
                    [(e, -1) for _, e in intervals])
    current = total = 0
    previous_time = 0
    for time, delta in events:
        if current > 0:
            total += time - previous_time    # this stretch was covered
            current += delta
            previous_time = time
    return total
```

Accumulating *before* applying the delta is the ordering that matters: the stretch between the previous event and this one was covered at the *old* count, not the new one.

And this is the merge from yesterday, computed without producing the blocks — which is worth noticing, because it means the sweep subsumes it.

A weighted sweep

```
def peak_bandwidth(streams: list[tuple[int, int, int]]) → int:
    """(start, end, mbps). +w and -w instead of +1 and -1. Nothing else changes."""
    events = sorted([(s, +w) for s, _, w in streams] +
                    [(e, -w) for _, e, w in streams])
    current = best = 0
    for _, delta in events:
        current += delta
        best = max(best, current)
    return best
```

The only change is the magnitude of the delta, which is the clearest demonstration that the sweep is a framework rather than one algorithm.

The heap, when you need the assignment

```
import heapq

def assign_rooms(intervals: list[tuple[int, int]]) → list[int]:
    """The sweep COUNTS; the heap ASSIGNS. Same answer for the count."""
    order = sorted(range(len(intervals)), key=lambda i: intervals[i][0])
    free: list[tuple[int, int]] = []          # (end_time, room_number)
    assignment = [0] * len(intervals)
    rooms_used = 0
    for i in order:
        start, end = intervals[i]
        if free and free[0][0] ≤ start:      # the earliest-freeing room is ready
            _, room = heapq.heappop(free)
        else:
            rooms_used += 1
            room = rooms_used
        assignment[i] = room
        heapq.heappush(free, (end, room))
    return assignment
```

`free[0][0] ≤ start` is the tie-break again — a room freed at 4 is reusable at 4 — and it must match whatever the sweep uses, or the two disagree.

`rooms_used` ends equal to the sweep's answer, which is a free consistency check worth running.

The difference array, for dense coordinates

```
def min_rooms_dense(intervals: list[tuple[int, int]], horizon: int = 1440) → int:
    """Times are minutes in a day: an array beats sorting events."""
    counts = [0] * (horizon + 1)
    for start, end in intervals:
        counts[start] += 1
        counts[end] -= 1                    # exclusive end: freed at `end`
    current = best = 0
    for delta in counts:
        current += delta
        best = max(best, current)
    return best
```

`O(n + horizon)` rather than `O(n log n)`, and for ten thousand meetings across 1,440 minutes it is faster and simpler than sorting events.

Reaching for the sweep when the coordinate space is small is over-engineering, and noticing that is worth a sentence in an interview.

The skyline: the same sweep, a different accumulator

```
import heapq

def skyline(buildings: list[tuple[int, int, int]]) → list[tuple[int, int]]:
    """
    (left, right, height) → the outline.

    THE SAME SWEEP. The accumulator is a MAX rather than a sum, so
    it needs a heap with lazy deletion instead of an integer.
    """
    events = sorted([(l, -h, r) for l, r, h in buildings] +
                    [(r, 0, 0) for _, r, _ in buildings])
    result: list[tuple[int, int]] = []
    live = [(0, float("inf"))]          # (-height, end): a ground plane
    for x, negative_height, right in events:
        while live[0][1] ≤ x:           # LAZY DELETION: drop the expired
            heapq.heappop(live)
        if negative_height:
            heapq.heappush(live, (negative_height, right))
        current = -live[0][0]
        if not result or result[-1][1] ≠ current:
            result.append((x, current))
    return result
```

`while live[0][1] ≤ x` is lazy deletion, and it is the trick: a heap cannot remove an arbitrary element, so instead you leave expired entries in and discard them when they surface. Each entry is popped at most once, so the total cost stays $O(n \log n)$.

`(l, -h, r)` sorts taller buildings first at the same x , which is what makes the outline correct when several begin together.

And the framing is the useful part: this is the meeting-rooms sweep with `max` instead of `+`. Seeing that makes a Hard problem approachable.

The complete solution

```
"""The sweep line: break intervals into events, discard which is which."""
import heapq
import random

def min_rooms(intervals: list[tuple[int, int]]) → int:
    """(time, delta) sorting puts -1 before +1: a room freed at 4 is reusable."""
    events = sorted([(s, +1) for s, _ in intervals] +
                    [(e, -1) for _, e in intervals])
    current = best = 0
    for _, delta in events:
        current += delta
        best = max(best, current)
    return best

def max_concurrent_inclusive(intervals: list[tuple[int, int]]) → int:
    """The OTHER convention: both endpoints count at the same instant."""
    events = sorted([(s, 0, +1) for s, _ in intervals] +
                    [(e, 1, -1) for _, e in intervals])
    current = best = 0
    for _, _, delta in events:
        current += delta
        best = max(best, current)
    return best

def min_rooms_two_pointer(intervals: list[tuple[int, int]]) → int:
    """The same algorithm; the tie-break is visible as '<'."""
    starts = sorted(s for s, _ in intervals)
    ends = sorted(e for _, e in intervals)
    rooms = best = 0
    i = j = 0
    while i < len(starts):
        if starts[i] < ends[j]:
            rooms += 1
            best = max(best, rooms)
            i += 1
        else:
            rooms -= 1
            j += 1
    return best

def busiest_moment(intervals: list[tuple[int, int]]) → tuple[int, int]:
    """> records the FIRST moment the peak is reached; ≥ the last."""
    events = sorted([(s, +1) for s, _ in intervals] +
                    [(e, -1) for _, e in intervals])
    current = best = at = 0
    for time, delta in events:
        current += delta
        if current > best:
            best, at = current, time
    return best, at

def total_covered(intervals: list[tuple[int, int]]) → int:
```

```

"""Accumulate BEFORE applying the delta: the stretch had the old count."""
events = sorted([(s, +1) for s, _ in intervals] +
                [(e, -1) for _, e in intervals])
current = total = previous_time = 0
for time, delta in events:
    if current > 0:
        total += time - previous_time
        current += delta
        previous_time = time
return total

def peak_weighted(items: list[tuple[int, int, int]]) → int:
    """(start, end, weight). Only the magnitude of the delta changes."""
    events = sorted([(s, +w) for s, _, w in items] +
                    [(e, -w) for _, e, w in items])
    current = best = 0
    for _, delta in events:
        current += delta
        best = max(best, current)
    return best

def assign_rooms(intervals: list[tuple[int, int]]) → list[int]:
    """The heap ASSIGNS what the sweep only COUNTS."""
    order = sorted(range(len(intervals)), key=lambda i: intervals[i][0])
    free: list[tuple[int, int]] = []
    assignment = [0] * len(intervals)
    rooms_used = 0
    for i in order:
        start, end = intervals[i]
        if free and free[0][0] ≤ start:
            _, room = heapq.heappop(free)
        else:
            rooms_used += 1
            room = rooms_used
        assignment[i] = room
        heapq.heappush(free, (end, room))
    return assignment

def min_rooms_dense(intervals: list[tuple[int, int]], horizon: int = 1440) → int:
    """Small coordinate space: a difference array beats sorting events."""
    counts = [0] * (horizon + 2)
    for start, end in intervals:
        counts[start] += 1
        counts[end] -= 1
    current = best = 0
    for delta in counts:
        current += delta
        best = max(best, current)
    return best

def skyline(buildings: list[tuple[int, int, int]]) → list[tuple[int, int]]:
    """The same sweep with MAX as the accumulator: a heap with lazy deletion."""
    events = sorted([(l, -h, r) for l, r, h in buildings] +
                    [(r, 0, 0) for _, r, _ in buildings])
    result: list[tuple[int, int]] = []
    live: list[tuple[float, float]] = [(0, float("inf"))]
    for x, negative_height, right in events:

```

```

        while live[0][1] ≤ x:                                # lazy deletion
            heapq.heappop(live)
        if negative_height:
            heapq.heappush(live, (negative_height, right))
        current = int(-live[0][0])
        if not result or result[-1][1] ≠ current:
            result.append((x, current))
    return result

def brute_max_concurrent(intervals: list[tuple[int, int]]) → int:
    """Check every interval's start as a candidate instant. Obviously correct."""
    best = 0
    for s, _ in intervals:
        overlapping = sum(1 for a, b in intervals if a ≤ s < b)
        best = max(best, overlapping)
    return best

if __name__ == "__main__":
    random.seed(0)
    meetings = [(1, 4), (2, 5), (3, 6), (7, 9)]
    print("THE SWEEP")
    print(" meetings          :", meetings)
    print(" rooms needed      :", min_rooms(meetings))
    print(" two-pointer agrees:", min_rooms_two_pointer(meetings))
    print(" busiest moment      :", busiest_moment(meetings))
    print(" assignment           :", assign_rooms(meetings))

    print("\nTHE TIE-BREAK – meetings that abut")
    abutting = [(1, 4), (4, 6), (6, 9)]
    print(" meetings          :", abutting)
    print(" end first (rooms)    :", min_rooms(abutting))
    print(" start first (inclusive):", max_concurrent_inclusive(abutting))

    print("\nTHE VARIANTS")
    print(" total covered       :", total_covered(meetings))
    streams = [(0, 10, 100), (5, 15, 250), (12, 20, 400)]
    print(" peak bandwidth      :", peak_weighted(streams), "mbps")
    print(" dense array agrees:", min_rooms_dense(meetings, horizon=20))

    print("\nTHE SKYLINE – the same sweep, MAX instead of sum")
    print(" buildings :", [(2, 9, 10), (3, 7, 15), (5, 12, 12),
                    (15, 20, 10), (19, 24, 8)])
    print(" outline   :", skyline([(2, 9, 10), (3, 7, 15), (5, 12, 12),
                           (15, 20, 10), (19, 24, 8)]))

    print("\nVERIFICATION")
    mismatches = 0
    for _ in range(500):
        n = random.randint(1, 10)
        ivs = [(lambda s: (s, s + random.randint(1, 8)))(random.randint(0, 15))
               for _ in range(n)]
        if min_rooms(ivs) ≠ brute_max_concurrent(ivs):
            mismatches += 1
    print(f" sweep vs brute force: {mismatches} mismatches in 500 cases")

```

Run it and you get:

```

THE SWEEP
meetings      : [(1, 4), (2, 5), (3, 6), (7, 9)]
rooms needed  : 3
two-pointer agrees: 3
busiest moment : (3, 3)
assignment    : [1, 2, 3, 1]

THE TIE-BREAK – meetings that abut
meetings      : [(1, 4), (4, 6), (6, 9)]
end first (rooms) : 1
start first (inclusive): 2

THE VARIANTS
total covered   : 7
peak bandwidth  : 650 mbps
dense array agrees: 3

THE SKYLINE – the same sweep, MAX instead of sum
buildings : [(2, 9, 10), (3, 7, 15), (5, 12, 12), (15, 20, 10), (19, 24, 8)]
outline   : [(2, 10), (3, 15), (7, 12), (12, 0), (15, 10), (20, 8), (24, 0)]

VERIFICATION
sweep vs brute force: 0 mismatches in 500 cases

```

`end first : 1` against `start first : 2` on three abutting meetings is the tie-break, made visible — **one comparison, and the answer doubles**. And abutting meetings are what a real calendar looks like.

And the skyline output is the same sweep with a different accumulator, which is the framing that makes a Hard problem tractable: **sort events, walk, accumulate** — **and only the accumulator changed**.

6 What it costs

The sweep.

```

build 2n events    0(n)
sort them          0(n log n)    ← dominates
one pass           0(n)
space              0(n) for the events

TOTAL: 0(n log n) time, 0(n) space

```

The two-pointer form, for comparison:

```
sort two arrays of n: 2 x  $O(n \log n)$   
against sorting 2n:  $O(2n \log 2n)$ 
```

→ $2n \log n$ against $2n \log n + 2n$
→ marginally better, same complexity, under 10% in practice

Pick whichever reads more clearly. The difference is not real.

Concretely:

```
n = 1,000          instant  
n = 100,000        sort 200,000 events ~0.2 s in Python  
n = 10,000,000     sort 20,000,000 events ~25 s
```

→ the sweep scales exactly as well as sorting, because it IS
sorting plus a linear pass

The difference array, when coordinates are dense:

```
sweep:           $O(n \log n)$   
difference array:  $O(n + \text{horizon})$ 
```

```
n = 10,000 meetings, horizon = 1,440 minutes:  
sweep: 20,000 events sorted ~200,000 operations  
array: 10,000 updates + 1,440 scan ~11,440 operations
```

→ ~18x faster AND simpler

```
and it inverts when the horizon is large:  
n = 100 meetings over a year in seconds (horizon 31,536,000):  
sweep: 200 events → trivial  
array: 31.5M entries → 250 MB and pointless
```

The crossover is roughly $\text{horizon} < n \log n$, and knowing which side you are on is worth ten seconds of arithmetic.

The heap version:

```
sort by start       $O(n \log n)$   
n heap pushes/pops  $O(n \log k)$ ,  $k = \text{rooms}$   
→  $O(n \log n)$  overall
```

space: $O(k)$, the number of rooms – often far smaller than n

→ same complexity as the sweep, and it produces the ASSIGNMENT
→ so the sweep's only advantage is simplicity and $O(1)$ working state

The skyline:

2n events sorted $O(n \log n)$
each pushed once, popped at most once (LAZY DELETION)
→ $O(n \log n)$ total

the lazy deletion is what keeps it $O(n \log n)$:

a heap cannot remove an arbitrary element, so expired entries stay in and are discarded when they surface

→ the heap can hold up to n entries, but each is popped once

without lazy deletion you would need a structure supporting arbitrary removal – a balanced BST or an indexed heap – which is the same complexity and considerably more code.

Space, across the variants:

sweep, counting only	$O(n)$ events, $O(1)$ working state
two-pointer	$O(n)$ for two arrays, $O(1)$ working
difference array	$O(\text{horizon})$
heap assignment	$O(k)$ where $k = \text{rooms}$
skyline	$O(n)$ for the heap in the worst case

Compared with the naive approach:

"maximum concurrent" by checking every instant against every interval:
 $O(n^2)$

$n = 100,000$: 10^{10} against 3.4×10^6 → $\sim 3,000\times$

and the naive version also has to CHOOSE the instants to check, which is only correct if you check every interval's start – a subtlety that the sweep removes entirely.

And the framework property, in cost terms:

changing the accumulator costs NOTHING asymptotically:

count	→ an integer	$O(1)$ per event
weighted	→ an integer	$O(1)$ per event
maximum	→ a heap	$O(\log n)$ per event
→ the last one adds a log factor and stays $O(n \log n)$ because the sort already costs that		

7 The traps

The tie-break, backwards.

```
>>> min_rooms([(1, 4), (4, 6), (6, 9)])
1
>>> max_concurrent_inclusive([(1, 4), (4, 6), (6, 9)])
2
```

One against two, on three meetings that abut. Both are correct for different questions, and the meeting-rooms answer is one — the room is freed at four and immediately reused.

And abutting meetings are what a real calendar looks like, so this fails on the most realistic input there is, by exactly one.

Encoding events as strings.

```
>>> sorted([(4, "start"), (4, "end")])
[(4, 'end'), (4, 'start')]
```

It happens to give the meeting-rooms convention, because "end" sorts before "start". That is a coincidence of the alphabet. Rename them to "begin" and "finish" and it silently reverses. Use numbers, where the ordering is intentional.

Forgetting that the sweep discards the assignment.

```
>>> min_rooms([(1, 4), (2, 5), (3, 6), (7, 9)])
3
>>> # "which room is the 7-9 meeting in?"
>>> # the sweep cannot say. It threw that away – which is what
>>> # made it cheap.
```

If the question asks which meeting goes where, the counter is not enough, and the heap is the answer. Reading the question for “how many” against “which” decides the algorithm.

Accumulating covered time in the wrong order.

```
>>> # if you apply the delta BEFORE adding the stretch:
>>> # the stretch between the previous event and this one gets
>>> # counted at the NEW count, not the old one
>>> # → the first and last stretches are wrong
```

The stretch that just ended had the *previous* count, so add it first and then apply the delta. It is one line’s ordering and it produces an answer that is close and wrong.

Using a difference array when the horizon is huge.


```
>>> counts = [0] * (31_536_000 + 2)    # a year in seconds
>>> # 250 MB for 100 meetings
```

No error, and it is absurd. The array version is right when the coordinate space is small and dense — minutes in a day — and wrong otherwise. Compare `horizon` against `n log n` before choosing.

Comparing floating-point times.

```
>>> events = sorted([(0.1 + 0.2, +1), (0.3, -1)])
>>> events
[(0.3, -1), (0.30000000000000004, 1)]
```

`0.1 + 0.2` is not `0.3`, so two events that should be simultaneous are ordered — and the tie-break you carefully designed does not apply, because there is no tie. Work in integers — minutes, seconds, milliseconds — not in floating-point hours.

Mismatched conventions between the sweep and the heap.

```
>>> # sweep uses (time, delta) → end before start → 1 room
>>> # heap uses `free[0][0] < start` (strict) → does NOT reuse at 4
>>> #                                     → 2 rooms
>>> # the two now DISAGREE, and both look right in isolation
```

If you implement both, they must use the same convention, and checking that they agree is a free consistency test worth running.

The skyline without lazy deletion.

```
>>> # trying to remove a specific building from a heap when it ends:
>>> # heapq has no such operation
>>> # → people write a linear scan-and-remove, which is O(n) per event
>>> # → O(n^2) overall
```

Lazy deletion — leave expired entries in and discard them when they surface — keeps it `O(n log n)`, because each entry is popped at most once.

Not checking whether an array beats the sort.

```
10,000 meetings, minutes in a day:
sweep: ~200,000 operations
array: ~11,400 operations

→ 18x, and the array version is fewer lines
```

Reaching for the sweep reflexively when the coordinates are small is over-engineering, and noticing it is a real signal.

8 In the interview

How it gets asked

- “How many meeting rooms do you need?” — LeetCode 253, the canonical form.
- “Do these meetings conflict?” — LeetCode 252, the easy opener.
- “What was the busiest minute?” — the variant.
- “Given buildings, produce the skyline.” — LeetCode 218, the hard version.
- “Which meeting goes in which room?” — the follow-up the sweep cannot answer.
- “Does `[2, 4]` conflict with `[4, 6]`?” — checking whether you ask.

“This sounds like a scheduling problem and it is a counting problem, **and noticing that is the whole technique.**

I do not need to know which meeting is in which room. I need to know how many are happening at the busiest moment. Those are different questions and the second one is much easier.

So: break every meeting into two independent events. Plus one at its start, minus one at its end. **Throw away which meeting they came from.** Sort every event by time, walk through with a running total, **and the maximum it reaches is the number of rooms.**

Six lines, and the discarding is the idea — once the events are independent there is no assignment problem left, only a list of numbers and a running sum.

The detail that matters is the tie-break, and I would settle it before writing. A meeting ends at four and another starts at four: **for rooms that is one room, because the first group leaves and the second arrives.** So at equal times, **process the end before the start** — the counter goes down before it goes up and never reaches two.

In Python that comes free by sorting `(time, delta)` **tuples**, because minus one sorts before plus one.

And getting it backwards gives an answer exactly one too high — on meetings that abut on the hour, which is what every real calendar looks like.

I would not encode the events as strings. `"end"` happens to sort before `"start"`, **which is a coincidence of the alphabet**, and renaming them silently reverses the convention.

Cost: `O(n log n)` **dominated by sorting the events,** `O(n)` **space,** `O(1)` **working state.**

There is an equivalent two-pointer form — sort the starts and ends separately and merge them — **which makes the tie-break visible as a** `<` **rather than hiding it in a sort key.** Same answer, same complexity.

And one question before I code: does the answer need to say which room each meeting is in? Because the sweep threw that away — **that is exactly what made it cheap** — and if you want the assignment I would use a min-heap of room end times instead.”

The follow-ups

“Why does the tie-break matter, and which way should it go?”

“Because it changes the answer by exactly one, on the most realistic input there is.

The situation: a meeting ends at four and another starts at four.

For meeting rooms, that is one room. The first group walks out, the second walks in, and the room was never occupied by both. **So the end event must be processed first** — the counter drops to zero and then rises to one, and never reaches two.

Process the start first and the counter goes one, two, one — and the maximum is two, **so the answer says two rooms where one is enough.**

And this fails on abutting meetings, which is what a real calendar is made of — nine to ten, ten to eleven, eleven to twelve. **So the wrong convention is wrong on almost every real input, by exactly one.**

The other convention is right for a different question. If I ask ‘how many people were present at time four’ with inclusive endpoints, **both the person leaving and the person arriving are present at that instant**, and the start should go first.

So the problem decides it, and I would ask if it is not stated — and say which I assumed either way.

On implementation: in Python, sorting `(time, delta)` tuples gives the meeting-rooms convention for free, because minus one sorts before plus one. **If I need the other convention I would add an explicit priority field** — zero for starts, one for ends — **rather than relying on the sign.**

And I would specifically not encode the events as strings. `(4, "end")` sorts before `(4, "start")` because `"e"` comes before `"s"` — **which gives the right answer for the wrong reason**, and renaming them to `"begin"` and `"finish"` silently reverses it.

One more: work in integers. Floating-point times break the tie-break entirely, because `0.1 + 0.2` is not `0.3` — **two events that should be simultaneous get ordered, and the tie-break you designed never applies.**“

“Now tell me which room each meeting is in.”

“Then the sweep cannot help, and the reason is the interesting part: it discarded exactly that information, and that is what made it cheap.

The sweep breaks meetings into independent events and throws away which meeting each event came from. That is why it is six lines and constant working state — and it is why it can only ever answer ‘how many’.

For the assignment I would use a min-heap of room end times.

Process the meetings in start order. Keep a heap of `(end_time, room_number)` for the rooms currently in use. For each meeting, look at the room that frees earliest: if it is free by now, reuse it; otherwise open a new room. Then push this meeting’s end time with whichever room it got.

The heap’s maximum size is the number of rooms, which agrees with the sweep’s answer — and that agreement is a free consistency check I would actually run, because if the two disagree it is almost always because their tie-break conventions differ.

The condition is `free[0][0] ≤ start` — a room freed at four is reusable at four — and it must match the sweep’s convention exactly.

Cost: $O(n \log n)$ for the sort plus $O(n \log k)$ for the heap operations, where k is the number of rooms. Same complexity as the sweep, and $O(k)$ space rather than $O(n)$, which is often much smaller.

So the honest comparison is: the heap gives strictly more information at the same complexity. The sweep’s only advantages are that it is simpler and has constant working state — and if the question only asks for a count, simpler is worth something.

One product note: ‘reuse the earliest-freeing room’ produces a valid assignment and not a nice one. Meetings hop between rooms in a way that looks arbitrary. If the requirement is that a series should stay in one room, that is a different and harder problem, and worth asking about rather than assuming.”

“Given buildings with heights, produce the skyline.”

“This is the same sweep with one thing changed, and saying that is what makes a Hard problem approachable rather than a new algorithm to learn.

The framework is identical: break each building into two events — one where it begins and one where it ends — sort them by position, and walk through maintaining an accumulator.

What changes is the accumulator. For meeting rooms it is a running sum, and an integer suffices. Here the outline at any point is the *maximum* height among the buildings currently active — so I need a structure that can give me the maximum, add a height, and remove a height.

A max-heap gives me the maximum and insertion cheaply, and it cannot remove an arbitrary element — which is the problem, because when a building ends I need to take its height out.

The answer is lazy deletion. Store `(height, right_edge)` in the heap. When I look at the top, first discard anything whose right edge is already behind me. Expired entries stay in the heap until they surface and are then dropped.

That keeps it $O(n \log n)$, because every entry is pushed once and popped at most once — even though the heap can temporarily hold entries that are no longer relevant.

Without lazy deletion the usual mistake is a linear scan to remove the element, which is $O(n)$ per event and $O(n^2)$ overall.

Two details. Sort so that taller buildings come first at the same position — encoding the height as negative does that — so that when several buildings start together the outline jumps straight to the tallest. And emit a point only when the current maximum actually changes, or the output is full of duplicate heights.

A ground plane in the heap — height zero with an infinite right edge — removes the empty-heap special case, which is a small thing that removes a whole branch.

The framing I would give first, though, is the one sentence: it is the meeting-rooms sweep with `max` instead of `+`.

The model answer

“You run a car park with a single entrance. You have a log of every vehicle’s arrival and departure time for the last month. Tell me how many spaces you actually needed, when the peak was, and how much of the day the car park was completely empty.”

“Three questions, and **all three are one sweep** — which is worth saying up front, because it means I write one pass and read three answers out of it.

The key move is that I do not care which car is which. The log pairs arrivals with departures, **and for all three questions I can throw that pairing away.** What I need is a list of times, each marked as an arrival or a departure.

So: plus one at every arrival, minus one at every departure, sorted by time, one pass with a running total.

Question one — how many spaces — is the maximum the running total reaches.

Question two — when the peak was — is one extra variable: record the time whenever the running total sets a new maximum. **And I would use a strict greater-than so it records the *first* moment the peak occurred,** which is what ‘when were we busiest’ usually means. **Greater-or-equal would report the last, which is a defensible different answer** — worth stating rather than choosing silently.

Question three — how long it was empty — is the sum of the stretches where the running total is zero. And the ordering matters: **the stretch between the previous event and this one had the *previous* count,** so I accumulate before applying the delta. **Getting that backwards gives an answer that is close and wrong.**

The tie-break, which I would settle explicitly. A car leaves at ten past two and another arrives at ten past two: **is that one space or two?** For a car park, one — **the space is freed and taken.** So at equal times, **process the departure first,** which sorting `(time, delta)` tuples gives me free because minus one sorts before plus one.

And I would ask about this rather than assume, because if the log records to the second, exact ties are rare and it barely matters; **if it records to the minute, ties are constant and the convention changes the answer.**

Now the sizing, which changes the choice of algorithm. A month of a busy car park might be a hundred thousand vehicles — two hundred thousand events, sorted in a fraction of a second. **The sweep is comfortable.**

But if the times are recorded to the minute, there are only 43,200 minutes in a month — and a difference array over those minutes is `O(n + 43,200)` rather than `O(n log n)`, and it is simpler. Increment at each arrival minute,

decrement at each departure minute, one prefix sum. **At a hundred thousand vehicles that is about eighteen times fewer operations.**

So I would ask what resolution the timestamps have, because dense small coordinates make the array version strictly better — **and reaching for the sweep reflexively there would be over-engineering.**

Two things I would raise.

Work in integers. If the log has times as floating-point hours, **two events that should be simultaneous get ordered by rounding error and the tie-break never applies.** Convert to minutes or seconds on the way in.

And if they later ask which space each car was in — for a numbered car park — **the sweep cannot answer, because it discarded exactly that.** That is a min-heap of space-free times: reuse the earliest-freeing space if it is free, otherwise open a new one. **Same complexity, and the heap's size agrees with the sweep's answer**, which is a consistency check worth running once.”

9 Recall card

The sweep is a COUNTING technique, and the idea is to discard which interval is which. Break each into `+1` at the start and `-1` at the end, throw away the association, sort by time, and take the running maximum. Six lines. “How many at once” and “which one is which” are different questions, and the first is far easier.

The tie-break is where half of implementations are wrong. For meeting rooms, an end at time 4 must be processed before a start at 4 — the room is freed and reused, giving one room, not two. Sorting `(time, delta)` tuples gives this for free because `-1 < +1`. Getting it backwards is exactly one too high on abutting meetings, which is what every real calendar looks like. Never encode events as strings — `"end" < "start"` is a coincidence of the alphabet — and work in integers, because floating-point times destroy the tie entirely.

Two equivalent forms: the event list (tie-break in the sort key) and two pointers over sorted starts and ends (tie-break visible as `<` against `≤`). Same answer, same complexity.

The sweep cannot give the ASSIGNMENT — it discarded it, which is what made it cheap. For “which room”, use a min-heap of end times: reuse the earliest-freeing room if free, else open a new one. Same $O(n \log n)$, $O(k)$ space, and the heap’s

size agrees with the sweep — a free consistency check (they disagree only when their conventions differ).

It is a framework: sort events, walk, accumulate — only the ACCUMULATOR changes. Count $\rightarrow$ integer; weighted (bandwidth) $\rightarrow +w / -w$; total covered $\rightarrow$ sum the stretches while the count is positive (accumulate BEFORE applying the delta); the SKYLINE $\rightarrow$ max instead of $+$, so the accumulator becomes a heap with LAZY DELETION (leave expired entries in, discard them when they surface — each popped once, so still $O(n \log n)$).

And when the coordinate space is small and dense, a difference array beats the sweep — $O(n + \text{horizon})$ against $O(n \log n)$, $\sim 18\times$ faster for 10,000 meetings over 1,440 minutes. Compare horizon against $n \log n$ before choosing.

Design an ad click aggregator

1 What this is, and why they ask it

An ad click aggregator counts events — impressions and clicks — and answers questions about them: **how many clicks did this campaign get in the last hour, broken down by country?**

They ask it because **it is the canonical streaming-aggregation problem, and it is the one design in this set where counting is the entire product.** Everything else here stores things and retrieves them. **Here the raw events are almost worthless individually and the aggregates are what people pay for.**

Three things make it hard, and they are all about counting rather than about scale.

The volume is enormous and the payload is tiny. A million events a second at a hundred bytes each. **That is a different shape from anything else this month** — it is not a storage problem, it is a *throughput and aggregation* problem.

And the money makes correctness matter in an unusual way. Advertisers pay **per click**, so an overcount is fraud and an undercount is lost revenue. **But the events arrive at-least-once**, so duplicates are guaranteed by the transport — **and reconciling “exactly-once counting” with “at-least-once delivery” is the design.**

Then late data, which is the part people underestimate. A phone goes into a tunnel and reports its clicks twenty minutes later. **Which hour do they belong to — the hour they happened, or the hour they arrived?** The answer decides whether your reports are correct or merely available, **and every stream-processing system has a name for this problem because it is unavoidable.**

And the fourth thing, which is the interesting inversion: the dashboard queries are not over raw events at all. Nobody scans a trillion rows. **The aggregates are computed as the data arrives and stored pre-summed** — which is yesterday’s sweep-line idea applied to a stream: **process events in order, accumulate, and never look back.**

By the end of this lesson you can design the ingestion path, aggregate in a stream with windows, handle late and duplicate events, serve the queries, and reconcile the fast path against a batch recomputation.

2 The story

The mill weighed lorries in and out and Sarojini had run the weighbridge for nineteen years, and the whole of her job was one number per lorry.

Weight in, minus weight out. That was the cane.

And the number mattered in a way nothing else at the mill mattered, **because the farmers were paid on it.**

Her first system was a register with every lorry in it. Registration number, time, weight in, weight out. Eleven hundred lorries a day in the season. **And when the office wanted to know how much cane had come in from the Nandigama villages that week, somebody sat down with the register and added it up, and it took two hours and it was wrong about a third of the time.**

So she started keeping the totals as she went.

A board on the wall with the villages down the side. **A lorry weighs out, she does the subtraction, and she adds it to that village's line immediately.** The register still recorded every lorry — **the totals were not instead of it, they were as well as it** — but the question the office asked was already answered before they asked it.

Two hours became a glance.

And then two things went wrong, in the second season, and both took her a while.

The first was the lorry that came back. A driver would weigh in, go off to the yard, and weigh out four hours later — **and sometimes the paperwork for a lorry from the morning arrived at her window in the evening,** because it had gone round the wrong way.

Which village's line did it go on? The one for the day it arrived, or the day it was weighed? **She had been adding it to today, and the Nandigama total for Tuesday was wrong for a week before anybody noticed.**

The second was worse and it took a season to find. A clerk, trying to be helpful, wrote a lorry into her board when it weighed in **and again when it weighed out.** Every lorry that clerk handled was counted twice.

And she could not tell from the board. The board only had totals. **The only way she found it was by adding up the register, by hand, for one day** — and comparing.

After that she did that every Sunday, for the whole of the rest of her career.

“The board is for answering. The register is for being right,” she told the man who replaced her. “They should agree. When they do not, the register is what happened.”

3 The idea in plain English

Sarojini’s board is the streaming aggregate, her register is the raw event log, and her Sunday is reconciliation. **Those three things are the whole architecture.**

Start with the shape, because it is unlike everything else this month.

EVENTS	AGGREGATES
1,000,000/second	a few thousand queries/second
~100 bytes each	"clicks for campaign X, last hour,
individually worthless	by country"
~100 GB/day	tiny
at-least-once	must be exactly right – money

The raw events are a firehose of tiny, individually meaningless records. The product is the sums, and nobody ever queries a single click.

Which means the central design move is: aggregate on the way in, not on the way out.

Nobody scans a trillion rows to answer “clicks last hour”. The counts are computed as the events arrive and stored already summed — and a query is a lookup, not a scan.

That is yesterday’s sweep line applied to a stream: process events in time order, accumulate into a running total, and never look back at individual events.

Now the pipeline, which is five stages.

1. INGEST	a tiny endpoint that does almost nothing
2. QUEUE	Kafka, partitioned – the durable buffer
3. AGGREGATE	stream processing into time windows
4. STORE	pre-summed counts, by dimension and window
5. SERVE	the dashboard, which does lookups

Stage one must be as small as possible. The click endpoint’s only job is **to write the event to the queue and return** — no validation beyond the minimum, no enrichment, no database write. **It sits on the path of every ad interaction on the internet,** so its latency and availability are the product’s.

Stage two is the durable buffer, and it is what makes everything downstream restartable. **If the aggregator falls over, the events are still in Kafka** and it resumes from its offset. **Without the queue, a crash loses whatever was in flight** — and those are clicks somebody is being charged for.

Partition by campaign rather than randomly, **so all the events for one campaign land on one partition and one aggregator instance**, which makes the aggregation local and removes cross-instance coordination entirely.

Stage three is the interesting one: windowed aggregation.

“How many clicks in the last hour” needs the events bucketed by time. A tumbling window — fixed, non-overlapping, one-minute buckets — is the usual choice, **and one-minute granularity is fine enough that hourly, daily and monthly totals are all sums over minute buckets.**

And that is why minute buckets are the right unit: they are small enough to answer any window a dashboard asks for and large enough that there are not too many of them.

Now the two hard problems: event time against processing time, and duplicates.

Event time is when the click happened. Processing time is when your system saw it. They differ by milliseconds usually and by **twenty minutes** for a phone that went into a tunnel.

Bucketing by processing time is easy and wrong: Tuesday’s report includes clicks from Monday, and Monday’s report changes if you rerun it. **Bucketing by event time is correct and means a window is never truly closed** — a late event can always arrive for an hour you already reported.

The standard resolution is a watermark: a moving estimate of “we have probably seen everything up to time T”. **Windows are closed when the watermark passes them, with a grace period** — typically minutes — **and anything later goes to a separate late-data path** rather than being silently dropped or silently incorporated.

And the honest framing is that there is no correct answer, only a stated one. Wait longer and the numbers are more complete and less timely; wait less and the reverse. Naming the trade-off is what is being tested.

Then duplicates, which are guaranteed rather than possible.

Kafka is at-least-once. Network retries happen. A client that does not get an acknowledgement resends. So an event will be delivered twice, and each duplicate is a click somebody pays for.

The answer is a client-generated event id and deduplication in a window. The client — the ad SDK — assigns an id when the click happens; **the aggregator keeps recently-seen ids and drops repeats.**

And the window is the compromise. Keeping every id forever is impossible at a million a second. **Keeping an hour of them is about 3.6 billion ids** — which is where a Bloom filter or a sketch becomes attractive, **with the honest cost that a false positive drops a real click.**

Then stage four: what to store.

Not the raw events, for querying. Pre-aggregated counts, keyed by the dimensions people actually filter on: **campaign, country, device, minute.**

And the dimensionality is the thing that explodes. Every extra dimension multiplies the number of rows — **campaign × country × device × ad-slot × minute is a very large number** — so **you store the combinations people actually query and not the full cross-product.**

Stage five is a lookup, and it is why the whole design exists: a dashboard asking for “campaign 4471, today, by country” **reads a few hundred pre-summed rows** rather than scanning a hundred billion events.

And finally: the two paths, which is Sarojini’s board and register.

The streaming aggregate is fast and approximately right. It has deduplicated within a window, closed windows on a watermark, and dropped some late data.

The raw event log is slow and exactly right. It is every event, kept.

So: recompute the aggregates from the raw log periodically — nightly — and reconcile. The batch numbers replace the streaming ones for closed periods. **The stream serves the last few hours; the batch serves everything older.**

That is the lambda architecture, and it is much criticised for the duplicated logic — **and for billing data it remains the honest answer,** because a fast approximate number and a slow exact one are genuinely different products and you need both.

4 The picture

The shape, which is unlike the rest of this month:

EVENTS IN

1,000,000/second
~100 bytes each
individually meaningless
~100 GB/day raw

QUERIES OUT

~1,000/second
"campaign 4471, last hour,
by country"
~200 rows

- a THROUGHPUT problem on the way in
- a LOOKUP problem on the way out
- and NOBODY EVER SCANS THE RAW EVENTS to answer a query

The pipeline:

```
ad SDK
| click event (id, campaign, country, device, timestamp)
v
[ INGEST ]  does almost nothing: write to the queue, return 202
|
v
[ KAFKA ]   partitioned BY CAMPAIGN
|           → all of one campaign's events on one partition
|           → aggregation is local, no cross-instance coordination
v
[ AGGREGATOR ] dedupe → bucket by EVENT TIME → accumulate
|
+-----+
v               v
[ PRE-SUMMED COUNTS ]  [ RAW EVENT LOG ]
fast, approximate      slow, exact
serves the last hours  serves everything older
|                       |
|                       v
|                       [ NIGHTLY BATCH ] → overwrite closed periods
v
[ DASHBOARD ]  a lookup, never a scan

"The board is for answering. The register is for being right."
```

Event time against processing time:

a phone goes into a tunnel at 14:58, clicks at 14:59, surfaces at 15:20

BUCKET BY PROCESSING TIME (when we saw it) EASY AND WRONG
the click lands in the 15:20 bucket
→ the 14:59 report is missing a click, forever
→ and RERUNNING the pipeline gives DIFFERENT numbers,
because processing time depends on when you ran it

BUCKET BY EVENT TIME (when it happened) CORRECT AND HARD
the click belongs to 14:59
→ but 14:59 was reported 20 minutes ago
→ so a window is NEVER truly closed

THE RESOLUTION: a WATERMARK
a moving estimate of "we have probably seen everything up to T"
close windows when the watermark passes them, plus a grace period
anything later → a separate LATE-DATA path, corrected explicitly

THERE IS NO CORRECT ANSWER, ONLY A STATED ONE:
wait longer → more complete, less timely
wait less → more timely, less complete

Duplicates, which are guaranteed:

the client does not get an ack and RESENDS
Kafka delivers at-least-once
the aggregator crashes after counting and before committing its offset

→ THE SAME CLICK IS COUNTED TWICE
→ and each duplicate is money an advertiser is charged

THE FIX: a CLIENT-GENERATED event id, deduplicated in a window

seen = a set of recent ids
if event.id in seen: drop
else: count it, and remember the id

AND THE WINDOW IS THE COMPROMISE:
 $1,000,000 \text{ events/second} \times 3,600 \text{ s} = 3.6 \text{ BILLION ids per hour}$
as 16-byte ids: 58 GB → too much to keep exactly
as a Bloom filter at 1%: ~4 GB → and 1% of real clicks DROPPED

→ so: exact dedup over a SHORT window (a few minutes),
plus batch dedup over the raw log for the authoritative number

Dimensional explosion, which is where storage goes:

dimensions and their cardinalities

campaign	100,000
country	200
device	5
ad slot	50
minute	1,440/day

FULL CROSS-PRODUCT:

$100,000 \times 200 \times 5 \times 50 \times 1,440 = 7.2 \times 10^{12}$ rows/day

→ IMPOSSIBLE, and almost all of it is zero

WHAT YOU ACTUALLY STORE:

- only combinations that OCCURRED (sparse)
- only combinations people QUERY (a handful of dimension sets)

campaign x minute	~100M rows/day
campaign x country x minute	~500M rows/day
campaign x device x minute	~10M rows/day

→ ~600M rows/day, not 7×10^{12} .

The cross-product is a trap; sparsity and query-driven rollups are the answer.

The two paths, and why both exist:

STREAMING

latency: seconds
deduped over minutes
windows closed on a watermark
late data partly dropped
→ FAST and APPROXIMATE

SERVE: the last few hours
the batch OVERWRITES the streaming numbers for closed periods

BATCH

latency: hours
deduped over everything
every event, in event-time order
late data included
→ SLOW and EXACT

Criticised as "lambda architecture" for duplicating the logic.
FOR BILLING DATA IT IS STILL THE HONEST ANSWER, because
"fast and roughly right" and "slow and exactly right" are
genuinely different products and you need both.

5 How it actually works

Ingestion: do almost nothing

```
@app.post("/click")
def record_click(payload: dict) → tuple[str, int]:
    """On the path of every ad interaction. Write and return."""
    event = {
        "id": payload["event_id"],          # CLIENT-generated
        "campaign": payload["campaign_id"],
        "country": geo_of(request.remote_addr),
        "device": payload.get("device", "unknown"),
        "event_time": payload["timestamp"],  # WHEN IT HAPPENED
        "received_at": time.time(),         # when we saw it
    }
    producer.send("clicks", key=str(event["campaign"]), value=event)
    return "", 202
```

`202`, not `200` — accepted for processing, not processed. **And no database write, no validation beyond parsing, no enrichment that needs a lookup**, because this end-point sits in front of every ad on the internet and its latency is the product's.

`key=campaign` is the partitioning decision: all of a campaign's events go to one partition, so one aggregator instance owns that campaign and no cross-instance coordination is needed.

And both timestamps are carried. `event_time` is what the aggregation uses; `received_at` is what tells you how late the data was, **which is the metric you need to choose the watermark**.

Deduplication

```
DEDUP_WINDOW = 300

def is_duplicate(event_id: str) → bool:
    """Exact, over a short window. Long windows need a sketch."""
    return not redis.set(f"seen:{event_id}", "1", nx=True, ex=DEDUP_WINDOW)
```

`SET NX` is atomic, so two aggregator instances racing on the same id cannot both proceed — a check-then-write would let both through.

Five minutes is the compromise. It covers retries and short outages; it does not cover a phone that surfaces after twenty minutes, and that gap is closed by the batch layer rather than by a bigger window.

Because the arithmetic is unforgiving: an hour of ids at a million a second is 3.6 billion, and even a Bloom filter at one percent drops one percent of real clicks — which is revenue.

Windowed aggregation

```
from collections import defaultdict

WINDOW_SECONDS = 60
GRACE_SECONDS = 120

class Aggregator:
    def __init__(self) → None:
        self.windows: dict[int, dict[tuple, int]] = defaultdict(
            lambda: defaultdict(int))
        self.watermark = 0.0

    def consume(self, event: dict) → None:
        if is_duplicate(event["id"]):
            return

        bucket = int(event["event_time"]) // WINDOW_SECONDS # BY EVENT TIME
        self.watermark = max(self.watermark,
                             event["event_time"] - GRACE_SECONDS)

        if bucket * WINDOW_SECONDS < self.watermark - GRACE_SECONDS:
            late_events.send(event) # NOT dropped
            return

        for key in rollup_keys(event):
            self.windows[bucket][key] += 1

    def flush_closed(self) → None:
        """Emit windows the watermark has passed."""
        for bucket in sorted(self.windows):
            if bucket * WINDOW_SECONDS + WINDOW_SECONDS < self.watermark:
                store.write(bucket, self.windows.pop(bucket))
```

`event["event_time"] // WINDOW_SECONDS` is the whole correctness decision — bucketing by when it happened, not by when it arrived. **The alternative is one character shorter and produces reports that change when you rerun them.**

The watermark is derived from the events themselves — the maximum event time seen, minus a grace period — **so it advances with the data rather than with the clock**, which is what makes it work when a partition is delayed.

And `late_events.send(event)` **rather than a silent drop** is the honest handling: late data goes somewhere it can be counted later, and its volume is a metric worth alerting on.

The rollups people actually query

```
def rollup_keys(event: dict) → list[tuple]:
    """Only the combinations people query – NOT the full cross-product."""
    c, country, device = event["campaign"], event["country"], event["device"]
    return [
        ("campaign", c),
        ("campaign_country", c, country),
        ("campaign_device", c, device),
        ("country", country),
    ]
```

Four rollups rather than the cross-product, which would be seven trillion rows a day. **The list is driven by the dashboard's filters**, and adding a new one means backfilling from the raw log — **which is a real cost and the reason to think about this in advance.**

Storage

```
def write_window(bucket: int, counts: dict[tuple, int]) → None:
    """Pre-summed, keyed by dimension and minute. A query is a lookup."""
    rows = [{"bucket": bucket, "dimension": key[0], "values": key[1:],
            "count": count}
            for key, count in counts.items()]
    store.bulk_upsert(rows)  # upsert: the batch layer overwrites

def query(dimension: str, values: tuple, start: float, end: float) → int:
    """Sum the minute buckets in the range. A few hundred rows, not billions."""
    lo, hi = int(start) // WINDOW_SECONDS, int(end) // WINDOW_SECONDS
    return store.sum_counts(dimension, values, lo, hi)
```

bulk_upsert rather than insert is what lets the batch layer overwrite the streaming numbers for a closed period **without a delete-then-insert race.**

And an hour's query is sixty minute-rows, a day's is 1,440 — **which is why minute granularity is the right unit: fine enough for any window, coarse enough that the sums stay small.**

The batch reconciliation

```
def reconcile(day: str) → dict:
    """Sarojini's Sunday. The register is what happened."""
    exact = spark.sql(f"""
        SELECT dimension, values, bucket, COUNT(DISTINCT event_id) AS count
        FROM raw_events
        WHERE event_date = '{day}'
        GROUP BY dimension, values, bucket
    """)
    streaming = store.read_day(day)

    report = {"matched": 0, "differ": []}
    for row in exact:
        streamed = streaming.get((row.dimension, row.values, row.bucket), 0)
        if streamed == row.count:
            report["matched"] += 1
        else:
            report["differ"].append((row, streamed))
    store.overwrite_day(day, exact)      # the batch number wins
    return report
```

COUNT(DISTINCT event_id) is the batch deduplication, and it is over the whole day rather than a five-minute window — **which is exactly the gap the streaming layer could not close.**

And the streaming numbers are overwritten rather than corrected, because the batch layer's answer is authoritative. **The comparison is kept as a report,** because a growing discrepancy is the earliest signal that something in the fast path is broken.

Fraud filtering, which is not optional

```
def looks_fraudulent(event: dict) → str | None:
    if click_rate(event["ip"], window=60) > 20:
        return "ip_rate"
    if event["event_time"] - impression_time(event["impression_id"]) < 0.1:
        return "too_fast"          # clicked before it rendered
    if not impression_exists(event["impression_id"]):
        return "no_impression"     # a click with no ad shown
    return None
```

A click that arrives less than a hundred milliseconds after the impression is not a human, and **a click with no matching impression is not a click at all** — those two rules catch a large fraction of simple automation.

And the flagged events are stored separately rather than discarded, because advertisers dispute charges and you must be able to show what was excluded and why.

Approximate counting, where exactness is not needed

```
def unique_users(campaign: int, bucket: int) → int:
    """HyperLogLog: ~1.6% error in 12 KB, against gigabytes for a set."""
    return redis.pfcount(f"hll:{campaign}:{bucket}")

def record_unique(campaign: int, bucket: int, user_id: str) → None:
    redis.pfadd(f"hll:{campaign}:{bucket}", user_id)
```

Unique users is a cardinality question, not a count — and an exact set of a hundred million user ids is gigabytes per campaign per hour.

HyperLogLog gives about 1.6% error in twelve kilobytes, which is completely acceptable for reach and completely unacceptable for billing. Approximate the reporting metrics; count the billable ones exactly.

The real systems

Kafka	the durable buffer; partition by campaign
Flink / Spark Streaming	stream processing with event-time windows and watermarks — both have first-class support for exactly this problem
Druid / ClickHouse	the pre-aggregated store; both designed for time-series rollups with dimensional filters
Redis	short-window deduplication, HyperLogLog
S3 + Spark	the raw event log and the nightly batch
Kappa architecture	the alternative to lambda: one code path, replay the log to recompute

Naming Flink's watermarks specifically is worth doing, because **event-time windowing with watermarks is the thing the framework exists to provide** — and knowing that it is a solved problem with a name is better than describing it as if it were novel.

6 The numbers

Volume.

```
1,000,000 events/second (impressions and clicks)
= 86,400,000,000 events/day
```

```
each event ~100 bytes
= 8.6 TB/day of raw events
= 3.1 PB/year
```

```
compressed (~5:1, they are highly repetitive): ~1.7 TB/day
```

Eight terabytes a day of hundred-byte records is the shape: **enormous count, trivial size each.**

Ingestion.

```
1,000,000 requests/second at peak
each doing: parse + one Kafka produce
```

```
per instance: ~20,000 requests/second (network-bound)
→ 50 instances at peak
```

```
Kafka: ~1,000,000 messages/second across the cluster
      at ~100 bytes = 100 MB/s
      → ~20 brokers, comfortably
```

Fifty machines for a million requests a second — and the endpoint is small precisely so that this number stays small.

Aggregation.

```
1,000,000 events/second, 4 rollups each = 4,000,000 counter updates/second
```

```
in-memory, per instance: ~500,000 updates/second
→ ~8 aggregator instances
```

```
and because Kafka is partitioned BY CAMPAIGN, each instance owns a
disjoint set of campaigns
→ no cross-instance coordination at all
```

Partitioning by campaign is what removes the coordination, and it is worth stating as the reason rather than as a detail.

Storage: the raw log against the aggregates.

RAW EVENTS

8.6 TB/day compressed to ~1.7 TB
x 90 days retention = ~155 TB
on object storage at \$0.023/GB = ~\$3,600/month

AGGREGATES

~600,000,000 rows/day (sparse, query-driven rollups)
each ~50 bytes = 30 GB/day
x 400 days = 12 TB
→ 13x SMALLER than 90 days of raw events, and it answers every query

And the cross-product, for contrast:

campaign x country x device x slot x minute
= 100,000 x 200 x 5 x 50 x 1,440 = 7.2×10^{12} rows/day

at 50 bytes: 360 TB/day

→ against 30 GB/day for the sparse, query-driven rollups
→ 12,000x, and almost every one of those rows would be ZERO

Twelve thousand times, entirely from storing what occurred rather than what could occur — which is the single most important sizing insight in this design.

Deduplication memory.

1,000,000 ids/second

5-minute exact window:	300,000,000 ids	
	x ~50 B in Redis = 15 GB	feasible
1-hour exact window:	3,600,000,000 ids	
	x ~50 B = 180 GB	not feasible
1-hour Bloom filter, 1%:	~4 GB	
	→ and 1% of REAL clicks dropped	
	→ 1% of revenue. NOT acceptable.	

→ exact dedup over minutes in the stream
→ COUNT(DISTINCT) over the whole day in the batch layer

The Bloom filter arithmetic is the argument: a structure that is fine for a crawler is unacceptable here, because the false positives are money.

Query cost.

```
"campaign 4471, last hour, by country"
60 minute-buckets x ~50 countries = ~3,000 pre-summed rows
→ ~10 ms

the same question over RAW events:
1 hour x 1,000,000/second = 3,600,000,000 events to scan
→ minutes, on a cluster

→ ~5 orders of magnitude, which is why the aggregation exists
```

Late data, measured.

```
typical distribution of (received_at - event_time):

< 1 second      ~95%
1-10 seconds    ~4%
10-60 seconds   ~0.9%
> 60 seconds    ~0.1%      ← 1,000,000 events/day

with a 2-minute grace period: ~99.98% included in the window
→ ~0.02% goes to the late path = ~17,000,000 events/day

→ which is why the late path must EXIST and be counted,
   not silently dropped
```

Seventeen million late events a day is not an edge case, and it is the number that justifies the batch layer.

Cost, roughly:

ingestion fleet (50-80 machines)	~\$80,000/month
Kafka cluster (20 brokers)	~\$40,000/month
aggregation (10 machines)	~\$15,000/month
aggregate store (Druid/ClickHouse)	~\$50,000/month
raw log on object storage	~\$4,000/month
nightly batch (Spark)	~\$30,000/month

	~\$220,000/month

→ for a system whose entire output is sums.
And it is justified because those sums ARE the billing.

7 The trade-offs

Event time against processing time. Processing time is trivial to implement and **makes reports non-reproducible** — rerun the pipeline and the numbers change, because they depend on when you ran it. Event time is correct and **means no window**

is ever truly closed. For billing there is no choice: the number must be the same tomorrow as today.

Watermark grace period. Longer means more complete numbers and more delay before a window can be reported. Shorter means timely numbers and more late data. **There is no correct value — only a stated one**, and the right way to choose it is to measure the actual distribution of lateness and pick a percentile.

Exact deduplication against a sketch. Exact over five minutes costs fifteen gigabytes and misses the twenty-minute stragglers. **A Bloom filter over an hour is four gigabytes and drops one percent of real clicks** — which is one percent of revenue, so it is unacceptable here even though the same structure was fine for a web crawler. The resolution is a short exact window plus batch deduplication.

Which rollups to precompute. The full cross-product is seven trillion rows a day and almost entirely zero. **Query-driven rollups are thirty gigabytes** — twelve thousand times smaller. **The cost is that a new query dimension requires a backfill from the raw log**, which takes hours, so the set of dimensions is a decision with real inertia and worth getting roughly right in advance.

Lambda against kappa. Lambda — a fast approximate path and a slow exact one — duplicates the aggregation logic in two places, which is the standard criticism and is fair. Kappa keeps one code path and recomputes by replaying the log, which is elegant and means the fast path's approximations are the only numbers you have until a replay completes. For billing, lambda is still the honest answer, because a fast rough number and a slow exact one are genuinely different products.

Approximate metrics against exact ones. HyperLogLog gives unique-user counts within about 1.6% in twelve kilobytes rather than gigabytes. **That is right for reach and reporting and wrong for anything billable. Approximate what is reported; count exactly what is charged** — and say which is which on the dashboard.

And the honest one: aggregation is lossy by construction. Once you have stored “4,412 clicks for campaign X in minute Y”, **you cannot answer a question you did not anticipate** without going back to the raw log. **That is why the raw log is kept**, and it is why deleting it to save four thousand dollars a month would be a false economy.

When would I not build this? Below a few thousand events a second, a database with an index on (campaign, timestamp) and a GROUP BY is the whole system, and every component here is overhead. **Managed analytics — BigQuery, Snowflake, Redshift — will ingest and aggregate at billions of rows a day and cost less**

than the team. **Building this is justified by the latency requirement on the fast path** — advertisers want near-real-time spend — **and by the volume**, not by the aggregation, which is the easy part.

8 In the interview

How it gets asked

- *“Design an ad click aggregator.”* — usually with a scale like a million events a second.
- *“How do you handle events that arrive late?”* — the event-time question, and the core of it.
- *“An event is delivered twice. What happens?”* — deduplication, and the money.
- *“How do you answer ‘clicks in the last hour by country’ quickly?”*
- *“How do you know your numbers are right?”* — reconciliation.
- *“What if I want a breakdown you did not precompute?”*

“The shape of this is unlike anything else, so let me start there. **A million events a second at a hundred bytes each, and a few thousand queries a second.** The raw events are individually worthless — **the product is the sums.**

Which means the central move is: aggregate on the way in, not on the way out. Nobody scans a trillion rows to answer ‘clicks last hour’. **The counts are computed as the events arrive and stored pre-summed, and a query is a lookup.**

The pipeline is five stages. A tiny ingest endpoint that writes to a queue and returns. **Kafka, partitioned by campaign** — so one instance owns each campaign and there is no cross-instance coordination. A stream aggregator. A pre-summed store. And a dashboard that does lookups.

The ingest endpoint must do almost nothing — no database write, no enrichment — **because it sits in front of every ad interaction and its latency is the product’s.**

Now the two things that are actually hard, and both are about counting rather than scale.

First, event time against processing time. A phone goes into a tunnel and reports its clicks twenty minutes later. **Bucketing by when we saw it is easy and wrong** — the numbers change if you rerun the pipeline. **Bucketing by when it happened is correct and means a window is never truly closed.**

The resolution is a watermark: a moving estimate of ‘we have probably seen everything up to T’, closing windows with a grace period, **and sending anything later to an explicit late-data path rather than silently dropping it.** And I would be honest that **there is no correct grace period, only a stated one** — longer is more complete and less timely.

Second, duplicates, which are guaranteed rather than possible. Kafka is at-least-once, clients retry, and **every duplicate is money an advertiser is charged.** So: a client-generated event id and deduplication in a window.

And the window is the compromise, because the arithmetic is unforgiving. An hour of ids at a million a second is 3.6 billion — **and a Bloom filter at one percent would drop one percent of real clicks, which is one percent of revenue.** So: exact dedup over a few minutes in the stream, and `COUNT(DISTINCT)` over the whole day in a batch layer.

Which brings me to the last piece: two paths. The stream is fast and approximate; **a nightly batch over the raw log is slow and exact and overwrites the closed periods.** For billing data that duplication is the honest answer.”

The follow-ups

“How do you handle events that arrive late?”

“This is the question that decides whether the reports are correct or merely available, **so let me be precise about what ‘late’ means first.**

Every event has two timestamps: when it happened, and when we saw it. They differ by milliseconds normally, **and by twenty minutes for a phone that went into a tunnel and surfaced.**

Bucketing by processing time — when we saw it — is one character easier and it is wrong. The click gets counted in the fifteen-twenty bucket instead of fourteen-fifty-nine, **and worse, rerunning the pipeline gives different numbers,** because processing time depends on when you happened to run it. **For billing, non-reproducible numbers are not acceptable.**

So: bucket by event time. And the consequence is that **a window is never truly closed** — a late event can always arrive for an hour you already reported.

The standard resolution is a watermark, and it is what stream processors like Flink exist to provide. **A watermark is a moving estimate of ‘we have probably seen everything up to time T’** — typically the maximum event time seen so far, minus a grace period. **It advances with the data rather than with the clock,** which matters when a partition is delayed.

Windows are closed when the watermark passes them. And **anything arriving after that goes to an explicit late-data path** — not dropped silently, and not silently folded into a report somebody has already seen.

The grace period is a genuine trade with no correct answer: wait longer and the numbers are more complete and less timely; wait less and the reverse. **The right way to pick it is to measure the actual distribution of lateness.** In practice about ninety-five percent of events arrive within a second and **about a tenth of a percent take more than a minute** — which at a million a second is **a million events a day,** so this is not an edge case.

And the batch layer is what makes the trade survivable. The nightly recomputation over the raw log sees every event regardless of when it arrived, **so the authoritative number is eventually right even though the streaming number was not.** The stream serves the last few hours; the batch overwrites everything older.”

“An event is delivered twice. What happens?”

“It gets counted twice, **and an advertiser is charged twice** — so this is a correctness problem with money attached rather than a hygiene issue.

And duplicates are guaranteed, not possible. Three separate sources: **a client that does not receive an acknowledgement and resends; Kafka’s at-least-once delivery; and an aggregator that crashes after counting an event and before committing its offset**, so it reprocesses from the last commit.

The answer is a client-generated event id. The ad SDK assigns an id when the click happens — **not the server on receipt**, because a retried request would get a different server-generated id and deduplicate nothing. **That is the same rule as payment idempotency keys.**

Then the aggregator keeps recently-seen ids and drops repeats, using an atomic set-if-absent — **a check-then-write would let two racing instances both through.**

The hard part is the window, and the arithmetic is unforgiving. At a million events a second, **an hour of ids is 3.6 billion.** As sixteen-byte ids with overhead that is **around 180 gigabytes** — not feasible.

And the obvious answer is a Bloom filter, which is wrong here for an interesting reason. Four gigabytes at one percent false positive — **and a false positive means dropping a real click. One percent of revenue.** The same structure was completely fine for a web crawler, where a false positive meant skipping a page nobody would miss. **Here the false positives are money, so it is unacceptable.**

So the resolution is two-layered. Exact deduplication over a short window in the stream — five minutes, about fifteen gigabytes, which covers retries and short outages. **And `COUNT(DISTINCT event_id)` over the whole day in the batch layer**, which catches the twenty-minute stragglers the short window could not.

The streaming number may be slightly high; the batch number is right and overwrites it. And I would track the difference as a metric, because a growing gap is the earliest signal that something in the fast path has broken.”

“What if I want a breakdown you did not precompute?”

“Then it is slow, and I would rather say that plainly than pretend the design covers everything — because the reason it is fast is precisely that it does not.

The aggregates are precomputed for specific dimension combinations — campaign by minute, campaign by country by minute, and so on. A query in that set is a lookup over a few hundred pre-summed rows: about ten milliseconds.

A combination I did not precompute has to come from the raw log, which for an hour is 3.6 billion events — minutes on a cluster, not milliseconds.

And the reason I do not precompute everything is the cross-product. Campaign times country times device times ad slot times minute is about seven trillion rows a day, and almost every one of them is zero. Against thirty gigabytes a day for the sparse, query-driven rollups — twelve thousand times smaller.

So the design decision is: store what occurred, in the combinations people query. Not the full space.

Which means the set of dimensions has real inertia, and adding one requires backfilling from the raw log — hours of batch work. So I would want the likely query patterns established early, and I would treat “which breakdowns do advertisers actually look at” as a product question to be answered before building rather than after.

Two things that soften it. A slow ad-hoc path is fine as a feature, as long as it is honest — a query that says ‘this will take a few minutes’ is better than one that quietly times out. And keeping the raw log is what makes any of this possible, which is why deleting it to save a few thousand dollars a month would be a false economy — the aggregates are lossy by construction, and the log is the only thing that can answer a question you did not anticipate.

That is Sarojini’s point: the board is for answering, and the register is for being right.”

The model answer

“Design a system that counts ad impressions and clicks: a million events a second, and advertisers see their spend updated within a minute.”

“Two constraints in that sentence drive everything: **a million events a second**, and **within a minute** — so this needs a fast path, and the money means it also needs a correct one.

The shape first. A million hundred-byte events a second is **8.6 terabytes a day**, against a few thousand queries. **The raw events are individually worthless; the product is the sums.** So the central move is **aggregate on the way in, not on the way out** — nobody scans a trillion rows.

Ingestion does almost nothing: parse, write to Kafka, return 202. **No database write, no enrichment**, because it sits in front of every ad interaction on the internet. **Fifty machines at a million requests a second**, and it is small so that number stays small.

Kafka partitioned by campaign, so all of a campaign’s events land on one partition and one aggregator owns it — **no cross-instance coordination at all**. About twenty brokers for a hundred megabytes a second.

Aggregation into one-minute tumbling windows, bucketed by EVENT time. Minute granularity because hourly, daily and monthly are all sums over minute buckets — **fine enough for any window a dashboard asks for**.

And bucketing by event time rather than arrival time is the correctness decision. Arrival time is easier and **makes the numbers change when you rerun the pipeline**, which for billing is not acceptable.

Windows close on a watermark with a grace period of a couple of minutes, which covers about 99.98% of events. **The remaining tenth of a percent — a million events a day — goes to an explicit late path**, not a silent drop.

Deduplication with a client-generated event id, exact over a five-minute window in Redis. **And I would explicitly reject a Bloom filter here:** four gigabytes for an hour at one percent false positive sounds attractive, **but a false positive drops a real click, which is one percent of revenue**. The same structure was fine for a crawler; here the errors are money.

Storage: query-driven rollups, not the cross-product. The full space is seven trillion rows a day and almost all zero; **the four or five combinations people actually filter on are about six hundred million rows a day, thirty gigabytes. Twelve thousand times smaller**, entirely from storing what occurred.

And the two paths, which is the part I would insist on. The stream is fast and approximate — deduplicated over minutes, windows closed on a watermark, some late data missed. **A nightly batch over the raw log is slow and**

exact, deduplicating with `COUNT(DISTINCT)` over the whole day, and it overwrites the streaming numbers for closed periods.

Advertisers see the streaming number within a minute, which satisfies the requirement. The invoice uses the batch number. And the difference between them is a metric I would alert on, because a growing gap is the first sign the fast path is broken.

Two things I would raise unprompted.

Fraud filtering is not optional. A click arriving less than a hundred milliseconds after its impression is not a human; a click with no matching impression is not a click. And flagged events must be stored separately rather than discarded, because advertisers dispute charges and you have to show what was excluded and why.

And approximate where it is not billable. Unique reach via HyperLogLog is 1.6% error in twelve kilobytes against gigabytes for an exact set — **completely fine for a reach report and completely wrong for a charge.** Approximate what is reported; count exactly what is billed — and label which is which on the dashboard, because an advertiser who discovers the distinction themselves will not be pleased.”

9 Recall card

The shape is unlike the rest: a million tiny events a second in, a few thousand queries out, and the raw events are individually worthless. So the central move is **aggregate on the way IN, not on the way out** — a query becomes a lookup over pre-summed rows, never a scan.

Five stages: a tiny ingest endpoint (write to Kafka, return `202`, no DB write), Kafka **PARTITIONED BY CAMPAIGN** (so one instance owns each campaign — no cross-instance coordination), stream aggregation into one-minute windows, a pre-summed store, and a dashboard that does lookups.

Event time versus processing time is the correctness decision. Processing time is easier and **makes reports change when you rerun them.** Event time is correct and means **no window is ever truly closed** — resolved by a **watermark** (max event time seen minus a grace period, advancing with the data, not the clock) plus an **explicit late-data path**. ~0.1% arrive over a minute late — a million events a day, not an edge case. There is no correct grace period, only a stated one.

Duplicates are GUARANTEED (client retries, Kafka at-least-once, aggregator restarts) **and each one is money**. **Client-generated event id**, atomic set-if-absent, exact over ~ 5 minutes (~ 15 GB). **Explicitly reject a Bloom filter**: 4 GB for an hour at 1% sounds good, **but a false positive drops a real click — 1% of revenue**. The same structure was fine for a crawler because there the errors were free.

Never store the cross-product: campaign $\times$ country $\times$ device $\times$ slot $\times$ minute is 7×10^{12} rows/day, **almost all zero**, against ~ 30 GB/day for sparse query-driven rol-lups — $12,000 \times$. The cost is that a new dimension needs a backfill, so the dimension set has real inertia.

Two paths, deliberately. Stream = fast and approximate; **nightly batch over the raw log = slow and exact** (`COUNT(DISTINCT)` over the whole day) and it **OVERWRITES closed periods**. Advertisers see the streaming number in a minute; **the invoice uses the batch number, and the gap between them is the alert**. Approximate what is reported (HyperLogLog, 1.6% in 12 KB), count exactly what is billed.

Practice — Meeting rooms and the sweep line

DSA topic: Meeting rooms and the sweep line **System design topic:** Design an ad click aggregator

Code these, in this order

One rule for the whole set: **state the tie-break convention before you write the sort**. Does an event ending at time `t` conflict with one starting at `t`? It is one comparison, the problem decides it, **and getting it backwards gives an answer exactly one too high on the most realistic inputs there are**.

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Meeting Rooms	LeetCode 252 (Easy)	The opener — and whether you sort at all.
2	Meeting Rooms II	LeetCode 253 (Medium)	The sweep, and the tie-break.
3	Car Pooling	LeetCode 1094 (Medium)	A weighted sweep, and dense coordinates.
4	My Calendar III	LeetCode 732 (Hard)	A sweep over a stream of arrivals.
5	The Skyline Problem	LeetCode 218 (Hard)	The same sweep with <code>max</code> as the accumulator.
6	Minimum Interval to Include Each Query	LeetCode 1851 (Hard)	A sweep over two sorted streams at once.

On problem 2, get the tie-break wrong on purpose

Encode the events so that starts sort before ends at equal times. Run on `[(1,4), (4,6), (6,9)]` and record the answer against 1. **Then run the correct version**. Say why abutting meetings are the normal case rather than an edge case.

On problem 2, encode the events as strings

Use `(time, "start")` and `(time, "end")`. Confirm it gives the right answer. **Then rename them to "begin" and "finish" and run it again**. Say what happened and why numbers are the right encoding.

On problem 2, produce the assignment

Extend your solution to say which meeting is in which room. **Say why the sweep alone cannot do this**, and check that the heap's size equals the sweep's answer. **If they disagree, find which convention differs.**

On problem 3, notice the coordinates are dense

The problem caps locations at 1,000. **Write both the sweep and a difference array.** Time both at $n = 10,000$ trips and record the ratio. **Say roughly where the crossover between the two approaches is.**

On problem 4, notice it is a stream

You cannot sort — bookings arrive one at a time and each must be answered immediately. **Say what changes**, and implement it with a running count in a sorted map. **Then say what the per-insert cost is.**

On problem 5, start from the meeting-rooms code

Take your sweep and change only the accumulator. **Write down what the accumulator must support** — add, remove, report the maximum — and say why a plain heap does not support all three.

Then implement lazy deletion. **Say why it stays $O(n \log n)$ despite the heap holding stale entries.**

On problem 6, find the two streams

There are intervals and there are queries, both sortable. **Say what you sweep over and what the accumulator is.** Then implement it.

Then the floating-point drill

Build a set of intervals with times like $0.1 + 0.2$ and 0.3 . Run your sweep. **Record what happens to the tie-break** and say why integers are the only safe representation.

Then the verification drill

Write a brute force that, for every interval's start, counts how many intervals contain that instant. Check your sweep against it on five hundred random inputs. **Say why checking only at interval starts is sufficient.**

The decomposition drill

1. State the technique in one sentence.
2. Say what you deliberately discard, and what that buys.

3. Say what question the sweep answers and what it cannot.
4. Give the complexity and what dominates it.

The tie-break drill

1. Give both conventions and the question each answers.
2. Say which one meeting rooms needs, and why.
3. Say what (time, delta) sorting gives you for free, and why.
4. Say why string encoding is a coincidence rather than a design.
5. Say by how much the wrong convention is wrong, and on what input.
6. Say what floating-point times do to it.

The forms drill

1. Give the event-list form.
2. Give the two-pointer form and say where the tie-break lives in it.
3. Say whether they differ in complexity.
4. Give the heap form and what it adds.
5. Say what consistency check the heap gives you free.

The variants drill

For each, say what changes and what the accumulator becomes:

1. How many rooms.
2. The busiest moment.
3. Total covered length.
4. Peak bandwidth.
5. The skyline.

The when-not-to drill

1. Give the condition under which a difference array wins.
2. Compute both costs at 10,000 meetings over 1,440 minutes.
3. Say when the array version becomes absurd.
4. Give the rough crossover rule.
5. Say what two dimensions require instead.

The break-it drill

For each, say what happens and whether anything reports it:

1. Starts before ends at equal times.
 2. Events encoded as `"begin"` and `"finish"`.
 3. Applying the delta before accumulating covered time.
 4. A difference array over a year in seconds.
 5. Floating-point times.
 6. Sweep and heap with mismatched conventions.
 7. Skyline without lazy deletion.
-

The shape drill

1. Give events in and queries out, with volumes.
2. Say what the central design move is, and why.
3. Say what nobody ever does, and why that matters.

The pipeline drill

1. Name the five stages.
2. Say what the ingest endpoint must not do, and why.
3. Say what the queue buys you.
4. Say what partitioning by campaign removes.
5. Say what status code the endpoint returns and why.

The time drill

1. Give both timestamps and what each is for.
2. Say what bucketing by arrival time gets wrong, in two ways.
3. Say what bucketing by event time costs.
4. Define a watermark and say what it advances with.
5. Say what happens to data past the grace period.
6. Give the lateness distribution and the volume at the tail.
7. State the trade honestly.

The duplicates drill

1. Name three sources of duplicates.
2. Say where the event id must come from and why.
3. Say why the dedup check must be atomic.
4. Compute exact dedup memory for five minutes and for an hour.

5. Say why a Bloom filter is right for a crawler and wrong here.
6. Give the two-layer resolution.

The storage drill

1. Compute the full cross-product and say what fraction is zero.
2. Compute the sparse query-driven total.
3. Give the ratio.
4. Say what adding a new dimension costs.
5. Say what that means for when the decision must be made.

The two-paths drill

1. Give both paths with their properties.
2. Say which serves what.
3. Say which number goes on the invoice.
4. Say what metric you would alert on.
5. Give the standard criticism and your answer to it.

The honesty drill

1. Say what aggregation is lossy about.
 2. Say why the raw log is kept.
 3. Say what you approximate and what you count exactly.
 4. Say what must be labelled on the dashboard, and why.
 5. Say why flagged fraudulent events are stored rather than dropped.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *How many meeting rooms do you need?* The decomposition and what it discards, the tie-break with its convention and its cost, the two equivalent forms, and what the sweep cannot answer.
2. *Now give me the skyline.* That it is the same sweep with a different accumulator, what the accumulator must support, lazy deletion, and why it stays $O(n \log n)$.

3. *Design an ad click aggregator.* Aggregate on the way in, event time versus processing time with watermarks, why a Bloom filter is wrong here, the cross-product trap, and the two paths.

Before you move on

- [] I can state the sweep in one sentence, including what it discards.
- [] I know “how many at once” and “which is which” are different questions.
- [] I settle the tie-break before writing the sort.
- [] I know `(time, delta)` gives the meeting-rooms convention free, and why.
- [] I never encode events as strings.
- [] I know the wrong convention is exactly one too high, on abutting inputs.
- [] I work in integers, and know what floats do to the tie-break.
- [] I can write both the event-list and two-pointer forms.
- [] I know the sweep cannot give the assignment, and what can.
- [] I know the heap’s size agrees with the sweep, and use it as a check.
- [] I can do the busiest-moment variant and know `>` versus `≥`.
- [] I can do total-covered and know to accumulate before applying the delta.
- [] I can do a weighted sweep.
- [] I know the skyline is the same sweep with `max` as the accumulator.
- [] I can explain lazy deletion and why it stays `O(n log n)`.
- [] I know when a difference array beats the sweep, with the arithmetic.
- [] I know the rough crossover rule.
- [] I can give the ad aggregator’s shape: events in, queries out.
- [] I know the central move is to aggregate on the way in.
- [] I can name the five stages and what the endpoint must not do.
- [] I know why partitioning by campaign removes coordination.
- [] I know both timestamps and what arrival-time bucketing gets wrong.
- [] I can define a watermark and say what it advances with.
- [] I know the lateness distribution and the daily volume at the tail.
- [] I can name three sources of duplicates.
- [] I know where the event id must come from.
- [] I can say why a Bloom filter is wrong here and right for a crawler.

- ☐ I can compute the cross-product against the sparse rollups.
- ☐ I know what a new dimension costs and when the decision must be made.
- ☐ I know which number goes on the invoice and what I would alert on.
- ☐ I answered all three questions above out loud.

Jump game and reachability

DATA STRUCTURES AND ALGORITHMS

Jump game and reachability

SYSTEM DESIGN

Design a distributed job scheduler

1 What this is, and why they ask it

You are at the start of an array. Each entry says how far you may jump from that position. Can you reach the end?

[2, 3, 1, 1, 4] — yes. [3, 2, 1, 0, 4] — no, because whatever you do you land on the zero and stop.

They ask it because **it is the clearest example in the course of a problem that looks like dynamic programming and is greedy**. Every instinct after a month of DP says “let `dp[i]` be whether position `i` is reachable” — **and that is $O(n^2)$ and correct, and there is an $O(n)$ one-pass answer that most people do not see.**

The insight is small and it is the whole lesson. **You do not need to know which positions are reachable. You need to know how far you can reach.** One number, updated as you walk. **That collapse — from a set of reachable positions to a single frontier — is the idea**, and it recurs in a family of problems.

The second reason is that **the follow-ups form a ladder and each rung uses the same idea differently**. Can I reach the end? What is the fewest jumps? Which positions are reachable going backwards? **And the fewest-jumps version is a BFS in disguise**, which is a genuinely nice thing to notice out loud.

And there is a specific detail that separates people: the loop must stop when the frontier falls behind. Walking past a position you cannot reach and continuing to update the frontier from it **gives an answer that is too optimistic**, and it fails only on inputs containing a zero — which many test sets do not.

By the end of this lesson you can write the one-pass reachability check, extend it to minimum jumps, explain why it is greedy and prove it, and recognise the family.

2 The story

The canal had stepping stones and Bhaskar crossed it twice a day for thirty-one years and he was the only person in the village who could tell you, standing on the bank, whether you would get across.

The stones were not evenly spaced and they were not all the same size.

A big flat one you could take a long stride from. A small round one you could barely stand on, and from that one you went to the very next stone or nowhere.

And in the monsoon the water came up and some of them went under, and the arrangement changed every year.

The way people worked it out was to start walking and find out, which was how people ended up in the canal.

Bhaskar's method took about four seconds and he had never explained it to anybody because nobody had asked.

He did not think about the stones. He thought about one number: how far along he could get.

Standing on the bank, looking at the first stone: **from there he could reach the fourth stone.** So — as far as the fourth.

Then he looked at the second stone, which was inside what he could already reach. **From the second stone you could get to the sixth.** So now — as far as the sixth.

Third stone. **A small one, you could only manage the fourth from it.** No change; he could already reach further than that.

And he went along like that, one number in his head, never more.

The thing that stopped him was always the same. He would reach a stone — say the seventh — **and find that the furthest he could get was also the seventh.**

Which meant the eighth was unreachable, and it did not matter what was beyond it.

His nephew, who had watched him do this and had tried to copy it, said he found it hard because he kept trying to remember which stones were good.

“You are remembering the wrong thing,” Bhaskar said. “It does not matter which stones. It matters how far. One number. And when the number stops moving before you do, you have your answer.”

3 The idea in plain English

Bhaskar's one number is the frontier, and his stopping rule is the loop condition. **Together they are the whole algorithm.**

Start with the naive answer, because it is what everybody writes first and it is worth knowing why it is wrong to write it.

`dp[i]` is `True` if position `i` is reachable. For each reachable `i`, mark everything from `i+1` to `i + nums[i]` as reachable. Answer: `dp[n-1]`.

Correct, $O(n^2)$, and it is remembering more than it needs.

The insight: you do not need to know *which* positions are reachable. You need to know *how far* you can reach.

Because reachability here is contiguous. If you can reach position seven, you can reach every position before it — so the set of reachable positions is always a prefix, and a prefix is described by one number.

That collapse — from a set to a frontier — is the idea, and it turns $O(n^2)$ into $O(n)$ with $O(1)$ space.

```
furthest = 0
for i, jump in enumerate(nums):
    if i > furthest:
        return False          # cannot even stand here
    furthest = max(furthest, i + jump)
return True
```

Five lines. And each one earns its place.

`if i > furthest: return False` is the stopping rule, and it is the line people omit. Without it, the loop walks past a position it cannot reach and keeps updating the frontier from it — which is Bhaskar's nephew stepping onto a stone that is not there. The answer comes out too optimistic, and it only fails on inputs containing a zero, which many test sets do not have.

`furthest = max(furthest, i + jump)` is the frontier update. Note the `max`: a small jump from a later position does not shrink what you could already reach.

And the loop ends without needing to check `furthest ≥ n-1` explicitly, because if the frontier ever falls behind, the guard catches it — so surviving the loop means you got there.

Now the ladder, and each rung is the same idea used differently.

Rung one: can I reach the end? The five lines above.

Rung two: what is the fewest jumps? And this is the one worth understanding properly, because it is a breadth-first search in disguise.

Think of it as levels. From the start, one jump reaches some range of positions — that is level one. **From anywhere in level one, another jump reaches some further range — level two.** The answer is which level contains the end.

And you do not need a queue, because the levels are contiguous ranges. **Two numbers describe the current level: where it ends, and the furthest anything in it can reach.**

```
jumps = current_end = furthest = 0
for i in range(len(nums) - 1):
    furthest = max(furthest, i + nums[i])
    if i == current_end:           # the end of this level
        jumps += 1
        current_end = furthest    # the next level ends here
return jumps
```

if i == current_end is the level boundary, and it is the line that makes this a BFS. **Reaching the end of the current level means you must take another jump,** and the new level extends as far as anything in the old one could reach.

And `range(len(nums) - 1)` — stopping one short — is deliberate. **Landing on the last position does not require a jump *from* it,** and including it counts one jump too many.

Rung three: the backwards version, which is a different way to see it.

Walk from the end. Keep the leftmost position from which you can reach the end — call it the target. **Moving right to left, if position `i` can reach the target, then `i` becomes the new target.** If the target ends up at zero, you can get there.

Same complexity, and it is the version that generalises better when the question asks *which* positions can reach the end rather than just whether the start can.

Now: why is this greedy correct? Because yesterday's lesson says you owe an argument.

The exchange argument for minimum jumps. Take any optimal sequence of jumps. **At each level, greedy jumps to whatever gives the furthest reach.** If an optimal solution jumps somewhere else, **swap greedy's choice in:** greedy's landing spot can reach at least as far, **so anything the optimal solution could do next, greedy can also do.** The jump count is unchanged. **So some optimal solution agrees with greedy's choice, and induction finishes it.**

And the key property that makes it work is contiguity again: from anywhere in a level you can reach anywhere in the range up to the frontier, so “the best landing spot” is well-defined and there is no trade-off between reach and position.

Which is exactly what breaks when the problem changes.

Rung four: where greedy stops. Jump Game II with costs — each jump costs something different — is no longer greedy, **because a jump that reaches furthest may cost more than two cheaper jumps.** That is dynamic programming.

And **Jump Game III** — you may jump left or right by a fixed amount — **is not a frontier problem at all**, because reachability is no longer contiguous. **It is a graph, and the answer is a plain BFS or DFS.**

Noticing which of those you have is the actual skill, and the test is: **is the set of reachable positions always a contiguous prefix?** If yes, one number suffices. **If not, you need the set** — and a set of positions is a graph traversal.

4 The picture

The collapse from a set to a number:

```
nums = [2, 3, 1, 1, 4]
index: 0 1 2 3 4
```

THE DP VERSION — remembers WHICH

```
dp = [T, ?, ?, ?, ?]
```

```
from 0 (jump 2): mark 1, 2      dp = [T, T, T, ?, ?]
```

```
from 1 (jump 3): mark 2, 3, 4   dp = [T, T, T, T, T]
```

```
from 2 (jump 1): mark 3
```

```
...
```

```
→  $O(n^2)$ , and it is remembering more than it needs
```

THE FRONTIER — remembers HOW FAR

```
i=0, jump 2: furthest = max(0, 0+2) = 2
```

```
i=1, jump 3: furthest = max(2, 1+3) = 4 ← reached the end
```

```
i=2, jump 1: furthest = max(4, 2+1) = 4
```

```
i=3, jump 1: furthest = max(4, 3+1) = 4
```

```
i=4      done
```

ONE NUMBER. $O(n)$ time, $O(1)$ space.

WHY IT WORKS: reachability is CONTIGUOUS. If you can reach 7, you can reach everything before it — so the reachable set is always a PREFIX, and a prefix is one number.

The stopping rule, which is the line people omit:

```
nums = [3, 2, 1, 0, 4]
index: 0 1 2 3 4
```

```
i=0, jump 3: furthest = 3
i=1, jump 2: furthest = max(3, 3) = 3
i=2, jump 1: furthest = max(3, 3) = 3
i=3, jump 0: furthest = max(3, 3) = 3
i=4:         i (4) > furthest (3) → STOP. Cannot reach here.
```

WITHOUT THE GUARD:

```
the loop reaches i=4 anyway and computes furthest = max(3, 4+4) = 8
→ returns True
→ IT STOOD ON A STONE THAT WAS NOT THERE
```

And it fails ONLY on inputs containing a zero, which many hand-written test sets do not have.

Minimum jumps as a breadth-first search:

```
nums = [2, 3, 1, 1, 4]
```

```
LEVEL 0: position 0
         reach from here: up to index 2
LEVEL 1: positions 1..2
         reach from anywhere in here: max(1+3, 2+1) = 4
LEVEL 2: positions 3..4 ← contains the end
```

→ 2 jumps

THE LEVELS ARE CONTIGUOUS RANGES, so no queue is needed –
TWO NUMBERS describe the current level:

```
current_end where this level stops
furthest     how far anything in it can reach
```

`if i == current\_end` is the level boundary, and it is the line that makes this a BFS.

Why the loop stops one short:

```
for i in range(len(nums) - 1):
    ^^^^^
```

LANDING on the last position does not require a jump FROM it.

```
nums = [2, 1]
with range(len(nums)):      i=0 → jumps=1
                           i=1 = current_end → jumps=2  WRONG
with range(len(nums)-1):    i=0 → jumps=1                CORRECT
```

One character, one extra jump, and no error.

The exchange argument:

ANY OPTIMAL SEQUENCE

at level k, it jumps to x

SWAP x FOR g:

g reaches at least as far as x, by definition of "furthest"

→ anything the optimal solution could do NEXT, greedy can also do

→ the jump count is UNCHANGED

→ some optimal solution agrees with greedy's choice

→ induct on the remainder

GREEDY

at level k, jumps to the position
with the FURTHEST reach, g

THE PROPERTY THAT MAKES IT WORK: contiguity.

From anywhere in a level you can reach anywhere up to the
frontier, so "the best landing spot" is well defined and there
is NO trade-off between reach and position.

Where the frontier stops working:

JUMP GAME (this one)

[2, 3, 1, 1, 4]

reachable set is a PREFIX

→ ONE NUMBER

→ $O(n)$ greedy

JUMP GAME WITH COSTS

each jump has a price

a further jump may cost MORE

→ reach and cost trade off

→ DYNAMIC PROGRAMMING

JUMP GAME III

reachable set is scattered

jump LEFT or RIGHT by `nums[i]`

→ not a prefix, not one number

→ A GRAPH: plain BFS or DFS

THE TEST: is the reachable set always a contiguous prefix?

yes → one number suffices

no → you need the SET, which is a graph traversal

5 The code, built step by step

Can I reach the end?

```
def can_jump(nums: list[int]) → bool:
    """One number: how far I can reach. O(n) time, O(1) space."""
    furthest = 0
    for i, jump in enumerate(nums):
        if i > furthest:
            return False                # cannot even stand here
        furthest = max(furthest, i + jump)
    return True
```

Five lines, and the guard is the one people omit. Without `if i > furthest`, the loop walks past unreachable positions and updates the frontier from them — and returns `True` for `[3,2,1,0,4]`.

max on the update matters too: a small jump from a later position **must not shrink** what was already reachable.

And no explicit `furthest ≥ n-1` check is needed at the end, because falling behind is caught by the guard — **so surviving the loop means the end is reachable**.

The naive version, for comparison

```
def can_jump_dp(nums: list[int]) → bool:
    """Correct, O(n^2), and it remembers more than it needs."""
    n = len(nums)
    reachable = [False] * n
    reachable[0] = True
    for i in range(n):
        if not reachable[i]:
            continue
        for step in range(1, nums[i] + 1):
            if i + step < n:
                reachable[i + step] = True
    return reachable[n - 1]
```

Writing both and comparing is the point of the lesson. The DP records which positions are reachable; the greedy records how far — and because reachability is contiguous, the set and the number carry the same information.

Minimum jumps

```
def min_jumps(nums: list[int]) → int:
    """A BFS over contiguous levels. Two numbers instead of a queue."""
    jumps = current_end = furthest = 0
    for i in range(len(nums) - 1):          # STOP ONE SHORT
        furthest = max(furthest, i + nums[i])
        if i == current_end:                # the end of this level
            jumps += 1
            current_end = furthest          # the next level ends here
    return jumps
```

`range(len(nums) - 1)` is deliberate: landing on the last position does not require a jump from it, and including it counts one too many.

`if i == current_end` is the level boundary, and it is what makes this a breadth-first search without a queue — the levels are contiguous ranges, so two numbers describe one.

The BFS it actually is

```
from collections import deque

def min_jumps_bfs(nums: list[int]) → int:
    """The same algorithm with an explicit queue, to see the levels."""
    n = len(nums)
    if n ≤ 1:
        return 0
    visited = [False] * n
    visited[0] = True
    queue = deque([0])
    jumps = 0
    while queue:
        jumps += 1
        for _ in range(len(queue)):          # one whole LEVEL
            i = queue.popleft()
            for j in range(i + 1, min(i + nums[i], n - 1) + 1):
                if j == n - 1:
                    return jumps
                if not visited[j]:
                    visited[j] = True
                    queue.append(j)
    return -1
```

Same answer, $O(n^2)$ in the worst case, and much clearer about what is happening. Writing it once makes the two-number version obvious — the queue always contains a contiguous range, so tracking its endpoints is enough.

The backwards version

```
def can_jump_backwards(nums: list[int]) → bool:
    """Walk right to left, tracking the leftmost position that reaches the end."""
    target = len(nums) - 1
    for i in range(len(nums) - 2, -1, -1):
        if i + nums[i] ≥ target:
            target = i          # i can reach the target
    return target == 0
```

Different reasoning, same complexity, and this is the version that generalises when the question asks *which* positions can reach the end rather than whether the start can.

Jump Game III: where the frontier does not apply

```
def can_reach_zero(nums: list[int], start: int) → bool:
    """
    Jump LEFT or RIGHT by nums[i]. Reachability is NOT contiguous,
    so there is no frontier – it is a graph, and this is a plain BFS.
    """
    n = len(nums)
    seen = [False] * n
    queue = deque([start])
    seen[start] = True
    while queue:
        i = queue.popleft()
        if nums[i] == 0:
            return True
        for j in (i + nums[i], i - nums[i]):
            if 0 ≤ j < n and not seen[j]:
                seen[j] = True
                queue.append(j)
    return False
```

No frontier anywhere, because the reachable set is scattered rather than a prefix.
Recognising that is the skill — and the test is exactly the contiguity question.

The weighted version: where greedy fails

```
def min_cost_jumps(nums: list[int], cost: list[int]) → int:
    """
    Each position has a cost to jump FROM. Greedy fails: the furthest
    jump may be the most expensive. This is DP.
    """
    n = len(nums)
    dp = [float("inf")] * n
    dp[0] = 0
    for i in range(n):
        if dp[i] == float("inf"):
            continue
        for j in range(i + 1, min(i + nums[i], n - 1) + 1):
            dp[j] = min(dp[j], dp[i] + cost[i])
    return int(dp[n - 1]) if dp[n - 1] != float("inf") else -1

def min_cost_greedy(nums: list[int], cost: list[int]) → int:
    """The greedy that breaks, for comparison."""
    total = i = 0
    n = len(nums)
    while i < n - 1:
        best = max(range(i + 1, min(i + nums[i], n - 1) + 1),
                    key=lambda j: j + nums[j])
        total += cost[i]
        i = best
    return total
```

Adding a cost breaks the contiguity argument, because **now there is a trade-off between reach and price** — the furthest landing spot may be the most expensive place to jump from. **The greedy choice is no longer safe, and the exchange argument fails at exactly that point.**

Which positions can reach the end

```
def reachable_positions(nums: list[int]) → list[bool]:
    """Every position from which the end is reachable. The backwards sweep."""
    n = len(nums)
    result = [False] * n
    result[n - 1] = True
    target = n - 1
    for i in range(n - 2, -1, -1):
        if i + nums[i] ≥ target:
            result[i] = True
            target = i
    return result
```

One pass, and it answers a strictly stronger question than `can_jump` — which is the reason the backwards formulation is worth knowing.

The complete solution

```
"""Jump game: the frontier collapse, and where it stops working."""

import random
from collections import deque

def can_jump(nums: list[int]) → bool:
    """ONE NUMBER: how far I can reach. The guard is the line people omit."""
    furthest = 0
    for i, jump in enumerate(nums):
        if i > furthest:
            return False
        furthest = max(furthest, i + jump)
    return True

def can_jump_no_guard(nums: list[int]) → bool:
    """THE BUG: no stopping rule. Fails only on inputs with a zero."""
    furthest = 0
    for i, jump in enumerate(nums):
        furthest = max(furthest, i + jump)
    return furthest ≥ len(nums) - 1

def can_jump_dp(nums: list[int]) → bool:
    """Correct,  $O(n^2)$ , and it remembers WHICH rather than HOW FAR."""
    n = len(nums)
    reachable = [False] * n
    reachable[0] = True
    for i in range(n):
        if not reachable[i]:
            continue
        for step in range(1, nums[i] + 1):
            if i + step < n:
                reachable[i + step] = True
    return reachable[n - 1]

def can_jump_backwards(nums: list[int]) → bool:
    """Right to left, tracking the leftmost position that reaches the end."""
    target = len(nums) - 1
    for i in range(len(nums) - 2, -1, -1):
        if i + nums[i] ≥ target:
            target = i
    return target == 0

def min_jumps(nums: list[int]) → int:
    """A BFS over contiguous levels. Two numbers instead of a queue."""
    jumps = current_end = furthest = 0
    for i in range(len(nums) - 1):
        furthest = max(furthest, i + nums[i])
        if i == current_end:
            jumps += 1
            current_end = furthest
    return jumps

def min_jumps_off_by_one(nums: list[int]) → int:
    """THE BUG: iterating to the end counts a jump FROM the last position."""
    jumps = current_end = furthest = 0
```



```

    for i in range(len(nums)):
        furthest = max(furthest, i + nums[i])
        if i == current_end:
            jumps += 1
            current_end = furthest
    return jumps

def min_jumps_bfs(nums: list[int]) → int:
    """The same algorithm with an explicit queue, to see the levels."""
    n = len(nums)
    if n ≤ 1:
        return 0
    visited = [False] * n
    visited[0] = True
    queue = deque([0])
    jumps = 0
    while queue:
        jumps += 1
        for _ in range(len(queue)):
            i = queue.popleft()
            for j in range(i + 1, min(i + nums[i], n - 1) + 1):
                if j == n - 1:
                    return jumps
                if not visited[j]:
                    visited[j] = True
                    queue.append(j)
    return -1

def reachable_positions(nums: list[int]) → list[bool]:
    """Every position from which the end is reachable. A stronger answer."""
    n = len(nums)
    result = [False] * n
    result[n - 1] = True
    target = n - 1
    for i in range(n - 2, -1, -1):
        if i + nums[i] ≥ target:
            result[i] = True
            target = i
    return result

def can_reach_zero(nums: list[int], start: int) → bool:
    """Jump LEFT or RIGHT: reachability is not contiguous → a plain BFS."""
    n = len(nums)
    seen = [False] * n
    queue = deque([start])
    seen[start] = True
    while queue:
        i = queue.popleft()
        if nums[i] == 0:
            return True
        for j in (i + nums[i], i - nums[i]):
            if 0 ≤ j < n and not seen[j]:
                seen[j] = True
                queue.append(j)
    return False

def min_cost_jumps(nums: list[int], cost: list[int]) → int:
    """Costs break contiguity: reach and price trade off. This is DP."""
    n = len(nums)
    dp = [float("inf")] * n

```

```

dp[0] = 0
for i in range(n):
    if dp[i] == float("inf"):
        continue
    for j in range(i + 1, min(i + nums[i], n - 1) + 1):
        dp[j] = min(dp[j], dp[i] + cost[i])
return int(dp[n - 1]) if dp[n - 1] != float("inf") else -1

def min_cost_greedy(nums: list[int], cost: list[int]) → int:
    """The greedy that breaks when costs appear."""
    total = i = 0
    n = len(nums)
    while i < n - 1:
        best = max(range(i + 1, min(i + nums[i], n - 1) + 1),
                    key=lambda j: j + nums[j])
        total += cost[i]
        i = best
    return total

if __name__ == "__main__":
    random.seed(0)

    print("CAN I REACH THE END?")
    for nums in ([2, 3, 1, 1, 4], [3, 2, 1, 0, 4], [0], [0, 1], [1, 0]):
        print(f" {str(nums):16} greedy {can_jump(nums)!s:5} "
              f"dp {can_jump_dp(nums)!s:5} backwards {can_jump_backwards(nums)}")

    print("\nTHE MISSING GUARD – fails only on inputs with a zero")
    for nums in ([3, 2, 1, 0, 4], [2, 3, 1, 1, 4]):
        print(f" {str(nums):16} correct {can_jump(nums)!s:5} "
              f"no guard {can_jump_no_guard(nums)}")

    print("\nMINIMUM JUMPS")
    for nums in ([2, 3, 1, 1, 4], [2, 3, 0, 1, 4], [1, 1, 1, 1], [0], [2, 1]):
        print(f" {str(nums):16} greedy {min_jumps(nums)} "
              f"bfs {min_jumps_bfs(nums)} "
              f"off-by-one {min_jumps_off_by_one(nums)}")

    print("\nWHICH POSITIONS CAN REACH THE END?")
    nums = [2, 3, 1, 1, 4]
    print(f" {nums} → {reachable_positions(nums)}")
    nums = [3, 2, 1, 0, 4]
    print(f" {nums} → {reachable_positions(nums)}")

    print("\nWHERE THE FRONTIER DOES NOT APPLY")
    print(" jump left or right, [4,2,3,0,3,1,2] from 5 →",
          can_reach_zero([4, 2, 3, 0, 3, 1, 2], 5))
    print(" jump left or right, [3,0,2,1,2] from 2 →",
          can_reach_zero([3, 0, 2, 1, 2], 2))

    print("\nWHERE GREEDY BREAKS – costs")
    nums, cost = [3, 1, 3, 1, 1, 1], [1, 1, 100, 1, 1, 0]
    print(f" nums {nums}, cost {cost}")
    print(f" greedy {min_cost_greedy(nums, cost)} dp {min_cost_jumps(nums, cost)}")

    print("\nVERIFICATION – greedy against the O(n^2) DP")
    mismatches = 0
    for _ in range(2000):
        n = random.randint(1, 10)
        case = [random.randint(0, 4) for _ in range(n)]

```

```

    if can_jump(case) != can_jump_dp(case):
        mismatches += 1
    print(f" {mismatches} mismatches in 2,000 random cases")

```

Run it and you get:

```

CAN I REACH THE END?
[2, 3, 1, 1, 4] greedy True dp True backwards True
[3, 2, 1, 0, 4] greedy False dp False backwards False
[0] greedy True dp True backwards True
[0, 1] greedy False dp False backwards False
[1, 0] greedy True dp True backwards True

THE MISSING GUARD – fails only on inputs with a zero
[3, 2, 1, 0, 4] correct False no guard True
[2, 3, 1, 1, 4] correct True no guard True

MINIMUM JUMPS
[2, 3, 1, 1, 4] greedy 2 bfs 2 off-by-one 3
[2, 3, 0, 1, 4] greedy 2 bfs 2 off-by-one 3
[1, 1, 1, 1] greedy 3 bfs 3 off-by-one 4
[0] greedy 0 bfs 0 off-by-one 1
[2, 1] greedy 1 bfs 1 off-by-one 1

WHICH POSITIONS CAN REACH THE END?
[2, 3, 1, 1, 4] → [True, True, True, True, True]
[3, 2, 1, 0, 4] → [False, False, False, False, True]

WHERE THE FRONTIER DOES NOT APPLY
jump left or right, [4,2,3,0,3,1,2] from 5 → True
jump left or right, [3,0,2,1,2] from 2 → False

WHERE GREEDY BREAKS – costs
nums [3, 1, 3, 1, 1, 1], cost [1, 1, 100, 1, 1, 0]
greedy 101 dp 3

VERIFICATION – greedy against the O(n^2) DP
0 mismatches in 2,000 random cases

```

`correct False` **against** `no guard True` **on** `[3,2,1,0,4]` is the missing stopping rule, made visible — **and the second line shows it agreeing on the input without a zero**, which is why the bug survives.

And `off-by-one` **giving one more on four of the five inputs** is the `range(len(nums) - 1)` decision — **one character, one extra jump, no error**. Note that `[2, 1]` agrees, **which is exactly why the bug survives**: when the last position is not itself a level boundary, the extra iteration changes nothing.

And the cost example is the clearest thing here: greedy 101 against 3. Greedy jumps to position 2 because it reaches furthest — and jumping *from* position 2 costs a hundred. **Three cheap hops down the middle cost three.**

6 What it costs

The frontier version.

```
one pass, constant work per element  
→  $O(n)$  time,  $O(1)$  space
```

```
n = 1,000,000: ~0.2 s in Python. And no allocation at all.
```

Against the naive DP:

```
for each reachable position, mark up to nums[i] positions  
worst case (every entry large):  $O(n^2)$ 
```

n = 1,000	1,000,000 operations	instant
n = 100,000	10^{10}	~3 hours
n = 1,000,000	10^{12}	not happening

```
→ and  $O(n)$  space for the reachable array
```

```
at n = 100,000 the greedy is instant and the DP does not finish.  
That is not a constant factor.
```

And the memory difference matters as much:

```
DP:      an array of n booleans →  $O(n)$   
greedy:  one integer           →  $O(1)$ 
```

```
n = 10,000,000: DP is ~80 MB of pointers; greedy is 8 bytes
```

Minimum jumps.

```
greedy (two numbers):   $O(n)$  time,  $O(1)$  space  
BFS with a queue:      $O(n^2)$  worst case,  $O(n)$  space  
                      (each position can be enqueued once, but  
                      scanning the range from each is  $O(n)$ )
```

```
→ the greedy version is the BFS with the queue collapsed into  
  two numbers, and it removes the inner scan entirely
```

The backwards version:

one pass right to left: $O(n)$ time, $O(1)$ space
→ identical cost, different reasoning

and `reachable\_positions` answers a STRICTLY STRONGER question
at the same cost, which is the argument for knowing it

Where the frontier does not apply:

JUMP GAME III (left or right)

BFS over n positions, 2 edges each

→ $O(n)$ time, $O(n)$ space for the visited array

→ still linear, but it NEEDS the set, so the $O(1)$ space
collapse is gone

WITH COSTS

DP over n positions, scanning up to n forward from each

→ $O(n^2)$ time, $O(n)$ space

→ and there is no frontier at all, because reach and cost
trade off

The verification cost:

an $O(n^2)$ DP for $n \leq 10$:	~5 minutes to write
2,000 random trials:	< 1 second

→ and it settles definitively whether the greedy is right,
which for a greedy algorithm is worth doing once

The comparison that matters:

the DP remembers WHICH positions are reachable:	n booleans
the greedy remembers HOW FAR:	one integer

and because reachability is CONTIGUOUS, they carry the
SAME INFORMATION.

→ the collapse is not an approximation. It is exact, and it
is available precisely because the reachable set is a prefix.

7 The traps

The missing stopping rule.

```
>>> can_jump([3, 2, 1, 0, 4])
False
>>> can_jump_no_guard([3, 2, 1, 0, 4])
True
```

Without `if i > furthest: return False`, the loop walks past position three — which it cannot reach — and updates the frontier from position four. It stood on a stone that was not there.

And it agrees on every input without a zero, which is why it survives review and passes hand-written tests.

The off-by-one in minimum jumps.

```
>>> min_jumps([2, 1])
1
>>> min_jumps_off_by_one([2, 1])
2
```

One too many. Landing on the final position does not require a jump from it, so the loop must stop one short. **No error, and the answer is wrong by one whenever the last index happens to be a level boundary** — which is most inputs, and not all of them, so a small test set can miss it entirely.

Shrinking the frontier.

```
>>> # furthest = i + jump      instead of  max(furthest, i + jump)
>>> # on [5, 1, 1, 1, 1, 1]:
>>> #   i=0 → furthest = 5
>>> #   i=1 → furthest = 2      ← WENT BACKWARDS
```

A small jump from a later position must not shrink what was already reachable. The `max` is not decoration — it is the statement that reachability only ever grows.

Writing the DP because it looks like DP.

```
>>> # after a month of dynamic programming, "dp[i] = is i reachable"
>>> # is the first thought, and it is O(n^2)
>>> # → correct, and it fails at n = 100,000
```

The test is contiguity: is the reachable set always a prefix? If yes, a set is more than you need. Asking that question takes ten seconds and saves the quadratic solution.

Using the frontier where reachability is not contiguous.

```
>>> # Jump Game III: jump LEFT or RIGHT by nums[i]
>>> # the reachable set is SCATTERED, not a prefix
>>> # → a single "furthest" number cannot describe it
>>> # → and a frontier-based solution gives nonsense
```

Recognising which problem you have is the actual skill. Contiguous reach means a number; scattered reach means a graph.

Assuming greedy survives a cost.

```
>>> # each jump has a price
>>> # the furthest landing spot may be the most expensive to jump from
>>> # → reach and cost trade off
>>> # → the exchange argument fails, and it is DP
```

Adding any second objective breaks contiguity, which is the same lesson as weighted interval scheduling — one word in the problem statement changes the algorithm.

A single-element array.

```
>>> can_jump([0])
True
>>> min_jumps([0])
0
```

You are already at the end, so a zero jump is irrelevant and no jumps are needed. Both are correct and both catch out solutions that assume at least one jump happens — and `min_jumps` returning zero rather than one depends entirely on the `range(len(nums) - 1)` decision.

Off-by-one in the inner range.

```
>>> # from position i with jump j, you can reach i+1 .. i+j
>>> # range(i + 1, i + nums[i] + 1)    correct
>>> # range(i + 1, i + nums[i])        one short
>>> # range(i, i + nums[i] + 1)        includes staying put
```

Three plausible ranges and only one is right. In the greedy version this does not arise, because `i + jump` is a single expression — which is a small argument for the greedy form beyond its complexity.

8 In the interview

How it gets asked

- “*Can you reach the last index?*” — LeetCode 55, the opener.
- “*What is the minimum number of jumps?*” — LeetCode 45, the follow-up.
- “*Why is greedy correct here?*” — the question that matters.
- “*Now you can jump left or right.*” — LeetCode 1306, where the frontier does not apply.
- “*Now each jump has a cost.*” — where it becomes DP.
- “*Can you do it in constant space?*” — which the frontier already does.

The first ninety seconds

“My first instinct after a month of dynamic programming is `dp[i] = is position i reachable`, and that is $O(n^2)$ and correct and I would not write it — because there is a linear answer and the reason is worth saying.

The insight is that I do not need to know *which* positions are reachable. I need to know *how far* I can reach.

And the reason one number suffices is contiguity: if I can reach position seven, I can reach every position before it. So the reachable set is always a prefix — and a prefix is described by a single number.

So: walk left to right, keeping `furthest`. At each position, if the index is past `furthest`, I cannot even stand here — return false. Otherwise extend `furthest` to the maximum of itself and `i + nums[i]`.

Five lines, $O(n)$ time, $O(1)$ space.

And the guard is the line I would call out, because it is the one people omit. Without it the loop walks past a position it cannot reach and keeps updating the frontier from it — which returns true for `[3,2,1,0,4]`. And it only fails on inputs containing a zero, so it passes most hand-written tests.

The `max` matters too: a small jump from a later position must not shrink what was already reachable.

For the minimum number of jumps, the same idea becomes a breadth-first search. From the start, one jump reaches a range — that is level one. From anywhere in level one, another jump reaches a further range. The answer is which level contains the end.

And no queue is needed, because the levels are contiguous ranges — two numbers describe one: where this level ends, and how far anything in it can reach. When I arrive at the end of the current level, I take a jump and the next level extends to the frontier.

One detail: the loop stops one short of the end, because landing on the last position does not require a jump from it — and including it counts one jump too many, on every input, with no error.

And I would offer the correctness argument unprompted, because greedy always owes one.”

The follow-ups

“Why is the greedy correct?”

“An exchange argument, and the property it depends on is worth naming because it is exactly what breaks in the variants.

Take the minimum-jumps version. At each level, greedy jumps to whatever landing spot gives the furthest subsequent reach.

Take any optimal sequence of jumps. If at some level it jumps somewhere other than greedy’s choice, swap greedy’s in.

Is that legal? Yes — both landing spots are within the current level, so both are reachable.

Is it worse? No — greedy’s spot reaches at least as far by definition, **so anything the optimal solution could do on its next jump, greedy can also do.** The jump count is unchanged.

So some optimal solution agrees with greedy’s choice, and induction on the remainder finishes it.

Now the property that makes it work, which is the important bit: contiguity. From anywhere in a level I can reach anywhere up to the frontier — so ‘the best landing spot’ is well defined, **and there is no trade-off between reaching far and being in a good position.** Further is simply better.

And that is exactly what breaks when the problem changes.

Add a cost per jump and the furthest landing spot may be the most expensive to jump from. Now reach and price trade off, **the swap can make things worse, and the exchange argument fails at precisely that step.** That is dynamic programming.

Allow jumps left as well as right and the reachable set is no longer a prefix at all — it is scattered — **so there is no frontier to be greedy about.** That is a graph, and a plain BFS.

So the test I apply is: is the set of reachable positions always a contiguous prefix? If yes, one number suffices and greedy works. If no, I need the set, which means a traversal.

And I verified it empirically once — the greedy against the $O(n^2)$ DP on a couple of thousand random small inputs — **which for a greedy algorithm is five minutes well spent.**“

“What is the minimum number of jumps?”

“The same idea, and the nice thing is that **it is a breadth-first search with the queue collapsed into two numbers** — which is worth saying, because it explains why the greedy is correct rather than merely asserting it.

Think in levels. Level zero is the start. **One jump from there reaches some contiguous range — that is level one.** From anywhere in level one, one more jump reaches a further range — level two. **The answer is which level contains the last position,** which is exactly what BFS computes.

And I do not need a queue, because every level is a contiguous range of positions. A range is described by two numbers: where this level ends, and how far anything in it can reach.

So: walk left to right. Extend `furthest` at every position. When I arrive at `current_end` — the boundary of the current level — I must take another jump, so increment the count and set `current_end` to `furthest`.

`O(n)` time, `O(1)` space, against the explicit BFS which is $O(n^2)$ in the worst case because scanning the reachable range from each position is linear.

The detail I would say out loud: the loop stops one short of the end. Landing on the final position does not require a jump from it. Including it counts one extra jump — **on every input, consistently, with no error** — which is easy to mistake for a definitional disagreement rather than a bug.

And the single-element case falls out correctly: `[0]` gives zero jumps, because you are already there.

I would also mention that this assumes the end is reachable, which the problem usually guarantees. **If it does not, the same loop can detect it** — if `current_end` stops advancing before reaching the end, there is no answer.”

“Now you can jump left or right by `nums[i]`. Does your approach still work?”

“No, and the reason is the interesting part — **it is the same property that made the greedy correct, and it is gone.**

The frontier works because reachability is contiguous: if I can reach position seven, I can reach everything before it. That is what lets a single number describe the whole reachable set.

Allow jumps in both directions and that stops being true. I might be able to reach positions three and eleven and nothing in between — **the reachable set is scattered, not a prefix.** A single ‘furthest’ number cannot describe it, and a frontier-based solution gives nonsense rather than a slightly worse answer.

So this is a graph. Each position is a node with at most two edges — `i + nums[i]` and `i - nums[i]` — **and the question is plain reachability, which is a BFS or a DFS with a visited set.**

`O(n)` time and `O(n)` space, because I now genuinely need the set. **The constant-space collapse is gone, and that is the cost of losing contiguity.**

What is worth noticing is that **the complexity did not get worse** — it is still linear — **only the space.** Because each node has a bounded number of edges, the traversal visits each position once.

And I would generalise the lesson rather than just answering the question. The test for whether a frontier applies is: **is the reachable set always a contiguous prefix?** If yes, one number. If no, a traversal.

The same test settles the cost variant, which is the other natural follow-up. Adding a price per jump keeps reachability contiguous but breaks the greedy choice, because now the furthest landing spot may be the most expensive — **so reach and cost trade off, and ‘further is simply better’ stops being true.** That one is dynamic programming, not a graph.

Three variants, three different techniques, and the question that separates them is about contiguity and trade-offs rather than about the code.“

The model answer

“A delivery drone flies along a straight route with charging pads at fixed points. At each pad, the amount of charge available tells you how many pads further you can fly before needing to land again. Given the charge at each pad, can the drone reach the last one — and if so, what is the fewest landings?”

“Two questions, and **they are the same idea used twice**, so let me establish the shape once and then answer both.

My first instinct is dynamic programming — ‘is pad i reachable’ — and I would not write it, because it is $O(n^2)$ and there is a linear answer.

The insight is that I do not need to know which pads are reachable. I need to know how far I can get.

And that works because reachability here is contiguous: if the drone can reach pad seven, it can reach every pad before it — **it can always land early. So the reachable set is always a prefix, and a prefix is one number.**

I would confirm that assumption explicitly, because it is the thing the whole approach depends on: **can the drone land at any pad within range, or must it fly the full distance its charge allows? If it must fly the maximum, reachability is not contiguous and this becomes a graph.**

Question one, assuming it can land early: one pass with a single number. Walk the pads left to right keeping `furthest`. **At each pad, if its index is beyond `furthest`, the drone cannot get here — return false.** Otherwise extend `furthest` to the maximum of itself and `i + charge[i]`.

$O(n)$ time, $O(1)$ space, and the guard is the important line — without it the loop reasons from a pad the drone never reached, **and a pad with zero charge stops it in reality and not in the code.**

Question two — the fewest landings — is the same idea as a breadth-first search over levels. From the start, one flight reaches a contiguous range of pads. From anywhere in that range, another flight reaches further. **The answer is which level contains the last pad.**

And the levels are contiguous, so two numbers describe one — where this level ends and how far anything in it can reach. **When I arrive at the level boundary, that is another landing.**

The loop stops one short of the end, because arriving at the final pad does not require taking off from it.

Both are $O(n)$ and constant space, so a route with a million pads is a fraction of a second.

Now three things about the domain that I would raise, because the textbook version hides them.

First, the charge probably is not an integer number of pads. It is a distance, and the pads are at irregular positions. **That does not change the algorithm** — the frontier becomes a distance rather than an index, and I advance through the pads while their position is within it — **but it does mean working in distances, and I would want to know the units.**

Second, wind and payload change the range, so the charge-to-distance conversion is not fixed. **The algorithm still applies with a conservative estimate,** and I would want a safety margin as an explicit parameter rather than baked into the numbers.

And third, the question that would change the algorithm: is there a cost to landing? Time, battery wear, a fee at the pad. **If landings have different costs, greedy breaks** — the furthest pad may be the most expensive place to land — **and it becomes dynamic programming. Fewest landings and cheapest route are different problems,** and I would want to know which one is actually wanted before committing to the greedy answer.”

9 Recall card

This looks like DP and is greedy. The naive `dp[i] = is position i reachable` is $O(n^2)$ and correct and **remembers more than it needs.** **The insight: you do not need to know WHICH positions are reachable, only HOW FAR you can reach** — one number.

It works because reachability is CONTIGUOUS: reach position 7 and you reach everything before it, **so the reachable set is always a prefix and a prefix is one number.** $O(n)$ time, $O(1)$ space.

The stopping rule is the line people omit: `if i > furthest: return False`. Without it the loop walks past an unreachable position **and updates the frontier from it** — returning `True` for `[3,2,1,0,4]`. **It fails only on inputs containing a zero,** which is why it survives. **And the update must be** `max(furthest, i+jump)` — reachability only ever grows.

Minimum jumps is a BFS with the queue collapsed into two numbers, because the levels are contiguous ranges: `current_end` (where this level stops) and `furthest` (how far anything in it reaches). `if i == current_end` **is the level boundary. The loop stops ONE SHORT** — landing on the last position needs no jump from it — **and including it is exactly one too many on every input, with no error.**

The exchange argument: greedy jumps to the spot with the furthest reach; swapping it into any optimal sequence is legal (both are in the current level) and no worse (it reaches at least as far), **so anything the optimum could do next, greedy can too.** The property it depends on is contiguity — “further is simply better”, with no trade-off.

The test for which technique you have: is the reachable set always a contiguous prefix? Yes → one number, greedy. Jump left OR right → scattered, not a prefix → a graph, plain BFS, and the $O(1)$ space is gone. Add a cost per jump → contiguity survives but “further is better” does not, because reach and price trade off → dynamic programming.

Design a distributed job scheduler

1 What this is, and why they ask it

A job scheduler runs work at a specified time. **“Send this email at 09:00 tomorrow.” “Run this report every night at two.” “Retry this webhook in five minutes.”**

They ask it because **it is the infrastructure every company builds and almost nobody gets right the first time**, and because the difficulties are not where people expect them.

The scheduling is easy. A sorted structure keyed by run time, and a loop that pulls whatever is due. **That part is twenty lines.**

What is hard is everything about failure. A worker picks up a job and dies halfway through — **is the job lost, or run twice?** A job runs for six hours — **how do you know it is still alive rather than stuck?** A scheduler instance is partitioned from the network but still running — **does another instance take over, and do they both then run everything?**

And the second hard thing is that “exactly once” is not available and the honest design says so. You get at-least-once with idempotent jobs, or at-most-once with the risk of loss. **Choosing which, per job, and saying why, is the interview.**

The third is a scale detail that catches people: a hundred million jobs all scheduled for 09:00. Everything in the system is fine until the thundering herd at the top of the hour, **and the fix is jitter, which is one line and is never in the first design.**

By the end of this lesson you can design the store and the polling loop, handle leases and worker death, support recurring jobs, avoid the herd, and say precisely what delivery guarantee you are offering.

2 The story

The hospital had a diary on the counter of the nurses’ station and everything that had to happen at a particular time was written in it.

Injectons at six. Dressings at ten. The dialysis lot at two.

And for eleven years the system was that whoever was on duty read the diary, **and the trouble was never the reading.**

The first trouble was the handover.

Because a nurse would look at ten o'clock, see four dressings, start the first one — **and go off shift**. And the next nurse would look at ten o'clock, see four dressings, and start the first one.

Which was done. And bed eleven got two dressings and bed fourteen got none.

The fix was a pencil tick, and it took an argument to introduce. You ticked the line **when you started**, not when you finished, **and you wrote your initials**.

Then the second trouble, which took a year.

A nurse ticked a line, started, and was called to something else and did not come back. And the tick said somebody was doing it, **so nobody else did**, and at four o'clock in the afternoon somebody noticed.

The rule they added was a time next to the tick. If your initials had been against a line for more than an hour, **anybody could rub it out and take it**.

Which caused the third trouble almost immediately, **because some things genuinely took two hours**, and the nurse doing them would come back to find somebody else halfway through.

So the tick had to be renewed. You came back to the station, and you moved the time along, **and that meant you were still on it**.

And the fourth thing was the one nobody planned for.

Because eleven years of “everything at six, everything at ten, everything at two” meant that at six o'clock there were nineteen things to do and at seven o'clock there were none.

The ward sister who fixed it did not change the medicine. She changed the diary. **She wrote things at five past, and ten past, and quarter past** — the same medicines, the same intervals, **spread across the hour**.

“It was never that we had too much work,” she said. “It was that we had written it all down at the same time.”

3 The idea in plain English

The nurses' diary is the job store, the tick is a lease, the time next to it is the lease expiry, renewing it is the heartbeat, and the ward sister's fix is jitter. **Those five things are the design.**

Start with the model, because the vocabulary matters.

JOB	what to run, and with what arguments
SCHEDULE	when: once at a time, or a repeating rule
RUN	one attempt at executing a job
LEASE	a claim on a run, held by one worker, with an expiry

A job is not a run. A daily report is one job and three hundred and sixty-five runs a year — **and conflating them is the modelling mistake that makes recurring jobs painful later.**

Now the store, which is a sorted structure keyed by time.

“What is due?” is the only query that matters, so the schema is built around it:

```
CREATE TABLE runs (
  id          BIGINT PRIMARY KEY,
  job_id      BIGINT NOT NULL,
  run_at      TIMESTAMPTZ NOT NULL,
  state       TEXT NOT NULL,          -- pending, leased, done, failed
  lease_until TIMESTAMPTZ,
  attempt     INT NOT NULL DEFAULT 0
);
CREATE INDEX ON runs (state, run_at);
```

One index on (state, run_at) and the whole scheduling problem is a range query — `WHERE state = 'pending' AND run_at ≤ now()`.

And a relational database is genuinely the right answer here for most systems, because the volume is modest and you need transactions.

Now claiming work, which is where the correctness is.

Two workers must not run the same job. A `SELECT` followed by an `UPDATE` is a race — **both see it pending, both claim it.**

The claim must be one atomic operation:

```

UPDATE runs
  SET state = 'leased', lease_until = now() + interval '60 seconds',
      worker = %(me)s, attempt = attempt + 1
  WHERE id IN (
    SELECT id FROM runs
    WHERE state = 'pending' AND run_at ≤ now()
    ORDER BY run_at
    LIMIT 10
    FOR UPDATE SKIP LOCKED          -- the important part
  )
RETURNING *;

```

FOR UPDATE SKIP LOCKED is the line that makes this scale. Without it, ten workers polling simultaneously **all block on the same rows** and the throughput is that of one worker. **With it, each worker takes rows nobody else has locked and moves on** — no coordination, no contention.

Then the lease, which is the nurses’ tick with a time on it.

A worker that dies mid-job must not hold the job forever. So the claim carries an expiry, and an expired lease means the run is available again.

And the renewal is what handles long jobs. A worker running a six-hour job extends its lease periodically — a heartbeat — so the job is not stolen while it is genuinely progressing.

Which produces the fundamental trade, and it is worth stating precisely:

SHORT LEASE	dead workers are detected fast a slow-but-alive worker gets its job STOLEN and it runs TWICE
LONG LEASE	slow workers are safe a dead worker's job sits idle for the whole lease

And there is no setting that avoids both, because you cannot distinguish a dead worker from a slow one over a network. That is the same failure-detection result that made presence hard in chat, applied to work instead of people.

So the honest answer is: heartbeat to make the lease short *and* safe, and accept that a worker partitioned from the database will have its job taken while still running it.

Which brings the delivery guarantee, and this is the thing to be precise about.

Exactly-once execution is not available. The worker can finish the work and die before recording that it did. **You cannot make “do the thing” and “record that I did the thing” atomic when the thing is an external side effect.**

So there are two honest choices, per job:

At-least-once — mark the run complete *after* the work. A **crash between them means it runs again. This is the default, and it requires the job to be idempotent.**

At-most-once — mark it complete *before* the work. A **crash means it never runs.** Correct for anything where a duplicate is worse than a miss, and rare.

And “make the job idempotent” is the actual answer, exactly as it was for payments and notifications: **an idempotency key derived from `(job_id, scheduled_time)`**, checked by whatever the job touches.

Then recurring jobs, where the modelling matters.

Do not store “every day at 09:00” as a single row you keep updating. Store the rule on the job, and materialise the next run as its own row — created when the previous one completes.

Two properties follow, and both are worth having. A history: every past run is a row, with its outcome. **And no drift:** the next run is computed from the schedule, not from when the last one finished.

And the awkward question: what if a run is late — do you catch up? A nightly report that was missed for three days: **run it three times, or once? There is no default answer**, and the honest design makes it a per-job policy with a “do not run more than N missed occurrences” bound.

Then the herd, which is the ward sister’s problem.

Everybody schedules things at the top of the hour. Nine o’clock, midnight, the start of the month. **A hundred million jobs at 09:00:00 is a spike that no amount of capacity planning survives**, because the average rate is fine and the instantaneous rate is not.

Two fixes, both trivial. Jitter: spread each job by a random offset within an acceptable window — **“at 09:00” becomes “between 09:00 and 09:05”**, which is almost always acceptable and removes the spike entirely. **And rate-limited dispatch:** the scheduler pulls at a bounded rate regardless of how much is due, so the backlog drains smoothly instead of arriving at once.

Finally, the scaling shape, which is not what people expect.

The scheduler is small. Even a hundred million scheduled jobs a day is about a thousand a second — **an ordinary database workload.** The polling loop is a query every second.

The workers are where the capacity is, and they scale horizontally because `SKIP LOCKED` means they need no coordination at all.

And the one thing that does need care at very large scale is the poll itself, because every scheduler instance querying the same index every second is contention on one hot range. **Partitioning the run table by time bucket, or sharding by job id, removes it** — and at most realistic scales it is not needed.

4 The picture

Job, schedule, run — three things, not one:

```
JOB  "send the weekly report"
+ SCHEDULE "every Monday at 09:00"
|
+→ RUN  2026-08-31 09:00    state: done        attempt 1
+→ RUN  2026-09-07 09:00    state: done        attempt 2 (retried)
+→ RUN  2026-09-14 09:00    state: pending      ← materialised
                                                    when the last
                                                    one completed
```

ONE job, MANY runs.

Conflating them — one row you keep updating — costs you the history AND makes retries impossible to model.

Claiming work without a race:

BROKEN – select then update

```
worker A: SELECT ... WHERE state='pending' → job 42
worker B: SELECT ... WHERE state='pending' → job 42    ← both see it
worker A: UPDATE 42 SET state='leased'
worker B: UPDATE 42 SET state='leased'
→ BOTH RUN JOB 42
```

CORRECT – one atomic statement

```
UPDATE runs SET state='leased', lease_until=now()+60s, worker=me
WHERE id IN (SELECT id FROM runs
             WHERE state='pending' AND run_at ≤ now()
             ORDER BY run_at LIMIT 10
             FOR UPDATE SKIP LOCKED)
RETURNING *
```

`SKIP LOCKED` IS WHAT MAKES IT SCALE:

without it, 10 workers all BLOCK on the same rows

→ throughput of ONE worker

with it, each takes rows nobody else holds

→ no coordination at all, workers scale horizontally

The lease, and the trade with no good setting:

```
worker claims job 42, lease_until = now + 60s
|
+-- finishes in 10s    → mark done, lease irrelevant
|
+-- still working at 50s → HEARTBEAT: extend the lease
|
+-- dies at 20s        → at 60s the lease EXPIRES
                        → another worker claims it
                        → the job runs (again)
```

THE TRADE:

SHORT LEASE (10s)	dead workers detected fast
	a slow-but-alive worker gets its job STOLEN
	→ the job runs TWICE

LONG LEASE (1 hour)	slow workers are safe
	a dead worker's job sits idle for an hour

AND THERE IS NO SETTING THAT AVOIDS BOTH, because you cannot distinguish a DEAD worker from a SLOW one over a network.

The mitigation is the HEARTBEAT: short leases, renewed while genuinely working.

Why exactly-once is not available:

```

do the work                                mark it done
-----                                -----
      |                                |
      +--- crash HERE -----+

```

the side effect HAPPENED and the record does NOT exist

AT-LEAST-ONCE: mark done AFTER the work
 crash → it runs AGAIN
 → requires the job to be IDEMPOTENT
 → THE DEFAULT

AT-MOST-ONCE: mark done BEFORE the work
 crash → it NEVER runs
 → correct only when a duplicate is worse
 than a miss

"EXACTLY ONCE" would need the side effect and the record to be
 ATOMIC – and the side effect is in somebody else's system.

→ the real answer is: at-least-once + an idempotency key
 derived from (job_id, scheduled_time)

The thundering herd, and the one-line fix:

WITHOUT JITTER

```

jobs due:    09:00:00 ##### 100,000,000
              09:00:01
              09:00:02
              ...
              09:04:59

```

→ the average rate is fine; the INSTANTANEOUS rate is not
 → every worker wakes, the database is hammered, everything
 times out, and the retries make it worse

WITH JITTER (spread over 5 minutes)

```

09:00:00 ####
09:00:30 ####
09:01:00 #### ~333,000 per second, evenly
...
09:04:30 ####

```

ONE LINE: `run_at = scheduled + random(0, jitter_window)`

"It was never that we had too much work. It was that we had
 written it all down at the same time."

Where the capacity actually is:

THE SCHEDULER

100,000,000 jobs/day
= ~1,200/second
one indexed range query
per second

→ AN ORDINARY DATABASE
WORKLOAD

→ the scheduling is NOT
the hard part

THE WORKERS

the actual work: sending emails,
generating reports, calling webhooks

→ seconds to hours each
→ THIS is where the machines are

→ and they scale horizontally,
because SKIP LOCKED means they
need no coordination

5 How it actually works

The schema

```
CREATE TABLE jobs (  
  id          BIGINT PRIMARY KEY,  
  payload     JSONB NOT NULL,  
  schedule    TEXT,                -- a cron expression, or NULL for one-off  
  max_attempts INT NOT NULL DEFAULT 3,  
  jitter_seconds INT NOT NULL DEFAULT 0  
);  
  
CREATE TABLE runs (  
  id          BIGINT PRIMARY KEY,  
  job_id      BIGINT NOT NULL REFERENCES jobs(id),  
  run_at      TIMESTAMPTZ NOT NULL,  
  state       TEXT NOT NULL,        -- pending, leased, done, failed  
  lease_until TIMESTAMPTZ,  
  worker      TEXT,  
  attempt     INT NOT NULL DEFAULT 0,  
  UNIQUE (job_id, run_at)           -- one run per job per scheduled time  
);  
  
CREATE INDEX ON runs (state, run_at) WHERE state IN ('pending', 'leased');
```

UNIQUE (job_id, run_at) is the line that prevents duplicate scheduling — two scheduler instances both materialising the next run of a recurring job **cannot both succeed**.

And the partial index — only **pending** and **leased** — keeps it small: **completed runs** are the vast majority of the table and are never queried by the polling loop.

Claiming work

```
LEASE_SECONDS = 60

def claim(worker_id: str, batch: int = 10) → list[dict]:
    """One atomic statement. SKIP LOCKED is what makes workers scale."""
    return db.query("""
        UPDATE runs
        SET state = 'leased',
            lease_until = now() + interval '%(lease)s seconds',
            worker = %(worker)s,
            attempt = attempt + 1
        WHERE id IN (
            SELECT id FROM runs
            WHERE state = 'pending' AND run_at ≤ now()
            ORDER BY run_at
            LIMIT %(batch)s
            FOR UPDATE SKIP LOCKED
        )
        RETURNING id, job_id, attempt
    """, {"lease": LEASE_SECONDS, "worker": worker_id, "batch": batch})
```

FOR UPDATE SKIP LOCKED is the whole scaling story. Without it, **ten workers polling at the same moment all queue behind the same rows** and the system's throughput is one worker's. **With it, each takes what nobody else holds.**

attempt = attempt + 1 on the claim rather than on completion matters: a worker that dies has already consumed an attempt, which is what stops an infinitely-crashing job retrying forever.

And ORDER BY run_at inside the subquery makes it fair — the oldest due work goes first, so a backlog drains in order rather than starving old jobs.

The lease heartbeat

```
def heartbeat(run_id: int, worker_id: str) → bool:
    """Extend the lease while genuinely working. Returns False if it was stolen."""
    rows = db.execute("""
        UPDATE runs
        SET lease_until = now() + interval '%(lease)s seconds'
        WHERE id = %(id)s AND worker = %(worker)s AND state = 'leased'
    """, {"lease": LEASE_SECONDS, "id": run_id, "worker": worker_id})
    return rows == 1
```

AND worker = %(worker)s is what makes this safe. If the lease expired and another worker took the run, **this update affects zero rows and returns False** — and the **original worker must then stop**, because somebody else is now running it.

A heartbeat that does not check ownership will happily extend a lease it no longer holds, and then two workers both believe they own the run.

Executing a run

```
def execute(run: dict, worker_id: str) → None:
    job = job_store.get(run["job_id"])
    idempotency_key = f"job:{run['job_id']}:{run['run_at']}"

    stop = threading.Event()
    threading.Thread(target=_heartbeat_loop,
                    args=(run["id"], worker_id, stop), daemon=True).start()

    try:
        handler(job["payload"], idempotency_key=idempotency_key)
        complete(run["id"], worker_id, "done")          # AFTER the work
    except Exception as error:
        if run["attempt"] ≥ job["max_attempts"]:
            complete(run["id"], worker_id, "failed", str(error))
        else:
            reschedule(run["id"], worker_id, backoff(run["attempt"]))
    finally:
        stop.set()
```

`complete` after the work is what makes this at-least-once. A crash between the two means the run executes again — which is why the idempotency key is passed to the handler.

And the key is derived from `(job_id, run_at)`, not generated per attempt: a retry must carry the same key, or deduplication downstream does nothing.

The heartbeat runs on its own thread because the work is blocking, and `finally: stop.set()` guarantees it stops even when the handler throws.

Reclaiming expired leases

```
def reclaim_expired() → int:
    """A dead worker's lease lapses and the run becomes available again."""
    return db.execute("""
        UPDATE runs
        SET state = 'pending', worker = NULL, lease_until = NULL
        WHERE state = 'leased' AND lease_until < now()
    """)
```

This is the only thing that recovers from a dead worker, and it needs no failure detection — **the absence of a heartbeat is the signal**, exactly as with presence and distributed locks.

And it must be idempotent and safe to run from several scheduler instances, which the conditional `WHERE` makes it: a run already reclaimed by another instance no longer matches.

Recurring jobs

```
from croniter import croniter

def materialise_next(job_id: int, after: datetime) → None:
    """Compute the next run from the SCHEDULE, not from when the last finished."""
    job = job_store.get(job_id)
    if not job["schedule"]:
        return
    next_at = croniter(job["schedule"], after).get_next(datetime)
    if job["jitter_seconds"]:
        next_at += timedelta(seconds=random.randint(0, job["jitter_seconds"]))
    db.execute("""
        INSERT INTO runs (id, job_id, run_at, state)
        VALUES (%(id)s, %(job)s, %(at)s, 'pending')
        ON CONFLICT (job_id, run_at) DO NOTHING
    """, {"id": snowflake.next_id(), "job": job_id, "at": next_at})
```

`croniter(schedule, after)` computes from the *schedule*, not from now — so a job that ran late does not drift. A daily job delayed by twenty minutes still runs at nine tomorrow, not at nine twenty.

And `ON CONFLICT DO NOTHING` against the unique constraint means two scheduler instances materialising the same next run is harmless — one wins, the other is a no-op.

Jitter, which is one line

```
def schedule_with_jitter(job_id: int, at: datetime, window_seconds: int) → datetime:
    """'At 09:00' becomes 'between 09:00 and 09:05'."""
    return at + timedelta(seconds=random.randint(0, window_seconds))
```

One line, and it removes the entire thundering-herd problem — which is why it is worth naming as a design decision rather than a detail.

The window is a product question: “send at nine” usually tolerates five minutes; “expire this token at nine” does not, so jitter is per job and defaults to zero.

Missed runs, which need a policy

```
CATCH_UP_LIMIT = 3

def handle_missed(job_id: int, now: datetime) → list[datetime]:
    """
    A nightly job missed for three days: run it three times, or once?
    There is no default answer. Make it a per-job policy.
    """
    job = job_store.get(job_id)
    policy = job.get("catch_up", "skip")
    missed = list(missed_occurrences(job["schedule"], job["last_run"], now))
    if policy == "skip":
        return missed[-1:] # only the most recent
    if policy == "all":
        return missed[-CATCH_UP_LIMIT:] # bounded, or an outage floods
    return [] # "none": drop them
```

The bound on `all` is not optional. A job that was down for a week with an hourly schedule would otherwise materialise a hundred and sixty-eight runs at once — a self-inflicted herd, at exactly the moment the system is recovering.

Backoff for retries

```
def backoff(attempt: int) → int:
    """Exponential, capped, with jitter – the same rule as everywhere else."""
    base = min(2 ** attempt, 3600)
    return base + random.randint(0, base // 2)
```

Jitter here for the same reason as in scheduling: without it, a hundred jobs that failed together retry together, and the failure repeats.

The scheduler loop

```
def scheduler_loop(worker_id: str) → None:
    while True:
        runs = claim(worker_id, batch=10)
        if not runs:
            time.sleep(1) # nothing due
            continue
        for run in runs:
            execute(run, worker_id)
```

Polling once a second is genuinely fine for almost every scheduler, and it is much simpler than anything event-driven. Sub-second precision is a requirement worth questioning — very few scheduled jobs need it, and it changes the design substantially.

The real systems

PostgreSQL	the job store; SELECT ... FOR UPDATE SKIP LOCKED
Quartz	is the standard queue pattern and is genuinely good
Airflow	the Java scheduler; clustered mode uses row locks
Temporal / Cadence	DAG-oriented; the scheduler/executor split is the same shape, with dependencies added
Sidekiq / Celery	same shape, with dependencies added
cron	queue-backed workers, Redis or a broker
	one machine, no coordination, no history –
	and correct for a great many cases

Temporal is worth naming as the different approach: instead of making jobs idempotent, **it persists the execution state so a crashed job resumes where it stopped** — which changes the guarantee genuinely rather than working around it.

6 The numbers

Scale.

```
100,000,000 scheduled jobs/day
= ~1,200/second average

but the distribution is NOT uniform:
  ~40% are scheduled on the hour
  ~15% at midnight
  → the instantaneous rate at 09:00:00 could be MILLIONS

→ the average is comfortable; the SPIKE is the whole problem
```

The scheduler's own load.

```
polling every second, 10 scheduler instances:
  10 queries/second, each an indexed range scan

the pending set at any moment (with jitter, steady state):
  ~1,200/second x a few seconds of lag = ~5,000 rows

→ a trivial database workload
→ THE SCHEDULING IS NOT THE HARD PART
```

Storage.

```
run row: ~200 bytes  
100,000,000 runs/day = 20 GB/day
```

```
with 90-day retention: 1.8 TB  
+ replicas: 5.4 TB
```

→ and the PARTIAL INDEX on (state, run_at) covers only pending and leased rows – a few thousand at a time – rather than 9 billion completed ones

That partial index is the difference between a usable table and an unusable one, and it is one `WHERE` clause.

The herd, quantified.

```
100,000,000 jobs, 40% on the hour = 40,000,000 at 09:00:00
```

WITHOUT JITTER:

the claim query returns whatever it can, but every worker polls simultaneously; the database sees 10 concurrent range scans over 40,000,000 matching rows

→ the query slows from ~1 ms to seconds

→ workers time out, retry, and make it worse

WITH 5 MINUTES OF JITTER:

```
40,000,000 / 300 seconds = ~133,000/second, evenly
```

at 10 jobs per claim and ~50 ms per job:

each worker does ~200 jobs/second

→ ~700 workers, steadily

→ ONE LINE turns an unbounded spike into a flat 133,000/second

Lease settings.

LEASE = 10 seconds, no heartbeat:
any job taking > 10 s gets stolen and runs twice
→ unusable for anything real

LEASE = 1 hour, no heartbeat:
a dead worker's job sits idle for up to an hour
→ unacceptable latency for a retry

LEASE = 60 seconds WITH a 20-second heartbeat:
dead worker detected in ≤ 60 s
a 6-hour job renews 1,080 times and is never stolen
heartbeat load: 700 workers / 20 s = 35 updates/second

→ 35 writes a second buys correct behaviour for both cases.
That is the argument for heartbeating rather than tuning the lease.

Worker capacity, which is where the machines are.

job duration varies enormously:
send an email ~100 ms
call a webhook ~500 ms (external!)
generate a report ~30 s to minutes

at 1,200 jobs/second average and ~200 ms mean:
 $1,200 \times 0.2 = 240$ concurrent jobs
at 50 concurrent per worker (I/O bound): ~5 workers

at the jittered peak of 133,000/second:
 $133,000 \times 0.2 = 26,600$ concurrent
→ ~530 workers

→ the fleet is sized for the PEAK, and jitter is what stops the peak being unbounded

Retention and cleanup.

9,000,000,000 completed runs over 90 days

```
DELETE FROM runs WHERE state IN ('done','failed') AND run_at < now() - 90 days
```

→ a DELETE over billions of rows is a disaster: it locks, bloats, and never finishes

→ PARTITION BY MONTH and DROP the old partition
→ instant, no locking, no bloat

Same lesson as the pastebin's expiry: at this size, deletion is a partition drop, not a DELETE.

Duplicate rate, which is the honest number:

at-least-once, 60-second lease with heartbeat:

duplicates occur when a worker is partitioned from the database for longer than the lease while still executing

realistically: ~0.01% - 0.1% of runs

at 100,000,000/day: 10,000 - 100,000 duplicate executions/day

→ which is why "make the job idempotent" is not advice, it is a REQUIREMENT. Ten thousand duplicate emails a day is a product failure; ten thousand idempotent no-ops is nothing.

Latency.

poll interval 1 s	→ up to 1 s of scheduling lag
claim query	→ ~1-5 ms
job dispatch	→ ~1 ms

→ a job scheduled for 09:00:00 starts by about 09:00:01

sub-second precision would need event-driven scheduling — a timer wheel or a delay queue — which is a substantially different design and is very rarely required.

7 The trade-offs

Lease length: short against long. Short detects dead workers quickly and **steals jobs from slow-but-alive workers, causing duplicate execution.** Long protects slow workers and **leaves a dead worker's job idle for the whole lease.** **There is no setting that avoids both, because a dead worker and a slow one are indistinguishable over a network** — and the resolution is heartbeating, which makes short leases safe at the cost of a small continuous write load.

At-least-once against at-most-once. At-least-once is the default and **requires every job to be idempotent** — which is a real constraint on job authors, and it is not optional at 100,000 duplicates a day. At-most-once loses work on a crash **and is correct when a duplicate is worse than a miss**, which is rare and should be an explicit per-job choice rather than an accident of where the `complete` call sits.

Polling against event-driven. Polling every second is simple, robust, and adds up to a second of lag. **Event-driven — a timer wheel, or a delay queue — gives sub-second precision and is substantially more complex**, with its own recovery problems. **Almost no scheduled job needs sub-second precision**, and asking whether this one does is worth ten seconds.

A database against a dedicated queue. Postgres with `SKIP LOCKED` handles thousands of jobs a second, gives transactions, history and ad-hoc queries for free, **and is one system rather than two**. A dedicated queue scales further and **loses the ability to ask “what is scheduled for tomorrow”**, which is a question operators ask constantly.

Catch-up policy. Running every missed occurrence is correct for accounting jobs and **catastrophic for notifications** — a week’s outage becomes a hundred and sixty-eight emails at once. Skipping to the most recent is right for most things and **silently loses work** for some. **There is no default, and the bound on catch-up is not optional**, or recovery from an outage becomes a second outage.

Jitter against precision. Jitter removes the herd for one line of code and **makes the run time approximate**. “Send at nine” tolerates it; **“expire this token at nine” does not**. So it is per job and defaults to zero, **which means the herd comes back for any job type where somebody set it to zero across the board**.

When would I not build this? `cron` on one machine is correct for a great many systems — no coordination, no lease, no distributed anything — and the failure mode, that the machine dying stops everything, is often acceptable. **Managed schedulers exist:** cloud cron services, Temporal, Airflow. **And if the jobs have dependencies between them** — “run B after A succeeds” — **this is a workflow engine and not a scheduler**, which is a genuinely different and larger system.

8 In the interview

How it gets asked

- “Design a distributed job scheduler.” or “Design cron as a service.”
- “A worker picks up a job and dies. What happens?” — the lease question.
- “Can you guarantee a job runs exactly once?” — where honesty scores.
- “A million jobs are all scheduled for 09:00. What happens?” — the herd.
- “How do you handle jobs that run for hours?”
- “How do recurring jobs work?”

“The scheduling part is easy and I want to say that first, because it is not where the design is.

A table of runs keyed by time, an index on (state, run_at), and a loop that claims whatever is due. Even a hundred million jobs a day is about twelve hundred a second — **an ordinary database workload.**

What is hard is failure, and there are three distinct problems.

First, two workers must not claim the same run. A select followed by an update is a race. **The claim has to be one atomic statement** — an `UPDATE ... WHERE id IN (SELECT ... FOR UPDATE SKIP LOCKED)`.

And SKIP LOCKED is what makes it scale: without it, ten workers polling simultaneously all block on the same rows and the throughput is one worker’s. **With it, each takes rows nobody else holds and there is no coordination at all.**

Second, a worker that dies must not hold the job forever. So the claim carries a lease with an expiry, and an expired lease means the run is available again.

And that produces a trade with no good setting. A short lease detects dead workers fast and steals jobs from slow-but-alive ones, so they run twice. **A long lease protects slow workers and leaves a dead worker’s job idle for the whole lease. You cannot distinguish dead from slow over a network** — so the resolution is a **heartbeat**: keep the lease short and renew it while genuinely working.

Third, and I would raise it unprompted: exactly-once execution is not available.

A worker can finish the work and die before recording that it did. You cannot make the side effect and the record atomic when the side effect is in somebody else’s system. **So it is at-least-once with idempotent jobs, or at-most-once with the risk of loss** — and that is a per-job decision, not a system-wide one.

And I would model jobs and runs separately. A daily report is **one job and three hundred and sixty-five runs** — which gives a history, makes retries natural, and stops recurring schedules drifting.

One question before I go further: does anything here need sub-second precision? Because polling once a second is simple and robust, **and event-driven scheduling is a substantially different design that very few jobs actually**

need.“

The follow-ups

“A worker picks up a job and dies. What happens?”

“The lease expires and another worker takes it — **and the interesting part is the trade that sets the lease length, because there is no setting that is right.**

When a worker claims a run it sets `lease_until` to now plus some period. A background sweep resets anything whose lease has passed back to pending. **So a dead worker’s job becomes available again with no failure detection needed** — the absence of a renewal is the signal.

Now the trade.

A short lease — say ten seconds — **detects death fast, and steals the job from any worker that is merely slow.** A job taking thirty seconds gets taken away and run again by somebody else, **so it executes twice and both workers think they own it.**

A long lease — an hour — **protects slow workers, and a dead worker’s job sits idle for up to an hour,** which for a retry is unacceptable latency.

And there is no value that avoids both, because a dead worker and a slow one look identical from the database’s point of view. That is the failure-detection result: **you cannot distinguish a crashed process from a slow one over a network.**

So the resolution is not to tune the lease — it is to heartbeat. Keep the lease short, and have the worker extend it periodically while genuinely working. A sixty-second lease with a twenty-second heartbeat **detects death within a minute and lets a six-hour job run untouched.**

And the heartbeat must check ownership. `UPDATE ... WHERE id = ? AND worker = ?` — if the lease already expired and somebody else took the run, **the update affects zero rows and the original worker must stop.** Without that check, a worker happily extends a lease it no longer holds, **and two workers both believe they own the run.**

One more detail: the attempt counter increments on the CLAIM, not on completion. A worker that dies has already consumed an attempt, which is what stops a job that crashes the worker every time from retrying forever.”

“Can you guarantee a job runs exactly once?”

“No, and I would rather say that plainly than describe something that is not exactly-once and call it that.

The reason is specific. A worker does the work — sends the email, calls the webhook — and then records that it did. If it crashes between those two moments, the side effect has happened and the record does not exist.

And you cannot make those atomic, because the side effect is in somebody else’s system. A database transaction cannot cover an email that has already been sent.

So there are exactly two honest choices, and they are a per-job decision.

At-least-once: mark the run complete after the work. A crash means it runs again. This is the default, and it requires the job to be idempotent — which is a real constraint on whoever writes the job, not a nice-to-have.

At-most-once: mark it complete before the work. A crash means it never runs. Correct when a duplicate is worse than a miss, which is rare — and it should be an explicit choice, not an accident of where somebody put the `complete` call.

And the practical answer is: at-least-once plus an idempotency key, derived from `(job_id, scheduled_time)` — not generated per attempt, because a retry must carry the same key or deduplication downstream does nothing. That is the same rule as payments and notifications.

I would give the honest number too, because it turns this from a theoretical concern into a requirement. Duplicates happen when a worker is partitioned from the database for longer than its lease while still executing — realistically a hundredth to a tenth of a percent of runs. At a hundred million runs a day, that is ten to a hundred thousand duplicate executions.

Ten thousand duplicate emails a day is a product failure. Ten thousand idempotent no-ops is nothing.

And there is a genuinely different approach worth naming: durable execution, as in Temporal. Instead of making jobs idempotent, it persists the execution state so a crashed job resumes where it stopped. That changes the guarantee rather than working around it — at the cost of a much heavier programming model.“

“A million jobs are all scheduled for 09:00. What happens?”

“Everything falls over, and the reason is that the average rate is fine and the instantaneous rate is not.

A hundred million jobs a day is twelve hundred a second, which is comfortable. But the distribution is nothing like uniform — **people schedule on the hour.** Perhaps forty percent of everything lands at some `:00`, and a large share at midnight.

So at nine o'clock exactly, the claim query has forty million matching rows, every scheduler instance polls at once, **the query goes from a millisecond to seconds, workers time out — and the retries make it worse.** The classic pattern where the recovery attempt is the second failure.

The fix is jitter, and it is one line. When materialising a run, add a random offset within an acceptable window. ‘At nine’ becomes ‘between nine and five past’.

Forty million jobs spread over three hundred seconds is a hundred and thirty-three thousand a second, evenly — which is a capacity question rather than a crisis.

And the window is a product decision, per job. ‘Send the daily digest at nine’ tolerates five minutes happily. ‘**Expire this token at nine**’ does not — so jitter defaults to zero and is set deliberately.

Which is worth flagging as a risk: if job authors leave it at zero everywhere, the herd comes straight back. So I would make it a default on the job type rather than something each author remembers.

The second mechanism is rate-limited dispatch. The scheduler claims at a bounded rate regardless of how much is due, **so a backlog drains smoothly rather than arriving at once —** and it protects against the case where jitter was not applied, or where a recovery has produced a genuine backlog.

And that connects to the catch-up policy, which is the same problem wearing a different hat. If the system was down for a week and a job runs hourly, **materialising every missed occurrence creates a hundred and sixty-eight runs at once — a self-inflicted herd at exactly the moment the system is recovering.** So the catch-up bound is not optional, and neither is the choice between running all missed occurrences, only the most recent, or none — **which has no default answer and belongs on the job.**“

The model answer

“Design a job scheduler for a platform where customers can schedule arbitrary tasks: a hundred million jobs a day, tasks lasting from milliseconds to hours, and it must survive workers dying.”

“The scheduling is the easy part and I want to establish that first so we spend the time correctly.

A hundred million jobs a day is twelve hundred a second — an ordinary database workload. A table of runs with an index on `(state, run_at)`, and a loop that claims what is due. **The difficulty is entirely in failure and in the distribution.**

Modelling: jobs and runs are separate. A job has a payload and optionally a schedule; **each occurrence is its own run row.** That gives a history, makes retries natural, and — because the next run is computed from the schedule rather than from when the last one finished — **recurring jobs do not drift.**

A unique constraint on `(job_id, run_at)` so two scheduler instances materialising the same next occurrence cannot both succeed.

Claiming: one atomic `UPDATE ... WHERE id IN (SELECT ... FOR UPDATE SKIP LOCKED)`. A select-then-update is a race. **And `SKIP LOCKED` is what lets workers scale horizontally with zero coordination** — without it, every worker blocks on the same rows and the fleet has the throughput of one.

The attempt counter increments on the claim, so a worker that dies has already consumed an attempt and a job that crashes its worker cannot retry forever.

Leases with heartbeats, because tasks run from milliseconds to hours. A **sixty-second lease renewed every twenty seconds** detects a dead worker within a minute **and lets a six-hour job run untouched.** **The heartbeat must check ownership** — if the lease lapsed and somebody else took the run, the update affects zero rows and the original worker stops.

And I would say plainly why there is no lease length that works without heartbeating: a dead worker and a slow one are indistinguishable over a network.

The delivery guarantee: at-least-once, with an idempotency key derived from `(job_id, run_at)`. Exactly once is not available — the work and the record of it cannot be made atomic when the work is an external side effect. **And I would give the number: roughly 0.01 to 0.1% of runs duplicate, which at this volume is ten to a hundred thousand a day.** So idempotency is a requirement on job authors, not advice.

At-most-once is available per job for anything where a duplicate is worse than a miss — **and it should be an explicit flag, not an accident of where the complete call sits.**

Now the thing that actually breaks this system: the herd. Forty percent of jobs get scheduled on the hour. **Forty million at 09:00:00 is not a capacity problem, it is an outage.**

Jitter is one line and removes it — spread each run over an acceptable window. **Forty million over five minutes is a hundred and thirty-three thousand a second, evenly.** Per-job, defaulting to zero, **and I would set it on the job type rather than trusting each author** — because if it is left at zero everywhere the herd returns.

Plus rate-limited dispatch and a bounded catch-up policy, because a week's outage on an hourly job would otherwise materialise a hundred and sixty-eight runs at once — **a self-inflicted herd during recovery.**

Sizing: the scheduler is a handful of instances polling once a second. The workers are where the machines are — about five hundred at the jittered peak, sized by job duration rather than by count.

And retention: partition the runs table by month and drop old partitions. A DELETE over nine billion rows locks, bloats and never finishes. Plus a **partial index covering only pending and leased rows,** which is what keeps the polling query fast against a table that is 99.99% completed runs.

Two closing points. I would question whether anything needs sub-second precision, because polling once a second is simple and robust and event-driven scheduling is a much larger design.

And if the tasks have dependencies on each other — ‘run B after A succeeds’ — this is the wrong system. That is a workflow engine, and building it as a scheduler with jobs that schedule other jobs **is how people end up with an undebuggable pile of scheduled work.** Worth establishing early.”

9 Recall card

The scheduling is the easy part — a table indexed on `(state, run_at)` and a poll loop; even 100M jobs/day is ~1,200/second. **Everything hard is failure and distribution.**

Model JOBS and RUNS separately (one job, many runs) for history, natural retries, and **no drift** — the next run is computed from the schedule, not from when the last finished. `UNIQUE (job_id, run_at)` stops two schedulers materialising the same occurrence.

Claim in ONE atomic statement with `FOR UPDATE SKIP LOCKED` — a select-then-update is a race, and without `SKIP LOCKED` every worker blocks on the same rows and the fleet has the throughput of one. **Increment the attempt counter on the CLAIM**, so a worker that dies has consumed an attempt.

Leases have no good length, because a dead worker and a slow one are indistinguishable over a network. Short steals jobs from slow workers (duplicate execution); long leaves a dead worker's job idle. **The resolution is a HEARTBEAT** — 60-second lease, 20-second renewal — **and the heartbeat MUST check ownership**, or two workers both believe they own the run.

Exactly-once is not available: the side effect and the record of it cannot be atomic when the side effect is in someone else's system. **At-least-once + an idempotency key from** `(job_id, run_at)` (not per attempt) is the default; at-most-once is an explicit per-job flag. **~0.01–0.1% of runs duplicate — 10,000–100,000/day at this volume — so idempotency is a REQUIREMENT, not advice.**

The herd is what actually breaks it: ~40% of jobs are scheduled on the hour, so the average rate is fine and the instantaneous rate is an outage. **Jitter is one line** (40M over 5 minutes = 133k/second, evenly) — **per job, defaulting to zero, so set it on the job TYPE or it will be left at zero.** Plus rate-limited dispatch and a **bounded catch-up policy**, or recovering from an outage becomes a second one. **And partition by month and DROP** — a `DELETE` over 9 billion rows never finishes.

DSA topic: Jump game and reachability **System design topic:** Design a distributed job scheduler

Code these, in this order

One rule for the whole set: **before writing a line, say what single number you are carrying**. Every problem here collapses a set into one value — the furthest reachable index, the end of the current level, the largest gap. **If you cannot name the number and say why the set collapses to it, you are about to write a set.**

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Jump Game	LeetCode 55 (Medium)	The collapse: a reachable set that is a prefix.
2	Jump Game II	LeetCode 45 (Medium)	BFS levels without a queue, and the loop bound.
3	Gas Station	LeetCode 134 (Medium)	The same frontier idea wearing a different hat.
4	Jump Game III	LeetCode 1306 (Medium)	Where the collapse fails — and you need a real search.
5	Jump Game VI	LeetCode 1696 (Medium)	Reachability plus a value, so greedy dies and a deque arrives.
6	Minimum Number of Taps to Water a Garden	LeetCode 1326 (Hard)	Jump Game II in disguise, with intervals.

On problem 1, delete the guard

Remove `if i > furthest: return False` and run on `[0, 1]` and `[3, 2, 1, 0, 4]`. **Record what each returns**. Then say in one sentence what the loop is doing after position 0 on the first input, and why nothing complains.

On problem 1, prove the prefix claim

Say out loud why the reachable set is always a prefix of the array. **Then construct an input where it is not** — you cannot, with non-negative jumps. **Say what would have to change for it to fail**, and connect that to problem 4.

On problem 2, get the loop bound wrong on purpose

Iterate to `len(nums)` instead of `len(nums) - 1`. Run on `[2, 3, 1, 1, 4]` and on `[2, 1]`. **Record both answers.** One is wrong and one is right — say why, and say what that means for a test set of three cases.

On problem 2, name the BFS

Write the explicit BFS with a queue that answers the same question. Run both on the same input and check they agree. **Then say which two variables in the greedy version are the queue**, and why the levels are contiguous.

On problem 3, find the frontier

Gas Station does not look like jumping. **Say what the running total is, and what resetting it to zero at a negative means.** Then say why the answer index is the one after the last reset, and why one pass suffices.

On problem 4, say why greedy dies

Jumps go both directions here. **Say precisely which property broke** — name it — and why the reachable set is no longer a prefix. Then write the BFS. **Compare its shape to your problem-2 solution and say what was implicit there.**

On problem 5, add a cost

Now each landing has a value and you want the best total. **Say why the furthest-reach heuristic gives the wrong answer**, and construct a small input where it does. Then write the DP with a monotonic deque.

On problem 6, do the translation explicitly

Convert each tap to an interval, then to a jump array. **Write down the conversion before coding it.** Solve it with your problem-2 code unchanged. **Then solve it directly on intervals** and say which you would write in an interview.

Then the counter-example drill

Build a version of Jump Game II where each jump has a cost and greedy loses. **Write the $O(n^2)$ DP and the greedy side by side** and search random inputs until they disagree. Record the smallest disagreement you find.

Then the verification drill

Write the $O(n^2)$ DP for minimum jumps. Check the greedy against it on two thousand random arrays with a fixed seed. **Say why zero mismatches is evidence and not proof**, and what would constitute proof.

The collapse drill

1. State what the naive solution tracks.
2. State the single number that replaces it.
3. State the property that lets the set collapse, in one sentence.
4. Say what breaks the property.
5. Give the complexity before and after.

The guard drill

1. Write the reachability loop from memory.
2. Name the one line most people omit.
3. Give an input where omitting it returns the wrong answer.
4. Say why the wrong answer is silent.
5. Say what invariant the guard is really checking.

The levels drill

1. Explain minimum jumps as a BFS.
2. Name the two variables that stand in for the queue.
3. Say why the levels are contiguous ranges.
4. Give the loop bound and why it is $n - 1$.
5. Say what answer you get with the wrong bound, and on what inputs.

The variants drill

For each, say what changes and whether greedy survives:

1. Can you reach the end.
2. Fewest jumps to the end.
3. Jumps in both directions.
4. Each landing has a cost.
5. Each landing has a reward.
6. Taps watering a garden.

The break-it drill

For each, say what happens and whether anything reports it:

1. The guard removed, on $[0, 1]$.
2. The loop running to n instead of $n - 1$.

3. Negative jump values.
 4. `furthest` initialised to `nums[0]` instead of `0`.
 5. Greedy applied when jumps have costs.
 6. Greedy applied when jumps go backwards.
-

The shape drill

1. Say what the scheduler stores and what the only query is.
2. Give the index, and why that index.
3. Say what fraction of the difficulty is scheduling, and what the rest is.
4. Give the average rate at 100M jobs/day.

The model drill

1. Say why jobs and runs are separate rows.
2. Give the three things that separation buys.
3. Say what drift is and how the model avoids it.
4. Give the unique constraint and what it prevents.
5. Say why the index is partial and what it costs otherwise.

The claiming drill

1. Say why select-then-update is wrong.
2. Write the atomic claim from memory.
3. Say what `SKIP LOCKED` does and what the throughput is without it.
4. Say why the attempt counter increments on the claim.
5. Say why the subquery is ordered.

The lease drill

1. Say what a lease is and what expiry means.
2. Give the short-lease failure and the long-lease failure.
3. State why no setting avoids both.
4. Give the heartbeat resolution with concrete numbers.
5. Say what the heartbeat must check, and what happens without that check.
6. Compute the heartbeat write load for 700 workers.

The guarantee drill

1. Say why exactly-once is not available, in two sentences.

2. Give both honest options and where the `complete` call sits in each.
3. Say what an idempotency key is derived from, and why not per attempt.
4. Give the duplicate rate and the daily count at 100M runs.
5. Say what durable execution changes, and what it costs.

The herd drill

1. Give the average rate and the instantaneous rate at 09:00.
2. Say why the average being fine is irrelevant.
3. Give the jitter fix and the arithmetic after applying it.
4. Say why jitter defaults to zero, and what risk that creates.
5. Say what rate-limited dispatch adds.
6. Say why catch-up is the same problem wearing a different hat.

The operations drill

1. Compute daily and 90-day storage.
 2. Say why a `DELETE` over that table fails.
 3. Give the alternative.
 4. Say where the machines actually are, and size the fleet at peak.
 5. Say what question you would ask before designing anything, and why.
 6. Say when this is the wrong system entirely.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *Can you reach the end of the array?* The collapse and the property that permits it, the guard and what it catches, and the complexity before and after.
 2. *Now give me the fewest jumps.* That it is a BFS over contiguous levels, which two variables are the queue, the loop bound and what the wrong bound gives, and what breaks greedy.
 3. *Design a distributed job scheduler.* That scheduling is the easy part, `SKIP LOCKED`, the lease trade with no good setting and why, that exactly-once is unavailable, and the thundering herd.
-

Before you move on

- [] I can name the single number the reachability solution carries.
- [] I can state the property that lets the reachable set collapse to one number.
- [] I can say what breaks that property.
- [] I write the guard, and I know what it catches.
- [] I know the guard's absence is silent on most inputs.
- [] I can write reachability from memory in five lines.
- [] I can explain minimum jumps as a BFS over levels.
- [] I can name the two variables that replace the queue.
- [] I know the loop bound is `n - 1`, and what the wrong bound returns.
- [] I know the wrong bound is right on some inputs, which is why it survives testing.
- [] I can construct an input where costed jumps break greedy.
- [] I know Jump Game III needs a real search, and why.
- [] I can translate the taps problem into a jump array.
- [] I checked the greedy against an `O(n^2)` DP and know why that is evidence, not proof.
- [] I can say what the scheduler stores and what the only query is.
- [] I know jobs and runs are separate, and the three things that buys.
- [] I can give the unique constraint and what it prevents.
- [] I know why the index is partial.
- [] I can write the atomic claim and explain `SKIP LOCKED`.
- [] I know the throughput without `SKIP LOCKED` is one worker's.
- [] I know why the attempt counter increments on the claim.
- [] I can give both lease failure modes and why no setting avoids both.
- [] I can give the heartbeat resolution with numbers.
- [] I know the heartbeat must check ownership, and what breaks without it.
- [] I can say in two sentences why exactly-once is not available.
- [] I know both honest guarantees and where the `complete` call sits in each.
- [] I know the idempotency key comes from `(job_id, run_at)`, not per attempt.
- [] I can give the duplicate rate and the daily count.
- [] I can give the herd arithmetic before and after jitter.
- [] I know jitter defaults to zero and what risk that creates.

- [] I know why a `DELETE` fails and what replaces it.
- [] I know when this is a workflow engine and not a scheduler.
- [] I answered all three questions above out loud.

Greedy and intervals revision and mock round

DATA STRUCTURES AND ALGORITHMS

Greedy and intervals revision and mock round

SYSTEM DESIGN

High-level design revision and full mock

Greedy and intervals revision and mock round

1 What this is, and why they ask it

This is the last day of greedy, and it is a **mock round**: three unseen problems under the clock, talked through out loud, with the solutions afterwards.

The revision half is one thing — a **decision procedure**. Given an unseen problem, **how do you decide, in under two minutes, whether greedy is safe?** And if it is, **which key do you sort by?**

Because those two questions are the whole of this phase. **Almost every greedy problem you will meet is “sort by the right thing, then take”**, and almost every greedy *failure* is either the wrong sort key or a greedy choice that was never safe.

They ask it because **greedy is the technique candidates most often get right for the wrong reason**. You write it, it passes the examples, and the interviewer asks “why does that work?” — **and there is nothing there**. A candidate who says “sort by end time, and here is the exchange argument” is in a different category from one who says “it seemed right”.

And the mock format matters as much as the problems. In a real interview you are choosing out loud, under time, without being able to run the code — **and the failure is almost never the technique**. It is freezing, or going silent, or writing before deciding.

By the end of this lesson you have the decision procedure, a sort-key table you can recall under pressure, the exchange argument in a form you can say in thirty seconds, the “greedy with an undo” family that bridges to DP, and three solved mock problems with a self-scoring rule.

2 The story

Anasuya had failed the driving test once, and it was not the driving.

Her uncle, who had taught her in the empty ground behind the sugar factory, said the same thing every evening for a month.

“Say it out loud. Whatever you are looking at, say it.”

She thought it was silly. She could see the gap. Why say it?

The second test was on a Tuesday morning and the examiner was a small man with a clipboard who said good morning and then nothing at all for eleven minutes.

And the silence was the hard part. Because with nobody speaking, her own head got loud, and the loudness was always the same shape — *was that the right moment, should I have waited, is he writing something down.*

So she did what her uncle had drilled into her, and she felt foolish doing it.

“Bike on the left, I am staying wide of him.” “Slowing for the school gate.” “There is a gap coming, I am taking it.”

At the roundabout by the bus stand there was a gap — **not a lovely gap, an adequate one** — and she took it, and she said why. *“That one is big enough and the next one might not come.”*

The examiner still said nothing. But she noticed, afterwards, that he had stopped writing at about the point she started talking.

At the end he signed the sheet and handed it over and said one thing, and she thought about it for years.

“Most of the people I fail can drive perfectly well.”

“They freeze at the junction. Not because they do not know what to do — **because they are waiting for a better moment than the one in front of them.** And the moment goes.”

“You took an ordinary gap and you told me why. That is the whole test.”

3 The idea in plain English

Greedy is: take the best-looking option now, and never revisit it. No stack of alternatives, no undoing — one pass, one decision at each step, committed.

That is why it is fast, and that is why it is usually wrong. Anasuya’s ordinary gap is right in traffic and wrong in a great many problems, **and the entire skill is telling the two apart before you write code.**

The decision procedure

Four questions, in this order. It takes under two minutes.

One: is there an obvious best-looking choice at each step? If you cannot name it in a sentence — “take the one that finishes earliest”, “take the largest coin” — **there is no greedy to try.**

Two: try to build a counter-example. Take the greedy choice, and ask **whether taking it can ever prevent a better total later.** Try small: three items, two of them conflicting. **This is where most greedy attempts die, and it should take thirty seconds.**

Three: state the exchange argument. *“Take any optimal solution. If it does not start with my greedy choice, I can swap my choice in for its first choice without making it worse. So there is an optimal solution starting with my choice, and by induction greedy is optimal.”* **If you can fill in the swap concretely, greedy is proved.** If you cannot, you do not have it.

Four: if two or three failed, it is DP. Greedy failing is not a dead end — **it is the signal that subproblems overlap and you need the table.**

Say all four out loud in an interview. The candidate who says “greedy fails here, and here is the three-element input where it fails, so I will do the DP” **is better than the one who guesses correctly**, because the reasoning transfers to a problem you have not seen.

The sort-key table

Almost every interval problem is “sort by the right key, then one pass.” The whole difficulty is choosing the key, so this is the thing to be able to recall cold:

THE QUESTION	SORT BY	WHY
most non-overlapping intervals (activity selection, arrows, non-overlapping intervals)	END	earliest finish leaves the most room behind it
merge overlapping intervals (insert, merge, free slots)	START	you can only merge with something already seen
how many overlap at once (rooms, peak load, skyline)	BOTH, as EVENTS	a start is +1, an end is -1; sweep and track the running total
can I reach the end / fewest jumps (jump game, gas station)	NOTHING (one pass)	the reachable set is a prefix, so it collapses to one frontier number
fit as many deadlines as possible (course schedule III)	DEADLINE + A HEAP	take everything, then UNDO the longest when you overrun
minimum stops with fuel/refuels (refuelling, IPO)	POSITION + A HEAP	pass everything, then take the biggest you passed, retroactively

Two families are worth naming separately, because they are the ones people do not recognise.

Greedy with an undo

Course Schedule III is the shape. Sort by deadline, **take every course**, and **when the running total overruns the current deadline, remove the longest course you have already taken.**

That removal is the exchange argument made executable. You are saying: *given that I must drop one, the longest is never a worse thing to drop* — because dropping it frees the most time and costs the same one course.

This is not pure greedy — it revisits decisions. **And it is not DP** — it never builds a table. It is the middle ground, and **it exists precisely where the greedy choice is right about “how many” but wrong about “which”.**

Regret greedy

Minimum Refuelling Stops is the shape. You do not decide at each station whether to stop. **You drive past everything you can reach, keeping what you passed, and when you run dry you retroactively take the biggest tank you went by.**

The trick is deferring the decision until you know you needed it. At the moment you run out of fuel you know exactly which stations were available, **so you can pick the best one with hindsight rather than guessing.**

Both families use a heap, and the heap is the tell. If a problem feels greedy but the greedy choice is clearly sometimes wrong, **ask whether you can defer or undo the choice with a heap** before reaching for DP.

Where greedy definitively dies

Add a cost to the choice and greedy usually dies. Jump Game is greedy; jump game *with a cost per landing* is DP. Coin change with the Indian coin set is greedy; **coin change with $[1, 3, 4]$ and target 6 is not** — greedy gives $4+1+1$ and the answer is $3+3$.

The pattern is: greedy survives when the choice affects only what is still available, and dies when the choice also carries a value you are optimising.

4 The picture

The decision procedure, as a chart:

an unseen problem

|

v

Can I name an obvious best-looking choice in one sentence?

|

+- NO ----> not greedy. Look for DP state.

|

YES

|

v

Can I build a small counter-example in 30 seconds?

|

+- YES ----> greedy is WRONG. Say so out loud, show the
counter-example, and go to DP.

|

NO

|

v

Can I state the exchange argument concretely?

|

+- NO ----> I do not have it. Try harder for a counter-
example; one of the two will give way.

|

YES

|

v

GREEDY, and I can defend it.

The middle branch is worth as many marks as the last one.

"Greedy fails, here is the input" is a correct answer.

The exchange argument in one picture:

greedy picks A (the earliest-finishing interval)
some optimal B ... (starts with something else)

OPTIMAL: [-----B-----][---C---][--D--]
GREEDY: [--A--]

A finishes no later than B, because greedy took the
earliest finish.

|

∨

SWAP: replace B with A in the optimal solution

SWAPPED: [--A--] [---C---][--D--]
 ^

A ends EARLIER than B did, so nothing that
followed B can now conflict.

→ same number of intervals, still valid
→ so there IS an optimal solution starting with A
→ induct on the rest

THAT IS THE WHOLE PROOF. It fits in thirty seconds
out loud, and it is what "why does that work?" wants.

Why “sort by end” and not “sort by start”:

[[1, 100], [11, 22], [1, 11], [2, 12]]

BY START: [1,100] [1,11] [2,12] [11,22]
take [1,100] first → it blocks EVERYTHING
keeps 1, removes 3 ← WRONG

BY END: [1,11] [2,12] [11,22] [1,100]
take [1,11], then [11,22]
keeps 2, removes 2 ← CORRECT

The greedy choice must be the one that COSTS THE LEAST
FUTURE. Earliest finish costs the least future. Earliest
start costs nothing predictable at all.

Greedy with an undo:

courses (duration, deadline), sorted by DEADLINE

```
[5, 5] take it. total 5. 5 ≤ 5 ok
[4, 6] take it. total 9. 9 > 6 OVERRUN
      |
      v
      undo the LONGEST taken (5)
      total 4. count still 2 in the heap? no –
      one was pushed and one popped, so 1... then
[2, 6] take it. total 6. 6 ≤ 6 ok
```

→ 2 courses

TAKE-AND-NEVER-UNDO gives 1: it takes [5,5], then nothing else fits.

The undo is the exchange argument, executed: given that I must drop one, dropping the LONGEST frees the most time at the same cost of one course.

Regret greedy:

target 100, start with 10 fuel
stations (position, fuel): [10,60] [20,30] [30,30] [60,40]

```
fuel 10 → can reach position 10
          pass [10,60], bank it           banked: {60}
          out of range. run dry.
          TAKE THE BIGGEST BANKED: 60
          fuel 70, stops 1
```

```
fuel 70 → can reach 20, 30, 60
          bank [20,30] [30,30] [60,40]    banked: {30,30,40}
          still short of 100. run dry.
          TAKE THE BIGGEST BANKED: 40
          fuel 110, stops 2
```

→ 2 stops

You never DECIDE at a station. You drive past everything, and you buy fuel WITH HINDSIGHT, at the moment you know you needed it.

5 The code, built step by step

This is a mock round. Read the problem, set a clock, and **say your decision procedure out loud before writing anything.** The solutions are below each one — do not read ahead.

Mock problem one, warm-up: fifteen minutes

Given a set of intervals, remove the minimum number so that the rest do not overlap. (Non-overlapping Intervals, LeetCode 435.)

The two minutes of thinking first. “Remove the fewest” is “**keep the most**”, which is activity selection. **The greedy choice is the earliest finishing time**, because it leaves the most room behind it. **The exchange argument: swap the earliest finisher into any optimal solution’s first slot; it ends no later, so nothing conflicts.**

```
def erase_overlap_intervals(intervals: list[list[int]]) → int:
    """SORT BY END. Keep greedily, count what you dropped."""
    if not intervals:
        return 0
    intervals = sorted(intervals, key=lambda pair: pair[1])
    kept, kept_end = 1, intervals[0][1]
    for start, end in intervals[1:]:
        if start ≥ kept_end:           # no overlap with the last one kept
            kept += 1
            kept_end = end
    return len(intervals) - kept
```

`start ≥ kept_end` is the tie-break, and it says touching intervals do not conflict — which is what this problem wants, and is a sentence worth saying out loud rather than assuming.

And answering “keep the most” and subtracting is easier than counting removals directly, which is worth noticing: **the greedy is on the keeping, and the removal count falls out.**

Mock problem two, the main one: twenty-five minutes

Each course takes some days and must finish by a deadline. Starting on day 1, take as many as possible. (Course Schedule III, LeetCode 630.)

The two minutes. Sorting by deadline is obviously right — **you must attempt earlier deadlines earlier, because a course you can take later you can also take now.**

But then the greedy choice fails, and you should find this yourself: `[[5,5], [4,6], [2,6]]`. Take-if-it-fits takes `[5,5]` and then nothing. **The answer is 2** — take `[4,6]` and `[2,6]`.

So greedy is right about the ordering and wrong about the commitment. Which is the tell for an undo.

```
import heapq

def schedule_course(courses: list[list[int]]) → int:
    """SORT BY DEADLINE. Take everything; when you overrun, undo the longest."""
    courses = sorted(courses, key=lambda course: course[1])
    taken: list[int] = [] # a max-heap of durations, negated
    total = 0
    for duration, deadline in courses:
        heapq.heappush(taken, -duration)
        total += duration
        if total > deadline: # the undo – the exchange argument, executed
            total += heapq.heappop(taken)
    return len(taken)
```

`total += heapq.heappop(taken)` **subtracts**, because the heap holds negated durations. **Write the comment, because you will misread it otherwise.**

And the argument for undoing the *longest* is the whole solution, so say it: **when you must drop one course, dropping the longest frees the most days and costs exactly the same one course.** There is never a reason to drop a shorter one.

Note what the counter is. `len(taken)` — **the heap’s size is the answer**, because every push that survived is a course you kept.

Mock problem three, the hard one: twenty minutes

A car starts with some fuel and must reach a target. Stations along the way each give a fixed amount. Fewest stops? (Minimum Number of Refueling Stops, LeetCode 871.)

The two minutes. The obvious greedy — “stop when you are about to run out” — **is a decision you cannot make correctly at the time**, because you do not yet know how far you must go.

So defer it. Drive as far as the fuel allows, **banking every station you pass**, and when you run dry, **buy the biggest tank you went past.**

```
def min_refuel_stops(target: int, start_fuel: int,
                    stations: list[list[int]]) → int:
    """REGRET GREEDY. Drive past everything; refuel retroactively, biggest first."""
    passed: list[int] = [] # fuel you drove past and could still take
    fuel, stops, i = start_fuel, 0, 0
    while fuel < target:
        while i < len(stations) and stations[i][0] ≤ fuel:
            heapq.heappush(passed, -stations[i][1])
            i += 1
        if not passed:
            return -1
        fuel += -heapq.heappop(passed)
        stops += 1
    return stops
```

`fuel` is doing two jobs and that is the elegant part: it is the fuel in the tank *and* the furthest position reachable, **because starting from zero they are the same number**. Say that out loud — it is the observation the whole solution rests on.

`stations[i][0] ≤ fuel` with `≤`, not `<`. A station exactly at the limit of your range **is reachable** — you arrive with an empty tank, which is fine. **With `<` the whole thing returns `-1` on the example**, and it is the single most likely bug here.

And `i` **never resets**. Each station is banked once, so the inner loop is amortised — **the two loops together are one pass, not a nested one**.

The self-scoring harness

Score yourself honestly on four things per problem, because the technique is rarely what fails:

1. Did I say the decision procedure OUT LOUD before writing?	yes / no
2. Did I name the sort key and JUSTIFY it?	yes / no
3. Did I try a counter-example before committing?	yes / no
4. Did I state the complexity without being asked?	yes / no

Three out of four on all three problems is an interview pass. Getting all three problems right silently is not.

The verification habit

Every greedy you write should be checked against a brute force on small random inputs. It takes four minutes, it is the only evidence you will ever have, **and it has caught more wrong sort keys than reasoning ever has**.

The complete file below does exactly that for all three.

The complete solution

```
"""Day 170 - the greedy mock round, solved, checked, and timed."""

from __future__ import annotations

import heapq
import itertools
import random

# ----- problem one
def erase_overlap_intervals(intervals: list[list[int]]) → int:
    """SORT BY END. Keep greedily, count what you dropped."""
    if not intervals:
        return 0
    intervals = sorted(intervals, key=lambda pair: pair[1])
    kept, kept_end = 1, intervals[0][1]
    for start, end in intervals[1:]:
        if start ≥ kept_end:
            # no overlap with the last one kept
            kept += 1
            kept_end = end
    return len(intervals) - kept

# ----- problem two
def schedule_course(courses: list[list[int]]) → int:
    """SORT BY DEADLINE. Take everything; when you overrun, undo the longest."""
    courses = sorted(courses, key=lambda course: course[1])
    taken: list[int] = []
    total = 0
    for duration, deadline in courses:
        heapq.heappush(taken, -duration)
        total += duration
        if total > deadline:
            # the undo - this is the exchange argument
            total += heapq.heappop(taken)
    return len(taken)

# ----- problem three
def min_refuel_stops(target: int, start_fuel: int,
                    stations: list[list[int]]) → int:
    """REGRET GREEDY. Drive past everything; refuel retroactively, biggest first."""
    passed: list[int] = []
    # fuel you drove past and could still take
    fuel, stops, i = start_fuel, 0, 0
    while fuel < target:
        while i < len(stations) and stations[i][0] ≤ fuel:
            heapq.heappush(passed, -stations[i][1])
            i += 1
        if not passed:
            return -1
        fuel += -heapq.heappop(passed)
        stops += 1
    return stops

# ----- the brute forces
def erase_brute(intervals: list[list[int]]) → int:
    """Every subset. Correct, and unusable past about eighteen intervals."""
    n = len(intervals)
    best = 0
    for size in range(n, 0, -1):
        for combo in itertools.combinations(intervals, size):
            ordered = sorted(combo)
            if all(ordered[i][1] ≤ ordered[i + 1][0] for i in range(len(ordered) - 1)):
                best = size
                break
    if best:
        break
```

```

return n - best

def schedule_brute(courses: list[list[int]]) → int:
    """Every subset in every order. Correct, and factorial."""
    best = 0
    n = len(courses)
    for size in range(n, 0, -1):
        found = False
        for combo in itertools.combinations(courses, size):
            for order in itertools.permutations(combo):
                clock = 0
                if all((clock := clock + d) ≤ dl for d, dl in order):
                    found = True
                    break
            if found:
                break
        if found:
            best = size
            break
    return best

def refuel_brute(target: int, start_fuel: int, stations: list[list[int]]) → int:
    """DP: furthest reachable with exactly k stops."""
    n = len(stations)
    reach = [start_fuel] + [0] * n
    for position, fuel in stations:
        for k in range(n - 1, -1, -1):
            if reach[k] ≥ position:
                reach[k + 1] = max(reach[k + 1], reach[k] + fuel)
    for k, distance in enumerate(reach):
        if distance ≥ target:
            return k
    return -1

# ----- the checks

def check_erase(trials: int = 2000) → int:
    bad = 0
    for _ in range(trials):
        n = random.randint(1, 8)
        intervals = []
        for _ in range(n):
            a = random.randint(0, 12)
            intervals.append([a, a + random.randint(1, 5)])
        if erase_overlap_intervals(intervals) ≠ erase_brute(intervals):
            bad += 1
    return bad

def check_schedule(trials: int = 400) → int:
    bad = 0
    for _ in range(trials):
        n = random.randint(1, 6)
        courses = [[random.randint(1, 6), random.randint(1, 15)] for _ in range(n)]
        if schedule_course(courses) ≠ schedule_brute(courses):
            bad += 1
    return bad

def check_refuel(trials: int = 2000) → int:
    bad = 0
    for _ in range(trials):
        n = random.randint(0, 6)
        target = random.randint(1, 40)
        start = random.randint(0, 20)
        positions = sorted(random.sample(range(1, 40), n))
        stations = [[p, random.randint(1, 15)] for p in positions]
        if min_refuel_stops(target, start, stations) ≠ refuel_brute(target, start, stations):

```

```

        bad += 1
    return bad

if __name__ == "__main__":
    random.seed(0)

    print("PROBLEM ONE - Non-overlapping Intervals (sort by END)")
    one = [[1, 2], [2, 3], [3, 4], [1, 3]]
    print(f" {one} → remove {erase_overlap_intervals(one)}")
    two = [[1, 100], [11, 22], [1, 11], [2, 12]]
    print(f" {two} → remove {erase_overlap_intervals(two)}")
    print(" sorted by START instead:")
    by_start = sorted(two, key=lambda p: p[0])
    kept, kept_end = 1, by_start[0][1]
    for s, e in by_start[1:]:
        if s ≥ kept_end:
            kept += 1
            kept_end = e
    print(f" keeps {kept}, so removes {len(two) - kept} (correct answer: "
          f"{erase_overlap_intervals(two)})")

    print()
    print("PROBLEM TWO - Course Schedule III (sort by DEADLINE, undo the longest)")
    courses = [[100, 200], [200, 1300], [1000, 1250], [2000, 3200]]
    print(f" {courses} → {schedule_course(courses)}")
    greedy_no_undo = [[5, 5], [4, 6], [2, 6]]
    print(f" {greedy_no_undo} → {schedule_course(greedy_no_undo)}")
    print(" take-and-never-undo would take [5,5] and then fit nothing: 1")

    print()
    print("PROBLEM THREE - Minimum Refueling Stops (regret greedy)")
    print(f" target 100, start 10, stations [[10,60],[20,30],[30,30],[60,40]] → "
          f"{min_refuel_stops(100, 10, [[10, 60], [20, 30], [30, 30], [60, 40]])}")
    print(f" target 100, start 1, stations [[10,100]] → "
          f"{min_refuel_stops(100, 1, [[10, 100]])}")
    print(f" target 1, start 1, stations [] → "
          f"{min_refuel_stops(1, 1, [])}")

    print()
    print("VERIFICATION - greedy against brute force")
    print(f" problem one: {check_erase()} mismatches in 2,000 random cases")
    print(f" problem two: {check_schedule()} mismatches in 400 random cases")
    print(f" problem three: {check_refuel()} mismatches in 2,000 random cases")

```

Running it:

PROBLEM ONE – Non-overlapping Intervals (sort by END)
 [[1, 2], [2, 3], [3, 4], [1, 3]] → remove 1
 [[1, 100], [11, 22], [1, 11], [2, 12]] → remove 2
 sorted by START instead:
 keeps 1, so removes 3 (correct answer: 2)

PROBLEM TWO – Course Schedule III (sort by DEADLINE, undo the longest)
 [[100, 200], [200, 1300], [1000, 1250], [2000, 3200]] → 3
 [[5, 5], [4, 6], [2, 6]] → 2
 take-and-never-undo would take [5,5] and then fit nothing: 1

PROBLEM THREE – Minimum Refueling Stops (regret greedy)
 target 100, start 10, stations [[10,60],[20,30],[30,30],[60,40]] → 2
 target 100, start 1, stations [[10,100]] → -1
 target 1, start 1, stations [] → 0

VERIFICATION – greedy against brute force
 problem one: 0 mismatches in 2,000 random cases
 problem two: 0 mismatches in 400 random cases
 problem three: 0 mismatches in 2,000 random cases

The **-1 case matters** — the first station is at 10 and you start with 1, **so you cannot even reach it**, and the `if not passed: return -1` is what catches that. And **target 1, start 1 with no stations gives 0**, because the `while fuel < target` loop never runs.

6 What it costs

Problem one.

```
sort:          n log n
one pass:      n comparisons

total:         0(n log n) time, 0(1) extra space
               (0(n) if the sort copies, which sorted() does)
```

THE SORT DOMINATES. Every interval problem in this phase has the same sentence, and it is worth saying: the greedy pass is linear and free; you pay for the ordering.

Problem two.

```

sort:                                n log n
one pass, n items:
  each push:                         log n
  at most n pops total:              n log n

total:                               O(n log n) time
                                       O(n) space for the heap

```

Count it concretely: 10,000 courses

```

sort: 10,000 x log2(10,000) ~= 10,000 x 13 = 130,000
pushes: 10,000 x 13 = 130,000
pops: at most 10,000 x 13 = 130,000
~= 400,000 operations. Instant.

```

The “at most n pops” is the amortised argument and it is worth stating: **you cannot pop more than you pushed**, so the total pop work over the whole run is bounded by the total push work — **not by n pops per step**.

Problem three.

```

each station is pushed AT MOST ONCE:  n pushes, n log n
each pop is one stop, at most n stops: n log n
the pointer i never resets:           n total advances

total:                               O(n log n) time
                                       O(n) space

```

The two nested while loops LOOK quadratic. They are not, because i only ever moves forward. Say this out loud — the interviewer is often checking whether you noticed.

Against the brute forces, so the verification cost is honest.

```

problem one, brute force:  every subset = 2^n
n = 8:  256 subsets        instant
n = 20: 1,048,576          a second or two
n = 40: 1,099,511,627,776  no

problem two, brute force:  every subset IN EVERY ORDER
n = 6:  sum over sizes of C(6,k) x k! = 1,957 orderings
n = 10: ~9,864,101
n = 12: ~1.3 billion       no

→ which is exactly why the checks in the file use n ≤ 8
  and n ≤ 6. The brute force is the ORACLE, not the
  solution, and it only has to be right on small inputs.

```

The whole phase, in one table.

PROBLEM FAMILY	TIME	SPACE	DOMINATED BY
activity selection	$n \log n$	$O(1)$	the sort
merging intervals	$n \log n$	$O(n)$	the sort
sweep line	$n \log n$	$O(n)$	the sort
jump game / reachability	$O(n)$	$O(1)$	NOTHING – no sort!
greedy with an undo	$n \log n$	$O(n)$	sort + heap
regret greedy	$n \log n$	$O(n)$	sort + heap

Jump game is the odd one out, and that is the point of it: when the greedy structure collapses to ONE NUMBER, you do not even pay for the sort.

7 The traps

The wrong sort key, which is silent.

Sorting by start on problem one:

```
[[1, 100], [11, 22], [1, 11], [2, 12]]
  by START: keeps 1, so removes 3
  by END:   keeps 2, so removes 2      ← correct
```

Nothing errors. The code runs, the answer is a plausible integer, and it passes `[[1,2],[2,3],[3,4]]` and every other example where the intervals happen to be in start order already. This is the trap of the phase: the wrong key is right on tidy inputs and wrong on real ones.

Sorting courses by duration instead of deadline:

```
[[1, 5], [2, 10], [6, 8]]
  by deadline: 3      ← correct
  by duration: 2
```

By duration takes 1, then 2 (total 3), then tries 6 — total 9, over the deadline of 8 — and undoes the 6. Two courses. By deadline it attempts the tight one while there is still room.

Take-and-never-undo on problem two.

```
[[5, 5], [4, 6], [2, 6]]
  with the undo: 2
  without it:    1
```

And the `>` in `if total > deadline` must not be `>=`:

```
[[5, 5], [4, 6], [2, 6]]
with `>`: 2      ← correct
with `≥`: 1
```

A course finishing exactly on its deadline is fine, and `≥` throws it away. One character, and it costs you a course on every input where the fit is exact — which is most of the interesting ones.

`<` instead of `≤` on problem three.

```
target 100, start 10, stations [[10,60],[20,30],[30,30],[60,40]]
with `≤`: 2      ← correct
with `<`: -1
```

A station exactly at the edge of your range is reachable. You arrive with an empty tank, which is legal. With `<` you never bank the first station, `passed` is empty, and the function reports the target unreachable — a wrong answer that looks like a legitimate “impossible”.

No guard on the empty input.

```
intervals = []
intervals.sort(key=lambda p: p[1])
kept_end = intervals[0][1]
```

```
Traceback (most recent call last):
  File "<stdin>", line 14, in empty_first
IndexError: list index out of range
```

This one at least announces itself, which puts it in the good category. The silent ones are worse.

Negating the wrong thing in the heap.

```
for course in courses:
    heapq.heappush(taken, -course)    # course is [duration, deadline]
```

```
Traceback (most recent call last):
  File "<stdin>", line 30, in heap_of_lists
TypeError: bad operand type for unary -: 'list'
```

Loud, and easy. The dangerous version is pushing `+duration` instead of `-duration`: no error, and the heap now pops the *shortest* course, so the undo removes the wrong one and the answer is quietly too small.

Reaching into the wrong tuple position.

```
courses.sort(key=lambda c: c[2])
```

```
Traceback (most recent call last):
  File "<stdin>", line 36, in sort_no_key
  File "<stdin>", line 36, in <lambda>
IndexError: list index out of range
```

And the two-subscript slip, which is the most common typing error under time pressure:

```
while i < len(stations) and stations[i, 0] ≤ fuel:
```

```
Traceback (most recent call last):
  File "<stdin>", line 22, in tuple_index
TypeError: list indices must be integers or slices, not tuple
```

“It worked on the examples” as a proof. The deepest trap of the phase. **Every wrong sort key passes the examples**, because example inputs are small and tidy and usually already in the right order. **The only evidence is a brute-force check on random inputs**, and the only *argument* is the exchange argument. **Have one of the two, and say which one you have.**

8 In the interview

How it gets asked

- “Two problems, no hints, talk as you go.” — the mock format itself.
- “Why does sorting by end time work?” — the exchange argument, and the real question of the phase.
- “How do you know greedy is correct here?”
- “Can you do better than the DP?” — often a hint that greedy exists.
- “What if each choice also had a cost?” — usually a hint that greedy just died.

The first ninety seconds

Said before writing anything, on any greedy-looking problem:

“Before I write, let me decide whether greedy is even safe here, because that is where these problems are won or lost.

First: is there an obvious best-looking choice at each step? Here it is **the interval that finishes earliest** — I can say it in a sentence, so there is a greedy to try.

Second: can I break it? Let me try three intervals with two of them conflicting... **I cannot construct a counter-example**, which is encouraging but not sufficient.

Third, and this is the real test: can I state the exchange argument?

Take any optimal solution. If it does not start with the earliest-finishing interval, swap that interval in for whatever it does start with. My interval finishes no later, **so nothing that followed can now conflict** — the solution is still valid and still the same size. **So there is an optimal solution starting with my greedy choice, and the same argument applies to what remains.**

That is a proof, and it is why I am confident rather than hopeful.

So: sort by end time, one pass, keep anything that starts at or after the last kept end. $O(n \log n)$, dominated entirely by the sort — the greedy pass itself is linear.

One clarification before I write: do intervals that touch count as overlapping? $[1, 2]$ and $[2, 3]$ — **it is one comparison and the problem decides it**, so I would rather ask than assume.

And when I am done I would check it against a brute force on small random inputs, because a wrong sort key passes every example and fails on real data.”

The follow-ups

“How do you decide whether greedy works, in general?”

“Four questions, in order, and it takes under two minutes.

One: can I name an obvious best-looking choice in one sentence? ‘Take the earliest finisher.’ ‘Take the largest coin.’ **If I cannot say it in a sentence, there is no greedy to try** and I go straight to DP state.

Two: can I build a small counter-example? I try three items with two of them in conflict. **This is where most greedy attempts die, and it should take thirty seconds.** Coin change is the standard example — greedy works for the coins in your pocket, and `[1,3,4]` with a target of 6 gives `4+1+1` when the answer is `3+3`.

Three: can I state the exchange argument concretely? Not the shape of one — **the actual swap.** ‘Replace the optimal’s first choice with mine; mine finishes no later; therefore nothing conflicts.’ **If I cannot fill in that middle clause, I do not have the argument** and I go back to hunting for a counter-example. One of the two always gives way.

Four: if two or three failed, it is DP — and I say that out loud rather than treating it as a failure. **‘Greedy fails here and here is the three-element input where it fails’ is a correct answer**, and it is worth as much as getting greedy right.

The pattern behind all of it: greedy survives when the choice affects only what is still available, and dies when the choice also carries a value you are optimising. Jump Game is greedy; jump game where each landing costs something is DP.

And there is a middle ground people forget. If greedy feels right but the choice is clearly sometimes wrong, **ask whether you can defer or undo it with a heap** before reaching for a table.”

“Which sort key, and why?”

“There are about five, and I would rather recall the reason than the key.

The rule underneath is: sort so the greedy choice costs the least future.

Sort by end when you want the most non-overlapping intervals — activity selection, arrows, non-overlapping intervals. **Earliest finish leaves the most room behind it**, which is precisely ‘costs the least future’.

Sort by start when merging. **You can only merge with something you have already seen**, so you need them in the order they begin.

Sort as events — a start is `+1`, an end is `-1` — when the question is ‘how many at once’. Rooms, peak load, the skyline. **And the tie-break at equal times is the whole difficulty there.**

Do not sort at all for reachability. **Jump Game is `O(n)`** because the reachable set is a prefix and collapses to one frontier number — **the odd one out in the phase, and the one worth remembering for that reason.**

Sort by deadline plus a heap when you want to fit as many deadlines as possible and pure greedy fails.

And here is the concrete evidence for why the key matters. On `[[1,100], [11,22], [1,11], [2,12]]`, **sorting by start keeps one interval and sorting by end keeps two.** Nothing errors. **The wrong key is silent, and it is right on every tidy example** — which is why I check against a brute force rather than against the samples.”

“What if each choice also had a cost?”

“Then greedy is probably dead, and I would say so immediately rather than trying to patch it.

The reason is structural. Greedy works when the choice only affects **what is still available**. As soon as the choice also carries **a value you are optimising**, the locally cheapest choice can lock you out of a globally cheaper path, **and there is no exchange argument** — the swap changes the total.

Concretely: Jump Game II is greedy and $O(n)$. Add a cost per landing and it is DP. On `[3,1,3,1,1,1]` with costs `[1,1,100,1,1,0]`, **greedy jumps to the position that reaches furthest — and that landing costs a hundred.** Three cheap hops down the middle cost three.

But before conceding to DP I would check for the two middle-ground families, because they catch a lot of problems that look like this.

Greedy with an undo. Course Schedule III: sort by deadline, take everything, **and when you overrun, remove the longest course you already took.** That removal is the exchange argument executed — **given that I must drop one, dropping the longest frees the most time at the same cost of one course.**

Regret greedy. Minimum Refuelling Stops: **do not decide at each station**, because you cannot yet know how far you must go. **Drive past everything, bank what you passed, and when you run dry take the biggest tank you went by** — deferring the decision until hindsight is available.

Both use a heap, and the heap is the tell. If a problem feels greedy but the choice is sometimes clearly wrong, **try deferring or undoing it before building a table.** If neither works, it is DP, and that is a fine place to end up.”

The model answer

“Here is a problem you have not seen. Talk me through it: given courses with a duration and a deadline, starting on day one, take as many as you can.”

“Let me decide the technique before I write anything.

Is there an obvious best-looking choice? Something about deadlines — **and I can justify the ordering straight away.** Sort by deadline, **because a course you can take later you can also take now**, so attempting tight deadlines first can never hurt.

Now can I break the naive greedy — sort by deadline and take anything that fits? Let me try three. `[[5,5],[4,6],[2,6]]`. Take `[5,5]`: five days used, deadline five, fine. Then `[4,6]` needs day nine — too late. `[2,6]` needs day seven — too late. So one course.

But `[4,6]` and `[2,6]` together take six days and both meet their deadlines. The answer is two.

So greedy is right about the ordering and wrong about the commitment. And that is a specific diagnosis rather than ‘greedy fails’ — **which tells me to try the undo family before reaching for DP.**

Here is the fix. Sort by deadline. **Take every course as you meet it.** When the running total exceeds the current deadline, **remove the longest course you have already taken.**

And the justification is the exchange argument, made executable: given that I must drop exactly one course, **dropping the longest frees the most days at exactly the same cost of one course.** There is never a reason to drop a shorter one. **So the count is never worse and the remaining budget is never tighter.**

A max-heap of durations gives me the longest in $\log n$. In Python that is `heapq` with negated values, and I will comment the sign because it is genuinely easy to misread.

The answer is the heap’s size — every push that survived is a course kept.

Cost: $O(n \log n)$. Sort, then n pushes and **at most n pops in total, not n pops per step** — you cannot pop more than you pushed. $O(n)$ space for the heap.

Two things I would check before saying I am done.

The comparison is `total > deadline`, not `>=`. A course finishing exactly on its deadline is fine, and `>=` throws it away — **on that same `[[5,5],[4,6],[2,6]]` it gives one instead of two.** One character.

And I would run it against a brute force — every subset in every order, on inputs of six or fewer — for a few hundred random cases. **Because a wrong sort key here is completely silent.** Sorting by duration instead of deadline gives two on `[[1,5],[2,10],[6,8]]` where the answer is three, **and nothing errors and every tidy example still passes.**

That check is the only evidence I have that the greedy is right, other than the exchange argument — and I would rather have both.“

9 Recall card

The decision procedure, four questions, under two minutes. (1) Can I name the best-looking choice in one sentence? (2) Can I build a counter-example in thirty seconds? (3) Can I state the **exchange argument concretely** — the actual swap, not its shape? (4) If 2 or 3 failed, **it is DP — and saying so with the counter-example is a correct answer.**

The exchange argument in thirty seconds: *take any optimal solution; swap my greedy choice in for its first choice; mine finishes no later, so nothing that followed can conflict; same size, still valid; induct.* **This is what “why does that work?” wants.**

The sort-key table, and the rule underneath it — sort so the greedy choice costs the least future. **END** for most non-overlapping. **START** for merging. **EVENTS** $(+1/-1)$ for how-many-at-once. **NO SORT**, `0(n)` for reachability — the frontier collapses to one number. **DEADLINE** + **heap** for fitting deadlines. **POSITION** + **heap** for regret greedy.

Two middle-ground families, and the heap is the tell. Greedy with an undo (Course Schedule III: take everything, drop the longest on overrun — dropping the longest frees the most time at the same cost of one course). **Regret greedy** (refuelling: never decide at a station, drive past, buy the biggest tank you passed once you know you needed it). **Try these before reaching for DP.**

Greedy survives when the choice affects only what is still available; it dies when the choice carries a value you are optimising. Jump Game is `0(n)` greedy; jump game with a cost per landing is DP.

The wrong sort key is SILENT and passes every tidy example. By start instead of end: keeps 1 not 2. By duration instead of deadline: 2 not 3. `>=` instead of `>`: 1 not 2. `<` instead of `<=` in refuelling: `-1` not 2. **Your only evidence is a brute-force check**

on small random inputs; your only argument is the exchange. Have one, and say which.

1 What this is, and why they ask it

This is the last day of the case studies, and it is not another case study. **It is the framework** — the thing that lets you take a prompt you have never seen to a defended architecture in forty-five minutes.

Because that is the actual test. **They will not ask you to design Twitter.** They know you have read the Twitter answer. **They will ask you to design something adjacent to it**, or something from their own product, **and what they are measuring is whether you have a method or a memory.**

A method looks like this: five minutes on requirements, five on the arithmetic, five on the interface and the data, fifteen on the architecture, ten on one deep dive, five on failure. Every prompt, every time.

And the single most common failure has nothing to do with knowledge. It is starting to draw boxes in minute two. The candidate who names a message queue before anyone has said how many users there are **has told the interviewer that they are recalling, not designing.**

They ask it because **the framework is what transfers.** A candidate who can be handed an unseen prompt and produce structure — requirements, numbers, the one bottleneck that matters, an honest trade — **is the same candidate on the job**, where every problem is unseen.

By the end of this lesson you have the minute-by-minute clock, the four questions to open with, the back-of-envelope numbers to know cold, the recurring toolkit, and **a full mock on a prompt this course has never covered**, worked through end to end.

2 The story

Shakuntala had been doing weddings for twenty-two years and she had one rule that annoyed everybody who walked in.

She would not talk about the food.

A man came in on a Thursday in a very good shirt, sat down, and started straight in on the menu. Two kinds of rice. Paneer done the way they do it at that place near the station. A sweet counter with three men behind it.

He had it all worked out and he wanted a price.

She let him finish, and then she asked him how many people.

“Four hundred. Maybe five.”

“Which is it?”

He said he would find out.

Then she asked when they eat. Not the date — the hour. He said eight, and she said eight or nine thirty, because at a wedding it is never eight, and if it is nine thirty the rice sits for ninety minutes and half of what he had just described cannot be made at all.

Then where. Then whether there was water at the place, and shade, and how far a van could get to the back of it.

And then the question the man did not expect, which was: **“What is the one thing that must not go wrong?”**

He thought about it and said that his mother-in-law was coming and there had to be enough of the sweet.

Shakuntala wrote one number on the board behind her — the hour they would actually eat — and circled it.

“Now I can tell you about food.”

He said, a little stiffly, that he had come in with the menu already decided.

“Everybody comes in with the menu already decided,” she said. **“And everybody who fails at this trade fails the same way. They start with it.”**

“Tell me how many people there are and when they eat, and I can build you a menu. Tell me the menu, and I know nothing at all.”

3 The idea in plain English

Shakuntala’s rule is the framework. **How many, when, and what must not go wrong — before a single box gets drawn.**

Here is the clock. **Forty-five minutes, six blocks, and the discipline is announcing each one out loud** so the interviewer knows where you are and can redirect you cheaply.

0-5	REQUIREMENTS	what it does, what it must not do, and what I am explicitly leaving out
5-10	THE NUMBERS	users, reads/sec, writes/sec, storage/day → and the ONE number that drives the design
10-15	INTERFACE + DATA	the three or four calls, and the data model
15-30	ARCHITECTURE	the boxes, in the order requests flow
30-40	DEEP DIVE	one thing, in real detail – theirs or mine
40-45	FAILURE	what breaks, what I would measure, what I would do with another hour

The first five minutes are worth more than the last twenty, and they are the ones candidates skip.

The four questions to open with

Ask these before anything else. They take ninety seconds and they change the design.

One: who uses it and how many of them? “Everyone” is not an answer — a thousand users and a hundred million are different systems, and the interviewer will tell you if you ask.

Two: what is the read-to-write ratio? This is the single most decision-relevant number in system design. Read-heavy means caching, replicas, fan-out on write, denormalisation. Write-heavy means the opposite of all four. Everything downstream depends on it.

Three: what must not go wrong? Shakuntala’s question. “We may lose a like” and “we may not lose a payment” produce different systems from the first box onwards. Ask it explicitly.

Four: what am I not building? Say what is out of scope and get agreement. “I am not doing search, analytics, or the admin panel.” This is not laziness — it is the only way to finish, and interviewers almost always agree.

The estimation ladder

Every back-of-envelope in system design is the same four steps, in this order:

1. USERS	daily active users
2. ACTIONS	per user, per day
3. RATE	divide by 86,400 → per second then multiply by 10 for peak
4. SIZE	per action, in bytes → per day, per year

Round aggressively. 86,400 seconds a day is **100,000**. A million times a thousand is **a billion**. **Nobody wants three significant figures**, and precision here reads as not knowing what matters.

And say the conclusion, not the number. “Twelve hundred writes a second — that is one ordinary database, so writes are not the problem here.” The number without the conclusion is arithmetic; with it, it is design.

The numbers worth knowing cold

seconds in a day	~100,000 (86,400, rounded)
1 million/day	~12/second
1 billion/day	~12,000/second
a tweet, a comment	~200 bytes - 1 KB
a photo	~200 KB - 2 MB
a minute of video (1080p)	~50 MB
one commodity database	~5,000 writes/second ~50,000 reads/second with replicas
one cache node	~100,000 reads/second
one machine, in memory	~64-256 GB
memory read	~100 ns
SSD read	~100 microseconds (1,000x memory)
disk seek	~10 ms (100,000x memory)
same-datacentre round trip	~0.5 ms
cross-continent round trip	~150 ms (the speed of light; unfixable)

The last one is the one to have ready, because it is the only number in the list that no amount of engineering will improve.

The toolkit

There are about a dozen building blocks and every case study is a rearrangement of them. Knowing *when* each one is the answer is the skill:

LOAD BALANCER	spread requests; health-check; the front door
CDN	static and semi-static content, near the user
CACHE	read-heavy, tolerant of slightly stale data
QUEUE	decouple; absorb bursts; retry; at-least-once
REPLICATION	read scale and durability, at the cost of staleness
SHARDING	write scale and storage, at the cost of cross-shard anything
CONSISTENT HASHING	shard without reshuffling everything on a change
BLOB STORE	big immutable things; never in the database
SEARCH INDEX	text queries the main store cannot answer
STREAM PROCESSOR	aggregate on the way in, not at read time
RATE LIMITER	protect everything behind it
IDEMPOTENCY KEY	the answer to every at-least-once delivery

And the recurring trade-offs, which are what “why?” is asking about:

fan-out on WRITE vs on READ	→ read-heavy? write. celebrities? read.
strong vs eventual consistency	→ what breaks if it is 200 ms stale?
SQL vs NoSQL	→ do you need transactions across rows?
sync vs async	→ does the user have to wait for it?
push vs pull	→ how many recipients, how urgent?
precompute vs compute on read	→ read:write ratio decides it

What senior sounds like

Three habits separate a strong answer from a complete one.

Volunteering the failure mode. Do not wait to be asked what happens when the cache dies. **Say it:** “if this cache goes cold, every request hits the database at once and takes it down — **so I would warm it on deploy and add a request-collapsing layer.**”

Giving the number with the trade. Not “we would cache it” but “**we would cache it; at a 90% hit rate that turns 50,000 reads a second into 5,000, which one database handles — and the cost is up to sixty seconds of staleness, which for a follower count is fine.**”

Saying what you do not know. “I have not built one, but I would expect X, and I would verify it by measuring Y.” **This is a strong answer.** Confident invention is the weakest thing you can do in the room, and it is always audible.

4 The picture

The clock, and what each block produces:

```
0-5    REQUIREMENTS
      | functional: 3-5 bullet points, no more
      | non-functional: availability, consistency, latency
      | OUT OF SCOPE: say it, get agreement
      v
5-10   THE NUMBERS
      | users → actions → per second → bytes
      | THE OUTPUT IS ONE SENTENCE:
      | "this is a <read-heavy / write-heavy / storage> problem"
      v
10-15  INTERFACE + DATA
      | 3-4 calls with their arguments
      | the tables, and what is sharded by what
      v
15-30  ARCHITECTURE
      | boxes in REQUEST-FLOW order, not importance order
      | say why each box exists as you draw it
      v
30-40  DEEP DIVE
      | ONE thing. Offer two, let them pick.
      v
40-45  FAILURE + WRAP
      | what breaks, what I would measure,
      | what I would do with another hour
```

THE MOST COMMON FAILURE IS DRAWING BOXES IN MINUTE TWO.
It tells the interviewer you are recalling, not designing.

The one number that decides everything:

READ : WRITE RATIO

1000:1 (social feed, product page, news)

- CACHE everything
- read REPLICAS
- FAN OUT ON WRITE (precompute the answer)
- DENORMALISE freely
- eventual consistency is almost always fine

1:1 (chat, collaborative editing)

- caching buys little
- the write path IS the design

1:1000 (metrics, logs, click events, sensor data)

- caching is USELESS
- batch and compress on the way in
- AGGREGATE at write time; never store raw at read scale
- a time-series or columnar store, not a row store

ASK FOR THIS NUMBER IN THE FIRST TWO MINUTES.

Every box you draw afterwards depends on it.

The estimation ladder, worked:

"Design a photo sharing app"

USERS	100,000,000 daily active
ACTIONS	2 uploads/day, 100 views/day
RATE	uploads: $200,000,000 / 100,000 \text{ s} = 2,000/\text{second}$ views: $10,000,000,000 / 100,000 \text{ s} = 100,000/\text{second}$
SIZE	PEAK: $\times 10 \rightarrow 20,000$ and $1,000,000$ photo ~1 MB storage: $200,000,000 \times 1 \text{ MB} = 200 \text{ TB/DAY}$ $= 73 \text{ PB/year}$

THE SENTENCE: "50:1 read-heavy, and the storage is the real problem, not the compute. Photos go in a blob store behind a CDN; the database holds only metadata."

THAT SENTENCE is what the five minutes was for.

The toolkit, as a decision table:

SYMPTOM

same data read many times
large static files, global users
traffic spikes, slow work
reads exceed one machine
writes or storage exceed one machine
adding a machine reshuffles all keys
big immutable blobs
"find posts containing X"
counting at read time is too slow
one client can hurt everyone
a retry might do it twice

REACH FOR

CACHE
CDN
QUEUE
REPLICAS
SHARDING
CONSISTENT HASHING
BLOB STORE + CDN
SEARCH INDEX
STREAM PROCESSOR
RATE LIMITER
IDEMPOTENCY KEY

5 How it actually works

A full mock, on a prompt this course has not covered. Read the prompt, set a clock at forty-five minutes, and do it yourself before reading on.

"Design an online examination platform. Schools across the country run exams on it. When an exam starts, everyone taking it starts at the same moment."

Minutes 0–5 · Requirements

Say the four questions out loud.

"Four things before I draw anything.

How many students, and are they concurrent? I will assume **ten million students in a single national exam, all starting at 10:00 exactly** — and I want to flag straight away that the simultaneity is the whole problem, not the volume.

What is the read-to-write ratio? Unusual here: **there is one enormous read at the start — the question paper — and then a steady stream of writes for three hours.** So it is two systems, not one, and I will design them separately.

What must not go wrong? I will assume: **an answer, once written by a student, is never lost, and nobody sees the paper before 10:00.** Those two are non-negotiable and everything else bends around them.

What am I not building? No question authoring, no marking, no proctoring, no analytics. **Just: deliver the paper, accept answers, submit.**"

Functional: start an exam at a fixed time; fetch the paper; save answers continuously; submit.

Non-functional: durability of answers above everything; the paper is secret until the start instant; **degrade gracefully on a bad connection** — a student on a village link must not lose an hour of work.

Minutes 5–10 · The numbers

The paper delivery, first, because it looks like the hard part:

```
10,000,000 students x 500 KB paper = 5 TB
delivered within 60 seconds of 10:00:00

5 TB / 60 s = 83 GB/second = 667 Gbps
```

Say the conclusion: that is a content-delivery problem, not a compute problem — and 667 Gbps in one minute from an origin is not something you build.

Then the answers:

```
autosave every 30 seconds, 10,000,000 students:
  10,000,000 / 30 = ~333,000 writes/second
  at ~2 KB each = ~667 MB/second

over a 3-hour exam:
  333,000 x 10,800 s = ~3.6 BILLION writes
```

And the submission spike:

```
most students submit in the final two minutes:
~5,000,000 / 120 s = ~42,000 submissions/second
```

The sentence: “This is a burst problem twice — once at the start and once at the end — with a sustained write load in between. Nothing here is read-heavy, so caching barely features.”

Minutes 10–15 · Interface and data

```
GET  /exam/{id}/paper      → the encrypted paper (available from 09:00)
GET  /exam/{id}/key       → the decryption key (available from 10:00:00)
POST /exam/{id}/answers   → {question_id, answer, version, client_time}
POST /exam/{id}/submit    → finalise
```

The two-call split for the paper is the central design decision, and it is worth stating as one.

```
CREATE TABLE answers (
  student_id BIGINT,
  exam_id BIGINT,
  question_id INT,
  answer TEXT,
  version INT,          -- monotonic per student, for ordering
  updated_at TIMESTAMPTZ,
  PRIMARY KEY (student_id, exam_id, question_id)
);
-- sharded by student_id: every write touches exactly one shard,
-- and no query ever crosses students
```

Sharding by `student_id` is free of cross-shard problems, because **no operation in this system ever spans two students**. Say that — it is the reason the sharding key is obvious, and obvious sharding keys are worth naming.

Minutes 15–30 · The architecture

Paper delivery, and the move that makes it trivial:

Encrypt the paper and push it to the CDN an hour early. Every student downloads it before 10:00 — **the 5 TB is spread over an hour instead of a minute**. At 10:00:00 exactly, **release only the decryption key**.

5 TB in 60 seconds	→ 667 Gbps	impossible
5 TB over an hour	→ 11 Gbps	ordinary CDN traffic
the key: 10,000,000 x 100 bytes = 1 GB		
1 GB in 60 seconds	→ 133 Mbps	one machine
→ the burst shrinks by a factor of FIVE THOUSAND		

This is the whole trick, and it generalises: when a huge payload must appear at an instant, pre-position the payload and release something tiny. The same shape as a ticket sale that pre-generates the seat map, or a game launch that pre-downloads and unlocks.

The answer path, and the durability requirement:

```

browser
| 1. writes to LOCAL STORAGE first, always
|   → a dropped connection loses nothing
v
load balancer
|
v
stateless write service
| 2. appends to a DURABLE LOG (Kafka), acks only after
|   the log confirms
v
append-only log, replicated 3x
|
+→ 3. consumer materialises the latest answer per question
|   into the sharded store
|
+→ 4. the log itself is the audit trail and the dispute record

```

Acking from the log rather than from the database is what meets “never lose an answer” — the log is a sequential append, replicated, and it absorbs 333,000 writes a second without difficulty. **The database is allowed to lag.**

And writing to the browser’s local storage first is not a nicety — it is what makes a bad connection survivable, which was a stated requirement. **The client retries from its own copy.**

Submission, and why the spike is not a spike:

Because answers were autosaved throughout, “submit” is a state change on one row, not an upload. Forty-two thousand small state transitions a second across two hundred shards is **two hundred a second per shard.**

Say this explicitly, because it is the payoff of the earlier decision: **the continuous autosave is what turns the end-of-exam spike into nothing.**

Minutes 30–40 · The deep dive

Offer two and let them choose. “I could go deep on **keeping answers consistent when the same student has two tabs open**, or on **what happens when a whole region loses connectivity mid-exam**. Which is more interesting?”

Take the second, since it is the harder one:

“A region goes dark at 11:00, an hour in. Two hundred thousand students.

Because every client holds its own answers locally, nothing is lost — that was the point of writing locally first.

When the region comes back, two hundred thousand clients reconnect at once and replay their unsent writes. That is a thundering herd, and it arrives at exactly the moment the system is recovering.

Three things. Jittered reconnect — clients wait a random interval before retrying, so the return is spread over minutes rather than arriving in one second. **Exponential backoff** on failure. **And the writes are idempotent**, keyed by `(student_id, question_id, version)`, so replaying them is safe.

The harder question is the exam clock. Do those students get their hour back? **That is a policy decision, not a technical one**, and I would raise it rather than assume — but technically it means the deadline is **per student, stored, and adjustable**, rather than a single global timestamp. **Which is a data-model decision that has to be made on day one**, so it is worth deciding now even though it looks like a product question.”

Minutes 40–45 · Failure and wrap

Say what breaks, unprompted:

- **The key-release endpoint is a single point of failure at 10:00:00.** Serve it from the CDN too, with a hard cache expiry set to the start instant.
- **A shard being unavailable loses writes for its students.** The log in front means they are only delayed, not lost.
- **Clock skew on clients** means “the exam started” is not simultaneous. The deadline is enforced **server-side**, and the client clock is advisory.

Say what you would measure: write acknowledgement latency at the 99th percentile, unacknowledged writes per client, and **the count of students with no successful write in the last two minutes** — which is the metric that would actually detect a regional outage.

Say what you would do with another hour: proctoring, marking, and **the hardest remaining problem, which is preventing the paper leaking through a compromised client** — and that this is not solvable in software alone.

The real systems

CDN (CloudFront, Cloudflare)	pre-positioned encrypted papers
Kafka	the durable write log
Cassandra / DynamoDB	the sharded answer store
Postgres	exam metadata, students, schedules
Redis	per-student session state, exam clocks
S3	the immutable archive for disputes

6 The numbers

The paper delivery, and what pre-positioning buys.

NAIVE – everyone downloads at 10:00:00
 $10,000,000 \times 500 \text{ KB} = 5 \text{ TB}$ in 60 seconds
 $= 83 \text{ GB/second} = 667 \text{ Gbps}$
→ not something you build; it is a bad afternoon for a CDN

PRE-POSITIONED – encrypted paper available from 09:00
 $5 \text{ TB over } 3,600 \text{ seconds} = 1.4 \text{ GB/second} = 11 \text{ Gbps}$
→ ordinary CDN traffic

then at 10:00:00, release the key:
 $10,000,000 \times 100 \text{ bytes} = 1 \text{ GB}$ in 60 seconds
 $= 17 \text{ MB/second} = 133 \text{ Mbps}$
→ ONE MACHINE

RATIO: $5 \text{ TB} \rightarrow 1 \text{ GB}$. A factor of 5,000.

The general form: when a huge payload must appear at an INSTANT, pre-position the payload and release something tiny.

The write load.

autosave every 30 s, 10,000,000 students:
 $333,000 \text{ writes/second}$ sustained, for 3 hours
 $\times 2 \text{ KB} = 667 \text{ MB/second}$

sharded by student_id across 200 shards:
 $1,667 \text{ writes/second}$ per shard
→ comfortably one ordinary node each

the durable log absorbs it undivided:
a sequential append at 667 MB/second is well within
a modest Kafka cluster

Total volume, and why you do not keep everything.

every autosave stored:

$333,000/s \times 10,800\text{ s} = 3.6\text{ billion writes}$
 $\times 2\text{ KB} = 7.2\text{ TB per exam}$

only the LATEST answer per student per question:

$10,000,000\text{ students} \times 40\text{ questions} \times 2\text{ KB} = 800\text{ GB}$

→ the materialised store is 800 GB

→ the LOG is 7.2 TB, kept 90 days for disputes, then dropped

and if the client sends only CHANGED answers – perhaps 20% of autosave ticks – both numbers fall fivefold: 67,000 writes/second and a 1.4 TB log.

That optimisation is one client-side comparison, and it is worth naming as a decision rather than leaving implicit.

The submission spike, and why it is not one.

5,000,000 students submit in the final 120 seconds
= 42,000 submissions/second

BUT the answers are already saved. "Submit" is:

UPDATE exams SET submitted_at = now() WHERE student_id = ?

~50 bytes, one row, one shard

$42,000 / 200\text{ shards} = 210\text{ writes/second per shard}$

→ THE CONTINUOUS AUTOSAVE IS WHAT TURNS THE END-OF-EXAM SPIKE INTO NOTHING.

Without it, 5,000,000 x 80 KB of answers in 120 seconds
= 3.3 GB/second of uploads at the worst possible moment –
and any student whose upload fails has lost three hours.

Compare that last pair honestly, because it is the strongest argument in the design: **the same total data either arrives smoothly over three hours or all at once in two minutes**, and the design choice is which.

The estimation ladder in general, so it is reusable.

1 million actions/day	= ~12/second
1 billion actions/day	= ~12,000/second
peak	= ~10x average
1 KB x 1 million/day	= 1 GB/day = ~365 GB/year
1 MB x 1 million/day	= 1 TB/day = ~365 TB/year
one database	~5,000 writes/second
one cache node	~100,000 reads/second
one machine's memory	~64-256 GB

Round 86,400 to 100,000. Nobody wants three significant figures, and precision here reads as not knowing what matters.

7 The trade-offs

How to spend the forty-five minutes. More time on requirements means less on the deep dive. **The requirements time is the better investment** — a beautifully detailed design of the wrong system scores nothing, and the interviewer will happily spend fifteen minutes on the deep dive if you leave time for it. **The failure mode is the reverse: thirty minutes of architecture and no time to say what breaks.**

Breadth against depth. Covering everything shallowly is safe and unmemorable. **Going deep on one thing risks going deep on the wrong thing** — which is exactly why you **offer two and let them choose**. That one sentence converts a gamble into a collaboration.

Volunteering weaknesses against defending the design. Saying “this cache going cold takes the database down with it” **sounds like an admission and reads as seniority**, because the alternative is the interviewer finding it. **The cost is that you must then have the mitigation ready** — an unmitigated weakness you raised yourself is worse than one nobody mentioned.

Naming specific technologies against staying generic. “Kafka” is more credible than “a message queue”, **and it invites a question about Kafka you may not be able to answer**. Name it when you can defend it, stay generic when you cannot — **and never name something purely to sound current**.

Following their hints against your plan. When an interviewer says “interesting, but what about X”, **X is the thing they want**. Abandon your plan immediately. **Candidates who finish their prepared answer while the interviewer waits are marked down**, and it is the most common way a strong candidate loses a round.

Admitting ignorance against covering. “I have not built one, but I would expect X and I would verify it by measuring Y” **is a strong answer.** Confident invention is the weakest possible move **and it is always audible** — the follow-up question that exposes it is the one thing an interviewer never forgets.

When is this framework wrong? For a prompt with a genuine novelty in it, the arithmetic block can mislead — if the interesting part is a consistency requirement rather than a scale one, five minutes of multiplication is five wasted minutes. **Read the prompt for where the difficulty actually is,** and if it is not scale, say so and spend the time elsewhere.

8 In the interview

How it gets asked

- “*Design a system you have never seen before. Forty-five minutes.*”
- “*How would you approach a design problem you know nothing about?*”
- Any unseen prompt — an exam platform, a warehouse tracker, a queue for a temple.
- “*You have ten minutes. Just the high level.*” — the same framework, compressed.

The first ninety seconds

Said on any unseen prompt, before drawing:

“Let me ask four things before I draw anything, because the answers change the design and I would rather not guess.

One: how many users, and how many at once? A thousand and a hundred million are different systems.

Two: what is the read-to-write ratio? This is the number I most want, **because everything downstream depends on it.** Read-heavy means caching, replicas, precomputing. Write-heavy means **none of those help and the write path is the design.**

Three: what must not go wrong? ‘We may lose a like’ and ‘we may not lose a payment’ produce different systems **from the first box onwards.**

Four: what am I explicitly not building? I would like to agree that up front — **it is the only way I finish in forty-five minutes.**

Then here is how I will use the time, so you can redirect me cheaply if I am on the wrong thing.

Five minutes on requirements. Five on the arithmetic, so we know which part is actually hard. Five on the interface and the data model. Fifteen on the architecture. Ten going deep on one piece — and I will offer you two and let you pick. Five at the end on what breaks and what I would measure.

The reason I do the arithmetic before the boxes is that it tells me which problem I am solving. If it comes out at a thousand writes a second, most of what I might have drawn is unnecessary, and saying that is worth more than drawing it.”

The follow-ups

“How do you approach a problem you know nothing about?”

“The same way every time, and the method is the point — because they will not ask me to design Twitter. They know I have read that answer. They will ask for something adjacent, and what is being measured is whether I have a method or a memory.

I start with the four questions: how many users, what is the read-to-write ratio, what must not go wrong, and what am I not building.

Then the arithmetic, and it produces exactly one sentence: ‘this is a read-heavy problem’, or ‘the storage is the real constraint’, or ‘this is two bursts with a steady load in between’. **That sentence is what the five minutes was for.** The numbers on their own are just multiplication.

Then the interface and the data model, because they force the requirements to become concrete. You cannot write down four calls without discovering an ambiguity you skated over.

Then the architecture, drawn in request-flow order rather than importance order, saying why each box exists as it appears. **A box I cannot justify does not get drawn.**

Then one deep dive, and I offer two options — that turns a guess about what interests them into a question.

And I always leave five minutes for failure, because volunteering what breaks is the difference between a complete answer and a good one.

The single most common mistake is drawing boxes in minute two. Naming a message queue before anyone has said how many users there are **tells the interviewer I am recalling, not designing** — and once they think that, everything after it is read as recall too.”

“Walk me through the arithmetic on something.”

“Take a photo-sharing app. **Four steps, and I round hard.**

Users: a hundred million daily active. **Actions:** two uploads and a hundred views each per day.

Rate: two hundred million uploads a day. **I round 86,400 seconds to a hundred thousand** — so two thousand uploads a second. Ten billion views a day is **a hundred thousand a second. Peak is ten times average:** twenty thousand and a million.

Size: a photo is about a megabyte, so **two hundred terabytes a day** — seventy-three petabytes a year.

Now the sentence, which is the actual output: ‘It is fifty-to-one read-heavy, and the storage is the real problem, not the compute.’

And that sentence decides three things immediately. Photos go in a blob store behind a CDN, **never in the database.** The database holds only metadata — a few hundred bytes a photo, so a few hundred gigabytes a day, **which is an entirely different problem from two hundred terabytes.** And at fifty-to-one, **caching and precomputing are worth a great deal.**

I would round 86,400 to 100,000 out loud rather than quietly, because the alternative is spending thirty seconds on long division while the interviewer waits. **Three significant figures here reads as not knowing what matters.**

And if the arithmetic comes out small, I say so and move on. ‘That is twelve hundred writes a second — **one ordinary database, so this is not where the difficulty is.**’ **Discovering that a problem is easy is a result,** and it buys me time for the part that is not.”

“What separates a good answer from a complete one?”

“Three habits, and none of them is knowledge.

Volunteering the failure mode. Not waiting to be asked what happens when the cache dies. **‘If this cache goes cold, every request hits the database at once and takes it down — so I would warm it on deploy and collapse duplicate requests.’** Saying it first reads as seniority; **having it found for me reads as a gap.**

Giving the number with the trade. Not ‘we would cache it’, but **‘we would cache it; at a ninety percent hit rate that turns fifty thousand reads a second into five thousand, which one database handles — and the cost is up to sixty seconds of staleness, which for a follower count is fine.’** Same decision, and one of them is a design.

Saying what I do not know. ‘I have not run one at this size, but I would expect X, and I would verify it by measuring Y.’ **Confident invention is the weakest thing I can do in the room, and it is always audible.**

And one more that is really about listening. When an interviewer says ‘interesting, but what about X’, **X is the thing they want.** I abandon my plan immediately. **Finishing a prepared answer while the interviewer waits is the most common way a strong candidate loses a round, and it is entirely avoidable.”**

The model answer

“Design an online examination platform. Ten million students, one national exam, everyone starts at ten o’clock.”

“Four questions first.

The read-to-write ratio here is unusual and it shapes everything: one enormous read at the start — the paper — and then three hours of steady writes. So it is two systems, and I will design them separately.

What must not go wrong: an answer, once written, is never lost; and nobody sees the paper before ten. Everything else bends around those two.

Out of scope: authoring, marking, proctoring, analytics.

The arithmetic, and it names the real problem. Ten million students times a five-hundred-kilobyte paper is five terabytes, wanted inside a minute — six hundred and sixty-seven gigabits a second. That is not something you build.

So: pre-position and release a key. Push the encrypted paper to the CDN an hour early. Everyone downloads it before ten — five terabytes over an hour is eleven gigabits a second, ordinary CDN traffic. At ten exactly, release only the decryption key: ten million times a hundred bytes is one gigabyte, which is one machine. A factor of five thousand, and it is the central decision in the design.

The write path, and the durability requirement. Autosave every thirty seconds is three hundred and thirty thousand writes a second, sustained for three hours.

The client writes to its own local storage first — that is what makes a bad connection survivable, which was a stated requirement. Then to a stateless write service, which appends to a durable replicated log and acks from the log, not from the database. A sequential append absorbs that rate; the database is allowed to lag behind it. The log is also the audit trail for disputes.

Sharded by student id across two hundred shards — sixteen hundred writes a second each. And the sharding key is obvious because no operation in this system ever spans two students, which is worth saying out loud.

Now the end-of-exam spike, and this is the payoff. Five million students submitting in the last two minutes is forty-two thousand a second — but the answers are already saved, so submit is a one-row state change, about two hundred a second per shard. Without the autosave it would be three point three gigabytes a second of uploads at the worst possible moment, and any student whose upload failed would have lost three hours. Same data — it either arrives smoothly over three hours or all at once.

For the deep dive I could go into two tabs open on the same exam, or a region losing connectivity mid-exam. Which would you prefer?

What breaks. The key endpoint is a single point of failure at exactly ten o'clock — **serve it from the CDN with an expiry set to the start instant.** Client clocks are skewed, so **the deadline is enforced server-side and the client clock is advisory.** And when a region reconnects, **two hundred thousand clients replay at once** — so **jittered reconnect, exponential backoff, and idempotent writes keyed by student, question and version.**

What I would measure: **acknowledgement latency at the ninety-ninth percentile, unacknowledged writes per client, and the number of students with no successful write in the last two minutes** — that last one is what would actually detect a regional outage.

And one thing I would flag as a decision rather than assume. If students lose an hour, **do they get it back?** That is a policy question, **but it decides whether the deadline is one global timestamp or a stored per-student value** — a data-model choice that has to be made on day one. **So I would want an answer now, even though it looks like a product question.**

9 Recall card

Six blocks, forty-five minutes, announced out loud: **0–5 requirements** (functional, non-functional, and what is explicitly out of scope), **5–10 the arithmetic**, **10–15 interface and data model**, **15–30 architecture in request-flow order**, **30–40 one deep dive** — **offer two and let them pick**, **40–45 failure and what you would measure.** **The most common failure is drawing boxes in minute two**, which says you are recalling rather than designing.

Four opening questions: how many users and how many at once; **the read-to-write ratio** (the single most decision-relevant number — it decides caching, replicas, fan-out and denormalisation); what must not go wrong; and what you are **not** building.

The estimation ladder: **users → actions → per second ($\div 100,000$, $\times 10$ for peak) → bytes.** **Round hard** — three significant figures reads as not knowing what matters. **The output is ONE SENTENCE**, not a number: *“read-heavy 50:1, and storage is the real problem.”* **Discovering a problem is easy is a result.**

Numbers cold: 1M/day $\approx$ 12/s; 1B/day $\approx$ 12,000/s; one database $\sim$ 5,000 writes/s; one cache node $\sim$ 100,000 reads/s; memory 100 ns, SSD 100 μ s, disk 10 ms; **cross-continent 150 ms and no engineering fixes it.**

The move worth stealing from the mock: when a huge payload must appear at an INSTANT, pre-position it and release something tiny. 5 TB in a minute becomes 5 TB over an hour plus a 1 GB key — a factor of 5,000. **And continuous autosave turns an end-of-exam spike into nothing: the same data either arrives smoothly or all at once, and that is the design choice.**

Three habits that read as senior: volunteer the failure mode before you are asked (with the mitigation ready); **give the number WITH the trade** (“90% hit rate turns 50,000 reads into 5,000, at up to 60 seconds of staleness”); and say what you do not know. **And when they say “interesting, but what about X” — X is what they want. Abandon your plan.**

DSA topic: Greedy and intervals revision and mock round **System design topic:** High-level design revision and full mock

Code these, in this order

One rule for the whole set, and it is the rule of the phase: **before you write a line, say the four questions out loud.** Can I name the best-looking choice in one sentence? Can I break it in thirty seconds? Can I state the exchange argument concretely? If not, is it DP? **Solving one of these silently and correctly is worth less than solving it out loud and getting it wrong.**

#	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
1	Non-overlapping Intervals	LeetCode 435 (Medium)	The sort key, and “remove fewest” = “keep most”.
2	Minimum Number of Arrows to Burst Balloons	LeetCode 452 (Medium)	The same greedy in disguise — do you see it?
3	Course Schedule III	LeetCode 630 (Hard)	Greedy with an undo, and why the longest.
4	Minimum Number of Refueling Stops	LeetCode 871 (Hard)	Regret greedy — deferring the choice to hindsight.
5	IPO	LeetCode 502 (Hard)	Two heaps, and a greedy that needs both orderings.
6	Candy	LeetCode 135 (Hard)	Two greedy passes, and why one pass cannot work.

On problem 1, use the wrong sort key on purpose

Sort by start instead of end. Run on `[[1,100],[11,22],[1,11],[2,12]]` and record the answer against the correct one. **Then run it on `[[1,2],[2,3],[3,4]]` and note that it agrees.** Say in one sentence why that combination is the trap of the whole phase.

On problem 2, notice it is problem 1

Solve it. **Then say precisely which words in the two problem statements differ,** and why the greedy is identical. Check the tie-break — do balloons touching at a point count as overlapping? **Say what changes if you get it backwards.**

On problem 3, find the counter-example first

Before writing anything, find a three-course input where take-if-it-fits gives the wrong answer. **Write it down and say what the correct answer is.** Only then write the heap version.

Then flip `>` to `≥` in the overrun check. **Run it on your counter-example and record what happens.**

On problem 3, sort by duration instead

Keep the heap and change only the sort key. Run on `[[1,5],[2,10],[6,8]]`. **Record both answers**, and say why sorting by deadline is the one that can be justified.

On problem 4, change `<=` to `<`

One character in the inner loop. Run on `target 100, start 10, stations [[10,60],[20,30],[30,30],[60,40]]` and record what it returns. **Say why that particular wrong answer is more dangerous than a crash.**

On problem 4, prove the two loops are linear

They look nested. **Say why they are not**, in one sentence about the pointer. Then instrument it: count the total inner-loop iterations for `n = 10,000` and check it is `n`, not `n2`.

On problem 5, name the two orderings

There is an ordering you sort by and an ordering you pop by, **and they are different fields**. Say which is which and why one heap cannot do both.

On problem 6, try one pass

Attempt it with a single left-to-right pass. **Find the input where it fails and say what information the first pass cannot have.** Then write the two-pass version.

Then the verification drill

For every problem above, write a brute force and check the greedy on a few hundred small random inputs. **Say for each one why your brute force is small enough to be an oracle** — give the input size where it stops being usable.

Then the mock drill

Set a clock. Fifteen minutes for one unseen greedy problem, talking the whole time. Score yourself on four things: did you say the decision procedure out loud, did you name and justify the sort key, did you try a counter-example, did you give the cost unprompted. **Three out of four is a pass.**

The decision-procedure drill

1. Give the four questions in order.
2. Say what each one rules out.
3. Say what you do when question two succeeds.
4. Say why “greedy fails, here is the input” is a correct answer.
5. Give the rule for when greedy survives, in one sentence.

The exchange-argument drill

1. State it in thirty seconds for activity selection.
2. Say what the swap is, concretely.
3. Say what the middle clause must establish.
4. Say what it means if you cannot fill that clause in.
5. Say how the induction finishes it.

The sort-key drill

1. Give all six rows of the table from memory.
2. Give the rule underneath all of them.
3. Say which family needs no sort at all, and why.
4. Give the input where by-start and by-end disagree, with both answers.
5. Say why the wrong key is silent.

The middle-ground drill

1. Name both families and their tell.
2. Give the Course Schedule III argument for undoing the longest.
3. Say what regret greedy defers, and until when.
4. Say why `fuel` doubles as a position.
5. Say when you would still fall back to DP.

The break-it drill

For each, say what happens and whether anything reports it:

1. Sorting intervals by start instead of end.
2. Sorting courses by duration instead of deadline.
3. `>=` instead of `>` in the overrun check.
4. `<` instead of `<=` in the refuelling loop.

5. Pushing `+duration` instead of `-duration`.
 6. No guard on an empty interval list.
-

The clock drill

1. Give the six blocks with their minute ranges.
2. Say what each block produces.
3. Say what the most common failure is, and what it signals.
4. Say which block candidates skip, and why that is the expensive one.

The four-questions drill

1. Give all four.
2. Say which is the most decision-relevant, and name four things it decides.
3. Give the three read:write regimes and what each implies.
4. Say why asking what you are *not* building is not laziness.

The estimation drill

1. Give the four steps of the ladder.
2. Say what you round 86,400 to, and why.
3. Work the photo-sharing example end to end.
4. Give the one sentence that is the actual output.
5. Say what you do when the arithmetic comes out small.

The numbers drill

1. Give per-second rates for 1M/day and 1B/day.
2. Give one database's write rate and one cache node's read rate.
3. Give memory, SSD and disk access times as a ratio.
4. Give the cross-continent round trip and say what is special about it.

The mock drill

1. State the exam platform's two systems and why they are separate.
2. Give the naive paper-delivery arithmetic and the pre-positioned version.
3. Give the ratio between them and state the general rule.
4. Compute the autosave write rate and the per-shard rate.
5. Say why the end-of-exam spike is not a spike, with both numbers.
6. Give three things that break, and what you would measure.

The seniority drill

1. Name the three habits and give an example of each.
 2. Say why volunteering a weakness needs the mitigation ready.
 3. Say what “interesting, but what about X” means.
 4. Say what confident invention costs you.
 5. Say when the framework itself is the wrong tool.
-

Say these out loud

Three questions. Answer each one in two minutes, standing up, without looking at the lesson.

1. *How do you decide whether greedy works?* The four questions in order, the exchange argument stated concretely, the two middle-ground families with their tell, and the rule for when greedy survives.
 2. *Which sort key, and why?* The rule underneath the table, all six rows, the family that needs no sort, and the input where by-start and by-end disagree.
 3. *Design a system you have never seen. Forty-five minutes.* The six blocks, the four opening questions, the estimation ladder ending in one sentence, offering two deep dives, and volunteering what breaks.
-

Before you move on

- [] I can give the four questions of the decision procedure, in order.
- [] I can state the exchange argument concretely in thirty seconds.
- [] I know what it means when I cannot fill in the swap clause.
- [] I know “greedy fails, here is the input” is a correct answer.
- [] I can give the rule for when greedy survives and when it dies.
- [] I can recall all six rows of the sort-key table.
- [] I can give the rule underneath the table in one sentence.
- [] I know reachability needs no sort, and why.
- [] I know the input where by-start and by-end disagree, with both answers.
- [] I know the wrong sort key is silent and passes tidy examples.
- [] I can name both middle-ground families and their tell.
- [] I can justify undoing the longest in Course Schedule III.

- [] I know what regret greedy defers, and until when.
- [] I know `fuel` doubles as a position, and why.
- [] I checked every greedy I wrote against a brute force.
- [] I know where each brute force stops being usable.
- [] I scored myself on the four mock criteria and got three of four.
- [] I can give the six blocks of the clock with their minutes.
- [] I know the most common failure is drawing boxes in minute two.
- [] I can give the four opening questions.
- [] I know the read:write ratio decides four things, and can name them.
- [] I can give the three read:write regimes and what each implies.
- [] I can work the estimation ladder and end with one sentence.
- [] I round 86,400 to 100,000 out loud.
- [] I know what to say when the arithmetic comes out small.
- [] I know the per-second rates, database and cache rates, and access times.
- [] I know the cross-continent round trip and why it is unfixable.
- [] I can give the pre-position-and-release-a-key rule with its ratio.
- [] I can say why the end-of-exam spike is not a spike, with numbers.
- [] I can name the three habits that read as senior.
- [] I know what “interesting, but what about X” means and what to do.
- [] I answered all three questions above out loud.

The nine sections, and what each is for

Every lesson in this book, on both tracks, carries the same nine sections in the same order. This is what each one is for, and how to read it.

1. What this is, and why they ask it

The topic in plain words, and the reason an interviewer spends time on it. This section exists so you never learn something without knowing what it buys you.

2. The story

A scene with a person in it, told in ordinary language, with no technical words at all. Two hundred to four hundred words about matching socks, or a canteen queue, or laying tables.

This is the most important section for a beginner and the easiest to skip. The story gives you a place to put the idea before you have the vocabulary for it. Later, when you are asked the question under pressure, the story is what you will reach for first.

3. The idea in plain English

The story, translated. Every term is defined the first time it appears, and defined terms are set in **bold** so you can find them again. Appendix D collects them all.

4. The picture

A diagram. Arrays, memory and pointers are drawn as ASCII boxes with indices above and values below, because adjacency matters and a box drawing keeps it. Trees, graphs, architectures and request flows are drawn as proper figures.

Every diagram is followed by a line telling you what to notice in it.

5. The code, built step by step

Fragments of ten lines or fewer, each one followed by prose explaining it. Then the complete solution in one block, ready to run.

The full answer is always here. This book does not withhold solutions.

6. What it costs

The arithmetic, done where you can see it.

On the DSA track that means counting the loop iterations out loud: not “this is quadratic” but “the inner loop runs sixty times for each of sixty items, so about three thousand comparisons”. On the system design track it means doing the multiplication: not “that is a lot of storage” but $50\text{M} \times 20 \times 2\text{KB} = 2\text{TB}$.

7. The traps

The mistakes people actually make, with the real error text printed exactly as it appears on screen. When your own program breaks this way, you will recognise it.

8. In the interview

The point of the whole document.

Real interviewer phrasings. A script for the first ninety seconds, because that is when candidates either take control of the question or lose it. The three follow-ups that usually come next. And a model answer, written out in full, so you have something to measure yours against.

9. Recall card

The two or three sentences that are the entire lesson. Say them aloud, from memory, in full sentences.

If you can do that, you have finished the lesson. If you cannot, you have read it, which is a different thing.

Recall cards

Section 9 of every lesson in this volume, collected. Cover the card, say it aloud, then check. Cards are grouped by the phase they belong to, on both tracks.

Building blocks of big systems

Day 143 · system-design — Bloom filters and probabilistic structures

A Bloom filter is a bit array plus k hashes: “no” is certain, “maybe” is not. No false negatives, some false positives — and the direction is the whole point, because a “no” can be acted on and a “maybe” always needs the real check.

Sizing: $m = -n \ln(p) / (\ln 2)^2$ bits, $k = (m/n) \ln 2$ hashes. For 1%: 9.6 bits per item, 7 hashes — a billion URLs in 1.2 GB instead of 100 GB, and roughly 5 more bits per extra factor of ten in accuracy.

Place it so a false positive costs a cheap extra lookup, never a wrong answer. The canonical use is an SSTable filter in an LSM store: a “no” skips a disk seek, a “maybe” costs one that would have happened anyway, and the file is immutable so deletion never comes up.

It cannot delete, list, or count, and overfilling degrades silently — five times the design capacity takes 1% to 63% with no error. Monitor the fill ratio. Counting or cuckoo filters if you need removal.

Relatives: HyperLogLog — distinct count in ~12 KB at ~0.8% error at any cardinality, and mergeable (PFMERGE) so hours combine into days without double counting. Count-Min Sketch — frequency in fixed memory, never under-estimates, accurate for heavy hitters and noise for the tail.

Day 144 · system-design — Building blocks revision and interview questions

Every component here is a derived copy of something in the database, arranged for a different question. Each one needs a staleness number, a sync mechanism, and a reconciliation job — and the reconciliation job is the part that never makes it onto the diagram.

Three questions place almost anything: does the user need the answer to continue (sync or async)? how stale can it be ($0 \rightarrow$ database; seconds $\rightarrow$ cache/search/stream; hours $\rightarrow$ warehouse)? what is the read-to-write ratio (read-heavy $\rightarrow$ cache and precompute; write-heavy $\rightarrow$ queue, partition, and ask if it must be durable)?

Work $\rightarrow$ task queue (per-message retry, DLQ). Events $\rightarrow$ log (many readers, replay). Postgres FTS to $\sim 1\text{--}5\text{M}$ docs; a replica to ~ 100 GB; SSE unless the client also sends; Redis GEO for anything moving.

The numbers that carry an argument: egress $\$0.09/\text{GB}$ vs storage $\$0.023$ (40 TB $\Rightarrow$ $\$54,000$ vs $\$900$); row-store scan 16 GB vs 320 MB needed ($\sim 100\times$); polling 176 Mbps vs WebSocket 40 KB/s; Bloom 1.2 GB vs 100 GB; time-series 16 B $\rightarrow$ 1.5 B per point.

The two failure modes: one component doing every job, and an imported system for a two-room flat. Every addition is a permanent operational cost, so **the burden of proof is on leaving the database.**

Dynamic programming

Day 143 · dsa — What dynamic programming actually is

DP is recursion plus memory. Two conditions: **optimal substructure** (the big answer is built from smaller answers to the same problem) and **overlapping subproblems** (those smaller problems repeat). No overlap means divide-and-conquer, not DP — a cache would never get a hit.

Derive the recurrence in words first, write it as plain recursion, then add memory. Writing the table before the recurrence is how people get stuck. And **say what `dp[i]` means as a full sentence** before the loop — if the state omits something that affects the answer, two subproblems share a cache entry and you get a confident wrong number.

Cost = (number of distinct states) $\times$ (work per state). `n` states $\times$ `0(1)` is `0(n)`; `n·m` states with a `k`-loop inside is `0(n·m·k)`.

Top-down first: same shape as your reasoning, and it only computes reachable states. **Convert to bottom-up for two reasons only:** recursion depth (~ 960 frames in Python), and because the space optimisation becomes obvious.

The numbers: naive stairs at $n=35$ is 18.5 million calls for **35 distinct answers**; memoised it is 35 computations. And each cell reading only a fixed window below it means $O(n)$ space collapses to $O(1)$.

Day 144 · dsa — Fibonacci: memoisation versus tabulation

Same recurrence, two directions. Top-down starts at the answer and recurses to the base cases; bottom-up starts at the base cases and fills forwards. **Identical time** — states $\times$ work per state.

The conversion is five mechanical steps: parameters $\rightarrow$ dimensions; base cases $\rightarrow$ initial values; recursive calls $\rightarrow$ table reads; **work out the loop order**; return `dp[original argument]`. **The loop always runs opposite to the recursion.**

Convert for two reasons: the recursion depth (~ 960 frames, and **memoisation does not help** — the first descent reaches the base before anything is cached), and because the space optimisation becomes visible.

Do not convert when the state space is sparse. Coin change with denominations of 1000 and 5000 at amount 100,000: a table fills 100,001 cells, top-down visits **105**.

Space window = how far back the recurrence reaches. Two variables for $i-1$, $i-2$; one row when only the previous row is read — and then **the iteration direction is part of the correctness**. Optimising throws away the ability to reconstruct the path, so ask whether the value alone is enough first.

Day 145 · dsa — Climbing stairs and the one-dimensional habit

The habit: “what was the last move, and where was I before it?” Enumerate the possible last moves — usually two — and each points at a strictly earlier index. That produces the recurrence for every linear DP.

Say what `dp[i]` means as a full sentence before the loop. It gives you the base cases (the indices the recurrence cannot compute) **and where the answer lives:** `dp[n-1]` for “using everything up to here”, `max(dp)` for “ending exactly here”, something else when the finish is past the end.

Same skeleton, three operators: `+` for counting, `max`/`min` for optimisation, `any` for feasibility — and the operator also tells you what to initialise the table with.

Cost = n states $\times$ the length of the last-move enumeration. Two options $\rightarrow O(n)$; “any earlier index” $\rightarrow O(n^2)$, and **bounding that enumeration** (the last word is at most `L` long) is often a $500 \times$ win for free.

Nearly all of them collapse to $O(1)$ space — a window as deep as the recurrence reaches. And if the answer is $\max(dp)$, carry the running best in a second variable, because the table is gone.

Day 146 · dsa — House robber and the choice at each step

A binary choice per position where taking forbids a neighbour: $dp[i] = \max(\text{money}[i] + dp[i-2], dp[i-1])$ — take, or skip.

$dp[i]$ = the maximum considering houses $0..i$ (not “if I rob i ”), which is what makes $dp[1] = \max(\text{money}[0], \text{money}[1])$ — the **larger**, not the second. Derive base cases from the sentence.

Two counter-examples to have ready. Greedy by value fails on $[900, 1600, 900]$ (1600 vs 1800). Alternating indices fails on $[2, 1, 1, 2]$ (3 vs 4) — **non-adjacent does not mean alternating**.

Circular: run the linear version twice, excluding the last house and excluding the first, and take the max — because every valid selection omits at least one endpoint, and neither run can produce an invalid one. **Guard** $n = 1$.

$O(n)$ time, $O(1)$ space with two variables (which also removes the empty/single guards). Reconstruction needs the **full table** — walk backwards, $dp[i] = dp[i-1]$ means skipped — so it is mutually exclusive with the space optimisation. And **delete-and-earn** is this problem after indexing by value, at $O(n + \max)$.

Day 147 · dsa — Finding the state: the hardest part of DP

The state is what identifies a subproblem, and the test is one sentence: *given only the state, can I decide what happens next without knowing anything else about how I got here?* If no, whatever is missing is the next dimension.

An incomplete state fails silently — two different situations share a cell, and the answer is not approximate, it is the answer to a different question.

Five things that usually need to be in the state: a **resource** being consumed (capacity, money), a **count** of something limited (transactions, stops), a **mode** you are in (holding, on cooldown), a position in a **second sequence**, and a **set already used** (bitmask, only for $n \leq 22$).

Incomplete and awkward are different. Incomplete needs another dimension; awkward needs the *meaning* changed — “best subarray **ending exactly at** i ” instead of “using elements up to i ”. **Redefining is often better than extending**, and adding a dimension when redefining would do turns $O(n)$ into $O(n^2)$.

Count the state space before writing. Adding a dimension **multiplies**; the practical ceiling is $\sim 10^7$ states in Python; and `dp[i][capacity]` is **pseudo-polynomial** — polynomial in the *value* of `W`, not in its number of digits, which is why knapsack has a DP solution and is still NP-hard.

Day 148 · dsa — The 0/1 knapsack

`dp[i][r]` = the best value from items `i` onwards with `r` capacity left. One index is not a question — “at item five” has no answer without the room remaining. Take (if it fits) or skip.

Greedy fails twice, and both counter-examples are worth memorising. By value: `W=10`, `(10,60)`, `(5,50)`, `(5,50)` — 60 against 100. By ratio: `W=10`, `(6,60)`, `(5,40)`, `(5,40)` — 60 against 80. **The ratio rule is correct for the fractional knapsack; indivisibility is what breaks it.**

One row, and the direction is the problem. `for r in range(W, w-1, -1)` is 0/1; `range(w, W+1)` is **unbounded**. Same three lines, no error, and the wrong one gives a *larger* answer.

`O(n × W)` is **pseudo-polynomial** — polynomial in the *value* of `W`, not its digit count — which is why a DP exists and the problem is still NP-hard. Compute `n × W` before writing; above $\sim 10^7$, use meet-in-the-middle (small `n`), swap the dimension to index by **value** (small values), or approximate.

The family is the same table: subset sum (booleans, and `reachable[0] = True` is the forgotten base case), equal partition (target `total/2`, and **reject an odd total first**), target sum, minimum difference. Reconstruction needs the full table, so it is mutually exclusive with the one-row form.

Day 149 · dsa — Subset sum and partition problems

Subset sum is knapsack with the value column deleted: `dp[t]` is `True` if some subset of the numbers seen so far adds to exactly `t`. Same two branches, **or where the max was**, base case `dp[0] = True` — the empty subset makes zero.

One row, inner loop backwards, `range(target, number - 1, -1)`, so `dp[t - number]` still holds the value from before this number existed. **Forwards solves the unbounded problem instead, with no error either way** — say the direction out loud as you write it.

Four disguises, all the same table. Equal partition = subset sum to $S/2$, **odd S fails immediately**. Minimum difference = scan the finished row downwards for the largest reachable $a \leq S/2$, answer $S - 2a$. Target sum = $P = (\text{target} + \text{total})/2$ by algebra, count instead of decide ($\text{ways}[0] = 1$, $+=$), zero if $\text{target} + \text{total}$ is odd or $|\text{target}| > \text{total}$. Last Stone Weight II = minimum difference.

$O(n \times T)$ **time**, $O(T)$ **space** — 200 numbers to target 10,000 is 2M cells and instant; target 10^9 is a `MemoryError` before the first comparison. **The bound on the values, not on n , decides whether this approach exists.**

Pseudo-polynomial: input size is $n \log T$ bits, running time is $n \times T = n \times 2^{(\log T)}$ — exponential in the input size, which is how NP-completeness and this table are both true. **Bitset trick:** $\text{reachable} \models \text{reachable} \ll \text{number}$, roughly $80 \times$ faster for free.

Day 150 · dsa — Unbounded knapsack and coin change

Unbounded knapsack is 0/1 with the inner loop forwards. $\text{range}(\text{coin}, \text{amount} + 1)$ so $\text{dp}[t - \text{coin}]$ may already include this coin — that is unlimited supply. Backwards means each item once. **One range reversed, two different problems, no error either way** — say the direction out loud as you write it.

Minimum coins: $\text{dp}[t] = 1 + \min(\text{dp}[t - c])$ over coins that fit. Base $\text{dp}[0] = 0$, rest `float('inf')` — zero makes every amount free, `-1` wins every `min`. Convert to `-1` only at the return, and check for infinity *before* `int()`, which raises `OverflowError`.

Greedy is wrong: coins $[1, 5, 7]$, amount 10 $\rightarrow$ greedy 4 ($7+1+1+1$), optimal 2 ($5+5$). It is correct for canonical systems like Indian denominations, which is why nobody notices.

Counting: $\text{ways}[0] = 1$, $+=$ instead of `min`, and the loop order decides the question. Coins outside = combinations ($[1, 2] \rightarrow 3$ gives 2); amount outside = permutations (gives 3). Coin Change II wants combinations; Combination Sum IV wants permutations despite its name. Read the examples.

$O(\text{coins} \times \text{amount})$ **time**, $O(\text{amount})$ **space** — pseudo-polynomial, so **the size of the amount, not the number of coins, decides whether this works**. Reconstruction is cheap here: one array of last-coin-used. **Minimum coins is also a BFS** over amounts with unit edges, and it wins hugely when the answer is small and the amount is large.

Day 151 · dsa — Coin Change II: counting

Counting is unforgiving where minimising is not: two paths to the same state are harmless for `min` and fatal for a count. The recurrence is always **sum over every possible last move**, and the base case is always `ways[0] = 1` — one way to make nothing — which is the seed every count grows from.

The loop nesting decides the question, and both versions are correct programs. Coins outside = **combinations** (each coin fully processed before the next, so every set is built in one canonical order); amount outside = **permutations** (every coin offered at every amount, so orderings separate). `[1,2]` to 3 gives 2 and 3 — that six-second hand-check catches the mistake every time.

Coin Change II wants combinations. Combination Sum IV wants permutations despite its name. Climbing stairs wants permutations (1-then-2 is a different climb) and is Fibonacci.

Permutations explode: coins `[1,2,5]` to 30 is **58 combinations** against **5,508,222 permutations** — so an answer far larger than expected means the nesting, before anything else.

Modulus at every addition, never only at the end — ways to make 5,000 from coins 1–200 has **74 digits** and a Java `long` holds 19. `10^9 + 7` because it is prime and its square fits in 64 bits. **In Python nothing overflows, so the bug only appears when you port the code** — and never add a modulus the problem did not ask for.

Counting is one integer; listing is backtracking and can be gigabytes (6.3M combinations $\approx$ 3.5 GB). **Count first, enumerate only if the count is small.**

Day 152 · dsa — Longest increasing subsequence

The natural state fails: “longest LIS in the first `i` elements” is incomplete — it gives a length and not the ending value, which is what decides whether the next element can follow. **Redefine, don’t extend:** `dp[i]` = the longest increasing subsequence ending *exactly* at `i`. Same `n` cells; redefining is free.

`dp[i] = 1 + max(dp[j])` over `j < i` with `nums[j] < nums[i]`, each starting at 1. The answer is `max(dp)`, never `dp[-1]` — on `[1,2,3,0]`, `dp` is `[1,2,3,1]`.

`O(n log n)`: `tails[k]` is the smallest value that can end an LIS of length `k+1`. Smaller endings are never worse (anything extending 9 also extends 7); `tails` is therefore sorted, so binary search is legal. Per number: `bisect_left` → append if past the end, else overwrite. The answer is `len(tails)`.

`tails` is NOT the subsequence — on `[1,6,7,2]` it ends as `[1,2,7]`, which is not in that order in the input. **Length right, contents meaningless.** For the actual sequence, record **parent pointers** during the pass.

`bisect_left` = strictly increasing; `bisect_right` = non-decreasing — `[1,3,3,5]` gives 3 and 4. **Russian doll envelopes** = sort width ascending, height DESCENDING within equal widths, then LIS on heights: the minus sign is what stops two same-width envelopes both being picked. `n2` at `n=100,000` is ~17 minutes; `n log n` is under a second.

Day 153 · dsa — Longest common subsequence

The first two-dimensional state: `dp[i][j]` = LCS of `a`'s first `i` characters and `b`'s first `j`. **Prefix lengths, not indices** — so `i = 0` is the empty prefix and the zero row and column are the base cases, free from the allocation. **The price is one off-by-one: the character is `a[i-1]`**, and that is where every bug lives.

Match → `dp[i-1][j-1] + 1` (the diagonal). **Mismatch** → `max(dp[i-1][j], dp[i][j-1])` — skip a character of `a` or of `b`, because you cannot tell which is out. **The answer is `dp[n][m]`**, genuinely the last cell, unlike LIS.

`O(n × m)` time and space; two rows gives `O(min(n, m))` after swapping so the shorter string is inner — `10,000 × 10` becomes 22 cells instead of 20,002. **The collapse gives up reconstruction** (Hirschberg's algorithm recovers it in linear space).

Reconstruction walks back from `dp[n][m]`: match → take the character, move diagonally; else move to the larger neighbour. `O(n + m)`. **The LCS is not unique** — the tie-break picks which one.

Substring differs on exactly two lines: mismatch writes 0 (contiguity broken) and the answer is the **table maximum**, not the last cell. `"abcdef" / "abzdef"` → subsequence 5, substring 3.

LCS is `diff`: in the LCS = unchanged, unmatched in `a` = deletions, unmatched in `b` = insertions. **Git uses Myers**, `O((n+m)×D)`, which gets faster the more similar the files are — `125×` on two 5,000-line files differing in 20. **Derived one-liners:** supersequence `n+m-LCS`; min deletions `n+m-2·LCS`; longest palindromic subsequence `LCS(s, reversed(s))`.

Day 154 · dsa — Edit distance

$dp[i][j]$ = minimum edits turning a 's first i characters into b 's first j . Prefix lengths, so the character is $a[i-1]$. Base cases are $dp[i][0] = i$ and $dp[0][j] = j$, NOT zeros — that is the LCS muscle memory that gives an answer far too small.

Match is FREE: $dp[i-1][j-1]$, no $+1$. Mismatch is $1 + \min$ of three, and name all three: replace = diagonal (both consumed), delete $a[i-1] = dp[i-1][j]$ (only i moves), insert $b[j-1] = dp[i][j-1]$ (only j moves). The index that moves belongs to the string whose character you just used up.

Edit distance is symmetric, so swapping delete and insert passes every test that checks the number — the bug only appears in the reconstruction. Name the branches in comments as you write them.

Reconstruction walks back asking which of four cases produced the value; the loop is `while i > 0 or j > 0`, not `and` — with `and`, a run of trailing insertions is silently dropped. Needs the full table.

$O(n \times m)$ time and space; two rows for $O(\min(n, m))$, and `current = [i] + ...` is the column base case people forget. Weighted costs change only the constants — and the base cases become running sums, and the distance stops being symmetric. Damerau adds transposition: `"hte" → "the"` is 1, not 2.

A spell-checker does not call this 100,000 times — 3 seconds per keystroke. Length filter, row-minimum early exit, then a BK-tree or a trie carrying the DP row down and pruning whole subtrees — a few hundred candidates instead of the whole dictionary, and about a millisecond.

Day 155 · dsa — String DP: palindromic substrings

$dp[i][j]$ is true when $s[i] = s[j]$ AND $dp[i+1][j-1]$ — the ends match and the inside is already a palindrome. The dependency is below-and-left, so the natural fill order reads cells that do not exist yet and silently misses every palindrome longer than 3. Fill by increasing substring length (or with i decreasing). Handle length 2 separately, or guard with $j - i < 2$.

Expand around centre is the answer to write: same $O(n^2)$ time, $O(1)$ space, fewer lines. The DP costs n^2 memory and buys nothing — a million booleans against four integers at $n = 1,000$.

There are $2n - 1$ centres, not $n - n$ characters (odd palindromes) and $n - 1$ gaps (even). Checking only characters reports "abba" as one letter, with no error. The expansion overshoots by one, so return `left + 1, right` — and a negative index slices from the end rather than raising.

Counting is the same loop with `count += 1` inside the while — each successful step outwards is one more distinct palindromic substring.

The table earns its place only for palindrome partitioning, which needs constant-time "is `s[i..j]` a palindrome?" $O(n^2)$ times — $O(n^3)$ with expansion.

Substring $\neq$ subsequence: "bbbab" gives 3 as a substring and 4 as a subsequence ("bbbb"), and the subsequence version is `LCS(s, reversed(s))` — expansion does not apply, because a subsequence has no contiguous centre. Manacher's is $O(n)$ (reuse mirrored results inside the rightmost known palindrome) — name it, don't write it; on real text expansion is already near-linear.

Day 156 · dsa — Grid DP: unique paths

The most direct state in DP: `dp[r][c]` is the answer for that cell — the grid is the table. Every path arrives from above or left, never both, so `dp[r][c] = dp[r-1][c] + dp[r][c-1]` counts each path exactly once. Base cases: first row and column are 1 (initialise the whole table to 1 and get them free).

The one-row collapse is clearest here: iterate left to right, and `row[c]` (not yet overwritten) is the cell above while `row[c-1]` (just overwritten) is the cell to the left. $O(\text{cols})$ space — 8 KB against 40 MB at $1,000^2$, and 80 KB against 4 GB at $10,000^2$. It fails when a cell reads the previous row to its right (minimum falling path) — that needs two arrays.

Obstacles: a blocked cell is 0 and the addition propagates it — but the first row is now ones until the first block and zeros after, because you cannot go round. The initialise-to-1 trick becomes wrong, giving 11 instead of 7.

Minimum path sum: same state, `min` instead of `+`, and the base cases become RUNNING SUMS — keeping the counting base cases is the standard bug. A blocked cell is `inf` here, not 0 — zero would mean free.

Closed form for the plain count: a path is $m+n-2$ moves of which $m-1$ are down, so `C(m+n-2, m-1)` — 28 for 3×7 , $O(1)$ space. Always say the limitation in the same breath: one obstacle destroys it, and costs have no closed form at all.

Counts grow fast ($40 \times 40 \approx 10^{22}$, overflowing a 64-bit integer silently) — which is why a modulus appears. And `maximal_square`'s state is “ending at”, so its answer is `max` over the table, not the last cell.

Day 157 · dsa — DP on decisions: buy and sell stock

The state is `(day, holding)` — and the second dimension is a **MODE**, not a position. “Best profit in the first `n` days” is incomplete: you cannot tell whether selling is legal without knowing whether you hold. **Track money, not profit** — buying subtracts, selling adds.

```
dp[d][0] = max(dp[d-1][0], dp[d-1][1] + price) (sell); dp[d][1] =
max(dp[d-1][1], dp[d-1][0] - price) (buy). Base: dp[0][0] = 0, dp[0][1] = -
price[0]. The answer is the not-holding state — ending while holding is never
optimal.  $O(n)$  time,  $O(1)$  space, two variables.
```

Greedy solves exactly two variants. One transaction: min-so-far and best difference. **Unlimited:** sum the positive day-to-day differences, because `buy@1/sell@3` then `buy@3/sell@5` telescopes into `buy@1/sell@5`. Any transaction limit breaks greedy — on `[3,3,5,0,0,3,1,4]` greedy says 8 whatever `k` is, while the answers are 6 at `k=2` and 4 at `k=1`.

At most `k`: add a dimension, and **STATE** where the transaction is consumed. Count it on the buy (`buy[t]` from `sell[t-1]`); **counting on both halves the limit and gives a plausible smaller number with no error.** The `k ≥ n/2` shortcut is required, not an optimisation — each transaction needs two days, and `k = 10^9` otherwise means `MemoryError`.

A fee is one constant on the sell — no new state. A cooldown is a third state (free / holding / cooling), still $O(n)$ and $O(1)$ — **adding a state to a machine is free; adding a dimension to a table is not.** In the cooldown version **save yesterday's values**, or you sell and buy on the same day; and the answer is `max(free, cooling)` — every state where you hold nothing.

Day 158 · dsa — Interval DP

Interval DP: `dp[i][j]` is the answer for the range `i..j`, and the recurrence tries every split. `dp[i][j] = best over k of dp[i][k] + dp[k+1][j] + join(i,k,j)`. Three nested loops — length, start, split — and length **MUST** be outermost, because both dependencies are strictly shorter ranges. A small test can pass by accident with the wrong order, which is the worst kind of bug.

$O(n^3)$ time, $O(n^2)$ space, and there is NO space collapse — `dp[i][j]` reads many different rows, not just the previous one. **Saying that is the right answer to “reduce the space”.** `n ≤ 100` or `n ≤ 500` in the constraints is the signal that cubic is expected.

Matrix chain: `join = dims[i] × dims[k+1] × dims[j+1]` with matrix `i` being `dims[i] × dims[i+1]`. `[10,100,5,50]` costs 7,500 one way and 75,000 the other — **ten times, for the same answer.** Brute force is the Catalan numbers: 1.3×10^{12} at `n = 25` against ~2,600 for the DP.

Burst balloons is the inversion, and it is the whole problem: ask which balloon is burst LAST, not first. Thinking “first” leaves the two halves entangled, because the left half’s last burst depends on what survives on the right. **Thinking “last” fixes its neighbours to the range boundaries**, so the halves become independent. **Pad both ends with 1** (negative indexing silently wraps otherwise), and **the `k` loop is INCLUSIVE of `j`** — `k` is a member, not a split — unlike matrix chain, which is the most common bug here.

Initialise to `inf` before the `k` loop when minimising (0 wins every `min`). **Write it memoised top-down**, because then the fill order cannot be wrong. **Stone game has no `k` loop** — the split is always at an end — so it is $O(n^2)$, and the minus sign (`piles[i] - dp[i+1][j]`) handles both players in one expression.

Day 159 · dsa — DP on trees

A subtree is a subproblem, and post-order traversal visits every child before its parent — so the recursion IS the fill order. No table, no loops, nothing to get wrong: **the hardest mechanical part of DP disappears when the data is a tree.** $O(n)$ time, $O(\text{depth})$ space.

The state is often a pair, not a number. House robber on a tree: `(take, skip)`, where `take = value + ∑ child.skip` and `skip = ∑ max(child.take, child.skip)`. **One number fails because the parent’s options depend on the child’s CHOICE, not just its score** — and it fails silently, too high, with an illegal selection behind it.

Record ≠ return, and this is the defining bug. Diameter: **record `left + right`** (the path *through* this node) and **return `1 + max(left, right)`** (the path *down* from it) — a path cannot fork at a node and still continue upwards. Returning the recorded value gives no error and numbers that grow with the tree.

Max path sum adds two details: clamp each child at `max(0, ...)` so a negative branch is skipped, and **initialise `best` to `-inf`**, not 0, because an all-negative tree has a negative answer.

Ask whether the answer is a property of the root or of the whole tree — height is returned; diameter and max path sum are **recorded**, because they can occur anywhere.

Depth is the practical trap: a line-shaped tree is valid, and Python's limit is 1,000 frames. `setrecursionlimit` raises Python's limit but not the C stack, so it **segfaults instead of raising** — use an explicit stack, pushing each node twice with a `processed` flag to get post-order. **Rerooting** answers a question at *every* node in $O(n)$ instead of $O(n^2)$: one post-order pass for subtree answers, one pre-order pass where moving the root across an edge costs one subtraction.

Day 160 · dsa — Bitmask DP

An integer's bits are a set. Five operations: `mask | (1<<i)` add, `mask & ~(1<<i)` remove, `mask & (1<<i)` test, `mask.bit_count()` size, `(1<<n)-1` full. **Always parenthesise the shift** — `=` binds tighter than `&`.

The test before committing: does the ORDER matter, or only the SET? If only the set, `dp[mask]` is complete — and in the assignment problem `popcount(mask)` even tells you which job you are on, so one dimension suffices. **If the last thing done affects what comes next — travelling salesman — you need `dp[mask][last]`**, and getting this wrong returns a smaller number with no error.

The fill order is free: adding a bit always increases the integer, so `for mask in range(1 << n)` visits every state after its dependencies — ascending works for both the push and the pull pattern.

TSP: `dp[1][0] = 0`, and the answer is `min over last of dp[full][last] + dist[last][0]`. **Forgetting the trip home** solves the open-path problem instead — 62 against 97 on the worked example, both plausible.

$O(2^n \cdot n^2)$, and say **BOTH halves**: at $n = 20$ that is $\sim 4 \times 10^8$ operations against $(n-1)! = 1.2 \times 10^{17}$ — **290 billion times better, and still exponential**. Practical ceiling ~ 20 in Python (memory fails before time — $n = 24$ is ~ 10 GB), ~ 24 in C. **For the assignment problem specifically, the Hungarian algorithm is $O(n^3)$** — volunteer that your answer is exponential where a polynomial one exists.

Submask enumeration: `submask = (submask - 1) & mask`. Total work is 3^n not 4^n — each element is in the submask, in the mask but not the submask, or in neither — which is 43M against 4.3B at $n = 16$.

Day 161 · dsa — Space optimisation in DP

One rule: look at which cells the recurrence reads and keep only those. `dp[i-1]`, `dp[i-2]` → **two variables**. Previous row only → **one or two rows** (swap so the shorter dimension is inner — `O(min(n,m))`). Previous row **plus the diagonal** → one row **plus one saved variable**. Many rows at once (interval DP) → **no collapse exists, and saying so is the right answer**.

The one-row array holds two rows at once, split at the cursor: everything left of it is updated (the current row), everything at or right of it is not (the previous row). **So** `row[c] += row[c-1]` **is exactly “above + left”** — pointing at those two values is the real demonstration that you understand the recurrence.

Direction decides which generation you read. Backwards guarantees the previous row (0/1 knapsack, each item once); **forwards allows the current row** (unbounded, reusable). Same six lines, different answers — 9 against 10 on the worked example — **and no error either way**.

Precondition: every dependency must be above or to the left. Minimum falling path reads the previous row to the *right*, which is already overwritten, so it needs **two arrays**.

The collapse is free in time — the same cells are computed, and one row fits in cache, so it is often ~25% *faster*. **What it costs is the reconstruction:** value in `O(min(n,m))` space, or path in `O(n×m)`. **Both is Hirschberg’s** — divide and conquer on the middle row, add the forward and backward arrays to find where the optimal path crosses, recurse on the halves: **exactly 2× the time, linear space, and the path**.

Re-establish the base cases on every row (`row[0] = i` is the most-forgotten line). **And write the clear table first, then collapse** — a wrong recurrence is nearly undebuggable in an array that is half one row and half another.

Day 162 · dsa — Recognising dynamic programming

Four signals — any one means “consider DP”: it asks for **max/min/count/possible**; there is a **small set of choices** at each step; the **naive recursion is exponential**; and the **constraint on n** is small in a telling way. **Read the constraints first** — `$n \leq 20$` → bitmask, `$\leq 100$` → `$O(n^3)$` , `$\leq 1,000$` → `$O(n^2)$` , `$\leq 10^5$` → one-dimensional or not DP at all. Every row lands near 10^6 – 10^7 , because the constraint is chosen so the intended solution fits and the next-worse one does not.

Then the test, and BOTH halves matter: optimal substructure AND overlapping subproblems. No overlap = divide and conquer (merge sort: 2,047 calls, 2,047 distinct arguments — a table would never be read). Overlap = DP (fib(20): 13,529 calls, 21 distinct arguments). **Broken substructure means your STATE is incomplete, not that DP is wrong.**

The shape follows one question: what do I need to know to decide the next step? Position → `dp[i]`; position + resource → `dp[i][cap]`; position + mode → `dp[i][mode]`; two sequences → `dp[i][j]`; a grid → `dp[r][c]`; a **range and how you split it** → interval DP; a subtree → recursion; a **set**, $n \leq 20$ → `dp[mask]`.

Then, in order and in ninety seconds: (1) state as a full SENTENCE — cannot finish it, you have no state; **(2) completeness check** — given only the state, can you decide the next step; **(3) count states × work against $\sim 10^7$** (Python does $\sim 10^7$ ops/second, C $\sim 10^9$). **Then recurrence, base cases, memoise** (the fill order cannot be wrong), tabulate and collapse only if asked.

Not DP when: no overlap (divide and conquer); greedy is provably safe (**spend 30 seconds trying to build a counter-example** — coins `[1,5,7]`, amount 10: greedy 4, optimal 2); **a closed form exists** (grid paths are a binomial coefficient); or the problem says “**list all**” rather than “how many” — that is backtracking, and the output is gigabytes whatever you do.

Day 163 · dsa — Dynamic programming revision and mock round

DP in one sentence: find a state that completely describes a situation, write a recurrence over smaller states, compute each once. Everything else is mechanics.

Eight shapes: `dp[i]` / `dp[i][cap]` / `dp[i][mode]` / `dp[i][j]` two sequences / `dp[r][c]` grid / `dp[i][j]` interval / recursion for trees / `dp[mask]`. **The one question that picks the shape: what do I need to know to decide the next step?**

Three non-obvious fill orders, all silent when wrong: knapsack **descending** for 0/1 and **ascending** for unbounded; interval DP by **increasing length**; tree DP **post-order** (free, from the recursion).

Fifteen bugs that raise no exception — loop direction, loop nesting, incomplete state, `dp[-1]` vs `max(dp)`, zero base cases in edit distance, `+1` on a match, no clamp on negatives, `best = 0` not `-inf`, interval fill order, record-vs-return in trees, a resource counted twice, the missing trip home, the missing `k ≥ n/2` shortcut, a dependency to the right, and the missing `row[0] = i` after collapsing. **Random testing against a brute force catches all fifteen in five minutes.**

The rubric, in weight order: state stated before coding, complexity given unprompted, recurrence correct, edge cases, communication — and only then whether the code ran. “Finished it first try” barely counts. No code before minute thirteen: the nine minutes at the start buy the last fifteen.

Say the state as a full SENTENCE, run the completeness check, count states $\times$ work against $\sim 10^7$ — ninety seconds, before anything else. Then memoise (the fill order cannot be wrong), tabulate and collapse only if asked. And reading is not retrieval: close everything and produce it out loud, because that is the only thing that tells you what you actually have.

Greedy and intervals

Day 164 · dsa — Greedy: when taking the best option now is safe

Greedy is a CLAIM, not a technique: the locally best choice is always part of *some* optimal solution. Two properties are needed — optimal substructure (shared with DP) and the greedy choice property (what greedy adds, and what is usually false). Note the phrasing: *some* optimal solution, not *the* one.

The decision procedure is thirty seconds: TRY TO BREAK IT. Look where the greedy choice is large and blocks two smaller things that together beat it. Coins [1, 5, 7] to 10: greedy 4, optimal 2. House robber [5, 10, 6] : greedy 10, optimal 11. 0/1 knapsack by ratio, cap 10, (6, 60), (5, 40), (5, 40) : greedy 60, optimal 80. All three are three items long.

The asymmetry is the whole point: a counter-example PROVES greedy fails; failing to find one proves nothing. If you cannot construct the exchange argument either, say so and write the DP — unnecessary DP costs code; unjustified greedy costs a wrong answer.

The exchange argument: take any optimal solution, swap the greedy choice in, show it does not get worse. For activity selection — swap in the earliest finisher; it finishes no later, so it conflicts with nothing that followed.

Four provably-correct greedies: activity selection (sort by END time), fractional knapsack (value per kilo — and the SAME rule fails for 0/1, because indivisibility is the difference), Huffman (merge the two rarest), and MST (Kruskal/Prim). All are dominated by a sort or a heap, because greedy is “process in the right order”.

The sort key IS the algorithm. By start time or by duration both fail on activity selection, **and by duration often gives the right answer on your test**, which is worse. And “**maximum NUMBER**” versus “**maximum VALUE**” is **one word apart and is greedy versus DP** — with values, weighted interval scheduling, still $O(n \log n)$.

Day 165 · dsa — Proving a greedy choice, simply

Two arguments, both short, both proof *sketches* rather than proofs. What you must establish is weaker than it looks: **there is AT LEAST ONE optimal solution containing your choice** — not that every optimum contains it, and not that the choice is “best”.

THE EXCHANGE ARGUMENT (use this by default), in three steps: take *any* optimal solution; if it contains your choice you are done; **otherwise swap your choice in for whatever it used there, and show BOTH that the result is still LEGAL and that it is NO WORSE.** Omitting the legality half is the most common incomplete argument.

Activity selection in two sentences: the earliest finisher ends no later than whatever it replaced, **so nothing that fitted after the old one now conflicts**, and the count is unchanged.

The ADJACENT-SWAP form is the easiest — look for it first, because you only reason about two elements. Shortest job first: any out-of-order adjacent pair can be swapped to reduce the total wait by exactly (longer – shorter), **so a schedule containing one is not optimal.**

“GREEDY STAYS AHEAD” is the other form: after every step, greedy’s partial answer is at least as good as anyone else’s, **so greedy can never run out of room first.** Use it when there is a natural order and a running measure; use exchange when the answer is a set.

A FAILED PROOF NAMES THE COUNTER-EXAMPLE. 0/1 knapsack by ratio breaks at “*swapping g in may not make room*” — which describes a heavy high-ratio item plus two lighter ones: $(6, 60)$ against two $(5, 40)$ in a 10-kg bag, 60 against 80. **The fractional version survives the same argument because you can swap an equal WEIGHT** — one word in the statement decides it.

Say it in thirty seconds, three sentences, unprompted. And “**I cannot construct the exchange argument, so I will write the DP**” is an excellent answer — an unnecessary table costs code, an unjustified greedy costs a wrong answer, and those are not symmetric.

Day 166 · dsa — Interval scheduling

Every interval problem starts with a sort, and the whole difficulty is SORT BY WHAT. Three questions that read almost identically need three different keys — and the wrong key gives a plausible answer with no error and the same complexity.

“How many can I fit?” → greedy, sort by END. Take anything starting at or after `last_end`. End time is correct because the only thing constraining the future is when the resource becomes free. “Minimum removals to make disjoint” is the same problem: `n` minus the answer.

“Merge what overlaps” → one pass, sort by START. Start order guarantees that anything overlapping the current block begins before it ends, which is what makes one comparison enough. The extension is `max(current_end, next_end)`, never `= next_end` — a contained interval would shrink the block, and containment is rare in test data.

“How many at once?” → SWEEP LINE, and the move is to stop caring which interval is which. Break each into `+1` at the start and `-1` at the end, sort the events, take the running maximum. Sorting `(time, delta)` tuples gives the right tie-break free (`-1 < +1`), so an end at time 4 is processed before a start at 4 — getting it backwards is exactly one too high on abutting inputs. The sweep COUNTS; a min-heap of end times ASSIGNS.

Settle the boundary convention before coding — `≥` against `>` — the problem decides it, and say which you assumed.

All techniques are $O(n \log n)$, dominated by the sort. Add values and greedy breaks: `[0,10]` worth 100 against three worth 10 gives greedy 30 and the answer 100. Weighted interval scheduling is DP — `dp[i] = max(dp[i-1], value + dp[p(i)])` with `p(i)` found by BINARY SEARCH (a linear scan makes it $O(n^2)$) — and it is the same $O(n \log n)$, so greedy failing costs a table, not a complexity penalty.

Day 167 · dsa — Merging intervals

Sort by START, one pass: if the next interval begins at or before the current block ends, extend the block; otherwise close it and start a new one. Sorting by start is what makes a single pass legal — anything overlapping the current block must BEGIN before it ends, so you never look backwards and never reopen a closed block.

The bug, and it is the point of the lesson: extend to `max(current_end, next_end)`, NEVER `= next_end`. An interval entirely inside the block would otherwise shrink it — `[1,10]` then `[2,3]` becomes `[1,3]`. And it gives exactly the right answer on typical hand-written tests, because partial overlap is what people write and containment is what real data contains.

Sorted by END the invariant fails: `[1,10]`, `[2,3]`, `[11,12]` gives `[(2,10)]` — the block starts at 2 and no code path reaches backwards to fix a start, while the block *count* stays right.

The boundary convention is one character — `≤` merges touching intervals, `<` does not — and the problem decides it. Ask if it is not stated.

Five variants: insert into a sorted list (`O(n)`), three phases — re-sorting is the mistake being tested; gaps (the complement, and the two ends are where the bugs live — needs explicit bounds, and `cursor = max(cursor, end)`); intersect two lists (two pointers, `[max(starts), min(ends)]`), advance whichever ENDS first); total covered (merge first, or overlaps are double-counted); and streaming.

Streaming: the one-pass algorithm does not apply. Keep the blocks in a sorted structure, binary search for the position, and absorb every overlapping block — one arrival can swallow several at once. `O(log n + k)` per insert, amortised `O(n log n)` overall because a block can be absorbed only once — the same total as sorting once, paid incrementally.

Day 168 · dsa — Meeting rooms and the sweep line

The sweep is a COUNTING technique, and the idea is to discard which interval is which. Break each into `+1` at the start and `-1` at the end, throw away the association, sort by time, and take the running maximum. Six lines. “How many at once” and “which one is which” are different questions, and the first is far easier.

The tie-break is where half of implementations are wrong. For meeting rooms, an end at time 4 must be processed before a start at 4 — the room is freed and reused, giving one room, not two. Sorting `(time, delta)` tuples gives this for free because `-1 < +1`. Getting it backwards is exactly one too high on abutting meetings, which is what every real calendar looks like. Never encode events as strings — `"end" < "start"` is a coincidence of the alphabet — and work in integers, because floating-point times destroy the tie entirely.

Two equivalent forms: the event list (tie-break in the sort key) and **two pointers over sorted starts and ends** (tie-break visible as `<` against `≤`). Same answer, same complexity.

The sweep cannot give the ASSIGNMENT — it discarded it, which is what made it cheap. For “which room”, use a **min-heap of end times**: reuse the earliest-freeing room if free, else open a new one. Same $O(n \log n)$, $O(k)$ space, **and the heap’s size agrees with the sweep** — a **free consistency check** (they disagree only when their conventions differ).

It is a framework: sort events, walk, accumulate — **only the ACCUMULATOR changes**. Count $\rightarrow$ integer; weighted (bandwidth) $\rightarrow$ `+w / -w`; total covered $\rightarrow$ sum the stretches while the count is positive (**accumulate BEFORE applying the delta**); the **SKYLINE** $\rightarrow$ `max` instead of `+`, so the accumulator becomes a heap with **LAZY DELETION** (leave expired entries in, discard them when they surface — each popped once, so still $O(n \log n)$).

And when the coordinate space is small and dense, a difference array beats the sweep — $O(n + \text{horizon})$ against $O(n \log n)$, $\sim 18\times$ faster for 10,000 meetings over 1,440 minutes. **Compare horizon against $n \log n$ before choosing.**

Day 169 · dsa — Jump game and reachability

This looks like DP and is greedy. The naive `dp[i] = is position i reachable` is $O(n^2)$ and correct and **remembers more than it needs**. **The insight: you do not need to know WHICH positions are reachable, only HOW FAR you can reach** — one number.

It works because reachability is CONTIGUOUS: reach position 7 and you reach everything before it, **so the reachable set is always a prefix and a prefix is one number**. $O(n)$ time, $O(1)$ space.

The stopping rule is the line people omit: `if i > furthest: return False`. Without it the loop walks past an unreachable position **and updates the frontier from it** — returning `True` for `[3,2,1,0,4]`. **It fails only on inputs containing a zero**, which is why it survives. **And the update must be** `max(furthest, i+jump)` — reachability only ever grows.

Minimum jumps is a BFS with the queue collapsed into two numbers, because the levels are contiguous ranges: `current_end` (where this level stops) and `furthest` (how far anything in it reaches). `if i == current_end` is the level boundary. **The loop stops ONE SHORT** — landing on the last position needs no jump from it — **and including it is exactly one too many on every input, with no error.**

The exchange argument: greedy jumps to the spot with the furthest reach; swapping it into any optimal sequence is legal (both are in the current level) and no worse (it reaches at least as far), **so anything the optimum could do next, greedy can too.** The property it depends on is contiguity — “further is simply better”, with no trade-off.

The test for which technique you have: is the reachable set always a contiguous prefix? Yes → one number, greedy. Jump left OR right → scattered, not a prefix → a graph, plain BFS, and the $O(1)$ space is gone. Add a cost per jump → contiguity survives but “further is better” does not, because reach and price trade off → dynamic programming.

Day 170 · dsa — Greedy and intervals revision and mock round

The decision procedure, four questions, under two minutes. (1) Can I name the best-looking choice in one sentence? (2) Can I build a counter-example in thirty seconds? (3) Can I state the **exchange argument concretely** — the actual swap, not its shape? (4) If 2 or 3 failed, it is DP — and saying so with the counter-example is a correct answer.

The exchange argument in thirty seconds: *take any optimal solution; swap my greedy choice in for its first choice; mine finishes no later, so nothing that followed can conflict; same size, still valid; induct.* This is what “why does that work?” wants.

The sort-key table, and the rule underneath it — sort so the greedy choice costs the least future. END for most non-overlapping. START for merging. EVENTS (+1/−1) for how-many-at-once. NO SORT, $O(n)$ for reachability — the frontier collapses to one number. DEADLINE + heap for fitting deadlines. POSITION + heap for regret greedy.

Two middle-ground families, and the heap is the tell. Greedy with an undo (Course Schedule III: take everything, drop the longest on overrun — dropping the longest frees the most time at the same cost of one course). **Regret greedy** (refuelling: never decide at a station, drive past, buy the biggest tank you passed once you know you needed it). Try these before reaching for DP.

Greedy survives when the choice affects only what is still available; it dies when the choice carries a value you are optimising. Jump Game is $O(n)$ greedy; jump game with a cost per landing is DP.

The wrong sort key is SILENT and passes every tidy example. By start instead of end: keeps 1 not 2. By duration instead of deadline: 2 not 3. $\geq$ instead of $>$: 1 not 2. $<$ instead of $\leq$ in refuelling: -1 not 2. **Your only evidence is a brute-force check**

on small random inputs; your only argument is the exchange. Have one, and say which.

High-level design case studies

Day 145 · system-design — How to run a high-level design interview: the forty-five-minute script

Six phases, forty-five minutes: requirements (5), scale (5), API and data model (5), high-level design (10), deep dive (15), wrap-up (5). The first fifteen minutes are setup and should feel mechanical; **the deep dive is what is being marked.**

Phase 1: ask, and state the exclusions. Phase 2: users → per second → peak → storage per year, and end with **the sentence that turns arithmetic into a decision** (“writes fit on one machine; reads do not”).

Phase 4: draw the boring version and trace one write and one read out loud. Phase 5: **offer a choice**, then cover mechanism, numbers, failure mode, and **the alternative you rejected** — that last one is what makes it a trade-off.

Four ways to lose it, all about time: drawing before asking; no numbers; breadth with no depth; and losing the clock. **Narrate the time** — “I’ve used ten minutes on the high-level, let me go deep now.”

Interruptions are hints — answer, then reorient. And **not knowing is fine if you say what you would do instead:** “I don’t know that product’s consistency levels; the property I need is read-your-writes, and I’d check whether it gives me that.”

Day 146 · system-design — Design a URL shortener

Do the arithmetic first: 100M URLs/month is ~40 writes/s, ~200 peak, ~600 GB/year. That is **one database** — so the design problem is code generation and redirect latency, and **inventing a sharding problem is how this round is lost.**

Code generation: counter + base-62, with two fixes. **Block allocation** (10,000 ids per server, so coordination once per 10,000 and a crash wastes a rounding error) and **a scrambling multiply** so codes are not enumerable — while saying plainly that this is obfuscation, not security.

Length from arithmetic: $62^6 = 57 \text{ billion} \approx 47 \text{ years}$; $62^7 = 3.5 \text{ trillion}$. Six is enough, seven is the safe choice, and base 62 rather than 64 because `+` and `/` need escaping.

301 vs 302 is a product decision. 301 is cached by the browser — $5\text{--}10\times$ less traffic, **no analytics after the first click**, and **the link can never be repointed**. 302 gives every click and the ability to disable a malicious link, which is why commercial shorteners use it.

The click count never goes on the read path — publish to a queue, aggregate asynchronously. And **abuse is the requirement people forget**: rate-limit creation, scan destinations at creation *and periodically*, offer a preview, and **disable rather than delete** so codes are never reissued.

Day 147 · system-design — Design a pastebin

Same shape as a shortener, one difference: the payload is 10 KB, not 200 bytes. Fewer writes ($\sim 12/\text{s}$) and **six times the storage** (3.6 TB/year) — so the storage decision is the design, not the traffic.

Metadata in the database, content in object storage — $5\times$ cheaper per GB, and a restore of 6 minutes instead of 5 hours. **With an inline threshold at ~ 4 KB**, because fetching a 1 KB paste from object storage is a 30 ms round trip to move 1 KB, and it covers $\sim 80\%$ of pastes with one database read.

A paste is immutable, so `Cache-Control: immutable` with a long `max-age` — and that single header absorbs a viral paste (100,000 reads in 20 minutes ≈ 83 req/s on one key) at a $\sim 99\%$ CDN hit rate.

Expiry: check on read (that is the correctness mechanism), **partition by expiry day** so removal is a `DROP TABLE`, and **let object-storage lifecycle rules delete the blobs** via retention-class prefixes. A `DELETE ... WHERE expires_at < now()` job is the answer that does not scale.

“Unlisted” means the id is hard to guess, not private — say so in the product. And abuse is the forgotten requirement: **scan for credentials on write**, rate-limit creation, cap the size, and `is_removed` rather than delete — which conflicts with long cache TTLs, so shorten them until a paste is scanned.

Day 148 · system-design — Design a rate limiter, at system scale

Fixed window is the cheapest and allows $2\times$ the limit at the boundary — 100 at 10:00:59 and 100 at 10:01:00. **Sliding log is exact and costs a timestamp per request** (8 GB vs 32 MB at 1M keys, limit 1,000). **Sliding window counter is the practical answer**: two counters, weight the previous window by the fraction still in view, error under $\sim 5\%$.

Token bucket separates two numbers with two jobs — rate `R` is the sustained limit, capacity `B` is how much burst is forgiven — with **lazy refill** on access and `time.monotonic()`, never wall-clock. **Leaky bucket eliminates bursts entirely**, at the cost of queueing latency.

The distributed count is the hard half. Shared Redis is exact and adds ~ 0.5 ms to every request; local counters at limit/n are wrong because traffic is never even; **local-with-sync is what to use, and the answer states the over-admission bound.**

Decide the failure policy in advance: fail open with a generous local fallback, plus a **circuit breaker on the limiter itself** — a slow Redis that you wait for on every request is how the limiter causes the outage.

Limit on several things at once (account, IP for anonymous — crude because of NAT, endpoint via per-request **cost**, and a **global** limit for the system itself), layer a crude limit at the edge, and return `429` with a **jittered** `Retry-After` or you have built your own thundering herd.

Day 149 · system-design — Design a distributed key-value store

Consistent hashing, not modulo: modulo remaps $\sim 91\%$ of keys when 10 machines becomes 11; the ring moves $\sim 1/n$. **256 virtual nodes per machine** evens the arcs to within $\sim 10\%$ and spreads a dead machine's load across many survivors. **The preference list must skip virtual nodes of a machine already in it** (and skip AZs).

$R + W > N$ is a counting argument, not a protocol — the read set and write set cannot be disjoint, so one responding replica has the newest write. **$N=3, W=2, R=2$ by default;** **$R=3$** makes every read wait for the slowest of three replicas (p99 ~ 2.5 ms $\rightarrow \sim 8$ ms). It bounds staleness for completed writes; **it is not linearizability.**

No global clock means concurrent writes cannot be ordered. Last-write-wins is one timestamp and silently loses writes under clock skew (Cassandra's default). **Vector clocks keep both versions,** `get` returns a list, the application merges — cart = union. Truncate past ~ 10 entries; false conflicts are safe.

Three repair mechanisms, all of them: hinted handoff keeps writes available during a failure; read repair is free but only fixes what is read; **Merkle-tree anti-entropy** finds 1 differing key in 1,000,000 in ~ 20 comparisons instead of 20 MB of hashes. **Gossip for membership**, no central registry.

Per node: an LSM tree (memtable $\rightarrow$ WAL $\rightarrow$ SSTables $\rightarrow$ compaction) with a **Bloom filter per SSTable.** **Dynamo is AP by choice** — both sides of a partition accept writes. **And below ~ 1 TB, one Postgres does this with transactions and indexes for free;** reaching for this design too early is the expensive mistake.

Day 150 · system-design — Design a distributed unique ID generator

Snowflake, 64 bits: 1 sign + 41 ms + 10 machine + 12 sequence. Timestamp in the **high** bits, which is the only reason the IDs sort by time — integer comparison compares high bits first. **Custom epoch at launch, not 1970:** 69 years from launch instead of exhausting in 2039.

41 bits = 69.7 years. 10 bits = 1,024 machines. 12 bits = 4,096/ms = 4.1M/s per machine — nothing real approaches it, so the sequence field is for burst correctness, not throughput. On exhaustion, **wait for the next millisecond**; never wrap, never carry into the machine field.

Why not UUIDv4: 16 bytes not 8 (~64 GB extra at 1B rows × 8 index copies) and random keys destroy the B-tree — every insert hits a different page instead of the hot rightmost one, **2–10× fewer inserts/second**, degrading invisibly as the index outgrows memory. **UUIDv7 (RFC 9562, 2024) fixes sortability** and needs no machine id — the real modern alternative at 16 bytes.

Clock going backwards is the failure mode: reissued timestamps with a reset sequence = silent duplicates. **Slew NTP, never step; wait on a small step; refuse loudly on a large one**, and persist the last timestamp across restarts.

Machine ids from ZooKeeper ephemeral sequential nodes, the private IP's low bits, or Kubernetes StatefulSet ordinals — never a config file, because two machines sharing an id produce duplicates with no error. **Sortable IDs leak your creation rate**, so keep the Snowflake internal and expose an encrypted or random public identifier.

Day 151 · system-design — Design a web crawler

The loop is BFS: frontier → fetch → store → extract links → add unseen. **The frontier is the whole design.**

Politeness inverts the obvious structure. A single FIFO clusters by host (pages link to their own site), so a hundred workers hit one server at once. **Queue per host, one worker per host, a heap keyed on next-allowed time** (monotonic clock). `robots.txt` per host, cached a day, obeyed — and decide in advance what an unreachable one means.

Two-tier frontier: front queues for priority, back queues for politeness, with a router between — a single priority queue cannot enforce politeness and a single host queue cannot express priority.

Three levels of dedup, and only the first saves bandwidth. Normalise + hash the URL (sort query params!); SHA-256 the body (~25% exact dupes → 25 TB/month saved); **SimHash + Hamming** ≤ 3 for near-duplicates. **Seen set: 80 GB as strings, 8 GB as hashes, 1.2 GB as a Bloom filter at 1% —** and 1% of a billion is ten million pages silently never crawled.

The number that reframes the problem: 386 pages/s at 1 req/host/s means 386 distinct hosts in flight. Not bandwidth-bound, not CPU-bound — **host-diversity-bound**. A 100,000-page site takes 28 hours.

Traps are calendars, facets and session IDs, not malice. Depth cap, per-domain cap, URL length, repeated segments — all heuristics, all sometimes wrong, all necessary; log them. **If-Modified-Since** cuts recrawl bandwidth ~5× (500 TB → 101 TB at 80% unchanged). **Partition the frontier by host** so politeness stays a local invariant.

Day 152 · system-design — Design a notification system at scale

Five stages: ingest → resolve → build → send → track. Ingest is **asynchronous** with a **202** so a provider outage cannot break checkout. **One queue and worker pool per channel** — a bulkhead, so slow SMS cannot starve email.

Four gates in the resolver, cheapest first: preferences → dedup → rate limit → quiet hours (in the user's timezone, and $h \geq 22$ or $h < 8$, not a range check). The per-user hourly cap is the most valuable feature and is always built last.

Dedup with an event-derived key — `hash(user, type, event_id)`, never generated during processing — via Redis **SET NX EX** (atomic; **GET** -then- **SET** is a race). Set the key **before** sending, and pass `apns-collapse-id` so the provider deduplicates too. **No Bloom filter here** — 1% silently unsent includes 1% of OTPs.

Retry classification is the thing people get wrong. `429 / 5xx / timeout` → back-off with jitter; `400 / 410 / 403` → **never retry, and act**: delete the dead token, record the opt-out. **Every message has a TTL** and expired ones are dropped and counted — that counter is the earliest signal of a backed-up pipeline.

Fan-out in two stages (one fan-out job → per-1,000 batches as separate messages): 10M followers is 5.5 hours on one worker, ~100 s batched, **~10 s with engagement filtering** — and the filter, not the parallelism, is most of the win. **Batching 5 likes into 1 push cuts volume ~30%, costs less, and raises open rates.**

SMS is ~1% of volume and ~88% of cost (\$60k/day at 1B/day) — prefer push. **And one Twilio number sends 1 SMS/second**, so high volume is a procurement problem, not a runtime one.

Day 153 · system-design — Design a news feed

Compute the ratio first: ~100 reads per write. That is what justifies expensive writes. **Fan-out on write (push)** = `O(followers)` per post, ~12 ms reads. **Fan-out on read (pull)** = free writes, ~180 ms reads querying hundreds of timelines. **15× slower, 100× as often — write wins.**

And write breaks on the celebrity: 50M followers = 50M inserts = ~17 minutes, with the pipeline blocked throughout. **So: hybrid.** Under ~10,000 followers → fan out on write; above → store only, and **readers merge in the ~30 celebrities they follow.** The asymmetry is the point: few celebrities followed, many followers each. **The threshold is a tuning parameter — be able to derive it, not recite it.**

Feeds hold post IDs, never content — 16 bytes vs 1 KB, and content is stored once (8 TB vs 240 TB at 300M users). **Redis sorted sets**, `ZREVRANGE` to read, `ZREMRANGEBYRANK` to cap at ~800 — uncapped feeds grow forever.

Cursor pagination, never offsets — new posts shift an offset and page two repeats page one's items. The cursor carries **score + id**, and for a ranked feed **a ranking session id too**, or changing scores reintroduce duplicates.

Hydrate in 3 batched multi-gets, not 60 round trips (~5 ms vs ~30). **Deletes and unfollows are filtered at read**, not scrubbed from millions of feeds — the batched fetch drops missing posts for free. **Ranking = candidates → features → score → filter**, with **time decay** (`score / (age+2)^1.5`) so a viral old post does not outrank everything new.

Day 154 · system-design — Design Twitter

Scope out loud first (in: tweet, follow, both timelines, search, trending; out: DMs, lists, ads, media, moderation) — **that choice is a large part of what is assessed.** Then size: **3B reads / 30M writes a day = 100:1**, which justifies fan-out on write.

Two timelines, built differently. **User timeline is just a query on** `(author_id, tweet_id)` — do not precompute it. **Home timeline is precomputed** into a Redis sorted set per user, capped ~800, **with the Snowflake id as both member and score** so it needs no timestamp and merging is free.

Hybrid fan-out at ~10,000 followers. A 100M-follower account = **~33 minutes of inserts**, and an **800 MB follower id list** that cannot be fetched at once — which is why the graph service streams a paged cursor. **Store the graph BOTH ways** (by follower and by followee, ~2 TB total), so both questions are single-shard; the copies reconcile asynchronously.

Search needs a different index entirely — inverted, keyed by term, **sharded by time** so a hot in-memory index over recent tweets answers most queries. **Asynchronous, so near-real-time**: putting it on the write path would make a tweet fail when the index is slow.

Trending is rate of change, not rate — ranking by count returns “the” every day. **Count-min sketch** (min across rows, never undercounts, 2.6 MB) scored against a **seasonal baseline**; the killer property is that **sketches merge across shards by addition**.

Hydrate in 3 batched multi-gets (~10 ms end to end). **Tweets are the smallest data here** — 3.3 TB/year against 4 TB of timelines and 2 TB of graph. **And below ~1M users this is a follows table and a join** — building the fan-out first is the mistake the question tempts you into.

Day 155 · system-design — Design Instagram

Two systems, and say so in the first minute: metadata is ~500 bytes a post and is the standard feed design; **media is ~2 MB a photo — 7,000× a tweet** — and is 99.94% of the bytes. All the interesting database work is on 0.06% of the data.

Uploads never touch your servers: pre-signed URLs. The API returns a short-lived signed permission and the client PUTs directly to object storage — **2.3 GB/s removed from the application tier**, at the cost of after-the-fact validation and moderation.

Transcoding is asynchronous and must be idempotent (storage events are at-least-once). **Honour EXIF orientation** or a quarter of photos come out sideways; **strip GPS** — a privacy requirement, not a size one. **Pre-generate 4 variants**, don’t resize on demand: 30% more storage against a compute service on the hottest read path.

The CDN is not an optimisation — it is the only affordable design. 3.75 PB/day from origin is ~\$345K/day; at a 95% hit rate ~\$90K/day. **So the most expensive possible mistake is a per-user signed URL**, which makes every URL unique and destroys the cache — **sign the path and expiry instead**.

Storage never shrinks: 95 PB/year. All-standard \$2.2M/month, **tiered by age** ~\$600K — 80% of views are of content under a week old. Anything pulled from archive must be promoted back.

The feed returns URLs, not bytes, plus dimensions and a **blurhash** (20 bytes, instant render, no layout jump) — and **the client picks the variant**, which is a 10× difference in bytes on a slow connection. **Video’s adaptive ladder costs ~1× real-**

time per rendition and becomes the dominant compute bill. **And delete is not delete:** metadata instant, CDN purge minutes and not guaranteed, objects last.

Day 156 · system-design — Design WhatsApp

Opposite shape from the social systems: write-heavy, point-to-point, latency-critical, and duplicates and reordering are user-visible. 100B messages/day is $\sim 1\text{M/s}$, but at 300 bytes that is **2.8 Gbps** against Instagram's 350 — **bounded by connections and latency, not bytes.**

WebSockets, one per device, so the servers are STATEFUL. A registry (`user → connection server`), a hash per user for multiple devices, **with a heartbeat-refreshed TTL** because unclean disconnects are normal). $500\text{M connections} \times \sim 10\text{ KB} = 5\text{ TB of RAM}$; WhatsApp ran 2M connections on one machine.

Three distinct events: sent / delivered / read = one tick / two ticks / blue ticks. Persist BEFORE acking — ack-then-crash loses a message the sender believes was sent, and the client will never retry. **Idempotency on a CLIENT-generated message id. Ordering is per conversation, not global.**

Offline queue = a sorted set drained in order, and messages are removed only after acknowledgement — delete-on-push loses messages. At-least-once, absorbed by client-side dedup. 30-day give-up.

Store-until-delivered is the biggest number in the design: 33 PB/year → a few TB. The phone becomes the archive. **Media is the real storage** ($\sim 1\text{ PB/day}$ unique after content-hash dedup, which is why forwarding is instant).

E2E encryption removes capabilities, and that is the answer: no server-side search, **spam fought on metadata only**, no server-side backup without a user-held key, and multi-device needs per-device keys — which makes a 1,000-member group **2,000 encryptions ≈ 2 seconds on a phone**, hence **sender keys**. **Groups are capped at $\sim 1,000$, which deletes the celebrity fan-out problem** — a product decision doing an architecture's job. **And presence, built naively, is larger than the message traffic itself.**

Day 157 · system-design — Design a chat system with presence

Inverts three WhatsApp decisions: history is kept forever ($\approx 91\text{ TB/year}$ vs a few TB), channels have unbounded membership, and encryption is server-side — which makes search possible and makes it a first-class system. The WebSocket transport is unchanged; say so and move on.

Storage: partition by `(channel_id, month)`, **cluster by** **Snowflake id**. “The last fifty” is one sequential read. **The month bucket is required** — a busy channel over five years is ~ 900 MB in one partition against a ~ 100 MB guideline (Discord’s published migration is the reference).

Fan-out by pub/sub, not per-member lookups. Each connection server subscribes **once per channel**, not per client; one publish replaces hundreds of registry lookups on the sender’s request. **Offline members get no queue** — `last_read_message_id` is one row per user per channel and the unread count is **computed lazily and cached** (eager recomputation is $> 1\text{M}$ range counts/second).

Presence is the surprise: built naively it is $\sim 70 \times$ the message traffic — $17,000 \text{ events/s} \times \sim 1,000 \text{ viewers} = 17\text{M pushes/s}$ against 230,000 for messages. **Fix by scoping (only people currently viewing a shared channel, ~ 20 not 1,000) and throttling (coalesce to one update per 5 s) $\rightarrow \sim 20,000/\text{s}$.**

Presence must be an expiring key with NO explicit offline event — closed laptops, lost signal and killed tabs never send a disconnect. **Absence of a heartbeat is the offline signal**, and the TTL is the dial between staleness and heartbeat traffic. **Typing = the same thing with a 3 s TTL and `SET NX` as the throttle**, never persisted. **Presence is approximate by design, and that is correct.**

Search permissions go IN the query as a `channel_id IN (...)` filter, never as a post-filter — result counts would leak the existence and activity of private channels.

Day 158 · system-design — Design YouTube

Scope out loud (in: upload, transcode, playback, storage, views; out: recommendations, comments, live, monetisation, Content ID) — the product is too big to design whole. **Video is $\sim 7,000 \times$ a photo and $\sim 10^6 \times$ a tweet**, and the bill is the design constraint.

Transcoding is $\sim 1 \times$ real-time per rendition — $720,000 \text{ hours/day} \times 6 = 4.3\text{M CPU-hours/day}$, $\sim \$4\text{M/month}$, which is why custom encoding silicon exists. **It parallelises by chunk: split at KEYFRAMES** (fixed-time splits are silently corrupt), one job per segment per rendition — **an hour of wall time becomes ~ 20 seconds with identical CPU**. Make it **idempotent** (the most expensive op in the system), **publish renditions progressively**, and **generate 2160p lazily**.

Adaptive bitrate inverts the usual relationship: the CLIENT decides. It fetches a manifest, starts low for a ~ 330 ms first frame, measures throughput and switches quality between segments. **Every segment is a static file**, so the CDN needs no video

logic and there is no per-viewer connection. **Segment length trades adaptation speed against overhead — 4–6 s is the compromise.**

Storage is ~15 GB per hour of video → 11 PB/day → 4 EB/year. Untiered \$92M/month; **tiered by POPULARITY, not age** (top 1% gets ~90% of views; the bottom 89% is most of the bytes) → ~\$20M. **The expedited restore path is mandatory** — a cold video that trends is unwatchable exactly when the views arrive. **Keep the original** for future codec migrations (AV1 saves 30–50% of the biggest line); re-encoding a rendition loses quality permanently.

Delivery is the largest line: 900 PB/day. At commercial CDN rates that is \$270M/month, so at this scale you build your own edge network and embed caches in ISPs. Adaptive bitrate itself is a $2.5\times$ cost control. **View counts are two systems** — instant approximate for display, batch authoritative with fraud filtering for money — which is why counts sometimes go down.

Day 159 · system-design — Design Netflix

One fact drives everything: the catalogue is SMALL (~20,000 titles, ~450 TB — 0.01% of a year of YouTube uploads) and KNOWN WEEKS IN ADVANCE. Lead with that comparison; every other difference follows from it.

Delivery is PREDICTIVE, not reactive. Files are pushed to every edge days before release, at 03:00 local when bandwidth is idle — **so at midnight there are no cache misses and origin traffic is ~zero.** A reactive CDN would pull ~270 TB from origin in the first minutes with the first viewer at every edge waiting.

Open Connect: appliances placed INSIDE ISP networks. Both sides save (the ISP avoids transit, Netflix avoids per-GB egress), which is why ISPs rack them. ~\$5M/month amortised against ~\$300M/month to buy the same delivery — $50\times$ — and it only works because the catalogue fits on a box.

Per-title and per-shot encoding, measured with a perceptual metric (VMAF): ~20% bitrate saving. Justified purely by amortisation — encode once, serve millions of times. ~3M CPU-hours for the **WHOLE catalogue (~\$90k, once)** against YouTube's 4.3M CPU-hours **PER DAY.** A cartoon needs 2 Mbps where a grainy thriller needs 8; a fixed ladder is wrong in both directions.

Recommendations are the interface, not a feature — no query, so the home page decides what is watched. Rows, each a ranking problem, **precomputed nightly** with a thin real-time layer; feasible over 20,000 items and not over billions.

Playback adds three things to yesterday's ABR: DRM (three ecosystems; CMAF + common encryption stores one copy not three), **entitlements** (rights are per country and per date), and **resume** — 40M concurrent streams reporting every ~5 s is **8M writes/second of 30 bytes**, the highest write rate here. **The release spike is a control-plane problem** — the CDN is pre-warmed, the licence server cannot be.

Day 160 · system-design — Design Google Drive or Dropbox

The difficulty is correctness, not scale — ~5,000 changes/second, no fan-out, no hot spots. **Files are metadata plus a list of content-addressed chunk hashes**; the bytes live in a separate immutable store.

Content-defined chunking, not fixed: a rolling hash picks boundaries so they move with the content — **with fixed boundaries, inserting one byte at the start changes every hash and re-uploads the whole file**. Upload = hash locally → ask which chunks are missing → send only those (4 MB, not 50).

Sync is a monotonic per-user change log with a cursor: “what since 4,417?” is **one request whatever the account size**, versus polling 100,000 files. **The push notification carries no data**, so losing one is harmless — the cursor is the authority. **Save the cursor AFTER applying**, so a crash replays (idempotent) rather than skips (permanent loss). **Compaction after ~30 days needs a “cursor too old, resync” path**.

Conflicts are detected by BASE VERSION, never timestamps — clocks disagree, and a fast clock wins every conflict forever. **There is no correct merge for a binary file, so keep both** as a named conflicted copy. **Last-write-wins is not simpler, it is wrong** — it destroys work silently and the user finds out a week later.

Deduplication is an information leak, and this is the best follow-up in the topic: the server telling you it already has a chunk lets an attacker who can guess a file's contents learn that someone else has it. **Fix: dedup storage globally, but check ownership on every chunk FETCH** — knowing a hash must not be enough to read.

Dedup saves ~30% personal / ~70% corporate; delta sync saves ~5× on bandwidth (every change syncs to ~3 devices). **The metadata is 50 billion rows** — tiny in bytes, huge in rows, queried far more than the chunks. **Client-side: check (mtime, size) before hashing**, with filesystem watch APIs replacing the scan.

Day 161 · system-design — Design Uber

Two systems with opposite requirements: locations (250,000 writes/s, 50-byte payloads, overwrite-only, LOSSY BY DESIGN, in-memory, sharded by city) and **trips/payments** (~5,000 writes/s, durable, transactional). The location pipeline is

250 × the write volume of the business it supports — do not put them in one store.

A normal index cannot answer “within 3 km” — latitude and longitude B-trees each give a *band*, and intersecting them still leaves a large share of a city. **Collapse two dimensions into one:** geohashing interleaves the bits so **nearby points share a prefix**, turning it into a prefix range scan.

Always search the cell PLUS its eight neighbours — two points 20 m apart across a boundary share no prefix, and this fails precisely for riders near boundaries. **H3 uses hexagons because all six neighbours are equidistant**, where a square has edge and corner neighbours at different distances. **Then an exact haversine filter decides** — the index narrows, it does not answer.

The live index is only ~200 MB for a million drivers. **A TTL does offline detection for free** — no “going offline” message to lose. **Search radius adapts:** one ring downtown, widen in the suburbs.

Greedy matching is measurably wrong. Distance $\neq$ time (batch the ETA calls), and assigning one rider at a time strands later riders — **22 minutes greedy against 12 batched on the worked example**. Batched matching is the assignment problem (Hungarian, $O(n^3)$, ~30 ms for 100×300), **at the cost of a few seconds’ delay for everyone**.

Dispatch takes a lock on the driver with a TTL — mutual exclusion and automatic release from one `SET NX EX`, because the explicit unlock is the message that gets lost. **Re-check the lock on acceptance**. Trips need an explicit transition table, or a retried “complete” charges twice. **Surge is a control loop and needs rate-limiting**, or it oscillates.

Day 162 · system-design — Design Google Maps

Dijkstra is off by FOUR ORDERS OF MAGNITUDE, not a constant factor. London→Edinburgh settles ~5,000,000 nodes (1–2 s); at 40,000 queries/second that is ~1,300 machines for routing alone. **Bidirectional gives 2–4 ×, A* gives 5–10 × — neither is close.** So the question is not “which algorithm” but “**what can I compute in advance**”.

Contraction hierarchies: road networks are hierarchical — a 200 km journey is 190 km of motorway. Order nodes by importance, **contract from the bottom up**, adding a shortcut only where the path through the node was the sole shortest route (the bounded local search is what stops the shortcuts exploding). **The query is bidirectional search that only moves UPWARDS** — ~500 nodes, ~0.1 ms, **10,000 × faster for a few hours of offline work per continent**.

Traffic defeats naive preprocessing: weights are baked in, and rebuilding takes hours while traffic changes every 2 minutes. **Split the preprocessing — a slow TOPOLOGY phase (roads change ~never) and a fast WEIGHTS phase (minutes) —** because traffic changes weights, not topology. **Plus a live overlay** checked only for the few hundred edges a query touches, with stale readings falling back to the model.

Traffic comes from the phones: ~5M reports/second, 17 TB/day. **Map matching is a real subsystem** — GPS accuracy is comparable to the gap between a flyover and the road beneath it. **Median, not mean** (a parked phone is not a jam) and a **minimum contributor count, which is simultaneously the noise filter and the privacy guarantee.** The raw stream is the most sensitive data in the system.

ETA is a prediction, not a sum: advance the clock as you walk the route, so a segment 40 minutes ahead is evaluated at its predicted conditions then — and **return a range**, since a point estimate is overconfident exactly when traffic is worst. **Tiles are 4ⁿ per zoom level** — pre-render to $z=12$ (~22M tiles), render deeper lazily ($z=20$ is 10^{12} tiles). **Vector tiles are ~50× cheaper than raster overall. And routing is ~2% of the bill** — the money is tiles and telemetry.

Day 163 · system-design — Design an e-commerce system

Two systems with 2,000:1 between them: browse/search is a cacheable read problem tolerating minutes of staleness; **checkout/orders is small (~100k/day, <200 GB/year) and must be exactly right.** Orders belong in a **single relational primary** — reaching for a distributed store trades the one guarantee you need for scale you do not.

The read-then-write race is the central bug: two requests both read 1, both pass the check, both write 0. **It needs a few milliseconds of overlap — never in testing, constantly in production.** Fix with an **atomic conditional update** (`WHERE sku = ? AND available ≥ ?`), then check the **affected row count**, not a re-read). Optimistic concurrency wastes a round trip per conflict; pessimistic locking serialises everything on the row.

Stock has THREE states: available / reserved / sold. Basket reserves nothing; **checkout reserves with a TTL;** payment converts it; abandonment lets it lapse — **because nobody sends “I gave up”.** The expiry job is trivial and its absence is catastrophic: the store shows sold out with a full warehouse.

Idempotency key generated by the CLIENT when checkout begins, not per request — a per-request key differs on the retry and deduplicates nothing. **The provider is the source of truth for whether money moved,** so add a reconciliation job

and an idempotent webhook handler.

Two-phase payment, and the ordering is the design: authorise → check inventory → capture. A void is free and invisible; a refund costs the fee again and takes days — ~\$570k/year at 1% refunds, which beats the correctness argument for persuasiveness.

The order is a SAGA with an explicit transition table (which is what stops a retried “paid” charging twice) and **compensations that are not symmetric. State change and event publication go in one transaction via an outbox. Flash sales break the single inventory row** (~3,000 updates/s): admit fewer, decrement in Redis (~100,000/s), shard the counter — **and sharding strands the last units, which is a real cost.** “Sold out” is a better answer than “we oversold”.

Day 164 · system-design — Design a payment system

The mismatch is the design: the business needs exactly-once, the network offers at-least-once. What you actually get is **at-least-once attempted + deduplication on a key = exactly-once recorded** — say it that way rather than claiming a guarantee.

Double-entry: every transfer is TWO signed entries summing to ZERO. Errors become detectable by arithmetic; **the balance is DERIVED (a snapshot is a cache you can throw away and recompute)**; and the ledger is **append-only** — corrections are compensating pairs, never edits. A five-hundred-year-old idea, and the correct data model.

Money is signed integers in the smallest unit, never floats ($0.1 + 0.2 \neq 0.3$), and the **exponent is not always 2** — JPY has none, BHD has three. Every amount carries its currency.

On a timeout, mark it unknown — NEVER **failed**. That single line is the most common bug in payment code. Resolve with three mechanisms, all needed: **client-generated idempotency keys** (per checkout, not per request), **a job that asks the provider** (which is the source of truth after a timeout, not your database), and **signature-verified, deduplicated webhooks** that may arrive before your own response.

The idempotency insert must BE the lock — a unique constraint or `SET NX`, because check-then-write has a window and two concurrent retries both charge.

Authorise → check → capture, because a VOID is free and a REFUND costs the fee twice and takes days. At 10M payments/day averaging £40, a 1% failure-after-capture rate is ~£100,000/day.

Reconcile daily against the settlement file, in four categories — timing, missed webhooks, **amount mismatches (the alarming one)**, and cut-off effects. **You will always find discrepancies; finding none means the job is broken.** Track the **age of the oldest unresolved item**, not the count. And **infrastructure is ~0.007% of the card fees** — always take the safe option.

Day 165 · system-design — Design a ticket booking system

Every unit is UNIQUE and non-substitutable, so there is NO oversell recovery — you cannot ship an equivalent seat. **And the contention is extreme AND predictable:** 100,000 people at a published time, $\sim 800 \times$ the ordinary rate for sixty seconds. **Predictability is the advantage** — pre-scale, pre-warm, pre-queue.

Three seat states — available / held / sold — and the hold must be **ATOMIC**: `SET NX EX` or a conditional `UPDATE` with a row-count check. **The TTL is the entire safety mechanism, because nobody sends “I gave up”.** Release must be a **Lua script checking a session id** — `GET` then `DEL` can release somebody else’s hold after an expiry. **Group bookings acquire in SORTED order** so contention is always in the same sequence and one side fails fast instead of deadlocking.

99% of requests must never reach the seat map — not because of the operations but because **they all fail, all retry, and double the load**. Three layers: **a waiting room with a VISIBLE position** (a countdown is qualitatively different from a spinner), **randomised admission within a window** (FCFS rewards fast networks and bots — it is a different unfairness, not fairness), and **rejecting early**: the number of disappointed people is fixed, the time they wait to find out is not.

The seat map is cached (~2 s) and stale, and that is CORRECT — the atomic hold is the authority, so a stale map produces a rejected attempt and a refresh. **Push over WebSockets, never poll**: ~50 changes/second broadcast against 100,000 clients polling every 2 s (1 GB/s for one show).

Payment: hold → authorise → RE-CHECK the hold → capture. 3-D Secure takes 30–60 seconds of the customer’s time, so **“hold expired mid-payment” is a real path** — void (free) and tell them.

The hold TTL has no derived answer. Short breaks slow payments; long wastes seats. **Overselling is visible and underselling is invisible**, so the number drifts long unless you measure the expired-hold rate against the failed-payment rate. **And the durable write path is trivial (~17 sales/second) — all the difficulty is upstream.**

Day 166 · system-design — Design search autocomplete at scale

Two numbers settle the design. The **latency budget**: $\sim 100\text{--}200$ ms between key-strokes minus $\sim 50\text{--}80$ ms of network leaves **20–50 ms of server time**, and a late suggestion is worse than none. **And the traffic is per KEYSTROKE, not per search** — $5\text{B searches} \times \sim 15 \text{ characters} = 75\text{B requests/day}$, $\sim 15 \times$ **search traffic**.

The central inversion: do not search, PRECOMPUTE. Every prefix maps to its **top five, already ranked** — one hash lookup, no traversal. **The cost is staleness**, patched by a **second, small, real-time trending layer** (~ 50 MB against the index's ~ 22 GB) merged at request time. **Two structures, two update frequencies.**

A plain trie is ~ 1 TB for a billion queries — the pointers outweigh the strings. Use a **radix tree or an FST** (shared prefixes *and* suffixes, $\sim 10\text{--}20$ GB), **or just a flat hash map**, since precomputed answers never need walking. **Cap prefixes at ~ 20 characters** or storage grows without bound.

Server work is ~ 5 ms and network is $50\text{--}80$ ms, so the biggest win is NOT BEING ON THE NETWORK. **Debounce** ($\sim 4\times$), **client-side filtering** ($\sim 2\times$, guarded on still having five results — the cached top-5 for "man" is not necessarily the top-5 for "manch"), and **edge serving** the ~ 100 MB popular head. **$\sim 75\times$ fewer requests reach origin, entirely from client and edge behaviour.** Keep connections alive, and **make requests cancellable** or a slow short-prefix response overwrites a fast long-prefix one.

Ranking: log-damped frequency, recency decay, and click-through (the strongest server-side signal — it measures usefulness, not volume). **Personal history is the strongest signal overall and can live entirely on the device** — free, private, instant.

Filter at build time AND at serve time, because removal cannot wait for the next build — completing a real person's name defamatorily has produced lawsuits, and **the asymmetry favours over-blocking** since nobody reports a suggestion that never appeared. **And the trending layer is trivially cheap to attack** (~ 500 repetitions in a 10-minute window): fix it with the **counting rule** — **distinct users, rate-limited** — not with a longer window.

Day 167 · system-design — Design a leaderboard

Two questions with different costs, and separating them is the opening move: the **TOP TEN** is easy in any sorted store; the **RANK** is a count over everyone **unless the structure supports it**. Almost every naive design gets the first and fails the second.

Redis sorted sets — and say what is underneath. A skip list with SPANS on the forward pointers (rank is the sum of spans along the search path, so $O(\log n)$ rather than a count) plus a hash map for $O(1)$ score lookup on update. The spans are precisely the feature that makes rank cheap. ~ 100 bytes/entry $\rightarrow$ 50M players $\approx$ 5 GB, ~ 50 – 100 μ s per operation, about half of one instance at peak.

Do NOT shard the sorted set: rank is inherently GLOBAL, so a split makes every rank query hit every shard and get slower as you add shards. Three better answers: **partition the leaderboard by a product dimension** (region, level, season — usually what the product wants anyway); **exact top 10,000 in a bounded set** (~ 1 MB, trimmed with ZREMRANGEBYRANK on every write); **or a score histogram** (~ 200 KB, $25,000\times$ smaller, accurate to a bucket).

Approximate is the better product below the top few hundred — “about 2,100th of 50,000” beats a precise number that changes every few seconds — **and the interface should say “about” rather than imply precision.**

Ties break lexicographically by member name by default — stable, arbitrary, and visible. **Encode the tie-break into the score** ($\text{score} \times 10^{10} + \text{inverted timestamp}$), and assert the 53-bit double precision bound, because exceeding it silently reorders players. **Display the real score, never the composite.**

Windows are separate sorted sets with TTLs (3 – $4\times$ write amplification, every read one lookup; shard by window, which is easy because no query crosses them). **“Rolling last 24 hours” has nothing to expire** — hourly buckets and an hour of granularity is the honest answer. **ZADD ... GT makes best-score atomic**; **ZREVRANK is zero-indexed**. **Redis is the serving structure, not the source of truth** — rebuild into a temp key and RENAME, ~ 10 minutes, which is affordable precisely because it is a leaderboard.

Day 168 · system-design — Design an ad click aggregator

The shape is unlike the rest: a million tiny events a second in, a few thousand queries out, and the raw events are individually worthless. So the central move is **aggregate on the way IN, not on the way out** — a query becomes a lookup over pre-summed rows, never a scan.

Five stages: a tiny ingest endpoint (write to Kafka, return 202, no DB write), **Kafka PARTITIONED BY CAMPAIGN** (so one instance owns each campaign — no cross-instance coordination), **stream aggregation into one-minute windows**, a **pre-summed store**, and a **dashboard that does lookups**.

Event time versus processing time is the correctness decision. Processing time is easier and **makes reports change when you rerun them.** Event time is correct and means **no window is ever truly closed** — resolved by a **watermark** (max event time seen minus a grace period, advancing with the data, not the clock) plus an **explicit late-data path**. **~0.1% arrive over a minute late — a million events a day, not an edge case.** There is no correct grace period, only a stated one.

Duplicates are GUARANTEED (client retries, Kafka at-least-once, aggregator restarts) and **each one is money.** **Client-generated event id**, atomic set-if-absent, exact over ~5 minutes (~15 GB). **Explicitly reject a Bloom filter:** 4 GB for an hour at 1% sounds good, **but a false positive drops a real click — 1% of revenue.** The same structure was fine for a crawler because there the errors were free.

Never store the cross-product: campaign × country × device × slot × minute is **7×10^{12} rows/day, almost all zero**, against **~30 GB/day** for sparse query-driven rol-lups — **12,000×**. The cost is that a new dimension needs a backfill, so the dimension set has real inertia.

Two paths, deliberately. Stream = fast and approximate; **nightly batch over the raw log = slow and exact** (**COUNT(DISTINCT)** over the whole day) and it **OVERWRITES closed periods.** Advertisers see the streaming number in a minute; the invoice uses the batch number, and the gap between them is the alert. Approximate what is reported (HyperLogLog, 1.6% in 12 KB), count exactly what is billed.

Day 169 · system-design — Design a distributed job scheduler

The scheduling is the easy part — a table indexed on (state, run_at) and a poll loop; even 100M jobs/day is ~1,200/second. **Everything hard is failure and distribution.**

Model JOBS and RUNS separately (one job, many runs) for history, natural retries, and **no drift** — the next run is computed from the schedule, not from when the last finished. **UNIQUE (job_id, run_at)** stops two schedulers materialising the same occurrence.

Claim in ONE atomic statement with **FOR UPDATE SKIP LOCKED** — a select-then-update is a race, and without **SKIP LOCKED** every worker blocks on the same rows and the fleet has the throughput of one. **Increment the attempt counter on the CLAIM**, so a worker that dies has consumed an attempt.

Leases have no good length, because a dead worker and a slow one are indistinguishable over a network. Short steals jobs from slow workers (duplicate execution); long leaves a dead worker's job idle. **The resolution is a HEARTBEAT** — 60-second lease, 20-second renewal — **and the heartbeat MUST check ownership**, or two workers both believe they own the run.

Exactly-once is not available: the side effect and the record of it cannot be atomic when the side effect is in someone else's system. **At-least-once + an idempotency key from (job_id, run_at)** (not per attempt) is the default; at-most-once is an explicit per-job flag. **~0.01–0.1% of runs duplicate — 10,000–100,000/day at this volume — so idempotency is a REQUIREMENT, not advice.**

The herd is what actually breaks it: ~40% of jobs are scheduled on the hour, so the average rate is fine and the instantaneous rate is an outage. **Jitter is one line** (40M over 5 minutes = 133k/second, evenly) — **per job, defaulting to zero, so set it on the job TYPE or it will be left at zero.** Plus rate-limited dispatch and a **bounded catch-up policy**, or recovering from an outage becomes a second one. **And partition by month and DROP** — a **DELETE** over 9 billion rows never finishes.

Day 170 · system-design — High-level design revision and full mock

Six blocks, forty-five minutes, announced out loud: 0–5 requirements (functional, non-functional, and what is explicitly out of scope), **5–10 the arithmetic**, **10–15 interface and data model**, **15–30 architecture in request-flow order**, **30–40 one deep dive** — **offer two and let them pick**, **40–45 failure and what you would measure.** **The most common failure is drawing boxes in minute two**, which says you are recalling rather than designing.

Four opening questions: how many users and how many at once; **the read-to-write ratio** (the single most decision-relevant number — it decides caching, replicas, fan-out and denormalisation); what must not go wrong; and what you are **not** building.

The estimation ladder: users → actions → per second ($\div 100,000$, $\times 10$ for peak) → bytes. **Round hard** — three significant figures reads as not knowing what matters. **The output is ONE SENTENCE**, not a number: *“read-heavy 50:1, and storage is the real problem.”* **Discovering a problem is easy is a result.**

Numbers cold: 1M/day $\approx$ 12/s; 1B/day $\approx$ 12,000/s; one database $\sim$ 5,000 writes/s; one cache node $\sim$ 100,000 reads/s; memory 100 ns, SSD 100 μ s, disk 10 ms; **cross-continent 150 ms and no engineering fixes it.**

The move worth stealing from the mock: when a huge payload must appear at an INSTANT, pre-position it and release something tiny. 5 TB in a minute becomes 5 TB over an hour plus a 1 GB key — a factor of 5,000. **And continuous autosave turns an end-of-exam spike into nothing: the same data either arrives smoothly or all at once, and that is the design choice.**

Three habits that read as senior: volunteer the failure mode before you are asked (with the mitigation ready); **give the number WITH the trade** (“90% hit rate turns 50,000 reads into 5,000, at up to 60 seconds of staleness”); and say what you do not know. **And when they say “interesting, but what about X” — X is what they want. Abandon your plan.**

The complete 180-day map

All one hundred and eighty days, both tracks, in order. Days in **bold** are in this volume.

DAY	DATA STRUCTURES AND ALGORITHMS	SYSTEM DESIGN
143	What dynamic programming actually is	Bloom filters and probabilistic structures
144	Fibonacci: memoisation versus tabulation	Building blocks revision and interview questions
145	Climbing stairs and the one-dimensional habit	How to run a high-level design interview: the forty-five-minute script
146	House robber and the choice at each step	Design a URL shortener
147	Finding the state: the hardest part of DP	Design a pastebin
148	The 0/1 knapsack	Design a rate limiter, at system scale
149	Subset sum and partition problems	Design a distributed key-value store
150	Unbounded knapsack and coin change	Design a distributed unique ID generator
151	Coin Change II: counting	Design a web crawler
152	Longest increasing subsequence	Design a notification system at scale
153	Longest common subsequence	Design a news feed
154	Edit distance	Design Twitter
155	String DP: palindromic substrings	Design Instagram
156	Grid DP: unique paths	Design WhatsApp
157	DP on decisions: buy and sell stock	Design a chat system with presence
158	Interval DP	Design YouTube
159	DP on trees	Design Netflix
160	Bitmask DP	Design Google Drive or Dropbox
161	Space optimisation in DP	Design Uber
162	Recognising dynamic programming	Design Google Maps
163	Dynamic programming revision and mock round	Design an e-commerce system
164	Greedy: when taking the best option now is safe	Design a payment system
165	Proving a greedy choice, simply	Design a ticket booking system
166	Interval scheduling	Design search autocomplete at scale

DAY	DATA STRUCTURES AND ALGORITHMS	SYSTEM DESIGN
167	Merging intervals	Design a leaderboard
168	Meeting rooms and the sweep line	Design an ad click aggregator
169	Jump game and reachability	Design a distributed job scheduler
170	Greedy and intervals revision and mock round	High-level design revision and full mock

Glossary

Every term this book defines, with the day it first appears. Definitions are the sentence that introduced the term.

4

4 kb [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

A

a replicated log [Day 119](#) — Real systems need a never-ending sequence of decisions, which is why every practical consensus system is built around a replicated log: an ordered, numbered list of entries, identical on every machine.

abstract class [Day 052](#) — Definition. An abstract class is a partly-built class: it can hold fields, constructors and working methods, and it declares some methods that subclasses must fill in.

acid [Day 025](#) — When Postgres says COMMIT succeeded, the data is on disk, and that promise is the D in ACID — Atomicity, Consistency, Isolation, Durability — the four letters you will meet properly on [day 033](../day-033-window-with-a-map/README.md).

adaptee [Day 068](#) — The adaptee — the washing machine, which has a different shape and cannot be changed.

adapter [Day 059](#) — An adapter is an implementation that translates to a specific technology (PostgresOrderRepository).

add a tie-breaker [Day 027](#) — And add a tie-breaker — without one, two customers with the same total may swap places between runs, which makes paginated results duplicate and skip.

algorithm [Day 076](#) — A strategy is an algorithm plugged into a caller; a command is a request with its receiver and arguments already bound, so it can be stored, queued and undone.

always comparable [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

amortised [Day 005](#) — The word for that is amortised: list.append is O(1) amortised, meaning any single append might be expensive but a long run of them averages out to constant time.

anaemic model [Day 044](#) — A design where classes hold only data and all the rules live in some `IdolService` is called an anaemic model — objects in name only — and it is the most common way candidates lose this round.

and a direction [Day 079](#) — It carries a floor and a direction — "I am on 4 and I want to go up".

anti-diagonals [Day 016](#) — To collect the down-left diagonals — usually called the anti-diagonals — group cells by the value of $r + c$.

anti-sniping [Day 095](#) — Starting the count again when somebody speaks at the last moment is anti-sniping: extending the deadline whenever a bid arrives near the end.

approximate [Day 102](#) — Also: Redis's LRU is approximate — it samples a handful of keys and evicts the least recently used among them, rather than maintaining a true global ordering, because exact LRU would cost a linked-list update on every access.

array [Day 015](#) — Padma's shelf is an array — the fixed row of numbered boxes you met on [\[day 009\]\(../day-009-what-an-array-is/README.md\)](#).

artefact [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

assignment rule [Day 108](#) — "Walk forwards until you reach a point" is the assignment rule: a key belongs to the first node clockwise.

at-least-once [Day 129](#) — Two: you gain duplicates. Queues deliver at-least-once: if the consumer does not acknowledge in time, the message comes back.

atomic [Day 008](#) — The fix is to make check-and-act atomic — indivisible, so that no other thread can act in the middle of it.

atomicity [Day 025](#) — That is atomicity — the A in ACID — and it is impossible to get right with a file unless you write the whole log-and-replay machinery yourself, at which point you have written a database.

availability [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that **every** non-failing node answers.

aws ecs [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, systemd on a virtual machine.

B

backend [Day 002](#) — The backend is the code that never leaves the kitchen.

bearer token [Day 019](#) — The token is a bearer token: whoever bears it is treated as the user.

best case [Day 002](#) — The best case is 1, the worst case is n.

binary tree [Day 098](#) — A binary tree is a tree where every node has at most two children, called left and right.

blast radius [Day 111](#) — The eleven days revealing water, diesel and medicine is blast radius: what actually stops.

block [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

blocked [Day 007](#) — When the barber stood there holding a head while the boy fetched hair colour, he was blocked — occupying a chair while doing no work.

both [Day 030](#) — Once both are inside the loop, look at the gap between them, measured the way round the loop from fast to slow.

boundary [Day 118](#) — The unifying idea worth stating: **two heaps facing each other let you maintain a boundary in a changing set — and the boundary is the answer.**

bounded context [Day 051](#) — The name for the boundary between those worlds is a bounded context: each context gets its own model of "customer", and they are linked by an id rather than merged.

bounded optimism [Day 116](#) — Spending up to two thousand is bounded optimism — act early where the downside is small.

breadth-first [Day 100](#) — The other family is breadth-first — one whole level at a time — which needs a queue instead of a stack and is [\[tomorrow\]\(../day-101-bfs-level-order/README.md\)](#).

break point [Day 045](#) — The result has one break point — one place where a value is followed by a smaller value.

broken links [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

bucket by time [Day 157](#) — The one thing that breaks this model is a channel that never stops growing. A partition with ten million messages is too large for most stores, so bucket by time: the partition key becomes (channel_id, month).

bucket counts [Day 171](#) — The fix is histograms. Each machine exports bucket counts — "how many requests fell in 0–10 ms, 10–50 ms, 50–100 ms" — and you sum the buckets across machines and compute the percentile from the sum. That is exactly what Prometheus's histogram_quantile does, and it is why counters and histograms are the metric types that aggregate and...

bulkhead [Day 111](#) — A bulkhead is a partition that stops a failure spreading — from ships, where a holed compartment does not sink the vessel.

burst size [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

C

cache [Day 003](#) — Saving it in his phone is caching. A cache is a local copy of an answer, kept so that you do not have to ask again.

cache it [Day 107](#) — Fix: you cannot split a single key across shards, so cache it — the photocopy — or replicate it to every shard, or give it its own shard.

cache stampede [Day 090](#) — And the first cold morning is a cache stampede.

candidates [Day 093](#) — The trays are the candidates — the values you are allowed to pick from.

capacity [Day 005](#) — A list keeps a capacity — how many slots it has room for — which is bigger than its length, the number of slots actually used.

cdn [Day 070](#) — CDN — a geographically distributed caching proxy.

certificate [Day 006](#) — When your browser connects to google.com, the machine sends back a certificate — a small document containing the name it claims (google.com), a public key, an expiry date, and a signature from a Certificate Authority.

chaining [Day 060](#) — The standard fix is chaining: each bucket holds a small list of the (key, value) pairs that landed there, and a lookup checks the two or three entries in that one bucket.

channels [Day 092](#) — The group message, the direct message and the phone call are channels — the different ways a message can physically reach a person.

checks permission [Day 011](#) — The kernel checks the arguments and checks permission — this is Balan saying no.

chunks [Day 160](#) — A file is not stored as a file. It is split into chunks — fixed or variable-size blocks, typically around 4 MB — and each chunk is identified by the hash of its contents. The file itself becomes a small piece of metadata: a name, a version, and an ordered list of chunk hashes.

circular buffer [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position 7 + 1 is position 0.

class [Day 043](#) — A class is the description; an object is one thing made from that description.

client [Day 068](#) — The client — the wall socket, which will only accept one shape.

collapse key [Day 141](#) — Collapsing is Devaraj refusing to carry the same message four times. Both platforms support a collapse key: a new notification with the same key replaces any undelivered one.

collapsing writes [Day 102](#) — Its real advantage is collapsing writes — a video watched ten thousand times in a minute becomes one database write instead of ten thousand.

collection [Day 016](#) — A collection is the set of things: /bags.

column [Day 026](#) — A column is one attribute of it, with a type that constrains what can go there.

compact [Day 011](#) — The price is that you now need a separate way to know which slots are real, and you must eventually compact: one pass that closes all the holes at once.

comparator [Day 058](#) — And the two uncles are the one case that needs a comparator — a rule about a pair, not a fact about an item.

comparison [Day 052](#) — A comparison is one question: *is this one bigger than that one?* A swap is exchanging two values so each sits where the other was.

complete [Day 114](#) — Insert appends first, and that is not a detail. Putting the new value at the end of the array is the only place that keeps the tree complete — any other position leaves a gap and destroys the array layout.

component [Day 069](#) — The tumblers are the component — the real object that does the actual job.

composite [Day 029](#) — This only ever applies when the primary key is composite — two or more columns together.

composite key [Day 026](#) — Its primary key is usually the two columns together — a composite key — which also enforces that the same pair cannot appear twice.

composition [Day 071](#) — Strategy uses composition — the varying part is an object passed in.

composition root [Day 053](#) — The composition root is the one place in the program where the real concrete classes are named and wired together — usually `main()` or a startup module.

connection [Day 041](#) — Under all of this, every query needs a connection — and opening one is genuinely expensive: TCP, then TLS ([[day 006](#)](../day-006-python-strings-dicts-sets/README.md)'s handshake), authentication, and in Postgres a whole operating-system process per connection ([[day 008](#)](../day-008-reading-a-problem/README.md)'s processes, ~5–10 MB each).

connection-oriented [Day 004](#) — This is why TCP is called connection-oriented — there is a real, agreed-upon connection with state on both sides.

consequence [Day 109](#) — The lorries are the consequence — the number that changes a decision, which is the only number that actually mattered.

consistency [Day 114](#) — The words also do not mean what they normally mean: consistency here is a very strong guarantee about a single value, and availability is a very strong claim that *every* non-failing node answers.

constraints [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

constructor [Day 044](#) — `_init_` is the constructor: the code that runs when a new object is made, whose job is to leave the object in a valid state.

container [Day 013](#) — A container is an ordinary process — the same kind of process you met on [[day 008](#)](../day-008-reading-a-problem/README.md) — that has been given a restricted view of the machine it is running on and a hard limit on what it may consume.

container image [Day 012](#) — The output is one immutable artefact — most commonly a container image — containing your code, its dependencies, and the exact runtime version, built the same way every time.

containerd [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

context [Day 073](#) — The object — the context — holds one of those classes and forwards every request to it.

coordinator [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

correlation id [Day 091](#) — So every record carries a correlation id — generated at the edge, stored in a context variable, and attached automatically by the framework rather than passed through every function signature.

cost [Day 032](#) — It enumerates the plausible combinations, assigns each a cost — an abstract number, not milliseconds — and picks the smallest.

count map [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

counter [Day 063](#) — Going through the messages once and bumping a number is building a frequency map, also called a count map or a counter — all three names mean the same thing and interviewers use all three.

cri-o [Day 013](#) — Docker's contribution was making it usable, and the format is now standardised by the OCI (Open Container Initiative) so that other tools — Podman, containerd, CRI-O — run the same images.

critical path [Day 134](#) — The number of levels is the critical path — the minimum time to finish everything if you had unlimited workers.

cycle [Day 125](#) — Cyclic or acyclic is the third. A cycle is a path that starts and ends at the same vertex without reusing an edge.

D

dangerous [Day 104](#) — Compact, and dangerous: any statement that is not deterministic gives a different result on the follower.

dashed [Day 050](#) — Four: implements an interface — the same triangle, but a dashed line:

dau [Day 097](#) — DAU — daily active users.

deep-copied [Day 067](#) — The cooker and the dabbas were deep-copied — genuinely new things that happen to look the same.

deleted [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

delta of delta [Day 137](#) — So you store the difference of the differences — the delta of delta — which is almost always zero, and zero costs one bit.

dependency [Day 053](#) — A dependency is anything a class needs in order to do its job but does not own — a repository, a mailer, a clock, a payment gateway.

depth [Day 046](#) — First, depth: with Vehicle → MotorVehicle → Car → ElectricCar → LeasedElectricCar, understanding one method means reading five files, and a change at the top can break anything below it, at any depth.

depth-first [Day 100](#) — Going depth-first means you follow one branch all the way to the bottom before backing up and trying the next one, and there are exactly three places you can do the visiting: before the children, between them, or after them.

deque [Day 031](#) — The fix for max is to keep a deque — a double-ended queue, from [\[day 074\]\(../day-074-deques-and-window-max/README.md\)](#) — holding indices, arranged so that their values are decreasing from front to back.

diminishing returns [Day 135](#) — But with diminishing returns: the eighth mention adds much less than the second.

direct addressing [Day 066](#) — The wall of numbered holes is direct addressing: because the keys are the numbers 1 to 120, you do not need to hash anything.

domain name [Day 003](#) — "Anjali Sharma" is the domain name. A domain name is a human-readable label for a machine or a group of machines: google.com, leetcode.com, api.github.com.

draw two diagrams [Day 050](#) — If your design genuinely has fifteen classes, draw two diagrams: the core five or six, and then a second showing one subsystem in detail.

duck typing [Day 047](#) — This is duck typing: if it has a send method that takes a string, it is a notifier.

dummy node [Day 080](#) — Sister Mary is a dummy node — sometimes called a sentinel head.

E

earlier [Day 118](#) — The safe pattern: the leader treats its lease as expiring earlier than the grantor does, so there is a gap in which nobody believes they are leader.

edge cases [Day 008](#) — A problem statement has four things in it: what you are given, what you must return, the constraints that bound the input, and the edge cases that the ordinary solution gets wrong.

edge compute [Day 103](#) — There is a real modern exception, and mentioning it is a good signal: edge compute — Cloudflare Workers, Lambda@Edge — runs your code at the edge, so some personalisation and some logic can happen there without a trip to the origin.

effect [Day 113](#) — Say it that way: "exactly-once delivery is impossible; exactly-once effect is achievable, with at-least-once delivery and an idempotency key." Systems that advertise exactly-once are doing precisely this.

elastic scaling [Day 100](#) — The locum starting with no handover is elastic scaling: add a machine and it is immediately useful.

election timeout [Day 118](#) — "By half past five" is the election timeout — the guess that the leader is gone.

empty [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

etl [Day 139](#) — ETL versus ELT is about where the reshaping happens. ETL — extract, transform, load — transforms on a separate machine before loading, which is what you did when warehouse compute was scarce.

event object [Day 072](#) — In practice you push a small, immutable event object — a plain record describing what happened, like `OrderPlaced(order_id, user_id, total, items)`.

eventual consistency [Day 115](#) — At the bottom is eventual consistency: if writes stop, the copies will agree — eventually, with no promise about when or what you see meanwhile.

exactly [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

exactly correct [Day 089](#) — This is exactly correct — no boundary effect, no approximation — and it is the version you already wrote on [\[day 077\]\(../day-077-stacks-queues-revision/README.md\)](#) as a deque of timestamps.

exactly one [Day 171](#) — XOR is exactly one — which is the same as "different", and that is the property the whole of the next XOR day rests on.

exclusive lock [Day 035](#) — When a transaction updates a row, the database gives it an exclusive lock on that row: nobody else may change the row until the transaction commits or rolls back.

expiry [Day 090](#) — The dates are expiry — a completely separate mechanism.

F

facts [Day 082](#) — So: a FinePolicy. The loan holds the facts — who, what, when it went out, when it was due, when it came back.

false negative [Day 124](#) — A false negative is failing to notice a machine that really has died.

false positive [Day 124](#) — A false positive is declaring a healthy machine dead.

fanout [Day 030](#) — The critical property is the fanout: because a page is 8 KB and an entry is small, one node holds hundreds of entries.

fee [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day.*

fifo [Day 090](#) — FIFO — evict the oldest inserted.

file [Day 093](#) — A file is a thing that is not a box: a plate, a lamp.

filled diamond [Day 050](#) — A filled diamond means *composition* — the parts die with the whole, so Order ♦ — OrderLine because a line item has no meaning without its order.

finite state machine [Day 073](#) — This is a finite state machine — a fixed set of states, a fixed set of events, and a table saying which event moves you from which state to which.

float [Day 081](#) — Devi's second tin is a float: coins reserved so tomorrow can start.

four-direction move [Day 096](#) — "Each one immediately beside the one before" is the four-direction move: up, down, left, right.

fragile [Day 021](#) — It is also fragile: it silently breaks on capital letters, spaces, digits and anything non-English, and §7 shows exactly how silently.

frontend [Day 002](#) — The frontend is the code that crosses the counter.

G

galloping mode [Day 057](#) — When merging, if one run keeps winning, Timsort switches to galloping mode — it binary-searches ahead to find how many elements it can take in one go instead of comparing one at a time.

gap [Day 047](#) — The gap is the answer she is searching for — not something in the array, a number she is choosing.

generator expression [Day 020](#) — That is a generator expression — it produces each item as join asks for it.

greedy simulation [Day 046](#) — It is a greedy simulation: load as much as fits, then start a new trip.

guest operating system [Day 013](#) — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

H

hash map [Day 066](#) — Kiran's phone is a hash map: a general system that can look up anything by any key, at the price of storing the keys and computing where they go.

hash ring [Day 108](#) — The eleven-kilometre loop is the hash ring — the space of all hash values, joined end to end.

hateoas [Day 016](#) — And then there is the seventh idea, part of the uniform interface, that everyone quotes and almost nobody implements: HATEOAS — Hypermedia As The Engine Of Application State.

header interface [Day 058](#) — A header interface is named after the implementation and lists everything it can do: Repository, Machine, UserService.

heap [Day 117](#) — The six letters on the table are the heap: one candidate per list.

heapsort [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A* — the nearest unvisited node - top-k problems — [[day 116](#)](../day-116-top-k/README.md) - merging k sorted lists — [[day 117](#)](../day-117-merge-k-sorted/README.md) - the running median — [[day 118](#)](../day-118-two-heaps/README.md), with two heaps -...

heartbeat [Day 124](#) — The whistle is a heartbeat. A heartbeat is a small, regular message that says "I am alive".

hidden spof [Day 111](#) — The shared approach road is a hidden SPOF — two components that are not independent.

hiding mechanism [Day 052](#) — Abstraction is about hiding mechanism: the caller says "charge this card" and does not learn which payment provider ran it.

highest node [Day 104](#) — Every path has exactly one highest node — the point where it turns around.

historical snapshot [Day 029](#) — | Copy | Instead of | Why | |---|---|---| | posts.comment_count | COUNT(*) on comments | counting on every read is unaffordable at a high read ratio | | orders.customer_name | joining to customers | a historical snapshot: the name at the time of the order | | order_items.unit_price | joining to products | the price *then*, which must...

hollow triangle [Day 050](#) — Three: inheritance — a line with a hollow triangle pointing at the parent:

hook [Day 075](#) — award is a hook: the base class implements it as *doing nothing*, so a subclass may override it and most do not.

horizontal scaling [Day 098](#) — Horizontal scaling means putting more machines beside it and dividing the work.

hypervisor [Day 013](#) — A hypervisor — VMware ESXi, KVM, Xen, Microsoft Hyper-V — divides the physical machine into virtual ones, and each of those runs its own complete guest operating system: its own kernel, its own init system, its own device drivers, its own scheduled background jobs.

I

image [Day 013](#) — An image is a read-only stack of filesystem layers — a base operating system's files, then your dependencies, then your code.

implicit graph [Day 130](#) — This is an implicit graph — the edges exist by rule rather than by record — and you met the idea on [[day 125](#)](../day-125-what-a-graph-is/README.md).

in parallel [Day 111](#) — Components in parallel — any one will do — multiply their *failure* probabilities, which is a completely different shape.

in series [Day 111](#) — Components in series — every one is needed — multiply their availabilities, so the total is always worse than the worst one.

in-degree [Day 125](#) — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

index [Day 025](#) — A database keeps an index — a separate structure, usually a B-tree, that maps a value to the location of the matching rows.

indexed heap [Day 114](#) — Hence the indexed heap: a hash map from value to index, updated on every swap.

inheritance [Day 071](#) — Template Method uses inheritance — an abstract base with the skeleton and subclasses filling in the holes.

inorder [Day 110](#) — The standing order is inorder: left subtree, then root, then right subtree.

input [Day 008](#) — In a problem: the input — its type, its size, and what is in it.

integrity [Day 006](#) — TLS also guarantees integrity: each piece carries a check value, so if anything in transit is altered by even one bit, the other end notices and drops the connection.

intent plus multiplicity [Day 070](#) — The honest distinction is intent plus multiplicity: you stack decorators and expect several; a proxy is usually one and is about access rather than behaviour.

interface [Day 052](#) — An interface is a pure list of method signatures with no state and no implementation — a contract that says what a type can do.

invariant [Day 028](#) — That is called an invariant — something true before the loop starts, still true after every turn, and therefore true when the loop ends.

is possible [Day 046](#) — $\rightarrow$ *Find the smallest X such that some task is possible.* $\rightarrow$ *Find the largest X such that some condition still holds.*

item [Day 016](#) — An item is one of them: /bags/213.

iterator [Day 075](#) — Iterator is the pattern where "go through this collection one item at a time" is separated from "this collection".

J

json [Day 015](#) — JSON — JavaScript Object Notation — is the plain-text format most web APIs use for both request bodies and responses, and you met it on [day 007](../day-007-space-complexity/README.md); it looks like {"id": 42, "name": "Zoya"}.

jump [Day 094](#) — A snake and a ladder are both a jump: a pair (from, to).

jwt [Day 020](#) — a session id — an opaque reference, meaningless by itself, that the server looks up; - a JWT — a self-contained signed token that carries the facts inside it, so no lookup is needed; - OAuth 2.0 and OpenID Connect — a protocol for getting one of the above from *somebody else*, so a user can log in with Google without your service ever...

K

k closest points [Day 116](#) — k closest points — the same shape with a computed key.

k most frequent [Day 116](#) — k most frequent — count first, then top-k on the counts.

k-th largest specifically [Day 116](#) — k-th largest specifically — quickselect is the natural fit, because you want one element rather than a set, and quickselect gives it directly.

key [Day 023](#) — The key is as much of the design as the algorithm.

key property [Day 108](#) — Adding the temple point disturbing only sixty houses is the key property: adding a node moves only the keys between it and its predecessor.

L

lag [Day 112](#) — | Move | Price | |---|---| | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

largest [Day 055](#) — the 1st smallest is at position 0, so the k-th smallest is at position k - 1; - the 1st largest is at position n - 1, so the k-th largest is at position n - k.

latency [Day 089](#) — The cost is latency — a request may wait in the queue — and that makes it wrong for a user-facing API, where you would rather refuse quickly than hold a connection.

lazy [Day 117](#) — Merging sorted streams — `heapq.merge(*iterables)` does exactly this and is lazy, so it works on inputs larger than memory.

lazy deletion [Day 114](#) — Python has neither. The idiomatic workaround is lazy deletion: push a new entry and mark the old one stale, skipping stale entries as they surface.

lazy loading [Day 041](#) — That is lazy loading: the ingredient bought at the moment it is needed.

lazy refill [Day 089](#) — The lazy refill is the implementation trick: no timer, no background job.

leader [Day 105](#) — Counter two is the leader: writes go there, and reads from it are always current.

leaf [Day 098](#) — He is a leaf — an item with no children.

lease [Day 118](#) — A lease is leadership with an expiry: you are the leader until time T unless you renew.

ledger entries [Day 085](#) — An expense produces a set of ledger entries: the payer is owed, each participant owes their share.

left [Day 040](#) — Remove the strip to the left: `prefix[r2+1][c1]`.

lfu [Day 090](#) — LFU — evict the least frequently used.

linear scan [Day 062](#) — Scrolling the whole list of names is a linear scan: to answer one question you touch everything.

linearizability [Day 114](#) — C — Consistency. Specifically linearizability: every read sees the most recent write, and the system behaves as if there were one copy of the data.

list [Day 001](#) — heap is a list: a row of values kept in order, like the socks piled on the bed.

list of lists [Day 016](#) — A matrix is a list of lists: the outer list holds the rows, and each row is itself a list holding that row's values.

load balancer [Day 098](#) — Sending students to the right counter is a load balancer.

load factor [Day 060](#) — The number that controls this is the load factor: the number of items divided by the number of buckets.

load shedding [Day 007](#) — This idea has a name — load shedding — and it is one of the things that separates a system that degrades from one that collapses.

log-structured merge tree [Day 031](#) — A log-structured merge tree — used by Cassandra, RocksDB, LevelDB and ScyllaDB — takes the opposite trade.

logical cohesion [Day 061](#) — Anjali's first kitchen is logical cohesion — grouped by kind, which looks tidy and is wrong, because the unit of use is the job, not the kind.

logical time [Day 113](#) — The replacement is logical time — Lamport clocks and vector clocks — which order events by **causality** rather than by wall time.

loop [Day 001](#) — A loop is a statement that says, "do the following again and again".

loop body [Day 002](#) — A loop body is the indented block underneath the loop header — the part that runs again and again.

low cohesion [Day 061](#) — The kitchen arranged by size is low cohesion — things used together kept apart, so every job takes trips.

lru [Day 090](#) — LRU — evict the least recently used.

M

manifest [Day 158](#) — Adaptive bitrate streaming. The client downloads a manifest — an HLS `.m3u8` or DASH `.mpd` — listing every rendition and every segment.

many times [Day 149](#) — Virtual nodes fix that. Each physical machine is placed on the circle many times — 100 to 256 positions each, by hashing machine-1#0, machine-1#1, and so on.

many-to-many [Day 026](#) — | Shape | Example | Where the key goes | |---|---|---| | one-to-many | a user has many posts | on the many side: posts.author_id | | many-to-many | a post has many tags, a tag has many posts | a third table: post_tags(post_id, tag_id) | | one-to-one | a user has one profile | either side; usually the optional one |

matrix [Day 016](#) — A matrix is a rectangle of values addressed by two numbers instead of one: which row, then which column.

mau [Day 097](#) — MAU — monthly active users.

max-heap [Day 116](#) — The mirror rule: to find the k smallest, keep a max-heap of size k — in Python, a min-heap of negated values.

method resolution order [Day 052](#) — The Python answer. The method resolution order — `D._mro_` — is computed by a rule called C3 linearisation, and `super()` follows that order rather than jumping straight to a parent.

milliseconds [Day 009](#) — About 10 milliseconds — a hundred times slower again than SSD, and a hundred thousand times slower than RAM.

min-heap [Day 118](#) — Each half is a heap, and they face each other: a max-heap holding the lower half so its largest is at the top, and a min-heap holding the upper half so its smallest is at the top — the two elements at the boundary are exactly the ones you need. And the only real work is keeping the two halves the same size, which is two lines and is...

minimum spanning forest [Day 139](#) — Both algorithms then produce a minimum spanning forest — one tree per component — and you detect it by counting: fewer than $V - 1$ edges taken means the graph was disconnected.

mixin [Day 049](#) — A mixin is a small class carrying one piece of behaviour, meant to be inherited alongside others:

monotone [Day 043](#) — A yes-or-no question with that shape is called monotone: once the answer flips from no to yes, it stays yes.

monotonic deque [Day 074](#) — The good solution keeps a small monotonic deque of "elements that could still be the maximum" and is $O(n)$ overall.

N

name mangling [Day 045](#) — Two underscores triggers name mangling: inside the class the attribute is rewritten as `_Account_key`, so a subclass defining its own `_key` does not collide.

namespace [Day 013](#) — A namespace is a kernel feature that gives a process its own private version of one part of the system.

negative [Day 035](#) — A sum over values that can be negative is not monotonic — the window is the wrong room, and the case goes to prefix sums, which arrive on [day 037](../day-037-prefix-sums/README.md).

negative infinity [Day 049](#) — LeetCode 162 states it: treat `nums[-1]` and `nums[n]` as negative infinity — imaginary values smaller than anything real, just off each end.

negatives [Day 038](#) — The map earns its space exactly when negatives are possible or the question is exactly — the two places monotonicity fails.

nested loop [Day 001](#) — A nested loop is a loop inside another loop.

network partition [Day 114](#) — The exchange fault is a network partition — the nodes are alive but cannot talk.

never equal [Day 115](#) — Two properties, and it needs both: it is always comparable (integers), and it is never equal (strictly increasing).

never forgets [Day 090](#) — Needs a third structure and a min_frequency counter to stay $O(1)$, and it never forgets: an entry that was hot last month holds a huge count and will not leave.

never persisted [Day 157](#) — Typing indicators are presence with a shorter fuse. Same mechanism, a two-to-three second TTL, throttled so a keystroke does not become a network message, and never persisted — a typing event that arrives late is worse than one that is dropped.

no children [Day 121](#) — It has no children — nothing continues past it.

node [Day 125](#) — The word node means the same thing and both are used; this course will say vertex when talking about the structure and node when talking about code.

not a map [Day 067](#) — | not a map — heap or sorted array |

O

$o(1)$ [Day 079](#) — tail — a reference to the last node, which makes append $O(1)$ instead of $O(n)$.

object [Day 043](#) — An object is data and the behaviour that belongs to that data, kept together in one place.

observers [Day 072](#) — The objects on the list are the observers.

occupied [Day 061](#) — A slot is empty, occupied, or deleted — a marker meaning "something was here; keep probing".

offset [Day 130](#) — Each viewer's position is an offset. An offset is just a number — the position in the log that a particular reader has got to.

on first touch [Day 041](#) — Each `post.author` is a *different table*, not yet loaded — so the ORM, helpfully and silently, fetches it on first touch.

one-shot [Day 075](#) — A list can be walked many times; a generator is one-shot — walk it twice and the second walk is empty, which is a bug people hit constantly.

opens [Day 126](#) — When failures exceed a threshold, it opens: from then on, calls to that dependency fail instantly without being attempted.

orchestrator [Day 012](#) — That something is an orchestrator — Kubernetes, AWS ECS, Nomad — or, on a smaller system, systemd on a virtual machine.

origin [Day 101](#) — The storeroom is the origin — the source of truth.

origin pull [Day 103](#) — Sending the finished pages down the wire is origin pull: the edge fetches from the origin on demand.

orphan rows [Day 026](#) — Without it you get orphan rows — a post whose author does not exist — and they are impossible to clean up later, because you cannot tell which ones were mistakes.

orphans [Day 134](#) — You get orphans — objects nobody references — and broken links — rows pointing at nothing.

out-degree [Day 125](#) — In a directed graph you count separately: in-degree is arrows coming in, out-degree is arrows going out.

overloading [Day 047](#) — Compile-time polymorphism, or overloading: several methods with the same name and different argument types, and the compiler picks.

overriding [Day 047](#) — Runtime polymorphism, or overriding: a subclass replaces a parent's method, and which one runs is decided by the object at run time.

P

page [Day 009](#) — A disk never reads one byte; it reads a block or page, typically 4 KB — and PostgreSQL reads 8 KB pages, and SSDs erase in blocks of megabytes.

parallel edges [Day 132](#) — And the case where excluding by vertex is not enough. If the graph can have two separate edges between the same pair — parallel edges — then A—B twice genuinely **is** a cycle, and skipping "the parent vertex" wrongly ignores it.

parameters [Day 015](#) — In an API those are the parameters — the values you must supply with a request — and leaving a required one out gets you an error, not a guess.

parent [Day 132](#) — The fix: pass down where you came from, and skip it. Each recursive call carries the vertex that called it — its parent — and the check becomes "seen, and not my parent".

partial failure [Day 113](#) — The one that matters most is partial failure: a call over a network has a third outcome that does not exist locally, *"I do not know"*, and almost every difficult problem in the rest of this phase is a consequence of it.

participants [Day 120](#) — The roles. One coordinator — Kalpana — and several participants, the machines holding the data.

partition key [Day 039](#) — A wide-column store — Cassandra is the name to know — organises data by two keys: a partition key that decides which machine (and which bucket on it) a row belongs to, and clustering columns that keep rows **sorted inside** that bucket.

partitions [Day 130](#) — It is split into partitions — independent logs that together make up the topic.

path [Day 005](#) — A request has four parts: a method saying what you want done, a path saying what you want it done to, headers carrying facts about the request itself, and an optional body carrying the content.

port [Day 003](#) — A port is a second number that says which program on that machine you want, the way a flat number says which door inside the building.

prefix sums [Day 037](#) — His noted meter readings are the prefix sums — and his godown reading, taken before anything was driven, is the extra zero at the front that today's code lives or dies by.

preorder [Day 110](#) — The calling order is preorder: root, then left subtree, then right subtree.

priority queue [Day 139](#) — Finding "the cheapest edge leaving the tree" efficiently means a priority queue — which makes Prim's structurally almost identical to [Dijkstra's](../day-136-dijkstra/README.md).

procedure [Day 062](#) — It is a procedure: a fixed order in which to read a piece of code you have never seen, so that by the end of it you can say what is wrong, what evidence you have, and what you would change first.

process [Day 008](#) — A process is a running program with its own private memory.

program [Day 001](#) — A program is a sequence of statements.

protecting state [Day 052](#) — Definition. Encapsulation is about protecting state: the balance cannot go negative because the only way to change it runs a check.

Q

qps [Day 097](#) — QPS stands for **queries per second** — the number of requests arriving at your system each second.

queue [Day 127](#) — The queue holds the frontier in order. A queue is a line where things come out in the order they went in — first in, first out — which you met on [day 73](../day-073-queues/README.md).

quorum [Day 118](#) — "Three of the four must agree" is a quorum — the majority from [day 117](../day-117-merge-k-sorted/README.md).

R

race condition [Day 008](#) — A race condition is when the result depends on the exact timing of two things running at once.

ram [Day 009](#) — The suitcase on the almirah — main memory, RAM. RAM — random access memory — is where your running program's data lives.

rank [Day 138](#) — Union by rank is the same idea using an upper bound on height rather than the exact size; both work, size is easier to reason about, and size gives you group sizes for free, which problems often want.

rebalancing [Day 107](#) — Moving twenty bundles is rebalancing — moving whole logical shards, not rehashing keys.

rebuilding the tree [Day 109](#) — Repacking the whole cart is rebuilding the tree — $O(n)$.

recursion [Day 053](#) — That is recursion: a function whose body calls the same function on a smaller piece of the problem.

references [Day 009](#) — A Python list stores references — memory addresses of objects that live elsewhere:

refill rate [Day 023](#) — | The tank | The algorithm | |---|---| | 500-litre capacity | burst size — the most you can spend at once | | 20 litres an hour going in | refill rate — the sustained rate you are allowed | | taking water to wash | spending a token; one request costs one token | | the tank being empty | no tokens left → the request is refused, 429 | |...

register [Day 009](#) — The chair beside the bed — registers. A register is a tiny store inside the processor itself, holding one value.

relative epsilon [Day 048](#) — The fix is a relative epsilon — stop when the gap is small *compared to the value*:

removal [Day 118](#) — The same two heaps, plus removal, which a heap cannot do.

replica [Day 136](#) — A replica is a copy of a shard on another node, serving reads and taking over if the primary fails.

replica divergence [Day 115](#) — Different people knowing different things is replica divergence — and none of them is wrong.

request-aware routing [Day 099](#) — The joint-family card going to counter one is request-aware routing — some requests are much more expensive than others.

resolver [Day 021](#) — Each field in the schema has a resolver — a function that knows how to produce that field.

resources [Day 017](#) — Before writing a single path, list the resources — the things the feature is about.

returned [Day 044](#) — If a copy is > returned late, a fee is charged at ₹2 a day."

ring buffer [Day 073](#) — That is a circular buffer, also called a ring buffer: a fixed block of slots where position $7 + 1$ is position 0.

role interface [Day 058](#) — A role interface is named after what a client needs: UserReader, Printer, PriceQuoter.

root [Day 093](#) — The root is the loft itself, written /.

rotation [Day 109](#) — The fix is a rotation: a local rearrangement of three or four pointers that reduces the height by one without touching the rest of the tree or changing the inorder order.

routine [Day 109](#) — Counting paces and multiplying is the routine — the same sequence every time, said out loud.

row [Day 026](#) — A row is one of those things.

run [Day 044](#) — So the copies of a value form one run: a contiguous stretch, described completely by its two ends.

S

same remainder [Day 038](#) — A stretch is divisible by k when the prefixes at its two ends leave the same remainder — so carry running $\% k$ and count matching remainders.

schema [Day 021](#) — A GraphQL API is defined by a schema: every type, every field, and every field's type, written down and strongly typed.

scoping [Day 096](#) — The two or three narrowing questions are scoping — deciding out loud what you are *not* building.

script [Day 077](#) — Today is the script — the fixed order of moves you run on every one of these prompts for the next twenty days.

seam [Day 053](#) — A seam is a place where you can change behaviour without editing the class.

search space [Day 042](#) — The stretch `nums[low..high]` is called the search space — the part of the array you have not ruled out yet.

selectivity [Day 032](#) — From those numbers the planner estimates selectivity: what fraction of rows a condition keeps.

sentinel [Day 037](#) — That extra entry is called a sentinel: a value placed at the boundary so that the boundary needs no special treatment.

separator [Day 019](#) — The string before the `.join` is the separator — `"".join` glues with nothing between, `",".join` puts a comma and a space between each pair.

sequence diagram [Day 050](#) — That is a sequence diagram: participants as columns, time running down, and each message an arrow from one column to the next.

serialisable [Day 033](#) — The strongest form is serialisable: the result is as if the transactions had run one after another, in some order.

service credits [Day 173](#) — An SLA is a contract with a customer: a promise, and a consequence when you break it. The consequence is almost always service credits — money back off the next bill — rather than damages.

session [Day 020](#) — A session is a record the server keeps about a logged-in user.

session guarantees [Day 115](#) — And the interesting rungs are in the middle, particularly causal consistency and the four session guarantees, because they are what users actually notice and they are much cheaper than the top.

set [Day 006](#) — A set stores values and answers "is this in here?" in a single step, no matter how many it holds.

shallow-copied [Day 067](#) — The milk and the newspaper were shallow-copied — the new kitchen got a *reference* to the same arrangement, and changing it from either end changed it for both.

shape [Day 003](#) — It throws away every detail that does not change as the input grows — the exact number of steps, how fast your laptop is, whether one line takes two nanoseconds or twenty — and keeps only the shape of the growth.

shard [Day 106](#) — Each room is a shard — a machine holding a subset of the rows.

shared lock [Day 035](#) — A shared lock is for reading: many transactions can hold one on the same row at once, because readers do not disturb each other.

shared store [Day 100](#) — The shared cabinet at the front desk is a shared store — a database, or Redis.

singleton [Day 064](#) — Singleton is the pattern that guarantees a class has exactly one instance, and gives everybody a global way to reach it.

singly linked list [Day 076](#) — A singly linked list — where each node knows only the **next** one — cannot do that.

slice [Day 019](#) — `s[1:4]` is a slice — a new string made of part of the old one.

smallest [Day 046](#) — `>` **Find the smallest X such that some task is possible.** `>` **Find the largest X such that some condition still holds.**

snapshot [Day 034](#) — The transaction takes a snapshot at its first read and every statement in it sees that snapshot, however long it runs.

snipe [Day 095](#) — A snipe is a bid placed in the last second, leaving nobody time to respond.

sorted region [Day 052](#) — And the sorted region is the part of the list you have already finished; every one of these methods grows a sorted region and shrinks an unsorted one, and they differ in **which end** the region grows from and **how** the next value gets there.

sorting everything [Day 116](#) — The nephew laying out the whole load is sorting everything — correct, and far more work than asked for.

splits [Day 031](#) — If the leaf is full, it splits: half the keys move to a new page, and a key is pushed up to the parent.

spof [Day 111](#) — The bridge is an SPOF — one component, and everything depends on it.

squares [Day 003](#) — | Example | `---|---|---|` | $O(1)$ | constant | does not change | reading items[5] | | $O(\log n)$ | logarithmic | goes up by one step | halving until you reach 1 | | $O(n)$ | linear | doubles | one loop over the list | | $O(n \log n)$ | linearithmic | slightly more than doubles | sorting | | $O(n^2)$ | quadratic | goes up four times | every...

stable [Day 051](#) — Python's sort is Timsort: $O(n \log n)$ worst case, stable, and adaptive — it finds runs that are already ordered and merges them, so nearly-sorted input runs close to $O(n)$.

stack [Day 075](#) — The fridge is a stack: last in, first out.

stack frame [Day 068](#) — When a function calls another function, the machine pushes a stack frame — the local variables and the place to return to — onto a stack.

stack overflow [Day 068](#) — And the bad Sunday is a stack overflow: too many unfinished things, and the one at the bottom is forgotten.

state [Day 143](#) — And the state is the thing to get right. The state is what identifies a subproblem — the arguments to the recursive function.

statements [Day 001](#) — A program is a sequence of statements.

status [Day 086](#) — It has a status — available, locked, booked — and a price, because the same seat costs more on a Saturday night.

still one machine [Day 112](#) — | Move | Price | `---|---` | Scale up | a ceiling, downtime to resize, and still one machine — no availability gain | | Cache | staleness, invalidation, and a stampede when a hot key expires | | CDN | purges are slow; nothing personalised; cache-key explosion from query strings | | Stateless | a shared session store becomes a hot...

stonith [Day 118](#) — STONITH — "shoot the other node in the head": the new leader forcibly disables the old one, by cutting power or revoking its storage lease, before accepting any write.

strategy [Day 071](#) — Strategy turns an algorithm into a value.

strictly shorter [Day 072](#) — `left[i]` — the index of the nearest bar to the left that is strictly shorter.

structural [Day 058](#) — In Python, typing.Protocol is structural — it matches on the shape of the methods, and the implementing class inherits nothing and imports nothing ([\[day 052\]\(../day-052-quadratic-sorts/README.md\)](#)).

subarray [Day 024](#) — There is a third word you will meet: a subarray is the same thing as a substring, for arrays.

subject [Day 072](#) — Observer is the pattern where one object — the subject — keeps a list of other objects that want to be told when something happens to it, and tells all of them with one call.

subscription list [Day 072](#) — The group itself is the subscription list, and the one message is the notification.

substring [Day 155](#) — First the definitions. A substring is contiguous — "aba" is a substring of "xabay", but "aa" is not.

subtree [Day 122](#) — If you get to the end, the node you are standing on is the root of a subtree — the whole set of words that begin with what the user typed.

surge multiplier especially [Day 088](#) — The surge multiplier especially is frozen.

swap [Day 052](#) — A comparison is one question: *is this one bigger than that one?* A swap is exchanging two values so each sits where the other was.

system call [Day 011](#) — The mechanism for asking is a system call, and the boundary it crosses is the most important line in a computer.

T

table [Day 026](#) — A table holds one kind of thing.

tail call [Day 088](#) — A tail call is a recursive call whose result is returned directly, with nothing left to do afterwards:

tail-recursive [Day 090](#) — This version is also tail-recursive — nothing happens after the call returns — which a language with tail-call elimination would turn into a loop for free.

tcp [Day 004](#) — TCP and UDP are the two ways one machine sends data to another.

template [Day 092](#) — The saved wording with the date and the hours swapped in is a template — a fixed piece of text with holes in it, filled in per message.

test double [Day 053](#) — A test double is a stand-in you pass in during a test.

the return address [Day 088](#) — the arguments it was called with, - the local variables it creates, - the return address — the exact point in the caller to resume at, - and a link to the frame below it.

the running median [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and Λ^* — the nearest unvisited node - top-k problems — [\[day 116\]\(../day-116-top-k/README.md\)](#) - merging k sorted lists — [\[day 117\]\(../day-117-merge-k-sorted/README.md\)](#) - the running median — [\[day 118\]\(../day-118-two-heaps/README.md\)](#), with two heaps ...

third table [Day 026](#) — | Shape | Example | Where the key goes | `|-|-|-|-|-|` | one-to-many | a user has many posts | on the many side: `posts.author_id` | | many-to-many | a post has many tags, a tag has many posts | a third table: `post_tags(post_id, tag_id)` | | one-to-one | a user has one profile | either side; usually the optional one |

thread [Day 008](#) — A thread is one line of execution inside a process, and all the threads in a process share that memory.

three distinct orderings [Day 092](#) — = 6 decision paths — but only three distinct orderings: [1,1,2], [1,2,1], [2,1,1].

three-way partition [Day 054](#) — The fix is a three-way partition: split into **less than**, **equal to**, and **greater than**, and recurse only into the outer two.

tight coupling [Day 061](#) — The balcony light on the reading lamp's switch is tight coupling — two things wired together that have no reason to be, so you cannot change one without changing the other.

time-to-live [Day 037](#) — Bhaskar's clear-out is time-to-live: attach an expiry to a key and the store deletes it, automatically.

tiny constraint [Day 097](#) — A tiny constraint: $n \leq 8$ means factorial, $n \leq 20$ means 2^n , target ≤ 500 with small candidates means combination sum.

toast [Day 134](#) — And there is a middle option worth knowing. Postgres stores large values out-of-line automatically — TOAST — in a side table, compressed, and only reads them when the column is selected.

token [Day 019](#) — The server verifies them and issues a token — a session id or a signed token.

tombstone [Day 011](#) — There is a real technique that does leave holes — marking a slot as deleted instead of removing it, called a tombstone — and databases and hash tables use it constantly.

top-k problems [Day 113](#) — task schedulers and event loops — the next event by time - Dijkstra's algorithm and A^* — the nearest unvisited node - top-k problems — [[day 116](#)](../day-116-top-k/README.md) - merging k sorted lists — [[day 117](#)](../day-117-merge-k-sorted/README.md) - the running median — [[day 118](#)](../day-118-two-heaps/README.md), with two heaps -...

total order [Day 123](#) — It gives you a total order — sort by the number, break ties by machine name — that never contradicts causality.

transitions itself [Day 071](#) — In State, the object transitions itself: a Draft order moves itself to Submitted, and the states know about each other.

transitive dependency [Day 029](#) — This is a transitive dependency: $id \rightarrow department_id \rightarrow department_name$.

tree [Day 040](#) — A domain of independent records — a tree — tolerates anything.

ttl [Day 003](#) — The family moving out is the staleness problem, and it is the reason for TTL. Every DNS answer comes with a TTL — time to live — which is a number of seconds saying how long the answer may be trusted before it must be looked up again.

tuple [Day 005](#) — A tuple is written with round brackets — (3, 7, 2) — and is a list you cannot change.

two [Day 087](#) — In fib, each call makes two — so the work explodes.

two implementations [Day 079](#) — And, as always, two implementations: NearestSuitable as above, and ZonedDispatcher where each car owns a band of floors.

U

udp [Day 004](#) — TCP and UDP are the two ways one machine sends data to another.

under-fetching [Day 021](#) — And from [[day 015](#)](../day-015-the-write-pointer/README.md)'s arithmetic: a screen needing four different resources makes four round trips, which on a mobile network is roughly 480 ms instead of 120 — under-fetching, usually called the $N+1$ problem when it happens in a list.

union filesystem [Day 013](#) — The stack of layers is a union filesystem — several directories presented as one merged tree.

upper bound [Day 044](#) — The upper bound is the first index where $\text{nums}[i] > 4$, which is index 4 — one past the run.

uri [Day 016](#) — In REST the thing is called a resource, and its address is a URI — the path in the URL.

url [Day 001](#) — A URL is an address for something on the internet.

V

value object [Day 051](#) — A value object is defined entirely by its values.

version [Day 117](#) — And the second half, which people forget: you must be able to tell which value is newest. The overlapping replica has the new value, but the others may return an old one, so every record carries a version — a counter, a timestamp, or a vector clock — and the reader takes the highest.

vertex [Day 125](#) — The places are the vertices. A vertex is one of the things in your collection — a bus stop, a person, a city, a course, a web page, a cell in a grid.

vertical scaling [Day 098](#) — Vertical scaling means replacing the machine with a bigger one — more cores, more memory, faster disk.

victim [Day 035](#) — It watches who waits for whom, and when it finds a circle, it picks a victim — one transaction in the cycle — and kills it with an error.

virtual address space [Day 011](#) — Memory. Each process gets its own virtual address space — its own set of addresses that the OS maps onto real physical memory.

virtual machine [Day 013](#) — A virtual machine is an emulated computer.

volatile [Day 009](#) — It is also volatile: switch the power off and it is gone.

W

weight [Day 122](#) — To do that in code, every word needs a weight — a number saying how often it is chosen.

which class [Day 076](#) — Factory vs Builder — a factory decides which class; a builder assembles one complicated object step by step.

which is hours [Day 166](#) — The cost is that it must be rebuilt, and that is the trade to state. New queries do not appear until the next rebuild, which is hours — and there is a separate fast path for trending terms, which is the third idea below.

within [Day 131](#) — Consumer group — Kafka's answer, and it is genuinely both: within a group, consumers compete like a queue; between groups, each gets everything like a topic.

worst case [Day 002](#) — The best case is 1, the worst case is n.

write skew [Day 034](#) — It appears in backend interviews at every product company, usually right after ACID, and the follow-up about write skew is where senior candidates are separated from the table-reciters.

write-around with invalidation [Day 102](#) — Wiping a line when the price moves is write-around with invalidation: update the truth, delete the cache entry, and let the next reader repopulate it.

write-through [Day 102](#) — Changing the board at four fifteen is write-through: update the cache when you update the truth.

Y

yes [Day 093](#) — | Problem | Recurse on | Duplicates in input | Extra rule | |---|---|---|---| | Combinations (LC 77) | $i + 1$ | no | stop at size k | | Combination Sum (LC 39) | i — reuse allowed | no | stop when remaining $= 0$ | | Combination Sum II (LC 40) | $i + 1$ — each used once | yes | sort, and skip when $i > \text{start}$ and $c[i] == c[i-1]$ | |...

References and further reading

Further reading, arranged by the part of the book it belongs to. Everything listed is a primary source, an official specification, or reference material that is free to read. None of this book is copied from any of them. They are here for the reader who wants the long version, the formal proof, or the original paper.

FOR THE WHOLE BOOK

Python Software Foundation. *The Python Language Reference and Standard Library*. <https://docs.python.org/3/> [PSF Licence, free to read]

The authority for every language behaviour this book relies on.

Python Wiki. *TimeComplexity - cost of the built-in container operations*. <https://wiki.python.org/moin/TimeComplexity> [Free to read]

The table behind almost every complexity claim about list, dict and set.

David Galles, University of San Francisco. *Data Structure Visualizations*. <https://www.cs.usfca.edu/~galles/visualization/Algorithms.html> [Free to use]

Step-through animations for most structures in Parts II to XV.

Steven Halim and others. *VisuAlgo - visualising data structures and algorithms*. <https://visualgo.net/en> [Free for self-study]

Useful when a diagram in this book is not enough and you want to press play.

MIT OpenCourseWare. *6.006 Introduction to Algorithms*. <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/> [CC BY-NC-SA]

Full lecture videos and notes. The formal treatment this book deliberately avoids.

LeetCode. *Problem set*. <https://leetcode.com/problemset/> [Free tier]

Where every problem named in the practice sheets is solved.

DATA STRUCTURES AND ALGORITHMS, BY PART

PART XVI · DYNAMIC PROGRAMMING (DAYS 143–163)

Wikipedia. *Dynamic programming*. https://en.wikipedia.org/wiki/Dynamic_programming [CC BY-SA]

Optimal substructure and overlapping subproblems, defined properly.

MIT OpenCourseWare. *6.006 dynamic programming lectures*. <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/> [CC BY-NC-SA]

Four lectures that cover the same ground as Part XVI, formally.

Wikipedia. *Knapsack problem*. https://en.wikipedia.org/wiki/Knapsack_problem [CC BY-SA]

The 0/1 and unbounded variants, and what pseudo-polynomial means.

PART XVII · GREEDY AND INTERVALS (DAYS 164–170)

Wikipedia. *Greedy algorithm*. https://en.wikipedia.org/wiki/Greedy_algorithm [CC BY-SA]

When greedy is provably correct, which is the whole question in Part XVII.

Wikipedia. *Interval scheduling*. https://en.wikipedia.org/wiki/Interval_scheduling [CC BY-SA]

The exchange argument behind sorting by end time.

Wikipedia. *Matroid*. <https://en.wikipedia.org/wiki/Matroid> [CC BY-SA]

The structure that explains why greedy works when it works. Optional.

SYSTEM DESIGN, BY PHASE

BUILDING BLOCKS OF BIG SYSTEMS (DAYS 129–144)

Apache Software Foundation. *Kafka design documentation*. <https://kafka.apache.org/documentation/#design> [Apache 2.0]

Partitions, offsets and delivery guarantees, from the source.

Jeffrey Dean and Sanjay Ghemawat, Google. *MapReduce: Simplified Data Processing on Large Clusters*. <http://research.google/pubs/mapreduce-simplified-data-processing-on-large-clusters/> [Free to read]

The paper that started the batch-processing chapter of every system design book.

Fay Chang and others, Google. *Bigtable: A Distributed Storage System for Structured Data*. <https://research.google/pubs/bigtable-a-distributed-storage-system-for-structured-data/> [Free to read]

LSM trees and column families, which underlie most wide-column stores.

Elastic. *Elasticsearch guide*. <https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html> [Free to read]

Inverted indexes in production, for the search-service designs.

Adam Wiggins. *The Twelve-Factor App*. <https://12factor.net/> [CC BY-SA 3.0]

Short. Explains configuration, statelessness and disposability better than most books.

HIGH-LEVEL DESIGN CASE STUDIES (DAYS 145–170)

Netflix. *Netflix Technology Blog*. <https://netflixtechblog.com/> [Free to read]

Real architecture write-ups for streaming, caching and chaos engineering.

Uber. *Uber Engineering Blog*. <https://www.uber.com/en-IN/blog/engineering/> [Free to read]

Geospatial indexing and dispatch, which is the ride-hailing case study.

Discord. *Discord Engineering Blog*. <https://discord.com/category/engineering> [Free to read]

Messaging at scale, with real numbers attached.

Meta. *Engineering at Meta*. <https://engineering.fb.com/> [Free to read]

News feed, TAO and photo storage - the source material for several case studies.

Todd Hoff. *High Scalability*. <http://highscalability.com/> [Free to read]

An archive of architecture write-ups going back years.

Colophon: how this book was made

This book was typeset from the course sources with free software only. Every typeface is published under the SIL Open Font Licence, every library under a permissive licence, and the page engine is the open-source Chromium print pipeline. Licence labels below describe the linked project, not this book.

TYPE, TOOLS AND LIBRARIES

SIL International, after Matthew Carter. *Charis SIL*. <https://software.sil.org/charis/> [SIL Open Font Licence 1.1]

The text face. An extension of Bitstream Charter, which Carter drew in 1987 to stay readable on coarse output. It is the standard free choice for technical books.

Bold Monday for IBM. *IBM Plex Sans and Plex Sans Condensed*. <https://github.com/IBM/plex> [SIL Open Font Licence 1.1]

Headings, labels, tables, folios and the covers.

JetBrains. *JetBrains Mono*. <https://github.com/JetBrains/JetBrainsMono> [SIL Open Font Licence 1.1]

All code, output and ASCII diagrams.

Google. *Noto Serif Devanagari*. <https://github.com/notofonts/devanagari> [SIL Open Font Licence 1.1]

The Devanagari on the covers and title page.

Knut Sveidqvist and contributors. *Mermaid*. <https://github.com/mermaid-js/mermaid> [MIT Licence]

Renders every flow chart, sequence diagram and tree.

Georg Brandl and contributors. *Pygments*. <https://pygments.org/> [BSD 2-Clause]

Colours the code. The palette used here was written for this book.

Waylan Limberg and contributors. *Python-Markdown*. <https://python-markdown.github.io/> [BSD 3-Clause]

Turns the lesson sources into the pages you are reading.

Martin Thoma and contributors. *pypdf*. <https://github.com/py-pdf/pypdf> [BSD 3-Clause]

Joins the covers, front matter and body, and writes the bookmarks.

The Chromium Authors. *Chromium print-to-PDF, driven by Puppeteer*. <https://pptr.dev/> [Apache 2.0 / BSD 3-Clause]

The typesetting engine. No proprietary software was used to make this book.

HOW A PAGE WAS MADE

The lesson sources are Markdown. Fenced blocks are lifted out before the Markdown is parsed, so diagrams keep every space they have and code reaches the colouring pass untouched. Each monospace block is measured, and its type size chosen so the widest line fits the column rather than falling off the page.

The result is rendered in a browser engine at 6.75 by 9.5 inches and printed to PDF. The body is printed first so that the contents and the index can be built from the page numbers the printing actually produced, then the covers and front matter are printed and the three are joined.

Problem index

Every problem named in a practice sheet, in the order the book reaches it, with its source and the chapter that sets it. Problem statements are not reproduced anywhere in this book. Solve them where they live.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
143	Fibonacci Number	LeetCode 509 (Easy)	The four versions: naive, memo, table, two variables.
143	Climbing Stairs	LeetCode 70 (Easy)	The same recurrence, different base cases. Get them right.
143	Min Cost Climbing Stairs	LeetCode 746 (Easy)	min instead of +, and where the answer lives.
143	Unique Paths	LeetCode 62 (Medium)	Two dimensions, and the base row and column.
143	Unique Paths II	LeetCode 63 (Medium)	Obstacles — the base cases stop being trivial.
143	House Robber	LeetCode 198 (Medium)	The first genuine choice at each step.
144	Fibonacci Number	LeetCode 509 (Easy)	The five conversion steps, on the simplest possible case.
144	Climbing Stairs	LeetCode 70 (Easy)	Same shape, different base cases.
144	Min Cost Climbing Stairs	LeetCode 746 (Easy)	Where the answer lives — $\min(dp[n-1], dp[n-2])$, not $dp[n-1]$.
144	Unique Paths	LeetCode 62 (Medium)	Two dimensions; the one-row collapse and its direction.
144	House Robber	LeetCode 198 (Medium)	A real choice per step, and the two-variable window.
144	Delete and Earn	LeetCode 740 (Medium)	Transform the input first, then it *is* House Robber.
145	Climbing Stairs	LeetCode 70 (Easy)	The habit, on the simplest possible case.
145	House Robber	LeetCode 198 (Medium)	A genuine choice, and the base case $\max(m[0], m[1])$.
145	Maximum Subarray	LeetCode 53 (Medium)	"Ending exactly at i" — so the answer is $\max(dp)$.
145	Min Cost Climbing Stairs	LeetCode 746 (Easy)	The answer is past the end of the array.
145	Decode Ways	LeetCode 91 (Medium)	Two moves, five conditions — the conditions are the problem.
145	Word Break	LeetCode 139 (Medium)	An enumeration of length n, and how to bound it.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
145	House Robber II	LeetCode 213 (Medium)	A circle: run the linear version twice and take the max.
146	House Robber	LeetCode 198 (Medium)	The take-or-skip recurrence and $dp[1] = \max(m[0], m[1])$.
146	House Robber II	LeetCode 213 (Medium)	The circle, the two runs, and the $n = 1$ guard.
146	Delete and Earn	LeetCode 740 (Medium)	Index by value — then it *is* problem 1.
146	Best Time to Buy and Sell Stock with Cooldown	LeetCode 309 (Medium)	The same forbidding, expressed as three states per day.
146	House Robber III	LeetCode 337 (Medium)	The same choice, on a tree instead of a line.
147	Best Time to Buy and Sell Stock IV	LeetCode 188 (Hard)	A mode *and* a count, and where the count is consumed.
147	Best Time to Buy and Sell with Cooldown	LeetCode 309 (Medium)	A mode with three values instead of two.
147	Edit Distance	LeetCode 72 (Medium)	Two positions, and prefix-length versus index conventions.
147	Cheapest Flights Within K Stops	LeetCode 787 (Medium)	A count dimension on a graph.
147	Partition Equal Subset Sum	LeetCode 416 (Medium)	A resource dimension, and the pseudo-polynomial point.
148	Partition Equal Subset Sum	LeetCode 416 (Medium)	Subset sum in disguise, and spotting the odd-total shortcut.
148	Target Sum	LeetCode 494 (Medium)	An algebraic rewrite into subset-sum counting.
148	Last Stone Weight II	LeetCode 1049 (Medium)	The same rewrite again, hidden much better.
148	Ones and Zeroes	LeetCode 474 (Medium)	Two resource dimensions at once — a three-dimensional table.
148	Coin Change	LeetCode 322 (Medium)	Unbounded, so tomorrow's loop direction. Do it to feel the difference.
149	Partition Equal Subset Sum	LeetCode 416 (Medium)	The reduction to $T = S/2$, and the odd-total exit.
149	Partition Array Into Two Arrays to Minimize Sum Difference	LeetCode 2035 (Hard)	The same table, scanned instead of indexed.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
149	Target Sum	LeetCode 494 (Medium)	Two lines of algebra, and counting instead of deciding.
149	Last Stone Weight II	LeetCode 1049 (Medium)	The best-hidden disguise in the set.
149	Partition to K Equal Sum Subsets	LeetCode 698 (Medium)	Where subset sum stops working and backtracking starts.
149	Ones and Zeroes	LeetCode 474 (Medium)	Two targets at once — the same idea in three dimensions.
150	Coin Change	LeetCode 322 (Medium)	Forwards, min, and the infinity sentinel.
150	Coin Change II	LeetCode 518 (Medium)	Counting combinations — coins on the outside.
150	Combination Sum IV	LeetCode 377 (Medium)	Counting permutations — amount on the outside.
150	Perfect Squares	LeetCode 279 (Medium)	Coin change where you generate the coins.
150	Minimum Cost For Tickets	LeetCode 983 (Medium)	Unbounded, with an unusual "coin" — a duration.
150	Integer Break	LeetCode 343 (Medium)	Unbounded knapsack maximising a product.
151	Coin Change II	LeetCode 518 (Medium)	Combinations, and the coins-outside nesting.
151	Combination Sum IV	LeetCode 377 (Medium)	Permutations, and a name that points the wrong way.
151	Climbing Stairs	LeetCode 70 (Easy)	Permutations you have been writing as Fibonacci all along.
151	Decode Ways	LeetCode 91 (Medium)	Sum over last moves, with validity conditions.
151	Unique Paths	LeetCode 62 (Medium)	Counting on a grid, and the one-row collapse.
151	Number of Dice Rolls With Target Sum	LeetCode 1155 (Medium)	Permutations with a fixed count — a second dimension.
152	Longest Increasing Subsequence	LeetCode 300 (Medium)	The state redefinition, and max(dp) not dp[-1].
152	Russian Doll Envelopes	LeetCode 354 (Hard)	Sorting to remove a dimension, and the tie-break minus sign.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
152	Number of Longest Increasing Subsequences	LeetCode 673 (Medium)	Counting alongside, and the replace-versus-add branches.
152	Longest Increasing Path in a Matrix	LeetCode 329 (Hard)	The same idea on a graph — memoised DFS, not a table.
152	Maximum Length of Pair Chain	LeetCode 646 (Medium)	Where greedy beats LIS, and why.
152	Longest Arithmetic Subsequence	LeetCode 1027 (Medium)	LIS with a second dimension in the state.
153	Longest Common Subsequence	LeetCode 1143 (Medium)	The two-dimensional state and both branches.
153	Longest Palindromic Subsequence	LeetCode 516 (Medium)	The LCS(s, reversed(s)) reduction.
153	Delete Operation for Two Strings	LeetCode 583 (Medium)	$n + m - 2 \cdot \text{LCS}$, in one line.
153	Shortest Common Supersequence	LeetCode 1092 (Hard)	Reconstruction, not just the length.
153	Uncrossed Lines	LeetCode 1035 (Medium)	LCS with the strings replaced by number arrays.
154	Edit Distance	LeetCode 72 (Medium)	Three branches, named, and the non-zero base cases.
154	One Edit Distance	LeetCode 161 (Medium)	Where the table is overkill and two pointers win.
154	Delete Operation for Two Strings	LeetCode 583 (Medium)	Edit distance with replace removed.
154	Minimum ASCII Delete Sum	LeetCode 712 (Medium)	Weighted costs, and what happens to the base cases.
154	Regular Expression Matching	LeetCode 10 (Hard)	The same table shape with a different move set.
154	Wildcard Matching	LeetCode 44 (Hard)	The same again, and where greedy beats the table.
155	Longest Palindromic Substring	LeetCode 5 (Medium)	Expand around centre, and the two kinds of centre.
155	Palindromic Substrings	LeetCode 647 (Medium)	The same loop, counting instead of measuring.
155	Longest Palindromic Subsequence	LeetCode 516 (Medium)	A different problem — the LCS reduction.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
155	Palindrome Partitioning II	LeetCode 132 (Hard)	Where the table is required and expansion is not enough.
155	Shortest Palindrome	LeetCode 214 (Hard)	Prefix palindromes, and a linear approach.
155	Valid Palindrome II	LeetCode 680 (Easy)	Two pointers, and knowing when no DP is needed.
156	Unique Paths	LeetCode 62 (Medium)	The cleanest state in DP, and the one-row collapse.
156	Unique Paths II	LeetCode 63 (Medium)	Obstacles, and the first row that is no longer all ones.
156	Minimum Path Sum	LeetCode 64 (Medium)	Same state, min, and running-sum base cases.
156	Triangle	LeetCode 120 (Medium)	A grid that is not rectangular, and filling upwards.
156	Minimum Falling Path Sum	LeetCode 931 (Medium)	Where the one-row trick breaks, and why.
156	Maximal Square	LeetCode 221 (Medium)	An "ending at" state, so the answer is not the last cell.
157	Best Time to Buy and Sell Stock	LeetCode 121 (Easy)	Greedy: min-so-far and best difference.
157	Best Time to Buy and Sell Stock II	LeetCode 122 (Medium)	The two-state machine, and why greedy also works.
157	Best Time to Buy and Sell Stock with Cooldown	LeetCode 309 (Medium)	A third state, and saving yesterday's values.
157	Best Time to Buy and Sell Stock with Transaction Fee	LeetCode 714 (Medium)	One constant, no new state.
157	Best Time to Buy and Sell Stock III	LeetCode 123 (Hard)	$k = 2$, and where the transaction is consumed.
157	Best Time to Buy and Sell Stock IV	LeetCode 188 (Hard)	General k , and the $k \geq n/2$ shortcut.
158	Burst Balloons	LeetCode 312 (Hard)	The last-not-first inversion, and the inclusive k .
158	Minimum Score Triangulation of Polygon	LeetCode 1039 (Medium)	The same shape once more, so it stops feeling special.
158	Stone Game	LeetCode 877 (Medium)	Two players, and the minus sign that handles both.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
158	Palindrome Partitioning II	LeetCode 132 (Hard)	An interval table feeding a one-dimensional DP.
158	Remove Boxes	LeetCode 546 (Hard)	Where two dimensions are not enough.
159	Diameter of Binary Tree	LeetCode 543 (Easy)	Record versus return, in its clearest form.
159	Binary Tree Maximum Path Sum	LeetCode 124 (Hard)	The same, plus the clamp and the -inf base.
159	House Robber III	LeetCode 337 (Medium)	The (take, skip) state, and why one number fails.
159	Longest Univalue Path	LeetCode 687 (Medium)	Record/return again, with a condition on the edge.
159	Sum of Distances in Tree	LeetCode 834 (Hard)	Rerooting, and why the naive version is $O(n^2)$.
159	Binary Tree Cameras	LeetCode 968 (Hard)	Three states per node, and a greedy that also works.
160	Partition to K Equal Sum Subsets	LeetCode 698 (Medium)	Bitmask over items, with pruning that matters.
160	Shortest Path Visiting All Nodes	LeetCode 847 (Hard)	Bitmask plus BFS, and multiple start states.
160	Find the Shortest Superstring	LeetCode 943 (Hard)	TSP in disguise, with overlap as the distance.
160	Number of Ways to Wear Different Hats	LeetCode 1434 (Hard)	Masking the smaller dimension, deliberately.
161	Climbing Stairs	LeetCode 70 (Easy)	The two-variable collapse, and simultaneous assignment.
161	Unique Paths	LeetCode 62 (Medium)	One row, and naming which half is which.
161	Edit Distance	LeetCode 72 (Medium)	One row plus the diagonal variable.
161	Partition Equal Subset Sum	LeetCode 416 (Medium)	The backwards inner loop, and why.
161	Minimum Falling Path Sum	LeetCode 931 (Medium)	Where the collapse is impossible in place.
161	Longest Common Subsequence	LeetCode 1143 (Medium)	Length in linear space; then the string.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
162	Jump Game II	LeetCode 45 (Medium)	Looks like DP, and greedy is better.
162	Decode Ways	LeetCode 91 (Medium)	One dimension, and a validity condition.
162	Longest Increasing Subsequence	LeetCode 300 (Medium)	The incomplete state, caught before coding.
162	Word Break	LeetCode 139 (Medium)	Position plus a dictionary — which dimension is real?
162	Combination Sum	LeetCode 39 (Medium)	"List all" — backtracking, not DP.
162	Super Egg Drop	LeetCode 887 (Hard)	Two dimensions, and a constraint that rules out the obvious one.
163	House Robber III	LeetCode 337 (Medium)	Tree DP under a clock, and the (take, skip) state.
163	Extra Characters in a String	LeetCode 2707 (Medium)	1-D DP, and noticing the hidden cubic cost.
163	Partition Array for Maximum Sum	LeetCode 1043 (Medium)	The last-group decomposition.
163	Best Time to Buy and Sell Stock IV	LeetCode 188 (Hard)	Three dimensions, and the $k \geq n/2$ shortcut.
163	Burst Balloons	LeetCode 312 (Hard)	Interval DP, and the last-not-first inversion.
163	Minimum Cost to Cut a Stick	LeetCode 1547 (Hard)	Interval DP you have not seen, from the shape alone.
164	Assign Cookies	LeetCode 455 (Easy)	The simplest correct greedy, and its exchange argument.
164	Non-overlapping Intervals	LeetCode 435 (Medium)	Sort by end time, and why not by start.
164	Maximum Subarray	LeetCode 53 (Medium)	Greedy and DP are the same algorithm here.
164	Partition Labels	LeetCode 763 (Medium)	A greedy whose rule is not a sort.
164	Coin Change	LeetCode 322 (Medium)	Greedy fails — construct the counter-example first.
164	Best Time to Buy and Sell Stock IV	LeetCode 188 (Hard)	Greedy fails as soon as the limit binds.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
165	Minimum Number of Arrows to Burst Balloons	LeetCode 452 (Medium)	The same exchange as activity selection, in disguise.
165	Queue Reconstruction by Height	LeetCode 406 (Medium)	A greedy whose argument is genuinely subtle.
165	Task Scheduler	LeetCode 621 (Medium)	Greedy plus a formula, and why the formula is right.
165	Course Schedule III	LeetCode 630 (Hard)	An exchange with an undo — greedy that reconsiders.
165	Minimum Cost to Connect Sticks	LeetCode 1167 (Medium)	Huffman, under a different name.
166	Merge Intervals	LeetCode 56 (Medium)	Sort by start, and max on the extension.
166	Insert Interval	LeetCode 57 (Medium)	Whether you notice the input is already sorted.
166	Non-overlapping Intervals	LeetCode 435 (Medium)	Sort by end, and that it is a subtraction.
166	Meeting Rooms II	LeetCode 253 (Medium)	The sweep line, and the tie-break at equal times.
166	Minimum Number of Arrows	LeetCode 452 (Medium)	The same greedy in disguise.
166	Maximum Profit in Job Scheduling	LeetCode 1235 (Hard)	Weighted — greedy fails, DP with a binary search.
167	Merge Intervals	LeetCode 56 (Medium)	The core loop, and the max on the extension.
167	Insert Interval	LeetCode 57 (Medium)	Whether you re-sort a sorted input.
167	Interval List Intersections	LeetCode 986 (Medium)	Two pointers, and which one to advance.
167	Employee Free Time	LeetCode 759 (Hard)	The complement, and its two boundary cases.
167	Data Stream as Disjoint Intervals	LeetCode 352 (Hard)	Streaming, and multi-block absorption.
167	Remove Covered Intervals	LeetCode 1288 (Medium)	Containment as the subject rather than the bug.
168	Meeting Rooms	LeetCode 252 (Easy)	The opener — and whether you sort at all.

DAY	PROBLEM	SOURCE	WHAT IT IS REALLY TESTING
168	Meeting Rooms II	LeetCode 253 (Medium)	The sweep, and the tie-break.
168	Car Pooling	LeetCode 1094 (Medium)	A weighted sweep, and dense coordinates.
168	My Calendar III	LeetCode 732 (Hard)	A sweep over a stream of arrivals.
168	The Skyline Problem	LeetCode 218 (Hard)	The same sweep with max as the accumulator.
168	Minimum Interval to Include Each Query	LeetCode 1851 (Hard)	A sweep over two sorted streams at once.
169	Jump Game	LeetCode 55 (Medium)	The collapse: a reachable set that is a prefix.
169	Jump Game II	LeetCode 45 (Medium)	BFS levels without a queue, and the loop bound.
169	Gas Station	LeetCode 134 (Medium)	The same frontier idea wearing a different hat.
169	Jump Game III	LeetCode 1306 (Medium)	Where the collapse fails — and you need a real search.
169	Jump Game VI	LeetCode 1696 (Medium)	Reachability plus a value, so greedy dies and a deque arrives.
169	Minimum Number of Taps to Water a Garden	LeetCode 1326 (Hard)	Jump Game II in disguise, with intervals.
170	Non-overlapping Intervals	LeetCode 435 (Medium)	The sort key, and "remove fewest" = "keep most".
170	Minimum Number of Arrows to Burst Balloons	LeetCode 452 (Medium)	The same greedy in disguise — do you see it?
170	Course Schedule III	LeetCode 630 (Hard)	Greedy with an undo, and why the longest.
170	Minimum Number of Refueling Stops	LeetCode 871 (Hard)	Regret greedy — deferring the choice to hindsight.
170	IPO	LeetCode 502 (Hard)	Two heaps, and a greedy that needs both orderings.
170	Candy	LeetCode 135 (Hard)	Two greedy passes, and why one pass cannot work.

H

Index

Page numbers in **bold** are where the term is defined. Terms are those the book defines; the glossary in Appendix D gives each one a sentence.

#

4 kb, 224, 229, 239, 1580, 1602, 1603, 1614

A

a replicated log, 1602
abstract class, 1602
acid, 1602, 1603, 1620
adaptee, 1602
adapter, 1602
add a tie-breaker, 1602
algorithm, 8, 11, 17, 18, 63, 149, 157, 162, 249, ...
always comparable, 1602, 1613
amortised, 382, 860, 866, 868, 1323, 1334, 1602
anaemic model, 1602
and a direction, 1602
anti-diagonals, 1602
anti-sniping, 1602
approximate, 275, 278, 382, 807, 1352, 1355, 1356, 1359, 1412, ...
array, 6, 7, 12, 17, 20, 25, 29, 35, 38, ...
artefact, 1603, 1605
assignment rule, 1603
at-least-once, 76, 85, 89, 476, 486, 489, 640, 648, 695, ...
atomic, 83, 85, 176, 177, 273, 276, 277, 285, 288, ...
atomicity, 1602, 1603
availability, 80, 125, 330, 1532, 1603, 1605, 1610, 1617
aws ecs, 1603, 1613

B

backend, 82, 635, 637, 657, 1280, 1603, 1620
bearer token, 237, 1603
best case, 1603, 1620
binary tree, 1603, 1630
blast radius, 231, 243, 1366, 1603
block, 1, 21, 22, 23, 48, 71, 76, 78, 84, ...
blocked, 21, 22, 23, 421, 424, 426, 433, 437, 440, ...
both, 4, 5, 30, 39, 41, 45, 47, 49, 53, ...
boundary, 274, 919, 921, 979, 1270, 1271, 1311, 1332, 1333, ...

bounded context, 1603
bounded optimism, 1603
breadth-first, 421, 433, 1443, 1446, 1461, 1463, 1465, 1603
break point, 562, 1603
broken links, 231, 1604, 1613
bucket by time, 743, 1604
bucket counts, 1604
bulkhead, 1604
burst size, 1604, 1615

C

cache, 6, 7, 8, 9, 10, 12, 13, 14, 15, ...
cache it, 172, 186, 223, 233, 424, 447, 706, 1113, 1531, ...
cache stampede, 1604
candidates, 535, 537, 568, 588, 978, 982, 987, 988, 1073, ...
capacity, 200, 201, 202, 203, 205, 206, 207, 208, 209, ...
cdn, 31, 38, 73, 75, 77, 82, 83, 88, 127, ...
certificate, 475, 1604
chaining, 1604
channels, 78, 478, 743, 747, 750, 751, 752, 754, 1604
checks permission, 1604
chunks, 636, 641, 657, 802, 803, 806, 817, 820, **916**, ...
circular buffer, 1604, 1615
class, 15, 29, 131, 176, 227, 234, 236, 239, 241, ...
client, 75, 78, 80, 127, 171, 173, 174, 223, 224, ...
collapse key, 1605
collapsing writes, 1605
collection, 154, 155, 168, 357, 358, 428, 1015, 1019, 1140, ...
column, 11, 22, 31, 39, 42, 56, 76, 78, 80, ...
compact, 31, 329, 514, 926, 1605, 1606
comparator, 1147, 1605
comparison, 309, 326, 328, 332, 360, 375, 381, 453, 461, ...
complete, 10, 57, 107, 155, 201, 202, 204, 206, 211, ...
component, 125, 1172, 1293, 1605, 1609, 1612, 1617
composite, 1349, 1352, 1354, 1605

- composite key, 1605
- composition, 1434, 1605, 1608, 1631
- composition root, 1605
- connection, 78, 79, 130, 486, 635, 637, 688, 691, 693, ...
- connection-oriented, 1605
- consequence, 1605, 1614, 1616
- consistency, 1531, 1532, 1533, 1602, 1603, 1605, 1607, 1611, 1616
- constraints, 1022, 1071, 1605, 1607
- constructor, 1602, 1605
- container, 1335, 1603, 1605, 1606, 1621
- container image, 1603, 1605
- containerd, 1606
- context, 1603, 1606
- coordinator, 324, 325, 328, 338, 340, 1606, 1614
- correlation id, 1606
- cost, 11, 12, 17, 19, 20, 22, 23, 25, 26, ...
- count map, 1606
- counter, 10, 11, 26, 30, 31, 33, 39, 100, 136, ...
- cri-o, 1606
- critical path, 136, 170, 174, 183, 192, 288, 328, 473, 869, ...
- cycle, 29, 223, 225, 227, 231, 236, 239, 243, 247, ...

D

- dangerous, 1606
- dashed, 1606
- dau, 125, 126, 131, 399, 400, 579, 580, 589, 755, ...
- deep-copied, 1606
- deleted, 223, 226, 227, 236, 298, 319, 324, 338, 476, ...
- delta of delta, 1606
- dependency, 7, 275, 834, 1070, 1606, 1613, 1619
- depth, 7, 11, 12, 13, 15, 17, 18, 19, 23, ...
- depth-first, 1606
- deque, 357, 358, 359, 361, 428, 1449, 1450, 1452, 1453, ...
- diminishing returns, 1085, 1607
- direct addressing, 1607
- domain name, 1607
- draw two diagrams, 1607
- duck typing, 1607
- dummy node, 1607

E

- earlier, 100, 101, 109, 114, 119, 124, 135, 217, 267, ...
- edge cases, 318, 680, 784, 1067, 1070, 1071, 1093, 1094, 1122, ...
- edge compute, 1607
- effect, 178, 187, 234, 270, 280, 284, 375, 438, 538, ...

- elastic scaling, 1607
- election timeout, 1607
- empty, 13, 106, 108, 110, 111, 112, 118, 120, 132, ...
- etl, 34, 339, 369, 427, 429, 515, 543, 937, 1429, ...
- event object, 1607
- eventual consistency, 1531, 1533, 1607
- exactly, 6, 7, 9, 14, 15, 20, 22, 31, 33, ...
- exactly correct, 1607
- exactly one, 14, 22, 102, 106, 113, 118, 160, 164, 203, ...
- exclusive lock, 1607
- expiry, 136, 137, 169, 179, 188, 220, 221, 222, 223, ...

F

- facts, 84, 298, 704, 1084, 1608, 1610, 1614
- false negative, 1608
- false positive, 26, 27, 28, 29, 428, 431, 485, 1608
- fanout, 480, 483, 530, 531, 533, 535, 584, 586, 587, ...
- fee, 6, 13, 75, 82, 83, 99, 100, 122, 123, ...
- fifo, 421, 423, 425, 426, 435, 437, 442, 446, 447, ...
- file, 15, 31, 32, 38, 41, 46, 62, 73, 74, ...
- filled diamond, 1608
- finite state machine, 1608
- float, 29, 275, 293, 312, 344, 351, 354, 355, 356, ...
- four-direction move, 1608
- fragile, 1608
- frontend, 1608

G

- galloping mode, 1608
- gap, 6, 17, 42, 46, 127, 135, 149, 150, 159, ...
- generator expression, 1608
- greedy simulation, 1608
- guest operating system, 1608, 1609

H

- hash map, 33, 39, 46, 1286, 1287, 1290, 1306, 1308, 1313, ...
- hash ring, 324, 1608
- hateoas, 1608
- header interface, 1609
- heap, 21, 24, 26, 27, 29, 33, 34, 36, 39, ...
- heapsort, 1609
- heartbeat, 693, 696, 747, 749, 1474, 1477, 1478, 1483, 1484, ...
- hidden spof, 1609
- hiding mechanism, 1609
- highest node, 1609
- historical snapshot, 1609
- hollow triangle, 1609

hook, 476, 478, 488, 1116, 1120, 1121, 1161, 1164, 1165, ...
horizontal scaling, 1609
hypervisor, 1608, 1609

I

image, 77, 80, 82, 83, 88, 95, 545, 546, 632, ...
implicit graph, 1609
in parallel, 328, 480, 803, 1609
in series, 81, 1609
in-degree, 1609, 1613
index, 19, 27, 53, 54, 60, 67, 69, 71, 73, ...
indexed heap, 1395, 1609
inheritance, 1609
inorder, 1609, 1616
input, 14, 63, 65, 91, 96, 108, 111, 112, 117, ...
integrity, 1610
intent plus multiplicity, 1610
interface, 860, 1529, 1532, 1535, 1606, 1608, 1609, 1610, 1615
invariant, 1166, 1322, 1610
is possible, 325, 811, 1009, 1610, 1617
item, 12, 15, 24, 25, 26, 27, 28, 29, 30, ...
iterator, 1610

J

json, 483, 534, 543, 546, 634, 636, 637, 643, 644, ...
jump, 99, 131, 153, 163, 199, 374, 382, 387, 390, ...
jwt, 1610

K

k closest points, 1610
k most frequent, 1610
k-th largest specifically, 1610
key, 14, 26, 32, 34, 35, 38, 40, 44, 47, ...
key property, 1172, 1610

L

lag, 73, 80, 113, 125, 129, 157, 161, 182, 222, ...
largest, 83, 113, 123, 131, 146, 147, 148, 150, 156, ...
latency, 36, 73, 74, 75, 76, 78, 79, 89, 95, ...
lazy, 236, 276, 288, 289, 295, 544, 763, 817, 821, ...
lazy deletion, 1384, 1389, 1391, 1392, 1395, 1406, 1577, 1611
lazy loading, 1611
lazy refill, 289, 1581, 1611
leader, 1316, 1346, 1348, 1349, 1350, 1352, 1353, 1354, 1355, ...
leaf, 835, 1218, 1611, 1617
lease, 86, 169, 374, 708, 764, 858, 859, 860, 862, ...
ledger entries, 1171, 1611

left, 20, 21, 22, 53, 56, 57, 61, 65, 80, ...
lfu, 1611
linear scan, 1273, 1397, 1611
linearizability, 1611
list, 2, 13, 14, 24, 26, 33, 35, 37, 41, ...
list of lists, 258, 310, 514, 621, 787, 901, 961, 1611
load balancer, 127, 174, 1531, 1537, 1611
load factor, 1611
load shedding, 1611
log-structured merge tree, 1611
logical cohesion, 1611
logical time, 1611
loop, 9, 12, 21, 22, 23, 43, 44, 51, 53, ...
loop body, 1611
low cohesion, 1611
lru, 6, 8, 10, 13, 14, 15, 44, 53, 54, ...

M

manifest, 641, 803, 804, 806, 807, 808, 811, 863, 1611
many times, 22, 27, 121, 170, 183, 198, 280, 322, 349, ...
many-to-many, 1612, 1618
matrix, 671, 673, 777, 778, 779, 780, 781, 784, 786, ...
mau, 1612
max-heap, 1402, 1510, 1512, 1524, 1612
method resolution order, 1612
milliseconds, 373, 380, 699, 844, 1039, 1047, 1100, 1289, 1606, ...
min-heap, 1279, 1281, 1283, 1384, 1399, 1401, 1405, 1575, 1577, ...
minimum spanning forest, 1612
mixin, 261, 1612
monotone, 1612
monotonic deque, 1612

N

name mangling, 1612
namespace, 77, 1612
negative, 32, 140, 305, 678, 836, 838, 841, 843, 844, ...
negative infinity, 1612
negatives, 880, 1072, 1607, 1612
nested loop, 777, 1612
network partition, 326, 1613
never equal, 1602, 1613
never forgets, 1613
never persisted, 1613
no children, 1611, 1613
node, 6, 18, 76, 79, 80, 94, 191, 279, 280, ...
not a map, 1613

O

$o(1)$, 9, 10, 12, 17, 20, 23, 54, 57, 59, ...
object, 15, 31, 38, 40, 71, 73, 75, 76, 77, ...
observers, 1613
occupied, 1275, 1606, 1607, 1613
offset, 327, 382, 532, 1357, 1413, 1613, 1622
on first touch, 1613
one-shot, 1613
opens, 400, 535, 581, 589, 599, 688, 707, 756, 859, ...
orchestrator, 1603, 1613
origin, 51, 70, 131, 230, 380, 535, 589, 635, 637, ...
origin pull, 1613
orphan rows, 1613
orphans, 231, 1604, 1613
out-degree, 1609, 1613
overloading, 1613
overriding, 1614

P

page, 39, 40, 41, 50, 75, 81, 136, 170, 179, ...
parallel edges, 1614
parameters, 1614
parent, 60, 457, 459, 460, 745, 748, 781, 784, 786, ...
partial failure, 1614
participants, 1606, 1614, 1616
partition key, 746, 1604, 1614
partitions, 326, 754, 1172, 1614, 1622
path, 1, 20, 21, 22, 23, 28, 32, 33, 34, ...
port, 8, 10, 18, 29, 30, 32, 39, 41, 54, ...
prefix sums, 1612, 1614
preorder, 1614
priority queue, 425, 1014, 1614
procedure, 380, 1019, 1144, 1503, 1511, 1552, 1614
process, 75, 77, 252, 354, 379, 402, 407, 482, 635, ...
program, 1, 2, 3, 10, 42, 1006, 1007, 1059, 1066, ...
protecting state, 1614

Q

qps, 1614
queue, 71, 73, 74, 75, 76, 80, 82, 83, 84, ...
quorum, 320, 323, 325, 328, 330, 335, 336, 339, 345, ...

R

race condition, 1615
ram, 1, 2, 3, 4, 10, 12, 15, 18, 19, ...
rank, 73, 77, 89, 93, 125, 129, 525, 528, 529, ...
rebalancing, 1103, 1615
rebuilding the tree, 1615
recursion, 7, 10, 12, 13, 15, 53, 57, 58, 59, ...

references, 479, 924, 1165, 1476, 1604, 1613, 1615, 1621
refill rate, 1604, 1615
register, 133, 321, 323, 326, 742, 743, 946, 947, 1158, ...
relative epsilon, 1615
removal, 33, 41, 227, 231, 235, 238, 239, 544, 715, ...
replica, 75, 76, 77, 80, 325, 326, 328, 330, 331, ...
replica divergence, 1615
request-aware routing, 1615
resolver, 432, 441, 478, 484, 492, 493, 1583, 1615
resources, 343, 1615, 1619
returned, 68, 174, 222, 236, 534, 543, 544, 655, 833, ...
ring buffer, 1604, 1615
role interface, 1615
root, 37, 110, 324, 326, 332, 338, 777, 793, 832, ...
rotation, 639, 1042, 1049, 1616
routine, 613, 908, 1244, 1616
row, 6, 8, 11, 18, 19, 20, 22, 27, 31, ...
run, 11, 27, 30, 37, 43, 51, 58, 60, 61, ...

S

same remainder, 1616
schema, 78, 177, 225, 1165, 1470, 1476, 1615, 1616
scoping, 605, 826, 1062, 1616
script, 98, 120, 121, 123, 124, 125, 131, 132, 135, ...
seam, 701, 1616
search space, 797, 1616
selectivity, 1616
sentinel, 362, 393, 394, 1607, 1616, 1627
separator, 1616
sequence diagram, 1616, 1623
serialisable, 1616
service credits, 1616
session, 534, 754, 866, 1107, 1225, 1228, 1229, 1231, 1234, ...
session guarantees, 1616
set, 1, 12, 20, 24, 25, 26, 27, 29, 30, ...
shallow-copied, 1616
shape, 3, 6, 8, 9, 19, 23, 24, 26, 27, ...
shard, 34, 40, 41, 76, 80, 94, 120, 121, 122, ...
shared lock, 1617
shared store, 274, 278, 286, 388, 528, 1617
singleton, 1617
singly linked list, 1617
slice, 15, 18, 40, 41, 160, 162, 191, 312, 623, ...
smallest, 307, 455, 456, 459, 867, 1137, 1159, 1165, 1214, ...
snapshot, 926, 1167, 1172, 1609, 1617
snipe, 1617

sorted region, 1617
sorting everything, 1617
splits, 30, 37, 268, 774, 776, 777, 787, 792, 821, ...
spof, 1609, 1617
squares, 392, 671, 980, 1617, 1627
stable, 31, 32, 34, 41, 46, 329, 339, 341, 374, ...
stack, 6, 12, 59, 66, 70, 231, 450, 454, 470, ...
stack frame, 59, 834, 1617
stack overflow, 1617
state, 1, 3, 5, 7, 12, 14, 16, 17, 19, ...
statements, 1614, 1617, 1624
status, 82, 89, 172, 294, 481, 482, 485, 486, 638, ...
still one machine, 1610, 1617
stonith, 1618
strategy, 121, 525, 874, 1003, 1063, 1602, 1605, 1618
strictly shorter, 773, 775, 777, 792, 794, 796, 797, 1072, 1568, ...
structural, 1614, 1618
subarray, 101, 103, 104, 106, 107, 108, 109, 110, 111, ...
subject, 1618, 1632
subscription list, 1618
substring, 1, 109, 110, 507, 510, 511, 512, 513, 608, ...
subtree, 836, 837, 838, 839, 848, 1011, 1014, 1609, 1614, ...
surge multiplier especially, 1618
swap, 266, 269, 355, 393, 394, 401, 402, 409, 411, ...
system call, 1618

T

table, 3, 6, 9, 10, 11, 13, 14, 19, 20, ...
tail call, 1618
tail-recursive, 1618
tcp, 1605, 1618, 1619
template, 426, 475, 1017, 1609, 1618
test double, 1618
the return address, 1618
the running median, 1609, 1618, 1619
third table, 1612, 1618
thread, 286, 289, 377, 592, 600, 741, 745, 756, 846, ...
three distinct orderings, 1619
three-way partition, 1619
tight coupling, 1619
time-to-live, 1619
tiny constraint, 1619

toast, 1619
token, 78, 237, 271, 272, 273, 274, 275, 276, 277, ...
tombstone, 1619
top-k problems, 1609, 1618, 1619
total order, 1619
transitions itself, 1619
transitive dependency, 1619
tree, 1, 5, 6, 17, 18, 38, 46, 77, 79, ...
ttl, 6, 24, 34, 46, 64, 79, 83, 89, 120, ...
tuple, 15, 55, 61, 153, 155, 206, 225, 227, 230, ...
two, 3, 4, 5, 6, 7, 9, 10, 11, 12, ...
two implementations, 1619

U

udp, 1618, 1619
under-fetching, 1619
union filesystem, 1619
upper bound, 587, 1342, 1372, 1615, 1620
uri, 179, 187, 331, 585, 687, 723, 860, 904, 1039, ...
url, 24, 26, 28, 31, 33, 34, 35, 37, 38, ...

V

value object, 1620
version, 10, 43, 57, 62, 91, 92, 159, 160, 240, ...
vertex, 1606, 1613, 1614, 1620
vertical scaling, 1620
victim, 1620
virtual address space, 1620
virtual machine, 387, 1603, 1613, 1620
volatile, 1620

W

weight, 132, 200, 201, 202, 203, 206, 210, 213, 217, ...
which class, 1620
which is hours, 795, 969, 1286, 1298, 1620
within, 76, 83, 86, 94, 110, 131, 133, 164, 179, ...
worst case, 280, 304, 345, 361, 620, 621, 622, 627, 629, ...
write skew, 1620
write-around with invalidation, 1620
write-through, 1620

Y

yes, 24, 26, 35, 36, 40, 46, 74, 75, 129, ...

Two subjects, every day, for one hundred and eighty days.

Most preparation teaches algorithms for three months, then starts again from nothing on system design. Interviews do not work that way. A single loop will ask you to reverse a linked list in one room and design a rate limiter in the next.

So this book teaches both, every day, for one hundred and eighty days. Written for someone starting from zero - not a graduate, not somebody brushing up - and finishing able to answer either kind of question out loud, to a stranger, without notes.

- ◆ One hundred and eighty chapters. Two lessons and a practice sheet in every one.
- ◆ The same nine sections every time, so you learn one reading rhythm and keep it.
- ◆ Every lesson opens with a story about a person, told without a single technical word.
- ◆ Every solution shown in full, built in fragments and then given complete.
- ◆ The arithmetic done out loud, and error messages printed exactly as they appear.
- ◆ Real interviewer phrasings, an opening script, the follow-ups, and a model answer.